

Conan Documentation

Release 2.34.0

The Conan team

Nov 12, 2026

CONTENTS

1 Introduction **3**
- 1.1 Open Source . . . 3
- 1.2 Decentralized package manager . . . 3
- 1.3 Binary management . . . 4
- 1.4 All platforms, all build systems and compilers . . . 5
- 1.5 Stable . . . 6
- 1.6 Community . . . 7
- 1.7 Navigating the documentation . . . 7

2 Installation **9**
- 2.1 Install with pip (recommended) . . . 9
- 2.2 Install with pipx . . . 10
- 2.3 Use a system installer or create a self-contained executable . . . 11
- 2.4 Install from source . . . 11

3 Tutorial **13**
- 3.1 Consuming packages . . . 13
- 3.2 Creating packages . . . 42
- 3.3 Working with Conan repositories . . . 95
- 3.4 Developing packages locally . . . 101
- 3.5 Versioning . . . 122
- 3.6 Other important Conan features . . . 145

4 Continuous Integration (CI) tutorial **147**
- 4.1 Packages and products pipelines . . . 149
- 4.2 Repositories and promotions . . . 149

5 Devops guide **173**
- 5.1 Using ConanCenter packages in production environments . . . 173
- 5.2 Local Recipes Index Repository . . . 176
- 5.3 Backing up third-party sources with Conan . . . 181
- 5.4 Managing package metadata files . . . 186
- 5.5 Versioning . . . 194
- 5.6 Save and restore packages from/to the cache . . . 197
- 5.7 Vendoring dependencies in Conan packages . . . 198
- 5.8 Package promotions . . . 199
- 5.9 Checking package vulnerabilities . . . 202
- 5.10 Package compression format . . . 205

6 Security **207**
- 6.1 Scanning dependencies with `conan audit` . . . 207

6.2 Software Bills of Materials (SBOM) . . . 208
6.3 Security guidelines . . . 210
6.4 C, C++ Compiler Sanitizers . . . 212

7 Integrations **221**
7.1 CMake . . . 221
7.2 CLion . . . 222
7.3 Visual Studio . . . 228
7.4 Autotools . . . 233
7.5 Bazel . . . 234
7.6 Makefile . . . 234
7.7 Xcode . . . 235
7.8 Meson . . . 235
7.9 Emscripten . . . 236
7.10 Premake . . . 236
7.11 Android . . . 237
7.12 JFrog . . . 237
7.13 ROS . . . 238
7.14 Python . . . 243
7.15 GitHub . . . 244
7.16 Community . . . 246

8 Examples **247**
8.1 ConanFile methods examples . . . 247
8.2 Conan extensions examples . . . 258
8.3 Conan recipe tools examples . . . 281
8.4 Cross-building examples . . . 334
8.5 Configuration files examples . . . 357
8.6 Graph examples . . . 362
8.7 Developer tools and flows . . . 373
8.8 Conan commands examples . . . 380
8.9 Conan runners examples . . . 383
8.10 Conan security examples . . . 394

9 Reference **397**
9.1 Binary model . . . 397
9.2 Commands . . . 415
9.3 conanfile.py . . . 605
9.4 conanfile.txt . . . 695
9.5 Conan Server . . . 698
9.6 Configuration files . . . 703
9.7 Environment variables . . . 754
9.8 Extensions . . . 756
9.9 Policies . . . 817
9.10 Recipe tools . . . 818
9.11 Runners . . . 1008
9.12 Workspace . . . 1010

10 Knowledge **1017**
10.1 Cheat sheet . . . 1017
10.2 Core guidelines . . . 1018
10.3 FAQ . . . 1020
10.4 Videos . . . 1027
10.5 JFrog Academy: Conan 2 Training . . . 1031
10.6 Community Resources . . . 1032

11 Incubating features **1033**
11.1 New CMakeConfigDeps generator . . . 1033
11.2 Workspaces . . . 1033

12 What's new in Conan 2 **1035**
12.1 Conan 2 migration guide . . . 1035
12.2 New graph model . . . 1035
12.3 New public Python API . . . 1035
12.4 New build system integrations . . . 1036
12.5 New custom user commands . . . 1036
12.6 New CLI . . . 1036
12.7 New deployers . . . 1036
12.8 New package_id . . . 1037
12.9 compatibility.py . . . 1037
12.10 New lockfiles . . . 1037
12.11 New configuration and environment management . . . 1037
12.12 Multi-revision cache . . . 1038
12.13 New extension plugins . . . 1038
12.14 Package immutability optimizations . . . 1038
12.15 Package lists . . . 1039
12.16 Metadata files . . . 1039
12.17 Third-party backup sources . . . 1039
12.18 Installing configuration from Conan packages . . . 1039

13 Changelog **1041**
13.1 2.34.0 (28-Oct-2026) . . . 1041
13.2 2.33.0 (29-Sept-2026) . . . 1043
13.3 2.32.0 (31-Aug-2026) . . . 1044
13.4 2.31.2 (04-Aug-2026) . . . 1045
13.5 2.31.1 (24-Jul-2026) . . . 1045
13.6 2.31.0 (23-Jul-2026) . . . 1046
13.7 2.30.0 (29-Jun-2026) . . . 1047
13.8 2.29.1 (12-Jun-2026) . . . 1048
13.9 2.29.0 (28-May-2026) . . . 1048
13.10 2.28.1 (30-Apr-2026) . . . 1049
13.11 2.28.0 (28-Apr-2026) . . . 1049
13.12 2.27.1 (13-Apr-2026) . . . 1051
13.13 2.27.0 (25-Mar-2026) . . . 1051
13.14 2.26.2 (05-Mar-2026) . . . 1052
13.15 2.26.1 (27-Feb-2026) . . . 1052
13.16 2.26.0 (25-Feb-2026) . . . 1052
13.17 2.25.2 (04-Feb-2026) . . . 1053
13.18 2.25.1 (29-Jan-2026) . . . 1053
13.19 2.25.0 (28-Jan-2026) . . . 1053
13.20 2.24.0 (15-Dec-2025) . . . 1055
13.21 2.23.0 (25-Nov-2025) . . . 1056
13.22 2.22.2 (07-Nov-2025) . . . 1057
13.23 2.22.1 (30-Oct-2025) . . . 1057
13.24 2.22.0 (29-Oct-2025) . . . 1057
13.25 2.21.0 (29-Sept-2025) . . . 1058
13.26 2.20.1 (04-Sept-2025) . . . 1060
13.27 2.20.0 (01-Sept-2025) . . . 1060
13.28 2.19.1 (30-Jul-2025) . . . 1061
13.29 2.19.0 (23-Jul-2025) . . . 1061

13.30 2.18.1 (04-Jul-2025) . . . 1062
13.31 2.18.0 (30-Jun-2025) . . . 1062
13.32 2.17.1 (23-Jun-2025) . . . 1063
13.33 2.17.0 (28-May-2025) . . . 1064
13.34 2.16.1 (29-Apr-2025) . . . 1065
13.35 2.16.0 (29-Apr-2025) . . . 1065
13.36 2.15.1 (14-Apr-2025) . . . 1066
13.37 2.15.0 (31-Mar-2025) . . . 1066
13.38 2.14.0 (12-Mar-2025) . . . 1067
13.39 2.13.0 (26-Feb-2025) . . . 1068
13.40 2.12.2 (12-Feb-2025) . . . 1069
13.41 2.12.1 (28-Jan-2025) . . . 1069
13.42 2.12.0 (27-Jan-2025) . . . 1069
13.43 2.11.0 (18-Dec-2024) . . . 1070
13.44 2.10.3 (18-Dec-2024) . . . 1071
13.45 2.10.2 (10-Dec-2024) . . . 1071
13.46 2.10.1 (04-Dec-2024) . . . 1071
13.47 2.10.0 (02-Dec-2024) . . . 1071
13.48 2.9.3 (21-Nov-2024) . . . 1072
13.49 2.9.2 (07-Nov-2024) . . . 1072
13.50 2.9.1 (30-Oct-2024) . . . 1072
13.51 2.9.0 (29-Oct-2024) . . . 1072
13.52 2.8.1 (17-Oct-2024) . . . 1074
13.53 2.8.0 (30-Sept-2024) . . . 1074
13.54 2.7.1 (11-Sept-2024) . . . 1075
13.55 2.7.0 (28-Aug-2024) . . . 1075
13.56 2.6.0 (01-Aug-2024) . . . 1076
13.57 2.5.0 (03-Jul-2024) . . . 1077
13.58 2.4.1 (10-Jun-2024) . . . 1078
13.59 2.4.0 (05-Jun-2024) . . . 1078
13.60 2.3.2 (28-May-2024) . . . 1079
13.61 2.3.1 (16-May-2024) . . . 1080
13.62 2.3.0 (06-May-2024) . . . 1080
13.63 2.2.3 (17-Apr-2024) . . . 1082
13.64 2.2.2 (25-Mar-2024) . . . 1082
13.65 2.2.1 (20-Mar-2024) . . . 1082
13.66 2.2.0 (20-Mar-2024) . . . 1082
13.67 2.1.0 (15-Feb-2024) . . . 1084
13.68 2.0.17 (10-Jan-2024) . . . 1087
13.69 2.0.16 (21-Dec-2023) . . . 1087
13.70 2.0.15 (20-Dec-2023) . . . 1087
13.71 2.0.14 (14-Nov-2023) . . . 1089
13.72 2.0.13 (28-Sept-2023) . . . 1090
13.73 2.0.12 (26-Sept-2023) . . . 1090
13.74 2.0.11 (18-Sept-2023) . . . 1091
13.75 2.0.10 (29-Aug-2023) . . . 1092
13.76 2.0.9 (19-Jul-2023) . . . 1093
13.77 2.0.8 (13-Jul-2023) . . . 1094
13.78 2.0.7 (21-Jun-2023) . . . 1095
13.79 2.0.6 (26-May-2023) . . . 1096
13.80 2.0.5 (18-May-2023) . . . 1096
13.81 2.0.4 (11-Apr-2023) . . . 1098
13.82 2.0.3 (03-Apr-2023) . . . 1099
13.83 2.0.2 (15-Mar-2023) . . . 1100

13.84 2.0.1 (03-Mar-2023) . . . 1101
13.85 2.0.0 (22-Feb-2023) . . . 1102
13.86 2.0.0-beta10 (16-Feb-2023) . . . 1102
13.87 2.0.0-beta9 (31-Jan-2023) . . . 1102
13.88 2.0.0-beta8 (12-Jan-2023) . . . 1103
13.89 2.0.0-beta7 (22-Dec-2022) . . . 1103
13.90 2.0.0-beta6 (02-Dec-2022) . . . 1104
13.91 2.0.0-beta5 (11-Nov-2022) . . . 1104
13.92 2.0.0-beta4 (11-Oct-2022) . . . 1104
13.93 2.0.0-beta3 (12-Sept-2022) . . . 1105
13.94 2.0.0-beta2 (27-Jul-2022) . . . 1105
13.95 2.0.0-beta1 (20-Jun-2022) . . . 1106

Index **1107**

Welcome! This is the user documentation for Conan, an open source, decentralized C/C++ package manager that works in all platforms and with all build systems and compilers. Other relevant resources:

- Conan home page. Entry point to the project, with links to docs, blog, social, downloads, release mailing list, etc.
- Github project and issue tracker. The main support channel, file issues here for questions, bug reports and feature requests.

Table of contents:

CHAPTER

ONE

INTRODUCTION

Conan is a dependency and package manager for C and C++ languages. It is free and open-source, works in all platforms (Windows, Linux, macOS, FreeBSD, Solaris, etc.), and can be used to develop for all targets including embedded, mobile (iOS, Android), and bare metal. It also integrates with all build systems like CMake, Visual Studio (MSBuild), Makefiles, SCons, etc., including proprietary ones.

It is specifically designed and optimized for accelerating the development and Continuous Integration of C and C++ projects. With full binary management, it can create and reuse any number of different binaries (for different configurations like architectures, compiler versions, etc.) for any number of different versions of a package, using exactly the same process in all platforms. As it is decentralized, it is easy to run your own server to host your own packages and binaries privately, without needing to share them. The free JFrog Artifactory Community Edition (CE) is the recommended Conan server to host your own packages privately under your control.

Conan is mature and stable, with a strong commitment to forward compatibility (non-breaking policy), and has a complete team dedicated full time to its improvement and support. It is backed and used by a great community, from open source contributors and package creators in ConanCenter to thousands of teams and companies using it.

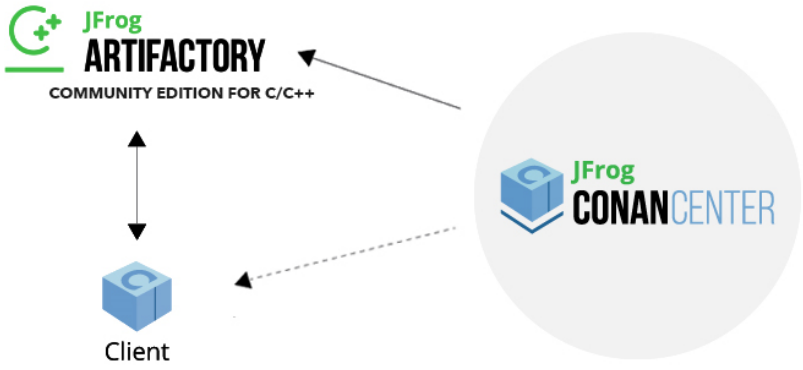
1.1 Open Source

Conan is Free and Open Source, with a permissive MIT license. Check out the source code and issue tracking (for questions and support, reporting bugs and suggesting feature requests and improvements) at https://github.com/conan-io/conan

1.2 Decentralized package manager

Conan is a decentralized package manager with a client-server architecture. This means that clients can fetch packages from, as well as upload packages to, different servers ("remotes"), similar to the "git" push-pull model to/from git remotes.

At a high level, the servers are just storing packages. They do not build nor create the packages. The packages are created by the client, and if binaries are built from sources, that compilation is also done by the client application.



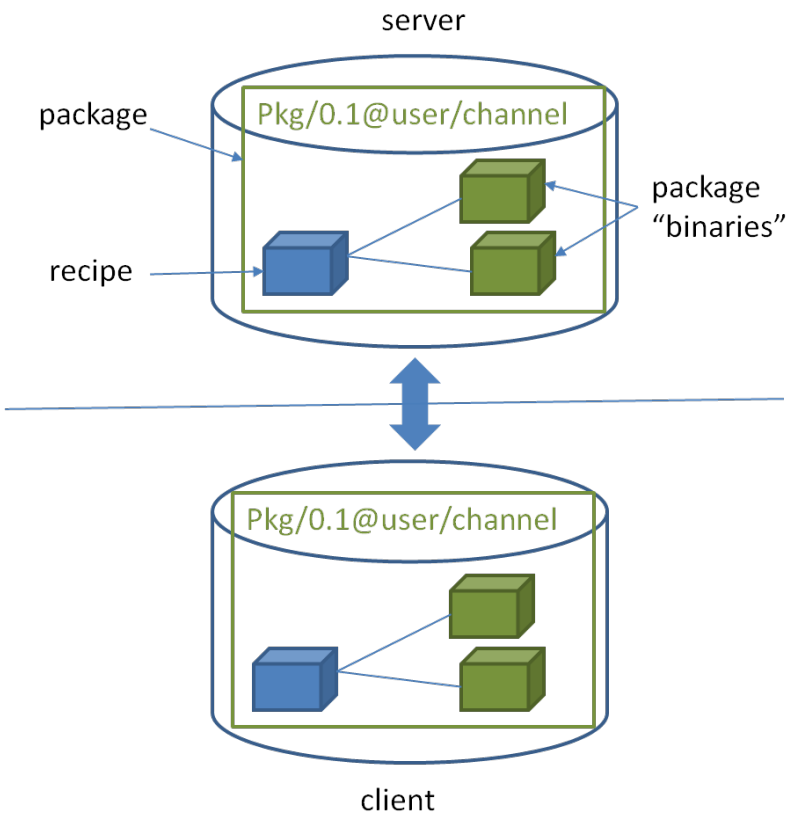
The different applications in the image above are:

- The Conan client: this is a console/terminal command-line application, containing the heavy logic for package creation and consumption. Conan client has a local cache for package storage, and so it allows you to fully create and test packages offline. You can also work offline as long as no new packages are needed from remote servers.
- JFrog Artifactory Community Edition (CE) is the recommended Conan server to host your own packages privately under your control. It is a free community edition of JFrog Artifactory for Conan packages, including a WebUI, multiple auth protocols (LDAP), Virtual and Remote repositories to create advanced topologies, a Rest API, and generic repositories to host any artifact.
- The conan_server is a small server distributed together with the Conan client. It is a simple open-source implementation and provides basic functionality, but no WebUI or other advanced features.
- ConanCenter is a central public repository where the community contributes packages for popular open-source libraries like Boost, Zlib, OpenSSL, Poco, etc.

1.3 Binary management

One of the most powerful features of Conan is that it can create and manage pre-compiled binaries for any possible platform and configuration. By using pre-compiled binaries and avoiding repeated builds from source, it saves significant time for developers and Continuous Integration servers, while also improving the reproducibility and traceability of artifacts.

A package is defined by a "conanfile.py". This is a file that defines the package's dependencies, sources, how to build the binaries from sources, etc. One package "conanfile.py" recipe can generate any arbitrary number of binaries, one for each different platform and configuration: operating system, architecture, compiler, build type, etc. These binaries can be created and uploaded to a server with the same commands in all platforms, having a single source of truth for all packages and not requiring a different solution for every different operating system.



Installation of packages from servers is also very efficient. Only the necessary binaries for the current platform and configuration are downloaded, not all of them. If the compatible binary is not available, the package can be built from sources in the client too.

1.4 All platforms, all build systems and compilers

Conan works on Windows, Linux (Ubuntu, Debian, Red Hat, ArchLinux, Raspbian), OSX, FreeBSD, and SunOS, and, as it is portable, it might work in any other platform that can run Python. It can target any existing platform: ranging from bare metal to desktop, mobile, embedded, servers, and cross-building.

Conan works with any build system too. There are built-in integrations to support the most popular ones like CMake, Visual Studio (MSBuild), Autotools and Makefiles, Meson, SCons, etc., but it is not a requirement to use any of them. It is not even necessary that all packages use the same build system: each package can use their own build system, and depend on other packages using different build systems. It is also possible to integrate with any build system, including proprietary ones.

Likewise, Conan can manage any compiler and any version. There are default definitions for the most popular ones: gcc, cl.exe, clang, apple-clang, intel, with different configurations of versions, runtimes, C++ standard library, etc. This model is also extensible to any custom configuration.

1.5 Stable

From Conan 2.0 and onwards, there is a commitment to stability, with the goal of not breaking user space while evolving the tool and the platform. This means:

- Moving forward to following minor versions 2.1, 2.2, ..., 2.X should never break existing recipes, packages or command line flows
- If something is breaking, it will be considered a regression and reverted.
- Bug fixes will not be considered breaking, recipes and packages relying on the incorrect behavior of such bugs will be considered already broken.
- Only documented features in https://docs.conan.io are considered part of the public interface of Conan. Private implementation details, and everything not included in the documentation is subject to change.
- The compatibility is always considered forward. New APIs, tools, methods, helpers can be added in following 2.X versions. Recipes and packages created with these features will be backwards incompatible with earlier Conan versions.
- Only the latest released patch (major.minor.patch) of every minor version is supported and stable.

There are some things that are not included in this commitment:

- Public repositories, like **ConanCenter**, assume the use of the latest version of the Conan client, and using an older version may result in failure of packages and recipes created with a newer version of the client. It is recommended to use your own private repository to store your own copy of the packages for production, or as a secondary alternative, to use some locking mechanism to avoid possible disruption from packages in ConanCenter that are updated and require latest Conan version.
- Configuration and automatic tools detection, like the detection of the default profile (`conan profile detect`) can and will change at any time. Users are encouraged to define their configurations in their own profiles files for repeatability. New versions of Conan might detect different default profiles.
- Builtin default implementation of extension points as plugins or hooks can also change with every release. Users can provide their own ones for stability.
- Output of packages templates with `conan new` can update at any time to use latest features.
- The output streams stdout, stderr, i.e. the terminal output can change at any time. Do not parse the terminal output for automation.
- Anything that is explicitly labeled as `experimental` or `preview` in the documentation, or in the Conan cli output. Read the section below for a detailed definition of these labels.
- Anything that is labeled as `deprecated` in the documentation should not get new usages, as it will not get new fixes and it will be removed in the next major version.
- Other tools and repositories outside of the Conan client

Conan needs Python>=3.8 to run. Conan will deprecate support for Python versions 1 year after those versions have been declared End Of Life (EOL).

If you have any question regarding Conan updates, stability, or any clarification about this definition of stability, please report in the documentation issue tracker: https://github.com/conan-io/docs.

1.6 Community

Conan is being used in production by thousands of companies like TomTom, Audi, RTI, Continental, Plex, Electrolux and Mercedes-Benz and many thousands of developers around the world. But an essential part of Conan is that many of those users will contribute back, creating an amazing and helpful community:

- The https://github.com/conan-io/conan project has around 8.6K stars in Github and counts with contributions from more than 400 different users (this is just the client tool).
- Many other users contribute recipes for ConanCenter via the https://github.com/conan-io/conan-center-index repo, creating packages for popular Open Source libraries, contributing many thousands of Pull Requests per year.
- More than two thousands Conan users hang around the CppLang Slack #conan channel, and help responding to questions, discussing problems and approaches, making it one of the most active channels in the whole CppLang slack.
- There is a Conan channel in #include<cpp> discord.

1.7 Navigating the documentation

This documentation has very different sections:

- The **tutorial** is an actual hands-on tutorial, with examples and real code, intended to be played sequentially from beginning to end, running the exercises in your own computer. There is a "narrative" to this section and the exercises might depend on some previous explanations and code - building on the previous example. This is the recommended approach for learning Conan.
- The **examples** also contain hands-on, fully operational examples with code, aimed to explain some very specific feature, tool or behavior. They do not have a conducting thread, they should be navigated by topic.
- The **reference** is the source of truth for the interfaces of every public command, class, method, helper, API and configuration file that can be used. It is not designed to be read fully, but to check for individual items when necessary.
- The **knowledge** base contains things like the FAQ, a very important section about general guidelines, good practices and bad practices, videos from conference talks, etc.

Features in this documentation might be labeled as:

- **experimental**: This feature is released and can be used, but it is under active development and the interfaces, APIs or behavior might change as a result of evolution, and this will not be considered breaking. If you are interested in these features you are encouraged to try them and give feedback, because that is exactly what allows to stabilize them.
- **preview**: When a feature is released in preview mode, this means it aims to be as final and stable as possible. Users are encouraged to use them, and the maintainers team will try not to break them unless necessary. But if necessary, they might change and break.
- **deprecated**: This feature should no longer be used, and it will be fully removed in next major release. Other alternatives or approaches should be used instead of it, and if using it, migrating to the other alternatives should be done as soon as possible. They will not be maintained or get fixes.

Everything else that is not labeled should be considered stable and won't be broken, unless something that is declared a bugfix.

Have any questions? Please check out our *FAQ section* or .

CHAPTER

TWO

INSTALLATION

Conan can be installed on many operating systems. It has been extensively used and tested on Windows, Linux (different distros), macOS, and is also actively used in FreeBSD and Solaris SunOS. There are also several additional operating systems on which it has been reported to work.

There are different ways to install Conan:

1. The preferred and **strongly recommended way to install Conan** is from PyPI, the Python Package Index, using the `pip` command.
2. Use a system installer, or create your own self-contained Conan executable, to not require Python on your system.
3. Running Conan from sources.

2.1 Install with pip (recommended)

To install latest Conan 2 version using `pip`, you need a Python >= 3.8 distribution installed on your machine. Modern Python distros come with pip pre-installed. However, if necessary you can install pip by following the instructions in pip docs.

Install Conan:

```
$ pip install conan
```

Important: Please READ carefully:

- Make sure that your **pip** installation matches your **Python (>= 3.8)** version.
- On **Linux**, you may need **sudo** permissions to install Conan globally.
- We strongly recommend using **virtualenvs** (virtualenvwrapper works great) for everything related to Python (check https://virtualenvwrapper.readthedocs.io/en/stable/, or https://pypi.org/project/virtualenvwrapper-win/ on Windows). With Python 3, the built-in module `venv` can also be used instead (check https://docs.python.org/3/library/venv.html). If not using a **virtualenv** it is possible that conan dependencies will conflict with previously existing dependencies, especially if you are using Python for other purposes.
- On **macOS**, especially the latest versions that may have **System Integrity Protection**, pip may fail. Try using virtualenvs, or install it to the Python user install directory with `$ pip install --user conan`.
- Some Linux distros, such as Linux Mint, require a restart (shell restart, or logout/system if not enough) after installation, so Conan is found in the path.

2.1.1 Known installation issues with pip

When Conan is installed with **pip install --user conan**, a new directory is usually created for it. However, the directory is not appended automatically to the *PATH* and the **conan** commands do not work. This can usually be solved by restarting the session of the terminal or running the following command:

```
$ source ~/.profile
```

2.1.2 Update

If installed via `pip`, your Conan version can be updated with:

```
$ pip install conan --upgrade  # Might need sudo or --user
```

The upgrade shouldn't affect the installed packages or cache information. If the cache becomes inconsistent somehow, you may want to remove its content by deleting it (`<userhome>/.conan2`).

2.2 Install with pipx

In certain scenarios, attempting to install with `pip` may yield the following error:

```
error: externally-managed-environment

x This environment is externally managed
    To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.
...
```

This is because some modern Linux distributions have started marking their Python installations as "externally managed", which means that the system's package manager is responsible for managing Python packages. Installing packages globally or even in the user space can interfere with system operations and potentially break system tools (check PEP-668 for more detailed information).

For those cases, it's recommended to use `pipx` to install Conan. `pipx` creates a virtual environment for each Python application, ensuring that dependencies do not conflict. The advantage is that it isolates Conan and its dependencies from the system Python and avoids potential conflicts with system packages while providing a clean environment for Conan to run.

To install Conan with `pipx`:

1. Ensure `pipx` is installed on your system. If it isn't, check the installation guidelines in the pipx documentation. For Debian-based distributions, you can install `pipx` using the system package manager:

   ```
   $ apt-get install pipx
   $ pipx ensurepath
   ```

   (Note: The package name might vary depending on the distribution)

2. Restart your terminal and then install Conan using `pipx`:

   ```
   $ pipx install conan
   ```

3. Now you can use Conan as you typically would.

2.3 Use a system installer or create a self-contained executable

There are a number of existing installers in Conan downloads for macOS Homebrew, Debian, Windows, Arch Linux, that will not require a Python installation.

We also distribute Conan binaries for Windows, Linux, and macOS in a compressed file that you can uncompress on your system and run directly.

> **Warning:** If you are using **macOS**, please be aware of the Gatekeeper feature that may quarantine the compressed binaries if downloaded directly using a web browser. To avoid this issue, download them using a tool such as *curl*, *wget*, or similar.

If there is no installer for your platform, you can create your own Conan executable with the `pyinstaller.py` utility in the repo. This process is able to create a self-contained Conan executable that contains all it needs, including the Python interpreter, so it wouldn't be necessary to have Python installed in the system.

You can do it with:

```
$ git clone https://github.com/conan-io/conan conan_src
$ cd conan_src
$ pip install -e .
$ python pyinstaller.py
```

It is important to install the dependencies and the project first with `pip install -e .` which configures the project as "editable", that is, to run from the current source folder. After creating the executable, it can be uninstalled with pip.

This has to run on the same platform that will be using the executable, pyinstaller does not cross-build. The resulting executable can be just copied and put in the system PATH of the running machine to be able to run Conan.

2.4 Install from source

You can run Conan directly from source code. First, you need to install Python and pip. Then, clone (or download and unzip) the Conan git repository and install it.

For the latest development version, checkout the *develop2* branch of the repository:

```
# clone folder name matters, to avoid imports issues
$ git clone https://github.com/conan-io/conan.git conan_src
$ cd conan_src
$ python -m pip install -e .
```

Now test your `conan` installation by running:

```
$ conan
```

You should see the Conan commands help.

CHAPTER

THREE

TUTORIAL

The purpose of this section is to guide you through the most important Conan features with practical examples. From using libraries already packaged by Conan, to how to package your libraries and store them on a remote server alongside all the precompiled binaries.

3.1 Consuming packages

This section shows how to build your projects using Conan to manage your dependencies. We will begin with a basic example of a C project that uses CMake and depends on the **zlib** library. This project will use a *conanfile.txt* file to declare its dependencies.

We will also cover how you can not only use 'regular' libraries with Conan but also manage tools you may need to use while building: like CMake, msys2, MinGW, etc.

Then, we will explain different Conan concepts like settings and options and how you can use them to build your projects for different configurations like Debug, Release, with static or shared libraries, etc.

Also, we will explain how to transition from the *conanfile.txt* file we used in the first example to a more powerful *conanfile.py*.

After that, we will introduce the concept of Conan build and host profiles and explain how you can use them to cross-compile your application to different platforms.

Then, in the "Introduction to versioning" we will learn about using different versions, defining requirements with version ranges, the concept of revisions and a brief introduction to lockfiles to achieve reproducibility of the dependency graph.

3.1.1 Build a simple CMake project using Conan

Let's get started with an example: we are going to create a string compressor application that uses one of the most popular C++ libraries: Zlib.

We'll use CMake as build system in this case but keep in mind that Conan **works with any build system** and is not limited to using CMake. You can check more examples with other build systems in the *Read More section*.

Please, first clone the sources to recreate this project, you can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/simple_cmake_project
```

We start from a very simple C language project with this structure:

```
.
├── CMakeLists.txt
└── src
    └── main.c
```

This project contains a basic *CMakeLists.txt* including the **zlib** dependency and the source code for the string compressor program in *main.c*.

Let's have a look at the *main.c* file:

Listing 1: **main.c**

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include <zlib.h>

int main(void) {
    char buffer_in [256] = {"Conan is a MIT-licensed, Open Source package manager for C␣
↪and C++ development "
                            "for C and C++ development, allowing development teams to␣
↪easily and efficiently "
                            "manage their packages and dependencies across platforms and␣
↪build systems."};
    char buffer_out [256] = {0};

    z_stream defstream;
    defstream.zalloc = Z_NULL;
    defstream.zfree = Z_NULL;
    defstream.opaque = Z_NULL;
    defstream.avail_in = (uInt) strlen(buffer_in);
    defstream.next_in = (Bytef *) buffer_in;
    defstream.avail_out = (uInt) sizeof(buffer_out);
    defstream.next_out = (Bytef *) buffer_out;

    deflateInit(&defstream, Z_BEST_COMPRESSION);
    deflate(&defstream, Z_FINISH);
    deflateEnd(&defstream);

    printf("Uncompressed size is: %lu\n", strlen(buffer_in));
    printf("Compressed size is: %lu\n", strlen(buffer_out));

    printf("ZLIB VERSION: %s\n", zlibVersion());

    return EXIT_SUCCESS;
}
```

Also, the contents of *CMakeLists.txt* are:

Listing 2: **CMakeLists.txt**

```
cmake_minimum_required(VERSION 3.15)
project(compressor C)
```

(continues on next page)

(continued from previous page)

```
find_package(ZLIB REQUIRED)

add_executable(${PROJECT_NAME} src/main.c)
target_link_libraries(${PROJECT_NAME} ZLIB::ZLIB)
```

Our application relies on the **Zlib** library. Conan, by default, tries to install libraries from a remote server called ConanCenter. You can search there for libraries and also check the available versions. In our case, after checking the available versions for Zlib we choose to use one of the latest versions: **zlib/1.3.1**.

The easiest way to install the **Zlib** library and find it from our project with Conan is using a *conanfile.txt* file. Let's create one with the following content:

Listing 3: **conanfile.txt**

```
[requires]
zlib/1.3.1

[generators]
CMakeDeps
CMakeToolchain
```

As you can see we added two sections to this file with a syntax similar to an *INI* file.

- The **[requires]** section is where we declare the libraries we want to use in the project, in this case, **zlib/1.3.1**.
- The **[generators]** section tells Conan to generate the files that the compilers or build systems will use to find the dependencies and build the project. In this case, as our project is based on *CMake*, we will use *CMakeDeps* to generate information about where the **Zlib** library files are installed and *CMakeToolchain* to pass build information to *CMake* using a *CMake* toolchain file.

Besides the *conanfile.txt*, we need a **Conan profile** to build our project. Conan profiles allow users to define a configuration set for things like the compiler, build configuration, architecture, shared or static libraries, etc. Conan, by default, will not try to detect a profile automatically, so we need to create one. To let Conan try to guess the profile, based on the current operating system and installed tools, please run:

```
conan profile detect --force
```

This will detect the operating system, build architecture and compiler settings based on the environment. It will also set the build configuration as *Release* by default. The generated profile will be stored in the Conan home folder with name *default* and will be used by Conan in all commands by default unless another profile is specified via the command line. An example of the output of this command for macOS would be:

```
$ conan profile detect --force
Found apple-clang 14.0
apple-clang>=13, using the major as version
Detected profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos
```

Note: A note about the detected C++ standard by Conan

Conan will always set the default C++ standard as the one that the detected compiler version uses by default, except for the case of macOS using apple-clang. In this case, for apple-clang>=11, it sets `compiler.cppstd=gnu17`. If you want to use a different C++ standard, you can edit the default profile file directly. First, get the location of the default profile using:

```
$ conan profile path default
/Users/user/.conan2/profiles/default
```

Then open and edit the file and set `compiler.cppstd` to the C++ standard you want to use.

Note: Using a compiler other than the auto-detected one

If you want to change a Conan profile to use a compiler different from the default one, you need to change the `compiler` setting and also tell Conan explicitly where to find it using the *tools.build:compiler_executables configuration*.

We will use Conan to install **Zlib** and generate the files that CMake needs to find this library and build our project. We will generate those files in the folder *build*. To do that, run:

```
$ conan install . --output-folder=build --build=missing
```

You will get something similar to this as the output of that command:

```
$ conan install . --output-folder=build --build=missing
...
-------- Computing dependency graph ----------
zlib/1.3.1: Not found in local cache, looking in remotes...
zlib/1.3.1: Checking remote: conancenter
zlib/1.3.1: Trying with 'conancenter'...
Downloading conanmanifest.txt
Downloading conanfile.py
Downloading conan_export.tgz
Decompressing conan_export.tgz
zlib/1.3.1: Downloaded recipe revision f1fadf0d3b196dc0332750354ad8ab7b
Graph root
    conanfile.txt: /home/conan/examples2/tutorial/consuming_packages/simple_cmake_
↪project/conanfile.txt
Requirements
    zlib/1.3.1#f1fadf0d3b196dc0332750354ad8ab7b - Downloaded (conancenter)

-------- Computing necessary packages ----------
Requirements
    zlib/1.3.1#f1fadf0d3b196dc0332750354ad8ab7b:cdc9a35e010a17fc90bb845108cf86cfcbce64bf
↪#dd7bf2a1ab4eb5d1943598c09b616121 - Download (conancenter)

-------- Installing packages ----------

Installing (downloading, building) binaries...
zlib/1.3.1: Retrieving package cdc9a35e010a17fc90bb845108cf86cfcbce64bf from remote
↪'conancenter'
Downloading conanmanifest.txt
```

(continues on next page)

(continued from previous page)

```
Downloading conaninfo.txt
Downloading conan_package.tgz
Decompressing conan_package.tgz
zlib/1.3.1: Package installed cdc9a35e010a17fc90bb845108cf86cfcbce64bf
zlib/1.3.1: Downloaded package revision dd7bf2a1ab4eb5d1943598c09b616121

-------- Finalizing install (deploy, generators) ----------
conanfile.txt: Generator 'CMakeToolchain' calling 'generate()'
conanfile.txt: Generator 'CMakeDeps' calling 'generate()'
conanfile.txt: Generating aggregated env files
```

As you can see in the output, there are a couple of things that happened:

- Conan installed the *Zlib* library from the remote server, which should be the Conan Center server by default if the library is available. This server stores both the Conan recipes, which are the files that define how libraries must be built, and the binaries that can be reused so we don't have to build from sources every time.
- Conan generated several files under the **build** folder. Those files were generated by both the `CMakeToolchain` and `CMakeDeps` generators we set in the **conanfile.txt**. `CMakeDeps` generates files so that CMake finds the Zlib library we have just downloaded. On the other side, `CMakeToolchain` generates a toolchain file for CMake so that we can transparently build our project with CMake using the same settings that we detected for our default profile.

Now we are ready to build and run our **compressor** app:

Listing 4: Windows

```
$ cd build
# assuming Visual Studio 15 2017 is your VS version and that it matches your default␣
↪profile
$ cmake .. -G "Visual Studio 15 2017" -DCMAKE_TOOLCHAIN_FILE="conan_toolchain.cmake"
$ cmake --build . --config Release
...
[100%] Built target compressor
$ Release\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
```

Listing 5: Linux, macOS

```
$ cd build
$ cmake .. -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake -DCMAKE_BUILD_TYPE=Release
$ cmake --build .
...
[100%] Built target compressor
$ ./compressor
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
```

Note that `CMakeToolchain` might generate CMake **preset** files, that allows users with a modern CMake (`>=3.23`) to use them with `cmake --preset` instead of passing the toolchain file argument. See *Building with CMake presets*

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 1: Building A Simple CMake Project From Conan
- *Building with CMake presets*
- *Getting started with Autotools*
- *Getting started with Meson*
- *Getting started with Bazel*
- *Getting started with Bazel 7.x*

3.1.2 Using build tools as Conan packages

In the previous example, we built our CMake project and used Conan to install and locate the **Zlib** library. We used the CMake already installed in our system to build our compressor binary. However, what happens if you want to build our project with a specific CMake version, different from the one already installed system-wide? Conan can also help you install these tools and use them to compile consumer projects or other Conan packages. In this case, you can declare this dependency in Conan using a type of requirement named `tool_requires`. Let's see an example of how to add a `tool_requires` to our project and use a different CMake version to build it.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/tool_requires
```

The structure of the project is the same as the one of the previous example:

```
.
├── conanfile.txt
├── CMakeLists.txt
└── src
    └── main.c
```

The main difference is the addition of the *[tool_requires]* section in the **conanfile.txt** file. In this section, we declare that we want to build our application using CMake **v3.27.9**.

Listing 6: **conanfile.txt**

```
[requires]
zlib/1.3.1

[tool_requires]
cmake/3.27.9

[generators]
CMakeDeps
CMakeToolchain
```

Important: Please note that this *conanfile.txt* will install *zlib/1.3.1* and *cmake/3.27.9* separately. However, if Conan does not find a binary for Zlib in Conan Center and it needs to be built from sources, a CMake installation must already be present on your system, because the `cmake/3.27.9` declared in your `conanfile.txt` only applies to your current project, not all dependencies. If you want to use that *cmake/3.27.9* to also build Zlib, when installing if necessary, you may add the `[tool_requires]` section to the profile you are using. Please check *the profile doc* for more information.

We also added a message to the *CMakeLists.txt* to output the CMake version:

Listing 7: **CMakeLists.txt**

```
cmake_minimum_required(VERSION 3.15)
project(compressor C)

find_package(ZLIB REQUIRED)

message("Building with CMake version: ${CMAKE_VERSION}")

add_executable(${PROJECT_NAME} src/main.c)
target_link_libraries(${PROJECT_NAME} ZLIB::ZLIB)
```

Now, as in the previous example, we will use Conan to install **Zlib** and **CMake 3.27.9** and generate the files to find both of them. We will generate those files in the folder *build*. To do that, just run:

```
$ conan install . --output-folder=build --build=missing
```

Note: **PowerShell** users will find the `.ps1` files generated next to the `.bat` (Windows) or `.sh` (Linux, Mac) ones for the local `conanfile`, in any platform (Windows, Linux and Mac). Conan itself always uses the `.bat` files in its subprocesses, unless `--conf=tools.env.virtualenv:powershell=<executable>` (e.g., `powershell.exe` or `pwsh`) is defined, which makes it generate and use the `.ps1` files for all packages. Setting this configuration to `True` or `False` is no longer supported.

To avoid the need to add this line every time, we recommend configuring it in the `[conf]` section of your profile. For detailed information, please refer to the *profiles section*.

You can check the output:

```
-------- Computing dependency graph ----------
cmake/3.27.9: Not found in local cache, looking in remotes...
cmake/3.27.9: Checking remote: conancenter
cmake/3.27.9: Trying with 'conancenter'...
Downloading conanmanifest.txt
Downloading conanfile.py
cmake/3.27.9: Downloaded recipe revision 3e3d8f3a848b2a60afafbe7a0955085a
Graph root
    conanfile.txt: /Users/user/Documents/developer/conan/examples2/tutorial/consuming_
↪packages/tool_requires/conanfile.txt
Requirements
    zlib/1.3.1#f1fadf0d3b196dc0332750354ad8ab7b - Cache
Build requirements
    cmake/3.27.9#3e3d8f3a848b2a60afafbe7a0955085a - Downloaded (conancenter)

-------- Computing necessary packages ----------
Requirements
    zlib/1.3.1#f1fadf0d3b196dc0332750354ad8ab7b:2a823fda5c9d8b4f682cb27c30caf4124c5726c8
↪#48bc7191ec1ee467f1e951033d7d41b2 - Cache
Build requirements
    cmake/3.27.9
↪#3e3d8f3a848b2a60afafbe7a0955085a:f2f48d9745706caf77ea883a5855538256e7f2d4
↪#6c519070f013da19afd56b52c465b596 - Download (conancenter)
```

(continues on next page)

(continued from previous page)

```
-------- Installing packages ----------

Installing (downloading, building) binaries...
cmake/3.27.9: Retrieving package f2f48d9745706caf77ea883a5855538256e7f2d4 from remote
↪'conancenter'
Downloading conanmanifest.txt
Downloading conaninfo.txt
Downloading conan_package.tgz
Decompressing conan_package.tgz
cmake/3.27.9: Package installed f2f48d9745706caf77ea883a5855538256e7f2d4
cmake/3.27.9: Downloaded package revision 6c519070f013da19afd56b52c465b596
zlib/1.3.1: Already installed!

-------- Finalizing install (deploy, generators) ----------
conanfile.txt: Generator 'CMakeToolchain' calling 'generate()'
conanfile.txt: Generator 'CMakeDeps' calling 'generate()'
conanfile.txt: Generating aggregated env files
```

Now, if you check the folder you will see that Conan generated a new file called `conanbuild.{sh,bat}`. This is the result of automatically invoking a `VirtualBuildEnv` generator when we declared the `tool_requires` in the **conanfile.txt**. This file sets some environment variables like a new `PATH` that we can use to inject the location of CMake v3.27.9 into our environment.

Activate the virtual environment, and run `cmake --version` to check that you have installed the new CMake version in the path.

Listing 8: Windows

```
$ cd build
$ conanbuild.bat
# conanbuild.ps1 if using Powershell
```

Listing 9: Linux, macOS

```
$ cd build
$ source conanbuild.sh
Capturing current environment in deactivate_conanbuildenv-release-x86_64.sh
Configuring environment variables
```

Run `cmake` and check the version:

```
$ cmake --version
cmake version 3.27.9
...
```

As you can see, after activating the environment, the CMake v3.27.9 binary folder was added to the path and is the currently active version now. Now you can build your project as you previously did, but this time Conan will use CMake 3.27.9 to build it:

Listing 10: Windows

```
# assuming Visual Studio 15 2017 is your VS version and that it matches your default␣
```

(continues on next page)

(continued from previous page)

```
↪profile
$ cmake .. -G "Visual Studio 15 2017" -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake
$ cmake --build . --config Release
...
Building with CMake version: 3.27.9
...
[100%] Built target compressor
$ Release\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
```

Listing 11: Linux, macOS

```
$ cmake .. -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake -DCMAKE_BUILD_TYPE=Release
$ cmake --build .
...
Building with CMake version: 3.27.9
...
[100%] Built target compressor
$ ./compressor
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
```

Note that when we activated the environment, a new file named `deactivate_conanbuild.{sh,bat}` was created in the same folder. If you source this file you can restore the environment as it was before.

Listing 12: Windows

```
$ deactivate_conanbuild.bat
```

Listing 13: Linux, macOS

```
$ source deactivate_conanbuild.sh
Restoring environment
```

Run `cmake` and check the version, it will be the version that was installed previous to the environment activation:

```
$ cmake --version
cmake version 3.22.0
...
```

Note: Best practice

`tool_requires` and tool packages are intended for executable applications, like `cmake` or `ninja`. Do not use `tool_requires` to depend on library or library-like dependencies.

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 4: Using Build Tools As Conan Packages
- *Using [platform_tool_requires] in your profiles*.

- *Creating recipes for tool_requires: packaging build tools*.
- *Using the same requirement as a requires and as a tool_requires*
- *Using a MinGW as tool_requires to build with gcc in Windows*
- *Using tool_requires in profiles*
- Using conf to set a toolchain from a tool requires

3.1.3 Building for multiple configurations: Release, Debug, Static and Shared

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/different_configurations
```

So far, we built a simple CMake project that depended on the **zlib** library and learned about `tool_requires`, a special type of `requirements` for build-tools like CMake. In both cases, we did not specify anywhere that we wanted to build the application in *Release* or *Debug* mode, or if we wanted to link against *static* or *shared* libraries. That is because Conan, if not instructed otherwise, will use a default configuration declared in the 'default profile'. This default profile was created in the first example when we run the **`conan profile detect`** command. Conan stores this file in the **/profiles** folder, located in the Conan user home. You can check the contents of your default profile by running the **`conan config home`** command to get the location of the Conan user home and then showing the contents of the default profile in the **/profiles** folder:

```
$ conan config home
Current Conan home: /Users/tutorial_user/.conan2

# output the file contents
$ cat /Users/tutorial_user/.conan2/profiles/default
[settings]
os=Macos
arch=x86_64
compiler=apple-clang
compiler.version=14.0
compiler.libcxx=libc++
compiler.cppstd=gnu11
build_type=Release
[options]
[tool_requires]
[env]

# The default profile can also be checked with the command "conan profile show"
```

As you can see, the profile has different sections. The `[settings]` section is the one that has information about things like the operating system, architecture, compiler, and build configuration.

When you call a Conan command setting the `--profile` argument, Conan will take all the information from the profile and apply it to the packages you want to build or install. If you don't specify that argument it's equivalent to call it with `--profile=default`. These two commands will behave the same:

```
$ conan install . --build=missing
$ conan install . --build=missing --profile=default
```

You can store different profiles and use them to build for different settings. For example, to use a `build_type=Debug`, or adding a `tool_requires` to all the packages you build with that profile. We will create a *debug* profile to try building with different configurations:

Listing 14: <conan home>/profiles/debug

```
[settings]
os=Macos
arch=x86_64
compiler=apple-clang
compiler.version=14.0
compiler.libcxx=libc++
compiler.cppstd=gnu11
build_type=Debug
```

Modifying settings: use Debug configuration for the application and its dependencies

Using profiles is not the only way to set the configuration you want to use. You can also override the profile settings in the Conan command using the `--settings` argument. For example, you can build the project from the previous examples in *Debug* configuration instead of *Release*.

Before building, please check that we modified the source code from the previous example to show the build configuration the sources were built with:

```
#include <stdlib.h>
...

int main(void) {
    ...
    #ifdef NDEBUG
    printf("Release configuration!\n");
    #else
    printf("Debug configuration!\n");
    #endif

    return EXIT_SUCCESS;
}
```

Now let's build our project for *Debug* configuration:

```
$ conan install . --output-folder=build --build=missing --settings=build_type=Debug
```

As we explained above, this is the equivalent of having *debug* profile and running these command using the `--profile=debug` argument instead of the `--settings=build_type=Debug` argument.

This **`conan install`** command will check if we already have the required libraries in the local cache (Zlib) for Debug configuration and obtain them if not. It will also update the build configuration in the `conan_toolchain.cmake` and `CMakePresets.json` files that the `CMakeToolchain` generator creates so that when we build the application it's built in *Debug* configuration. Now build your project as you did in the previous examples and check in the output how it was built in *Debug* configuration:

Listing 15: Windows

```
# assuming Visual Studio 15 2017 is your VS version and that it matches your default␣
↪profile
$ cd build
$ cmake .. -G "Visual Studio 15 2017" -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake
$ cmake --build . --config Debug
$ Debug\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
Debug configuration!
```

Listing 16: Linux, macOS

```
$ cd build
$ cmake .. -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake -DCMAKE_BUILD_TYPE=Debug
$ cmake --build .
$ ./compressor
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
Debug configuration!
```

Modifying options: linking the application dependencies as shared libraries

So far, we have been linking *Zlib* statically in our application. That's because in the Zlib's Conan package there's an attribute set to build in that mode by default. We can change from **static** to **shared** linking by setting the `shared` option to `True` using the `--options` argument. To do so, please run:

```
$ conan install . --output-folder=build --build=missing --options=zlib/1.3.1:shared=True
```

Doing this, Conan will install the *Zlib* shared libraries, generate the files to build with them and, also the necessary files to locate those dynamic libraries when running the application.

Note: Options are defined per-package. In this case we were defining that we wanted that specific version of zlib/1.3.1 as a shared library. If we had other dependencies and we want all of our dependencies (whenever possible) as shared libraries, we would use `-o *:shared=True`, with the * pattern that matches all package references.

Let's build the application again after configuring it to link *Zlib* as a shared library:

Listing 17: Windows

```
$ cd build
# assuming Visual Studio 15 2017 is your VS version and that it matches your default␣
↪profile
$ cmake .. -G "Visual Studio 15 2017" -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake
$ cmake --build . --config Release
...
[100%] Built target compressor
```

Listing 18: Linux, macOS

```
$ cd build
$ cmake .. -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake -DCMAKE_BUILD_TYPE=Release
$ cmake --build .
...
[100%] Built target compressor
```

Now, if you try to run the compiled executable you will see an error because the executable can't find the shared libraries for *Zlib* that we just installed.

Listing 19: Windows

```
$ Release\compressor.exe
(on a pop-up window) The code execution cannot proceed because zlib1.dll was not found.␣
↪Reinstalling the program may fix this problem.
# This error depends on the console being used and may not always pop up.
# It could run correctly if the console gets the zlib dll from a different path.
```

Listing 20: Linux

```
$ ./compressor
./compressor: error while loading shared libraries: libz.so.1: cannot open shared object␣
↪file: No such file or directory
```

Listing 21: Macos

```
$ ./compressor
./compressor: dyld[41259]: Library not loaded: @rpath/libz.1.dylib
```

This is because shared libraries (*.dll* in windows, *.dylib* in macOS and *.so* in Linux), are loaded at runtime. That means that the application executable needs to know where the required shared libraries are when it runs. On Windows, the dynamic linker will search in the same directory, then in the *PATH* directories. On macOS, it will search in the directories declared in *DYLD_LIBRARY_PATH*, and on Linux it will use *LD_LIBRARY_PATH*.

Conan provides a mechanism to define those variables and make it possible, for executables, to find and load these shared libraries. This mechanism is the `VirtualRunEnv` generator. If you check the output folder, you will see that Conan generated a new file called `conanrun.{sh,bat}`. This is the result of automatically invoking that `VirtualRunEnv` generator when we activated the `shared` option when doing the **`conan install`**. This generated script will set the *PATH*, *LD_LIBRARY_PATH*, *DYLD_LIBRARY_PATH* and *DYLD_FRAMEWORK_PATH* environment variables so that executables can find the shared libraries.

Activate the virtual environment, and run the executables again:

Listing 22: Windows

```
$ conanrun.bat
$ Release\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
...
```

Listing 23: Linux, macOS

```
$ source conanrun.sh
$ ./compressor
Uncompressed size is: 233
Compressed size is: 147
...
```

Just as in the previous example with the `VirtualBuildEnv` generator, when we run the `conanrun.{sh,bat}` script, a deactivation script called `deactivate_conanrun.{sh,bat}` is created to restore the environment. Source or run it to do so:

Listing 24: Windows

```
$ deactivate_conanrun.bat
```

Listing 25: Linux, macOS

```
$ source deactivate_conanrun.sh
```

Difference between settings and options

You may have noticed that for changing between *Debug* and *Release* configuration we used a Conan **setting**, but when we set *shared* mode for our executable we used a Conan **option**. Please note the difference between **settings** and **options**:

- **settings** are typically a project-wide configuration defined by the client machine. Things like the operating system, compiler or build configuration that will be common to several Conan packages and would not make sense to define one default value for only one of them. For example, it doesn't make sense for a Conan package to declare "Visual Studio" as a default compiler because that is something defined by the end consumer, and unlikely to make sense if they are working in Linux.
- **options** are intended for package-specific configuration that can be set to a default value in the recipe. For example, one package can define that its default linkage is static, and this is the linkage that should be used if consumers don't specify otherwise.

Introducing the concept of Package ID

When consuming packages like Zlib with different *settings* and *options*, you might wonder how Conan determines which binary to retrieve from the remote. The answer lies in the concept of the *package_id*.

The *package_id* is an identifier that Conan uses to determine the binary compatibility of packages. It is computed based on several factors, including the package's *settings*, *options*, and dependencies. When you modify any of these factors, Conan computes a new *package_id* to reference the corresponding binary.

Here's a breakdown of the process:

1. **Determine Settings and Options**: Conan first retrieves the user's input settings and options. These can come from the command line or profiles like *–settings=build_type=Debug* or *–profile=debug*.
2. **Compute the Package ID**: With the current values for *settings*, *options*, and dependencies, Conan computes a hash. This hash serves as the *package_id*, representing the binary package's unique identity.

3. **Fetch the Binary**: Conan then checks its cache or the specified remote for a binary package with the computed *package_id*. If it finds a match, it retrieves that binary. If not, Conan can build the package from source or indicate that the binary is missing.

In the context of our tutorial, when we consumed Zlib with different *settings* and *options*, Conan used the *package_id* to ensure that it fetched the correct binary that matched our specified configuration.

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 2: Building For Multiple Configurations With Conan And CMake Presets
- *VirtualRunEnv reference*
- *Cross-compiling using –profile:build and –profile:host*
- *Conan packages binary compatibility: the package ID*
- *Installing configurations with conan config install*
- VS Multi-config
- *How settings and options influence the package id*
- *Using patterns for settings and options*

3.1.4 Understanding the flexibility of using conanfile.py vs conanfile.txt

In the previous examples, we declared our dependencies (*Zlib* and *CMake*) in a *conanfile.txt* file. Let's have a look at that file:

Listing 26: **conanfile.txt**

```
[requires]
zlib/1.3.1

[tool_requires]
cmake/3.27.9

[generators]
CMakeDeps
CMakeToolchain
```

Using a *conanfile.txt* to build your projects using Conan is enough for simple cases, but if you need more flexibility you should use a *conanfile.py* file where you can use Python code to make things such as adding requirements dynamically, changing options depending on other options or setting options for your requirements. Let's see an example on how to migrate to a *conanfile.py* and use some of those features.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/conanfile_py
```

Check the contents of the folder and note that the contents are the same as in the previous examples but with a *conanfile.py* instead of a *conanfile.txt*.

```
.
├── CMakeLists.txt
├── conanfile.py
```

(continues on next page)

(continued from previous page)

```
└── src
    └── main.c
```

Remember that in the previous examples the *conanfile.txt* had this information:

Listing 27: **conanfile.txt**

```
[requires]
zlib/1.3.1

[tool_requires]
cmake/3.27.9

[generators]
CMakeDeps
CMakeToolchain
```

We will translate that same information to a *conanfile.py*. This file is what is typically called a **"Conan recipe"**. It can be used for consuming packages, like in this case, and also to create packages. For our current case, it will define our requirements (both libraries and build tools) and logic to modify options and set how we want to consume those packages. In the case of using this file to create packages, it can define (among other things) how to download the package's source code, how to build the binaries from those sources, how to package the binaries, and information for future consumers on how to consume the package. We will explain how to use Conan recipes to create packages in the *Creating Packages* section later.

The equivalent of the *conanfile.txt* in the form of a Conan recipe could look like this:

Listing 28: **conanfile.py**

```
from conan import ConanFile


class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/1.3.1")

    def build_requirements(self):
        self.tool_requires("cmake/3.27.9")
```

To create the Conan recipe, we declared a new class that inherits from the `ConanFile` class. This class has different class attributes and methods:

- The **settings** class attribute defines the project-wide variables, like the compiler, its version, or the OS itself that may change when we build our project. This is related to how Conan manages binary compatibility as these values will affect the value of the **package ID** for Conan packages. We will explain how Conan uses this value to manage binary compatibility later.
- The **generators** class attribute specifies which Conan generators will be run when we call the **`conan install`** command. In this case, we added **CMakeToolchain** and **CMakeDeps** as in the *conanfile.txt*.
- In the **requirements()** method, we use the `self.requires()` method to declare the *zlib/1.3.1* dependency.
- In the **build_requirements()** method, we use the `self.tool_requires()` method to declare the *cmake/3.27.9*

dependency.

Note: It's not strictly necessary to add the dependencies to the tools in `build_requirements()`, as in theory everything within this method could be done in the `requirements()` method. However, `build_requirements()` provides a dedicated place to define `tool_requires` and `test_requires`, which helps in keeping the structure organized and clear. For more information, please check the *requirements()* and *build_requirements()* docs.

You can check that running the same commands as in the previous examples will lead to the same results as before.

Listing 29: Windows

```
$ conan install . --output-folder=build --build=missing
$ cd build
$ conanbuild.bat
# assuming Visual Studio 15 2017 is your VS version and that it matches your default␣
↪profile
$ cmake .. -G "Visual Studio 15 2017" -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake
$ cmake --build . --config Release
...
Building with CMake version: 3.27.9
...
[100%] Built target compressor

$ Release\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
$ deactivate_conanbuild.bat
```

Listing 30: Linux, macOS

```
$ conan install . --output-folder build --build=missing
$ cd build
$ source conanbuild.sh
Capturing current environment in deactivate_conanbuildenv-release-x86_64.sh
Configuring environment variables
$ cmake .. -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake -DCMAKE_BUILD_TYPE=Release
$ cmake --build .
...
Building with CMake version: 3.27.9
...
[100%] Built target compressor

$ ./compressor
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
$ source deactivate_conanbuild.sh
```

So far, we have achieved the same functionality we had using a *conanfile.txt*. Let's see how we can take advantage of the capabilities of the *conanfile.py* to define the project structure we want to follow and also to add some logic using Conan settings and options.

Use the layout() method

In the previous examples, every time we executed a *conan install* command, we had to use the *–output-folder* argument to define where we wanted to create the files that Conan generates. There's a neater way to decide where we want Conan to generate the files for the build system that will allow us to decide, for example, if we want different output folders depending on the type of CMake generator we are using. You can define this directly in the *conanfile.py* inside the *layout()* method and make it work for every platform without adding more changes.

Listing 31: **conanfile.py**

```
import os

from conan import ConanFile


class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/1.3.1")
        if self.settings.os == "Windows":
            self.requires("base64/0.4.0")

    def build_requirements(self):
        if self.settings.os != "Windows":
            self.tool_requires("cmake/3.27.9")

    def layout(self):
        # We make the assumption that if the compiler is msvc the
        # CMake generator is multi-config
        multi = True if self.settings.get_safe("compiler") == "msvc" else False
        if multi:
            self.folders.generators = os.path.join("build", "generators")
            self.folders.build = "build"
        else:
            self.folders.generators = os.path.join("build", str(self.settings.build_
↪type), "generators")
            self.folders.build = os.path.join("build", str(self.settings.build_type))
```

As you can see, we defined the **self.folders.generators** attribute in the *layout()* method. This is the folder where all the auxiliary files generated by Conan (CMake toolchain and cmake dependencies files) will be placed.

Note that the definitions of the folders is different if it is a multi-config generator (like Visual Studio), or a single-config generator (like Unix Makefiles). In the first case, the folder is the same irrespective of the build type, and the build system will manage the different build types inside that folder. But single-config generators like Unix Makefiles must use a different folder for each configuration (as a different build_type Release/Debug). In this case we added a simple logic to consider multi-config if the compiler name is *msvc*.

Check that running the same commands as in the previous examples without the *–output-folder* argument will lead to the same results as before:

Listing 32: Windows

```
$ conan install . --build=missing
$ cd build
$ generators\conanbuild.bat
# assuming Visual Studio 15 2017 is your VS version and that it matches your default␣
↪profile
$ cmake .. -G "Visual Studio 15 2017" -DCMAKE_TOOLCHAIN_FILE=generators\conan_toolchain.
↪cmake
$ cmake --build . --config Release
...
Building with CMake version: 3.27.9
...
[100%] Built target compressor

$ Release\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
$ generators\deactivate_conanbuild.bat
```

Listing 33: Linux, macOS

```
$ conan install . --build=missing
$ cd build/Release
$ source ./generators/conanbuild.sh
Capturing current environment in deactivate_conanbuildenv-release-x86_64.sh
Configuring environment variables
$ cmake ../.. -DCMAKE_TOOLCHAIN_FILE=generators/conan_toolchain.cmake -DCMAKE_BUILD_
↪TYPE=Release
$ cmake --build .
...
Building with CMake version: 3.27.9
...
[100%] Built target compressor

$ ./compressor
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.3.1
$ source ./generators/deactivate_conanbuild.sh
```

There's no need to always write this logic in the *conanfile.py*. There are some pre-defined layouts you can import and directly use in your recipe. For example, for the CMake case, there's a *cmake_layout()* already defined in Conan:

Listing 34: **conanfile.py**

```
from conan import ConanFile
from conan.tools.cmake import cmake_layout


class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
```

(continues on next page)

(continued from previous page)

```
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/1.3.1")

    def build_requirements(self):
        self.tool_requires("cmake/3.27.9")

    def layout(self):
        cmake_layout(self)
```

Use the validate() method to raise an error for non-supported configurations

The *validate() method* is evaluated when Conan loads the *conanfile.py* and you can use it to perform checks of the input settings. If, for example, your project does not support *armv8* architecture on macOS, you can raise the *ConanInvalidConfiguration* exception to make Conan return with a special error code. This will indicate that the configuration used for settings or options is not supported.

Listing 35: **conanfile.py**

```
...
from conan.errors import ConanInvalidConfiguration

class CompressorRecipe(ConanFile):
    ...

    def validate(self):
        if self.settings.os == "Macos" and self.settings.arch == "armv8":
            raise ConanInvalidConfiguration("ARM v8 not supported in Macos")
```

Conditional requirements using a conanfile.py

You could add some logic to the *requirements() method* to add or remove requirements conditionally. Imagine, for example, that you want to add an additional dependency on Windows or that you want to use the system's CMake installation instead of using the Conan *tool_requires*:

Listing 36: **conanfile.py**

```
from conan import ConanFile


class CompressorRecipe(ConanFile):
    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/1.3.1")

        # Add base64 dependency for Windows
```

(continues on next page)

(continued from previous page)

```
        if self.settings.os == "Windows":
            self.requires("base64/0.4.0")

    def build_requirements(self):
        # Use the system's CMake for Windows
        if self.settings.os != "Windows":
            self.tool_requires("cmake/3.27.9")
```

Use the generate() method to copy resources from packages

In some scenarios, Conan packages include files that are useful or even necessary for the consumption of the libraries they package. These files can range from configuration files, assets, to specific files required for the project to build or run correctly. Using the *generate() method* you can copy these files from the Conan cache to your project's folder, ensuring that all required resources are directly available for use.

Here's an example that shows how to copy all resources from a dependency's `resdirs` directory to an `assets` directory within your project:

```
import os
from conan import ConanFile
from conan.tools.files import copy

class MyProject(ConanFile):

    ...

    def generate(self):
        # Copy all resources from the dependency's resource directory
        # to the "assets" folder in the source directory of your project
        dep = self.dependencies["dep_name"]
        copy(self, "*", dep.cpp_info.resdirs[0], os.path.join(self.source_folder, "assets
↪"))
```

Then, after the `conan install` step, all those resource files will be copied locally, allowing you to use them in your project's build process. For a complete example of how to import files from a package in the `generate()` method, you can refer to the blog post about using the Dear ImGui library, which demonstrates how to import bindings for the library depending on the graphics API.

Note: It's important to clarify that copying libraries, whether static or shared, is not necessary. Conan is designed to use the libraries from their locations in the Conan local cache using *generators* and *environment tools* without the need to copy them to the local folder.

Use the build() method and `conan build` command

If you have a recipe that implements a `build()` method, then it is possible to automatically call the full `conan install` + `cmake <configure>` + `cmake <build>` (or call the build system that the `build()` method uses) flow with a single command. Though this might not be the typical developer flow, it can be a convenient shortcut in some cases.

Let's add this `build()` method to our recipe:

```
from conan import ConanFile
from conan.tools.cmake import CMake

class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    ...

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
```

So now we can just call `conan build .`:

```
$ conan build .
...
Graph root
    conanfile.py: ...\conanfile.py
Requirements
    zlib/1.3.1#bfceb3f8904b735f75c2b0df5713b1e6 - Downloaded (conancenter)
Build requirements
    cmake/3.27.9#32cced101c6df0fab43e8d00bd2483eb - Downloaded (conancenter)

======== Calling build() ========
conanfile.py: Calling build()
conanfile.py: Running CMake.configure()
conanfile.py: RUN: cmake -G "Visual Studio 17 2022" -DCMAKE_TOOLCHAIN_FILE="conan_
↪toolchain.cmake"
...
conanfile.py: Running CMake.build()
conanfile.py: RUN: cmake --build "...\conanfile_py" --config Release
```

And we will see how it manages first to install the dependencies, then it will be calling the `build()` method, that calls CMake configure and build step for us. Because the `conan build .` does internally a `conan install`, it can receive the same arguments (profile, settings, options, lockfile, etc.) as `conan install`.

Note: Best practices

The `conan build` command does not intend to replace or change the typical developer flow using CMake and other build tools, and using their IDEs. It is just a convenient shortcut for cases in which we want to locally build a project easily without having to type several commands or use the IDE.

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 3: The Flexibility Of Using A conanfile.py

- *Using "cmake_layout" + "CMakeToolchain" + "CMakePresets feature" to build your project*.
- *Understanding the Conan Package layout*.
- *Documentation for all conanfile.py available methods*.

3.1.5 How to cross-compile your applications using Conan: host and build contexts

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/cross_building
```

In the previous examples, we learned how to use a *conanfile.py* or *conanfile.txt* to build an application that compresses strings using the *Zlib* and *CMake* Conan packages. Also, we explained that you can set information like the operating system, compiler or build configuration in a file called the Conan profile. You can use that profile as an argument (**`--profile`**) to invoke the **`conan install`** command. We also explained that not specifying that profile is equivalent to using the **`--profile=default`** argument.

For all those examples, we used the same platform for building and running the application. But, what if you want to build the application on your machine running Ubuntu Linux and then run it on another platform like a Raspberry Pi? Conan can model that case using two different profiles, one for the machine that **builds** the application (Ubuntu Linux) and another for the machine that **runs** the application (Raspberry Pi). We will explain this "two profiles" approach in the next section.

Conan two profiles model: build and host profiles

Even if you specify only one **`--profile`** argument when invoking Conan, Conan will internally use two profiles. One for the machine that **builds** the binaries (called the **build** profile) and another for the machine that **runs** those binaries (called the **host** profile). Calling this command:

```
$ conan install . --build=missing --profile=someprofile
```

Is equivalent to:

```
$ conan install . --build=missing --profile:host=someprofile --profile:build=default
```

As you can see we used two new arguments:

- `profile:host`: This is the profile that defines the platform where the built binaries will run. For our string compressor application, this profile would be the one applied for the *Zlib* library that will run on a **Raspberry Pi**.
- `profile:build`: This is the profile that defines the platform where the binaries will be built. For our string compressor application, this profile would be the one used by the *CMake* tool that will compile it on the **Ubuntu Linux** machine.

Note that when you just use one argument for the profile `--profile` is equivalent to `--profile:host`. If you don't specify the `--profile:build` argument, Conan will use the *default* profile internally.

So, if we want to build the compressor application on the Ubuntu Linux machine but run it on a Raspberry Pi, we should use two different profiles. For the **build** machine, we could use the default profile that in our case looks like this:

Listing 37: <conan home>/profiles/default

```
[settings]
os=Linux
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=12
```

And the profile for the Raspberry Pi that is the **host** machine:

Listing 38: <local folder>/profiles/raspberry

```
[settings]
os=Linux
arch=armv7hf
compiler=gcc
build_type=Release
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=12
[buildenv]
CC=arm-linux-gnueabihf-gcc-12
CXX=arm-linux-gnueabihf-g++-12
LD=arm-linux-gnueabihf-ld
```

Important: Please, take into account that in order to build this example successfully, you should have installed a toolchain that includes the compiler and all the tools to build the application for the proper architecture. In this case, the host machine is a Raspberry Pi 3 with *armv7hf* architecture operating system and we have the *arm-linux-gnueabihf* toolchain installed on the Ubuntu machine.

If you have a look at the *raspberry* profile, there is a section named `[buildenv]`. This section is used to set the environment variables that are needed to build the application. In this case, we declare the `CC`, `CXX` and `LD` variables pointing to the cross-build toolchain compilers and linker, respectively. Adding this section to the profile will invoke the *VirtualBuildEnv* generator everytime we do a **`conan install`**. This generator will add that environment information to the `conanbuild.sh` script that we will source before building with CMake so that it can use the cross-build toolchain.

Note: In some cases, you don't have the toolchain available on the build platform. For those cases, you can use a Conan package for the cross-compiler and add it to the `[tool_requires]` section of the profile. For an example of cross-building using a toolchain package, please check *this example*.

Build and host contexts

Now that we have our two profiles prepared, let's have a look at our *conanfile.py*:

Listing 39: **conanfile.py**

```
from conan import ConanFile
from conan.tools.cmake import cmake_layout

class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/1.3.1")

    def build_requirements(self):
        self.tool_requires("cmake/3.27.9")

    def layout(self):
        cmake_layout(self)
```

As you can see, this is practically the same *conanfile.py* we used in the *previous example*. We will require **zlib/1.3.1** as a regular dependency and **cmake/3.27.9** as a tool needed for building the application.

We will need the application to build for the Raspberry Pi with the cross-build toolchain and also link the **zlib/1.3.1** library built for the same platform. On the other side, we need the **cmake/3.27.9** binary to run on Ubuntu Linux. Conan manages this internally in the dependency graph differentiating between what we call the "build context" and the "host context":

- The **host context** is populated with the root package (the one specified in the **`conan install`** or **`conan create`** command) and all its requirements added via `self.requires()`. In this case, this includes the compressor application and the **zlib/1.3.1** dependency.
- The **build context** contains the tool requirements used on the build machine. This category typically includes all the developer tools like CMake, compilers and linkers. In this case, this includes the **cmake/3.27.9** tool.

These contexts define how Conan will manage each of the dependencies. For example, as **zlib/1.3.1** belongs to the **host context**, the `[buildenv]` build environment we defined in the **raspberry** profile (profile host) will only apply to the **zlib/1.3.1** library when building and won't affect anything that belongs to the **build context** like the **cmake/3.27.9** dependency.

Now, let's build the application. First, call **`conan install`** with the profiles for the build and host platforms. This will install the **zlib/1.3.1** dependency built for the *armv7hf* architecture and a **cmake/3.27.9** version that runs on a 64-bit architecture.

```
$ conan install . --build missing -pr:b=default -pr:h=./profiles/raspberry
```

Then, let's call CMake to build the application. As we did in the previous example, we have to activate the **build environment** running `source Release/generators/conanbuild.sh`. That will set the environment variables needed to locate the cross-build toolchain and build the application.

```
$ cd build
$ source Release/generators/conanbuild.sh
Capturing current environment in deactivate_conanbuildenv-release-armv7hf.sh
Configuring environment variables
```

(continues on next page)

(continued from previous page)

```
$ cmake .. -DCMAKE_TOOLCHAIN_FILE=Release/generators/conan_toolchain.cmake -DCMAKE_BUILD_
↪TYPE=Release
$ cmake --build .
...
-- Conan toolchain: C++ Standard 17 with extensions ON
-- The C compiler identification is GNU 12.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/arm-linux-gnueabihf-gcc-12 - skipped
-- Detecting C compile features
-- Detecting C compile features - done    [100%] Built target compressor
...
$ source Release/generators/deactivate_conanbuild.sh
```

You could check that we built the application for the correct architecture by running the `file` Linux utility:

```
$ file compressor
compressor: ELF 32-bit LSB shared object, ARM, EABI5 version 1 (SYSV), dynamically
linked, interpreter /lib/ld-linux-armhf.so.3,
BuildID[sha1]=2a216076864a1b1f30211debf297ac37a9195196, for GNU/Linux 3.2.0, not
stripped
```

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 5: Cross-Compiling Your Applications With Conan
- *Creating a Conan package for a toolchain*
- *Cross building to Android with the NDK*
- *VirtualBuildEnv reference*
- Cross-build using a tool_requires
- *How to require test frameworks like gtest: using test_requires*
- Using Conan to build for iOS

3.1.6 Introduction to versioning

So far we have been using requires with fixed versions like `requires = "zlib/1.2.12"`. But sometimes dependencies evolve, new versions are released and consumers want to update to those versions as easily as possible.

It is always possible to edit the `conanfiles` and explicitly update the versions to the new ones, but there are mechanisms in Conan to allow such updates without even modifying the recipes.

Version ranges

A `requires` can express a dependency to a certain range of versions for a given package, with the syntax `pkgname/[version-range-expression]`. Let's see an example. Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/versioning
```

We can see that we have there:

Listing 40: **conanfile.py**

```
from conan import ConanFile


class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/[~1.2]")
```

That `requires` contains the expression `zlib/[~1.2]`, which means "approximately" `1.2` version. That means, it can resolve to any `zlib/1.2.8`, `zlib/1.2.11` or `zlib/1.2.12`, but it will not resolve to something like `zlib/1.3.0`. Among the available matching versions, a version range will always pick the latest one.

If we do a **`conan install`**, we would see something like:

```
$ conan install .

Graph root
    conanfile.py: .../conanfile.py
Requirements
    zlib/1.2.12#87a7211557b6690ef5bf7fc599dd8349 - Downloaded
Resolved version ranges
    zlib/[~1.2]: zlib/1.2.12
```

If we tried instead to use `zlib/[<1.2.12]`, that means that we would like to use a version lower than `1.2.12`, but that one is excluded, so the latest one to satisfy the range would be `zlib/1.2.11`:

```
$ conan install .

Resolved version ranges
    zlib/[<1.2.12]: zlib/1.2.11
```

The same applies to other type of requirements, like `tool_requires`. Let's add one to the recipe:

Listing 41: **conanfile.py**

```
from conan import ConanFile


class CompressorRecipe(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def requirements(self):
        self.requires("zlib/[~1.2]")

    def build_requirements(self):
        self.tool_requires("cmake/[>3.10]")
```

We see it resolved to the latest available CMake package, with at least version `3.11`:

```
$ conan install .
...
Graph root
    conanfile.py: .../conanfile.py
Requirements
    zlib/1.2.12#87a7211557b6690ef5bf7fc599dd8349 - Cache
Build requirements
    cmake/3.27.9#f305019023c2db74d1001c5afa5cf362 - Downloaded
Resolved version ranges
    cmake/[>3.10]: cmake/3.27.9
    zlib/[~1.2]: zlib/1.2.12
```

Revisions

What happens when a package creator does some change to the package recipe or to the source code, but they don't bump the `version` to reflect those changes? Conan has an internal mechanism to keep track of those modifications, and it is called **revisions**.

The recipe revision is the hash that can be seen together with the package name and version in the form `pkgname/version#recipe_revision` or `pkgname/version@user/channel#recipe_revision`. The recipe revision is a hash of the contents of the recipe and the source code. So if something changes either in the recipe, its associated files or in the source code that this recipe is packaging, it will create a new recipe revision.

You can list existing revisions with the **`conan list`** command:

```
$ conan list "zlib/1.2.12#*" -r=conancenter
conancenter
  zlib
    zlib/1.2.12
      revisions
        82202701ea360c0863f1db5008067122 (2022-03-29 15:47:45 UTC)
        bd533fb124387a214816ab72c8d1df28 (2022-05-09 06:59:58 UTC)
        3b9e037ae1c615d045a06c67d88491ae (2022-05-13 13:55:39 UTC)
        ...
```

Revisions always resolve to the latest (chronological order of creation or upload to the server) revision. Though it is not a common practice, it is possible to explicitly pin a given recipe revision directly in the `conanfile`, like:

```
def requirements(self):
    self.requires("zlib/1.2.12#87a7211557b6690ef5bf7fc599dd8349")
```

This mechanism can, however, be tedious to maintain and update when new revisions are created, so probably in the general case, this shouldn't be done.

Lockfiles

The usage of version ranges, and the possibility of creating new revisions of a given package without bumping the version allows to do fast, automatic and convenient updates, without the need to edit recipes.

But in some occasions, there is also a need to provide an immutable and reproducible set of dependencies. This process is known as "locking", and the mechanism to allow it is "lockfile" files. A lockfile is a file that contains a fixed list of dependencies, specifying the exact version and exact revision. So, for example, a lockfile will never contain a version range with an expression, but only pinned dependencies.

A lockfile can be seen as a snapshot of a given dependency graph at some point in time. Such snapshot must be "realizable", that is, it needs to be a state that can be actually reproduced from the conanfile recipes. And this lockfile can be used at a later point in time to force that same state, even if there are new created package versions.

Let's see lockfiles in action. First, let's pin the dependency to `zlib/1.2.11` in our example:

```
def requirements(self):
    self.requires("zlib/1.2.11")
```

Then, let's capture a lockfile:

```
conan lock create .

-------- Computing dependency graph ----------
Graph root
    conanfile.py: .../conanfile.py
Requirements
    zlib/1.2.11#4524fcdd41f33e8df88ece6e755a5dcc - Cache

Generated lockfile: .../conan.lock
```

Let's see what the lockfile `conan.lock` contains:

```
{
    "version": "0.5",
    "requires": [
        "zlib/1.2.11#4524fcdd41f33e8df88ece6e755a5dcc%1650538915.154"
    ],
    "build_requires": [],
    "python_requires": []
}
```

Now, let's restore the original `requires` version range:

```
def requirements(self):
    self.requires("zlib/[~1.2]")
```

And run **`conan install .`**, which by default will find the `conan.lock`, and run the equivalent **`conan install . --lockfile=conan.lock`**

```
conan install .

Graph root
    conanfile.py: .../conanfile.py
Requirements
    zlib/1.2.11#4524fcdd41f33e8df88ece6e755a5dcc - Cache
```

Note how the version range is no longer resolved, and it doesn't get the `zlib/1.2.12` dependency, even if it is the allowed range `zlib/[~1.2]`, because the `conan.lock` lockfile is forcing it to stay in `zlib/1.2.11` and that exact revision too.

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 6: Intro To Versioning
- *Introduction to Versioning*

Note: The Conan 2 Essentials training course is available for free at the JFrog Academy, which covers the same topics as this documentation but in a more interactive way. You can access it here.

3.2 Creating packages

This section shows how to create Conan packages using a Conan recipe. We begin by creating a basic Conan recipe to package a simple C++ library that you can scaffold using the **`conan new`** command. Then, we will explain the different methods that you can define inside a Conan recipe and the things you can do inside them:

- Using the `source()` method to retrieve sources from external repositories and apply patches to those sources.
- Add requirements to your Conan packages inside the `requirements()` method.
- Use the `generate()` method to prepare the package build, and customize the toolchain.
- Configure settings and options in the `configure()` and `config_options()` methods and how they affect the packages' binary compatibility.
- Use the `build()` method to customize the build process and launch the tests for the library you are packaging.
- Select which files will be included in the Conan package using the `package()` method.
- Define the package information in the `package_info()` method so that consumers of this package can use it.
- Use a *test_package* to test that the Conan package can be consumed correctly.

After this walkthrough around some Conan recipe methods, we will explain some peculiarities of different types of Conan packages like, for example, header-only libraries, packages for pre-built binaries, packaging tools for building other packages or packaging your own applications.

3.2.1 Create your first Conan package

In previous sections, we *consumed* Conan packages (like the *Zlib* one), first using a *conanfile.txt* and then with a *conanfile.py*. But a *conanfile.py* recipe file is not only meant to consume other packages, it can be used to create your own packages, as well. In this section, we explain how to create a simple Conan package with a *conanfile.py* recipe and how to use Conan commands to build those packages from sources.

Important: This is a **tutorial** section. You are encouraged to execute these commands. For this concrete example, you will need **CMake** installed in your path. It is not strictly required by Conan to create packages, you can use other build systems (such as VS, Meson, Autotools, and even your own) to do that, without any dependency on CMake.

Use the **conan new** command to create a "Hello World" C++ library example project:

```
$ conan new cmake_lib -d name=hello -d version=1.0
```

This will create a Conan package project with the following structure.

```
.
├── CMakeLists.txt
├── conanfile.py
├── include
│   └── hello.h
├── src
│   └── hello.cpp
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

The generated files are:

- **conanfile.py**: On the root folder, there is a *conanfile.py* which is the main recipe file, responsible for defining how the package is built and consumed.
- **CMakeLists.txt**: A simple generic *CMakeLists.txt*, with nothing specific about Conan in it.
- **src** and **include** folders: the folders that contain the simple C++ "hello" library.
- **test_package** folder: contains an *example* application that will require and link with the created package. It is not mandatory, but it is useful to check that our package is correctly created.

Let's have a look at the package recipe *conanfile.py*:

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout, CMakeDeps


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    # Optional metadata
    license = "<Put the package license here>"
    author = "<Put your name here> <And your email here>"
```

(continues on next page)

(continued from previous page)

```
    url = "<Package recipe repository url here, for issues about the package>"
    description = "<Description of hello package here>"
    topics = ("<Put some tag here>", "<here>", "<and here>")

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    # Sources are located in the same place as this recipe, copy them to the recipe
    exports_sources = "CMakeLists.txt", "src/*", "include/*"

    def config_options(self):
        if self.settings.os == "Windows":
            del self.options.fPIC

    def layout(self):
        cmake_layout(self)

    def generate(self):
        deps = CMakeDeps(self)
        deps.generate()
        tc = CMakeToolchain(self)
        tc.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def package(self):
        cmake = CMake(self)
        cmake.install()

    def package_info(self):
        self.cpp_info.libs = ["hello"]
```

Let's explain the different sections of the recipe briefly:

First, you can see the **name and version** of the Conan package defined:

- `name`: a string, with a minimum of 2 and a maximum of 100 **lowercase** characters that defines the package name. It should start with an alphanumeric character or underscore and can contain alphanumeric, underscore, +, ., - characters.
- `version`: It is a string, and can take any value, matching the same constraints as the `name` attribute. In case the version follows semantic versioning in the form `X.Y.Z-pre1+build2`, that value might be used for requiring this package through version ranges instead of exact versions.

Then you can see some attributes defining **metadata**. These are optional but recommended and define things like a short `description` for the package, the `author` of the packaged library, the `license`, the `url` for the package repository, and the `topics` that the package is related to.

After that, there is a section related with the binary configuration. This section defines the valid settings and options for the package. As we explained in the *consuming packages section*:

- `settings` are project-wide configuration that cannot be defaulted in recipes. Things like the operating system, compiler or build configuration that will be common to several Conan packages
- `options` are package-specific configuration and can be defaulted in recipes. In this case, we have the option of creating the package as a shared or static library, with static being the default.

Important: The `shared` option implicitly defines that this package is a `package_type = "library"` and it will be either a `shared-library` or a `static-library` depending on the value. Defining a `package_type` is important, so if this option is not defined, it is strongly recommended to define a explicit `package_type` see *package_type reference*.

After that, the `exports_sources` attribute is set to define which sources are part of the Conan package. These are the sources for the library you want to package. In this case, the sources for our "hello" library.

Then, several methods are declared:

- The `config_options()` method (together with the `configure()` one) allows fine-tuning the binary configuration model. For example, on Windows, there is no `fPIC` option, so it can be removed.
- The `layout()` method declares the locations where we expect to find the source files and destinations for the files generated during the build process. Example destination folders are those for the generated binaries and all the files that the Conan generators create in the `generate()` method. In this case, as our project uses CMake as build system, we call `cmake_layout()`. Calling this function will set the expected locations for a CMake project.
- The `generate()` method prepares the build of the package from source. In this case, it could be simplified to an attribute `generators = "CMakeToolchain"`, but it is left to show this important method. In this case, the execution of `CMakeToolchain` `generate()` method will create a *conan_toolchain.cmake* file that translates the Conan `settings` and `options` to CMake syntax. The `CMakeDeps` generator is added for completeness, but it is not strictly necessary until `requires` are added to the recipe.
- The `build()` method uses the `CMake` wrapper to call CMake commands. It is a thin layer that, in this case, will pass the `-DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake` argument. It will configure the project and build it from source.
- The `package()` method copies artifacts (headers, libs) from the build folder to the final package folder. It can be done with bare "copy" commands, but in this case, it is leveraging the already existing CMake install functionality. If the CMakeLists.txt didn't implement it, it would be easy to write an equivalent using the *copy() tool* in the `package()` method.
- Finally, the `package_info()` method defines that consumers must link with a "hello" library when using this package. Other information such as include or lib paths can be defined as well. This information is used for files created by generators (as `CMakeDeps`) to be used by consumers. This is generic information about the current package, and is available to the consumers irrespective of the build system they are using and irrespective of the build system we have used in the `build()` method.

The **test_package** folder is not critical now for understanding how packages are created. The important bits are:

- The **test_package** folder is different from unit or integration tests. These tests are "package" tests, and validate that the package is properly created and that the package consumers will be able to link against it and reuse it.
- It is a small Conan project itself. It contains its own `conanfile.py` and its source code including build scripts, that depends on the package being created, and builds and executes a small application that requires the library in the package.
- It doesn't belong in the package. It only exists in the source repository, not in the package.

Let's build the package from sources with the current default configuration, and then let the `test_package` folder test the package:

```
$ conan create .

======== Exporting recipe to the cache ========
hello/1.0: Exporting package recipe
...
hello/1.0: Exported: hello/1.0#dcbfe21e5250264b26595d151796be70 (2024-03-04 17:52:39 UTC)

======== Installing packages ========
-------- Installing package hello/1.0 (1 of 1) --------
hello/1.0: Building from source
hello/1.0: Calling build()
...
hello/1.0: Package '9bdee485ef71c14ac5f8a657202632bdb8b4482b' built

======== Testing the package: Building ========
...
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example

======== Testing the package: Executing test ========
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release!
  hello/1.0: __aarch64__ defined
  hello/1.0: __cplusplus201703
  hello/1.0: __GNUC__4
  hello/1.0: __GNUC_MINOR__2
  hello/1.0: __clang_major__15
  hello/1.0: __apple_build_version__15000309
...
```

If "Hello world Release!" is displayed, it worked. This is what has happened:

- The *conanfile.py* together with the contents of the *src* folder have been copied (**exported**, in Conan terms) to the local Conan cache.
- A new build from source for the `hello/1.0` package starts, calling the `generate()`, `build()` and `package()` methods. This creates the binary package in the Conan cache.
- Conan then moves to the *test_package* folder and executes a **`conan install`** + **`conan build`** + `test()` method, to check if the package was correctly created.

We can now validate that the recipe and the package binary are in the cache:

```
$ conan list hello
Local Cache
  hello
    hello/1.0
```

The **`conan create`** command receives the same parameters as **`conan install`**, so you can pass to it the same settings and options. If we execute the following lines, we will create new package binaries for the Debug configuration and build the hello library as a shared library:

```
$ conan create . -s build_type=Debug
...
hello/1.0: Hello World Debug!

$ conan create . -o hello/1.0:shared=True
...
hello/1.0: Hello World Release!
```

These new package binaries will be also stored in the Conan cache, ready to be used by any project in this computer. We can see them with:

```
# list all the binaries built for the hello/1.0 package in the cache
$ conan list "hello/1.0:*"
Local Cache
  hello
    hello/1.0
      revisions
        dcbfe21e5250264b26595d151796be70 (2024-05-10 09:40:15 UTC)
          packages
            2505f7ebb5a4cca156b2d6b8534f415a4a48b5c9
              info
                settings
                  arch: armv8
                  build_type: Release
                  compiler: apple-clang
                  compiler.cppstd: gnu17
                  compiler.libcxx: libc++
                  compiler.version: 15
                  os: Macos
                options
                  shared: True
            39f48664f195e0847f59889d8a4cdfc6bca84bf1
              info
                settings
                  arch: armv8
                  build_type: Release
                  compiler: apple-clang
                  compiler.cppstd: gnu17
                  compiler.libcxx: libc++
                  compiler.version: 15
                  os: Macos
                options
                  fPIC: True
                  shared: False
            814ddaac84bc84f3595aa076660133b88e49fb11
              info
                settings
                  arch: armv8
                  build_type: Debug
                  compiler: apple-clang
                  compiler.cppstd: gnu17
                  compiler.libcxx: libc++
                  compiler.version: 15
```

(continues on next page)

(continued from previous page)

```
                os: Macos
              options
                fPIC: True
                shared: False
```

Now that we have created a simple Conan package, we will explain each of the methods of the Conanfile in more detail. You will learn how to modify those methods to achieve things like retrieving the sources from an external repository, adding dependencies to our package, customising our toolchain and much more.

A note about the Conan cache

When you did the **conan create** command, the build of your package did not take place in your local folder but in another folder inside the *Conan cache*. This cache is located in the user home folder under the `.conan2` folder. Conan will use the `~/.conan2` folder to store the built packages and also different configuration files. You already used the **conan list** command to list the recipes and binaries stored in the local cache.

An **important** note: the Conan cache is private to the Conan client - modifying, adding, removing or changing files inside the Conan cache is undefined behaviour likely to cause breakages.

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 8: Creating Your First Conan Package
- *Create your first Conan package with Visual Studio/MSBuild*.
- *Create your first Conan package with Meson*.
- *Create your first Conan package with Autotools (only Linux)*.
- *CMake built-in integrations reference*.
- *conan create command reference* and *Conan list command reference*.

3.2.2 Handle sources in packages

In the *previous tutorial section*, we created a Conan package for a "Hello World" C++ library. We used the `exports_sources` attribute of the Conanfile to declare the location of the sources for the library. This method is the simplest way to define the location of the source files when they are in the same folder as the Conanfile. However, sometimes the source files are stored in a repository or a file in a remote server, and not in the same location as the Conanfile. In this section, we will modify the recipe we created previously by adding a `source()` method and explain how to:

- Retrieve the sources from a *zip* file stored in a remote repository.
- Retrieve the sources from a branch of a *git* repository.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/handle_sources
```

The structure of the project is the same as the one in the previous example but without the library sources:

```
.
├── CMakeLists.txt
├── conanfile.py
```

(continues on next page)

(continued from previous page)

```
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

Sources from a *zip* file stored in a remote repository

Let's have a look at the changes in the *conanfile.py*:

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout
from conan.tools.files import get


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    def source(self):
        # Please, be aware that using the head of the branch instead of an immutable tag
        # or commit is a bad practice and not allowed by Conan
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",
                  strip_root=True)

    def config_options(self):
        if self.settings.os == "Windows":
            del self.options.fPIC

    def layout(self):
        cmake_layout(self)

    def generate(self):
        tc = CMakeToolchain(self)
        tc.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def package(self):
        cmake = CMake(self)
        cmake.install()
```

(continues on next page)

(continued from previous page)

```
    def package_info(self):
        self.cpp_info.libs = ["hello"]
```

As you can see, the recipe is the same but instead of declaring the `exports_sources` attribute as we did previously, i.e.

```
exports_sources = "CMakeLists.txt", "src/*", "include/*"
```

we declare a `source()` method with this information:

```
def source(self):
    # Please, be aware that using the head of the branch instead of an immutable tag
    # or commit is strongly discouraged, unsupported by Conan and likely to cause issues
    get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",
              strip_root=True)
```

We used the *conan.tools.files.get()* tool that will first **download** the *zip* file from the URL that we pass as an argument and then **unzip** it. Note that we pass the `strip_root=True` argument so that if all the unzipped contents are in a single folder, all the contents are moved to the parent folder (check the *conan.tools.files.unzip()* reference for more details).

Warning: It is expected that retrieving the sources in the future produces the same results. Using mutable source origins, like a moving reference in git (e.g HEAD branch), or the URL to a file where the contents may change over time, is strongly discouraged and not supported. Not following this practice will result in undefined behavior likely to cause breakages

The contents of the zip file are the same as the sources we previously had beside the Conan recipe, so if you do a **`conan create`** the results will be the same as before.

```
$ conan create .

...

-------- Installing packages ----------

Installing (downloading, building) binaries...
hello/1.0: Calling source() in /Users/user/.conan2/p/0fcb5ffd11025446/s/.
Downloading update_source.zip

hello/1.0: Unzipping 3.7KB
Unzipping 100 %
hello/1.0: Copying sources to build folder
hello/1.0: Building your package in /Users/user/.conan2/p/tmp/369786d0fb355069/b

...

-------- Testing the package: Running test() ----------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release!
hello/1.0: __x86_64__ defined
```

(continues on next page)

(continued from previous page)

```
hello/1.0: __cplusplus199711
hello/1.0: __GNUC__4
hello/1.0: __GNUC_MINOR__2
hello/1.0: __clang_major__13
hello/1.0: __clang_minor__1
hello/1.0: __apple_build_version__13160021
```

Please, check the highlighted lines with the messages about the download and unzip operation.

Sources from a branch in a *git* repository

Now, let's modify the `source()` method to bring the sources from a *git* repository instead of a *zip* file. We show just the relevant parts:

```
...

from conan.tools.scm import Git


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...

    def source(self):
        git = Git(self)
        git.clone(url="https://github.com/conan-io/libhello.git", target=".")

    ...
```

Here, we use the *conan.tools.scm.Git()* tool. The `Git` class implements several methods to work with *git* repositories. In this case, we call the `clone()` method to clone the https://github.com/conan-io/libhello.git repository in the default branch using the same folder for cloning the sources instead of a subfolder (passing the `target="."` argument).

Warning: As above, this is only a simple example. The source origin for `Git()` also has to be immutable. It is necessary to checkout an immutable tag or a specific commit to guarantee the correct behavior. Using the HEAD of the repository is not allowed and can cause undefined behavior and breakages.

To checkout a commit or tag in the repository, we use the `checkout()` method of the Git tool:

```
def source(self):
    git = Git(self)
    git.clone(url="https://github.com/conan-io/libhello.git", target=".")
    git.checkout("<tag> or <commit hash>")
```

For more information about the `Git` class methods, please check the *conan.tools.scm.Git()* reference.

Note that it's also possible to run other commands by invoking the `self.run()` method.

Using the conandata.yml file

We can write a file named `conandata.yml` in the same folder as the `conanfile.py`. This file will be automatically exported and parsed by Conan and we can read that information from the recipe. This is handy, for example, to extract the URLs of the external source repositories, zip files etc. This is an example of `conandata.yml`:

```
sources:
  "1.0":
    url: "https://github.com/conan-io/libhello/archive/refs/heads/main.zip"
    sha256: "7bc71c682895758a996ccf33b70b91611f51252832b01ef3b4675371510ee466"
    strip_root: true
  "1.1":
    url: ...
    sha256: ...
```

The recipe doesn't need to be modified for each version of the code. We can pass all the `keys` of the specified version (`url`, `sha256`, and `strip_root`) as arguments to the `get` function, that, in this case, allow us to verify that the downloaded zip file has the correct `sha256`. So we could modify the source method to this:

```
def source(self):
    get(self, **self.conan_data["sources"][self.version])
    # Equivalent to:
    # data = self.conan_data["sources"][self.version]
    # get(self, data["url"], sha256=data["sha256"], strip_root=data["strip_root"])
```

The information of the `conandata.yml` applicable to the current version is also available in the serialized recipe information, see the *graph info json format*

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 8: Package Creation and Uploading
- *Patching sources*
- *Capturing Git SCM source information* instead of copying sources with `exports_sources`.
- *source() method reference*

3.2.3 Add dependencies to packages

In the *previous tutorial section*, we created a Conan package for a "Hello World" C++ library. We used the *conan.tools.scm.Git()* tool to retrieve the sources from a git repository. So far, the package does not have any dependency on other Conan packages. Let's explain how to add a dependency to our package in a very similar way to how we did in the *consuming packages section*. We will add some fancy colour output to our "Hello World" library using the fmt library.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/add_requires
```

You will notice some changes in the *conanfile.py* file from the previous recipe. Let's check the relevant parts:

```
...
from conan.tools.build import check_max_cppstd, check_min_cppstd
...
```

(continues on next page)

(continued from previous page)

```
class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...
    generators = "CMakeDeps"
    ...

    def validate(self):
        check_min_cppstd(self, "11")
        check_max_cppstd(self, "20")

    def requirements(self):
        self.requires("fmt/8.1.1")

    def source(self):
        git = Git(self)
        git.clone(url="https://github.com/conan-io/libhello.git", target=".")
        # Please, be aware that using the head of the branch instead of an immutable tag
        # or commit is not a good practice in general
        git.checkout("require_fmt")
```

- First, we set the `generators` class attribute to make Conan invoke the *CMakeDeps* generator. This was not needed in the previous recipe as we did not have dependencies. `CMakeDeps` will generate all the config files that CMake needs to find the `fmt` library.
- Next, we use the *requires()* method to add the fmt dependency to our package.
- Note that we added an extra line in the *source()* method. We use the `Git().checkout()` method to checkout the source code in the require_fmt branch. This branch contains the changes in the source code to add colours to the library messages, and also in the `CMakeLists.txt` to declare that we are using the `fmt` library.
- Finally, note we added the *validate()* method to the recipe. We already used this method in the *consuming packages section* to raise an error for non-supported configurations. Here, we call the functions *check_min_cppstd()* and *check_max_cppstd()* to verify that we are using at least C++11 and at most C++20 standards in our settings.

You can check the new sources using the fmt library in the require_fmt branch. You will see that the hello.cpp file adds colours to the output messages:

```
#include <fmt/color.h>

#include "hello.h"

void hello(){
    #ifdef NDEBUG
    fmt::print(fg(fmt::color::crimson) | fmt::emphasis::bold, "hello/1.0: Hello World␣
↪Release!\n");
    #else
    fmt::print(fg(fmt::color::crimson) | fmt::emphasis::bold, "hello/1.0: Hello World␣
↪Debug!\n");
    #endif
    ...
```

Let's build the package from sources with the current default configuration, and then let the `test_package` folder test

the package. You should see the output messages with colour now:

```
$ conan create . --build=missing
-------- Exporting the recipe ----------
...
-------- Testing the package: Running test() ----------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release!
  hello/1.0: __x86_64__ defined
  hello/1.0: __cplusplus 201103
  hello/1.0: __GNUC__ 4
  hello/1.0: __GNUC_MINOR__ 2
  hello/1.0: __clang_major__ 13
  hello/1.0: __clang_minor__ 1
  hello/1.0: __apple_build_version__ 13160021
```

Headers transitivity

By default, Conan assumes that the required dependency headers are an implementation detail of the current package, to promote good software engineering practices like low coupling and encapsulation. In the example above, `fmt` is purely an implementation detail in the `hello/1.0` package. Consumers of `hello/1.0` will not know anything about `fmt`, or has access to its headers, if a consumer of `hello/1.0` would try to add a `#include <fmt/color.h>`, it will fail, not being able to find that headers.

But if the public headers of the `hello/1.0` package have the `#include` to `fmt` headers, that means that such headers must be propagated down to allow consumers of `hello/1.0` to be compiled successfully. As this is not the default expected behavior, recipes must declare it as:

```
class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    def requirements(self):
        self.requires("fmt/8.1.1", transitive_headers=True)
```

That will propagate the necessary compilation flags and headers `includedirs` to the consumers of `hello/1.0`.

Note: Best practices

If a consumer of `hello/1.0` had a direct inclusion to `fmt` headers such as `#include <fmt/color.h>`, then, such a consumer should declare its own `self.requires("fmt/8.1.1")` requirement, as that is a direct requirement. In other words, even if the dependency to `hello/1.0` was removed from that consumer, it would still depend on `fmt`, and consequently it cannot abuse the transitivity of the `fmt` headers from `hello`, but declare them explicitly.

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 9: Dependencies, Generators And Building
- *Reference for requirements() method*.
- *Introduction to versioning*.

3.2.4 Preparing the build

In the *previous tutorial section*, we added the fmt requirement to our Conan package to provide colour output to our "Hello World" C++ library. In this section, we focus on the `generate()` method of the recipe. The aim of this method is to generate all the information that could be needed while running the build step. That means things like:

- Writing files to be used in the build step, like *scripts* that inject environment variables, files to pass to the build system, etc.
- Configuring the toolchain to provide extra information based on the settings and options or removing information from the toolchain that Conan generates by default and may not apply for certain cases.

We explain how to use this method for a simple example based on the previous tutorial section. We add a *with_fmt* option to the recipe, and depending on its value we either require the *fmt* library or not. We use the `generate()` method to modify the toolchain so that it passes a variable to CMake so that we can conditionally add that library and use *fmt* or not in the source code.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/preparing_the_build
```

You will notice some changes in the *conanfile.py* file from the previous recipe. Let's check the relevant parts:

```
...
from conan.tools.build import check_max_cppstd, check_min_cppstd
...

class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...
    options = {"shared": [True, False],
               "fPIC": [True, False],
               "with_fmt": [True, False]}

    default_options = {"shared": False,
                       "fPIC": True,
                       "with_fmt": True}
    ...

    def validate(self):
        if self.options.with_fmt:
            check_min_cppstd(self, "11")
            check_max_cppstd(self, "14")

    def source(self):
        git = Git(self)
        git.clone(url="https://github.com/conan-io/libhello.git", target=".")
        # Please, be aware that using the head of the branch instead of an immutable tag
        # or commit is not a good practice in general
        git.checkout("optional_fmt")

    def requirements(self):
        if self.options.with_fmt:
```

(continues on next page)

(continued from previous page)

```
            self.requires("fmt/8.1.1")

    def generate(self):
        tc = CMakeToolchain(self)
        if self.options.with_fmt:
            tc.variables["WITH_FMT"] = True
        tc.generate()

    ...
```

As you can see:

- We declare a new `with_fmt` option with the default value set to `True`.
- Based on the value of the `with_fmt` option:
  - We conditionally install the `fmt/8.1.1` Conan package.
  - We conditionally require a minimum and a maximum C++ standard as the *fmt* library requires at least C++11 and it will not compile if we try to use a standard above C++14 (just an example, *fmt* can actually build with more modern standards).
  - We conditionally inject the `WITH_FMT` variable with the value `True` to the *CMakeToolchain* so that we can use it in the *CMakeLists.txt* of the **hello** library to add the CMake **fmt::fmt** target.
- We are cloning another branch of the library. The *optional_fmt* branch contains some changes in the code. Let's see what changed on the CMake side:

Listing 42: **CMakeLists.txt**

```
cmake_minimum_required(VERSION 3.15)
project(hello CXX)

add_library(hello src/hello.cpp)
target_include_directories(hello PUBLIC include)
set_target_properties(hello PROPERTIES PUBLIC_HEADER "include/hello.h")

if (WITH_FMT)
    find_package(fmt)
    target_link_libraries(hello fmt::fmt)
    target_compile_definitions(hello PRIVATE USING_FMT=1)
endif()

install(TARGETS hello)
```

As you can see, we use the `WITH_FMT` we injected in the *CMakeToolchain*. Depending on the value we will try to find the fmt library and link our hello library with it. Also, check that we add the `USING_FMT=1` compile definition that we use in the source code depending on whether we choose to add support for `fmt` or not.

Listing 43: **hello.cpp**

```
#include <iostream>
#include "hello.h"

#if USING_FMT == 1
#include <fmt/color.h>
```

(continues on next page)

(continued from previous page)

```
#endif

void hello(){
    #if USING_FMT == 1
        #ifdef NDEBUG
        fmt::print(fg(fmt::color::crimson) | fmt::emphasis::bold, "hello/1.0: Hello␣
↪World Release! (with color!)\n");
        #else
        fmt::print(fg(fmt::color::crimson) | fmt::emphasis::bold, "hello/1.0: Hello␣
↪World Debug! (with color!)\n");
        #endif
    #else
        #ifdef NDEBUG
        std::cout << "hello/1.0: Hello World Release! (without color)" << std::endl;
        #else
        std::cout << "hello/1.0: Hello World Debug! (without color)" << std::endl;
        #endif
    #endif
}
```

Let's build the package from sources first using `with_fmt=True` and then using `with_fmt=False`. When *test_package* runs it will show different messages depending on the value of the option.

```
$ conan create . --build=missing -o with_fmt=True
-------- Exporting the recipe ----------
...

-------- Testing the package: Running test() ----------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release! (with color!)

$ conan create . --build=missing -o with_fmt=False
-------- Exporting the recipe ----------
...

-------- Testing the package: Running test() ----------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release! (without color)
```

This is just a simple example of how to use the `generate()` method to customize the toolchain based on the value of one option, but there are lots of other things that you could do in the `generate()` method like:

- Create a complete custom toolchain based on your needs to use in your build.
- Access certain information about the package dependencies, like:
  - The configuration via *conf_info*.
  - The options of dependencies.
  - Import files from dependencies using the *copy tool*. You could also import the files to create manifests for the package, collecting all versions and licenses of dependencies.
- Use the *Environment tools* to generate information for the system environment.

- Adding custom configurations besides *Release* and *Debug*, taking into account the settings, like *ReleaseShared* or *DebugShared*.

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 9: Dependencies, Generators And Building
- Use the `generate()` method to *import files from dependencies*.
- *generate() method reference*

3.2.5 Configure settings and options in recipes

We already explained *Conan settings and options* and how to use them to build your projects for different configurations like Debug, Release, with static or shared libraries, etc. In this section, we explain how to configure these settings and options in the case that one of them does not apply to a Conan package. We will introduce briefly how Conan models binary compatibility and how that relates to options and settings.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/configure_options_settings
```

You will notice some changes in the **conanfile.py** file from the previous recipe. Let's check the relevant parts:

```
class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...
    options = {"shared": [True, False],
               "fPIC": [True, False],
               "with_fmt": [True, False]}

    default_options = {"shared": False,
                       "fPIC": True,
                       "with_fmt": True}
    ...

    def config_options(self):
        if self.settings.os == "Windows":
            del self.options.fPIC

    def configure(self):
        if self.options.shared:
            # If os=Windows, fPIC will have been removed in config_options()
            # use rm_safe to avoid double delete errors
            self.options.rm_safe("fPIC")
    ...
```

You can see that we added a *configure() method* to the recipe. Let's explain what the objective of this method is and how it's different from the `config_options()` method we already had defined in the recipe:

- `configure()`: use this method to configure which options or settings of the recipe are available. For example, in this case, we **delete the fPIC option**, because it should only be **True** if we are building the library as shared (in fact, some build systems will add this flag automatically when building a shared library).

- `config_options()`: This method is used to **constrain** the available options in a package **before they take a value**. If a value is assigned to a setting or option that is deleted inside this method, Conan will raise an error. In this case we are **deleting the fPIC option** on Windows because that option does not exist for that operating system. Note that this method is executed before the `configure()` method.

Be aware that deleting an option using the `config_options()` method has a different result from using the `configure()` method. Deleting the option in `config_options()` **is like we never declared it in the recipe** which will raise an exception saying that the option does not exist. However, if we delete it in the `configure()` method we can pass the option but it will have no effect. For example, if you try to pass a value to the `fPIC` option on Windows, Conan will raise an error that the option does not exist:

Listing 44: Windows

```
$ conan create . --build=missing -o fPIC=True
...
-------- Computing dependency graph --------
ERROR: option 'fPIC' doesn't exist
Possible options are ['shared', 'with_fmt']
```

As you have noticed, the `configure()` and `config_options()` methods **delete an option** if certain conditions are met. Let's explain why we are doing this and the implications of removing that option. It is related to how Conan identifies packages that are binary compatible with the configuration set in the profile. In the next section, we introduce the concept of the **Conan package ID**.

Conan packages binary compatibility: the package ID

In previous examples, we used Conan to build for different configurations like *Debug* and *Release*. Each time you create the package for one of those configurations, Conan will build a new binary. Each of them is related to a **generated hash** called **the package ID**. The package ID is just a way to convert a set of settings, options and information about the requirements of the package to a unique identifier.

Let's build our package for *Release* and *Debug* configurations and check the generated binaries package IDs.

```
$ conan create . --build=missing -s build_type=Release -tf="" # -tf="" will skip␣
↪building the test_package
...
[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX static library libhello.a
[100%] Built target hello
hello/1.0: Package '738feca714b7251063cc51448da0cf4811424e7c' built
hello/1.0: Build folder /Users/user/.conan2/p/tmp/7fe7f5af0ef27552/b/build/Release
hello/1.0: Generated conaninfo.txt
hello/1.0: Generating the package
hello/1.0: Temporary package folder /Users/user/.conan2/p/tmp/7fe7f5af0ef27552/p
hello/1.0: Calling package()
hello/1.0: CMake command: cmake --install "/Users/user/.conan2/p/tmp/7fe7f5af0ef27552/b/
↪build/Release" --prefix "/Users/user/.conan2/p/tmp/7fe7f5af0ef27552/p"
hello/1.0: RUN: cmake --install "/Users/user/.conan2/p/tmp/7fe7f5af0ef27552/b/build/
↪Release" --prefix "/Users/user/.conan2/p/tmp/7fe7f5af0ef27552/p"
-- Install configuration: "Release"
-- Installing: /Users/user/.conan2/p/tmp/7fe7f5af0ef27552/p/lib/libhello.a
-- Installing: /Users/user/.conan2/p/tmp/7fe7f5af0ef27552/p/include/hello.h
hello/1.0 package(): Packaged 1 '.h' file: hello.h
hello/1.0 package(): Packaged 1 '.a' file: libhello.a
```

(continues on next page)

(continued from previous page)

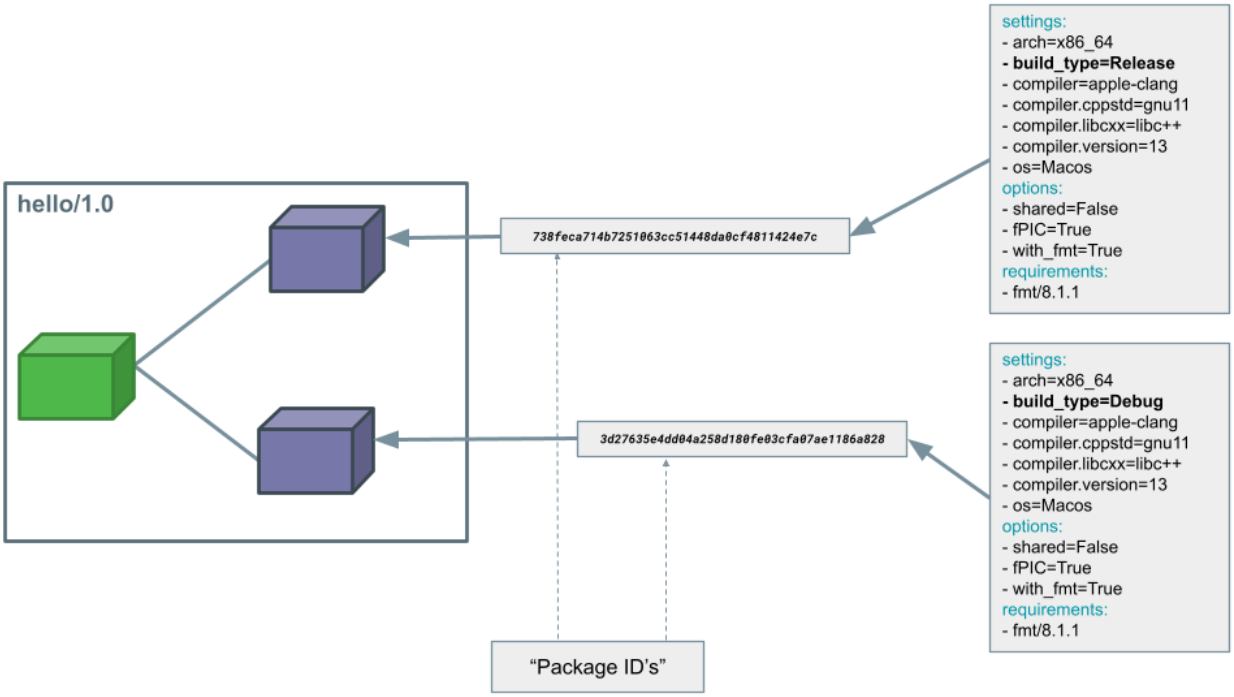
```
hello/1.0: Package '738feca714b7251063cc51448da0cf4811424e7c' created
hello/1.0: Created package revision 3bd9faedc711cbb4fdf10b295268246e
hello/1.0: Full package reference: hello/1.0
↪#e6b11fb0cb64e3777f8d62f4543cd6b3:738feca714b7251063cc51448da0cf4811424e7c
↪#3bd9faedc711cbb4fdf10b295268246e
hello/1.0: Package folder /Users/user/.conan2/p/5c497cbb5421cbda/p

$ conan create . --build=missing -s build_type=Debug -tf="" # -tf="" will skip building␣
↪the test_package
...
[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX static library libhello.a
[100%] Built target hello
hello/1.0: Package '3d27635e4dd04a258d180fe03cfa07ae1186a828' built
hello/1.0: Build folder /Users/user/.conan2/p/tmp/19a2e552db727a2b/b/build/Debug
hello/1.0: Generated conaninfo.txt
hello/1.0: Generating the package
hello/1.0: Temporary package folder /Users/user/.conan2/p/tmp/19a2e552db727a2b/p
hello/1.0: Calling package()
hello/1.0: CMake command: cmake --install "/Users/user/.conan2/p/tmp/19a2e552db727a2b/b/
↪build/Debug" --prefix "/Users/user/.conan2/p/tmp/19a2e552db727a2b/p"
hello/1.0: RUN: cmake --install "/Users/user/.conan2/p/tmp/19a2e552db727a2b/b/build/Debug
↪" --prefix "/Users/user/.conan2/p/tmp/19a2e552db727a2b/p"
-- Install configuration: "Debug"
-- Installing: /Users/user/.conan2/p/tmp/19a2e552db727a2b/p/lib/libhello.a
-- Installing: /Users/user/.conan2/p/tmp/19a2e552db727a2b/p/include/hello.h
hello/1.0 package(): Packaged 1 '.h' file: hello.h
hello/1.0 package(): Packaged 1 '.a' file: libhello.a
hello/1.0: Package '3d27635e4dd04a258d180fe03cfa07ae1186a828' created
hello/1.0: Created package revision 67b887a0805c2a535b58be404529c1fe
hello/1.0: Full package reference: hello/1.0
↪#e6b11fb0cb64e3777f8d62f4543cd6b3:3d27635e4dd04a258d180fe03cfa07ae1186a828
↪#67b887a0805c2a535b58be404529c1fe
hello/1.0: Package folder /Users/user/.conan2/p/c7796386fcad5369/p
```

As you can see, Conan generated two package IDs:

- Package *738feca714b7251063cc51448da0cf4811424e7c* for Release
- Package *3d27635e4dd04a258d180fe03cfa07ae1186a828* for Debug

These two package IDs are calculated by taking the **set of settings, options and some information about the requirements** (we will explain this later in the documentation) and **calculating a hash** of them. So, for example, in this case, they are the result of the information depicted in the diagram below.



Those package IDs are different because the **build_type** is different. Now, when you want to install a package, Conan will:

- Collect the settings and options applied, along with some information about the requirements and calculate the hash for the corresponding package ID.
- If that package ID matches one of the packages stored in the local Conan cache, Conan will use that. If not, and we have any Conan remote configured, it will search for a package with that package ID in the remotes.
- If that calculated package ID does not exist in the local cache and remotes, Conan will fail with a "missing binary" error message, or will try to build that package from sources (this depends on the value of the `--build` argument). This build will generate a new package ID in the local cache.

These steps are simplified, there is far more to package ID calculation than what we explain here. Recipes themselves can even adjust their package ID calculations, we can have different recipe and package revisions besides package IDs, and there's also a built-in mechanism in Conan that can be configured to declare that some packages with a certain package ID are compatible with each other.

Maybe you have now the intuition of why we delete settings or options in Conan recipes. If you do that, those values will not be included in the computation of the package ID, so even if you define them, the resulting package ID will be the same. You can check this behaviour, for example, with the fPIC option that is deleted when we build with the option `shared=True`. Regardless of the value you pass for the fPIC option, the generated package ID will be the same for the **hello/1.0** binary:

Listing 45: Windows

```
$ conan create . --build=missing -o shared=True -o fPIC=True -tf=""
```

Listing 46: Linux, macOS

```
$ conan create . --build=missing -o shared=True -o -tf=""
```

```
...
hello/1.0 package(): Packaged 1 '.h' file: hello.h
hello/1.0 package(): Packaged 1 '.dylib' file: libhello.dylib
hello/1.0: Package '2a899fd0da3125064bf9328b8db681cd82899d56' created
hello/1.0: Created package revision f0d1385f4f90ae465341c15740552d7e
hello/1.0: Full package reference: hello/1.0
↪#e6b11fb0cb64e3777f8d62f4543cd6b3:2a899fd0da3125064bf9328b8db681cd82899d56
↪#f0d1385f4f90ae465341c15740552d7e
hello/1.0: Package folder /Users/user/.conan2/p/8a55286c6595f662/p

$ conan create . --build=missing -o shared=True -o fPIC=False -tf=""
...
-------- Computing dependency graph --------
Graph root
    virtual
Requirements
    fmt/8.1.1#601209640bd378c906638a8de90070f7 - Cache
    hello/1.0#e6b11fb0cb64e3777f8d62f4543cd6b3 - Cache

-------- Computing necessary packages --------
Requirements
    fmt/8.1.1#601209640bd378c906638a8de90070f7:d1b3f3666400710fec06446a697f9eeddd1235aa
↪#24a2edf207deeed4151bd87bca4af51c - Skip
    hello/1.0#e6b11fb0cb64e3777f8d62f4543cd6b3:2a899fd0da3125064bf9328b8db681cd82899d56
↪#f0d1385f4f90ae465341c15740552d7e - Cache

-------- Installing packages --------

-------- Installing (downloading, building) binaries... --------
hello/1.0: Already installed!
```

As you can see, the first run created the `2a899fd0da3125064bf9328b8db681cd82899d56` package, and the second one, regardless of the different value of the fPIC option, said we already had the `2a899fd0da3125064bf9328b8db681cd82899d56` package installed.

C libraries

There are other typical cases where you want to delete certain settings. Imagine that you are packaging a C library. When you build this library, there are settings like the compiler C++ standard (`self.settings.compiler.cppstd`) or the standard library used (`self.settings.compiler.libcxx`) that won't affect the resulting binary, at all. Then it does not make sense that they affect the package ID computation, so a typical pattern is to delete them in the `configure()` method:

```
def configure(self):
    self.settings.rm_safe("compiler.cppstd")
    self.settings.rm_safe("compiler.libcxx")
```

Please, note that deleting these settings in the `configure()` method will not only modify the package ID calculation but will also affect how the toolchain and the build system integrations work because the C++ settings do not exist.

Note: From Conan 2.4, the above `configure()` is not necessary if the `languages = "C"` recipe attribute is defined

(experimental).

Header-only libraries

A similar case happens with packages that package *header-only libraries*. In that case, there's no binary code we need to link with, but just some header files to add to our project. Thus, the package ID of the Conan package should not be affected by settings or options. For that case, there's a simpler way of declaring that the generated package ID should not take into account settings, options or any information from the requirements: calling the `self.info.clear()` method inside another recipe method called `package_id()`:

```
def package_id(self):
    self.info.clear()
```

We will explain the `package_id()` method later and explain how you can customize the way the package ID for the package is calculated. You can also check the *Conanfile's methods reference* if you want to know how this method works in more detail.

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 11: Configuring Settings And Options
- *Header-only packages*.
- Check the binary compatibility *compatibility.py extension*.
- Conan *package types*.
- *Setting package_id_mode for requirements*.
- Read the *binary model reference* for a full view of the Conan binary model.

3.2.6 Build packages: the build() method

We already used a Conan recipe that has a *build() method* and learned how to use that to invoke a build system and build our packages. In this tutorial, we will modify that method and explain how you can use it to do things like:

- Building and running tests
- Conditional patching of the source code
- Selecting the build system you want to use conditionally

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/build_method
```

Build and run tests for your project

You will notice some changes in the **conanfile.py** file from the previous recipe. Let's check the relevant parts:

Changes introduced in the recipe

Listing 47: *conanfile.py*

```
class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...

    def source(self):
        git = Git(self)
        git.clone(url="https://github.com/conan-io/libhello.git", target=".")
        # Please, be aware that using the head of the branch instead of an immutable tag
        # or commit is not a good practice in general
        git.checkout("with_tests")

    ...

    def requirements(self):
        if self.options.with_fmt:
            self.requires("fmt/8.1.1")

    def build_requirements(self):
        self.test_requires("gtest/1.17.0")

    ...

    def generate(self):
        tc = CMakeToolchain(self)
        if self.options.with_fmt:
            tc.variables["WITH_FMT"] = True
        tc.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
        if not self.conf.get("tools.build:skip_test", default=False):
            test_folder = os.path.join("tests")
            if self.settings.os == "Windows":
                test_folder = os.path.join("tests", str(self.settings.build_type))
            self.run(os.path.join(test_folder, "test_hello"))

    ...
```

- We added the *gtest/1.17.0* requirement to the recipe as a `test_requires()`. It's a type of requirement intended for testing libraries like **Catch2** or **gtest**, and we declare it inside the `build_requirements()` method, which is the recommended place for test dependencies.

- We use the `tools.build:skip_test` configuration (`False` by default), to tell CMake whether to build and run the tests or not. A couple of things to bear in mind:
  - If we set the `tools.build:skip_test` configuration to `True`, Conan will automatically inject the `BUILD_TESTING` variable to CMake set to `OFF`. You will see in the next section that we are using this variable in our *CMakeLists.txt* to decide whether to build the tests or not.
  - We use the `tools.build:skip_test` configuration in the `build()` method, after building the package and tests, to decide if we want to run the tests or not.
  - In this case we are using **gtest** for testing and we have to check if the build method is to run the tests or not. This configuration also affects the execution of `CMake.test()` if you are using CTest and `Meson.test()` for Meson.

Changes introduced in the library sources

First, please note that we are using another branch from the **libhello** library. This branch has two novelties on the library side:

- We added a new function called `compose_message()` to the library sources so we can add some unit tests over this function. This function is just creating an output message based on the arguments passed.
- As we mentioned in the previous section, the CMakeLists.txt for the library uses the `BUILD_TESTING` CMake variable that conditionally adds the *tests* directory.

Listing 48: *CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(hello CXX)

...

if (NOT BUILD_TESTING STREQUAL OFF)
    add_subdirectory(tests)
endif()

...
```

The `BUILD_TESTING` CMake variable is declared and set to `OFF` by Conan (if not already defined) whenever the `tools.build:skip_test` configuration is set to value `True`. This variable is typically declared by CMake when you use CTest but using the `tools.build:skip_test` configuration you can use it in your *CMakeLists.txt* even if you are using another testing framework.

We have a CMakeLists.txt in the *tests* folder using googletest for testing:

Listing 49: *tests/CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(PackageTest CXX)

find_package(GTest REQUIRED CONFIG)

add_executable(test_hello test.cpp)
target_link_libraries(test_hello GTest::gtest GTest::gtest_main hello)
```

With basic tests on the functionality of the `compose_message()` function:

Listing 50: *tests/test.cpp*

```
#include "../include/hello.h"
#include "gtest/gtest.h"

namespace {
    TEST(HelloTest, ComposeMessages) {
      EXPECT_EQ(std::string("hello/1.0: Hello World Release! (with color!)\n"), compose_
↪message("Release", "with color!"));
      ...
    }
}
```

Now that we have gone through all the changes in the code, let's try them out:

```
$ conan create . --build=missing -tf=""
...
[ 25%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[ 50%] Linking CXX static library libhello.a
[ 50%] Built target hello
[ 75%] Building CXX object tests/CMakeFiles/test_hello.dir/test.cpp.o
[100%] Linking CXX executable test_hello
[100%] Built target test_hello
hello/1.0: RUN: ./tests/test_hello
Capturing current environment in /Users/user/.conan2/p/tmp/c51d80ef47661865/b/build/
↪generators/deactivate_conanbuildenv-release-x86_64.sh
Configuring environment variables
Running main() from /Users/user/.conan2/p/tmp/3ad4c6873a47059c/b/googletest/src/gtest_
↪main.cc
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from HelloTest
[ RUN      ] HelloTest.ComposeMessages
[       OK ] HelloTest.ComposeMessages (0 ms)
[----------] 1 test from HelloTest (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
hello/1.0: Package '82b6c0c858e739929f74f59c25c187b927d514f3' built
...
```

As you can see, the tests were built and run. Let's use now the `tools.build:skip_test` configuration in the command line to skip the test building and running:

```
$ conan create . -c tools.build:skip_test=True -tf=""
...
[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX static library libhello.a
[100%] Built target hello
hello/1.0: Package '82b6c0c858e739929f74f59c25c187b927d514f3' built
...
```

You can see now that only the library target was built and that no tests were built or run.

Conditionally patching the source code

If you need to patch the source code, the recommended approach is to do that in the `source()` method. Sometimes, if that patch depends on settings or options, you have to use the `build()` method to apply patches to the source code before launching the build. There are *several ways to do this* in Conan. One of them would be using the *replace_in_file* tool:

```
import os
from conan import ConanFile
from conan.tools.files import replace_in_file


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    def build(self):
        replace_in_file(self, os.path.join(self.source_folder, "src", "hello.cpp"),
                        "Hello World",
                        "Hello {} Friends".format("Shared" if self.options.shared else
↪"Static"))
```

Please note that patching in `build()` should be avoided if possible and only be done for very particular cases as it will make it more difficult to develop your packages locally (we will explain more about this in the *local development flow section* later).

Conditionally select your build system

It's not uncommon that some packages need one build system or another depending on the platform we are building on. For example, the *hello* library could build on Windows using CMake and on Linux and macOS using Autotools. This can be easily handled in the `build()` method like this:

```
...

class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    ...

    def generate(self):
        if self.settings.os == "Windows":
            tc = CMakeToolchain(self)
```

(continues on next page)

(continued from previous page)

```
            tc.generate()
            deps = CMakeDeps(self)
            deps.generate()
        else:
            tc = AutotoolsToolchain(self)
            tc.generate()
            deps = PkgConfigDeps(self)
            deps.generate()

    ...

    def build(self):
        if self.settings.os == "Windows":
            cmake = CMake(self)
            cmake.configure()
            cmake.build()
        else:
            autotools = Autotools(self)
            autotools.autoreconf()
            autotools.configure()
            autotools.make()

    ...
```

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 9: Dependencies, Generators And Building
- *Patching sources*

3.2.7 Package files: the package() method

We already used the `package()` method in our *hello* package to invoke CMake's install step. In this tutorial, we will explain the use of *CMake.install()* in more detail and also how to modify this method to do things like:

- Using *conan.tools.files* utilities to copy the generated artifacts from the build folder to the package folder
- Copying package licenses
- Manage packaging of symlinks

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/package_method
```

Using CMake install step in the package() method

This is the simplest choice when you have already defined in your *CMakeLists.txt* the functionality of extracting the artifacts (headers, libraries, binaries) from the build and source folder to a predetermined place and maybe do some post-processing of those artifacts. This will work without changes in your *CMakeLists.txt* because Conan will set the `CMAKE_INSTALL_PREFIX` CMake variable to point to the recipe's *package_folder* attribute. Then, just calling *install()* in the *CMakeLists.txt* over the created target is enough for Conan to move the built artifacts to the correct location in the Conan local cache.

Listing 51: *CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(hello CXX)

add_library(hello src/hello.cpp)
target_include_directories(hello PUBLIC include)
set_target_properties(hello PROPERTIES PUBLIC_HEADER "include/hello.h")

...

install(TARGETS hello)
```

Listing 52: *conanfile.py*

```
def package(self):
    cmake = CMake(self)
    cmake.install()
```

Let's build our package again and pay attention to the lines regarding the packaging of files in the Conan local cache:

```
$ conan create . --build=missing -tf=""
...
hello/1.0: Build folder /Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/b/build/Release
hello/1.0: Generated conaninfo.txt
hello/1.0: Generating the package
hello/1.0: Temporary package folder /Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/p
hello/1.0: Calling package()
hello/1.0: CMake command: cmake --install "/Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/b/
↪build/Release" --prefix "/Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/p"
hello/1.0: RUN: cmake --install "/Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/b/build/
↪Release" --prefix "/Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/p"
-- Install configuration: "Release"
-- Installing: /Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/p/lib/libhello.a
-- Installing: /Users/user/.conan2/p/tmp/b5857f2e70d1b2fd/p/include/hello.h
hello/1.0 package(): Packaged 1 '.h' file: hello.h
hello/1.0 package(): Packaged 1 '.a' file: libhello.a
hello/1.0: Package 'fd7c4113dad406f7d8211b3470c16627b54ff3af' created
hello/1.0: Created package revision bf7f5b9a3bb2c957742be4be216dfcbb
hello/1.0: Full package reference: hello/1.0
↪#25e0b5c00ae41ef9fbfbbb1e5ac86e1e:fd7c4113dad406f7d8211b3470c16627b54ff3af
↪#bf7f5b9a3bb2c957742be4be216dfcbb
hello/1.0: Package folder /Users/user/.conan2/p/47b4c4c61c8616e5/p
```

As you can see, both the *include* and *library* files were copied to the package folder after calling the `cmake.install()`

method.

Use conan.tools.files.copy() in the package() method and packaging licenses

For the cases that you don't want to rely on CMake's install functionality or that you are using another build-system, Conan provides the tools to copy the selected files to the *package_folder*. In this case, you can use the *tools.files.copy* function to make that copy. We can replace the previous `cmake.install()` step with a custom copy of the files and the result would be the same.

Note that we are also packaging the `LICENSE` file from the library sources in the *licenses* folder. This is a common pattern in Conan packages and could also be added to the previous example using `cmake.install()` as the *CMakeLists.txt* will not copy this file to the *package folder*.

Listing 53: *conanfile.py*

```
def package(self):
    copy(self, "LICENSE", src=self.source_folder, dst=os.path.join(self.package_folder,
↪"licenses"))
    copy(self, pattern="*.h", src=os.path.join(self.source_folder, "include"), dst=os.
↪path.join(self.package_folder, "include"))
    copy(self, pattern="*.a", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "lib"), keep_path=False)
    copy(self, pattern="*.so", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "lib"), keep_path=False)
    copy(self, pattern="*.lib", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "lib"), keep_path=False)
    copy(self, pattern="*.dll", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "bin"), keep_path=False)
    copy(self, pattern="*.dylib", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "lib"), keep_path=False)
```

Let's build our package one more time and pay attention to the lines regarding the packaging of files in the Conan local cache:

```
$ conan create . --build=missing -tf=""
...
hello/1.0: Build folder /Users/user/.conan2/p/tmp/222db0532bba7cbc/b/build/Release
hello/1.0: Generated conaninfo.txt
hello/1.0: Generating the package
hello/1.0: Temporary package folder /Users/user/.conan2/p/tmp/222db0532bba7cbc/p
hello/1.0: Calling package()
hello/1.0: Copied 1 file: LICENSE
hello/1.0: Copied 1 '.h' file: hello.h
hello/1.0: Copied 1 '.a' file: libhello.a
hello/1.0 package(): Packaged 1 file: LICENSE
hello/1.0 package(): Packaged 1 '.h' file: hello.h
hello/1.0 package(): Packaged 1 '.a' file: libhello.a
hello/1.0: Package 'fd7c4113dad406f7d8211b3470c16627b54ff3af' created
hello/1.0: Created package revision 50f91e204d09b64b24b29df3b87a2f3a
hello/1.0: Full package reference: hello/1.0
↪#96ed9fb1f78bc96708b1abf4841523b0:fd7c4113dad406f7d8211b3470c16627b54ff3af
↪#50f91e204d09b64b24b29df3b87a2f3a
hello/1.0: Package folder /Users/user/.conan2/p/21ec37b931782de8/p
```

Check how the *include* and *library* files are packaged. The LICENSE file is also copied as we explained above.

Managing symlinks in the package() method

Another thing you can do in the package method is controlling how to package symlinks. Conan won't manipulate symlinks by default, so we provide several *tools* to convert absolute symlinks to relative ones and to remove external or broken symlinks.

Imagine that some of the files packaged in the last example were symlinks that point to an absolute location inside the Conan cache. Then, calling `conan.tools.files.symlinks.absolute_to_relative_symlinks()` would convert those absolute links into relative paths and make the package relocatable.

Listing 54: *conanfile.py*

```
from conan.tools.files.symlinks import absolute_to_relative_symlinks

def package(self):
    copy(self, "LICENSE", src=self.source_folder, dst=os.path.join(self.package_folder,
↪"licenses"))
    copy(self, pattern="*.h", src=os.path.join(self.source_folder, "include"), dst=os.
↪path.join(self.package_folder, "include"))
    copy(self, pattern="*.a", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "lib"), keep_path=False)
    ...

    absolute_to_relative_symlinks(self, self.package_folder)
```

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 10: The package() And package_info() Methods
- *package() method reference*

3.2.8 Define information for consumers: the package_info() method

In the previous tutorial section, we explained how to store the headers and binaries of a library in a Conan package using the *package method*. Consumers that depend on that package will reuse those files, but we have to provide some additional information so that Conan can pass that to the build system and consumers can use the package.

For instance, in our example, we are building a static library named *hello* that will result in a *libhello.a* file in Linux and macOS or a *hello.lib* file in Windows. Also, we are packaging a header file *hello.h* with the declaration of the library functions. The Conan package ends up with the following structure in the Conan local cache:

```
.
├── include
│   └── hello.h
└── lib
    └── libhello.a
```

Then, consumers that want to link against this library will need some information:

- The location of the *include* folder in the Conan local cache to search for the *hello.h* file.
- The name of the library file to link against it (*libhello.a* or *hello.lib*)
- The location of the *lib* folder in the Conan local cache to search for the library file.

Conan provides an abstraction over all the information consumers may need in the *cpp_info* attribute of the ConanFile. The information for this attribute must be set in the *package_info() method*. Let's have a look at the `package_info()` method of our *hello/1.0* Conan package:

Listing 55: *conanfile.py*

```
...

class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...

    def package_info(self):
        self.cpp_info.libs = ["hello"]
```

We can see a couple of things:

- We are adding a *hello* library to the `libs` property of the `cpp_info` to tell consumers that they should link the libraries from that list.
- We are **not adding** information about the *lib* or *include* folders where the library and header files are packaged. The `cpp_info` object provides the `.includedirs` and `.libdirs` properties to define those locations but Conan sets their value as `lib` and `include` by default so it's not needed to add those in this case. If you were copying the package files to a different location then you have to set those explicitly. The declaration of the `package_info` method in our Conan package would be equivalent to this one:

Listing 56: *conanfile.py*

```
...

class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...

    def package_info(self):
        self.cpp_info.libs = ["hello"]
        # conan sets libdirs = ["lib"] and includedirs = ["include"] by default
        self.cpp_info.libdirs = ["lib"]
        self.cpp_info.includedirs = ["include"]
```

Setting information in the package_info() method

Besides what we explained above about the information you can set in the `package_info()` method, there are some typical use cases:

- Define information for consumers depending on settings or options
- Customizing certain information that generators provide to consumers, like the target names for CMake or the generated file names for pkg-config, for example
- Propagating configuration values to consumers
- Propagating environment information to consumers

- Define components for Conan packages that provide multiple libraries

Let's see some of those in action. First, clone the project sources if you haven't done so yet. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/package_information
```

Define information for consumers depending on settings or options

For this section of the tutorial we introduced some changes in the library and recipe. Let's check the relevant parts:

Changes introduced in the library sources

First, please note that we are using another branch from the **libhello** library. Let's check the library's *CMakeLists.txt*:

Listing 57: *CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(hello CXX)

...

add_library(hello src/hello.cpp)

if (BUILD_SHARED_LIBS)
    set_target_properties(hello PROPERTIES OUTPUT_NAME hello-shared)
else()
    set_target_properties(hello PROPERTIES OUTPUT_NAME hello-static)
endif()

...
```

As you can see, we are setting the output name for the library depending on whether we are building the library as static (*hello-static*) or as shared (*hello-shared*). Now let's see how to translate these changes to the Conan recipe.

Changes introduced in the recipe

To update our recipe according to the changes in the library's *CMakeLists.txt*, we have to conditionally set the library name depending on the `self.options.shared` option in the `package_info()` method:

Listing 58: *conanfile.py*

```
class helloRecipe(ConanFile):
    ...

    def source(self):
        git = Git(self)
        git.clone(url="https://github.com/conan-io/libhello.git", target=".")
        # Please, be aware that using the head of the branch instead of an immutable tag
        # or commit is not a good practice in general
```

(continues on next page)

(continued from previous page)

```
        git.checkout("package_info")

    ...

    def package_info(self):
        if self.options.shared:
            self.cpp_info.libs = ["hello-shared"]
        else:
            self.cpp_info.libs = ["hello-static"]
```

Now, let's create the Conan package with `shared=False` (that's the default so no need to set it explicitly) and check that we are packaging the correct library (*libhello-static.a* or *hello-static.lib*) and that we are linking the correct library in the *test_package*.

```
$ conan create . --build=missing
...
-- Install configuration: "Release"
-- Installing: /Users/user/.conan2/p/tmp/a311fcf8a63f3206/p/lib/libhello-static.a
-- Installing: /Users/user/.conan2/p/tmp/a311fcf8a63f3206/p/include/hello.h
hello/1.0 package(): Packaged 1 '.h' file: hello.h
hello/1.0 package(): Packaged 1 '.a' file: libhello-static.a
hello/1.0: Package 'fd7c4113dad406f7d8211b3470c16627b54ff3af' created
...
-- Build files have been written to: /Users/user/.conan2/p/tmp/a311fcf8a63f3206/b/build/
↪Release
hello/1.0: CMake command: cmake --build "/Users/user/.conan2/p/tmp/a311fcf8a63f3206/b/
↪build/Release" -- -j16
hello/1.0: RUN: cmake --build "/Users/user/.conan2/p/tmp/a311fcf8a63f3206/b/build/Release
↪" -- -j16
[ 25%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[ 50%] Linking CXX static library libhello-static.a
[ 50%] Built target hello
[ 75%] Building CXX object tests/CMakeFiles/test_hello.dir/test.cpp.o
[100%] Linking CXX executable test_hello
[100%] Built target test_hello
hello/1.0: RUN: tests/test_hello
...
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example

-------- Testing the package: Running test() --------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release! (with color!)
```

As you can see both the tests for the library and the Conan *test_package* linked against the *libhello-static.a* library successfully.

Properties model: setting information for specific generators

The *CppInfo* object provides the `set_property` method to set information specific to each generator. For example, in this tutorial, we use the *CMakeDeps* generator to generate the information that CMake needs to build a project that requires our library. `CMakeDeps`, by default, will set a target name for the library using the same name as the Conan package. If you have a look at that *CMakeLists.txt* from the *test_package*:

Listing 59: test_package *CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(PackageTest CXX)

find_package(hello CONFIG REQUIRED)

add_executable(example src/example.cpp)
target_link_libraries(example hello::hello)
```

You can see that we are linking with the target name `hello::hello`. Conan sets this target name by default, but we can change it using the *properties model*. Let's try to change it to the name `hello::myhello`. To do this, we have to set the property `cmake_target_name` in the `package_info` method of our *hello/1.0* Conan package:

Listing 60: *conanfile.py*

```
class helloRecipe(ConanFile):
    ...

    def package_info(self):
        if self.options.shared:
            self.cpp_info.libs = ["hello-shared"]
        else:
            self.cpp_info.libs = ["hello-static"]

        self.cpp_info.set_property("cmake_target_name", "hello::myhello")
```

Then, change the target name we are using in the *CMakeLists.txt* in the *test_package* folder to `hello::myhello`:

Listing 61: test_package *CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(PackageTest CXX)
# ...
target_link_libraries(example hello::myhello)
```

And re-create the package:

```
$ conan create . --build=missing
Exporting the recipe
hello/1.0: Exporting package recipe
hello/1.0: Using the exported files summary hash as the recipe revision:␣
↪44d78a68b16b25c5e6d7e8884b8f58b8
hello/1.0: A new conanfile.py version was exported
hello/1.0: Folder: /Users/user/.conan2/p/a8cb81b31dc10d96/e
hello/1.0: Exported revision: 44d78a68b16b25c5e6d7e8884b8f58b8
...
-------- Testing the package: Building --------
```

(continues on next page)

(continued from previous page)

```
hello/1.0 (test package): Calling build()
...
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Conan: Target declared 'hello::myhello'
...
[100%] Linking CXX executable example
[100%] Built target example

-------- Testing the package: Running test() --------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release! (with color!)
```

You can see how Conan now declares the target `hello::myhello` instead of the default `hello::hello` and the *test_package* builds successfully.

Similarly, if the CMake config file name differs from the Conan package name, set the `cmake_file_name` property so consumers can call `find_package()` with the right name, for example if the consumers are expected to do `find_package(MyHello CONFIG REQUIRED)`

Listing 62: *conanfile.py*

```
def package_info(self):
    self.cpp_info.set_property("cmake_file_name", "MyHello")
```

The target name is not the only property you can set in the `CMakeDeps` generator. For a complete list of properties that affect the `CMakeDeps` generator behaviour, please check the *reference*.

Propagating environment or configuration information to consumers

You can provide environment information to consumers in the `package_info()`. To do so, you can use the ConanFile's *runenv_info* and *buildenv_info* properties:

- `runenv_info` *Environment* object that defines environment information that consumers that use the package may need when **running**.
- `buildenv_info` *Environment* object that defines environment information that consumers that use the package may need when **building**.

Please note that it's not necessary to add `cpp_info.bindirs` to `PATH` or `cpp_info.libdirs` to `LD_LIBRARY_PATH`, those are automatically added by the *VirtualBuildEnv* and *VirtualRunEnv*.

You can also define configuration values in the `package_info()` so that consumers can use that information. To do this, set the *conf_info* property of the `ConanFile`.

To learn more about this use case, please check the *corresponding example*.

Define components for Conan packages that provide multiple libraries

There are cases in which a Conan package may provide multiple libraries, for these cases you can set the separate information for each of those libraries using the components attribute from the *CppInfo* object.

To know more about this use case, please check the *components example* in the examples section.

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 10: The package() And package_info() Methods
- *Propagating environment and configuration information to consumers example*
- *Define components for Conan packages that provide multiple libraries example*
- *package_info() reference*

3.2.9 Testing Conan packages

In all the previous sections of the tutorial, we used the *test_package*. It was invoked automatically at the end of the `conan create` command after building our package verifying that the package is created correctly. Let's explain the *test_package* in more detail in this section.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/testing_packages
```

Some important notes to have in mind about the *test_package*:

- The *test_package* folder is different from unit or integration tests. These tests are "package" tests, and validate that the package is properly created, and that the package consumers will be able to link against it and reuse it.
- It is a small Conan project itself, that contains its own *conanfile.py* and its source code including build scripts. It depends on the package being created and builds and executes a small application that requires the library in the package.
- It doesn't belong to the package. It only exist in the source repository, not in the package.
- The *test_package* folder is the default one, but a different one can be defined via the command line argument `--test-folder` or with the `test_package_folder` recipe attribute.

The *test_package* folder for our hello/1.0 Conan package has the following contents:

```
test_package
  ├── CMakeLists.txt
  ├── conanfile.py
  └── src
      └── example.cpp
```

Let's have a look at the different files that are part of the *test_package*. First, *example.cpp* is just a minimal example of how to use the *libhello* library that we are packaging:

Listing 63: *test_package/src/example.cpp*

```
#include "hello.h"

int main() {
```

(continues on next page)

(continued from previous page)

```
    hello();
}
```

Then the *CMakeLists.txt* file to tell CMake how to build the example:

Listing 64: *test_package/CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.15)
project(PackageTest CXX)

find_package(hello CONFIG REQUIRED)

add_executable(example src/example.cpp)
target_link_libraries(example hello::hello)
```

Finally, the recipe for the *test_package* that consumes the *hello/1.0* Conan package:

Listing 65: *test_package/conanfile.py*

```
import os

from conan import ConanFile
from conan.tools.cmake import CMake, cmake_layout
from conan.tools.build import can_run


class helloTestConan(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeDeps", "CMakeToolchain"

    def requirements(self):
        self.requires(self.tested_reference_str)

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def layout(self):
        cmake_layout(self)

    def test(self):
        if can_run(self):
            cmd = os.path.join(self.cpp.build.bindir, "example")
            self.run(cmd, env="conanrun")
```

Let's go through the most relevant parts:

- We add the requirements in the `requirements()` method, but in this case we use the `tested_reference_str` attribute that Conan passes to the test_package. This is a convenience attribute to avoid hardcoding the package name in the test_package so that we can reuse the same test_package for several versions of the same Conan package. In our case, this variable will take the `hello/1.0` value.
- We define a `test()` method. This method will only be invoked in *test_package* recipes. It executes immediately

after `build()` is called and is meant to run some executable or tests on binaries to prove the package is correctly created. A couple of comments about the contents of our `test()` method:

- We are using the *conan.tools.build.cross_building* tool to check if we can run the built executable on the current platform. This tool will return the value of the `tools.build.cross_building:can_run` configuration in case it's set. Otherwise it will return if we are cross-building or not. It's a useful feature for the case that your architecture can run more than one target. For instance, Mac M1 machines can run both *armv8* and *x86_64*.
- We run the example binary that was generated in the `self.cpp.build.bindir` folder using the environment information that Conan put in the run environment. Conan will then invoke a launcher containing the runtime environment information, anything that is necessary for the environment to run the compiled executables and applications.

Now that we have gone through all the important bits of the code, let's try our *test_package*. Although we already learned that the *test_package* is invoked when we call `conan create`, you can also just create the *test_package* if you have already created the `hello/1.0` package in the Conan cache. This is done with the *conan test* command:

```
$ conan test test_package hello/1.0

...

-------- test_package: Computing necessary packages --------
Requirements
    fmt/8.1.1#cd132b054cf999f31bd2fd2424053ddc:ff7a496f48fca9a88dc478962881e015f4a5b98f
↪#1d9bb4c015de50bcb4a338c07229b3bc - Cache
    hello/1.0#25e0b5c00ae41ef9fbfbbb1e5ac86e1e:fd7c4113dad406f7d8211b3470c16627b54ff3af
↪#4ff3fd65a1d37b52436bf62ea6eaac04 - Cache
Test requirements
    gtest/1.17.0
↪#d136b3379fdb29bdfe31404b916b29e1:656efb9d626073d4ffa0dda2cc8178bc408b1bee
↪#ee8cbd2bf32d1c89e553bdd9d5606127 - Skip

...

[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example

-------- Testing the package: Running test() --------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release! (with color!)
```

As you can see in the output, our *test_package* builds successfully testing that the *hello/1.0* Conan package can be consumed with no problem.

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 12: Testing Conan Packages

3.2.10 Other types of packages

In the previous sections, we saw how to create a new recipe for a classic C++ library but there are other types of packages apart from libraries.

In this section, we will review how to create a recipe for *header-only* libraries, how to package already *built libraries*, and how to create recipes for *tool requires* and *applications*.

Header-only packages

In this section, we are going to learn how to create a recipe for a header-only library.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/other_packages/header_only
```

A header-only library is composed only of header files. That means a consumer doesn't link with any library but includes headers, so we need only one binary configuration for a header-only library.

In the *Create your first Conan package* section, we learned about settings and how building the recipe applying different `build_type` (Release/Debug) generates a new binary package.

As we only need one binary package, we don't need to declare the *settings* attribute. This is a basic recipe for a header-only recipe:

Listing 66: conanfile.py

```
from conan import ConanFile
from conan.tools.files import copy


class SumConan(ConanFile):
    name = "sum"
    version = "0.1"
    # No settings/options are necessary, this is header only
    exports_sources = "include/*"
    # We can avoid copying the sources to the build folder in the cache
    no_copy_source = True
    # Important, define the package_type
    package_type = "header-library"

    def package(self):
        # This will also copy the "include" folder
        copy(self, "*.h", self.source_folder, self.package_folder)

    def package_info(self):
        # For header-only packages, libdirs and bindirs are not used
        # so it's necessary to set those as empty.
        self.cpp_info.bindirs = []
        self.cpp_info.libdirs = []
```

Please note that we are setting `cpp_info.bindirs` and `cpp_info.libdirs` to `[]` because header-only libraries don't have compiled libraries or binaries, and since they default to `["bin"]` and `["lib"]`, it is necessary to override them.

Also check that we are setting the *no_copy_source* attribute to `True` so that the source code will not be copied from the `source_folder` to the `build_folder`. This is a typical optimization for header-only libraries to avoid extra copies.

Our header-only library is this simple function that sums two numbers:

Listing 67: include/sum.h

```
inline int sum(int a, int b){
    return a + b;
}
```

The folder *examples2/tutorial/creating_packages/other_packages/header_only* in the cloned project contains a `test_package` folder with an example of an application consuming the header-only library. So we can run the `conan create .` command to build the package and test the package:

```
$ conan create .
...
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example

-------- Testing the package: Running test() ----------
sum/0.1 (test package): Running test()
sum/0.1 (test package): RUN: ./example
1 + 3 = 4
```

After running `conan create`, a new binary package is created for the header-only library, and we can see how the `test_package` project can use it correctly.

We can list the binary packages created running this command:

```
$ conan list "sum/0.1#:*"
Local Cache
  sum
    sum/0.1
      revisions
        c1a714a086933b067bcbf12002fb0780 (2024-05-09 15:28:51 UTC)
          packages
            da39a3ee5e6b4b0d3255bfef95601890afd80709
              info
```

We get one package with the package ID `da39a3ee5e6b4b0d3255bfef95601890afd80709`. Let's see what happens if we run `conan create` specifying `-s build_type=Debug`:

```
$ conan create . -s build_type=Debug
$ conan list "sum/0.1#:*"
Local Cache
  sum
    sum/0.1
      revisions
        c1a714a086933b067bcbf12002fb0780 (2024-05-09 15:28:51 UTC)
          packages
            da39a3ee5e6b4b0d3255bfef95601890afd80709
              info
```

Even with the `test_package` executable being built for Debug, we get the same binary package for the header-only

library. This is because we didn't specify the `settings` attribute in the recipe, so the changes in the input settings (`-s build_type=Debug`) do not affect the recipe and therefore the generated binary package is always the same.

Header-only library with tests

In the previous example, we saw why a recipe for a header-only library shouldn't declare the `settings` attribute, but sometimes the recipe needs them to build some executable, for example, for testing the library. Nonetheless, the binary package of the header-only library should still be unique, so we are going to review how to achieve that.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/other_packages/header_only_gtest
```

We have the same header-only library that sums two numbers, but now we have this recipe:

```
import os
from conan import ConanFile
from conan.tools.files import copy
from conan.tools.cmake import cmake_layout, CMake


class SumConan(ConanFile):
    name = "sum"
    version = "0.1"
    settings = "os", "arch", "compiler", "build_type"
    exports_sources = "include/*", "test/*"
    no_copy_source = True
    generators = "CMakeToolchain", "CMakeDeps"
    # Important, define the package_type
    package_type = "header-library"

    def requirements(self):
        self.test_requires("gtest/1.17.0")

    def validate(self):
        check_min_cppstd(self, 11)

    def layout(self):
        cmake_layout(self)

    def build(self):
        if not self.conf.get("tools.build:skip_test", default=False):
            cmake = CMake(self)
            cmake.configure(build_script_folder="test")
            cmake.build()
            self.run(os.path.join(self.cpp.build.bindir, "test_sum"))

    def package(self):
        # This will also copy the "include" folder
        copy(self, "*.h", self.source_folder, self.package_folder)

    def package_info(self):
```

(continues on next page)

(continued from previous page)

```
        # For header-only packages, libdirs and bindirs are not used
        # so it's necessary to set those as empty.
        self.cpp_info.bindirs = []
        self.cpp_info.libdirs = []

    def package_id(self):
        self.info.clear()
```

These are the changes introduced in the recipe:

- We are introducing a `test_require` to `gtest/1.17.0`. A `test_require` is similar to a regular requirement but it is not propagated to the consumers and cannot conflict.
- `gtest` needs at least C++11 to build. So we introduced a `validate()` method calling `check_min_cppstd`.
- As we are building the `gtest` examples with CMake, we use the generators `CMakeToolchain` and `CMakeDeps`, and we declared the `cmake_layout()` to have a known/standard directory structure.
- We have a `build()` method, building the tests, but only when the standard conf `tools.build:skip_test` is not `True`. Use that conf as a standard way to enable/disable the testing. It is used by the helpers like `CMake` to skip the `cmake.test()` in case we implement the tests in CMake.
- We have a `package_id()` method calling `self.info.clear()`. This is internally removing all the information (settings, options, requirements) from the package_id calculation so we generate only one configuration for our header-only library.

We can call `conan create` to build and test our package.

```
$ conan create . -s compiler.cppstd=14 --build missing
...
Running main() from /Users/luism/.conan2/p/tmp/9bf83ef65d5ff0d6/b/googletest/
↪src/gtest_main.cc
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from SumTest
[ RUN      ] SumTest.BasicSum
[       OK ] SumTest.BasicSum (0 ms)
[----------] 1 test from SumTest (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
sum/0.1: Package 'da39a3ee5e6b4b0d3255bfef95601890afd80709' built
...
```

We can run `conan create` again specifying a different `compiler.cppstd` and the built package would be the same:

```
$ conan create . -s compiler.cppstd=17
...
sum/0.1: RUN: ./test_sum
Running main() from /Users/luism/.conan2/p/tmp/9bf83ef65d5ff0d6/b/googletest/
↪src/gtest_main.cc
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from SumTest
```

(continues on next page)

(continued from previous page)

```
[ RUN      ] SumTest.BasicSum
[       OK ] SumTest.BasicSum (0 ms)
[----------] 1 test from SumTest (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
sum/0.1: Package 'da39a3ee5e6b4b0d3255bfef95601890afd80709' built
```

Note: Once we have the `sum/0.1` binary package available (in a server, after a `conan upload`, or in the local cache), we can install it even if we don't specify input values for `os`, `arch`, ... etc. This is a new feature of Conan 2.X.

> We could call `conan install --require sum/0.1` with an empty profile and would get the binary package from the server. But if we miss the binary and we need to build the package again, it will fail because of the lack of settings.

See also:

- JFrog Academy Conan 2 Essentials Module 3, Lesson 14: Creating A Recipe For Header-Only Libraries

Package prebuilt binaries

There are specific scenarios in which it is necessary to create packages from existing binaries, for example from 3rd parties or binaries previously built by another process or team that is not using Conan. Under these circumstances, building from sources is not what you want.

You can package the local files in the following scenarios:

1. When you are developing your package locally and you want to quickly create a package with the built artifacts, but as you don't want to rebuild again (clean copy) your artifacts, you don't want to call **`conan create`**. This method will keep your local project build if you are using an IDE.
2. When you cannot build the packages from sources (when only pre-built binaries are available) and you have them in a local directory.
3. Same as 2 but you have the precompiled libraries in a remote repository.

Locally building binaries

Use the **`conan new`** command to create a "Hello World" C++ library example project:

```
$ conan new cmake_lib -d name=hello -d version=0.1
```

This will create a Conan package project with the following structure.

```
.
├── CMakeLists.txt
├── conanfile.py
├── include
│   └── hello.h
├── src
│   └── hello.cpp
```

(continues on next page)

(continued from previous page)

```
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

We have a `CMakeLists.txt` file in the root, a `src` folder with the `cpp` files, and an `include` folder for the headers. There's also a `test_package/` folder to test that the exported package is working correctly.

Now, for every different configuration (different compilers, architectures, build_type, ...):

1. We call **`conan install`** to generate the `conan_toolchain.cmake` file and the `CMakeUserPresets.json` that can be used in our IDE or when calling CMake (only >= 3.23).

   ```
   $ conan install . -s build_type=Release
   ```

2. We build our project calling CMake, our IDE, etc.:

   Listing 68: Linux, macOS

   ```
   $ mkdir -p build/Release
   $ cd build/Release
   $ cmake ../.. -DCMAKE_BUILD_TYPE=Release -DCMAKE_TOOLCHAIN_FILE=../Release/
   ↪generators/conan_toolchain.cmake
   $ cmake --build .
   ```

   Listing 69: Windows

   ```
   $ mkdir -p build
   $ cd build
   $ cmake ..  -DCMAKE_TOOLCHAIN_FILE=generators/conan_toolchain.cmake
   $ cmake --build . --config Release
   ```

   Note: As we are directly using our IDE or CMake to build the library, the `build()` method of the recipe is never called and could be removed.

3. We call **`conan export-pkg`** to package the built artifacts.

   ```
   $ cd ../..
   $ conan export-pkg . -s build_type=Release
   ...
   hello/0.1: Calling package()
   hello/0.1 package(): Packaged 1 '.h' file: hello.h
   hello/0.1 package(): Packaged 1 '.a' file: libhello.a
   ...
   hello/0.1: Package '54a3ab9b777a90a13e500dd311d9cd70316e9d55' created
   ```

   Let's look more closely at the package method. The generated `package()` method is using `cmake.install()` to copy the artifacts from our local folders to the Conan package.

   There is an alternative and generic `package()` method that could be used for any build system:

```
def package(self):
    local_include_folder = os.path.join(self.source_folder, self.cpp.source.
↪includedirs[0])
    local_lib_folder = os.path.join(self.build_folder, self.cpp.build.libdirs[0])
    copy(self, "*.h", local_include_folder, os.path.join(self.package_folder,
↪"include"), keep_path=False)
    copy(self, "*.lib", local_lib_folder, os.path.join(self.package_folder, "lib"),␣
↪keep_path=False)
    copy(self, "*.a", local_lib_folder, os.path.join(self.package_folder, "lib"),␣
↪keep_path=False)
```

This `package()` method is copying artifacts from the following directories that, thanks to the `layout()` method, will always point to the correct places:

- **os.path.join(self.source_folder, self.cpp.source.includedirs[0])** will always point to our local include folder.
- **os.path.join(self.build_folder, self.cpp.build.libdirs[0])** will always point to the location of the libraries when they are built, no matter if using a single-config CMake Generator or a multi-config one.

4. We can test the built package calling **`conan test`**:

```
$ conan test test_package/conanfile.py hello/0.1 -s build_type=Release

-------- Testing the package: Running test() ----------
hello/0.1 (test package): Running test()
hello/0.1 (test package): RUN: ./example
hello/0.1: Hello World Release!
  hello/0.1: __x86_64__ defined
  hello/0.1: __cplusplus199711
  hello/0.1: __GNUC__4
  hello/0.1: __GNUC_MINOR__2
  hello/0.1: __clang_major__13
  hello/0.1: __clang_minor__1
  hello/0.1: __apple_build_version__13160021
```

Now you can try to generate a binary package for `build_type=Debug` running the same steps but changing the `build_type`. You can repeat this process any number of times for different configurations.

Packaging already pre-built binaries

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/other_packages/prebuilt_binaries
```

This is an example of scenario 2 explained in the introduction. If you have a local folder containing the binaries for different configurations, you can package them using the following approach.

These are the files of our example, (be aware that the library files are only empty files so not valid libraries):

```
.
├── conanfile.py
└── vendor_hello_library
```

(continues on next page)

(continued from previous page)

```
    ├── linux
    │   ├── armv8
    │   │   ├── include
    │   │   │   └── hello.h
    │   │   └── libhello.a
    │   └── x86_64
    │       ├── include
    │       │   └── hello.h
    │       └── libhello.a
    ├── macos
    │   ├── armv8
    │   │   ├── include
    │   │   │   └── hello.h
    │   │   └── libhello.a
    │   └── x86_64
    │       ├── include
    │       │   └── hello.h
    │       └── libhello.a
    └── windows
        ├── armv8
        │   ├── hello.lib
        │   └── include
        │       └── hello.h
        └── x86_64
            ├── hello.lib
            └── include
                └── hello.h
```

We have folders and subfolders corresponding to the settings `os` and `arch`. This the recipe of our example:

```
import os
from conan import ConanFile
from conan.tools.files import copy


class helloRecipe(ConanFile):
    name = "hello"
    version = "0.1"
    settings = "os", "arch"

    def layout(self):
        _os = str(self.settings.os).lower()
        _arch = str(self.settings.arch).lower()
        self.folders.build = os.path.join("vendor_hello_library", _os, _arch)
        self.folders.source = self.folders.build
        self.cpp.source.includedirs = ["include"]
        self.cpp.build.libdirs = ["."]

    def package(self):
        local_include_folder = os.path.join(self.source_folder, self.cpp.source.
↪includedirs[0])
        local_lib_folder = os.path.join(self.build_folder, self.cpp.build.libdirs[0])
```

(continues on next page)

(continued from previous page)

```
        copy(self, "*.h", local_include_folder, os.path.join(self.package_folder,
↪"include"), keep_path=False)
        copy(self, "*.lib", local_lib_folder, os.path.join(self.package_folder, "lib"),␣
↪keep_path=False)
        copy(self, "*.a", local_lib_folder, os.path.join(self.package_folder, "lib"),␣
↪keep_path=False)

    def package_info(self):
        self.cpp_info.libs = ["hello"]
```

- We are not building anything, so the `build` method is not useful here.
- We can keep the same `package` method from the previous example because the location of the artifacts is declared by the `layout()`.
- Both the source folder (with headers) and the build folder (with libraries) are in the same location, in a path that follows:

  ```
  vendor_hello_library/{os}/{arch}
  ```

- The headers are in the `include` subfolder of the `self.source_folder` (we declare it in `self.cpp.source.includedirs`).
- The libraries are in the root of the `self.build_folder` folder (we declare `self.cpp.build.libdirs = ["."]`).
- We removed the `compiler` and the `build_type` because we only have different libraries depending on the operating system and the architecture (it might be a pure C library).

Now, for each different configuration, we run the **conan export-pkg** command. Later, we can list the binaries so we can check we have one package for each precompiled library:

```
$ conan export-pkg . -s os="Linux" -s arch="x86_64"
$ conan export-pkg . -s os="Linux" -s arch="armv8"
$ conan export-pkg . -s os="Macos" -s arch="x86_64"
$ conan export-pkg . -s os="Macos" -s arch="armv8"
$ conan export-pkg . -s os="Windows" -s arch="x86_64"
$ conan export-pkg . -s os="Windows" -s arch="armv8"

$ conan list "hello/0.1#:*"
Local Cache
  hello
    hello/0.1
      revisions
        9c7634dfe0369907f569c4e583f9bc50 (2024-05-10 08:28:31 UTC)
          packages
            522dcea5982a3f8a5b624c16477e47195da2f84f
              info
                settings
                  arch: x86_64
                  os: Windows
            63fead0844576fc02943e16909f08fcdddd6f44b
              info
                settings
                  arch: x86_64
                  os: Linux
```

(continues on next page)

(continued from previous page)

```
            82339cc4d6db7990c1830d274cd12e7c91ab18a1
              info
                settings
                  arch: x86_64
                  os: Macos
            a0cd51c51fe9010370187244af885b0efcc5b69b
              info
                settings
                  arch: armv8
                  os: Windows
            c93719558cf197f1df5a7f1d071093e26f0e44a0
              info
                settings
                  arch: armv8
                  os: Linux
            dcf68e932572755309a5f69f3cee1bede410e907
              info
                settings
                  arch: armv8
                  os: Macos
```

In this example, we don't have a `test_package/` folder but you can provide one to test the packages like in the previous example.

Downloading and Packaging Pre-built Binaries

This is an example of scenario 3 explained in the introduction. If we are not building the libraries, we likely have them somewhere in a remote repository. In this case, creating a complete Conan recipe with the detailed retrieval of the binaries could be the preferred method, because it is reproducible, and the original binaries might be traced.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/other_packages/prebuilt_remote_binaries
```

Listing 70: conanfile.py

```
import os
from conan.tools.files import get, copy
from conan import ConanFile


class HelloConan(ConanFile):
    name = "hello"
    version = "0.1"
    settings = "os", "arch"

    def build(self):
        base_url = "https://github.com/conan-io/libhello/releases/download/0.0.1/"

        _os = {"Windows": "win", "Linux": "linux", "Macos": "macos"}.get(str(self.
↪settings.os))
```

(continues on next page)

(continued from previous page)

```
        _arch = str(self.settings.arch).lower()
        url = "{}/{}_{}.tgz".format(base_url, _os, _arch)
        get(self, url)

    def package(self):
        copy(self, "*.h", self.build_folder, os.path.join(self.package_folder, "include
↪"))
        copy(self, "*.lib", self.build_folder, os.path.join(self.package_folder, "lib"))
        copy(self, "*.a", self.build_folder, os.path.join(self.package_folder, "lib"))

    def package_info(self):
        self.cpp_info.libs = ["hello"]
```

Typically, pre-compiled binaries come for different configurations, so the only task that the `build()` method has to implement is to map the `settings` to the different URLs.

We only need to call **conan create** with different settings to generate the needed packages:

```
$ conan create . -s os="Linux" -s arch="x86_64"
$ conan create . -s os="Linux" -s arch="armv8"
$ conan create . -s os="Macos" -s arch="x86_64"
$ conan create . -s os="Macos" -s arch="armv8"
$ conan create . -s os="Windows" -s arch="x86_64"
$ conan create . -s os="Windows" -s arch="armv8"

$ conan list "hello/0.1#:*"
Local Cache
  hello
    hello/0.1
      revisions
        d8e4debf31f0b7b5ec7ff910f76f1e2a (2024-05-10 09:13:16 UTC)
          packages
            522dcea5982a3f8a5b624c16477e47195da2f84f
              info
                settings
                  arch: x86_64
                  os: Windows
            63fead0844576fc02943e16909f08fcdddd6f44b
              info
                settings
                  arch: x86_64
                  os: Linux
            82339cc4d6db7990c1830d274cd12e7c91ab18a1
              info
                settings
                  arch: x86_64
                  os: Macos
            a0cd51c51fe9010370187244af885b0efcc5b69b
              info
                settings
                  arch: armv8
                  os: Windows
            c93719558cf197f1df5a7f1d071093e26f0e44a0
```

(continues on next page)

(continued from previous page)

```
            info
              settings
                arch: armv8
                os: Linux
          dcf68e932572755309a5f69f3cee1bede410e907
            info
              settings
                arch: armv8
                os: Macos
```

It is recommended to include also a small consuming project in a `test_package` folder to verify the package is built correctly, and then upload it to a Conan remote with **`conan upload`**.

The same building policies apply: having a recipe fail if no Conan packages are created, and the **`--build`** argument is not defined. A typical approach for this kind of package could be to define a **`build_policy="missing"`**, especially if the URLs are also under the team's control. If they are external (on the internet), it could be better to create the packages and store them on your own Conan repository, so that the builds do not rely on third-party URLs being available.

See also:

- JFrog Academy Conan 2 Essentials Module 3, Lesson 15: Creating a Recipe For Prebuilt Binaries

Tool requires packages

In the "*Using build tools as Conan packages*" section, we learned how to use a tool require to build (or help building) our project or Conan package. In this section, we are going to learn how to create a recipe for a tool require.

Note: Best practice

`tool_requires` and tool packages are intended for executable applications, like `cmake` or `ninja` that can be used as `tool_requires("cmake/[>=3.25]")` by other packages to put those executables in their path. They are not intended for library-like dependencies (use `requires` for them), for test frameworks (use `test_requires`) or in general for anything that belongs to the "host" context of the final application. Do not abuse `tool_requires` for other purposes.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/creating_packages/other_packages/tool_requires/tool
```

A simple tool require recipe

This is a recipe for a (fake) application that receiving a path returns 0 if the path is secure. We can check how the following simple recipe covers most of the `tool-require` use-cases:

Listing 71: conanfile.py

```
import os
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout
from conan.tools.files import copy

```

(continues on next page)

(continued from previous page)

```
class secure_scannerRecipe(ConanFile):
    name = "secure_scanner"
    version = "1.0"
    package_type = "application"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"

    # Sources are located in the same place as this recipe, copy them to the recipe
    exports_sources = "CMakeLists.txt", "src/*"

    def layout(self):
        cmake_layout(self)

    def generate(self):
        tc = CMakeToolchain(self)
        tc.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def package(self):
        extension = ".exe" if self.settings_build.os == "Windows" else ""
        copy(self, "*secure_scanner{}".format(extension),
             self.build_folder, os.path.join(self.package_folder, "bin"), keep_
↪path=False)

    def package_info(self):
        self.buildenv_info.define("MY_VAR", "23")
```

There are a few relevant things in this recipe:

1. It declares `package_type = "application"`. This is optional but convenient, and will indicate to Conan that the current package doesn't contain headers or libraries to be linked. Consumers will know that this package is an application.
2. The `package()` method is packaging the executable into the `bin/` folder, that is declared by default as a bindir: `self.cpp_info.bindirs = ["bin"]`.
3. In the `package_info()` method, we are using `self.buildenv_info` to define an environment variable `MY_VAR` that will also be available to consumers.

Let's create a binary package for the `tool_require`:

```
$ conan create . --build-require
...
secure_scanner/1.0: Calling package()
secure_scanner/1.0: Copied 1 file: secure_scanner
secure_scanner/1.0 package(): Packaged 1 file: secure_scanner
...
Security Scanner: The path 'mypath' is secure!
```

Important: Use `--build-require` argument.

The `conan create` command by default creates packages for the "host" context, using the "host" profile. But if the package we are creating is intended to be used as a tool with `tool_requires`, then it needs to be built for the "build" context.

The `--build-require` argument specifies this. When this argument is provided, the current recipe binary will be built for the "build" context. Because the `secure_scanner/1.0` package is a package which executables run in the current "build" machine, not necessarily in the final "host" machine, that could be different to the build one, for example in the case of a cross-build.

The `--build-require` argument is necessary to build the `secure_scanner` package correctly as a build tool.

Let's review the `test_package/conanfile.py`:

```
from conan import ConanFile


class secure_scannerTestConan(ConanFile):
    settings = "os", "compiler", "build_type", "arch"

    def build_requirements(self):
        self.tool_requires(self.tested_reference_str)

    def test(self):
        extension = ".exe" if self.settings_build.os == "Windows" else ""
        self.run("secure_scanner{} mypath".format(extension))
```

We are requiring the `secure_scanner` package as `tool_require` doing `self.tool_requires(self.tested_reference_str)`. In the `test()` method, we are running the application because it is available in the PATH. In the next example, we are going to see why the executables from a `tool_require` are available to consumers.

Let's create a consumer recipe to test if we can run the `secure_scanner` application of the `tool_require` and read the environment variable. Go to the *examples2/tutorial/creating_packages/other_packages/tool_requires/consumer* folder:

Listing 72: conanfile.py

```
from conan import ConanFile

class MyConsumer(ConanFile):
    name = "my_consumer"
    version = "1.0"
    settings = "os", "arch", "compiler", "build_type"
    tool_requires = "secure_scanner/1.0"

    def build(self):
        extension = ".exe" if self.settings_build.os == "Windows" else ""
        self.run("secure_scanner{} {}".format(extension, self.build_folder))
        if self.settings_build.os != "Windows":
            self.run("echo MY_VAR=$MY_VAR")
        else:
            self.run("set MY_VAR")
```

In this simple recipe we are declaring a `tool_require` to `secure_scanner/1.0` and we are calling directly the

packaged application `secure_scanner` in the `build()` method, also printing the value of the `MY_VAR` env variable.

If we build the consumer:

```
$ conan build .

-------- Installing (downloading, building) binaries... --------
secure_scanner/1.0: Already installed!

-------- Finalizing install (deploy, generators) --------
...
conanfile.py (my_consumer/1.0): RUN: secure_scanner /Users/luism/workspace/examples2/
↪tutorial/creating_packages/other_packages/tool_requires/consumer
...
Security Scanner: The path '/Users/luism/workspace/examples2/tutorial/creating_packages/
↪other_packages/tool_requires/consumer' is secure!
...
MY_VAR=23
```

We can see that the executable returned 0 (because our folder is secure) and it printed `Security Scanner: The path is secure!` message. It also printed the "23" value assigned to `MY_VAR` but, why are these automatically available?

- The generators `VirtualBuildEnv` and `VirtualRunEnv` are automatically used.
- The `VirtualRunEnv` is reading the `tool-requires` and is creating a launcher like `conanbuildenv-release-x86_64.sh` appending all `cpp_info.bindirs` to the `PATH`, all the `cpp_info.libdirs` to the `LD_LIBRARY_PATH` environment variable and declaring each variable of `self.buildenv_info`.
- Every time conan executes `self.run`, it, by default, activates the `conanbuild.sh` file before calling any command. The `conanbuild.sh` is including the `conanbuildenv-release-x86_64.sh`, so the application is in the PATH and the environment variable "MYVAR" has the value declared in the `tool-require`.

Removing settings in package_id()

With the previous recipe, if we call **`conan create`** with different settings like different compiler versions, we will get different binary packages with a different `package ID`. This might be convenient to, for example, keep better traceability of our tools. In this case, the *compatibility.py* plugin can help to locate the best matching binary in case Conan doesn't find the binary for our specific compiler version.

But in some cases we might want to just generate a binary taking into account only the `os`, `arch` or at most adding the `build_type` to know if the application is built for Debug or Release. We can add a `package_id()` method to remove them:

Listing 73: conanfile.py

```
import os
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout
from conan.tools.files import copy


class secure_scannerRecipe(ConanFile):
    name = "secure_scanner"
    version = "1.0"
```

(continues on next page)

(continued from previous page)

```
    settings = "os", "compiler", "build_type", "arch"
    ...

    def package_id(self):
        del self.info.settings.compiler
        del self.info.settings.build_type
```

So, if we call **conan create** with different `build_type` we will get exactly the same `package_id`.

```
$ conan create .
...
Package '82339cc4d6db7990c1830d274cd12e7c91ab18a1' created

$ conan create . -s build_type=Debug
...
Package '82339cc4d6db7990c1830d274cd12e7c91ab18a1' created
```

We got the same binary `package_id`. The second command `conan create . -s build_type=Debug` created and overwrote the previous Release binary (it created a newer package revision), because they have the same `package_id` identifier. It is typical to create only the `Release` one, and if for any reason managing both Debug and Release binaries is intended, then the approach would be not removing the `del self.info.settings.build_type`.

See also:

- JFrog Academy Conan 2 Essentials Module 3, Lesson 16: Creating Tool Require Packages
- *Using the same requirement as a requires and as a tool_requires*
- Toolchains (compilers)
- *Usage of runenv_info*
- *More info on settings_target*

Note: The Conan 2 Essentials training course is available for free at the JFrog Academy, which covers the same topics as this documentation but in a more interactive way. You can access it here.

3.3 Working with Conan repositories

We already *learned how to download and use packages* from Conan Center that is the official repository for open source Conan packages. We also *learned how to create our own packages* and store them in the Conan local cache for reusing later. In this section, we cover how you can use Conan repositories to upload your recipes and binaries and store them for later use on another machine, project, or for sharing purposes.

First, we will cover how you can setup a Conan repository locally (you can skip this part if you already have a Conan remote configured). Then, we will explain how to upload packages to your own repositories and how to operate when you have multiple Conan remotes configured. We will also briefly cover how you can contribute to the Conan Center central repository.

Finally, we will explain the *local_recipes_index*, a special type of remote that allows the use of a source folder with recipes as a Conan remote repository.

3.3.1 Setting up a Conan remote

There are several options to set up a Conan repository:

For private development:

- *Artifactory Community Edition for C/C++*: Artifactory Community Edition (CE) for C/C++ is a completely free Artifactory server that implements both Conan and generic repositories. It is the recommended server for companies and teams wanting to host their own private repository. It has a web UI, advanced authentication and permissions, very good performance and scalability, a REST API, and can host generic artifacts (tarballs, zips, etc). Check *Artifactory Community Edition for C/C++* for more information.
- *Conan server*: Simple, free and open source, MIT licensed server that is part of the conan-io organization project. Check *Setting up a Conan Server* for more information.

Enterprise solutions:

- **Artifactory Pro**: Artifactory is the binary repository manager for all major packaging formats. It is the recommended remote type for enterprise and professional package management. Check the Artifactory Documentation for more information. For a comparison between Artifactory editions, check the Artifactory Comparison Matrix.

Artifactory Community Edition for C/C++

Artifactory Community Edition (CE) for C/C++ is the recommended server for development and hosting private packages for a team or company. It is completely free, and features a WebUI, advanced authentication and permissions, great performance and scalability, a REST API, a generic CLI tool and generic repositories to host any kind of source or binary artifact.

This is a very brief introduction to Artifactory CE. For the complete Artifactory CE documentation, visit Artifactory docs.

Running Artifactory CE

The recommended way of running Artifactory CE is using Docker. The latest image is `releases-docker.jfrog.io/jfrog/artifactory-cpp-ce:latest`:

```
$ docker run --name artifactory -d -e JF_SHARED_DATABASE_TYPE=derby -e JF_SHARED_
↪DATABASE_ALLOWNONPOSTGRESQL=true -e JF_JFCONNECT_ENABLED=false -p 8081:8081 -p␣
↪8082:8082 releases-docker.jfrog.io/jfrog/artifactory-cpp-ce:latest
```

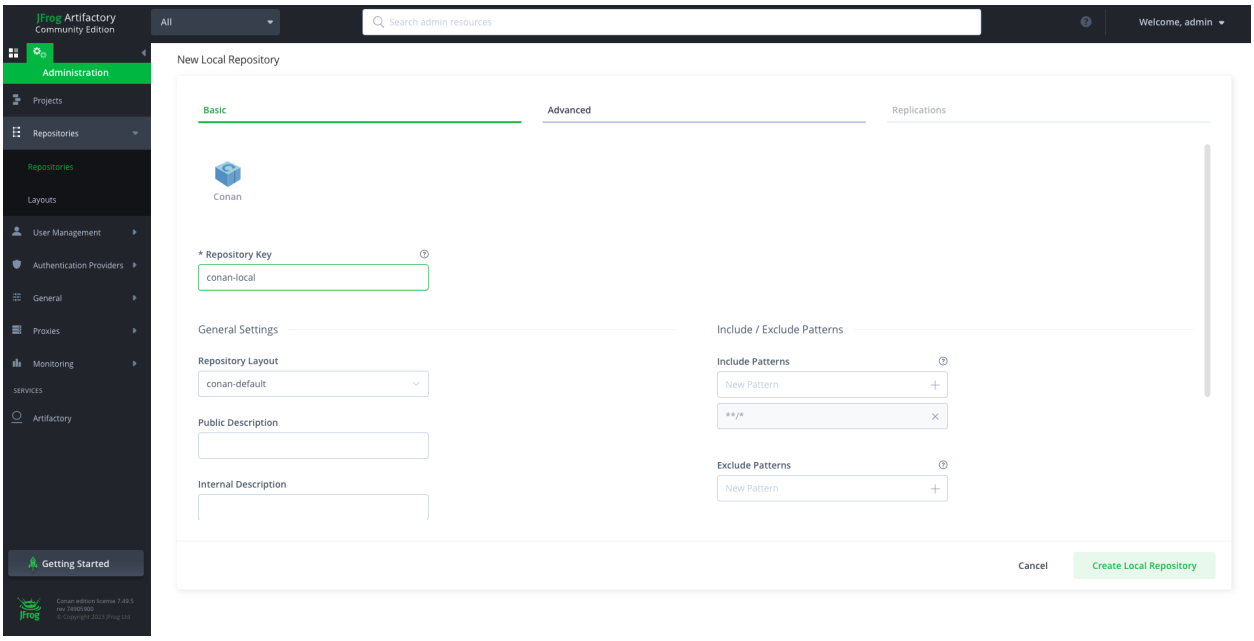
This is running Artifactory CE with an embedded Derby database. For better performance in production, you might want to check the Single node Artifactory installation and the full Artifactory installation guide.

For versions older than Artifactory 7.77, alternative installation methods like downloading installers from Download Page are available. After unzipping these installers, Artifactory can be launched by double-clicking the `artifactory.bat` on Windows or `artifactory.sh` script in the *app/bin* subfolder, depending on the OS.

Once Artifactory has started, navigate to the default URL *http://localhost:8082* where the Web UI should be running. The default user and password are `admin:password`.

Creating and Using a Conan Repo

Navigate to Administration -> Repositories -> Repositories, then click on the "Add Repositories" button and select "Local Repository". A dialog for selecting the package type will appear. Select **Conan**, then type a "Repository Key" (the name of the repository you are about to create), for example "conan-local", and click on "Create Local Repository". You can create multiple repositories to serve different flows, teams, or projects.



Now, let's configure the Conan client to connect with the "conan-local" repository. First, add the remote to the Conan remote registry:

```
$ conan remote add artifactory http://localhost:8081/artifactory/api/conan/conan-local
```

Then configure the credentials for the remote:

```
$ conan remote login artifactory <user> -p <password>
```

From now, you can upload, download, search, etc. the remote repos similarly to the other repo types.

```
$ conan upload <package_name> -r=artifactory
$ conan search "*" -r=artifactory
```

Setting up a Conan Server

Important: This server is mainly used for testing (though it might work fine for small teams). We recommend using the free *Artifactory Community Edition for C/C++* for private development or **Artifactory Pro** as Enterprise solution.

The **Conan Server** is a free and open source server that implements Conan remote repositories. It is a very simple application, used for testing inside the Conan client and distributed as a separate pip package.

Install the **Conan Server** using pip:

```
$ pip install conan-server
```

Then you can run the server:

```
$ conan_server
 ************************
 Using config: /Users/user/.conan_server/server.conf
 Storage: /Users/user/.conan_server/data
 Public URL: http://localhost:9300/v2
 PORT: 9300
 ************************
 Bottle v0.12.24 server starting up (using WSGIRefServer())...
 Listening on http://0.0.0.0:9300/
 Hit Ctrl-C to quit.
```

Note: On Windows, you may experience problems with the server if you run it under Bash/MSYS2. It is recommended to launch it in a regular `cmd` window.

See also:

- *Conan Server reference*

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 13: Working with Conan Repositories

3.3.2 Uploading Packages

In the previous section, we learned how to *set up a Conan repository*. Now we will go through the process of uploading both recipes and binaries to this remote and store them for later use on another machine, project, or for sharing purposes.

First, check if the remote you want to upload to is already in your current remote list:

```
$ conan remote list
```

You can search any remote in the same way as you search your Conan local cache. Actually, many Conan commands optionally accept a specific remote:

```
$ conan search "*" -r=my_local_server
```

Now, upload the package recipe and all the packages to your remote. In this example, we are using our `my_local_server` remote, but you could use any other:

```
$ conan upload hello -r=my_local_server
```

Now try again to read the information from the remote. We refer to it as remote, even if it is running on your local machine, as it could be running on another server in your LAN:

```
$ conan search hello -r=my_local_server
```

Now we can check if we can download the packages and use them in a project. For that purpose, we first have to **remove the local copies**, otherwise the remote packages will not be downloaded. Since we have just uploaded them, they are identical to the local ones.

```
$ conan remove hello -c
$ conan list hello
```

Now, to install the uploaded package from the Conan repository just run:

```
$ conan install --requires=hello/1.0 -r=my_local_server
```

You can check whether the package exists on your local computer again with:

```
$ conan list hello
```

See also:

- JFrog Academy Conan 2 Essentials Module 2, Lesson 13: Working with Conan Repositories
- *conan upload command reference*
- *conan remote command reference*
- *conan search command reference*

3.3.3 Contributing to Conan Center

Note: Default Remote Update in Conan 2.9.2

Starting from **Conan version 2.9.2**, the default remote has been changed to *https://center2.conan.io*. The previous default remote *https://center.conan.io* is now frozen and will no longer receive updates. It is recommended to update your remote configuration to use the new default remote to ensure access to the latest recipes and package updates (for more information, please read this post).

If you still have the deprecated remote configured as the default, please update using the following command:

```
conan remote update conancenter --url="https://center2.conan.io"
```

Contribution of packages to ConanCenter is done via pull requests to the Github repository in https://github.com/conan-io/conan-center-index. The C3I (ConanCenter Continuous Integration) service will build binaries automatically from those pull requests, and once merged, will upload them to ConanCenter package repository.

Read more about how to submit a pull request to conan-center-index source repository.

3.3.4 Local Recipes Index Repository

The *local_recipes_index* repository is an **experimental** special type of repository to which you cannot upload packages or store binaries. The purpose of this remote is:

- Enable contributors to share package recipes with the community, particularly for libraries that might not be suitable for ConanCenter.
- Simplify the process of building binaries from a private *conan-center-index* fork, allowing absolute control over recipes, customization, and maintaining a stable repository snapshot. This ensures robustness against upstream changes in ConanCenter. For detailed setup and usage instructions, see the dedicated section in the Conan DevOps Guide *Local Recipes Index Repository*.

Setup

To set up a local recipes index repository to share your own recipes, you need to organize your recipes in a folder structure that mimics that of *conan-center-index*. To get started, you can use the *local_recipes_index* template for the *conan new* command. For demonstration purposes, let's create a *local-recipes-index* repository for a hypothetical *hello* library, with a license incompatible with Conan Center, using the *local_recipes_index* template for the **conan new** command:

```
$ mkdir repo && cd repo
$ conan new local_recipes_index -d name=hello -d version=0.1 \
    -d url=https://github.com/conan-io/libhello/archive/refs/tags/0.0.1.zip \
    -d sha256=1dfb66cfd1e2fb7640c88cc4798fe25853a51b628ed9372ffc0ca285fe5be16b
$ cd ..
```

The **conan new local_recipes_index** command creates a template that assumes CMake as the build system alongside other heavy assumptions. In practice, it will require further customization, but for this demo, it works as-is. It will create a folder layout equal to the *conan-center-index* GitHub repository:

```
.
└── repo
    └── recipes
        └── hello
            ├── all
            │   ├── conandata.yml
            │   ├── conanfile.py
            │   └── test_package
            │       ├── CMakeLists.txt
            │       ├── conanfile.py
            │       └── src
            │           └── example.cpp
            └── config.yml
```

After setting up the repository, we add it as a local remote to Conan:

```
$ conan remote add mylocalrepo ./repo --allowed-packages="hello/*"
```

Please pay special attention to the **--allowed-packages** argument. This argument ensures that all packages other than *hello* are discarded by Conan. This can be used to minimize the surface area for a potential supply chain attack.

Now you can list and install packages from this new repository:

```
$ conan list "*" -r=mylocalrepo
$ conan install --requires=hello/0.1 -r=mylocalrepo --build=missing
```

At this point, you could push this repository to your GitHub account and share it with the community. Users then simply need to clone the GitHub repository and add the cloned folder as a local repository themselves.

Note: Please be aware that, as we commented earlier, this feature is specifically tailored for scenarios where certain libraries are not suitable for ConanCenter. Remember, a "local-recipes-index" repository has limitations: it is not fully reproducible as it models only versions and not revisions, and it does not provide binaries. Therefore, outside of these cases, it is advised to use a remote package server such as *Artifactory*.

See also:

- *DevOps guide*

- Introducing the Local-Recipes-Index Post

Note: The Conan 2 Essentials training course is available for free at the JFrog Academy, which covers the same topics as this documentation but in a more interactive way. You can access it here.

3.4 Developing packages locally

As we learned in *previous sections* of the tutorial, the most straightforward way to work when developing a Conan package is to run **`conan create`**. This means that every time it is run, Conan performs a series of costly operations in the Conan cache, such as downloading, decompressing, copying sources, and building the entire library from scratch. Sometimes, especially with large libraries, while we are developing the recipe, these operations cannot be performed every time.

This section will first show the **Conan local development flow**, that is, working on packages in your local project directory without having to export the contents of the package to the Conan cache first.

We will also cover how other packages can consume packages under development using **editable mode**.

Finally, we will explain the **Conan package layouts** in depth. It is the key feature that makes it possible to work with Conan packages in the Conan cache or locally without making any changes.

3.4.1 Package Development Flow

This section introduces the **Conan local development flow**, which allows you to work on packages in your local project directory without having to export the contents of the package to the Conan cache first.

This local workflow encourages users to perform trial-and-error in a local sub-directory relative to their recipe, much like how developers typically test building their projects with other build tools. The strategy is to test the *conanfile.py* methods individually during this phase.

Let's use this flow for the `hello` package we created in *the previous section*.

Please clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/developing_packages/local_package_development_flow
```

You can check the contents of the folder:

```
.
├── conanfile.py
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

conan source

You will generally want to start with the **conan source** command. The strategy here is that you're testing your source method in isolation and downloading the files to a temporary sub-folder relative to the *conanfile.py*. This relative folder is defined by the `self.folders.source` property in the `layout()` method. In this case, as we are using the pre-defined `cmake_layout`, we set the value with the `src_folder` argument.

Note: In this example, we are packaging a third-party library from a remote repository. In case you have your sources beside your recipe in the same repository, running **conan source** will not be necessary for most of the cases.

Let's have a look at the recipe's `source()` and `layout()` method:

```
...

def source(self):
    # Please be aware that using the head of the branch instead of an immutable tag
    # or commit is not a good practice in general.
    get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",
        strip_root=True)

def layout(self):
    cmake_layout(self, src_folder="src")

...
```

Now run the **conan source** command and check the results:

```
$ conan source .
conanfile.py (hello/1.0): Calling source() in /Users/.../local_package_development_flow/
↪src
Downloading main.zip
conanfile.py (hello/1.0): Unzipping 3.7KB
Unzipping 100%
```

You can see that a new *src* folder has appeared containing all the *hello* library sources.

```
.
├── conanfile.py
├── src
│   ├── CMakeLists.txt
│   ├── LICENSE
│   ├── README.md
│   ├── include
│   │   └── hello.h
│   └── src
│       └── hello.cpp
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

Now it's easy to check the sources and validate them. Once you've got your source method right and it contains the files you expect, you can move on to testing the various attributes and methods related to downloading dependencies.

conan install

After running the **conan source** command, you can run the **conan install** command. This command will install all the recipe requirements if needed and prepare all the files necessary for building by running the `generate()` method.

We can check all the parts from our recipe that are involved in this step:

```
...

class helloRecipe(ConanFile):

    ...

    generators = "CMakeDeps"

    ...

    def layout(self):
        cmake_layout(self, src_folder="src")

    def generate(self):
        tc = CMakeToolchain(self)
        tc.generate()

    ...
```

Now run the **conan install** command and check the results:

```
$ conan install .
...
-------- Finalizing install (deploy, generators) --------
conanfile.py (hello/1.0): Writing generators to ...
conanfile.py (hello/1.0): Generator 'CMakeDeps' calling 'generate()'
conanfile.py (hello/1.0): Calling generate()
...
conanfile.py (hello/1.0): Generating aggregated env files
```

You can see that a new *build* folder appeared with all the files that Conan needs for building the library like a toolchain for CMake and several environment configuration files.

```
.
├── build
│   └── Release
│       └── generators
│           ├── CMakePresets.json
│           ├── cmakedeps_macros.cmake
│           ├── conan_toolchain.cmake
│           ├── conanbuild.sh
│           ├── conanbuildenv-release-x86_64.sh
│           ├── conanrun.sh
│           ├── conanrunenv-release-x86_64.sh
│           ├── deactivate_conanbuild.sh
│           └── deactivate_conanrun.sh
├── conanfile.py
```

(continues on next page)

(continued from previous page)

```
├── src
│   ├── CMakeLists.txt
│   ├── CMakeUserPresets.json
│   ├── LICENSE
│   ├── README.md
│   ├── include
│   │   └── hello.h
│   └── src
│       └── hello.cpp
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

Now that all the files necessary for building are generated, you can move on to testing the `build()` method.

conan build

Running the **conan build** command will invoke the `build()` method:

```
...

class helloRecipe(ConanFile):

    ...

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    ...
```

Let's run **conan build**:

```
$ conan build .
...
-- Conan toolchain: C++ Standard 11 with extensions ON
-- Conan toolchain: Setting BUILD_SHARED_LIBS = OFF
-- Configuring done
-- Generating done
-- Build files have been ...
conanfile.py (hello/1.0): CMake command: cmake --build ...
conanfile.py (hello/1.0): RUN: cmake --build ...
[100%] Built target hello
```

For most recipes, the `build()` method should be very simple, and you can also invoke the build system directly, without invoking Conan, as you have all the necessary files available for building. If you check the contents of the *src* folder, you'll find a *CMakeUserPresets.json* file that you can use to configure and build the *conan-release* preset. Let's try it:

```
$ cd src
$ cmake --preset conan-release
...
-- Configuring done
-- Generating done

$ cmake --build --preset conan-release
...
[100%] Built target hello
```

You can check that the results of invoking CMake directly are equivalent to the ones we got using the **conan build** command.

Note: We use CMake presets in this example. This requires CMake >= 3.23 because the "include" from `CMakeUserPresets.json` to `CMakePresets.json` is only supported since that version. If you prefer not to use presets you can use something like:

```
cmake <path> -G <CMake generator> -DCMAKE_TOOLCHAIN_FILE=<path to
conan_toolchain.cmake> -DCMAKE_BUILD_TYPE=Release
```

Conan will show the exact CMake command everytime you run `conan install` in case you can't use the presets feature.

---

conan export-pkg

Now that we've built the package binaries locally, we can also package those artifacts in the Conan local cache using the **conan export-pkg** command. Please note that this command will create the package in the Conan cache and test it by running the *test_package* afterwards.

```
$ conan export-pkg .
conanfile.py (hello/1.0) package(): Packaged 1 '.h' file: hello.h
conanfile.py (hello/1.0) package(): Packaged 1 '.a' file: libhello.a
conanfile.py (hello/1.0): Package 'b1d267f77ddd5d10d06d2ecf5a6bc433fbb7eeed' created
conanfile.py (hello/1.0): Created package revision f09ef573c22f3919ba26ee91ae444eaa
...
conanfile.py (hello/1.0): Package folder /Users/...
conanfile.py (hello/1.0): Exported package binary
...
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example

-------- Testing the package: Running test() --------
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: ./example
hello/1.0: Hello World Release!
hello/1.0: __x86_64__ defined
hello/1.0: __cplusplus201103
hello/1.0: __GNUC__4
hello/1.0: __GNUC_MINOR__2
```

(continues on next page)

(continued from previous page)

```
hello/1.0: __clang_major__14
hello/1.0: __apple_build_version__14000029
```

Now you can list the packages in the local cache and check that the `hello/1.0` package was created:

```
$ conan list hello/1.0
Local Cache
  hello
    hello/1.0
```

See also:

- Reference for conan *source*, *install*, *build*, *export-pkg* and *test* commands.
- Packaging prebuilt binaries *example*
- When you are locally developing packages, at some point you might need to step into dependencies code while debugging. Please read this *example how to debug and step into dependencies* for more information about this use case.

3.4.2 Packages in editable mode

The normal way of working with Conan packages is to run a `conan create` or `conan export-pkg` to store them in the local cache, so that consumers use the packages stored in the cache. In some cases, when you want to consume these packages while developing them, it can be tedious to run `conan create` each time you make changes to the package. For those cases, you can put your package in editable mode and consumers will be able to find the headers and artifacts in your local working directory, eliminating the need for packaging.

Let's see how we can put a package in editable mode and consume it from the local working directory.

Please, first of all, clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/developing_packages/editable_packages
```

There are two folders inside this project:

```
.
├── hello
│   ├── CMakeLists.txt
│   ├── conanfile.py
│   └── src
│       └── hello.cpp
└── say
    ├── CMakeLists.txt
    ├── conanfile.py
    ├── include
    │   └── say.h
    └── src
        └── say.cpp
```

- A "say" folder containing a full-fledged package, with its `conanfile.py` and its source code.
- A "hello" folder containing a simple consumer project with a `conanfile.py` and its source code, which depends on the `say/1.0` requirement.

We will put `say/1.0` in editable mode and show how the `hello` consumer can find `say/1.0` headers and binaries in its local working directory.

Put say/1.0 package in editable mode

To avoid creating the package `say/1.0` in the cache for every change, we are going to put that package in editable mode, creating **a link from the reference in the cache to the local working directory**:

```
$ conan editable add say
$ conan editable list
say/1.0
    Path: /Users/.../examples2/tutorial/developing_packages/editable_packages/say/
↪conanfile.py
```

From now on, every usage of `say/1.0` by any other Conan package or project will be redirected to the `/Users/.../examples2/tutorial/developing_packages/editable_packages/say/conanfile.py` user folder instead of using the package from the Conan cache.

Note that the key of editable packages is a correct definition of the `layout()` of the package. Read the *package layout() section* to learn more about this method.

In this example, the `say conanfile.py` recipe is using the predefined `cmake_layout()` which defines the typical CMake project layout that can be different depending on the platform and generator used.

Now that the `say/1.0` package is in editable mode, let's build it locally:

```
$ cd say

# Windows: we will build two configurations to show multi-config
$ conan install . -s build_type=Release
$ conan install . -s build_type=Debug
$ cmake --preset conan-default
$ cmake --build --preset conan-release
$ cmake --build --preset conan-debug

# Linux, macOS: we will build only one configuration
$ conan install .
$ cmake --preset conan-release
$ cmake --build --preset conan-release
```

Note: We use CMake presets in this example. This requires CMake >= 3.23 because the "include" from `CMakeUserPresets.json` to `CMakePresets.json` is only supported since that version. If you prefer not to use presets you can use something like:

```
cmake <path> -G <CMake generator> -DCMAKE_TOOLCHAIN_FILE=<path to
conan_toolchain.cmake> -DCMAKE_BUILD_TYPE=Release
```

Conan will show the exact CMake command everytime you run `conan install` in case you can't use the presets feature.

Using say/1.0 package in editable mode

Consuming a package in editable mode is transparent from the consumer perspective. In this case we can build the `hello` application as usual:

```
$ cd ../hello

# Windows: we will build two configurations to show multi-config
$ conan install . -s build_type=Release
$ conan install . -s build_type=Debug
$ cmake --preset conan-default
$ cmake --build --preset conan-release
$ cmake --build --preset conan-debug
$ build\Release\hello.exe
say/1.0: Hello World Release!
...
$ build\Debug\hello.exe
say/1.0: Hello World Debug!
...

# Linux, macOS: we will only build one configuration
$ conan install .
$ cmake --preset conan-release
$ cmake --build --preset conan-release
$ ./build/Release/hello
say/1.0: Hello World Release!
```

As you can see, `hello` can successfully find the header and library files of the `say/1.0` package.

Working with editable packages

Once the above steps have been completed, you can work with your build system or IDE without involving Conan and make changes to the editable packages. Any changes will directly apply to consumers. Let's see how this works by making a change in the `say` source code:

```
$ cd ../say
# Edit src/say.cpp and change the error message from "Hello" to "Bye"

# Windows: we will build two configurations to show multi-config
$ cmake --build --preset conan-release
$ cmake --build --preset conan-debug

# Linux, macOS: we will only build one configuration
$ cmake --build --preset conan-release
```

And build and run the "hello" project:

```
$ cd ../hello

# Windows
$ cd build
$ cmake --build --preset conan-release
$ cmake --build --preset conan-debug
```

(continues on next page)

(continued from previous page)

```
$ Release\hello.exe
say/1.0: Bye World Release!
$ Debug\hello.exe
say/1.0: Bye World Debug!

# Linux, macOS
$ cmake --build --preset conan-release
$ ./hello
say/1.0: Bye World Release!
```

In this manner, you can develop both the `say` library and the `hello` application simultaneously without executing any Conan command in between. If you have both open in your IDE, you can simply build one after the other.

Building editable dependencies

If there are many editable dependencies, it might be inconvenient to go one by one, building them in the right order. It is possible to do an ordered build of the editable dependencies with the `--build` argument.

Let's clean the previous local executables first:

```
$ git clean -xdf
```

Using the `build()` method in the `hello/conanfile.py` recipe that we haven't really used so far (because we have been building directly calling `cmake`, not by calling `conan build` command), we can do such build with just:

```
$ conan build hello
```

Note that all we had to do to perform a full build of this project is these two commands. Starting from scratch in a different folder:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/developing_packages/editable_packages
$ conan editable add say
$ conan build hello --build=editable
```

Note that if we don't pass the `--build=editable` to `conan build hello`, the binaries for `say/0.1` that is in editable mode won't be available and it will fail. With the `--build=editable`, first a build of the `say` binaries is done locally and incrementally, and then another incremental build of `hello` will be done. Everything will still happen locally, with no packages built in the cache. If there are multiple `editable` dependencies, with nested transitive dependencies, Conan will build them in the right order.

If editable packages have dependants in the Conan cache, it is possible to force the rebuild from source of the cache dependants by using `--build=editable --build=cascade`. In general, this should be avoided, however. If it is needed to rebuild those dependencies, the recommendation is to put them in editable mode too.

Note that it is possible to build and test a package in editable mode with its own `test_package` folder. If a package is put in editable mode, and if it contains a `test_package` folder, the **`conan create`** command will still do a local build of the current package.

Revert the editable mode

In order to disable editable mode for a package, just remove the link using:

```
$ conan editable remove --refs=say/1.0
```

It will remove the link (the local directory won't be affected) and all the packages consuming this requirement will get it from the cache again.

Warning: Packages that are built while consuming an editable package in their upstreams can generate binaries and packages that are incompatible with the released version of the editable package. Avoid uploading these packages without re-creating them with the in-cache version of all libraries.

Note: For packages in editable mode, the `package_folder` of the dependency, as seen from the consumers (`self.dependencies["say"].package_folder`), points to the local project root folder, where there is no actual "package". Recipes, generators and deployers should use `cpp_info` to locate the artifacts, which works transparently for editable and cache packages. Note that `package_folder` can also be `None` for other dependencies, when the binary is skipped because it is not necessary. See the *dependencies reference* and the *full_deploy deployer* for more information.

3.4.3 Understanding the Conan Package layout

In the previous section, we introduced the concept of *editable packages* and mentioned that the reason they work *out of the box* when put in editable mode is due to the current definition of the information in the `layout()` method. Let's examine this feature in more detail.

In this tutorial, we will continue working with the `say/1.0` package and the `hello/1.0` consumer used in the *editable packages* tutorial.

Please, first of all, clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/developing_packages/package_layout
```

Note: We use CMake presets in this example. This requires CMake >= 3.23 because the "include" from `CMakeUserPresets.json` to `CMakePresets.json` is only supported since that version. If you prefer not to use presets you can use something like:

```
cmake <path> -G <CMake generator> -DCMAKE_TOOLCHAIN_FILE=<path to
conan_toolchain.cmake> -DCMAKE_BUILD_TYPE=Release
```

Conan will show the exact CMake command everytime you run `conan install` in case you can't use the presets feature.

As you can see, the main folder structure is the same:

```
.
├── hello
│   ├── CMakeLists.txt
│   ├── conanfile.py
```

(continues on next page)

(continued from previous page)

```
│   └── src
│       └── hello.cpp
└── say
    ├── CMakeLists.txt
    ├── conanfile.py
    ├── include
    │   └── say.h
    └── src
        └── say.cpp
```

The main difference here is that we are not using the predefined *cmake_layout()* in the `say/1.0` ConanFile, but instead, we are declaring our own custom layout. Let's see how we describe the information in the `layout()` method so that it works both when we create the package in the Conan local cache and also when the package is in editable mode.

Listing 74: say/conanfile.py

```
import os
from conan import ConanFile
from conan.tools.cmake import CMake


class SayConan(ConanFile):
    name = "say"
    version = "1.0"

    exports_sources = "CMakeLists.txt", "src/*", "include/*"

    ...

    def layout(self):

        ## define project folder structure

        self.folders.source = "."
        self.folders.build = os.path.join("build", str(self.settings.build_type))
        self.folders.generators = os.path.join(self.folders.build, "generators")

        ## cpp.package information is for consumers to find the package contents in the␣
↪Conan cache

        self.cpp.package.libs = ["say"]
        self.cpp.package.includedirs = ["include"] # includedirs is already set to
↪'include' by
                                                   # default, but declared for completion
        self.cpp.package.libdirs = ["lib"]         # libdirs is already set to 'lib' by
                                                   # default, but declared for completion

        ## cpp.source and cpp.build information is specifically designed for editable␣
↪packages:

        # this information is relative to the source folder that is '.'
        self.cpp.source.includedirs = ["include"] # maps to ./include
```

(continues on next page)

(continued from previous page)

```
        # this information is relative to the build folder that is './build/<build_type>',
↪ so it will
        self.cpp.build.libdirs = ["."]  # map to ./build/<build_type> for libdirs

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
```

Let's review the `layout()` method. You can see that we are setting values for `self.folders` and `self.cpp`. Let's explain what these values do.

self.folders

Defines the structure of the `say` project for the source code and the folders where the files generated by Conan and the built artifacts will be located. This structure is independent of whether the package is in editable mode or exported and built in the Conan local cache. Let's define the folder structure for the `say` package:

```
say
  ├── CMakeLists.txt
  ├── conanfile.py
  ├── include
  │   └── say.h
  ├── src
  │   └── say.cpp
  └── build
      ├── Debug            --> Built artifacts for Debug
      │   └── generators   --> Conan generated files for Debug config
      └── Release          --> Built artifacts for Release
          └── generators   --> Conan generated files for Release config
```

- As we have our `CMakeLists.txt` in the `.` folder, `self.folders.source` is set to `.`.
- We set `self.folders.build` to be **./build/Release** or **./build/Debug** depending on the `build_type` setting. These are the folders where we want the built binaries to be located.
- The `self.folders.generators` folder is the location we set for all the files created by the Conan generators. In this case, all the files generated by the `CMakeToolchain` generator will be stored there.

Note: Please note that the values above are for a single-configuration CMake generator. To support multi-configuration generators, such as Visual Studio, you should make some changes to this layout. For a complete layout that supports both single-config and multi-config, please check the *cmake_layout()* in the Conan documentation.

---

self.cpp

This attribute is used to define **where consumers will find the package contents** (header files, libraries, etc.) depending on whether the package is in editable mode or not.

cpp.package

First, we set the information for `cpp.package`. This defines the contents of the package and its location relative to the folder where the package is stored in the local cache. Please note that defining this information is equivalent to defining `self.cpp_info` in the `package_info()` method. This is the information we defined:

- `self.cpp.package.libs`: we add the `say` library so that consumers know that they should link with it. This is equivalent to declaring `self.cpp_info.libs` in the `package_info()` method.
- `self.cpp.package.libdirs`: we add the `lib` folder so that consumers know that they should search there for the libraries. This is equivalent to declaring `self.cpp_info.libdirs` in the `package_info()` method. Note that the default value for `libdirs` in both `cpp_info` and `cpp.package` is `["lib"]`, so we could have omitted that declaration.
- `self.cpp.package.includedirs`: we add the `include` folder so that consumers know that they should search there for the library headers. This is equivalent to declaring `self.cpp_info.includedirs` in the `package_info()` method. Note that the default value for `includedirs` in both `cpp_info` and `cpp.package` is `["include"]`, so we could have omitted that declaration.

To check how this information affects consumers, we are going to first do a **`conan create`** on the `say` package:

```
$ cd say
$ conan create . -s build_type=Release
```

When we call **`conan create`**, Conan moves the recipe and sources declared in the recipe to be exported to the local cache to a recipe folder and after that, it will create a separate package folder to build the binaries and store the actual package contents. If you check in the `[YOUR_CONAN_HOME]/p` folder, you will find two new folders similar to these:

Tip: You could get the exact locations for these folders using the **`conan cache`** command or checking the output of the **`conan create`** command.

```
<YOUR_CONAN_HOME>/p
├── sayb3ea744527a91      --> folder for sources
│   └── ...
│
└── say830097e941e10      --> folder for building and storing the package binaries
    ├── b
    │   ├── build
    │   │   └── Release
    │   ├── include
    │   │   └── say.h
    │   └── src
    │       ├── hello.cpp
    │       └── say.cpp
    └── p
        ├── include       --> defined in cpp.package.includedirs
        │   └── say.h
```

(continues on next page)

(continued from previous page)

```
        └── lib           --> defined in cpp.package.libdirs
            └── libsay.a  --> defined in self.cpp.package.libs
```

You can identify there the structure we defined in the `layout()` method. If you build the `hello` consumer project now, it will search for all the headers and libraries of `say` in that folder inside the local cache in the locations defined by `cpp.package`:

```
$ cd ../hello
$ conan install . -s build_type=Release

# Linux, MacOS
$ cmake --preset conan-release --log-level=VERBOSE
# Windows
$ cmake --preset conan-default --log-level=VERBOSE

...
-- Conan: Target declared 'say::say'
-- Conan: Library say found <YOUR_CONAN_HOME>p/say8938ceae216fc/p/lib/libsay.a
-- Created target CONAN_LIB::say_say_RELEASE STATIC IMPORTED
-- Conan: Found: <YOUR_CONAN_HOME>p/p/say8938ceae216fc/p/lib/libsay.a
-- Configuring done
...

$ cmake --build --preset conan-release
[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX executable hello
[100%] Built target hello
```

cpp.source and cpp.build

We also defined `cpp.source` and `cpp.build` attributes in our recipe. These are only used when the package is in editable mode and point to the locations that consumers will use to find headers and binaries. We defined:

- `self.cpp.source.includedirs` set to `["include"]`. This location is relative to the `self.folders.source` that we defined to `.`. In the case of editable packages, this location will be the local folder where our project resides.
- `self.cpp.build.libdirs` set to `["."]`. This location is relative to the `self.folders.build` that we defined to **./build/<build_type>**. In the case of editable packages, this location will point to **<local_folder>/build/<build_type>**.

Note that other `cpp.source` and `cpp.build` definitions are also possible, with different meanings and purposes. For example:

- `self.cpp.source.libdirs` and `self.cpp.source.libs` could be used if we had pre-compiled libraries in the source repo, committed to git, for example. They are not a product of the build, but rather part of the sources.
- `self.cpp.build.includedirs` could be used for folders containing headers generated at build time, as it usually happens by some code generators that are invoked by the build before starting to compile the project.

To check how this information affects consumers, we are going to first put the `say` package in editable mode and build it locally.

```
$ cd ../say
$ conan editable add . --name=say --version=1.0
$ conan install . -s build_type=Release
$ cmake --preset conan-release
$ cmake --build --preset conan-release
```

If you check the contents of the say project's folder now, you can see that the output folders match the ones we defined with `self.folders`:

```
.
├── CMakeLists.txt
├── CMakeUserPresets.json
├── build
│   └── Release       --> defined in cpp.build.libdirs
│       ├── ...
│       ├── generators
│       │   ├── CMakePresets.json
│       │   ├── ...
│       │   └── deactivate_conanrun.sh
│       └── libsay.a  --> no need to define
├── conanfile.py
├── include           --> defined in cpp.source.includedirs
│   └── say.h
└── src
    ├── hello.cpp
    └── say.cpp
```

Now that we have the `say` package in editable mode, if we build the `hello` consumer project, it will search for all the headers and libraries of `say` in the folders defined by `cpp.source` and `cpp.build`:

```
$ cd ../hello
$ conan install . -s build_type=Release

# Linux, MacOS
$ cmake --preset conan-release --log-level=VERBOSE
# Windows
$ cmake --preset conan-default --log-level=VERBOSE

...
-- Conan: Target declared 'say::say'
-- Conan: Library say found <local_folder>/examples2/tutorial/developing_packages/
↪package_layout/say/build/Release/libsay.a
-- Conan: Found: <local_folder>/examples2/tutorial/developing_packages/package_layout/
↪say/build/Release/libsay.a
-- Configuring done
...

$ cmake --build --preset conan-release
[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX executable hello
[100%] Built target hello

$ conan editable remove --refs=say/1.0
```

Note: Please, note that we did not define `self.cpp.build.libs = ["say"]`. This is because the information set in `self.cpp.source` and `self.cpp.build` will be merged with the information set in `self.cpp.package` so that you only have to define things that change for the editable package. For the same reason, you could also omit setting `self.cpp.source.includedirs = ["include"]` but we left it there to show the use of `cpp.source`.

See also:

- Define the layout() *when you package third-party libraries*
- Define the layout() *when you have the conanfile in a subfolder*
- Define the layout() *when you want to handle multiple subprojects*

3.4.4 Workspaces

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

In the previous section, we worked with *editable packages* and how to define a custom layout. Let's introduce the concept of *workspace* and how to use it.

Introduction

A Conan *workspace* gives you the chance to manage several packages as `editable` mode in an *orchestrated* or *monolithic* (also called *super-build*) way:

- *orchestrated*, we denote Conan building the editable packages one by one starting on the applications/consumers if exist.
- *monolithic*, we denote the editable packages built as a monolith, generating a single result (generators, etc) for the whole workspace.

Notice that the packages added to the workspace are automatically resolved as `editable` ones. Those editable packages are named as workspace's `packages`.

How to define a workspace

Workspaces are defined by the files `conanws.yml` and/or `conanws.py` files. Any Conan workspace command will traverse up the file system from the current working directory to the filesystem root, until it finds one of those files. That will define the "root" workspace folder. The paths in the `conanws` file are intended to be relative to be relocatable if necessary, or could be committed to Git in monorepo-like projects.

Through the `conan workspace` command, we can open, add, and/or remove `packages` from the current workspace.

See also:

Read the *workspace* section. Read the *conan workspace command* section.

Monolithic build

Conan workspaces can be built as a single monolithic project (super-project), which can be very convenient. Let's see it with an example:

```
$ conan new workspace
$ conan workspace super-install
$ cmake --preset conan-release # use conan-default in Win
$ cmake --build --preset conan-release
```

Let's explain a bit what happened. At first, the `conan new workspace` created a template project with some relevant files and the following structure:

```
.
├── CMakeLists.txt
├── app1
│   ├── CMakeLists.txt
│   ├── conanfile.py
│   ├── src
│   │   ├── app1.cpp
│   │   ├── app1.h
│   │   └── main.cpp
│   └── test_package
│       └── conanfile.py
├── conanws.py
├── conanws.yml
├── liba
│   ├── CMakeLists.txt
│   ├── conanfile.py
│   ├── include
│   │   └── liba.h
│   ├── src
│   │   └── liba.cpp
│   └── test_package
│       ├── CMakeLists.txt
│       ├── conanfile.py
│       └── src
│           └── example.cpp
└── libb
    ├── CMakeLists.txt
    ├── conanfile.py
    ├── include
    │   └── libb.h
    ├── src
    │   └── libb.cpp
    └── test_package
        ├── CMakeLists.txt
        ├── conanfile.py
        └── src
            └── example.cpp
```

The root `CMakeLists.txt` defines the super-project with:

Listing 75: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.25)
project(monorepo CXX)

include(FetchContent)

function(add_project PACKAGE_NAME SUBFOLDER)
    message(STATUS "Adding project: ${PACKAGE_NAME}. Folder: ${SUBFOLDER}")
    FetchContent_Declare(
        ${PACKAGE_NAME}
        SOURCE_DIR ${CMAKE_CURRENT_LIST_DIR}/${SUBFOLDER}
        SYSTEM
        OVERRIDE_FIND_PACKAGE
    )
    FetchContent_MakeAvailable(${PACKAGE_NAME})
endfunction()

include(build/conanws_build_order.cmake)

foreach(pair ${CONAN_WS_BUILD_ORDER})
    string(FIND "${pair}" ":" pos)
    string(SUBSTRING "${pair}" 0 "${pos}" pkg)
    math(EXPR pos "${pos} + 1")  # Skip the separator
    string(SUBSTRING "${pair}" "${pos}" -1 folder)

    add_project(${pkg} ${folder})
    # This target should be defined in the liba/CMakeLists.txt, but we can fix it here
    get_target_property(target_type ${pkg} TYPE)
    if (NOT target_type STREQUAL "EXECUTABLE")
        add_library(${pkg}::${pkg} ALIAS ${pkg})
    endif()
endforeach()
```

So basically, the super-project uses `FetchContent` to add the subfolders' sub-projects. For this to work correctly, the subprojects must be CMake based subprojects with `CMakeLists.txt`. Also, the subprojects must define the correct targets as would be defined by the `find_package()` scripts, like `liba::liba`. If this is not the case, it is always possible to define some local `ALIAS` targets.

This super-build `CMakeLists.txt` defines dynamically the correct sub-projects order. Note that the `FetchContent` strategy requires to define the different sub-projects in the correct build-order. While this is easy for workspaces with very few packages, this can become a burden for larger workspaces. The definition of the build-order is done in the generated `conanws_build_order.cmake` file, that is created by the `conan workspace super-install` command calling the `conanws.py` workspace `build_order()` method. It is the responsibility of the workspace to translate the information of the `build_order()` to specifics of the build system. The exact implementation, like the `conanws_build_order.cmake` file, is not a "built-in" workspace feature, note this is only the example approach provided by the `conan new workspace` default template. Users can implement their own logic in their `conanws.py` files.

The other important part is the `conanws.py` file:

Listing 76: conanws.py

```
from conan import Workspace
from conan import ConanFile
from conan.tools.files import save
from conan.tools.cmake import CMakeDeps, CMakeToolchain, cmake_layout


class MyWs(ConanFile):
    """ This is a special conanfile, used only for workspace definition of layout
    and generators. It shouldn't have requirements, tool_requirements. It shouldn't have
    build() or package() methods
    """
    settings = "os", "compiler", "build_type", "arch"

    def generate(self):
        deps = CMakeDeps(self)
        deps.generate()
        tc = CMakeToolchain(self)
        tc.generate()

    def layout(self):
        cmake_layout(self)


class Ws(Workspace):
    def root_conanfile(self):
        return MyWs  # Note this is the class name

    def build_order(self, order):
        super().build_order(order)  # default behavior prints the build order
        pkglist = " ".join([f'{it["ref"].name}:{it["folder"]}' for level in order for it␣
↪in level])
        save(self, "build/conanws_build_order.cmake", f"set(CONAN_WS_BUILD_ORDER
↪{pkglist})")
```

The role of the `class MyWs(ConanFile)` embedded conanfile is important, it defines the super-project necessary generators and layout.

The `conan workspace super-install` does not install the different packages separately, for this command, the packages of the workspace no longer exist as independent entities, they are just treated as a single "node" in the dependency graph, as they will be part of the super-project build. So there is only a single generated `conan_toolchain.cmake` and a single common set of dependencies `xxx-config.cmake` files for all super-project external dependencies.

Note: Ambiguous dependencies

Because there is only a single generated set of dependency files for the whole super-project, monolithic builds do not support crossbuilding when a dependency is present both in the host and build context, because a dependency can only be represented by one single node, even if in a regular (non-workspace) graph it could legitimately appear more than once, for example once in the `host` context and once in the `build` context (a `tool_requires`), or once as a private (`visible=False`) requirement and once as a public one. Since Conan 2.33, when this ambiguity arises Conan resolves it with the following priority, emitting a warning each time it does so:

- If the nodes belong to different contexts, the `host` context one is used.

- Otherwise, if the nodes have different visibility, the visible one is used.

If neither rule resolves the ambiguity, for example two unrelated editable packages that each privately require a different version of a same-named dependency, no warning is emitted and the resulting node is arbitrary. It is recommended that workspace packages built as a monolith stick to a single context and avoid this kind of conflicting dependency.

In the `build_order(self, order)` method above, the `order` argument is an ordered list of lists representing the topologically sorted order of building. The elements of the inner lists represent packages in the workspace, and are dictionaries with the reference `ref` (of type *RecipeReference*) and the source folder `folder` of every package. This `folder` will be the `source_folder`, specified in the recipe `layout()`.

Note: Best practices

For workspace recipes, the recommendation is to keep a simple layout, with the `conanfile.py` in the root of each package repository, the source folder also being the root of the repository and the `CMakeLists.txt` in the root of the repository. That simplifies many tasks, like `git clone <repo> && cd repo && conan install/build`, and also makes workspaces easier to define and manage.

The template above worked without external dependencies, but everything would work the same when there are external dependencies. This can be tested with:

```
$ conan new cmake_lib -d name=mymath
$ conan create .
$ conan new workspace -d requires=mymath/0.1
$ conan workspace super-install
$ cmake ...
```

Note: The current `conan new workspace` generates a CMake based super project. But it is possible to define a super-project using other build systems, like a MSBuild solution file that adds the different `.vcxproj` subprojects. As long as the super-project knows how to aggregate and manage the sub-projects, this is possible.

It might also be possible for the `add()` method in the `conanws.py` to manage the addition of the subprojects to the super-project, if there is some structure.

Orchestrated build

Conan workspaces can also build the different `packages` separately, and taking into account if there are packages defined as consumers of the other ones.

Let's use another structure to understand better how it works. Now, let's create it from scratch with the `conan workspace init .` that creates an almost empty conanws.py/conanws.yml, and using the `conan new cmake_lib/cmake_exe` basic templates, that create regular CMake-based conan packages:

```
$ mkdir myproject && cd myproject
$ conan workspace init .
$ conan new cmake_lib -d name=hello -d version=1.0 -o hello
$ conan new cmake_exe -d name=app -d version=1.0 -d requires=hello/1.0 -o app
```

Those commands created a file structure like this:

```
.
├── conanws.py
├── conanws.yml
├── app
│   ├── CMakeLists.txt
│   ├── conanfile.py
│   ├── src
│   │   ├── app.cpp
│   │   ├── app.h
│   │   └── main.cpp
│   └── test_package
│       └── conanfile.py
└── hello
    ├── CMakeLists.txt
    ├── conanfile.py
    ├── include
    │   └── hello.h
    ├── src
    │   └── hello.cpp
    └── test_package
        ├── CMakeLists.txt
        ├── conanfile.py
        └── src
            └── example.cpp
```

Now, the `conanws.yml` is empty and the `conanws.py` has a quite minimal definition. Let's add the `app` application (consumes `hello`) and the `hello` lib as new `packages` to the workspace:

```
$ conan workspace add hello
Reference 'hello/1.0' added to workspace
$ conan workspace add app
Reference 'app/1.0' added to workspace
```

Defined the workspace's `packages`, we can build them and execute the application:

```
$ conan workspace build
$ app/build/Release/app
hello/1.0: Hello World Release!
...
app/1.0: Hello World Release!
...
```

For any feedback, please open new tickets in https://github.com/conan-io/conan/issues.

3.5 Versioning

This section of the tutorial introduces several concepts about versioning of packages.

First, explicit version updates and how to define versions of packages is explained.

Then, it will be introduced how `requires` with version ranges can help to automate updating to the latest versions.

There are some situations when recipes or source code are changed, but the version of the package is not increased. For those situations, Conan uses automatic `revisions` to be able to provide traceability and reproducibility of those changes.

Lockfiles are a common mechanism in package managers to be able to reproduce the same dependency graph later in time, even when new versions or revisions of dependencies are uploaded. Conan also provides lockfiles to be able to guarantee this reproducibility.

Finally, when different branches of a dependency graph `requires` different versions of the same package, that is called a "version conflict". The tutorial will also introduce these errors and how to address them.

3.5.1 Versions

This section explains how different versions of a given package can be created, first starting with manually changing the version attribute in the `conanfile.py` recipe, and then introducing the `set_version()` method as a mechanism to automate the definition of the package version.

Note: This section uses very simple, empty recipes without building any code, so without `build()`, `package()`, etc., to illustrate the versioning with the simplest possible recipes, and allowing the examples to run easily and to be very fast and simple. In real life, the recipes would be full-blown recipes as seen in previous sections of the tutorial, building actual libraries and packages.

Let's start with a very simple recipe:

Listing 77: conanfile.py

```
from conan import ConanFile

class pkgRecipe(ConanFile):
    name = "pkg"
    version = "1.0"

    # The recipe would export files and package them, but not really
    # necessary for the purpose of this part of the tutorial
    # exports_sources = "include/*"
    # def package(self):
    #     ...
```

That we can create `pkg/1.0` package with:

```
$ conan create .
...
pkg/1.0 .
...

$ conan list "pkg/*"
```

(continues on next page)

(continued from previous page)

```
Local Cache
  pkg
    pkg/1.0
```

If we now did some changes to the source files of this library, this would be a new version, and we could change the `conanfile.py` version to `version = "1.1"` and create the new `pkg/1.1` version:

```
# Make sure you modified conanfile.py to version=1.1
$ conan create .
...
pkg/1.1 .
...

$ conan list "pkg/*"
Local Cache
  pkg
    pkg/1.0
    pkg/1.1
```

As we can see, now we see in our cache both `pkg/1.0` and `pkg/1.1`. The Conan cache can store any number of different versions and configurations for the same `pkg` package.

Automating versions

Instead of manually changing the version in `conanfile.py`, it is possible to automate it with 2 different approaches.

First it is possible to provide the `version` directly in the command line. In the example above, we could remove the `version` attribute from the recipe and do:

```
# Make sure you removed the version attribute in conanfile.py
$ conan create . --version=1.2
...
pkg/1.2 .
...

$ conan list "pkg/*"
Local Cache
  pkg
    pkg/1.0
    pkg/1.1
    pkg/1.2
```

The other possibility is to use the `set_version()` method to define the version dynamically, for example, if the version already exists in the source code or in a text file, or it should be deduced from the git version.

Let's assume that we have a `version.txt` file in the repo, that contains just the version string `1.3`. Then, this can be done:

Listing 78: conanfile.py

```
from conan import ConanFile
from conan.tools.files import  load

```

(continues on next page)

(continued from previous page)

```
class pkgRecipe(ConanFile):
    name = "pkg"

    def set_version(self):
        self.version = load(self, "version.txt")
```

```
# No need to specify the version in CLI arg or in recipe attribute
$ conan create .
...
pkg/1.3 .
...

$ conan list "pkg/*"
Local Cache
  pkg
    pkg/1.0
    pkg/1.1
    pkg/1.2
    pkg/1.3
```

It is also possible to combine the command line version definition, falling back to reading from file if the command line argument is not provided with the following syntax:

Listing 79: conanfile.py

```
def set_version(self):
    # if self.version is already defined from CLI --version arg, it will
    # not load version.txt
    self.version = self.version or load(self, "version.txt")
```

```
# This will create the "1.4" version even if the version.txt file contains "1.3"
$ conan create . --version=1.4
...
pkg/1.4 .
...

$ conan list "pkg/*"
Local Cache
  pkg
    pkg/1.0
    pkg/1.1
    pkg/1.2
    pkg/1.3
    pkg/1.4
```

Likewise, it is possible to obtain the version from a Git tag:

Listing 80: conanfile.py

```
from conan import ConanFile
from conan.tools.scm import Git
```

(continues on next page)

(continued from previous page)

```
class pkgRecipe(ConanFile):
    name = "pkg"

    def set_version(self):
        git = Git(self)
        tag = git.run("describe --tags")
        self.version = tag
```

```
# assuming this is a git repo, and it was tagged to 1.5
$ git init .
$ git add .
$ git commit -m "initial commit"
$ git tag 1.5
$ conan create .
    ...
    pkg/1.5 .
    ...

$ conan list "pkg/*"
Local Cache
  pkg
    pkg/1.0
    pkg/1.1
    pkg/1.2
    pkg/1.3
    pkg/1.4
    pkg/1.5
```

Note: Best practices

- We could try to use something like the branch name or the commit as the version number. However this might have some disadvantages, for example, when this package is being required, it will need a explicit `requires = "pkg/commit"` in every other package recipe requiring this one, and it might be difficult to update consumers consistently, and to know if a newer or older dependency is being used.

Requiring the new versions

When a new package version is created, if other package recipes requiring this one contain a explicit `requires`, pinning the exact version like:

Listing 81: app/conanfile.py

```
from conan import ConanFile

class AppRecipe(ConanFile):
    name = "app"
    version = "1.0"
    requires = "pkg/1.0"
```

Then, installing or creating the app recipe will keep requiring and using the pkg/1.0 version and not the newer ones. To start using the new pkg versions, it is necessary to explicitly update the requires like:

Listing 82: app/conanfile.py

```
from conan import ConanFile

class AppRecipe(ConanFile):
    name = "app"
    version = "1.0"
    requires = "pkg/1.5"
```

This process, while it achieves very good reproducibility and traceability, can be a bit tedious if we are managing a large dependency graph and we want to move forward to use the latest dependencies versions faster and with less manual intervention. To automate this, the *version-ranges* explained in the next section can be used.

3.5.2 Version ranges

In the previous section, we ended with several versions of the pkg package. Let's remove them and create the following simple project:

Listing 83: pkg/conanfile.py

```
from conan import ConanFile

class pkgRecipe(ConanFile):
    name = "pkg"
```

Listing 84: app/conanfile.py

```
from conan import ConanFile

class appRecipe(ConanFile):
    name = "app"
    requires = "pkg/1.0"
```

Let's create pkg/1.0 and install app, to see it requires pkg/1.0:

```
$ conan remove "pkg*" -c
$ conan create pkg --version=1.0
... pkg/1.0 ...
$ conan install app
...
Requirements
    pkg/1.0
```

Then, if we create a new version of pkg/1.1, it will not automatically be used by app:

```
$ conan create pkg --version=1.1
... pkg/1.0 ...
# Note how this still uses the previous 1.0 version
$ conan install app
...
```

(continues on next page)

(continued from previous page)

```
Requirements
    pkg/1.0
```

So we could modify app conanfile to explicitly use the new pkg/1.1 version, but instead of that, let's use the following version-range expression (introduced by the [expression] brackets):

Listing 85: app/conanfile.py

```
from conan import ConanFile

class appRecipe(ConanFile):
    name = "app"
    requires = "pkg/[>=1.0 <2.0]"
```

When we now install the dependencies of app, it will automatically use the latest version in the range, even if we create a new one, without needing to modify the app conanfile:

```
# this will now use the newer 1.1
$ conan install app
...
Requirements
    pkg/1.1

$ conan create pkg --version=1.2
... pkg/1.2 ...
# Now it will automatically use the newest 1.2
$ conan install app
...
Requirements
    pkg/1.2
```

This holds as long as the newer version lies within the defined range, if we create a pkg/2.0 version, app will not use it:

```
$ conan create pkg --version=2.0
... pkg/2.0 ...
# Conan will use the latest in the range
$ conan install app
...
Requirements
    pkg/1.2
```

When using version ranges, versions in the cache are preferred over remote ones, so if you have a local pkg/1.2 package, it will be used instead of the remote one, even if the remote one is newer. To ensure you use the latest available one, you can use the --update argument in the install/create command. Note that the --update argument will look into all the remotes specified in the command for possible newer versions, and won't stop at the first newer one found.

Version ranges can be defined in several places:

- In conanfile.py recipes requires, tool_requires, test_requires, python_requires
- In conanfile.txt files in [requires], [tool_requires], [test_requires] sections
- In command line arguments like --requires= and --tool_requires.

- In profiles `[tool_requires]` section

Semantic versioning

The semantic versioning specification or semver, specifies that packages should be versioned using always 3 dot-separated digits like `MAJOR.MINOR.PATCH`, with very specific meanings for each digit.

Conan extends the semver specification to any number of digits, and also allows to include lowercase letters in it. This was done because during 1.X a lot of experience and feedback from users was gathered, and it became evident than in C++ the versioning scheme is often more complex, and users were demanding more flexibility, allowing versions like `1.2.3.a.8` if necessary.

Conan versions non-digit identifiers follow the same rules as package names, they can only contain lowercase letters. This is to avoid `1.2.3-Beta` to be a different version than `1.2.3-beta` which can be problematic, even a security risk.

The ordering of versions when necessary (for example to decide which is the latest version in a version range) is done by comparing individually each dot-separated entity in the version, from left to right. Digits will be compared numerically, so 2 < 11, and entries containing letters will be compared alphabetically (even if they also contain some numbers).

Similarly to the semver specification, Conan can manage **prereleases** and **builds** in the form: `VERSION-prerelease+build`. Conan will also order pre-releases and builds according to the same rules, and each one of them can also contain an arbitrary number of items, like `1.2.3-pre.1.2.1+build.45.a`. Note that the semver standard does not apply any ordering to builds, but Conan does, with the same logic that is used to order the main version and the pre-releases.

Important: Note that the ordering of pre-releases can be confusing at times. A pre-release happens earlier in time than the release it is qualifying. So `1.1-alpha.1` is older than `1.1`, not newer.

Range expressions

Range expressions can have comparison operators for the lower and higher bounds, separated with a space. Also, lower bounds and upper bounds in isolation are permitted, though they are generally not recommended under normal versioning schemes, specially the lower bound only. `requires = "pkg/[>=1.0 <2.0]"` will include versions like 1.0, 1.2.3 and 1.9, but will not include 0.3, 2.0 or 2.1 versions.

The tilde ~ operator can be used to define an "approximately" equal version range. `requires = "pkg/[~1]"` will include versions 1.3 and 1.8.1, but will exclude versions like 0.8 or 2.0. Likewise `requires = "pkg/[~2.5]"` will include 2.5.0 and 2.5.3, but exclude 2.1, 2.7, 2.8. As conan semver implementation allows multiple digits, expressions like `requires = "pkg/[~2.5.2]"` will be equivalent to `requires = "pkg/[>=2.5.2 < 2.6.0]"` and `requires = "pkg/[~2.5.1.3]"` will be equivalent to `requires = "pkg/[>=2.5.1.3 < 2.6.0.0]"`.

The caret ^ operator is very similar to the tilde, but allows variability over the digit following the first non-zero digit. `requires = "pkg/[^1.2]"` will include 1.2.1, 1.3 and 1.51, but will exclude 1.0, 2, 2.0. `^1.2.0` would act the same, while `^0.1.2` would include 0.1.2.1 and 0.1.3, but exclude 0.1.1 and 0.2.0.

It is also possible to apply multiple conditions with the OR operator, like `requires = "pkg/[>1 <2.0 || ^3.2]"` but this kind of complex expressions is not recommended in practice and should only be used in very extreme cases.

There is the possibility to use string-matching over the end of the string to simplify some ranges that otherwise would require more complicated conditions (this feature is experimental). Defining `requires=pkg/[1.2.3.*]` will match any version that starts exactly with `1.2.3.`, and discard others. For example it will match `1.2.3.5` and `1.2.3.abc`, but it will discard `1.2.3` (as it doesn't have the final dot).

Finally, note that pre-releases are not resolved by default. The way to include them in the range is to explicitly enable them with either the `include_prerelease` option (`requires = "pkg/[>1 <2, include_prerelease]"`), or via

the `core.version_ranges:resolve_prereleases=True` configuration, *which you can read more about here*. In this example, 1.0-pre.1 and 1.5.1-pre1 will be included, but 2.0-pre1 would be excluded.

Note: While it is possible to hardcode the `include_prerelease` in the `requires` version range, it is not recommended generally. Pre-releases should be opt-in, and controlled by the user, who decides if they want to use pre-releases. Also, note that the `include_prerelease` receives no argument, hence it's not possible to deactivate prereleases with `include_prerelease=False`.

For more information about valid range expressions go to *Requires reference*

Note that version ranges only control which version is resolved. Whether a new version within the range requires the consumer to rebuild depends on the `package_type` of both packages — see *The effect of dependencies on package_id*.

3.5.3 Revisions

This sections introduces how doing modifications to a given recipe or source code without explicitly creating new versions, will still internally track those changes with a mechanism called revisions.

Creating different revisions

Let's start with a basic "hello" package:

```
$ mkdir hello && cd hello
$ conan remove hello* -c # clean possible existing ones
$ conan new cmake_lib -d name=hello -d version=1.0
$ conan create .
hello/1.0: Hello World Release!
...
```

We can now list the existing recipe revisions in the cache:

```
$ conan list "hello/1.0#*"
Local Cache
  hello
    hello/1.0
      revisions
        2475ece651f666f42c155623228c75d2 (2023-01-31 23:08:08 UTC)
```

If we now edit the `src/hello.cpp` file, to change the output message from "Hello" to "Bye"

Listing 86: hello/src/hello.cpp

```
void hello(){

    #ifdef NDEBUG
    std::cout << "hello/1.0: Bye World Release!\n";
    ...
```

So if we create the package again, without changing the version `hello/1.0`, we will get a new output:

```
$ conan create .
hello/1.0: Bye World Release!
...
```

But even if the version is the same, internally a new revision 2b547b7f20f5541c16d0b5cbcf207502 has been created.

```
$ conan list "hello/1.0#*"
Local Cache
  hello
    hello/1.0
      revisions
        2475ece651f666f42c155623228c75d2 (2023-01-31 23:08:08 UTC)
        2b547b7f20f5541c16d0b5cbcf207502 (2023-01-31 23:08:25 UTC)
```

This recipe **revision** is the hash of the contents of the recipe, including the `conanfile.py`, and the exported sources (`src/main.cpp`, `CMakeLists.txt`, etc., that is, all files exported in the recipe).

We can now edit the `conanfile.py`, to define the `license` value:

Listing 87: hello/conanfile.py

```
class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    # Optional metadata
    license = "MIT"
    ...
```

So if we create the package again, the output will be the same, but we will also get a new revision, as the `conanfile.py` changed:

```
$ conan create .
hello/1.0: Bye World Release!
...
$ conan list "hello/1.0#*"
Local Cache
  hello
    hello/1.0
      revisions
        2475ece651f666f42c155623228c75d2 (2023-01-31 23:08:08 UTC)
        2b547b7f20f5541c16d0b5cbcf207502 (2023-01-31 23:08:25 UTC)
        1d674b4349d2b1ea06aa6419f5f99dd9 (2023-01-31 23:08:34 UTC)
```

Important: The recipe **revision** is the hash of the contents. It can be changed to be the Git commit hash with `revision_mode = "scm"`. But in any case it is critical that every revision represents an immutable source, including the recipe and the source code:

- If the sources are managed with `exports_sources`, then they will be automatically be part of the hash
- If the sources are retrieved from a external location, like a downloaded tarball or a git clone, that should guarantee uniqueness, by forcing the checkout of a unique immutable tag, or a commit. Moving targets like branch names or HEAD would be broken, as revisions are considered immutable.

Any change in source code or in recipe should always imply a new revision.

Warning: Line Endings Issue

Git, by default, will checkout files on Windows systems using CRLF line endings. This results in different files compared to Linux systems where files will use LF line endings. Since the files are different, the Conan recipe revision computed on Windows will differ from the revisions on other platforms like Linux. Please, check more about this issue and how to solve it in the *FAQ dedicated section*.

Using revisions

The recipe revisions are resolved by default to the latest revision for every given version. In the case above, we could have a `chat/1.0` package that consumes the above `hello/1.0` package:

```
$ cd ..
$ mkdir chat && cd chat
$ conan new cmake_lib -d name=chat -d version=1.0 -d requires=hello/1.0
$ conan create .
...
Requirements
chat/1.0#17b45a168519b8e0ed178d822b7ad8c8 - Cache
hello/1.0#1d674b4349d2b1ea06aa6419f5f99dd9 - Cache
...
hello/1.0: Bye World Release!
chat/1.0: Hello World Release!
```

We can see that by default, it is resolving to the latest revision `1d674b4349d2b1ea06aa6419f5f99dd9`, so we also see the `hello/1.0: Bye World` modified message.

It is possible to explicitly depend on a given revision in the recipes, so it is possible to modify the `chat/1.0` recipe to define it requires the first created revision:

Listing 88: chat/conanfile.py

```
def requirements(self):
    self.requires("hello/1.0#2475ece651f666f42c155623228c75d2")
```

So creating `chat` will now force the first revision:

```
$ conan create .
...
Requirements
chat/1.0#12f87e1b8a881da6b19cc7f229e16c76 - Cache
hello/1.0#2475ece651f666f42c155623228c75d2 - Cache
...
hello/1.0: Hello World Release!
chat/1.0: Hello World Release!
```

Note: Note that pinning a revision when using version ranges has not effect and Conan will warn about it.

Uploading revisions

The upload command will upload only the latest revision by default:

```
# upload latest revision only, all package binaries
$ conan upload hello/1.0 -c -r=myremote
```

If for some reason we want to upload all existing revisions, it is possible with:

```
# upload all revisions, all binaries for each revision
$ conan upload hello/1.0#* -c -r=myremote
```

In the server side, the latest uploaded revision becomes the latest one, and the one that will be resolved by default. For this reason, the above command uploads the different revisions in order (from older revision to latest revision), so the relative order of revisions is respected in the server side.

Note that if another machine decides to upload a revision that was created some time ago, it will still become the latest in the server side, because it is created in the server side with that time.

Package revisions

Package binaries when created also compute the hash of their contents, forming the **package revision**. But they are very different in nature to **recipe revisions**. Recipe revisions are naturally expected, every change in source code or in the recipe would cause a new recipe revision. But package binaries shouldn't have more than one **package revision**, because binaries variability would be already encoded in a unique `package_id`. Put in other words, if the recipe revision is the same (exact same input recipe and source code) and the `package_id` is the same (exact same configuration profile, settings, etc.), then that binary should be built only once.

As C and C++ build are not deterministic, it is possible that subsequents builds of the same package, without modifying anything will be creating new package revisions:

```
# Build again 2 times the latest
$ conan create .
$ conan create .
```

In some OSs like Windows, this build will not be reproducible, and the resulting artifacts will have different checksums, resulting in new package revisions:

```
$ conan list "hello/1.0:*#*"
Local Cache
  hello
    hello/1.0
      revisions
        1d674b4349d2b1ea06aa6419f5f99dd9 (2023-02-01 00:03:29 UTC)
          packages
            2401fa1d188d289bb25c37cfa3317e13e377a351
              revisions
                8b8c3deef5ef47a8009d4afaebfe952e (2023-01-31 23:08:40 UTC)
                8e8d380347e6d067240c4c00132d42b1 (2023-02-01 00:03:12 UTC)
                c347faaedc1e7e3282d3bfed31700019 (2023-02-01 00:03:35 UTC)
              info
                settings
                arch: x86_64
                build_type: Release
                ...
```

By default, the package revision will also be resolved to the latest one. However, it is not possible to pin a package revision explicitly in recipes, recipes can only require down to the recipe revision as we defined above.

Warning: Best practices

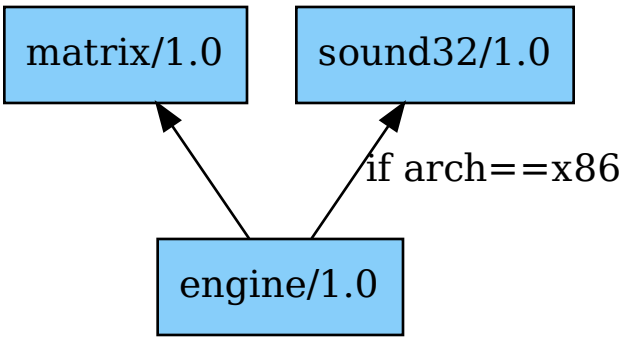
Having more than 1 package revision for any given recipe revision + `package_id` is a smell or a potential bad practice. It means that something was rebuilt when it was not necessary, wasting computing and storage resources. There are ways to avoid doing it: for example `conan create . --build=missing:hello*` only builds `hello` from source when its binary is still missing. You can also use `--build=missing:&`: the `&` pattern refers to the package being created, so if a compatible binary for it already exists, `conan create` will not rebuild it. `conan graph info` can also show what would need to be built.

3.5.4 Lockfiles

Lockfiles are a mechanism to achieve reproducible dependencies, even when new versions or revisions of those dependencies are created. Let's see it with a practical example, start cloning the examples2 repository:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/versioning/lockfiles/intro
```

In this folder we have a small project, consisting in 3 packages: a `matrix` package, emulating some mathematical library, an `engine` package emulating some game engine, and a `sound32` package, emulating a sound library for some 32bits systems. These packages are actually most empty, they do not build any code, but they are good to learn the concepts of lockfiles.



We will start by creating the first `matrix/1.0` version:

```
$ conan create matrix --version=1.0
```

Now we can check in the `engine` folder its recipe:

```
class Engine(ConanFile):
    name = "engine"
    settings = "arch"

    def requirements(self):
        self.requires("matrix/[>=1.0 <2.0]")
        if self.settings.arch == "x86":
            self.requires("sound32/[>=1.0 <2.0]")
```

Lets move to the `engine` folder and install its dependencies:

```
$ cd engine
$ conan install .
...
Requirements
    matrix/1.0#905c3f0babc520684c84127378fefdd0 - Cache
Resolved version ranges
    matrix/[>=1.0 <2.0]: matrix/1.0
```

As the `matrix/1.0` version is in the valid range, it is resolved and used. But if someone creates a new `matrix/1.1` or `1.X` version, it would also be automatically used, because it is also in the valid range. To avoid this, we will capture a "snapshot" of the current dependencies creating a `conan.lock` lockfile:

```
$ conan lock create .
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "matrix/1.0#905c3f0babc520684c84127378fefdd0%1675278126.0552447"
    ],
    "build_requires": [],
    "python_requires": []
}
```

We can see how the created `conan.lock` lockfile contains the `matrix/1.0` version and its revision. But `sound32/1.0` is not in the lockfile, because for the default configuration profile (not `x86`), this `sound32` is not a dependency.

Now, a new `matrix/1.1` version is created:

```
$ cd ..
$ conan create matrix --version=1.1
$ cd engine
```

And see what happens when we issue a new `conan install` command for the engine:

```
$ conan install .
# equivalent to conan install . --lockfile=conan.lock
...
Requirements
   matrix/1.0#905c3f0babc520684c84127378fefdd0 - Cache
```

As we can see, the new `matrix/1.1` was not used, even if it is in the valid range! This happens because by default the `--lockfile=conan.lock` will be used if the `conan.lock` file is found. The locked `matrix/1.0` version and revision will be used to resolve the range, and the `matrix/1.1` will be ignored.

Likewise, it is possible to issue other Conan commands, and if the `conan.lock` is there, it will be used:

```
$ conan graph info . --filter=requires # --lockfile=conan.lock is implicit
# display info for matrix/1.0
$ conan create . --version=1.0 # --lockfile=conan.lock is implicit
# creates the engine/1.0 package, using matrix/1.0 as dependency
```

If using a lockfile is intended, like in CI, it is better that the argument `--lockfile=conan.lock` explicit.

Multi-configuration lockfiles

We saw above that the `engine` has a conditional dependency to the `sound32` package, in case the architecture is `x86`. That also means that such `sound32` package version was not captured in the above lockfile.

Lets create the `sound32/1.0` package first, then try to install `engine`:

```
$ cd ..
$ conan create sound32 --version=1.0
$ cd engine
$ conan install . -s arch=x86 # FAILS!
ERROR: Requirement 'sound32/[>=1.0 <2.0]' not in lockfile
```

This happens because the `conan.lock` lockfile doesn't contain a locked version for `sound32`. By default lockfiles are strict, if we are locking dependencies, a matching version inside the lockfile must be found. We can relax this assumption with the `--lockfile-partial` argument:

```
$ conan install . -s arch=x86 --lockfile-partial
...
Requirements
    matrix/1.0#905c3f0babc520684c84127378fefdd0 - Cache
    sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7 - Cache
Resolved version ranges
    sound32/[>=1.0 <2.0]: sound32/1.0
```

This will manage to partially lock to `matrix/1.0`, and resolve `sound32` version range as usual. But we can do better, we can extend our lockfile to also lock `sound32/1.0` version, to avoid possible disruptions caused by new `sound32` unexpected versions:

```
$ conan lock create . -s arch=x86
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7%1675278904.0791488",
        "matrix/1.0#905c3f0babc520684c84127378fefdd0%1675278900.0103245"
    ],
    "build_requires": [],
    "python_requires": []
}
```

Now, both `matrix/1.0` and `sound32/1.0` are locked inside our `conan.lock` lockfile. It is possible to use this lockfile for both configurations (64bits, and x86 architectures), having versions in a lockfile that are not used for a given configuration is not an issue, as long as the necessary dependencies for that configuration find a matching version in it.

Important: Lockfiles contains sorted lists of requirements, ordered by versions and revisions, so latest versions and revisions are the ones that are prioritized when resolving against a lockfile. A lockfile can contain two or more different versions of the same package, just because different version ranges require them. The sorting will provide the right logic so each range resolves to each valid versions.

If a version in the lockfile doesn't fit in a valid range, it will not be used. It is not possible for lockfiles to force a dependency that goes against what `conanfile` requires define, as they are "snapshots" of an existing/realizable dependency graph, but cannot define an "impossible" dependency graph.

Evolving lockfiles

Even if lockfiles enforce and constraint the versions that can be resolved for a graph, it doesn't mean that lockfiles cannot evolve. Actually, controlled evolution of lockfiles is paramount to important processes like Continuous Integration, when the effect of one change in the graph wants to be tested in isolation of other possible concurrent changes.

In this section we will introduce some of the basic functionality of lockfiles that allows such evolution.

First, if we would like now to introduce and test the new `matrix/1.1` version in our `engine`, without necessarily pulling many other dependencies that could have got new versions too, we could manually add `matrix/1.1` to the lockfile:

```
$ Running: conan lock add --requires=matrix/1.1
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7%1675278904.0791488",
        "matrix/1.1",
        "matrix/1.0#905c3f0babc520684c84127378fefdd0%1675278900.0103245"
    ],
    "build_requires": [],
    "python_requires": []
}
```

To be clear: manually adding with `conan lock add` is not necessarily a recommended flow, it is possible to automate the task with other approaches, that will be explained later. This is just an introduction to the principles and concepts.

The important idea is that now we got 2 versions of `matrix` in the lockfile, and `matrix/1.1` is before `matrix/1.0`, so for the range `matrix/[>=1.0 <2.0]`, the first one (`matrix/1.1`) would be prioritized. That means that when now the new lockfile is used, it will resolve to `matrix/1.1` version (even if a `matrix/1.2` or higher version existed in the system):

```
$ conan install . -s arch=x86 --lockfile-out=conan.lock
Requirements
    matrix/1.1#905c3f0babc520684c84127378fefdd0 - Cache
    sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7 - Cache
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7%1675278904.0791488",
        "matrix/1.1#905c3f0babc520684c84127378fefdd0%1675278901.7527816",
        "matrix/1.0#905c3f0babc520684c84127378fefdd0%1675278900.0103245"
```

(continues on next page)

(continued from previous page)

```
    ],
    "build_requires": [],
    "python_requires": []
}
```

Note that now `matrix/1.1` was resolved, and it also got its `revision` stored in the lockfile (because `--lockfile-out=conan.lock` was passed as argument).

It is true that the former `matrix/1.0` version was not used. As said above, having old versions in the lockfile that are not used is not harmful. However, if we want to prune the unused versions and revisions, we could use the `--lockfile-clean` for that purpose:

```
$ conan install . -s arch=x86 --lockfile-out=conan.lock --lockfile-clean
...
Requirements
    matrix/1.1#905c3f0babc520684c84127378fefdd0 - Cache
    sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7 - Cache
...
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7%1675278904.0791488",
        "matrix/1.1#905c3f0babc520684c84127378fefdd0%1675278901.7527816"
    ],
    "build_requires": [],
    "python_requires": []
}
```

It is relevant to note that the `--lockfile-clean` could remove locked versions in given configurations. For example, if instead of the above, the `x86_64` architecture is used, the `--lockfile-clean` will prune the "unused" `sound32`, because in that configuration is not used. It is possible to evaluate new lockfiles for every different configuration, and then merge them:

```
$ conan lock create . --lockfile-out=64.lock --lockfile-clean
$ conan lock create . -s arch=x86 --lockfile-out=32.lock --lockfile-clean
$ cat 64.lock
{
    "version": "0.5",
    "requires": [
        "matrix/1.1#905c3f0babc520684c84127378fefdd0%1675294635.6049662"
    ],
    "build_requires": [],
    "python_requires": []
}
$ cat 32.lock
{
    "version": "0.5",
    "requires": [
        "sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7%1675294637.9775107",
        "matrix/1.1#905c3f0babc520684c84127378fefdd0%1675294635.6049662"
    ],
    "build_requires": [],
```

(continues on next page)

(continued from previous page)

```
    "python_requires": []
}
$ conan lock merge --lockfile=32.lock --lockfile=64.lock --lockfile-out=conan.lock
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "sound32/1.0#83d4b7bf607b3b60a6546f8b58b5cdd7%1675294637.9775107",
        "matrix/1.1#905c3f0babc520684c84127378fefdd0%1675294635.6049662"
    ],
    "build_requires": [],
    "python_requires": []
}
```

This multiple-clean + merge operation is not something that developers should do, only CI scripts, and for some advanced CI flows that will be explained later.

See also:

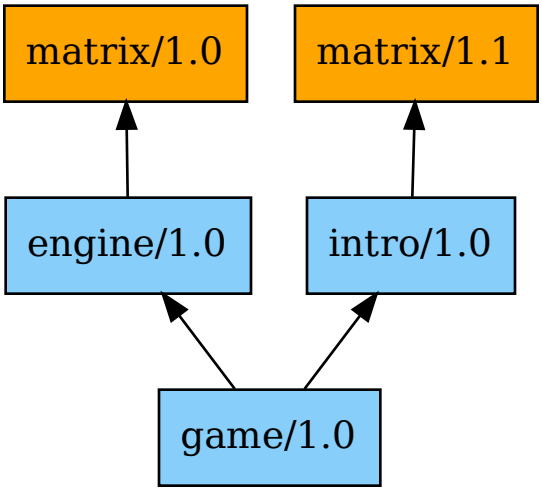
- *CI tutorial*.

3.5.5 Dependencies conflicts

In a dependency graph, when different packages depends on different versions of the same package, this is called a dependency version conflict. It is relatively easy to produce one. Let's see it with a practical example, start cloning the examples2 repository:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/versioning/conflicts/versions
```

In this folder we have a small project, consisting in several packages: `matrix` (a math library), `engine/1.0` video game engine that depends on `matrix/1.0`, `intro/1.0`, a package implementing the intro credits and functionality for the videogame that depends on `matrix/1.1` and finally the `game` recipe that depends simultaneously on `engine/1.0` and `intro/1.0`. All these packages are actually empty, but they are enough to produce the conflicts.



Let's create the dependencies:

```
$ conan create matrix --version=1.0
$ conan create matrix --version=1.1  # note this is 1.1!
$ conan create engine --version=1.0 # depends on matrix/1.0
$ conan create intro --version=1.0 # depends on matrix/1.1
```

And when we try to install `game`, we will get the error:

```
$ conan install game
Requirements
    engine/1.0#0fe4e6890766f7b8e21f764f0049aec7 - Cache
    intro/1.0#d639998c2e55cf36d261ab319801c322 - Cache
    matrix/1.0#905c3f0babc520684c84127378fefdd0 - Cache
Graph error
    Version conflict: intro/1.0->matrix/1.1, game/1.0->matrix/1.0.
ERROR: Version conflict: intro/1.0->matrix/1.1, game/1.0->matrix/1.0.
```

This is a version conflict, and Conan will not decide automatically how to resolve the conflict, but the user should explicitly resolve such conflict.

Resolving conflicts

Of course, the most direct and straightforward way to solve such a conflict is going to the dependencies `conanfile.py` and upgrading their `requirements()` so they point now to the same version. However this might not be practical in some cases, or it might be even impossible to fix the dependencies conanfiles.

For that case, it should be the consuming `conanfile.py` the one that can resolve the conflict (in this case, `game`) by explicitly defining which version of the dependency should be used, with the following syntax:

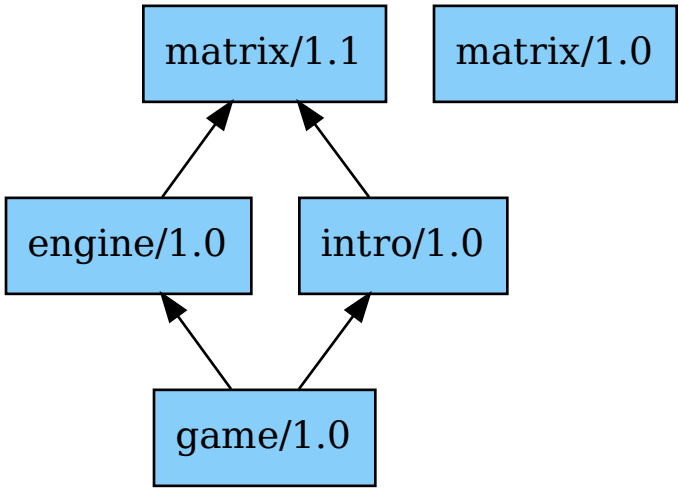
Listing 89: game/conanfile.py

```
class Game(ConanFile):
    name = "game"
    version = "1.0"

    def requirements(self):
        self.requires("engine/1.0")
        self.requires("intro/1.0")
        self.requires("matrix/1.1", override=True)
```

This is called an `override`. The `game` package do not directly depend on `matrix`, this `requires` declaration will not introduce such a a direct dependency. But the `matrix/1.1` version will be propagated upstream in the dependency graph, overriding the `requires` of packages that do depend on any `matrix` version, forcing the consistency of the graph, as all upstream packages will now depend on `matrix/1.1`:

```
$ conan install game
...
Requirements
    engine/1.0#0fe4e6890766f7b8e21f764f0049aec7 - Cache
    intro/1.0#d639998c2e55cf36d261ab319801c322 - Cache
    matrix/1.1#905c3f0babc520684c84127378fefdd0 - Cache
```



Note: In this case, a new binary for `engine/1.0` was not necessary, but in some situations the above could fail with a `engine/1.0` "binary missing error". Because previously `engine/1.0` binaries were built against `matrix/1.0`. If the `package_id` rules and configuration define that `engine` should be rebuilt when minor versions of the dependencies change, then it will be necessary to build a new binary for `engine/1.0` that builds and links against the new `matrix/1.1` dependency.

What happens if `game` had a direct dependency to `matrix/1.2`? Lets create the version:

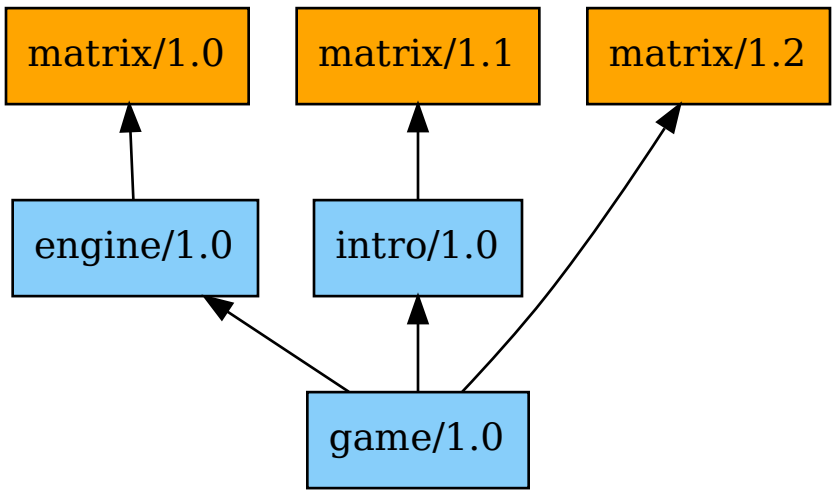
```
$ conan create matrix --version=1.2
```

Now let's modify `game/conanfile.py` to introduce this as a direct dependency:

Listing 90: game/conanfile.py

```
class Game(ConanFile):
    name = "game"
    version = "1.0"

    def requirements(self):
        self.requires("engine/1.0")
        self.requires("intro/1.0")
        self.requires("matrix/1.2")
```



So installing it will raise a conflict error again:

```
$ conan install game
...
ERROR: Version conflict: engine/1.0->matrix/1.0, game/1.0->matrix/1.2.
```

As this time, we want to respect the direct dependency between `game` and `matrix`, we will define the `force=True` requirement trait, to indicate that this dependency version will also be forcing the overrides upstream:

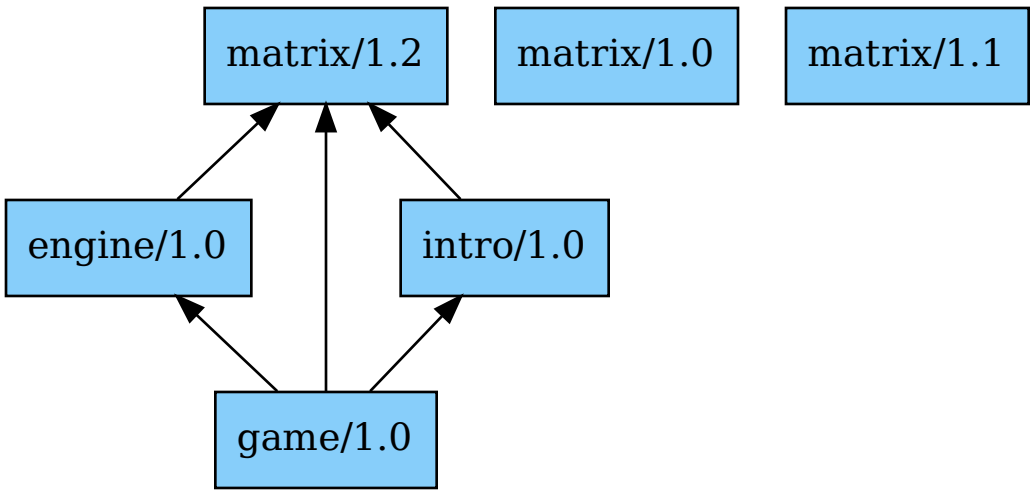
Listing 91: game/conanfile.py

```
class Game(ConanFile):
    name = "game"
    version = "1.0"

    def requirements(self):
        self.requires("engine/1.0")
        self.requires("intro/1.0")
        self.requires("matrix/1.2", force=True)
```

And that will now solve again the conflict (as commented above, note that in real applications this could mean that binaries for `engine/1.0` and `intro/1.0` would be missing, and need to be built to link against the new forced `matrix/1.2` version):

```
$ conan install game
Requirements
    engine/1.0#0fe4e6890766f7b8e21f764f0049aec7 - Cache
    intro/1.0#d639998c2e55cf36d261ab319801c322 - Cache
    matrix/1.2#905c3f0babc520684c84127378fefdd0 - Cache
```



Note: Best practices

- Resolving version conflicts by overrides/forces should in general be the exception and avoided when possible, applied as a temporary workaround. The real solution is to move forward the dependencies `requires` so they naturally converge to the same versions of upstream dependencies.
- A key takeaway is that the `force` trait will create a direct dependency between the consumer and the required package, while the `override` won't, it will only instruct Conan to prefer the required version if the package is already in the dependency graph.
- Version-ranges can also produce some version conflicts, even if Conan tries to reduce them. This *FAQ about version conflicts* discusses the graph resolution algorithm and strategies to minimize the conflicts.

Overriding options

It is possible that when there are diamond structures in a dependency graph, like the one seen above, different recipes might be defining different values for the upstream `options`. In this case, this is not directly causing a conflict, but instead the first value to be defined is the one that will be prioritized and will prevail.

In the above example, if `matrix/1.0` can be both a static and a shared library, and `engine` decides to define that it should be a static library (not really necessary, because that is already the default):

Listing 92: engine/conanfile.py

```
class Engine(ConanFile):
    name = "engine"
    version = "1.0"
    # Not strictly necessary because this is already the matrix default
    default_options = {"matrix*:shared": False}
```

Warning: Defining options values in recipes does not have strong guarantees, please check *this FAQ about options values for dependencies*. The recommended way to define options values is in profile files.

And also `intro` recipe would do the same, but instead define that it wants a shared library, and adds a `validate()` method, because for some reason the `intro` package can only be built against shared libraries and otherwise crashes:

Listing 93: intro/conanfile.py

```
class Intro(ConanFile):
    name = "intro"
    version = "1.0"
    default_options = {"matrix*:shared": True}

    def requirements(self):
        self.requires("matrix/1.0")

    def validate(self):
        if not self.dependencies["matrix"].options.shared:
            raise ConanInvalidConfiguration("Intro package doesn't work with static␣
↪matrix library")
```

Then, this will cause an error, because as the first one to define the option value is `engine` (it is declared first in the `game` conanfile `requirements()` method). In the examples2 repository, go to the "options" folder, and create the different packages:

```
$ cd ../options
$ conan create matrix
$ conan create matrix -o matrix/*:shared=True
$ conan create engine
$ conan create intro
$ conan install game  # FAILS!
...
-------- Installing (downloading, building) binaries... --------
ERROR: There are invalid packages (packages that cannot exist for this configuration):
intro/1.0: Invalid: Intro package doesn't work with static matrix library
```

Following the same principle, the downstream consumer recipe, in this case `game` conanfile.py can define the options values, and those will be prioritized:

Listing 94: game/conanfile.py

```
class Game(ConanFile):
    name = "game"
    version = "1.0"
    default_options = {"matrix*:shared": True}

    def requirements(self):
        self.requires("engine/1.0")
        self.requires("intro/1.0")
```

And that will force now `matrix` being a shared library, no matter if `engine` defined `shared=False`, because the downstream consumers always have priority over the upstream dependencies.

```
$ conan install game
...
-------- Installing (downloading, building) binaries... --------
matrix/1.0: Already installed!
matrix/1.0: I am a shared-library library!!!
engine/1.0: Already installed!
intro/1.0: Already installed!
```

Note: Best practices

As a general rule, avoid modifying or defining values for dependencies `options` in consumers `conanfile.py`. The declared `options` defaults should be good for the majority of cases, and variations from those defaults can be defined better in profiles better.

Note: The Conan 2 Essentials training course is available for free at the JFrog Academy, which covers the same topics as this documentation but in a more interactive way. You can access it here.

3.6 Other important Conan features

3.6.1 python_requires

It is possible to reuse code from other recipes using the *python_requires feature*.

If you maintain many recipes for different packages that share some common logic and you don't want to repeat the code in every recipe, you can put that common code in a Conan `conanfile.py`, upload it to your server, and have other recipe conanfiles do a `python_requires = "mypythoncode/version"` to depend on it and reuse it.

3.6.2 Packages lists

It is possible to manage a list of packages, recipes and binaries together with the "packages-list" feature. Several commands like `upload`, `download`, and `remove` allow receiving a list of packages file as an input, and they can do their operations over that list. A typical use case is to "upload to the server the packages that have been built in the last `conan create`", which can be done with:

```
$ conan create . --format=json > build.json
$ conan list --graph=build.json --graph-binaries=build --format=json > pkglist.json
$ conan upload --list=pkglist.json -r=myremote -c
```

See the *examples in this section*.

3.6.3 Removing unused packages from the cache

Warning: The (lru) least recently used feature is in **preview**. See *the Conan stability* section for more information.

The Conan cache does not implement any automatic expiration policy, so its size will be always increasing unless packages are removed or the cache is removed from time to time. It is possible to remove recipes and packages that haven't been used recently, while keeping those that have been used in a given time period (Least Recently Used LRU policy). This can be done with the `--lru` argument to `conan remove` and `conan list` commands:

```
# remove all binaries (but not recipes) not used in the last 4 weeks
$ conan remove "*:*" --lru=4w -c
# remove all recipes that have not been used in the last 4 weeks (with their binaries)
$ conan remove "*" --lru=4w -c
```

Note that the LRU time follows the rules of the `remove` command. If we are removing recipes with a `"*"` pattern, only the LRU times for recipes will be checked. If a recipe has been recently used, it will keep all the binaries, and if the recipe has not been recently used, it will remove itself and all its binaries. If we use a `"*:*"` pattern, it will check for binaries only, and remove those unused, but always leaving the recipe.

Using `conan list` first (take into account that `conan list` do not default to list all revisions, as opposed to `remove`, so it is necessary to explicit the `#*` to select all revisions if that is the intention) it is possible to create a list of least recently used packages:

```
# List all unused (last 4 weeks) recipe revisions
$ conan list "*#*" --lru=4w --format=json > old.json
# Remove those recipe revisions (and their binaries)
$ conan remove --list=old.json -c
```

See commands help *conan remove* and *conan list*.

Note: The **Conan 2 Essentials** and **Conan 2 Advanced** training paths are available for free at the JFrog Academy. They cover the same topics as this documentation in a more interactive way. Conan 2 Essentials | Conan 2 Advanced | See *JFrog Academy: Conan 2 Training* for the full course overview.

CHAPTER

FOUR

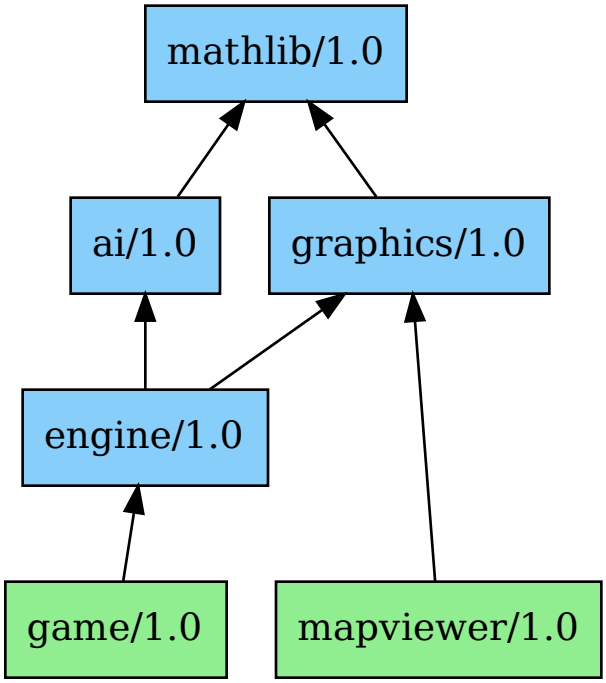
CONTINUOUS INTEGRATION (CI) TUTORIAL

Note:

- This is an advanced topic, previous knowledge of Conan is necessary. Please *read and practice the user tutorial* first.
- This section is intended for devops and build engineers designing and implementing a CI pipeline involving Conan packages, if it is not the case, you can skip this section.
- There is a conference talk *Continuous Integration for Large Scale C/C++ Projects With Conan2 at ACCU-2025* that is based in this tutorial, that might provide some extra information and some implementation details.

Continuous Integration has different meanings for different users and organizations. In this tutorial we will cover the scenarios when users are doing changes to the source code of their packages and want to automatically build new binaries for those packages and also compute if those new package changes integrate cleanly or break the organization main products.

In this tutorial we will use this small project that uses several packages (static libraries by default) to build a couple of applications, a video game and a map viewer utility. The `game` and `mapviewer` are our final "**products**", what we distribute to our users:



All of the packages in the dependency graph have a `requires` to its direct dependencies using version ranges, for example, `game` contains a `requires("engine/[>=1.0 <2]")` so new patch and minor versions of the dependencies will automatically be used without needing to modify the recipes.

Note: Important notes

- This section is written as a hands-on tutorial. It is intended to be reproduced by copying the commands in your machine.
- The tutorial presents some of the tools, good practices and common approaches to the CI problem. But there are no silver bullets. This tutorial is not the unique way that things should be done. Different organizations might have different needs and priorities, different build services power and budget, different sizes, etc. The principles and practices presented in the tutorial might need to be adapted.
- If you have any questions or feedback, please submit a new issue in https://github.com/conan-io/conan/issues
- However some of the principles and best practices would be general for all approaches. Things like package immutability, using promotions between repositories and not using the `channel` for that purpose are good practices that should be followed.

4.1 Packages and products pipelines

When a developer is doing some changes to a package source code, we will consider 2 different parts or pipelines of the overall system CI: the **packages pipeline** and the **products pipeline**

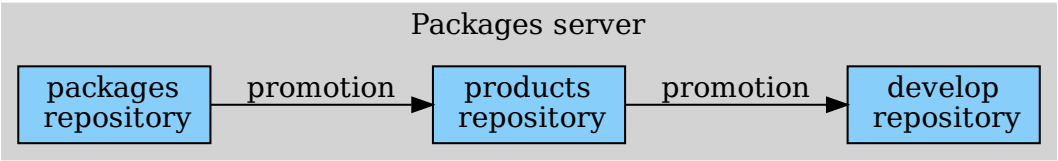
- The **packages pipeline** takes care of building one single package when its code is changed. If necessary it will build it for different configurations.
- The **products pipeline** takes care of building the main organization "products" (the packages that implement the final applications or deliverables), and making sure that changes and new versions in dependencies integrate correctly, rebuilding any intermediate packages in the graph if necessary.

The idea is that if some developer does changes to the `ai` package, producing a new `ai/1.1.0` version, the packages pipeline will first build this new version. But this new version might accidentally break or require rebuilding some consumer packages. If our organization main **products** are `game/1.0` and `mapviewer/1.0`, then the products pipeline can be triggered, in this case it would rebuild `engine/1.0` and `game/1.0` as they are affected by the change.

4.2 Repositories and promotions

The concept of multiple server side repositories is very important for CI. In this tutorial we will use 3 repositories:

- `develop`: This repository is the main one that developers have configured in their machines to be able to `conan install` dependencies and work. As such it is expected to be quite stable, similar to a shared "develop" branch in git, and the repository should contain pre-compiled binaries for the organization's pre-defined platforms, so developers and CI don't need to do `--build=missing` and build again and again from source.
- `packages`: This repository will be used to temporarily upload the packages built by the "packages pipeline", to not upload them directly to the `develop` repo and avoid disruption until these packages are fully validated.
- `products`: This repository will be used to temporarily upload the packages built by the "products pipeline", while building and testing that new dependencies changes do not break the main "products".



Promotions are the mechanism used to make packages available from one pipeline to the other. Connecting the above packages and product pipelines with the repositories, there will be 2 promotions:

- When all the different binaries for the different configurations have been built for a single package with the `packages pipeline`, and uploaded to the `packages` repository, the new version and changes to the package can be considered "correct" and promoted (copied) to the `products` repository.
- When the `products pipeline` has built from source all the necessary packages that need a re-build because of the new package versions in the `products` repository and has checked that the organization "products" (such `game/1.0` and `mapviewer/1.0`) are not broken, then the packages can be promoted (copied) from the `products` repo to the `develop` repo, to make them available for all other developers and CI.

Note:

- The concept of **immutability** is important in package management and devops. Modifying `channel` is strongly discouraged, see *Package promotions*.
- The versioning approach is important. This tutorial will be following *the default Conan versioning approach, see details here*

This tutorial is just modeling the **development** flow. In production systems, there will be other repositories and promotions, like a `testing` repository for the QA team, and a final `release` repository for final users, such that packages can be promoted from `develop` to `testing` to `release` as they pass validation. Read more about promotions in *Package promotions*.

Let's start with the tutorial, move to the next section to do the project setup:

4.2.1 Project setup

The code necessary for this tutorial is found in the `examples2` repo, clone it and move to the folder:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/ci/game
```

Server repositories setup

We need 3 different repositories in the same server. Make sure to have an Artifactory running and available. You can download the free *Artifactory CE* from the downloads page and run it in your own computer, or you can use docker:

```
$ docker run --name artifactory -d -e JF_SHARED_DATABASE_TYPE=derby -e JF_SHARED_
↪DATABASE_ALLOWNONPOSTGRESQL=true -e JF_JFCONNECT_ENABLED=false -p 8081:8081 -p␣
↪8082:8082 releases-docker.jfrog.io/jfrog/artifactory-cpp-ce:latest
# Can be stopped with "docker stop artifactory"
```

When you launch it, you can go to http://localhost:8082/ to check it (user: "admin", password: "password"). If you have another available Artifactory, it can be used too if you can create new repositories there.

As a first step, log into the web UI and **create 3 different local repositories** called `develop`, `packages` and `products`.

Then according to the `project_setup.py` file, these are the necessary environment variables to configure the server. Please define `ARTIFACTORY_URL`, `ARTIFACTORY_USER` and/or `ARTIFACTORY_PASSWORD` if necessary to adapt to your setup:

```
# TODO: This must be configured by users
SERVER_URL = os.environ.get("ARTIFACTORY_URL", "http://localhost:8081/artifactory/api/
↪conan")
USER = os.environ.get("ARTIFACTORY_USER", "admin")
PASSWORD = os.environ.get("ARTIFACTORY_PASSWORD", "password")
```

Initial dependency graph

Warning:

- The initialization of the project will remove the contents of the 3 `develop`, `products` and `packages` repositories in the server.
- The `examples2/ci/game` folder contains a `.conanrc` file that defines a local cache, so commands executed in this tutorial do not pollute or alter your main Conan cache.

```
$ python project_setup.py
```

This will do several tasks, clean the server repos, create initial `Debug` and `Release` binaries for the dependency graph and upload them to the `develop` repo, then clean the local cache. Note in this example we are using `Debug` and `Release` as our different configurations for convenience, but in real cases these would be different configurations such as Windows/X86_64, Linux/x86_64, Linux/armv8, etc., running in different computers.

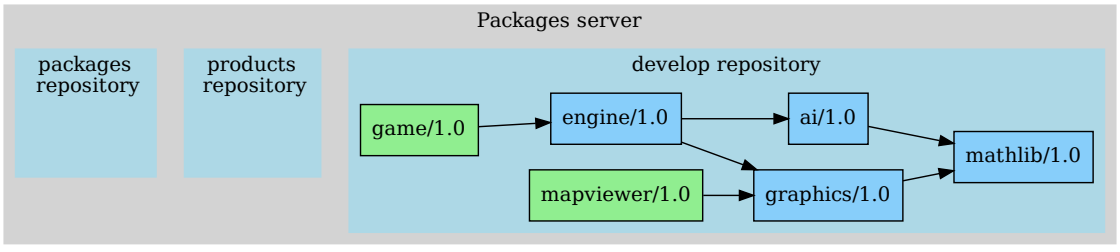
After the setup, it can be checked that the 3 remotes are defined, but only `develop` remote is enabled, and there are no packages in the local cache:

```
$ conan remote list
products: http://localhost:8081/artifactory/api/conan/products [Verify SSL: True,␣
↪Enabled: False]
develop: http://localhost:8081/artifactory/api/conan/develop [Verify SSL: True, Enabled:␣
↪True]
packages: http://localhost:8081/artifactory/api/conan/packages [Verify SSL: True,␣
↪Enabled: False]

$ conan list *
Found 0 pkg/version recipes matching * in local cache
Local Cache
WARN: There are no matching recipe references
```

Important: The order of the remotes is important. If the `products` repository is enabled, it will have higher priority than the `develop` one, so if it contains new versions, they will be picked from there.

This dependency graph of packages in the `develop` repo is the starting point for our tutorial, assumed as a functional and stable "develop" state of the project that developers can `conan install` to work in any of the different packages.



4.2.2 Packages pipeline

The **packages pipeline** will build, create and upload the package binaries for the different configurations and platforms, when some developer is submitting some changes to one of the organization repositories source code. For example if a developer is doing some changes to the `ai` package, improving some of the library functionality, and bumping the version to `ai/1.1.0`. If the organization needs to support both Windows and Linux platforms, then the package pipeline will build the new `ai/1.1.0` both for Windows and Linux, before considering the changes are valid. If some of the configurations fail to build under a specific platform, it is common to consider the changes invalid and stop the processing of those changes, until the code is fixed.

For the `package pipeline` we will start with a simple source code change in the `ai` recipe, simulating some improvements in the `ai` package, providing some better algorithms for our game.

Let's do the following changes in the ai package:

- Let's change the implementation of the `ai/src/ai.cpp` function and change the message from `Some Artificial` to `SUPER BETTER Artificial`
- Let's change the default `intelligence=0` value in `ai/include/ai.h` to a new `intelligence=50` default.
- Finally, let's bump the version. As we did some changes to the package public headers, it would be adviced to bump the `minor` version, so let`s edit the `ai/conanfile.py` file and define `version = "1.1.0"` there (instead of the previous `1.0`). Note that if we did some breaking changes to the `ai` public API, the recommendation would be to change the major instead and create a new `2.0` version.

The **packages pipeline** will take care of building the different packages binaries for the new `ai/1.1.0` and upload them to the `packages` binary repository to avoid disrupting or causing potential issues to other developers and CI jobs. If the pipeline succeed it will promote (copy) them to the `products` binary repository, and stop otherwise.

There are different aspects that need to be taken into account when building these binary packages for `ai/1.1.0`. The following tutorial sections do the same job, but under different hypothesis. They are explained in increasing complexity.

Note all of the commands can be found in the repository `run_example.py` file. This file is mostly intended for maintainers and testing, but it might be useful as a reference in case of issues.

Package pipeline: single configuration

We will start with the most simple case, in which we only had to build 1 configuration, and that configuration can be built in the current CI machine.

As we described before while presenting the different server binary repositories, the idea is that package builds will use by default the `develop` repo only, which is considered the stable one for developer and CI jobs.

This pipeline starts from a clean state, with no packages in the cache, and only the `develop` repository enabled.

With this configuration the CI job could just do:

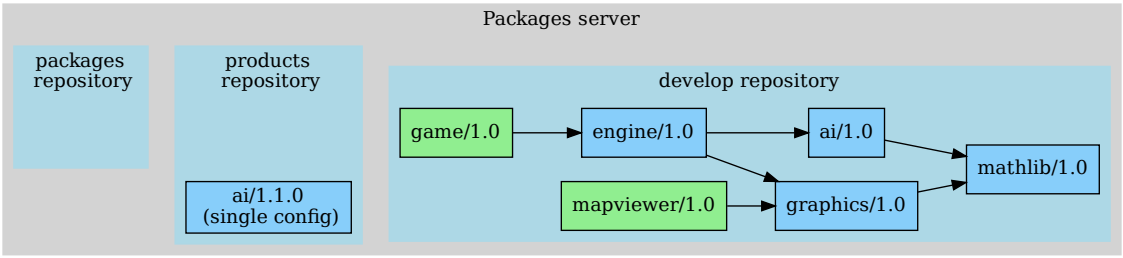
```
$ cd ai
$ conan create . --build="missing:ai/*"
...
ai/1.1.0: SUPER BETTER Artificial Intelligence for aliens (Release)!
ai/1.1.0: Intelligence level=50
```

Note the `--build="missing:ai/*"` might not be fully necessary in some cases, but it can save time in other situations. For example, if the developer did some changes just to the repo README, and didn't bump the version at all, Conan will not generate a new `recipe revision`, and detect this as a no-op, avoiding having to unnecessarily rebuild binaries from source.

If we are in a single-configuration scenario and it built correctly, for this simple case we don't need a promotion, and just uploading directly the built packages to the `products` repository will be enough, where the `products pipeline` will pick it later.

```
# We don't want to disrupt developers or CI, upload to products
$ conan remote enable products
$ conan upload "ai*" -r=products -c
$ conan remote disable products
```

As the cache was initially clean, all `ai` packages would be the ones that were built in this pipeline.



This was a very simple scenario, let's move to a more realistic one: having to build more than one configuration.

Package pipeline: multi configuration

In the previous section we were building just 1 configuration. This section will cover the case in which we need to build more than 1 configuration. We will use the `Release` and `Debug` configurations here for convenience, as it is easier to follow, but in real case these configurations will be more like Windows, Linux, OSX, building for different architectures, cross building, etc.

Let's begin cleaning our cache:

```
$ conan remove "*" -c  # Make sure no packages from last run
```

We will create the packages for the 2 configurations sequentially in our computer, but note these will typically run in different computers, so it is typical for CI systems to launch the builds of different configurations in parallel.

Listing 1: Release build

```
$ cd ai  # If you were not inside "ai" folder already
$ conan create . --build="missing:ai/*" -s build_type=Release --format=json > graph.json
$ conan list --graph=graph.json --graph-binaries=build --format=json > built.json

$ conan remote enable packages
$ conan upload -l=built.json -r=packages -c --format=json > uploaded_release.json
$ conan remote disable packages
```

We have done a few changes and extra steps:

- First step is similar to the one in the previous section, a `conan create`, just making it explicit our configuration `-s build_type=Release` for clarity, and capturing the output of the `conan create` in a `graph.json` file.

- The second step is create from the `graph.json` a `built.json` **package list** file, with the packages that needs to be uploaded, in this case, only the packages that have been built from source (`--graph-binaries=build`) will be uploaded. This is done for efficiency and faster uploads.
- Third step is to enable the `packages` repository. It was not enabled to guarantee that all possible dependencies came from `develop` repo only.
- Then, we will upload the `built.json` package list to the `packages` repository, creating the `uploaded_release.json` package list with the new location of the packages (the server repository).
- Finally, we will disable again the `packages` repository

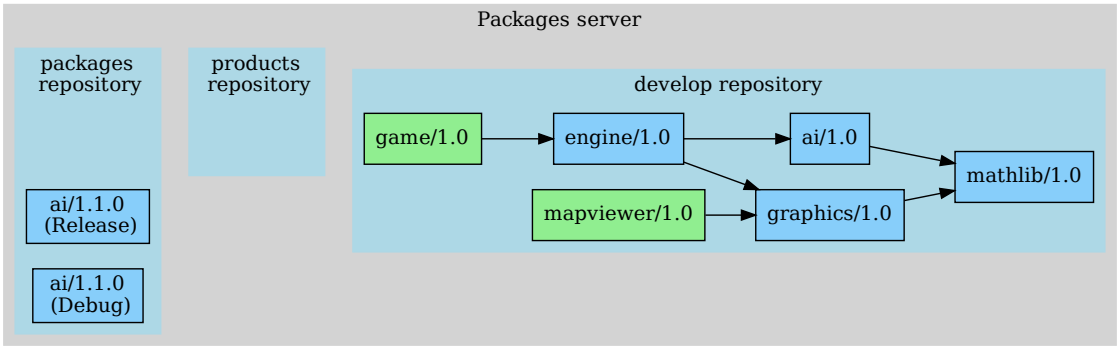
Likewise, the Debug build will do the same steps:

Listing 2: Debug build

```
$ conan create . --build="missing:ai/*" -s build_type=Debug --format=json > graph.json
$ conan list --graph=graph.json --graph-binaries=build --format=json > built.json

$ conan remote enable packages
$ conan upload -l=built.json -r=packages -c --format=json > uploaded_debug.json
$ conan remote disable packages
```

When both Release and Debug configuration finish successfully, we would have these packages in the repositories:



When all the different binaries for `ai/1.1.0` have been built correctly, the `package pipeline` can consider its job succesfull and decide to promote those binaries. But further package builds and checks are necessary, so instead of promoting them to the `develop` repository, the `package pipeline` can promote them to the `products` binary repository. As all other developers and CI use the `develop` repository, no one will be broken at this stage either:

Listing 3: Promoting from packages->product

```
# aggregate the package list
$ conan pkglist merge -l uploaded_release.json -l uploaded_debug.json --format=json >␣
↪uploaded.json

$ conan remote enable packages
$ conan remote enable products
# Promotion using Conan download/upload commands
# (slow, can be improved with art:promote custom command)
$ conan download --list=uploaded.json -r=packages --format=json > promote.json
```

(continues on next page)

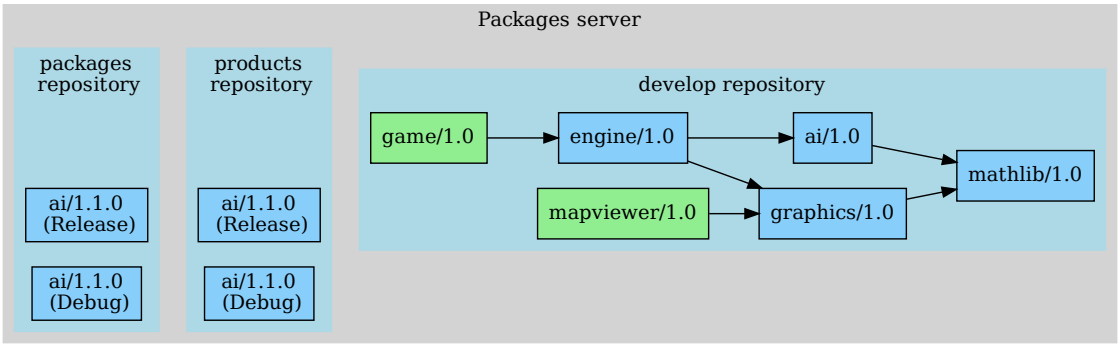
(continued from previous page)

```
$ conan upload --list=promote.json -r=products -c
$ conan remote disable packages
$ conan remote disable products
```

The first step uses the `conan pkglist merge` command to merge the package lists from the "Release" and "Debug" configurations and merge it into a single `uploaded.json` package list. This list is the one that will be used to run the promotion.

In this example we are using a slow `conan download` + `conan upload` promotion. This can be way more efficient with the `conan art:promote` extension command.

After running the promotion we will have the following packages in the server:



To summarize:

- We built 2 different configurations, `Release` and `Debug` (could have been Windows/Linux or others), and uploaded them to the `packages` repository.
- When all package binaries for all configurations were successfully built, we promoted them from the `packages` to the `products` repository, to make them available for the `products pipeline`.
- **Package lists** were captured in the package creation process and merged into a single one to run the promotion.

There is still an aspect that we haven't considered yet, the possibility that the dependencies of `ai/1.1.0` change during the build. Move to the next section to see how to use lockfiles to achieve more consistent multi-configuration builds.

Package pipeline: multi configuration using lockfiles

In the previous example, we built both `Debug` and `Release` package binaries for `ai/1.1.0`. In real world scenarios the binaries to build would be different platforms (Windows, Linux, embedded), different architectures, and very often it will not be possible to build them in the same machine, requiring different computers.

The previous example had an important assumption: the dependencies of `ai/1.1.0` do not change at all during the building process. In many scenarios, this assumption will not hold, for example if there are any other concurrent CI jobs, and one succesfull job publishes a new `mathlib/1.1` version in the `develop` repo.

Then it is possible that one build of `ai/1.1.0`, for example, the one running in the Linux servers starts earlier and uses the previous `mathlib/1.0` version as dependency, while the Windows servers start a bit later, and then their build will use the recent `mathlib/1.1` version as dependency. This is a very undesirable situation, having binaries for the same `ai/1.1.0` version using different dependencies versions. This can lead in later graph resolution problems, or even worse, get to the release with different behavior for different platforms.

The way to avoid this discrepancy in dependencies is to force the usage of the same dependencies versions and revisions, something that can be done with *lockfiles*.

Creating and applying lockfiles is relatively straightforward. The process of creating and promoting the configurations will be identical to the previous section, but just applying the lockfiles.

Creating the lockfile

Let's make sure as usual that we start from a clean state:

```
$ conan remove "*" -c  # Make sure no packages from last run
```

Then we can create the lockfile `conan.lock` file:

```
# Capture a lockfile for the Release configuration
$ conan lock create . -s build_type=Release --lockfile-out=conan.lock
# extend the lockfile so it also covers the Debug configuration
# in case there are Debug-specific dependencies
$ conan lock create . -s build_type=Debug --lockfile=conan.lock --lockfile-out=conan.lock
```

Note that different configurations, using different profiles or settings could result in different dependency graphs. A lockfile file can be used to lock the different configurations, but it is important to iterate the different configurations/profiles and capture their information in the lockfile.

Note: The `conan.lock` is the default argument, and if a `conan.lock` file exists, it might be automatically used by `conan install/create` and other graph commands. This can simplify many of the commands, but this tutorial is showing the full explicit commands for clarity and didactical reasons.

The `conan.lock` file can be inspected, it will be something like:

```
{
    "version": "0.5",
    "requires": [
        "mathlib/1.0#f2b05681ed843bf50d8b7b7bdb5163ea%1724319985.398"
    ],
    "build_requires": [],
    "python_requires": [],
    "config_requires": []
}
```

As we can see, it is locking the `mathlib/1.0` dependency version and revision.

With the lockfile, creating the different configurations is exactly the same, but providing the `--lockfile=conan.lock` argument to the `conan create` step, it will guarantee that `mathlib/1.0#f2b05681ed843bf50d8b7b7bdb5163ea` will always be the exact dependency used, irrespective if there exist new `mathlib/1.1` versions or new revisions available. The following builds could be launched in parallel but executed at different times, and still they will always use the same `mathlib/1.0` dependency:

Listing 4: Release build

```
$ cd ai  # If you were not inside "ai" folder already
$ conan create . --build="missing:ai/*" --lockfile=conan.lock -s build_type=Release --
↪format=json > graph.json
```

(continues on next page)

(continued from previous page)

```
$ conan list --graph=graph.json --graph-binaries=build --format=json > built.json
$ conan remote enable packages
$ conan upload -l=built.json -r=packages -c --format=json > uploaded_release.json
$ conan remote disable packages
```

Listing 5: Debug build

```
$ conan create . --build="missing:ai/*" --lockfile=conan.lock -s build_type=Debug --
↪format=json > graph.json
$ conan list --graph=graph.json --graph-binaries=build --format=json > built.json
$ conan remote enable packages
$ conan upload -l=built.json -r=packages -c --format=json > uploaded_debug.json
$ conan remote disable packages
```

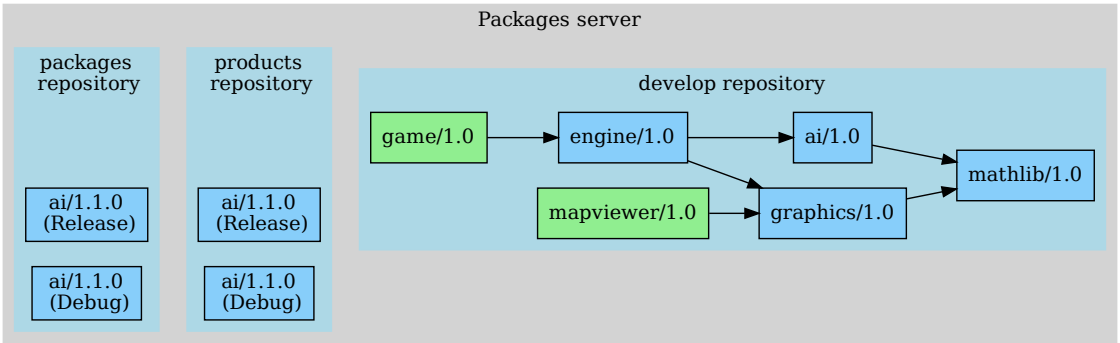
Note the only modification to the previous example is the addition of `--lockfile=conan.lock`. The promotion will also be identical to the previous one:

Listing 6: Promoting from packages->product

```
# aggregate the package list
$ conan pkglist merge -l uploaded_release.json -l uploaded_debug.json --format=json >␣
↪uploaded.json

$ conan remote enable packages
$ conan remote enable products
# Promotion using Conan download/upload commands
# (slow, can be improved with art:promote custom command)
$ conan download --list=uploaded.json -r=packages --format=json > promote.json
$ conan upload --list=promote.json -r=products -c
$ conan remote disable packages
$ conan remote disable products
```

And the final result will be the same as in the previous section, but this time just with the guarantee that both `Debug` and `Release` binaries were built using exactly the same `mathlib` version:



Now that we have the new `ai/1.1.0` binaries in the `products` repo, we can consider the `packages pipeline` finished and move to the next section, and build and check our products to see if this new `ai/1.1.0` version integrates correctly.

4.2.3 Products pipeline

The **products pipeline** responds to a more challenging question: do my "products" build correctly with the new versions of the packages? to the packages and their dependencies? This is the real "Continuous Integration" part, in which changes in different packages are really tested against the organization important products to check if things integrate cleanly or break.

Let's continue with the example above, if we now have a new `ai/1.1.0` package, is it going to break the existing `game/1.0` and/or `mapviewer/1.0` applications? Is it necessary to re-build from source some of the existing packages that depend directly or indirectly on `ai` package? In this tutorial we use `game/1.0` and `mapviewer/1.0` as our "products", but this concept will be further explained later, and specially why it is important to think in terms of "products" instead of trying to explicitly model the dependencies top-bottom in the CI.

The essence of this **products pipeline** in our example is that the new `ai/1.1.0` version that was uploaded to the `products` repository automatically falls into the valid version ranges, and our versioning approach means that such a minor version increase will require building from source its consumers, in this case `engine/1.0` and `game/1.0` and in that specific sequential order, while all the other packages will remain the same. Knowing which packages need to be built from source and in which order, and executing that build to check if the main organization products keep working correctly with the new dependencies versions is the responsibility of the products pipeline.

What are the products

The **products** are the main software artifact that a organization (a company, a team, a project) is delivering as final result and provide some value for users of those artifacts. In this example we will consider `game/1.0` and `mapviewer/1.0` the "products". Note that it is possible to define different versions of the same package as products, for example, if we had to maintain different versions of the `game` for different customers, we could have `game/1.0` and `game/2.3` as well as different versions of `mapviewer` as products.

The "products" approach, besides the advantage of focusing on the business value, has another very important advantage: it avoids having to model the dependency graph at the CI layer. It is a frequent attempt trying to model the inverse dependency model, that is, representing at the CI level the dependants or consumers of a given package. In our example, if we had configured a job for building the `ai` package, we could have another job for the `engine` package, that is triggered after the `ai` one, configuring such topology somehow in the CI system.

But this approach does not scale at all and have very important limitations:

- The example above is relatively simple, but in practice dependency graphs can have many more packages, even several hundreds, making it very tedious and error prone to define all dependencies among packages in the CI
- Dependencies evolve over time, and new versions are used, some dependencies are removed and newer dependencies are added. The simple relationship between repositories modeled at the CI level can result in a very inefficient, slow and time consuming CI, if not a fragile one that continuously breaks because some dependencies change.
- The combinatorial nature that happens downstream a dependency graph, where a relatively stable top dependency, lets say `mathlib/1.0` might be used by multiple consumers such as `ai/1.0`, `ai/1.1`, `ai/1.2` which in turn each one might be used by multiple `engine` different versions and so on. Building only the latest version of the consumers would be insufficient in many cases and building all of them would be extremely costly.
- The "inverse" dependency model, that is, asking what are the "dependants" of a given package is extremely challeging in practice, specially in a decentralized approach like Conan in which packages can be stored in different repositories, including different servers, and there isn't a central database of all packages and their relations. Also, the "inverse" dependency model is, similar to the direct one, conditional. As a dependency can be conditional on any configuration (settings, options), the inverse is also conditioned to the same logic, and such logic also evolves and changes with every new revision and version.

In C and C++ projects the "products" pipeline becomes more necessary and critical than in other languages due to the compilation model with headers textual inclusions becoming part of the consumers' binary artifacts and due to the native artifacts linkage models.

Building intermediate packages new binaries

A frequently asked question is what would be the version of a consumer package when it builds against a new dependency version. Put it explicitly for our example, where we have defined that we need to build again the `engine/1.0` package because now it is depending on `ai/1.1.0` new version:

- Should we create a new `engine/1.1` version to build against the new `ai/1.1.0`?
- Or should we keep the `engine/1.0` version?

The answer lies in the *binary model and how dependencies affect the package_id*. Conan has a binary model that takes into account both the versions, revisions and `package_id` of the dependencies, as well as the different package types (`package_type` attribute).

The recommendation is to keep the package versions aligned with the source code. If `engine/1.0` is building from a specific commit/tag of its source repository, and the source of that repository doesn't change at all, then it becomes very confusing to have a changing package version that deviate from the source version. With the Conan binary model what we will have is 2 different binaries for `engine/1.0`, with 2 different `package_id`. One binary will be built against the `ai/1.0` version and the other binary will be built against the `ai/1.1.0`, something like:

```
$ conan list engine:* -r=develop
engine/1.0
    revisions
      fba6659c9dd04a4bbdc7a375f22143cb (2024-08-22 09:46:24 UTC)
        packages
          2c5842e5aa3ed21b74ed7d8a0a637eb89068916e
            info
              settings
                ...
              requires
                ai/1.0.Z
                graphics/1.0.Z
                mathlib/1.0.Z
          de738ff5d09f0359b81da17c58256c619814a765
            info
              settings
                ...
              requires
                ai/1.1.Z
                graphics/1.0.Z
                mathlib/1.0.Z
```

Let's see how a product pipeline can build such `engine/1.0` and `game/1.0` new binaries using the new dependencies versions. In the following sections we will present a products pipeline in an incremental way, the same as the packages pipeline.

Products pipeline: single configuration

In this section we will implement a very basic products pipeline, without distributing the build, without using lockfiles or building multiple configurations.

The main idea is to illustrate the need to rebuild some packages because there is a new `ai/1.1.0` version that can be integrated by our main products. This new `ai` version is in the `products` repository, as it was already succesfully built by the "packages pipeline". Let's start by making sure we have a clean environment with the right repositories defined:

```
# First clean the local "build" folder
$ pwd  # should be <path>/examples2/ci/game
$ rm -rf build  # clean the temporary build folder
$ mkdir build && cd build # To put temporary files

# Now clean packages and define remotes
$ conan remove "*" -c  # Make sure no packages from last run
# NOTE: The products repo is first, it will have higher priority.
$ conan remote enable products
```

Recall that the `products` repo has higher priority than the `develop` repo. It means Conan will resolve first in the `products` repo, if it finds a valid version for the defined version ranges, it will stop there and return that version, without checking the `develop` repo (checking all repositories can be done with `--update`, but that would be slower and with the right repository ordering, it is not necessary).

As we have already defined, our main products are `game/1.0` and `mapviewer/1.0`, let's start by trying to install and use `mapviewer/1.0`:

```
$ conan install --requires=mapviewer/1.0
...
Requirements
    graphics/1.0#24b395ba17da96288766cc83accc98f5 - Downloaded (develop)
    mapviewer/1.0#c4660fde083a1d581ac554e8a026d4ea - Downloaded (develop)
    mathlib/1.0#f2b05681ed843bf50d8b7b7bdb5163ea - Downloaded (develop)
...
Install finished successfully

# Activate the environment and run the executable
# Use "conanbuild.bat && mapviewer" in Windows
$ source conanrun.sh && mapviewer
...
graphics/1.0: Checking if things collide (Release)!
mapviewer/1.0:serving the game (Release)!
```

As we can see, `mapviewer/1.0` doesn't really depend on `ai` package at all, not any version. So if we install it, we would already have a pre-compiled binary for it and everything works.

But if we now try the same with `game/1.0`:

```
$ conan install --requires=game/1.0
...
======== Computing necessary packages ========
...
ERROR: Missing binary: game/1.0:bac7cd2fe1592075ddc715563984bbe000059d4c

game/1.0: WARN: Cant find a game/1.0 package binary␣
```

(continues on next page)

(continued from previous page)

```
↪bac7cd2fe1592075ddc715563984bbe000059d4c for the configuration:
...
[requires]
ai/1.1.0#01a885b003190704f7617f8c13baa630
```

It will fail, because it will get `ai/1.1.0` from the `products` repo, and there will be no pre-compiled binary for `game/1.0` against this new version of `ai`. This is correct, `ai` is a static library, so we need to re-build `game/1.0` against it, let's do it using the `--build=missing` argument:

```
$ conan install --requires=game/1.0 --build=missing
...
======== Computing necessary packages ========
Requirements
    ai/1.1.0:8b108997a4947ec6a0487a0b6bcbc0d1072e95f3 - Download (products)
    engine/1.0:de738ff5d09f0359b81da17c58256c619814a765 - Build
    game/1.0:bac7cd2fe1592075ddc715563984bbe000059d4c - Build
    graphics/1.0:8b108997a4947ec6a0487a0b6bcbc0d1072e95f3 - Download (develop)
    mathlib/1.0:4d8ab52ebb49f51e63d5193ed580b5a7672e23d5 - Download (develop)

-------- Installing package engine/1.0 (4 of 5) --------
engine/1.0: Building from source
...
engine/1.0: Package de738ff5d09f0359b81da17c58256c619814a765 created
-------- Installing package game/1.0 (5 of 5) --------
game/1.0: Building from source
...
game/1.0: Package bac7cd2fe1592075ddc715563984bbe000059d4c created
Install finished successfully
```

Note the `--build=missing` knows that `engine/1.0` also needs a new binary as a result of its dependency to the new `ai/1.1.0` version. Then, Conan proceeds to build the packages in the right order, first `engine/1.0` has to be built, because `game/1.0` depends on it. After the build we can list the new built binaries and see how they depend on the new versions:

```
$ conan list engine:*
Local Cache
  engine
    engine/1.0
      revisions
        fba6659c9dd04a4bbdc7a375f22143cb (2024-09-30 12:19:54 UTC)
          packages
            de738ff5d09f0359b81da17c58256c619814a765
              info
                ...
                requires
                  ai/1.1.Z
                  graphics/1.0.Z
                  mathlib/1.0.Z

$ conan list game:*
Local Cache
  game
```

(continues on next page)

(continued from previous page)

```
    game/1.0
      revisions
        1715574045610faa2705017c71d0000e (2024-09-30 12:19:55 UTC)
          packages
            bac7cd2fe1592075ddc715563984bbe000059d4c
              info
                ...
                requires
                  ai/1.1.0
↪#01a885b003190704f7617f8c13baa630:8b108997a4947ec6a0487a0b6bcbc0d1072e95f3
                  engine/1.0
↪#fba6659c9dd04a4bbdc7a375f22143cb:de738ff5d09f0359b81da17c58256c619814a765
                  graphics/1.0
↪#24b395ba17da96288766cc83accc98f5:8b108997a4947ec6a0487a0b6bcbc0d1072e95f3
                  mathlib/1.0
↪#f2b05681ed843bf50d8b7b7bdb5163ea:4d8ab52ebb49f51e63d5193ed580b5a7672e23d5
```

The new `engine/1.0:de738ff5d09f0359b81da17c58256c619814a765` binary depends on `ai/1.1.Z`, because as it is a static library it will only require re-builds for changes in the minor version, but not patches. While the `game/1.0` new binary will depend on the full exact `ai/1.1.0#revision:package_id`, and also on the new `engine/1.0:de738ff5d09f0359b81da17c58256c619814a765` new binary that depends on `ai/1.1.Z`.

Now the game can be executed:

```
# Activate the environment and run the executable
# Use "conanbuild.bat && game" in Windows
$ source conanrun.sh && game
mathlib/1.0: mathlib maths (Release)!
ai/1.1.0: SUPER BETTER Artificial Intelligence for aliens (Release)!
ai/1.1.0: Intelligence level=50
graphics/1.0: Checking if things collide (Release)!
engine/1.0: Computing some game things (Release)!
game/1.0:fun game (Release)!
```

We can see that the new `game/1.0` binary incorporates the improvements in `ai/1.1.0`, and links correctly with the new binary for `engine/1.0`.

And this is a basic "products pipeline", we manage to build and test our main products when necessary (recall that `mapviewer` wasn't really affected, so no rebuilds were necessary at all). In general, a production "products pipeline" will finish uploading the built packages to the repository and running a new promotion to the `develop` repo. But as this was a very basic and simple pipeline, let's wait a bit for that, and let's continue with more advanced scenarios.

Products pipeline: the build-order

The previous section used `--build=missing` to build all the necessary packages in the same CI machine. This is not always desired, or even possible, and in many situations it is preferable to do a distributed build, to achieve faster builds and better usage the CI resources. The most natural distribution of the build load is to build different packages in different machines. Let's see how this is possible with the `conan graph build-order` command.

Let's start as usual making sure we have a clean environment with the right repositories defined:

```
# First clean the local "build" folder
$ pwd  # should be <path>/examples2/ci/game
```

(continues on next page)

(continued from previous page)

```
$ rm -rf build  # clean the temporary build folder
$ mkdir build && cd build # To put temporary files

$ conan remove "*" -c  # Make sure no packages from last run
# NOTE: The products repo is first, it will have higher priority.
$ conan remote enable products
```

We will obviate by now the `mapviewer/1.0` product and focus this section in the `game/1.0` product. The first step is to compute the "build-order", that is, the list of packages that need to be built, and in what order. This is done with the following `conan graph build-order` command:

```
$ conan graph build-order --requires=game/1.0 --build=missing --order-by=recipe --reduce␣
↪--format=json > game_build_order.json
```

Note a few important points:

- It is necessary to use the `--build=missing`, in exactly the same way than in the previous section. Failing to provide the intended `--build` policy and argument will result in incomplete or erroneous build-orders.
- The `--reduce` argument eliminates all elements in the resulting order that don't have the `binary: Build` policy. This means that the resulting "build-order" cannot be merged with other build order files for aggregating them into a single one, which is important when there are multiple configurations and products.
- The `--order-by` argument allows to define different orders, by "recipe" or by "configuration". In this case, we are using `--order-by=recipe` which is intended to parallelize builds per recipe, that means, that all possible different binaries for a given package like `engine/1.0` should be built first before any consumer of `engine/1.0` can be built.

The resulting `game_build_order.json` looks like:

Listing 7: game_build_order.json

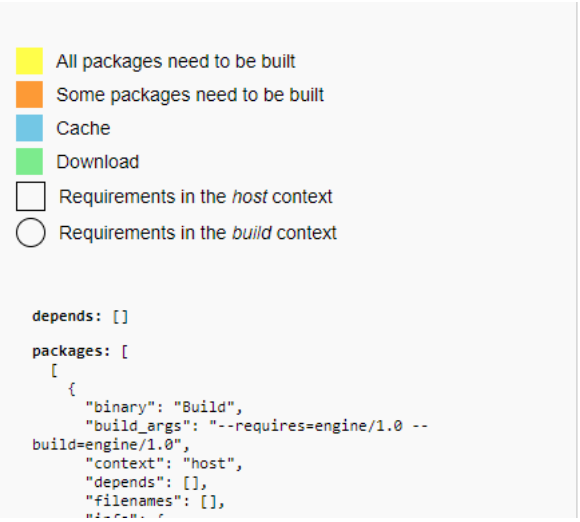
```
{
    "order_by": "recipe",
    "reduced": true,
    "order": [
        [
            {
                "ref": "engine/1.0#fba6659c9dd04a4bbdc7a375f22143cb",
                "packages": [
                    [
                        {
                            "package_id": "de738ff5d09f0359b81da17c58256c619814a765",
                            "binary": "Build",
                            "build_args": "--requires=engine/1.0 --build=engine/1.0",
                        }
                    ]
                ]
            }
        ],
        [
            {
                "ref": "game/1.0#1715574045610faa2705017c71d0000e",
                "depends": [
                    "engine/1.0#fba6659c9dd04a4bbdc7a375f22143cb"
```

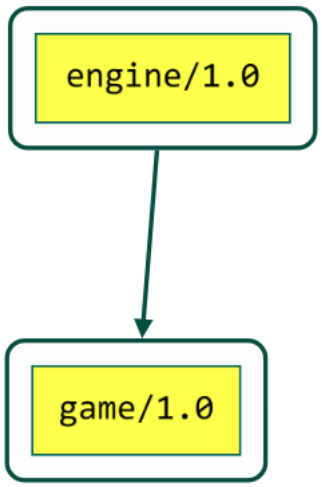
(continues on next page)

(continued from previous page)

```
                ],
                "packages": [
                    [
                        {
                            "package_id": "bac7cd2fe1592075ddc715563984bbe000059d4c",
                            "binary": "Build",
                            "build_args": "--requires=game/1.0 --build=game/1.0",
                        }
                    ]
                ]
            }
        ]
    ]
}
```

For convenience, in the same way that `conan graph info ... --format=html > graph.html` can generate a file with an HTML interactive dependency graph, the `conan graph build-order ... --format=html > build_order.html` can generate an HTML visual representation of the above json file:





The resulting json contains an `order` element which is a list of lists. This arrangement is important, every element in the top list is a set of packages that can be built in parallel because they do not have any relationship among them. You can view this list as a list of "levels", in level 0, there are packages that have no dependencies to any other package being built, in level 1 there are packages that contain dependencies only to elements in level 0 and so on.

Then, the order of the elements in the outermost list is important and must be respected. Until the build of all the packages in one list item has finished, it is not possible to start the build of the next "level".

Using the information in the `graph_build_order.json` file, it is possible to execute the build of the necessary packages, in the same way that the previous section's `--build=missing` did, but not directly managed by us.

Taking the arguments from the json, the commands to execute would be:

```
$ conan install --requires=engine/1.0 --build=engine/1.0
$ conan install --requires=game/1.0 --build=game/1.0
```

We are executing these commands manually, but in practice, it would be a `for` loop in CI executing over the json output. We will see some Python code later for this. At this point we wanted to focus on the `conan graph build-order` command, but we haven't really explained how the build is distributed.

Also note that inside every element there is an inner list of lists, the "packages" section, for all the binaries that must be built for a specific recipe for different configurations.

Let's move now to see how a multi-product, multi-configuration build order can be computed.

Products pipeline: multi-product multi-configuration builds

In the previous section we computed a `conan graph build-order` with several simplifications, we didn't take the `mapviewer` product into account, and we processed only 1 configuration.

In real scenarios, it will be necessary to manage more than one product and the most common case is that there is more than one configuration for every product. If we build these different cases sequentially it will be much slower and inefficient, and if we try to build them in parallel there will easily be many duplicated and unnecessary builds of the same packages, wasting resources and even producing issues as race conditions or traceability problems.

To avoid this issue, it is possible to compute a single unified "build-order" that aggregates all the different build-orders that are computed for the different products and configurations.

Let's start as usual cleaning the local cache and defining the correct repos:

```
# First clean the local "build" folder
$ pwd  # should be <path>/examples2/ci/game
$ rm -rf build  # clean the temporary build folder
$ mkdir build && cd build # To put temporary files

$ conan remove "*" -c  # Make sure no packages from last run
# NOTE: The products repo is first, it will have higher priority.
$ conan remote enable products
```

Now, we will start computing the build-order for `game/1.0` for the 2 different configurations that we are building in this tutorial, debug and release:

```
$ conan graph build-order --requires=game/1.0 --build=missing --order-by=recipe --
↪format=json > game_release.json
$ conan graph build-order --requires=game/1.0 --build=missing --order-by=recipe -s build_
↪type=Debug --format=json > game_debug.json
```

These commands are basically the same as in the previous section, each one with a different configuration and creating a different output file `game_release.json` and `game_debug.json`. These files will be similar to the previous ones, but as we haven't used the `--reduce` argument (this is important!) they will actually contain a "build-order" of all elements in the graph, even if only some contain the `binary: Build` definition, and others will contain other `binary: Download|Cache|etc`.

Now, let's compute the build-order for `mapviewer/1.0`:

```
$ conan graph build-order --requires=mapviewer/1.0 --build=missing --order-by=recipe --
↪format=json > mapviewer_release.json
$ conan graph build-order --requires=mapviewer/1.0 --build=missing --order-by=recipe -s␣
↪build_type=Debug --format=json > mapviewer_debug.json
```

Note that in the generated `mapviewer_xxx.json` build-order files, there will be only 1 element for `mapviewer/1.0` that contains a `binary: Download`, because there is really no other package to be built, and as `mapviewer` is an application linked statically, Conan knows that it can "skip" its dependencies binaries. If we had used the `--reduce` argument we would have obtained an empty `order`. But this is not an issue, as the next final step will really compute what needs to be built.

Let's take all the 4 different "build-order" files (2 products x 2 configurations each), and merge them together:

```
$ conan graph build-order-merge --file=game_release.json --file=game_debug.json --
↪file=mapviewer_release.json --file=mapviewer_debug.json --reduce --format=json > build_
↪order.json
```

Now we have applied the `--reduce` argument to produce a final `build_order.json` that is ready for distribution to the build agents and it only contains those specific packages that need to be built:

```
{
    "order_by": "recipe",
    "reduced": true,
    "order": [
        [
            {
                "ref": "engine/1.0#fba6659c9dd04a4bbdc7a375f22143cb",
                "packages": [
                    [
                        {
                            "package_id": "de738ff5d09f0359b81da17c58256c619814a765",
                            "filenames": ["game_release"],
                            "build_args": "--requires=engine/1.0 --build=engine/1.0",
                        },
                        {
                            "package_id": "cbeb3ac76e3d890c630dae5c068bc178e538b090",
                            "filenames": ["game_debug"],
                            "build_args": "--requires=engine/1.0 --build=engine/1.0",

                        }
                    ]
                ]
            }
        ],
        [
            {
                "ref": "game/1.0#1715574045610faa2705017c71d0000e",
                "packages": [
                    [
                        {
                            "package_id": "bac7cd2fe1592075ddc715563984bbe000059d4c",
                            "filenames": ["game_release"],
                            "build_args": "--requires=game/1.0 --build=game/1.0",
                        },
                        {
                            "package_id": "01fbc27d2c156886244dafd0804eef1fff13440b",
                            "filenames": ["game_debug"],
                            "build_args": "--requires=game/1.0 --build=game/1.0",
                        }
                    ]
                ]
            }
        ]
    ],
    "profiles": {
        "game_release": {"args": ""},
```

(continues on next page)

(continued from previous page)

```
        "game_debug": {"args": "-s:h=\"build_type=Debug\""},
        "mapviewer_release": {"args": ""},
        "mapviewer_debug": {"args": "-s:h=\"build_type=Debug\""}
    }
}
```

This build order summarizes the necessary builds. First it is necessary to build all different binaries for `engine/1.0`. This recipe contains 2 different binaries, one for Release and the other for Debug. These binaries belong to the same element in the `packages` list, which means they do not depend on each other and can be built in parallel. Each binary tracks its own original build-order file with `"filenames": ["game_release"]`, so it is possible to deduce the necessary profiles to apply to it. The `build_order.json` file contains a `profiles` section that helps recovering the profile and settings command line arguments that were used to create the respective original build-order files.

Then, after all binaries of `engine/1.0` have been built, it is possible to proceed to build the different binaries for `game/1.0`. It also contains 2 different binaries for its debug and release configurations, which can be built in parallel.

In practice, this would mean something like:

```
# This 2 could be executed in parallel
# (in different machines, or different Conan caches)
$ conan install --requires=engine/1.0 --build=engine/1.0
$ conan install --requires=engine/1.0 --build=engine/1.0 -s build_type=Debug

# Once engine/1.0 builds finish, it is possible
# to build these 2 binaries in parallel (in different machines or caches)
$ conan install --requires=game/1.0 --build=game/1.0
$ conan install --requires=game/1.0 --build=game/1.0 -s build_type=Debug
```

In this section we have still omitted some important implementation details that will follow in next sections. The goal was to focus on the `conan graph build-order-merge` command and how different products and configurations can be merged in a single "build-order". The next section will show with more details how this build-order can be really distributed in CI, using lockfiles to guarantee constant dependencies.

Products pipeline: distributed full pipeline with lockfiles

This section will present the full and complete implementation of a multi-product, multi-configuration distributed CI pipeline. It will cover important implementation details:

- Using lockfiles to guarantee a consistent and fixed set of dependencies for all configurations.
- Uploading built packages to the `products` repository.
- Capturing "package lists" and using them to run the final promotion.
- How to iterate the "build-order" programmatically

Let's start as usual cleaning the local cache and defining the correct repos:

```
# First clean the local "build" folder
$ pwd  # should be <path>/examples2/ci/game
$ rm -rf build  # clean the temporary build folder
$ mkdir build && cd build # To put temporary files

$ conan remove "*" -c  # Make sure no packages from last run
```

(continues on next page)

(continued from previous page)

```
# NOTE: The products repo is first, it will have higher priority.
$ conan remote enable products
```

Similarly to what we did in the `packages pipeline` when we wanted to ensure that the dependencies are exactly the same when building the different configurations and products, the first necessary step is to compute a `conan.lock` lockfile that we can pass to the different CI build agents to enforce the same set of dependencies everywhere. This can be done incrementally for the different `products` and configurations, aggregating it in the final single `conan.lock` lockfile. This approach assumes that both `game/1.0` and `mapviewer/1.0` will be using the same versions and revisions of the common dependencies.

```
$ conan lock create --requires=game/1.0 --lockfile-out=conan.lock
$ conan lock create --requires=game/1.0 -s build_type=Debug --lockfile=conan.lock --
↪lockfile-out=conan.lock
$ conan lock create --requires=mapviewer/1.0 --lockfile=conan.lock --lockfile-out=conan.
↪lock
$ conan lock create --requires=mapviewer/1.0 -s build_type=Debug --lockfile=conan.lock --
↪lockfile-out=conan.lock
```

Note: Recall that the `conan.lock` arguments are mostly optional, as that is the default lockfile name. The first command can be typed as `conan lock create --requires=game/1.0`. Also, all commands, including `conan install`, if they find a existing `conan.lock` file they will use it automatically, without an explicit `--lockfile=conan.lock`. The commands in this tutorial are shown explicitly complete for completeness and didactical reasons.

Then, we can compute the build order for each product and configuration. These commands are identical to the ones in the previous section, with the only difference of adding a `--lockfile=conan.lock` argument:

```
$ conan graph build-order --requires=game/1.0 --lockfile=conan.lock --build=missing --
↪order-by=recipe --format=json > game_release.json
$ conan graph build-order --requires=game/1.0 --lockfile=conan.lock --build=missing -s␣
↪build_type=Debug --order-by=recipe --format=json > game_debug.json
$ conan graph build-order --requires=mapviewer/1.0 --lockfile=conan.lock --build=missing␣
↪--order-by=recipe --format=json > mapviewer_release.json
$ conan graph build-order --requires=mapviewer/1.0 --lockfile=conan.lock --build=missing␣
↪-s build_type=Debug --order-by=recipe --format=json > mapviewer_debug.json
```

Likewise the `build-order-merge` command will be identical to the previous one. In this case, as this command doesn't really compute a dependency graph, a `conan.lock` argument is not necessary, dependencies are not being resolved:

```
$ conan graph build-order-merge --file=game_release.json --file=game_debug.json --
↪file=mapviewer_release.json --file=mapviewer_debug.json --reduce --format=json > build_
↪order.json
```

So far, this process has been almost identical to the previous section one, just with the difference of capturing and using a lockfile. Now, we will explain the "core" of the `products` pipeline: iterating the build-order and distributing the build, and gathering the resulting built packages.

This would be an example of some Python code that performs the iteration sequentially (a real CI system would distribute the builds to different agents in parallel):

```
build_order = open("build_order.json", "r").read()
build_order = json.loads(build_order)
to_build = build_order["order"]

pkg_lists = []  # to aggregate the uploaded package-lists
for level in to_build:
    for recipe in level:  # This could be executed in parallel
        ref = recipe["ref"]
        # For every ref, multiple binary packages are being built.
        # This can be done in parallel too. Often it is for different platforms
        # they will need to be distributed to different build agents
        for packages_level in recipe["packages"]:
            # This could be executed in parallel too
            for package in packages_level:
                build_args = package["build_args"]
                filenames = package["filenames"]
                build_type = "-s build_type=Debug" if any("debug" in f for f in␣
↪filenames) else ""
                run(f"conan install {build_args} {build_type} --lockfile=conan.lock --
↪format=json", file_stdout="graph.json")
                run("conan list --graph=graph.json --format=json", file_stdout="built.
↪json")
                filename = f"uploaded{len(pkg_lists)}.json"
                run(f"conan upload -l=built.json -r=products -c --format=json", file_
↪stdout=filename)
                pkg_lists.append(filename)
```

Note:

- This code is specific for the `--order-by=recipe` build-order, if chosing the `--order-by=configuration`, the json is different and it would require a different iteration.

These are the tasks that the above Python code is doing:

- For every `package` in the build-order, a `conan install --require=<pkg> --build=<pkg>` is issued, and the result of this command is stored in a `graph.json` file
- The `conan list` command transform this `graph.json` into a package list called `built.json`. Note that this package list actually stores both the built packages and the necessary transitive dependencies. This is done for simplicity, as later these package lists will be used for running a promotion, and we also want to promote the dependencies such as `ai/1.1.0` that were built in the `packages pipeline` and not by this job.
- The `conan upload` command uploads the package list to the `products` repo. Note that the `upload` first checks what packages already exist in the repo, avoiding costly transfers if they already exist.
- The result of the `conan upload` command is captured in a new package list called `uploaded<index>.json`, that we will accumulate later, that will serve for the final promotion.

In practice this translates to the following commands (that you can execute to continue the tutorial):

```
# engine/1.0 release
$ conan install --requires=engine/1.0 --build=engine/1.0 --lockfile=conan.lock --
↪format=json > graph.json
$ conan list --graph=graph.json --format=json > built.json
```

(continues on next page)

(continued from previous page)

```
$ conan upload -l=built.json -r=products -c --format=json > uploaded1.json

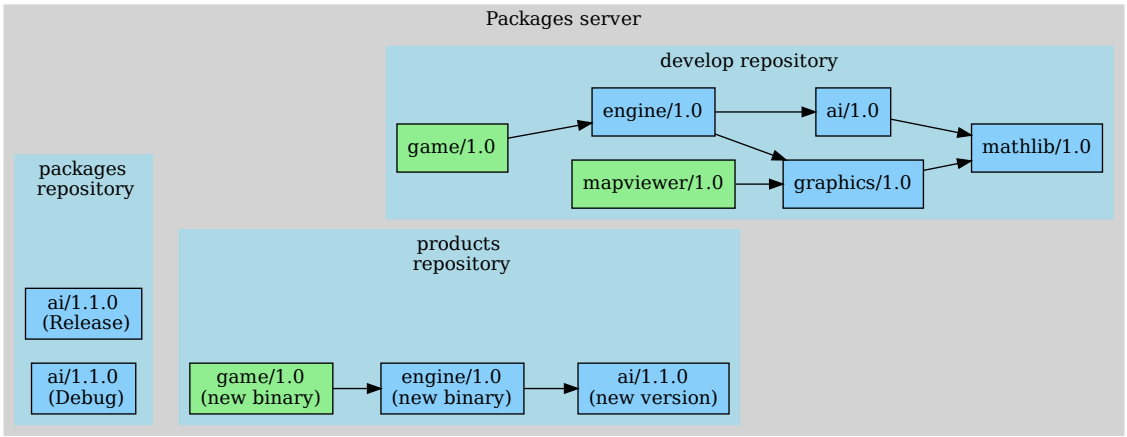
# engine/1.0 debug
$ conan install --requires=engine/1.0 --build=engine/1.0 --lockfile=conan.lock -s build_
↪type=Debug --format=json > graph.json
$ conan list --graph=graph.json --format=json > built.json
$ conan upload -l=built.json -r=products -c --format=json > uploaded2.json

# game/1.0 release
$ conan install --requires=game/1.0 --build=game/1.0 --lockfile=conan.lock --format=json␣
↪> graph.json
$ conan list --graph=graph.json --format=json > built.json
$ conan upload -l=built.json -r=products -c --format=json > uploaded3.json

# game/1.0 debug
$ conan install --requires=game/1.0 --build=game/1.0 --lockfile=conan.lock -s build_
↪type=Debug --format=json > graph.json
$ conan list --graph=graph.json --format=json > built.json
$ conan upload -l=built.json -r=products -c --format=json > uploaded4.json
```

After this step the newly built packages will be in the `products` repo and we will have 4 `uploaded1.json` - `uploaded4.json` files.

Simplifying the different release and debug configurations, the state of our repositories would be something like:



We can now accumulate the different `uploadedX.json` files into a single package list `uploaded.json` that contains everything:

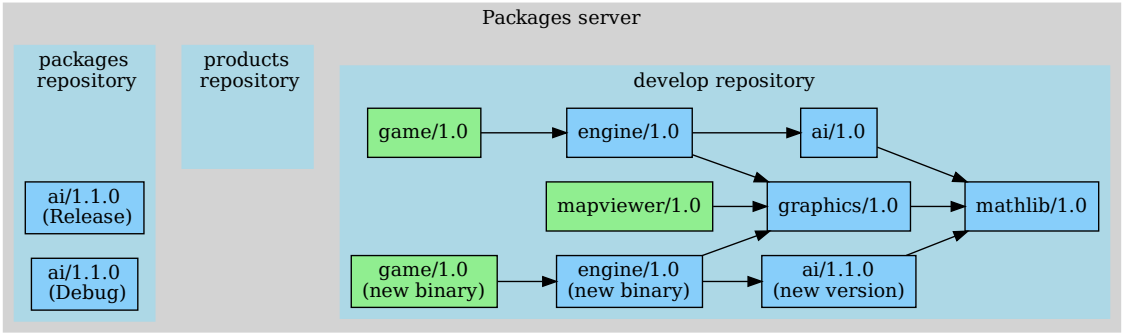
```
$ conan pkglist merge -l uploaded0.json -l uploaded1.json -l uploaded2.json -l uploaded3.
↪json --format=json > uploaded.json
```

And finally, if everything worked well, and we consider this new set of versions and new package binaries is ready to be used by developers and other CI jobs, then we can run the final promotion from the `products` to the `develop` repository:

Listing 8: Promoting from products->develop

```
# Promotion using Conan download/upload commands
# (slow, can be improved with art:promote custom command)
$ conan download --list=uploaded.json -r=products --format=json > promote.json
$ conan upload --list=promote.json -r=develop -c
```

And our final `develop` repository state will be:



This state of the `develop` repository will have the following behavior:

- Developers installing `game/1.0` or `engine/1.0` will by default resolve to latest `ai/1.1.0` and use it. They will find pre-compiled binaries for the dependencies too, and they can continue developing using the latest set of dependencies.
- Developers and CI that were using a lockfile that was locking `ai/1.0` version, will still be able to keep working with that dependency without anything breaking, as the new versions and package binaries do not break or invalidate the previous existing binaries.

At this point, the question of what to do with the lockfile used in the Ci could arise. Note that the `conan.lock` now contains the `ai/1.1.0` version locked. There could be different strategies, like storing this lockfile in the "products" git repositories, making it easily available when developers checkout those repos. Note, however, that this lockfile matches the latest state of the `develop` repo, so developers checking out one of the "products" git repositories and doing a `conan install` against the `develop` server repository will naturally resolve to the same dependencies stored in the lockfile.

It is a good idea to at least store this lockfile in any release bundle, if the "products" are bundled somehow (a installer, a debian/rpm/choco/etc package), to include or attach to this bundled release for the final users of the software, the lockfile used to produce it, so no matter what changes in development repositories, those lockfiles can be recovered from the release information later in time.

Final remarks

As commented in this CI tutorial introduction, this doesn't pretend to be a silver bullet, a CI system that you can deploy as-is in your organization. This tutorial so far presents a "happy path" of a Continuous Integration process for developers, and how their changes in packages that are part of larger products can be tested and validated as part of those products.

The focus of this CI tutorial is to introduce some important concepts, good practices and tools such as:

- The importance of defining the organization "products", the main deliverables that need to be checked and built against new dependencies versions created by developers.
- How new dependencies versions of developers shouldn't be uploaded to the main development repositories until validated, to not break other developers and CI jobs.
- How multiple repositories can be used to build a CI pipeline that isolate non validated changes and new versions.
- How large dependency graphs can be built efficiently in CI with the `conan graph build-order`, and how build-orders for different configurations and products can be merged together.
- Why `lockfiles` are necessary in CI when there are concurrent CI builds.
- The importance of versioning, and the role of `package_id` to re-build only what is necessary in large dependency graphs.
- Not using `user/channel` as variable and dynamic qualifiers of packages that change across the CI pipeline, but using instead different server repositories.
- Running package promotions (copies) across server repositories when new package versions are validated.

There are still many implementation details, strategies, use cases, and error scenarios that are not covered in this tutorial yet:

- How to integrate breaking changes of a package that requires a new breaking major version.
- Different versioning strategies, using pre-releases, using versions or relying on recipe revisions in certain cases.
- How lockfiles can be stored and used across different builds, if it is good to persist them and where.
- Different branching and merging strategies, nightly builds, releases flows.

We plan to extend this CI tutorial, including more examples and use cases. If you have any question or feedback, please create a ticket in https://github.com/conan-io/conan/issues.

CHAPTER

FIVE

DEVOPS GUIDE

The previous *tutorial* section was aimed at users in general and developers.

The *Continuous Integration tutorial* explained the basics on how to implement Continuous Integration involving Conan packages.

This section is intended for DevOps users, build and CI engineers, administrators, and architects adopting, designing and implementing Conan in production in their teams and organizations. If you plan to use Conan in production in your project, team, or organization, this section contains the necessary information.

5.1 Using ConanCenter packages in production environments

Note: Default Remote Update in Conan 2.9.2

Starting from **Conan version 2.9.2**, the default remote has been changed to *https://center2.conan.io*. The previous default remote *https://center.conan.io* is now frozen and will no longer receive updates. It is recommended to update your remote configuration to use the new default remote to ensure access to the latest recipes and package updates (for more information, please read this post).

If you still have the deprecated remote configured as the default, please update using the following command:

```
conan remote update conancenter --url="https://center2.conan.io"
```

ConanCenter is a fantastic resource that contains reference implementations of recipes for over 1500 libraries and applications contributed by the community. As such, it is a great knowledge base on how to create and build Conan packages for open source dependencies.

ConanCenter also builds and provides binary packages for a wide range of configurations: multiple operating systems (Windows, Linux, macOS), compilers, compiler versions, and library variants (shared, static). On top of this, for a lot of libraries community contributors ensure that recipes are compatible for additional operating systems (Android, iOS, FreeBSD, QNX) and CPU architectures. The recipes in Conan Center are the greatest example of Conan's universality promise.

Unlike other package managers or repositories, ConanCenter does not maintain a fixed snapshot of versions. On the contrary, for a given library (e.g. OpenCV), multiple versions are actively maintained at the same time. This gives users greater control of which versions to use, rather than having to remain fixed to an older version, or pushing them to always be on the latest version.

In order to support this ecosystem, ConanCenter recipes are updated very frequently. Recipes themselves may be updated to support a new platform, bug fixes, or to require newer versions of their dependencies. On the other hand, each user of ConanCenter may have a different combination of versions in their requirements. This means that given the same input list of requirements, Conan may resolve the graph differently at different points in time - resolving to

different recipe revisions, versions, or packages. This is similar to the default behavior of package managers in other languages (pip/PyPi, npm, cargo, etc). In production environments where reproducibility is important, it is therefore discouraged to depend directly on Conan Center in an unconstrained manner.

The following guidelines contain a series of recommendations to ensure repeatability, reliability, compliance and, where applicable, control to enable customization. As a summary, it is highly recommended to follow these approaches when using packages from ConanCenter:

- Lock the versions and revisions you depend on using *lockfiles*
- Host your own copy of ConanCenter recipes and package binaries *in a server under your control*

5.1.1 Repeatability and reproducibility

As mentioned earlier - given a set of requirements, changes in ConanCenter can cause the Conan dependency solver to resolve different graphs over time. This does not only apply to the actual versions of libraries (e.g. `opencv/4.5.0` instead `opencv/4.2.1`) - but also the recipes themselves. That is, there may exist multiple revisions of the `opencv/4.5.0` recipe, which can have side effects for consumers. Changes in recipes typically address a problem (bugfixes), target functionality (e.g. adding a conditional option, support for a new platform), or change versions of dependencies.

In order to ensure repeatability, the use of lockfiles on the consumer side is greatly encouraged: please check *the lockfile docs* for more information.

Lockfiles ensure that Conan will resolve the same graph in a repeatable and consistent manner - thus making sure the same versions are used across multiple systems (CI, developers, etc).

Lockfiles are also used in other package managers like Python pip, Rust Cargo, npm - these recommendations are in line with the practices of these other technologies.

Additionally, it is highly recommended to host your recipes and packages in your own server (see below). Both of these approaches help you achieve having control on when upstream changes from ConanCenter are propagated across your team and systems.

5.1.2 Service reliability

Consuming recipes and packages from the ConanCenter remote can be impacted during periods of downtime (scheduled or otherwise). While every effort is made to ensure that the ConanCenter is always available, and unscheduled downtime is rare and treated with urgency - this can impact users that depend on ConanCenter directly. Additionally, when building recipes from source, this requires retrieving the source packages (typically zip or tar files) from remote servers outside of the control of ConanCenter. Occasionally, these too can suffer from unscheduled downtime.

In enterprise production environments with strong uptime is required, it is strongly recommended to host recipes and binary packages in a server under your control.

- Read more: *creating and hosting your own Conan Center binaries*

This can also protect against transient network issues, and issues caused by transfer of binary data from external sources. These recommendations also apply when consuming packages from external sources in any package manager.

5.1.3 Compliance and security

Some industries such as finance, robotics and embedded, have stronger requirements around change management, open source licenses and reproducibility. For example, changes in recipes could result in a new version being resolved for a dependency, in a way that the license for that version has changed and needs to be validated and audited by your organization. In some industries like medical or automotive, you may be required to ensure all your dependencies can be built from source in a repeatable way, and thus using binaries provided by Conan Center may not be advisable. In these instances, we recommend building your own binary packages from source:

- Read more: *creating and hosting your own Conan Center binaries*

If the `conancenter` remote is used directly, your organization might require that the binaries are built from source, and not downloaded directly from ConanCenter. This can exceptionally be achieved with the `recipes_only` field *in the remote configuration*, but it is highly likely that you will also want to build the packages from source, and not depend on the recipes and binaries provided by ConanCenter. See the above link for more information.

5.1.4 Control and customization

It is very common for users of dependencies to require custom changes to external libraries - typically to support specific platform configurations not considered by either ConanCenter or the original library authors, backport bug fixes, etc. Some of these changes may not be suitable to be merged in ConanCenter, and it may not happen until this has been reviewed and validated by ConanCenter maintainers. For this reason, if you need tight control over the changes in recipes, it is highly recommended to host not only a Conan remote, but your own fork of the conan-center-index recipe repository.

- Read more: *creating and hosting your own Conan Center binaries*

The following subsections describe in more details the above strategies:

Creating and hosting your own ConanCenter binaries

Hosting your own copy of the packages you need in your server could be done by just downloading binaries from ConanCenter and then uploading them to your own server. However, it is much better to fully own the complete supply chain and create the binaries in your own CI systems. So the recommended flow to use ConanCenter packages in production would be:

- Create a fork of the ConanCenter Github repository: https://github.com/conan-io/conan-center-index
- Create a list of the packages and versions you need for your projects. This list can be added to the fork too, and maintained there (packages can be added and removed with PRs when the teams need them).
- Create a script that first `conan export` all the packages in your list, then `conan create --build=missing` them.
- Upload your build packages to your own server, that you use in production, instead of ConanCenter.

Note: Best practices

- Do not add `user/channel` to packages created from ConanCenter forks, it is way simpler to create and use them as `zlib/1.2.13` without user-channel. The `user/channel` part would be mostly recommended for your own proprietary packages, but not for open source ConanCenter packages. It adds more divergence from the upstream and consequently more maintenance with little added value.
- Do not mix packages and recipes created from your fork of `conan-center-index` Github repo with the ones from ConanCenter central server. Once you create some binaries for third parties from your fork, it is strongly recommended to fully disconnect from ConanCenter (you can remove the `remote`, and you can add

the `remotes.json` file with your own remotes to the configuration you can distribute and install with `conan config install/install-pkg`), and create all your third-party packages from your fork.

This is the basic flow idea. We will be adding examples and tools to further automate this flow as soon as possible.

This flow is relatively straightforward, and has many advantages that mitigate the risks described before:

- No central repository outage can affect your builds.
- No changes in the central repository can break your projects, you are in full control when and how those changes are updated in your packages (as explained below).
- You can customize, adapt, fix and perfectly control what versions are used, and release fixes in minutes, not weeks. You can apply customizations that wouldn't be accepted in the central repository.
- You fully control the binaries supply chain, from the source (recipes) to the binaries, eliminating in practice the majority of potential supply chain attacks of central repositories.

Updating from upstream

Updating from the upstream `conan-center-index` Github repo is still possible, and it can be done in a fully controlled way:

- Merge the latest changes in the upstream main fork of `conan-center-index` into your fork.
- You can check and audit those changes if you want to, analyzing the diffs (some automation that trims the diffs of recipes that you don't use could be useful)
- Firing the above process will efficiently rebuild the new binaries that are needed. If your recipes are not affected by changes, the process will avoid rebuilding binaries (thanks to `--build=missing`).
- You can upload the packages to a secondary "test" server repository. Then test your project against that test server, to check that your project is not broken by the new ConanCenter packages.
- Once you verify that everything is good with the new packages, you can copy them from the secondary "test" repository to your main production repository to start using them.

5.2 Local Recipes Index Repository

The **Local Recipes Index** is an **experimental** repository type introduced in Conan to enhance flexibility in managing C/C++ package recipes. This repository type allows users to use a local directory as a Conan remote, where the directory structure mirrors that of the *conan-center-index* GitHub repository.

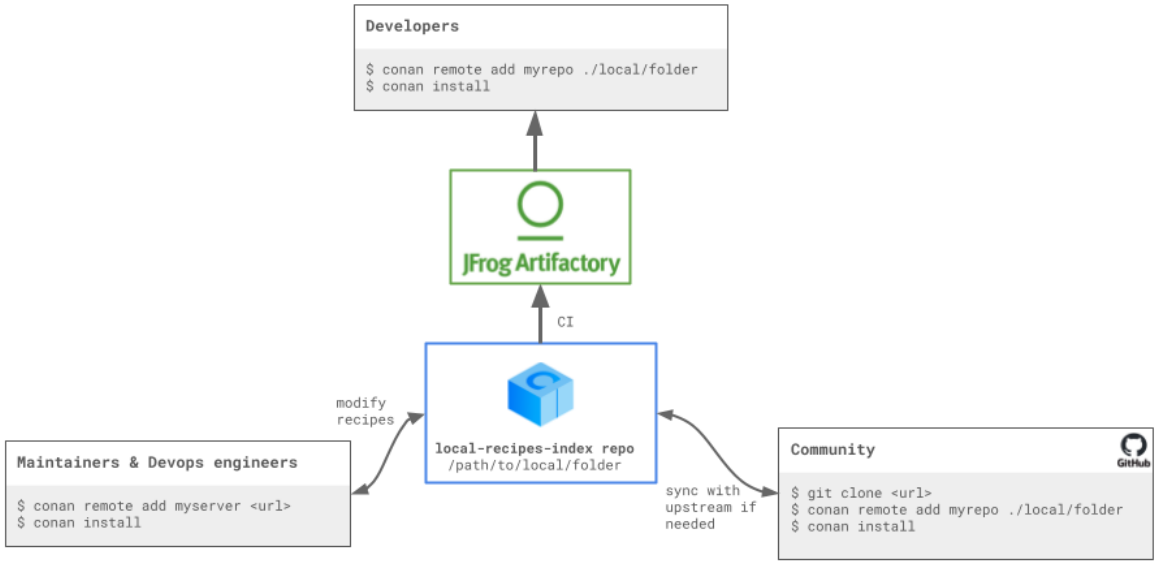
This setup is particularly useful for:

- Building binaries from a private *conan-center-index* fork.
- Sharing your own recipes for certain libraries or tools that, due to licensing restrictions or proprietary nature, are not suitable for ConanCenter. Check how you can use it for this purpose in the dedicated section of the documentation *Local Recipes Index Repository*.

5.2.1 Building Binaries from a private *conan-center-index* fork

As we already introduced in the *previous section of the Conan DevOps Guide* some organizations, particularly large enterprises, prefer not to use binaries downloaded from the internet. Instead, they build their own binaries in-house using the *conan-center-index* recipes. These organizations often need to customize these recipes to meet unique requirements that are not applicable to the broader community, making such contributions unsuitable for the upstream repository.

The *local-recipes-index* allows users to maintain a local folder with the same structure as the *conan-center-index* GitHub repository, using it as a source for package recipes. This new type of repository is recipes-only, necessitating the construction of package binaries from source on each machine where the package is used. For sharing binaries across teams, we continue to recommend *using a Conan remote server like Artifactory* for production purposes.



The *local-recipes-index* repository allows you to easily build binaries from a fork of *conan-center-index*, and then hosting them on a Conan remote repository like Artifactory. The main difference with the process explained in the *previous section* is the ability to immediately test multiple local changes without the need to export each time a recipe is modified.

Note that in this case, mixing binaries from ConanCenter with locally built binaries is not recommended for several reasons:

- Binary compatibility: There may be small differences in setup between the ConanCenter CI and the user's CI. Maintaining a consistent setup for all binaries can mitigate some issues.
- Full control over builds: Building all binaries yourself ensures you have complete control over the compilation environment and dependency versions.

Instead, it's recommended to build all your direct and transitive dependencies from the fork. To begin, remove the upstream ConanCenter as it will not be used, everything will come from our own fork:

```
$ conan remote remove conancenter
```

Then we will clone our fork (in this case, we are cloning directly the upstream for demo purposes, but you would be cloning your fork instead):

```
$ git clone https://github.com/conan-io/conan-center-index
```

Add this as our *mycenter* remote:

```
# Add the mycenter remote pointing to the local folder
$ conan remote add mycenter ./conan-center-index
```

And that's all! Now you're set to list and use packages from your *conan-center-index* local folder:

```
$ conan list "zlib/*" -r=mycenter
mycenter
  zlib
    zlib/1.2.11
    zlib/1.2.12
    zlib/1.2.13
    zlib/1.3
    zlib/1.3.1
```

We can also install packages from this repo, for example we can do:

```
$ conan install --requires=zlib/1.3
...
======== Computing dependency graph ========
zlib/1.3: Not found in local cache, looking in remotes...
zlib/1.3: Checking remote: mycenter
zlib/1.3: Downloaded recipe revision 5c0f3a1a222eebb6bff34980bcd3e024
Graph root
    cli
Requirements
    zlib/1.3#5c0f3a1a222eebb6bff34980bcd3e024 - Downloaded (mycenter)

======== Computing necessary packages ========
Requirements
    zlib/1.3#5c0f3a1a222eebb6bff34980bcd3e024:72c852c5f0ae27ca0b1741e5fd7c8b8be91a590a -␣
↪Missing
ERROR: Missing binary: zlib/1.3:72c852c5f0ae27ca0b1741e5fd7c8b8be91a590a
```

As we can see, Conan managed to get the recipe for `zlib/1.3` from `mycenter`, but then it failed because there is no binary. This is expected, **the repository only contains the recipes, but not the binaries**. We can build the binary from source with `--build=missing` argument:

```
$ conan install --requires=zlib/1.3 --build=missing
...
zlib/1.3: package(): Packaged 2 '.h' files: zconf.h, zlib.h
zlib/1.3: package(): Packaged 1 file: LICENSE
zlib/1.3: package(): Packaged 1 '.a' file: libz.a
zlib/1.3: Created package revision 0466b3475bcac5c2ce37bb5deda835c3
zlib/1.3: Package '72c852c5f0ae27ca0b1741e5fd7c8b8be91a590a' created
zlib/1.3: Full package reference: zlib/1.3
↪#5c0f3a1a222eebb6bff34980bcd3e024:72c852c5f0ae27ca0b1741e5fd7c8b8be91a590a
↪#0466b3475bcac5c2ce37bb5deda835c3
zlib/1.3: Package folder /home/conan/.conan2/p/b/zlib1ed9fe13537a2/p
WARN: deprecated: Usage of deprecated Conan 1.X features that will be removed in Conan 2.
```

(continues on next page)

(continued from previous page)

```
↪X:
WARN: deprecated:     'cpp_info.names' used in: zlib/1.3

======== Finalizing install (deploy, generators) ========
cli: Generating aggregated env files
cli: Generated aggregated env files: ['conanbuild.sh', 'conanrun.sh']
Install finished successfully
```

We can see now the binary package in our local cache:

```
$ conan list "zlib:*"
Local Cache
  zlib
    zlib/1.3
    revisions
      5c0f3a1a222eebb6bff34980bcd3e024 (2024-04-10 11:50:34 UTC)
      packages
        72c852c5f0ae27ca0b1741e5fd7c8b8be91a590a
        info
          settings
            arch: x86_64
            build_type: Release
            compiler: gcc
            compiler.version: 9
            os: Linux
          options
            fPIC: True
            shared: False
```

Finally, upload the binary package to our Artifactory repository to make it available for our organization, users and CI jobs:

```
$ conan remote add myartifactoryrepo <artifactory_url>
$ conan upload zlib* -r=myartifactoryrepo -c
```

This way, consumers of the packages will not only enjoy the pre-compiled binaries and avoid having to always re-build from source all dependencies, but that will also provide stronger guarantees that the dependencies build and work correctly, that all dependencies and transitive dependencies play well together, etc. Decoupling the binary creation process from the binary consumption process is the way to achieve faster and more reliable usage of dependencies.

Remember, in a production setting, the *conan upload* command should be executed by CI, not developers, following the *Conan guidelines*. This approach ensures that package consumers enjoy pre-compiled binaries and consistency across dependencies.

5.2.2 Modifying the local-recipes-index repository files

One of the advantages of this approach is that all the changes that we do in every single recipe are automatically available for the Conan client. For example, changes to the *recipes/zlib/config.yml* file are immediately recognized by the Conan client. If you edit that file and remove all versions but the latest and then we *list* the recipes:

```
$ conan list "zlib/*" -r=mycenter
mycenter
  zlib
    zlib/1.3.1
```

When some of the recipes change, then note that the current Conan home already contains a cached copy of the package, so it will not update it unless we explicitly use the `--update`, as any other Conan remote.

So if we do a change in the `zlib` recipe in `recipes/zlib/all/conanfile.py` and repeat:

```
$ conan install --requires=zlib/1.3.1 -r=mycenter --update --build=missing
```

We will immediately have the new package binary locally built from source from the new modified recipe in our Conan home.

5.2.3 Using local-recipes-index Repositories in Production

Several important points should be considered when using this new feature:

- It is designed for **third-party packages**, where recipes in one repository are creating packages with sources located elsewhere. To package your own code, the standard practice of adding *conanfile.py* recipes along with the source code and using the standard *conan create* flow is recommended.
- The *local-recipes-index* repositories point to **local folders in the filesystem**. While users may choose to sync that folder with a git repository or other version control mechanisms, Conan is agnostic to this, as it is only aware of the folder in the filesystem that points to the (current) state of the repository. Users may choose to run git commands directly to switch branches/commit/tags and Conan will automatically recognise the changes
- This approach operates at the source level and does not generate package binaries. For deployment for development and production environments, the use of a remote package server such as Artifactory is crucial. It's important to note that this feature is not a replacement for Conan's remote package servers, which play a vital role in hosting packages for regular use.
- Also, note that a server remote can retain a history of changes storing multiple recipe revisions. In contrast, a *local-recipes-index* remote can only represent a single snapshot at any given time.
- ConanCenter does not use `python-requires`, as this is a mechanism more intended for first-party packages. Using `python-requires` in a `local-recipes-index` repository is possible (and experimental) at this moment, but only if the `python-requires` are also in the same index repository. It is not intended or planned to support having these `python-requires` in other repositories or in the user Conan cache.

Furthermore, this feature does not support placing server URLs directly in recipes; remote repositories must be explicitly added with *conan remote add*. Decoupling abstract package requirements, such as "zlib/1.3.1", from their specific origins is crucial to resolving dependencies correctly and leveraging Conan's graph capabilities, including version conflict detection and resolution, version-ranges resolution, *opting into pre-releases*, *platform_requires*, *replace_requires*, etc. This separation also facilitates the implementation of modern DevOps practices, such as package immutability, full relocatability and package promotions.

See also:

- *Using Local-Recipes-Index repositories to share your libraries*

- Introducing the Local-Recipes-Index Post

5.3 Backing up third-party sources with Conan

For recipes and build scripts for open source, publicly available libraries, it is common practice to download the sources from a canonical source, like Github releases, or project download web pages. Keeping a record of the origin of these files is useful for traceability purposes, however, it is often not guaranteed that the files will be available in the long term, and a user in the future building the same recipe from source may encounter a problem. Conan can thus be configured to transparently retrieve sources from a configured mirror, without modifying the recipes or *conandata.yml*. Additionally, these sources can be transparently uploaded alongside the packages via **conan upload**.

The *sources backup* feature is intended for storing the downloaded recipe sources in a file server in your own infrastructure, allowing future reproducibility of your builds even in the case where the original download URLs are no longer accessible.

The backup is triggered for calls to the *download* and *get* methods when a `sha256` file hash is provided.

5.3.1 Configuration overview

This feature is controlled by a few *global.conf* items:

- `core.sources:download_cache`: Local path to store the sources backups to. *If not set, the default Conan home cache path will be used.*
- `core.sources:download_urls`: Ordered list of URLs that Conan will try to download the sources from, where `origin` represents the original URL passed to `get`/`download` from *conandata.yml*. This allows to control the fetch order, either `["origin", "https://your.backup/remote/"]` to look into and fetch from your backup remote only if and when the original source is not present, or `["https://your.backup/remote/", "origin"]` to prefer your backup server ahead of the recipes' canonical links. Being a list, multiple remotes are also possible. `["origin"]` *by default*
- `core.sources:upload_url`: URL of the remote to upload the backups to when calling **conan upload**, which might or might not be different from any of the URLs defined for download. *Empty by default*
- `core.sources:exclude_urls`: List of origins to skip backing up. If the URL passed to `get`/`download` starts with any of the origins included in this list, the source won't be uploaded to the backup remote when calling **conan upload**. *Empty by default*

Note: When adding a backup source remote to `core.sources:download_urls`, if the server is not reachable, (or if it requires authentication and it is not provided), Conan will **NOT** skip it and continue to the next URL in the list, instead it will raise an error and the download will fail. If your remote backup server is down and you want to be able to continue downloading from the original URLs, remove the backup remote from the list temporarily.

5.3.2 Usage

Let's overview how the feature works by providing an example usage from beginning to end:

In summary, it looks something like:

- A remote backup repository is set up. This should allow `PUT` and `GET` HTTP methods to modify and fetch its contents. If access credentials are desired (which is strongly recommended for uploading permissions), you can use the *source_credentials.json* feature. *See below* if you are in need for configuring your own.
- The remote's URL can then be set in `core.sources:download_urls` and `core.sources:upload_url`.
- In your recipe's `source()` method, ensure the relevant `get/download` calls supply the `sha256` hash of the downloaded files.
- Set `core.sources:download_cache` in your *global.conf* file if a custom location is desired, else the default cache folder will be used
- Run Conan normally, creating packages etc.
- **Once some sources have been locally downloaded, the folder pointed to by `core.sources:download_cache` will contain, for each downloaded file:**
  - A blob file (no extensions) with the name of the `sha256` hash provided in `get/download`.
  - A `.json` file which will also have the name of the `sha256` hash, that will contain information about which references and which mirrors this blob belongs to.
- Calling `conan upload` will now optionally upload the backups for the matching references if `core.sources:upload_url` is set.

Note: *See below* for a guide on how to configure your own backup server

Setting up the necessary configs

The *global.conf* file should contain the `core.sources:download_urls` if downloading from a custom backup source remote is desired, and `core.sources:download_cache` if a custom local cache path to download the backups to is desired.

Listing 1: global.conf

```
core.sources:download_urls=["https://myteam.myorg.com/artifactory/backup-sources/",
↪"origin"]
core.sources:download_cache=/path/to/backup/sources
```

Note: Either `core.sources:download_urls` or `core.sources:download_cache` should be defined for the feature to be enabled.

You might want to add extra confs based on your use case, as described *in the beginning of this document*.

Note: The recommended approach for dealing with the configuration of CI workers and developers in your organization is to install the configs using the `conan config install` command on a repository. Read more *here*

Run Conan as normal

With the above steps completed, Conan can now be used as normal, and for every downloaded source, Conan will first look into the folder indicated in `core.sources:download_cache`, and if not found there, will traverse `core.sources:download_urls` until it find the file or fails, and store a local copy in the same `core.sources:download_cache` location.

When the backup is fetched from the the backup remote, a message like what follows will be shown to the user:

Listing 2: The client will now print information regarding from which remote it was capable of downloading the sources

```
$ conan create . --version=1.3

...

======== Installing packages ========
zlib/1.3: Calling source() in /Users/ruben/.conan2/p/zlib0f4e45286ecd1/s/src
zlib/1.3: Sources for ['https://zlib.net/fossils/zlib-1.3.tar.gz', 'https://github.com/
↪madler/zlib/releases/download/v1.3/zlib-1.3.tar.gz']
          found in remote backup https://myteam.myorg.com/artifactory/backup-sources

-------- Installing package zlib/1.3 (1 of 1) --------

...
```

If we now again try to run this, we'll find that no download is performed and the locally stored version of the files is used.

Original URLs sent to the backup server

New in version 2.34.

Backup sources are stored and retrieved by the `sha256` of the file, so the backup server does not know by default where the requested file originally came from. To make this information available to the server, Conan sends the original URLs in the `X-Conan-Origin-Urls` HTTP header when requesting the file from a backup remote defined in `core.sources:download_urls`.

The value of the header is a JSON-encoded list with the URLs passed to the `get/download` call (a single URL is also sent as a list with one element). For example, for the `zlib` call shown above, the request to the backup remote would include:

```
GET /artifactory/backup-sources/<sha256> HTTP/1.1
X-Conan-Origin-Urls: ["https://zlib.net/fossils/zlib-1.3.tar.gz", "https://github.com/
↪madler/zlib/releases/download/v1.3/zlib-1.3.tar.gz"]
```

Note:

- The header is only sent in the request for the source file itself, not in the one for its `.json` metadata file.
- The header is never sent to the `origin` URLs, only to the backup remotes.
- The URLs are sent as they are specified in the recipe, so make sure they don't contain secrets (like tokens in the URL), as they would be visible to the backup server.

Upload the packages

Once a package has been created as shown above, when a call to `conan upload zlib/1.3 -c` is performed to upload the resulting binary to your Conan repository, it will also upload the source backups for that same reference to your backups remote if configured to do so, and future source downloads of this recipe will use the newly updated contents when necessary.

Note: See *the packages list feature* for a way to only upload the packages that have been built

In case there's a need to upload backups for sources not linked to any package, or for packages that are already on the remote and would therefore be skipped during upload, the **`conan cache backup-upload`** command can be used to address this scenario.

Creating the backup repository

You can also set up your own remote backup repository instead of relying on an already available one. While an Artifactory generic repository (available for free with Artifactory CE) is recommend for this purpose, any simple server that allows `PUT` and `GET` HTTP methods to modify and fetch its contents is sufficient.

Read the following section for instructions on how to create a generic Artifactory backup repo and how to give it public read permissions, while keeping write access only for authorized agents

Creating an Artifactory backup repo for your sources

For the backup repository, we'll create a generic Artifactory repo using the free Community Edition version.

For this, in the repositories section of the administration tab, we'll create a new generic repository, and in this example we'll imaginatively give it the name of *backup-sources*.

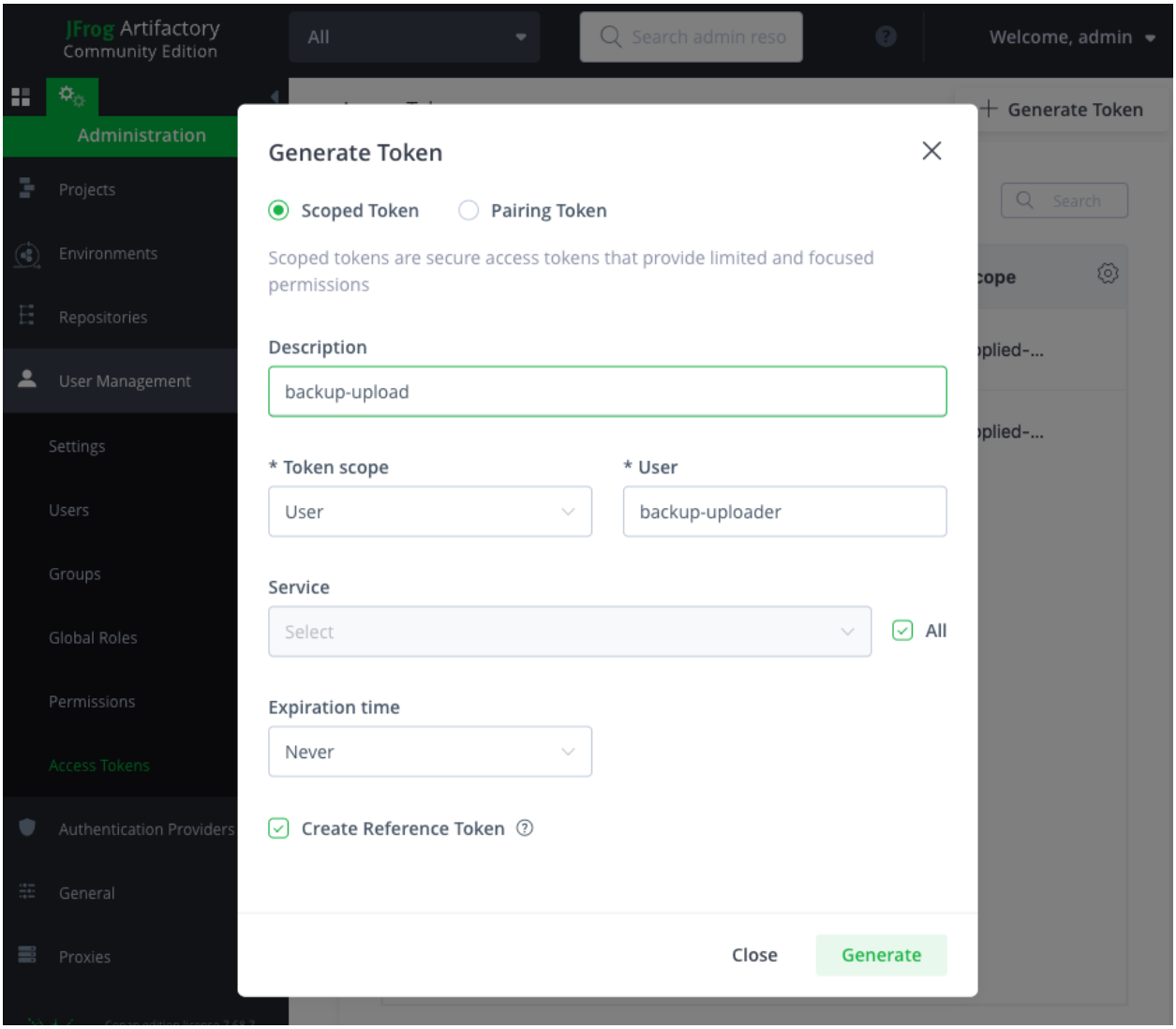
The URL of the remote should now be added to the *global.conf* file's `core.sources:upload_url` conf

Listing 3: global.conf

```
core.sources:upload_url=https://myteam.myorg.com/artifactory/backup-sources/
```

Next, as we want this to be a public read repo, we'll allow anonymous read access to our repo. See the official Artifactory documentation for a step-by-step guide on how to create one.

Now, to be able to upload contents, we'll also create a new user from the User Management section, called *backup uploader*, and from the Access Tokens section, we'll generate a reference token associated with the user



The generated token should now live in the *source_credentials.json* file:

Listing 4: source_credentials.json

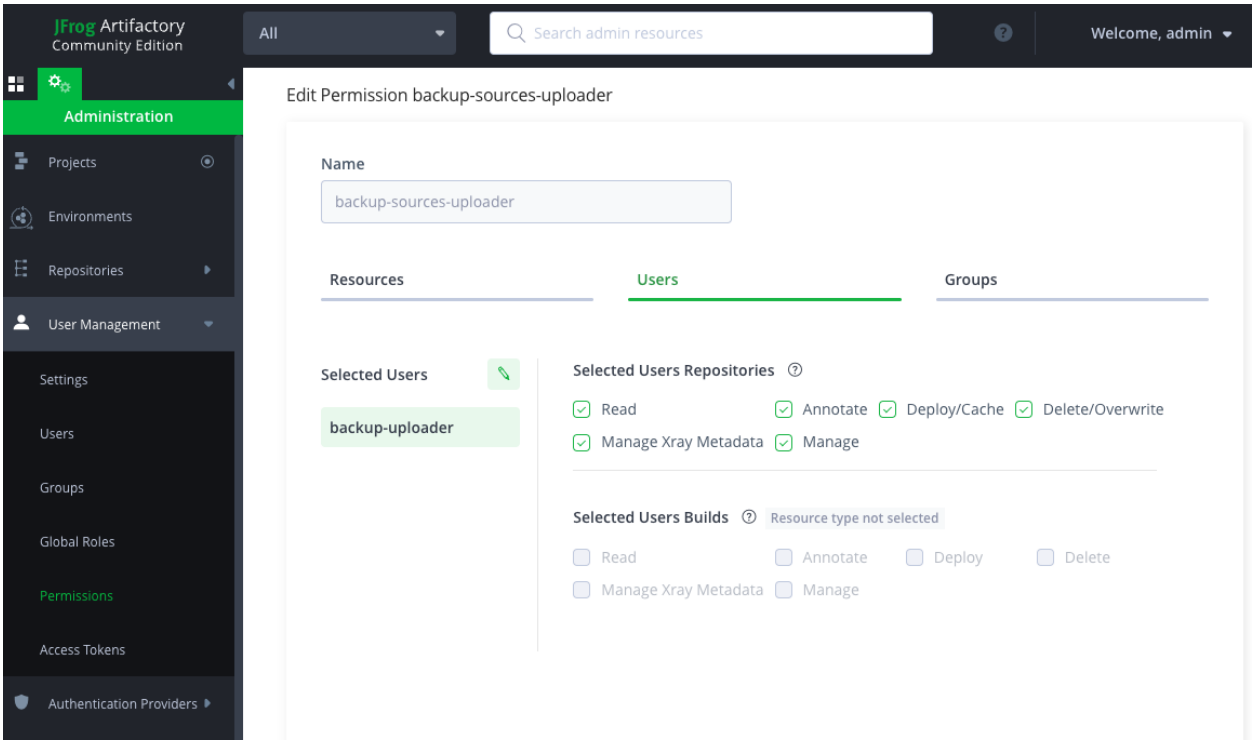
```
{
   "credentials": [
      {
         "url": "https://myteam.myorg.com/artifactory/backup-sources/",
         "token": "cmVmdGtu1234567890abcdefghijklmnopqrstuvwxyz"
```

(continues on next page)

(continued from previous page)

```
        }
    ]
}
```

And last but not least, from the Permissions section we'll give the user manage access to the new repository (which will automatically give it every other permission available, feel free to modify them according to your needs)



With this, access to our remote backup is now configured to allow anonymous read but authenticated upload.

5.4 Managing package metadata files

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

A Conan package is typically composed by several C and C++ artifacts, headers, compiled libraries and executables. But there are other files that might not be necessary for the normal consumption of such a package, but which could be very important for compliance, technical or business reasons, for example:

- Full build logs
- The tests executables
- The tests results from running the test suite
- Debugging artifacts like heavy .pdb files
- Coverage, sanitizers, or other source or binary analysis tools results
- Context and metadata about the build, exact machine, environment, author, CI data

- Other compliance and security related files

There are several important reasons to store and track these files like regulations, compliance, security, reproducibility and traceability. The problem with these files is that they can be large/heavy, if we store them inside the package (just copying the artifacts in the `package()` method), this will make the packages much larger, and it will affect the speed of downloading, unzipping and using packages in general. And this typically happens a lot of times, both in developer machines but also in CI, and it can have an impact on the developer experience and infrastructure costs. Furthermore, packages are immutable, that is, once a package has been created, it shouldn't be modified. This might be a problem if we want to add extra metadata files after the package has been created, or even after the package has been uploaded.

The **metadata files** feature allows to create, upload, append and store metadata associated to packages in an integrated and unified way, while avoiding the impact on developers and CI speed and costs, because metadata files are not downloaded and unzipped by default when packages are used.

It is important to highlight that there are two types of metadata:

- Recipe metadata, associated to the `conanfile.py` recipe, the metadata should be common to all binaries created from this recipe (package name, version and recipe revision). This metadata will probably be less common, but for example results of some scanning of the source code, that would be common for all configurations and builds, can be recipe metadata.
- Package binary metadata, associated to the package binary for a given specific configuration and represented by a `package_id`. Build logs, tests reports, etc, that are specific to a binary configuration will be package metadata.

5.4.1 Creating metadata in recipes

Recipes can directly define metadata in their methods. A common use case would be to store logs. Using the `self.recipe_metadata_folder` and `self.package_metadata_folder`, the recipe can store files in those locations.

```
import os
from conan import ConanFile
from conan.tools.files import save, copy

class Pkg(ConanFile):
    name = "pkg"
    version = "0.1"

    def layout(self):
        # Or something else, like the "cmake_layout(self)" built-in layout
        self.folders.build = "mybuild"
        self.folders.generators = "mybuild/generators"

    def export(self):
        # logs that might be generated in the recipe folder at "export" time.
        # these would be associated with the recipe repo and original source of the recipe␣
↪repo
        copy(self, "*.log", src=self.recipe_folder,
             dst=os.path.join(self.recipe_metadata_folder, "logs"))

    def source(self):
        # logs originated in the source() step, for example downloading files, patches or␣
↪other stuff
        save(self, os.path.join(self.recipe_metadata_folder, "logs", "src.log"), "srclog!!
↪")
```

(continues on next page)

(continued from previous page)

```
    def build(self):
        # logs originated at build() step, the most common ones
        save(self, "mylogs.txt", "some logs!!!")
        copy(self, "mylogs.txt", src=self.build_folder,
             dst=os.path.join(self.package_metadata_folder, "logs"))
```

Note that "recipe" methods (those that are common for all binaries, like `export()` and `source()`) should use `self.recipe_metadata_folder`, while "package" specific methods (`build()`, `package()`) should use the `self.package_metadata_folder`.

Doing a `conan create` over this recipe, will create "metadata" folders in the Conan cache. We can have a look at those folders with:

```
$ conan create .
$ conan cache path pkg/0.1 --folder=metadata
# folder containing the recipe metadata
$ conan cache path pkg/0.1:package_id --folder=metadata
# folder containing the specific "package_id" binary metadata
```

It is also possible to use the "local flow" commands and get local "metadata" folders. If we want to do this, it is very recommended to use a `layout()` method like above to avoid cluttering the current folder. Then the local commands will allow to test and debug the functionality:

```
$ conan source .
# check local metadata/logs/src.log file
$ conan build .
# check local mybuild/metadata/logs/mylogs.txt file
```

NOTE: Note that the locally created metadata will not be exported to the Conan cache during the `conan export-pkg` command. Some metadata, as the one generated in `export()` method can be generated in the cache, as the `conan export-pkg` command calls that method, but the metadata inside the "build" folder will not be exported. If you want to add that metadata to the exported package, you can copy it after the `conan export-pkg` using the paths reported by `conan cache path`, as described below in the "Adding metadata with commands" section.

5.4.2 Creating metadata with hooks

If there is some common metadata across recipes, it is possible to capture it without modifying the recipes, using hooks. Let's say that we have a simpler recipe:

```
import os
from conan import ConanFile
from conan.tools.files import save, copy

class Pkg(ConanFile):
    name = "pkg"
    version = "0.1"
    no_copy_source = True

    def layout(self):
        self.folders.build = "mybuild"
        self.folders.generators = "mybuild/generators"
```

(continues on next page)

(continued from previous page)

```
    def source(self):
        save(self, "logs/src.log", "srclog!!")

    def build(self):
        save(self, "logs/mylogs.txt", "some logs!!!")
```

As we can see, this is not using the metadata folders at all. Let's define now the following hooks:

```
import os
from conan.tools.files import copy

def post_export(conanfile):
      conanfile.output.info("post_export")
      copy(conanfile, "*.log", src=conanfile.recipe_folder,
         dst=os.path.join(conanfile.recipe_metadata_folder, "logs"))

def post_source(conanfile):
      conanfile.output.info("post_source")
      copy(conanfile, "*", src=os.path.join(conanfile.source_folder, "logs"),
         dst=os.path.join(conanfile.recipe_metadata_folder, "logs"))

def post_build(conanfile):
      conanfile.output.info("post_build")
      copy(conanfile, "*", src=os.path.join(conanfile.build_folder, "logs"),
         dst=os.path.join(conanfile.package_metadata_folder, "logs"))
```

The usage of these hooks will have a very similar effect to the in-recipe approach: the metadata files will be created in the cache when `conan create` executes, and also locally for the `conan source` and `conan build` local flow.

5.4.3 Adding metadata with commands

Metadata files can be added or modified after the package has been created. To achieve this, using the `conan cache path` command will return the folders to do that operation, so copying, creating or modifying files in that location will achieve this.

```
$ conan create . --name=pkg --version=0.1
$ conan cache path pkg/0.1 --folder=metadata
# folder to put the metadata, initially empty if we didn't use hooks
# and the recipe didn't store any metadata. We can copy and put files
# in the folder
$ conan cache path pkg/0.1:package_id --folder=metadata
# same as above, for the package metadata, we can copy and put files in
# the returned folder
```

This metadata is added locally, in the Conan cache. If you want to update the server metadata, uploading it from the cache is necessary.

5.4.4 Uploading metadata

So far the metadata has been created locally, stored in the Conan cache. Uploading the metadata to the server is integrated with the existing `conan upload` command:

```
$ conan upload "*" -c -r=default
# Uploads recipes, packages and metadata to the "default" remote
...
pkg/0.1: Recipe metadata: 1 files
pkg/0.1:da39a3ee5e6b4b0d3255bfef95601890afd80709: Package metadata: 1 files
```

By default, `conan upload` will upload recipes and packages metadata when a recipe or a package is uploaded to the server. But there are some situations that Conan will completely avoid this upload, if it detects that the revisions do already exist in the server, it will not upload the recipes or the packages. If the metadata has been locally modified or added new files, we can force the upload explicitly with:

```
# We added some metadata to the packages in the cache
# But those packages already exist in the server
$ conan upload "*" -c -r=default --metadata="*"
...
pkg/0.1: Recipe metadata: 1 files
pkg/0.1:da39a3ee5e6b4b0d3255bfef95601890afd80709: Package metadata: 1 files
```

The `--metadata` argument allows to specify the metadata files that we are uploading. If we structure them in folders, we could specify `--metadata="logs*"` to upload only the logs metadata, but not other possible ones like `test` metadata.

```
# Upload only the logs metadata of the zlib/1.2.13 binaries
# This will upload the logs even if zlib/1.2.13 is already in the server
$ conan upload "zlib/1.2.13:*" -r=remote -c --metadata="logs/*"
# Multiple patterns are allowed:
$ conan upload "*" -r=remote -c --metadata="logs/*" --metadata="tests/*"
```

Sometimes it might be useful to upload packages without uploading the metadata, even if the metadata cache folders contain files. To ignore uploading the metadata, use an empty argument as metadata pattern:

```
# Upload only the packages, not the metadata
$ conan upload "*" -r=remote -c --metadata=""
```

The case of mixing `--metadata=""` with `--metadata="*"` is not allowed, and it will raise an error.

```
# Invalid command, it will raise an error
$ conan upload "*" -r=remote -c --metadata="" --metadata="logs/*"
ERROR: Empty string and patterns can not be mixed for metadata.
```

5.4.5 Downloading metadata

As described above, metadata is not downloaded by default. When packages are downloaded with a `conan install` or `conan create` fetching dependencies from the servers, the metadata from those servers will not be downloaded.

The way to recover the metadata from the server is to explicitly specify it with the `conan download` command:

```
# Get the metadata of the "pkg/0.1" package
$ conan download pkg/0.1 -r=default --metadata="*"
...
$ conan cache path pkg/0.1 --folder=metadata
# Inspect the recipe metadata in the returned folder
$ conan cache path pkg/0.1:package_id --folder=metadata
# Inspect the package metadata for binary "package_id"
```

The retrieval of the metadata is done with `download` per-package. If we want to download the metadata for a whole dependency graph, it is necessary to use "package-lists":

```
$ conan install . --format=json -r=remote > graph.json
$ conan list --graph=graph.json --format=json > pkglist.json
# the list will contain the "remote" origin of downloaded packages
$ conan download --list=pkglist.json --metadata="*" -r=remote
```

Note that the "package-list" will only contain associated to the "remote" origin the packages that were downloaded. If they were previously in the cache, then, they will not be listed under the "remote" origin and the metadata will not be downloaded. If you want to collect the dependencies metadata, recall to download it when the package is installed from the server. There are other possibilities, like a custom command that can automatically collect and download dependencies metadata from the servers.

Downloading metadata to an arbitrary folder

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Since Conan 2.32, the Python API provides two methods to fetch only the metadata of a recipe or package revision into an arbitrary folder, bypassing the Conan cache. This is useful for custom commands and CI workflows that need to inspect metadata (build logs, test outputs…) without changing the local cache state:

- *DownloadAPI.recipe_metadata*
- *DownloadAPI.package_metadata*

Both methods require a fully resolved reference (with recipe revision, and package revision for the package case), the target `Remote`, the metadata patterns to fetch, and the destination folder. The metadata files are written under `<folder>/metadata/`. This can be wrapped in a custom command as follows:

```
import os
from conan.api.model import PkgReference, RecipeReference
from conan.cli.command import conan_command


@conan_command(group="custom commands")
def metadata_download(conan_api, parser, *args):
```

(continues on next page)

(continued from previous page)

```
    """Download the metadata of a recipe/package revision into cwd (no cache)."""
    parser.add_argument("reference")
    parser.add_argument("-r", "--remote", required=True)
    a = parser.parse_args(*args)
    remote = conan_api.remotes.get(a.remote)
    if ":" in a.reference:
        pref = PkgReference.loads(a.reference)
        if pref.ref.revision is None:
            pref.ref = conan_api.list.latest_recipe_revision(pref.ref, remote)
        if pref.revision is None:
            pref = conan_api.list.latest_package_revision(pref, remote)
        conan_api.download.package_metadata(pref, remote, ["*"], os.getcwd())
    else:
        ref = RecipeReference.loads(a.reference)
        if ref.revision is None:
            ref = conan_api.list.latest_recipe_revision(ref, remote)
        conan_api.download.recipe_metadata(ref, remote, ["*"], os.getcwd())
```

5.4.6 Removing metadata

At the moment it is not possible to remove metadata from the server side using Conan, as the metadata are "additive", it is possible to add new data, but not to remove it (otherwise it would not be possible to add new metadata without downloading first all the previous metadata, and that can be quite inefficient and more error prone, specially sensitive to possible race conditions).

The recommendation to remove metadata from the server side would be to use the tools, web interface or APIs that the server might provide.

Note:

Best practices

- Metadata shouldn't be necessary for using packages. It should be possible to consume recipes and packages without downloading their metadata. If metadata is mandatory for a package to be used, then it is not metadata and should be packaged as headers and binaries.
- Metadata reading access should not be a frequent operation, or something that developers have to do. Metadata read is intended for exceptional cases, when some build logs need to be recovered for compliance, or some test executables might be needed for debugging or re-checking a crash.
- Conan does not do any compression or decompression of the metadata files. If there are a lot of metadata files, consider zipping them yourself, otherwise the upload of those many files can take a lot of time. If you need to handle different types of metadata (logs, tests, reports), zipping the files under each category might be better to be able to filter with the `--metadata=xxx` argument.

5.4.7 test_package as metadata

This is an illustrative example of usage of metadata, storing the full `test_package` folder as metadata to later recover it and execute it. Note that this is not necessarily intended for production.

Let's start with a hook that automatically stores as **recipe metadata** the `test_package` folder

```
import os
from conan.tools.files import copy

def post_export(conanfile):
   conanfile.output.info("Storing test_package")
   folder = os.path.join(conanfile.recipe_folder, "test_package")
   copy(conanfile, "*", src=folder,
         dst=os.path.join(conanfile.recipe_metadata_folder, "test_package"))
```

Note that this hook doesn't take into account that `test_package` can be dirty with tons of temporary build objects (it should be cleaned before being added to metadata), and it doesn't check that `test_package` might not exist at all and crash.

When a package is created and uploaded, it will upload to the server the recipe metadata containing the `test_package`:

```
$ conan create ...
$ conan upload "*" -c -r=default  # uploads metadata
...
pkg/0.1: Recipe metadata: 1 files
```

Let's remove the local copy, and assume that the package is installed, but the metadata is not:

```
$ conan remove "*" -c  # lets remove the local packages
$ conan install --requires=pkg/0.1 -r=default  # this will not download metadata
```

If at this stage the installed package is failing in our application, we could recover the `test_package`, downloading it, and copying it to our current folder:

```
$ conan download pkg/0.1 -r=default --metadata="test_package*"
$ conan cache path pkg/0.1 --folder=metadata
# copy the test_package folder from the cache, to the current folder
# like `cp -R ...`

# Execute the test_package
$ conan test metadata/test_package pkg/0.1
pkg/0.1 (test package): Running test()
```

See also:

- TODO: Examples how to collect the metadata of a complete dependency graph with some custom deployer or command

This is an **experimental** feature. We are looking forward to hearing your feedback, use cases and needs, to keep improving this feature. Please report it in Github issues

5.5 Versioning

This section deals with different versioning topics:

5.5.1 Default versioning approach

When doing changes to the source code of a package, and creating such a package, one good practice is to increase the version of the package to represent the scope and impact of those changes. The "semver" standard specification defines a `MAJOR.MINOR.PATCH` versioning approach with a specific meaning for changing each digit.

Conan implements versioning based on the "semver" specification, but with some extended capabilities that were demanded by the C and C++ ecosystems:

- Conan versions can have any number of digits, like `MAJOR.MINOR.PATCH.MICRO.SUBMICRO...`
- Conan versions can contain also letters, not only digits, and they are also ordered in alphabetical order, so `1.a.2` is older tha `1.b.1` for example.
- The version ranges can be equally defined for any number of digits, like `dependency/[>=1.0.0.0 <1.0.0.10]`

Read the *introduction to versioning* in the tutorial.

But one very different aspect of C and C++ building model compared to other languages is how the dependencies affect the binaries of the consumers requiring them. This is described in the *Conan binary model* reference.

Basically, when some package changes its version, this can have different effects on the "consumers" of this package, requiring such "consumers" to do a rebuild from source or not to integrate the new dependency changes. This also depends on the package types, as the logic changes when linking a shared library or a static library. Conan binary model with `dependency traits`, `package_type`, and the `package_id` modes is able to represent this logic and compute efficiently what needs to be rebuilt from source.

The default Conan behavior can give some hints of what version changes would be recommended when doing different changes to the packages source code:

- Not modifying the version typically means that we want Conan automatic **recipe revisions** to handle that. A common use case is when the C/C++ source code is not modified at all, and only changes to the `conanfile.py` recipe are done. As the source code is the same, we might want to keep the same version number, and just have a new revision of that version.
- **Patch**: Increasing the **patch** version of a package means that only internal changes were done, in practice it means change to files that are not public headers of the package. This "patch" version can avoid having to re-build consumers of this package, for example if the current package getting a new "patch" version is a static library, all other packages that implement static libraries that depend on this one do not need to be re-built from source, as depending on the same public interface headers guarantee the same binary.
- **Minor**: If changes are done to package public headers, in an API source compatible way, then the recommendation would be to increase the **minor** verson of a package. That means that other packages that depend on it will be able to compile without issues, but as there were modifications in public headers (that could contain C++ templates or other things that could be inlined in the consumer packages), then those consumer packages need to be rebuilt from source to incorporate these changes.
- **Major**: If API breaking changes are done to the package public headers, then increasing the **major** version is recommended. As the most common recommended version-range is something like `dependency/[>1.0 <2]`, where the next major is excluded, that means that publishing these new versions will not break existing consumers, because they will not be used at all by those consumers, because their version ranges will exclude them. It will be necessary to modify the consumers recipes and source code (to fix the API breaking changes) to be able to use the new major version.

Note that while this is close to the standard "semver" definition of version and version ranges, the C/C++ compilation model needs to introduce a new side effect, that of "needing to rebuild the consumers", following the logic explained above in the `embed` and `non_embed` cases.

This is just the default recommended versioning approach, but Conan allows to change these defaults, as it implements an extension of the "semver" standard that allows any number of digits, letters, etc, and it also allows to change the `package_id` modes to define how different versions of the dependencies affect the consumers binaries. See *how to customize the dependencies package_id modes*.

Note: Best practices

- It is not recommended to use other package reference fields, as the `user` and `channel` to represent changes in the source code, or other information like the git branch, as this becomes "viral" requiring changes in the `requires` of the consumers. Furthermore, they don't implement any logic in the build model with respect to which consumers need to be rebuilt.
- The recommended approach is to use versioning and multiple server repositories to host the different packages, so they don't interfere with other builds, read *the Continuous Integration tutorial* for more details.

5.5.2 Handling version ranges and pre-releases

When developing a package and using version ranges for defining our dependencies, there might come a time when a new version of a dependency gets a new pre-release version that we would like to test before it's released to have a change to validate the new version ahead of time.

At first glance, it could be expected that the new version matches our range if it intersect it, but *as described in the version ranges tutorial*, by default Conan does not match pre-release versions to ranges that don't specify it.

Conan provides the *global.conf* `core.version_ranges:resolve_prereleases` conf, a tri-state configuration option that controls if version ranges should resolve to pre-releases.

When _not_ set, the default behavior is to listen to the version range expression, and only match if the version range explicitly allows pre-release versions, like for `[>=1 <2, include_prerelease]`.

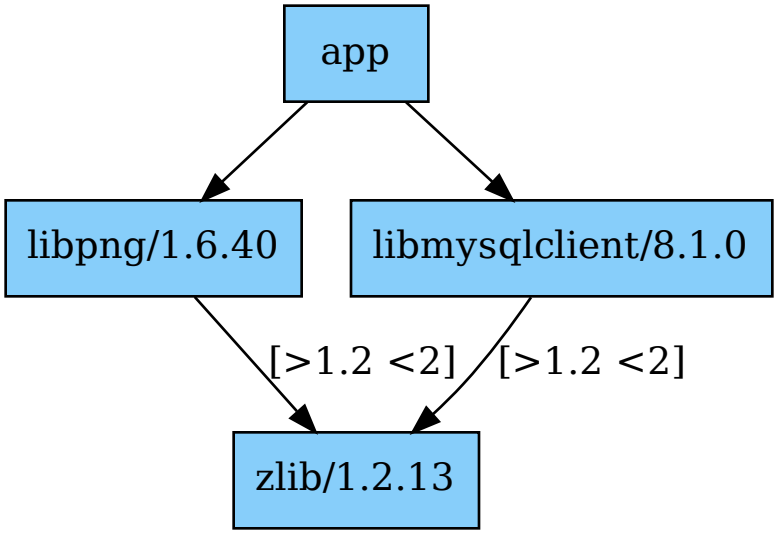
If the `include_prerelease` flag is not specified, pre-release versions are ignored in this case.

When set to `False`, it will not match pre-release versions, even if the version range expression allows it.

When set to `True`, it globally enables pre-release matching in version ranges, even if the version range expression does not explicitly allow it. This avoids having to modify and export the recipes of your dependency graph, which would become unfeasible for large ones.

This conf has the added benefit of affecting the whole dependency graph, so that if any of our dependencies also define a requirement to our library of interest, the new version will also be picked up by it.

Let's see this in action. Imagine we have the following (summarized) dependency graph, in which we depend on `libpng` and `libmysqlclient`, both of which depend on `zlib` via the `[>1.2 <2]` version range:



If `zlib/1.3-pre` is now published, using it is as easy as modifying your *global.conf* file and adding the line `core.version_ranges:resolve_prereleases=True` (or adding the `--core-conf core.version_ranges:resolve_prereleases=True` CLI argument to your command invocations), after which, running `conan create` will now output the expected prerelease version of `zlib` being used:

```
...

======== Computing dependency graph ========
Graph root
   cli
Requirements
   libmysqlclient/8.1.0#493d36bd9641e15993479706dea3c341 - Cache
   libpng/1.6.40#2ba025f1324ff820cf68c9e9c94b7772 - Cache
   lz4/1.9.4#b572cad582ca4d39c0fccb5185fbb691 - Cache
   openssl/3.1.2#f2eb8e67d3f5513e8a9b5e3b62d87ea1 - Cache
   zlib/1.3-pre#f2eb8e6ve24ff825bca32bea494b77dd - Cache
   zstd/1.5.5#54d99a44717a7ff82e9d37f9b6ff415c - Cache
Build requirements
   cmake/3.27.1#de7930d308bf5edde100f2b1624841d9 - Cache
Resolved version ranges
   cmake/[>=3.18 <4]: cmake/3.27.1
   openssl/[>=1.1 <4]: openssl/3.1.2
   zlib/[>1.2 <2]: zlib/1.3-pre
...
```

Now our package can be tested and validated against this new version, and the conf be afterwards removed once the testing is over to go back to the usual Conan behaviour.

5.6 Save and restore packages from/to the cache

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

With the `conan cache save` and `conan cache restore` commands, it is possible to create a .tgz from one or several packages from a Conan cache and later restore those packages into another Conan cache. There are some scenarios this can be useful:

- In Continuous Integration, specially if doing distributed builds, it might be very convenient to be able to move temporary packages recently built. Most CI systems have the capability of transferring files between jobs for this purpose. The Conan cache is not concurrent, sometimes for parallel jobs different caches have to be used.
- For air-gapped setups, in which packages can only be transferred via client side.
- Developers directly sharing some packages with other developers for testing or inspection.

The process of saving the packages is using the `conan cache save` command. It can use a pattern, like the `conan list` command, but it can also accept a package-list, like other commands like `remove, upload, download`. For example:

```
$ conan cache save "pkg/*:*"
Saving pkg/1.0: p/pkg1df6df1a3b33c
Saving pkg/1.0:9a4eb3c8701508aa9458b1a73d0633783ecc2270: p/b/pkgd573962ec2c90/p
Saving pkg/1.0:9a4eb3c8701508aa9458b1a73d0633783ecc2270 metadata: p/b/pkgd573962ec2c90/p
...
# creates conan_cache_save.tgz
```

The `conan_cache_save.tgz` file contains the packages named `pkg` (any version), the last recipe revision, and the last package revision of all the package binaries. The name of the file can be changed with the optional `--file=xxxx` argument. Some important considerations:

- The command saves the contents of the cache "recipe" folders, containing the subfolders "export", "export_sources", "source" and recipe "metadata".
- The "source" folder in the cache can be skipped with the `conan cache save --no-source` argument. That means that if the restored recipe needs to build a new binary in the restored cache, it will not have the sources and it will try to download them if the recipe `source()` method says so.
- The command saves the contents of the "package" and the package "metadata" folders, but not the binary "build" or "download", that are considered temporary folders.
- If the user doesn't want any of those folders to be saved, they can be cleaned before saving them with `conan cache clean` command
- The command saves the cache files and artifacts as well as the metadata (revisions, package_id) to be able to restore those packages in another cache. But it doesn't save any other cache state like `settings.yml`, `global.conf`, `remotes`, etc. If the saved packages require any other specific configuration, it should be managed with `conan config install`.

The compression format can be defined by the file extension, supported formats are `.tgz`, `.txz` (experimental) and `.tzst` (experimental, requires Python>=3.14). The compression level can be defined via the `core:compresslevel` configuration.

We can move this `conan_cache_save.tgz` file to another Conan cache and restore it as:

```
$ conan cache restore conan_cache_save.tgz
Restore: pkg/1.0 in p/pkg1df6df1a3b33c
Restore: pkg/1.0:9a4eb3c8701508aa9458b1a73d0633783ecc2270 in p/b/pkg773791b8c97aa/p
Restore: pkg/1.0:9a4eb3c8701508aa9458b1a73d0633783ecc2270 metadata in p/b/
↪pkg773791b8c97aa/d/metadata
...
```

The restore process will overwrite existing packages if they already exist in the cache.

Note: Best practices

- Saving and restoring packages is not a substitute for proper storage (upload) of packages in a Conan server repository. It is only intended as a transitory mechanism, in CI systems, to save an air-gap, etc., but not as a long-term storage and retrieval.
- Saving and restoring packages is not a substitute for proper backup of server repositories. The recommended way to implement long term backup of Conan packages is using some server side backup strategy.
- The storage format and serialization is not guaranteed at this moment to be future-proof and stable. It is expected to work in the same Conan version, but future Conan versions might break the storage format created with previous versions. (this is aligned with the above recommendation to not use it as a backup strategy)

5.7 Vendoring dependencies in Conan packages

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

From Conan 2.4 it is possible to create and use Conan packages that completely vendor their dependencies, that is, they completely hide and isolate their dependencies from their consumers. This can be useful in some different cases:

- When sharing Conan packages with other organizations which vendor (copy, embed or link) the dependencies, so it is not necessary for the consumers of their packages to have access to those dependencies and the intention is that they always use the shared precompiled binaries.
- To introduce a hard decoupling between parts of a project.

To make a package vendor its dependencies, define in its recipe the following attribute:

```
class MyPkg(ConanFile):
    name = "mypkg"
    version = "0.1"

    vendor = True

    requires = "somedep/1.2"
```

When we have this recipe, we can create its binaries with a normal `conan create .`. But when we use this package as a requirement for other packages, its dependencies will be fully invisible. The graph will not even expand the `somedep/1.2` requirement. This dependency doesn't even need to be available in the remotes for the consumers, it will not be checked.

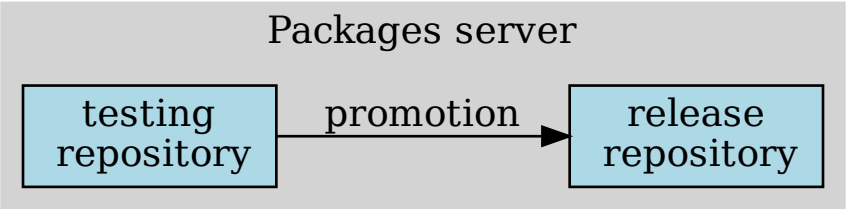
Some important notes:

- A package that vendors its dependencies is intended to be consumed always in binary form.
- The dependencies of a vendoring package always form a fully private and isolated dependency graph, decoupled from the rest of the dependency graph that uses this package.
- It is the responsibility of the vendoring package and its users to guarantee that vendored dependencies do not collide. If a vendoring package vendors for example `libssl.a` as a static library doing a regular copy of it in its package, and there is another package in the graph that also provides `libssl`, there will be a conflict that Conan cannot detect as `libssl.a` is vendored as an internal implementation detail of the package, but not explicitly modeled. Mechanisms like `provides` can be used for this purpose, but it is the responsibility of the recipe authors to take it into account.
- The `package_id` of a package that defines `vendor=True` is fully independent of its dependencies. The dependencies versions will never affect the `package_id` of the vendoring package, so it is important to note that the version of the vendoring package represents a full private dependency graph.
- The regular `default_options` or options values definitions from consumer `conanfile.py` recipes do not propagate over vendoring packages, as they don't even expand their dependencies.
- If a vendoring package binary is missing and/or the user request to build such a package from sources, Conan will fail, raising an error that it is not possible to build it.
- To allow the expansion of the private dependency the `tools.graph:vendor=build` configuration can be activated. If that is the case, the private dependency graph of the package will be computed and expanded and the package will be allowed to build.

5.8 Package promotions

Package promotions are the recommended devops practice to handle quality, maturity or stages of packages in different technologies, and of course, also for Conan packages.

The principle of package promotions is that there are multiple server package repositories defined and packages are uploaded and copied among repositories depending on the stage. For example we could have two different server package repositories called "testing" and "release":

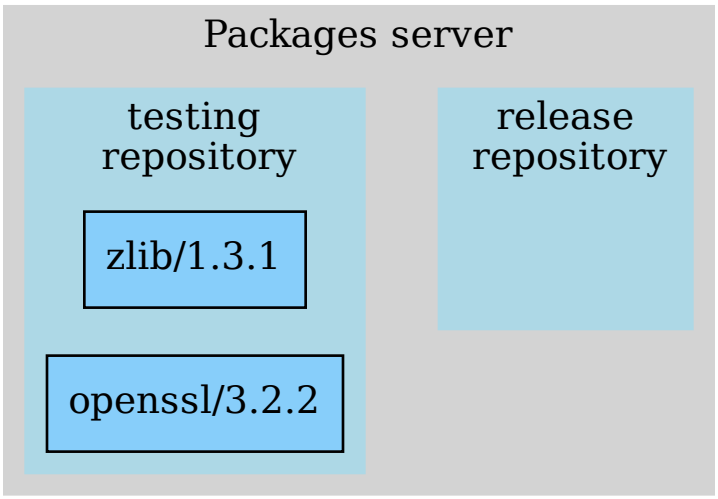


Note: Best practices

- Using different `user/channel` to try to denote maturity is strongly discouraged. It was described in the early Conan 1 days years ago, before the possibility of having multiple repositories, but it shouldn't be used anymore.

- Packages should be completely immutable across pipelines and stages, a package cannot rename or change its `user/channel`, and re-building it from source to have a new `user/channel` is also a strongly discourage devops practice.

Between those repositories there will be some quality gates. In our case, some packages will be put in the "testing" repository, for the QA team to test them, for example `zlib/1.3.1` and `openssl/3.2.2`:



When the QA team tests and approves these packages, they can be promoted to the "release" repository. Basically, a promotion is a copy of the packages, including all the artifacts and metadata from the "testing" to the "release" repository.

There are different ways to implement and execute a package promotion. Artifactory has some APIs that can be used to move individual files or folders. The Conan extensions repository contains the `conan art:promote` command that can be used to promote Conan "package lists" from one server repository to another repository.

If we have a package list `pkglist.json` that contains the above `zlib/1.3.1` and `openssl/3.2.2` packages, then the command would look like:

Listing 5: Promoting from testing->release

```
$ conan art:promote pkglist.json --from=testing --to=release --url=https://<url>/
↪artifactory --user=<user> --password=<password>
```

Note that the `conan art:promote` command doesn't work with ArtifactoryCE, Pro editions of Artifactory are needed. The promote functionality can be implemented in these cases with a simple download+upload flow:

Listing 6: Promoting from testing->release

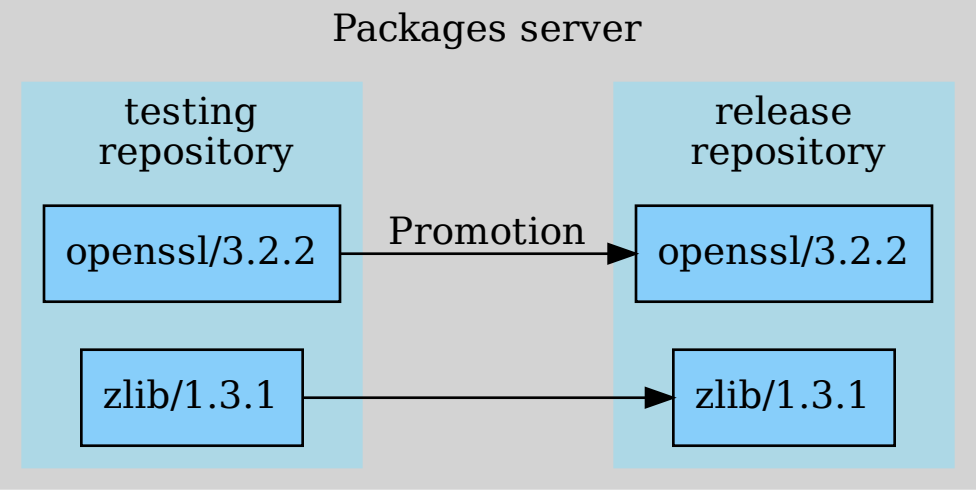
```
# Promotion using Conan download/upload commands
# (slow, can be improved with art:promote custom command)
$ conan download --list=promote.json -r=testing --format=json > downloaded.json
```

(continues on next page)

(continued from previous page)

```
$ conan upload --list=downloaded.json -r=release -c
```

After the promotion from "testing" to "release" repository, the packages would be like:



Note: Best practices

- In modern package servers such as Artifactory package artifacts are **deduplicated**, that is, they do not take any extra storage when they are copied in different locations, including different repositories. The **deduplication** is checksum based, so the system is also smart to avoid re-uploading existing artifacts. This is very important for the "promotions" mechanism: this mechanism is only copying some metadata, so it can be very fast and it is storage efficient. Pipelines can define as many repositories and promotions as necessary without concerns about storage costs.
- Promotions can also be done in JFrog platform with `Release Bundles`. The Conan extensions repository also contains one command to generate a release bundle (that can be promoted using the Artifatory API).

See also:

- *Using package lists examples*
- *Promotions usage in CI*

5.9 Checking package vulnerabilities

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `conan audit` command (introduced in Conan 2.14.0) is used to check for known vulnerabilities in your Conan packages.

By default, Conan provides access to a ConanCenter provider, which is a public provider that checks for vulnerabilities in ConanCenter packages, which uses JFrog Advanced Security to scan packages.

5.9.1 Requesting a token

To use the command, you will first need to register for the free service at https://audit.conan.io/register. After registering, you will receive an email with an activation link. Clicking this link will take you to a page where your personal access token is displayed.

Once you have your token, you can authenticate the `conancenter` provider with it:

```
$ conan audit provider auth conancenter --token=<your_token>
```

Note: Using `--token` in the command line may expose your token in the shell history. To prevent this, set it as an environment variable named after the provider in uppercase. For example, for *conancenter*, use: `CONAN_AUDIT_PROVIDER_TOKEN_CONANCENTER=<token>`.

5.9.2 Scanning packages

Once you have authenticated, you can check for vulnerabilities in your packages with the `conan audit scan` and `conan audit list` commands.

- `conan audit scan` will check for the vulnerabilities of the given package(s) and their dependencies.
- `conan audit list` will list the vulnerabilities of the given package(s) without checking their dependencies.

```
$ conan audit list openssl/1.1.1w

Requesting vulnerability info for: openssl/1.1.1w

******************
* openssl/1.1.1w *
******************

2 vulnerabilities found:

- CVE-2023-5678 (Severity: Medium, CVSS: 5.3)

  Issue summary: Generating excessively long X9.42 DH keys or checking
  excessively long X9.42 DH keys or parameters may be very slow.  Impact summary:
  Applications that use the functions DH_generate_key() to generate an X9.42 DH
  key may exper...
```

(continues on next page)

(continued from previous page)

```
  url: https://git.openssl.org/gitweb/?p=openssl.git;a=commitdiff;
↪h=db925ae2e65d0d925adef429afc37f75bd1c2017

- CVE-2024-0727 (Severity: Medium, CVSS: 5.5)

  Issue summary: Processing a maliciously formatted PKCS12 file may lead OpenSSL
  to crash leading to a potential Denial of Service attack  Impact summary:
  Applications loading files in the PKCS12 format from untrusted sources might
  terminate ...
  url: https://github.com/alexcrichton/openssl-src-rs/commit/
↪add20f73b6b42be7451af2e1044d4e0e778992b2

Total vulnerabilities found: 2


Summary:

- openssl/1.1.1w 2 vulnerabilities found

Vulnerability information provided by JFrog. Please check https://jfrog.com/advanced-
↪security/ for more information.
You can send questions and report issues about the returned vulnerabilities to conan-
↪research@jfrog.com.
```

To scan the entire dependency graph of a package, the simplest way is using the `conan audit scan` command and providing a path to your conanfile, just as you would do with other Conan commands such as `conan install`.

For example, for a project with a conanfile.txt:

```
[requires]
libpng/1.5.30
openssl/1.1.1w
```

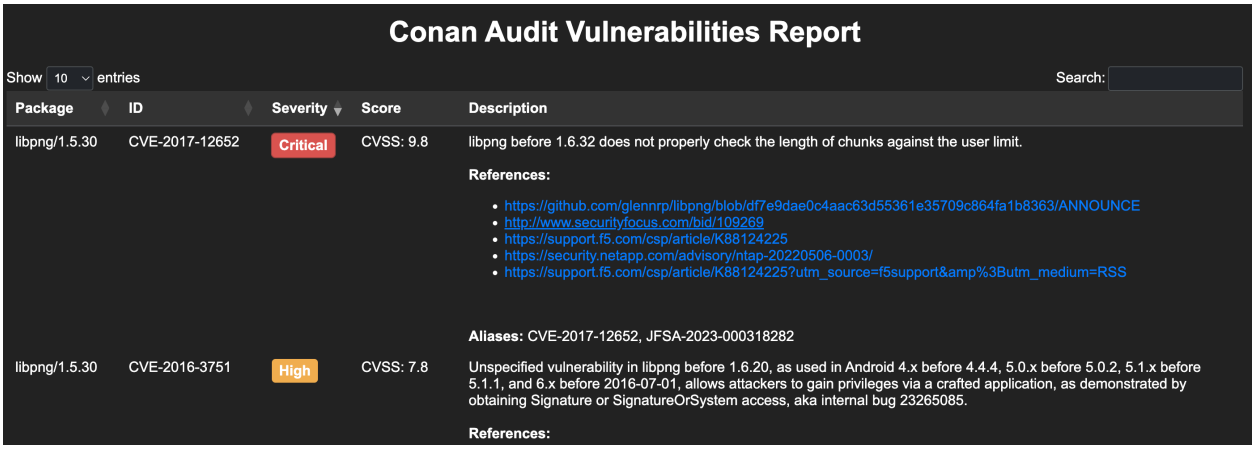
You can run:

```
$ conan audit scan .
```

Note that all of these commands support various output formats, such as JSON and HTML.

```
$ conan audit scan . -f=html > report.html
```

This generates an HTML report with the vulnerabilities found in the given package(s) and their dependencies, which will look something like:



The scan also has the threshold option `--severity-level`, which allows you to set a minimum severity level for the vulnerabilities. In case the threshold value is surpassed by any of the vulnerabilities found, the command will return a non-zero exit code. By default, it's set to 9.0 (Critical), but you can set it to a lower value to include lower severity vulnerabilities in the report. To disable the threshold, set it to 100.0.

```
$ conan audit scan . --severity-level=5.0
...
The package openssl/1.1.1w has a CVSS score 5.3 and exceeded the threshold severity␣
↪level 5.0.
```

5.9.3 Adding private providers

You can add your own private providers to the list of providers used by the `conan audit` subcommands. For now, only JFrog Advanced Security providers are supported.

Note: To use these private providers, your Artifactory license should include a subscription to JFrog Curation

---

To add a provider, the recommended way is to first create a specific user in Artifactory to use as the read-only user, which can be given no extra permissions. Then, after creating an access token for the user, you can add the provider with the following command:

```
$ conan audit provider add myprovider --type=private --url=https://your.artifactory.url -
↪-token=<your_token>
```

Note: Instead of using the `--token` argument in the command line, which may expose your token in the shell history, you can authenticate with the provider using an environment variable. Set the `CONAN_AUDIT_PROVIDER_TOKEN_<PROVIDER_NAME>` environment variable with the token value, replacing *<PROVIDER_NAME>* with the provider name in uppercase and using underscores (_) instead of hyphens (-).

For example, for *myprovider*, use: `CONAN_AUDIT_PROVIDER_TOKEN_MYPROVIDER=<token>`.

Note the `--type=private` argument, which specifies that the provider is a private provider, and that the supplied URL should be the base URL of the Artifactory instance.

You can now use the provider with the `conan audit scan` and `conan audit list` commands **without any limitation on the number of requests**, by specifying the provider name using the `-p` / `--provider` argument.

```
$ conan audit scan . -p=myprovider
```

See also:

- For detailed reference documentation on all `conan audit` subcommands and their options, consult the *conan audit command reference*.
- Read more in the dedicated blog post.
- Please check the *conan audit command reference* for other security related features.
- Check out our *security conference in using std::cpp 2025* for a deeper insight in package vulnerabilities.

5.10 Package compression format

Conan compresses different artifacts before uploading them to the servers for faster uploads and downloads, and lower storage needs. Specifically for C and C++ packages that could contain hundreds of different files, for example multiple header files, it is very inefficient to upload and store them one by one.

For this reason Conan creates some compressed artifacts like `conan_export.tgz`, `conan_sources.tgz` and `conan_package.tgz`, for the recipe extra files, the exported sources and the final package binary respectively. These are the files that are uploaded to the servers, together with the `conanfile.py`, the `conanmanifest.txt` and the `conaninfo.txt` files.

The compression happens when a locally created artifact is being uploaded with the `conan upload` command. If the recipe and package artifacts have been downloaded from the server, the compressed artifacts are cached and it is not necessary to compress them again. Furthermore, uploading them to a server that contains those artifacts will skip the actual upload transfer when the `revisions` match, or even avoid the transfer when uploading to a repository without the revision, but the file already exists in the server if the server has file de-duplication capabilities, like Artifactory.

These artifacts are automatically extracted when a package is downloaded or installed.

> **Warning:** The different compressed artifacts are an internal implementation detail, and it is not allowed to manipulate, change, remove or alter them.

Conan has traditionally used only the built-in `tgz` format to compress the artifacts, and allowed the `core.gzip:compresslevel` to select different compression levels (a tradeoff between speed and compression ratio).

From Conan 2.25 it is possible to (experimentally) select other compression formats that might be more efficient.

5.10.1 Using `xz` or `zstd` compression formats

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

From Conan 2.25 it is possible to choose between `gz`, `xz` and `zst` compression formats with the configuration: `core.upload:compression_format`. This configuration can be defined in the `global.conf` file. Recall that this file can also be distributed to all developers and CI machines easily with `conan config install/install-pkg`.

The compressed artifacts will be named after the compression format, with extensions such as `conan_package.txz`, `conan_package.tzst` or `conan_package.tgz`.

The `core:compresslevel` allows to select the compression level for the different algorithms. It supersedes the previous `core.gzip:compresslevel`.

Important: The `zstd` compression is using Python>=3.14 built-in features. It requires then Python>=3.14, both for compressing and uploading, and for consuming recipes and packages that were compressed with `zstd`. Conan will fail with an error message in both cases if Python<3.14.

Previous Conan versions (Conan<2.25), only understand `.tgz` artifacts and `gz` compression, and will fail to process artifacts compressed with other formats, Make sure that all your Conan clients have updated to >=2.25 before using this feature.

CHAPTER

SIX

SECURITY

Security is a critical aspect of many software development projects and products. Conan implements several security features to allow C and C++ developer and organizations to streamline security in their processes.

6.1 Scanning dependencies with `conan audit`

The `conan audit` commands provide a built-in way to **scan your dependencies for known CVEs**.

For a step-by-step guide on authentication, usage examples, output formats, and setting up private providers, see *Checking package vulnerabilities*. In short:

1. **Register** at audit.conan.io.
2. **Activate your account** via the confirmation email you receive.
3. **Save your token**, which is displayed on the page after activation.
4. **Configure Conan to use your token**:

```
conan audit provider auth conancenter --token=<token>
```

5. Run a scan:

```
# Check a specific reference
conan audit list zlib/1.2.13

# Scan the entire dependency graph
conan audit scan .  # Path to the conanfile.py/txt
```

This command also supports using your own JFrog Platform as a private provider for vulnerability scanning. See the *Adding private providers* section for more details.

6.1.1 Filtering queried packages

By default, the `conan audit scan` command will query all packages in the dependency graph. You can filter the packages to be queried based on their context using the `--context` option, which accepts `"host"`, or `"build"` as values, and when omitted, defaults to quering both contexts.

This allows you to skip checking for CVEs in build requirements, which are not part of the final product and therefore less relevant (but still important!) for vulnerability scanning.

It's also possible to perform this filter using the `conan audit list` command, by leveraging the packages list filtering from the `conan list` command. For example:

```
# Generate the dependency graph in JSON format
$ conan graph info . --format=json > graph.json
# Create a packages list for the resolved dependency graph, filtering to only contain␣
↪the `host` context packages
$ conan list --graph=graph.json --graph-context=host --format=json > pkglist.json
# Scan the filtered packages list for vulnerabilities
$ conan audit list --list=pkglist.json
```

See also:

- JFrog Academy Conan 2 Essentials Module 1, Lesson 7: Scanning C++ packages for Vulnerabilities using Conan Audit
- For detailed reference documentation on all `conan audit` subcommands and their options, consult the *conan audit command reference*.
- Read more in the dedicated blog post.
- Check out our *security conference in using std::cpp 2025* for a deeper insight of package vulnerabilities.

6.2 Software Bills of Materials (SBOM)

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

A Software Bill of Materials (SBOM) is a document that lists all the components, libraries, dependencies, and other elements that make up a specific piece of software. Similar to a bill of materials in manufacturing, which details the parts and materials used to build a product, an SBOM provides transparency about what is contained "inside" an application or software system.

Conan allows you to generate SBOMs natively by using a resolved dependency graph. This way, you can create the SBOM for your program at the same time you build it.

This feature only supports CycloneDX version 1.4 and 1.6. If you need a different standard, another version, or if you encounter any potential improvements, please feel free to open an issue on our GitHub . We would be delighted to hear your feedback!

6.2.1 CycloneDX

Conan supports CycloneDX out-of-the-box, which is one of the **most widely used standards** for SBOMs.

The CycloneDX tool is available in the `conan.tools.sbom` module. It provides the `cyclonedx_1_4` and `cyclonedx_1_6` functions which receives a `conanfile` and returns a dictionary with the SBOM data in the CycloneDX JSON format.

Using this feature is as simple as implementing a *hook* in your client which uses this tool to create the SBOM and stores it in the appropriate location.

Usage examples

Let's look at two examples:

In the first one, we want to generate the SBOM at the moment we create our app, after the package method. This is very **useful for keeping track of the components and dependencies** of that went into building our software. In the example, we save the generated sbom in the package metadata folder to keep our project organized

```
import json
import os
from conan.api.output import ConanOutput
from conan.tools.sbom import cyclonedx_1_6

def post_package(conanfile, **kwargs):
    sbom_cyclonedx_1_6 = cyclonedx_1_6(conanfile)
    metadata_folder = conanfile.package_metadata_folder
    file_name = "sbom.cdx.json"
    with open(os.path.join(metadata_folder, file_name), 'w') as f:
        json.dump(sbom_cyclonedx_1_6, f, indent=4)
    ConanOutput().success(f"CYCLONEDX CREATED - {conanfile.package_metadata_folder}")
```

See also:

- *See here for more information on the metadata feature*.

In the second example, we generate our SBOM after the generate method. This allows us to create the SBOMs when we install the dependencies from Conan. This can be very useful for generating SBOMs for different versions of our dependencies. Note that this time we're saving the SBOM in the generators folder, so that the user installing the dependencies has easy access to the SBOM.

```
import json
import os
from conan.api.output import ConanOutput
from conan.tools.sbom import cyclonedx_1_6

def post_generate(conanfile, **kwargs):
    sbom_cyclonedx_1_6 = cyclonedx_1_6(conanfile)
    generators_folder = conanfile.generators_folder
    file_name = "sbom.cdx.json"
    os.mkdir(os.path.join(generators_folder, "sbom"))
    with open(os.path.join(generators_folder, "sbom", file_name), 'w') as f:
        json.dump(sbom_cyclonedx_1_6, f, indent=4)
    ConanOutput().success(f"CYCLONEDX CREATED - {conanfile.generators_folder}")
```

Both hooks can coexist in such a way that we can generate the SBOMs for our application and our dependencies separately. This can greatly assist us in conducting continuous analysis of our development process and ensuring software quality.

See also:

- *SBOM tools*.

6.2.2 Generating a Conan-based SBOM

Instead of using a standard, we can take a "Conan-based approach". Thanks to the `conanfile.subgraph.serialize()` function, we can directly obtain information about the dependencies of our package. In the following example, we can see a hook that generates a simplified SBOM consisting of the serialization of the subgraph, which includes all data Conan has about the specific dependencies. Note that this serialization is **not a standard SBOM format**, and is not standardized in any way. The information is similar to the one provided by the **`conan graph info ... --format=json`** command.

```
import json
import os
from conan.api.output import ConanOutput

def post_package(conanfile, **kwargs):
    metadata_folder = conanfile.package_metadata_folder
    file_name = "sbom.conan.json"
    with open(os.path.join(metadata_folder, file_name), 'w') as f:
        json.dump(conanfile.subgraph.serialize(), f, indent=2)
    ConanOutput().success(f"CONAN SBOM CREATED - {conanfile.package_metadata_folder}")
```

6.2.3 Artifactory Build Info

With Conan, you also have the option to create a **"build info"**, which provides detailed information about the build generated in your **Artifactory**. It allows you to see, among other things, the history of versions, artifacts, modules, and dependencies that were necessary to create your build.

It is an SBOM focused on the process within Artifactory itself, making it perfect for maintaining traceability in the lifecycle of your binaries.

Unfortunately, it is not available natively in Conan, so it will be necessary to install the command from `conan-extensions`. You can find more information at the link below.

See also:

- *How to install the build info extension and how to generate your build info*.
- Check out our *security conference in using std::cpp 2025* to learn more about SBOM.

6.3 Security guidelines

This is an incomplete and preliminary, not exhaustive security related recommendations when using Conan:

- Avoid using tokens and passwords in URLs, they can easily be leaked in logs. For example, if the `source()` method of recipes implement a `git clone`, do not use git credentials in the URL, but instead use ssh-keys configured in the system. Same for downloading tarballs with `tools.get()` and `tools.download()`.
- In general, developers shouldn't have write permissions on servers, just read permissions to download and install packages. Only the CI should have write/upload permissions. As an exception, it might be possible to use some "playground" repository that developers use to share packages for debugging and testing purposes with other colleagues, but that "playground" repository should be isolated from the normal testing and production repositories.
- Use tokens with limited permissions in CI. If a job only needs read permissions, use a token with read-permissions only. Use write credentials exclusively in the "upload" parts of the CI pieplines.
- Enable dependencies vulnerability checking with `conan audit`, check *the conan audit docs*

- In many production cases, it is very recommended to fully own the SW lifecycle (SWLC) of the dependencies, including the third party dependencies. For this reason, the *Using ConanCenter packages in production environments* section recommends building your own binaries from source and storing those binaries in your own private server, without downloading packages directly from ConanCenter. The *local-recipes-index feature* was designed to help in this process.
- To avoid being disrupted by internet outages and possible tampering of tarballs downloaded from the internet, the *Backup sources* feature can be used.

6.3.1 Archive extraction and trust model

Conan extracts `tar` archives (`.tar`, `.tgz`, `.tar.gz`, `.txz`, `.tar.bz2`, etc.) in several places: the `unzip()` and `get()` tools in recipes, the package and recipe tarballs downloaded from remotes, and `conan cache restore`. In all of them, **Conan uses the Python `tarfile` extraction filter `fully_trusted` by default** (see the Python tarfile extraction filters docs). That means that the archive contents are extracted as they are, without the protections of the `data` filter: absolute paths, paths with `..` that escape the destination folder, links pointing outside the destination, device files, and special permission bits (setuid, setgid, sticky) are not rejected or sanitized. This is also the behavior of Python < 3.14, but Conan sets it explicitly, so it doesn't change when running Conan with Python >= 3.14, where `data` became the Python default.

The reasons for this default are:

- The data filter is not backwards compatible for binary packages. For example, it strips setuid/setgid/sticky bits, removes group/other write permissions and rejects some links. Changing the default could silently alter or break existing packages.
- The content being extracted by Conan is either C/C++ source code that is going to be compiled and executed, or binaries that are going to be executed. Extracting such content from an untrusted archive is dangerous regardless of the extraction filter, so the only model that makes sense is that **the archives are fully trusted**.

This implies that **a malicious or compromised archive can write files outside of the destination folder**, and it is the responsibility of the user to only consume archives from trusted sources:

- Only use Conan remotes that you trust, and limit who can upload to them (see the recommendations above).
- Do not run `conan cache restore` on archives from untrusted origins.
- In recipes, only use `get()` and `unzip()` with sources downloaded from trusted locations, and always provide a checksum (`sha256`) for downloaded tarballs. Consider the *Backup sources* feature.
- For tarballs from less trusted origins, the `data` filter can be requested with the `extract_filter="data"` argument in `unzip()` and `get()`, or globally with the `tools.files.unzip:filter=data` conf. Note that it applies only to these recipe helpers, and not to the extraction of Conan packages downloaded from remotes or to `conan cache restore`. It will not make it safe to compile or execute the extracted contents.

Issues that only rely on an archive being extracted with the `fully_trusted` filter are considered as part of this trust model, not as vulnerabilities.

6.4 C, C++ Compiler Sanitizers

Warning: Do not use sanitizers for production builds, especially for binaries with elevated privileges (e.g., SUID). Sanitizer runtimes rely on environment variables and can enable privilege escalation. Use only in development and testing.

Sanitizers are powerful runtime instrumentation tools that detect issues such as:

- Buffer overflows (stack/heap), use-after-free, double-free
- Data races in multithreaded code
- Memory leaks
- Use of uninitialized memory
- A wide range of undefined behaviors

Compilers such as GCC, Clang, and MSVC support sanitizers via compiler and linker flags.

This page explains recommended approaches for integrating compiler sanitizers into your workflow with Conan.

6.4.1 Compiler Sanitizer Support Comparison

Important: Always rebuild all dependencies when using MemorySanitizer (MSan) and generally for ThreadSanitizer (TSan) to avoid false positives/negatives. For AddressSanitizer, UBSan, and LeakSanitizer, mixing instrumented code with prebuilt libraries is typically safe, but may miss bugs inside those libraries.

Each compiler has different levels of support for various sanitizers, Clang being the most comprehensive so far. To help you choose the right sanitizer for your needs and compiler, here is a summary of the most common ones:

Sanitizer	GCC	Clang	MSVC	Notes
AddressSanitizer (ASan)	YES	YES	YES	MSVC: Supports x86, x64 and ARM64
ThreadSanitizer (TSan)	YES	YES	NO	Detects data races
MemorySanitizer (MSan)	NO	YES	NO	Clang-only, requires *-O1*
UndefinedBehaviorSanitizer (UBSan)	YES	YES	NO	Wide range of undefined behavior checks
LeakSanitizer (LSan)	YES	YES	NO	Often integrated with ASan
HardwareAddressSanitizer (HWASan)	NO	YES	NO	ARM64 only, lower overhead than ASan
KernelAddressSanitizer (KASan)	YES	YES	YES	MSVC: Requires Windows 11
DataFlowSanitizer (DFSan)	NO	YES	NO	Dynamic data flow analysis
Control Flow Integrity (CFI)	NO	YES	YES	MSVC: */guard:cf*

Besides MSVC having more limited support for sanitizers, it encourages the community to vote for new features at Developer Community. Very recently Visual Studio 2026 added support for AddressSanitizer on ARM64 architecture. This support should be straightforward when using Conan with MSVC.

Also, you can consider the typical use cases for each sanitizer:

- **AddressSanitizer (ASan)**: Great default for memory errors; often combined with UBSan for broader coverage.
- **ThreadSanitizer (TSan)**: Find data races in multithreaded code.

- **MemorySanitizer (MSan)**: Detects uninitialized memory reads (Clang-only). Requires all dependencies to be instrumented.
- **LeakSanitizer (LSan)**: Often included with ASan on Clang/GCC, can be enabled explicitly. Typically used to find memory leaks.
- **UndefinedBehaviorSanitizer (UBSan)**: Catches many undefined behaviors; often combined with ASan.

Common Sanitizer Combinations

The sanitizers can often be combined to provide more comprehensive coverage, but not all combinations are supported by every compiler. Here are some common combinations and their compatibility mostly used with GCC and Clang:

Combination	GCC	Clang	MSVC	Compatibility
ASan + UBSan	YES	YES	NO	Most common combination
TSan + UBSan	YES	YES	NO	Good for multithreaded code
ASan + LSan	YES	YES	NO	LSan often enabled by default with ASan
MSan + UBSan	NO	YES	NO	Requires careful dependency management

Notes on combinations:

- AddressSanitizer (ASan), ThreadSanitizer (TSan), and MemorySanitizer (MSan) **are mutually exclusive with one another**.
- MemorySanitizer often requires special flags such as `-O1`, `-fno-omit-frame-pointer` and fully-instrumented dependencies; mixing with non-instrumented code leads to crashes/false positives.

Compiler-Specific Flags

Each compiler requires specific flags to enable the desired sanitizers. Here is a summary of the most common sanitizers and their corresponding flags for GCC, Clang, and MSVC:

Sanitizer	GCC Flag	Clang Flag	MSVC Flag
AddressSanitizer	*-fsanitize=address*	*-fsanitize=address*	*/fsanitize=address*
ThreadSanitizer	*-fsanitize=thread*	*-fsanitize=thread*	N/A
MemorySanitizer	N/A	*-fsanitize=memory*	N/A
UndefinedBehavior	*-fsanitize=undefined*	*-fsanitize=undefined*	N/A
LeakSanitizer	*-fsanitize=leak*	*-fsanitize=leak*	N/A

It may seem like a large number of options, but for Clang, these are only a portion. To obtain the complete list, please refer to the official documentation for each compiler:

- Clang: AddressSanitizer, ThreadSanitizer, MemorySanitizer, UndefinedBehaviorSanitizer.
- GCC: Instrumentation Options.
- MSVC: MSVC Sanitizers.

6.4.2 Binary Compatibility

How sanitizers affect your binaries

Sanitizers instrument your code at compile time, adding runtime checks and metadata. This changes your binary's **Application Binary Interface (ABI)**, making instrumented code incompatible with non-instrumented code.

Key changes sanitizers make:

- **Memory layout**: Sanitizers add shadow memory, guard zones, or tracking metadata around your data
- **Function calls**: Standard library functions (`malloc`, `free`, etc.) are wrapped or intercepted
- **Runtime dependencies**: Instrumented code requires sanitizer runtime libraries (`libasan`, `libtsan`, etc.)
- **Linking**: Mixing instrumented and non-instrumented code can cause crashes, false positives, or undefined behavior

Handling external code

When using sanitizers, you must consider how to handle third-party dependencies. As mixing instrumented and non-instrumented code can lead to issues, here are some strategies:

Always require full instrumentation:

- **MemorySanitizer (MSan)**: Changes function ABIs to pass shadow state.
- **DataFlowSanitizer (DFSan)**: Explicitly modifies the ABI by appending label parameters to functions.
- **ThreadSanitizer (TSan)**: Changes memory layout and intercepts synchronization primitives. Some code may not be instrumented by ThreadSanitizer, but not recommended.

Usually require full instrumentation:

- **AddressSanitizer (ASan)**: Adds redzones and shadow memory; Works with non-instrumented code, but not recommended.
- **HardwareAddressSanitizer (HWASan)**: Similar to ASan but uses hardware tagging. Mixing is possible but not recommended.

Can often mix with non-instrumented code:

- **UndefinedBehaviorSanitizer (UBSan)**: Adds runtime checks for undefined behavior; Minimal ABI changes, safer to mix.
- **LeakSanitizer (LSan)**: Detects memory leaks at program exit; When standalone, has minimal ABI impact.

For reliable results, **always** rebuild your entire dependency tree with the same sanitizer configuration.

6.4.3 Enabling Sanitizers

Conan cannot infer sanitizer flags from settings automatically. You have to pass the appropriate compiler and linker flags (e.g., `-fsanitize=` or `/fsanitize=address`) via profiles or toolchains. Conan toolchains (e.g., `CMakeToolchain`, `MesonToolchain`) will propagate flags defined in `[conf]` sections.

Modeling and applying sanitizers using settings

If you want to model sanitizer options so that the package ID is affected by them, you can *customize new compiler sub-settings*. You should not need to modify `settings.yml` directly; instead add *the settings_user.yml*.

This approach is preferred because enabling a sanitizer alters the package ID, allowing you to build and use the same binary package with or without sanitizers. This is ideal for development and debugging workflows.

Configuring sanitizers as part of settings

If you typically use a specific set of sanitizers or combinations for your builds, you can specify a sub-setting as a list of values in your `settings_user.yml`. For example, for Clang, GCC and MSVC:

Listing 1: settings_user.yml

```
compiler:
  clang:
    sanitizer: [null, Address, Leak, Thread, Memory, UndefinedBehavior,␣
↪HardwareAssistanceAddress, KernelAddress, AddressUndefinedBehavior,␣
↪ThreadUndefinedBehavior]
  gcc:
    sanitizer: [null, Address, Leak, Thread, UndefinedBehavior, KernelAddress,␣
↪AddressUndefinedBehavior, ThreadUndefinedBehavior]
  msvc:
    sanitizer: [null, Address, KernelAddress]
```

This example defines a few common sanitizers. You can add any sanitizer your compiler supports. The `null` value represents a build without sanitizers. The above models for Clang the use of `-fsanitize=address`, `-fsanitize=thread`, `-fsanitize=memory`, `-fsanitize=leak`, `-fsanitize=undefined`, `-fsanitize=hwaddress`, `-fsanitize=kernel-address`, as well as combinations like `-fsanitize=address,undefined` and `-fsanitize=thread,undefined`.

As the `sanitizer` setting is a list, it can be choose by one single value at time. As an workaround to support mutiple sanitizers at same time, you can define combinations like `AddressUndefinedBehavior` and `ThreadUndefinedBehavior`, as listed above. There is no limitation on the number of combinations you can define, but keep in mind that these are only tags to help you manage your builds. You still need to pass the appropriate flags to the compiler and linker accordingly.

Adding sanitizers as part of the profile

Another option is to add the sanitizer values as part of a profile. This way, you can easily switch between different configurations by using dedicated profiles.

Listing 2: compiler_sanitizers/profiles/gcc_asan

```
[settings]
arch=x86_64
os=Linux
build_type=Debug
compiler=gcc
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=15
```

(continues on next page)

(continued from previous page)

```
compiler.sanitizer=Address

[conf]
tools.build:cflags+=["-fsanitize=address", "-fno-omit-frame-pointer"]
tools.build:cxxflags+=["-fsanitize=address", "-fno-omit-frame-pointer"]
tools.build:exelinkflags+=["-fsanitize=address"]
tools.build:sharedlinkflags+=["-fsanitize=address"]

[runenv]
ASAN_OPTIONS="halt_on_error=1:detect_leaks=1"
```

For Visual Studio (MSVC) we can obtain an equivalent profile for AddressSanitizer:

Listing 3: compiler_sanitizers/profiles/msvc_asan

```
[settings]
arch=x86_64
os=Windows
build_type=Debug
compiler=msvc
compiler.version=194
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.sanitizer=Address

[conf]
tools.build:cxxflags+=["/fsanitize=address", "/Zi"]
tools.build:exelinkflags+=["/fsanitize=address"]
```

The Conan client is not capable of deducing the necessary flags from the settings and applying them automatically during the build process. It is necessary to pass the expected sanitizer flags according to the `compiler.sanitizer` value as part of the compiler and linker flags. Conan's built-in toolchains (like `CMakeToolchain` and `MesonToolchain`) will automatically pick up the flags defined in the `[conf]` section and apply them to the build.

Managing sanitizers with a custom CMake toolchain

Besides using Conan profiles to manage sanitizer settings, you can also use other approaches.

If you already have a *custom CMake toolchain file* to manage compiler and build options, you can pass the necessary flags to enable sanitizers there instead of profiles.

Listing 4: cmake/my_toolchain.cmake

```
# Apply to all targets; consider per-target options for finer control
set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS} -fsanitize=address,undefined -fno-omit-frame-pointer
↪")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -fsanitize=address,undefined -fno-omit-frame-
↪pointer")
set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} -fsanitize=address,undefined")
set(CMAKE_SHARED_LINKER_FLAGS "${CMAKE_SHARED_LINKER_FLAGS} -fsanitize=address,undefined
↪")
```

Then, specify this toolchain file as part of your Conan profile:

Listing 5: profiles/gcc_asan_ubsan

```
[settings]
arch=x86_64
os=Linux
build_type=Debug
compiler=gcc
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=15
compiler.sanitizer=AddressUndefinedBehavior

[conf]
tools.cmake.cmaketoolchain:user_toolchain=["<path_to>/cmake/my_toolchain.cmake"]
```

This way, you can keep your existing CMake toolchain file and still leverage Conan profiles to manage other settings.

Note that this approach only works if all dependencies are built using CMake and the `CMakeToolchain` integration. If you have dependencies using other build systems (e.g., Meson, Autotools), those dependencies will not receive the sanitizer flags defined in your custom CMake toolchain file.

Managing sanitizers as a custom CMake Build Type

Another option, without using the custom `compiler.sanitizer` setting, is to define sanitizers as a custom CMake build type. In this approach, you select the sanitizer configuration by choosing the build type during the CMake configure step. To achieve this, you can create a custom CMake toolchain file that maps build types to sanitizer flags. For example:

Listing 6: cmake/sanitizer_toolchain.cmake

```
if(CMAKE_BUILD_TYPE STREQUAL "DebugASan")
    set(SANITIZER_FLAGS "-g -fsanitize=address -fno-omit-frame-pointer")
elseif(CMAKE_BUILD_TYPE STREQUAL "DebugUBSan")
    set(SANITIZER_FLAGS "-g -fsanitize=undefined -fno-omit-frame-pointer")
elseif(CMAKE_BUILD_TYPE STREQUAL "DebugASanUBSan")
    set(SANITIZER_FLAGS "-g -fsanitize=address,undefined -fno-omit-frame-pointer")
else()
    set(SANITIZER_FLAGS "")
endif()

set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS} ${SANITIZER_FLAGS}")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${SANITIZER_FLAGS}")
set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} ${SANITIZER_FLAGS}")
set(CMAKE_SHARED_LINKER_FLAGS "${CMAKE_SHARED_LINKER_FLAGS} ${SANITIZER_FLAGS}")
```

Also, you will need to update your `settings_user.yml` to include the new build types:

Listing 7: settings_user.yml

```
build_type: [DebugASan, DebugUBSan, DebugASanUBSan]
```

Then, in your Conan profile, specify this toolchain file:

Listing 8: profiles/gcc_asan_ubsan

```
[settings]
arch=x86_64
os=Linux
build_type=DebugASan
compiler=gcc
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=15

[conf]
tools.cmake.cmaketoolchain:user_toolchain=["<path_to>/sanitizer_toolchain.cmake"]
```

Be aware that this approach uses a custom build type, as a result, Conan will not automatically apply the standard flags associated with the Debug build type. Therefore, in your custom toolchain file, you need to also define the flags that correspond to the desired based build type, for example, that `-g` flag that matches the Debug `build_type` in gcc-like compilers.

Using this approach, you can easily switch between different sanitizer configurations and standard build types, preserving the package ID differentiation based on the build type.

6.4.4 Practical Usage Examples

For practical examples of using sanitizers with Conan, please refer to the *Building Examples Using Sanitizers* section in the examples.

6.4.5 Additional recommendations

- Debug info and optimization:
  - For ASan/TSan and when using GCC or Clang, the compiler flags `-O1` or `-O2` generally works; for MSan, prefer `-O1` and avoid aggressive inlining.
  - `-fno-omit-frame-pointer` helps stack traces.
- Runtime symbolization:

  Some sanitizers can be configured to provide better stack traces and error reports. These features can be enabled via environment variables, such as `ASAN_OPTIONS` and `UBSAN_OPTIONS` for compilers like GCC and Clang.
  - Useful settings for CI:
    - `ASAN_OPTIONS=halt_on_error=1:detect_leaks=1:log_path=asan`.

      This configuration makes AddressSanitizer stop execution on the first error, enables leak detection, and logs the output to a file named `asan`.
    - `UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1:log_path=ubsan`.

      This setup instructs UndefinedBehaviorSanitizer to print stack traces in a human-readable format, halt on the first error, and log the output to a file named `ubsan`.
- Suppressions:

  If certain known issues are not relevant for your testing, you can create suppression files to filter them out from the sanitizer reports.

  - For ASan: `ASAN_OPTIONS=suppressions=asan.supp`.

    Create a file named `asan.supp` with the following content:

    ```
    leak:FreeMyObject
    ```

    This example suppresses leak reports originating from `FreeMyObject`.

  - For UBSan: `UBSAN_OPTIONS=suppressions=ubsan.supp`.

    Create a file named `ubsan.supp` with the following content:

    ```
    signed-integer-overflow IncreaseCounter
    ```

    This example suppresses signed integer overflow reports from `IncreaseCounter`.

- Third-party dependencies:
  - Mixed instrumented/uninstrumented code can lead to false positives or crashes, especially with MSan.
  - Prefer building dependencies with the same sanitizer or limit sanitizers to leaf applications.
- MSVC and Windows notes:
  - ASan with MSVC/Clang-cl uses `/fsanitize=address` and PDBs via `/Zi`. Not supported for 32-bit targets.
  - KAsan requires Windows 11.
  - Some features are limited when using whole program optimization (`/GL`) or certain runtime libraries.

CHAPTER

SEVEN

INTEGRATIONS

Conan provides seamless integration with several platforms, build systems, and IDEs. Conan brings off-the-shelf support for some of the most important operating systems, including Windows, Linux, macOS, Android, and iOS. Some of the most important build systems supported by Conan include CMake, MSBuild, Meson, Autotools and Make. In addition to build systems, Conan also provides integration with popular IDEs, such as Visual Studio and Xcode.

7.1  CMake

Conan provides different tools to integrate with CMake in a transparent way. Using these tools, the consuming `CMakeLists.txt` file does not need to be aware of Conan at all. The CMake tools also provide better IDE integration via cmake-presets.

To learn how to integrate Conan with your current CMake project you can follow the *Conan tutorial* that uses CMake along all the sections.

Please also check the reference for the CMakeDeps, CMakeToolchain, and CMake tools:

- `CMakeDeps`: responsible for generating the CMake config files for all the required dependencies of a package.
- `CMakeConfigDeps`: A modern and better alternative to `CMakeDeps`, released in Conan 2.25 that has several improvements and fixes.
- `CMakeToolchain`: generates all the information needed for CMake to build the packages according to the information passed to Conan about things like the operating system, the compiler to use, architecture, etc. in a `conan_toolchain.cmake` toolchain file. It will also generate *cmake-presets* files for easy integration with some IDEs that support this CMake feature off-the-shelf.
- `CMake` build helper is the tool used by Conan `conanfile.py` recipes to run CMake and will pass all the arguments that CMake needs to build successfully, such as the toolchain file, build type file, and all the CMake definitions set in the recipe.

The `CMakeDeps` and `CMakeConfigDeps`, together with `CMakeToolchain` follow for the classic consumption flow described along the tutorial and many other sections in this documentation:

```
$ conan install ...
$ cmake --preset conan-xxxx
# or use the -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake
```

This flow is important, the `conan install` command generates CMake presets and `conan_toolchain.cmake` toolchain files that helps locating the dependencies, besides trying to align as best as possible with the profile information. This is the recommended flow for most cases.

In extraordinary and exceptional scenarios, it might be desired for the `CMake` execution to call `conan install` to simplify the flow, for example for some IDE integrations like the CLion one, so the users don't need to call `conan install` themselves.

For this purpose, the cmake-conan integration exists. It uses the CMake "dependency providers" feature to intercept the first `find_package()` and do a call to `conan install` to fetch the dependencies at that point.

This `cmake-conan` project stability is not guaranteed, and it has some known issues and limitations. Refer to the Github repository for more details. And note that calling `conan install` explicitly before calling `cmake` is still the preferred and most recommended flow for most cases.

See also:

- Check the *Building your project using CMakePresets* example
- Reference for *CMakeDeps*, *CMakeConfigDeps generator*, *CMakeToolchain* and *CMake build helper*
- *Conan tutorial*



7.2 CLion

7.2.1 Introduction

There's a plugin available in the JetBrains Marketplace that's compatible with CLion versions higher than *2022.3*. With this plugin, you can browse Conan packages available in Conan Center, add them to your project, and install them directly from the CLion IDE interface.

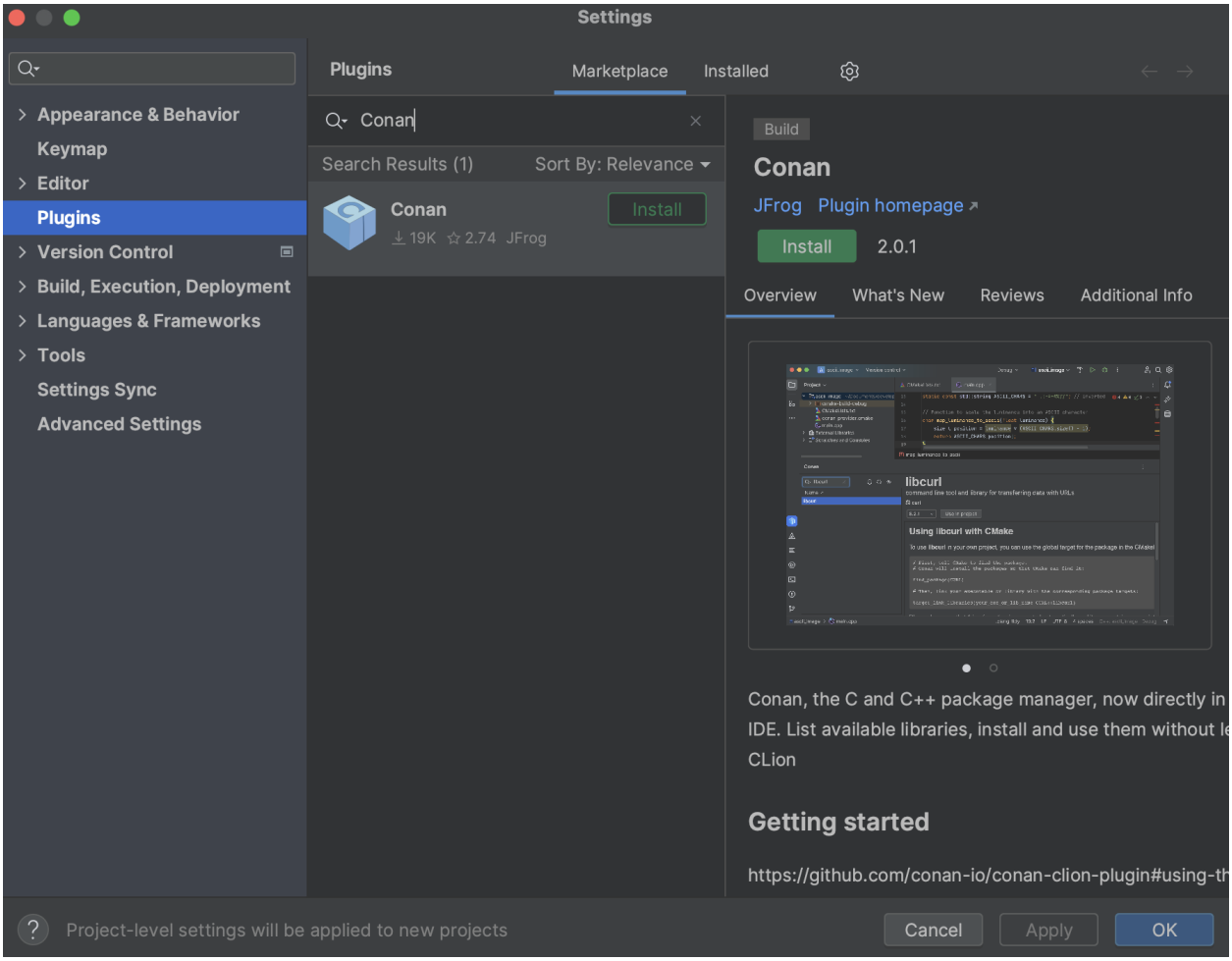
This plugin utilizes cmake-conan, a CMake dependency provider for Conan. It injects `conan_provider.cmake` using the `CMAKE_PROJECT_TOP_LEVEL_INCLUDES` definition. This dependency provider translates the CMake configuration to Conan. For instance, if you select a *Debug* profile in CLion, Conan will install and use the packages for *Debug*.

Bear in mind that *cmake-conan* activates the Conan integration every time CMake calls `find_package()`. This means that no library will be installed until the CMake configure step runs. At that point, Conan will attempt to install the required libraries and build them if necessary.

Also, note that dependency providers are a relatively new feature in CMake. Therefore, you will need CMake version >= 3.24 and Conan >= 2.0.5.

7.2.2 Installing the plugin

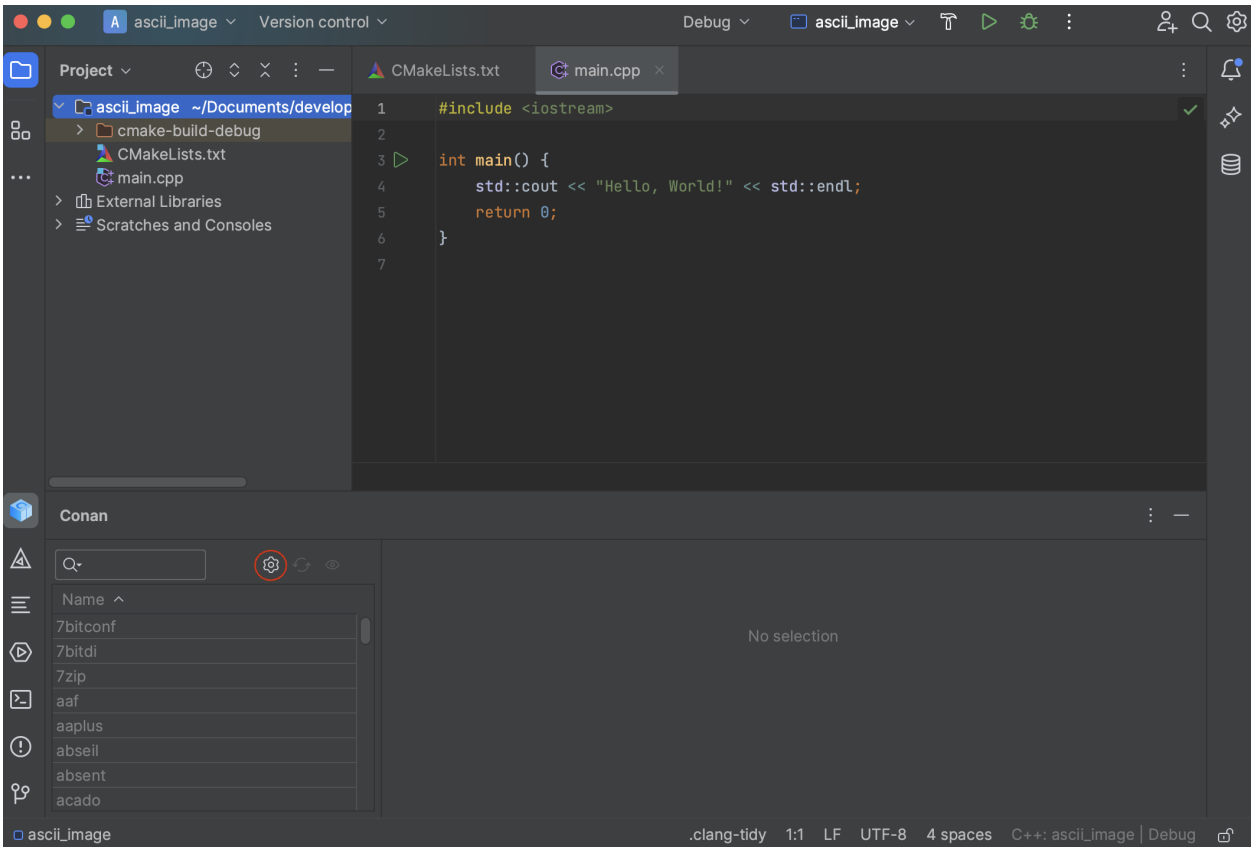
To install the new Conan CLion plugin, navigate to the JetBrains marketplace. Open CLion, go to *Settings* > *Plugins*, then select the *Marketplace* tab. Search for the Conan plugin and click on the Install button.



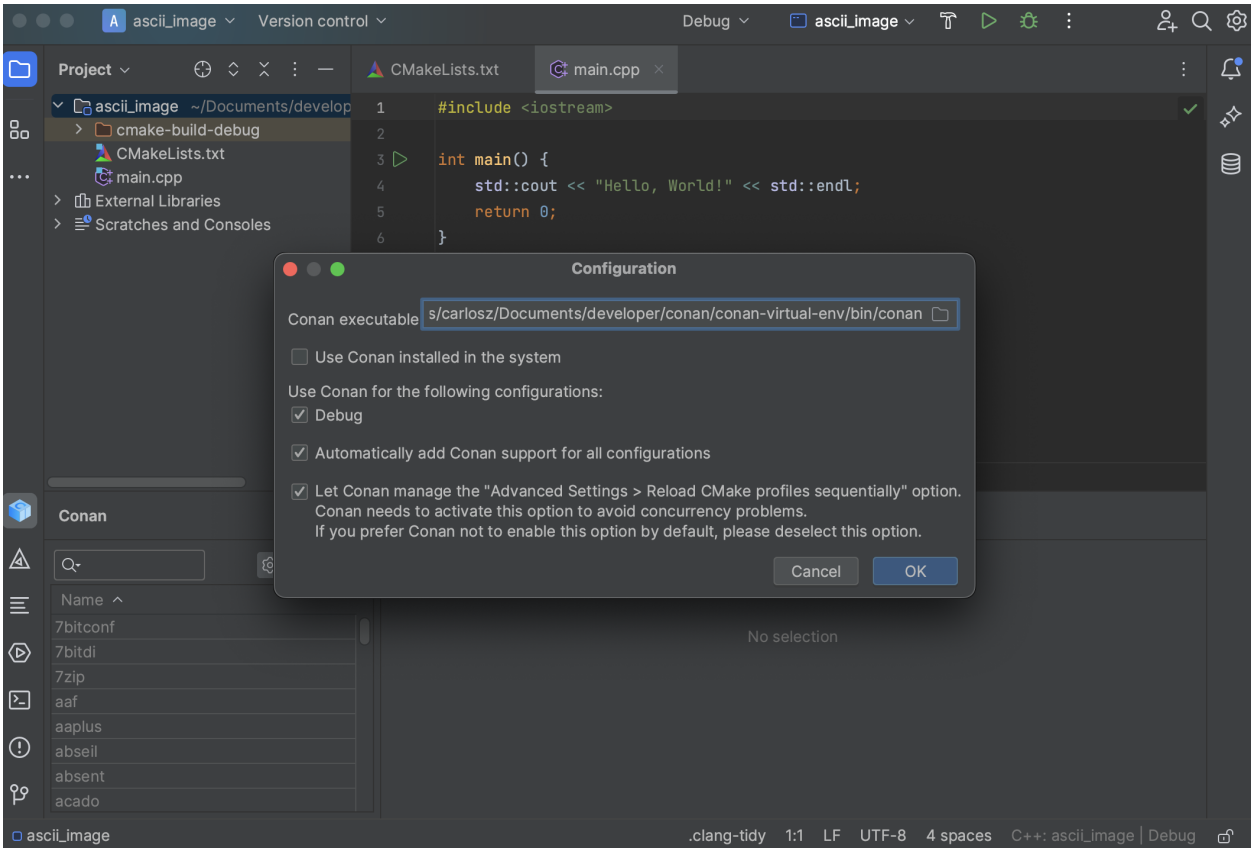
After restarting CLion, a new “Conan” tool tab will appear at the bottom of the IDE.

7.2.3 Configuring the plugin

Open a CMake project or create a new one in CLion. Then, go to the “Conan” tool tab at the bottom of the IDE. The only enabled action in the toolbar of the plugin will be the one with the “wheel” (configuration) symbol. Click on it.



The first thing you should do is configure the Conan client executable that will be used. You can point to a specific installation in an arbitrary location on your system, or you can select "Use Conan installed in the system" to use the system-level installation.



Several options are marked as default. Let's review them:

- You'll see checkboxes indicating which configurations Conan should manage. In our case, since we only have the Debug configuration, it's the only one checked. Below that, "Automatically add Conan support for all configurations" is checked by default. This means you don't need to manually add Conan support to new build configurations; the plugin will do it automatically.
- There's also a checkbox allowing Conan to modify the default CLion settings and run CMake sequentially instead of in parallel. This is necessary because the Conan cache isn't concurrent yet in Conan 2.

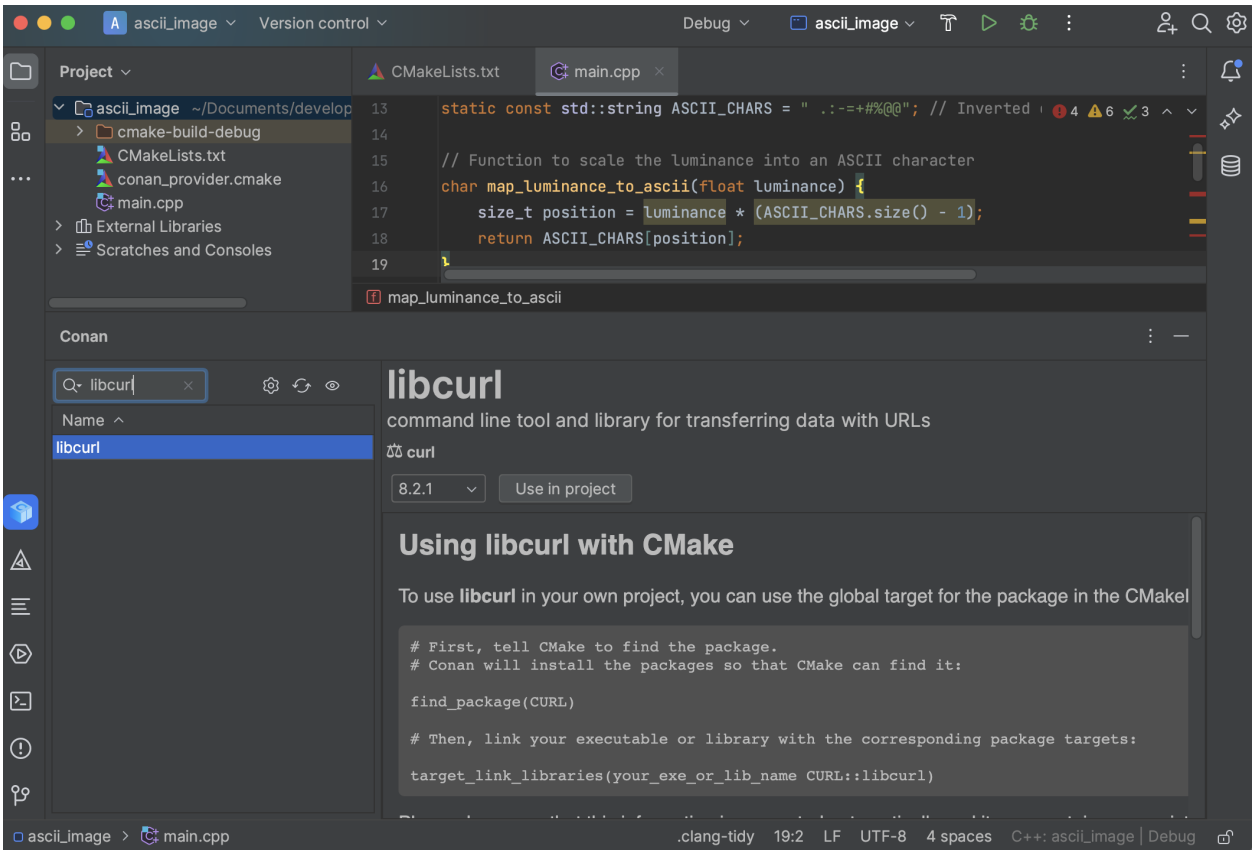
If you're using the Conan plugin, you typically wouldn't uncheck these options. After setting your preferences, click the OK button to finalize the configuration.

Note: At this point, CLion will run the configure step for CMake automatically. Since the plugin sets up the *conan.cmake* dependency provider, a warning will appear in the CMake output. This warning indicates that we haven't added a *find_package()* to our *CMakeLists.txt* yet. This warning will disappear once we add the necessary *find_package()* calls to the *CMakeLists.txt* file.

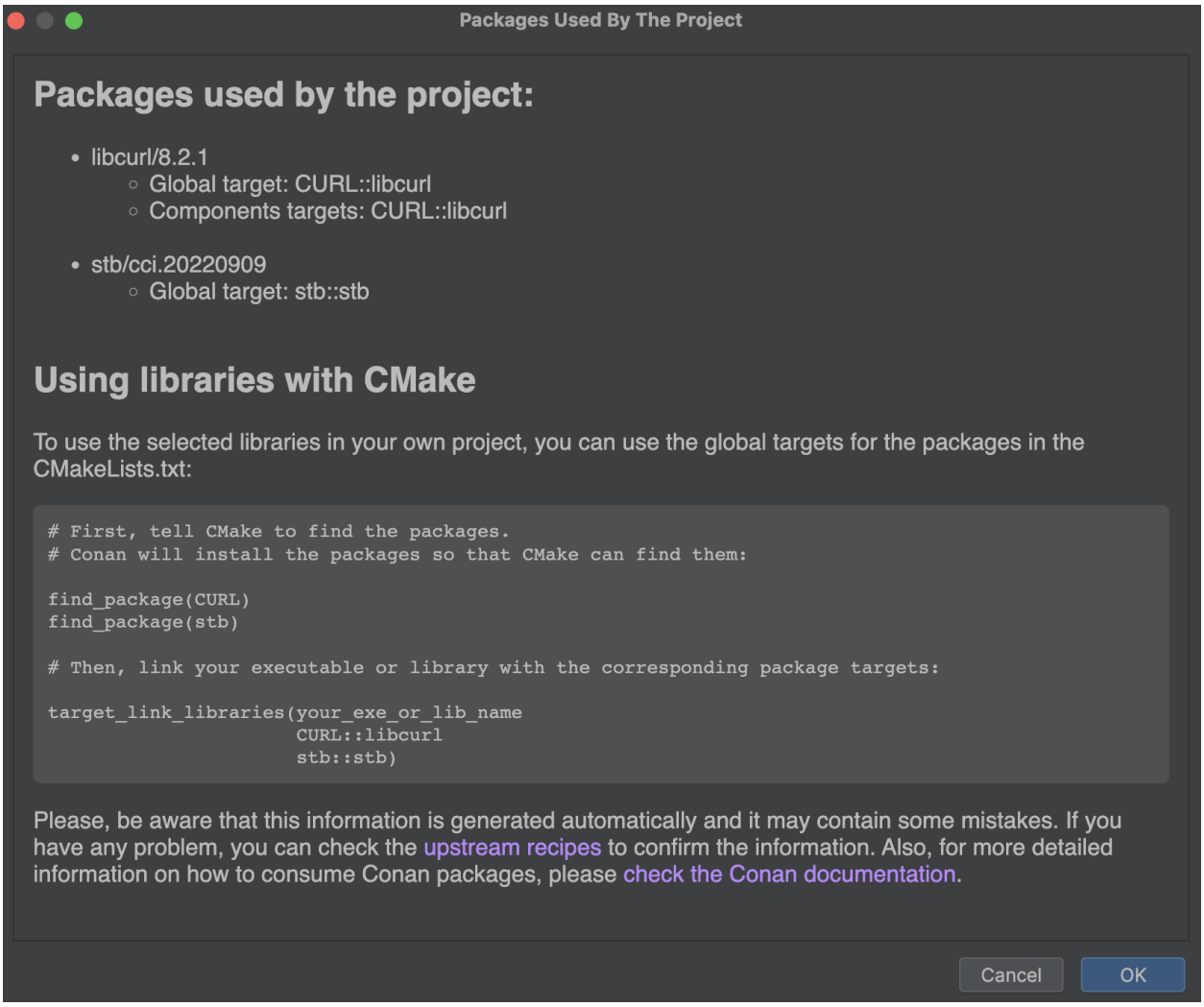
After the initial configuration, you'll notice that the list of libraries is enabled. The "update" and "inspect" buttons are also active. We'll explain these in detail later.

7.2.4 Using the plugin

With the plugin configured, you can browse available libraries and install them from CLion. For example, if you want to use libcurl to download an image from the Internet, navigate to the library list and search for *libcurl*. Information on how to add it to CMake will be displayed, along with a "Use in project" button. Select the version you want and click the button.



If you click on the "eye" (inspect) icon, you'll see all the libraries added to the project (assuming you added more than one). This view includes basic target information for CMake and the necessary code snippets to integrate them into CMake.



Conan stores information about the used packages in a *conandata.yml* file in your project folder. This file is read by a *conanfile.py*, which is also created during this process. You can customize these files for advanced plugin usage, but ensure you read the information in the corresponding files to do this correctly. Modify your *CMakeLists.txt* according to the instructions, which should look something like this:

```
cmake_minimum_required(VERSION 3.15) project(project_name) set(CMAKE_CXX_STANDARD 17)
find_package(CURL) add_executable(project_name main.cpp)
target_link_libraries(project_name CURL::libcurl)
```

After reloading the CMake project, you should see Conan installing the libraries in the CMake output tab.

See also:

- For more details, check the entry in the Conan blog about the plugin.



7.3 Visual Studio

7.3.1 Recipe tools for Visual Studio

Conan provides several tools to help manage your projects using Microsoft Visual Studio. These tools can be imported from `conan.tools.microsoft` and allow for native integration with Microsoft Visual Studio, without the need to use CMake and instead directly using Visual Studio solutions, projects, and property files. The most relevant tools are:

- *MSBuildDeps*: the dependency information generator for Microsoft MSBuild build system. It will generate multiple `xxxx.props` properties files, one per dependency of a package, to be used by consumers using MSBuild or Visual Studio, just by adding the generated properties files to the solution and projects.
- *MSBuildToolchain*: the toolchain generator for MSBuild. It will generate MSBuild properties files that can be added to the Visual Studio solution projects. This generator translates the current package configuration, settings, and options, into MSBuild properties files syntax.
- *MSBuild* build helper is a wrapper around the command line invocation of MSBuild. It will abstract the calls like `msbuild "MyProject.sln" /p:Configuration=<conf> /p:Platform=<platform>` into Python method calls.

For the full list of tools under `conan.tools.microsoft` please check the *reference* section.

7.3.2 Conan extension for Visual Studio

There's an extension available in the VisualStudio Marketplace that's compatible beginning from Visual Studio version *2022*. With this extension, you can browse Conan packages available in Conan Center, add them to your project, and they will be automatically installed before building your projects.

Note: The Visual Studio extension is only compatible with C/C++ projects based on MSBuild. It will not work with CMake-based projects or projects using other technologies. For CMake-based projects, please refer to the cmake-conan dependency provider.

Installation

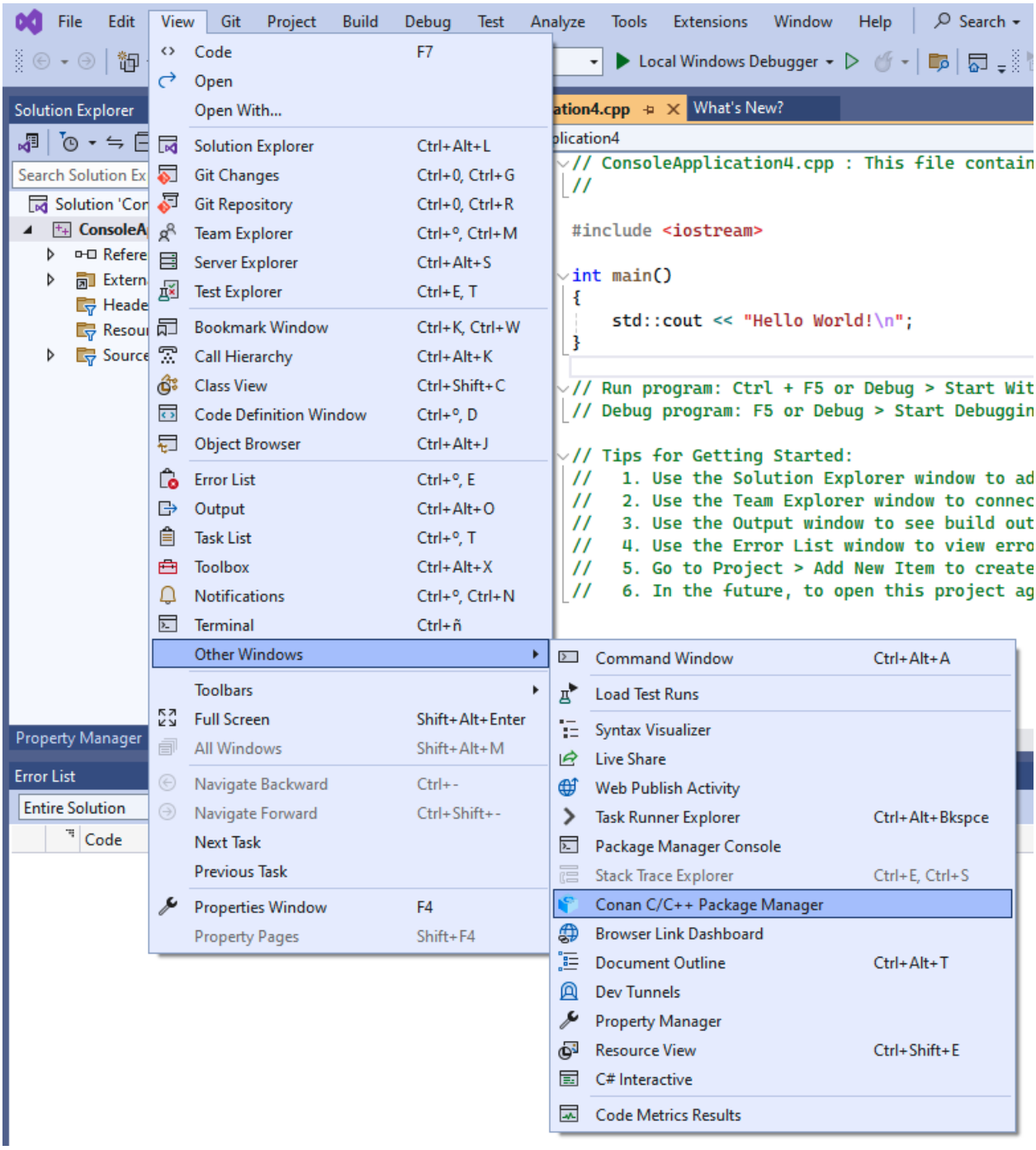
The Conan Visual Studio Extension can be installed directly from within Visual Studio:

- Open the Extensions menu.
- Select Manage Extensions.
- Search for "Conan" in the Online marketplace.
- Download and install the extension.

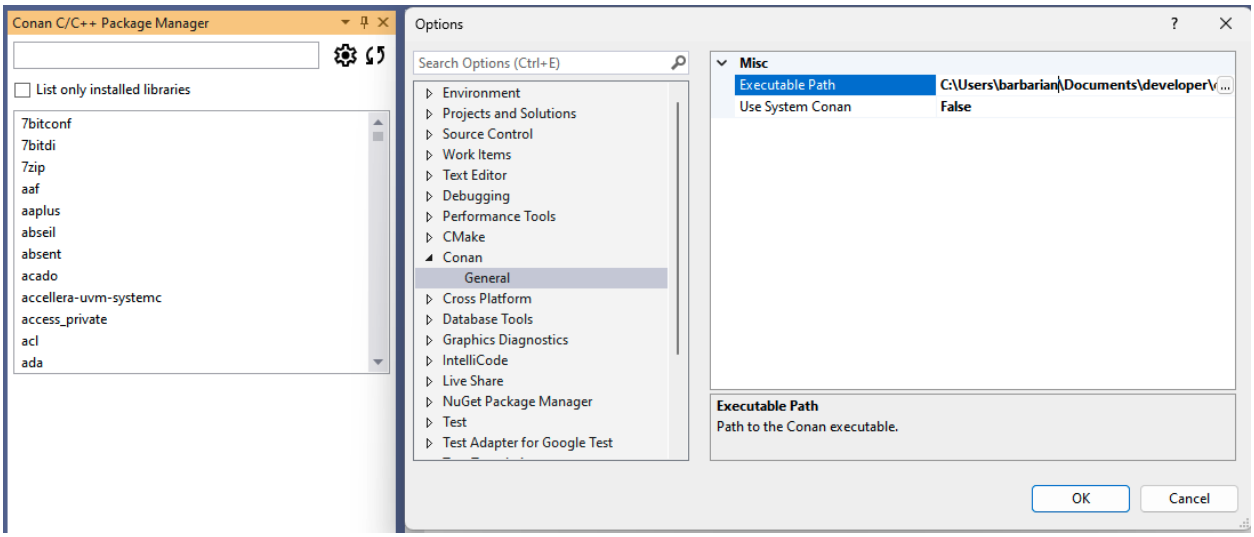
Alternatively, you can download the latest release from our releases page and install it manually.

Initial Configuration

After installing the Conan extension, you can access it from the "Conan" tool window in Visual Studio. To do so, go to **View > Other Windows > Conan C/C++ Package Manager**.



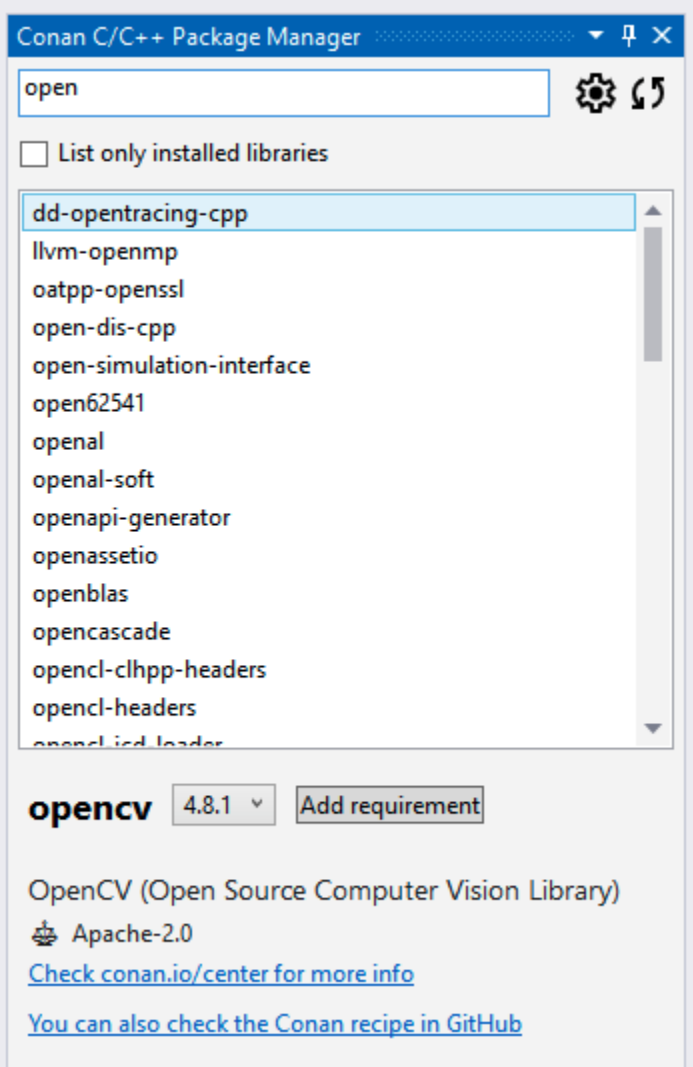
Initially, you will need to configure the Conan executable to be used by the extension. By clicking on the *configure* button (gear icon) from the extension's window, you can set up the path to the Conan client executable. You can either specify a custom path or choose to use the Conan client installed at the system level.



Once you have configured the Conan client, the extension is ready to use, and you can start adding libraries to your project.

Searching and Adding Libraries

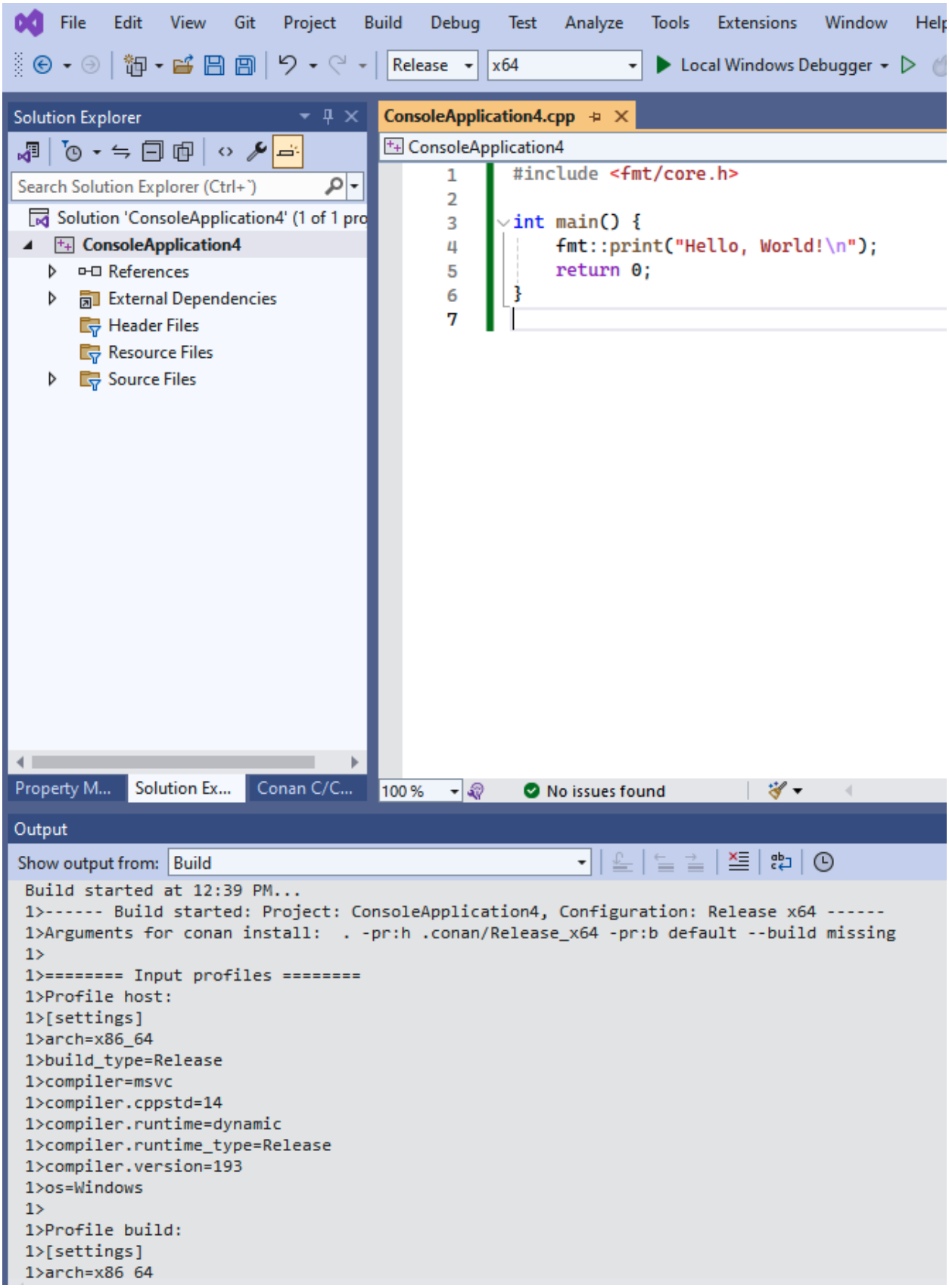
Once configured, the library list in the Conan tool window becomes active, and you can search for Conan packages using the search bar. Selecting a library will allow you to view its details, including available versions and integration options.



If you now click the *Add requirement* button, the extension will add a *conanfile.py* and a *conandata.yml* to your project with the necessary information to install the selected Conan packages. It will also add a prebuild event to the project to install those libraries on the next compilation of the project.

At any point, you can also use the *refresh* button (circular arrow icon) to update the list of available packages in Conan Center.

Now, if you initiate the build process for your project, the pre-build event will trigger Conan to install the packages and inject the necessary properties into the project, enabling Visual Studio to compile and link against those libraries.


File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help
Release
x64
Local Windows Debugger
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'ConsoleApplication4' (1 of 1 pro
ConsoleApplication4
References
External Dependencies
Header Files
Resource Files
Source Files
Property M...
Solution Ex...
Conan C/C...
ConsoleApplication4.cpp
ConsoleApplication4
#include <fmt/core.h>
int main() {
fmt::print("Hello, World!\n");
return 0;
}
100 %
No issues found
Output
Show output from: Build
Build started at 12:39 PM...
1>------ Build started: Project: ConsoleApplication4, Configuration: Release x64 ------
1>Arguments for conan install: . -pr:h .conan/Release_x64 -pr:b default --build missing
1>
1>======== Input profiles ========
1>Profile host:
1>[settings]
1>arch=x86_64
1>build_type=Release
1>compiler=msvc
1>compiler.cppstd=14
1>compiler.runtime=dynamic
1>compiler.runtime_type=Release
1>compiler.version=193
1>os=Windows
1>
1>Profile build:
1>[settings]
1>arch=x86_64

Warning: The initial compilation might fail if Visual Studio does not have sufficient time to process the injected properties. If this happens, simply rebuild the project, and it should build successfully.

For a more in-depth introduction to the Conan Visual Studio extension with a practical example, please check this example in Conan blog.

See also:

- Reference for *MSBuildDeps*, *MSBuildToolchain* and *MSBuild*.
- *CLion Conan plugin*.



7.4 Autotools

Conan provides different tools to help manage your projects using Autotools. They can be imported from `conan.tools.gnu`. The most relevant tools are:

- *AutotoolsDeps*: the dependencies generator for Autotools, which generates shell scripts containing environment variable definitions that the Autotools build system can understand.
- *AutotoolsToolchain*: the toolchain generator for Autotools, which generates shell scripts containing environment variable definitions that the Autotools build system can understand.
- *Autotools* build helper, a wrapper around the command line invocation of autotools that abstracts calls like *./configure* or *make* into Python method calls.
- *PkgConfigDeps*: the dependencies generator for *pkg-config* which generates *pkg-config* files for all the required dependencies of a package.

For the full list of tools under `conan.tools.gnu` please check the *reference* section.

See also:

- Reference for *AutotoolsDeps*, *AutotoolsToolchain*, *Autotools* and *PkgConfigDeps*.



7.5 Bazel

Conan provides different tools to help manage your projects using Bazel. They can be imported from `conan.tools.google`. The most relevant tools are:

- `BazelDeps`: the dependencies generator for Bazel, which generates a *[DEPENDENCY]/BUILD.bazel* file for each dependency and a *conan_deps.MODULE.bazel* file that loads all of them. Include it from your *MODULE.bazel* file (Bazel >= 7.2). The Bazel 6.x *dependencies.bzl* file loaded from a *WORKSPACE* is still generated, but it is deprecated since Conan 2.34.
- `BazelToolchain`: the toolchain generator for Bazel, which generates a `conan_bzl.rc` file that contains a build configuration `conan-config` to inject all the parameters into the **`bazel build`** command.
- `Bazel`: the Bazel build helper. It's simply a wrapper around the command line invocation of Bazel.

See also:

- Reference for *BazelDeps*.
- Reference for *BazelToolchain*.
- Reference for *Bazel*.
- *Build a simple Bazel project using Conan*
- *Build a simple Bazel 7.x project using Conan*



7.6 Makefile

Conan provides different tools to help manage your projects using Make. They can be imported from `conan.tools.gnu`. Besides the most popular variant, GNU Make, Conan also supports other variants like BSD Make. The most relevant tools are:

- *MakeDeps*: the dependencies generator for Make, which generates a Makefile containing definitions that the Make build tool can understand.

Currently, there is no `MakeToolchain` generator, it should be added in the future.

For the full list of tools under `conan.tools.gnu` please check the *reference* section.

See also:

- Reference for *MakeDeps*.



7.7 Xcode

Conan provides different tools to integrate with Xcode IDE, providing all the necessary information about the dependencies, build options and also to build projects created with Xcode in recipes. They can be imported from `conan.tools.apple`. The most relevant tools are:

- *XcodeDeps*: the dependency information generator for Xcode. It will generate multiple *.xcconfig* configuration files, that can be used by consumers using xcodebuild in the command line or adding them to the Xcode IDE.
- *XcodeToolchain*: the toolchain generator for Xcode. It will generate .xcconfig configuration files that can be added to Xcode projects. This generator translates the current package configuration, settings, and options, into Xcode .xcconfig files syntax.
- *XcodeBuild* build helper is a wrapper around the command line invocation of Xcode. It will abstract the calls like `xcodebuild -project app.xcodeproj -configuration <config> -arch <arch> ...`

For the full list of tools under `conan.tools.apple` please check the *reference* section.

See also:

- Reference for *XcodeDeps*, *XcodeToolchain* and *XcodeBuild build helper*



7.8 Meson

Conan provides different tools to help manage your projects using Meson. They can be imported from `conan.tools.meson`. The most relevant tools are:

- *MesonToolchain*: generates the .ini files for Meson with the definitions of all the Meson properties related to the Conan options and settings for the current package, platform, etc. MesonToolchain normally works together with *PkgConfigDeps* to manage all the dependencies.
- *Meson* build helper, a wrapper around the command line invocation of Meson.

See also:

- Reference for *MesonToolchain* and *Meson*.
- Build a simple Meson project using Conan *example*

Build a simple Meson project using Conan



7.9 Emscripten

Conan provides support for cross-building for both asm.js and WASM (Web Assembly) targets using Emscripten. This enables developers to compile C/C++ code for the browser and other JavaScript environments.

For detailed examples and step-by-step instructions, refer to:

- *Cross-building with Emscripten*

7.10 Premake

Conan provides different tools to help manage your projects using Premake. They can be imported from `conan.tools.premake`. The most relevant tools are:

- `PremakeDeps`: the dependencies generator for Premake, to allow consuming dependencies from Premake projects.
- `PremakeToolchain`: the toolchain generator for Premake. It will create a wrapper over premake scripts allowing premake workspace and projects customization.
- `Premake`: the Premake build helper. It's simply a wrapper around the command line invocation of Premake.

See also:

- Reference for *PremakeDeps*.
- Reference for *PremakeToolchain*.
- Reference for *Premake*.



7.11 android Android

Conan provides support for cross-building for Android, and it's easy to integrate with Android Studio. Please check these examples for more information on how to build your binaries for Android:

- *Cross building to Android with the NDK*
- *Integrating Conan in Android Studio*



7.12 JFrog

7.12.1 Artifactory Build Info

Warning: The support of Artifactory Build Info via extension commands is not covered by *the Conan stability commitment*.

The Artifactory build info is a recollection of the metadata of a build. This json-formatted file includes all the details about the build broken down into segments like version history, artifacts, project modules, dependencies, and everything that was required to create the build.

Build infos are identified with a `build name` and a `build number`, similar to how many CI services identify the builds. They are conveniently stored in Artifactory to keep track of the build metadata to later perform different operations.

Conan does not offer built-in support for the build info format. However, we have developed some *custom commands* at at the extensions repository using the feature, that provides support to create and manage the build info files.

How to install the build info extension commands

Using the dedicated repository for Conan extensions https://github.com/conan-io/conan-extensions, it is as easy as:

```
$ conan config install https://github.com/conan-io/conan-extensions.git -sf=extensions/
↪commands/art -tf=extensions/commands/art
```

Generating a Build Info

A Build Info can be generated from a create or install command:

```
$ conan create . --format json -s build_type=Release > create_release.json
```

Then upload the created package to your repository:

```
$ conan upload ... -c -r ...
```

Now, using the JSON output from the create/install commands, a build info file can be generated:

```
$ conan art:build-info create create_release.json mybuildname_release 1 <repo> --server␣
↪my_artifactory --with-dependencies > mybuildname_release.json
```

And then uploaded to Artifactory:

```
$ conan art:build-info upload mybuildname_aggregated.json --server my_artifactory
```

For more reference, see the full example at https://github.com/conan-io/conan-extensions/tree/main/extensions/commands/art#how-to-manage-build-infos-in-artifactory

See also:

- JFrog Artifactory has a dedicated API to manage build infos that has been integrated into the custom commands for Artifactory.
- Check the `conan art:build-info` documentation for reference: https://github.com/conan-io/conan-extensions/blob/main/extensions/commands/art/readme_build_info.md

7.13 ROS

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Conan provides integration for your Robot Operating System (ROS) C/C++ based projects. This will allow you to consume Conan packages inside your ROS package projects. The Conan packages can be installed and used in CMake with the help of the *ROSEnv generator* created for the purpose.

It provides a clean integration that requires no Conan-specific modifications in your *CMakeLists.txt*.

Important: This integration supports **ROS2**, it has been developed using the **Kilted version** and the aim is to **support newer versions going forward**. If you have any issues with other ROS versions, please let us know by opening an issue in our GitHub repository.

Note: Pre-requisites to run the example:

1. In order to run the example, it is expected that you have an Ubuntu environment with ROS2 installed. For convenience, you can also use this Dockerfile instead:

```
FROM osrf/ros:kilted-desktop
RUN apt-get update && apt-get install -y \
curl \
python3-pip \
git \
ros-kilted-nav2-msgs \
&& rm -rf /var/lib/apt/lists/*
RUN pip3 install --upgrade pip && pip3 install conan==2.*
RUN conan profile detect
CMD ["bash"]
```

   Simply copy the Dockerfile, build your image with `docker build -t conanio/ros-kilted .`, and finally run it with `docker run -it conanio/ros-kilted`.

There is also the possibility to run ROS2 on Windows. Follow the [installation instructions in the ROS 2 documentation](https://docs.ros.org/en/kilted/Installation/Windows-Install-Binary.html).

2. The files for this example can be found at our examples repository. Clone it like so to get started:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/ros/rosenv
```

7.13.1 Consuming Conan packages using the ROSEnv generator

Imagine we have a ROS C++ package called *str_printer* that uses some functionality from the third party string formatting library fmt to print fancy strings.

We have the following project structure:

```
$  tree /f
workspace
├──str_printer
│    │   CMakeLists.txt
│    │   conanfile.txt
│    │   package.xml
│    ├──include
│    │   └──str_printer
│    │        str_printer.h
│    └──src
│        str_printer.cpp
└──consumer
     │   CMakeLists.txt
```

(continues on next page)

(continued from previous page)

```
    │   package.xml
    └── src
        main.cpp
```

Where:

- The *str_printer* is a ROS package that implements a function and **depends on the fmt Conan package**.
- The *consumer* is also a ROS package that depends on the *str_printer* ROS package and uses its functionality in a **final executable**.

The only difference in the *str_printer* package with respect to a normal ROS package is that it includes a *conanfile.txt* file. This is the file used by Conan to install the required dependencies and generate the files needed to perform the build.

Listing 1: str_printer/conanfile.txt

```
[requires]
fmt/11.0.2

[generators]
CMakeDeps
CMakeToolchain
ROSEnv
```

In this case, we will install the 11.0.2 version of *fmt* and Conan will generate files for CMake and ROS so we can build the *str_printer* package later.

To install the *fmt* library using Conan we should do the following:

```
$ cd workspace
$ conan install str_printer/conanfile.txt --build missing --output-folder install/conan
======== Computing dependency graph ========
fmt/11.0.2: Not found in local cache, looking in remotes...
fmt/11.0.2: Checking remote: conancenter
fmt/11.0.2: Downloaded recipe revision 5c7438ef4d5d69ab106a41e460ce11f3
Graph root
    conanfile.txt: /home/user/examples2/examples/tools/ros/rosenv/workspace/str_printer/
↪conanfile.txt
Requirements
    fmt/11.0.2#5c7438ef4d5d69ab106a41e460ce11f3 - Downloaded (conancenter)

======== Computing necessary packages ========
Requirements
    fmt/11.0.2#5c7438ef4d5d69ab106a41e460ce11f3:29da3f322a17cc9826b294a7ab191c2f298a9f49
↪#d8d27fde7061f89f7992c671d98ead71 - Download (conancenter)

======== Installing packages ========

-------- Downloading 1 package --------
fmt/11.0.2: Retrieving package 29da3f322a17cc9826b294a7ab191c2f298a9f49 from remote
↪'conancenter'
fmt/11.0.2: Package installed 29da3f322a17cc9826b294a7ab191c2f298a9f49
fmt/11.0.2: Downloaded package revision d8d27fde7061f89f7992c671d98ead71
```

(continues on next page)

(continued from previous page)

```
======== Finalizing install (deploy, generators) ========
conanfile.txt: Writing generators to /home/user/examples2/examples/tools/ros/rosenv/
↪workspace/install/conan
conanfile.txt: Generator 'CMakeDeps' calling 'generate()'
conanfile.txt: CMakeDeps necessary find_package() and targets for your CMakeLists.txt
    find_package(fmt)
    target_link_libraries(... fmt::fmt)
conanfile.txt: Generator 'CMakeToolchain' calling 'generate()'
conanfile.txt: CMakeToolchain generated: conan_toolchain.cmake
conanfile.txt: Preset 'conan-release' added to CMakePresets.json. Invoke it manually␣
↪using 'cmake --preset conan-release' if using CMake>=3.23
conanfile.txt: If your CMake version is not compatible with CMakePresets (<3.23) call␣
↪cmake like: 'cmake <path> -G "Unix Makefiles" -DCMAKE_TOOLCHAIN_FILE=/home/danimtb/
↪examples2/examples/tools/ros/rosenv/workspace/install/conan/conan_toolchain.cmake -
↪DCMAKE_POLICY_DEFAULT_CMP0091=NEW -DCMAKE_BUILD_TYPE=Release'
conanfile.txt: CMakeToolchain generated: CMakePresets.json
conanfile.txt: CMakeToolchain generated: ../../str_printer/CMakeUserPresets.json
conanfile.txt: Generator 'ROSEnv' calling 'generate()'
conanfile.txt: Generated ROSEnv Conan file: conanrosenv.sh
Use 'source /home/user/examples2/examples/tools/ros/rosenv/workspace/install/conan/
↪conanrosenv.sh' to set the ROSEnv Conan before 'colcon build'
conanfile.txt: Generating aggregated env files
conanfile.txt: Generated aggregated env files: ['conanrosenv.sh']
Install finished successfully
```

This will download the *fmt* Conan package to the local cache and generate the CMake and ROS environment files in the *conan* subfolder of the *install* directory.

Now we can source our ROS environment, then **source the Conan ROSEnv environment**, so the conan-installed package are found by CMake, and then we can build the *str_printer* package as usual with Colcon.

```
$ source /opt/ros/kilted/setup.bash
$ source install/conan/conanrosenv.sh
$ colcon build --packages-select str_printer
Starting >>> str_printer
Finished <<< str_printer [10.8s]

Summary: 1 package finished [12.4s]
```

7.13.2 Bridging the Conan-provided transitive dependencies to another ROS package

As the *consumer* ROS package depends on *str_printer*, the targets of transitive dependencies should be exported. This is done as usual in the *str_printers*'s *CMakeLists.txt* using *ament_export_dependencies()*:

Listing 2: str_printer/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(str_printer)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
add_compile_options(-Wall -Wextra -Wpedantic)
```

(continues on next page)

(continued from previous page)

```
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
find_package(fmt REQUIRED)  # Retrieved with Conan C/C++ Package Manager

add_library(str_printer src/str_printer.cpp)

target_include_directories(str_printer PUBLIC
$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include/str_printer>
$<INSTALL_INTERFACE:include>)

target_compile_features(str_printer PUBLIC c_std_99 cxx_std_17)  # Require C99 and C++17
ament_target_dependencies(str_printer fmt)

ament_export_targets(str_printerTargets HAS_LIBRARY_TARGET)
ament_export_dependencies(fmt)

install(
DIRECTORY include/
DESTINATION include
)

install(
TARGETS str_printer
EXPORT str_printerTargets
LIBRARY DESTINATION lib
ARCHIVE DESTINATION lib
RUNTIME DESTINATION bin
INCLUDES DESTINATION include
)

ament_package()
```

To build the *consumer* ROS package, you can proceed as usual (make sure that you have both the ROS environment and the Conan ROSEnv environment *sourced* before building as in previous step):

```
$ colcon build --packages-select consumer
Starting >>> consumer
Finished <<< consumer [7.9s]

Summary: 1 package finished [9.4s]
```

And after this, our *consumer* application should be ready to run with just:

```
$ source install/setup.bash
$ ros2 run consumer main
Hi there! I am using fmt library fetched with Conan C/C++ Package Manager
```

See also:

- Reference for *ROSEnv generator*.

7.14 Python

7.14.1 conan-py-build

conan-py-build is a PEP 517 build backend that lets Conan manage the C/C++ side of Python packages that ship compiled extensions (via pybind11, nanobind, the Python C API, etc.).

Once it's declared as the `build-backend` in `pyproject.toml`, commands like `pip wheel`, `pip install` or `python -m build` will:

- Read the dependencies and build steps from a regular Conan `conanfile.py`.
- Resolve and install those dependencies with Conan, downloading precompiled binaries from ConanCenter or building them from source.
- Run the recipe's `build()`/`package()` methods with any build system Conan can drive (CMake, Meson, etc.).
- Copy what `package()` staged into the resulting wheel, alongside the pure Python part of the package.

Tip: See it in action: *build a simple Python package with a C++ extension (pybind11 + fmt) using conan-py-build*.

This removes the need for a separate, ad-hoc step to fetch and build C/C++ dependencies before packaging, and keeps that logic in the same `conanfile.py` format used for regular C/C++ packages. It also supports profiles and lockfiles for reproducible builds, dynamic versioning, PEP 621 entry points, and integrates with `auditwheel`/`delocate-wheel`/`delvewheel` and cibuildwheel to produce self-contained wheels with bundled shared libraries.

See also:

- Build a simple Python extension using Conan *example*.
- conan-py-build documentation, for the full getting started guide, configuration reference, and more examples (Meson, nanobind, cibuildwheel).
- Introducing conan-py-build blog post.

7.14.2 PyEnv

The *PyEnv* tool installs executable Python packages with `pip` inside a dedicated virtual environment, isolating them from the system Python and from the Conan package itself. It's meant for Python CLI tools needed during a recipe's build, such as a build system or code generator invoked from `build()`, not for Python libraries imported by the recipe.

See also:

- *PyEnv reference*, for the full API and a recipe example.
- *Using the PyEnv Python from CMake*, for the caveats when a CMake project looks for Python.

7.15 GitHub

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The Conan GitHub Actions integration allows you to setup Conan client in your GitHub Actions workflows in a simple and effective way.

The project can be found on its GitHub marketplace page, or its GitHub source page directly.

To use the integration, add a step in your workflow YAML file. The integration will install the Conan client and set up the environment for you.

You can customize the following parameters:

- **Conan version**: Specify the Conan version to install (e.g., *2.15.1*). Default: latest stable.
- **Configuration URLs**: A list of configuration URLs to download and install in Conan home. By default, no configuration is installed.
- **Conan Audit Token**: The *audit* token used for the audit command to scan vulnerabilities in packages. By default, no token is used.
- **Conan home path**: Set a custom location for the Conan home folder. By default, no custom path is used.
- **Cache Conan packages**: Cache all packages in your Conan cache automatically and re-use them in a next build. By default, no cache is used.
- **Python version**: You can specify the Python version to be installed with Conan, the same will be available in the environment. By default, Python 3.10 is installed.

The integration is available for all platforms supported by GitHub Actions, including Linux, Windows, and macOS.

7.15.1 Examples

This section provides some examples of how to use the integration in your GitHub Actions workflows.

Scanning Packages for Vulnerabilities in a Nightly Build

Warning: Do not share your Conan audit token or expose it in your code. Always use GitHub secrets for sensitive data.

First, you need to set up the Conan audit token in your GitHub secrets. Then, use the following example to scan for vulnerabilities in a package and its dependencies:

Listing 3: .github/workflows/ci.yml

```
name: Nightly security scan
on:
  schedule:
    - cron: "0 0 * * *"

jobs:
  scan-vulnerabilities:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Install and setup Conan
        uses: conan-io/setup-conan@v1
        with:
          audit_token: ${{ secrets.MY_CONAN_AUDIT_TOKEN }}

      - name: Scan for vulnerabilities with Conan Audit
        run: |
          conan audit scan .
```

This example scans all dependencies in a `conanfile.py` in the current directory. Note that it uses a GitHub schedule to run the scan every day at midnight, this is in the case of using the free service token, to avoid hitting the daily limits, but still having security checks every day.

Installing Conan configuration and building packages

This example installs a custom Conan configuration from a URL, restores cached packages from previous builds, builds the package defined in the `conanfile.py`, and uploads it to the Conan server.

Listing 4: .github/workflows/ci.yml

```
name: Build and upload Conan package
on:
  push:
    branches:
      - 'main'

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Install and setup Conan
        uses: conan-io/setup-conan@v1
        with:
          config_urls: https://mycompany.com/conan/configs.git
          cache_packages: true
```

(continues on next page)

(continued from previous page)

```
    - name: Build and upload package
      run: |
        conan create . -pr:a myprofile --build=missing
        conan remote login artifactory developer -p ${{ secrets.MY_CONAN_PASSWORD }}
        conan upload "*" --confirm --remote artifactory
```

In this example, the action's option `cache_packages` is set to true, so all packages in the Conan cache are cached for the next build. Remote information is expected from the configuration installed from the URL pointed by the option `config_urls`. Remote authentication uses GitHub secrets for security. The remote authentication is done using the GitHub secrets, which is a secure way to store sensitive information.

7.16 Community

Important: This section contains links to different integrations created and supported by the community. Conan is not responsible for any of them, and does not provide any guarantee of compatibility or support.

- **conan2-rs Rust-Conan integration**: A Rust wrapper of the conan C/C++ package manager (conan.io) to simplify usage in build scripts. For source and support go to the conan2-rs Github page. Created by @ravenexp
- **Cruiz Conan GUI**. A Graphical User Interface for Conan client from Mark Final. Source and support in cruiz Github page.
- **Gradle Conan plugin**: A plugin to use Conan from the Gradle build system. See it in the Gradle plugins site. Project wiki here, for source, suppport (tickets), go to project Sourceforge page
- **nvim-conan Neovim integration**: A Conan 2 integration for managing packages, profiles, remotes and common Conan workflows directly from Neovim. For source and support go to the nvim-conan Github page. Created by @mm4cN

Note: Notes

- If you have an integration that you think could be valuable for the community, and you are willing to support it, feel free to open a Pull Request to this page in the Github repo.
- Recall that these integrations are not supported by the Conan team. If you have any questions, issues or feedback about them, please reach out to their maintainers in their issue trackers.

CHAPTER

EIGHT

EXAMPLES

8.1 ConanFile methods examples

8.1.1 ConanFile package_info() examples

Propagating environment or configuration information to consumers

TBD

Define components for Conan packages that provide multiple libraries

At the *section of the tutorial about the package_info() method*, we learned how to define information in a package for consumers, such as library names or include and library folders. In the tutorial, we created a package with only one library that consumers linked to. However, in some cases, libraries provide their functionalities separated into different *components*. These components can be consumed independently, and in some cases, they may require other components from the same library or others. For example, consider a library like OpenSSL that provides *libcrypto* and *libssl*, where *libssl* depends on *libcrypto*.

Conan provides a way to abstract this information using the *components* attribute of the *CppInfo* object to define the information for each separate component of a Conan package. Consumers can also select specific components to link against but not the rest of the package.

Let's take a game-engine library as an example, which provides several components such as *algorithms*, *ai*, *rendering*, and *network*. Both *ai* and *rendering* depend on the *algorithms* component.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/conanfile/package_info/components
```

You can check the contents of the project:

```
.
├── CMakeLists.txt
├── conanfile.py
├── include
│   ├── ai.h
│   ├── algorithms.h
│   ├── network.h
│   └── rendering.h
├── src
```

(continues on next page)

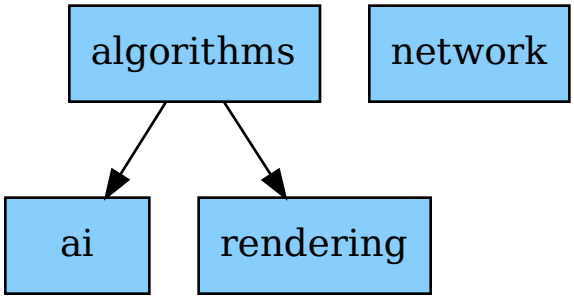


Fig. 1: components of the game-engine package

(continued from previous page)

```
    ├── ai.cpp
    ├── algorithms.cpp
    ├── network.cpp
    └── rendering.cpp
└── test_package
    ├── CMakeLists.txt
    ├── CMakeUserPresets.json
    ├── conanfile.py
    └── src
        └── example.cpp
```

As you can see, there are sources for each of the components and a CMakeLists.txt file to build them. We also have a *test_package* that we are going to use to test the consumption of the separate components.

First, let's have a look at package_info() method in the *conanfile.py* and how we declared the information for each component that we want to provide to the consumers of the game-engine package:

```
...

def package_info(self):
    self.cpp_info.components["algorithms"].libs = ["algorithms"]
    self.cpp_info.components["algorithms"].set_property("cmake_target_name", "algorithms
↪")

    self.cpp_info.components["network"].libs = ["network"]
    self.cpp_info.components["network"].set_property("cmake_target_name", "network")

    self.cpp_info.components["ai"].libs = ["ai"]
    self.cpp_info.components["ai"].requires = ["algorithms"]
    self.cpp_info.components["ai"].set_property("cmake_target_name", "ai")

    self.cpp_info.components["rendering"].libs = ["rendering"]
    self.cpp_info.components["rendering"].requires = ["algorithms"]
```

(continues on next page)

(continued from previous page)

```
    self.cpp_info.components["rendering"].set_property("cmake_target_name", "rendering")
```

There are a couple of relevant things:

- We declare the libraries generated by each of the components by setting information in the `cpp_info.components` attribute. You can set the same information for each of the components as you would for the `self.cpp_info` object. The `cpp_info` for components has some defaults defined, just like it does for *self.cpp_info*. For example, the `cpp_info.components` object provides the `.includedirs` and `.libdirs` properties to define those locations, but Conan sets their value as `["lib"]` and `["include"]` by default, so it's not necessary to add them in this case.
- We are also declaring the components' dependencies using the `.requires` attribute. With this attribute, you can declare requirements at the component level, not only for components in the same recipe but also for components from other packages that are declared as requires of the Conan package.
- We are changing the default target names for the components using the *properties model*. By default, Conan sets a target name for components like `<package_name::component_name>`, but for this tutorial we will set the component target names just with the component names omitting the `::`.
- When `cpp_info` has global build information (e.g. `cpp_info.defines`), it does not inherit to the components. If you want to share this information with the components, you need to set it explicitly for each component.

You can have a look at the consumer part by checking the *test_package* folder. First the *conanfile.py*:

```
...

def generate(self):
    deps = CMakeDeps(self)
    deps.check_components_exist = True
    deps.generate()
```

You can see that we are setting the *check_components_exist* property for CMakeDeps. This is not needed, just to show how you can do if you want your consumers to fail if the component does not exist. So, the CMakeLists.txt could look like this:

```
cmake_minimum_required(VERSION 3.15)
project(PackageTest CXX)

find_package(game-engine REQUIRED COMPONENTS algorithms network ai rendering)

add_executable(example src/example.cpp)

target_link_libraries(example algorithms
                              network
                              ai
                              rendering)
```

And the `find_package()` call would fail if any of the components targets do not exist.

Let's run the example:

```
$ conan create .
...
game-engine/1.0: RUN: cmake --build "/Users/barbarian/.conan2/p/t/game-d6e361d329116/b/
↪build/Release" -- -j16
[ 12%] Building CXX object CMakeFiles/algorithms.dir/src/algorithms.cpp.o
```

(continues on next page)

(continued from previous page)

```
[ 25%] Building CXX object CMakeFiles/network.dir/src/network.cpp.o
[ 37%] Linking CXX static library libnetwork.a
[ 50%] Linking CXX static library libalgorithms.a
[ 50%] Built target network
[ 50%] Built target algorithms
[ 62%] Building CXX object CMakeFiles/ai.dir/src/ai.cpp.o
[ 75%] Building CXX object CMakeFiles/rendering.dir/src/rendering.cpp.o
[ 87%] Linking CXX static library libai.a
[100%] Linking CXX static library librendering.a
[100%] Built target ai
[100%] Built target rendering
...

======== Launching test_package ========

...
-- Conan: Component target declared 'algorithms'
-- Conan: Component target declared 'network'
-- Conan: Component target declared 'ai'
-- Conan: Component target declared 'rendering'
...
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example


======== Testing the package: Executing test ========
game-engine/1.0 (test package): Running test()
game-engine/1.0 (test package): RUN: ./example
I am the algorithms component!
I am the network component!
I am the ai component!
└──> I am the algorithms component!
I am the rendering component!
└──> I am the algorithms component!
```

You could check that requiring a component that does not exist will raise an error. Add the *nonexistent* component to the find_package() call:

```
cmake_minimum_required(VERSION 3.15)
project(PackageTest CXX)

find_package(game-engine REQUIRED COMPONENTS nonexistent algorithms network ai rendering)

add_executable(example src/example.cpp)

target_link_libraries(example algorithms
                             network
                             ai
                             rendering)
```

And test the package again:

```
$ conan test test_package game-engine/1.0

...

Conan: Component 'nonexistent' NOT found in package 'game-engine'
Call Stack (most recent call first):
CMakeLists.txt:4 (find_package)

-- Configuring incomplete, errors occurred!

...

ERROR: game-engine/1.0 (test package): Error in build() method, line 22
        cmake.configure()
        ConanException: Error 1 while executing
```

See also:

If you want to use recipes defining components in `editable` mode, check the example in *Using components and editable packages*.

8.1.2 ConanFile layout() examples

Declaring the layout when the Conanfile is inside a subfolder

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/conanfile/layout/conanfile_in_subfolder
```

If we have a project intended to package the code that is in the same repo as the `conanfile.py`, but the `conanfile.py` is not in the root of the project:

```
.
├── CMakeLists.txt
├── conan
│   └── conanfile.py
├── include
│   └── say.h
└── src
    └── say.cpp
```

The `conanfile.py` would look like this:

```
import os
from conan import ConanFile
from conan.tools.files import load, copy
from conan.tools.cmake import CMake


class PkgSay(ConanFile):
    name = "say"
    version = "1.0"
```

(continues on next page)

(continued from previous page)

```
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain"

    def layout(self):
        # The root of the project is one level above
        self.folders.root = ".."
        # The source of the project (the root CMakeLists.txt) is the source folder
        self.folders.source = "."
        self.folders.build = "build"

    def export_sources(self):
        # The path of the CMakeLists.txt and sources we want to export are one level␣
↪above
        folder = os.path.join(self.recipe_folder, "..")
        copy(self, "*.txt", folder, self.export_sources_folder)
        copy(self, "src/*.cpp", folder, self.export_sources_folder)
        copy(self, "include/*.h", folder, self.export_sources_folder)

    def source(self):
        # Check that we can see that the CMakeLists.txt is inside the source folder
        cmake_file = load(self, "CMakeLists.txt")

    def build(self):
        # Check that the build() method can also access the CMakeLists.txt in the source␣
↪folder
        path = os.path.join(self.source_folder, "CMakeLists.txt")
        cmake_file = load(self, path)

        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def package(self):
        cmake = CMake(self)
        cmake.install()
```

You can try and create the `say` package:

```
$ cd conan
$ conan create .
```

See also:

- *layout method*
- *how the package layout works*.

Declaring the layout when creating packages for third-party libraries

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/conanfile/layout/third_party_libraries
```

If we have this project, intended to create a package for a third-party library whose code is located externally:

```
.
├── conanfile.py
└── patches
    └── mypatch
```

The `conanfile.py` would look like this:

```
...

class Pkg(ConanFile):
    name = "hello"
    version = "1.0"
    exports_sources = "patches*"

    ...

    def layout(self):
        cmake_layout(self, src_folder="src")
        # if you are declaring your own layout, just declare:
        # self.folders.source = "src"

    def source(self):
        # we are inside a "src" subfolder, as defined by layout
        # the downloaded soures will be inside the "src" subfolder
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",
            strip_root=True)
        # Please, be aware that using the head of the branch instead of an immutable tag
        # or commit is not a good practice in general as the branch may change the␣
↪contents

        # patching, replacing, happens here
        patch(self, patch_file=os.path.join(self.export_sources_folder, "patches/mypatch
↪"))

    def build(self):
        # If necessary, the build() method also has access to the export_sources_folder
        # for example if patching happens in build() instead of source()
        #patch(self, patch_file=os.path.join(self.export_sources_folder, "patches/mypatch
↪"))
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
        ...
```

We can see that the `ConanFile.export_sources_folder` attribute can provide access to the root folder of the sources:

- Locally it will be the folder where the `conanfile.py` lives
- In the cache it will be the "source" folder, that will contain a copy of `CMakeLists.txt` and `patches`, while the "source/src" folder will contain the actual downloaded sources.

We can check that everything runs fine now:

```
$ conan create .
...
Downloading main.zip
hello/1.0: Unzipping 3.7KB
Unzipping 100 %
...
[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX static library libhello.a
[100%] Built target hello
...
$ conan list hello/1.0
Local Cache
hello
    hello/1.0
```

See also:

- Read more about the *layout method* and *how the package layout works*.

Declaring the layout when we have multiple subprojects

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/conanfile/layout/multiple_subprojects
```

Let's say that we have a project that contains two subprojects: *hello* and *bye*, that need to access some information that is at their same level (sibling folders). Each subproject would be a Conan package. The structure could be something similar to this:

```
.
├── bye
│   ├── CMakeLists.txt
│   ├── bye.cpp        # contains an #include "../common/myheader.h"
│   └── conanfile.py   # contains include(../common/myutils.cmake)
├── common
│   ├── myheader.h
│   └── myutils.cmake
└── hello
    ├── CMakeLists.txt # contains include(../common/myutils.cmake)
    ├── conanfile.py
    └── hello.cpp      # contains an #include "../common/myheader.h"
```

Both *hello* and *bye* subprojects needs to use some of the files located inside the `common` folder (that might be used and shared by other subprojects too), and it references them by their relative location. Note that `common` is not intended to be a Conan package. It is just some common code that will be copied into the different subproject packages.

We can use the `self.folders.root = ".."` layout specifier to locate the root of the project, then use the `self.folders.subproject = "subprojectfolder"` to relocate back most of the layout to the current subproject folder,

as it would be the one containing the build scripts, sources code, etc., so other helpers like `cmake_layout()` keep working. Let's see how the *conanfile.py* of *hello* could look like:

Listing 1: ./hello/conanfile.py

```
import os
from conan import ConanFile
from conan.tools.cmake import cmake_layout, CMake
from conan.tools.files import copy


class hello(ConanFile):
    name = "hello"
    version = "1.0"

    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain"

    def layout(self):
        self.folders.root = ".."
        self.folders.subproject = "hello"
        cmake_layout(self)

    def export_sources(self):
        source_folder = os.path.join(self.recipe_folder, "..")
        copy(self, "hello/conanfile.py", source_folder, self.export_sources_folder)
        copy(self, "hello/CMakeLists.txt", source_folder, self.export_sources_folder)
        copy(self, "hello/hello.cpp", source_folder, self.export_sources_folder)
        copy(self, "common*", source_folder, self.export_sources_folder)

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
        self.run(os.path.join(self.cpp.build.bindirs[0], "hello"))
```

Let's build *hello* and check that it's building correctly, using the contents of the common folder.

```
$ conan install hello
$ conan build hello
...
[100%] Built target hello
conanfile.py (hello/1.0): RUN: ./hello
hello WORLD
```

You can also run a **`conan create`** and check that it works fine too:

```
$ conan create hello
...
[100%] Built target hello
conanfile.py (hello/1.0): RUN: ./hello
hello WORLD
```

Note: Note the importance of the `export_sources()` method, which is able to maintain the same relative layout of

the `hello` and `common` folders, both in the local developer flow in the current folder, but also when those sources are copied to the Conan cache, to be built there with `conan create` or `conan install --build=hello`. This is one of the design principles of the `layout()`, the relative location of things must be consistent in the user folder and in the cache.

See also:

- Read more about the *layout method* and *how the package layout works*.

Using components and editable packages

It is possible to define components in the `layout()` method, to support the case of `editable` packages. That is, if we want to put a package in `editable` mode, and that package defines `components`, it is necessary to define the components layout correctly in the `layout()` method. Let's see it in a real example.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/conanfile/layout/editable_components
```

There we find a `greetings` subfolder and package, that contains 2 libraries, the `hello` library and the `bye` library. Each one is modeled as a `component` inside the package recipe:

Listing 2: greetings/conanfile.py

```
class GreetingsConan(ConanFile):
    name = "greetings"
    version = "0.1"
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeDeps", "CMakeToolchain"
    exports_sources = "src/*"

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def layout(self):
        cmake_layout(self, src_folder="src")
        # This "includedirs" starts in the source folder, which is "src"
        # So the components include dirs is the "src" folder (includes are
        # intended to be included as ``#include "hello/hello.h"``)
        self.cpp.source.components["hello"].includedirs = ["."]
        self.cpp.source.components["bye"].includedirs = ["."]
        # compiled libraries "libdirs" will be inside the "build" folder, depending
        # on the platform they will be in "build/Release" or directly in "build" folder
        bt = "." if self.settings.os != "Windows" else str(self.settings.build_type)
        self.cpp.build.components["hello"].libdirs = [bt]
        self.cpp.build.components["bye"].libdirs = [bt]

    def package(self):
        copy(self, "*.h", src=self.source_folder,
             dst=join(self.package_folder, "include"))
        copy(self, "*.lib", src=self.build_folder,
```

(continues on next page)

(continued from previous page)

```
            dst=join(self.package_folder, "lib"), keep_path=False)
        copy(self, "*.a", src=self.build_folder,
            dst=join(self.package_folder, "lib"), keep_path=False)

    def package_info(self):
        self.cpp_info.components["hello"].libs = ["hello"]
        self.cpp_info.components["bye"].libs = ["bye"]

        self.cpp_info.set_property("cmake_file_name", "MYG")
        self.cpp_info.set_property("cmake_target_name", "MyGreetings::MyGreetings")
        self.cpp_info.components["hello"].set_property("cmake_target_name",
↪"MyGreetings::MyHello")
        self.cpp_info.components["bye"].set_property("cmake_target_name",
↪"MyGreetings::MyBye")
```

While the location of the `hello` and `bye` libraries in the final package is in the final `lib` folder, then nothing special is needed in the `package_info()` method, beyond the definition of the components. In this case, the customization of the CMake generated filenames and targets is also included, but it is not necessary for this example.

The important part is the `layout()` definition. Besides the common `cmake_layout`, it is necessary to define the location of the components headers (`self.cpp.source` as they are source code) and the location of the locally built libraries. As the location of the libraries depends on the platform, the final `self.cpp.build.components["component"].libdirs` depends on the platform.

With this recipe we can put the package in editable mode and locally build it with:

```
$ conan editable add greetings
$ conan build greetings
# we might want to also build the debug config
```

In the `app` folder we have a package recipe to build 2 executables, that link with the `greeting` package components. The `app/conanfile.py` recipe there is simple, the `build()` method builds and runs both `example` and `example2` executables that are built with `CMakeLists.txt`:

```
# Note the MYG file name, not matching the package name,
# because the recipe defined "cmake_file_name"
find_package(MYG)

add_executable(example example.cpp)
# Note the MyGreetings::MyGreetings target name, not matching the package name,
# because the recipe defined "cmake_target_name"
# "example" is linking with the whole package, both "hello" and "bye" components
target_link_libraries(example MyGreetings::MyGreetings)

add_executable(example2 example2.cpp)
# "example2" is only using and linking "hello" component, but not "bye"
target_link_libraries(example2 MyGreetings::MyHello)
```

```
$ conan build app
hello: Release!
bye: Release!
```

If you now go to the `bye.cpp` source file and modify the output message, then build `greetings` and `app` locally, the final output message for the "bye" component library should change:

```
$ conan build greetings
$ conan build app
hello: Release!
adios: Release!
```

8.2 Conan extensions examples

Note: Check the conan-extensions repository, which hosts useful extensions ready to use or to take inspiration from for your custom ones

8.2.1 Custom commands

Custom command: Clean old recipe and package revisions

Note: This is mostly an example command. The built-in `conan remove *#!latest` syntax, meaning "all revisions but the latest" would probably be enough for this use case, without needing this custom command.

Warning: Using this command requires Conan 2.21.0 or higher.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/extensions/commands/clean
```

In this example we are going to see how to create/use a custom command: **`conan clean`**. It removes every recipe and its package revisions from the local cache or the remotes, except the latest package revision from the latest recipe one.

Note: To understand better this example, it is highly recommended to read previously the *Custom commands reference*.

Locate the command

Copy the command file `cmd_clean.py` into your `[YOUR_CONAN_HOME]/extensions/commands/` folder (create it if it's not there). If you don't know where `[YOUR_CONAN_HOME]` is located, you can run **`conan config home`** to check it.

Run it

Now, you should be able to see the new command in your command prompt:

```
$ conan -h
...
Custom commands
clean        Deletes (from local cache or remotes) all recipe and package revisions but␣
↪the
               latest package revision from the latest recipe revision.

$ conan clean -h
usage: conan clean [-h] [-r REMOTE] [--force]

Deletes (from local cache or remotes) all recipe and package revisions but
the latest package revision from the latest recipe revision.

optional arguments:
  -h, --help            show this help message and exit
  -r REMOTE, --remote REMOTE
                        Will remove from the specified remote
  --force               Remove without requesting a confirmation
```

Finally, if you execute **conan clean**:

```
$ conan clean
Found 4 pkg/version recipes matching */* in local cache
Do you want to remove all the recipes revisions and their packages ones, except the␣
↪latest package revision from the latest recipe one? (yes/no): yes
Keeping recipe revision: other/1.0#31da245c3399e4124e39bd4f77b5261f and its latest␣
↪package revisions [Local cache]
Removed package revision: other/1.0
↪#31da245c3399e4124e39bd4f77b5261f:da39a3ee5e6b4b0d3255bfef95601890afd80709
↪#a16985deb2e1aa73a8480faad22b722c [Local cache]
Removed recipe revision: other/1.0#721995a35b1a8d840ce634ea1ac71161 and all its package␣
↪revisions [Local cache]
Keeping recipe revision: hello/1.0#9a77cdcff3a539b5b077dd811b2ae3b0 and its latest␣
↪package revisions [Local cache]
Removed package revision: hello/1.0
↪#9a77cdcff3a539b5b077dd811b2ae3b0:da39a3ee5e6b4b0d3255bfef95601890afd80709
↪#cee90a74944125e7e9b4f74210bfec3f [Local cache]
Removed package revision: hello/1.0
↪#9a77cdcff3a539b5b077dd811b2ae3b0:da39a3ee5e6b4b0d3255bfef95601890afd80709
↪#7cddd50952de9935d6c3b5b676a34c48 [Local cache]
Keeping recipe revision: libcxx/0.1#abcdef1234567890abcdef1234567890 and its latest␣
↪package revisions [Local cache]
```

Nothing should happen if you run it again:

```
$ conan clean
Do you want to remove all the recipes revisions and their packages ones, except the␣
↪latest package revision from the latest recipe one? (yes/no): yes
Keeping recipe revision: other/1.0#31da245c3399e4124e39bd4f77b5261f and its latest␣
↪package revisions [Local cache]
```

(continues on next page)

(continued from previous page)

```
Keeping recipe revision: hello/1.0#9a77cdcff3a539b5b077dd811b2ae3b0 and its latest␣
↪package revisions [Local cache]
Keeping recipe revision: libcxx/0.1#abcdef1234567890abcdef1234567890 and its latest␣
↪package revisions [Local cache]
```

Code tour

The `conan clean` command has the following code:

Listing 3: cmd_clean.py

```
from conan.api.conan_api import ConanAPI
from conan.api.model import PackagesList, ListPattern
from conan.api.input import UserInput
from conan.api.output import ConanOutput, Color
from conan.cli.command import OnceArgument, conan_command

recipe_color = Color.BRIGHT_BLUE
removed_color = Color.BRIGHT_YELLOW

@conan_command(group="Custom commands")
def clean(conan_api: ConanAPI, parser, *args):
    """
    Deletes (from local cache or remotes) all recipe and package revisions but
    the latest package revision from the latest recipe revision.
    """
    parser.add_argument('-r', '--remote', action=OnceArgument,
                        help='Will remove from the specified remote')
    parser.add_argument('--force', default=False, action='store_true',
                        help='Remove without requesting a confirmation')
    args = parser.parse_args(*args)

    def confirmation(message):
        return args.force or ui.request_boolean(message)

    ui = UserInput(non_interactive=False)
    out = ConanOutput()
    remote = conan_api.remotes.get(args.remote) if args.remote else None
    output_remote = remote or "Local cache"

    # List all recipes revisions and all their packages revisions as well
    pkg_list = conan_api.list.select(ListPattern("*/*#*:*#*", rrev=None, prev=None),␣
↪remote=remote)
    if pkg_list and not confirmation("Do you want to remove all the recipes revisions␣
↪and their packages ones, "
                                     "except the latest package revision from the latest␣
↪recipe one?"):
        out.writeln("Aborted")
        return

    # Split the package list into based on their recipe reference
```

(continues on next page)

(continued from previous page)

```
    for sub_pkg_list in pkg_list.split():
        latest = max(sub_pkg_list.items(), key=lambda item: item[0])[0]
        out.writeln(f"Keeping recipe revision: {latest.repr_notime()} "
                    f"and its latest package revisions [{output_remote}]", fg=recipe_
↪color)
        for rref, packages in sub_pkg_list.items():
            # For the latest recipe revision, keep the latest package revision only
            if latest == rref:
                # Get the latest package timestamp for each package_id
                latest_pref_list = [max([p for p in packages if p.package_id == pkg_id],␣
↪key=lambda p: p.timestamp)
                                    for pkg_id in {p.package_id for p in packages}]
                for pref in packages:
                    if pref not in latest_pref_list:
                        conan_api.remove.package(pref, remote=remote)
                        out.writeln(f"Removed package revision: {pref.repr_notime()} [
↪{output_remote}]", fg=removed_color)
            else:
                # Otherwise, remove all outdated recipe revisions and their packages
                conan_api.remove.recipe(rref, remote=remote)
                out.writeln(f"Removed recipe revision: {rref.repr_notime()} "
                            f"and all its package revisions [{output_remote}]",␣
↪fg=removed_color)
```

Let's analyze the most important parts.

parser

The `parser` param is an instance of the Python command-line parsing `argparse.ArgumentParser`, so if you want to know more about its API, visit its official website.

User output

`ConanOutput()`: class to manage user outputs. In this example, we're using only `out.writeln(message, fg=None, bg=None)` where `fg` is the font foreground, and `bg` is the font background. Apart from that, you have some predefined methods like `out.info()`, `out.success()`, `out.error()`, etc.

Conan public API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The most important part of this example is the usage of the Conan API via `conan_api` parameter. These are some examples which are being used in this custom command:

```
conan_api.remotes.get(args.remote)
conan_api.list.select(ListPattern("*/*#*:*#*", rrev=None, prev=None), remote=remote)
```

(continues on next page)

(continued from previous page)

```
conan_api.remove.recipe(rrev, remote=remote)
conan_api.remove.package(prev, remote=remote)
```

- `conan_api.remotes.get(...)`: `[RemotesAPI]` Returns a RemoteRegistry given the remote name.
- `conan_api.list.select(...)`: `[ListAPI]` Returns a list with all the recipes matching the given pattern.
- `conan_api.remove.recipe(...)`: `[RemoveAPI]` Removes the given recipe revision and all its package revisions.
- `conan_api.remove.package(...)`: `[RemoveAPI]` Removes the given package revision.

Besides that, it deserves especial attention these lines:

```
for sub_pkg_list in pkg_list.split():
    latest = max(sub_pkg_list.items(), key=lambda item: item[0])[0]

...

latest_pref_list = [max([p for p in packages if p.package_id == pkg_id], key=lambda p: p.
↪timestamp)
                                for pkg_id in {p.package_id for p in packages}]
```

Basically, the `pkg_list.split()` is returning a list for the same recipe reference. Then, `sub_pkg_list.items()` returns a list of tuples `(Recipe Reference, Packages References)`, so finally, `max(..., key=...)` is used to get the latest recipe reference based on its timestamp. Later, `latest_pref_list` is created to keep only the latest package revision for each package ID. It iterates over the set of package IDs to get the latest package revision based on its timestamp.

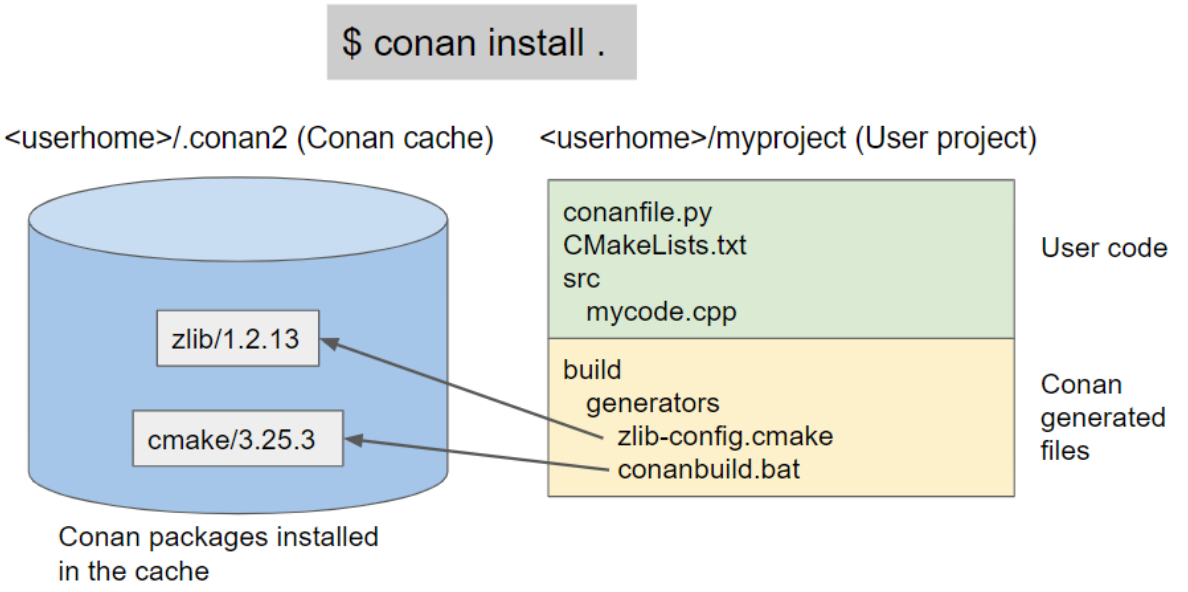
If you want to know more about the Conan API, visit the *ConanAPI section*

8.2.2 Builtin deployers

Creating a Conan-agnostic deploy of dependencies for developer use

With the `full_deploy` deployer it is possible to create a Conan-agnostic copy of dependencies that can be used by developers without even having Conan installed in their computers.

The common and recommended flow for most cases is using Conan packages directly from the Conan cache:



However, in some situations, it might be useful to be able to deploy a copy of the dependencies into a user folder, so the dependencies can be located there instead of in the Conan cache. This is possible using the Conan deployers.

Let's see it with an example. All the source code is in the examples2.0 Github repository

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/extensions/deployers/development_deploy
```

In the folder we can find the following `conanfile.txt`:

```
[requires]
zlib/1.2.13

[tool_requires]
cmake/3.25.3

[generators]
CMakeDeps
CMakeToolchain

[layout]
cmake_layout
```

The folder also contains a standard `CMakeLists.txt` and a `main.cpp` source file that can create an executable that links with `zlib` library.

We can install the Debug and Release dependencies, and deploy a local copy of the packages with:

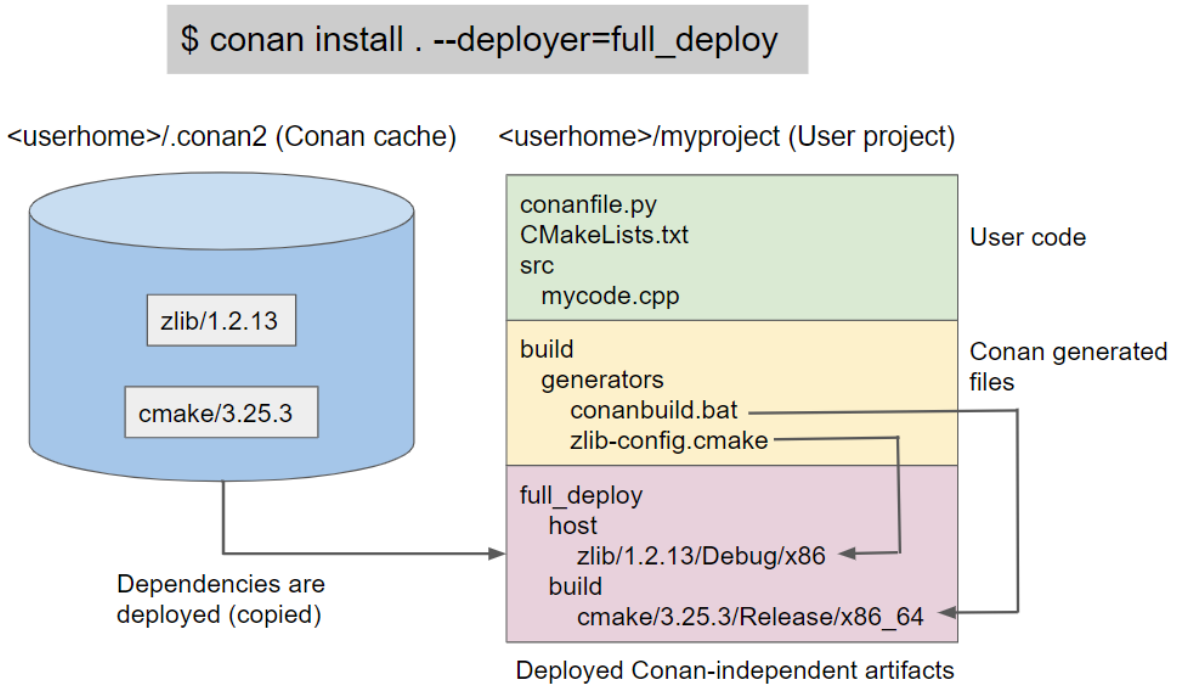
```
$ conan install . --deployer=full_deploy --build=missing
$ conan install . --deployer=full_deploy -s build_type=Debug --build=missing
```

This will create the following folders:

```
├──src
├──build
│   ├──generators
│         └── ZLibConfig.cmake
├──full_deploy
    ├──build
    │   └──cmake
    │       └──3.25.3
    │           └──x86_64
    │               ├──bin
    │
    └──host
        └──zlib
            └──1.2.13
                ├──Debug
                │   └──x86_64
                │       ├──include
                │       ├──lib
                └──Release
                    └──x86_64
                        ├──include
                        ├──lib
```

(Note that you could use the `--deployer-folder` argument to change the base folder output path for the deployer)

This folder is fully self-contained. It contains both the necessary tools (like `cmake` executable), the headers and compiled libraries of `zlib` and the necessary files like `ZLibConfig.cmake` in the `build/generators` folder, that point to the binaries inside `full_deploy` with a relative path.



The Conan cache can be removed, and even Conan uninstalled, then the folder could be moved elsewhere in the com-

puter or copied to another computer, assuming it has the same configuration of OS, compiler, etc.

```
$ cd ..
$ cp -R development_deploy /some/other/place
$ cd /some/other/place
```

And the files could be used by developers as:

Listing 4: Windows

```
$ cd build
# Activate the environment to use CMake 3.25
$ generators\conanbuild.bat
$ cmake --version
cmake version 3.25.3
# Configure, should match the settings used at install
$ cmake .. -G \"Visual Studio 17 2022\" -DCMAKE_TOOLCHAIN_FILE=generators/conan_
↪toolchain.cmake
$ cmake --build . --config Release
$ Release\compressor.exe
ZLIB VERSION: 1.2.13
```

The environment scripts in Linux and OSX are not relocatable, because they contain absolute paths and the `sh` shell does not have any way to provide access to the current script directory for sourced files.

This shouldn't be a big blocker, as a "search and replace" with `sed` in the generators folder can fix it:

Listing 5: Linux

```
$ cd build/Release/generators
# Fix folders in Linux
$ sed -i 's,{old_folder},{new_folder},g' *
# Fix folders in MacOS
$ sed -i '' 's,{old_folder},{new_folder},g' *
$ source conanbuild.sh
$ cd ..
$ cmake --version
cmake version 3.25.3
$ cmake ../.. -DCMAKE_TOOLCHAIN_FILE=generators/conan_toolchain.cmake -DCMAKE_BUILD_
↪TYPE=Release
$ cmake --build .
$ ./compressor
ZLIB VERSION: 1.2.13
```

Note: Best practices

The fact that this flow is possible doesn't mean that it is recommended for the majority of cases. It has some limitations:

- It is less efficient, requiring an extra copy of dependencies
- Only `CMakeDeps` and `CMakeToolchain` are relocatable at this moment. For other build system integrations, please create a ticket in Github
- Linux and OSX shell scripts are not relocatable and require a manual `sed`
- The binary variability is limited to Release/Debug. The generated files are exclusively for the current configuration, changing any other setting (os, compiler, architecture) will require a different deploy

In the general case, normal usage of the cache is recommended. This "relocatable development deployment" could be useful for distributing final products that looks like an SDK, to consumers of a project not using Conan.

8.2.3 Custom deployers

Copy sources from all your dependencies

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/extensions/deployers/sources
```

In this example we are going to see how to create and use a custom deployer. This deployer copies all the source files from your dependencies and puts them into a specific output folder

Note: To better understand this example, it is highly recommended to have previously read the *Deployers* reference.

Locate the deployer

In this case, the deployer is located in the same directory as our example conanfile, but as shown in *Deployers* reference, Conan will look for the specified deployer in a few extra places in order, namely:

1. Absolute paths
2. Relative to cwd
3. In the `[CONAN_HOME]/extensions/deployers` folder
4. Built-in deployers

Run it

For our example, we have a simple recipe that lists both `zlib` and `mcap` as requirements. With the help of the `tools.build:download_source=True` conf, we can force the invocation of its `source()` method, which will ensure that sources are available even if no build needs to be carried out.

Now, you should be able to use the new deployer in both `conan install` and `conan graph` commands for any given recipe:

```
$ conan graph info . -c tools.build:download_source=True --deployer=sources_deploy
```

Inspecting the command output we can see that it copied the sources of our direct dependencies `zlib` and `mcap`, **plus** the sources of our transitive dependencies, `zstd` and `lz4` to a `dependencies_sources` folder. After this is done, extra preprocessing could be done to accomplish more specific needs.

Note that you can pass the `--deployer-folder` argument to change the base folder output path for the deployer.

Code tour

The **source_deploy.py** file has the following code:

Listing 6: **sources_deploy.py**

```
from conan.errors import ConanException
from conan.tools.files import copy
import os


def deploy(graph, output_folder, **kwargs):
    # Note the kwargs argument is mandatory to be robust against future changes.
    for name, dep in graph.root.conanfile.dependencies.items():
        if dep.folders is None or dep.folders.source_folder is None:
            raise ConanException(f"Sources missing for {name} dependency.\n"
                                 "This deployer needs the sources of every dependency␣
↪present to work, either building from source, "
                                 "or by using the 'tools.build:download_source' conf.")
        copy(graph.root.conanfile, "*", dep.folders.source_folder, os.path.join(output_
↪folder, "dependency_sources", str(dep)))
```

deploy()

The `deploy()` method is called by Conan, and gets both a dependency graph and an output folder path as arguments. It iterates all the dependencies of our recipe and copies every source file to their respective folders under `dependencies_sources` using *conan.tools.copy*.

Note: If you're using this deployer as an example for your own, remember that `tools.build:download_source=True` is necessary so that `dep.folders.source_folder` is defined for the dependencies. Without the conf, said variable will not be defined for those dependencies that do not need to be built from sources nor in those commands that do not require building, such as **`conan graph`**.

Note: If your custom deployer needs access to the full dependency graph, including those libraries that might be skipped, use the `tools.graph:skip_binaries=False` conf. This is useful for collecting, for example, all the licenses in your graph.

Customize the full_deploy deployer: handling editable packages

The built-in *full_deploy* deployer copies the `package_folder` of every dependency to the deployer output folder. This is what it does with the different kinds of dependencies:

- Packages in the Conan cache: the package contents are copied.
- Dependencies with `package_folder=None`, which happens when the binary is skipped because it is not necessary: they are not deployed, as there is no folder to copy.
- Packages in *editable mode*: the `package_folder` is the local project root folder, where the `conanfile.py` of the editable is. There is no actual "package" there, but `full_deploy` copies the whole folder, including sources, build artifacts and anything else in it.

Deploying editables is not recommended in general. Deployers are intended to be customized. For real production needs, it is recommended to use your own deployer, and install it with `conan config install` or `conan config install-pkg`.

This is the implementation of the built-in `full_deploy`, slightly simplified, with some extra code to handle editables. Copy it to a `my_full_deploy.py` file:

Listing 7: **my_full_deploy.py**

```
import os
import shutil

from conan.errors import ConanException
from conan.tools.files import rmdir


def deploy(graph, output_folder, **kwargs):
    conanfile = graph.root.conanfile
    conanfile.output.info(f"Custom full deployer to {output_folder}")
    symlinks = conanfile.conf.get("tools.deployer:symlinks", check_type=bool,␣
↪default=True)
    for dep in conanfile.dependencies.values():
        # Skipped binaries, there is nothing to deploy
        if dep.package_folder is None:
            continue
        # Editable packages: the package_folder is the local project folder, not a␣
↪package.
        # Raise a ConanException instead of "continue" to fail when there are editables
        if dep.recipe == "Editable":
            conanfile.output.warning(f"Skipping deploy of editable package {dep}")
            continue

        folder_name = os.path.join("full_deploy", dep.context, dep.ref.name, str(dep.ref.
↪version))
        build_type = dep.info.settings.get_safe("build_type")
        arch = dep.info.settings.get_safe("arch")
        if build_type:
            folder_name = os.path.join(folder_name, build_type)
        if arch:
            folder_name = os.path.join(folder_name, arch)

        new_folder = os.path.join(output_folder, folder_name)
        rmdir(conanfile, new_folder)
        try:
            shutil.copytree(dep.package_folder, new_folder, symlinks=symlinks)
        except Exception as e:
            raise ConanException(f"full_deploy: The copy of '{dep}' files failed: {e}.\n"
                                 f"You can use 'tools.deployer:symlinks' conf to disable␣
↪symlinks")
        # Generators running after the deployer will point to the deployed folder
        dep.set_deploy_folder(new_folder)
```

Then it can be used as any other deployer:

```
$ conan install . --deployer=my_full_deploy
```

Some notes about the code:

- `dep.package_folder` can be `None` for skipped binaries, so any deployer needs to check it.
- `dep.recipe == "Editable"` is a valid way to detect editable packages. It can also be used to implement other behaviors, like deploying only the artifacts defined in the `cpp_info` of the editable package (`dep.cpp_info.includedirs`, `dep.cpp_info.libdirs`, etc.) instead of the whole folder.
- Generators like `CMakeDeps` do not need conditionals for editables: the `layout()` method makes `cpp_info` work transparently for packages in the cache and editable packages.

8.2.4 Package Signing Plugin

Signing packages with OpenSSL

This is an example of a Package Signing Plugin implementation using the OpensSSL digest tool. You will need to have `openssl` installed at the system level and available in your `PATH`.

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This example is available in the examples2 repository: examples/extensions/plugins/openssl_sign.

Note: OpenSSL is used here for demonstration purposes only. The Package Signing plugin mechanism is backend-agnostic, and you could implement a similar plugin using other tools available in your system (for example, `gpg`), with minimal changes to the signing and verification commands.

Generating the signing keys

To sign and verify the packages using the plugin, first, we will need a public and private key.

To generate the keys using the `openssl` executable, we can run:

```
$ openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

This will generate the private key used to sign the packages.

Now, we can get the public key from it with this command:

```
$ openssl pkey -in private_key.pem -pubout -out public_key.pem
```

The plugin will use this public key to verify the packages.

Configuring the plugin

Caution: This example stores a private key next to the plugin for simplicity. **Do not do this in production**. Instead, load the signing key from environment variables or a secret manager, or delegate signing to a remote signing service. **Always keep the private key out of the Conan cache and out of source control**.

1. Copy the `examples/extensions/plugins/openssl_sign/sign.py` file to your Conan home at `CONAN_HOME/extensions/plugins/sign/sign.py`.

1. Copy the `sign.py` file to your Conan home:

   `CONAN_HOME/extensions/plugins/sign/sign.py`

2. Place the generated keys in a folder named after your provider (`my-organization` in this example), next to `sign.py`:

Your final folder structure should look like this:

```
CONAN_HOME/
└── extensions/
    └── plugins/
        └── sign/
            ├── sign.py
            └── my-organization/
                ├── private_key.pem
                └── public_key.pem
```

The `my-organization` folder serves as the **provider** in this example, and it is used by the plugin to identify the organization that owns the keys.

Tip: The Package Signing plugin is installed in the Conan configuration folder, so they can be easily distributed as part of the client configuration using the *conan config install* command.

Implementation

The plugin's implementation is very straightforward.

For signing packages, the *sign()* function is defined, where the packages are signed by the **`openssl dgst`** command:

```
def sign(ref, artifacts_folder, signature_folder, **kwargs)
    ...
    openssl_sign_cmd = [
        "openssl",
        "dgst",
        "-sha256",
        "-sign", privkey_filepath,
        "-out", signature_filepath,
        manifest_filepath
    ]
    try:
        _run_command(openssl_sign_cmd)
```

(continues on next page)

(continued from previous page)

```
        ConanOutput().success(f"Package signed for reference {ref}")
    except Exception as exc:
        raise ConanException(f"Error signing artifact: {exc}")
    ...
```

There, the manifest `pkgsign-manifest.json` (created right before `sign()` function is called) is used to sign the package, as it contains the filenames and checksums of the artifacts of the package.

The signature file is saved into the `signature_filepath` (the signature folder at `<package_folder>/metadata/sign`), and finally, the metadata of the signature is returned as a dictionary in a list:

```
def sign(ref, artifacts_folder, signature_folder, **kwargs)
    ...
    return [{"method": "openssl-dgst",
         "provider": "my-organization",
         "sign_artifacts": {
            "manifest": "pkgsign-manifest.json",
            "signature": signature_filename}}]
```

This information saved in a file `pkgsign-signatures.json` placed in the signature folder, so it can be used in the *verify()* to verify the package signature against the correct provider keys, with the correct signing method (`openssl-dgst` for this example) and using the signature files in `sign_artifacts`.

For verifying packages, the *verify()* function is defined.

First, the `pkgsign-signatures.json` is loaded to read the metadata of the signatures (multiple signatures are supported):

```
def verify(ref, artifacts_folder, signature_folder, files, **kwargs):
    ...
    signatures = json.loads(f.read()).get("signatures")
    ...
    for signature in signatures:
        signature_filename = signature.get("sign_artifacts").get("signature")
        signature_filepath = os.path.join(signature_folder, signature_filename)
        ...
        provider = signature.get("provider")
        signature_method = signature.get("method")
        ...
```

Then, the `provider` information is used to select the correct public key for verification that use the right signature verification `method` (`openssl-dgst` for this example) and run the **`openssl dgst -verify`** command:

```
def verify(ref, artifacts_folder, signature_folder, files, **kwargs):
    ...
    openssl_verify_cmd = [
        "openssl",
        "dgst",
        "-sha256",
        "-verify", pubkey_filepath,
        "-signature", signature_filepath,
        manifest_filepath,
    ]
    try:
```

(continues on next page)

(continued from previous page)

```
        _run_command(openssl_verify_cmd)
        ConanOutput().success(f"Package verified for reference {ref}")
    except Exception as exc:
        raise ConanException(f"Error verifying signature {signature_filepath}: {exc}")
```

The `verify()` function does not return any value in case the package is correct. If the verification fails, then a `ConanException()` should be raised.

Signing packages

Now that the plugin is configured, we can create a package and sign it afterwards:

```
$ conan new cmake_lib -d name=hello -d version=1.0
$ conan create
```

For signing the recipe and package, use the dedicated command:

```
$ conan cache sign hello/1.0

hello/1.0: Compressing conan_sources.tgz
hello/1.0:dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b: Compressing conan_package.tgz
Running command: openssl dgst -sha256 -sign C:\Users\user\.conan2\extensions\plugins\
↪sign\my-organization\private_key.pem -out C:\Users\user\.conan2\p\hello092ffa809a9a1\d\
↪metadata\sign\pkgsign-manifest.json.sig C:\Users\user\.conan2\p\hello092ffa809a9a1\d\
↪metadata\sign\pkgsign-manifest.json
Package signed for reference hello/1.0
Running command: openssl dgst -sha256 -sign C:\Users\user\.conan2\extensions\plugins\
↪sign\my-organization\private_key.pem -out C:\Users\user\.conan2\p\b\hello5b13c694fef4a\
↪d\metadata\sign\pkgsign-manifest.json.sig C:\Users\user\.conan2\p\b\hello5b13c694fef4a\
↪d\metadata\sign\pkgsign-manifest.json
Package signed for reference hello/1.0:dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b
[Package sign] Results:

hello/1.0
revisions
    53321bba8793db6fea5ea1a98dd6f3d6
    packages
        dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b
        revisions
            4b1eaf2e27996cb39cb3774f185fcd8e

[Package sign] Summary: OK=2, FAILED=0
```

As you see, the command is executing the `sign()` function of the plugin that uses the `openssl` executable to sign the recipe and the package with a command similar to:

```
$ openssl dgst -sha256 -sign private_key.pem -out pkgsign-manifest.json.sig pkgsign-
↪manifest.json
```

And it is also using the conan-generated `pkgsign-manifest.json` file to create the signature. You can read more about this manifest file at *Package signing*.

Verifying packages

For verifying the recipe and package, use the dedicated command:

```
$ conan cache verify hello/1.0

[Package sign] Checksum verified for file conan_sources.tgz␣
↪(4ce077cbea9ce87a481b5d6dbb50bd791f4e37e931754cdeb40aeb017baed66c).
[Package sign] Checksum verified for file conanfile.py␣
↪(0ec44c268f0f255ab59a246c3d13ae6dbd487dea7635b584236b701047f92ba0).
[Package sign] Checksum verified for file conanmanifest.txt␣
↪(f7f00bb74ed8469a367ed02faded3c763130da9b63dae23916b2a4f099625b15).
Running command: openssl dgst -sha256 -verify C:\Users\user\.conan2\extensions\plugins\
↪sign\my-organization\public_key.pem -signature C:\Users\user\.conan2\p\
↪hello092ffa809a9a1\d\metadata\sign\pkgsign-manifest.json.sig C:\Users\user\.conan2\p\
↪hello092ffa809a9a1\d\metadata\sign\pkgsign-manifest.json
Package verified for reference hello/1.0
[Package sign] Checksum verified for file conan_package.tgz␣
↪(5cc1b9e330fe5bb6ad5904db45d78ecd6bdc71bcc18eff8d19a1ed126ba5a5aa).
[Package sign] Checksum verified for file conaninfo.txt␣
↪(f80367b17176346e10640ed813d6d2f1c45ed526822ff71066696179d16e2f2f).
[Package sign] Checksum verified for file conanmanifest.txt␣
↪(91429ce32c2d0a99de6459a589ac9c35933ed65165ee5c564b6534da57fdfa65).
Running command: openssl dgst -sha256 -verify C:\Users\user\.conan2\extensions\plugins\
↪sign\my-organization\public_key.pem -signature C:\Users\user\.conan2\p\b\
↪hello5b13c694fef4a\d\metadata\sign\pkgsign-manifest.json.sig C:\Users\user\.conan2\p\b\
↪hello5b13c694fef4a\d\metadata\sign\pkgsign-manifest.json
Package verified for reference hello/1.0:dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b
[Package sign] Results:

hello/1.0
revisions
    53321bba8793db6fea5ea1a98dd6f3d6
    packages
        dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b
        revisions
            4b1eaf2e27996cb39cb3774f185fcd8e

[Package sign] Summary: OK=2, FAILED=0
```

As you see, Conan is performing an internal checksum verification for the files and calling the `verify()` function of the plugin that uses the `openssl` executable to verify the recipe and the package with a command similar to:

```
$ openssl dgst -sha256 -verify public_key.pem -signature pkgsign-manifest.json.sig␣
↪pkgsign-manifest.json
```

See also:

If you want to create your own package signing plugin, check the reference documentation at *Package signing*.

Signing packages with Sigstore (Cosign)

This is an example of a package signing plugin implementation using Sigstore via Cosign. You need **Cosign** (version 3.0.0 or newer) on your `PATH`. See the Cosign releases page for binaries.

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This example is available in the examples2 repository: examples/extensions/plugins/sigstore_sign.

Note: Cosign is used here for demonstration only. The package signing plugin mechanism is backend-agnostic; you could implement a similar plugin with other tools (for example OpenSSL or GPG) by changing the commands invoked from `sign()` and `verify()`, as described in *Package signing*.

Generating the signing key pair

Generate a Cosign key pair (Cosign prompts for a passphrase to protect the private key):

```
$ cosign generate-key-pair --output-key-prefix signing
```

This creates `signing.key` (private) and `signing.pub` (public). Use the passphrase later to set the `COSIGN_PASSWORD` environment variable when configuring the plugin (see below).

Configuring the plugin

> **Caution:** This example stores a private key next to the plugin for simplicity. **Do not do this in production.** Instead, load the signing key from environment variables or a secret manager, or delegate signing to a remote signing service. **Always keep the private key out of the Conan cache and out of source control**.

1. Copy `sign.py` and `signing-config.json` from the examples2 folder into your Conan home:

   `CONAN_HOME/extensions/plugins/sign/sign.py`

   `CONAN_HOME/extensions/plugins/sign/signing-config.json`

2. Place the generated keys in a folder named after the **provider** used by the plugin. This example uses `my-organization` (the name is hardcoded in `sign.py`):

   `CONAN_HOME/extensions/plugins/sign/my-organization/signing.key`

   `CONAN_HOME/extensions/plugins/sign/my-organization/signing.pub`

3. Set the `COSIGN_PASSWORD` environment variable. The plugin **requires** this variable to be present when signing: Cosign reads it instead of prompting on the terminal. Set it to the **private key passphrase** you chose when generating the key pair. If the key has **no** passphrase, set `COSIGN_PASSWORD` to an empty value.

Your layout should look like this:

```
CONAN_HOME/
└── extensions/
```

(continues on next page)

(continued from previous page)

```
    └── plugins/
        └── sign/
            ├── sign.py
            ├── signing-config.json
            └── my-organization/
                ├── signing.key
                └── signing.pub
```

Tip: The package signing plugin lives under the Conan configuration directory, so you can distribute it with *conan config install* (for example from a fork or internal repo that contains the same files).

Implementation

Note: **Method name convention:** Use the literal string `sigstore` (lowercase) in the `method` field when your plugin uses this Cosign/Sigstore tools. This is a convenient way to identify the signing method used to sign the package and so the verifier can pick the right backend.

For signing, `sign()` invokes **`cosign sign-blob`** on Conan's `pkgsign-manifest.json`, writes a Sigstore **bundle** (`artifact.sigstore.json`) next to the manifest, and returns metadata for `pkgsign-signatures.json`:

```
def sign(ref, artifacts_folder, signature_folder, **kwargs):
    ...
    cosign_sign_cmd = [
        "cosign",
        "sign-blob",
        "--key",
        privkey_filepath,
        "--bundle",
        bundle_filepath,
        "-y",
        f"--signing-config={_signing_config_path()}",
        manifest_filepath,
    ]
    try:
        _run_command(cosign_sign_cmd)
        ConanOutput().success(f"Package signed for reference {ref}")
    except Exception as exc:
        raise ConanException(f"Error signing artifact: {exc}") from exc

    return [
        {
            "method": "sigstore",
            "provider": provider,
            "sign_artifacts": {
                "manifest": "pkgsign-manifest.json",
                "bundle": "artifact.sigstore.json",
            },
```

(continues on next page)

(continued from previous page)

```
        }
    ]
```

For verification, `verify()` reads `pkgsign-signatures.json`, resolves the manifest and bundle paths, loads the public key for the recorded **provider**, and runs **`cosign verify-blob`** (without Rekor support):

```
def verify(ref, artifacts_folder, signature_folder, files, **kwargs):
    ...
    cosign_verify_cmd = [
        "cosign",
        "verify-blob",
        "--key",
        pubkey_filepath,
        "--bundle",
        bundle_filepath,
        "--private-infrastructure=true",
        manifest_filepath,
    ]
    try:
        _run_command(cosign_verify_cmd)
        ConanOutput().success(f"Package verified for reference {ref}")
    except Exception as exc:
        raise ConanException(f"Error verifying signature {bundle_filepath}: {exc}") from␣
↪exc
```

If verification fails, the plugin raises `ConanException`. On success it does not return a value.

You can read more about `pkgsign-manifest.json` at *Package signing*.

Signing packages

Create a package and sign it:

```
$ conan new cmake_lib -d name=hello -d version=1.0
$ conan create
$ conan cache sign hello/1.0

hello/1.0: Compressing conan_sources.tgz
hello/1.0:dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b: Compressing conan_package.tgz
Running command: cosign sign-blob --key .../sign/my-organization/signing.key --bundle ...
↪/metadata/sign/artifact.sigstore.json -y --signing-config=.../sign/signing-config.json␣
↪.../metadata/sign/pkgsign-manifest.json
Package signed for reference hello/1.0
...
[Package sign] Summary: OK=2, FAILED=0
```

Note: Starting with Conan 2.26.0, **`conan upload`** does not sign packages automatically. Use **`conan cache sign`** before upload when remotes should store signatures. See *Package signing*.

Verifying packages

Verify recipe and package binaries in the cache:

```
$ conan cache verify hello/1.0

[Package sign] Checksum verified for file conan_sources.tgz (...)
...
Running command: cosign verify-blob --key .../sign/my-organization/signing.pub --bundle .
↪../metadata/sign/artifact.sigstore.json --private-infrastructure=true .../metadata/
↪sign/pkgsign-manifest.json
Package verified for reference hello/1.0
...
[Package sign] Summary: OK=2, FAILED=0
```

Packages downloaded from a remote are verified on install (for example **conan install**).

See also:

Plugin API and manifest details: *Package signing*.

8.2.5 Python build backend

Build a Python C/C++ extension using Conan and conan-py-build

In this example, we are going to create a small Python package with a C++ extension that uses pybind11 to expose the C++ code to Python and fmt as a regular C++ dependency. Both dependencies are downloaded and built by Conan, driven by conan-py-build, a PEP 517 build backend that lets `pip` (or `build`, or `uv`) build the C/C++ part of a Python package through Conan.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/extensions/python_build_backend
```

The project has this structure:

```
.
├── pyproject.toml
├── conanfile.py
├── CMakeLists.txt
├── LICENSE
├── src
│   └── myadder.cpp
└── python
    └── myadder
        └── __init__.py
```

Let's have a look at the *pyproject.toml* file:

Listing 8: **pyproject.toml**

```
[build-system]
requires = ["conan-py-build"]
```

(continues on next page)

(continued from previous page)

```
build-backend = "conan_py_build.build"

[project]
name = "myadder"
version = "0.1.0"
description = "A simple Python package with a C++ extension"
license = "MIT"
license-files = ["LICENSE"]
requires-python = ">=3.8"

[tool.conan-py-build.wheel]
packages = ["python/myadder"]
```

Setting `conan-py-build` as the `build-backend` means that every time this package is built (`pip wheel .`, `pip install .`, `python -m build`, etc.), the backend will read the recipe declared in the standard *conanfile.py*, resolve and install its dependencies through Conan, run the `build()` and `package()` methods, and copy everything `package()` staged into the resulting wheel.

The `[tool.conan-py-build.wheel]` section tells the backend where the pure Python part of the package lives, so it also gets copied into the wheel next to the compiled extension. See the configuration reference for all the available options (dynamic versioning, extra Conan profiles/arguments, entry points, sdist contents, etc.).

Now, the *conanfile.py* is a regular Conan recipe, no different from one used to create a C++ package:

Listing 9: **conanfile.py**

```
from conan import ConanFile
from conan.tools.cmake import CMake, cmake_layout


class MyAdderConan(ConanFile):
    name = "myadder"
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeToolchain", "CMakeDeps"

    def layout(self):
        cmake_layout(self)

    def requirements(self):
        self.requires("pybind11/3.0.1")
        self.requires("fmt/12.1.0")

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def package(self):
        cmake = CMake(self)
        cmake.install()
```

It declares **pybind11** and **fmt** as regular requirements, downloaded as precompiled binaries from ConanCenter (or built from source if needed), and uses *CMakeToolchain* and *CMakeDeps* to configure and build the extension with CMake.

Important: `conan-py-build` automatically points `CMakeToolchain` at the same Python interpreter driving the build (`Python3_EXECUTABLE`/`Python_EXECUTABLE`), so the recipe doesn't need to set it and CMake can't pick up a different, ABI-incompatible Python.

Important: For the compiled extension to be importable, it must land under a directory that matches the Python package name, so the `.so`/`.pyd` ends up next to `__init__.py` in the wheel. This is controlled by the `DESTINATION` argument of CMake's `install(TARGETS ...)`.

The *CMakeLists.txt* builds the `_core` extension module with pybind11, links it against `fmt`, and installs it into a `myadder` folder to match the Python package name:

Listing 10: **CMakeLists.txt**

```
cmake_minimum_required(VERSION 3.15)
project(myadder LANGUAGES CXX)

find_package(Python3 REQUIRED COMPONENTS Interpreter Development.Module)
set(PYBIND11_FINDPYTHON ON)
find_package(pybind11 CONFIG REQUIRED)
find_package(fmt REQUIRED)

pybind11_add_module(_core src/myadder.cpp)
target_link_libraries(_core PRIVATE pybind11::module fmt::fmt)

install(TARGETS _core DESTINATION myadder)
```

The C++ source exposes an `add()` function to Python, using **fmt** to print the result in bold green text:

Listing 11: **src/myadder.cpp**

```
#include <pybind11/pybind11.h>
#include <fmt/core.h>
#include <fmt/color.h>

namespace py = pybind11;

double add(double a, double b) {
    double result = a + b;
    fmt::print(fg(fmt::color::green) | fmt::emphasis::bold,
               "{} + {} = {}\n", a, b, result);
    return result;
}

PYBIND11_MODULE(_core, m) {
    m.doc() = "Simple Python extension using fmt via Conan.";
    m.def("add", &add, "Add two numbers and print the result formatted with fmt.",
          py::arg("a"), py::arg("b"));
}
```

And finally, *python/myadder/__init__.py* just re-exports the function from the compiled module:

Listing 12: **python/myadder/__init__.py**

```
from myadder._core import add

__all__ = ["add"]
```

Build and test

Build the wheel with `pip`. There's no need to install `conan-py-build` beforehand: since it's declared under `[build-system] requires` in *pyproject.toml*, `pip` installs it (and Conan itself) into an isolated build environment automatically:

```
$ pip wheel . -w dist/
```

Conan will install **pybind11** and **fmt**, build the extension with CMake, and `conan-py-build` will assemble the resulting wheel and save it into the *dist* folder, e.g. *dist/myadder-0.1.0-cp312-cp312-macosx_14_0_arm64.whl* (the exact filename depends on your platform and Python version).

A wheel is just a zip file, so you can check what got packaged without installing it:

```
$ python -m zipfile -l dist/myadder-*.whl
File Name                                             Modified             Size
myadder/__init__.py                            2026-07-08 14:01:30           49
myadder/_core.cpython-312-darwin.so            2026-07-08 14:41:12       233288
myadder-0.1.0.dist-info/METADATA               2026-07-08 14:41:08          212
myadder-0.1.0.dist-info/WHEEL                  2026-07-08 14:41:12          101
myadder-0.1.0.dist-info/licenses/LICENSE       2026-07-08 14:01:32         1083
myadder-0.1.0.dist-info/RECORD                 2026-07-08 14:41:12          471
```

Note how the compiled `_core` extension, the one Conan's `package()` staged during the build, ends up right next to `__init__.py` under `myadder/` — exactly the layout the `DESTINATION` in *CMakeLists.txt* was set up for.

Now install it and try it out:

```
$ pip install dist/myadder-*.whl
$ python -c "from myadder import add; add(2, 3)"
2 + 3 = 5
```

The line above is printed by the C++ extension in bold green, courtesy of **fmt** — a visible sign that the compiled code, not Python, produced it.

See also:

- *conan-py-build integration*
- conan-py-build documentation, for configuration options like dynamic versioning, custom Conan profiles, entry points, and shared library handling with `auditwheel`/`delocate`/`delvewheel`.
- Introducing conan-py-build blog post.
- conan-py-build examples, including Meson, nanobind, and cibuildwheel-based builds.

8.3 Conan recipe tools examples

8.3.1 CMake

CMakeToolchain: Building your project using CMakePresets

In this example we are going to see how to use `CMakeToolchain`, predefined layouts like `cmake_layout` and the `CMakePresets` CMake feature.

Let's create a basic project based on the template `cmake_exe` as an example of a C++ project:

```
$ conan new cmake_exe -d name=foo -d version=1.0
```

Generating the toolchain

The recipe from our project declares the generator "CMakeToolchain".

We can call **`conan install`** to install both `Release` and `Debug` configurations. Conan will generate a `conan_toolchain.cmake` at the corresponding *generators* folder:

```
$ conan install .
$ conan install . -s build_type=Debug
```

Building the project using `CMakePresets`

A `CMakeUserPresets.json` file is generated in the same folder of your `CMakeLists.txt` file, so you can use the `--preset` argument from `cmake >= 3.23` or use an IDE that supports it.

The `CMakeUserPresets.json` is including the `CMakePresets.json` files located at the corresponding *generators* folder.

The `CMakePresets.json` contain information about the `conan_toolchain.cmake` location and even the `binaryDir` set with the output directory.

Note: We use CMake presets in this example. This requires CMake >= 3.23 because the "include" from `CMakeUserPresets.json` to `CMakePresets.json` is only supported since that version. If you prefer not to use presets you can use something like:

```
cmake <path> -G <CMake generator> -DCMAKE_TOOLCHAIN_FILE=<path to
conan_toolchain.cmake> -DCMAKE_BUILD_TYPE=Release
```

Conan will show the exact CMake command everytime you run `conan install` in case you can't use the presets feature.

If you are using a multi-configuration generator:

```
$ cmake --preset conan-default
$ cmake --build --preset conan-debug
$ build\Debug\foo.exe
foo/1.0: Hello World Debug!
```

(continues on next page)

(continued from previous page)

```
$ cmake --build --preset conan-release
$ build\Release\foo.exe
foo/1.0: Hello World Release!
```

If you are using a single-configuration generator:

```
$ cmake --preset conan-debug
$ cmake --build --preset conan-debug
$ ./build/Debug/foo
foo/1.0: Hello World Debug!


$ cmake --preset conan-release
$ cmake --build --preset conan-release
$ ./build/Release/foo
foo/1.0: Hello World Release!
```

Note that we didn't need to create the `build/Release` or `build/Debug` folders, as we did *in the tutorial*. The output directory is declared by the `cmake_layout()` and automatically managed by the CMake Presets feature.

This behavior is also managed automatically by Conan (with CMake >= 3.15) when you build a package in the Conan cache (with **`conan create`** command). The CMake >= 3.23 is not required.

Read More:

- `cmake_layout()` *reference*
- Conanfile *layout() method reference*
- Package layout tutorial *tutorial*
- Understanding *Conan package layouts*

CMakeToolchain: Extending your CMakePresets with Conan generated ones

In this example we are going to see how to extend your own CMakePresets to include Conan generated ones.

Note: We use CMake presets in this example. This requires CMake >= 3.23 because the "include" from `CMakeUserPresets.json` to `CMakePresets.json` is only supported since that version. If you prefer not to use presets you can use something like:

```
cmake <path> -G <CMake generator> -DCMAKE_TOOLCHAIN_FILE=<path to
conan_toolchain.cmake> -DCMAKE_BUILD_TYPE=Release
```

Conan will show the exact CMake command everytime you run `conan install` in case you can't use the presets feature.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/cmake/cmake_toolchain/extend_own_cmake_presets
```

Please open the *conanfile.py* and check how it sets `tc.user_presets_path = 'ConanPresets.json'`. By modifying this attribute of *CMakeToolchain*, you can change the default filename of the generated preset.

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.user_presets_path = 'ConanPresets.json'
    tc.generate()
    ...
```

Now you can provide your own `CMakePresets.json`, besides the `CMakeLists.txt`:

Listing 13: CMakePresets.json

```
{
"version": 4,
"include": ["./ConanPresets.json"],
"configurePresets": [
    {
        "name": "default",
        "displayName": "multi config",
        "inherits": "conan-default"
    },
    {
        "name": "release",
        "displayName": "release single config",
        "inherits": "conan-release"
    },
    {
        "name": "debug",
        "displayName": "debug single config",
        "inherits": "conan-debug"
    }
],
"buildPresets": [
    {
        "name": "multi-release",
        "configurePreset": "default",
        "configuration": "Release",
        "inherits": "conan-release"
    },
    {
        "name": "multi-debug",
        "configurePreset": "default",
        "configuration": "Debug",
        "inherits": "conan-debug"
    },
    {
        "name": "release",
        "configurePreset": "release",
        "configuration": "Release",
        "inherits": "conan-release"
    },
    {
        "name": "debug",
        "configurePreset": "debug",
        "configuration": "Debug",
```

(continues on next page)

(continued from previous page)

```
        "inherits": "conan-debug"
    }
]
}
```

Note how the `"include": ["./ConanPresets.json"]`, and that every preset `inherits` a Conan generated one.

We can now install for both Release and Debug (and other configurations also, with the help of `build_folder_vars` if we want):

```
$ conan install .
$ conan install . -s build_type=Debug
```

And build and run our application, by using **our own presets** that extend the Conan generated ones:

```
# Linux (single-config, 2 configure, 2 builds)
$ cmake --preset debug
$ cmake --build --preset debug
$ ./build/Debug/foo
> Hello World Debug!

$ cmake --preset release
$ cmake --build --preset release
$ ./build/Release/foo
> Hello World Release!

# Windows VS (Multi-config, 1 configure 2 builds)
$ cmake --preset default

$ cmake --build --preset multi-debug
$ build\\Debug\\foo
> Hello World Debug!

$ cmake --build --preset multi-release
$ build\\Release\\foo
> Hello World Release!
```

CMakeToolchain: Inject arbitrary CMake variables into dependencies

You can find the sources to recreate this project in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/cmake/cmake_toolchain/user_toolchain_profile
```

In the general case, Conan package recipes provide the necessary abstractions via settings, confs, and options to control different aspects of the build. Many recipes define `options` to activate or deactivate features, optional dependencies, or binary characteristics. Configurations like `tools.build:cxxflags` can be used to inject arbitrary C++ compile flags.

In some exceptional cases, it might be desired to inject CMake variables directly into dependencies doing CMake builds. This is possible when these dependencies use the `CMakeToolchain` integration. Let's check it in this simple example.

If we have the following package recipe, with a simple `conanfile.py` and a `CMakeLists.txt` printing a variable:

Listing 14: conanfile.py

```
from conan import ConanFile
from conan.tools.cmake import CMake

class AppConan(ConanFile):
    name = "foo"
    version = "1.0"
    settings = "os", "compiler", "build_type", "arch"
    exports_sources = "CMakeLists.txt"

    generators = "CMakeToolchain"

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
```

Listing 15: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(foo LANGUAGES NONE)
message(STATUS "MYVAR1 ${MY_USER_VAR1}!!")
```

We can define a profile file and a `myvars.cmake` file (both in the same folder) like the following:

Listing 16: myprofile

```
include(default)
[conf]
tools.cmake.cmaketoolchain:user_toolchain+={{profile_dir}}/myvars.cmake
```

Note the `{{profile_dir}}` is a jinja template expression that evaluates to the current profile folder, allowing to compute the necessary path to `myvars.cmake` file. The `tools.cmake.cmaketoolchain:user_toolchain` is a **list** of files to inject to the generated `conan_toolchain.cmake`, so the += operator is used to append to it.

The `myvars.cmake` can define as many variables as we want:

Listing 17: myvars.cmake

```
set(MY_USER_VAR1 "MYVALUE1")
```

Applying this profile, we can see that the package CMake build effectively uses the variable provided in the external `myvars.cmake` file:

```
$ conan create . -pr=myprofile
...
-- MY_USER_VAR1 MYVALUE1
```

Note that using `user_toolchain` while defining values for confs like `tools.cmake.cmaketoolchain:system_name` is supported.

Also, `user_toolchain` files can define variables for cross-building, such as `CMAKE_SYSTEM_NAME`, `CMAKE_SYSTEM_VERSION` and `CMAKE_SYSTEM_PROCESSOR`. If these variables are defined in the user toolchain file, they will be respected, and the `conan_toolchain.cmake` deduced ones will not overwrite the user defined ones.

If those variables are not defined in the user toolchain file, then the Conan automatically deduced ones will be used.

The `tools.cmake.cmaketoolchain:user_toolchain` conf value might also be passed in the command line `-c` argument, but the location of the `myvars.cmake` needs to be absolute to be found, as jinja replacement doesn't happen in the command line.

CMakeToolchain: Using xxx-config.cmake files inside packages

Conan relies in the general case in the `package_info()` abstraction to allow packages built with any build system to be usable from any other package built with any other build system. In the CMake case, Conan relies on the `CMakeDeps` generator to generate `xxxx-config.cmake` files for every dependency, even if those dependencies didn't generate one or aren't built with CMake at all.

ConanCenter users this abstraction, not packaging the `xxx-config.cmake` files, and using the information in `package_info()`. This is very important to provide as build-system agnostic as possible packages and be fair with different build systems, vendors and users. For example, there are many Conan users happily using native MSBuild (VS) projects without any CMake at all. If ConanCenter packages were only built using the in-package `config.cmake` files, this wouldn't be possible.

But the fact that ConanCenter does that, doesn't mean that this is not possible or mandatory. It is perfectly possible to use the in-packages `xxx-config.cmake` files, dropping the usage of `CMakeDeps` generator.

You can find the sources to recreate this example in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/cmake/pkg_config_files
```

If we have a look to the `conanfile.py`:

```
class pkgRecipe(ConanFile):
    name = "pkg"
    version = "0.1"
    ...

    def package_info(self):
        # No information provided, only the in-package .cmake is used here
        # Other build systems or CMake via CMakeDeps will fail
        self.cpp_info.builddirs = ["pkg/cmake"]
        self.cpp_info.set_property("cmake_find_mode", "none")
```

This is a very typical recipe, the main difference is the `package_info()` method. Three important things to notice:

- It doesn't define fields like `self.cpp_info.libs = ["mypkg"]`. Conan will not be propagating this information to the consumer, the only place this information will be is inside the in-package `xxx-config.cmake` file
- Just in case there are some users still instantiating `CMakeDeps`, it is disabling the client side generation of the `xxx-config.cmake` file with `set_property("cmake_find_mode", "none")`
- It is defining that it will contain the build scripts (like the `xxx-config.cmake` package) inside that folder, to be located by consumers.

Note: If the in-package `xxx-config.cmake` file has a different name than the Conan package name, set the `cmake_file_name` property with the right value, so that `find_package()` can locate it:

```
self.cpp_info.set_property("cmake_file_name", "MyPkg")
```

So the responsibility of defining the package details has been transferred to the `CMakeLists.txt` that contains:

```
add_library(mylib src/pkg.cpp)  # Use a different name than the package, to make sure

set_target_properties(mylib PROPERTIES PUBLIC_HEADER "include/pkg.h")
target_include_directories(mylib PUBLIC
        $<BUILD_INTERFACE:${PROJECT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:${CMAKE_INSTALL_INCLUDEDIR}>
    )

# Use non default mypkgConfig name
install(TARGETS mylib EXPORT mypkgConfig)
export(TARGETS mylib
    NAMESPACE mypkg::  # to simulate a different name and see it works
    FILE "${CMAKE_CURRENT_BINARY_DIR}/mypkgConfig.cmake"
)
install(EXPORT mypkgConfig
    DESTINATION "${CMAKE_INSTALL_PREFIX}/pkg/cmake"
    NAMESPACE mypkg::
)
```

With that information, when `conan create` is executed:

- The `build()` method will build the package
- The `package()` method will call `cmake install`, which will create the `mypkgConfig.cmake` file
- It will be created in the package folder `pkg/cmake/mypkgConfig.cmake` file
- It will contain enough information for the headers, and it will create a `mypkg::mylib` target.

Note that the details of the config filename, the namespace and the target are also not known by Conan, so this is also something that the consumer build scripts should know.

This is enough to have a package with an internal `mypkgConfig.cmake` file that can be used by consumers. In this example code, the consumer is just the `test_package/conanfile.py`, but exactly the same wouldn apply to any arbitrary consumer.

The consumer `conanfile.py` doesn't need to use `CMakeDeps` at all, only `generators = "CMakeToolchain"`. Note that the `CMakeToolchain` generator is still necessary, because the `mypkgConfig.cmake` needs to be found inside the Conan cache. The `CMakeToolchain` generated `conan_toolchain.cmake` file contains these paths defined.

The consumer `CMakeLists.txt` would be standard:

```
find_package(mypkg CONFIG REQUIRED)

add_executable(example src/example.cpp)
target_link_libraries(example mypkg::mylib)
```

You can verify it works with:

```
$ conan create .

======== Testing the package: Executing test ========
pkg/0.1 (test package): Running test()
pkg/0.1 (test package): RUN: Release\example
```

(continues on next page)

(continued from previous page)

```
pkg/0.1: Hello World Release!
pkg/0.1: _M_X64 defined
pkg/0.1: MSVC runtime: MultiThreadedDLL
pkg/0.1: _MSC_VER1939
pkg/0.1: _MSVC_LANG201402
pkg/0.1: __cplusplus199711
pkg/0.1 test_package
```

Important considerations

The presented approach has one limitation, it doesn't work for multi-configuration IDEs. Implementing this approach won't allow developers to directly switch from IDEs like Visual Studio from Release to Debug and vice versa, and it will require a `conan install` to change. It is not an issue at all for single-config setups, but for VS developers it can be a bit inconvenient. The team is working on the VS plugin that might help to mitigate this. The reason is a CMake limitation, `find_package()` can only find one configuration, and with `CMakeDeps` being dropped here, there is nothing that Conan can do to avoid this limitation.

It is important to know that it is also the package author and the package `CMakeLists.txt` responsibility to correctly manage transitivity to other dependencies, and this is not trivial in some cases. There are risks that if not done correctly the in-package `xxx-config.cmake` file can locate its transitive dependencies elsewhere, like in the system, but not in the transitive Conan package dependencies.

Finally, recall that these packages won't be usable by other build systems rather than CMake.

Using CMakeToolchain with different generators: Ninja example

This guide demonstrates how to use *CMakeToolchain* with predefined generators like Ninja and how to configure it to use different generators.

Note: We assume you have already installed Ninja in your system. In case you do not have Ninja installed in your system, you can use the Ninja Conan package in your profile (default or custom) by adding *tool-requires*.

Understanding CMake generators

The CMake client offers a variety of generators to create build system files. If you want to use a generator other than the default chosen by CMake, you can configure `tools.cmake.cmaketoolchain:generator`.

Note: Please, note that CMake client is not the same as the Conan *CMake* helper.

To see which generators are available on your system, run:

```
$ cmake --help
```

You can set this *configuration in your profile*, directly in the command line, or even in your *global configuration*.

Using the Ninja generator by default in a profile

First, let's create a profile file name `my_custom_profile`, so we can set the Ninja generator as the default for all Conan packages built with this profile.

```
$ conan profile detect --name=my_custom_profile
```

To set the Ninja generator as the default in `my_custom_profile` profile, add the entry `[conf]` with the generator value in the file:

```
[settings]
os=Linux
arch=x86_64
compiler=gcc
compiler.version=13
compiler.libcxx=libstdc++11
compiler.cppstd=20
build_type=Release

[conf]
tools.cmake.cmaketoolchain:generator=Ninja
```

Now, we will create a basic project based on the `cmake_exe` template as an example of a C++ project:

```
$ conan new cmake_exe -d name=foo -d version=0.1.0
```

Then, we can build your project using the profile we just created:

```
$ conan create . -pr=my_custom_profile
```

This configuration will be passed to the `conan_toolchain.cmake` file, generated by `CMakeToolchain`, then the Ninja generator will be used. You should see the following output snippet indicating the Ninja generator is being used:

```
Profile host:
[settings]
...
[conf]
tools.cmake.cmaketoolchain:generator=Ninja

...
foo/0.1.0: Calling build()
foo/0.1.0: Running CMake.configure()
foo/0.1.0: RUN: cmake -G "Ninja" ...
```

Note that same configuration can be passed to the default profile, and used for all Conan packages built with that profile.

In case passing the generator configuration by command line, the same will override the profile configuration.

CMakeToolchain: Using LLVM/Clang Windows compiler

The Clang compiler in Windows can come from 2 different installations or distributions:

- The LLVM/Clang compiler, that uses the MSVC runtime
- The Msys2 Clang compiler that uses the Msys2 runtime (libstdc++6.dll)

This example explains the LLVM/Clang with the MSVC runtime. This Clang distribution can in turn be used in three different ways:

- Using the Clang component installed by Visual Studio installer as part of VS
- Using the LLVM/Clang downloaded compiler (it still uses the MSVC runtime), via the GNU-like frontend `clang`
- Using the LLVM/Clang downloaded compiler (it still uses the MSVC runtime), via the MSVC-like frontend `clang-cl`

Let's start from a simple `cmake_exe` template:

```
$ conan new cmake_exe -d name=mypkg -d version=0.1
```

This creates a simple CMake based project and Conan package recipe that uses `CMakeToolchain`.

LLVM/Clang with `clang` GNU-like frontend

To build this configuration we will use the following profile:

Listing 18: llvm_clang

```
[settings]
os=Windows
arch=x86_64
build_type=Release
compiler=clang
compiler.version=18
compiler.cppstd=14
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.runtime_version=v144

[buildenv]
PATH=+(path)C:/ws/LLVM/18.1/bin

[conf]
tools.cmake.cmaketoolchain:generator=Ninja
tools.compilation:verbosity=verbose

[tool_requires]
ninja/[*]
```

Quick explanation of the profile:

- The `compiler.runtime` definition is the important differentiator to distinguish between Msys2-Clang and LLVM/Clang with the MSVC runtime. The LLVM/Clang defines this `compiler.runtime`, while the Msys2-Clang doesn't.

- The MSVC runtime can be either dynamic or static. It is important also to define the runtime version (toolset version `v144`) of this runtime, as it is possible to use different ones.
- The `[buildenv]` allows to point to the LLVM/Clang compiler, in case it is not already in the path. **Note** the `PATH=+(path)` syntax, to **prepend** that path, so it has higher priority, otherwise it is possible that CMake would find and use the Clang component installed inside Visual Studio.
- We are using the `Ninja` CMake generator, and installing it from a `[tool_requires]`, but this might not be necessary if Ninja is installed in your system.

Let's build it:

```
$ conan build . -pr=llvm_clang
...
-- The CXX compiler identification is Clang 18.1.8 with GNU-like command-line
-- Check for working CXX compiler: C:/ws/LLVM/18.1/bin/clang++.exe - skipped
...
[1/3] C:\ws\LLVM\18.1\bin\clang++.exe   -O3 -DNDEBUG -std=c++14 -D_DLL -D_MT -Xclang --
↪dependent-lib=msvcrt -MD -MT CMakeFiles/mypkg.dir/src/main.cpp.obj -MF CMakeFiles\
↪mypkg.dir\src\main.cpp.obj.d -o CMakeFiles/mypkg.dir/src/main.cpp.obj -c C:/Users/
↪Diego/conanws/kk/clang/src/main.cpp
[2/3] C:\ws\LLVM\18.1\bin\clang++.exe   -O3 -DNDEBUG -std=c++14 -D_DLL -D_MT -Xclang --
↪dependent-lib=msvcrt -MD -MT CMakeFiles/mypkg.dir/src/mypkg.cpp.obj -MF CMakeFiles\
↪mypkg.dir\src\mypkg.cpp.obj.d -o CMakeFiles/mypkg.dir/src/mypkg.cpp.obj -c C:/Users/
↪Diego/conanws/kk/clang/src/mypkg.cpp
[3/3] cmd.exe /C "cd . && C:\ws\LLVM\18.1\bin\clang++.exe -fuse-ld=lld-link -
↪nostartfiles -nostdlib -O3 -DNDEBUG -D_DLL -D_MT -Xclang --dependent-lib=msvcrt -
↪Xlinker /subsystem:console CMakeFiles/mypkg.dir/src/mypkg.cpp.obj CMakeFiles/mypkg.dir/
↪src/main.cpp.obj -o mypkg.exe -Xlinker /MANIFEST:EMBED -Xlinker /implib:mypkg.lib -
↪Xlinker /pdb:mypkg.pdb -Xlinker /version:0.0   -lkernel32 -luser32 -lgdi32 -lwinspool -
↪lshell32 -lole32 -loleaut32 -luuid -lcomdlg32 -ladvapi32 -loldnames  && cd ."
```

See how the desired LLVM/Clang compiler installed in the `C:/ws` folder is used, and how the `GNU-like` command line syntax is used. The GNU-like syntax requires the `--dependent-lib=msvcrt` (added automatically by CMake) compiler and linker flags to define linking against the dynamic MSVC runtime, as otherwise LLVM/Clang link it statically. Also note that the `-MD -MT` flags are not related to the <MSVC runtime, in the GNU-like frontend, they have a completely different meaning.

We can run our executable, and see how the Clang compiler version and the MSVC runtime match the defined ones:

```
$ build\Release\mypkg.exe
mypkg/0.1: Hello World Release!
    mypkg/0.1: _M_X64 defined
    mypkg/0.1: __x86_64__ defined
    mypkg/0.1: MSVC runtime: MultiThreadedDLL
    mypkg/0.1: _MSC_VER1943
    mypkg/0.1: _MSVC_LANG201402
    mypkg/0.1: __cplusplus201402
    mypkg/0.1: __clang_major__18
    mypkg/0.1: __clang_minor__1
```

LLVM/Clang with `clang-cl` MSVC-like frontend

To build this configuration we will use the following profile:

Listing 19: llvm_clang_cl

```
[settings]
os=Windows
arch=x86_64
build_type=Release
compiler=clang
compiler.version=18
compiler.cppstd=14
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.runtime_version=v144

[buildenv]
PATH=+(path)C:/ws/LLVM/18.1/bin

[conf]
tools.cmake.cmaketoolchain:generator=Ninja
tools.build:compiler_executables = {"c": "clang-cl", "cpp": "clang-cl"}
tools.compilation:verbosity=verbose

[tool_requires]
ninja/[*]
```

The profile is almost identical to the above one, the main difference is the definition of `tools.build:compiler_executables`, defining the `clang-cl` compiler.

Note: The definition of `tools.build:compiler_executables` using the `clang-cl` compiler is what is used by Conan to differentiate among the frontends, also in other build systems. This frontend is not a `setting`, because the compiler is still the same, and the resulting binary should be binary compatible.

Let's build it:

```
$ conan build . -pr=llvm_clang_cl
...
-- The CXX compiler identification is Clang 18.1.8 with MSVC-like command-line
-- Check for working CXX compiler: C:/ws/LLVM/18.1/bin/clang-cl.exe - skipped
...
[1/3] C:\ws\LLVM\18.1\bin\clang-cl.exe  /nologo -TP   /DWIN32 /D_WINDOWS /GR /EHsc /O2 /
↪Ob2 /DNDEBUG -std:c++14 -MD /showIncludes /FoCMakeFiles\mypkg.dir\src\main.cpp.obj /
↪FdCMakeFiles\mypkg.dir\ -c -- C:\project\src\main.cpp
[2/3] C:\ws\LLVM\18.1\bin\clang-cl.exe  /nologo -TP   /DWIN32 /D_WINDOWS /GR /EHsc /O2 /
↪Ob2 /DNDEBUG -std:c++14 -MD /showIncludes /FoCMakeFiles\mypkg.dir\src\mypkg.cpp.obj /
↪FdCMakeFiles\mypkg.dir\ -c -- C:\project\src\mypkg.cpp
[3/3] cmd.exe /C "cd . && C:\ws\cmake\cmake-3.27.9-windows-x86_64\bin\cmake.exe -E vs_
↪link_exe --intdir=CMakeFiles\mypkg.dir --rc=C:\PROGRA~2\WI3CF2~1\10\bin\100226~1.0\x64\
↪rc.exe --mt=C:\PROGRA~2\WI3CF2~1\10\bin\100226~1.0\x64\mt.exe --manifests  -- C:\ws\
↪LLVM\18.1\bin\lld-link.exe /nologo CMakeFiles\mypkg.dir\src\mypkg.cpp.obj CMakeFiles\
```

(continues on next page)

(continued from previous page)

```
↪mypkg.dir\src\main.cpp.obj  /out:mypkg.exe /implib:mypkg.lib /pdb:mypkg.pdb /version:0.
↪0 /machine:x64 /INCREMENTAL:NO /subsystem:console  kernel32.lib user32.lib gdi32.lib␣
↪winspool.lib shell32.lib ole32.lib oleaut32.lib uuid.lib comdlg32.lib advapi32.lib &&␣
↪cd ."
```

See how the desired LLVM/Clang compiler installed in the `C:/ws` folder is used, and how the `MSVC-like` command line syntax is used. This MSVC-like syntax uses the `-MD/-MT` flags to differentiate across the dynamic/static MSVC runtimes.

We can run our executable, and see how the Clang compiler version and the MSVC runtime match the defined ones:

```
$ build\Release\mypkg.exe
mypkg/0.1: Hello World Release!
    mypkg/0.1: _M_X64 defined
    mypkg/0.1: __x86_64__ defined
    mypkg/0.1: MSVC runtime: MultiThreadedDLL
    mypkg/0.1: _MSC_VER1943
    mypkg/0.1: _MSVC_LANG201402
    mypkg/0.1: __cplusplus201402
    mypkg/0.1: __clang_major__18
    mypkg/0.1: __clang_minor__1
```

As expected, the output is identical to the previous one, as nothing changed except the compiler frontend.

MSVC Clang component (ClangCL Visual Studio toolset)

To build this configuration we will use the following profile:

Listing 20: llvm_clang_vs

```
[settings]
os=Windows
arch=x86_64
build_type=Release
compiler=clang
compiler.version=19
compiler.cppstd=14
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.runtime_version=v144

[conf]
tools.cmake.cmaketoolchain:generator=Visual Studio 17
tools.compilation:verbosity=verbose
```

This profile will use the CMake "Visual Studio" generator. This indicates that the Clang compiler will be the one provided by Visual Studio, and installed as a component of Visual Studio via the Visual Studio installer. Note the `compiler.version=19` is a different version than the one used above, which was `compiler.version=18`, as the version inside Visual is defined automatically by its installer.

This setup will always use the MSVC-like `clang-cl` frontend, and the `ClangCL` toolset will be activated to let Visual Studio that this is the compiler that it should use. It is not necessary to define the `tools.build:compiler_executable` here.

Let's build it:

```
$ conan build . -pr=llvm_clang_vs
...
-- Conan toolchain: CMAKE_GENERATOR_TOOLSET=ClangCL
-- The CXX compiler identification is Clang 19.1.1 with MSVC-like command-line
...
ClCompile:
    C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\Llvm\x64\bin\clang-
↪cl.exe /c /nologo /W1 /WX- /diagnostics:column /O2 /Ob2 /D _MBCS /D WIN
32 /D _WINDOWS /D NDEBUG /D "CMAKE_INTDIR=\"Release\"" /EHsc /MD /GS /fp:precise /GR /
↪std:c++14 /Fo"mypkg.dir\Release\\" /Gd /TP --target=amd64-pc-windows-
msvc  C:\project\src\mypkg.cpp C:\project\src\main.cpp
Link:
C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\Llvm\x64\bin\lld-link.
↪exe /OUT:"C:\project\build\Release\mypkg.exe" /
INCREMENTAL:NO kernel32.lib user32.lib gdi32.lib winspool.lib shell32.lib ole32.lib␣
↪oleaut32.lib uuid.lib comdlg32.lib advapi32.lib /MANIFEST /MANIFESTUAC:
"level='asInvoker' uiAccess='false'" /manifest:embed /PDB:"C:/Users/Diego/conanws/kk/
↪clang/build/Release/mypkg.pdb" /SUBSYSTEM:CONSOLE /DYNAMICBASE /NXCOMP
AT /IMPLIB:"C:/Users/Diego/conanws/kk/clang/build/Release/mypkg.lib"   /machine:x64␣
↪mypkg.dir\Release\mypkg.obj
mypkg.dir\Release\main.obj
mypkg.vcxproj -> C:\project\build\Release\mypkg.exe
```

The `CMAKE_GENERATOR_TOOLSET=ClangCL` is defined, and also the internal VS Clang component is used, and the `19.1.1` version is also displayed. Then, the regular `MSVC-like` syntax, including the definition of the runtime via `/MD` flags is used.

We can run our executable, and see how the Clang compiler version (19) and the MSVC runtime match the defined ones:

```
$ build\Release\mypkg.exe
mypkg/0.1: Hello World Release!
    mypkg/0.1: _M_X64 defined
    mypkg/0.1: __x86_64__ defined
    mypkg/0.1: MSVC runtime: MultiThreadedDLL
    mypkg/0.1: _MSC_VER1943
    mypkg/0.1: _MSVC_LANG201402
    mypkg/0.1: __cplusplus201402
    mypkg/0.1: __clang_major__19
    mypkg/0.1: __clang_minor__1
```

8.3.2 File interaction

Patching sources

In this example we are going to see how to patch the source code. This is necessary sometimes, specially when you are creating a package for a third party library. A patch might be required in the build system scripts or even in the source code of the library if you want, for example, to apply a security patch.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples/tools/files/patches
```

Patching using 'replace_in_file'

The simplest way to patch a file is using the `replace_in_file` tool in your recipe. It searches in a file the specified string and replaces it with another string.

in source() method

The source() method is called only once for all the configurations (different calls to **`conan create`** for different settings/options) so you should patch only in the `source()` method if the changes are common for all the configurations.

Look at the `source()` method at the `conanfile.py`:

```
import os
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout
from conan.tools.files import get, replace_in_file


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    def source(self):
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",␣
↪strip_root=True)
        replace_in_file(self, os.path.join(self.source_folder, "src", "hello.cpp"),
↪"Hello World", "Hello Friends!")

    ...
```

We are replacing the `"Hello World"` string with "Hello Friends!". We can run `conan create .` and verify that if the replace was done:

```
$ conan create .
...
-------- Testing the package: Running test() --------
hello/1.0: Hello Friends! Release!
...
```

Regular expressions and backreferences are also supported by enabling `regex`. For example, the following call captures the numeric part of `version=123` with `(\d+)` and reuses it in the replacement with `\1`, producing `version=v123`:

```
replace_in_file(self, "file.txt", r"version=(\d+)", r"version=v\1", regex=True)
```

in build() method

In this case, we need to apply a different patch depending on the configuration (*self.settings*, *self.options*...), so it has to be done in the `build()` method. Let's modify the recipe to introduce a change that depends on the `self.options.shared`:

```
class helloRecipe(ConanFile):

    ...

    def source(self):
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",␣
↪strip_root=True)

    def build(self):
        replace_in_file(self, os.path.join(self.source_folder, "src", "hello.cpp"),
                        "Hello World",
                        "Hello {} Friends!".format("Shared" if self.options.shared else
↪"Static"))
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    ...
```

If we call `conan create` with different `option.shared` we can check the output:

```
$ conan create .
...
hello/1.0: Hello Static Friends! Release!
...

$ conan create . -o shared=True
...
hello/1.0: Hello Shared Friends! Debug!
...
```

Patching using "patch" tool

If you have a patch file (diff between two versions of a file), you can use the `conan.tools.files.patch` tool to apply it. The rules about where to apply the patch (`source()` or `build()` methods) are the same.

We have this patch file, where we are changing again the message to say "Hello Patched World Release!":

```
--- a/src/hello.cpp
+++ b/src/hello.cpp
@@ -3,9 +3,9 @@

 void hello(){
     #ifdef NDEBUG
-    std::cout << "hello/1.0: Hello World Release!\n";
+    std::cout << "hello/1.0: Hello Patched World Release!\n";
```

(continues on next page)

(continued from previous page)

```
     #else
-    std::cout << "hello/1.0: Hello World Debug!\n";
+    std::cout << "hello/1.0: Hello Patched World Debug!\n";
     #endif

     // ARCHITECTURES
```

Edit the `conanfile.py` to:

1. Import the `patch` tool.
2. Add `exports_sources` to the patch file so we have it available in the cache.
3. Call the `patch` tool.

```
import os
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout
from conan.tools.files import get, replace_in_file, patch


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}
    exports_sources = "*.patch"

    def source(self):
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",␣
↪strip_root=True)
        patch_file = os.path.join(self.export_sources_folder, "hello_patched.patch")
        patch(self, patch_file=patch_file)

    ...
```

We can run "conan create" and see that the patch worked:

```
$ conan create .
...
-------- Testing the package: Running test() --------
hello/1.0: Hello Patched World Release!
...
```

We can also use the `conandata.yml` *introduced in the tutorial* so we can declare the patches to apply for each version:

```
patches:
  "1.0":
    - patch_file: "hello_patched.patch"
```

And there are the changes we introduce in the `source()` method:

```
.. code-block:: python

    def source(self):
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",␣
↪strip_root=True)
        patches = self.conan_data["patches"][self.version]
        for p in patches:
            patch_file = os.path.join(self.export_sources_folder, p["patch_file"])
            patch(self, patch_file=patch_file)
```

Check *patch* for more details.

If we run the **conan create**, the patch is also applied:

```
$ conan create .
...
-------- Testing the package: Running test() --------
hello/1.0: Hello Patched World Release!
...
```

Patching using "apply_conandata_patches" tool

The example above works but it is a bit complex. If you follow the same yml structure (check the *apply_conandata_patches* to see the full supported yml) you only need to call `apply_conandata_patches`:

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain, CMake, cmake_layout
from conan.tools.files import get, apply_conandata_patches


class helloRecipe(ConanFile):
    name = "hello"
    version = "1.0"

    ...

    def source(self):
        get(self, "https://github.com/conan-io/libhello/archive/refs/heads/main.zip",␣
↪strip_root=True)
        apply_conandata_patches(self)
```

Let's check if the patch is also applied:

```
$ conan create .
...
-------- Testing the package: Running test() --------
hello/1.0: Hello Patched World Release!
...
```

8.3.3 Meson

Build a simple Meson project using Conan

In this example, we are going to create a string compressor application that uses one of the most popular C++ libraries: Zlib.

Note: This example is based on the main *Build a simple CMake project using Conan* tutorial. So we highly recommend reading it before trying out this one.

We'll use Meson as build system and pkg-config as helper tool in this case, so you should get them installed before going forward with this example.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/meson/mesontoolchain/simple_meson_project
```

We start from a very simple C language project with this structure:

```
.
├── meson.build
└── src
    └── main.c
```

This project contains a basic *meson.build* including the **zlib** dependency and the source code for the string compressor program in *main.c*.

Let's have a look at the *main.c* file:

Listing 21: **main.c**

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include <zlib.h>

int main(void) {
    char buffer_in [256] = {"Conan is a MIT-licensed, Open Source package manager for C␣
↪and C++ development "
                            "for C and C++ development, allowing development teams to␣
↪easily and efficiently "
                            "manage their packages and dependencies across platforms and␣
↪build systems."};
    char buffer_out [256] = {0};

    z_stream defstream;
    defstream.zalloc = Z_NULL;
    defstream.zfree = Z_NULL;
    defstream.opaque = Z_NULL;
    defstream.avail_in = (uInt) strlen(buffer_in);
    defstream.next_in = (Bytef *) buffer_in;
    defstream.avail_out = (uInt) sizeof(buffer_out);
```

(continues on next page)

(continued from previous page)

```
    defstream.next_out = (Bytef *) buffer_out;

    deflateInit(&defstream, Z_BEST_COMPRESSION);
    deflate(&defstream, Z_FINISH);
    deflateEnd(&defstream);

    printf("Uncompressed size is: %lu\n", strlen(buffer_in));
    printf("Compressed size is: %lu\n", strlen(buffer_out));

    printf("ZLIB VERSION: %s\n", zlibVersion());

    return EXIT_SUCCESS;
}
```

Also, the contents of *meson.build* are:

Listing 22: **meson.build**

```
project('tutorial', 'c')
zlib = dependency('zlib', version : '1.2.11')
executable('compressor', 'src/main.c', dependencies: zlib)
```

Let's create a *conanfile.txt* with the following content to install **Zlib**:

Listing 23: **conanfile.txt**

```
[requires]
zlib/1.3.1

[generators]
PkgConfigDeps
MesonToolchain
```

In this case, we will use *PkgConfigDeps* to generate information about where the **Zlib** library files are installed thanks to the **.pc* files and *MesonToolchain* to pass build information to *Meson* using a *conan_meson_[native|cross].ini* file that describes the native/cross compilation environment, which in this case is a *conan_meson_native.ini* one.

We will use Conan to install **Zlib** and generate the files that Meson needs to find this library and build our project. We will generate those files in the folder *build*. To do that, run:

```
$ conan install . --output-folder=build --build=missing
```

Now we are ready to build and run our **compressor** app:

Listing 24: Windows

```
$ cd build
$ meson setup --native-file conan_meson_native.ini .. meson-src
$ meson compile -C meson-src
$ meson-src\compressor.exe
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.2.11
```

Listing 25: Linux, macOS

```
$ cd build
$ meson setup --native-file conan_meson_native.ini .. meson-src
$ meson compile -C meson-src
$ ./meson-src/compressor
Uncompressed size is: 233
Compressed size is: 147
ZLIB VERSION: 1.2.11
```

Create your first Conan package with Meson

In the *Create your first Conan package tutorial* CMake was used as the build system. If you haven't read that section, read it first to familiarize yourself with the `conanfile.py` and `test_package` concepts, then come back to read about the specifics of the `Meson` package creation.

Use the **conan new** command to create a "Hello World" C++ library example project:

```
$ conan new meson_lib -d name=hello -d version=1.0
```

This will create a Conan package project with the following structure.

```
├── conanfile.py
├── meson.build
├── hello.vcxproj
├── src
│   ├── hello.h
│   └── hello.cpp
└── test_package
    ├── conanfile.py
    ├── meson.build
    └── src
        └── example.cpp
```

The structure and files are very similar to the previous CMake example:

- **conanfile.py**: On the root folder, there is a *conanfile.py* which is the main recipe file, responsible for defining how the package is built and consumed.
- **meson.build**: A Meson build script. This script doesn't need to contain anything Conan-specific, it is completely agnostic of Conan, because the integration is transparent.
- **src** folder: the folder that contains the simple C++ "hello" library.
- **test_package** folder: contains an *example* application that will require and link with the created package. In this case the `test_package` also contains a `meson.build`, but it is possible to have the `test_package` using other build system as CMake if desired. It is not mandatory that the test_package is using the same build system as the package.

Let's have a look at the package recipe *conanfile.py* (only the relevant new parts):

```
exports_sources = "meson.build", "src/*"

def layout(self):
    basic_layout(self)
```

(continues on next page)

(continued from previous page)

```
def generate(self):
    tc = MesonToolchain(self)
    tc.generate()

def build(self):
    meson = Meson(self)
    meson.configure()
    meson.build()

def package(self):
    meson = Meson(self)
    meson.install()
```

Let's explain the different sections of the recipe briefly:

- The `layout()` defines a `basic_layout()`, this is less flexible than a CMake one, so it doesn't allow any parametrization.
- The `generate()` method calls `MesonToolchain` that can generate `conan_meson_native.ini` and `conan_meson_cross.ini` Meson toolchain files for cross builds. If the project had dependencies with Conan `requires`, it should add `PkgConfigDeps` too
- The `build()` method uses the `Meson()` helper to drive the build
- The `package()` method uses the `Meson` install functionality to define and copy to the package folder the final artifacts.

The **test_package** folder also contains a `meson.build` file that declares a dependency to the tested package, and links an application, to verify the package was correctly created and contains that library:

Listing 26: test_package/meson.build

```
project('Testhello', 'cpp')
hello = dependency('hello', version : '>=0.1')
executable('example', 'src/example.cpp', dependencies: hello)
```

Note the `test_package/conanfile.py` contains also a `generators = "PkgConfigDeps", "MesonToolchain"`, because the `test_package` has the "hello" package as dependency, and `PkgConfigDeps` is necessary to locate it.

Note: This example assumes Meson, Ninja and PkgConfig are installed in the system, which might not always be the case. If they are not, you can create a profile `myprofile` with:

```
include(default)

[tool_requires]
meson/[*]
pkgconf/[*]
```

We added *Meson* and *pkg-config* as *tool requirements to the profile*. By executing `conan create . -pr=myprofile`, those tools will be installed and made available during the package's build process.

Let's build the package from sources with the current default configuration, and then let the `test_package` folder test the package:

```
$ conan create .

...
======== Testing the package: Executing test ========
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: .\example
hello/1.0: Hello World Release!
  hello/1.0: _M_X64 defined
  hello/1.0: MSVC runtime: MultiThreadedDLL
  hello/1.0: _MSC_VER1939
  hello/1.0: _MSVC_LANG201402
  hello/1.0: __cplusplus201402
hello/1.0 test_package
```

We can now validate that the recipe and the package binary are in the cache:

```
$ conan list "hello/1.0:*"
Local Cache:
  hello
    hello/1.0
      revisions
        856c535669f78da11502a119b7d8a6c9 (2024-03-04 17:52:39 UTC)
          packages
            c13a22a41ecd72caf9e556f68b406569547e0861
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: msvc
                  compiler.cppstd: 14
                  compiler.runtime: dynamic
                  compiler.runtime_type: Release
                  compiler.version: 193
                  os: Windows
```

See also:

- *Meson built-in integrations reference*.
- *PkgConfigDeps built-in integrations reference*.

8.3.4 Bazel

Build a simple Bazel project using Conan

Warning: This example is Bazel 6.x compatible. The Bazel 6.x support in `BazelDeps` is deprecated since Conan 2.34 and will be removed in a future release. See *Build a simple Bazel 7.x project using Conan* for the recommended approach.

In this example, we are going to create a Hello World program that uses one of the most popular C++ libraries: fmt.

Note: This example is based on the main *Build a simple CMake project using Conan* tutorial. So we highly recommend reading it before trying out this one.

We'll use Bazel as the build system and helper tool in this case, so you should get it installed before going forward with this example. See how to install Bazel.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/google/bazeltoolchain/6_x/string_formatter
```

We start from a very simple C++ language project with this structure:

```
.
├── WORKSPACE
├── conanfile.txt
└── main
    ├── BUILD
    └── demo.cpp
```

This project contains a *WORKSPACE* file loading the Conan dependencies (in this case only `fmt`) and a *main/BUILD* file which defines the *demo* bazel target and it's in charge of using `fmt` to build a simple Hello World program.

Let's have a look at each file's content:

Listing 27: **main/demo.cpp**

```
#include <cstdlib>
#include <fmt/core.h>

int main() {
    fmt::print("{} - The C++ Package Manager!\n", "Conan");
    return EXIT_SUCCESS;
}
```

Listing 28: **WORKSPACE**

```
load("@//conan:dependencies.bzl", "load_conan_dependencies")
load_conan_dependencies()
```

Listing 29: **main/BUILD**

```
cc_binary(
    name = "demo",
    srcs = ["demo.cpp"],
    deps = [
        "@fmt//:fmt"
    ],
)
```

Listing 30: **conanfile.txt**

```
[requires]
```

(continues on next page)

(continued from previous page)

```
fmt/10.1.1

[generators]
BazelDeps
BazelToolchain

[layout]
bazel_layout
```

Conan uses the *BazelToolchain* to generate a `conan_bzl.rc` file which defines the `conan-config` bazel-build configuration. This file and the configuration are passed as parameters to the `bazel build` command. Apart from that, Conan uses the *BazelDeps* generator to create all the bazel files (*[DEP]/BUILD.bazel* and *dependencies.bzl*) which define all the dependencies as public bazel targets. The *WORKSPACE* above is already ready to load the *dependencies.bzl* which will tell the *main/BUILD* all the information about the `@fmt//:fmt` bazel target.

As the first step, we should install all the dependencies listed in the `conanfile.txt`. The command *conan install* does not only install the `fmt` package, it also builds it from sources in case your profile does not match with a pre-built binary in your remotes. Furthermore, it will save all the files created by the generators listed in the `conanfile.txt` in a folder named *conan/* (default folder defined by the `bazel_layout`).

```
$ conan install . --build=missing
# ...
======== Finalizing install (deploy, generators) ========
conanfile.txt: Writing generators to /Users/user/develop/examples2/examples/tools/google/
↪bazeltoolchain/6_x/string_formatter/conan
conanfile.txt: Generator 'BazelDeps' calling 'generate()'
conanfile.txt: Generator 'BazelToolchain' calling 'generate()'
conanfile.txt: Generating aggregated env files
conanfile.txt: Generated aggregated env files: ['conanbuild.sh', 'conanrun.sh']
Install finished successfully
```

Now we are ready to build and run our application:

```
$ bazel --bazelrc=./conan/conan_bzl.rc build --config=conan-config //main:demo
Starting local Bazel server and connecting to it...
INFO: Analyzed target //main:demo (38 packages loaded, 272 targets configured).
INFO: Found 1 target...
INFO: From Linking main/demo:
ld: warning: ignoring duplicate libraries: '-lc++'
Target //main:demo up-to-date:
  bazel-bin/main/demo
INFO: Elapsed time: 60.180s, Critical Path: 7.68s
INFO: 6 processes: 4 internal, 2 darwin-sandbox.
INFO: Build completed successfully, 6 total actions
```

```
$ ./bazel-bin/main/demo
Conan - The C++ Package Manager!
```

Build a simple Bazel 7.x project using Conan

Warning: This example is Bazel >= 7.2 compatible.

In this example, we are going to create a Hello World program that uses one of the most popular C++ libraries: fmt.

Note: This example is based on the *Build a simple CMake project using Conan* tutorial. So we highly recommend reading it before trying out this one.

We'll use Bazel as the build system and helper tool in this case, so you should get it installed before going forward with this example. See how to install Bazel.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/google/bazeltoolchain/7_x/string_formatter
```

We start from a very simple C++ language project with this structure:

```
.
├── MODULE.bazel
├── conanfile.txt
└── main
    ├── BUILD
    └── demo.cpp
```

This project contains a *MODULE.bazel* file loading the Conan dependencies (in this case only `fmt`) and a *main/BUILD* file which defines the *demo* bazel target and it's in charge of using `fmt` to build a simple Hello World program.

Let's have a look at each file's content:

Listing 31: **main/demo.cpp**

```
#include <cstdlib>
#include <fmt/core.h>

int main() {
    fmt::print("{} - The C++ Package Manager!\n", "Conan");
    return EXIT_SUCCESS;
}
```

Listing 32: **MODULE.bazel**

```
include("//conan:conan_deps.MODULE.bazel")
```

Listing 33: **main/BUILD**

```
load("@rules_cc//cc:cc_binary.bzl", "cc_binary")

cc_binary(
    name = "demo",
    srcs = ["demo.cpp"],
```

(continues on next page)

(continued from previous page)

```
    deps = [
        "@fmt//:fmt"
    ],
)
```

Listing 34: **conanfile.txt**

```
[requires]
fmt/10.1.1

[generators]
BazelDeps
BazelToolchain

[layout]
bazel_layout
```

Conan uses the *BazelToolchain* to generate a `conan_bzl.rc` file which defines the `conan-config` bazel-build configuration. This file and the configuration are passed as parameters to the `bazel build` command. Apart from that, Conan uses the *BazelDeps* generator to create all the bazel files (*[DEP]/BUILD.bazel*, *[DEP]/BUILD.rules_cc.bazel*, *conan_deps.MODULE.bazel*, *conan_deps_module_extension.bzl*, and *conan_deps_repo_rules.bzl*) which define the rule and all the dependencies to create/load them as Bazel repositories. The *MODULE.bazel* above includes the generated *conan_deps.MODULE.bazel* snippet, which will tell the *main/BUILD* all the information about the `@fmt//:fmt` bazel target.

As the first step, we should install all the dependencies listed in the `conanfile.txt`. The command *conan install* does not only install the `fmt` package, it also builds it from sources in case your profile does not match with a pre-built binary in your remotes. Furthermore, it will save all the files created by the generators listed in the `conanfile.txt` in a folder named *conan/* (default folder defined by the `bazel_layout`).

```
$ conan install . --build=missing
# ...
======== Finalizing install (deploy, generators) ========
conanfile.txt: Writing generators to /Users/user/develop/examples2/examples/tools/google/
↪bazeltoolchain/7_x/string_formatter/conan
conanfile.txt: Generator 'BazelDeps' calling 'generate()'
conanfile.txt: Generator 'BazelToolchain' calling 'generate()'
conanfile.txt: Generating aggregated env files
conanfile.txt: Generated aggregated env files: ['conanbuild.sh', 'conanrun.sh']
Install finished successfully
```

Now we are ready to build and run our application:

```
$ bazel --bazelrc=./conan/conan_bzl.rc build --config=conan-config //main:demo
Computing main repo mapping:
Loading:
Loading: 0 packages loaded
Analyzing: target //main:demo (1 packages loaded, 0 targets configured)
Analyzing: target //main:demo (1 packages loaded, 0 targets configured)
[0 / 1] [Prepa] BazelWorkspaceStatusAction stable-status.txt
INFO: Analyzed target //main:demo (69 packages loaded, 369 targets configured).
[5 / 7] Compiling main/demo.cpp; 0s darwin-sandbox
INFO: Found 1 target...
```

(continues on next page)

(continued from previous page)

```
Target //main:demo up-to-date:
  bazel-bin/main/demo
INFO: Elapsed time: 2.955s, Critical Path: 1.70s
INFO: 7 processes: 5 internal, 2 darwin-sandbox.
INFO: Build completed successfully, 7 total actions
```

```
$ ./bazel-bin/main/demo
Conan - The C++ Package Manager!
```

8.3.5 Autotools

Build a simple Autotools project with Conan dependencies

Warning: This example will only work for Linux and OSX environments and does not support Windows directly, including msys2/cygwin subsystems. However, Windows Subsystem for Linux (WSL) should work since it provides a Linux environment. While Conan offers *win_bash = True* for some level of support in Windows environments with Autotools, it's not applicable in this tutorial.

In this example, we are going to create a string formatter application that uses one of the most popular C++ libraries: fmt.

We'll use Autotools as build system and pkg-config as a helper tool in this case, so you should get them installed on Linux and Mac before going forward with this example.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
git clone https://github.com/conan-io/examples2.git
cd examples2/examples/tools/autotools/autotoolstoolchain/string_formatter
```

We start with a very simple C++ language project with the following structure:

```
.
├── configure.ac
├── Makefile.am
├── conanfile.txt
└── src
    └── main.cpp
```

This project contains a basic *configure.ac <https://www.gnu.org/software/autoconf/manual/autoconf-2.60/html_node/Writing-configure_002eac.html>_* including the **fmt** pkg-config dependency and the source code for the string formatter program in *main.cpp*.

Let's have a look at the *main.cpp* file, it only prints a simple message but uses `fmt::print` method for it.

Listing 35: **main.cpp**

```
#include <cstdlib>
#include <fmt/core.h>

int main() {
    fmt::print("{} - The C++ Package Manager!\n", "Conan");
```

(continues on next page)

(continued from previous page)

```
    return EXIT_SUCCESS;
}
```

The `configure.ac` file checks for a C++ compiler using the `AC_PROG_CXX` macro and also checks for the `fmt.pc` pkg-config module using the `PKG_CHECK_MODULES` macro.

Listing 36: **configure.ac**

```
AC_INIT([stringformatter], [0.1.0])
AM_INIT_AUTOMAKE([1.10 -Wall no-define foreign])
AC_CONFIG_SRCDIR([src/main.cpp])
AC_CONFIG_FILES([Makefile])
PKG_CHECK_MODULES([fmt], [fmt])
AC_PROG_CXX
AC_OUTPUT
```

The *Makefile.am* specifies that `string_formatter` is the expected executable and that it should be linked to the `fmt` library.

Listing 37: **Makefile.am**

```
AUTOMAKE_OPTIONS = subdir-objects
ACLOCAL_AMFLAGS = ${ACLOCAL_FLAGS}

bin_PROGRAMS = string_formatter
string_formatter_SOURCES = src/main.cpp
string_formatter_CPPFLAGS = $(fmt_CFLAGS)
string_formatter_LDADD = $(fmt_LIBS)
```

The *conanfile.txt* looks simple as it just installs the **fmt** package and uses two generators to build our project.

Listing 38: **conanfile.txt**

```
[requires]
fmt/9.1.0

[generators]
AutotoolsToolchain
PkgConfigDeps
```

In this case, we will use *PkgConfigDeps* to generate information about where the **fmt** library files are installed thanks to the **.pc* files and *AutotoolsToolchain* to pass build information to *autotools* using a *conanbuild[.sh|.bat]* file that describes the compilation environment.

We will use Conan to install **fmt** library, generate a toolchain for Autotools, and, .pc files for find **fmt** by pkg-config.

Building on Linux and macOS

First, we should install some requirements. On Linux you need to have `automake` , `pkgconf` and `make` packages installed, their packages names should vary according to the Linux distribution, but essentially, it should include all tools (aclocal, automake, autoconf and make) that you will need to build the following example.

For this example, we will not consider a specific Conan profile, but `fmt` is highly compatible with many different configurations. So it should work mostly with versions of GCC and Clang compiler.

As the first step, we should install all dependencies listed in the `conanfile.txt`. The command :ref: *conan install<reference_commands_install>* will not only install the `fmt` package, but also build it from sources in case your profile does not match with a pre-built binary in your remotes. Plus, it will provide these generators listed in the `conanfile.txt`

```
conan install . --build=missing
```

After running `conan install` command, we should have new files present in the *string_formatter* folder:

```
└── string_formatter
    ├── Makefile.am
    ├── conanautotoolstoolchain.sh
    ├── conanbuild.conf
    ├── conanbuild.sh
    ├── conanbuildenv-release-armv8.sh
    ├── conanfile.txt
    ├── conanrun.sh
    ├── conanrunenv-release-armv8.sh
    ├── configure.ac
    ├── deactivate_conanbuild.sh
    ├── deactivate_conanrun.sh
    ├── fmt-_fmt.pc
    ├── fmt.pc
    ├── run_example.sh
    └── src
        └── main.cpp
```

These files are the result of those generators listed in the `conanfile.txt`. Once all files needed to build the example are generated and `fmt` is installed, now we can load the script `conanbuild.sh`.

```
source conanbuild.sh
```

The `conanbuild.sh` is a default file generated by the *VirtualBuildEnv* and helps us to load other script files, so we don't need to execute more manual steps to load each generator file. It will load `conanautotoolstoolchain.sh`, generated by *AutotoolsToolchain*, which defines environment variables according to our Conan profile, used when running `conan install` command. Those environment variables configured are related to the compiler and `autotools`, like `CFLAGS`, `CPPFLAGS`, `LDFLAGS`, and `PKG_CONFIG_PATH`.

As the next step, we can configure the project by running the following commands in sequence:

```
aclocal
automake --add-missing
autoconf
./configure
```

The aclocal command will read the file `configure.ac` and generate a new file named `aclocal.m4`, which contains macros needed by the `automake`. As the second step, the automake command will read the `Makefile.am`, and will generate the file `Makefile.in`. So the command autoconf will use those files and generate the `configure` file. Once we run `configure`, all environment variables will be consumed. The `fmt.pc` will be loaded at this step too, as `autotools` uses the custom `PKG_CONFIG_PATH` to find it.

Then, finally, we can build the project to generate the string formatter application. Now we run the `make` command, which will consume the `Makefile` generated by `autotools`.

```
make
```

The `make` command will read the `Makefile` and invoke the compiler, then, build the `main.cpp`, generating the executable `string_formatter` in the same folder.

```
./string_formatter
Conan - The C++ Package Manager!
```

The final output is the result of a new application, printing a message with the help of `fmt` library, and built by `Autotools`.

Create your first Conan package with Autotools

> **Warning:** This example will only work for Linux and OSX environments and does not support Windows directly, including msys2/cygwin subsystems. However, Windows Subsystem for Linux (WSL) should work since it provides a Linux environment. While Conan offers *win_bash = True* for some level of support in Windows environments with Autotools, it's not applicable in this tutorial.

In the *Create your first Conan package tutorial* CMake was used as the build system. If you haven't read that section, read it first to familiarize yourself with the `conanfile.py` and `test_package` concepts, then come back to read about the specifics of the `Autotools` package creation.

Use the **`conan new`** command to create a "Hello World" C++ library example project:

```
$ conan new autotools_lib -d name=hello -d version=0.1
```

This will create a Conan package project with the following structure.

```
├── conanfile.py
├── configure.ac
├── Makefile.am
├── src
│   ├── hello.h
│   ├── hello.cpp
│   └── Makefile.am
└── test_package
    ├── conanfile.py
    ├── configure.ac
    ├── mainc.pp
    └── Makefile.am
```

The structure and files are very similar to the previous CMake example:

- **conanfile.py**: On the root folder, there is a *conanfile.py* which is the main recipe file, responsible for defining how the package is built and consumed.
- **configure.ac**: An autotools configuration script, that contains the necessary macros and references the `Makefiles` it needs to configure.
- **Makefile.am**: A Makefile configuration file, defining only `SUBDIRS = src`
- **src** folder: the folder that contains the simple C++ "hello" library.
- **src/Makefile.am**: Makefile configuration file containing the library definition and source files like `libhello_la_SOURCES = hello.cpp hello.h`
- **test_package** folder: contains an *example* application that will require and link with the created package. In this case the `test_package` also contains an autotools project, but it is possible to have the `test_package` using other build system as CMake if desired. It is not mandatory that the test_package is using the same build system as the package.

Let's have a look at the package recipe *conanfile.py* (only the relevant new parts):

```
exports_sources = "configure.ac", "Makefile.am", "src/*"

def layout(self):
    basic_layout(self)

def generate(self):
    at_toolchain = AutotoolsToolchain(self)
    at_toolchain.generate()

def build(self):
    autotools = Autotools(self)
    autotools.autoreconf()
    autotools.configure()
    autotools.make()

def package(self):
    autotools = Autotools(self)
    autotools.install()
    fix_apple_shared_install_name(self)
```

Let's explain the different sections of the recipe briefly:

- The `layout()` defines a `basic_layout()`, this is less flexible than a CMake one, so it doesn't allow any parametrization.
- The `generate()` method calls `AutotoolsToolchain` that can generate a `conanautotoolstoolchain` environment script defining environment variables like `CXXFLAGS` or `LDFLAGS` that will be used by the `Makefiles` to map the Conan input settings into compile flags. If the project had dependencies with Conan `requires`, it should add `PkgConfigDeps` too
- The `build()` method uses the `Autotools()` helper to drive the build, calling the different configure and build steps.
- The `package()` method uses the `Autotools` install functionality to define and copy to the package folder the final artifacts. Note the template also includes a call to `fix_apple_shared_install_name()` that uses OSX *install_name_tool* utility to set `@rpath``to fix the ``LC_ID_DYLIB` and `LC_LOAD_DYLIB` fields on Apple dylibs, because it is very unusual that autotools project will manage to do this (CMake can do it) .

Let's build the package from sources with the current default configuration, and then let the `test_package` folder test the package:

```
$ conan create .

...
======== Testing the package: Executing test ========
hello/0.1 (test package): Running test()
hello/0.1 (test package): RUN: ./main
hello/0.1: Hello World Release!
  hello/0.1: __x86_64__ defined
  hello/0.1: _GLIBCXX_USE_CXX11_ABI 1
  hello/0.1: __cplusplus201703
  hello/0.1: __GNUC__11
  hello/0.1: __GNUC_MINOR__1
hello/0.1 test_package
```

We can now validate that the recipe and the package binary are in the cache:

```
$ conan list "hello/1.0:*"
Local Cache:
  hello
    hello/1.0
      revisions
        5b151b3f08144bf25131266eb306ddff (2024-03-06 12:03:52 UTC)
          packages
            8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: gcc
                  compiler.cppstd: gnu17
                  compiler.libcxx: libstdc++11
                  compiler.version: 11
                  os: Linux
                options
                  fPIC: True
                  shared: False
```

See also:

- *GNU built-in integrations reference*.

Create your first Conan package with Autotools in Windows (msys2)

Warning: This example is intended for the Windows OS, using the `msys2` subsystem to run the autotools build system. The support is **limited**, the `AutotoolsDeps` generator still doesn't work for Windows, so the `test_package` in the default template will fail.

Note this example is building with the MSVC compiler, not with MinGW/gcc. Even if the build system is autotools, the example is targeting the MSVC compiler, and the resulting package will be binary compatible and can be used from other packages using MSVC with other build systems. It is not necessary to force MinGW/gcc to use some open source dependencies that use autotools, and ConanCenter builds all of them with MSVC.

In the *Create your first Conan package with Autotools* tutorial, the autotools integrations are presented. Please read first that section, to understand them, as this section will only introduce the Windows/msys2 specific issues.

We will use the same the **conan new** command to create a "Hello World" C++ library example project:

```
$ conan new autotools_lib -d name=mypkg -d version=0.1
```

Check the above tutorial to understand the created files.

Besides these files, we will create a profile file:

Listing 39: msys2_profile

```
include(default)

[conf]
tools.microsoft.bash:subsystem=msys2
tools.microsoft.bash:path=C:\ws\msys64\usr\bin\bash
# since Conan 2.9, this "cl" compiler definition is not necessary
# by default for the 'compiler=msvc'
# tools.build:compiler_executables={"c": "cl", "cpp": "cl"}
```

Note that you might need to adapt the path to the `bash` system of `msys2`.

In the package recipe *conanfile.py* we will have:

```
win_bash = True
```

This is very important, it tells Conan that when this package is to be built, it has to launch a `bash` shell to execute the build in it.

Note: It is not necessary, and in fact it is not recommended for most cases to be already running inside an `msys2` terminal. Conan will automatically run the build subprocess for autotools in the defined bash shell.

If already running in a bash shell, it is necessary to activate the `tools.microsoft.bash:active=True` conf.

Let's build the package from sources with the current default configuration, making sure to deactivate the `test_package`, because otherwise it will fail.

```
# Deactivating the test_package, as AutotoolsDeps doesn't work yet.
$ conan create . -pr=msys2_profile -tf=""

...
mypkg/0.1: package(): Packaged 1 '.h' file: mypkg.h
mypkg/0.1: package(): Packaged 1 '.la' file: libmypkg.la
mypkg/0.1: package(): Packaged 1 '.lib' file: mypkg.lib
mypkg/0.1: Created package revision fa661758835cf6f7f311c857447393cc
mypkg/0.1: Package '9bdee485ef71c14ac5f8a657202632bdb8b4482b' created
```

We can now validate that the recipe and the package binary are in the cache:

```
$  conan list "mypkg:*"
Found 1 pkg/version recipes matching mypkg in local cache
Local Cache
  mypkg
    mypkg/0.1
      revisions
        6e85b0c27c7fbc8eddc1994dbb543b52 (2024-04-30 18:29:44 UTC)
          packages
            9bdee485ef71c14ac5f8a657202632bdb8b4482b
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: msvc
                  compiler.cppstd: 14
                  compiler.runtime: dynamic
                  compiler.runtime_type: Release
                  compiler.version: 193
                  os: Windows
                options
                  shared: False
```

Note how the binary is a `compiler=msvc` one.

See also:

- *GNU built-in integrations reference*.

AutoTools: Using LLVM/Clang Windows compiler

The Clang compiler in Windows can come from 2 different installations or distributions:

- The LLVM/Clang compiler, that uses the MSVC runtime
- The Msys2 Clang compiler that uses the Msys2 runtime (libstdc++6.dll)

This example explains the LLVM/Clang with the MSVC runtime. This Clang distribution can in turn be used in two different ways:

- Using the LLVM/Clang downloaded compiler (it still uses the MSVC runtime), via the GNU-like frontend `clang`
- Using the LLVM/Clang downloaded compiler (it still uses the MSVC runtime), via the MSVC-like frontend `clang-cl`

Let's start from a simple `autotools_exe` template:

```
$ conan new autotools_exe -d name=mypkg -d version=0.1
```

This creates a simple Autotools based project and Conan package recipe that uses `AutotoolsToolchain`.

Autotools: LLVM/Clang with `clang` GNU-like frontend

To build this configuration we will use the following profile:

Listing 40: llvm_clang

```
[settings]
os=Windows
arch=x86_64
build_type=Release
compiler=clang
compiler.version=18
compiler.cppstd=14
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.runtime_version=v144

[buildenv]
PATH=+(path)C:\ws\LLVM\18.1\bin

[conf]
tools.compilation:verbosity=verbose
tools.build:compiler_executables = {"c": "clang", "cpp": "clang++"}
tools.microsoft.bash:subsystem=msys2
tools.microsoft.bash:path=C:\ws\msys64\usr\bin\bash.exe
```

Quick explanation of the profile:

- The `compiler.runtime` definition is the important differentiator to distinguish between Msys2-Clang and LLVM/Clang with the MSVC runtime. The LLVM/Clang defines this `compiler.runtime`, while the Msys2-Clang doesn't.
- The MSVC runtime can be either dynamic or static. It is important also to define the runtime version (toolset version `v144`) of this runtime, as it is possible to use different ones.
- The `[buildenv]` allows to point to the LLVM/Clang compiler, in case it is not already in the path. **Note** the `PATH=+(path)` syntax, to **prepend** that path, so it has higher priority
- While defining `tools.microsoft.bash:path`, the full path to the `msys2` `bash.exe` has been used. Otherwise, it is possible that it can find another `bash.exe` in the Windows system that will not be valid.

Let's build it:

```
$ conan build . -pr=llvm_clang
...
conanfile.py (mypkg/0.1): Calling build()
conanfile.py (mypkg/0.1): RUN: autoreconf --force --install
conanfile.py (mypkg/0.1): RUN: "/c/projectpath/clang/configure" --prefix=/ --bindir=$
↪{prefix}/bin --sbindir=${prefix}/bin --libdir=${prefix}/lib --includedir=${prefix}/
↪include --oldincludedir=${prefix}/include
conanfile.py (mypkg/0.1): RUN: make -j8
```

(continues on next page)

(continued from previous page)

```
...
clang++ -DPACKAGE_NAME=\"mypkg\" -DPACKAGE_TARNAME=\"mypkg\" -DPACKAGE_VERSION=\"0.1\" -
↪DPACKAGE_STRING=\"mypkg\ 0.1\" -DPACKAGE_BUGREPORT=\"\" -DPACKAGE_URL=\"\" -DPACKAGE=\
↪"mypkg\" -DVERSION=\"0.1\" -I. -I/c/projectpath/clang/src   -DNDEBUG  -std=c++14 -D_
↪DLL -D_MT -Xclang --dependent-lib=msvcrt -O3 -c -o main.o /c/projectpath/clang/src/
↪main.cpp
source='/c/projectpath/clang/src/mypkg.cpp' object='mypkg.o' libtool=no \
DEPDIR=.deps depmode=none /bin/sh /c/projectpath/clang/depcomp \
clang++ -DPACKAGE_NAME=\"mypkg\" -DPACKAGE_TARNAME=\"mypkg\" -DPACKAGE_VERSION=\"0.1\" -
↪DPACKAGE_STRING=\"mypkg\ 0.1\" -DPACKAGE_BUGREPORT=\"\" -DPACKAGE_URL=\"\" -DPACKAGE=\
↪"mypkg\" -DVERSION=\"0.1\" -I. -I/c/projectpath/clang/src   -DNDEBUG  -std=c++14 -D_
↪DLL -D_MT -Xclang --dependent-lib=msvcrt -O3 -c -o mypkg.o /c/projectpath/clang/src/
↪mypkg.cpp
clang++  -std=c++14 -D_DLL -D_MT -Xclang --dependent-lib=msvcrt -O3  -fuse-ld=lld-link -
↪o mypkg.exe main.o mypkg.o
```

Note how the `clang++` compiler is used, the runtime is selected with `-D_DLL -D_MT -Xclang --dependent-lib=msvcrt`.

We can run our executable, and see how the Clang compiler version and the MSVC runtime match the defined ones:

```
$ build-release\src\mypkg.exe
mypkg/0.1: Hello World Release!
    mypkg/0.1: _M_X64 defined
    mypkg/0.1: __x86_64__ defined
    mypkg/0.1: MSVC runtime: MultiThreadedDLL
    mypkg/0.1: _MSC_VER1943
    mypkg/0.1: _MSVC_LANG201402
    mypkg/0.1: __cplusplus201402
    mypkg/0.1: __clang_major__18
    mypkg/0.1: __clang_minor__1
```

Autotools: LLVM/Clang with `clang-cl` MSVC-like frontend

To build this configuration we will use the following profile:

Listing 41: llvm_clang_cl

```
[settings]
os=Windows
arch=x86_64
build_type=Release
compiler=clang
compiler.version=18
compiler.cppstd=14
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.runtime_version=v144

[buildenv]
PATH=+(path)C:/ws/LLVM/18.1/bin
```

(continues on next page)

(continued from previous page)

```
[conf]
tools.compilation:verbosity=verbose
tools.microsoft.bash:subsystem=msys2
tools.build:compiler_executables = {"c": "clang-cl", "cpp": "clang-cl"}
tools.microsoft.bash:path=C:\ws\msys64\usr\bin\bash.exe
```

The profile is almost identical to the above one, the main difference is the definition of `tools.build:compiler_executables`, defining the `clang-cl` compiler.

Note: The definition of `tools.build:compiler_executables` using the `clang-cl` compiler is what is used by Conan to differentiate among the frontends, also in other build systems. This frontend is not a `setting`, because the compiler is still the same, and the resulting binary should be binary compatible.

Let's build it:

```
$ conan build . -pr=llvm_clang_cl
...
clang-cl -DPACKAGE_NAME=\"mypkg\" -DPACKAGE_TARNAME=\"mypkg\" -DPACKAGE_VERSION=\"0.1\" -
↪DPACKAGE_STRING=\"mypkg\ 0.1\" -DPACKAGE_BUGREPORT=\"\" -DPACKAGE_URL=\"\" -DPACKAGE=\
↪"mypkg\" -DVERSION=\"0.1\" -I. -I/c/projectpath/clang/src   -DNDEBUG  -std:c++14 -MD -
↪O2 -c -o main.obj `cygpath -w '/c/projectpath/clang/src/main.cpp'`
source='/c/projectpath/clang/src/mypkg.cpp' object='mypkg.obj' libtool=no \
DEPDIR=.deps depmode=msvc7msys /bin/sh /c/projectpath/clang/depcomp \
clang-cl -DPACKAGE_NAME=\"mypkg\" -DPACKAGE_TARNAME=\"mypkg\" -DPACKAGE_VERSION=\"0.1\" -
↪DPACKAGE_STRING=\"mypkg\ 0.1\" -DPACKAGE_BUGREPORT=\"\" -DPACKAGE_URL=\"\" -DPACKAGE=\
↪"mypkg\" -DVERSION=\"0.1\" -I. -I/c/projectpath/clang/src   -DNDEBUG  -std:c++14 -MD -
↪O2 -c -o mypkg.obj `cygpath -w '/c/projectpath/clang/src/mypkg.cpp'`
clang-cl  -std:c++14 -MD -O2   -o mypkg.exe main.obj mypkg.obj
...
```

See how the desired `clang-cl` is used, and how the `MSVC-like` command line syntax is used, like `-std:c++14`. This MSVC-like syntax uses the `-MD/-MT` flags to differentiate across the dynamic/static MSVC runtimes.

We can run our executable, and see how the Clang compiler version and the MSVC runtime match the defined ones:

```
$ build\Release\mypkg.exe
mypkg/0.1: Hello World Release!
    mypkg/0.1: _M_X64 defined
    mypkg/0.1: __x86_64__ defined
    mypkg/0.1: MSVC runtime: MultiThreadedDLL
    mypkg/0.1: _MSC_VER1943
    mypkg/0.1: _MSVC_LANG201402
    mypkg/0.1: __cplusplus201402
    mypkg/0.1: __clang_major__18
    mypkg/0.1: __clang_minor__1
```

As expected, the output is identical to the previous one, as nothing changed except the compiler frontend.

Note: It might be possible to build using the `clang-cl` distributed as a Visual Studio component for autotools-like projects. But it is necessary to provide the full path to that Clang component within the Visual Studio installed folder, so it can be found, via `[buildenv]` and or `tools.build:compiler_executables`, because it is basically an

LLVM/Clang compiler, packaged and distributed by the Visual Studio installer.

8.3.6 GNU

Using PkgConfig.fill_cpp_info with PKG_CONFIG_PATH from a profile

The *PkgConfig* helper can read a `.pc` file and use its contents to fill a `cpp_info` object with `fill_cpp_info()`. When `pkg_config_path` is not passed explicitly, `pkg-config` locates the `.pc` file via the `PKG_CONFIG_PATH` environment variable, which can be defined in the profile `[buildenv]` section.

A common scenario is cross-compilation with an external toolchain: the toolchain provides a sysroot that already ships `.pc` files describing its libraries (`gl.pc`, `EGL.pc`, `zlib.pc`, ...), and the host profile points `PKG_CONFIG_PATH` at the sysroot's `pkgconfig` folder. A wrapper recipe can then read those `.pc` files at `package()` time and expose the information to consumers without hard-coding include and library paths.

The challenge comes when users try to define `PKG_CONFIG_PATH` in the `[buildenv]` section of a profile, because that is not available in the `package_info()` method of the recipes, as the package information cannot rely on the "build environment", as the build environment is only available when the package is being built, but not necessarily when the package is being consumed.

The example below uses a fabricated `mylibastral.pc` next to the profile to keep it self-contained, but the mechanism is exactly the same as pointing to a real sysroot folder, to illustrate how this could be implemented if necessary.

Important: This strategy is valid only for "system" packages, that is, recipes that act as a wrapper around a system or sysroot dependency (typical in cross-compilation with an external toolchain). Regular Conan packages that build and package their own artifacts should not use this.

The **profile** below adds a folder containing `.pc` files to `PKG_CONFIG_PATH`. The `profile_dir` Jinja variable resolves to the folder that contains the profile itself, so a `mypcs/` folder next to the profile can be used directly:

Listing 42: *myprofile*

```
[settings]
os=Linux
arch=armv8

[buildenv]
PKG_CONFIG_PATH+=(path){{ profile_dir }}/mypcs
```

In a real cross-compilation setup, that path would typically point to the sysroot folder shipped by the toolchain, for example `{{ sysroot }}/usr/lib/pkgconfig`.

The `.pc` file located in the `mypcs` folder besides the profile describes the library (include folders, library folders, libs, defines, link flags, ...):

Listing 43: *mypcs/mylibastral.pc*

```
prefix=/usr/local
exec_prefix=${prefix}
libdir=${exec_prefix}/lib
includedir=${prefix}/include

Name: mylibastral
```

(continues on next page)

(continued from previous page)

```
Description: Interface library for Astral data flows
Version: 6.6.6
Libs: -L${libdir}/libastral -lastral -lm -Wl,--whole-archive
Cflags: -I${includedir}/libastral -D_USE_LIBASTRAL
```

The `conanfile.py` recipe uses `PkgConfig` to read `mylibastral.pc` and populate a `CppInfo` object. It does it at `package()` time, when the package is being built (the policies are added so the package is automatically build when necessary in the current machine, and binaries are never uploaded, as the information from that .pc file will only be valid in the current machine). Notice that `pkg_config_path` is not passed: it is picked up from `PKG_CONFIG_PATH` defined in the profile. The resulting `CppInfo` is serialized to the package folder in `package()`, and reloaded in `package_info()`:

Listing 44: *conanfile.py*

```
import os
from conan import ConanFile
from conan.tools.gnu import PkgConfig
from conan.tools import CppInfo


class Pkg(ConanFile):
    name = "mypkg"
    version = "0.1"
    build_policy = "missing"
    upload_policy = "skip"

    def package(self):
        # This method executes at package build time, it has
        # available the build environment "buildenv"
        pkg_config = PkgConfig(self, "mylibastral")
        cpp_info = CppInfo(self)
        pkg_config.fill_cpp_info(cpp_info, is_system=False, system_libs=["m"])
        cpp_info.save(os.path.join(self.package_folder, "cpp_info.json"))

    def package_info(self):
        # This method executes at package consumption time, it will
        # NOT have available the build environment, but it can load a file
        # generated at build/package time
        self.cpp_info = CppInfo(self).load("cpp_info.json")
```

Let's now create and consume the package:

```
$ conan create . -pr=myprofile
$ conan install --requires=mypkg/0.1 -pr=myprofile -g CMakeDeps
```

The generated `mypkg-none-armv8-data.cmake` will contain the include/lib folders, libs, defines and link flags read from `mylibastral.pc`, something like:

```
set(mypkg_INCLUDE_DIRS_NONE "/usr/local/include/libastral")
set(mypkg_LIB_DIRS_NONE "/usr/local/lib/libastral")
set(mypkg_LIBS_NONE astral)
set(mypkg_SYSTEM_LIBS_NONE m)
set(mypkg_DEFINITIONS_NONE "-D_USE_LIBASTRAL")
set(mypkg_COMPILE_DEFINITIONS_NONE "_USE_LIBASTRAL")
```

(continues on next page)

(continued from previous page)

```
set(mypkg_SHARED_LINK_FLAGS_NONE "-Wl,--whole-archive")
```

8.3.7 Capturing Git scm information

There are 2 main strategies to handle source code in recipes:

- **Third-party code**: When the `conanfile.py` recipe is packaging third party code, like an open source library, it is typically better to use the `source()` method to download or clone the sources of that library. This is the approach followed by the `conan-center-index` repository for ConanCenter.
- **Your own code**: When the `conanfile.py` recipe is packaging your own code, it is typically better to have the `conanfile.py` in the same repository as the sources. Then, there are 2 alternatives for achieving reproducibility:
  - Using the `exports_sources` (or `export_source()` method) to capture a copy of the sources together with the recipe in the Conan package. This is very simple and pragmatic and would be recommended for the majority of cases.
  - For cases when it is not possible to store the sources beside the Conan recipe, for example when the package is to be consumed for someone that shouldn't have access to the source code at all, then the current **scm capture** method would be the way.

In the **scm capture** method, instead of capturing a copy of the code itself, the "coordinates" for that code are captured instead, in the `Git` case, the `url` of the repository and the `commit`. If the recipe needs to build from source, it will use that information to get a clone, and if the user who tries that is not authorized, the process will fail. They will still be able to use the pre-compiled binaries that we distribute, but not build from source or have access to the code.

Let's see how it works with an example. Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/scm/git/capture_scm
```

There we will find a small "hello" project, containing this `conanfile.py`:

```
from conan import ConanFile
from conan.tools.cmake import CMake, cmake_layout
from conan.tools.scm import Git


class helloRecipe(ConanFile):
    name = "hello"
    version = "0.1"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}
    generators = "CMakeDeps", "CMakeToolchain"

    def export(self):
        git = Git(self, self.recipe_folder)
        # save the url and commit in conandata.yml
        git.coordinates_to_conandata()
```

(continues on next page)

(continued from previous page)

```
    def source(self):
        # we recover the saved url and commit from conandata.yml and use them to get␣
↪sources
        git = Git(self)
        git.checkout_from_conandata_coordinates()

    ...
```

We need this code to be in its own Git repository, to see how it works in the real case, so please create a folder outside of the `examples2` repository, and copy the contents of the current folder there, then:

```
$ mkdir /home/myuser/myfolder # or equivalent in other OS
$ cp -R . /home/myuser/myfolder # or equivalent in other OS
$ cd /home/myuser/myfolder # or equivalent in other OS

# Initialize the git repo
$ git init .
$ git add .
$ git commit . -m wip
# Finally create the package
$ conan create .
...
======== Exporting recipe to the cache ========
hello/0.1: Exporting package recipe: /myfolder/conanfile.py
hello/0.1: Calling export()
hello/0.1: RUN: git status . --short --no-branch --untracked-files
hello/0.1: RUN: git rev-list HEAD -n 1 --full-history -- "."
hello/0.1: RUN: git remote -v
hello/0.1: RUN: git branch -r --contains cb7815a58529130b49da952362ce8b28117dee53
hello/0.1: RUN: git fetch origin --dry-run --depth=1␣
↪cb7815a58529130b49da952362ce8b28117dee53
hello/0.1: WARN: Current commit cb7815a58529130b49da952362ce8b28117dee53 doesn't exist␣
↪in remote origin
This revision will not be buildable in other computer
hello/0.1: RUN: git rev-parse --show-toplevel
hello/0.1: Copied 1 '.py' file: conanfile.py
hello/0.1: Copied 1 '.yml' file: conandata.yml
hello/0.1: Exported to cache folder: /.conan2/p/hello237d6f9f65bba/e
...
======== Installing packages ========
hello/0.1: Calling source() in /.conan2/p/hello237d6f9f65bba/s
hello/0.1: Cloning git repo
hello/0.1: RUN: git clone "<hidden>"  "."
hello/0.1: Checkout: cb7815a58529130b49da952362ce8b28117dee53
hello/0.1: RUN: git checkout cb7815a58529130b49da952362ce8b28117dee53
```

Let's explain step by step what is happening:

- When the recipe is exported to the Conan cache, the `export()` method executes, `git.coordinates_to_conandata()`, which stores the Git URL and commit in the `conandata.yml` file by internally calling `git.get_url_and_commit()`. See the *Git reference* for more information about these methods.
- This obtains the URL of the repo pointing to the local `<local-path>/capture_scm` and the commit

```
8e8764c40bebabbe3ec57f9a0816a2c8e691f559
```

- It warns that this information will **not** be enough to re-build from source this recipe once the package is uploaded to the server and is tried to be built from source in other computer, which will not contain the path pointed by <local-path>/capture_scm. This is expected, as the repository that we created doesn't have any remote defined. If our local clone had a remote defined and that remote contained the `commit` that we are building, the `scm_url` would point to the remote repository instead, making the build from source fully reproducible.
- The `export()` method stores the `url` and `commit` information in the `conandata.yml` for future reproducibility.
- When the package needs to be built from sources and it calls the `source()` method, it recovers the information from the `conandata.yml` file inside the `git.checkout_from_conandata_coordinates()` method, which internally calls `git.clone()` with it to retrieve the sources. In this case, it will be cloning from the local checkout in <local-path>/capture_scm, but if it had a remote defined, it will clone from it.

Warning: To achieve reproducibility, it is very important for this **scm capture** technique that the current checkout is not dirty If it was dirty, it would be impossible to guarantee future reproducibility of the build, so `git.get_url_and_commit()` can raise errors, and require to commit changes. If more than 1 commit is necessary, it would be recommended to squash those commits before pushing changes to upstream repositories.

If we do now a second `conan create .`, as the repo is dirty we would get:

```
$ conan create .
hello/0.1: Calling export()
ERROR: hello/0.1: Error in export() method, line 19
    scm_url, scm_commit = git.get_url_and_commit()
    ConanException: Repo is dirty, cannot capture url and commit: .../capture_scm
```

This could be solved by cleaning the repo with `git clean -xdf`, or by adding a `.gitignore` file to the repo with the following contents (which might be a good practice anyway for source control):

Listing 45: .gitignore

```
test_package/build
test_package/CMakeUserPresets.json
```

The capture of coordinates uses the `Git.get_url_and_commit()` method, that by default does:

- If the repository is dirty, it will raise an exception
- If the repository is not dirty, but the commit doesn't exist in the remote, it will warn, but it will return the local folder as repo `url`. This way, local commits can be tested without needing to push them to the server. The `core.scm:local_url=allow` can silence the warning and the `core.scm:local_url=block` will immediately raise an error: This last value can be useful for CI scenarios, to fail fast and save a build that would have been blocked later in the `conan upload`.
- Packages built with local commit will fail if trying to upload them to the server with `conan upload` as those local commits are not in the server and then the package might not be reproducible. This upload error can be avoided by setting `core.scm:local_url=allow`.
- If the repository is not dirty, and the commit exists in the server, it will return the remote URL and the commit.

Credentials management

In the example above, credentials were not necessary, because our local repo didn't require them. But in real world scenarios, the credentials can be required.

The first important bit is that `git.get_url_and_commit()` will capture the url of the `origin` remote. This url must not encode tokens, users or passwords, for several reasons. First because that will make the process not repeatable, and different builds, different users would get different urls, and consequently different recipe revisions. The `url` should always be the same. The recommended approach is to manage the credentials in an orthogonal way, for example using ssh keys. The provided example contains a Github action that does this:

Listing 46: .github/workflows/hello-demo.yml

```
name: Build "hello" package capturing SCM in Github actions
run-name: ${{ github.actor }} checking hello-ci Git scm capture
on: [push]
jobs:
Build:
    runs-on: ubuntu-latest
    steps:
    - name: Check out repository code
        uses: actions/checkout@v3
        with:
        ssh-key: ${{ secrets.SSH_PRIVATE_KEY }}
    - uses: actions/setup-python@v4
        with:
        python-version: '3.10'
    - uses: webfactory/ssh-agent@v0.7.0
        with:
        ssh-private-key: ${{ secrets.SSH_PRIVATE_KEY }}
    - run: pip install conan
    - run: conan profile detect
    - run: conan create .
```

This `hello-demo.yml` takes care of the following:

- The checkout `actions/checkout@v3` action receives the `ssh-key` to checkout as `git@` instead of `https`
- The `webfactory/ssh-agent@v0.7.0` action takes care that the ssh key is also activated during the execution of the following tasks, not only during the checkout.
- It is necessary to setup the `SSH_PRIVATE_KEY` secret in the Github interface, as well as the `deploy key` for the repo (with the private and public parts of the ssh-key)

In this way, it is possible to keep completely separated the authentication and credentials from the recipe functionality, without any risk to leaking credentials.

Note: Best practices

- Do not use an authentication mechanism that encodes information in the urls. This is risky, can easily disclose credentials in logs. It is recommended to use system mechanisms like ssh keys.
- Doing `conan create` is not recommended for local development, but instead running `conan install` and building locally, to avoid too many unnecessary commits. Only when everything works locally, it is time to start checking the `conan create` flow.

8.3.8 MSBuild

Create your first Conan package with Visual Studio/MSBuild

In the *Create your first Conan package tutorial* CMake was used as the build system. If you haven't read that section, read it first to familiarize yourself with the `conanfile.py` and `test_package` concepts, then come back to read about the specifics of the `Visual Studio` package creation.

Use the **`conan new`** command to create a "Hello World" C++ library example project:

```
$ conan new msbuild_lib -d name=hello -d version=1.0
```

This will create a Conan package project with the following structure.

```
.
├── conanfile.py
├── hello.sln
├── hello.vcxproj
├── include
│   └── hello.h
├── src
│   └── hello.cpp
└── test_package
    ├── conanfile.py
    ├── test_hello.sln
    ├── test_hello.vcxproj
    └── src
        └── test_hello.cpp
```

The structure and files are very similar to the previous CMake example:

- **conanfile.py**: On the root folder, there is a *conanfile.py* which is the main recipe file, responsible for defining how the package is built and consumed.
- **hello.sln**: A Visual Studio solution file that can be opened with the IDE.
- **hello.vcxproj**: A Visual Studio C/C++ project, part of the solution above.
- **src** and **include** folders: the folders that contains the simple C++ "hello" library.
- **test_package** folder: contains an *example* application that will require and link with the created package. In this case the `test_package` also contains a Visual Studio solution and project, but it is possible to have the `test_package` using other build system as CMake if desired. It is not mandatory that the test_package is using the same build system as the package.

Let's have a look at the package recipe *conanfile.py* (only the relevant new parts):

```
# Sources are located in the same place as this recipe, copy them to the recipe
exports_sources = "hello.sln", "hello.vcxproj", "src/*", "include/*"

def layout(self):
    vs_layout(self)

def generate(self):
    tc = MSBuildToolchain(self)
    tc.generate()
```

(continues on next page)

(continued from previous page)

```
def build(self):
    msbuild = MSBuild(self)
    msbuild.build("hello.sln")

def package(self):
    copy(self, "*.h", os.path.join(self.source_folder, "include"),
         dst=os.path.join(self.package_folder, "include"))
    copy(self, "*.lib", src=self.build_folder, dst=os.path.join(self.package_folder, "lib
↪"),
         keep_path=False)
```

Let's explain the different sections of the recipe briefly:

- Note there are no `options` like the `shared` option in this recipe. The current project always builds a static library, so it is not optional.
- The `layout()` defines a typical VS layout, this is less flexible than a CMake one, so it doesn't allow any parametrization.
- The `generate()` method calls `MSBuildToolchain` to generate a `conantoolchain.props` file, that the project must add to its properties. If the project had dependencies with Conan `requires`, it should add `MSBuildDeps` too and add the relevant generated files property sheets.
- The `build()` method uses the `MSBuild()` helper to drive the build of the solution
- As the project doesn't have any "install" functionality in the build scripts, the `package()` method can manually define which files must be copied.

The `hello.vcxproj` project file adds the generated property sheets like `conantoolchain.props` to the project, so the build can receive the Conan input `settings` and act accordingly.

Listing 47: hello.vcxproj

```
 <ImportGroup Label="PropertySheets">
   <Import Project="conan\conantoolchain.props" />
 </ImportGroup>
```

If the project had dependencies, it should add the dependencies generated `.props` files too.

The **test_package** folder also contains a `test_hello.vcxproj` file, that includes both the toolchain and the dependencies property sheets:

Listing 48: test_package/test_hello.vcxproj

```
<ImportGroup Label="PropertySheets">
    <Import Project="conan\conantoolchain.props" />
    <Import Project="conan\conandeps.props" />
</ImportGroup>
```

Note the `test_package/conanfile.py` contains also a `generators="MSBuildDeps"`.

Let's build the package from sources with the current default configuration, and then let the `test_package` folder test the package:

```
$ conan create .

...
```

(continues on next page)

(continued from previous page)

```
======== Testing the package: Executing test ========
hello/1.0 (test package): Running test()
hello/1.0 (test package): RUN: x64\Release\test_hello
hello/1.0: Hello World Release!
  hello/1.0: _M_X64 defined
  hello/1.0: MSVC runtime: MultiThreadedDLL
  hello/1.0: _MSC_VER1939
  hello/1.0: _MSVC_LANG201402
  hello/1.0: __cplusplus199711
hello/1.0 test_package
```

We can now validate that the recipe and the package binary are in the cache:

```
$ conan list hello/1.0:*
Local Cache:
  hello
    hello/1.0
      revisions
        856c535669f78da11502a119b7d8a6c9 (2024-03-04 17:52:39 UTC)
          packages
            c13a22a41ecd72caf9e556f68b406569547e0861
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: msvc
                  compiler.cppstd: 14
                  compiler.runtime: dynamic
                  compiler.runtime_type: Release
                  compiler.version: 193
                  os: Windows
```

See also:

- Check the *Conan Visual Studio Extension*.
- *MSBuild built-in integrations reference*.

8.3.9 System Packages

Wrapping system requirements in a Conan package

Conan can manage system packages, allowing you to install platform-specific dependencies easily. This is useful when you need to install platform-specific system packages. For example, you may need to install a package that provides a specific driver or graphics library that only works on a specific platform.

Conan provides a way to install system packages using the *system package manager* tool.

In this example, we are going to explore the steps needed to create a wrapper package around a system library and what is needed to consume it in a Conan package. Note that the package will not contain the binary artifacts, it will just manage to check/install them calling `system_requirements()` and the respective system package managers (e.g Apt, Yum). In this example, we are going to create a Conan package to wrap the system ncurses requirement and then show how to use this requirement in an application.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/tools/system/package_manager/
```

You will find the following tree structure:

```
.
├── conanfile.py
└── consumer
    ├── CMakeLists.txt
    ├── conanfile.py
    └── ncurses_version.c
```

The `conanfile.py` file is the recipe that wraps the ncurses system library. Finally, the **consumer** directory contains a simple C application that uses the ncurses library, we will visit it later.

When wrapping a pre-built system library, we do not need to build the project from source, only install the system library and package its information. In this case, we are going to check the **conanfile.py** file that packages the ncurses library first:

```
from conan import ConanFile
from conan.tools.system import package_manager
from conan.tools.gnu import PkgConfig
from conan.errors import ConanInvalidConfiguration

required_conan_version = ">=2.0"


class SysNcursesConan(ConanFile):
    name = "ncurses"
    version = "system"
    description = "A textual user interfaces that work across a wide variety of terminals
↪"
    topics = ("curses", "terminal", "toolkit")
    homepage = "https://invisible-mirror.net/archives/ncurses/"
    license = "MIT"
    package_type = "shared-library"
    settings = "os", "arch", "compiler", "build_type"

    def package_id(self):
        self.info.clear()

    def validate(self):
        supported_os = ["Linux", "Macos", "FreeBSD"]
        if self.settings.os not in supported_os:
            raise ConanInvalidConfiguration(f"{self.ref} wraps a system package only␣
↪supported by {supported_os}.")

    def system_requirements(self):
        dnf = package_manager.Dnf(self)
        dnf.install(["ncurses-devel"], update=True, check=True)

        yum = package_manager.Yum(self)
```

(continues on next page)

(continued from previous page)

```
        yum.install(["ncurses-devel"], update=True, check=True)

        apt = package_manager.Apt(self)
        apt.install(["libncurses-dev"], update=True, check=True)

        pacman = package_manager.PacMan(self)
        pacman.install(["ncurses"], update=True, check=True)

        zypper = package_manager.Zypper(self)
        zypper.install(["ncurses"], update=True, check=True)

        brew = package_manager.Brew(self)
        brew.install(["ncurses"], update=True, check=True)

        pkg = package_manager.Pkg(self)
        pkg.install(["ncurses"], update=True, check=True)

    def package_info(self):
        self.cpp_info.bindirs = []
        self.cpp_info.includedirs = []
        self.cpp_info.libdirs = []

        self.cpp_info.set_property("cmake_file_name", "Curses")
        self.cpp_info.set_property("cmake_target_name", "Curses::Curses")
        self.cpp_info.set_property("cmake_additional_variables_prefixes", ["CURSES",])

        pkg_config = PkgConfig(self, 'ncurses')
        pkg_config.fill_cpp_info(self.cpp_info, is_system=True)
```

In this **conanfile.py** file, we are using the *system package manager* tool to install the ncurses library based on different package managers, under the *system_requirements* method. It's important to note that the `system_requirements` method is called always, when building, or even if the package is already installed. This is useful to ensure that the package is installed in the system.

Each package manager may vary the package name used to install the ncurses library, so we need to check the package manager documentation to find the correct package name first.

Another important detail is the **package_info** method. In this method, we are using the *PkgConfig* tool to fill the **cpp_info** data, based on the file `ncurses.pc` installed by the system package manager.

See *Using PkgConfig.fill_cpp_info with PKG_CONFIG_PATH from a profile* for a variant of this pattern in a cross-compilation scenario, where the `.pc` files are provided by a sysroot pointed to by `PKG_CONFIG_PATH` in the profile `[buildenv]`.

Now, let's install the ncurses library using the **conanfile.py** file:

```
$ conan create . --build=missing -c tools.system.package_manager:mode=install -c tools.
↪system.package_manager:sudo=true
```

Note that we are using the *Conan configuration* `tools.system.package_manager:mode` as **install**, otherwise, Conan will not install the system package, but check if it is installed only. The same for `tools.system.package_manager:sudo` as **True** to run the package manager with root privileges. As a result of this command, you should be able to see the **ncurses** library installed in your system, in case not been installed yet.

Now, let's check the **consumer** directory. This directory contains a simple C application that uses the ncurses library.

The **conanfile.py** file in the **consumer** directory is:

```
from conan import ConanFile
from conan.tools.build import can_run
from conan.tools.cmake import cmake_layout, CMake
import os


class AppNCursesVersionConan(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "CMakeDeps", "CMakeToolchain"
    package_type = "application"
    exports_sources = "CMakeLists.txt", "ncurses_version.c"

    def requirements(self):
        if self.settings.os in ["Linux", "Macos", "FreeBSD"]:
            self.requires("ncurses/system")

    def layout(self):
        cmake_layout(self)

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

        app_path = os.path.join(self.build_folder, "ncurses_version")
        self.output.info(f"The example application has been successfully built.\nPlease␣
↪run the executable using: '{app_path}'")
```

The recipe is simple. It requires the **ncurses** package we just created and uses the **CMake** tool to build the application. Once the application is built, it shows the **ncurses_version** application path, so you can run it manually as you wish and check its output.

The **ncurses_version.c** file is a simple C application that uses the ncurses library to print the ncurses version, but using white background and blue text:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include <ncurses.h>


int main(void) {
    int max_y, max_x;
    char message [256] = {0};

    initscr();

    start_color();
    init_pair(1, COLOR_BLUE, COLOR_WHITE);
    getmaxyx(stdscr, max_y, max_x);
```

(continues on next page)

(continued from previous page)

```
    snprintf(message, sizeof(message), "Conan 2.x Examples - Installed ncurses version:
↪%s\n", curses_version());
    attron(COLOR_PAIR(1));
    mvprintw(max_y / 2, max_x / 2 - (strlen(message) / 2), "%s", message);
    attroff(COLOR_PAIR(1));

    refresh();

    return EXIT_SUCCESS;
}
```

The **CMakeLists.txt** file is a simple CMake file that builds the **ncurses_version** application:

```
cmake_minimum_required(VERSION 3.15)
project(ncurses_version C)

find_package(Curses CONFIG REQUIRED)

add_executable(${PROJECT_NAME} ncurses_version.c)
target_link_libraries(${PROJECT_NAME} PRIVATE Curses::Curses)
```

The CMake target **Curses::Curses** is provided by the **ncurses** package we just created. It follows the official CMake module for FindCurses. The information about libraries and include directories is now available in the **cpp_info** object, as we filled it using the **PkgConfig** tool.

Now, let's build the application:

```
$ cd consumer/
$ conan build . --name=ncurses-version --version=0.1.0
  ...
  conanfile.py (ncurses-version/0.1.0): The example application has been successfully␣
↪built.
  Please run the executable using: '/tmp/consumer/build/Release/ncurses_version'
```

After building the application, it will show the executable path. You can run it to check the output:

```
$ /tmp/consumer/build/Release/ncurses_version

Conan 2.x Examples - Installed ncurses version: ncurses 6.0.20160213
```

Don't worry if the displayed version is different from the one shown here or the executable path different. It depends on the version installed in your system and where you built the application.

That's it! You have successfully packaged a system library and consumed it in a Conan package.

Wrapping a library installed in the system as a Conan package

As a variant of the above case, it is also possible to apply the above strategy to libraries that are installed in the system, but not necessarily installed by the system package manager, nor necessarily in the common system locations where the compilers will find them by default.

Suppose that there is an existing library, already compiled in a user folder such as:

```
/home/myuser/mymath
                └── include
                    ├── mymath.h
                └── lib
                    ├── mymath.lib
```

And `/home/myuser/mymath` is not added to the compilers default paths or anything like that.

In general, a more recommended approach is to create a full package from those precompiled binaries, and upload that package, and then manage it as any other regular package. See the tutorial about *creating packages from pre-compiled binaries here*.

But in some scenarios, it might still be desirable to use that library from its installed location `/home/myuser/mymath` without putting the artifacts inside a Conan package. This can be done with a "wrapper" recipe, similar to the one above, but which does not have any `system_requirements()` method.

It could be something like:

```
from conan import ConanFile

class MyMath(ConanFile):
    name = "mymath"
    version = "1.2"  # In this case an actual version might make more sense
    package_type = "static-library"

    def package_info(self):
        self.cpp_info.bindirs = []
        # Absolute paths are allowed here
        self.cpp_info.includedirs = ["/home/myuser/mymath/include"]
        self.cpp_info.libdirs = ["/home/myuser/mymath/lib"]
        self.cpp_info.libs = ["mymath"]
```

Note that it is also possible to still do conditions based on settings, in case that the library is installed in the system in different locations based on the platform:

```
settings = "os"

def package_info(self):
    self.cpp_info.bindirs = []
    # Absolute paths are allowed here
    if self.settings.os == "Windows":
        self.cpp_info.includedirs = ["C:/Users/myuser/mymath/include"]
        self.cpp_info.libdirs = ["C:/Users/myuser/mymath/lib"]
    else:
        self.cpp_info.includedirs = ["/home/myuser/mymath/include"]
        self.cpp_info.libdirs = ["/home/myuser/mymath/lib"]
    self.cpp_info.libs = ["mymath"]
```

It might even be possible to parametrize those absolute paths with some environment variable specific for that platform too.

Note: Best practices

- The use of "wrapper" recipes like this one should be minimized, as it makes reproducibility and traceability harder. Creating a real package putting the headers and libraries inside it, uploading it to the server, makes it possible to achieve such traceability and reproducibility.
- This type of "wrapper" recipe can be convenient together with the `[replace_requires]` feature, for specific platform constraints, like a platform that mandates that some `openssl` library must be the one contained in a sysroot, not the one from the Conan package `openssl/version`, but in general, such a dependency to `openssl/version` is required by other packages. In those cases, writing a wrapper recipe around the sysroot `openssl` and using `[replace_requires]` to force the dependency graph to resolve to it could make sense.

Consuming system requirements only when building a package

In some cases, you may want to consume system requirements only when building a package, but not when installing it. It can be useful when you want to build a package in a CI/CD pipeline, but you don't want to run the system package manager when installing the Conan package in a different environment. For those cases, there are few approaches that can be used to achieve this goal.

Consume a Conan package wrapper for a system package as build requirement

In this approach, you can use a Conan package for a *wrapped system package*. Then, the package can be consumed regularly by the method *build_requirements()*.

```
from conan import ConanFile

class MyPackage(ConanFile):
    name = "mypackage"
    settings = "os", "compiler", "build_type", "arch"

    def build_requirements(self):
        self.tool_requires("ncurses/system")

    ...
```

This ensures that downstream consumers of the package *mypackage* will not directly invoke the system package manager (e.g., apt-get). Only the direct package consumer of the system wrap package for `ncurses` will execute the system package manager when building the package.

Centralizing and wrapping `ncurses` in a separated recipe makes it reusable across multiple cases and is good practice to avoid code duplication.

Consume the system package directly in the build() method

In case wanting to run the system package manager only when building the package, but not having a Conan package to wrap the system library information, it's possible to run the system package manager in the **build()** method:

```
from conan import ConanFile
from conan.tools.system import package_manager

class MyPackage(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    ...

    def build(self):
        if self.settings.os == "Linux":
            apt = package_manager.Apt(self)
            apt.install(["libncurses-dev"], update=True, check=True)
```

This way, the system package manager will be called only when building the package, not when installing it. There is the advantage of not needed to create a separated Conan package to wrap the system library information, this is a much simpler case, when only a single recipe need to install the system package.

Still, this approach may lead to code duplication if multiple recipes consume the same system package. It is recommended to use this method sparingly and only for well-contained cases.

8.4 Cross-building examples

8.4.1 Creating a Conan package for a toolchain

After learning how to create recipes for tool requires that package applications, we are going to show an example on how to create a recipe that packages a precompiled toolchain or compiler for building other packages.

In the "*How to cross-compile your applications using Conan: host and build contexts*" tutorial section, we discussed the basics of cross-compiling applications using Conan with a focus on the "build" and "host" contexts. We learned how to configure Conan to use different profiles for the build machine and the target host machine, enabling us to cross-compile applications for platforms like Raspberry Pi from an Ubuntu Linux machine.

However, in that section, we assumed the existence of a cross-compiling toolchain or compiler as part of the build environment, set up through Conan profiles. Now, we will take a step further by demonstrating how to create a Conan package for such a toolchain. This package can then be used as a *tool_require* in other Conan recipes, simplifying the process of setting up the environment for cross-compilation.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/cross_build/toolchain_packages/toolchain
```

Here, you will find a Conan recipe (and the *test_package*) to package an ARM toolchain for cross-compiling to Linux ARM for both 32 and 64 bits. To simplify a bit, we are assuming that we can just cross-build from Linux x86_64 to Linux ARM, both 32 and 64 bits. If you're looking for another example, you can explore an additional MacOs to Linux cross-build example right here .

```
.
├── conanfile.py
```

(continues on next page)

(continued from previous page)

```
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── test_package.cpp
```

Let's check the recipe and go through the most relevant parts:

Listing 49: conanfile.py

```
import os
from conan import ConanFile
from conan.tools.files import get, copy, save
from conan.errors import ConanInvalidConfiguration
from conan.tools.scm import Version

class ArmToolchainPackage(ConanFile):
    name = "arm-toolchain"
    version = "13.2"
    ...
    settings = "os", "arch"
    package_type = "application"

    def _archs32(self):
        return ["armv6", "armv7", "armv7hf"]

    def _archs64(self):
        return ["armv8", "armv8.3"]

    def _get_toolchain(self, target_arch):
        if target_arch in self._archs32():
            return ("arm-none-linux-gnueabihf",
                    "df0f4927a67d1fd366ff81e40bd8c385a9324fbdde60437a512d106215f257b3")
        else:
            return ("aarch64-none-linux-gnu",
                    "12fcdf13a7430655229b20438a49e8566e26551ba08759922cdaf4695b0d4e23")

    def validate(self):
        if self.settings.arch != "x86_64" or self.settings.os != "Linux":
            raise ConanInvalidConfiguration(f"This toolchain is not compatible with
↪{self.settings.os}-{self.settings.arch}. "
                                            "It can only run on Linux-x86_64.")

        valid_archs = self._archs32() + self._archs64()
        if self.settings_target.os != "Linux" or self.settings_target.arch not in valid_
↪archs:
            raise ConanInvalidConfiguration(f"This toolchain only supports building for␣
↪Linux-{valid_archs.join(',')}. "
                                            f"{self.settings_target.os}-{self.settings_
↪target.arch} is not supported.")

        if self.settings_target.compiler != "gcc":
            raise ConanInvalidConfiguration(f"The compiler is set to '{self.settings_
```

(continues on next page)

(continued from previous page)

```
↪target.compiler}', but this "
                                            "toolchain only supports building with gcc.")

        if Version(self.settings_target.compiler.version) >= Version("14") or␣
↪Version(self.settings_target.compiler.version) < Version("13"):
            raise ConanInvalidConfiguration(f"Invalid gcc version '{self.settings_target.
↪compiler.version}'. "
                                            "Only 13.X versions are supported for the␣
↪compiler.")

    def source(self):
        # The ARM toolchain is distributed under GPL-3.0-only license
        # Reference: https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads
        save(self, "LICENSE", "ARM GNU Toolchain\n"
             "License: GNU General Public License v3.0 (GPL-3.0-only)\n"
             "https://www.gnu.org/licenses/gpl-3.0.html\n\n"
             "EULA: https://developer.arm.com/GetEula?Id=37988a7c-c40e-4b78-9fd1-
↪62c20b507aa8\n")

    def build(self):
        toolchain, sha = self._get_toolchain(self.settings_target.arch)
        get(self, f"https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu/
↪13.2.rel1/binrel/arm-gnu-toolchain-13.2.rel1-x86_64-{toolchain}.tar.xz",
            sha256=sha, strip_root=True)

    def package_id(self):
        self.info.settings_target = self.settings_target
        # We only want the ``arch`` setting
        self.info.settings_target.rm_safe("os")
        self.info.settings_target.rm_safe("compiler")
        self.info.settings_target.rm_safe("build_type")

    def package(self):
        toolchain, _ = self._get_toolchain(self.settings_target.arch)
        dirs_to_copy = [toolchain, "bin", "include", "lib", "libexec"]
        for dir_name in dirs_to_copy:
            copy(self, pattern=f"{dir_name}/*", src=self.build_folder, dst=self.package_
↪folder, keep_path=True)
        copy(self, "LICENSE", src=self.build_folder, dst=os.path.join(self.package_
↪folder, "licenses"), keep_path=False)

    def package_info(self):
        toolchain, _ = self._get_toolchain(self.settings_target.arch)
        self.cpp_info.bindirs.append(os.path.join(self.package_folder, toolchain, "bin"))

        self.conf_info.define("tools.build:compiler_executables", {
            "c":   f"{toolchain}-gcc",
            "cpp": f"{toolchain}-g++",
            "asm": f"{toolchain}-as"
        })
```

Validating the toolchain package: settings, settings_build and settings_target

As you may recall, the *validate() method* is used to indicate that a package is not compatible with certain configurations. As mentioned earlier, we are limiting the usage of this package to a *Linux x86_64* platform for cross-compiling to a *Linux ARM* target, supporting both 32-bit and 64-bit architectures. Let's check how we incorporate this information into the `validate()` method and discuss the various types of settings involved:

Validating the build platform

```
...

settings = "os", "arch"

...

def validate(self):

    if self.settings.arch != "x86_64" or self.settings.os != "Linux":
        raise ConanInvalidConfiguration(f"This toolchain is not compatible with {self.
↪settings.os}-{self.settings.arch}. "
                                        "It can only run on Linux-x86_64.")
    ...
```

First, it's important to acknowledge that only the `os` and `arch` settings are declared. These settings represent the machine that will compile the package for the toolchain, so we only need to verify that they correspond to `Linux` and `x86_64`, as these are the platforms for which the toolchain binaries are intended.

It is important to note that for this package, which is to be used as a `tool_requires`, these settings do not relate to the `host` profile but to the `build` profile. This distinction is recognized by Conan when creating the package with the `--build-require` argument. This will make the `settings` and the `settings_build` to be equal within the context of package creation.

Validating the target platform

In scenarios involving cross-compilation, validations regarding the target platform, where the executable generated by the toolchain's compilers will run, must refer to the `settings_target`. These settings come from the information in the `host` profile. For instance, if compiling for a Raspberry Pi, that will be the information stored in the `settings_target`. Again, Conan is aware that `settings_target` should be populated with the `host` profile information due to the use of the `--build-require` flag during package creation.

```
def validate(self):
    ...

    valid_archs = self._archs32() + self._archs64()
    if self.settings_target.os != "Linux" or self.settings_target.arch not in valid_
↪archs:
        raise ConanInvalidConfiguration(f"This toolchain only supports building for␣
↪Linux-{valid_archs.join(',')}. "
                                        f"{self.settings_target.os}-{self.settings_target.
↪arch} is not supported.")

    if self.settings_target.compiler != "gcc":
        raise ConanInvalidConfiguration(f"The compiler is set to '{self.settings_target.
↪compiler}', but this "
                                        "toolchain only supports building with gcc.")
```

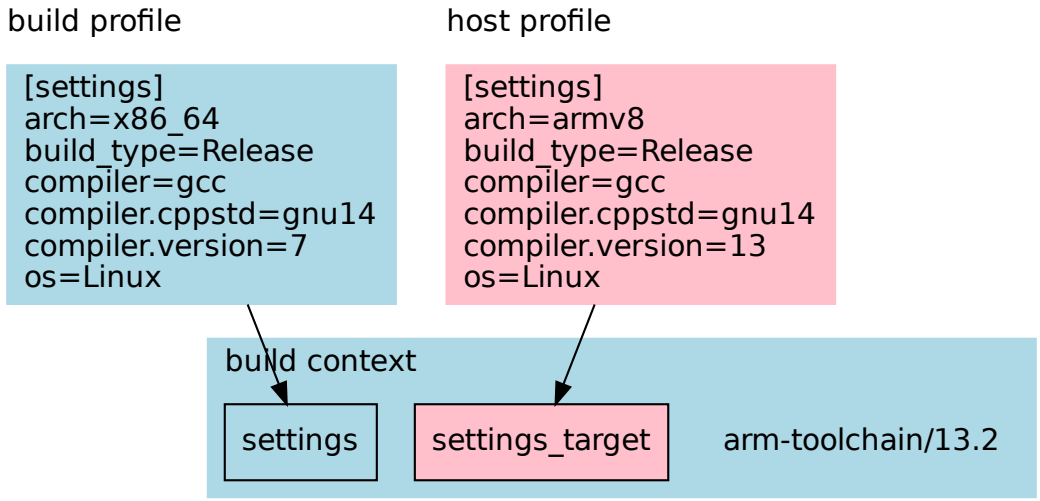
(continues on next page)

(continued from previous page)

```
    if Version(self.settings_target.compiler.version) >= Version("14") or Version(self.
↪settings_target.compiler.version) < Version("13"):
        raise ConanInvalidConfiguration(f"Invalid gcc version '{self.settings_target.
↪compiler.version}'. "
                                        "Only 13.X versions are supported for the␣
↪compiler.")
```

As you can see, several verifications are made to ensure the validity of the operating system and architectures for the resulting binaries' execution environment. Additionally, it verifies that the compiler's name and version align with the expectations for the `host` context.

Here, the diagram shows both profiles and which settings are picked for the **arm-toolchain** recipe that is in the *build* context.



Downloading the binaries for the toolchain and packaging it

```
...

def _archs32(self):
    return ["armv6", "armv7", "armv7hf"]

def _archs64(self):
    return ["armv8", "armv8.3"]

def _get_toolchain(self, target_arch):
    if target_arch in self._archs32():
        return ("arm-none-linux-gnueabihf",
                "df0f4927a67d1fd366ff81e40bd8c385a9324fbdde60437a512d106215f257b3")
    else:
```

(continues on next page)

(continued from previous page)

```
        return ("aarch64-none-linux-gnu",
                "12fcdf13a7430655229b20438a49e8566e26551ba08759922cdaf4695b0d4e23")

def source(self):
    # The ARM toolchain is distributed under GPL-3.0-only license
    # Reference: https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads
    save(self, "LICENSE", "ARM GNU Toolchain\n"
         "License: GNU General Public License v3.0 (GPL-3.0-only)\n"
         "https://www.gnu.org/licenses/gpl-3.0.html\n\n"
         "EULA: https://developer.arm.com/GetEula?Id=37988a7c-c40e-4b78-9fd1-
↪62c20b507aa8\n")

def build(self):
    toolchain, sha = self._get_toolchain(self.settings_target.arch)
    get(self, f"https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu/13.2.
↪rel1/binrel/arm-gnu-toolchain-13.2.rel1-x86_64-{toolchain}.tar.xz",
        sha256=sha, strip_root=True)

def package(self):
    toolchain, _ = self._get_toolchain(self.settings_target.arch)
    dirs_to_copy = [toolchain, "bin", "include", "lib", "libexec"]
    for dir_name in dirs_to_copy:
        copy(self, pattern=f"{dir_name}/*", src=self.build_folder, dst=self.package_
↪folder, keep_path=True)
    copy(self, "LICENSE", src=self.source_folder, dst=os.path.join(self.package_folder,
↪"licenses"), keep_path=False)

...
```

The *source()* method is used to download or reference the recipe license. In this case, we reference the ARM toolchain's GPL-3.0 license and EULA. However, this is the only action performed there. The actual toolchain binaries are fetched in the *build()* method. This approach is necessary because the toolchain package is designed to support both 32-bit and 64-bit architectures, requiring us to download two distinct sets of toolchain binaries. Which binary the package ends up with depends on the *settings_target* architecture. This conditional downloading process can't happen in the *source()* method, as it *caches the downloaded contents*.

The *package()* method doesn't have anything out of the ordinary; it simply copies the downloaded files into the package folder, license included.

Adding `settings_target` to the Package ID information

In recipes designed for cross-compiling scenarios, particularly those involving toolchains that target specific architectures or operating systems, and the binary package can be different based on the target platform we may need to modify the `package_id()` to ensure that Conan correctly identifies and differentiates between binaries based on the target platform they are intended for.

In this case, we extend the `package_id()` method to include `settings_target`, which encapsulates the target platform's configuration (in this case if it's 32 or 64 bit):

```
def package_id(self):
    # Assign settings_target to the package ID to differentiate binaries by target␣
↪platform.
```

(continues on next page)

(continued from previous page)

```
    self.info.settings_target = self.settings_target

    # We only want the ``arch`` setting
    self.info.settings_target.rm_safe("os")
    self.info.settings_target.rm_safe("compiler")
    self.info.settings_target.rm_safe("build_type")
```

By specifying `self.info.settings_target = self.settings_target`, we explicitly instruct Conan to consider the target platform's settings when generating the package ID. In this case we remove `os`, `compiler` and `build_type` settings as changing them will not be relevant for selecting the toolchain we will use for building and leave only the `arch` setting that will be used to decide if want to produce binaries for 32 or 64 bits.

Define information for consumers

In the `package_info()` method we define all the information that consumers need to have available when using the toolchain:

```
def package_info(self):
    toolchain, _ = self._get_toolchain(self.settings_target.arch)
    self.cpp_info.bindirs.append(os.path.join(self.package_folder, toolchain, "bin"))

    self.conf_info.define("tools.build:compiler_executables", {
        "c":   f"{toolchain}-gcc",
        "cpp": f"{toolchain}-g++",
        "asm": f"{toolchain}-as"
    })
```

In this case, we need to define the following information:

- Add directories containing toolchain tools that may be required during compilation. The toolchain we download will store its tools in both `bin` and `<toolchain_triplet>/bin`. Since `self.cpp_info.bindirs` defaults to `bin`, we only need to add the directory specific to the triplet. Note that it's not necessary to define environment information to add these directories to the `PATH`, as Conan will manage this through the *VirtualRunEnv*.
- We define the `tools.build:compiler_executables` configuration. This configuration will be considered in several generators, like *CMakeToolchain*, *MesonToolchain*, or *AutotoolsToolchain*, to direct to the appropriate compiler binaries.

Testing the Conan toolchain package

We also added a simple *test_package* to test the toolchain:

Listing 50: test_package/conanfile.py

```
import os
from io import StringIO

from conan import ConanFile
from conan.tools.cmake import CMake, cmake_layout


class TestPackageConan(ConanFile):
```

(continues on next page)

(continued from previous page)

```
    settings = "os", "arch", "compiler", "build_type"
    generators = "CMakeToolchain", "VirtualBuildEnv"

    def build_requirements(self):
        self.tool_requires(self.tested_reference_str)

    def layout(self):
        cmake_layout(self)

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def test(self):
        if self.settings.arch in ["armv6", "armv7", "armv7hf"]:
            toolchain = "arm-none-linux-gnueabihf"
        else:
            toolchain = "aarch64-none-linux-gnu"
        self.run(f"{toolchain}-gcc --version")
        test_file = os.path.join(self.cpp.build.bindirs[0], "test_package")
        stdout = StringIO()
        self.run(f"file {test_file}", stdout=stdout)
        if toolchain == "aarch64-none-linux-gnu":
            assert "ELF 64-bit" in stdout.getvalue()
        else:
            assert "ELF 32-bit" in stdout.getvalue()
```

This test package ensures that the toolchain is functional, building a minimal *hello world* program and that binaries produced with it are correctly targeted for the specified architecture.

Cross-build an application using the toolchain

Having detailed the toolchain recipe, it's time to proceed with package creation:

```
$ conan create . -pr:b=default -pr:h=../profiles/raspberry-64 --build-require

======== Exporting recipe to the cache ========
...
======== Input profiles ========
Profile host:
[settings]
arch=armv8
build_type=Release
compiler=gcc
compiler.cppstd=gnu14
compiler.libcxx=libstdc++11
compiler.version=13
os=Linux

Profile build:
```

(continues on next page)

(continued from previous page)

```
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu14
compiler.libcxx=libstdc++11
compiler.version=7
os=Linux
...
======== Testing the package: Executing test ========
arm-toolchain/13.2 (test package): Running test()
arm-toolchain/13.2 (test package): RUN: aarch64-none-linux-gnu-gcc --version
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 13.2.rel1 (Build arm-13.7)) 13.2.1 20231009
Copyright (C) 2023 Free Software Foundation, Inc.
...
```

Important: Use `--build-require` argument.

The `conan create` command by default creates packages for the "host" context, using the "host" profile. But if the package we are creating is intended to be used as a tool with `tool_requires`, then it needs to be built for the "build" context instead.

The `--build-require` argument specifies this. When this argument is provided, the current recipe binary will be built for the "build" context, in this case using the `default` profile, and it will receive the `raspberry-64` "host" profile settings as `settings_target`. The `arm-toolchain/13.2` package is a package which executables run in the current "build" machine, not in the RaspberryPI, but it is a tool that targets the RaspberryPI.

The `--build-require` argument is necessary to build the `arm-toolchain` package correctly as a build tool.

With the toolchain package prepared, we proceed to build an actual application. This will be the same application previously cross-compiled in the *How to cross-compile your applications using Conan: host and build contexts* section. However, this time, we incorporate the toolchain package as a dependency within the host profile. This ensures the toolchain is used to build the application and all its dependencies

```
$ cd .. && cd consumer
$ conan install . -pr:b=default -pr:h=../profiles/raspberry-64 -pr:h=../profiles/arm-
↪toolchain --build missing
$ cmake --preset conan-release
$ cmake --build --preset conan-release
$ file ./build/Release/compressor
compressor: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), dynamically
linked, interpreter /lib/ld-linux-aarch64.so.1, for GNU/Linux 3.7.0, with debug_info,
not stripped
```

We composed the already existing profile with another profile called `arm-toolchain` that just has the `tool_requires` added:

```
[tool_requires]
arm-toolchain/13.2
```

During this procedure, the zlib dependency will also be compiled for ARM 64-bit architecture if it hasn't already been. Additionally, it's important to verify the architecture of the resulting executable, confirming its alignment with the targeted 64-bit architecture.

See also:

- *More info on settings_target*
- *Cross-compile your applications using Conan*
- Another example of cross-compilation from Macos to Linux

8.4.2 Cross building to Android with the NDK

In this example, we are going to see how to cross-build a Conan package to Android.

First of all, download the Android NDK from the download page and unzip it. In MacOS you can also install it with `brew install android-ndk`.

Then go to the `profiles` folder in the conan config home directory (check it running **`conan config home`**) and create a file named `android` with the following contents:

```
include(default)

[settings]
# Just an example, you need to use your real settings
os=Android
os.api_level=27
arch=armv8
compiler=clang
compiler.version=18
compiler.libcxx=c++_shared
compiler.cppstd=17

[conf]
# Use your path here
tools.android:ndk_path=/usr/local/share/android-ndk
```

You might need to modify:

- `compiler.version`: Check the NDK documentation or find a `bin` folder containing the compiler executables like `x86_64-linux-android31-clang`. In a Macos installation it is found in the NDK path + `toolchains/llvm/prebuilt/darwin-x86_64/bin`. Run `./x86_64-linux-android31-clang --version` to check the running `clang` version and adjust the profile.
- `compiler.libcxx`: The supported values are `c++_static` and `c++_shared`.
- `compiler.cppstd`: The C++ standard version, adjust as your needs.
- `os.api_level`: You can check here the usage of each Android Version/API level and choose the one that fits better with your requirements. This is typically a balance between new features and more compatible applications.
- `arch`: There are several architectures supported by Android: `x86`, `x86_64`, `armv7`, and `armv8`.
- `tools.android:ndk_path` conf: Write the location of the unzipped NDK.

If you are in Windows, it is necessary to have a make-like build system like MinGW-Make or Ninja. We can provision for Ninja directly in our profile with `[tool_requires]`:

```
...
[conf]
# Use your path here
tools.android:ndk_path=C:\ws\android\android-ndk-r27
```

(continues on next page)

(continued from previous page)

```
tools.cmake.cmaketoolchain:generator=Ninja

[tool_requires]
ninja/[*]
```

Use the **conan new** command to create a "Hello World" C++ library example project:

```
$ conan new cmake_lib -d name=hello -d version=1.0
```

Then we can specify the `android` profile and our hello library will be built for Android:

```
$ conan create . --profile android

[ 50%] Building CXX object CMakeFiles/hello.dir/src/hello.cpp.o
[100%] Linking CXX static library libhello.a
[100%] Built target hello
...
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
[100%] Linking CXX executable example
[100%] Built target example
```

Both the library and the `test_package` executable are built for Android, so we cannot use them in our local computer.

Unless you have access to a *root* Android device, running the test application or using the built library is not possible directly so it is more common to build an Android application that uses the `hello` library.

It is also possible to use the `android-ndk` from a Conan `tool-requires`. There is already a Conan package in ConanCenter containing the AndroidNDK, so writing a profile like:

```
[settings]
os=Android
os.api_level=27
arch=armv8
compiler=clang
compiler.version=18
compiler.libcxx=c++_shared
compiler.cppstd=17
build_type=Release

# You might need Ninja conf and tool-requires in Windows too
[tool_requires]
android-ndk/[*]
```

And this will download automatically the latest android-ndk from ConanCenter and inject and apply it automatically to build the package. Note that to use packages from ConanCenter in production the *following approach is recommended*

See also:

- Check the example *Integrating Conan in Android Studio* to know how to use your c++ libraries in a native Android application.
- Check the tutorial *How to cross-compile your applications using Conan*.

8.4.3 Integrating Conan in Android Studio

At the *Cross building to Android with the NDK* we learned how to build a package for Android using the NDK. In this example we are going to learn how to do it with the Android Studio and how to use the libraries in a real Android application.

Creating a new project

First of all, download and install the Android Studio IDE.

Then create a `new project` selecting `Native C++` from the templates.

In the next wizard window, select a name for your application, for example *MyConanApplication*, you can leave the "Minimum SDK" with the suggested value (21 in our case), but remember the value as we are using it later in the Conan profile at `os.api_level`

In the "Build configuration language" you can choose between `Groovy DSL (build.gradle)` or `Kotlin DSL (build.gradle.kts)` in order to use *conanInstall* task bellow.

Select a "C++ Standard" in the next window, again, remember the choice as later we should use the same in the profile at `compiler.cppstd`.

In the project generated with the wizard we have a folder `cpp` with a `native-lib.cpp`. We are going to modify that file to use `zlib` and print a message with the used `zlib` version. Copy only the highlighted lines, it is important to keep the function name.

Listing 51: native-lib.cpp

```
#include <jni.h>
#include <string>
#include "zlib.h"

extern "C" JNIEXPORT jstring JNICALL
Java_com_example_myconanapp_MainActivity_stringFromJNI(
        JNIEnv* env,
        jobject /* this */) {
    std::string hello = "Hello from C++, zlib version: ";
    hello.append(zlibVersion());
    return env->NewStringUTF(hello.c_str());
}
```

Now we are going to learn how to introduce a requirement to the `zlib` library and how to prepare our project.

Introducing dependencies with Conan

conanfile.txt

We need to provide the `zlib` package with Conan. Create a file `conanfile.txt` in the `cpp` folder:

Listing 52: conanfile.txt

```
[requires]
zlib/1.2.12

[generators]
```

(continues on next page)

(continued from previous page)

```
CMakeToolchain
CMakeDeps

[layout]
cmake_layout
```

build.gradle

We are going to automate calling `conan install` before building the Android project, so the requires are prepared, open the `build.gradle` file in the `My_Conan_App.app` (Find it in the *Gradle Scripts* section of the Android project view). Paste the `task conanInstall` contents after the `plugins` and before the `android` elements:

Groovy

Kotlin

Listing 53: build.gradle

```
plugins {
 ...
}

task conanInstall {
    def conanExecutable = "conan" // define the path to your conan installation
    def buildDir = new File("app/build")
    buildDir.mkdirs()
    ["Debug", "Release"].each { String build_type ->
        ["armv7", "armv8", "x86", "x86_64"].each { String arch ->
            def cmd = conanExecutable + " install " +
                      "../src/main/cpp --profile android -s build_type="+ build_type +" -
↪s arch=" + arch +
                      " --build missing -c tools.cmake.cmake_layout:build_folder_vars=[
↪'settings.arch']"
            print(">> ${cmd} \n")

            def sout = new StringBuilder(), serr = new StringBuilder()
            def proc = cmd.execute(null, buildDir)
            proc.consumeProcessOutput(sout, serr)
            proc.waitFor()
            println "$sout $serr"
            if (proc.exitValue() != 0) {
                throw new Exception("out> $sout err> $serr" + "\nCommand: ${cmd}")
            }
        }
    }
}

android {
    compileSdk 32

    defaultConfig {
```

(continues on next page)

(continued from previous page)

```
...
```

Listing 54: build.gradle.kts

```
plugins {
 ...
}

tasks.register("conanInstall") {
    val conanExecutable = "conan" // define the path to your conan installation
    val buildDir = file("app/build")
    buildDir.mkdirs()

    val buildTypes = listOf("Debug", "Release")
    val architectures = listOf("armv7", "armv8", "x86", "x86_64")

    doLast {
        buildTypes.forEach { buildType ->
            architectures.forEach { arch ->
                val cmd = "$conanExecutable install ../../src/main/cpp --profile android-
↪studio " +
                          "-s build_type=$buildType -s arch=$arch --build missing " +
                          "-c tools.cmake.cmake_layout:build_folder_vars=['settings.arch
↪']"

                println(">> $cmd")

                val proc = ProcessBuilder(cmd.split(" "))
                    .directory(buildDir)
                    .start()

                val result = proc.inputStream.bufferedReader().readText()
                val errors = proc.errorStream.bufferedReader().readText()

                proc.waitFor()

                if (proc.exitValue() != 0) {
                    throw Exception("Execution failed! Output: $result Error: $errors")
                }
                println(result)
                if (errors.isNotBlank()) {
                    println("Errors: $errors")
                }
            }
        }
    }
}

tasks.named("preBuild").configure {
    dependsOn("conanInstall")
}
```

(continues on next page)

(continued from previous page)

```
android {
   compileSdk 32

   defaultConfig {

...
```

The `conanInstall` task is calling **`conan install`** for Debug/Release and for each architecture we want to build, you can adjust these values to match your requirements.

If we focus on the `conan install` task we can see:

1. We are passing a `--profile android`, so we need to create the profile. Go to the `profiles` folder in the conan config home directory (check it running **`conan config home`**) and create a file named `android` with the following contents:

   System NDK

   Conan NDK package

   ```
   include(default)

   [settings]
   os=Android
   os.api_level=27
   compiler=clang
   compiler.version=18
   compiler.libcxx=c++_static
   compiler.cppstd=17

   [conf]
   tools.android:ndk_path=/opt/homebrew/share/android-ndk
   ```

   ```
   include(default)

   [settings]
   os=Android
   os.api_level=27
   compiler=clang
   compiler.version=18
   compiler.libcxx=c++_static
   compiler.cppstd=17

   [tool_requires]
   *: android-ndk/r27
   ```

   You might need to modify:

   - `tools.android:ndk_path` conf: The location of the NDK provided by Android Studio. You should be able to see the path to the NDK if you open the `cpp/includes` folder in your IDE.
   - `compiler.version`: Check the NDK documentation or find a `bin` folder containing the compiler executables like `x86_64-linux-android31-clang`. In a Macos installation it is found in the NDK path `+ toolchains/llvm/prebuilt/darwin-x86_64/bin`. Run `./x86_64-linux-android31-clang --version` to check the running `clang` version and adjust the profile.

- `compiler.libcxx`: The supported values are `c++_static` and `c++_shared`.
- `compiler.cppstd`: The C++ standard version, this should be the value you selected in the Wizard.
- `os.api_level`: Use the same value you selected in the Wizard.

2. We are passing `-c tools.cmake.cmake_layout:build_folder_vars=['settings.arch']`, thanks to that, Conan will create a different folder for the specified `settings.arch` so we can have all the configurations available at the same time.

To make Conan work we need to pass CMake a custom toolchain. We can do it introducing a single line in the same file, in the `android/defaultConfig/externalNativeBuild/cmake` element:

Listing 55: build.gradle

```
android {
    compileSdk 32

    defaultConfig {
        applicationId "com.example.myconanapp"
        minSdk 27
        targetSdk 27
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
        externalNativeBuild {
            cmake {
                cppFlags '-v'
                arguments("-DCMAKE_TOOLCHAIN_FILE=conan_android_toolchain.cmake")
            }
        }
```

conan_android_toolchain.cmake

Create a file called `conan_android_toolchain.cmake` in the `cpp` folder, that file will be responsible of including the right toolchain depending on the `ANDROID_ABI` variable that indicates the build configuration that the IDE is currently running:

Listing 56: conan_android_toolchain.cmake

```
# During multiple stages of CMake configuration, the toolchain file is processed and␣
↪command-line
# variables may not be always available. The script exits prematurely if essential␣
↪variables are absent.

if ( NOT ANDROID_ABI OR NOT CMAKE_BUILD_TYPE )
    return()
endif()
if(${ANDROID_ABI} STREQUAL "x86_64")
    include("${CMAKE_CURRENT_LIST_DIR}/build/x86_64/${CMAKE_BUILD_TYPE}/generators/
↪conan_toolchain.cmake")
elseif(${ANDROID_ABI} STREQUAL "x86")
    include("${CMAKE_CURRENT_LIST_DIR}/build/x86/${CMAKE_BUILD_TYPE}/generators/conan_
```

(continues on next page)

(continued from previous page)

```
↪toolchain.cmake")
 elseif(${ANDROID_ABI} STREQUAL "arm64-v8a")
     include("${CMAKE_CURRENT_LIST_DIR}/build/armv8/${CMAKE_BUILD_TYPE}/generators/conan_
↪toolchain.cmake")
 elseif(${ANDROID_ABI} STREQUAL "armeabi-v7a")
     include("${CMAKE_CURRENT_LIST_DIR}/build/armv7/${CMAKE_BUILD_TYPE}/generators/conan_
↪toolchain.cmake")
 else()
     message(FATAL "Not supported configuration")
 endif()
```

CMakeLists.txt

Finally, we need to modify the `CMakeLists.txt` to link with the `zlib` library:

Listing 57: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.18.1)
project("myconanapp")
add_library(myconanapp SHARED native-lib.cpp)

find_library(log-lib log)

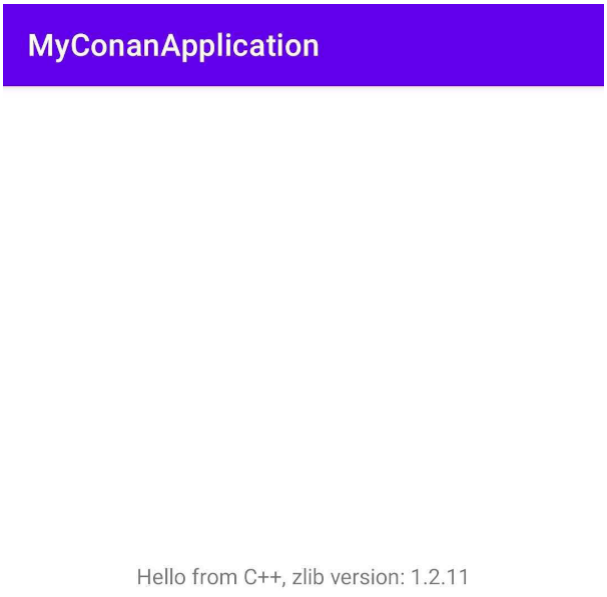
find_package(ZLIB CONFIG)

target_link_libraries(myconanapp ${log-lib} ZLIB::ZLIB)
```

Building the application

If we build our project we can see that *conan install* is called multiple times building the different configurations of `zlib`.

Then if we run the application in a Virtual Device or in a real device pairing it with the QR code we can see:



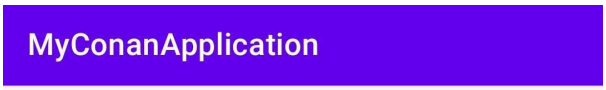
Once we have our project configured, it is very easy to change our dependencies and keep developing the application, for example, we can edit the `conanfile.txt` file and change the `zlib` to the version `1.12.2`:

```
[requires]
zlib/1.2.12

[generators]
CMakeToolchain
CMakeDeps

[layout]
cmake_layout
```

If we click build and then run the application, we will see that the zlib dependency has been updated:



Hello from C++, zlib version: 1.2.12

8.4.4 Cross-building with Emscripten - WebAssembly and asm.js

This example demonstrates how to cross-build a simple C++ project using Emscripten and Conan.

Conan supports WASM cross compilation, giving you the flexibility to target different JavaScript/WebAssembly runtimes in the browser.

We recommend creating separate Conan profiles for each target. Below are recommended profiles and instructions on how to build with them.

Setting up Conan profile for WebAssembly (WASM)

```
[settings]
arch=wasm
build_type=Release
compiler=emcc
compiler.cppstd=17
compiler.libcxx=libc++
# Optional settings to enable multithreading (see note below)
# compiler.threads=posix
compiler.version=4.0.10
os=Emscripten

[tool_requires]
emsdk/4.0.10

[conf]
# Optional settings to enable memory allocation
tools.build:exelinkflags=['-sALLOW_MEMORY_GROWTH=1', '-sMAXIMUM_MEMORY=4GB', '-sINITIAL_
↪MEMORY=64MB']
tools.build:sharedlinkflags=['-sALLOW_MEMORY_GROWTH=1', '-sMAXIMUM_MEMORY=4GB', '-
↪sINITIAL_MEMORY=64MB']
```

Note: Conan also supports building for asm.js targets, which is a nowadays considered deprecated.

What's the difference between asm.js and WASM?

- **asm.js** is a subset of JavaScript optimized for speed. It is fully supported by all browsers (even older ones) and compiles to a large `.js` file.
- **WebAssembly (WASM)** is a binary format that is smaller and faster to load and execute. Most modern browsers support it, and it is generally recommended for new projects. **WASM** is also easier to integrate with native browser APIs compared to **asm.js**.

Even though Emscripten is not a true runtime environment (like Linux or Windows), it is part of a toolchain ecosystem that compiles C/C++ to WebAssembly (WASM) and asm.js.

Conan uses `os=Emscripten` to:

- Align with the toolchain: Emscripten integrates the compiler, runtime glue, and JavaScript environment, making it practical to treat as an "OS-like" target.
- Support backward compatibility: Many recipes in Conan Center Index use `os=Emscripten` to enable or disable features and dependencies that specifically target Emscripten.
- Maintain stability: Changing this setting would break recipes that rely on it, and would complicate compatibility with alternative WASM toolchains.

Note: `wasm` arch refers to `WASM 32-bit` target architecture, which is the default. If you wish to target `WASM64`, set `arch=wasm64` in your profile. **Note that WASM64 is still experimental** and requires Node.js v20+ and a browser that supports it.

Important: According to emscripten documentation Emscripten supports two multithreading APIs:

- POSIX Threads API (`posix` in conan profile)
- Wasm Workers API (`wasm_workers` in conan profile)

These two APIs are incompatible with each other and incompatibles with binaries compiled without threading support. This incompatibility necessitates the modeling of threading usage within the compiler's binary model, allowing conan to distinguish between binaries compiled with threading and those compiled without it.

Conan will automatically set compiler and linker flags to enable threading if configured in the profile.

The profiles above use the `emsdk` package from Conan Center Index repository, which provides the Emscripten SDK, including `emcc`, `em++`, and tools like `emrun` and `node`.

If you prefer to use your system-installed Emscripten instead of the Conan-provided one, `tool_requires` could be replaced by custom `compiler_executables` and `buildenv`:

```
[conf]
tools.build:compiler_executables={'c':'/path/to/emcc', 'cpp':'/path/to/em++'}

[buildenv]
CC=emcc
CXX=em++
AR=emar
NM=emnm
RANLIB=emranlib
STRIP=emstrip
```

This way conan could configure *emsdk* local installation to be used from *CMake*, *Meson*, *Autotools* or other build systems.

In some cases, you might also need the `Emscripten.cmake` toolchain file for advanced scenarios. This toolchais is already added in our packaged *emsdk* but if you are using your own Emscripten installation, you can specify it in the profile by using *tools.cmake.cmaketoolchain:user_toolchain* and providing the absolute path to your toolchain file.

Note: The `tools.build:exelinkflags` and `tools.build:sharedlinkflags` in previous profiles are recomendations but users can modify them or define their values in the CMakeLists.txt file using the `set_target_properties()` command.

- By enabling `ALLOW_MEMORY_GROWTH` we allow the runtime to grow its memory dynamically at runtime by calling `emscripten_resize_heap()`. Without this flag, memory is allocated at startup and cannot grow.
- The `MAXIMUM_MEMORY` and `INITIAL_MEMORY` values specifies the maximum and initial memory size for the Emscripten runtime. These values can be adjusted based on your application's needs.

  Take into account that `arch=wasm64` has a theorical exabytes maximum memory size, but runtime currently limits it to 16GB, while `arch=wasm32` has a maximum memory size of 4GB and `arch=asm.js` has a maximum memory size of 2GB.

Important: `emcc` compiler does not guarantee any ABI compatibility between different versions (patches included) To ensure a new `package_id` is generated when the Emscripten version changes, it is recommended to update the `compiler.version` setting in your profile accordingly.

This will ensure that the package ID is generated based on the Emscripten version, allowing Conan to detect changes in the Emscripten toolchain and rebuild the project accordingly.

8.4.5 Building packages for TriCore

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

TriCore is an embedded microcontroller architecture used in multiple domains such as automotive. There are multiple compilers for TriCore, some of which can be found here. There are also `gcc` implementations for TriCore; this is what we will be using in the examples on this page.

Since Conan 2.7 there is some built-in support for this architecture:

- The default `settings.yml` contains architectures: `'tc131', 'tc16', 'tc161', 'tc162', 'tc18'`
- `CMakeToolchain` defines `CMAKE_SYSTEM_NAME=Generic-ELF` and `CMAKE_SYSTEM_PROCESSOR=tricore` for these architectures
- The compiler flags `-m<architecture>"` are injected as compiler and linker flags in `CMakeToolchain` and `AutotoolsToolchain`

That means that it is possible to define a profile like:

Listing 58: tricore.profile

```
[settings]
os=baremetal
arch=tc162
compiler=gcc
compiler.version=11
compiler.cppstd=20
compiler.libcxx=libstdc++11

[options]
*:fPIC=False
*:shared=False

[conf]
tools.build:compiler_executables={"c":"tricore-elf-gcc","cpp":"tricore-elf-g++"}
```

This assumes the compiler is installed in the system path, and its executables are called `tricore-elf-gcc` and `tricore-elf-g++`. And then, cross-build and create a package for TriCore using this profile, for example the default `cmake_lib`:

```
$ conan new cmake_lib -d name=mypkg -d version=0.1
$ conan create . -pr=tricore.profile
```

Note:

- This support is new and experimental. Please create a ticket in https://github.com/conan-io/conan/issues for any feedback or issues
- Linking applications (like if using `conan new cmake_exe`) requires a specific linker script, definition of entry-points, etc. Trying to build it as above will produce linking errors. We will try to add further examples for this case.

8.4.6 Cross-compiling from Linux to Windows with MinGW

It is possible to cross-build from Linux to Windows using the MinGW cross-compiler. Note that such a compiler won't be using the MSVC runtime, but the MinGW one, which uses the `libstdc++6.dll` runtime.

This blog post about Clang in Windows describes the different runtimes for the different Windows subsystems, which is equally applicable to MinGW.

The first step would be to install the compiler. In Debian based systems:

```
$ sudo apt install gcc-mingw-w64-x86-64-posix
$ sudo apt install g++-mingw-w64-x86-64-posix
```

If the compiler is installed in the system path, then we could write a profile like:

Listing 59: mingw

```
[settings]
os=Windows
compiler=gcc
compiler.version=10
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
arch=x86_64
build_type=Release

[buildenv]
CC=x86_64-w64-mingw32-gcc-posix
CXX=x86_64-w64-mingw32-g++-posix
```

Then, let's say that we have a basic CMake project, which we can create with the `conan new`:

```
$ conan new cmake_lib -d name=mypkg -d version=0.1
$ conan create . -pr=mingw

...
-- Using Conan toolchain: .../conan_toolchain.cmake
-- Conan toolchain: Defining architecture flag: -m64
-- Conan toolchain: C++ Standard 17 with extensions ON
-- The CXX compiler identification is GNU 10.0.0
-- Check for working CXX compiler: /usr/bin/x86_64-w64-mingw32-g++-posix - skipped

mypkg/0.1 (test package): Running CMake.build()
mypkg/0.1 (test package): RUN: cmake --build ...
gcc-10-x86_64-gnu17-release" -- -j8
[ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.obj
[100%] Linking CXX executable example.exe
[100%] Built target example
```

The example.exe will not be executed in the Linux machine, because the `test_package` contains a `if can_run(self)` branch to not run it in cross-build scenarios.

We can now take the `example.exe` and run it in a Windows machine:

```
mypkg/0.1: Hello World Release!
mypkg/0.1: _M_X64 defined
```

(continues on next page)

(continued from previous page)

```
mypkg/0.1: __x86_64__ defined
mypkg/0.1: _GLIBCXX_USE_CXX11_ABI 1
mypkg/0.1: MSVC runtime: MultiThreadedDLL
mypkg/0.1: __cplusplus201402
mypkg/0.1: __GNUC__10
mypkg/0.1: __MINGW32__1
mypkg/0.1: __MINGW64__1
```

Note:

- It is very possible that some recipes in ConanCenter are not prepared to be cross-built from Linux to Windows. The recommended way to build ConanCenter recipes is to build them with MSVC in Windows, as there might be limitations for the specific build-systems of the recipes, and MinGW support is not guaranteed.
- Trying to run the executables with some emulators like `wine` might require extra effort, because the runtime environment is intended to be Windows, and as such a `conanrun.bat` environment file will be created, but that cannot be executed in Linux. Using configurations like `-c tools.build.cross_building:can_run=True -c tools.microsoft.bash:subsystem=mingw -c tools.microsoft.bash:active=True` can allow to force the generation and execution of `conanrun.sh`.

8.5 Configuration files examples

8.5.1 Customize your settings: create your settings_user.yml

Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/examples/config_files/settings_user
```

In this example we are going to see how to customize your settings without overwriting the original **settings.yml** file.

Note: To understand better this example, it is highly recommended to read previously the reference about *settings.yml*.

Locate the settings_user.yml

First of all, let's have a look at the proposed `source/settings_user.yml`:

Listing 60: **settings_user.yml**

```
os:
    webOS:
        sdk_version: [null, "7.0.0", "6.0.1", "6.0.0"]
arch: ["cortexa15t2hf"]
compiler:
    gcc:
        version: ["13.0-rc"]
```

As you can see, we don't have to rewrite all the settings because they will be merged with the already defined in **settings.yml**.

Then, what are we adding through that `settings_user.yml` file?

- New OS: `webOS`, and its sub-setting: `sdk_version`.
- New `arch` available: `cortexa15t2hf`.
- New gcc version: `13.0-rc`.

Now, it's time to copy the file `source/settings_user.yml` into your `[CONAN_HOME]/` folder:

```
$ conan config install sources/settings_user.yml
Copying file settings_user.yml to /Users/myuser/.conan2/.
```

Use your new settings

After having copied the `settings_user.yml`, you should be able to use them for your recipes. Add this simple one into your local folder:

Listing 61: **conanfile.py**

```
from conan import ConanFile

class PkgConan(ConanFile):
    name = "pkg"
    version = "1.0"
    settings = "os", "compiler", "build_type", "arch"
```

Then, create several Conan packages (not binaries, as it does not have any source file for sure) to see that it's working correctly:

Listing 62: **Using the new OS and its sub-setting**

```
$ conan create . -s os=webOS -s os.sdk_version=7.0.0
...
Profile host:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu98
compiler.libcxx=libc++
compiler.version=12.0
os=webOS
os.sdk_version=7.0.0

Profile build:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu98
compiler.libcxx=libc++
```

(continues on next page)

(continued from previous page)

```
compiler.version=12.0
os=Macos
...
-------- Installing (downloading, building) binaries... --------
pkg/1.0: Copying sources to build folder
pkg/1.0: Building your package in /Users/myuser/.conan2/p/t/pkg929d53a5f06b1/b
pkg/1.0: Generating aggregated env files
pkg/1.0: Package 'a0d37d10fdb83a0414d7f4a1fb73da2c210211c6' built
pkg/1.0: Build folder /Users/myuser/.conan2/p/t/pkg929d53a5f06b1/b
pkg/1.0: Generated conaninfo.txt
pkg/1.0: Generating the package
pkg/1.0: Temporary package folder /Users/myuser/.conan2/p/t/pkg929d53a5f06b1/p
pkg/1.0 package(): WARN: No files in this package!
pkg/1.0: Package 'a0d37d10fdb83a0414d7f4a1fb73da2c210211c6' created
pkg/1.0: Created package revision 6a947a7b5669d6fde1a35ce5ff987fc6
pkg/1.0: Full package reference: pkg/1.0
↪#637fc1c7080faaa7e2cdccde1bcde118:a0d37d10fdb83a0414d7f4a1fb73da2c210211c6
↪#6a947a7b5669d6fde1a35ce5ff987fc6
pkg/1.0: Package folder /Users/myuser/.conan2/p/pkgb3950b1043542/p
```

Listing 63: **Using new gcc compiler version**

```
$ conan create . -s compiler=gcc -s compiler.version=13.0-rc -s compiler.
↪libcxx=libstdc++11
...
Profile host:
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.libcxx=libstdc++11
compiler.version=13.0-rc
os=Macos

Profile build:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu98
compiler.libcxx=libc++
compiler.version=12.0
os=Macos
...
-------- Installing (downloading, building) binaries... --------
pkg/1.0: Copying sources to build folder
pkg/1.0: Building your package in /Users/myuser/.conan2/p/t/pkg918904bbca9dc/b
pkg/1.0: Generating aggregated env files
pkg/1.0: Package '44a4588d3fe63ccc6e7480565d35be38d405718e' built
pkg/1.0: Build folder /Users/myuser/.conan2/p/t/pkg918904bbca9dc/b
pkg/1.0: Generated conaninfo.txt
pkg/1.0: Generating the package
```

(continues on next page)

(continued from previous page)

```
pkg/1.0: Temporary package folder /Users/myuser/.conan2/p/t/pkg918904bbca9dc/p
pkg/1.0 package(): WARN: No files in this package!
pkg/1.0: Package '44a4588d3fe63ccc6e7480565d35be38d405718e' created
pkg/1.0: Created package revision d913ec060e71cc56b10768afb9620094
pkg/1.0: Full package reference: pkg/1.0
↪#637fc1c7080faaa7e2cdccde1bcde118:44a4588d3fe63ccc6e7480565d35be38d405718e
↪#d913ec060e71cc56b10768afb9620094
pkg/1.0: Package folder /Users/myuser/.conan2/p/pkg789b624c93fc0/p
```

Listing 64: **Using the new OS and the new architecture**

```
$ conan create . -s os=webOS -s arch=cortexa15t2hf
...
Profile host:
[settings]
arch=cortexa15t2hf
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu98
compiler.libcxx=libc++
compiler.version=12.0
os=webOS

Profile build:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu98
compiler.libcxx=libc++
compiler.version=12.0
os=Macos
...
-------- Installing (downloading, building) binaries... --------
pkg/1.0: Copying sources to build folder
pkg/1.0: Building your package in /Users/myuser/.conan2/p/t/pkgde9b63a6bed0a/b
pkg/1.0: Generating aggregated env files
pkg/1.0: Package '19cf3cb5842b18dc78e5b0c574c1e71e7b0e17fc' built
pkg/1.0: Build folder /Users/myuser/.conan2/p/t/pkgde9b63a6bed0a/b
pkg/1.0: Generated conaninfo.txt
pkg/1.0: Generating the package
pkg/1.0: Temporary package folder /Users/myuser/.conan2/p/t/pkgde9b63a6bed0a/p
pkg/1.0 package(): WARN: No files in this package!
pkg/1.0: Package '19cf3cb5842b18dc78e5b0c574c1e71e7b0e17fc' created
pkg/1.0: Created package revision f5739d5a25b3757254dead01b30d3af0
pkg/1.0: Full package reference: pkg/1.0
↪#637fc1c7080faaa7e2cdccde1bcde118:19cf3cb5842b18dc78e5b0c574c1e71e7b0e17fc
↪#f5739d5a25b3757254dead01b30d3af0
pkg/1.0: Package folder /Users/myuser/.conan2/p/pkgd154182aac59e/p
```

As you could observe, each command has created a different package. That was completely right because we were using different settings for each one. If you want to see all the packages created, you can use the *conan list* command:

Listing 65: List all the *pkg/1.0*'s packages

```
$ conan list pkg/1.0:*
Local Cache
  pkg
    pkg/1.0
      revisions
        637fc1c7080faaa7e2cdccde1bcde118 (2023-02-16 06:42:10 UTC)
          packages
            19cf3cb5842b18dc78e5b0c574c1e71e7b0e17fc
              info
                settings
                  arch: cortexa15t2hf
                  build_type: Release
                  compiler: apple-clang
                  compiler.cppstd: gnu98
                  compiler.libcxx: libc++
                  compiler.version: 12.0
                  os: webOS
            44a4588d3fe63ccc6e7480565d35be38d405718e
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: gcc
                  compiler.libcxx: libstdc++11
                  compiler.version: 13.0-rc
                  os: Macos
            a0d37d10fdb83a0414d7f4a1fb73da2c210211c6
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: apple-clang
                  compiler.cppstd: gnu98
                  compiler.libcxx: libc++
                  compiler.version: 12.0
                  os: webOS
                  os.sdk_version: 7.0.0
```

Try any other custom setting!

See also:

- *profiles*.
- *Conan packages binary compatibility: the package ID*

8.6 Graph examples

This section contains examples about different types of advanced graphs, using different types of `requires` and `tool_requires`, advanced usage of requirement traits, etc.

8.6.1 Use a CMake macro packaged in a dependency

When a package recipe wants to provide a CMake functionality via a macro, it can be done as follows. Let's say that we have a `pkg` recipe, that will "export" and "package" a `Macros.cmake` file that contains a `pkg_macro()` CMake macro:

Listing 66: pkg/conanfile.py

```
from conan import ConanFile
from conan.tools.files import copy

class Pkg(ConanFile):
    name = "pkg"
    version = "0.1"
    package_type = "static-library"
    # Exporting, as part of the sources
    exports_sources = "*.cmake"

    def package(self):
        # Make sure the Macros.cmake is packaged
        copy(self, "*.cmake", src=self.source_folder, dst=self.package_folder)

    def package_info(self):
        # We need to define that there are "build-directories", in this case
        # the current package root folder, containing build files and scripts
        self.cpp_info.builddirs = ["."]
```

Listing 67: pkg/Macros.cmake

```
function(pkg_macro)
    message(STATUS "PKG MACRO WORKING!!!")
endfunction()
```

When this package is created (`cd pkg && conan create .`), it can be consumed by other package recipes, for example this application:

Listing 68: app/conanfile.py

```
from conan import ConanFile
from conan.tools.cmake import CMake

class App(ConanFile):
    package_type = "application"
    generators = "CMakeToolchain", "CMakeDeps"
    settings = "os", "compiler", "arch", "build_type"
    requires = "pkg/0.1"
```

(continues on next page)

(continued from previous page)

```
    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
```

That has this `CMakeLists.txt`:

Listing 69: app/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(App LANGUAGES NONE)

include(Macros)  # include the file with the macro (note no .cmake extension)
pkg_macro()  # call the macro
```

So when we run a local build, we will see how the file is included and the macro called:

```
$ cd app
$ conan build .
PKG MACRO WORKING!!!
```

8.6.2 Use cmake modules inside a `tool_requires` transparently

When we want to reuse some `.cmake` scripts that are inside another Conan package there are several possible different scenarios, like if the `.cmake` scripts are inside a regular `requires` or a `tool_requires`.

Also, it is possible to want 2 different approaches:

- The consumer of the scripts can do a explicit `include(MyScript)` in their CMakeLists.txt. This approach is nicely explicit and simpler to setup, just define `self.cpp_info.builddirs` in the recipe, and consumers with `CMakeToolchain` will automatically be able to do the `include()` and use the functionality. See the *example here*
- The consumer wants to have the dependency cmake modules automatically loaded when the `find_package()` is executed. This current example implements this case.

Let's say that we have a package, intended to be used as a `tool_require`, with the following recipe:

Listing 70: myfunctions/conanfile.py

```
import os
from conan import ConanFile
from conan.tools.files import copy

class Conan(ConanFile):
    name = "myfunctions"
    version = "1.0"
    exports_sources = ["*.cmake"]

    def package(self):
        copy(self, "*.cmake", self.source_folder, self.package_folder)

    def package_info(self):
        self.cpp_info.set_property("cmake_build_modules", ["myfunction.cmake"])
```

And a `myfunction.cmake` file in:

Listing 71: myfunctions/myfunction.cmake

```
function(myfunction)
    message("Hello myfunction!!!!")
endfunction()
```

We can do a `cd myfunctions && conan create .` which will create the `myfunctions/1.0` package containing the cmake script.

Then, a consumer package will look like:

Listing 72: consumer/conanfile.py

```
from conan import ConanFile
from conan.tools.cmake import CMake, CMakeDeps, CMakeToolchain

class Conan(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    tool_requires = "myfunctions/1.0"

    def generate(self):
        tc = CMakeToolchain(self)
        tc.generate()

        deps = CMakeDeps(self)
        # By default 'myfunctions-config.cmake' is not created for tool_requires
        # we need to explicitly activate it
        deps.build_context_activated = ["myfunctions"]
        # and we need to tell to automatically load 'myfunctions' modules
        deps.build_context_build_modules = ["myfunctions"]
        deps.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
```

And a `CMakeLists.txt` like:

Listing 73: consumer/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.0)
project(test)
find_package(myfunctions CONFIG REQUIRED)
myfunction()
```

Then, the consumer will be able to automatically call the `myfunction()` from the dependency module:

```
$ conan build .
...
Hello myfunction!!!!
```

If for some reason the consumer wants to force the usage from the `tool_requires()` as a CMake module, the consumer could do `deps.set_property("myfunctions", "cmake_find_mode", "module", build_context=True)`, and then `find_package(myfunctions MODULE REQUIRED)` will work.

8.6.3 Depending on different versions of the same tool-require

Note: This is an **advanced** use case. It shouldn't be necessary in the vast majority of cases.

In the general case, trying to do something like this:

```
def build_requirements(self):
    self.tool_requires("gcc/1.0")
    self.tool_requires("gcc/2.0")
```

Will generate a "conflict", showing an error like `Duplicated requirement`. This is correct in most situations, when it is obvious that it is not possible to use 2 versions of the same compiler to build the current package.

However there are some exceptional situations when something like that is desired. Let's recreate the potential scenario. Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
git clone https://github.com/conan-io/examples2.git
cd examples2/examples/graph/tool_requires/different_versions
```

There we have a `gcc` fake recipe with:

```
class Pkg(ConanFile):
    name = "gcc"

    def package(self):
        echo = f"@echo off\necho MYGCC={self.version}!!"
        save(self, os.path.join(self.package_folder, "bin", f"mygcc{self.version}.bat"),␣
↪echo)
        save(self, os.path.join(self.package_folder, "bin", f"mygcc{self.version}.sh"),␣
↪echo)
        os.chmod(os.path.join(self.package_folder, "bin", f"mygcc{self.version}.sh"),␣
↪0o777)
```

This is not an actual compiler, it fakes it with a shell or bat script that prints `MYGCC=current-version` when executed. Note the binary itself is called `mygcc1.0` and `mygcc2.0`, that is, it contains the version in the executable name itself.

We can create 2 different versions for `gcc/1.0` and `gcc/2.0` with:

```
$ conan create gcc --version=1.0
$ conan create gcc --version=2.0
```

Now, in the `wine` folder there is a `conanfile.py` like this:

```
class Pkg(ConanFile):
    name = "wine"
    version = "1.0"

    def build_requirements(self):
        # If we specify "run=False" they no longer conflict
        self.tool_requires("gcc/1.0", run=False)
        self.tool_requires("gcc/2.0", run=False)

    def generate(self):
        # It is possible to individually reference each one
```

(continues on next page)

(continued from previous page)

```
        gcc1 = self.dependencies.build["gcc/1.0"]
        assert gcc1.ref.version == "1.0"
        gcc2 = self.dependencies.build["gcc/2.0"]
        assert gcc2.ref.version == "2.0"

    def build(self):
        ext = "bat" if platform.system() == "Windows" else "sh"
        self.run(f"mygcc1.0.{ext}")
        self.run(f"mygcc2.0.{ext}")
```

The first important point is the `build_requirements()` method, that does a `tool_requires()` to both versions, but defining `run=False`. **This is very important**: we are telling Conan that we actually don't need to run anything from those packages. As `tool_requires` are not visible, they don't define headers or libraries, there is nothing that makes Conan identify those 2 `tool_requires` as conflicting. So the dependency graph can be constructed without errors, and the `wine/1.0` package will contain 2 different tool-requires to both `gcc/1.0` and `gcc/2.0`.

Of course, it is not true that we won't run anything from those `tool_requires`, but now Conan is not aware of it, and it is completely the responsibility of the user to manage it.

> **Warning:** Using `run=False` makes the `tool_requires()` completely invisible, that means that profile `[tool_requires]` will not be able to override its version, but it would create an extra tool-require dependency with the version injected from the profile. You might want to exclude specific packages with something like `!wine/*: gcc/3.0`.

The recipe has still access in the `generate()` method to each different `tool_require` version, just by providing the full reference like `self.dependencies.build["gcc/1.0"]`.

Finally, the most important part is that the usage of those tools is completely the responsibility of the user. The `bin` folder of both `tool_requires` containing the executables will be in the path thanks to the `VirtualBuildEnv` generator that by default updates the PATH env-var. In this case the executables are different like `mygcc1.0.sh` and `mygcc2.0.sh`, so it is not an issue, and each one will be found inside its package.

But if the executable file was exactly the same like `gcc.exe`, then it would be necessary to obtain the full folder (typically in the `generate()` method) with something like `self.dependencies.build["gcc/1.0"].cpp_info.bindir` and use the full path to disambiguate.

Let's see it working. If we execute:

```
$ conan create wine
...
wine/1.0: RUN: mygcc1.0.bat
MYGCC=1.0!!

wine/1.0: RUN: mygcc2.0.bat
MYGCC=2.0!!
```

8.6.4 Depending on same version of a tool-require with different options

Note: This is an **advanced** use case. It shouldn't be necessary in the vast majority of cases.

In the general case, trying to do something like this:

```
def build_requirements(self):
    self.tool_requires("gcc/1.0")
    self.tool_requires("gcc/1.0")
```

Will generate a "conflict", showing an error like `Duplicated requirement`.

However there are some exceptional situations that we could need to depend on the same `tool_requires` version, but using different binaries of that `tool_requires`. This can be achieved by passing different `options` to those `tool_requires`. Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
git clone https://github.com/conan-io/examples2.git
cd examples2/examples/graph/tool_requires/different_options
```

There we have a `gcc` fake recipe with:

```
class Pkg(ConanFile):
    name = "gcc"
    version = "1.0"
    options = {"myoption": [1, 2]}

    def package(self):
        # This fake compiler will print something different based on the option
        echo = f"@echo off\necho MYGCC={self.options.myoption}!!"
        save(self, os.path.join(self.package_folder, "bin", f"mygcc{self.options.
↪myoption}.bat"), echo)
        save(self, os.path.join(self.package_folder, "bin", f"mygcc{self.options.
↪myoption}.sh"), echo)
        os.chmod(os.path.join(self.package_folder, "bin", f"mygcc{self.options.myoption}.
↪sh"), 0o777)
```

This is not an actual compiler, it fakes it with a shell or bat script that prints `MYGCC=current-option` when executed. Note the binary itself is called `mygcc1` and `mygcc2`, that is, it contains the option in the executable name itself.

We can create 2 different binaries for `gcc/1.0` with:

```
$ conan create gcc -o myoption=1
$ conan create gcc -o myoption=2
```

Now, in the `wine` folder there is a `conanfile.py` like this:

```
class Pkg(ConanFile):
    name = "wine"
    version = "1.0"

    def build_requirements(self):
        self.tool_requires("gcc/1.0", run=False, options={"myoption": 1})
        self.tool_requires("gcc/1.0", run=False, options={"myoption": 2})
```

(continues on next page)

(continued from previous page)

```
    def generate(self):
        gcc1 = self.dependencies.build.get("gcc", options={"myoption": 1})
        assert gcc1.options.myoption == "1"
        gcc2 = self.dependencies.build.get("gcc", options={"myoption": 2})
        assert gcc2.options.myoption == "2"

    def build(self):
        ext = "bat" if platform.system() == "Windows" else "sh"
        self.run(f"mygcc1.{ext}")
        self.run(f"mygcc2.{ext}")
```

The first important point is the `build_requirements()` method, that does a `tool_requires()` to both binaries, but defining `run=False` and `options={"myoption": value}` traits. **This is very important**: we are telling Conan that we actually don't need to run anything from those packages. As `tool_requires` are not visible, they don't define headers or libraries and they define different `options`, there is nothing that makes Conan identify those 2 `tool_requires` as conflicting. So the dependency graph can be constructed without errors, and the `wine/1.0` package will contain 2 different tool-requires to both `gcc/1.0` with `myoption=1` and with `myoption=2`.

Of course, it is not true that we won't run anything from those `tool_requires`, but now Conan is not aware of it, and it is completely the responsibility of the user to manage it.

> **Warning:** Using `run=False` makes the `tool_requires()` completely invisible, that means that profile `[tool_requires]` will not be able to override its version, but it would create an extra tool-require dependency with the version injected from the profile. You might want to exclude specific packages with something like `!wine/*: gcc/3.0`.

The recipe still has access in the `generate()` method to each different `tool_require` version, just by providing the options values for the dependency that we want `self.dependencies.build.get("gcc", options={"myoption": 1})`.

Finally, the most important part is that the usage of those tools is completely the responsibility of the user. The `bin` folder of both `tool_requires` containing the executables will be in the path thanks to the `VirtualBuildEnv` generator that by default updates the PATH env-var. In this case the executables are different like `mygcc1.sh` and `mygcc2.sh`, so it is not an issue, and each one will be found inside its package.

But if the executable file was exactly the same like `gcc.exe`, then it would be necessary to obtain the full folder (typically in the `generate()` method) with something like `self.dependencies.build.get("gcc", options={"myoption": 1}).cpp_info.bindir` and use the full path to disambiguate.

Let's see it working. If we execute:

```
$ conan create wine
...
wine/1.0: RUN: mygcc1.bat
MYGCC=1!!

wine/1.0: RUN: mygcc2.bat
MYGCC=2!!
```

8.6.5 Using the same requirement as a requires and as a tool_requires

There are libraries which could behave as a library and as a tool requirement, e.g., protobuf Those libraries normally contains headers/sources of the library itself, and, perhaps, some extra tools (compilers, shell scripts, etc.). Both parts are used in different contexts, let's think of this scenario using *protobuf* for instance:

- I want to create a library which includes a compiled protobuf message. The protobuf compiler (build context) needs to be invoked at build time, and the library with the compiled *.pb.cc* file needs to be linked against the protobuf library (host context).

Given that, we should be able to use protobuf in build/host context in the same Conan recipe. Basically, your package recipe should look like:

```
def requirements(self):
    self.requires("protobuf/3.18.1")

def build_requirements(self):
    self.tool_requires("protobuf/<host_version>")
```

Note: The `protobuf/<host_version>` expression ensures that the same version of the library is used in both contexts. You can read more about it *here*.

This is the way to proceed with any other library used in both contexts. Nonetheless, let's see a detailed example to see how the example looks like.

Please, first clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
git clone https://github.com/conan-io/examples2.git
cd examples2/examples/graph/tool_requires/using_protobuf/myaddresser
```

The structure of the project is the following:

```
./
├── conanfile.py
├── CMakeLists.txt
├── addressbook.proto
├── apple-arch-armv8
├── apple-arch-x86_64
└── src
    └── myaddresser.cpp
└── include
    └── myaddresser.h
└── test_package
    ├── conanfile.py
    ├── CMakeLists.txt
    └── src
        └── example.cpp
```

The `conanfile.py` looks like:

Listing 74: ./conanfile.py

```
from conan import ConanFile
from conan.tools.cmake import CMake, cmake_layout
```

(continues on next page)

(continued from previous page)

```
class myaddresserRecipe(ConanFile):
    name = "myaddresser"
    version = "1.0"
    package_type = "library"
    settings = "os", "compiler", "build_type", "arch"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}
    generators = "CMakeDeps", "CMakeToolchain"
    # Sources are located in the same place as this recipe, copy them to the recipe
    exports_sources = "CMakeLists.txt", "src/*", "include/*", "addressbook.proto"

    def config_options(self):
        if self.settings.os == "Windows":
            self.options.rm_safe("fPIC")

    def configure(self):
        if self.options.shared:
            self.options.rm_safe("fPIC")

    def requirements(self):
        self.requires("protobuf/3.18.1")

    def build_requirements(self):
        self.tool_requires("protobuf/<host_version>")

    def layout(self):
        cmake_layout(self)

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

    def package(self):
        cmake = CMake(self)
        cmake.install()

    def package_info(self):
        self.cpp_info.libs = ["myaddresser"]
        self.cpp_info.requires = ["protobuf::libprotobuf"]
```

As you can see, we're using *protobuf* at the same time but in different contexts.

The `CMakeLists.txt` shows how this example uses protobuf compiler and library:

Listing 75: ./CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(myaddresser LANGUAGES CXX)

find_package(protobuf CONFIG REQUIRED)
```

(continues on next page)

(continued from previous page)

```
protobuf_generate_cpp(PROTO_SRCS PROTO_HDRS addressbook.proto)

add_library(myaddresser src/myaddresser.cpp ${PROTO_SRCS})
target_include_directories(myaddresser PUBLIC include)

target_include_directories(myaddresser PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<BUILD_INTERFACE:${CMAKE_CURRENT_BINARY_DIR}>
  $<INSTALL_INTERFACE:include>
)

target_link_libraries(myaddresser PUBLIC protobuf::libprotobuf)

set_target_properties(myaddresser PROPERTIES PUBLIC_HEADER "include/myaddresser.h;$
↪{PROTO_HDRS}")
install(TARGETS myaddresser)
```

Where the library itself defines a simple *myaddresser.cpp* which uses the generated *addressbook.pb.h* header:

Listing 76: ./src/myaddresser.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include "addressbook.pb.h"
#include "myaddresser.h"

void myaddresser(){
  // Testing header generated by protobuf
  GOOGLE_PROTOBUF_VERIFY_VERSION;

  tutorial::AddressBook address_book;
  auto * person = address_book.add_people();
  person->set_id(1337);
  std::cout << "myaddresser(): created a person with id 1337\n";
  // Optional:  Delete all global objects allocated by libprotobuf.
  google::protobuf::ShutdownProtobufLibrary();
}
```

Finally, the *test_package* example simply calls the `myaddresser()` function to check that everything works correctly:

Listing 77: ./test_package/src/example.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include "myaddresser.h"


int main(int argc, char* argv[]) {
  myaddresser();
  return 0;
```

(continues on next page)

(continued from previous page)

```
}
```

So, let's see if it works fine:

```
$ conan create . --build missing
...

Requirements
    myaddresser/1.0
↪#71305099cc4dc0b08bb532d4f9196ac1:c4e35584cc696eb5dd8370a2a6d920fb2a156438 - Build
    protobuf/3.18.1
↪#ac69396cd9fbb796b5b1fc16473ca354:e60fa1e7fc3000cc7be2a50a507800815e3f45e0
↪#0af7d905b0df3225a3a56243841e041b - Cache
    zlib/1.2.13#13c96f538b52e1600c40b88994de240f:d0599452a426a161e02a297c6e0c5070f99b4909
↪#69b9ece1cce8bc302b69159b4d437acd - Cache
Build requirements
    protobuf/3.18.1
↪#ac69396cd9fbb796b5b1fc16473ca354:e60fa1e7fc3000cc7be2a50a507800815e3f45e0
↪#0af7d905b0df3225a3a56243841e041b - Cache
...

-- Install configuration: "Release"
-- Installing: /Users/myuser/.conan2/p/b/myser03f790a5a5533/p/lib/libmyaddresser.a
-- Installing: /Users/myuser/.conan2/p/b/myser03f790a5a5533/p/include/myaddresser.h
-- Installing: /Users/myuser/.conan2/p/b/myser03f790a5a5533/p/include/addressbook.pb.h

myaddresser/1.0: package(): Packaged 2 '.h' files: myaddresser.h, addressbook.pb.h
myaddresser/1.0: package(): Packaged 1 '.a' file: libmyaddresser.a
....

======== Testing the package: Executing test ========
myaddresser/1.0 (test package): Running test()
myaddresser/1.0 (test package): RUN: ./example
myaddresser(): created a person with id 1337
```

After seeing it's running OK, let's try to use cross-building. Notice that this part is based on MacOS Intel systems, and cross-compiling for MacOS ARM ones, but you could use your own profiles depending on your needs for sure.

Warning: MacOS system is required to run this part of the example.

```
$ conan create . --build missing -pr:b apple-arch-x86_64 -pr:h apple-arch-armv8
...

-- Install configuration: "Release"
-- Installing: /Users/myuser/.conan2/p/b/myser03f790a5a5533/p/lib/libmyaddresser.a
-- Installing: /Users/myuser/.conan2/p/b/myser03f790a5a5533/p/include/myaddresser.h
-- Installing: /Users/myuser/.conan2/p/b/myser03f790a5a5533/p/include/addressbook.pb.h

myaddresser/1.0: package(): Packaged 2 '.h' files: myaddresser.h, addressbook.pb.h
myaddresser/1.0: package(): Packaged 1 '.a' file: libmyaddresser.a
....
```

(continues on next page)

(continued from previous page)

```
======== Testing the package: Executing test ========
myaddresser/1.0 (test package): Running test()
```

Now, we cannot see the example running because of the host architecture. If we want to check that the *example* executable is built for the correct one:

```
$ file test_package/build/apple-clang-13.0-armv8-gnu17-release/example
test_package/build/apple-clang-13.0-armv8-gnu17-release/example: Mach-O 64-bit␣
↪executable arm64
```

Everything works as expected, and the executable was built for 64-bit executable arm64 architectures.

8.7 Developer tools and flows

8.7.1 Debugging and stepping into dependencies

Sometimes, when developing and debugging your own code, it could be useful to be able to step-into the dependencies source code too. There are a couple of things to take into account:

- Recipes and packages from ConanCenter do not package always all the debug artifacts necessary to debug. For example in Windows, the `*.pdb` files are not packaged, because they are very heavy, and in practice barely used. It is possible to have your own packages to package the PDB files if you want, but that still won't solve the next point.
- Debug artifacts are often not relocatable, that means that such artifacts can only be used in the location they were built from sources. But packages that are uploaded to a server and downloaded to a different machine can put those artifacts in a different folder. Then, the debug artifacts might not correctly locate the source code, the symbols, etc.

Building from source

The recommended approach for debugging dependencies is building them from source in the local cache. This approach should work out of the box for most recipes, including ConanCenter recipes.

We can reuse the code from the very first example in the tutorial for this use case. Please, first clone the sources to recreate this project. You can find them in the examples2 repository in GitHub:

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/simple_cmake_project
```

Then, lets make sure the dependency is built from source:

```
$ conan install . -s build_type=Debug --build="zlib/*"
...
Install finished successfully
```

Assuming that we have CMake>=3.23, we can use the presets (otherwise, please use the `-DCMAKE_TOOLCHAIN_FILE` arguments):

```
$ cmake . --preset conan-default
```

This will create our project, that we can start building and debugging.

Step into a dependency with Visual Studio

Once the project is created, in Visual Studio, we can double-click on the `compressor.sln` file, or open the file from the open Visual Studio IDE.

Once the project is open, the first step is building it, making sure the `Debug` configuration is the active one, going to `Build -> Build Solution` will do it. Then we can define `compressor` as the "Startup project" in project view.

Going to the `compressor/main.c` source file, we can introduce a breakpoint in some line there:

Listing 78: main.c

```
int main(void) {
    ...

    // add a breakpoint in deflateInit line in your IDE
    deflateInit(&defstream, Z_BEST_COMPRESSION);
    deflate(&defstream, Z_FINISH);
```

Clicking on the `Debug -> Start Debugging` (or F5), the program will start debugging and stop at the `deflateInit()` line. Clicking on the `Debug -> Step Into`, the IDE should be able to navigate to the `deflate.c` source code. If we check this file, its path will be inside the Conan cache, like `C:\Users\<myuser>\.conan2\p\b\zlib4f7275ba0a71f\b\src\deflate.c`

Listing 79: deflate.c

```
int ZEXPORT deflateInit_(strm, level, version, stream_size)
 z_streamp strm;
 int level;
 const char *version;
 int stream_size;
 {
     return deflateInit2_(strm, level, Z_DEFLATED, MAX_WBITS, DEF_MEM_LEVEL,
                          Z_DEFAULT_STRATEGY, version, stream_size);
     /* To do: ignore strm->next_in if we use it as window */
 }
```

See also:

- Modifying the dependency source code while debugging is not possible with this approach. If that is the intended flow, the recommended approach is to use *editable package*.

8.7.2 Debugging shared libraries with Visual Studio

In the previous example we discussed how to debug dependencies in Visual Studio, but when using Conan dependencies in a project it is possible that the original build folder and build files don't exist. Conan packages don't contain the necessary information for debugging libraries with Visual Studio by default, this information is stored in PDBs that are generated during the compilation of the libraries. When using Conan these PDBs are generated in the build folder, which is only needed during the building of the libraries. For that reason it's a common operation to clean the Conan cache with `conan cache clean` to remove the build folder and save disk space.

For these cases where the build folder is not present we created a hook that copies the PDBs generated in the build folder to the package folder. This behavior can't be forced by default because PDB files are usually larger than the whole package, and it would greatly increase the package sizes.

This section will follow some examples on how to debug a project in different cases to show how users can make use of the PDB hook.

Creating a project and debugging as usual

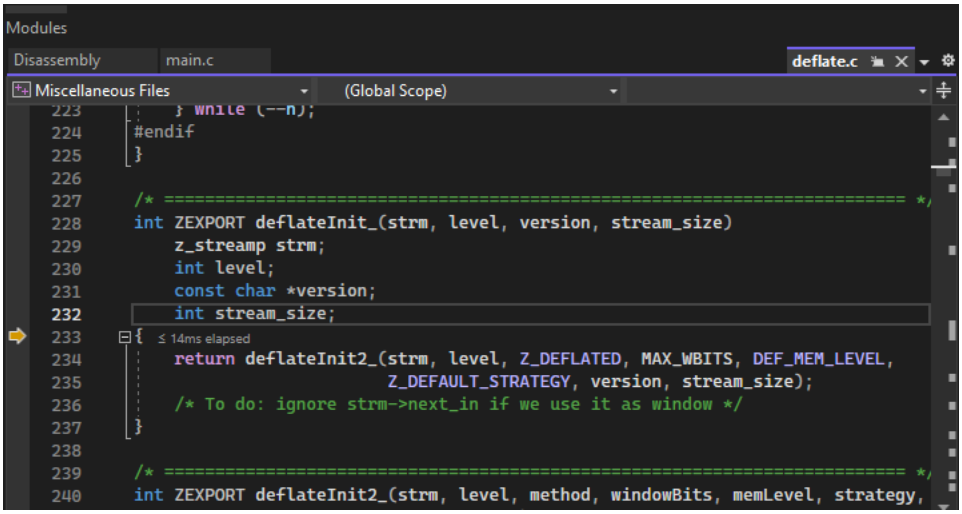
First we will debug our project as usual, as it is explained in more detail in the *previous example*. We can start building our dependencies from sources as in the previous section, only this time we will build them as shared. To begin with, clone the sources needed for the example from the examples2 repository in GitHub and create the project.

```
$ git clone https://github.com/conan-io/examples2.git
$ cd examples2/tutorial/consuming_packages/simple_cmake_project
$ conan install . -o="*:shared=True" -s build_type=Debug --build="zlib/*"
...
Install finished successfully

# CMake presets require CMake>=3.23
$ cmake --preset=conan-default
```

Note: We will only cover the case where the dependencies are built as shared because the PDBs and how they are linked to the libraries works differently for static libraries.

We can now open the solution `compressor.sln` to open our project in Visual Studio and debug it as explained in the previous example. Setting a breakpoint in line 22, running the debugger and using the step into will allow us to debug inside our dependency file `deflate.c`.



In this case the original build files were all present so the debugger worked as usual. Next we will see how the debugger works after removing the build files from the Conan cache.

Removing build files from the Conan cache

There are multiple reasons that can cause the build files to not be present after the dependencies are compiled. We will clean our build files from the cache to simulate one of those cases using `conan cache clean`. The `--build` flag makes sure that we only remove the build files, as we will need our source files for this example.

```
$ conan list "zlib/1.2.11:*"
$ conan cache path --folder build zlib/1.2.11:17b26a16efb893750e4481f98a154db2934ead88
$ conan cache clean zlib/1.2.11 --build
$ conan cache path --folder build zlib/1.2.11:17b26a16efb893750e4481f98a154db2934ead88
```

After closing and reopening our solution in Visual Studio, we can try to debug again. If you try to step into the dependency, with the breakpoint on line 22, you will notice it will directly skip over to the next line as Visual Studio doesn't have any information on the dependencies to debug.

Installing a hook to copy the PDBs to the package folder

To solve the issue of not having the PDBs in the package folder, we created a hook that copies the PDBs from the build folder to the package folder. The hook is available in the conan-extensions repository. Installing the whole repository will work, but we recommend to only install the hooks folder from the `conan-extensions` repository with:

```
$ conan config install https://github.com/conan-io/conan-extensions.git -sf=extensions/
↪hooks -tf=extensions/hooks
```

The hook is made so it won't run by default, as it can increase the package size significantly. As explained in the *hooks documentation*, we need to change the name of our hook to start with `hook_`. To locate the path where the hook was placed, run the command `conan config home` to find your local cache path and go to the `extensions/hooks` folder to rename the `_hook_copy_pdbs_to_package.py` file. Be aware that this hook will run everytime a `package()` method is run, to disable the hook just rename the hook back to start with `_hook_`.

The hook is implemented as a post-package hook, which means that it will execute after the package is created through the `package()` method of a recipe. This avoids any potential issue, as the order will be as follows:

- The `build()` method of the recipe is executed, generating the DLLs and PDBs
- The `package()` method of the recipe is executed, copying the necessary files to the package folder (in this case the DLLs but not the PDBs)
- The hook is executed copying the PDBs from the build folder next to its DLL for every DLL in the package

The hook makes use of the `dumpbin` tool which is included in the Visual Studio installation. This tool allows us to get information of a DLL, in this case the path where its associated PDB is located. It will be used for every DLL in the package to locate its PDB to copy it to the package folder.

For more information on how PDBs work with Visual Studio and how we used it to create the hook can be found in the hook readme.

Debugging without build files

After installing the hook we will create again the project from sources so the hook can now copy the PDBs to the package folder alongside the package DLLs so they can be found by the debugger.

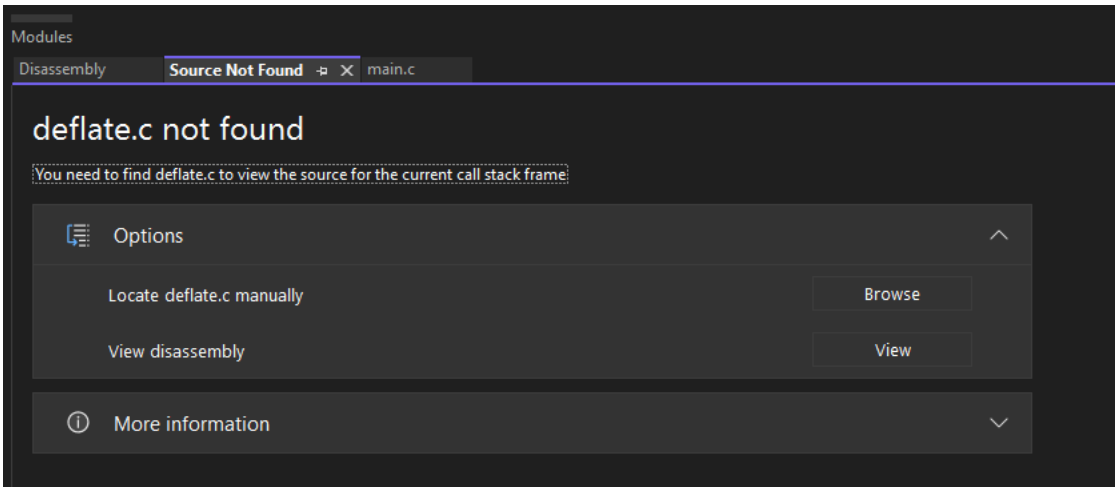
```
$ conan install . -o="*:shared=True" -s build_type=Debug --build="zlib/*"
...
zlib/1.2.11: Calling package()
...
[HOOK - hook_copy_pdbs_to_package.py] post_package(): PDBs post package hook running
...
Install finished successfully

# CMake presets require CMake>=3.23
$ cmake --preset=conan-default
```

Notice that when running the conan install now you will see the outputs of the hook running after the call to `package()`. To test the hook we can clean the cache again to remove the build files, this includes the sources used to build the library and the PDBs that were originally generated.

```
$ conan cache clean zlib/1.2.11 --build
```

Open the solution in Visual Studio again and start the debugger. When you try to step into the dependency in line 22, an error message will pop up telling us the file was not found and it will ask where the file is located. We can close this window and it will give the option to view the disassembly which can be debugged thanks to the PDB. The PDB only contains the debugging information but Visual Studio is missing the source files, so it won't be able to debug over those as it did initially.



Locating the sources path for the debugger

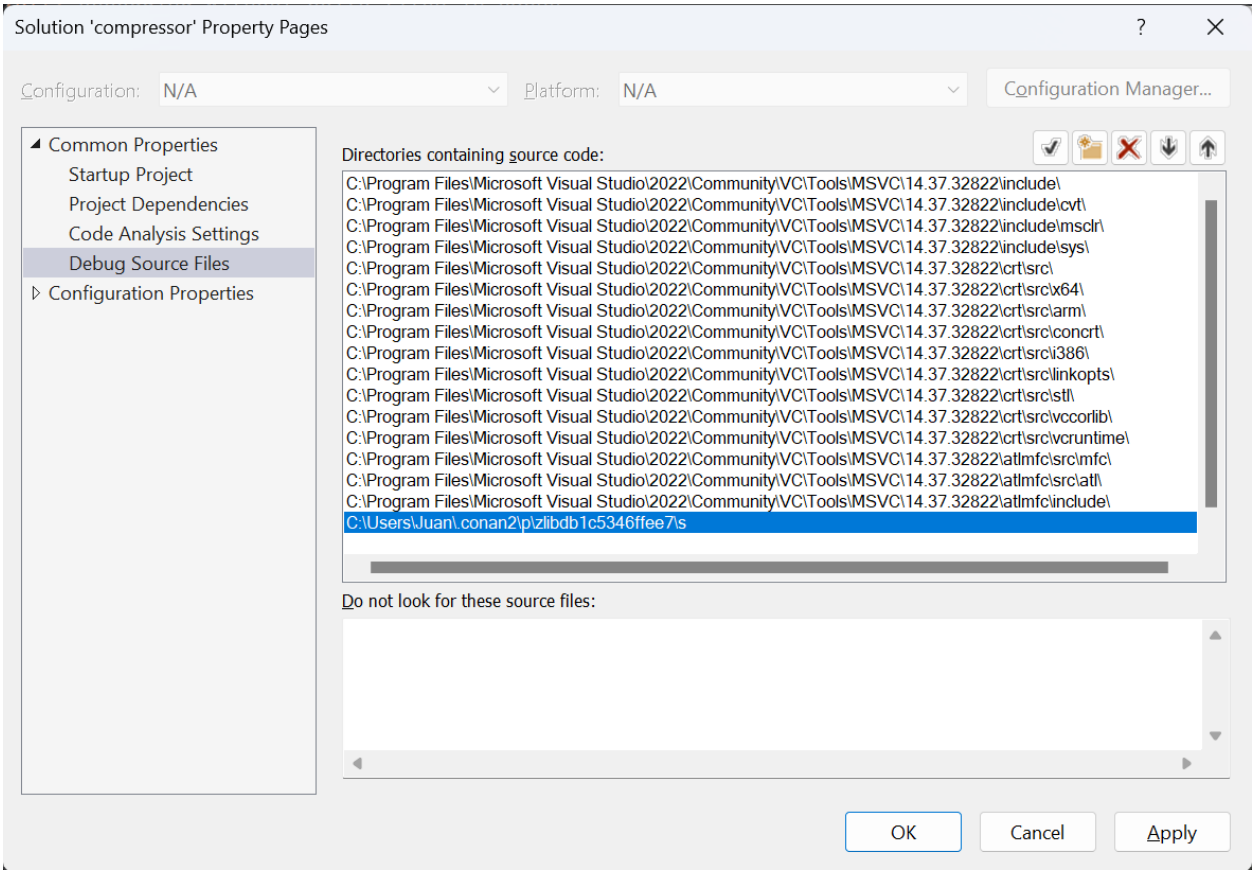
Visual Studio won't be able to find the source files by itself after deleting the original build files. To be able to debug over the source files, there's an option to manually set the source folder path so that it's possible to debug over the source files. This requires that the source files for the dependency exist. In our case we can get the location of this source files by running a `conan cache path`.

```
$ conan cache path --folder source zlib/1.2.11
```

In case this source path is not present we can use a config to download the sources again.

```
$ conan install . -o="*:shared=True" -s build_type=Debug -c:a="tools.build:download_
↪source=True"
```

Once we have the source path we can set it in Visual Studio so the debugger can find the source files. Right click on the solution in the Solution Explorer and select Properties. Go to Debug Source Files in the Common Properties section and add our source path.



Starting the debugger again will allow to step into the code of the dependency as in the first example we did.

Note: If there are patches to the source files we won't be able to debug over the modified files, as we are using the files from the source folder and the patches are applied in a later step right before being compiled in the build folder.

Any modification to the source files will not allow debugging over them, as Visual Studio does a checksum check, so they need to be the exact same files as when the libraries were compiled.

8.7.3 Using a MinGW as tool_requires to build with gcc in Windows

If we had MinGW installed in our environment, we could define a profile like:

```
[settings]
os=Windows
compiler=gcc
compiler.version=12
compiler.libcxx=libstdc++11
compiler.threads=posix
compiler.exception=sjlj
arch=x86_64
build_type=Release

[buildenv]
PATH+=(path)C:/path/to/mingw/bin
# other environment we might need like
CXX=C:/path/to/mingw/bin/g++
# etc

[conf]
# some configuration like 'tools.build:compiler_executables' might be needed for some␣
↪cases
```

But we can also use a Conan package that contains a copy of the MinGW compiler and use it as a `tool_requires` instead:

Listing 80: mingw-profile.txt

```
[settings]
os=Windows
compiler=gcc
compiler.version=12
compiler.libcxx=libstdc++11
compiler.threads=posix
compiler.exception=seh
arch=x86_64
build_type=Release


[tool_requires]
mingw-builds/12.2.0
```

With this profile we can for example create a package in Windows with:

```
# Using a basic template project
$ conan new cmake_lib -d name=mypkg -d version=0.1
$ conan create . -pr=mingw
...
-- The CXX compiler identification is GNU 12.2.0
...


======== Testing the package: Executing test ========
```

(continues on next page)

(continued from previous page)

```
mypkg/0.1 (test package): Running test()
mypkg/0.1 (test package): RUN: .\example
mypkg/0.1: Hello World Release!
mypkg/0.1: _M_X64 defined
mypkg/0.1: __x86_64__ defined
mypkg/0.1: _GLIBCXX_USE_CXX11_ABI 1
mypkg/0.1: MSVC runtime: MultiThreadedDLL
mypkg/0.1: __cplusplus201703
mypkg/0.1: __GNUC__12
mypkg/0.1: __GNUC_MINOR__2
mypkg/0.1: __MINGW32__1
mypkg/0.1: __MINGW64__1
mypkg/0.1 test_package
```

See also:

- The ConanCenter web page for the mingw-builds package
- The `conan-center-index` mingw-builds Github repo recipe

8.8 Conan commands examples

8.8.1 Using packages-lists

Packages lists are a powerful and convenient Conan feature that allows to automate and concatenate different Conan commands. Let's see some common use cases:

Listing packages and downloading them

A first simple use case could be listing some recipes and/or binaries in a server, and then downloading them.

We can do any `conan list`, for example, to list all `zlib` versions above `1.2.11`, the latest recipe revision, all Windows binaries for that latest recipe revision, and finally the latest package revision for every binary.

Note: If we want to actually download something later, it is necessary to specify a package revision in the `conan list` pattern, such as `latest`, otherwise only the recipes will be downloaded.

```
$ conan list "zlib/[>1.2.11]#latest:*#latest" -p os=Windows --format=json -r=conancenter␣
↪> pkglist.json
```

The output of the command is sent in `json` format to the file `pkglist.json` that looks like:

Listing 81: pkglist.json (simplified)

```
"conancenter": {
    "zlib/1.2.12": {
        "revisions": {
            "b1fd071d8a2234a488b3ff74a3526f81": {
                "timestamp": 1667396813.987,
                "packages": {
```

(continues on next page)

(continued from previous page)

```
                    "ae9eaf478e918e6470fe64a4d8d4d9552b0b3606": {
                        "revisions": {
                            "19808a47de859c2408ffcf8e5df1fdaf": {
                            }
                        },
                        "info": {
                            "settings": {
                                "arch": "x86_64",
                                "os": "Windows"
                            }
                        }
                    }
                }
            }
        },
    "zlib/1.2.13": {
    }
}
```

The first level in the `pkglist.json` is the "origin" remote or "Local Cache" if the list happens in the cache. In this case, as we listed the packages in `conancenter` remote, that will be the origin.

We can now do a download of these recipes and binaries with a single `conan download` invocation:

```
$ conan download --list=pkglist.json -r=conancenter
# Download the recipes and binaries in pkglist.json
# And displays a report of the downloaded things
```

Downloading from one remote and uploading to a different remote

Let's say that we create a new package list from the packages downloaded in the previous step:

```
$ conan download --list=pkglist.json -r=conancenter --format=json > downloaded.json
# Download the recipes and binaries in pkglist.json
# And stores the result in "downloaded.json"
```

The resulting `downloaded.json` will be almost the same as the `pkglist.json` file, but in this case, the "origin" of those packages is the `"Local Cache"` (as the downloaded packages will be in the cache):

Listing 82: downloaded.json (simplified)

```
"Local Cache": {
        "zlib/1.2.12": {
            "revisions": {
            }
        }
    }
```

That means that we can now upload this same set of recipes and binaries to a different remote:

```
$ conan upload --list=downloaded.json -r=myremote -c
# Upload those artifacts to the same remote
```

Note: Best practices

This would be a **slow** mechanism to run promotions between different server repositories. Servers like Artifactory provide ways to directly copy packages from one repository to another without using a client, that are orders of magnitude faster because of file deduplication, so that would be the recommended approach. The presented approach in this section might be used for air-gapped environments and other situations in which it is not possible to do a server-to-server copy.

Building and uploading packages

One of the most interesting flows is the one when some packages are being built in the local cache, with a `conan create` or `conan install --build=xxx` command. Typically, we would like to upload the locally built packages to the server, so they don't have to be re-built again by others. But we might want to upload only the built binaries, but not all others transitive dependencies, or other packages that we had previously in our local cache.

It is possible to compute a package list from the output of a `conan install`, `conan create` and `conan graph info` commands. Then, that package list can be used for the upload. Step by step:

First let's say that we have our own package `mypkg/0.1` and we create it:

```
$ conan new cmake_lib -d name=mypkg -d version=0.1
$ conan create . --format=json > create.json
```

This will create a json representation of the graph, with information of what packages have been built `"binary": "Build"`:

Listing 83: create.json (simplified)

```
{
"graph": {
    "nodes": {
        "0": {
            "ref": "conanfile",
            "id": "0",
            "recipe": "Cli",
            "context": "host",
            "test": false
        },
        "1": {
            "ref": "mypkg/0.1#f57cc9a1824f47af2f52df0dbdd440f6",
            "id": "1",
            "recipe": "Cache",
            "package_id": "2401fa1d188d289bb25c37cfa3317e13e377a351",
            "prev": "75f44d989175c05bc4be2399edc63091",
            "build_id": null,
            "binary": "Build"
        }
    }
}
```

We can compute a package list from this file, and then upload those artifacts to the server with:

```
$ conan list --graph=create.json --graph-binaries=build --format=json > pkglist.json
# Create a pkglist.json with the known list of recipes and binaries built from sources
$ conan upload --list=pkglist.json -r=myremote -c
```

Removing packages lists

It is also possible to first `conan list` and create a list of things to remove, and then remove them:

```
# Removes everything from the cache
$ conan list "*#*" --format=json > pkglist.json
$ conan remove --list=pkglist.json  -c
```

Note that in this case, the default patterns are different in `list` and `remove`, because of the destructive nature of `conan remove`:

- When a recipe is passed to `remove` like `conan remove zlib/1.2.13`, it will remove the recipe of `zlib/1.2.13` and all of its binaries, because the binaries cannot live without the recipe.
- When a `package_id` is passed, like `conan remove zlib/1.2.13:package_id`, then that specific `package_id` will be removed, but the recipe will not

Then the pattern to remove everything will be different if we call directly `conan remove` or if we call first `conan list`, for example:

```
# Removes everything from the cache
$ conan remove "*"
# OR via list, we need to explicitly include all revisions
$ conan list "*#*" --format=json > pkglist.json
$ conan remove --list=pkglist.json  -c

# Removes only the binaries from the cache (leave recipes)
$ conan remove "*:*"
# OR via list, we need to explicitly include all revisions
$ conan list "*#*:*" --format=json > pkglist.json
$ conan remove --list=pkglist.json  -c
```

For more information see the *Reference commands section*

8.9 Conan runners examples

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

8.9.1 Creating a Conan package using a Docker runner

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

First of all you need to have the Docker daemon installed and running, plus Conan and the `docker` Python package. This tutorial assumes that you are running Conan inside a Python virtual environment, skip the first line if you already have the `docker` Python package installed in your virtual environment.

```
# install docker in your virtual environment if you don't have it already installed
$ pip install conan docker
$ docker ps
$ CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Now we are going to create create simple `cmake_lib` Conan template to later run inside Docker using the runner feature. Let's create the Conan package and a Dockerfile inside our project folder.

```
$ cd </my/runner/folder>
$ mkdir mylib
$ cd mylib
$ conan new cmake_lib -d name=mylib -d version=0.1
$ tree
.
├── CMakeLists.txt
├── conanfile.py
├── include
│   └── mylib.h
├── src
│   └── mylib.cpp
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

Dockerfile

```
FROM ubuntu:22.04
RUN apt-get update \
    && DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
        build-essential \
        cmake \
        python3 \
        python3-pip \
        python3-venv \
    && rm -rf /var/lib/apt/lists/*
RUN pip install conan
```

```
$ cd </my/runner/folder>/mylib
$ tree
.
...
├── Dockerfile
...
```

Now, we need to define two new profiles inside the conan `profiles` folder. Replace `</my/runner/folder>` with your real project folder path.

`docker_example_host` profile

```
[settings]
build_type=Release
arch=x86_64
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=11
os=Linux

[runner]
type=docker
dockerfile=</my/runner/folder>/mylib
cache=copy
remove=true
platform=linux/amd64
```

Note: Users are free to configure architecture and platform on the host profile. Conan docker integration will build and run the image using the specified platform.

For example, if you are using a Mac Silicon, you can set the platform to `linux/arm64/v8` to build the image using the armv8 architecture.

```
[settings]
arch=armv8
# ...

[runner]
platform=linux/arm64/v8
```

`docker_example_build` profile

```
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=11
os=Linux
```

We are going to start from a totally clean environment, without any containers, images or conan package.

```
$ conan list "*:*"
Found 0 pkg/version recipes matching * in local cache
```

```
$ docker ps --all
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

```
$ docker images
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE
```

Now, it's time to create our library `mylib` using our new runner definition.

```
$ conan create . -pr:h docker_example_host -pr:b docker_example_build
```

If we split and analyze the command output, we can see what is happening and where the commands are being executed.

1. Standard conan execution.

```
======== Exporting recipe to the cache ========
mylib/0.1: Exporting package recipe: </my/runner/folder>/mylib/conanfile.py
mylib/0.1: Copied 1 '.py' file: conanfile.py
mylib/0.1: Copied 1 '.txt' file: CMakeLists.txt
mylib/0.1: Copied 1 '.h' file: mylib.h
mylib/0.1: Copied 1 '.cpp' file: mylib.cpp
mylib/0.1: Exported to cache folder: /Users/<user>/.conan2/p/mylib4abd06a04bdaa/e
mylib/0.1: Exported: mylib/0.1#8760bf5a311f01cc26f3b95428203210 (2024-07-08 12:22:01 UTC)

======== Input profiles ========
Profile host:
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=11
os=Linux

Profile build:
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=11
os=Linux
```

2. Build docker image

```
Building the Docker image: conan-runner-default
Dockerfile path: '</my/runner/folder>/mylib/Dockerfile'
Docker build context: '</my/runner/folder>/mylib'

Step 1/3 : FROM ubuntu:22.04

---> 97271d29cb79
Step 2/3 : RUN apt-get update     && DEBIAN_FRONTEND=noninteractive apt-get install -y --
↪no-install-recommends         build-essential         cmake         python3        ␣
↪python3-pip         python3-venv         g++-x86-64-linux-gnu     && rm -rf /var/lib/
↪apt/lists/*

...

---> 2bcf70201cce
Successfully built 2bcf70201cce
```

(continues on next page)

(continued from previous page)

```
Successfully tagged conan-runner-default:latest
```

3. Save the local cache running `conan cache save`.

```
Save host cache in: /Users/<user>/sources/test/mylib/.conanrunner/local_cache_save.tgz
Found 1 pkg/version recipes matching * in local cache
Saving mylib/0.1: mylib4abd06a04bdaa
```

4. Create and initialize the docker container.

```
Creating the docker container
Container conan-runner-docker running
```

5. Check if the container has a conan version with the runner feature.

```
conan-runner-docker | $ conan --version
conan-runner-docker | Conan version 2.12.1
```

6. Initialize the container conan cache using the host copy running `conan cache restore`.

```
conan-runner-docker | $ conan cache restore "/root/conanrunner/mylib/.conanrunner/local_
↪cache_save.tgz"
conan-runner-docker | Restore: mylib/0.1 in mylib4abd06a04bdaa
conan-runner-docker | Local Cache
conan-runner-docker |   mylib
conan-runner-docker |     mylib/0.1
conan-runner-docker |       revisions
conan-runner-docker |         8760bf5a311f01cc26f3b95428203210 (2025-01-31 12:34:25 UTC)
conan-runner-docker |           packages
conan-runner-docker |           recipe_folder: mylib4abd06a04bdaa
```

7. Run the conan create inside the container and build "mylib".

```
   conan-runner-docker | $ conan create /root/conanrunner/mylib -pr:h docker_param_
↪example_host -pr:b docker_param_example_build
-f json > create.json
   conan-runner-docker |
   conan-runner-docker | ======== Exporting recipe to the cache ========
   conan-runner-docker | mylib/0.1: Exporting package recipe: /root/conanrunner/mylib/
↪conanfile.py
   conan-runner-docker | mylib/0.1: Copied 1 '.py' file: conanfile.py
   conan-runner-docker | mylib/0.1: Copied 1 '.txt' file: CMakeLists.txt
   conan-runner-docker | mylib/0.1: Copied 1 '.h' file: mylib.h
   conan-runner-docker | mylib/0.1: Copied 1 '.cpp' file: mylib.cpp
   conan-runner-docker | mylib/0.1: Exported to cache folder: /root/.conan2/p/
↪mylib4abd06a04bdaa/e
   conan-runner-docker | mylib/0.1: Exported: mylib/0.1#8760bf5a311f01cc26f3b95428203210␣
↪(2025-01-31 12:34:26 UTC)
   conan-runner-docker |
   conan-runner-docker | ======== Input profiles ========
   conan-runner-docker | Profile host:
   conan-runner-docker | [settings]
   conan-runner-docker | arch=x86_64
```

(continues on next page)

(continued from previous page)

```
  conan-runner-docker | build_type=Release
  conan-runner-docker | compiler=gcc
  conan-runner-docker | compiler.cppstd=gnu17
  conan-runner-docker | compiler.libcxx=libstdc++11
  conan-runner-docker | compiler.version=11
  conan-runner-docker | os=Linux
  conan-runner-docker |
  conan-runner-docker | Profile build:
  conan-runner-docker | [settings]
  conan-runner-docker | arch=x86_64
  conan-runner-docker | build_type=Release
  conan-runner-docker | compiler=gcc
  conan-runner-docker | compiler.cppstd=gnu17
  conan-runner-docker | compiler.libcxx=libstdc++11
  conan-runner-docker | compiler.version=11
  conan-runner-docker | os=Linux
  conan-runner-docker |
  conan-runner-docker |
  conan-runner-docker | ======== Computing dependency graph ========
  conan-runner-docker | Graph root
  conan-runner-docker |     cli
  conan-runner-docker | Requirements
  conan-runner-docker |     mylib/0.1#8760bf5a311f01cc26f3b95428203210 - Cache
  conan-runner-docker |
  conan-runner-docker | ======== Computing necessary packages ========
  conan-runner-docker | mylib/0.1: Forced build from source
  conan-runner-docker | Requirements
  conan-runner-docker |     mylib/0.1
↪#8760bf5a311f01cc26f3b95428203210:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe - Build
  conan-runner-docker |

  ...

  conan-runner-docker | [ 50%] Building CXX object CMakeFiles/example.dir/src/example.
↪cpp.o
  conan-runner-docker | [100%] Linking CXX executable example
  conan-runner-docker | [100%] Built target example
  conan-runner-docker |
  conan-runner-docker |
  conan-runner-docker | ======== Testing the package: Executing test ========
  conan-runner-docker | mylib/0.1 (test package): Running test()
  conan-runner-docker | mylib/0.1 (test package): RUN: ./example
  conan-runner-docker | mylib/0.1: Hello World Release!
  conan-runner-docker | mylib/0.1: __x86_64__ defined
  conan-runner-docker | mylib/0.1: _GLIBCXX_USE_CXX11_ABI 1
  conan-runner-docker | mylib/0.1: __cplusplus201703
  conan-runner-docker | mylib/0.1: __GNUC__11
  conan-runner-docker | mylib/0.1: __GNUC_MINOR__4
  conan-runner-docker | mylib/0.1 test_package
```

8. Copy just the package created inside the container using the `pkglist.json` info from the previous `conan create`, restore this new package inside the host cache running a `conan cache save` and remove the container.

```
conan-runner-docker | $ conan cache save --list=pkglist.json --file "/root/conanrunner/
↪mylib"/.conanrunner/docker_cache_save.tgz
conan-runner-docker | Saving mylib/0.1: mylib4abd06a04bdaa
conan-runner-docker | Saving mylib/0.1:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe: b/
↪mylib11242e0a7e627/p
conan-runner-docker | Saving mylib/0.1:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe␣
↪metadata: b/mylib11242e0a7e627/d/metadata
conan-runner-docker | Local Cache
conan-runner-docker |   mylib
conan-runner-docker |     mylib/0.1
conan-runner-docker |       revisions
conan-runner-docker |         8760bf5a311f01cc26f3b95428203210 (2025-01-31 12:34:26 UTC)
conan-runner-docker |           packages
conan-runner-docker |             8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe
conan-runner-docker |               revisions
conan-runner-docker |                 ded6547554ff2306db5250451340fa43
conan-runner-docker |                   package_folder: b/mylib11242e0a7e627/p
conan-runner-docker |                   metadata_folder: b/mylib11242e0a7e627/d/metadata
conan-runner-docker |               info
conan-runner-docker |                 settings
conan-runner-docker |                   os: Linux
conan-runner-docker |                   arch: x86_64
conan-runner-docker |                   compiler: gcc
conan-runner-docker |                   compiler.cppstd: gnu17
conan-runner-docker |                   compiler.libcxx: libstdc++11
conan-runner-docker |                   compiler.version: 11
conan-runner-docker |                   build_type: Release
conan-runner-docker |                 options
conan-runner-docker |                   fPIC: True
conan-runner-docker |                   shared: False
conan-runner-docker |           recipe_folder: mylib4abd06a04bdaa
conan-runner-docker |
Restore host cache from: /Users/<user>/sources/test/mylib/.conanrunner/docker_cache_save.
↪tgz
Restore: mylib/0.1 in mylib4abd06a04bdaa
Restore: mylib/0.1:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe in b/mylib11242e0a7e627/p
Restore: mylib/0.1:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe metadata in b/
↪mylib11242e0a7e627/d/metadata
Stopping container
Removing container
```

If we now check the status of our conan and docker cache, we will see the new mylib package compile for Linux and the new docker image but we don't have any container because we define `remove=true`

```
$ conan list "*:*"
Found 1 pkg/version recipes matching * in local cache
Local Cache
mylib
    mylib/0.1
    revisions
        8760bf5a311f01cc26f3b95428203210 (2024-07-08 12:33:28 UTC)
        packages
            8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe
```

(continues on next page)

(continued from previous page)

```
            info
                settings
                arch: x86_64
                build_type: Release
                compiler: gcc
                compiler.cppstd: gnu17
                compiler.libcxx: libstdc++11
                compiler.version: 11
                os: Linux
                options
                fPIC: True
                shared: False
```

```
$ docker ps --all
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

```
$ docker images
REPOSITORY        TAG       IMAGE ID       CREATED          SIZE
my-conan-runner   latest    2bcf70201cce   11 minutes ago   531MB
```

What we have just done is to compile a library from scratch inside a Docker container without running any Docker command and retrieve the generated packages in a totally transparent and easily debuggable way thanks to our terminal output.

In this way, we can work as we have always done regardless of whether it is on our machine or in a container, without several open terminals and having the result of each operation in the same cache, being able to reuse the compiled packages from a previous compilation in another container automatically and transparently.

8.9.2 Using a docker runner configfile to parameterize a Dockerfile

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

If you need more control over the build and execution of the container, you can define more parameters inside a configfile yaml.

For example, you can add arguments in the build step or environment variables when you launch the container.

To use it, you just need to add it in the host profile.

```
[settings]
...
[runner]
type=docker
configfile=</my/runner/folder>/configfile
cache=copy
remove=false
```

How to use

Let's create a Dockerfile inside your project folder, a cmake_lib `myparamlib` like the *"Creating a Conan package using a Docker runner"* example and two profiles.

```
$ cd </my/runner/folder>
$ mkdir myparamlib
$ cd myparamlib
$ conan new cmake_lib -d name=myparamlib -d version=0.1
$ cd </my/runner/folder>
$ tree
.
├── CMakeLists.txt
├── conanfile.py
├── include
│   └── myparamlib.h
├── src
│   └── myparamlib.cpp
└── test_package
    ├── CMakeLists.txt
    ├── conanfile.py
    └── src
        └── example.cpp
```

```
ARG BASE_IMAGE
FROM $BASE_IMAGE
RUN apt-get update \
    && DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
        build-essential \
        cmake \
        python3 \
        python3-pip \
        python3-venv \
    && rm -rf /var/lib/apt/lists/*
RUN pip install conan
```

configfile

```
image: my-conan-runner-image
build:
    dockerfile: </my/runner/folder>
    build_context: </my/runner/folder>
    build_args:
        BASE_IMAGE: ubuntu:22.04
run:
    name: my-conan-runner-container
```

```
$ cd </my/runner/folder>/myparamlib
$ tree
.
...
├── Dockerfile
...
├── configfile
...
```

docker_param_example_host profile

```
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=11
os=Linux

[runner]
type=docker
configfile=</my/runner/folder>/myparamlib/configfile
cache=copy
remove=false
```

docker_param_example_build profile

```
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=11
os=Linux
```

Now it's time to create our new library.

```
$ conan create . -pr:h docker_param_example_host -pr:b docker_param_example_build

...

Building the Docker image: conan-runner-default
Dockerfile path: '</my/runner/folder>/myparamlib/Dockerfile'
Docker build context: '</my/runner/folder>/myparamlib'

Step 1/5 : ARG BASE_IMAGE

Step 2/5 : FROM $BASE_IMAGE

...

Successfully built caa8071cdff7
Successfully tagged my-conan-runner-image:latest

...

conan-runner-docker | $ conan create /root/conanrunner/myparamlib -pr:h docker_param_
↪example_host -pr:b docker_param_example_build -f json > create.json

...

conan-runner-docker | [ 50%] Building CXX object CMakeFiles/example.dir/src/example.cpp.o
```

(continues on next page)

(continued from previous page)

```
conan-runner-docker | [100%] Linking CXX executable example
conan-runner-docker | [100%] Built target example
conan-runner-docker |
conan-runner-docker | ======== Testing the package: Executing test ========
conan-runner-docker | myparamlib/0.1 (test package): Running test()
conan-runner-docker | myparamlib/0.1 (test package): RUN: ./example
conan-runner-docker | myparamlib/0.1: Hello World Release!
conan-runner-docker | myparamlib/0.1: __x86_64__ defined
conan-runner-docker | myparamlib/0.1: _GLIBCXX_USE_CXX11_ABI 1
conan-runner-docker | myparamlib/0.1: __cplusplus201703
conan-runner-docker | myparamlib/0.1: __GNUC__11
conan-runner-docker | myparamlib/0.1: __GNUC_MINOR__4
conan-runner-docker | myparamlib/0.1 test_package
conan-runner-docker |
conan-runner-docker | $ </my/runner/folder>/myparamlib/.conanrunner/docker_cache_save.tgz
conan-runner-docker |
conan-runner-docker | Saving myparamlib/0.1: mypar36e44205a36b9
conan-runner-docker | Saving myparamlib/0.1:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe: b/
↪mypare0dc449d4125d/p
conan-runner-docker | Saving myparamlib/0.1:8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe␣
↪metadata: b/mypare0dc449d4125d/d/metadata
```

If we now check the status of our conan cache, we will see the new `myparamlib` pacakge.

```
$ conan list "*:*"
Found 1 pkg/version recipes matching * in local cache
Local Cache
myparamlib
    myparamlib/0.1
    revisions
        11cb359a0526fe9ce3cfefb59c5d1953 (2024-07-08 12:47:21 UTC)
        packages
            8631cf963dbbb4d7a378a64a6fd1dc57558bc2fe
            info
                settings
                arch: x86_64
                build_type: Release
                compiler: gcc
                compiler.cppstd: gnu17
                compiler.libcxx: libstdc++11
                compiler.version: 11
                os: Linux
                options
                fPIC: True
                shared: False
```

8.10 Conan security examples

8.10.1 Using Compiler Sanitizers with Conan

To better illustrate the *sanitizers integration with Conan*, this section provides practical examples using AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) with simple C++ programs.

As a first step, please clone the sources to recreate this project. You can find them in the examples2 repository on GitHub:

```
git clone https://github.com/conan-io/examples2.git
cd examples2/examples/security/sanitizers/compiler_sanitizers
```

In this example we will see how to prepare Conan to use sanitizers in different ways.

To show how to use sanitizers in your builds, let's consider two examples.

AddressSanitizer: index out of bounds

In this example, we will build a simple C++ program that intentionally accesses an out-of-bounds index in an array, which should trigger ASan when running the program. We will be using a Conan profile to enable ASan:

Listing 84: profiles/gcc_asan

```
[settings]
arch=x86_64
os=Linux
build_type=Debug
compiler=gcc
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=15
compiler.sanitizer=Address

[conf]
tools.build:cflags=['-fsanitize=address']
tools.build:cxxflags=['-fsanitize=address']
tools.build:exelinkflags=['-fsanitize=address']
tools.build:sharedlinkflags+=["-fsanitize=address"]

[runenv]
ASAN_OPTIONS=halt_on_error=1:detect_leaks=1
```

Note that in this profile we set the `compiler.sanitizer=Address` does not define what compiler flags to use, but it is a settings to make explicit that both ASan and UBSan are intended to be used.

And for further illustration, we also use environment variable `ASAN_OPTIONS=halt_on_error=1:detect_leaks=1` for runtime configuration, to manage ASan to halt execution on the first error and to detect memory leaks when the program exits.

Listing 85: index_out_of_bounds/main.cpp

```
#include <iostream>
#include <cstdlib>
```

(continues on next page)

(continued from previous page)

```
int main() {
#ifdef __SANITIZE_ADDRESS__
  std::cout << "Address sanitizer enabled\n";
#else
  std::cout << "Address sanitizer not enabled\n";
#endif

  int foo[100];
  foo[100] = 42; // Out-of-bounds write

  return EXIT_SUCCESS;
}
```

Note: The preprocessor check above is portable for GCC, Clang and MSVC. The define `__SANITIZE_ADDRESS__` is present when **ASan** is active;

To build and run this example using Conan:

```
cd index_out_of_bounds/
conan build . -pr ../profiles/gcc_asan
build/Debug/index_out_of_bounds
```

Expected output (abbreviated):

```
Address sanitizer enabled
==32018==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fffbe04a6d0 ...
WRITE of size 4 at 0x7fffbe04a6d0 thread T0
#0 ... in main .../index_out_of_bounds+0x12ea
...
SUMMARY: AddressSanitizer: stack-buffer-overflow ... in main
This frame has 1 object(s):
[48, 448) 'foo' (line 11) <== Memory access at offset 448 overflows this variable
```

UndefinedBehaviorSanitizer: signed integer overflow

This example demonstrates how to use UBSan to detect signed integer overflow. It combines ASan and UBSan. Create a dedicated profile:

Listing 86: profiles/gcc_asan_ubsan

```
[settings]
arch=x86_64
os=Linux
build_type=Debug
compiler=gcc
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=15
compiler.sanitizer=AddressUndefinedBehavior

[conf]
```

(continues on next page)

(continued from previous page)

```
tools.build:cflags+=["-fsanitize=address,undefined", "-fno-omit-frame-pointer"]
tools.build:cxxflags+=["-fsanitize=address,undefined", "-fno-omit-frame-pointer"]
tools.build:exelinkflags+=["-fsanitize=address,undefined"]
tools.build:sharedlinkflags+=["-fsanitize=address,undefined"]
```

It is supported by GCC and Clang. MSVC does not support UBSan.

Source code:

Listing 87: signed_integer_overflow/main.cpp

```
#include <iostream>
#include <cstdlib>
#include <climits>

int main() {
#ifdef __SANITIZE_ADDRESS__
  std::cout << "Address sanitizer enabled\n";
#else
  std::cout << "Address sanitizer not enabled\n";
#endif

  int x = INT_MAX;
  x += 42;                    // signed integer overflow

  return EXIT_SUCCESS;
}
```

Build and run:

```
cd signed_integer_overflow/
conan build . -pr ../profiles/gcc_asan_ubsan
build/Debug/signed_integer_overflow
```

Expected output (abbreviated):

```
Address sanitizer enabled
.../main.cpp:16:9: runtime error: signed integer overflow: 2147483647 + 1 cannot be␣
↪represented in type 'int'
```

When executing the example application, UBSan detects the signed integer overflow and reports it as expected.

CHAPTER

NINE

REFERENCE

9.1 Binary model

This section introduces first how the `package_id`, the package binaries identifier is computed, hashing the configuration (settings + options + dependencies versions). While the effect of `settings` and `options` is more straightforward, understanding the effects of the dependencies requires more explanations, so that will be done in its own section.

Conan binary model is extensible, and users can define their custom settings, options and configuration to model their own binaries characteristics.

Finally, the default binary compatibility model will be described, and how it can be customized to adapt to different needs.

9.1.1 How the `package_id` is computed

Let's take some package and list its binaries, for example:

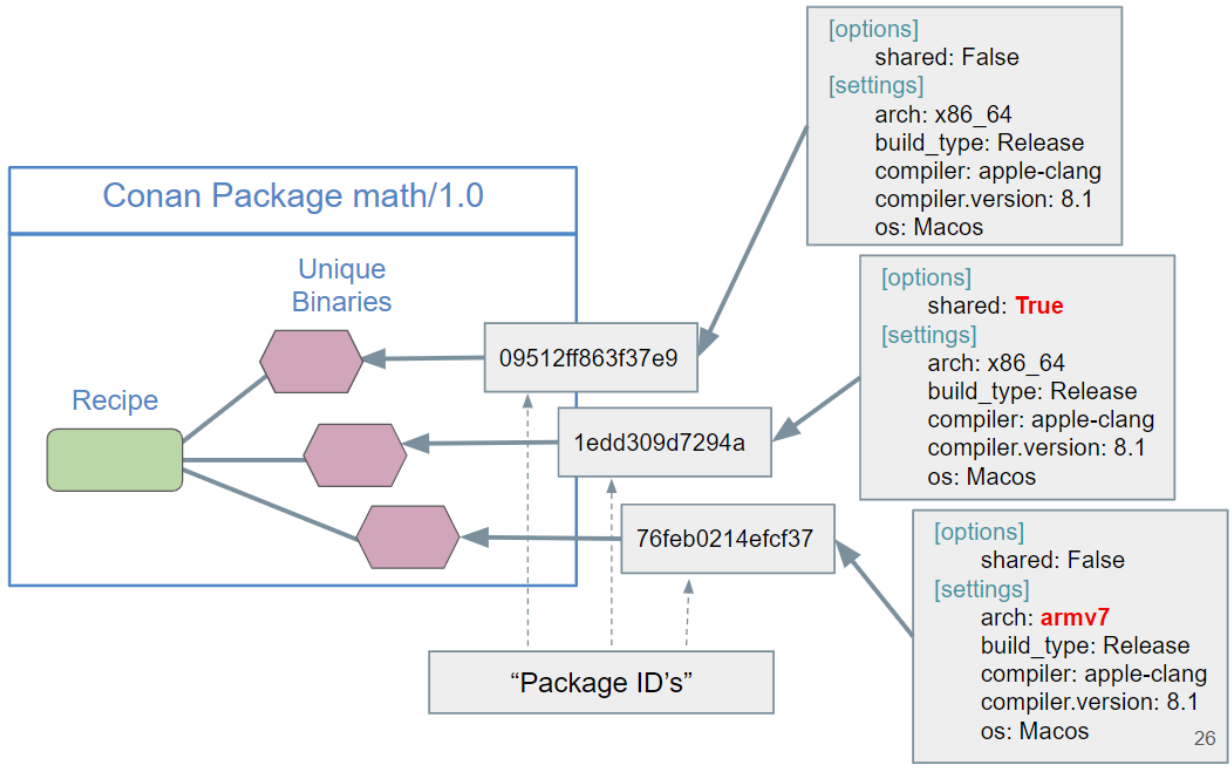
```
$ conan list "zlib/1.2.13:*" -r=conancenter
Local Cache
  zlib
    zlib/1.2.13
      revisions
        97d5730b529b4224045fe7090592d4c1 (2023-08-22 02:51:57 UTC)
          packages
            d62dff20d86436b9c58ddc0162499d197be9de1e  # package_id
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: apple-clang
                  compiler.version: 13
                  os: Macos
                options
                  fPIC: True
                  shared: False
            abe5e2b04ea92ce2ee91bc9834317dbe66628206  # package_id
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: gcc
```

(continues on next page)

(continued from previous page)

```
    compiler.version: 11
    os: Linux
  options
    shared: True
```

We can see several binaries for the latest recipe revision of `zlib/1.2.13`. Every binary is identified by its own `package_id`, and below it we can see some information for that binary under `info`. This information is the one used to compute the `package_id`. Every time something changes in this information, like the architecture, or being a static or a shared library, a new `package_id` is computed because it represents a different binary.



The `package_id` is computed as the sha1 hash of the `conaninfo.txt` file, containing the `info` displayed above. It is relatively easy to display such file:

```
$ conan install --requires=zlib/1.2.13 --build=missing
# Use the <package-id> listed in the install
$ conan cache path zlib/1.2.13:<package-id>
# cat the conaninfo.txt in the returned path
$ cat <path>/conaninfo.txt
[settings]
arch=x86_64
build_type=Release
compiler=msvc
compiler.runtime=dynamic
compiler.runtime_type=Release
compiler.version=193
os=Windows
[options]
shared=False
```

(continues on next page)

(continued from previous page)

```
$ sha1sum <path>/conaninfo.txt
# Should be the "package_id"!
```

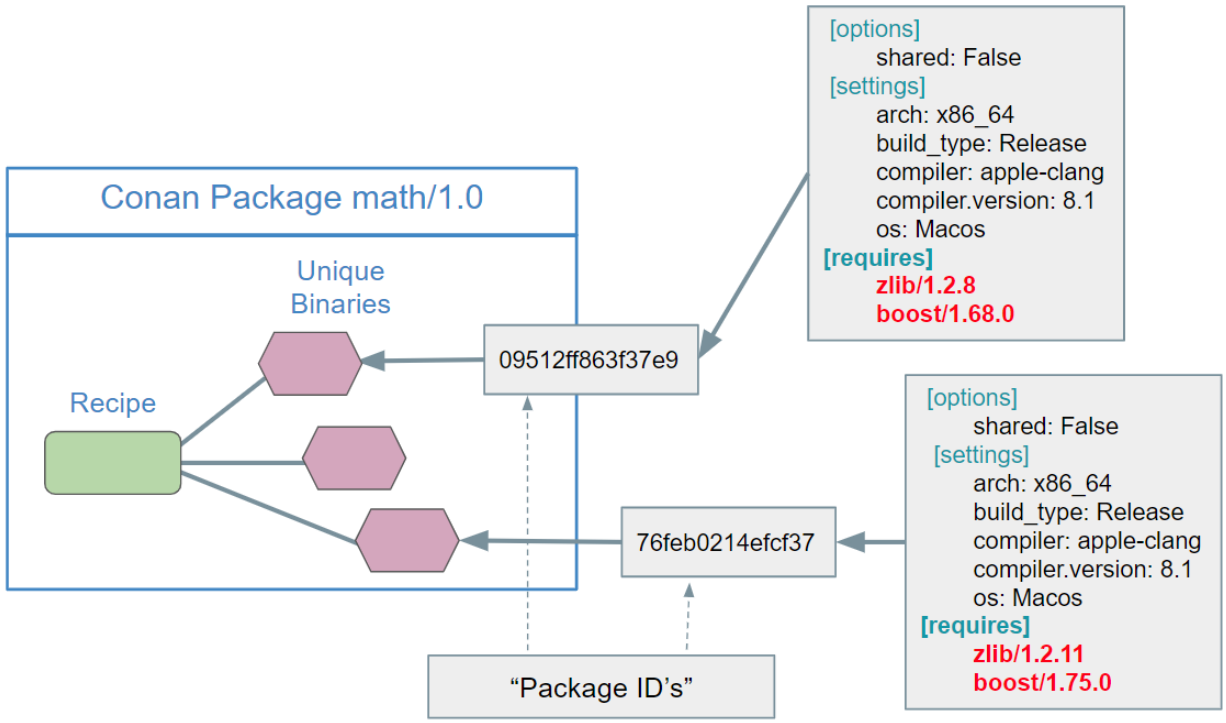
The `package_id` is the sha1 checksum of the `conaninfo.txt` file inside the package. You can validate it with the `sha1sum` utility.

If now we have a look to the binaries of `openssl` we can see something like:

```
$ conan list "openssl/3.1.2:*" -r=conancenter
conancenter
  openssl
    openssl/3.1.2
      revisions
        8879e931d726a8aad7f372e28470faa1 (2023-09-13 18:52:54 UTC)
          packages
            0348efdcd0e319fb58ea747bb94dbd88850d6dd1  # package_id
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: apple-clang
                  compiler.version: 13
                  os: Macos
                options
                  386: False
                  ...
                  shared: True
                requires
                  zlib/1.3.Z
```

We see now that the `conaninfo.txt` contains a new section the `requires` section. This happens because `openssl` depends on `zlib`, and due to the C and C++ compilation model, the dependencies can affect the binaries that use them. Some examples are when using inline or templates from `#include` header files of the dependency.

Expanding the image above:



As it can be seen, even if the `settings` and the `options` are the same, different binaries will be obtained if the dependencies versions change. In the next section *how the versions affect the package_id* is explained.

9.1.2 How settings and options of a recipe influence its package ID

In Conan, a package ID is a unique identifier for a package binary that takes into account all the factors that affect its binary compatibility. These factors include recipe options and settings as well as requirements or tool requirements.

Let's see how settings and options affect the package ID and some examples where they should not.

How settings influence the package ID

Settings are development project-wide variables, like the compiler, its version, or the OS itself. These variable values have to be defined, they should match the values of our development environment, and they cannot have a default value like options do.

For example, let's define a recipe that generates packages that are only OS dependent:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0.0"
    settings = "os"  # Only OS setting affects the package ID
```

If we generate a package from this recipe for Linux we will get the following package ID:

```
$ conan create . --settings os=Linux
...
```

(continues on next page)

(continued from previous page)

```
pkg/1.0.0: Package '9a4eb3c8701508aa9458b1a73d0633783ecc2270' created

$ conan list pkg/1.0.0:*
Local Cache
  pkg
    pkg/1.0.0
      revisions
          476929a74c859bb5f646363a4900f7cf (2024-03-07 09:13:43 UTC)
            packages
              9a4eb3c8701508aa9458b1a73d0633783ecc2270
                info
                  settings
                    os: Linux
```

If we do the same thing with Windows, now the package ID will be different:

```
$ conan create . --settings os=Windows
...
pkg/1.0.0: Package 'ebec3dc6d7f6b907b3ada0c3d3cdc83613a2b715' created

$ conan list pkg/1.0.0:*
Local Cache
  pkg
    pkg/1.0.0
      revisions
          476929a74c859bb5f646363a4900f7cf (2024-03-07 09:13:43 UTC)
            packages
              9a4eb3c8701508aa9458b1a73d0633783ecc2270
                info
                  settings
                    os: Linux
              ebec3dc6d7f6b907b3ada0c3d3cdc83613a2b715
                info
                  settings
                    os: Windows
```

Whenever a value of the settings or subsettings changes, the package ID will be different to reflect that.

The most common usage for settings is to model the different project-wide aspects that might influence the package ID. A recipe that does that will be:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0.0"
    settings = "os", "arch", "compiler", "build_type"
```

Now, compiling a package with different compiler versions will result into different package IDs:

```
$ conan create . --settings compiler.version=192
...
pkg/1.0.0: Package '4f267380690f99b3ef385199826c268f63147457' created
```

(continues on next page)

(continued from previous page)

```
$ conan create . --settings compiler.version=193
...
pkg/1.0.0: Package 'c13a22a41ecd72caf9e556f68b406569547e0861' created

$ conan list pkg/1.0.0:*
Local Cache
  pkg
    pkg/1.0.0
      revisions
        f1f48830ecb04f3b328429b390fc5de8 (2024-03-07 09:21:07 UTC)
          packages
            4f267380690f99b3ef385199826c268f63147457
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: msvc
                  compiler.cppstd: 14
                  compiler.runtime: dynamic
                  compiler.runtime_type: Release
                  compiler.version: 192
                  os: Windows
            c13a22a41ecd72caf9e556f68b406569547e0861
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: msvc
                  compiler.cppstd: 14
                  compiler.runtime: dynamic
                  compiler.runtime_type: Release
                  compiler.version: 193
                  os: Windows
```

Removing settings for a package used as a tool_require

There could be cases when a setting should not influence the resulting package ID. An example of this could be when a recipe packages a tool that would be used to build other packages via `tool_requires`

In that case, the value of the compiler used is needed for the compilation of the tool but not that relevant for consumers, as we only want to execute the tool to build other projects. So we could eventually remove the influence of the compiler from the pacakge ID:

```
from conan import ConanFile

class CMake(ConanFile):
    name = "cmake"
    version = "1.0.0"
    settings = "os", "arch", "compiler", "build_type"  # Only OS and architecture␣
↪influence the resulting package
```

(continues on next page)

(continued from previous page)

```
    def build(self):
        # self.settings.compiler value will be used here to compile cmake

    def package_id(self):
        # Remove compiler setting from package ID
        del self.info.settings.compiler
```

Why not removing the setting from the *settings* attribute? Because the compiler value is still needed in the *build()* method to perform the compilation of the executable.

Note: In the case we are generating our own executables (our own apps, not a `tool_require`), **removing the compiler setting from package ID is not recommended**, as we would always want to know that the package was generated with a specific compiler.

However, in case we are packaging a tool that does not even require a compiler input for building (a python script for example), we could also directly remove the settings attribute:

```
from conan import ConanFile

class MyPythonScripts(ConanFile):
    name = "my-python-scripts"
    version = "1.0.0"
    # No settings this time
```

Or, if the tool is platform specific we can just keep the OS and architecture information:

```
from conan import ConanFile

class MyScripts(ConanFile):
    name = "my-scripts"
    version = "1.0.0"
    settings = "os", "arch"
```

How options influence the package ID

Options are used to specify characteristics that are particular to a single recipe, contrasting with settings that generally remain consistent across recipes within a project. They are usually a set of particular characteristics of a library executable or conan package may have.

For example, a *shared* option is a very common option used in recipes that can produce shared libraries. However, it could not be a setting as not all recipes produce shared libraries.

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0.0"
    options = {"shared": [True, False]}
    default_options = {"shared": True}
```

As in the previous case with settings, the different values of an option will influence the package ID and therefore, generate different packages depending on it.

```
$ conan create . --options shared=True
...
pkg/1.0.0: Package '1744785cb24e3bdca70e27041dc5abd20476f947' created

$ conan create . --options shared=False
...
pkg/1.0.0: Package '55c609fe8808aa5308134cb5989d23d3caffccf2' created
```

In the same way, there might be "options" that are needed as input in a recipe to generate a package which shouldn't be taken into account in the package ID. An example of this could be an option to control something that during the build phase but that does not influence the package result, like the *verbosity* of a compilation. In that case, the recipe should remove the option in the *package_id() method*:

However, the general advice is that **options should always affect the package ID**, and in case we would like to have an input to the recipe that should **not** affect it, it should be done via the *conf section* of your profile. Then in the recipe we should just add:

```
from conan import ConanFile

class MyPkg(ConanFile):
    name = "my-pkg"
    version = "1.0.0"

    def build(self):
        verbosity = self.conf.get("user.my-pkg:verbosity")
        self.output.info(f"Using verbosity level: {verbosity})
        ...
```

Listing 1: *myprofile*

```
[conf]
user.my-pkg:verbosity=silent
```

That way the package ID will be not affected, the recipe will be cleaner (without irrelevant options for package ID) and the input is easily managed via the profile's conf section.

See also:

- *How the package_id is computed*
- *Configure settings and options in recipes*

9.1.3 The effect of dependencies on `package_id`

When a given package depends on a another package and uses it, the effect of dependencies can be different based on the package types:

For libraries:

- **Non-embed mode**: When an application, a shared library or a static library depends on another shared library, or when a static library depends on another static library, the "consumer" library does not do a copy of the binary artifacts of the "dependency" at all. We call it non-embed mode, the dependency binaries are not being linked or embedded in the consumer. This assumes that there are not inlined functionalities in the dependency headers, and the headers are pure interface and not implementation.

- **Embed mode**: When an application or a shared library depends on a header-only or a static-library, the dependencies binaries are copied or partially copied (depending on the linker) in the consumer binary. Also when a static library depends on a header-only library, it is considered that there will be embedding in the consumer binary of such headers, as they will also contain the implementation, it is impossible that they are a pure interface.

For header-only libraries:

- Header only libraries are always independent of their dependencies, because they do not link them. No change in the dependencies versions changes the final package of the header-only library.

For applications (`tool_requires`):

- **Build mode**: When some package uses a `tool_requires` of another package, the binary artifacts in the dependency are never copied or embedded.

When the package types are not known (the recipes do not define the `shared` option, or they do not define `package_type` attribute):

- **Unknown mode**: This mode is generally not recommended, as it is the result of not having enough information about the package types, and it results in a excesively optimistic `package_id` computation with a `semver` approach for binary rebuilds, which is generally insufficient for C and C++ libraries. The recommendation is to clear specify the `shared` option or to explicitly define `package_type`.

Non-embed mode

When we list the binaries of a package like `openssl` with dependencies:

```
$ conan list openssl/3.1.2:* -r=conancenter
conancenter
  openssl
    openssl/3.1.2
      revisions
        8879e931d726a8aad7f372e28470faa1 (2023-09-13 18:52:54 UTC)
          packages
            0348efdcd0e319fb58ea747bb94dbd88850d6dd1  # package_id
              info
                options
                  shared: True
                ...
                requires
                  zlib/1.3.Z
```

This binary was a `shared` library, linking with `zlib` as a shared library. This means it was using "non-embed" mode. The default of non-embed mode is `minor_mode`, which means:

- All `zlib` patch versions will be mapped to the same `zlib/1.3.Z`. This means that if our `openssl/3.1.2` package binary `0348efdcd0e319fb58ea747bb94dbd88850d6dd1` binary is considered binary compatible with all `zlib/1.3.Z` versions (for any `Z`), and will not require to rebuild the `openssl` binary.
- New `zlib` minor versions, like `zlib/1.4.0` will result in a "minor-mode" identifier like `zlib/1.4.Z`, and then, it will require a new `openssl/3.1.2` package binary, with a new `package_id`

Note: There was a bug in Conan versions previous to Conan 2.28 in this **non-embed mode**, in which transitive dependencies could affect the `package_id` of consumers, even if they were not direct dependencies, they were not being embedded, and they were not propagating headers to it. This was causing sub-optimal behavior, that could require some unnecessary builds from source for new `package_ids`. To avoid breaking existing users, this bug fix was introduced as opt-in via policies:

- If a recipe defines `required_conan_version = ">=2.28"` or higher, it will automatically enable the new fixed `package_id` computation. Recipes that don't update their required Conan version will still use the older `package_id`.
- If the global configuration in `global.conf` defines the `core:policies=["required_conan_version>=2.28"]` it will have the same behavior, enabling this bugfix for all packages. See the *documentation for policies* for more information.
- The recommendation is to activate the new behavior via policies. It will save resources like build time, and it will also be more future-proof, future Conan versions might enable this behavior unconditionally.

Embed mode

The following commands illustrate the concept of embed-mode. We create a `dep/0.1` package with a static library, and then we create a `app/0.1` package with an executable that links with static library inside `dep/0.1`. We can use the `conan new` command for quickly creating these two packages:

```
$ mkdir dep && cd dep
$ conan new cmake_lib -d name=dep -d version=0.1
$ conan create . -tf=""
$ cd .. && mkdir app && cd app
$ conan new cmake_exe -d name=app -d version=0.1 -d requires=dep/0.1
$ conan create .
dep/0.1: Hello World Release!
...
app/0.1: Hello World Release!
```

If we now list the `app/0.1` binaries, we will see the binary just created:

```
  $ conan list app/0.1:*
  Local Cache
    app/0.1
      revisions
        632e236936211ac2293ec33339ce582b (2023-09-25 22:34:17 UTC)
          packages
            3ca530d20914cf632eb00efbccc564da48190314
              info
                settings
                  ...
                requires
                  dep/0.1
↪#d125304fb1fb088d5b92d4f8135f4dff:9bdee485ef71c14ac5f8a657202632bdb8b4482b
```

It is now visible that the `app/0.1` package-id depends on the full identifier of the `dep/0.1` dependency, that includes both its recipe revision and `package_id`.

If we do a change now to the `dep` code, and re-create the `dep/0.1` package , even if we don't bump the version, it will create a new recipe revision:

```
$ cd ../dep
# Change the "src/dep.cpp" code to print a new message, like "Hello Moon"
$ conan create . -tf=""
# New recipe revision dep/0.1#1c90e8b8306c359b103da31faeee824c
```

So if we try now to install app/0.1 binary, it will fail with a "missing binary" error:

```
$ conan install --requires=app/0.1
ERROR: Missing binary: app/0.1:ef2b5ed33d26b35b9147c90b27b217e2c7bde2d0

app/0.1: WARN: Can't find a 'app/0.1' package binary
↪'ef2b5ed33d26b35b9147c90b27b217e2c7bde2d0' for the configuration:
[settings]
...
[requires]
dep/0.1#1c90e8b8306c359b103da31faeee824c:9bdee485ef71c14ac5f8a657202632bdb8b4482b

ERROR: Missing prebuilt package for 'app/0.1'
```

As the app executable links with the dep static library, it needs to be rebuilt to include the latest changes, even if dep/0.1 didn't bump its version, app/0.1 depends on "embed-mode" on dep/0.1, so it wil use down to the package_id of such dependency identifier.

Let's build the new app/0.1 binary:

```
$ cd ../app
$ conan create .
dep/0.1: Hello Moon Release!  # Message changed to Moon
...
app/0.1: Hello World Release!
```

Now we will have two app/0.1 different binaries:

```
$ conan list "app/0.1:*"
Local Cache
  app
    app/0.1
      revisions
        632e236936211ac2293ec33339ce582b (2023-09-25 22:49:32 UTC)
          packages
            3ca530d20914cf632eb00efbccc564da48190314
              info
                settings
                  ...
                requires
                  dep/0.1
↪#d125304fb1fb088d5b92d4f8135f4dff:9bdee485ef71c14ac5f8a657202632bdb8b4482b
            ef2b5ed33d26b35b9147c90b27b217e2c7bde2d0
              info
                settings
                  ...
                requires
                  dep/0.1
↪#1c90e8b8306c359b103da31faeee824c:9bdee485ef71c14ac5f8a657202632bdb8b4482b
```

We will have these two different binaries, one of them linking with the first revision of the dep/0.1 dependency (with the "Hello World" message), and the other binary with the other package_id linked with the second revision of the dep/0.1 dependency (with the "Hello Moon" message).

The above described mode is called full_mode, and it is the default for the embed_mode.

Package types and default modes

The "embed/non-embed/unknown" modes are derived from the recipes definitions of `package_type`. Recall that `package_type` can be explicitly defined as a recipe attribute, but it can also be implicity derived from the `shared` option. The resulting `package_id_mode` defines when a binary package needs to be built from source, depending on the chanages of the dependency version.

Important: It is very important to have a correct definition of `package_type`, and it is extremely recommended to define it, either explicitly with `package_type` recipe attribute, or by the definition of the `shared` options.

The following summarizes the defaults:

- Embed mode
  - Package types:
    - A `shared-library` or `application` consuming a `static-library`
    - An `application`, `shared-library` or `static-library` consuming a `header-library`
  - Default `package_id_mode`: `full_mode` any change (version, recipe, or binary) forces a consumer rebuild
- Non-embed mode
  - Package types:
    - A `static-library` linking with another `static-library`
    - An `application`, `shared-library` or `static-library` linking another `shared-library`.
  - Default `package_id_mode`: `minor_mode`: patch-version changes are ignored; minor or major changes force a rebuild
- Header-only library
  - Package types:
    - A `header-library` depending on any other `static-library`, `shared-library` or `header-library`.
  - Default `package_id_mode`: None. Header-only libraries are not affected in their `package_id` by their dependencies versions.
- Build mode
  - Package types:
    - Any package doing a `tool_requires` to other package (in recipe or by profile injection via `[tool_requires]`)
  - Default `package_id_mode`: None. By default, tools do not change the binary they help to produce (the binary is the same built with different versions of CMake, for example. For compilers this might not be true, but that effect is already captured by `compiler.version` setting).
- Unknown mode
  - Package types:
    - When any of the package types are not defined
  - Default `package_id_mode`: `semver_mode`: for versions >=`1.0` only major changes force a rebuild; for <`1.0` any change forces a rebuild

Note: There are different ways to change the defaults, like defining `core.package_id:default_embed_mode`, `core.package_id:default_non_embed_mode`, and `core.package_id:default_unknown_mode` in `global.conf`, or defining the equivalent recipe attributes. All of this will be explained in following sections.

9.1.4 Extending the binary model

There are a few mechanisms to extend the default Conan binary model:

Custom settings

It is possible to add new settings or subsettings in the *settings.yml* file, something like:

```
os:
    Windows:
        new_subsetting: [null, "subvalue1", "subvalue2"]
new_root_setting: [null, "value1", "value2"]
```

Where the `null` value allows leaving the setting undefined in profiles. If not including, it will be mandatory that profiles define a value for them.

The custom settings will be used explicitly or implicitly in recipes and packages:

```
class Pkg(ConanFile):
    # If we explicitly want this package binaries to vary according to 'new_root_setting'
    settings = "os", "compiler", "build_type", "arch", "new_root_setting"
    # While all packages with 'os=Windows' will implicitly vary according to 'new_
↪subsetting'
```

See also:

For the full reference of how `settings.yml` file can be customized *visit the settings section*. In practice, it is not necessary to modify the `settings.yml` file, and instead, it is possible to provide `settings_user.yml` file to extend the existing settings. See *the settings_user.yml documentation*.

Custom options

`Options` are custom to every recipe, there is no global definition of options like the `settings.yml` one.

Package `conanfile.py` recipes define their own options, with their own range of valid values and their own defaults:

```
class MyPkg(ConanFile):
    ...
    options = {"build_tests": [True, False],
               "option2": ["ANY"]}
    default_options = {"build_tests": True,
                       "option1": 42,
                       "z*:shared": True}
```

The options `shared`, `fPIC` and `header_only` have special meaning for Conan, and are considered automatically by most built-in build system integrations. They are also the recommended default to represent when a library is shared, static or header-only.

See also:

- *documentation for options*
- *documentation for default_options*.
- *Defining options for dependencies in recipes does not have strong guarantees*

Settings vs options vs conf

When to use settings or options or configuration?

- **Settings** are a project-wide configuration, something that typically affects the whole project that is being built and affects the resulting package binaries. For example, the operating system or the architecture would be naturally the same for all packages in a dependency graph, linking a Linux library to build a Windows app, or mixing architectures is impossible. Settings cannot be defaulted in a package recipe. A recipe for a given library cannot say that its default is `os=Windows`. The `os` will be given by the environment in which that recipe is processed. It is a mandatory input to be defined in the input profiles.
- On the other hand, **options** are a package-specific configuration that affects the resulting package binaries. Static or shared library are not settings that apply to all packages. Some can be header only libraries while other packages can be just data, or package executables. For example, `shared` is a common option (the default for specifying if a library can be static or shared), but packages can define and use any options they want. Options are defined in the package `conanfile.py` recipe, including their supported and default values with `options` and `default_options`.
- Configuration via `conf` is intended for configuration that does not affect the resulting package binaries in the general case. For example, building one library with the `tools.cmake.cmaketoolchain:generator=Ninja` shouldn't result in a binary different than if built with Visual Studio (just a typically faster build thanks to Ninja).

There are some exceptions to the above. For example, settings can be defined per-package using the `<pattern:>setting=value`, both in profiles and command line:

```
$ conan install . -s mypkg/*:compiler=gcc -s compiler=clang ..
```

This will use `gcc` for "mypkg" and `clang` for the rest of the dependencies (in most cases it is recommended to use the same compiler for the whole dependency graph, but some scenarios when strong binary compatibility is guaranteed, it is possible to mix libraries built with different compilers).

There are situations whereby many packages use the same option value, thereby allowing you to set its value once using patterns, like:

```
$ conan install . -o *:shared=True
```

Custom configuration

As commented above, the Conan `conf` configuration system is intended to tune some of the tools and behaviors, but without really affecting the resulting package binaries. Some typical `conf` items are activating parallel builds, configuring "retries" when uploading to servers, or changing the CMake generator. Read more about *the Conan configuration system in this section*.

There is also the possibility to define `user.xxxx:conf=value` for user-defined configuration, that in the same spirit as core and tools built-in configurations, do not affect the `package_id` of binaries.

But there might be some special situations in which it is really desired that some `conf` defines different `package_ids`, creating different package binaries. It is possible to do this in two different places:

- Locally, in the recipe's `package_id` method, via the `self.info.conf` attribute:

```
def package_id(self):
    # We can get the value from the actual current conf value, or define a new value
    value = self.conf.get("user.myconf:myitem")
    # This ``self.info.conf`` will become part of the ``package_id``
    self.info.conf.define("user.myconf:myitem", value)
```

- Globally, with the `tools.info.package_id:confs` configuration, receiving as argument a list of existing configuration to be part of the package ID, so you can define in profiles:

```
tools.info.package_id:confs=["tools.build:cxxflags", ...]
```

The value of the `package_id` will contain the value provided in the `tools.build:cxxflags` and other configurations. Note that this value is managed as a string, changing the string, will produce a different result and a different `package_id`, so if this approach is used, it is very important to be very consistent with the provided values for different configurations like `tools.build:cxxflags`.

It is also possible to use regex expressions to match several `confs`, instead of listing all of them, for example `.*cmake` could match any configuration that contains "cmake" in its name (not that this is recommended, see best practices below).

Note: Best practices

In general, defining variability of binaries `package_id` via `conf` should be reserved for special situations and always managed with care. Passing many different `confs` to the `tools.info.package_id:confs` can easily result in issues like missing binaries or unnecessarily building too many binaries. If that is the case, consider building higher level abstraction over your binaries with new custom settings or options.

Cross build target settings

The `self.settings_target` is a `conanfile.py` attribute that becomes relevant in cross-compilation scenarios for the `tool_requires` tools in the "build" context. When we have a `tool_requires` like CMake, lets say the `cmake/3.25.3`, the package binary is independent of the possible platform that cross-compiling will target, it is the same `cmake` executable for all different target platforms. The `settings` for a cross-building from Windows-X64 to Linux-armv8 scenario for the `cmake` conanfile recipe would be:

- `self.settings`: The settings where the current `cmake/3.25.3` will run. As it is a tool-require, it will run in the Windows machine, so `self.settings.os = Windows` and `self.settings.arch = x86_64`.
- `self.settings_build`: The settings of the current build machine that would build this package if necessary. This is also the Windows-x64 machine, so `self.settings_build.os = Windows` and `self.settings_build.arch = x86_64` too.
- `self.settings_target`: The settings that the current application outcome will target. In this case it will be `self.settings_target.os = Linux` and `self.settings_target.arch = armv8`

In the `cmake` package scenario, as we pointed out, the target is irrelevant. It is not used in the `cmake` conanfile recipe at all, and it doesn't affect the `package_id` of the `cmake` binary package.

But there are situations when the binary package can be different based on the target platform. For example a cross-compiler `gcc` that has a different `gcc` executable based on the target it will compile for. This is typical in the GNU ecosystem where we can find `arm-gcc` toolchains, for example, specific for a given architecture. This scenario can be reflected by Conan, extending the `package_id` with the value of these `settings_target`:

```
def package_id(self):
    self.info.settings_target = self.settings_target
    # If we only want the ``os`` and ``arch`` settings, then we remove the other:
    self.info.settings_target.rm_safe("compiler")
    self.info.settings_target.rm_safe("build_type")
```

9.1.5 Customizing the binary compatibility

The default binary compatibility requires an almost exact match of settings and options, and a versioned match of dependencies versions, as explained in the *previous section about dependencies*.

In summary, the required binaries `package_id` when installing dependencies should match by default:

- All the settings in the `package_id` except `compiler.cppstd` should match exactly the ones provided in the input profiles, including the compiler version. So `compiler.version=9` is different than `compiler.version=9.1`.
- The default behavior will assume binary compatibility among different `compiler.cppstd` values for C++ packages, being able to fall back to other values rather than the one specified in the input profiles, if the `cppstd` required by the input profile does not exist. This is controlled by the `compatibility.py` plugin, that can be customized by users.
- All the options in the `package_id` should match exactly the ones provided in the input profiles.
- The versions of the dependencies should match:
  - In case of "embedding dependencies", should match the exact version, including the recipe-revision and the dependency `package_id`. The `package_revision` is never included as it is assumed to be ill-formed to have more than one `package_revision` for the same `package_id`.
  - In case of "non-embedding dependencies", the versions of the dependencies should match down to the `minor` version, being the `patch`, `recipe_revision` and further information not taken into account.
  - In case of "tool dependencies", the versions of the dependencies do not affect at all by default to the consumer `package_id`.

These rules can be customized and changed using different approaches, depending on the needs, as explained in following sections

Customizing binary compatibility of settings and options

Information erasure in package_id() method

Recipes can **erase** information from their `package_id` using their `package_id()` method. For example, a package containing only an executable can decide to remove the information from `settings.compiler` and `settings.build_type` from their `package_id`, assuming that an executable built with any compiler will be valid, and that it is not necessary to store different binaries built with different compilers:

```
def package_id(self):
    del self.info.settings.compiler
    del self.info.settings.build_type
```

It is also possible to assign a value for a given setting, for example if we want to have one single binary for all gcc versions included in the [>=5 <7>] range, we could do:

```
def package_id(self):
    if self.info.settings.compiler == "gcc":
        version = Version(self.info.settings.compiler.version)
        if version >= "5.0" and version < "7.0":
            self.info.settings.compiler.version = "gcc5-6"
```

Note: Best practice

Note that information erasure in `package_id()` means that 1 single `package_id` will represent a whole range of different settings, but the information of what exact setting was used to create the binary will be lost, and only 1 binary can be created for that range. Re-creating the package with different settings in the range, will create a new binary that overwrites the previous one (with a new package-revision).

If we want to be able to create, store and manage different binaries for different input settings, information erasure can't be used, and using the below `compatibility` approaches is recommended.

See also:

- *Conan packages binary compatibility: the package ID*
- *package_id() method reference*

The compatibility() method

Recipes can define their binary compatibility rules, using their `compatibility()` method. For example, if we want that binaries built with gcc versions 4.8, 4.7 and 4.6 to be considered compatible with the ones compiled with 4.9 we could declare a `compatibility()` method like this:

```
def compatibility(self):
    if self.settings.compiler == "gcc" and self.settings.compiler.version == "4.9":
        return [{"settings": [("compiler.version", v)]}
                for v in ("4.8", "4.7", "4.6")]
```

Read more about the `compatibility()` method in *the compatibility() method reference*

The `compatibility.py` plugin

Compatibility can be defined globally via the `compatibility.py` plugin, in the same way that the `compatibility()` method does for one recipe, but for all packages globally.

Check the binary compatibility *compatibility.py extension*.

Customizing binary compatibility of dependencies versions

Global default package_id modes

The `core.package_id:default_xxx` configurations defined in `global.conf` can be used to globally change the defaults of how dependencies affect their consumers

```
core.package_id:default_build_mode: By default, 'None'
core.package_id:default_embed_mode: By default, 'full_mode'
core.package_id:default_non_embed_mode: By default, 'minor_mode'
core.package_id:default_python_mode: By default, 'minor_mode'
core.package_id:default_unknown_mode: By default, 'semver_mode'
```

These confs affect how *the package id* is calculated, so changing them will affect your generated binaries. It's thus recommended that they stay consistent across your organization.

Note: Best practices

It is strongly recommended that the `core.package_id:default_xxx` should be global, consistent and immutable across organizations. It can be confusing to change these defaults for different projects or teams, because it will result in missing binaries.

It should also be consistent and shared with the consumers of generated packages if those packages are shared outside the organization, in that case sharing the `global.conf` file via `conan config install` could be recommended.

Consider using the Conan defaults, they should be a good balance between efficiency and safety, ensuring exact re-building for embed cases, and good control via versions for non-embed cases.

Custom package_id modes for recipe consumers

Recipes can define their default effect to their consumers, via some `package_id_xxxx_mode` attributes.

The `package_id_embed_mode`, `package_id_non_embed_mode`, `package_id_unknown_mode` are class attributes that can be defined in recipes to define the effect they have on their consumers `package_id`, when they are consumed as `requires`. The `build_mode` (experimental) is a class attribute that affects the package consumers when these consumers use it as `tool_requires`. Can be declared as:

```
from conan import ConanFile

class Pkg(ConanFile):
    ...
    package_id_embed_mode = "full_mode"
    package_id_non_embed_mode = "patch_mode"
    package_id_unknown_mode = "minor_mode"
    build_mode = "patch_mode"  # when this is used with tool_requires
    # For exceptional cases when we explicitly want to make consumers
    # depend on this dependency option value
    package_id_abi_options = ["shared"]
```

Read more in *package_id_{embed,non_embed,python,unknown}_mode, build_mode* and in *package_id_abi_options*.

Custom package_id from recipe dependencies

Recipes can define how their dependencies affect their `package_id`, using the `package_id_mode` trait:

```
from conan import ConanFile

class Pkg(ConanFile):
    def requirements(self):
        self.requires("mydep/1.0", package_id_mode="patch_mode")
```

Using `package_id_mode` trait does not differentiate between the "embed" and "non-embed" cases, it is up to the user to define the correct value. It is likely that this approach should only be used for very special cases that do not have variability of shared/static libraries controlled via `options`.

Note that the `requirements()` method is evaluated while the graph is being expanded, the dependencies do not exist yet (haven't been computed), so it is not possible to know the dependencies options. In this case it might be preferred to use the `package_id()` method.

The `package_id()` method can define how the dependencies affect the current package with:

```
from conan import ConanFile

class Pkg(ConanFile):
    def package_id(self):
        self.info.requires["mydep"].major_mode()
```

The different modes that can be used are defined in *package_id_{embed,non_embed,python,unknown}_mode, build_mode*

9.2 Commands

This section describes the Conan built-in commands, like `conan install` or `conan search`.

It is also possible to create user custom commands, visit *custom commands reference* and these *custom command examples*

Consumer commands:

9.2.1 conan cache

Perform file operations in the local cache (of recipes and/or packages).

conan cache path

```
$ conan cache path -h
usage: conan cache path [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF]
                        [--folder {export_source,source,build,metadata}]
                        reference
```

(continues on next page)

(continued from previous page)

```
Show the path to the Conan cache for a given reference.

positional arguments:
  reference             Recipe reference or Package reference

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --folder {export_source,source,build,metadata}
                        Path to show. The 'build' requires a package
                        reference. If the argument is not passed, it shows
                        'exports' path for recipe references and 'package'
                        folder for package references.
```

The `conan cache path` returns the path in the cache of a given reference. Depending on the reference, it could return the path of a recipe, or the path to a package binary.

Let's say that we have created a package in our current cache with:

```
$ conan new cmake_lib -d name=pkg -d version=0.1
$ conan create .
...
Requirements
    pkg/0.1#cdc0d9d0e8f554d3df2388c535137d77 - Cache

Requirements
    pkg/0.1#cdc0d9d0e8f554d3df2388c535137d77:2401fa1d188d289bb25c37cfa3317e13e377a351 -␣
↪Build
```

And now we are interested in obtaining the path where our `pkg/0.1` recipe `conanfile.py` has been exported:

```
$ conan cache path pkg/0.1
<path to conan cache>/p/5cb229164ec1d245/e

$ ls <path to conan cache>/p/5cb229164ec1d245/e
conanfile.py  conanmanifest.txt
```

By default, if the recipe revision is not specified, it means the "latest" revision in the cache. This can also be made explicit by the literal `#latest`, and also any recipe revision can be explicitly defined, these commands are equivalent to the above:

```
$ conan cache path pkg/0.1#latest
```

(continues on next page)

(continued from previous page)

```
<path to conan cache>/p/5cb229164ec1d245/e

# The recipe revision might be different in your case.
# Check the "conan create" output to get yours
$ conan cache path pkg/0.1#cdc0d9d0e8f554d3df2388c535137d77
<path to conan cache>/p/5cb229164ec1d245/e
```

Together with the recipe folder, there are a two other folders that are common to all the binaries produced with this recipe: the "export_source" folder and the "source" folder. Both can be obtained with:

```
$ conan cache path pkg/0.1 --folder=export_source
<path to conan cache>/p/5cb229164ec1d245/es

$ ls <path to conan cache>/p/5cb229164ec1d245/es
CMakeLists.txt  include/  src/

$ conan cache path pkg/0.1 --folder=source
<path to conan cache>/p/5cb229164ec1d245/s

$ ls <path to conan cache>/p/5cb229164ec1d245/s
CMakeLists.txt  include/  src/
```

In this case the contents of the "source" folder are identical to the ones of the "export_source" folder because the recipe did not implement any `source()` method that could retrieve code or do any other operation over the code, like applying patches.

The recipe revision by default will be `#latest`, this follows the same rules as above.

Note that these two folders will not exist if the package has not been built from source, like when a precompiled binary is retrieve from a server.

It is also possible to obtain the folders of the binary packages providing the `package_id`:

```
# Your package_id might be different, it depends on the platform
# Check the "conan create" output to obtain yours
$ conan cache path pkg/0.1:2401fa1d188d289bb25c37cfa3317e13e377a351
<path to conan cache>/p/1cae77d6250c23b7/p

$ ls <path to conan cache>/p/1cae77d6250c23b7/p
conaninfo.txt  conanmanifest.txt  include/  lib/
```

As above, by default it will resolve to the "latest" recipe revision and package revision. The command above is equal to explicitly defining `#latest` or the exact revisions. All the commands below are equivalent to the above one:

```
$ conan cache path pkg/0.1#latest:2401fa1d188d289bb25c37cfa3317e13e377a351
<path to conan cache>/p/1cae77d6250c23b7/p

$ conan cache path pkg/0.1#latest:2401fa1d188d289bb25c37cfa3317e13e377a351#latest
<path to conan cache>/p/1cae77d6250c23b7/p

$ conan cache path pkg/0.1
↪#cdc0d9d0e8f554d3df2388c535137d77:2401fa1d188d289bb25c37cfa3317e13e377a351
<path to conan cache>/p/1cae77d6250c23b7/p
```

It is possible to access the "build" folder with all the temporary build artifacts:

```
$ conan cache path pkg/0.1:2401fa1d188d289bb25c37cfa3317e13e377a351 --folder=build
<path to conan cache>/p/1cae77d6250c23b7/b

ls -al <path to conan cache>/p/1cae77d6250c23b7/b
build/  CMakeLists.txt  CMakeUserPresets.json  conaninfo.txt  include/  src/
```

Again, the "build" folder will only exist if the package was built from source.

Note: Best practices

- This `conan cache path` command is intended for eventual inspection of the cache, but the cache package storage must be considered **read-only**. Do not modify, change, remove or add files from the cache.
- If you are using this command to obtain the path to artifacts and then copying them, consider the usage of a `deployer` instead. In the general case, extracting artifacts from the cache manually is discouraged.
- Developers can use the `conan list ... --format=compact` to get the full references in a compact way that can be copied and pasted into the `conan cache path` command

conan cache clean

```
$ conan cache clean -h
usage: conan cache clean [-h] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [-l LIST] [-s] [-b] [-d] [-t] [-bs]
                         [-p PACKAGE_QUERY]
                         [pattern]

Remove non-critical folders from the cache, like source, build and/or download
(.tgz store) ones.

positional arguments:
  pattern               Selection pattern for references to clean

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list of packages to clean
  -s, --source          Clean source folders
  -b, --build           Clean build folders
  -d, --download        Clean download and metadata folders
  -t, --temp            Clean temporary folders
```

(continues on next page)

(continued from previous page)

```
  -bs, --backup-sources
                        Clean backup sources
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Remove only the packages matching a specific query,
                        e.g., os=Windows AND (arch=x86 OR compiler=gcc)
```

This command will remove all temporary folders, along with the source, build and download folder that Conan generates in its execution. It will do so for every matching reference passed in *pattern*, or the contents of the pkglist file if the `--list` option is used. It's possible to limit the cleaning to certain kinds of folders with different flags.

Examples:

- Remove all non-critical files:

```
$ conan cache clean "*"
```

- Remove all temporary files:

```
$ conan cache clean "*" --temp
```

- Remove the download folders for the `zlib` recipe:

```
$ conan cache clean "zlib/*" --download
```

- Remove everything but the download folder for the `zlib` recipe:

```
$ conan cache clean "zlib/*" --source --build --temp
```

- Get a list of packages to remove temp files from, then remove them:

```
$ conan list "zlib/*" -f=json > pkglist.json
$ conan cache clean --list pkglist.json
```

conan cache check-integrity

```
$ conan cache check-integrity -h
usage: conan cache check-integrity [-h] [-f FORMAT] [--out-file OUT_FILE]
                                   [-v [{quiet,error,warning,notice,status,verbose,debug,
↪v,trace,vv}]]
                                   [-cc CORE_CONF] [-l LIST]
                                   [-p PACKAGE_QUERY]
                                   [pattern]

Check the integrity of the local cache for the given references

positional arguments:
  pattern               Selection pattern for references to check integrity
                        for

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
```

(continues on next page)

(continued from previous page)

```
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list of packages to check integrity for
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Only the packages matching a specific query, e.g.,
                        os=Windows AND (arch=x86 OR compiler=gcc)
```

The `conan cache check-integrity` command checks the integrity of Conan packages in the local cache that match the given *pattern*, or the contents of the pkglist file if the `--list` option is used. This means that it will throw an error if any file included in the `conanmanifest.txt` is missing or does not match the declared checksum in that file.

For example, to verify the integrity of the whole Conan local cache, do:

```
$ conan cache check-integrity "*"
mypkg/1.0: Integrity checked: ok
mypkg/1.0:454923cd42d0da27b9b1294ebc3e4ecc84020747: Integrity checked: ok
mypkg/1.0:454923cd42d0da27b9b1294ebc3e4ecc84020747: Integrity checked: ok
zlib/1.3.1: Integrity checked: ok
zlib/1.3.1:6fe7fa69f760aee504e0be85c12b2327c716f9e7: Integrity checked: ok
```

This command can also return a pkglist when the `--format=json` option is used. This returns the packages the are corrupted, which is useful for generating a list of packages that can later be used, for example, to remove all potentially corrupted packages in a single operation:

```
$ conan cache check-integrity "*" --format=json --out-file pkglist.json
$ conan remove --list pkglist.json
```

conan cache backup-upload

```
$ conan cache backup-upload -h
usage: conan cache backup-upload [-h] [--out-file OUT_FILE]
                                 [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                                 [-cc CORE_CONF]

Upload all the source backups present in the cache

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
```

(continues on next page)

(continued from previous page)

```
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

The `conan cache backup-upload` will upload all source backups present in the local cache to the backup server, (excluding those which have been fetched from the excluded urls listed in the `core.sources:exclude_urls` conf), regardless of which package they belong to, if any.

conan cache save

```
$ conan cache save -h
usage: conan cache save [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [-l LIST] [--file FILE] [--no-source]
                        [pattern]

Get the artifacts from a package list and archive them

positional arguments:
  pattern               A pattern in the form
                        'pkg/version#revision:package_id#revision', e.g:
                        zlib/1.2.13:* means all binaries for zlib/1.2.13. If
                        revision is not specified, it is assumed latest one.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list of packages to save
  --file FILE           Save to this file. Allowed extensions .tgz, .txz,
                        .tzst (.txz and .tzst experimental and .tzst requires
                        Python>=3.14)
  --no-source           Exclude the sources
```

Read more in *Save and restore packages from/to the cache*.

conan cache restore

```
$ conan cache restore -h
usage: conan cache restore [-h] [-f FORMAT] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF]
                           file

Put the artifacts from an archive into the cache

positional arguments:
  file                  Path to archive to restore

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Read more in *Save and restore packages from/to the cache*.

Warning: The archive is extracted with the `fully_trusted` tar filter. Only restore archives from trusted origins, see *the security guidelines*.

conan cache ref

```
$ conan cache ref -h
usage: conan cache ref [-h] [--out-file OUT_FILE]
                       [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                       [-cc CORE_CONF]
                       path

Show the reference for a given Conan cache folder

positional arguments:
  path                  Path to a Conan cache folder

options:
  -h, --help            show this help message and exit
```

(continues on next page)

(continued from previous page)

```
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

For a given cache folder, returns the Conan reference, that is, a recipe reference in the form `name/version#recipe_revision`, or a package reference in the form `name/version#recipe_revision:package_id#package_revision` (both could also have user/channel), depending on the contents of the folder.

This is a developer and debugging command, intended for occasional developer usage while debugging potential issues, but it is not recommended for any other use case.

Note: Best practices

Navigating the Conan cache is not an intended or supported use case. Using the `conan cache ref` command in any automation, CI or scripting is strongly discouraged. The `conan cache ref` is intended exclusively to be a helper command for developers while debugging.

---

conan cache sign

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan cache sign -h
usage: conan cache sign [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [-l LIST] [-p PACKAGE_QUERY]
                        [pattern]

Sign packages with the Package Signing Plugin

positional arguments:
  pattern               Selection pattern for references to be signed

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
```

(continues on next page)

(continued from previous page)

```
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list of packages to be signed
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Only the packages matching a specific query, e.g.,
                        os=Windows AND (arch=x86 OR compiler=gcc)
```

Signs the packages matching the pattern/reference or package list provided. For example:

```
$ conan list zlib/1.3.1:* --format=json > list.json

$ conan cache sign --list=list.json
[Package sign] Results:

zlib/1.3.1
revisions
    bfceb3f8904b735f75c2b0df5713b1e6
    packages
        7bfde258ff4f62f75668d0896dbddedaa7480a0f

[Package sign] Summary: OK=1, FAILED=0
```

This command requires a configured package signing plugin, read more in *Package signing*.

conan cache verify

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan cache verify -h
usage: conan cache verify [-h] [-f FORMAT] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF] [-l LIST] [-p PACKAGE_QUERY]
                          [pattern]

Check the signature of packages with the Package Signing Plugin

positional arguments:
  pattern               Selection pattern for references to verify their
                        signature

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
```

(continues on next page)

(continued from previous page)

```
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list of packages to verify their signature
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Only the packages matching a specific query, e.g.,
                        os=Windows AND (arch=x86 OR compiler=gcc)
```

Verifies the signatures of the packages matching the pattern/reference or a package list.

This command requires as configured package signing plugin, read more in *Package signing*.

9.2.2 conan config

Manage the Conan configuration in the Conan home.

conan config home

```
$ conan config home -h
usage: conan config home [-h] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF]

Show the Conan home folder.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

The `conan config home` command returns the path of the Conan home folder.

```
$ conan config home

/home/user/.conan2
```

conan config install

```
$ conan config install -h
usage: conan config install [-h] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF] [--verify-ssl [VERIFY_SSL] |
                            --insecure] [-t {git,dir,file,url}] [-a ARGS]
                            [-sf SOURCE_FOLDER] [-tf TARGET_FOLDER]
                            item

Install the configuration (remotes, profiles, conf), from git, http or a
folder, into the Conan home folder.

positional arguments:
  item                  git repository, local file or folder or zip file
                        (local or http) where the configuration is stored

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --verify-ssl [VERIFY_SSL]
                        Verify SSL connection when downloading file
  --insecure            Allow insecure server connections when using SSL.
                        Equivalent to --verify-ssl=False
  -t {git,dir,file,url}, --type {git,dir,file,url}
                        Type of remote config
  -a ARGS, --args ARGS  String with extra arguments for "git clone"
  -sf SOURCE_FOLDER, --source-folder SOURCE_FOLDER
                        Install files only from a source subfolder from the
                        specified origin
  -tf TARGET_FOLDER, --target-folder TARGET_FOLDER
                        Install to that path in the conan cache
```

The `conan config install` command is intended to install in the current home a common shared Conan configuration, like the definitions of `remotes`, `profiles`, `settings`, `hooks`, `extensions`, etc.

The command can use as source any of the following:

- A URL pointing to a zip archive containing the configuration files

- A git repository containing the files
- A local folder
- Just one file

Files in the current Conan home will be replaced by the ones from the installation source. All the configuration files can be shared and installed this way:

- `remotes.json` for the definition of remotes
- Any custom profile files inside a `profiles` subfolder
- Custom `settings.yml`
- Custom `global.conf`
- All the extensions, including plugins, hooks.
- Custom user commands.

This command reads a `.conanignore` file which, if present, filters which files and folders are copied over to the user's Conan home folder. This file uses fnmatch patterns to match over the folder contents, excluding those entries that match from the config installation. See conan-io/command-extensions's .conanignore for an example of such a file. You can force certain files to be copied over by using the `!` negation syntax:

```
# Ignore all files
*
# But copy the file named "settings.yml"
!settings.yml
```

Examples:

- Install the configuration from a URL:

```
$ conan config install http://url/to/some/config.zip
```

- Install the configuration from a URL, but only getting the files inside a *origin* folder inside the zip file, and putting them inside a *target* folder in the local cache:

```
$ conan config install http://url/to/some/config.zip -sf=origin -tf=target
```

- Install configuration from 2 different zip files from 2 different urls, using different source and target folders for each one, then update all:

```
$ conan config install http://url/to/some/config.zip -sf=origin -tf=target
$ conan config install http://url/to/some/config.zip -sf=origin2 -tf=target2
$ conan config install http://other/url/to/other.zip -sf=hooks -tf=hooks
```

- Install the configuration from a Git repository with submodules:

```
$ conan config install http://github.com/user/conan_config/.git --args="--recursive"
```

  You can also force the git download by using **`--type git`** (in case it is not deduced from the URL automatically):

```
$ conan config install http://github.com/user/conan_config/.git --type git
```

- Install the configuration from a specific Git branch:

```
$ conan config install http://github.com/user/conan_config/.git --args="--branch␣
↪mybranch"
```

- Install from a URL skipping SSL verification:

```
$ conan config install http://url/to/some/config.zip --verify-ssl=False
```

  This will disable the SSL check of the certificate.

- Install a specific file from a local path:

```
$ conan config install my_settings/settings.yml
```

- Install the configuration from a local path:

```
$ conan config install /path/to/some/config.zip
```

Note: Since Conan 2.31, `conan config install` will warn if a `config_version.json` file already exists in the Conan home (which is created by `conan config install-pkg` to track installed configuration package versions). Mixing `conan config install` with `conan config install-pkg` is not recommended, as the plain `conan config install` is not tracked in `config_version.json` and may result in a configuration that is inconsistent with the tracked versions. In that case, consider running `conan config clean` first, or avoid mixing the two commands.

conan config install-pkg

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan config install-pkg -h
usage: conan config install-pkg [-h] [--out-file OUT_FILE]
                                [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                                [-cc CORE_CONF] [-l LOCKFILE]
                                [--lockfile-partial]
                                [--lockfile-out LOCKFILE_OUT] [-f]
                                [--insecure] [--url URL] [-pr PROFILE]
                                [-s SETTINGS] [-o OPTIONS]
                                [reference]

(Experimental) Install the configuration (remotes, profiles, conf), from a
Conan package or from a conanconfig.yml file

positional arguments:
  reference             Package reference 'pkg/version' to install
                        configuration from or path to 'conanconfig.yml' file

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
```

(continues on next page)

(continued from previous page)

```
                      verbose to more verbose: -vquiet, -verror, -vwarning,
                      -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                      -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                      Define core configuration, overwriting global.conf
                      values. E.g.: -cc core:non_interactive=True
  -l LOCKFILE, --lockfile LOCKFILE
                      Path to a lockfile. Use --lockfile="" to avoid
                      automatic use of existing 'conan.lock' file
  --lockfile-partial  Do not raise an error if some dependency is not found
                      in lockfile
  --lockfile-out LOCKFILE_OUT
                      Filename of the updated lockfile
  -f, --force         Force the re-installation of configuration
  --insecure          Allow insecure server connections when using SSL
  --url URL           (Experimental) Provide Conan repository URL (for first
                      install without remotes)
  -pr PROFILE, --profile PROFILE
                      Profile to install config
  -s SETTINGS, --settings SETTINGS
                      Settings to install config
  -o OPTIONS, --options OPTIONS
                      Options to install config
```

This command allows to install configuration from a Conan package stored in a Conan server.

The packages containing configuration follow some special rules:

- They must define the `package_type = "configuration"`
- The configuration files must be packaged in the final "binary" package, following the same layout as they would for other `conan config install` cases.
- They cannot be used as `requires` of other packages, because that would result in a chicken-and-egg problem.
- They cannot contain `requires` to other packages
- The configuration packages are created with `conan create` and `conan export-pkg` as other packages, and uploaded to the servers with `conan upload`

To install a configuration from a Conan configuration package, it is possible:

- To generate a lockfile file with `--lockfile-out`. This lockfile file can be passed to `conan config install-pkg --lockfile` (it will automatically loaded it if is named `conan.lock` and found in the current directory) in the future to guarantee the same exact version.
- Version ranges can be used `conan config install-pkg "myconf/[>=1.0 <2]"` is correct, and it will install the latest one in that range.
- `conan config install-pkg` always look in the server for the latest version or revision.
- If the same version and revision was downloaded and installed from the server, `conan config install-pkg` will be a no-op unless `--force` is used, in this case the configuration will be overwritten.

It is also possible to make the version of the configuration affect all packages `package_id` and be part of the binary model, by activating the `core.package_id:config_mode` conf (this is also experimental), to any available mode, like `minor_mode`. Note that the order of the installation of packages in case multiple configuration packages are installed is important. This is why Conan will raise an error if the relative order of installed configuration packages changes as the result of installing updates for those configuration packages.

As the `conan config install-pkg` command downloads the package from a Conan remote server, it can download from an already existing remote, or it can download from a Conan remote directly specifying the repository URL:

```
$ conan config install-pkg myconf/version --url=<url/conan/remote/repo>
```

In the same way that `conan remote add` can define `--insecure` to disable the SSL verification for that remote, it is possible to disable it for `conan config install-pkg` with:

```
$ conan config install-pkg myconf/version --url=<url/conan/remote/repo> --insecure
```

When specifying the `--url` argument, a Conan remote named `config_install_url` is created on the fly. That means that if authentication is desired via env-vars, the env-var names will be `CONAN_LOGIN_USERNAME_CONFIG_INSTALL_URL` or `CONAN_PASSWORD_CONFIG_INSTALL_URL`.

Conan configuration packages can also be parameterized depending on profiles, settings and options. For example, if some organization would like to manage their configuration slightly differently for Windows and other platforms they could do:

```
import os
from conan import ConanFile
from conan.tools.files import copy

class Conf(ConanFile):
    name = "myconf"
    version = "0.1"
    settings = "os"
    package_type = "configuration"
    def package(self):
        f = "win" if self.settings.os == "Windows" else "nix"
        copy(self, "*.conf", src=os.path.join(self.build_folder, f), dst=self.package_
↪folder)
```

And if they had a layout with different `global.conf` for the different platforms, like:

```
conanfile.py
win/global.conf
nix/global.conf
```

They could create and upload their configuration package as:

```
$ conan export-pkg . -s os=Windows
$ conan export-pkg . -s os=Linux
$ conan upload "*" -r=remote -c
```

Then, developers could do:

```
$ conan config install-pkg "myconf/[*]" -s os=Linux
# or even implicitly, if they default build profile defines os=Linux
$ conan config install-pkg "myconf/[*]"
```

And they will get the correct configuration for their platform.

See also:

- If you lock installed configuration packages in a lockfile, you could use the *conan lock upgrade-config* command to update such a lockfile.

conanconfig.yml

The `conan config install-pkg` admits also as an input a yaml `conanconfig.yml` file that can contain more than one package requirement, something like:

```
packages:
    - myconf_a/0.1
    - myconf_b/0.1
    - myconf_c/[>=1 <2]
```

and be used like `conan config install-pkg .` or even just `conan config install-pkg`.

The file also admits the definition of `urls` with the same meaning as the `--url` command line argument, to simplify the initial installation of configuration when doing a Conan setup:

Listing 2: conanconfig.yml

```
packages:
    - myconf_a/0.1
    - myconf_b/0.1
    - myconf_c/[>=1 <2]
urls:
    - https://my/conan/remote/repo/url
```

Like in the `remotes.json` file, the `urls` in the `conanconfig.yml` file can also add the `verify_ssl` specifier to disable SSL verification, with the same behavior as the command line argument `--insecure`:

Listing 3: conanconfig.yml

```
packages:
    - myconf/0.1
urls:
    - url: https://some.server.com
      verify_ssl: false
```

Important: When installing more than 1 configuration package, the order of installation is important, as the later installed packages can overwrite configuration files installed by the previous ones. Consequently, if you decide to make the configuration part of the packages `package_id` via `core.package_id:config_mode` conf, the order is taken into account.

Then any installation or re-installation of packages or updates that change this order will be raised as an error. For example if after installing the configuration from the `conanconfig.yml` above we try to do a `conan config install-pkg myconf_a/0.2`, that will be raised as an error, because that would make `myconf_a` to be the latest installed one, not the first.

But on the other hand, doing an update with the previous file will not be an error, because it will re-install the `myconf_a`, `myconf_b` and `myconf_c` in order. Likewise, doing an update only for `myconf_c` wouldn't be an error, because it is the last one and preserves the relative order.

Configuration packages in lockfiles

When a configuration package is stored in a lockfile, with the `--lockfile-out` argument, it will create an entry in the lockfile `config_requires` entry. This entry has different purposes:

- When installing configuration packages with `conan config install-pkg` using command line arguments or a `conanconfig.yml` file that contains version ranges, or even pinned versions, but no recipe-revision, the provided lockfile can constraint that input to force and guarantee the exact version and recipe revision for that package defined in the lockfile.
- When using a lockfile as input in regular `conan install/build/create/graph-info`, etc, it will perform a check of the installed configuration packages, and if they are not aligned with the lockfile defined `config_requires` it will raise an error. Users then can issue a `conan config install-pkg` command to install the required configuration packages so their environments align. The idea is that lockfiles `config_requires` are there to guarantee the same configuration. The check goes in both directions, configuration packages already installed in the current user cache must satisfy the lockfile constraints, and lockfile declared `config_requires` must be installed. If for any reason, this behaviour wouldn't be desired, it is possible to use a different lockfile just for the configurations, independent from the regular packages lockfiles, avoiding in this way a populated `config_requires` when using regular packages installation commands.

conan config list

```
$ conan config list -h
usage: conan config list [-h] [-f FORMAT] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF]
                         [pattern]

Show all the Conan available configurations: core and tools.

positional arguments:
  pattern               Filter configuration items that matches this pattern

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Displays all the Conan built-in configurations. There are 2 groups:

- `core.xxxx`: These can only be defined in `global.conf` and are used by Conan internally
- `tools.xxxx`: These can be defined both in `global.conf` and profiles, and will be used by recipes and tools used within recipes, like `CMakeToolchain`

```
$ conan config list
core.cache:storage_path: Absolute path where the packages and database are stored
core.download:download_cache: Define path to a file download cache. Files uploaded to a␣
↪server are also stored here, so a later download of the same artifact can be skipped
core.download:parallel: Number of concurrent threads to download packages
core.download:retry:  (int, default: 2) Number of retries in case of failure when␣
↪downloading from Conan server
core.download:retry_wait: (int, default: 1s) Seconds to wait between download attempts␣
↪from Conan server
core.graph:compatibility_mode: (Experimental) Set this to 'optimized' to enable the␣
↪improved compatibility behaviour when querying multiple compatible binaries in remotes
core.gzip:compresslevel: The Gzip compression level for Conan artifacts (default=9)
core.net.http:cacert_path: Path containing a custom Cacert file
core.net.http:clean_system_proxy: If defined, the proxies system env-vars will be␣
↪discarded
core.net.http:client_cert: Path or tuple of files containing a client cert (and key)
core.net.http:max_retries: Maximum number of connection retries (requests library)
core.net.http:no_proxy_match: List of urls to skip from proxies configuration
core.net.http:proxies: Dictionary containing the proxy configuration
core.net.http:timeout: Number of seconds without response to timeout (requests library)
core.net.http:trust_store: (Experimental) If True, additionally verify HTTPS␣
↪certificates against the OS-native trust store via the 'truststore' (needs Python>=3.
↪10) package (used together with, not instead of, the certifi CA bundle).
core.package_id:config_mode: How the 'config_version' affects binaries. By default 'None'
core.package_id:default_build_mode: By default, 'None'
core.package_id:default_embed_mode: By default, 'full_mode'
core.package_id:default_non_embed_mode: By default, 'minor_mode'
core.package_id:default_python_mode: By default, 'minor_mode'
core.package_id:default_unknown_mode: By default, 'semver_mode'
core.scm:excluded: List of excluded patterns for builtin git dirty checks
core.scm:local_url: By default allows to store local folders as remote url, but not␣
↪upload them. Use 'allow' for allowing upload and 'block' to completely forbid it
core.sources.patch:extra_path: Extra path to search for patch files for conan create
core.sources:download_cache: Folder to store the sources backup
core.sources:download_urls: List of URLs to download backup sources from
core.sources:exclude_urls: URLs which will not be backed up
core.sources:upload_url: Remote URL to upload backup sources to
core.upload:compression_format: The compression format used when uploading Conan␣
↪packages. Possible values: 'zst', 'xz', 'gz' (default=gz)
core.upload:parallel: Number of concurrent threads to upload packages
core.upload:retry: (int, default: 1) Number of retries in case of failure when uploading␣
↪to Conan server
core.upload:retry_wait: (int, default: 5s) Seconds to wait between upload attempts to␣
↪Conan server
core.version_ranges:resolve_prereleases: Whether version ranges can resolve to pre-
↪releases or not
core:allow_uppercase_pkg_names: Temporarily (will be removed in 2.X) allow uppercase␣
↪names
core:compresslevel: The compression level for Conan artifacts (default zstd=3, gz=9)
core:default_build_profile: Defines the default build profile ('default' by default)
core:default_profile: Defines the default host profile ('default' by default)
core:non_interactive: Disable interactive user input, raises error if input necessary
core:policies: A list of opt-in behaviors that can be defined in the configuration to␣
```

(continues on next page)

(continued from previous page)

```
↪control specific aspects of Conan's behavior,
such as keeping deprecated behaviours:
   - If required_conan_version>=2.28, bugfix https://github.com/conan-io/conan/pull/
↪19705 for transitive static libraries package_id
   - If required_conan_version>=2.28, bugfix https://github.com/conan-io/conan/pull/
↪19849 for VirtualBuildEnv bindir path propagation based on requirement run trait
   - If required_conan_version>=2.28, https://github.com/conan-io/conan/pull/19286␣
↪defaults the new 'consistent' trait to True for the host context, even when
↪'visible=False'
   - If required_conan_version>=2.30, bugfix https://github.com/conan-io/conan/pull/
↪20073 for propagation of the 'transitive_header' trait
core:required_conan_version: Raise if current version does not match the defined range.
core:skip_warnings: Do not show warnings matching any of the patterns in this list.␣
↪Current warning tags are 'network', 'deprecated', 'experimental'
core:update_policy: (Legacy). If equal 'legacy' when multiple remotes, update based on␣
↪order of remotes, only the timestamp of the first occurrence of each revision counts.
core:warnings_as_errors: Treat warnings matching any of the patterns in this list as␣
↪errors and then raise an exception. Current warning tags are 'network', 'deprecated'
tools.android:cmake_legacy_toolchain: Define to explicitly pass ANDROID_USE_LEGACY_
↪TOOLCHAIN_FILE in CMake toolchain
tools.android:ndk_path: Argument for the CMAKE_ANDROID_NDK
tools.apple:enable_arc: (boolean) Enable/Disable ARC Apple Clang flags
tools.apple:enable_bitcode: (boolean) Enable/Disable Bitcode Apple Clang flags
tools.apple:enable_visibility: (boolean) Enable/Disable Visibility Apple Clang flags
tools.apple:sdk_path: Path to the SDK to be used
tools.build.cross_building:can_run: (boolean) Indicates whether is possible to run a non-
↪native app on the same architecture. It's used by 'can_run' tool
tools.build.cross_building:cross_build: (boolean) Decides whether cross-building or not␣
↪regardless of arch/OS settings. Used by 'cross_building' tool
tools.build:add_rpath_link: Add -Wl,-rpath-link flags pointing to all lib directories␣
↪for host dependencies (CMake and Meson toolchains)
tools.build:asmflags: List of extra ASM flags used by CMakeToolchain
tools.build:cflags: List of extra C flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:compiler_executables: Defines a Python dict-like with the compilers path to␣
↪be used. Allowed keys {'c', 'cpp', 'cuda', 'objc', 'objcxx', 'rc', 'fortran', 'asm',
↪'hip', 'ispc'}
tools.build:cxxflags: List of extra CXX flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:defines: List of extra definition flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:download_source: Force download of sources for every package
tools.build:exelinkflags: List of extra flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:install_strip: (boolean or list) True/False to strip on install for every␣
↪CMake, Meson and Autotools integration, or a list of 'cmake', 'meson', 'autotools' to␣
↪strip only for those.
tools.build:jobs: Default compile jobs number -jX Ninja, Make, /MP VS (default: max CPUs)
tools.build:linker_scripts: List of linker script files to pass to the linker used by␣
↪different toolchains like CMakeToolchain, AutotoolsToolchain, and MesonToolchain
tools.build:rcflags: List of extra RC (resource compiler) flags used by different␣
↪toolchains like CMakeToolchain, MSBuildToolchain and MesonToolchain
```

(continues on next page)

(continued from previous page)

```
tools.build:sharedlinkflags: List of extra flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:skip_test: Do not execute CMake.test() and Meson.test() when enabled
tools.build:sysroot: Pass the --sysroot=<tools.build:sysroot> flag if available. (None␣
↪by default)
tools.build:verbosity: Verbosity of build systems if set. Possible values are 'quiet'␣
↪and 'verbose'
tools.cmake.cmake_layout:build_folder: (Experimental) Allow configuring the base folder␣
↪of the build for local builds
tools.cmake.cmake_layout:build_folder_vars: Settings and Options that will produce a␣
↪different build folder and different CMake presets names
tools.cmake.cmake_layout:test_folder: (Experimental) Allow configuring the base folder␣
↪of the build for test_package
tools.cmake.cmakedeps:new: Use the new CMakeDeps generator
tools.cmake.cmaketoolchain:enabled_blocks: Select the specific blocks to use in the␣
↪conan_toolchain.cmake
tools.cmake.cmaketoolchain:extra_variables: Dictionary with variables to be injected in␣
↪CMakeToolchain (potential override of CMakeToolchain defined variables)
tools.cmake.cmaketoolchain:find_package_prefer_config: Argument for the CMAKE_FIND_
↪PACKAGE_PREFER_CONFIG
tools.cmake.cmaketoolchain:generator: User defined CMake generator to use instead of␣
↪default
tools.cmake.cmaketoolchain:presets_environment: String to define wether to add or not␣
↪the environment section to the CMake presets. Empty by default, will generate the␣
↪environment section in CMakePresets. Can take values: 'disabled'.
tools.cmake.cmaketoolchain:system_name: Define CMAKE_SYSTEM_NAME in CMakeToolchain
tools.cmake.cmaketoolchain:system_processor: Define CMAKE_SYSTEM_PROCESSOR in␣
↪CMakeToolchain
tools.cmake.cmaketoolchain:system_version: Define CMAKE_SYSTEM_VERSION in CMakeToolchain
tools.cmake.cmaketoolchain:toolchain_file: Use other existing file rather than conan_
↪toolchain.cmake one
tools.cmake.cmaketoolchain:toolset_arch: Toolset architecture to be used as part of␣
↪CMAKE_GENERATOR_TOOLSET in CMakeToolchain
tools.cmake.cmaketoolchain:toolset_cuda: (Experimental) Path to a CUDA toolset to use,␣
↪or version if installed at the system level
tools.cmake.cmaketoolchain:user_presets: (Experimental) Select a different name or␣
↪relative filepath instead of CMakeUserPresets.json, empty to disable. A relative␣
↪filepath (with directory component) enables CMake 4.4+ --preset-file mode
tools.cmake.cmaketoolchain:user_toolchain: Inject existing user toolchains at the␣
↪beginning of conan_toolchain.cmake
tools.cmake:cmake_program: Path to CMake executable
tools.cmake:configure_args: Add extra arguments to CMake.configure() command line
tools.cmake:ctest_args: Add extra arguments to CMake.ctest() runner command line
tools.cmake:install_strip: (Deprecated) Add --strip to cmake.install(). Use tools.
↪build:install_strip instead
tools.compilation:verbosity: Verbosity of compilation tools if set. Possible values are
↪'quiet' and 'verbose'
tools.deployer:symlinks: Set to False to disable deployers copying symlinks
tools.env.virtualenv:powershell: If specified, it generates PowerShell launchers (.ps1).␣
↪Use this configuration setting the PowerShell executable you want to use (e.g.,
↪'powershell.exe' or 'pwsh')
tools.env:deactivation_mode: (Experimental) If 'function', generate a deactivate␣
```

(continues on next page)

(continued from previous page)

```
↪function instead of a script to unset the environment variables
tools.env:dotenv: (Experimental) Generate dotenv environment files
tools.files.download:retry: (int, default: 2) Number of retries in case of failure when␣
↪downloading
tools.files.download:retry_wait: (int, default: 5s) Seconds to wait between download␣
↪attempts
tools.files.download:verify: If set, overrides recipes on whether to perform SSL␣
↪verification for their downloaded files. Only recommended to be set while testing
tools.files.unzip:filter: Define tar extraction filter: 'fully_trusted', 'tar', 'data'
tools.gnu:build_triplet: Custom build triplet to pass to Autotools scripts
tools.gnu:define_libcxx11_abi: Force definition of GLIBCXX_USE_CXX11_ABI=1 for␣
↪libstdc++11
tools.gnu:disable_flags: Disable the automatic addition of flags to some build systems.␣
↪List of possible values: ['arch', 'arch_link', 'libcxx', 'build_type', 'build_type_link
↪', 'threads','cppstd', 'cstd']
tools.gnu:extra_configure_args: List of extra arguments to pass to configure when using␣
↪AutotoolsToolchain and GnuToolchain
tools.gnu:host_triplet: Custom host triplet to pass to Autotools scripts
tools.gnu:make_program: Indicate path to make program
tools.gnu:pkg_config: Path to pkg-config executable used by PkgConfig build helper
tools.google.bazel:bazelrc_path: List of paths to bazelrc files to be used as 'bazel --
↪bazelrc=rcpath1 ... build'
tools.google.bazel:configs: List of Bazel configurations to be used as 'bazel build --
↪config=config1 ...'
tools.graph:skip_binaries: Allow the graph to skip binaries not needed in the current␣
↪configuration (True by default)
tools.graph:skip_build: (Experimental) Do not expand build/tool_requires
tools.graph:skip_test: (Experimental) Do not expand test_requires. If building it might␣
↪need 'tools.build:skip_test=True'
tools.graph:vendor: (Experimental) If 'build', enables the computation of dependencies␣
↪of vendoring packages to build them
tools.info.package_id:confs: List of existing configuration to be part of the package ID
tools.intel:installation_path: Defines the Intel oneAPI installation root path
tools.intel:setvars_args: Custom arguments to be passed onto the setvars.sh|bat script␣
↪from Intel oneAPI
tools.meson.mesontoolchain:backend: Any Meson backend: ninja, vs, vs2010, vs2012, vs2013,
↪ vs2015, vs2017, vs2019, xcode
tools.meson.mesontoolchain:extra_machine_files: List of paths for any additional native/
↪cross file references to be appended to the existing Conan ones
tools.meson.mesontoolchain:extra_variables: Dict of dicts defining extra variables per␣
↪meson file section: 'properties', 'binaries', 'project_options'
tools.microsoft.bash:active: Set True only when Conan runs in a POSIX Bash (MSYS2/
↪Cygwin) where Python's subprocess (shell=True) uses a POSIX-compatible shell (e.g., /
↪bin/sh). Do not set when using Conan from cmd/PowerShell or with native Windows Python␣
↪('win32').
tools.microsoft.bash:path: The path to the shell to run when conanfile.win_bash==True
tools.microsoft.bash:subsystem: The subsystem to be used when conanfile.win_bash==True.␣
↪Possible values: msys2, msys, cygwin, wsl, sfu
tools.microsoft.msbuild:installation_path: VS install path, to avoid auto-detect via␣
↪vswhere, like C:/Program Files (x86)/Microsoft Visual Studio/2019/Community. Use empty␣
↪string to disable
tools.microsoft.msbuild:max_cpu_count: Argument for the /m when running msvc to build␣
```

(continues on next page)

(continued from previous page)

```
↪parallel projects
tools.microsoft.msbuild:vs_version: Defines the IDE version (15, 16, 17) when using the␣
↪msvc compiler. Necessary if compiler.version specifies a toolset that is not the IDE␣
↪default
tools.microsoft.msbuilddeps:exclude_code_analysis: Suppress MSBuild code analysis for␣
↪patterns
tools.microsoft.msbuildtoolchain:compile_options: Dictionary with MSBuild compiler␣
↪options
tools.microsoft:msvc_update: Force the specific update irrespective of compiler.update␣
↪(CMakeToolchain and VCVars)
tools.microsoft:winsdk_version: Use this winsdk_version in vcvars
tools.system.package_manager:mode: Mode for package_manager tools: 'check', 'report',
↪'report-installed' or 'install'
tools.system.package_manager:sudo: Use 'sudo' when invoking the package manager tools in␣
↪Linux (False by default)
tools.system.package_manager:sudo_askpass: Use the '-A' argument if using sudo in Linux␣
↪to invoke the system package manager (False by default)
tools.system.package_manager:tool: Default package manager tool: 'apk', 'apt-get', 'yum',
↪ 'dnf', 'brew', 'pacman', 'choco', 'zypper', 'pkg' or 'pkgutil'
tools.system.pipenv:python_interpreter: (Deprecated) Use 'tools.system.pyenv:python_
↪interpreter' instead. Path to the Python interpreter to be used to create the␣
↪virtualenv
tools.system.pyenv:python_interpreter: (Experimental) Path to the Python interpreter to␣
↪be used to create the virtualenv
```

It is possible to list only the configurations that match a given pattern, like:

```
$  conan config list proxy
core.net.http:clean_system_proxy: If defined, the proxies system env-vars will be␣
↪discarded
core.net.http:no_proxy_match: List of urls to skip from proxies configuration
core.net.http:proxies: Dictionary containing the proxy configuration
```

See also:

- These configurations can be defined in `global.conf`, profile files and command line, see *Conan configuration files*

conan config show

```
$ conan config show -h
usage: conan config show [-h] [-f FORMAT] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF]
                         pattern

Get the value of the specified conf

positional arguments:
  pattern               Conf item(s) pattern for which to query their value
```

(continues on next page)

(continued from previous page)

```
options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Shows the values of the conf items that match the given pattern.

For a *global.conf* consisting of

```
tools.build:jobs=42
tools.files.download:retry_wait=10
tools.files.download:retry=7
core.net.http:timeout=30
core.net.http:max_retries=5
zlib*/:tools.files.download:retry_wait=100
zlib*/:tools.files.download:retry=5
```

You can get all the values:

```
$ conan config show "*"

core.net.http:max_retries: 5
core.net.http:timeout: 30
tools.files.download:retry: 7
tools.files.download:retry_wait: 10
tools.build:jobs: 42
zlib*/:tools.files.download:retry: 5
zlib*/:tools.files.download:retry_wait: 100
```

Or just those referring to the `tools.files` section:

```
$ conan config show "*tools.files*"

tools.files.download:retry: 7
tools.files.download:retry_wait: 10
zlib*/:tools.files.download:retry: 5
zlib*/:tools.files.download:retry_wait: 100
```

Notice the first * in the pattern. This will match all the package patterns. Removing it will make the command only show global confs:

```
$ conan config show "tools.files*"

```

(continues on next page)

(continued from previous page)

```
tools.files.download:retry: 7
tools.files.download:retry_wait: 10
```

conan config clean

```
$ conan config clean -h
usage: conan config clean [-h] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF]

(Experimental) Clean the configuration files in the Conan home folder, while
keeping installed packages

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Removes all the custom configuration from the Conan home, such as `remotes.json`, profiles, settings, plugins, extensions, etc. This does not remove packages, only the configuration files.

9.2.3 conan graph

The `conan graph` command contains several subcommands that return information of a dependency graph without needing to download the package binaries.

conan graph info

```
$ conan graph info -h
usage: conan graph info [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                        [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                        [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL] [-o OPTIONS]
                        [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                        [-o:a OPTIONS_ALL] [-s SETTINGS] [-s:b SETTINGS_BUILD]
                        [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL] [-c CONF]
                        [-c:b CONF_BUILD] [-c:h CONF_HOST] [-c:a CONF_ALL]
                        [--requires REQUIRES] [--tool-requires TOOL_REQUIRES]
```

(continues on next page)

(continued from previous page)

```
                        [--name NAME] [--version VERSION] [--user USER]
                        [--channel CHANNEL] [-l LOCKFILE] [--lockfile-partial]
                        [--lockfile-out LOCKFILE_OUT] [--lockfile-clean]
                        [--lockfile-overrides LOCKFILE_OVERRIDES]
                        [--check-updates] [--filter FILTER]
                        [--package-filter PACKAGE_FILTER] [-d DEPLOYER]
                        [-df DEPLOYER_FOLDER] [--build-require]
                        [path]

Compute the dependency graph and show information about it.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: html, json, dot
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
```

(continues on next page)

(continued from previous page)

```
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --check-updates       Check if there are recipe updates
  --filter FILTER       Show only the specified fields
  --package-filter PACKAGE_FILTER
                        Print information only for packages that match the
                        patterns
  -d DEPLOYER, --deployer DEPLOYER
                        Deploy using the provided deployer to the output
                        folder. Built-in deployers: 'full_deploy',
                        'direct_deploy'. Deployers will only deploy recipes,
                        as 'conan graph info' do not retrieve binaries
  -df DEPLOYER_FOLDER, --deployer-folder DEPLOYER_FOLDER
                        Deployer output folder, base build folder by default
                        if not set
  --build-require       Whether the provided reference is a build-require

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
```

(continues on next page)

(continued from previous page)

```
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan graph info` command shows information about the dependency graph for the recipe specified in `path`.

Examples:

```
$ conan graph info .
$ conan graph info myproject_folder
$ conan graph info myproject_folder/conanfile.py
$ conan graph info --requires=hello/1.0@user/channel
```

The output will look like:

```
$ conan graph info --require=binutils/2.38 -r=conancenter

...

======== Basic graph information ========
conanfile:
  ref: conanfile
  id: 0
  recipe: Cli
  package_id: None
  prev: None
  build_id: None
  binary: None
  invalid_build: False
  info_invalid: None
  revision_mode: hash
  package_type: unknown
  settings:
    os: Macos
    arch: armv8
    compiler: apple-clang
    compiler.cppstd: gnu17
    compiler.libcxx: libc++
    compiler.version: 14
    build_type: Release
  options:
  system_requires:
  recipe_folder: None
  source_folder: None
  build_folder: None
  generators_folder: None
  package_folder: None
  cpp_info:
    root:
      includedirs: ['include']
      srcdirs: None
      libdirs: ['lib']
      resdirs: None
      bindirs: ['bin']
      builddirs: None
      frameworkdirs: None
      system_libs: None
      frameworks: None
      libs: None
      defines: None
      cflags: None
      cxxflags: None
      sharedlinkflags: None
      exelinkflags: None
      objects: None
      sysroot: None
```

(continues on next page)

(continued from previous page)

```
      requires: None
      properties: None
  label: cli
  context: host
  test: False
  requires:
    1: binutils/2.38#0dc90586530d3e194d01d17cb70d9461
binutils/2.38#0dc90586530d3e194d01d17cb70d9461:
  ref: binutils/2.38#0dc90586530d3e194d01d17cb70d9461
  id: 1
  recipe: Downloaded
  package_id: 5350e016ee8d04f418b50b7be75f5d8be9d79547
  prev: None
  build_id: None
  binary: Invalid
  invalid_build: False
  info_invalid: cci does not support building binutils for Macos since binutils is␣
↪degraded there (no as/ld + armv8 does not build)
  url: https://github.com/conan-io/conan-center-index/
  license: GPL-2.0-or-later
  description: The GNU Binutils are a collection of binary tools.
  topics: ('gnu', 'ld', 'linker', 'as', 'assembler', 'objcopy', 'objdump')
  homepage: https://www.gnu.org/software/binutils
  revision_mode: hash
  package_type: application
  settings:
    os: Macos
    arch: armv8
    compiler: apple-clang
    compiler.version: 14
    build_type: Release
  options:
    multilib: True
    prefix: aarch64-apple-darwin-
    target_arch: armv8
    target_os: Macos
    target_triplet: aarch64-apple-darwin
    with_libquadmath: True
  system_requires:
  recipe_folder: /Users/barbarian/.conan2/p/binut53bd9b3ee9490/e
  source_folder: None
  build_folder: None
  generators_folder: None
  package_folder: None
  cpp_info:
    root:
      includedirs: ['include']
      srcdirs: None
      libdirs: ['lib']
      resdirs: None
      bindirs: ['bin']
      builddirs: None
```

(continues on next page)

(continued from previous page)

```
      frameworkdirs: None
      system_libs: None
      frameworks: None
      libs: None
      defines: None
      cflags: None
      cxxflags: None
      sharedlinkflags: None
      exelinkflags: None
      objects: None
      sysroot: None
      requires: None
      properties: None
  label: binutils/2.38
  context: host
  test: False
  requires:
    2: zlib/1.2.13#416618fa04d433c6bd94279ed2e93638
zlib/1.2.13#416618fa04d433c6bd94279ed2e93638:
  ref: zlib/1.2.13#416618fa04d433c6bd94279ed2e93638
  id: 2
  recipe: Cache
  package_id: 76f7d863f21b130b4e6527af3b1d430f7f8edbea
  prev: 866f53e31e2d9b04d49d0bb18606e88e
  build_id: None
  binary: Skip
  invalid_build: False
  info_invalid: None
  url: https://github.com/conan-io/conan-center-index
  license: Zlib
  description: A Massively Spiffy Yet Delicately Unobtrusive Compression Library (Also␣
↪Free, Not to Mention Unencumbered by Patents)
  topics: ('zlib', 'compression')
  homepage: https://zlib.net
  revision_mode: hash
  package_type: static-library
  settings:
    os: Macos
    arch: armv8
    compiler: apple-clang
    compiler.version: 14
    build_type: Release
  options:
    fPIC: True
    shared: False
  system_requires:
  recipe_folder: /Users/barbarian/.conan2/p/zlibbcf9063fcc882/e
  source_folder: None
  build_folder: None
  generators_folder: None
  package_folder: None
  cpp_info:
```

(continues on next page)

(continued from previous page)

```
    root:
      includedirs: ['include']
      srcdirs: None
      libdirs: ['lib']
      resdirs: None
      bindirs: ['bin']
      builddirs: None
      frameworkdirs: None
      system_libs: None
      frameworks: None
      libs: None
      defines: None
      cflags: None
      cxxflags: None
      sharedlinkflags: None
      exelinkflags: None
      objects: None
      sysroot: None
      requires: None
      properties: None
  label: zlib/1.2.13
  context: host
  test: False
  requires:
```

conan graph info builds the complete dependency graph, like **conan install** does. The main difference is that it doesn't try to install or build the binaries, but the package recipes will be retrieved from remotes if necessary.

It is very important to note that the **conan graph info** command outputs the dependency graph for a given configuration (settings, options), as the dependency graph can be different for different configurations. This means that the input to the **conan graph info** command is the same as **conan install**, the configuration can be specified directly with settings and options, or using profiles,and querying the graph of a specific recipe is possible by using the `--requires` flag as shown above.

You can additionally filter the output, both by filtering by fields (`--filter`) and by package (`--filter-package`). For example, to get the options of zlib, the following command could be run:

```
$ conan graph info --require=binutils/2.38 -r=conancenter --filter=options --package-
↪filter="zlib*"

...

======== Basic graph information ========
zlib/1.2.13#13c96f538b52e1600c40b88994de240f:
  ref: zlib/1.2.13#13c96f538b52e1600c40b88994de240f
  options:
    fPIC: True
    shared: False
```

The `--package-filter` accepts the `&` placeholder as `--package-filter="&"` to refer to the current "consumer" recipes, without needing to explicitly type its package name.

Available formatters

json formatter

For the documentation about the JSON formatter, please check the *dedicated section*.

dot formatter

To use the DOT format, execute the following command:

Listing 4: binutils/2.38 graph info DOT representation

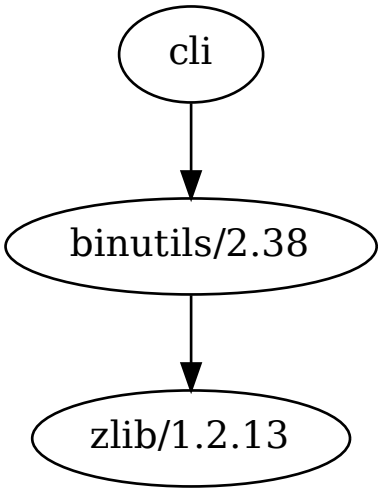
```
$ conan graph info --require=binutils/2.38 -r=conancenter --format=dot > graph.dot
```

This command generates a DOT file with the following content:

Listing 5: Contents of graph.dot

```
digraph {
    "cli" -> "binutils/2.38"
    "binutils/2.38" -> "zlib/1.2.13"
}
```

To visualize this graph, you can render it using Graphviz or any compatible tool.

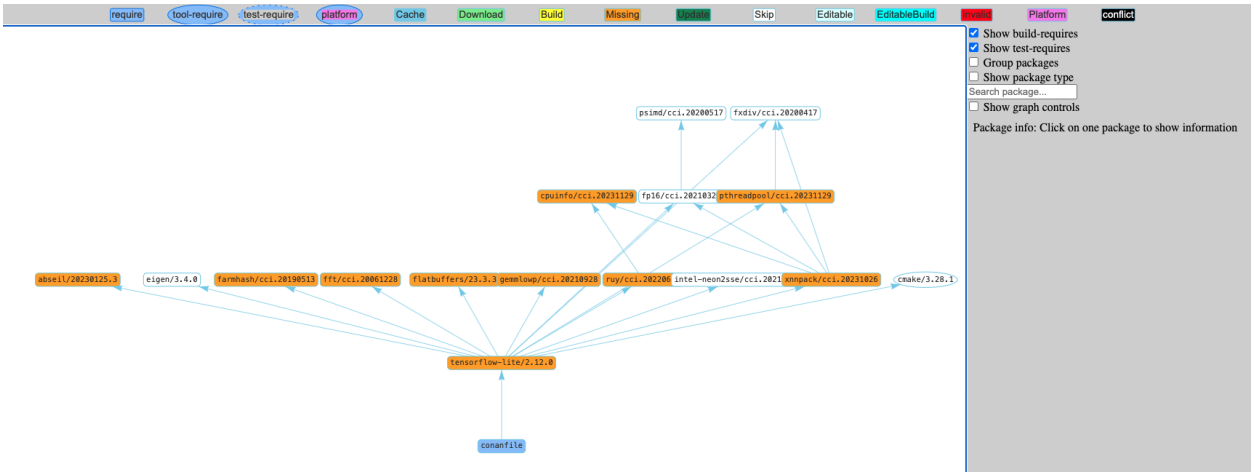


html formatter

The HTML formatter provides a visual representation of the dependency graph that is both interactive and user-friendly.

```
$ conan graph info --require=tensorflow-lite/2.12.0 -r=conancenter --format=html > graph.
↪html
```

The HTML output displays an interactive graph of your project's dependencies, featuring nodes for packages with versions, directional arrows for dependencies, and color-coded labels for dependency types. You can interact with the graph to filter visibility of dependencies and access package details and status.



Note: When using `format=html`, the generated HTML contains links to a third-party resource: the vis-network library trough the *vis-network.min.js* file. By default, this file is retrieved from Cloudflare. However, for environments without an internet connection, you will need to create a template for the file and place it in `CONAN_HOME/templates/graph.html` to point to a local version of the remote vis-network.min.js file

Use the template located in `<conan_sources>/conan/cli/formatters/graph/info_graph_html.py` as a starting point for your own.

See also:

- Check the *JSON format output* for this command.

conan graph build-order

```
$ conan graph build-order -h
usage: conan graph build-order [-h] [-f FORMAT] [--out-file OUT_FILE]
                               [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                               [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                               [-u [UPDATE]] [-pr PROFILE]
                               [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                               [-pr:a PROFILE_ALL] [-o OPTIONS]
                               [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                               [-o:a OPTIONS_ALL] [-s SETTINGS]
                               [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                               [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
```

(continues on next page)

(continued from previous page)

```
                              [-c:h CONF_HOST] [-c:a CONF_ALL]
                              [--requires REQUIRES]
                              [--tool-requires TOOL_REQUIRES] [--name NAME]
                              [--version VERSION] [--user USER]
                              [--channel CHANNEL] [-l LOCKFILE]
                              [--lockfile-partial]
                              [--lockfile-out LOCKFILE_OUT]
                              [--lockfile-clean]
                              [--lockfile-overrides LOCKFILE_OVERRIDES]
                              [--order-by {recipe,configuration}] [--reduce]
                              [path]

Compute the build order of a dependency graph.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
```

(continues on next page)

(continued from previous page)

```
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --order-by {recipe,configuration}
                        Select how to order the output, "recipe" by default if
                        not set.
  --reduce              Reduce the build order, output only those to build.
                        Use this only if the result will not be merged later
                        with other build-order

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
```

(continues on next page)

(continued from previous page)

```
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan graph build-order` command computes the build order of the dependency graph for the recipe specified in `path` or in `--requires/--tool-requires`.

There are 2 important arguments that affect how this build order is computed:

- The `--order-by` argument can take 2 values `recipe` and `configuration`, depending how we want to structure and parallelize our CI.
- The `--reduce` argument will strip all packages in the order that doesn't need to be built from source.

By default, the `conan graph build-order` will return the order for the full dependency graph, and it will annotate in each element what needs to be done, for example `"binary": "Cache"` if the binary is already in the Conan Cache and it doesn't need to be built from source, and `"binary": "Build"`, if it needs to be built from source. Having the full order is necessary if we want to `conan graph build-order-merge` several build-orders into a single one later, because having the full information allows to preserve the relative order that would otherwise be lost and broken. Consequently, the `--reduce` argument should only be used when we are directly going to use the result to do the build,

but not if we plan to later do a merge of the resulting build-order with other ones.

Let's consider installing *libpng* and wanting to see the build order for this requirement ordered by recipe:

Warning: Please be aware that starting with Conan 2.1.0, using the *–order-by* argument is recommended, and its absence is deprecated. This argument will be removed in the near future. It is maintained for backward compatibility. Note that the JSON output will differ if you use the *–order-by* argument, changing from a simple list to a dictionary with extended information.

```
$ conan graph build-order --requires=libpng/1.5.30 --format=json --order-by=recipe
...
======== Computing the build order ========
{
    "order_by": "recipe",
    "reduced": false,
    "order": [
        [
            {
                "ref": "zlib/1.3#06023034579559bb64357db3a53f88a4",
                "depends": [],
                "packages": [
                    [
                        {
                            "package_id": "d62dff20d86436b9c58ddc0162499d197be9de1e",
                            "prev": "54b9c3efd9ddd25eb6a8cbf01860b499",
                            "context": "host",
                            "binary": "Cache",
                            "options": [],
                            "filenames": [],
                            "depends": [],
                            "overrides": {},
                            "build_args": null
                        }
                    ]
                ]
            }
        ],
        [
            {
                "ref": "libpng/1.5.30#ed8593b3f837c6c9aa766f231c917a5b",
                "depends": [
                    "zlib/1.3#06023034579559bb64357db3a53f88a4"
                ],
                "packages": [
                    [
                        {
                            "package_id": "60778dfa43503cdcda3636d15124c19bf6546ae3",
                            "prev": "ad092d2e4aebcd9d48a5b1f3fd51ba9a",
                            "context": "host",
                            "binary": "Download",
                            "options": [],
                            "filenames": [],
```

(continues on next page)

(continued from previous page)

```
                            "depends": [],
                            "overrides": {},
                            "build_args": null
                        }
                    ]
                ]
            }
        ]
    ],
    "profiles": {
        "self": {
            "args": ""
        }
    }
}
```

Firstly, we can see the `zlib` package, as `libpng` depends on it. The output is sorted by recipes as we passed with the *–order-by* argument; however, we might prefer to see it sorted by configurations instead. For that purpose use the *–order-by* argument with value *configuration*.

At the end of the json, after the `order` field, we see a `profiles` field, which contains the profile related command line arguments for the current "build-order". As in this case we didn't provide any arguments, it is empty. But if we used something like `conan graph build-order ... -pr=default -s build_type=Debug > bo.json`, the `args` will contain those arguments (with json character escaping): `"args": "-pr:h=\"default\" -s:h=\"build_type=Debug\""`

Using `--order-by=configuration` we will get a different build-order format:

```
$ conan graph build-order --requires=libpng/1.5.30 --format=json --order-by=configuration
...
======== Computing the build order ========
{
    "order_by": "configuration",
    "reduced": false,
    "order": [
        [
            {
                "ref": "zlib/1.3#06023034579559bb64357db3a53f88a4",
                "pref": "zlib/1.3
↪#06023034579559bb64357db3a53f88a4:d62dff20d86436b9c58ddc0162499d197be9de1e
↪#54b9c3efd9ddd25eb6a8cbf01860b499",
                "package_id": "d62dff20d86436b9c58ddc0162499d197be9de1e",
                "prev": "54b9c3efd9ddd25eb6a8cbf01860b499",
                "context": "host",
                "binary": "Cache",
                "options": [],
                "filenames": [],
                "depends": [],
                "overrides": {},
                "build_args": null
            }
        ],
        [
```

(continues on next page)

(continued from previous page)

```
            {
                "ref": "libpng/1.5.30#ed8593b3f837c6c9aa766f231c917a5b",
                "pref": "libpng/1.5.30
↪#ed8593b3f837c6c9aa766f231c917a5b:60778dfa43503cdcda3636d15124c19bf6546ae3
↪#ad092d2e4aebcd9d48a5b1f3fd51ba9a",
                "package_id": "60778dfa43503cdcda3636d15124c19bf6546ae3",
                "prev": "ad092d2e4aebcd9d48a5b1f3fd51ba9a",
                "context": "host",
                "binary": "Download",
                "options": [],
                "filenames": [],
                "depends": [
                    "zlib/1.3
↪#06023034579559bb64357db3a53f88a4:d62dff20d86436b9c58ddc0162499d197be9de1e
↪#54b9c3efd9ddd25eb6a8cbf01860b499"
                ],
                "overrides": {},
                "build_args": null
            }
        ]
    ]
}
```

If we now apply the `--reduce`:

```
$ conan graph build-order --requires=libpng/1.5.30 --reduce --format=json --order-
↪by=configuration
...
======== Computing the build order ========
{
    "order_by": "configuration",
    "reduced": false,
    "order": []
}
```

As there are no binaries to build here, all binaries already exist. If we explicitly force to build some, the result would be only those that are going to be built:

```
$ conan graph build-order --requires=libpng/1.5.30 --build="libpng/*" --reduce --
↪format=json --order-by=configuration
...
======== Computing the build order ========
{
    "order_by": "configuration",
    "reduced": false,
    "order": [
        [
            {
                "ref": "libpng/1.5.30#ed8593b3f837c6c9aa766f231c917a5b",
                "pref": "libpng/1.5.30
↪#ed8593b3f837c6c9aa766f231c917a5b:60778dfa43503cdcda3636d15124c19bf6546ae3
↪#ad092d2e4aebcd9d48a5b1f3fd51ba9a",
```

(continues on next page)

(continued from previous page)

```
                "package_id": "60778dfa43503cdcda3636d15124c19bf6546ae3",
                "prev": null,
                "context": "host",
                "binary": "Build",
                "options": [],
                "filenames": [],
                "depends": [],
                "overrides": {},
                "build_args": "--require=libpng/1.5.30 --build=libpng/1.5.30"
            }
        ]
    ]
}
```

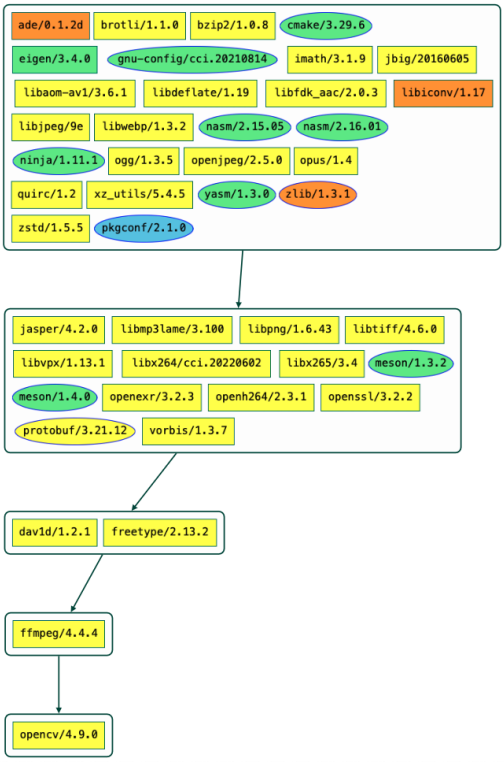
Then it will contain exclusively the `binary=Build` nodes, but not the rest. Note that it will also provide a `build_args` field with the arguments needed for a `conan install <args>` to fire the build of this package in the CI agent.

Getting a visual representation of the Build Order

You can obtain a visual representation of the build order by using the HTML formatter. For example:

```
$ conan graph build-order --requires=opencv/4.9.0 --order-by=recipe --build=missing --
↪format=html > build-order.html
```





conan graph build-order-merge

```
$ conan graph build-order-merge -h
usage: conan graph build-order-merge [-h] [-f FORMAT] [--out-file OUT_FILE]
                                     [-v [{quiet,error,warning,notice,status,verbose,
↪debug,v,trace,vv}]]
                                     [-cc CORE_CONF] [--file [FILE]]
                                     [--reduce]

Merge more than 1 build-order file.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --file [FILE]         Files to be merged
  --reduce              Reduce the build order, output only those to build.
                        Use this only if the result will not be merged later
                        with other build-order
```

As described in the `conan graph build-order` command, there are 2 types of order `recipe` and `configuration`. Only build-orders of the same type can be merged together, otherwise the command will return an error.

Note that only build-orders that haven't been reduced with `--reduce` can be merged.

The result of merging the different input files can be also reduced with the `conan graph build-order-merge --reduce` argument, and the behavior will be the same, leave only the elements that need to be built from source.

When 2 or more "build-order" files are merged, the resulting merge contains a `profiles` section like:

```
"profiles": {
    "build_order_win": {
        "args": "-pr:h=\"profile1\" -s:h=\"os=Windows\" ..."
    },
    "build_order_nix": {
        "args": "-pr:h=\"profile2\" -s:h=\"os=Linux\" ..."
    }
}
```

With the `build_order_win` and `build_order_nix` being the "build-order" filenames that were used as inputs to the merge, and which will be referenced in the `filenames` field of every `package` in the build order. This way, it is easier to obtain the necessary command line arguments to build a specific package binary in the build-order when building multiple configurations.

Note that when a merged build order containing multilpe `filenames` something like:

```
{
    "package_id": "efa83b160a55b033c4ea706ddb980cd708e3ba1b",
    "context": "build",
    "binary": "Build",
    "filenames": [
        "build_order_win",
        "build_order_nix"
    ],
    "build_args": "--tool-requires=dep/0.1 --build=dep/0.1",
    "info": {
        "settings": {
            "build_type": "Release"
        }
    }
}

"profiles": {
    "build_order_win": {
        "args": "-pr:h=\"profile1\" -s:h=\"os=Windows\" ..."
    },
    "build_order_nix": {
        "args": "-pr:h=\"profile2\" -s:h=\"os=Linux\" ..."
    }
}
```

Then, the `filename` to be used is the first one, in this case `build_order_win`, because the `context` and `build_args` arguments matches this profile information. The other filenames are provided as a reference to which other individual build-order files had this `package_id` listed for build.

conan graph explain

```
$ conan graph explain -h
usage: conan graph explain [-h] [-f FORMAT] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                           [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                           [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL]
                           [-o OPTIONS] [-o:b OPTIONS_BUILD]
                           [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL]
                           [-s SETTINGS] [-s:b SETTINGS_BUILD]
                           [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL] [-c CONF]
                           [-c:b CONF_BUILD] [-c:h CONF_HOST] [-c:a CONF_ALL]
                           [--requires REQUIRES]
                           [--tool-requires TOOL_REQUIRES] [--name NAME]
                           [--version VERSION] [--user USER]
                           [--channel CHANNEL] [-l LOCKFILE]
                           [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                           [--lockfile-clean]
                           [--lockfile-overrides LOCKFILE_OVERRIDES]
                           [--check-updates] [--build-require]
                           [--missing [MISSING]]
```

(continues on next page)

(continued from previous page)

```
                          [path]

Explain what is wrong with the dependency graph, like report missing binaries
closest alternatives, trying to explain why the existing binaries do not match

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
```

(continues on next page)

(continued from previous page)

```
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --check-updates       Check if there are recipe updates
  --build-require       Whether the provided reference is a build-require
  --missing [MISSING]   A pattern in the form
                        'pkg/version#revision:package_id#revision', e.g:
                        "zlib/1.2.13:*" means all binaries for zlib/1.2.13. If
                        revision is not specified, it is assumed latest one.

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
```

(continues on next page)

(continued from previous page)

```
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan graph explain` tries to give a more detailed explanation for a package that might be missing with the configuration provided and show the differences between the expected binary package and the available ones. It helps to understand what is missing from the package requested, whether it is different options, different settings or different dependencies.

Example:

Imagine that we want to install the *lib/1.0.0* that depends on *dep/2.0.0* but we don't have a binary yet, as the latest CI run only generated a binary for lib/1.0.0 using the previous version of *dep*. When we try to install the refere *lib/1.0.0* it says:

```
$ conan install --requires=lib/1.0.0
...
ERROR: Missing prebuilt package for 'lib/1.0.0'
```

Now we can try to find a explanation for this:

```
$ conan graph explain --requires=lib/1.0.0
requires: dep/1.Y.Z
diff
  dependencies
    expected: dep/2.Y.Z
    existing: dep/1.Y.Z
    explanation: This binary has same settings and options, but different dependencies
```

In the same way, it can report when a package has a different option value and the output is also available in JSON format:

```
$conan graph explain --requires=lib/1.0.0 -o lib/*:shared=True --format=json
...
{
    "closest_binaries": {
        "lib/1.0.0": {
            "revisions": {
                "dc0e384f0551386cd76dc29cc964c95e": {
                    "timestamp": 1692672717.68,
                    "packages": {
                        "b647c43bfefae3f830561ca202b6cfd935b56205": {
                            "info": {
                                "settings": {
                                    "arch": "x86_64",
                                    "build_type": "Release",
                                    "compiler": "gcc",
                                    "compiler.version": "11",
                                    "os": "Linux"
                                },
                                "options": {
                                    "shared": "False"
                                }
                            },
                            "diff": {
                                "platform": {},
                                "options": {
                                    "expected": [
                                        "shared=True"
                                    ],
                                    "existing": [
                                        "shared=False"
                                    ]
                                },
                                "settings": {},
                                "dependencies": {},
                                "explanation": "This binary was built with same settings␣
↪but different options."
                            },
                            "remote": "conancenter"
                        }
                    }
                }
            }
        }
    }
}
```

conan graph outdated

```
$ conan graph outdated -h
usage: conan graph outdated [-h] [-f FORMAT] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                            [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                            [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL]
                            [-o OPTIONS] [-o:b OPTIONS_BUILD]
                            [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL]
                            [-s SETTINGS] [-s:b SETTINGS_BUILD]
                            [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL] [-c CONF]
                            [-c:b CONF_BUILD] [-c:h CONF_HOST] [-c:a CONF_ALL]
                            [--requires REQUIRES]
                            [--tool-requires TOOL_REQUIRES] [--name NAME]
                            [--version VERSION] [--user USER]
                            [--channel CHANNEL] [-l LOCKFILE]
                            [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                            [--lockfile-clean]
                            [--lockfile-overrides LOCKFILE_OVERRIDES]
                            [--check-updates] [--build-require]
                            [path]

List the dependencies in the graph and it's newer versions in the remote

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
```

(continues on next page)

(continued from previous page)

```
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --check-updates       Check if there are recipe updates
  --build-require       Whether the provided reference is a build-require

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
```

(continues on next page)

(continued from previous page)

```
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan graph outdated` command provides details on libraries for which a newer version is available in a remote repository. This command helps users in identifying outdated libraries by displaying the latest version available in the remote repository and indicating which specific remote repository it was found in. Additionally, it presents information on the versions currently stored in the local cache and specifies the version ranges for each library.

It will display the information for every library on the dependency graph it is run on. For example if running the command with an older version of `libcurl` it will display:

```
$ conan graph outdated --requires=libcurl/[*]
```

```
======== Computing dependency graph ========
Graph root
    cli
Requirements
    libcurl/8.5.0#95279f20d2443016907657f081a79261 - Cache
    openssl/3.2.1#edbeabd3bfc383d2cca3858aa2a78a0d - Cache
    zlib/1.3.1#f52e03ae3d251dec704634230cd806a2 - Cache
Build requirements
    nasm/2.15.05#058c93b2214a49ca1cfe9f8f26205568 - Cache
    strawberryperl/5.32.1.1#8f83d05a60363a422f9033e52d106b47 - Cache
Resolved version ranges
    libcurl/[*]: libcurl/8.5.0
    openssl/[>=1.1 <4]: openssl/3.2.1
    zlib/[>=1.2.11 <2]: zlib/1.3.1

======== Checking remotes ========
Found 35 pkg/version recipes matching libcurl in conancenter
Found 46 pkg/version recipes matching openssl in conancenter
Found 6 pkg/version recipes matching zlib in conancenter
Found 5 pkg/version recipes matching nasm in conancenter
Found 3 pkg/version recipes matching strawberryperl in conancenter
======== Outdated dependencies ========
libcurl
    Current versions:  libcurl/8.5.0
    Latest in remote(s):  libcurl/8.6.0 - conancenter
    Version ranges: libcurl/[*]
nasm
    Current versions:  nasm/2.15.05
    Latest in remote(s):  nasm/2.16.01 - conancenter
```

9.2.4 conan inspect

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan inspect -h
usage: conan inspect [-h] [-f FORMAT] [--out-file OUT_FILE]
                     [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                     [-cc CORE_CONF] [-r REMOTE | -nr] [-l LOCKFILE]
                     [--lockfile-partial]
                     [path]

Inspect a conanfile.py to return its public fields.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py).
                        Defaults to current directory
```

(continues on next page)

(continued from previous page)

```
options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -r REMOTE, --remote REMOTE
                        Remote names. Accepts wildcards ('*' means all the
                        remotes available)
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
```

The **conan inspect** command shows the public attributes of any recipe (*conanfile.py*) as follows:

```
$ conan inspect .
default_options:
    shared: False
    fPIC: True
    neon: True
    msa: True
    sse: True
    vsx: True
    api_prefix:
description: libpng is the official PNG file format reference library.
generators: []
homepage: http://www.libpng.org
label:
license: libpng-2.0
name: libpng
options:
    api_prefix:
    fPIC: True
    msa: True
    neon: True
    shared: False
    sse: True
    vsx: True
options_definitions:
    shared: ['True', 'False']
```

(continues on next page)

(continued from previous page)

```
    fPIC: ['True', 'False']
    neon: ['True', 'check', 'False']
    msa: ['True', 'False']
    sse: ['True', 'False']
    vsx: ['True', 'False']
    api_prefix: ['ANY']
package_type: None
requires: []
revision_mode: hash
settings: ['os', 'arch', 'compiler', 'build_type']
topics: ['png', 'graphics', 'image']
url: https://github.com/conan-io/conan-center-index
```

`conan inspect` evaluates recipe methods such as `set_name()` and `set_version()`, and is capable of resolving `python_requires` dependencies (which can be locked with the `--lockfile` argument), so its base methods will also be properly executed.

Note: The `--remote` argument is used *only* for fetching remote `python_requires` in cases where they are needed, **not** to inspect recipes from a remote. Use *conan graph info* for such cases.

The **`conan inspect ... --format=json`** returns a JSON output format in `stdout` (which can be redirected to a file) with the following structure:

```
$ conan inspect . --format=json
{
    "name": "libpng",
    "url": "https://github.com/conan-io/conan-center-index",
    "license": "libpng-2.0",
    "description": "libpng is the official PNG file format reference library.",
    "homepage": "http://www.libpng.org",
    "revision_mode": "hash",
    "default_options": {
        "shared": false,
        "fPIC": true,
        "neon": true,
        "msa": true,
        "sse": true,
        "vsx": true,
        "api_prefix": ""
    },
    "topics": [
        "png",
        "graphics",
        "image"
    ],
    "package_type": "None",
    "settings": [
        "os",
        "arch",
        "compiler",
        "build_type"
```

(continues on next page)

(continued from previous page)


```
    ],
    "options": {
        "api_prefix": "",
        "fPIC": "True",
        "msa": "True",
        "neon": "True",
        "shared": "False",
        "sse": "True",
        "vsx": "True"
    },
    "options_definitions": {
        "shared": [
            "True",
            "False"
        ],
        "fPIC": [
            "True",
            "False"
        ],
        "neon": [
            "True",
            "check",
            "False"
        ],
        "msa": [
            "True",
            "False"
        ],
        "sse": [
            "True",
            "False"
        ],
        "vsx": [
            "True",
            "False"
        ],
        "api_prefix": [
            "ANY"
        ]
    },
    "generators": [],
    "requires": [],
    "source_folder": null,
    "build_folder": null,
    "generators_folder": null,
    "package_folder": null,
    "label": ""
}
```


Note: `conan inspect` does not list any requirements listed in the `requirements()` method, only those present in the `requires` attribute will be shown.

9.2.5 conan install

```
$ conan install -h
usage: conan install [-h] [-f FORMAT] [--out-file OUT_FILE]
                     [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                     [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                     [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                     [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL] [-o OPTIONS]
                     [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                     [-o:a OPTIONS_ALL] [-s SETTINGS] [-s:b SETTINGS_BUILD]
                     [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL] [-c CONF]
                     [-c:b CONF_BUILD] [-c:h CONF_HOST] [-c:a CONF_ALL]
                     [--requires REQUIRES] [--tool-requires TOOL_REQUIRES]
                     [--name NAME] [--version VERSION] [--user USER]
                     [--channel CHANNEL] [-l LOCKFILE] [--lockfile-partial]
                     [--lockfile-out LOCKFILE_OUT] [--lockfile-clean]
                     [--lockfile-overrides LOCKFILE_OVERRIDES] [-g GENERATOR]
                     [-of OUTPUT_FOLDER] [-d DEPLOYER]
                     [--deployer-folder DEPLOYER_FOLDER]
                     [--deployer-package DEPLOYER_PACKAGE] [--build-require]
                     [--envs-generation {false}]
                     [path]

Install the requirements specified in a recipe (conanfile.py or conanfile.txt).

It can also be used to install packages without a conanfile, using the
--requires and --tool-requires arguments.

If any requirement is not found in the local cache, it will iterate the remotes
looking for it. When the full dependency graph is computed, and all dependencies
recipes have been found, it will look for binary packages matching the current settings.
If no binary package is found for some or several dependencies, it will error,
unless the '--build' argument is used to build it from source.

After installation of packages, the generators and deployers will be called.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
```

(continues on next page)

(continued from previous page)

```
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  -g GENERATOR, --generator GENERATOR
                        Generators to use
  -of OUTPUT_FOLDER, --output-folder OUTPUT_FOLDER
                        The root output folder for generated and build files
  -d DEPLOYER, --deployer DEPLOYER
                        Deploy using the provided deployer to the output
                        folder. Built-in deployers: 'full_deploy',
                        'direct_deploy', 'runtime_deploy'
  --deployer-folder DEPLOYER_FOLDER
                        Deployer output folder, base build folder by default
                        if not set
  --deployer-package DEPLOYER_PACKAGE
                        Execute the deploy() method of the packages matching
                        the provided patterns
  --build-require       Whether the provided path is a build-require
  --envs-generation {false}
                        Generation strategy for virtual environment files for
                        the root

remote arguments:
```

(continues on next page)

(continued from previous page)

```
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL
```

(continues on next page)

(continued from previous page)

```
reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan install` command is one of the main Conan commands, and it is used to resolve and install dependencies.

This command does the following:

- Compute the whole dependency graph, for the current configuration defined by settings, options, profiles and configuration. It resolves version ranges, transitive dependencies, conditional requirements, etc, to build the dependency graph.
- Evaluate the existence of binaries for every package in the graph, whether or not there are precompiled binaries to download, or if they should be built from sources (as directed by the `--build` argument). If binaries are missing, it will not recompute the dependency graph to try to fallback to previous versions that contain binaries for that configuration. If a certain dependency version is desired, it should be explicitly required.
- Download precompiled binaries, or build binaries from sources in the local cache, in the right order for the dependency graph.
- Create the necessary files as requested by the "generators", so build systems and other tools can locate the locally installed dependencies
- Optionally, execute the desired `deployers`.

See also:

- Check the *JSON format output* for this command.

Conanfile path or –requires

The `conan install` command can use 2 different origins for information. The first one is using a local `conanfile.py` or `conanfile.txt`, containing definitions of the dependencies and generators to be used.

```
$ conan install .  # there is a conanfile.txt or a conanfile.py in the cwd
$ conan install conanfile.py  # also works, direct reference file
$ conan install myconan.txt  # explicit custom name
$ conan install myfolder  # there is a conanfile in "myfolder" folder
```

Even if it is possible to use a custom name, in the general case, it is recommended to use the default `conanfile.py` name, located in the repository root, so users can do a straightforward `git clone ... `` + ``conan install .`

The other possibility is to not have a `conanfile` at all, and define the requirements to be installed directly in the command line:

```
# Install the zlib/1.2.13 library
$ conan install --requires=zlib/1.2.13
# Install the zlib/1.2.13 and bzip2/1.0.8 libraries
$ conan install --requires=zlib/1.2.13 --requires=bzip2/1.0.8
# Install the cmake/3.23.5 and ninja/1.11.0 tools
$ conan install --tool-requires=cmake/3.23.5 --tool-requires=ninja/1.11.0
# Install the zlib/1.2.13 library and ninja/1.11.0 tool
$ conan install --requires=zlib/1.2.13 --tool-requires=ninja/1.11.0
```

When only `--tool-requires` arguments are provided, the `[tool_requires]` defined in the host profile are not installed, as they wouldn't be necessary to consume those installed tools.

In the general case, it is recommended to use a `conanfile` instead of defining things in the command line.

Profiles, Settings, Options, Conf

There are several arguments that are used to define the effective profiles that will be used, both for the "build" and "host" contexts.

By default the arguments refer to the "host" context, so `--settings:host, -s:h` is totally equivalent to `--settings, -s`. Also, by default, the `conan install` command will use the `default` profile both for the "build" and "host" context. That means that if a profile with the "default" name has not been created, it will error.

Multiple definitions of profiles can be passed as arguments, and they will compound from left to right (right has the highest priority)

```
# The values of myprofile3 will have higher priority
$ conan install . -pr=myprofile1 -pr=myprofile2 -pr=myprofile3
```

Note: Profiles are searched for in a variety of locations, *see here for more information*

If values for any of `settings`, `options` and `conf` are provided in the command line, they create a profile that is composed with the other provided `-pr` (or the "default" one if not specified) profiles, with higher priority, not matter what the order of arguments is.

```
# the final "host" profile will always be build_type=Debug, even if "myprofile"
# says "build_type=Release"
$ conan install . -pr=myprofile -s build_type=Debug
```

Generators and deployers

The `-g` argument allows to define in the command line the different built-in generators to be used:

```
$ conan install --requires=zlib/1.2.13 -g CMakeDeps -g CMakeToolchain
```

Note that in the general case, the recommended approach is to have the `generators` defined in the `conanfile`, and only for the `--requires` use case, it would be more necessary as command line argument.

Generators are intended to create files for the build systems to locate the dependencies, while the `deployers` main use case is to copy files from the Conan cache to user space, and performing any other custom operations over the dependency graph, like collecting licenses, generating reports, deploying binaries to the system, etc. The syntax for deployers is:

```
# does a full copy of the dependencies binaries to the current user folder
$ conan install . --deployer=full_deploy
```

There are 3 built-in deployers:

- *full_deploy* does a complete copy of the dependencies binaries in the local folder, with a minimal folder structure to avoid conflicts between files and artifacts of different packages
- *direct_deploy* does a copy of only the immediate direct dependencies, but does not include the transitive dependencies.
- *runtime_deploy* deploys all the shared libraries and the executables of the dependencies into a flat directory structure, preserving subdirectories as-is.

Some generators might have the capability of redefining the target "package folder". That means that if some other generator like `CMakeDeps` is used that is pointing to the packages, it will be pointing to the local deployed copy, and not to the original packages in the Conan cache. See the full example in *Creating a Conan-agnostic deploy of dependencies for developer use*.

It is also possible, and it is a powerful extension point, to write custom user deployers. Read more about custom deployers in *Deployers*.

It is possible to also invoke the package recipes `deploy()` method with the `--deployer-package`:

```
# Execute deploy() method of every recipe that defines it
$ conan install --requires=pkg/0.1 --deployer-package="*"
# Execute deploy() method only for "pkg" (any version) recipes
$ conan install --requires=pkg/0.1 --deployer-package="pkg/*"
# Execute deploy() method for all packages except the "zlib" (transitive dep) one
$ conan install --requires=pkg/0.1 --deployer-package="*" --deployer-package="~zlib/*"
```

The `--deployer-package` argument is a pattern and accepts multiple values, all package references matching any of the defined patterns will execute its `deploy()` method. This includes negated patterns, where for example `--deployer-package=~pkg/*` will execute the `deploy()` method for all packages except for that of the `pkg` recipe. The `--deployer-folder` argument will also affect the output location of this deployment. See the *deploy() method*.

If multiple deployed packages deploy to the same location, it is their responsibility to not mutually overwrite their binaries if they have the same filenames. For example if multiple packages `deploy()` a file called "License.txt", each recipe is responsible for creating an intermediate folder with the package name and/or version that makes it unique, so other recipes `deploy()` method do not overwrite previously deployed "License.txt" files.

Name, version, user, channel

The `conan install` command provides optional arguments for `--name, --version, --user, --channel`. These arguments might not be necessary in the majority of cases. Never for `conanfile.txt` and for `conanfile.py` only in the case that they are not defined in the recipe:

```
from conan import ConanFile
from conan.tools.scm import Version

class Pkg(ConanFile):
    name = "mypkg"

    def requirements(self):
        if Version(self.version) >= "3.23":
            self.requires("...")
```

```
# If we don't specify ``--version``, it will be None and it will fail
$ conan install . --version=3.24
```

Lockfiles

The `conan install` command has several arguments to load and produce lockfiles. By default, if a `conan.lock` file is located beside the recipe or in the current working directory if no path is provided, will be used as an input lockfile.

Lockfiles are strict by default, that means that if there is some `requires` and it cannot find a matching locked reference in the lockfile, it will error and stop. For cases where it is expected that the lockfile will not be complete, as there might be new dependencies, the `--lockfile-partial` argument can be used.

By default, `conan install` will not generate an output lockfile, but if the `--lockfile-out` argument is provided, pointing to a filename, like `--lockfile-out=result.lock`, then a lockfile will be generated from the current dependency graph. If `--lockfile-clean` argument is provided, all versions and revisions not used in the current dependency graph will be dropped from the resulting lockfile.

Let's say that we already have a `conan.lock` input lockfile, but we just added a new `requires = "newpkg/1.0"` to a new dependency. We could resolve the dependencies, locking all the previously locked versions, while allowing to resolve the new one, which was not previously present in the lockfile, and store it in a new location, or overwrite the existing lockfile:

```
# --lockfile=conan.lock is the default, not necessary
$ conan install . --lockfile=conan.lock --lockfile-partial --lockfile-out=conan.lock
```

Also, it is likely that the majority of lockfile operations are better managed by the `conan lock` command.

See also:

- *Lockfiles*.
- Read the tutorial about the *local package development flow*.

Update

The `conan install` command has an `--update` argument that will force the re-evaluation of the selected items of the dependency graph, allowing for the update of the dependencies to the latest version if using version ranges, or to the latest revision of the same version, when those versions are not locked in the given lockfile. Passing `--update` will check every package in the dependency graph, but it is also possible to pass a package name to the `--update` argument (it can be added to the command more than once with different names), to only update those packages, which avoids the re-evaluation of the whole graph.

```
$ conan install . --update  # Update all packages in the graph
$ conan install . --update=openssl  # Update only the openssl package
$ conan install . --update=openssl --update=boost  # Update both openssl and boost␣
↪packages
```

Note that the `--update` argument will look into all the remotes specified in the command for possible newer versions, and won't stop at the first newer one found.

Build modes

The `conan install --build=<mode>` argument controls the behavior regarding building packages from source. The default behavior is failing if there are no existing binaries, with the "missing binary" error message, except for packages that define a `build_policy = "missing"` policy, but this can be changed with the `--build` argument.

The possible values are:

```
--build=never      Disallow build for all packages, use binary packages or fail if a␣
↪binary
                   package is not found, it cannot be combined with other '--build'␣
↪options.
--build=missing    Build packages from source whose binary package is not found.
--build=cascade    Build packages from source that have at least one dependency being␣
↪built from
                   source. Legacy and discouraged, shouldnt be used in most cases.
--build=[pattern]  Build packages from source whose package reference matches the␣
↪pattern. The
                   pattern uses 'fnmatch' style wildcards, so '--build="*"' will build␣
↪everything
                   from source.
--build=~[pattern] Excluded packages, which will not be built from the source, whose␣
↪package
                   reference matches the pattern. The pattern uses 'fnmatch' style␣
↪wildcards.
                   Same as ``--build=![pattern]``
--build=missing:[pattern] Build from source if a compatible binary does not exist, only␣
↪for
                          packages matching pattern.
--build=missing:~[pattern] Build from source if a compatible binary does not exist, for
                          packages not matching the pattern.
                          Same as ``--build=missing:![pattern]``
--build=compatible:[pattern] (Experimental) Build from source if a compatible binary␣
↪does not
                             exist, and the requested package is invalid, the closest␣
↪package
```

(continues on next page)

(continued from previous page)

```
                              binary following the defined compatibility policies (method␣
↪and
                              compatibility.py)
```

The `--build=never` policy can be used to force never building from source, even for package recipes that define the `build_policy = "missing"` policy.

The `--build=compatible:[pattern]` is an **experimental** new mode that allows building missing binaries with a configuration different than the current one. For example if the current profile has `compiler.cppstd=14`, but some package raises an "invalid" configuration error, because it needs at least `compiler.cppstd=17`, and the binary compatibiliy (defined for example in `compatibility.py` plugin) allows that as a compatible binary, then, Conan will build from source that dependency package applying `compiler.cppstd=17`.

The `--build=[pattern]` uses a pattern, so it should use something like `--build="zlib/*"` to match any version of the `zlib` package, as doing `--build=zlib` will not work.

The `--build=missing:[pattern]` form uses the same kind of package patterns as in *Profile patterns* (`fnmatch`-style wildcards, references like `name/version@user/channel`, etc.). Also, you can use the `&` syntax to match the **consumer** conanfile (the root of the graph). That is useful with `conan create .` when you only want to build the package being created if its binary is missing, without retyping its name: `--build=missing:&` (equivalent to `--build=missing:current_pkg/current_version` in the case of `conan create .`).

The `--build=missing:[pattern]` also accepts negations like `--build=missing:!dep/* --build=missing:!lib/*` will build all packages except `dep` and `lib` ones.

Note: Best practices

Forcing the rebuild of existing binaries with `--build="*"` or any other `--build="pkg/*"` or similar pattern is not a recommended practice. If a binary is already existing there is no reason to rebuild it from source. CI pipelines should be specially careful to not do this, and in general the `--build=missing` and `--build=missing:[pattern]` are more recommended.

The `--build=cascade` mode is partly legacy, and shouldn't be used in most cases. The `package_id` computation should be the driver to decide what needs to be built. This mode has been left in Conan 2 only for exceptional cases, like recovering from broken systems, but it is not recommended for normal production usage.

9.2.6 conan list

```
$ conan list -h
usage: conan list [-h] [-f FORMAT] [--out-file OUT_FILE]
                  [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                  [-cc CORE_CONF] [-p PACKAGE_QUERY] [-fp FILTER_PROFILE]
                  [-fs FILTER_SETTINGS] [-fo FILTER_OPTIONS] [-r REMOTE] [-c]
                  [-g GRAPH] [-gb GRAPH_BINARIES] [-gr GRAPH_RECIPES]
                  [-gc {build,host,build-only,host-only}] [--lru LRU]
                  [pattern]

List existing recipes, revisions, or packages in the cache (by default) or the remotes.

positional arguments:
  pattern               A pattern in the form
                        'pkg/version#revision:package_id#revision', e.g:
```

(continues on next page)

(continued from previous page)

```
                        "zlib/1.2.13:*" means all binaries for zlib/1.2.13. If
                        revision is not specified, it is assumed latest one.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html, compact
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        List only the packages matching a specific query, e.g,
                        os=Windows AND (arch=x86 OR compiler=gcc)
  -fp FILTER_PROFILE, --filter-profile FILTER_PROFILE
                        Profiles to filter the binaries
  -fs FILTER_SETTINGS, --filter-settings FILTER_SETTINGS
                        Settings to filter the binaries
  -fo FILTER_OPTIONS, --filter-options FILTER_OPTIONS
                        Options to filter the binaries
  -r REMOTE, --remote REMOTE
                        Remote names. Accepts wildcards ('*' means all the
                        remotes available)
  -c, --cache           Search in the local cache
  -g GRAPH, --graph GRAPH
                        Graph json file
  -gb GRAPH_BINARIES, --graph-binaries GRAPH_BINARIES
                        Which binaries are listed
  -gr GRAPH_RECIPES, --graph-recipes GRAPH_RECIPES
                        Which recipes are listed
  -gc {build,host,build-only,host-only}, --graph-context {build,host,build-only,host-
↪only}
                        Filter the results by the given context
  --lru LRU             List recipes and binaries that have not been recently
                        used. Use a time limit like --lru=5d (days) or
                        --lru=4w (weeks), h (hours), m(minutes)
```

The `conan list` command can list recipes and packages from the local cache, from the specified remotes or from both. This command uses a *reference pattern* as input. The structure of this pattern is based on a complete Conan reference that looks like:

`name/version@user/channel#rrev:pkgid#prev`

This pattern supports using `*` as wildcard as well as `#latest` to specify the latest revision (though that might not be necessary in most cases, by default Conan will be listing the latest revisions).

Using it you can list:

- Recipe references (`name/version@user/channel`).

- Recipe revisions (`name/version@user/channel#rrev`).
- Package IDs and their configurations (`name/version@user/channel#rrev:pkgids`).
- Package revisions (`name/version@user/channel#rrev:pkgids#prev`).

Let's see some examples on how to use this pattern:

Listing recipe references

Listing 6: *list all references on local cache*

```
# Make sure to quote the argument
$ conan list
Local Cache
  hello
    hello/2.26.1@mycompany/testing
    hello/2.20.2@mycompany/testing
    hello/1.0.4@mycompany/testing
    hello/2.3.2@mycompany/stable
    hello/1.0.4@mycompany/stable
  string-view-lite
    string-view-lite/1.6.0
  zlib
    zlib/1.3.1
```

This command is equivalent to `$ conan list "*"` (make sure to quote the argument), if no argument is provided Conan will list all packages.

Listing 7: *list all versions of a reference*

```
$ conan list zlib
Local Cache
  zlib
    zlib/1.3.1
    zlib/1.2.12
```

As we commented, you can also use the * wildcard inside the reference you want to search.

Listing 8: *list all versions of a reference, equivalent to the previous one*

```
# Make sure to quote the argument
$ conan list "zlib/*"
Local Cache
  zlib
    zlib/1.3.1
    zlib/1.2.12
```

You can also use version ranges in the version field to define the versions you want:

Listing 9: *list version ranges*

```
# Make sure to quote the argument
$ conan list "zlib/[<1.2.12]" -r=conancenter
Local Cache
```

(continues on next page)

(continued from previous page)

```
  zlib
    zlib/1.3.1
$ conan list "zlib/[>1.2.11]" -r=conancenter
Local Cache
  zlib
    zlib/1.2.12
    zlib/1.2.13
```

Use the pattern for searching only references matching a specific channel:

Listing 10: *list references with 'stable' channel*

```
$ conan list "*/*@*/stable"
Local Cache
  hello
    hello/2.3.2@mycompany/stable
    hello/1.0.4@mycompany/stable
```

Use the `...@` pattern for searching only references that don't have *user* and *channel*:

Listing 11: *list references without user and channel*

```
$ conan list "*/*@"
Local Cache
  string-view-lite
    string-view-lite/1.6.0
  zlib
    zlib/1.3.1
```

Listing recipe revisions

To list recipe revisions the #<pattern> must be used. If we want just the latest revision we can use the `#latest` placeholder:

Listing 12: *list latest recipe revision*

```
$ conan list zlib/1.3.1#latest
Local Cache
  zlib
    zlib/1.3.1
      revisions
        ffa77daf83a57094149707928bdce823 (2022-11-02 13:46:53 UTC)
```

To list all recipe revisions use the * wildcard:

Listing 13: *list all recipe revisions*

```
  $ conan list "zlib/1.3.1#*"
  Local Cache
    zlib
      zlib/1.3.1
        revisions
```

(continues on next page)

(continued from previous page)

```
        ffa77daf83a57094149707928bdce823 (2022-11-02 13:46:53 UTC)
        8b23adc7acd6f1d6e220338a78e3a19e (2022-10-19 09:19:10 UTC)
        ce3665ce19f82598aa0f7ac0b71ee966 (2022-10-14 11:42:21 UTC)
        31ee767cb2828e539c42913a471e821a (2022-10-12 05:49:39 UTC)
        d77ee68739fcbe5bf37b8a4690eea6ea (2022-08-05 17:17:30 UTC)
```

Listing package IDs

The shortest way of listing all the package IDs belonging to the latest recipe revision is using `name/version@user/channel:*` as the pattern:

Listing 14: *list all package IDs for latest recipe revision*

```
# Make sure to quote the argument
$ conan list "zlib/1.3.1:*"
Local Cache
  zlib
    zlib/1.3.1
      revisions
        d77ee68739fcbe5bf37b8a4690eea6ea (2022-08-05 17:17:30 UTC)
        packages
          d0599452a426a161e02a297c6e0c5070f99b4909
            info
              settings
                arch: x86_64
                build_type: Release
                compiler: apple-clang
                compiler.version: 12.0
                os: Macos
              options
                fPIC: True
                shared: False
          ebec3dc6d7f6b907b3ada0c3d3cdc83613a2b715
            info
              settings
                arch: x86_64
                build_type: Release
                compiler: gcc
                compiler.version: 11
                os: Linux
              options
                fPIC: True
                shared: False
```

Note: Here the `#latest` for the recipe revision is implicit, i.e., that pattern is equivalent to `zlib/1.3.1#latest:*`

To list all the package IDs for all the recipe revisions use the * wildcard in the revision # part:

Listing 15: *list all the package IDs for all the recipe revisions*

```
# Make sure to quote the argument
$ conan list "zlib/1.3.1#*:*"
Local Cache
  zlib
    zlib/1.3.1
      revisions
        d77ee68739fcbe5bf37b8a4690eea6ea (2022-08-05 17:17:30 UTC)
          packages
            d0599452a426a161e02a297c6e0c5070f99b4909
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: apple-clang
                  compiler.version: 12.0
                  os: Macos
                options
                  fPIC: True
                  shared: False
        e4e1703f72ed07c15d73a555ec3a2fa1 (2022-07-04 21:21:45 UTC)
          packages
            d0599452a426a161e02a297c6e0c5070f99b4909
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: apple-clang
                  compiler.version: 12.0
                  os: Macos
                options
                  fPIC: True
                  shared: False
```

Listing package revisions

The shortest way of listing the latest package revision for a specific recipe revision and package ID is using the pattern `name/version@user/channel#rrev:pkgid`

Listing 16: *list latest package revision for a specific recipe revision and package ID*

```
$ conan list zlib/1.3.1
↪#8b23adc7acd6f1d6e220338a78e3a19e:fdb823f07bc228621617c6397210a5c6c4c8807b
Local Cache
  zlib
    zlib/1.3.1
      revisions
        8b23adc7acd6f1d6e220338a78e3a19e (2022-08-05 17:17:30 UTC)
        packages
          fdb823f07bc228621617c6397210a5c6c4c8807b
```

(continues on next page)

(continued from previous page)

```
            revisions
              4834a9b0d050d7cf58c3ab391fe32e25 (2022-11-18 12:33:31 UTC)
```

To list all the package revisions for for the latest recipe revision:

Listing 17: *list all the package revisions for all package-ids the latest recipe revision*

```
# Make sure to quote the argument
$ conan list "zlib/1.3.1:*#*"
Local Cache
  zlib
    zlib/1.3.1
      revisions
        6a6451bbfcb0e591333827e9784d7dfa (2022-12-29 11:51:39 UTC)
        packages
          b1d267f77ddd5d10d06d2ecf5a6bc433fbb7eeed
            revisions
              67bb089d9d968cbc4ef69e657a03de84 (2022-12-29 11:47:36 UTC)
              5e196dbea832f1efee1e70e058a7eead (2022-12-29 11:47:26 UTC)
              26475a416fa5b61cb962041623748d73 (2022-12-29 11:02:14 UTC)
          d15c4f81b5de757b13ca26b636246edff7bdbf24
            revisions
              a2eb7f4c8f2243b6e80ec9e7ee0e1b25 (2022-12-29 11:51:40 UTC)
```

Note: Here the `#latest` for the recipe revision is implicit, i.e., that pattern is equivalent to `zlib/1.3.1#latest:*#*`

Listing graph artifacts

When the `conan list --graph=<graph.json>` graph json file is provided, the command will list the binaries in it. By default, it will list all recipes and binaries included in the dependency graph. But the `--graph-recipes=<recipe-mode>` and `--graph-binaries=<binary-mode>` allow specifying what artifacts have to be listed in the final result, some examples:

- `conan list --graph=graph.json --graph-binaries=build` list exclusively the recipes and binaries that have been built from sources
- `conan list --graph=graph.json --graph-recipes="*"` list exclusively the recipes, all recipes, but no binaries
- `conan list --graph=graph.json --graph-binaries=download` list exclusively the binaries that have been downloaded in the last `conan create` or `conan install`

Additionally, the `--graph-context` argument allows to filter the output by the context of the package, allowing to list either `build` packages, `host` packages, and `build-only` packages or `host-only` packages when we want to list packages that are _only_ present in their respective context.

Filtering packages

There are a few ways to filter the packages that are returned by the command:

- The `--package-query` option allows to filter the packages that match a specific query, for example `--package-query="os=Windows AND (arch=x86 OR compiler=gcc)"` would match only Windows packages where the architecture is x86 or the compiler is gcc.
- You can filter packages by profiles (`--filter-profile`), settings (`--filter-settings`), or options (`--filter-options`). Note that only declared settings and options in the recipe will be considered for filtering, so that if for example a recipe does not declare the `shared` option, its packages will always be returned when using the `--filter-options="*:shared=True"` filter (regardless of the `shared` value used)

```
$ conan list "zlib/1.3.1:*" -fs="os=Macos" -fo="*:shared=True" -r=conancenter
conancenter
  zlib
    zlib/1.3.1
      revisions
        f52e03ae3d251dec704634230cd806a2 (2024-02-22 09:20:06 UTC)
          packages
            24612164eb0760405fcd237df0102e554ed1cb2f
              info
                settings
                  arch: x86_64
                  build_type: Release
                  compiler: apple-clang
                  compiler.version: 13
                  os: Macos
                options
                  shared: True
            a3c9d80d887539fac38b81ff8cd4585fe42027e0
              info
                settings
                  arch: armv8
                  build_type: Release
                  compiler: apple-clang
                  compiler.version: 13
                  os: Macos
                options
                  shared: True
```

Both ways can be used together, and only the packages that match both filters will be listed

List json output format

Note: Best practices

The text output in the terminal should never be parsed or relied on for automation, and it is intended for human reading only. For any automation, the recommended way is using the formatted output as *json*

The `conan list ... --format=json` will return a json output in `stdout` (which can be redirected to a file) with the following structure:

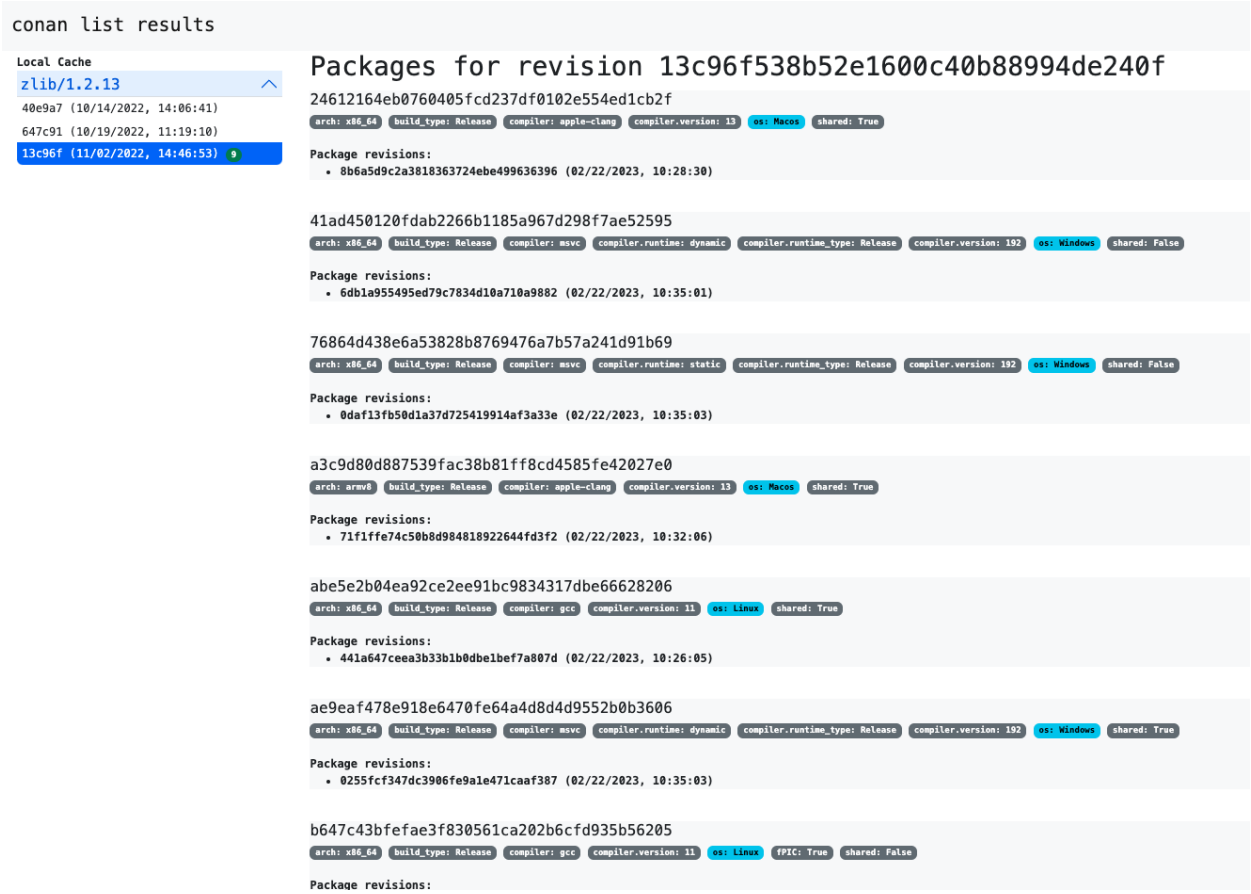
```
# Make sure to quote the argument
$ conan list "zlib/1.3.1:*#*" --format=json
{
  "Local Cache": {
    "zli/1.0.0": {
      "revisions": {
        "b58eeddfe2fd25ac3a105f72836b3360": {
          "timestamp": "2023-01-10 16:30:27 UTC",
          "packages": {
            "9a4eb3c8701508aa9458b1a73d0633783ecc2270": {
              "revisions": {
                "d9b1e9044ee265092e81db7028ae10e0": {
                  "timestamp": "2023-01-10 22:45:49 UTC"
                }
              },
              "info": {
                "settings": {
                   "os": "Linux"
                }
              }
            },
            "ebec3dc6d7f6b907b3ada0c3d3cdc83613a2b715": {
              "revisions": {
                "d9b1e9044ee265092e81db7028ae10e0": {
                  "timestamp": "2023-01-10 22:45:49 UTC"
                }
              },
              "info": {
                "settings": {
                  "os": "Windows"
                }
              }
            }
          }
        }
      }
    }
  }
}
```

List html output format

The `conan list ... --format=html` will return a html output in `stdout` (which can be redirected to a file) with the following structure:

```
$ conan list "zlib/1.2.13#*:*#*" --format=html -c > list.html
```

Here is the rendered generated HTML.



List compact output format

For developers, it can be convenient to use the `--format=compact` output, because it allows to copy and paste full references into other commands (like for example `conan cache path`):

```
$ conan list "zlib/1.2.13:*" -r=conancenter --format=compact
conancenter
  zlib/1.2.13
    zlib/1.2.13#97d5730b529b4224045fe7090592d4c1%1692672717.68 (2023-08-22 02:51:57 UTC)
      zlib/1.2.13
↪#97d5730b529b4224045fe7090592d4c1:d62dff20d86436b9c58ddc0162499d197be9de1e
        settings: Macos, x86_64, Release, apple-clang, 13
        options(diff): fPIC=True, shared=False
      zlib/1.2.13
↪#97d5730b529b4224045fe7090592d4c1:abe5e2b04ea92ce2ee91bc9834317dbe66628206
        settings: Linux, x86_64, Release, gcc, 11
        options(diff): shared=True
      zlib/1.2.13
↪#97d5730b529b4224045fe7090592d4c1:ae9eaf478e918e6470fe64a4d8d4d9552b0b3606
        settings: Windows, x86_64, Release, msvc, dynamic, Release, 192
        options(diff): shared=True
    ...
```

The `--format=compact` will show the list of values for `settings`, and it will only show the differences ("diff") for

options, that is, it will compute the common denominator of options for all displayed packages, and will print only those values that deviate from that common denominator.

See also:

- *Read the "package lists" example usages*

9.2.7 conan lock

The `conan lock` command contains several subcommands. In addition to these commands, most of the Conan commands that compute a graph, like `create`, `install`, `graph`, can both receive lockfiles as input and produce lockfiles as output.

conan lock add

```
$ conan lock add -h
usage: conan lock add [-h] [--out-file OUT_FILE]
                      [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                      [-cc CORE_CONF] [--requires REQUIRES]
                      [--build-requires BUILD_REQUIRES]
                      [--python-requires PYTHON_REQUIRES]
                      [--config-requires CONFIG_REQUIRES]
                      [--lockfile-out LOCKFILE_OUT] [--lockfile LOCKFILE]

Add requires, build-requires or python-requires to an existing or new
lockfile. The resulting lockfile will be ordered, newer versions/revisions
first. References can be supplied with and without revisions like "--
requires=pkg/version", but they must be recipe references, including at least
the version, and they cannot contain a version range.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --requires REQUIRES   Add references to lockfile.
  --build-requires BUILD_REQUIRES
                        Add build-requires to lockfile
  --python-requires PYTHON_REQUIRES
                        Add python-requires to lockfile
  --config-requires CONFIG_REQUIRES
                        Add config-requires to lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the created lockfile
  --lockfile LOCKFILE   Filename of the input lockfile
```

The `conan lock add` command is able to add a package version to an existing or new lockfile `requires`, `build_requires`, `python_requires` or `config_requires`.

For example, the following is able to create a lockfile (by default, named `conan.lock`):

```
$ conan lock add --requires=pkg/1.1 --build-requires=tool/2.2 --python-requires=mypytool/
↪3.3
Generated lockfile: ...conan.lock

$cat conan.lock
{
    "version": "0.5",
    "requires": [
        "pkg/1.1"
    ],
    "build_requires": [
        "tool/2.2"
    ],
    "python_requires": [
        "mypytool/3.3"
    ]
}
```

The `conan lock add` command also allows to provide an existing lockfile as an input, and it will add the arguments to the existing lockfile, maintaining the package versions sorted:

```
$ conan lock add --build-requires=tool/2.3 --lockfile=conan.lock
Using lockfile: '.../conan.lock'
Generated lockfile: .../conan.lock

$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "pkg/1.1"
    ],
    "build_requires": [
        "tool/2.3",
        "tool/2.2"
    ],
    "python_requires": [
        "mypytool/3.3"
    ]
}
```

The `conan lock add` command does not perform any checking on the lockfile, the packages, the existence of packages, the existence of package versions, or the existence of those packages in a given dependency graph, it is a basic manipulation of the json information. When that lockfile is applied to resolve a dependency graph, it is possible that the added versions do not exist, or do not resolve for the `conanfile.py` recipes defined version ranges.

Moreover, the list of versions is still sorted. Adding an older version like `tool/2.1` to the previous lockfile won't make that version being used automatically if the recipes contain the version range `tool/[>=2.0 <3]`, because the `tool/2.2` version is listed there and the range will resolve to it, not to the older `tool/2.1`.

Note that a lockfile created with `conan lock add` can be incomplete and not contain all necessary locked versions that a full dependency graph would need. For those cases, recall that the `--lockfile-partial` argument can be

applied. Note also that if a `conan.lock` file exist in the current folder, Conan commands like `conan install` will automatically use it. Please have a look to the *lockfiles tutorial*.

If explicitly adding revisions, please recall that the revisions are timestamp sorted. If more than one revision exists in the lockfile, it is mandatory to provide the timestamps of those revisions, so the sorting makes sense, which can be done with:

```
$ conan lock add --requires=pkg/1.1#revision%timestamp
```

Warning:

- It is forbidden to manually manipulate a Conan lockfile, changing the strict sorting of references, and that could result in any arbitrary undefined behavior.
- Recall that it is not possible to `conan lock add` a version range. The version might be not fully complete (like not providing the revision), but it must be an exact version.

Note: Best practices

This command will not be necessary in many situations. The existing `conan install`, `conan create`, `conan lock`, `conan export`, `conan graph` commands can directly update or produce new lockfiles with the new information of the packages they are creating, and those new or updated lockfiles can be used to continue with the processing.

conan lock create

```
$ conan lock create -h
usage: conan lock create [-h] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                         [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                         [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL] [-o OPTIONS]
                         [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                         [-o:a OPTIONS_ALL] [-s SETTINGS]
                         [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                         [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                         [-c:h CONF_HOST] [-c:a CONF_ALL]
                         [--requires REQUIRES] [--tool-requires TOOL_REQUIRES]
                         [--name NAME] [--version VERSION] [--user USER]
                         [--channel CHANNEL] [-l LOCKFILE]
                         [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                         [--lockfile-clean]
                         [--lockfile-overrides LOCKFILE_OVERRIDES]
                         [--build-require]
                         [path]

Create a lockfile from a conanfile or a reference.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
```

(continues on next page)

(continued from previous page)

```
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --build-require       Whether the provided reference is a build-require

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
```

(continues on next page)

(continued from previous page)

```
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
```

(continues on next page)

(continued from previous page)

```
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan lock create` command creates a lockfile for the recipe or reference specified in `path` or `--requires`. This command will compute the dependency graph, evaluate which binaries do exist or need to be built, but it will not try to install or build from source those binaries. In that regard, it is equivalent to the `conan graph info` command. Most of the arguments accepted by this command are the same as `conan graph info` (and `conan install`, `conan create`), because the `conan lock create` creates or update a lockfile for a given configuration.

A lockfile can be created from scratch, computing a new dependency graph from a local conanfile, or from requires, for example for this `conanfile.txt`:

Listing 18: conanfile.txt

```
[requires]
fmt/9.0.0

[tool_requires]
cmake/3.23.5
```

We can run:

```
$ conan lock create .

$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "fmt/9.0.0#ca4ae2047ef0ccd7d2210d8d91bd0e02%1675126491.773"
    ],
    "build_requires": [
        "cmake/3.23.5#5f184bc602682bcea668356d75e7563b%1676913225.027"
    ],
    "python_requires": []
}
```

`conan lock create` accepts a `--lockfile` input lockfile (if a `conan.lock` default one is found, it will be automatically used), and then it will add new information in the `--lockfile-out` (by default, also `conan.lock`). For example if we change the above `conanfile.txt`, removing the `tool_requires`, updating `fmt` to `9.1.0` and adding a new dependency to `zlib/1.2.13`:

Listing 19: conanfile.txt

```
[requires]
fmt/9.1.0
zlib/1.2.13

[tool_requires]
```

We will see how `conan lock create` **extends** the existing lockfile with the new configuration, but it doesn't remove unused versions or packages from it:

```
$ conan lock create .  # will use the existing conan.lock as base, and rewrite it
# use --lockfile and --lockfile-out to change that behavior

$ cat conan.lock
{
  "version": "0.5",
  "requires": [
      "zlib/1.2.13#13c96f538b52e1600c40b88994de240f%1667396813.733",
      "fmt/9.1.0#e747928f85b03f48aaf227ff897d9634%1675126490.952",
      "fmt/9.0.0#ca4ae2047ef0ccd7d2210d8d91bd0e02%1675126491.773"
  ],
  "build_requires": [
      "cmake/3.23.5#5f184bc602682bcea668356d75e7563b%1676913225.027"
  ],
  "python_requires": []
}
```

This behavior is very important to be able to capture multiple different configurations (Linux/Windows, shared/static, Debug/Release, etc) that might have different dependency graphs. See the *lockfiles tutorial*, to read more about lockfiles for multiple configurations.

If we want to trim unused versions and packages we can force it with the `--lockfile-clean` argument:

```
$ conan lock create . --lockfile-clean
# will use the existing conan.lock as base, and rewrite it, cleaning unused versions
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "zlib/1.2.13#13c96f538b52e1600c40b88994de240f%1667396813.733",
        "fmt/9.1.0#e747928f85b03f48aaf227ff897d9634%1675126490.952"
    ],
    "build_requires": [],
    "python_requires": []
}
```

See also:

The *lockfiles tutorial section* has more examples and hands on explanations of lockfiles.

conan lock merge

```
$ conan lock merge -h
usage: conan lock merge [-h] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [--lockfile LOCKFILE]
                        [--lockfile-out LOCKFILE_OUT]

Merge 2 or more lockfiles.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --lockfile LOCKFILE   Path to lockfile to be merged
  --lockfile-out LOCKFILE_OUT
                        Filename of the created lockfile
```

The `conan lock merge` command takes 2 or more lockfiles and aggregate them, producing one final lockfile. For example, if we have 2 lockfiles `lock1.lock` and `lock2.lock`, we can merge both in a final `conan.lock` one:

```
# we have 2 lockfiles lock1.lock and lock2.lock
$ conan lock add --requires=pkg/1.1 --lockfile-out=lock1.lock
$ cat lock1.lock
{
    "version": "0.5",
    "requires": [
        "pkg/1.1",
    ],
    "build_requires": [],
    "python_requires": []
}

$ conan lock add --requires=other/2.1 --build-requires=tool/3.2 --lockfile-out=lock2.lock
$ cat lock2.lock
{
    "version": "0.5",
    "requires": [
        "other/2.1"
    ],
    "build_requires": [
        "tool/3.2"
    ],
    "python_requires": []
```

(continues on next page)

(continued from previous page)

```
}

# we can merge both
$ conan lock merge --lockfile=lock1.lock --lockfile=lock2.lock
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "pkg/1.1",
        "other/2.1"
    ],
    "build_requires": [
        "tool/3.2"
    ],
    "python_requires": []
}
```

Similar to the `conan lock add` command, the `conan lock merge`:

- Does keep strict sorting of the lists of versions
- It does not perform any kind of validation if the packages or versions exist or not, or if they belong to a given dependency graph
- It is a basic processing of the json files, aggregating them.
- It doesn't guarantee that the lockfile will be complete, might require `--lockfile-partial` if not
- Recipe revisions, if defined, must contain the timestamp to be sorted correctly.

Warning:

- It is forbidden to manually manipulate a Conan lockfile, changing the strict sorting of references, and that could result in any arbitrary undefined behavior.
- Recall that it is not possible to `conan lock add` a version range. The version might be not fully complete (like not providing the revision), but it must be an exact version.

See also:

To better understand `conan lock merge`, it is recommended to first understand lockfiles in general, visit the *lockfiles tutorial* for a practical introduction to lockfiles.

This `conan lock merge` command can be useful to consolidate in a single lockfile when for some reasons there are several lockfiles that have diverged. A use case would be to create a multi-configuration lockfile that contains all necessary locked versions for all OSs (Linux, Windows, etc), even if there are conditional dependencies in the graph for the different OSs. At some point when testing a new dependency version, for example, `pkg/3.4` new version, when previously `pkg/3.3` was already in the graph, we might want to have such a new lockfile cleaning the previous `pkg/3.3`. If we apply the `--lockfile-clean` argument that will remove the non-used versions in the lockfile, but that will also remove the OS-dependant dependencies. So something like this could be done: lets say that we have this lockfile (simplified, removed revisions for simplicity) as the result of testing a new `pkgb/0.2` version for our main product `app1/0.1`:

Listing 20: app.lock

```
{
  "version": "0.5",
  "requires": [
      "pkgb/0.2",
      "pkgb/0.1",
      "pkgawin/0.1",
      "pkganix/0.1",
      "app1/0.1"
  ]
}
```

The `pkgawin` and `pkganix` are dependencies that exist exclusively in Windows and Linux respectively. Everything looks good, `pkgb/0.2` new version works fine with our app, and we want to clean the unused things from the lockfile:

```
$ conan lock create --requires=app1/0.1 --lockfile=app.lock --lockfile-out=win.lock -s␣
↪os=Windows --lockfile-clean
# Note how both pkgb/0.1 and pkganix are gone
$ cat win.lock
{
  "version": "0.5",
  "requires": [
      "pkgb/0.2",
      "pkgawin/0.1",
      "app1/0.1"
  ]
}
$ conan lock create --requires=app1/0.1 --lockfile=app.lock --lockfile-out=nix.lock -s␣
↪os=Linux --lockfile-clean
# Note how both pkgb/0.1 and pkgawin are gone
$ cat win.lock
{
  "version": "0.5",
  "requires": [
      "pkgb/0.2",
      "pkganix/0.1",
      "app1/0.1"
  ]
}
  # Finally, merge the 2 clean lockfiles, for keeping just 1 for next iteration
$ conan lock merge --lockfile=win.lock --lockfile=nix.lock --lockfile-out=final.lock
$ cat final.lock
{
  "version": "0.5",
  "requires": [
      "pkgb/0.2",
      "pkgawin/0.1",
      "pkganix/0.1",
      "app1/0.1"
  ]
}
```

conan lock remove

```
$ conan lock remove -h
usage: conan lock remove [-h] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [--requires REQUIRES]
                         [--build-requires BUILD_REQUIRES]
                         [--python-requires PYTHON_REQUIRES]
                         [--config-requires CONFIG_REQUIRES]
                         [--lockfile-out LOCKFILE_OUT] [--lockfile LOCKFILE]

Remove requires, build-requires or python-requires from an existing lockfile.
References can be supplied with and without revisions like "--
requires=pkg/version",

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --requires REQUIRES   Remove references to lockfile.
  --build-requires BUILD_REQUIRES
                        Remove build-requires from lockfile
  --python-requires PYTHON_REQUIRES
                        Remove python-requires from lockfile
  --config-requires CONFIG_REQUIRES
                        Remove config-requires from lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the created lockfile
  --lockfile LOCKFILE   Filename of the input lockfile
```

The `conan lock remove` command is able to remove `requires`, `build_requires`, `python_requires` or `config_requires` items from an existing lockfile.

For example, if we have the following `conan.lock`:

```
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "math/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
        "engine/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "build_requires": [
        "cmake/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
        "ninja/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
```

(continues on next page)

(continued from previous page)

```
    ],
    "python_requires": [
        "mytool/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
        "othertool/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ]
}
```

The `conan lock remove` command:

```
$ conan lock remove --requires="math/*" --build-requires=cmake/1.0 --python-requires=
↪"*tool/*"
```

Will result in the following `conan.lock`:

```
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "engine/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "build_requires": [
        "ninja/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "python_requires": [
    ]
}
```

It is possible to specify different patterns:

- Remove by version-ranges with expressions like `--requires="math/[>=1.0 <2]"`, and also
- Remove a specific revision: `--requires=math/1.0#revision`
- Remove locked dependencies for a given "team" user `--requires=*/*@team*`

The `conan lock remove` can be useful for:

- In combination with `conan lock add`, it can be used to force the downgrade of a locked version to an older one. As `conan lock add` always adds and sorts the order, resulting in newer versions with high priority, it is not possible to force going back to an older version with just `add`. But first using `conan lock remove`, then `conan lock add`, it is possible to do so.
- `conan lock remove` can unlock certain dependencies, resulting in an incomplete lockfile, that can be used with `--lockfile-partial` to resolve to the latest available versions for the unlocked dependencies, while keeping locked the rest.

conan lock update

```
$ conan lock update -h
usage: conan lock update [-h] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [--requires REQUIRES]
                         [--build-requires BUILD_REQUIRES]
                         [--python-requires PYTHON_REQUIRES]
                         [--config-requires CONFIG_REQUIRES]
                         [--lockfile-out LOCKFILE_OUT] [--lockfile LOCKFILE]

Update requires, build-requires or python-requires from an existing lockfile.
References that matches the arguments package names will be replaced by the
arguments. References can be supplied with and without revisions like "--
requires=pkg/version",

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --requires REQUIRES   Update references to lockfile.
  --build-requires BUILD_REQUIRES
                        Update build-requires from lockfile
  --python-requires PYTHON_REQUIRES
                        Update python-requires from lockfile
  --config-requires CONFIG_REQUIRES
                        Update config-requires from lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the created lockfile
  --lockfile LOCKFILE   Filename of the input lockfile
```

The `conan lock update` command is able to update `requires`, `build_requires`, `python_requires` or `config_requires` items from an existing lockfile.

For example, if we have the following `conan.lock`:

```
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "math/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
        "engine/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "build_requires": [
        "cmake/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
```

(continues on next page)

(continued from previous page)

```
        "ninja/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "python_requires": [
        "mytool/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
        "othertool/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ]
}
```

The `conan lock update` command:

```
$ conan lock update --requires=math/1.1 --build-requires=cmake/1.1
```

Will result in the following `conan.lock`:

```
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "math/1.1",
        "engine/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "build_requires": [
        "cmake/1.1",
        "ninja/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ],
    "python_requires": [
        "mytool/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
        "othertool/1.0#fd2b006646a54397c16a1478ac4111ac%1702683583.3544693"
    ]
}
```

The command will replace existing locked references that matches the same package name with the provided argument values. If the provided references does not exist in the lockfile, they will be added (same as `conan lock add` command).

This command is similar to do a `conan lock remove` followed by a `conan lock add` command.

conan lock upgrade

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan lock upgrade -h
usage: conan lock upgrade [-h] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                          [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                          [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL]
                          [-o OPTIONS] [-o:b OPTIONS_BUILD]
```

(continues on next page)

(continued from previous page)

```
                          [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL] [-s SETTINGS]
                          [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                          [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                          [-c:h CONF_HOST] [-c:a CONF_ALL]
                          [--requires REQUIRES]
                          [--tool-requires TOOL_REQUIRES] [--name NAME]
                          [--version VERSION] [--user USER]
                          [--channel CHANNEL] [-l LOCKFILE]
                          [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                          [--lockfile-clean]
                          [--lockfile-overrides LOCKFILE_OVERRIDES]
                          [-ur UPDATE_REQUIRES] [-ubr UPDATE_BUILD_REQUIRES]
                          [-upr UPDATE_PYTHON_REQUIRES] [--build-require]
                          [path]

(Experimental) Upgrade requires, build-requires or python-requires from an
existing lockfile given a conanfile or a reference.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
```

(continues on next page)

(continued from previous page)

```
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  -ur UPDATE_REQUIRES, --update-requires UPDATE_REQUIRES
                        Update requires from lockfile
  -ubr UPDATE_BUILD_REQUIRES, --update-build-requires UPDATE_BUILD_REQUIRES
                        Update build-requires from lockfile
  -upr UPDATE_PYTHON_REQUIRES, --update-python-requires UPDATE_PYTHON_REQUIRES
                        Update python-requires from lockfile
  --build-require       Whether the provided reference is a build-require

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
```

(continues on next page)

(continued from previous page)

```
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan lock upgrade` command is able to upgrade `requires`, `build_requires`, `python_requires` items from an existing lockfile.

For example, if we have the following `conan.lock`:

```
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "package/1.0#b0546195fd5bf19a0e6742510fff8855%1740472377.653885"
    ],
```

(continues on next page)

(continued from previous page)

```
    "build_requires": [
        "cmake/1.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
    ]
}
```

And these packages available in the cache:

```
$ conan list "*" --format=compact

Found 9 pkg/version recipes matching * in local cache
Local Cache
  package/1.0
  package/1.9
  cmake/3.29.0
  cmake/3.30.5
```

Using the `conan lock upgrade` command with the appropiate `--update-**` arguments:

```
$ conan lock upgrade --requires=package/[>=1.0 <2] --update-requires=package/[*]
```

Will result in the following `conan.lock`:

```
$ cat conan.lock
{
    "version": "0.5",
    "requires": [
        "package/1.9#b0546195fd5bf19a0e6742510fff8855%1740484122.108484"
    ],
    "build_requires": [
        "cmake/3.29.0#85d927a4a067a531b1a9c7619522c015%1702683583.3411012",
    ]
}
```

The same can be done for `build_requires` and `python_requires`.

The command will upgrade existing locked references that match the same package name with versions that match the version ranges provided by required arguments.

The `conan lock upgrade` command may also be able to upgrade `requires`, `build_requires`, `python_requires` from a conanfile. This use case enhances the functionality of version ranges.

Let's consider the following conanfile:

```
from conan import ConanFile
class HelloConan(ConanFile):
    requires = ("math/[>=1.0 <2]")
    tool_requires = "ninja/[>=1.0]"
```

```
$ conan list "*" --format=compact

Found 9 pkg/version recipes matching * in local cache
Local Cache
  math/1.0
  math/2.0
```

(continues on next page)

(continued from previous page)

```
  ninja/1.0
  ninja/1.1
```

Starting from the same environment and `conan.lock` file from previous example. Running the following command:

```
$ conan lock upgrade . --update-requires=math/1.0 --update-build-requires=ninja/[*]
```

Will result in the following `conan.lock`:

```
{
    "version": "0.5",
    "requires": [
        "math/1.0#b0546195fd5bf19a0e6742510fff8855%1740488410.356828"
    ],
    "build_requires": [
        "ninja/1.1#dc77a17d3e566df710241e3b1f380b8c%1740488410.371875"
    ]
}
```

`math` package have not been updated due to the version range specified in the conanfile, but `ninja` has been updated to the latest version available in the cache.

If a dependency is updated and in the new revision, a transitive dependency is added, the `lock upgrade` command will reflect the new transitive dependency in the lockfile. E.g.

- `liba/1.0` depends on `libb/1.0`
- `libb/1.0` depends on `libc/1.0`

If `libb/2.0` depends also on `libd/1.0`:

```
$ conan lock upgrade --requires=libb/[>=2] --update-requires=libb/*
```

The resulting lockfile will contain both `libc/1.0` and `libd/1.0`.

Note: Updating transitive dependencies is not supported yet. This is an experimental feature and it may change in the future.

---

conan lock upgrade-config

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan lock upgrade-config -h
usage: conan lock upgrade-config [-h] [--out-file OUT_FILE]
                                 [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                                 [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                                 [-u [UPDATE]] [-pr PROFILE]
                                 [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
```

(continues on next page)

(continued from previous page)

```
                                [-pr:a PROFILE_ALL] [-o OPTIONS]
                                [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                                [-o:a OPTIONS_ALL] [-s SETTINGS]
                                [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                                [-s:a SETTINGS_ALL] [-c CONF]
                                [-c:b CONF_BUILD] [-c:h CONF_HOST]
                                [-c:a CONF_ALL] [--requires REQUIRES]
                                [--tool-requires TOOL_REQUIRES] [-l LOCKFILE]
                                [--lockfile-partial]
                                [--lockfile-out LOCKFILE_OUT]
                                [--lockfile-clean]
                                [--lockfile-overrides LOCKFILE_OVERRIDES]
                                [--update-config-requires UPDATE_CONFIG_REQUIRES]
                                [path]

(Experimental) Upgrade config requires in a lockfile

positional arguments:
  path                  Path to a conanconfig.yml file (defaults to current
                        directory)

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
```

(continues on next page)

(continued from previous page)

```
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --update-config-requires UPDATE_CONFIG_REQUIRES
                        Update config-requires from lockfile

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
```

(continues on next page)

(continued from previous page)

```
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan lock upgrade-config` command is equivalent to the previous `conan lock upgrade`, but tailored specifically to upgrade the `config-requires` packages that can be installed with `conan config install-pkg`.

The upgrade can be done over individual requirements passed on the command line:

```
$ conan lock upgrade-config --requires=config/[*] --update-config-requires=config/*
```

Note that it is important to specify which packages are to be updated with `--update-config-requires`, because it is possible that the lockfile contains more than one configuration package.

Also note that the upgrade of the lockfile doesn't change yet or install the configuration. Until a `conan config install-pkg` happens, the active and current configuration will not be updated.

It is also possible to use a `conanconfig.yml` file as an input to the command, by passing the path to its containing folder:

```
$ conan lock upgrade-config . --update-config-requires=config/1.0
```

Important: The `path` argument must point to a folder containing a `conanconfig.yml` file (typically the folder where `conan config install-pkg` is executed), not to a `conanfile.py` or `conanfile.txt` recipe. Use `--requires` or `--tool-requires` to define recipe references directly on the command line.

See also:

- See the *conan config install-pkg* command.
- *conan lock add*: Manually add items to a lockfile
- *conan lock remove*: Manually remove items from a lockfile

- *conan lock create*: Evaluates a dependency graph and save a lockfile
- *conan lock merge*: Merge several existing lockfiles into one
- *conan lock update*: Manually update items from a lockfile
- *conan lock upgrade*: (Experimental) Upgrade items from a lockfile
- *conan lock upgrade-config*: (Experimental) Upgrade configuration packages from a lockfile

```
$ conan lock -h
usage: conan lock [-h] [--out-file OUT_FILE]
                  [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                  [-cc CORE_CONF]
                  {add,create,merge,remove,update,upgrade,upgrade-config} ...

Create or manage lockfiles.

positional arguments:
  {add,create,merge,remove,update,upgrade,upgrade-config}
                        sub-command help
    add                 Add requires, build-requires or python-requires to an
                        existing or new lockfile. The resulting lockfile will
                        be ordered, newer versions/revisions first. References
                        can be supplied with and without revisions like "--
                        requires=pkg/version", but they must be recipe
                        references, including at least the version, and they
                        cannot contain a version range.
    create              Create a lockfile from a conanfile or a reference.
    merge               Merge 2 or more lockfiles.
    remove              Remove requires, build-requires or python-requires
                        from an existing lockfile. References can be supplied
                        with and without revisions like "--
                        requires=pkg/version",
    update              Update requires, build-requires or python-requires
                        from an existing lockfile. References that matches the
                        arguments package names will be replaced by the
                        arguments. References can be supplied with and without
                        revisions like "--requires=pkg/version",
    upgrade             (Experimental) Upgrade requires, build-requires or
                        python-requires from an existing lockfile given a
                        conanfile or a reference.
    upgrade-config      (Experimental) Upgrade config requires in a lockfile

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

9.2.8 conan pkglist

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Perform different operations over package lists:

- Merge multiple package lists (deep merge) into a single one: `conan pkglist merge`
- Find in which remotes packages from the cache can be found: `conan pkglist find-remote`

conan pkglist merge

```
$ conan pkglist merge -h
usage: conan pkglist merge [-h] [-f FORMAT] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF] [-l LIST]

(Experimental) Merge several package lists into a single one

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list file
```

The `conan pkglist merge` command can merge multiple package lists into a single one:

```
$ conan pkglist merge --list=list1.json --list=list2.json --format=json > result.json
```

The merge will be a deep merge, different versions can be added, and within versions multiple revisions can be added, and for every recipe revision multiple package_ids can be also accumulated.

conan pkglist find-remote

```
$ conan pkglist find-remote -h
usage: conan pkglist find-remote [-h] [-f FORMAT] [--out-file OUT_FILE]
                                 [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                                 [-cc CORE_CONF] [-r REMOTE]
                                 list

(Experimental) Find the remotes of a list of packages in the cache

positional arguments:
  list                  Input package list

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -r REMOTE, --remote REMOTE
                        Remote names. Accepts wildcards ('*' means all the
                        remotes available)
```

The `conan pkglist find-remote` command will take a package list of packages in the cache (key `"Local Cache"`) and look for them in the defined remotes. For every exact occurrence in a remote matching the recipe, version, recipe-revision, etc, an entry in the resulting "package lists" will be added for that specific remote.

See also:

- *Read the "package lists" example usages*

9.2.9 conan profile

Manage profiles

conan profile detect

```
$ conan profile detect -h
usage: conan profile detect [-h] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF] [--name NAME] [-f] [-e]

Generate a profile using auto-detected values.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --name NAME           Profile name, 'default' if not specified
  -f, --force           Overwrite if exists
  -e, --exist-ok        If the profile already exist, do not detect a new one
```

Warning: The output of `conan profile detect` is **not stable**. It can change at any time in future Conan releases to adapt to latest tools, latest versions, or other changes in the environment. See *the Conan stability* section for more information.

You can create a new auto-detected profile for your configuration using:

Listing 21: *auto-detected profile*

```
$ conan profile detect
Found apple-clang 14.0
apple-clang>=13, using the major as version
Detected profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos
```

(continues on next page)

(continued from previous page)

```
WARN: This profile is a guess of your environment, please check it.
WARN: Defaulted to cppstd='gnu17' for apple-clang.
WARN: The output of this command is not guaranteed to be stable and can change in future␣
↪Conan versions.
WARN: Use your own profile files for stability.
Saving detected profile to /Users/barbarians/.conan2/profiles/default
```

Be aware that if the profile already exists you have to use `--force` to overwrite it. Otherwise it will fail

Listing 22: *force overwriting already existing default profile*

```
$ conan profile detect
ERROR: Profile '/Users/carlosz/.conan2/profiles/default' already exists
$ conan profile detect --force
Found apple-clang 14.0
...
Saving detected profile to /Users/carlosz/.conan2/profiles/default
```

Note: Best practices It is not recommended to use `conan profile detect` in production. To guarantee reproducibility, it is recommended to define your own profiles, store them in a git repo or in a zip in a server, and distribute it to your team and CI machines with `conan config install`, together with other configuration like custom settings, custom remotes definition, etc.

conan profile list

```
$ conan profile list -h
usage: conan profile list [-h] [-f FORMAT] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF]

List all profiles in the cache.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Listing 23: *force overwriting already existing default profile*

```
$ conan profile list
Profiles found in the cache:
default
ios_base
ios_simulator
clang_15
```

conan profile path

```
$ conan profile path -h
usage: conan profile path [-h] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF]
                          name

Show profile path location.

positional arguments:
  name                  Profile name

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Use to get the profile location in your [CONAN_HOME] folder:

```
$ conan profile path default
/Users/barbarians/.conan2/profiles/default
```

conan profile show

```
$ conan profile show -h
usage: conan profile show [-h] [-f FORMAT] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                          [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL]
                          [-o OPTIONS] [-o:b OPTIONS_BUILD]
```

(continues on next page)

(continued from previous page)

```
                          [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL] [-s SETTINGS]
                          [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                          [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                          [-c:h CONF_HOST] [-c:a CONF_ALL] [-cx {host,build}]

Show aggregated profiles from the passed arguments.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -cx {host,build}, --context {host,build}

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
```

(continues on next page)

(continued from previous page)

```
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL
```

Use **conan profile show** to compute the resulting build and host profiles from the command line arguments. For example, combining different options and settings with the default profile or with any other profile using the `pr:b` or `pr:h` arguments:

```
$ conan profile show -s:h build_type=Debug -o:h shared=False
Host profile:
[settings]
arch=x86_64
build_type=Debug
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos
[options]
shared=False
[conf]


Build profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos
[conf]
```

It's also useful to show the result of the evaluation of *jinja2 templates in the profiles*. For example, a profile like this:

Listing 24: *myprofile*

```
[settings]
os = {{ {"Darwin": "Macos"}.get(platform.system(), platform.system()) }}
```

Check the evaluated profile:

```
$ conan profile show -pr:h=myprofile
Host profile:
[settings]
os=Macos
```

(continues on next page)

(continued from previous page)

```
[conf]
...
```

The command can also output a json with the results:

```
$ conan profile show --format=json

{
"host": {
    "settings": {
        "arch": "armv8",
        "build_type": "Release",
        "compiler": "apple-clang",
        "compiler.cppstd": "gnu17",
        "compiler.libcxx": "libc++",
        "compiler.version": "15",
        "os": "Macos"
    },
    "package_settings": {},
    "options": {},
    "tool_requires": {},
    "conf": {},
    "build_env": ""
},
"build": {
    "settings": {
        "arch": "armv8",
        "build_type": "Release",
        "compiler": "apple-clang",
        "compiler.cppstd": "gnu17",
        "compiler.libcxx": "libc++",
        "compiler.version": "15",
        "os": "Macos"
    },
    "package_settings": {},
    "options": {},
    "tool_requires": {},
    "conf": {},
    "build_env": ""
}
}
```

See also:

- Read more about *profiles*

9.2.10 conan remove

```
$ conan remove -h
usage: conan remove [-h] [-f FORMAT] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF] [-c] [-p PACKAGE_QUERY] [-r REMOTE]
                    [-l LIST] [--lru LRU] [--dry-run]
                    [pattern]

Remove recipes or packages from local cache or a remote.

- If no remote is specified (-r), the removal will be done in the local conan cache.
- If a recipe reference is specified, it will remove the recipe and all the packages,␣
↪unless -p
  is specified, in that case, only the packages matching the specified query (and not␣
↪the recipe)
  will be removed.
- If a package reference is specified, it will remove only the package.

positional arguments:
  pattern               A pattern in the form
                        'pkg/version#revision:package_id#revision', e.g:
                        "zlib/1.2.13:*" means all binaries for zlib/1.2.13. If
                        revision is not specified, it is assumed latest one.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -c, --confirm         Remove without requesting a confirmation
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Remove all packages (empty) or provide a query:
                        os=Windows AND (arch=x86 OR compiler=gcc)
  -r REMOTE, --remote REMOTE
                        Will remove from the specified remote
  -l LIST, --list LIST  Package list file
  --lru LRU             Remove recipes and binaries that have not been
                        recently used. Use a time limit like --lru=5d (days)
                        or --lru=4w (weeks), h (hours), m(minutes)
  --dry-run             Do not remove any items, only print those which would
                        be removed
```

The `conan remove` command removes recipes and packages from the local cache or from a specified remote. Depending on the patterns specified as argument, it is possible to remove a complete package, or just remove the binaries,

leaving still the recipe available. You can also use the keyword `!latest` in the revision part of the pattern to avoid removing the latest recipe or package revision of a certain Conan package.

Use `--dry-run` to avoid performing actual deletions, and instead get a list of the elements that would have been removed.

It has 2 possible and mutually exclusive inputs:

- The `conan remove <pattern>` pattern-based matching of recipes.
- The `conan remove --list=<pkglist>` that will remove the artifacts specified in the `pkglist` json file

There are other commands like **`conan list`** (see the patterns documentation there *conan list*), **`conan upload`** and **`conan download`**, that take the same patterns.

To remove recipes and their associated package binaries from the local cache:

```
$ conan remove "*"
# Removes everything from the cache

$ conan remove "zlib/*""
# Remove all possible versions of zlib, including all recipes, revisions and packages

$ conan remove zlib/1.2.11
# Remove zlib/1.2.11, all its revisions and package binaries. Leave other zlib versions

$ conan remove "zlib/[<1.2.13]"
# Remove zlib/1.2.11 and zlib/1.2.12, all its revisions and package binaries.

$ conan remove zlib/1.2.11#latest
# Remove zlib/1.2.11, only its latest recipe revision and binaries of that revision
# Leave the other zlib/1.2.11 revisions intact

$ conan remove zlib/1.2.11#!latest
# Remove all the recipe revisions from zlib/1.2.11 but the latest one
# Leave the latest zlib/1.2.11 revision intact

$ conan remove zlib/1.2.11#<revision>
# Remove zlib/1.2.11, only its exact <revision> and binaries of that revision
# Leave the other zlib/1.2.11 revisions intact
```

To remove only package binaries, but leaving the recipes, it is necessary to specify the pattern including the `:` separator of the `package_id`:

```
$ conan remove "zlib/1.2.11:*"
# Removes all the zlib/1.2.11 package binaries from all the recipe revisions

$ conan remove "zlib/*:*"
# Removes all the binaries from all the recipe revisions from all zlib versions

$ conan remove "zlib/1.2.11#latest:*"
# Removes all the zlib/1.2.11 package binaries only from the latest zlib/1.2.11 recipe␣
↪revision

$ conan remove "zlib/1.2.11#!latest:*"
# Removes all the zlib/1.2.11 package binaries from all the recipe revisions but the␣
↪latest one
```

(continues on next page)

(continued from previous page)

```
$ conan remove zlib/1.2.11:<package_id>
# Removes the package binary <package_id> from all the zlib/1.2.11 recipe revisions

$ conan remove zlib/1.2.11:#latest<package_id>#latest
# Removes only the latest package revision of the binary identified with <package_id>
# from the latest recipe revision of zlib/1.2.11
# WARNING: Recall that having more than 1 package revision is a smell and shouldn't␣
↪happen
# in normal situations
```

Note that you can filter which packages will be removed using the `--package-query` argument:

```
$ conan remove zlib/1.2.11:* -p compiler=clang
# Removes all the zlib/1.2.11 packages built with Clang compiler
```

You can query packages by both their settings and options, including custom ones. To query for options you need to explicitly add the *options.* prefix, so that *-p options.shared=False* will work but *-p shared=False* won't.

All the above commands, by default, operate in the Conan cache. To remove artifacts from a server, use the `-r=myremote` argument:

```
$ conan remove zlib/1.2.11:* -r=myremote
# Removes all the zlib/1.2.11 package binaries from all the recipe revisions in
# the remote <myremote>
```

9.2.11 conan remote

Use this command to add, edit and remove Conan repositories from the Conan remote registry and also manage authentication to those remotes. For more information on how to work with Conan repositories, please check the *dedicated section*.

```
$ conan remote -h
usage: conan remote [-h] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF]
                    {add,auth,disable,enable,list,list-users,login,logout,remove,rename,
↪set-user,update}
                    ...

Manage the remote list and the users authenticated on them.

positional arguments:
  {add,auth,disable,enable,list,list-users,login,logout,remove,rename,set-user,update}
                        sub-command help
    add                 Add a remote.
    auth                Authenticate in the defined remotes. Use
                        CONAN_LOGIN_USERNAME* and CONAN_PASSWORD* variables if
                        available. Ask for username and password interactively
                        in case (re-)authentication is required and there are
                        no CONAN_LOGIN* and CONAN_PASSWORD* variables
                        available which could be used. Usually you'd use this
```

(continues on next page)

(continued from previous page)

```
                        method over conan remote login for scripting which
                        needs to run in CI and locally. By default, this
                        command returns exit code 0 even if authentication
                        fails for some remotes. Use --strict to return exit
                        code 1 if authentication fails for any remote.
    disable             Disable all the remotes matching a pattern.
    enable              Enable all the remotes matching a pattern.
    list                List current remotes.
    list-users          List the users logged into all the remotes.
    login               Login into the specified remotes matching a pattern.
    logout              Clear the existing credentials for the specified
                        remotes matching a pattern.
    remove              Remove remotes.
    rename              Rename a remote.
    set-user            Associate a username with a remote matching a pattern
                        without performing the authentication.
    update              Update a remote.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote add

```
$ conan remote add -h
usage: conan remote add [-h] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [--insecure] [--index INDEX] [-f]
                        [-ap ALLOWED_PACKAGES] [-t {local-recipes-index}]
                        [--recipes-only] [--force-auth]
                        name url

Add a remote.

positional arguments:
  name                  Name of the remote to add
  url                   Url of the remote

options:
  -h, --help            show this help message and exit
```

(continues on next page)

(continued from previous page)

```
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --insecure            Allow insecure server connections when using SSL
  --index INDEX         Insert the remote at a specific position in the remote
                        list
  -f, --force           Force the definition of the remote even if duplicated
  -ap ALLOWED_PACKAGES, --allowed-packages ALLOWED_PACKAGES
                        Add recipe reference pattern to list of allowed
                        packages for this remote
  -t {local-recipes-index}, --type {local-recipes-index}
                        Define the remote type
  --recipes-only        Disallow binary downloads from this remote, only
                        recipes will be downloaded
  --force-auth          Do not allow anonymous access to this remote, always
                        authenticate before attempting any request
```

conan remote auth

```
$ conan remote auth -h
usage: conan remote auth [-h] [-f FORMAT] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [--with-user] [--force] [--strict]
                         remote

Authenticate in the defined remotes. Use CONAN_LOGIN_USERNAME* and
CONAN_PASSWORD* variables if available. Ask for username and password
interactively in case (re-)authentication is required and there are no
CONAN_LOGIN* and CONAN_PASSWORD* variables available which could be used.
Usually you'd use this method over conan remote login for scripting which
needs to run in CI and locally. By default, this command returns exit code 0
even if authentication fails for some remotes. Use --strict to return exit
code 1 if authentication fails for any remote.

positional arguments:
  remote                Pattern or name of the remote/s to authenticate
                        against. The pattern uses 'fnmatch' style wildcards.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
```

(continues on next page)

(continued from previous page)

```
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --with-user           Only try to auth in those remotes that already have a
                        username or a CONAN_LOGIN_USERNAME* env-var defined
  --force               Force authentication for anonymous-enabled
                        repositories. Can be used for force authentication in
                        case your Artifactory instance has anonymous access
                        enabled and Conan would not ask for username and
                        password even for non-anonymous repositories if not
                        yet authenticated.
  --strict              Return exit code 1 if authentication fails for any
                        remote.
```

Note: If a remote which allows anonymous access matches the pattern given to the command, Conan won't try to authenticate with it by default. If you want to authenticate with a remote that allows anonymous access, you can use the `--force` option.

Note: By default, `conan remote auth` exits with code 0 even if some remotes fail to authenticate. Use `--strict` to exit with a non-zero code when any matched remote fails.

conan remote disable

```
$ conan remote disable -h
usage: conan remote disable [-h] [-f FORMAT] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF]
                            remote

Disable all the remotes matching a pattern.

positional arguments:
  remote                Pattern of the remote/s to disable. The pattern uses
                        'fnmatch' style wildcards.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
```

(continues on next page)

(continued from previous page)

```
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote enable

```
$ conan remote enable -h
usage: conan remote enable [-h] [-f FORMAT] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF]
                           remote

Enable all the remotes matching a pattern.

positional arguments:
  remote                Pattern of the remote/s to enable. The pattern uses
                        'fnmatch' style wildcards.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote list

```
$ conan remote list -h
usage: conan remote list [-h] [-f FORMAT] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF]

List current remotes.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote list-users

```
$ conan remote list-users -h
usage: conan remote list-users [-h] [-f FORMAT] [--out-file OUT_FILE]
                               [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                               [-cc CORE_CONF]

List the users logged into all the remotes.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote login

```
$ conan remote login -h
usage: conan remote login [-h] [-f FORMAT] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF] [-p [PASSWORD]]
                          remote [username]

Login into the specified remotes matching a pattern.

positional arguments:
  remote                Pattern or name of the remote to login into. The
                        pattern uses 'fnmatch' style wildcards.
  username              Username

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -p [PASSWORD], --password [PASSWORD]
                        User password. Use double quotes if password with
                        spacing, and escape quotes if existing. If empty, the
                        password is requested interactively (not exposed)
```

conan remote logout

```
$ conan remote logout -h
usage: conan remote logout [-h] [-f FORMAT] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF]
                           remote

Clear the existing credentials for the specified remotes matching a pattern.

positional arguments:
  remote                Pattern or name of the remote to logout. The pattern
                        uses 'fnmatch' style wildcards.

options:
```

(continues on next page)

(continued from previous page)

```
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote remove

```
$ conan remote remove -h
usage: conan remote remove [-h] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF]
                           remote

Remove remotes.

positional arguments:
  remote                Name of the remote to remove. Accepts 'fnmatch' style
                        wildcards.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote rename

```
$ conan remote rename -h
usage: conan remote rename [-h] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF]
                           remote new_name

Rename a remote.

positional arguments:
  remote                Current name of the remote
  new_name              New name for the remote

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote set-user

```
$ conan remote set-user -h
usage: conan remote set-user [-h] [-f FORMAT] [--out-file OUT_FILE]
                             [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                             [-cc CORE_CONF]
                             remote username

Associate a username with a remote matching a pattern without performing the
authentication.

positional arguments:
  remote                Pattern or name of the remote. The pattern uses
                        'fnmatch' style wildcards.
  username              Username

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
```

(continues on next page)

(continued from previous page)

```
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan remote update

```
$ conan remote update -h
usage: conan remote update [-h] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF] [--url URL] [--secure] [--insecure]
                           [--index INDEX] [-ap ALLOWED_PACKAGES]
                           [--recipes-only [{True,False}]]
                           [--force-auth [{True,False}]]
                           remote

Update a remote.

positional arguments:
  remote                Name of the remote to update

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --url URL             New url for the remote
  --secure              Don't allow insecure server connections when using SSL
  --insecure            Allow insecure server connections when using SSL
  --index INDEX         Insert the remote at a specific position in the remote
                        list
  -ap ALLOWED_PACKAGES, --allowed-packages ALLOWED_PACKAGES
                        Add recipe reference pattern to the list of allowed
                        packages for this remote
  --recipes-only [{True,False}]
                        Disallow binary downloads from this remote, only
                        recipes will be downloaded
  --force-auth [{True,False}]
```

(continues on next page)

(continued from previous page)

```
                        Do not allow anonymous access to this remote, always
                        authenticate before attempting any request
```

See also:

- *Uploading packages tutorial*
- *Working with Conan repositories*
- *Upload Conan packages to remotes using conan upload command*

9.2.12 conan search

Search existing recipes in remotes. This command is equivalent to `conan list <query> -r=*`, and is provided for simpler UX.

```
$ conan search -h
usage: conan search [-h] [-f FORMAT] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF] [-r REMOTE]
                    reference

Search for package recipes in all the remotes (by default), or a remote.

positional arguments:
  reference             Recipe reference to search for. It can contain * as
                        wildcard at any reference field.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -r REMOTE, --remote REMOTE
                        Remote names. Accepts wildcards. If not specified it
                        searches in all the remotes
```

```
$ conan search zlib
conancenter
  zlib
    zlib/1.2.8
    zlib/1.3.1
    zlib/1.2.12
```

(continues on next page)

(continued from previous page)

```
    zlib/1.2.13

$ conan search zlib -r=conancenter
conancenter
  zlib
    zlib/1.2.8
    zlib/1.3.1
    zlib/1.2.12
    zlib/1.2.13

$ conan search zlib/1.2.1* -r=conancenter
conancenter
  zlib
    zlib/1.3.1
    zlib/1.2.12
    zlib/1.2.13

$ conan search zlib/1.2.1* -r=conancenter --format=json
{
    "conancenter": {
        "zlib/1.3.1": {},
        "zlib/1.2.12": {},
        "zlib/1.2.13": {}
    }
}
```

9.2.13 conan version

Note: This feature is in **preview**. It means that it is very unlikely to be removed and unlikely to have breaking changes. Maintainers will try as much as possible to not break it, and only do it if very necessary. See *the Conan stability* section for more information.

```
$ conan version -h
usage: conan version [-h] [-f FORMAT] [--out-file OUT_FILE]
                     [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                     [-cc CORE_CONF]

Give information about the Conan client version.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
```

(continues on next page)

(continued from previous page)

```
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

The **conan version** command shows information about the system and Python environment, including Conan version, Python version, system platform, architecture, release, CPU, and more:

- **version**: The Conan version.
- **conan_path**: The path to the Conan script.
- ***python*: A sub-dictionary containing information about the Python environment, including:**
  - **version**: The version of Python being used.
  - **sys_version**: The full Python system version.
  - **sys_executable**: The path to the Python executable.
  - **is_frozen**: An indicator of whether the Python script is being run as a frozen file (e.g., using py2exe or PyInstaller).
  - **architecture**: The architecture detected by Python.
- ***system*: A sub-dictionary containing information about the operating system, including:**
  - **version**: The version of the operating system.
  - **platform**: The platform of the system.
  - **system**: The name of the operating system.
  - **release**: The release version of the operating system.
  - **cpu**: Information about the system's CPU.

```
$ conan version
version: 2.0.6
conan_path: /conan/venv/bin/conan
python
  version: 3.10.4
  sys_version: 3.10.4 (main, May 17 2022, 10:53:07) [Clang 13.1.6 (clang-1316.0.21.2.3)]
  sys_executable: /conan/venv/bin/python
  is_frozen: False
  architecture: arm64
system
  version: Darwin Kernel Version 23.4.0: Fri Mar 15 00:12:37 PDT 2024; root:xnu-10063.
↪101.17~1/RELEASE_ARM64_T6031
  platform: macOS-14.4.1-arm64-arm-64bit
  system: Darwin
  release: 23.4.0
  cpu: arm
```

The **conan version --format=json** returns a JSON output format in `stdout` (which can be redirected to a file) with the following structure:

```
$ conan version --format=json
{
```

(continues on next page)

(continued from previous page)

```
    "version": "2.0.6",
    "conan_path": "/Users/myUser/Documents/GitHub/conan/venv/bin/conan",
    "python": {
        "version": "3.10.4",
        "sys_version": "3.10.4 (main, May 17 2022, 10:53:07) [Clang 13.1.6 (clang-1316.0.
↪21.2.3)]",
        "sys_executable": "/conan/venv/bin/python",
        "is_frozen": false,
        "architecture": "arm64"
    },
    "system": {
        "version": "Darwin Kernel Version 23.4.0: Fri Mar 15 00:12:37 PDT 2024; root:xnu-
↪10063.101.17~1/RELEASE_ARM64_T6031",
        "platform": "macOS-14.4.1-arm64-arm-64bit",
        "system": "Darwin",
        "release": "23.4.0",
        "cpu": "arm"
    }
}
```

9.2.14 conan workspace

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `conan workspace` command allows to open, add, and remove packages from the current workspace. Check the `conan workspace -h` help and the help of the subcommands to check their usage.

```
$ conan workspace -h
usage: conan workspace [-h] [--out-file OUT_FILE]
                       [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                       [-cc CORE_CONF]
                       {add,build,clean,complete,create,info,init,install,open,remove,
↪root,source,super-install}
                       ...

Manage Conan workspaces (group of packages in editable mode)

positional arguments:
  {add,build,clean,complete,create,info,init,install,open,remove,root,source,super-
↪install}
                        sub-command help
    add                 Add packages to current workspace
    build               Call "conan build" for packages in the workspace, in
                        the right order
    clean               Clean the temporary build folders when possible
    complete            Complete the workspace, opening or adding intermediate
                        packages to it that have requirements to other
```

(continues on next page)

(continued from previous page)

```
                        packages in the workspace.
    create              Call "conan create" for packages in the workspace, in
                        the correct order. Packages will be created in the
                        Conan cache, not locally
    info                Display info for current workspace
    init                Initialize a workspace in the given path, creating an
                        empty conanws.yml and conanws.py if they dont exist.
    install             Call "conan install" for packages in the workspace, in
                        the right order
    open                Open specific references. If no reference is provided,
                        open every package in the current workspace definition
                        whose folder does not yet exist.
    remove              Remove packages from the current workspace
    root                Return the folder containing the
                        conanws.py/conanws.yml workspace file
    source              Call the source() method of packages in the workspace
    super-install       Install the workspace as a monolith, installing only
                        external dependencies to the workspace, generating a
                        single result (generators, etc) for the whole
                        workspace.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan workspace init

```
$ conan workspace init -h
usage: conan workspace init [-h] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF]
                            [path]

Initialize a workspace in the given path, creating an empty conanws.yml and
conanws.py if they dont exist.

positional arguments:
  path                  Path to a folder where the workspace will be
                        initialized. Defaults to the current directory
```

(continues on next page)

(continued from previous page)

```
options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

The command `conan workspace init [path]` creates an empty `conanws.yml` file and a minimal `conanws.py` within that path if they don't exist yet. That path can be relative to your current working directory.

```
$ conan workspace init myfolder
Created empty conanws.yml in myfolder
Created minimal conanws.py in myfolder
```

conan workspace [add | remove]

```
$ conan workspace add -h
usage: conan workspace add [-h] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF] [--name NAME] [--version VERSION]
                           [--user USER] [--channel CHANNEL] [--ref REF]
                           [--folder FOLDER] [-of OUTPUT_FOLDER]
                           [-r REMOTE | -nr]
                           [path]

Add packages to current workspace

positional arguments:
  path                  Path to the package folder in the user workspace

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --ref REF             Open and add this reference
  --folder FOLDER       Target folder for the opened package, relative to the
```

(continues on next page)

(continued from previous page)

```
                        workspace root. Subfolders are allowed (e.g.
                        libs/mypkg). Only valid together with '--ref'
  -of OUTPUT_FOLDER, --output-folder OUTPUT_FOLDER
                        The root output folder for generated and build files
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile
```

```
$ conan workspace remove -h
usage: conan workspace remove [-h] [--out-file OUT_FILE]
                              [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                              [-cc CORE_CONF]
                              path

Remove packages from the current workspace

positional arguments:
  path                  Path to the package folder in the user workspace

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Use these commands to add or remove editable packages to the current workspace. The `conan workspace add <path>` folder must contain a `conanfile.py`. That path can be relative to your current workspace.

When adding a package, the reference `conan workspace add --ref <ref>` can be used instead of the individual arguments `--name`, `--version`, etc. Also, the optional `--folder` argument can be used to control the relative path (to the workspace root) where the package will be added. Subfolders are allowed, e.g. `--folder=libs/mypkg`.

The `conanws.py` has a default implementation, but it is possible to override the default behavior:

Listing 25: conanws.py

```
import os
from conan import Workspace
```

(continues on next page)

(continued from previous page)

```
class MyWorkspace(Workspace):
   def name(self):
      return "myws"

   def add(self, ref, path, *args, **kwargs):
      self.output.info(f"Adding {ref} at {path}")
      super().add(ref, path, *args, **kwargs)

   def remove(self, path, *args, **kwargs):
      self.output.info(f"Removing {path}")
      return super().remove(path, *args, **kwargs)
```

See *conan workspace complete* command to open/add multiple packages that are missing in the package to connect different packages already existing in the workspace.

conan workspace info

```
$ conan workspace info -h
usage: conan workspace info [-h] [-f FORMAT] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF]

Display info for current workspace

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Use this command to show information about the current workspace

```
$ cd myfolder
$ conan new workspace
$ conan workspace info
WARN: Workspace found
WARN: Workspace is a dev-only feature, exclusively for testing
name: myfolder
folder: /path/to/myfolder
packages
```

(continues on next page)

(continued from previous page)

```
  - path: liba
    ref: liba/0.1
  - path: libb
    ref: libb/0.1
  - path: app1
    ref: app1/0.1
```

conan workspace clean

```
$ conan workspace clean -h
usage: conan workspace clean [-h] [--out-file OUT_FILE]
                             [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                             [-cc CORE_CONF]

Clean the temporary build folders when possible

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

The new `conan workspace clean` command removes by default the `output-folder` of every package in the workspace if it was defined. If it is not defined, it won't remove anything by default, as removing files in user space is dangerous, and could destroy user changes or files. It would be recommended that users manage that cleaning with `git clean -xdf` or similar strategies. It is also possible to define a custom clean logic by implementing the `clean()` method:

```
class Ws(Workspace):
   def name(self):
      return "my_workspace"
   def clean(self):
      self.output.info("MY CLEAN!!!!")
```

conan workspace open

```
$ conan workspace open -h
usage: conan workspace open [-h] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF] [--folder FOLDER]
                            [-r REMOTE | -nr]
                            [reference]

Open specific references. If no reference is provided, open every package in
the current workspace definition whose folder does not yet exist.

positional arguments:
  reference             Open this package source repository. If omitted, open
                        all packages in the current workspace definition

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --folder FOLDER       Target folder for the opened package, relative to the
                        workspace root. Subfolders are allowed (e.g.
                        libs/mypkg). Only valid together with a 'reference'
                        argument
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
```

The new `conan workspace open` command implements a new concept. The packages containing an `scm` information in the `conandata.yml` (with `git.coordinates_to_conandata()`) can be automatically cloned and checkout inside the current workspace from their Conan recipe reference (including recipe revision).

The optional `--folder` argument can be used together with an explicit reference to control the relative path (to the workspace root) where the package will be cloned. Subfolders are allowed, e.g. `--folder=libs/mypkg`:

```
$ conan workspace open mypkg/1.0 --folder=libs/mypkg
```

If `conan workspace open` is called without a reference argument, it will read the current `conanws` file and automatically open every package it defines whose folder doesn't already exist yet, cloning each one into its workspace-defined path:

```
$ conan workspace open
```

See *conan workspace complete* command to open/add multiple packages that are missing in the package to connect different packages already existing in the workspace.

conan workspace root

```
$ conan workspace root -h
usage: conan workspace root [-h] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF]

Return the folder containing the conanws.py/conanws.yml workspace file

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

Return the folder containing the conanws.py/conanws.yml workspace file.

conan workspace source

```
$ conan workspace source -h
usage: conan workspace source [-h] [--out-file OUT_FILE]
                              [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                              [-cc CORE_CONF] [--pkg PKG]

Call the source() method of packages in the workspace

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --pkg PKG             Define specific packages
```

The command `conan workspace source` performs the equivalent of `conan source <package-path>` for every package defined within the workspace.

conan workspace install

```
$ conan workspace install -h
usage: conan workspace install [-h] [--out-file OUT_FILE]
                               [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                               [-cc CORE_CONF] [--pkg PKG] [-b BUILD]
                               [-r REMOTE | -nr] [-u [UPDATE]] [-pr PROFILE]
                               [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                               [-pr:a PROFILE_ALL] [-o OPTIONS]
                               [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                               [-o:a OPTIONS_ALL] [-s SETTINGS]
                               [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                               [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                               [-c:h CONF_HOST] [-c:a CONF_ALL] [-l LOCKFILE]
                               [--lockfile-partial]

Call "conan install" for packages in the workspace, in the right order

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --pkg PKG             Define specific packages
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
```

(continues on next page)

(continued from previous page)

```
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
```

(continues on next page)

(continued from previous page)

```
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
```

The command `conan workspace install` performs the equivalent of `conan install <package-path>` for every `package` defined within the workspace in the correct order.

conan workspace build

```
$ conan workspace build -h
usage: conan workspace build [-h] [--out-file OUT_FILE]
                             [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                             [-cc CORE_CONF] [--pkg PKG] [-b BUILD]
                             [-r REMOTE | -nr] [-u [UPDATE]] [-pr PROFILE]
                             [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                             [-pr:a PROFILE_ALL] [-o OPTIONS]
                             [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                             [-o:a OPTIONS_ALL] [-s SETTINGS]
                             [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                             [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                             [-c:h CONF_HOST] [-c:a CONF_ALL] [-l LOCKFILE]
                             [--lockfile-partial]

Call "conan build" for packages in the workspace, in the right order

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --pkg PKG             Define specific packages
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
```

(continues on next page)

(continued from previous page)

```
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
```

(continues on next page)

(continued from previous page)

```
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
```

The command `conan workspace build` performs the equivalent of `conan build <package-path>` for every package defined within the workspace in the correct order.

conan workspace create

```
$ conan workspace create -h
usage: conan workspace create [-h] [--out-file OUT_FILE]
                              [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                              [-cc CORE_CONF] [--pkg PKG] [-b BUILD]
                              [-r REMOTE | -nr] [-u [UPDATE]] [-pr PROFILE]
                              [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                              [-pr:a PROFILE_ALL] [-o OPTIONS]
                              [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                              [-o:a OPTIONS_ALL] [-s SETTINGS]
                              [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                              [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                              [-c:h CONF_HOST] [-c:a CONF_ALL] [-l LOCKFILE]
```

(continues on next page)

(continued from previous page)

```
                              [--lockfile-partial]

Call "conan create" for packages in the workspace, in the correct order.
Packages will be created in the Conan cache, not locally

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --pkg PKG             Define specific packages
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
```

(continues on next page)

(continued from previous page)

```
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
```

The command `conan workspace create` performs the equivalent of `conan create <package-path>` for every package defined within the workspace in the correct order. They will be created in the Conan cache, not locally.

conan workspace super-install

```
$ conan workspace super-install -h
usage: conan workspace super-install [-h] [-f FORMAT] [--out-file OUT_FILE]
                                     [-v [{quiet,error,warning,notice,status,verbose,
↪debug,v,trace,vv}]]
                                     [-cc CORE_CONF] [--pkg PKG]
                                     [-g GENERATOR] [-of OUTPUT_FOLDER]
                                     [-d DEPLOYER]
                                     [--deployer-folder DEPLOYER_FOLDER]
                                     [--deployer-package DEPLOYER_PACKAGE]
                                     [--envs-generation {false}] [-b BUILD]
                                     [-r REMOTE | -nr] [-u [UPDATE]]
                                     [-pr PROFILE] [-pr:b PROFILE_BUILD]
                                     [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL]
                                     [-o OPTIONS] [-o:b OPTIONS_BUILD]
                                     [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL]
                                     [-s SETTINGS] [-s:b SETTINGS_BUILD]
                                     [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL]
                                     [-c CONF] [-c:b CONF_BUILD]
                                     [-c:h CONF_HOST] [-c:a CONF_ALL]
                                     [-l LOCKFILE] [--lockfile-partial]
                                     [--lockfile-out LOCKFILE_OUT]
                                     [--lockfile-clean]
                                     [--lockfile-overrides LOCKFILE_OVERRIDES]

Install the workspace as a monolith, installing only external dependencies to
the workspace, generating a single result (generators, etc) for the whole
workspace.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --pkg PKG             Define specific packages
  -g GENERATOR, --generator GENERATOR
                        Generators to use
  -of OUTPUT_FOLDER, --output-folder OUTPUT_FOLDER
                        The root output folder for generated and build files
```

(continues on next page)

(continued from previous page)

```
  -d DEPLOYER, --deployer DEPLOYER
                        Deploy using the provided deployer to the output
                        folder. Built-in deployers: 'full_deploy',
                        'direct_deploy', 'runtime_deploy'
  --deployer-folder DEPLOYER_FOLDER
                        Deployer output folder, base build folder by default
                        if not set
  --deployer-package DEPLOYER_PACKAGE
                        Execute the deploy() method of the packages matching
                        the provided patterns
  --envs-generation {false}
                        Generation strategy for virtual environment files for
                        the root
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
```

(continues on next page)

(continued from previous page)

```
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The command `conan workspace super-install` is useful to install and build the current workspace as a monolithic super-project of the editables.

By default it uses all the `editable` packages in the workspace. It is possible to select only a subset of them with the `conan workspace super-install --pkg=pkg_name1 --pkg=pkg_name2` optional arguments. Only the sub-graph of those packages, including their dependencies and transitive dependencies will be installed.

Note: As the super-project only generates one shared set of dependency files, a dependency reachable through different contexts or visibilities from different workspace packages can be ambiguous. Since Conan 2.33, Conan prioritizes the `host` context and visible requirements to resolve it, warning when it does so. See the *monolithic build tutorial* for more details.

---

conan workspace complete

```
$ conan workspace complete -h
usage: conan workspace complete [-h] [--out-file OUT_FILE]
                                [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                                [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                                [-u [UPDATE]] [-pr PROFILE]
                                [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                                [-pr:a PROFILE_ALL] [-o OPTIONS]
                                [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                                [-o:a OPTIONS_ALL] [-s SETTINGS]
                                [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                                [-s:a SETTINGS_ALL] [-c CONF]
                                [-c:b CONF_BUILD] [-c:h CONF_HOST]
                                [-c:a CONF_ALL] [-l LOCKFILE]
                                [--lockfile-partial]

Complete the workspace, opening or adding intermediate packages to it that
have requirements to other packages in the workspace.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
```

(continues on next page)

(continued from previous page)

```
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
```

(continues on next page)

(continued from previous page)

```
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
```

The `conan workspace complete` command is intended to complete the `conan workspace open/add` commands. When there are packages in a workspace that have dependencies on some packages in the Conan cache, and in turn those cache packages depend on packages that are in the workspace, this creates an undesired and risky situation.

Packages in the Conan cache must be reproducible, including their dependencies. Having binaries in the Conan cache that build against headers and libraries in a workspace, that are not really packages yet, and might never be, is a dangerous situation. It means that it is very easy to have Conan packages in the cache that build and link against code and binaries that never exist in Conan, that are never uploaded as packages. When the cache packages are uploaded and later deployed to production they will link and/or run with different packages, which can cause different issues, from compile or link problems to very difficult to debug and understand runtime errors.

So when a Conan `workspace` command detects this situation, it will raise an error like:

```
ERROR: Workspace definition error. Package mypkg/version in the Conan cache
has dependencies to packages in the workspace: ["dep1/1.0", "dep2/0.2"]
Try the 'conan workspace complete' to open/add intermediate packages
```

This could be solved by manually doing a `conan workspace open/add <dep>` for the missing packages, and add them to the workspace, then repeat the previous command until the error is gone. The `conan workspace complete` command is basically a helper to do this process automatically, detecting what are the missing packages and adding all of them to the workspace.

See also:

- Read the *Workspace tutorial* section.

- Read the *conan new workspace* command section.

9.2.15 conan run

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan run -h
usage: conan run [-h] [--out-file OUT_FILE]
                 [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                 [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr] [-u [UPDATE]]
                 [-pr PROFILE] [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                 [-pr:a PROFILE_ALL] [-o OPTIONS] [-o:b OPTIONS_BUILD]
                 [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL] [-s SETTINGS]
                 [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                 [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                 [-c:h CONF_HOST] [-c:a CONF_ALL] [--requires REQUIRES]
                 [--tool-requires TOOL_REQUIRES] [--name NAME]
                 [--version VERSION] [--user USER] [--channel CHANNEL]
                 [-l LOCKFILE] [--lockfile-partial]
                 [--lockfile-out LOCKFILE_OUT] [--lockfile-clean]
                 [--lockfile-overrides LOCKFILE_OVERRIDES]
                 [--context {host,build}] [--build-require]
                 [path] command

(Experimental) Run a command given a set of requirements from a recipe or from command␣
↪line.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given
  command               Command to run

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
```

(continues on next page)

(continued from previous page)

```
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --context {host,build}
                        Context to use, by default both contexts are activated
                        if not specified
  --build-require       Whether the provided path is a build-require

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
```

(continues on next page)

(continued from previous page)

```
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan run` command lets you directly execute a binary from a Conan package, automatically resolving and in-

stalling all its dependencies. There's no need to manually activate any environments generated by Conan: just pass the executable to run, and Conan will activate the necessary environments and execute it.

The command can receive either a `conanfile.py/conanfile.txt` or have the requirements specified directly from the CLI via `--requires` and `--tool-requires` arguments.

For example, if we call a specific version of `openssl` we would:

```
$ conan run "openssl --version" --tool-requires=openssl/3.5.4

Installing and building dependencies, this might take a while...
OpenSSL 3.5.4 30 Sep 2025 (Library: OpenSSL 3.5.4 30 Sep 2025)
```

This command is useful when you want to execute some specific binary from any package.

Note: This command activates both the `host` and `build` contexts, so that both contexts binaries are made available at once. In case that a package exists in both contexts, the `host` context binaries take precedence.

Running executables from profile `[tool_requires]`

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Starting in Conan 2.32, `conan run` can be invoked without a `conanfile` and without `--requires` / `--tool-requires` arguments. In that case, executables provided by the `[tool_requires]` section of the active profile can be executed directly:

Listing 26: *myprofile*

```
include(default)
[tool_requires]
cmake/3.29.3
```

```
$ conan run "cmake --version" -pr:h=myprofile
```

9.2.16 conan require

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

```
$ conan require -h
usage: conan require [-h] [--out-file OUT_FILE]
                     [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                     [-cc CORE_CONF]
                     {add,remove} ...

Adds/removes requirements to/from your local conanfile.
```

(continues on next page)

(continued from previous page)

```
positional arguments:
  {add,remove}          sub-command help
    add                 Add a new requirement to your local conanfile as a
                        version range. By default, it will look for the
                        requirement versions remotely.
    remove              Removes a requirement from your local conanfile.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

The **conan require** command helps to add any requirement as a version range or remove it from your *conanfile.py*.

Important: This command is only a UX utility. It's not aimed at replacing editing the conanfile, and it's not expected to cover all the use cases, i.e., conditional requirements, requirements with different traits, etc. For all those mentioned scenarios, we recommend editing the conanfile.py as usual.

conan require add

```
$ conan require add -h
usage: conan require add [-h] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [--folder FOLDER] [-tor TOOL]
                         [-ter TEST] [-r REMOTE | -nr]
                         [requires ...]

Add a new requirement to your local conanfile as a version range. By default,
it will look for the requirement versions remotely.

positional arguments:
  requires              Requirement name.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
```

(continues on next page)

(continued from previous page)

```
                      verbose to more verbose: -vquiet, -verror, -vwarning,
                      -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                      -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                      Define core configuration, overwriting global.conf
                      values. E.g.: -cc core:non_interactive=True
  --folder FOLDER     Path to a folder containing a recipe (conanfile.py).
                      Defaults to the current directory
  -tor TOOL, --tool TOOL
                      Tool requirement name.
  -ter TEST, --test TEST
                      Test requirement name.
  -r REMOTE, --remote REMOTE
                      Remote names. Accepts wildcards ('*' means all the
                      remotes available)
  -nr, --no-remote    Do not use remote, resolve exclusively in the cache
```

Add a new requirement to your local *conanfile.py* as a version range.

By default, it looks for the recipe name in any of your remotes. When a remote contains any result for the recipe required, the latest version is used and written as a version range between the version found and the next major one (if possible, as versions based on commits do not have that major version):

```
$ conan require add fmt
Connecting to remote 'conancenter' anonymously
Found 21 pkg/version recipes matching fmt/* in conancenter
Added 'fmt/[>=12.1.0 <13]' as a new requires.
```

It admits several arguments as new requirements:

```
$ conan require add fmt zlib
Connecting to remote 'conancenter' anonymously
Found 21 pkg/version recipes matching fmt/* in conancenter
Found 5 pkg/version recipes matching zlib/* in conancenter
Added 'fmt/[>=12.1.0 <13]' as a new requires.
Added 'zlib/[>=1.3.1 <2]' as a new requires.
```

Or even, you can directly put the requirement version:

```
$ conan require add boost/1.89.0
Added 'boost/[>=1.89.0 <2]' as a new requires.
```

Tool and test requirements are also supported:

```
$ conan require add --tool cmake --test gtest
Connecting to remote 'conancenter' anonymously
Found 54 pkg/version recipes matching cmake/* in conancenter
Found 10 pkg/version recipes matching gtest/* in conancenter
Added 'cmake/[>=4.2.2 <5]' as a new tool_requires.
Added 'gtest/cci.20210126' as a new test_requires.
```

Use `--no-remote` to resolve versions only from the local cache:

```
$ conan require add boost --no-remote
Found 2 pkg/version recipes matching boost/* in local cache
Added 'boost/[>=1.89.0 <2]' as a new requires.
```

Use `--folder` to point to a different recipe location:

```
$ conan require add fmt --folder=path/to/conanfile.py
```

conan require remove

```
$ conan require remove -h
usage: conan require remove [-h] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF] [--folder FOLDER] [-tor TOOL]
                            [-ter TEST]
                            [requires ...]

Removes a requirement from your local conanfile.

positional arguments:
  requires              Requirement name.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --folder FOLDER       Path to a folder containing a recipe (conanfile.py).
                        Defaults to the current directory
  -tor TOOL, --tool TOOL
                        Tool requirement name.
  -ter TEST, --test TEST
                        Test requirement name.
```

Remove any requirement from your *conanfile.py*:

```
$ conan require remove fmt zlib
Removed fmt dependency as requires.
Removed zlib dependency as requires.
```

Tool and test requirements are also supported:

```
$ conan require remove --tool cmake --test gtest
Removed cmake dependency as tool_requires.
```

(continues on next page)

(continued from previous page)

```
Removed gtest dependency as test_requires.
```

Use `--folder` to point to a different recipe location:

```
$ conan require remove fmt --folder=path/to/conanfile.py
```

- *conan cache*: Return the path of recipes and packages in the cache
- *conan config*: Manage Conan configuration (remotes, settings, plugins, etc)
- *conan graph*: Obtain information about the dependency graph without fetching binaries
- *conan inspect*: Inspect a conanfile.py to return the public fields
- *conan install*: Install dependencies
- *conan list*: List recipes, revisions and packages in the local cache or in remotes
- *conan lock*: Create and manage lockfiles
- *conan pkglist*: Manipulate package lists, merge them or find packages in remotes.
- *conan profile*: Display and manage profile files
- *conan remove*: Remove packages from the local cache or from remotes
- *conan remote*: Add, remove, login/logout and manage remote server
- *conan search*: Search packages matching a name
- *conan version*: Give information about the Conan client version
- *conan workspace*: Manage Conan workspaces
- *conan run*: Execute binaries with automatic environment activation
- *conan require*: Adds/removes requirements to/from your local conanfile

Creator commands:

9.2.17 conan build

```
$ conan build -h
usage: conan build [-h] [-f FORMAT] [--out-file OUT_FILE]
                   [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                   [-cc CORE_CONF] [--name NAME] [--version VERSION]
                   [--user USER] [--channel CHANNEL] [-g GENERATOR]
                   [-of OUTPUT_FOLDER] [-d DEPLOYER]
                   [--deployer-folder DEPLOYER_FOLDER] [--build-require]
                   [--envs-generation {false}] [-b BUILD] [-r REMOTE | -nr]
                   [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                   [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL] [-o OPTIONS]
                   [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL]
                   [-s SETTINGS] [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                   [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                   [-c:h CONF_HOST] [-c:a CONF_ALL] [-l LOCKFILE]
                   [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                   [--lockfile-clean]
                   [--lockfile-overrides LOCKFILE_OVERRIDES]
```

(continues on next page)

(continued from previous page)

```
                    [path]

Install dependencies and call the build() method.

positional arguments:
  path                  Path to a python-based recipe file or a folder
                        containing a conanfile.py recipe. conanfile.txt cannot
                        be used with conan build. Defaults to current
                        directory

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -g GENERATOR, --generator GENERATOR
                        Generators to use
  -of OUTPUT_FOLDER, --output-folder OUTPUT_FOLDER
                        The root output folder for generated and build files
  -d DEPLOYER, --deployer DEPLOYER
                        Deploy using the provided deployer to the output
                        folder. Built-in deployers: 'full_deploy',
                        'direct_deploy', 'runtime_deploy'
  --deployer-folder DEPLOYER_FOLDER
                        Deployer output folder, base build folder by default
                        if not set
  --build-require       Whether the provided path is a build-require
  --envs-generation {false}
                        Generation strategy for virtual environment files for
                        the root
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
```

(continues on next page)

(continued from previous page)

```
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
```

(continues on next page)

(continued from previous page)

```
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan build` command installs the recipe specified in `path` and calls its `build()` method.

See also:

- Read the tutorial about the *local package development flow*.

9.2.18 conan create

```
$ conan create -h
usage: conan create [-h] [-f FORMAT] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF] [--name NAME] [--version VERSION]
                    [--user USER] [--channel CHANNEL] [-l LOCKFILE]
                    [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                    [--lockfile-clean]
                    [--lockfile-overrides LOCKFILE_OVERRIDES] [-b BUILD]
                    [-r REMOTE | -nr] [-u [UPDATE]] [-pr PROFILE]
                    [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
```

(continues on next page)

(continued from previous page)

```
                    [-pr:a PROFILE_ALL] [-o OPTIONS] [-o:b OPTIONS_BUILD]
                    [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL] [-s SETTINGS]
                    [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                    [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                    [-c:h CONF_HOST] [-c:a CONF_ALL] [--build-require]
                    [-tf TEST_FOLDER] [-tm] [-bt BUILD_TEST]
                    [path]

Create a package.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py).
                        Defaults to current directory

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
```

(continues on next page)

(continued from previous page)

```
                        compatibility.py)
  --build-require       Whether the package being created is a build-require
                        (to be used as tool_requires() by other packages)
  -tf TEST_FOLDER, --test-folder TEST_FOLDER
                        Alternative test folder name. By default it is
                        "test_package". Use "" to skip the test stage
  -tm, --test-missing   Run the test_package checks only if the package is
                        built from source but not if it already existed (using
                        --build=missing)
  -bt BUILD_TEST, --build-test BUILD_TEST
                        Same as '--build' but only for the test_package
                        requires. By default if not specified it will take the
                        '--build' value if specified

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
```

(continues on next page)

(continued from previous page)

```
                      host context. Use -pr:b (--profile:build) to specify
                      the build context, or -pr:a (--profile:all) to specify
                      both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                      Apply the specified options. By default, or if
                      specifying -o:h (--options:host), it applies to the
                      host context. Use -o:b (--options:build) to specify
                      the build context, or -o:a (--options:all) to specify
                      both contexts at once. Example:
                      -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                      Apply the specified settings. By default, or if
                      specifying -s:h (--settings:host), it applies to the
                      host context. Use -s:b (--settings:build) to specify
                      the build context, or -s:a (--settings:all) to specify
                      both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                      -c:h (--conf:host), it applies to the host context.
                      Use -c:b (--conf:build) to specify the build context,
                      or -c:a (--conf:all) to specify both contexts at once.
                      Example:
                      -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL
```

The `conan create` command creates a package from the recipe specified in `path`.

This command will first **`export`** the recipe to the local cache and then build and create the package. If a `test_package` folder (you can change the folder name with the `-tf` argument or with the `test_package_folder` recipe attribute) is found, the command will run the consumer project to ensure that the package has been created correctly. Check *testing Conan packages* section to know more about how to test your Conan packages.

Tip: Sometimes you want to **skip/disable the test stage**. In that case you can skip/disable the test package stage by passing an empty value as the `-tf` argument:

```
$ conan create . --test-folder=""
```

You might also want to do `conan create . --build=missing` so the package is not built if a binary already exists in the servers. If you want to also avoid the `test_package` step when the binary already exists, you can apply the `conan create . --build=missing --test-missing`, and it will only launch the test-package when the binary is built from source.

Using conan create with build requirements

The `--build-require` argument allows to create the package using the configuration and settings of the "build" context, as it was a `build_require`. This feature allows to create packages in a way that is consistent with the way they will be used later.

```
$ conan create . --name=cmake --version=3.23.1 --build-require
```

Conan create output

The `conan create ... --format=json` creates a json output containing the full dependency graph information. This json is the same as the one created with `conan graph info` (see the *graph info json format*) with extended information about the binaries, like a more complete `cpp_info` field. This resulting json is the dependency graph of the package recipe being created, excluding all the `test_package` and other possible dependencies of the `test_package/conanfile.py`. These dependencies only exist in the `test_package` functionality, and as such, are not part of the "main" product or package. If you are interested in capturing the dependency graph including the `test_package` (most likely not necessary in most cases), then you can do it running the `conan test` command separately.

The same happens for lockfiles created with `--lockfile-out` argument. The lockfile will only contain the created package and its transitive dependencies versions, but it will not contain the `test_package` or the transitive dependencies of the `test_package/conanfile.py`. It is possible to capture a lockfile which includes those with the `conan test` command (though again, this might not be really necessary)

Note: Best practice

In general, having `test_package/conanfile.py` with dependencies other than the tested one should be avoided. The `test_package` functionality should serve as a simple check to ensure the package is correctly created. Adding extra dependencies to `test_package` might indicate that the check is not straightforward or that its functionality is being misused. If, for any reason, your `test_package` has additional dependencies, you can control their build using the `--build-test` argument.

Methods execution order

The `conan create` executes *methods* of a *conanfile.py* in the following order:

1. **Export recipe to the cache**
    1. `init()`
    2. `set_name()`
    3. `set_version()`
    4. `export()`
    5. `export_sources()`
2. **Compute dependency graph**
    1. `init()`
    2. `config_options()`
    3. `configure()`
    4. `requirements()`
    5. `build_requirements()`

3. **Compute necessary packages**
    1. `validate_build()`
    2. `validate()`
    3. `package_id()`
    4. `layout()`
    5. `system_requirements()`
    6. `build_system_requirements()`
4. **Install packages**
    1. `source()`
    2. `build_id()`
    3. `generate()`
    4. `build()`
    5. `package()`
    6. `package_info()`

Steps `generate()`, `build()`, `package()` from *Install packages* step will not be called if the package is not being built from sources. Likewise, `build_system_requirements()` is only called for packages that are being built from sources.

After that, if you have a folder named *test_package* in your project or you call the `conan create` command with the `--test-folder` flag, the command will invoke the methods of the *conanfile.py* file inside the folder in the following order:

1. **Launch test_package**
    1. (test package) `init()`
    2. (test package) `set_name()`
    3. (test package) `set_version()`
2. **Compute dependency graph**
    1. (test package) `config_options()`
    2. (test package) `configure()`
    3. (test package) `requirements()`
    4. (test package) `build_requirements()`
    5. `init()`
    6. `config_options()`
    7. `configure()`
    8. `requirements()`
    9. `build_requirements()`
3. **Compute necessary packages**
    1. `validate_build()`
    2. `validate()`

3. `package_id()`
4. `layout()`
5. (test package) `validate_build()`
6. (test package) `validate()`
7. (test package) `package_id()`
8. (test package) `layout()`
9. `system_requirements()`
10. `build_system_requirements()`
11. (test package) `system_requirements()`
12. (test package) `build_system_requirements()`

4. **Install packages**
    1. `build_id()`
    2. `generate()`
    3. `build()`
    4. `package_info()`
5. **Test the package**
    1. (test package) `build()`
    2. (test package) `test()`

The functions with *(test package)* belong to the *conanfile.py* in the *test_package* folder. The steps `build_id()`, `generate()`, `build()` inside the *Install packages* step will be skipped if the project is already installed. Typically, it should be installed just as it was installed in the previous "install packages" step.

When using the `cmake_layout()` functionality inside `test_package`, the conf `tools.cmake.cmake_layout:test_folder` can be used to define the location of the build artifacts for the `test_package`. See *cmake_layout() docs*. Likewise, the full path to the build artifacts will be defined by the `self.folders.build_folder_vars` attribute.

Build modes

The `conan create --build=<xxxx>` build modes are very similar to the `conan install` ones documented in *Build Modes*, with some differences.

By default, `conan create` defines the `--build=current_pkg/current_version` to force the build from source for the current revision. This assumes that the source code (recipe, C/C++ code) was changed and it will create a new revision. If that is not the case, then the `--build=missing:current_pkg/current_version`, or `--build="missing:&"` would be recommended to avoid rebuilding from source an already existing binary.

When a `--build=xxx` argument is defined in the command line, then the automatically defined `--build=current_pkg/current_version` is no longer passed, and it should be passed as a explicit argument too.

Note: Best practices

Having more than a `package_revision` for a given `recipe_revision` and `package_id` is discouraged in most cases, as it implies unnecessarily rebuilding from sources binaries that were already existing. For that reason, using

`conan create` repeatedly over the same recipe without any source changes that would cause a new `recipe_revision` is discouraged, and using `conan create . --build=missing:[pattern]` would be the recommended approach.

See also:

- Read more about creating packages in the *dedicated tutorial*
- Read more about *testing Conan packages*

9.2.19 conan download

```
$ conan download -h
usage: conan download [-h] [-f FORMAT] [--out-file OUT_FILE]
                      [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                      [-cc CORE_CONF] [--only-recipe] [-p PACKAGE_QUERY] -r
                      REMOTE [-m METADATA] [-l LIST]
                      [pattern]

Download (without installing) a single conan package from a remote server.

It downloads just the package, but not its transitive dependencies, and it will not call
any generate, generators or deployers.
It can download multiple packages if patterns are used, and also works with
queries over the package binaries.

positional arguments:
  pattern               A pattern in the form
                        'pkg/version#revision:package_id#revision', e.g:
                        "zlib/1.2.13:*" means all binaries for zlib/1.2.13. If
                        revision is not specified, it is assumed latest one.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --only-recipe         Download only the recipe/s, not the binary packages.
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Only download packages matching a specific query. e.g:
                        os=Windows AND (arch=x86 OR compiler=gcc)
  -r REMOTE, --remote REMOTE
                        Download from this specific remote
  -m METADATA, --metadata METADATA
                        Download the metadata matching the pattern, even if
```

(continues on next page)

(continued from previous page)

```
                        the package is already in the cache and not downloaded
  -l LIST, --list LIST  Package list file
```

Downloads recipe and binaries to the local cache from the specified remote.

Note: Please, be aware that **conan download** unlike **conan install**, will not download any of the transitive dependencies of the downloaded package.

The `conan download` command can download packages to 1 server repository specified by the `-r=myremote` argument.

It has 2 possible and mutually exclusive inputs:

- The `conan download <pattern>` pattern-based matching of recipes, with a pattern similar to the `conan list <pattern>`.
- The `conan download --list=<pkglist>` that will download the artifacts specified in the `pkglist` json file

You can use patterns to download specific references just like in other commands like **conan list** (see the patterns documentation there *conan list*) or **conan upload**:

```
# download latest revision and packages for all openssl versions in foo remote
$ conan download "openssl/*" -r foo
```

Note: **conan download** will download only the latest revision by default. If you want to download more revisions other than the latest one you can use wildcards in the revisions part of the reference pattern argument

---

You may also just download recipes (in this case selecting all the revisions in the pattern, not just the latest one):

```
# download all recipe revisions for zlib/1.2.13
$ conan download "zlib/1.2.13#*" -r foo --only-recipe
```

If you just want to download the packages belonging to a specific setting, use the `--package-query` argument:

```
$ conan download "zlib/1.2.13#*" -r foo --package-query="os=Linux and arch=x86"
```

If the `--format=json` formatter is specified, the result will be a "PackageList", compatible with other Conan commands, for example the `conan upload` command, so it is possible to concatenate a `download + upload`, using the generated json file. See the *Packages Lists examples*.

Downloading metadata

The metadata files of the recipes and packages are not downloaded by default. It is possible to explicitly retrieve them with the `conan download --metadata=xxx` argument. The main arguments are the same as above, and Conan will download the specified packages, or skip them if they are already in the cache:

```
$ conan download pkg/0.1 -r=default --metadata="*"
# will download pgkg/0.1 recipe with all the recipe metadata
# And also all package binaries (latest package revision)
# with all the binaries metadata
```

If only one or several metadata folders or sets of files are desired, it can also be specified:

```
$ conan download pkg/0.1 -r=default --metadata="logs/*" --metadata="tests/*"
# Will download only the logs and tests metadata, but not other potential metadata files
```

For more information see the *metadata section*.

Note: *See here for examples of using package lists for downloading.*

9.2.20 conan editable

Allow working with a package that resides in user folder.

conan editable add

```
$ conan editable add -h
usage: conan editable add [-h] [--out-file OUT_FILE]
                          [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                          [-cc CORE_CONF] [--name NAME] [--version VERSION]
                          [--user USER] [--channel CHANNEL] [--ref REF]
                          [-of OUTPUT_FOLDER] [-r REMOTE | -nr]
                          [path]

Define the given <path> location as the package <reference>, so when this
package is required, it is used from this <path> location instead of the
cache.

positional arguments:
  path                  Path to the package folder in the user workspace

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --ref REF             Full package reference (e.g. pkg/1.0@user/channel), as
                        a shortcut for --name/--version/--user/--channel
  -of OUTPUT_FOLDER, --output-folder OUTPUT_FOLDER
                        The root output folder for generated and build files
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
```

(continues on next page)

(continued from previous page)

```
reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile
```

conan editable remove

```
$ conan editable remove -h
usage: conan editable remove [-h] [--out-file OUT_FILE]
                             [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                             [-cc CORE_CONF] [-r REFS]
                             [path]

Remove the "editable" mode for this reference.

positional arguments:
  path                  Path to a folder containing a recipe conanfile.py or
                        to a recipe file. e.g., ./my_project/conanfile.py.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -r REFS, --refs REFS  Directly provide reference patterns
```

conan editable list

```
$ conan editable list -h
usage: conan editable list [-h] [-f FORMAT] [--out-file OUT_FILE]
                           [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                           [-cc CORE_CONF]

List all the packages in editable mode.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
```

(continues on next page)

(continued from previous page)

```
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

See also:

- Read the tutorial about editable packages *editable package*.

9.2.21 conan export

```
$ conan export -h
usage: conan export [-h] [-f FORMAT] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF] [--name NAME] [--version VERSION]
                    [--user USER] [--channel CHANNEL] [-r REMOTE | -nr]
                    [-l LOCKFILE] [--lockfile-out LOCKFILE_OUT]
                    [--lockfile-partial] [--build-require]
                    [path]

Export a recipe to the Conan package cache.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py).
                        Defaults to current directory

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, pkglist
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -l LOCKFILE, --lockfile LOCKFILE
```

(continues on next page)

(continued from previous page)

```
                        Path to a lockfile.
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --build-require       Whether the provided reference is a build-require

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile
```

The `conan export` command exports the recipe specified in `path` to the Conan package cache.

Output Formats

The **`conan export`** command accepts two types of formats for the `--format` argument:

- `json`: Creates a JSON file containing the information of the exported recipe reference.
- `pkglist`: Generates a JSON file in the *pkglist* format, which can be utilized as input for various commands such as **`upload`**, **`download`**, and **`remove`**.

9.2.22 conan export-pkg

```
$ conan export-pkg -h
usage: conan export-pkg [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [-of OUTPUT_FOLDER] [--build-require]
                        [-tf TEST_FOLDER] [-sb] [-r REMOTE | -nr]
                        [--name NAME] [--version VERSION] [--user USER]
                        [--channel CHANNEL] [-l LOCKFILE] [--lockfile-partial]
                        [--lockfile-out LOCKFILE_OUT] [--lockfile-clean]
                        [--lockfile-overrides LOCKFILE_OVERRIDES]
                        [-pr PROFILE] [-pr:b PROFILE_BUILD]
                        [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL] [-o OPTIONS]
                        [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                        [-o:a OPTIONS_ALL] [-s SETTINGS] [-s:b SETTINGS_BUILD]
                        [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL] [-c CONF]
                        [-c:b CONF_BUILD] [-c:h CONF_HOST] [-c:a CONF_ALL]
                        [path]

Create a package directly from pre-compiled binaries.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py).
                        Defaults to current directory
```

(continues on next page)

(continued from previous page)

```
options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -of OUTPUT_FOLDER, --output-folder OUTPUT_FOLDER
                        The root output folder for generated and build files
  --build-require       Whether the provided reference is a build-require
  -tf TEST_FOLDER, --test-folder TEST_FOLDER
                        Alternative test folder name. By default it is
                        "test_package". Use "" to skip the test stage
  -sb, --skip-binaries  Skip installing dependencies binaries
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
```

(continues on next page)

(continued from previous page)

```
-pr:h PROFILE_HOST, --profile:host PROFILE_HOST
-pr:a PROFILE_ALL, --profile:all PROFILE_ALL
-o OPTIONS, --options OPTIONS
                      Apply the specified options. By default, or if
                      specifying -o:h (--options:host), it applies to the
                      host context. Use -o:b (--options:build) to specify
                      the build context, or -o:a (--options:all) to specify
                      both contexts at once. Example:
                      -o="pkg/*:with_qt=True"
-o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
-o:h OPTIONS_HOST, --options:host OPTIONS_HOST
-o:a OPTIONS_ALL, --options:all OPTIONS_ALL
-s SETTINGS, --settings SETTINGS
                      Apply the specified settings. By default, or if
                      specifying -s:h (--settings:host), it applies to the
                      host context. Use -s:b (--settings:build) to specify
                      the build context, or -s:a (--settings:all) to specify
                      both contexts at once. Example: -s="compiler=gcc"
-s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
-s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
-s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
-c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                      -c:h (--conf:host), it applies to the host context.
                      Use -c:b (--conf:build) to specify the build context,
                      or -c:a (--conf:all) to specify both contexts at once.
                      Example:
                      -c="tools.cmake.cmaketoolchain:generator=Xcode"
-c:b CONF_BUILD, --conf:build CONF_BUILD
-c:h CONF_HOST, --conf:host CONF_HOST
-c:a CONF_ALL, --conf:all CONF_ALL
```

The `conan export-pkg` command creates a package binary directly from pre-compiled binaries in a user folder. This command can be useful in different cases:

- When creating a package for some closed source or pre-compiled binaries provided by a vendor. In this case, it is not necessary that the `conanfile.py` recipe contains a `build()` method, and providing the `package()` and `package_info()` method are enough to package those pre-compiled binaries. In this case the `build_policy = "never"` could make sense to indicate it is not possible to `conan install --build=this_pkg`, as it doesn't know how to build from sources when it is a dependency.
- When testing some recipe locally in the *local development flow*, it can be used to quickly put the locally built binaries in the cache to make them available to other packages for testing, without needing to go through a full `conan create` that would be slower.

In general, it is expected that when `conan export-pkg` executes, the possible Conan dependencies that were necessary to build this package had already been installed via `conan install`, so it is not necessary to download dependencies at `export-pkg` time. But if for some reason this is not the case, the command defines `--remote` and `--no-remote` arguments, similar to other commands, as well as the `--skip-binaries` optimization that could save some time installing dependencies binaries if they are not strictly necessary for the current `export-pkg`. But this is the responsibility of the user, as it is possible that such binaries are actually necessary, for example, if a `tool_requires = "cmake/x.y"` is used and the `package()` method implements a `cmake.install()` call, this will definitely need the binaries for the dependencies installed in the current machine to execute.

The `conan export-pkg` is a package creation command, it will create both a new recipe and a new package binary, in the same way that the `conan create` command does. Similarly, it will run after the creation of the package any

"test-package" functionality. If there is a `test_package` folder besides the `conanfile.py`, or a different test-package folder is defined via the `--test-folder/-tf` argument or in the recipe `test_package_folder` attribute, then, such test-package will be triggered to test and validate that the created package is usable by the simple consumer project in the test-package folder.

See also:

- Check the *JSON format output* for this command.
- Read the tutorial about the *local package development flow*.

9.2.23 conan new

Create a new recipe (with a conanfile.py and other associated files) from either a predefined or a user-defined template.

conan new

```
$ conan new -h
usage: conan new [-h] [--out-file OUT_FILE]
                 [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                 [-cc CORE_CONF] [-d DEFINE] [-f] [-o OUTPUT]
                 [template]

Create a new example recipe and source files from a template.

positional arguments:
  template              Template name, either a predefined built-in or a user-
                        provided one. Available built-in templates: basic,
                        cmake_lib, cmake_exe, header_lib, meson_lib,
                        meson_exe, msbuild_lib, msbuild_exe, bazel_lib,
                        bazel_exe (Bazel >= 7.2, Bzlmod), autotools_lib,
                        autotools_exe, premake_lib, premake_exe,
                        local_recipes_index, workspace. E.g. 'conan new
                        cmake_lib -d name=hello -d version=0.1'. You can
                        define your own templates too by inputting an absolute
                        path as your template, or a path relative to your
                        conan home folder.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -d DEFINE, --define DEFINE
                        Define a template argument as key=value, e.g., -d
                        name=mypkg
```

(continues on next page)

(continued from previous page)

```
-f, --force           Overwrite file if it already exists
-o OUTPUT, --output OUTPUT
                      Output folder for the generated files
```

The `conan new` command creates a new recipe in the current working directory, plus extra example files such as *CMakeLists.txt* or the *test_package* folder (as necessary), to either be used as a basis for your own project or aiding in the debugging process.

Note that each template has some required and some [optional] user-defined variables used to customize the resulting files.

The available templates are:

- *default* (no argument is required): Creates a simple and empty CMake consumer recipe. The use case would be an user having a local CMakeLists.txt and the sources, and want to build them likely using requirements through Conan:

  Its variables are: name, version, [requires1, requires2, ...], [tool_requires1, tool_requires2, ...]

- **basic**: Creates a simple recipe with some example code and helpful comments, and is a good starting point to avoid writing boilerplate code.

  Its variables are: [name], [version], [description], [requires1, requires2, ...], [tool_requires1, tool_requires2, ...]

- **alias**: Creates the minimal recipe needed to define an alias to a target recipe

  Its variables are: name, [version], target

- **cmake_lib**: Creates a cmake library target that defines a function called `name`, which will print some information about the compilation environment to stdout. You can add requirements to this template in the form of

  ```
  conan new cmake_lib -d name=ai -d version=1.0 -d requires=math/3.14 -d
  requires=magic/0.0
  ```

  This will add requirements for both `math/3.14` and `magic/0.0` to the *requirements()* method, will add the necessary `find_package``s in CMake, and add a call to ``math()` and `magic()` inside the generated `ai()` function.

  Its variables are: name, version, [requires1, requires2, ...], [tool_requires1, tool_requires2, ...]

- **cmake_exe**: Creates a cmake executable target that defines a function called `name`, which will print some information about the compilation environment to stdout. You can add requirements to this template in the form of

  ```
  conan new cmake_exe -d name=game -d version=1.0 -d requires=math/3.14 -d
  requires=ai/1.0
  ```

  This will add requirements for both `math/3.14` and `ai/1.0` to the *requirements()* method, will add the necessary `find_package``s in CMake, and add a call to ``math()` and `ai()` inside the generated `game()` function.

  Its variables are: name, version, [requires1, requires2, ...], [tool_requires1, tool_requires2, ...]

- **header_lib** Creates a header-only library that defines a function called `name`, which will print some output to stdout.

  You can add requirements to this template in the form of

  ```
  conan new header_lib -d name=foo -d version=1.0 -d requires=math/3.14 -d
  requires=magic/0.0
  ```

This will add requirements for both `math/3.14` and `ai/1.0` to the *requirements()* method, and add a call to `math()` and `ai()` inside the generated `foo()` function.

Its variables are: name, version, [requires1, requires2, ...]

- **autotools_lib**: Creates an Autotools library.

  Its variables are: `name`, `version`

- **autotools_exe**: Creates an Autotools executable

  Its variables are: `name`, `version`

- **bazel_lib**: **Bazel integration BazelDeps, BazelToolchain, Bazel is experimental**. Creates a Bazel library. Since Conan 2.34, it is the same as **bazel_7_lib**. It generates a *MODULE.bazel* based project that requires Bazel >= 7.2.

  Its variables are: `name`, `version`

- **bazel_exe**: **Bazel integration BazelDeps, BazelToolchain, Bazel is experimental**. Creates a Bazel executable. Since Conan 2.34, it is the same as **bazel_7_exe**. It generates a *MODULE.bazel* based project that requires Bazel >= 7.2.

  Its variables are: `name`, `version`

- **meson_lib**: Creates a Meson library.

  Its variables are: `name`, `version`

- **meson_exe**: Creates a Meson executable

  Its variables are: `name`, `version`

- **msbuild_lib**: Creates a MSBuild library.

  Its variables are: `name`, `version`

- **msbuild_exe**: Creates a MSBuild executable

  Its variables are: `name`, `version`

- **workspace**: Creates a ready-to-use workspace containing three editables: **liba**, **libb** (requires liba) and **app1** (requires libb), plus the top-level `CMakeLists.txt`, `conanws.yml` and `conanws.py` that describe the workspace.

  You can pass a `requires` variable like `-d requires=mymath/0.1` to add an external dependency to **liba**.

By default, all `name` and `version` variables are set to `mypkg` and `0.1`, respectively, if not provided by the user.

Warning: The output of the predefined built-in templates is **not stable**. It might change in future releases to adapt to the latest tools or good practices.

Examples

```
$ conan new
```

Generates a simple CMake consumer *conanfile.py*. Notice that neither the CMakeLists.txt nor the sources are created.

Note: You could be interested in the *conan require* command to add some remote/local requirements to your recipe.

```
$ conan new basic
```

Generates a basic *conanfile.py* that does not implement any custom functionality

```
$ conan new basic -d name=mygame -d requires=math/1.0 -d requires=ai/1.3
```

Generates a *conanfile.py* for `mygame` that depends on the packages `math/1.0` and `ai/1.3`

```
$ conan new cmake_lib
```

Creates a basic CMake library with default package `name = "mypkg"` and default package version `version = "0.1"`

```
$ conan new cmake_exe -d name=game -d version=1.0 -d requires=math/3.14 -d requires=ai/1.
↪0
```

Generates the necessary files for a CMake executable target. This will add requirements for both `math/3.14` and `ai/1.0` to the `requirements()` method, will add the necessary `find_package` in CMake, and add a call to `math()` and `ai()` inside the generated `game()` function.

Custom templates

There's also the possibility of creating your templates. Templates in the Conan home should be located in the `templates/command/new` folder, and each template should have a folder named like the template one. If we create the `templates/command/new/mytemplate` folder, the command will be called with the following:

```
$ conan new mytemplate
```

As with other files in the Conan home, you can manage these templates with `conan config install <url>`, putting them in a git repo or an http server and sharing them with your team. It is also possible to use templates from any folder, just passing the full path to the template in the `conan new <full_path>`, but in general it is more convenient to manage them in the Conan home.

The folder can contain as many files as desired. Both the filenames and the contents of the files can be templatized using Jinja2 syntax. The command `-d/--define` arguments will define the `key=value` inputs to the templates.

The file contents will be like (Jinja2 syntax):

```
# File "templates/command/new/mytemplate/conanfile.py"
from conan import ConanFile

class Conan(ConanFile):
    name = "{{name}}"
    version = "{{version}}"
    license = "{{license}}"
```

And it will require passing these values:

```
$ conan new mytemplate -d name=pkg -d version=0.1 -d license=MIT
```

and it will generate in the current folder a file:

```
# File "<cwd>/conanfile.py"
from conan import ConanFile

```

(continues on next page)

(continued from previous page)

```
class Conan(ConanFile):
    name = "pkg"
    version = "0.1"
    license = "MIT"
```

There are some special `-d/--defines` names. The `name` one is always mandatory. The `conan_version` definition will always be automatically defined. The `requires` and `tool_requires` definitions, if existing, will be automatically converted to lists. The `package_name` will always be defined, by default equals to `name`.

For parametrized filenames, the filenames themselves support Jinja2 syntax. For example if we store a file named literally `{{name}}` with the brackes in the template folder `templates/command/new/mytemplate/`, instead of the `conanfile.py` above:

Listing 27: File: "templates/command/new/mytemplate/{{name}}"

```
{{contents}}
```

Then, executing

```
$ conan new mytemplate -d name=file.txt -d contents=hello!
```

will create a file called `file.txt` in the current dir containing the string `hello!`.

If there are files in the template not to be rendered with Jinja2, like image files, then their names should be added to a file called `not_templates` inside the template directory, one filename per line. So we could have a folder with:

```
templates/command/new/mytemplate
                        |- not_templates
                        |- conanfile.py
                        |- image.png
                        |- image2.png
```

And the `not_templates` contains the string `*.png`, then `conan new mytemplate ...` will only render the `conanfile.py` through Jinja2, but both images will be copied as-is.

9.2.24 conan source

```
$ conan source -h
usage: conan source [-h] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF] [-l LOCKFILE] [--lockfile-partial]
                    [--name NAME] [--version VERSION] [--user USER]
                    [--channel CHANNEL]
                    [path]

Call the source() method.

positional arguments:
  path                  Path to a folder containing a conanfile.py. Defaults
                        to current directory

options:
  -h, --help            show this help message and exit
```

(continues on next page)

(continued from previous page)

```
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile for python-requires resolution. Use
                        --lockfile="" to avoid automatic use of existing
                        'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile
```

See also:

- Read the tutorial about the *local package development flow*.

9.2.25 conan test

```
$ conan test -h
usage: conan test [-h] [-f FORMAT] [--out-file OUT_FILE]
                  [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                  [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr] [-u [UPDATE]]
                  [-pr PROFILE] [-pr:b PROFILE_BUILD] [-pr:h PROFILE_HOST]
                  [-pr:a PROFILE_ALL] [-o OPTIONS] [-o:b OPTIONS_BUILD]
                  [-o:h OPTIONS_HOST] [-o:a OPTIONS_ALL] [-s SETTINGS]
                  [-s:b SETTINGS_BUILD] [-s:h SETTINGS_HOST]
                  [-s:a SETTINGS_ALL] [-c CONF] [-c:b CONF_BUILD]
                  [-c:h CONF_HOST] [-c:a CONF_ALL] [-l LOCKFILE]
                  [--lockfile-partial] [--lockfile-out LOCKFILE_OUT]
                  [--lockfile-clean] [--lockfile-overrides LOCKFILE_OVERRIDES]
                  [path] reference

Test a package from a test_package folder.

positional arguments:
  path                  Path to a test_package folder containing a
                        conanfile.py. Defaults to a 'test_package' folder in
                        the current directory
  reference             Provide a package reference to test
```

(continues on next page)

(continued from previous page)

```
options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
```

(continues on next page)

(continued from previous page)

```
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
```

(continues on next page)

(continued from previous page)

```
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan test` command uses the *test_package* folder specified in `path` to tests the package reference specified in `reference`.

When using the `cmake_layout()` functionality inside `test_package`, the conf `tools.cmake.cmake_layout:test_folder` can be used to define the location of the build artifacts for the `test_package`. See *cmake_layout() docs*. Likewise, the full path to the build artifacts will be defined by the `self.folders.build_folder_vars` attribute.

- **tools.cmake.cmake_layout:test_folder** (*new since Conan 2.2.0*)(*experimental*) uses its value as the base folder of the `conanfile.folders.build` for test_package builds. If that value is `$TMP`, Conan will create and use a temporal folder.

See also:

- Read the tutorial about *testing Conan packages*.

9.2.26 conan upload

Use this command to upload recipes and binaries to Conan repositories. For more information on how to work with Conan repositories, please check the *dedicated section*.

```
$ conan upload -h
usage: conan upload [-h] [-f FORMAT] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF] [-p PACKAGE_QUERY] -r REMOTE
                    [--only-recipe] [--force] [--check] [-c] [--dry-run]
                    [--allow-disabled] [-l LIST] [-m METADATA]
                    [pattern]

Upload packages to a remote.

By default, all the matching references are uploaded (all revisions).
By default, if a recipe reference is specified, it will upload all the revisions for all␣
↪the
binary packages, unless --only-recipe is specified. You can use the "latest" placeholder␣
↪at the
"reference" argument to specify the latest revision of the recipe or the package.

positional arguments:
  pattern               A pattern in the form
                        'pkg/version#revision:package_id#revision', e.g:
                        "zlib/1.2.13:*" means all binaries for zlib/1.2.13. If
                        revision is not specified, it is assumed latest one.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
```

(continues on next page)

(continued from previous page)

```
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -p PACKAGE_QUERY, --package-query PACKAGE_QUERY
                        Only upload packages matching a specific query. e.g:
                        os=Windows AND (arch=x86 OR compiler=gcc)
  -r REMOTE, --remote REMOTE
                        Upload to this specific remote
  --only-recipe         Upload only the recipe/s, not the binary packages.
  --force               Force the upload of the artifacts even if the revision
                        already exists in the server
  --check               Perform an integrity check, using the manifests,
                        before upload
  -c, --confirm         Upload all matching recipes without confirmation
  --dry-run             Do not execute the real upload (experimental)
  --allow-disabled      Allow uploading to disabled remote
  -l LIST, --list LIST  Package list file
  -m METADATA, --metadata METADATA
                        Upload the metadata, even if the package is already in
                        the server and not uploaded
```

The `conan upload` command can upload packages to 1 server repository specified by the `-r=myremote` argument.

It has 2 possible and mutually exclusive inputs: - The `conan upload <pattern>` pattern-based matching of recipes, with a pattern similar to the `conan list <pattern>`. - The `conan upload --list=<pkglist>` that will upload the artifacts specified in the `pkglist` json file

If the `--format=json` formatter is specified, the result will be a "PackageList", compatible with other Conan commands, for example the `conan remove` command, so it is possible to concatenate different commands using the generated json file. The resulting "PackageList" also includes the URLs where each file has been or will be uploaded, providing additional context for automation or inspection purposes. See the *Packages Lists examples*.

The `--dry-run` argument will prepare the packages for upload, zip files if necessary, check in the server to see what needs to be uploaded and what is already in the server, but it will not execute the actual upload.

Note: The `--dry-run` flag is useful as the resulting package list json can be used to feed a `conan upload --list=<pkglist>` call, allowing for the split of the preparation of artifacts step and the actual upload step, which can be useful in CI pipelines.

```
# Recipes are loaded and artifacts prepared, but no upload is performed
$ conan upload <pattern> --dry-run -r=myremote -f=json > pkglist.json
# Nothing is re-prepared, only the actual upload is performed
$ conan upload --list=pkglist.json -r=myremote
```

Upload policies and efficient uploads

The `conan upload` commmand performs a check in the server to see if the local Conan packages in the cache already exist in the server or not. This is done by comparing the local `recipe-revision` and `package-revision` against the existing server ones. If they already exist in the server, the actual upload can be skipped, as the revision system uses the artifacts checksums, so it is guaranteed that the same artifacts already exist in the server.

If for some reason it is desired to force the full transfer of the artifacts from the local filesystem to the server again, and assuming the server has overwrite/delete permissions (necessary for an overwrite), then the `conan upload --force` can be used. That will force a new upload to the server.

Uploading an older existing revision to the server with `--force` doesn't guarantee that such a revision will be made the latest one in the server, that is to update its timestamp to the current time. This behavior might depend on the server configuration. For example in **Artifactory** the default configuration, due to historic reasons and Conan 1.X compatibility the behavior is as follows:

- If the `conan upload --force` happens before 60 seconds of the original upload, it is not made latest.
- If the `conan upload --force` happens after 60 seconds of the original upload, it is made latest.

The time limit can be configured with `artifactory.conan.index.timestamp.override.threshold.millis`, for example, to completely opt-out of this behavior and `conan upload --force` not changing the revision timestamp and consequently never making it the latest, it is possible to define `artifactory.conan.index.timestamp.override.threshold.millis=Long.MAX_VALUE`.

Note: In general, the `conan upload --force` argument shouldn't be used in regular production pipelines. It is more intended for exceptional cases, like fixing some corrupted package.

Upload configurations

There are different configurations and parameters that can affect the uploads:

- The recipe `upload_policy = "skip"` attribute is intended to skip the upload of binaries for that package, only the recipe will ever be uploaded. This attribute is used for exceptional cases where a package can only be built in the installation machine, for example "system" package wrappers
- `core.scm:local_url`: By default allows to store local folders as remote url, but not upload them. Use 'allow' for allowing upload and 'block' to completely forbid it. By default `scm` captures that are not reproducible, that is, that point to a local folder, will be blocked at upload time. This configuration can avoid that block, but please note that this is not recommended in the general case, as those packages won't be able to be reproduce later, as their sources are pointing to a local machine folder that will dissapear.
- `core.sources:upload_url`: Remote URL to upload backup sources to. If the "backup-sources" system is configured with this URL, then the `conan upload` command will also upload the associated downloaded sources to this backup-sources repository.
- `core.upload:compression_format`: The compression format used when uploading Conan packages. Possible values: 'zst', 'xz', 'gz' (default=gz)`. Recall that `zst` requires at least Python>=3.14 to work. With this configuration, it is possible to change the compression format for Conan stored artifacts.
- `core.upload:parallel`: Number of concurrent threads to upload packages. Using this conf, it is possible to upload packages in parallel. By default, or when set to a value less than 2, no parallelization will take place, and any other value will be the number of parallel threads to utilize.
- `core.upload:retry`: (int, default: 1) Number of retries in case of failure when uploading to Conan server
- `core.upload:retry_wait`: (int, default: 5s) Seconds to wait between upload attempts to Conan server

See also:

- *Uploading packages tutorial*
- *Working with Conan repositories*
- *Managing remotes with conan remote command*
- *Uploading metadata files*.
- *conan build*: Install package and call its build method
- *conan create*: Create a package from a recipe
- *conan download*: Download (without install) a single conan package from a remote server.
- *conan editable*: Allows working with a package in user folder
- *conan export*: Export a recipe to the Conan package cache
- *conan export-pkg*: Create a package directly from pre-compiled binaries
- *conan new*: Create a new recipe from a predefined template
- *conan source*: Calls the source() method
- *conan test*: Test a package
- *conan upload*: Upload packages from the local cache to a specified remote

Security Commands

9.2.27 conan audit

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

New feature in Conan 2.14.0

The `conan audit` command is used to check for known vulnerabilities in your Conan packages.

See *the audit devops page* to see examples on how to use the `conan audit` command.

conan audit scan

```
$ conan audit scan -h
usage: conan audit scan [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [-b BUILD] [-r REMOTE | -nr]
                        [-u [UPDATE]] [-pr PROFILE] [-pr:b PROFILE_BUILD]
                        [-pr:h PROFILE_HOST] [-pr:a PROFILE_ALL] [-o OPTIONS]
                        [-o:b OPTIONS_BUILD] [-o:h OPTIONS_HOST]
                        [-o:a OPTIONS_ALL] [-s SETTINGS] [-s:b SETTINGS_BUILD]
                        [-s:h SETTINGS_HOST] [-s:a SETTINGS_ALL] [-c CONF]
                        [-c:b CONF_BUILD] [-c:h CONF_HOST] [-c:a CONF_ALL]
                        [--requires REQUIRES] [--tool-requires TOOL_REQUIRES]
                        [--name NAME] [--version VERSION] [--user USER]
```

(continues on next page)

(continued from previous page)

```
                        [--channel CHANNEL] [-l LOCKFILE] [--lockfile-partial]
                        [--lockfile-out LOCKFILE_OUT] [--lockfile-clean]
                        [--lockfile-overrides LOCKFILE_OVERRIDES]
                        [--build-require] [-sl SEVERITY_LEVEL]
                        [--context {host,build}] [-p PROVIDER]
                        [path]

Scan a given recipe for vulnerabilities in its dependencies.

positional arguments:
  path                  Path to a folder containing a recipe (conanfile.py or
                        conanfile.txt) or to a recipe file. e.g.,
                        ./my_project/conanfile.txt. Defaults to the current
                        directory when no --requires or --tool-requires is
                        given

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -b BUILD, --build BUILD
                        Optional, specify which packages to build from source.
                        Combining multiple '--build' options on one command
                        line is allowed. Possible values: --build=never
                        Disallow build for all packages, use binary packages
                        or fail if a binary package is not found, it cannot be
                        combined with other '--build' options. --build=missing
                        Build packages from source whose binary package is not
                        found. --build=cascade Build packages from source that
                        have at least one dependency being built from source.
                        --build=[pattern] Build packages from source whose
                        package reference matches the pattern. The pattern
                        uses 'fnmatch' style wildcards, so '--build="*"' will
                        build everything from source. --build=~[pattern]
                        Excluded packages, which will not be built from the
                        source, whose package reference matches the pattern.
                        The pattern uses 'fnmatch' style wildcards.
                        --build=missing:[pattern] Build from source if a
                        compatible binary does not exist, only for packages
                        matching pattern. --build=compatible:[pattern]
                        (Experimental) Build from source if a compatible
                        binary does not exist, and the requested package is
```

(continues on next page)

(continued from previous page)

```
                        invalid, the closest package binary following the
                        defined compatibility policies (method and
                        compatibility.py)
  --requires REQUIRES   Directly provide requires instead of a conanfile
  --tool-requires TOOL_REQUIRES
                        Directly provide tool-requires instead of a conanfile
  --build-require       Whether the provided reference is a build-require
  -sl SEVERITY_LEVEL, --severity-level SEVERITY_LEVEL
                        Set threshold for severity level to raise an error. By
                        default raises an error for any critical CVSS (9.0 or
                        higher). Use 100.0 to disable it.
  --context {host,build}
                        Context to scan, by default both contexts are scanned
                        if not specified
  -p PROVIDER, --provider PROVIDER
                        Provider to use for scanning

remote arguments:
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
  -nr, --no-remote      Do not use remote, resolve exclusively in the cache
  -u [UPDATE], --update [UPDATE]
                        Will install newer versions and/or revisions in the
                        local cache for the given references whose name
                        matches the given pattern, or all references in the
                        graph if no argument is supplied. When using version
                        ranges, it will install the latest version that
                        satisfies the range. It will update to the latest
                        revision for the resolved version range. The consumer
                        pattern (&) has no effect, and users should not
                        specify versions.

profile arguments:
  -pr PROFILE, --profile PROFILE
                        Apply the specified profile. By default, or if
                        specifying -pr:h (--profile:host), it applies to the
                        host context. Use -pr:b (--profile:build) to specify
                        the build context, or -pr:a (--profile:all) to specify
                        both contexts at once
  -pr:b PROFILE_BUILD, --profile:build PROFILE_BUILD
  -pr:h PROFILE_HOST, --profile:host PROFILE_HOST
  -pr:a PROFILE_ALL, --profile:all PROFILE_ALL
  -o OPTIONS, --options OPTIONS
                        Apply the specified options. By default, or if
                        specifying -o:h (--options:host), it applies to the
                        host context. Use -o:b (--options:build) to specify
                        the build context, or -o:a (--options:all) to specify
                        both contexts at once. Example:
                        -o="pkg/*:with_qt=True"
  -o:b OPTIONS_BUILD, --options:build OPTIONS_BUILD
  -o:h OPTIONS_HOST, --options:host OPTIONS_HOST
  -o:a OPTIONS_ALL, --options:all OPTIONS_ALL
```

(continues on next page)

(continued from previous page)

```
  -s SETTINGS, --settings SETTINGS
                        Apply the specified settings. By default, or if
                        specifying -s:h (--settings:host), it applies to the
                        host context. Use -s:b (--settings:build) to specify
                        the build context, or -s:a (--settings:all) to specify
                        both contexts at once. Example: -s="compiler=gcc"
  -s:b SETTINGS_BUILD, --settings:build SETTINGS_BUILD
  -s:h SETTINGS_HOST, --settings:host SETTINGS_HOST
  -s:a SETTINGS_ALL, --settings:all SETTINGS_ALL
  -c CONF, --conf CONF  Apply the specified conf. By default, or if specifying
                        -c:h (--conf:host), it applies to the host context.
                        Use -c:b (--conf:build) to specify the build context,
                        or -c:a (--conf:all) to specify both contexts at once.
                        Example:
                        -c="tools.cmake.cmaketoolchain:generator=Xcode"
  -c:b CONF_BUILD, --conf:build CONF_BUILD
  -c:h CONF_HOST, --conf:host CONF_HOST
  -c:a CONF_ALL, --conf:all CONF_ALL

reference arguments:
  --name NAME           Provide a package name if not specified in conanfile
  --version VERSION     Provide a package version if not specified in
                        conanfile
  --user USER           Provide a user if not specified in conanfile
  --channel CHANNEL     Provide a channel if not specified in conanfile

lockfile arguments:
  -l LOCKFILE, --lockfile LOCKFILE
                        Path to a lockfile. Use --lockfile="" to avoid
                        automatic use of existing 'conan.lock' file
  --lockfile-partial    Do not raise an error if some dependency is not found
                        in lockfile
  --lockfile-out LOCKFILE_OUT
                        Filename of the updated lockfile
  --lockfile-clean      Remove unused entries from the lockfile
  --lockfile-overrides LOCKFILE_OVERRIDES
                        Overwrite lockfile overrides
```

The `conan audit scan` command checks for vulnerabilities in the given references and their transitive dependencies. This command receives configuration arguments such as profiles and settings, to control the expansion of the graph.

conan audit list

```
$ conan audit list -h
usage: conan audit list [-h] [-f FORMAT] [--out-file OUT_FILE]
                        [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}
↪]]
                        [-cc CORE_CONF] [-l LIST] [-s SBOM] [-lock LOCKFILE]
                        [-r REMOTE] [-p PROVIDER]
                        [reference]
```

(continues on next page)

(continued from previous page)

```
List the vulnerabilities of the given reference.

positional arguments:
  reference             Reference to list vulnerabilities for

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  -l LIST, --list LIST  Package list file to list vulnerabilities for
  -s SBOM, --sbom SBOM  SBOM file to list vulnerabilities for
  -lock LOCKFILE, --lockfile LOCKFILE
                        Path to the lockfile to check for vulnerabilities
  -r REMOTE, --remote REMOTE
                        Remote to use for listing
  -p PROVIDER, --provider PROVIDER
                        Provider to use for scanning
```

The `conan audit list` command lists vulnerabilities for the given references, without checking their transitive dependencies. You can pass a single reference, a pkglist file with multiple references, a cyclonedx SBOM file generated with the *conan.tools.sbom* module, or a Conan lockfile.

conan audit provider

```
$ conan audit provider -h
usage: conan audit provider [-h] [-f FORMAT] [--out-file OUT_FILE]
                            [-v [{quiet,error,warning,notice,status,verbose,debug,v,
↪trace,vv}]]
                            [-cc CORE_CONF] [--url URL]
                            [--type {conan-center-proxy,private}]
                            [--token TOKEN]
                            {add,list,auth,remove} [name]

Manage security providers for the 'conan audit' command.

positional arguments:
  {add,list,auth,remove}
                        Action to perform from 'add', 'list' , 'remove' or
                        'auth'
  name                  Provider name
```

(continues on next page)

(continued from previous page)

```
options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
  --url URL             Provider URL
  --type {conan-center-proxy,private}
                        Provider type
  --token TOKEN         Provider token
```

The `conan audit provider` command manages the list of providers used to check for vulnerabilities.

By default the `conan audit` subcommands use the ConanCenter provider, but you can add your own providers to the list. For now, besides the default ConanCenter provider, only private JFrog Security providers are supported, see *the audit devops page* for more information.

There are 3 subcommands:

- `conan audit provider auth`: Authenticates a provider with a token.
- `conan audit provider add`: Adds a provider to the list.
- `conan audit provider remove`: Removes a provider from the list.

See also:

- Read more in the dedicated blog post.

9.2.28 conan report

The `conan report` command contains subcommands that return information about packages and libraries.

```
$ conan report -h
usage: conan report [-h] [--out-file OUT_FILE]
                    [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]]
                    [-cc CORE_CONF]
                    {diff} ...

Gets information about the recipe and its sources.

positional arguments:
  {diff}                sub-command help
    diff                Get the difference between two recipes with their
                        sources. It can be used to compare two different
                        versions of the same recipe, or two different recipe
                        revisions. Each old/new recipe can be specified by a
```

(continues on next page)

(continued from previous page)

```
                        path to a conanfile.py and a companion reference, or
                        by a reference only. If only a reference is specified,
                        it will be searched in the local cache, or downloaded
                        from the specified remotes. If no revision is
                        specified, the latest revision will be used.

options:
  -h, --help            show this help message and exit
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

conan report diff

```
$ conan report diff -h
usage: conan report diff [-h] [-f FORMAT] [--out-file OUT_FILE]
                         [-v [{quiet,error,warning,notice,status,verbose,debug,v,trace,
↪vv}]]
                         [-cc CORE_CONF] [-op OLD_PATH] -or OLD_REFERENCE
                         [-np NEW_PATH] -nr NEW_REFERENCE [-r REMOTE]

Get the difference between two recipes with their sources. It can be used to
compare two different versions of the same recipe, or two different recipe
revisions. Each old/new recipe can be specified by a path to a conanfile.py
and a companion reference, or by a reference only. If only a reference is
specified, it will be searched in the local cache, or downloaded from the
specified remotes. If no revision is specified, the latest revision will be
used.

options:
  -h, --help            show this help message and exit
  -f FORMAT, --format FORMAT
                        Select the output format: json, html
  --out-file OUT_FILE   Write the output of the command to the specified file
                        instead of stdout.
  -v [{quiet,error,warning,notice,status,verbose,debug,v,trace,vv}]
                        Level of detail of the output. Valid options from less
                        verbose to more verbose: -vquiet, -verror, -vwarning,
                        -vnotice, -vstatus, -v or -vverbose, -vv or -vdebug,
                        -vvv or -vtrace
  -cc CORE_CONF, --core-conf CORE_CONF
                        Define core configuration, overwriting global.conf
                        values. E.g.: -cc core:non_interactive=True
```

(continues on next page)

(continued from previous page)

```
  -op OLD_PATH, --old-path OLD_PATH
                        Path to the old recipe if comparing a local recipe is
                        desired
  -or OLD_REFERENCE, --old-reference OLD_REFERENCE
                        Old reference, e.g. 'mylib/1.0'. If used on its own,
                        it can contain a revision, which will be resolved to
                        the latest one if not provided, but it will be ignored
                        if a path is specified. If used with a path, it will
                        be used to create the reference for the recipe to be
                        compared.
  -np NEW_PATH, --new-path NEW_PATH
                        Path to the new recipe if comparing a local recipe is
                        desired
  -nr NEW_REFERENCE, --new-reference NEW_REFERENCE
                        New reference, e.g. 'mylib/1.0'. If used on its own,
                        it can contain a revision, which will be resolved to
                        the latest one if not provided, but it will be ignored
                        if a path is specified. If used with a path, it will
                        be used to create the reference for the recipe to be
                        compared.
  -r REMOTE, --remote REMOTE
                        Look in the specified remote or remotes server
```

The `conan report diff` command gets the differences between two recipes, also comparing their sources. This functionality allows you to compare either two versions of the same recipe or two entirely different recipes. Each recipe (old and new) can be identified in one of two ways: by providing both the path to its `conanfile.py` and its reference, or by specifying just the reference.

When only a reference is given, Conan will first search for the recipe in the local cache; if it is not found, it will attempt to download it from the configured remotes. If no revision is explicitly provided, Conan will default to using the latest available revision.

Examples

Remote Reference vs Remote Reference

If we want to compare versions 1.0 and 2.0 of *mylib* that are available on our *my-remote* remote, it would be:

```
$ conan report diff --old-reference="mylib/1.0" --new-reference="mylib/2.0" -r=my-remote
```

Remote Reference vs Local Reference

Let's suppose we're making changes to the recipe or adding a new version, and we want to compare our changes against a version that is in the remote. The version that is not on the remote requires the path to the recipe in order to compare it. If it's the old version that we're modifying and it's not found in the remotes, we would use `--old-path`:

```
$ conan report diff --old-reference="mylib/1.0" --old-path="path/to/recipe" --new-
↪reference="mylib/2.0"
```

If, on the other hand, it's the new version that we're modifying then we would use `--new-path`:

```
$ conan report diff --old-reference="mylib/1.0" --new-reference="mylib/2.0" --new-path=
↪"path/to/recipe"
```

Local Reference vs Local Reference

Finally, if we're modifying both versions, we'll need to provide both paths. They may or may not be the same.

```
$ conan report diff --old-reference="mylib/1.0" --old-path="path/to/recipe" --new-
↪reference="mylib/2.0" --new-path="path/to/recipe"
```

Specifying revision

The command allows you to specify the revision of the package you want to compare. By default, it uses the latest revision, but by providing a revision, you can target the exact package you want to compare. This makes it possible to do things like compare two identical versions with different revisions in order to check for differences between them.

```
$ conan report diff --old-reference="mylib/1.0#oldrev" --new-reference="mylib/1.0#newrev"
```

Available formatters

Text Formatter

By default, it displays this format, which is the format provided by a `git diff` between the packages.

JSON Formatter

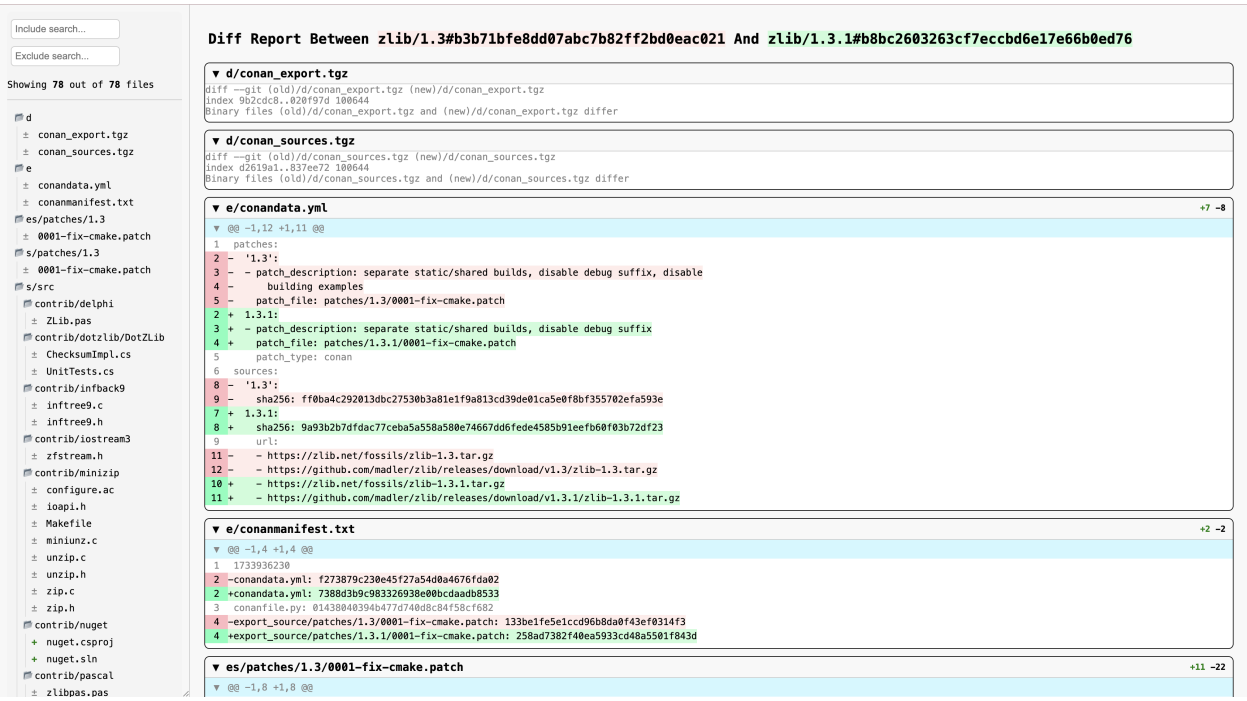
You can obtain the result in JSON format, providing a structured output that is perfect for consumption by other scripts.

```
$ conan report diff --old-reference="mylib/1.0" --new-reference="mylib/2.0" --format=json
```

HTML Formatter

The HTML format generates a small self-contained static web page in a single HTML file. This page lets you conveniently visualize the changes in the recipe as well as the changes in the source files of your libraries. It contains filters to include and exclude keywords and shortcuts to all the changed files.

```
$ conan report diff --old-reference="zlib/1.3" --new-reference="zlib/1.3.1" --
↪format=html > diff.html
```



- *conan audit*: Checks for vulnerabilities in your Conan packages.
- *conan report*: Get information about the packages

Commands Output to stdout and stderr

Conan commands output information following a deliberate design choice that aligns with common practices in many CLI tools and the POSIX standard:

- `stdout`: For final command results (e.g., JSON, HTML).
- `stderr`: For diagnostic output, including logs, warnings, errors, and progress messages.

More info can be found more in the *FAQ section*.

Redirecting Output to Files

You can redirect Conan output to files using shell redirection:

```
$ conan install . --format=json > output.json
```

Alternatively, use the `--out-file` argument (available since Conan 2.12.0) to specify an output file directly:

```
$ conan install . --format=json --out-file=output.json
```

9.2.29 Command formatters

Almost all the commands have the parameter `--format xxxx` which is used to apply an output conversion. The command formatters help users see the command output in a different way that could fit better with their needs. Here, there are only some of the most important ones whose details are worthy of having a separate section.

Formatter: Graph-info JSON

This section is aimed to show one example of the JSON format output when using any of these commands:

- `conan graph info xxxx --format=json`
- `conan create xxxx --format=json`
- `conan install xxxx --format=json`
- `conan export-pkg xxxx --format=json`

The output shows the graph information processed by Conan in each command.

The JSON output generated by **`conan graph info --require=zlib/1.2.11 -r=conancenter --format=json > graph.json`** for instance:

Listing 28: **graph.json**


```
{
    "graph": {
        "nodes": {
            "0": {
                "ref": "conanfile",
                "id": "0",
                "recipe": "Cli",
                "package_id": null,
                "prev": null,
                "rrev": null,
                "rrev_timestamp": null,
                "prev_timestamp": null,
                "remote": null,
                "binary_remote": null,
                "build_id": null,
                "binary": null,
                "invalid_build": false,
                "info_invalid": null,
                "name": null,
                "user": null,
                "channel": null,
                "url": null,
                "license": null,
                "author": null,
                "description": null,
                "homepage": null,
                "build_policy": null,
                "upload_policy": null,
                "revision_mode": "hash",
                "provides": null,
                "deprecated": null,
                "win_bash": null,
                "win_bash_run": null,
                "default_options": null,
                "options_description": null,
                "version": null,
                "topics": null,
```


(continues on next page)

(continued from previous page)


```
                "package_type": "unknown",
                "settings": {
                    "os": "Macos",
                    "arch": "x86_64",
                    "compiler": "apple-clang",
                    "compiler.cppstd": "gnu17",
                    "compiler.libcxx": "libc++",
                    "compiler.version": "12.0",
                    "build_type": "Release"
                },
                "options": {},
                "options_definitions": {},
                "generators": [],
                "python_requires": null,
                "system_requires": {},
                "recipe_folder": null,
                "source_folder": null,
                "build_folder": null,
                "generators_folder": null,
                "package_folder": null,
                "cpp_info": {
                    "root": {
                        "includedirs": [
                            "include"
                        ],
                        "srcdirs": null,
                        "libdirs": [
                            "lib"
                        ],
                        "resdirs": null,
                        "bindirs": [
                            "bin"
                        ],
                        "builddirs": null,
                        "frameworkdirs": null,
                        "system_libs": null,
                        "frameworks": null,
                        "libs": null,
                        "defines": null,
                        "cflags": null,
                        "cxxflags": null,
                        "sharedlinkflags": null,
                        "exelinkflags": null,
                        "objects": null,
                        "sysroot": null,
                        "requires": null,
                        "properties": null
                    }
                },
                "conf_info": {},
                "label": "cli",
                "dependencies": {
```


(continues on next page)

(continued from previous page)


```
                    "1": {
                        "ref": "zlib/1.2.11",
                        "run": false,
                        "libs": true,
                        "skip": false,
                        "test": false,
                        "force": false,
                        "direct": true,
                        "build": false,
                        "transitive_headers": null,
                        "transitive_libs": null,
                        "headers": true,
                        "package_id_mode": null,
                        "visible": true
                    }
                },
                "context": "host",
                "test": false
            },
            "1": {
                "ref": "zlib/1.2.11#ffa77daf83a57094149707928bdce823",
                "id": "1",
                "recipe": "Cache",
                "package_id": "d0599452a426a161e02a297c6e0c5070f99b4909",
                "prev": "1440f4f447208c8e6808936b4c6ff282",
                "rrev": "dc0e384f0551386cd76dc29cc964c95e",
                "rrev_timestamp": 1703667991.3458598,
                "prev_timestamp": 1703668372.8517942,
                "remote": null,
                "binary_remote": null,
                "build_id": null,
                "binary": "Missing",
                "invalid_build": false,
                "info_invalid": null,
                "name": "zlib",
                "user": null,
                "channel": null,
                "url": "https://github.com/conan-io/conan-center-index",
                "license": "Zlib",
                "author": null,
                "description": "A Massively Spiffy Yet Delicately Unobtrusive␣
↪Compression Library (Also Free, Not to Mention Unencumbered by Patents)",
                "homepage": "https://zlib.net",
                "build_policy": null,
                "upload_policy": null,
                "revision_mode": "hash",
                "provides": null,
                "deprecated": null,
                "win_bash": null,
                "win_bash_run": null,
                "default_options": {
                    "shared": false,
```


(continues on next page)

(continued from previous page)


```
                "fPIC": true
            },
            "options_description": null,
            "version": "1.2.11",
            "topics": [
                "zlib",
                "compression"
            ],
            "package_type": "static-library",
            "settings": {
                "os": "Macos",
                "arch": "x86_64",
                "compiler": "apple-clang",
                "compiler.cppstd": "gnu17",
                "compiler.libcxx": "libc++",
                "compiler.version": "12.0",
                "build_type": "Release"
            },
            "options": {
                "fPIC": "True",
                "shared": "False"
            },
            "options_definitions": {
                "shared": [
                    "True",
                    "False"
                ],
                "fPIC": [
                    "True",
                    "False"
                ]
            },
            "generators": [],
            "python_requires": null,
            "system_requires": {},
            "recipe_folder": "/Users/franchuti/.conan2/p/zlib774aa77541f8b/e",
            "source_folder": null,
            "build_folder": null,
            "generators_folder": null,
            "package_folder": null,
            "cpp_info": {
                "root": {
                    "includedirs": [
                        "include"
                    ],
                    "srcdirs": null,
                    "libdirs": [
                        "lib"
                    ],
                    "resdirs": null,
                    "bindirs": [
                        "bin"
```


(continues on next page)

(continued from previous page)


```
                        ],
                        "builddirs": null,
                        "frameworkdirs": null,
                        "system_libs": null,
                        "frameworks": null,
                        "libs": null,
                        "defines": null,
                        "cflags": null,
                        "cxxflags": null,
                        "sharedlinkflags": null,
                        "exelinkflags": null,
                        "objects": null,
                        "sysroot": null,
                        "requires": null,
                        "properties": null
                    }
                },
                "conf_info": {},
                "label": "zlib/1.2.11",
                "info": {
                    "settings": {
                        "os": "Macos",
                        "arch": "x86_64",
                        "compiler": "apple-clang",
                        "compiler.cppstd": "gnu17",
                        "compiler.libcxx": "libc++",
                        "compiler.version": "12.0",
                        "build_type": "Release"
                    },
                    "options": {
                        "fPIC": "True",
                        "shared": "False"
                    }
                },
                "vendor": false,
                "conandata": {
                    "patches": {
                        "1.2.11": [
                            {
                                "patch_description": "separate static/shared builds,␣
↪disable debug suffix, disable building examples",
                                "patch_file": "patches/1.2.x/0001-fix-cmake.patch",
                                "patch_type": "conan"
                            },
                            {
                                "patch_description": "fix condition for WIDECHAR usage",
                                "patch_file": "patches/1.2.x/0003-gzguts-fix-widechar-
↪condition.patch",
                                "patch_source": "https://github.com/madler/zlib/issues/
↪268",
                                "patch_type": "portability"
                            }
```


(continues on next page)

(continued from previous page)

```
                        ]
                    },
                    "sources": {
                        "1.2.11": {
                            "sha256":
↪"c3e5e9fdd5004dcb542feda5ee4f0ff0744628baf8ed2dd5d66f8ca1197cb1a1",
                            "url": "https://zlib.net/fossils/zlib-1.2.11.tar.gz"
                        }
                    }
                },
                "dependencies": {},
                "context": "host",
                "test": false
            }
        },
        "root": {
            "0": "None"
        },
        "overrides": {},
        "resolved_ranges": {},
        "replaced_requires": {}
    }
}
```

- *graph-info formatter*: Show the graph information in JSON format. It's used by several commands.

9.3 conanfile.py

The `conanfile.py` is the recipe file of a package, responsible for defining how to build it and consume it.

```
from conan import ConanFile

class HelloConan(ConanFile):
    ...
```

Important: *conanfile.py* recipes use a variety of attributes and methods to operate. In order to avoid collisions and conflicts, follow these rules:

- Public attributes and methods, like `build()`, `self.package_folder`, are reserved for Conan. Don't use public members for custom fields or methods in the recipes.
- Use "protected" access for your own members, like `self._my_data` or `def _my_helper(self):`. Conan only reserves "protected" members starting with `_conan`.

Contents:

9.3.1 Attributes

- *Package reference*
  - *name*
  - *version*
  - *user*
  - *channel*
- *Metadata*
  - *description*
  - *license*
  - *author*
  - *topics*
  - *homepage*
  - *url*
- *Requirements*
  - *requires*
  - *tool_requires*
  - *build_requires*
  - *test_requires*
  - *python_requires*
  - *python_requires_extend*
- *Sources*
  - *exports*
  - *exports_sources*
  - *conan_data*
  - *source_buildenv*
- *Binary model*
  - *package_type*
  - *settings*
  - *options*
  - *default_options*
  - *default_build_options*
  - *options_description*
  - *languages*
  - *info*
  - *package_id_{embed,non_embed,python,unknown}_mode, build_mode*

    - *package_id_abi_options*
    - *context*
- *Build*
    - *generators*
    - *build_policy*
    - *win_bash*
    - *win_bash_run*
- *Folders and layout*
    - *source_folder*
    - *export_sources_folder*
    - *build_folder*
    - *generators_folder*
    - *package_folder*
    - *recipe_folder*
    - *recipe_metadata_folder*
    - *package_metadata_folder*
    - *no_copy_source*
    - *test_package_folder*
- *Layout*
    - *folders*
    - *cpp*
    - *layouts*
- *Package information for consumers*
    - *cpp_info*
    - *buildenv_info*
    - *runenv_info*
    - *conf_info*
    - *generator_info*
    - *deprecated*
    - *provides*
    - *package_type_traits*
- *Other*
    - *dependencies*
    - *subgraph*
    - *conf*

- *Output*
- *Output contents*
- *revision_mode*
- *upload_policy*
- *required_conan_version*
- *implements*
- *alias*
- *extension_properties*

Package reference

Recipe attributes that can define the main `pkg/version@user/channel` package reference.

name

The name of the package. A valid name is all lowercase and has:

- A minimum of 2 and a maximum of 101 characters (though shorter names are recommended).
- **Matches the following regex `^[a-z0-9_][a-z0-9_+.-]{1,100}$`: so starts with alphanumeric or `_`,** then from 1 to 100 characters between alphanumeric, `_`, `+`, `.` or `-`.

The name is only necessary for `export`-ing the recipe into the local cache (`export`, `export-pkg` and `create` commands), if they are not defined in the command line with `--name=<pkgname>`.

version

The version of the package. A valid version follows the same rules than the `name` attribute. In case the version follows semantic versioning in the form `X.Y.Z-pre1+build2`, that value might be used for requiring this package through version ranges instead of exact versions.

The version is only strictly necessary for `export`-ing the recipe into the local cache (`export`, `export-pkg` and `create` commands), if they are not defined in the command line with `--version=<pkgversion>`

The `version` can be dynamically defined in the command line, and also programmatically in the recipe with the *set_version() method*.

user

A valid string for the `user` field follows the same rules than the `name` attribute. This is an optional attribute. It can be used to identify your own packages with `pkg/version@user/channel`, where `user` could be the name of your team, org or company. ConanCenter recipes don't have `user/channel`, so they are in the form of `pkg/version` only. You can also name your packages without user and channel, or using only the user as `pkg/version@user`.

The user can be specified in the command line with `--user=<myuser>`

channel

A valid string for the `channel` field follows the same rules than the `name` attribute. This is an optional attribute. It is sometimes used to identify a maturity of the package ("stable", "testing"...), but in general this is not necessary, and the maturity of packages is better managed by putting them in different server repositories.

The channel can be specified in the command line with `--channel=<mychannel>`. If a channel is specified, a user must also be specified, so the package reference is always complete as `pkg/version@user/channel`.

Metadata

Optional metadata, like license, description, author, etc. Not necessary for most cases, but can be useful to have.

description

This is an optional, but recommended text field, containing the description of the package, and any information that might be useful for the consumers. The first line might be used as a short description of the package.

```
class HelloConan(ConanFile):
    name = "hello"
    version = "0.1"
    description = """This is a Hello World library.
                     A fully featured, portable, C++ library to say Hello World in the␣
↪stdout,
                     with incredible iostreams performance"""
```

license

License of the **target** source code and binaries, i.e. the code that is being packaged, not the `conanfile.py` itself. Can contain several, comma separated licenses. It is a text string, so it can contain any text, but it is strongly recommended that recipes of Open Source projects use SPDX identifiers from the SPDX license list

This will help people wanting to automate license compatibility checks, like consumers of your package, or you if your package has Open-Source dependencies.

```
class Pkg(ConanFile):
    license = "MIT"
```

author

Main maintainer/responsible for the package, any format. This is an optional attribute.

```
class HelloConan(ConanFile):
    author = "John J. Smith (john.smith@company.com)"
```

topics

Tags to group related packages together and describe what the code is about. Used as a search filter in ConanCenter. Optional attribute. It should be a tuple of strings.

```
class ProtocInstallerConan(ConanFile):
    name = "protoc_installer"
    version = "0.1"
    topics = ("protocol-buffers", "protocol-compiler", "serialization", "rpc")
```

homepage

The home web page of the library being packaged.

Used to link the recipe to further explanations of the library itself like an overview of its features, documentation, FAQ as well as other related information.

```
class EigenConan(ConanFile):
    name = "eigen"
    version = "3.3.4"
    homepage = "http://eigen.tuxfamily.org"
```

url

URL of the package repository, i.e. not necessarily of the original source code. Recommended, but not mandatory attribute.

```
class HelloConan(ConanFile):
    name = "hello"
    version = "0.1"
    url = "https://github.com/conan-io/libhello.git"
```

Requirements

Attribute form of the dependencies simple declarations, like `requires`, `tool_requires`. For more advanced way to define requirements, use the `requirements()`, `build_requirements()` methods instead.

requires

List or tuple of strings for regular dependencies in the host context, like a library.

```
class MyLibConan(ConanFile):
    requires = "hello/1.0", "otherlib/2.1@otheruser/testing"
```

You can specify version ranges, the syntax is using brackets:

```
class HelloConan(ConanFile):
    requires = "pkg/[>1.0 <1.8]"
```

Accepted expressions would be:

Expression	Versions in range	Versions outside of range
[>=1.0 <2]	1.0.0, 1.0.1, 1.1, 1.2.3	0.2, 2.0, 2.1, 3.0
[<3.2.1]	0.1, 1.2, 2.4, 3.1.1	3.2.2
[>2.0]	2.1, 2.2, 3.1, 14.2	1.1, 1.2, 2.0

The caret `^` and tilde `~` operators are basically compact representations of lower and upper bounds:

- The `[~2.5.1]` range could be written as `[>=2.5.1 <2.6.0]`
- The `[^1.2.3]` range could be written as `[>=1.2.3 <2.0.0]`

In general, it is recommended to use the full expression `[>=lower <upper]` instead of the caret or tilde shortcuts, as it is more explicit and evident for all readers what the valid versions are in that range.

If pre-releases are activated, like defining configuration `core.version_ranges:resolve_prereleases=True`:

Expression	Versions in range	Versions outside of range
[>=1.0 <2]	1.0.0-pre.1, 1.0.0, 1.0.1, 1.1, 1.2.3	0.2, 2.0-pre.1, 2.0, 2.1, 3.0
[<3.2.1]	0.1, 1.2, 1.8-beta.1, 2.0-alpha.2, 2.4, 3.1.1	3.2.1-pre.1, 3.2.1, 3.2.2, 3.3
[>2.0]	2.1-pre.1, 2.1, 2.2, 3.1, 14.2	1.1, 1.2, 2.0-pre.1, 2.0

See also:

- Check *Range expressions* version_ranges tutorial section
- Check *requirements()* method docs

tool_requires

List or tuple of strings for dependencies. Represents a build tool like "cmake". If there is an existing pre-compiled binary for the current package, the binaries for the tool_require won't be retrieved. They cannot conflict.

```
class MyPkg(ConanFile):
    tool_requires = "tool_a/0.2", "tool_b/0.2@user/testing"
```

This is the declarative way to add `tool_requires`. Check the *tool_requires()* conanfile.py method to learn a more flexible way to add them.

build_requires

Warning: Deprecated since Conan 2.28. Please use `tool_requires` instead of `build_requires`.

build_requires are used in Conan 2 to provide compatibility with the Conan 1.X syntax, but their use is discouraged in Conan 2 and will be deprecated in future 2.X releases. Please use *tool_requires* instead of *build_requires* in your Conan 2 recipes.

test_requires

List or tuple of strings for dependencies in the host context only. Represents a test tool like "gtest". Used when the current package is built from sources. They don't propagate information to the downstream consumers. If there is an existing pre-compiled binary for the current package, the binaries for the test_require won't be retrieved. They cannot conflict.

```
class MyPkg(ConanFile):
    name = "mypkg"
    test_requires = "gtest/1.17.0", "other_test_tool/0.2@user/testing"
```

Note that `test_requires` are private (`visible=False`), only the recipe that declares them has visibility and can use them. The consumers of `MyPkg` will not see or know about the existence of `gtest` or `other_test_tool`. Consequently, they cannot be affected by consumers `options` values definitions, it doesn't matter that a consumer of `mypkg` defines options like `gtest*:some_option=somevalue`, because `gtest` will never receive that value.

This is the declarative way to add `test_requires`. Check the *test_requires() method* to learn a more flexible way to add them.

python_requires

This class attribute allows to define a dependency to another Conan recipe and reuse its code. Its basic syntax is:

```
from conan import ConanFile

class Pkg(ConanFile):
    python_requires = "pyreq/0.1@user/channel"  # recipe to reuse code from

    def build(self):
        self.python_requires["pyreq"].module # access to the whole conanfile.py module
        self.python_requires["pyreq"].module.myvar  # access to a variable
        self.python_requires["pyreq"].module.myfunct()  # access to a global function
        self.python_requires["pyreq"].path # access to the folder where the reused file␣
↪is
```

Read more about this attribute in *Python requires*

python_requires_extend

This class attribute defines one or more classes that will be injected in runtime as base classes of the recipe class. Syntax for each of these classes should be a string like `pyreq.MyConanfileBase` where the `pyreq` is the name of a `python_requires` and `MyConanfileBase` is the name of the class to use.

```
from conan import ConanFile

class Pkg(ConanFile):
    python_requires = "pyreq/0.1@user/channel", "utils/0.1@user/channel"
    python_requires_extend = "pyreq.MyConanfileBase", "utils.UtilsBase"  # class/es to␣
↪inject
```

Sources

exports

List or tuple of strings with *file names* or fnmatch patterns that should be exported and stored side by side with the *conanfile.py* file to make the recipe work: other python files that the recipe will import, some text file with data to read,...

For example, if we have some python code that we want the recipe to use in a `helpers.py` file, and have some text file *info.txt* we want to read and display during the recipe evaluation we would do something like:

```
exports = "helpers.py", "info.txt"
```

Exclude patterns are also possible, with the `!` prefix:

```
exports = "*.py", "!*tmp.py"
```

See also:

- *Check the export() conanfile.py method*.

exports_sources

List or tuple of strings with file names or fnmatch patterns that should be exported and will be available to generate the package. Unlike the `exports` attribute, these files shouldn't be used by the `conanfile.py` Python code, but to compile the library or generate the final package. And, due to its purpose, these files will only be retrieved if requested binaries are not available or the user forces Conan to compile from sources.

This is an alternative to getting the sources with the `source()` method. Used when we are not packaging a third party library and we have together the recipe and the C/C++ project:

```
exports_sources = "include*", "src*"
```

Exclude patterns are also possible, with the `!` prefix:

```
exports_sources = "include*", "src*", "!src/build/*"
```

Note, if the recipe defines the `layout()` method and specifies a `self.folders.source = "src"` it won't affect where the files (from the `exports_sources`) are copied. They will be copied to the base source folder. So, if you want to replace some file that got into the `source()` method, you need to explicitly copy it from the parent folder or even better, from `self.export_sources_folder`.

```
import os, shutil
from conan import ConanFile
from conan.tools.files import save, load

class Pkg(ConanFile):
    ...
    exports_sources = "CMakeLists.txt"

    def layout(self):
        self.folders.source = "src"
        self.folders.build = "build"
```

(continues on next page)

(continued from previous page)

```
    def source(self):
        # emulate a download from web site
        save(self, "CMakeLists.txt", "MISTAKE: Very old CMakeLists to be replaced")
        # Now I fix it with one of the exported files
        shutil.copy("../CMakeLists.txt", ".")
        shutil.copy(os.path.join(self.export_sources_folder, "CMakeLists.txt"), ".")
```

See also:

- *Check the export_sources() conanfile.py method*.

conan_data

Read only attribute with a dictionary with the keys and values provided in a *conandata.yml* file format placed next to the *conanfile.py*. This YAML file is automatically exported with the recipe and automatically loaded with it too.

You can declare information in the *conandata.yml* file and then access it inside any of the methods of the recipe. For example, a *conandata.yml* with information about sources that looks like this:

```
sources:
  "1.1.0":
    url: "https://www.url.org/source/mylib-1.0.0.tar.gz"
    sha256: "8c48baf3babe0d505d16cfc0cf272589c66d3624264098213db0fb00034728e9"
  "1.1.1":
    url: "https://www.url.org/source/mylib-1.0.1.tar.gz"
    sha256: "15b6393c20030aab02c8e2fe0243cb1d1d18062f6c095d67bca91871dc7f324a"
```

```
def source(self):
    get(self, **self.conan_data["sources"][self.version])
```

source_buildenv

Boolean attribute to opt-in injecting the *VirtualBuildEnv* generated environment while running the *source()* method.

Setting this attribute to *True* (default value *False*) will inject the `VirtualBuildEnv` generated environment from tool requires when executing the *source()* method.

```
class MyConan:
    name = "mylib"
    version = "1.0.0"
    source_buildenv = True
    tool_requires = "7zip/1.2.0"

    def source(self):
        get(self, **self.conan_data["sources"][self.version])
        self.run("7z x *.zip -o*")  ## Can run 7z in the source method
```

Binary model

Important attributes that define the package binaries model, which settings, options, package type, etc. affect the final packaged binaries.

package_type

Optional, but very strongly recommended. Declaring the `package_type` will help Conan:

- To choose better the default `package_id_mode` for each dependency, that is, how a change in a dependency should affect the `package_id` to the current package.
- Which information from the dependencies should be propagated to the consumers, like headers, libraries, runtime information. See *here* to see what traits are propagated based on the `package_type` information.

The valid values are:

- **application**: The package is an application.
- **library**: The package is a generic library. It will try to determine the type of library (from `shared-library`, `static-library`, `header-library`) reading the `self.options.shared` (if declared) and the `self.options.header_only`
- **shared-library**: The package is a shared library.
- **static-library**: The package is a static library.
- **header-library**: The package is a header only library.
- **build-scripts**: The package only contains build scripts.
- **python-require**: The package is a python require.
- **unknown**: The type of the package is unknown.

Note that relationships between packages might not always be defined and can lead to errors, for example, `build-scripts` cannot have regular `requires` dependencies to compiled libraries, and it is not known how these should be propagated through something that is intended to be used as a `tool_requires`. If some package want to use both some build scripts and link with a given library should define a `tool_requires()` to the `build-scripts` package and a regular `requires()` to the compiled library.

Important: The `package_type` defines how different information of C and C++ packages is propagated down the dependency graph: visibility of headers, linkage requirements, etc. It is very recommended to define it, and it should be defined in most cases.

Note: The runtime propagation implied by `package_type` can be overridden by declaring `"run": True` inside *package_type_traits* attribute. This is useful, for example, for a `static-library` that also ships a shared library that consumers need available at runtime, which would otherwise not be exposed, as `static-library` implies `run=False`.

Note: The `package_type` attribute can also be computed dynamically in both the `config_options()` and `configure()` methods, for example, a recipe might not support building shared libraries in a specific platform. This can be defined as:

```
package_type = "library"
options = {"shared": [True, False]}

def config_options(self):
    if self.settings.os == "Windows":
        del self.options.shared
        self.package_type = "static-library"
```

where care should be taken when accessing the `shared` option, as it will be removed for Windows, so the `self.options.get_safe("shared")` accessor should be used as appropiate.

settings

List of strings with the first level settings (from *settings.yml*) that the recipe needs, because:

- They are read for building (e.g: `if self.settings.compiler == "gcc"`)
- They affect the `package_id`. If a value of the declared setting changes, the `package_id` has to be different.

The most common is to declare:

```
settings = "os", "compiler", "build_type", "arch"
```

Once the recipe is loaded by Conan, the `settings` are processed and they can be read in the recipe, also the sub-settings:

```
settings = "os", "arch"

def build(self):
    if self.settings.compiler == "gcc":
        if self.settings.compiler.cppstd == "gnu20":
            # do some special build commands
```

If you try to access some setting that doesn't exist, like `self.settings.compiler.libcxx` for the `msvc` setting, Conan will fail telling that `libcxx` does not exist for that compiler.

If you want to do a safe check of settings values, you could use the `get_safe()` method:

```
def build(self):
    # Will be None if doesn't exist (not declared)
    arch = self.settings.get_safe("arch")
    # Will be None if doesn't exist (doesn't exist for the current compiler)
    compiler_version = self.settings.get_safe("compiler.version")
    # Will be the default version if the return is None
    build_type = self.settings.get_safe("build_type", default="Release")
```

The `get_safe()` method returns `None` if that setting or sub-setting doesn't exist and there is no default value assigned.

It's also feasible to check the possible values defined in *settings.yml* using the `possible_values()` method:

```
def generate(self):
    # Print if Android exists as OS in the whole settings.yml
    is_android = "Android" in self.settings.possible_values()["os"]
    self.output.info(f"Android in settings.yml: {is_android}")
    # Print the available versions for the compiler used by the HOST profile
```

(continues on next page)

(continued from previous page)

```
    compiler_versions = self.settings.compiler.version.possible_values()
    self.output.info(f"[HOST] Versions for {str(self.settings.compiler)}:  {', '.
↪join(compiler_versions)}")
    # Print the available versions for the compiler used by the BUILD profile
    compiler_versions = self.settings_build.compiler.version.possible_values()
    self.output.info(f"[BUILD] Versions for {str(self.settings_build.compiler)}:  {', '.
↪join(compiler_versions)}")
```

As you can see above, doing `self.settings.possible_values()` returns the whole *settings.yml* as a Python dict-like object, and doing `self.settings.compiler.version.possible_values()` for instance returns the available versions for the compiler used by the consumer.

If you want to do a safe deletion of settings, you could use the `rm_safe()` method. For example, in the `configure()` method a typical pattern for a C library would be:

```
def configure(self):
    self.settings.rm_safe("compiler.libcxx")
    self.settings.rm_safe("compiler.cppstd")
```

See also:

- *settings.yml*.
- *Removing settings in the package_id() method*.
- *Creating universal binaries using CMakeToolchain*.

options

Dictionary with traits that affects only the current recipe, where the key is the option name and the value is a list of different values that the option can take. By default any value change in an option, changes the `package_id`. Check the `default_options` and `default_build_options` fields to define default values for the options.

Values for each option can be typed or plain strings (`"value"`, `True`, `42`,...).

There are two special values:

- `None`: Allow the option to have a `None` value (not specified) without erroring.
- `"ANY"`: For options that can take any value, not restricted to a set.

```
class MyPkg(ConanFile):
    ...
    options = {
        "shared": [True, False],
        "option1": ["value1", "value2"],
        "option2": ["ANY"],
        "option3": [None, "value1", "value2"],
        "option4": [True, False, "value"],
}
```

Once the recipe is loaded by Conan, the `options` are processed and they can be read in the recipe. You can also use the method `.get_safe()` (see *settings attribute*) to avoid Conan raising an Exception if the option doesn't exist:

```
class MyPkg(ConanFile):
    options = {"shared": [True, False]}

    def build(self):
        if self.options.shared:
            # build the shared library
        if self.options.get_safe("foo", True):
            pass
```

In boolean expressions, like `if self.options.shared`:

- equals `True` for the values `True`, `"True"` and `"true"`, and any other value that would be evaluated the same way in Python code.
- equals `False` for the values `False`, `"False"` and `"false"`, also for the empty string and for `0` and `"0"` as expected.

Notice that a comparison using `is` is always `False` because the types would be different as it is encapsulated inside a Python class.

If you want to do a safe deletion of options, you could use the `rm_safe()` method. For example, in the `config_options()` method a typical pattern for Windows library would be:

```
def config_options(self):
    if self.settings.os == "Windows":
        self.options.rm_safe("fPIC")
```

Warning: Some names are not allowed as option names because they conflict with existing `Options` class methods. Some of them are already documented or available methods:

- `update`: used by `self.options.update()` to extend options from a `python_requires` base class. See the *init() method*.
- `clear`: used internally to clear all options, e.g. for header-only packages.
- `items`: used to iterate over option name/value pairs.
- The `get_safe` and `rm_safe` operations.

But there can be also other private methods that could eventually collide (Conan will try to reduce them, but at the moment they are still there and could shadow user definitions):

- Some verbs related to storage and serialization: `loads`, `dumps` and `serialize`.
- The `possible_values` access
- Anything that is private (starts with _ underscore, or starts with `conan`)

Using any of these names as option keys will cause the option value to be shadowed by the corresponding method, leading to unexpected behavior.

See also:

- Read the *Getting started, creating packages* to know how to declare and how to define a value to an option.
- *Removing options in the package_id() method*.
- Read *how the package_type attribute behaves when a shared option is declared*.

default_options

The attribute `default_options` defines the default values for the options, both for the current recipe and for any requirement. This attribute should be defined as a python dictionary.

```
class MyPkg(ConanFile):
    ...
    requires = "zlib/1.2.8", "zwave/2.0"
    options = {"build_tests": [True, False],
               "option2": "ANY"}
    default_options = {"build_tests": True,
                       "option1": 42,
                       "z*:shared": True}
```

You can also assign default values for options of your requirements using "<reference_pattern>: option_name", being a valid `reference_pattern` a `name/version` or any pattern with * like the example above.

Warning: Defining options values for dependencies in recipes does not have strong guarantees, please check *this FAQ about options values for dependencies*. The recommended way to define `default_options` values for dependencies is in **profile files**.

You can also set the options conditionally to a final value with `configure()` instead of using `default_options`:

```
class OtherPkg(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    options = {"some_option": [True, False]}
    # Do NOT declare 'default_options', use 'config_options()'

    def configure(self):
        if self.options.some_option == None:
            if self.settings.os == 'Android':
                self.options.some_option = True
            else:
                self.options.some_option = False
```

Take into account that if a value is assigned in the `configure()` method it cannot be overridden.

See also:

- *config_options() method*.

There are 2 different ways that a recipe can try to define options values for its dependencies. Using `default_options = {"mypkg/*:myoption", 123}` the current recipe can define the `123` value to the dependency `mypkg` `myoption`. This way of defining options for dependencies has some limitations:

- Any other downstream user of the current recipe that defines the same option for `mypkg` will have precedence, overwriting the current recipe `123` value. Also any definition in the profile or command line will also have precedence. The recipe `default_options` have the least precedence. If a recipe will not work at all with some dependencies options, then recipes can check and raise `ConanInvalidConfiguration` errors accordingly.
- Any *sibling* package that depends on `mypkg` will also define its options and it will be the only one being taken into account. In other words, the first time `mypkg` is required by any other package will "freeze" its currently assigned options values. Any other package that depends later on `mypkg`, closing the diamond structures in the dependency graph will not have any influence on the `mypkg` options. Only the first one requiring it will.

The second way to define the options values is defining them as `important!`.

Warning: The `important!` syntax is experimental and can be changed or removed at any time.

A recipe can define its dependencies options as `important!` with the syntax `default_options = {"mypkg/*:myoption!", 123}`. That means that the `mypkg` `myoption` will not be overriden by other downstream packages, profile or command line doing regular definition of options (like `-o *:myoption=234`).

But there are 2 cases in which this will still not define the final value of the dependency:

- If any downstream recipe, command line or profile also uses the `myoption!` syntax, that will also have precedence and override the value upstream
- If there is any other package that requires first `mypkg`, the values defined at that moment will still have precedence.

In general the recommendation for defining options values is to do it in `profile` files, not in recipes, as in-recipe definition can be more complicated specially for complex dependency graphs.

default_build_options

The attribute `default_build_options` defines the default values for the options in the build context and is typically used for defining options for `tool_requires`.

```
from conan import ConanFile
class Consumer(ConanFile):
    default_options = {"protobuf/*:shared": True}
    default_build_options = {"protobuf/*:shared": False}
    def requirements(self):
        self.requires("protobuf/1.0")
    def build_requirements(self):
        self.tool_requires("protobuf/1.0")
```

options_description

The `options_description` attribute is an optional attribute that can be defined in the form of a dictionary where the key is the option name and the value is a description of the option in text format. This attribute is useful for providing additional information about the functionality and purpose of each option, particularly when the option is not self-explanatory or has complex or special behavior.

The format for each dictionary entry should be:

- Key: Option name. Must be a string and must match one of the keys in the `options` dictionary.
- Value: Description of the option. Must be a string and can be as long as necessary.

For example:

```
class MyPkg(ConanFile):
    ...
    options = {"option1": [True, False],
               "option2": "ANY"}

    options_description = {
        "option1": "Describe the purpose and functionality of 'option1'. ",
        "option2": "Describe the purpose and functionality of 'option2'. ",
    }
```

languages

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

From Conan 2.4, the `conanfile.py` recipe attribute `languages` can be used to define the programming languages involved in this package. At the moment the `C` and `C++` languages are the possible values. For example a pure C package would define something as:

```
class ZLib(ConanFile):
    languages = "C"
```

It is possible to define more than one language, for example `languages = "C", "C++"` is the correct definition when a package is built from both C and C++ sources.

Regarding `languages` definition, the following will happen:

- If no `languages` is defined or `C` is not a declared language, `compiler.cstd` subsetting will be automatically removed at package `configure()` time (to achieve backward compatibility).
- If `languages` is defined, but it doesn't contain `C++`, `compiler.cppstd` and `compiler.libcxx` subsettings will be automatically removed at package `configure()` time.

info

Object used exclusively in `package_id()` method:

- The *:ref:package_id method<reference_conanfile_methods_package_id>* to control the unique ID for a package:

  ```
  def package_id(self):
      self.info.clear()
  ```

The `self.info.clear()` method removes all the settings, options, requirements (`requires`, `tool_requires`, `python_requires`) and configuration (`conf`) from the `package_id` computation, so the `package_id` will always result in the same binary, irrespective of all those things. This would be the typical case of a header-only library, in which the packaged artifacts (files) are always identical.

package_id_{embed,non_embed,python,unknown}_mode, build_mode

The `package_id_embed_mode`, `package_id_non_embed_mode`, `package_id_python_mode`, `package_id_unknown_mode` are class attributes that can be defined in recipes to define the effect they have on their consumers' `package_id`, when they are consumed as `requires`.

The `build_mode` (experimental) is a class attribute that affects the package consumers when these consumers use it as `tool_requires`. Can be declared as:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0.0"
    # They are not mandatory, and it is not necessary to define all
```

(continues on next page)

(continued from previous page)

```
    package_id_embed_mode = "full_mode"
    package_id_non_embed_mode = "patch_mode"
    package_id_unknown_mode = "minor_mode"
    package_id_python_mode = "major_mode"
    build_mode = "patch_mode"  # (experimental) when used as tool_requires
```

In general, the Conan defaults are good ones, and allow providing users good control over when the consumers need to be re-built from source or not. Also, the Conan defaults can be changed globally in the `global.conf` file (they should be changed globally for all users, CI, etc.) via the `core.package_id:xxxx` configurations. The in-recipe attribute definition is useful to define behavior that deviates from the defaults.

Possible values are (following the semver definition of MAJOR.MINOR.PATCH, with an extra MICRO digit for versioning schemes with 4 digits):

- `patch_mode`: New patches, minors, and major releases of the package will require a new binary (new `package_id`) of the consumers. New recipe revisions will not require new binaries of the consumers. For example if we create a new `pkg/1.0.1` version and some consumer has `requires = "pkg/[>=1.0 <2.0]"`, such a consumer will build a new binary against this specific new `1.0.1` version. But if we just change the recipe, producing a new `recipe_revision`, the consumers will not require building a new binary.
- `micro_mode`: (Available since Conan 2.31). Similar to `patch_mode` but factoring in a 4th `MICRO` digit for versioning schemes that use `MAJOR.MINOR.PATCH.MICRO`. New micro, patch, minor and major releases will require a new binary of the consumers, but new recipe revisions will not. Any additional digits beyond the 4th are ignored. Missing digits default to `0` (a dependency version `1.2.3` will be represented as `1.2.3.0`).
- `minor_mode`: New minor and major releases of this package will require a new binary of the consumers. New patches and new revisions will not require new binaries of the consumers. This is the default for the "non-embed-mode", as it allows fine control by the users to decide when to rebuild things or not.
- `major_mode`: Only new major releases will require new binaries. Any other modifications and new versions will not require new binaries from the consumers.
- `full_mode`: The full identifier of this package, including `pkgname/version@user/channel#recipe_revision:package_id` will be used in the consumers `package_id`, then requiring to build a new binary of the consumer for every change of this package (as any change either in source or configuration will produce a different `recipe_revision` or `package_id` respectively). This is the default for the "embed-mode".
- `unrelated_mode`: No change in this package will ever produce a new binary in the consumer.
- `revision_mode`: Uses the `pkgname/version@user/channel#recipe_revision` in the consumers' `package_id`, that is the full reference except the `package_id` of the dependency.
- `semver_mode`: Equivalent to `major_mode` if the version is `>=1.0`, or equivalent to `patch_mode` (or the full version if it has more than 3 digits) if the version is `<1.0`.

The 4 different attributes are:

- `package_id_embed_mode`: Define the mode for "embedding" cases, that is, a shared library linking a static library, an application linking a static library, an application or a library linking a header-only library. The default for this mode is `full_mode`
- `package_id_non_embed_mode`. Define the mode for "non-embedding" cases, that is, a shared library linking another shared library, a static library linking another static library, an application executable linking a shared library. The default for this mode is `minor_mode`.
- `package_id_unknown_mode`: Define the mode when the relationship between packages is unknown. If it is not possible to deduce the package type, because there are no `shared` or `header_only` options defined, or because

`package_type` is not defined, then, this mode will be used. The default for this mode is `semver_mode` (similar to Conan 1.X behavior).

- `package_id_python_mode`: Define the mode for consumers of `python_requires`. By default it will be `minor_mode`, and it is strongly recommended to use this default, and not define the `package_id_python_mode`. This attribute is provided for completeness and exceptional cases like temporary migrations.
- `build_mode`: (Experimental) Define the mode for consumers using this dependency as `tool_requires`. By default is `None`, which means that the `tool_requires` does not affect directly the `package_id` of their consumers. Enabling this `build_mode` introduces a harder dependency to the `tool_requires` that will be needed to resolve the `package_id` of the consumers in more cases.

See also:

Read the *binary model reference* for a full view of the Conan binary model.

package_id_abi_options

There are some scenarios when it might be desired to make the value of a given option to influence the `package_id` of the binaries consuming this package.

This is generally not necessary in most of the cases, as the default binary model is good for the majority of scenarios. For example, in all `embed` modes, the full dependency reference, including its `package_id`, that already encodes the dependency options, are already factored in into the consumer `package_id`. So for that case, the dependency's options already have this effect on the consumer `package_id`.

But there might be some cases for `non_embed`, like a static library that could be linking a dependency sometimes as a static library and sometimes as a shared library. In platforms such as Linux or Mac, the linkage doesn't really change the consumer. But this case for Windows `msvc` compiler, when the dependency is conditionally defining `dllimport` in their headers, the calling convention changes, and the binary of the consumer is different.

For shared libraries in Windows, it is common to find this idiom:

```
#ifdef WIN32
    #define HELLO_EXPORT __declspec(dllexport)
#else
    #define HELLO_EXPORT
#endif

HELLO_EXPORT void hello();
```

To export the `hello()` symbol into the shared library, as in Windows MSVC the symbols are not exported by default. This is not an issue, as the consumer packages and callers of `hello()` will not change its linkage.

But in some cases, some libraries might decide to define something like:

```
#ifdef WIN32
    #ifdef libhello_EXPORTS
        /* We are building this library */
        #define HELLO_EXPORT __declspec(dllexport)
    #else
        /* We are using this library */
        #define HELLO_EXPORT __declspec(dllimport)
    #endif
#else
    #define HELLO_EXPORT
#endif
```

(continues on next page)

(continued from previous page)

```
HELLO_EXPORT void hello();
```

And control via `libhello_EXPORTS` if the package is being built or consumed. In this case, the consumer will have a different linkage when linking this shared library with the `dllimport`, than when linking this library as a static library.

For those case, the package recipe can define which of its own options affect the consumers `package_id`, irrespective of the `non_embed` mode, so they can still generate different binaries for the different linkages, without necessarily resorting to a full `embed` mode that will require unnecessary rebuilds of binaries from source.

This can be defined with:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0.0"
    options = {"shared": [False, True]}

    package_id_abi_options = ["shared"]
```

And that will make all the consumers of `pkg/1.0.0` to automatically factor the `pkg/*:shared=True/False` value in their own `package_id`.

context

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `conanfile.py` recipe attribute `context` will contain either the "build" or "host" value to represent the context where the current package instance is being evaluated. Recall that it is possible that some recipes might exist both in the "build" and "host" contexts, depending on the usage.

This attribute shouldn't be necessary for the vast majority of cases, so it is recommended to avoid using it. One potential exception for this recommendation would be to break otherwise infinite dependency cycles, defining some conditional dependency as:

```
def requirements(self):
    if self.context == "host":
        self.tool_requires("mytool/1.0")
```

Build

generators

List or tuple of strings with names of generators.

```
class MyLibConan(ConanFile):
    generators = "CMakeDeps", "CMakeToolchain"
```

The generators can also be instantiated explicitly in the *generate() method*.

```
from conan.tools.cmake import CMakeToolchain

class MyLibConan(ConanFile):
    ...

    def generate(self):
        tc = CMakeToolchain(self)
        tc.generate()
```

build_policy

Controls when the current package is built during a `conan install`. The allowed values are:

- `"missing"`: Conan builds it from source if there is no binary available.
- `"never"`: This package cannot be built from sources, it is always created with `conan export-pkg`
- `None` (default value): This package won't be built unless the policy is specified in the command line (e.g `--build=foo*`)

```
class PocoTimerConan(ConanFile):
    build_policy = "missing"
```

win_bash

When `True` it enables the new run in a subsystem bash in Windows mechanism.

```
from conan import ConanFile

class FooRecipe(ConanFile):
    ...
    win_bash = True
```

It can also be declared as a `property` based on any condition:

```
from conan import ConanFile

class FooRecipe(ConanFile):
    ...


    @property
    def win_bash(self):
        return self.settings.arch == "armv8"
```

win_bash_run

When `True` it enables running commands in the `"run"` scope, to run them inside a bash shell.

```
from conan import ConanFile

class FooRecipe(ConanFile):

    ...

    win_bash_run = True
    def build(self):
        self.run(cmd, scope="run")  # will run <cmd> inside bash
```

Folders and layout

source_folder

The folder in which the source code lives. The path is built joining the base directory (a cache directory when running in the cache or the `output folder` when running locally) with the value of `folders.source` if declared in the `layout()` method.

Note that the base directory for the `source_folder` when running in the cache will point to the base folder of the build unless *no_copy_source* is set to `True`. But anyway it will always point to the correct folder where the source code is.

export_sources_folder

The value depends on the method you access it:

- At `source(self)`: Points to the base source folder (that means self.source_folder but without taking into account the `folders.source` declared in the `layout()` method). The declared *exports_sources* are copied to that base source folder always.
- At `exports_sources(self)`: Points to the folder in the cache where the export sources have to be copied.

See also:

- *Read about the export_sources() method*.
- *Read about the source() method*.

build_folder

The folder used to build the source code. The path is built joining the base directory (a cache directory when running in the cache or the `output folder` when running locally) with the value of `folders.build` if declared in the `layout()` method.

generators_folder

The folder where the files in the `generate()` method should be generated. The path is built from the layout's `self.folders.generators` attribute.

package_folder

The folder to copy the final artifacts for the binary package. In the local cache a package folder is created for every different package ID.

The most common usage of `self.package_folder` is to `copy` the files at the *package() method*:

```
import os
from conan import ConanFile
from conan.tools.files import copy

class MyRecipe(ConanFile):
    ...

    def package(self):
        copy(self, "*.so", self.build_folder, os.path.join(self.package_folder, "lib"))
        ...
```

recipe_folder

The folder where the recipe *conanfile.py* is stored, either in the local folder or in the cache. This is useful in order to access files that are exported along with the recipe, or the origin folder when exporting files in `export(self)` and `export_sources(self)` methods.

The most common usage of `self.recipe_folder` is in the `export(self)` and `export_sources(self)` methods, as the folder from where we copy the files:

```
from conan import ConanFile
from conan.tools.files import copy

class MethodConan(ConanFile):
    exports = "file.txt"
    def export(self):
        copy(self, "LICENSE.md", self.recipe_folder, self.export_folder)
```

recipe_metadata_folder

The `self.recipe_metadata_folder` (**experimental**) can be used in the `export()` and `export_sources()` and `source()` methods to save or copy **recipe** metadata files. See *metadata section* for more information.

package_metadata_folder

The `self.package_metadata_folder` (**experimental**) can be used in the `generate()`, `build()` and `package()` methods to save or copy **package** metadata files. See *metadata section* for more information.

no_copy_source

The attribute `no_copy_source` tells the recipe that the source code will not be copied from the `source_folder` to the `build_folder`. This is mostly an optimization for packages with large source codebases or header-only, to avoid extra copies.

If you activate `no_copy_source=True`, it is **mandatory** that the source code must not be modified at all by the configure or build scripts, as the source code will be shared among all builds.

The recipes should always use `self.source_folder` attribute, which will point to the `build` folder when `no_copy_source=False` and will point to the `source` folder when `no_copy_source=True`.

See also:

Read *header-only packages section* for an example using `no_copy_source` attribute.

test_package_folder

The `test_package_folder` class attribute allows defining in recipe a different default `test_package` folder for `conan create` commands. When a `conan create` runs, after the package is created in the cache, it will look for a `test_package` folder, or for the folder specified in the `--test-folder=xxx` argument, and launch the package test.

This attribute allows to change that default name:

```
import os
from conan import ConanFile

class Pkg(ConanFile):
    test_package_folder = "my/test/folder"
```

It allows to define any folder, always relative to the location of the `conanfile.py`.

Layout

folders

The `folders` attribute has to be set only in the `layout()` method. Please check the *layout() method documentation* to learn more about this attribute.

cpp

Object storing all the information needed by the consumers of a package: include directories, library names, library paths... Both for editable and regular packages in the cache. It is only available at the `layout()` method.

- `self.cpp.package`: For a regular package being used from the Conan cache. Same as declaring `self.cpp_info` at the `package_info()` method.
- `self.cpp.source`: For "editable" packages, to describe the artifacts under `self.source_folder`
- `self.cpp.build`: For "editable" packages, to describe the artifacts under `self.build_folder`.

The `cpp` attribute has to be set only in the `layout()` method. Please check the *layout() method documentation* to learn more about this attribute.

layouts

The `layouts` attribute has to be set only in the `layout()` method. Please check the *layout() method documentation* to learn more about this attribute.

The `layouts` attribute contains information about environment variables and `conf` that would be path-dependent, and as a result it would contain a different value when the package is in editable mode, or when the package is in the cache. The `layouts` sub-attributes are:

- `self.layouts.build`: information related to the relative `self.folders.build`
- `self.layouts.source`: information related to the relative `self.folders.source`
- `self.layouts.package`: information related to the final `package_folder`

Each one of those will contain:

- `buildenv_info`: environment variables build information for consumers (equivalent to `self.buildenv_info` in `package_info()`)
- `runenv_info`: environment variables run information for consumers (equivalent to `self.runenv_info` in `package_info()`)
- `conf_info`: configuration information for consumers (equivalent to `self.conf_info` in `package_info()`). Note this is only automatically propagated to `self.conf` of consumers when this package is a direct `tool_require`.

For example, if we had an `androidndk` recipe that contains the AndroidNDK, and we want to have that recipe in "editable" mode, it is necessary where the androidndk will be locally, before being in the created package:

```
import os
from conan import ConanFile
from conan.tools.files import copy

class AndroidNDK(ConanFile):

    def layout(self):
        # When developing in user space it is in a "mybuild" folder (relative to current␣
↪dir)
        self.layouts.build.conf_info.define_path("tools.android:ndk_path", "mybuild")
        # but when packaged it will be in a "mypkg" folder (inside the cache package␣
↪folder)
        self.layouts.package.conf_info.define_path("tools.android:ndk_path", "mypkg")
```

(continues on next page)

(continued from previous page)

```
    def package(self):
        copy(self, "*", src=os.path.join(self.build_folder, "mybuild"),
             dst=os.path.join(self.package_folder, "mypkg"))
```

Package information for consumers

cpp_info

Same as using `self.cpp.package` in the `layout()` method. Use it if you need to read the `package_folder` to locate the already located artifacts.

See also:

- *CppInfo* model.

Important: This attribute is only defined inside `package_info()` method being *None* elsewhere.

---

buildenv_info

For the dependant recipes, the declared environment variables will be present during the build process. Should be only filled in the `package_info()` method.

Important: This attribute is only defined inside `package_info()` method being *None* elsewhere.

```
def package_info(self):
    self.buildenv_info.append_path("PATH", self.package_folder)
```

See also:

Check the reference of the *Environment* object to know how to fill the `self.buildenv_info`.

runenv_info

For the dependant recipes, the declared environment variables will be present at runtime. Should be only filled in the `package_info()` method.

Important: This attribute is only defined inside `package_info()` method being *None* elsewhere.

```
def package_info(self):
    self.runenv_info.define_path("RUNTIME_VAR", "c:/path/to/exe")
```

See also:

Check the reference of the *Environment* object to know how to fill the `self.runenv_info`.

conf_info

Configuration variables to be passed to the dependant recipes. Should be only filled in the `package_info()` method.

```
class Pkg(ConanFile):
    name = "pkg"

    def package_info(self):
        self.conf_info.define("tools.build:verbosity", "debug")
        self.conf_info.get("tools.build:verbosity")  # == "debug"
        self.conf_info.append("user.myconf.build:ldflags", "--flag3")  # == ["--flag1",
↪"--flag2", "--flag3"]
        self.conf_info.update("tools.microsoft.msbuildtoolchain:compile_options", {
↪"ExpandAttributedSource": "false"})
        self.conf_info.unset("tools.microsoft.msbuildtoolchain:compile_options")
        self.conf_info.remove("user.myconf.build:ldflags", "--flag1")  # == ["--flag0",
↪"--flag2", "--flag3"]
        self.conf_info.pop("tools.system.package_manager:sudo")
```

See also:

Read here *the complete reference of self.conf_info*.

generator_info

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Generators to be passed to the dependant recipes. Should be only filled in the `package_info()` method, `None` by default.

See also:

See *an example usage here* and *the complete reference of self.generator_info*.

deprecated

This attribute declares that the recipe is deprecated, causing a user-friendly warning message to be emitted whenever it is used

For example, the following code:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "cpp-taskflow"
    version = "1.0"
    deprecated = True
```

may emit a `risk` warning like:

```
Deprecated
    cpp-taskflow/1.0

WARN: risk: There are deprecated packages in the graph
```

Optionally, the attribute may specify the name of the suggested replacement:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "cpp-taskflow"
    version = "1.0"
    deprecated = "Not secure, use better taskflow>1.2.3"
```

This will emit a `risk` warning like:

```
Deprecated
    cpp-taskflow/1.0: Not secure, use better taskflow>1.2.3

WARN: risk: There are deprecated packages in the graph
```

If the value of the attribute evaluates to `False`, no warning is printed.

provides

This attribute declares that the recipe provides the same functionality as other recipe(s). The attribute is usually needed if two or more libraries implement the same API to prevent link-time and run-time conflicts (ODR violations). One typical situation is forked libraries. Some examples are:

- LibreSSL, BoringSSL and OpenSSL
- libav and ffmpeg
- MariaDB client and MySQL client

If Conan encounters two or more libraries providing the same functionality within a single graph, it raises an error:

```
At least two recipes provides the same functionality:
- 'libjpeg' provided by 'libjpeg/9d', 'libjpeg-turbo/2.0.5'
```

The attribute value should be a string with a recipe name or a tuple of such recipe names.

For example, to declare that `libjpeg-turbo` recipe offers the same functionality as `libjpeg` recipe, the following code could be used:

```
from conan import ConanFile

class LibJpegTurbo(ConanFile):
    name = "libjpeg-turbo"
    version = "1.0"
    provides = "libjpeg"
```

To declare that a recipe provides the functionality of several different recipes at the same time, the following code could be used:

```
from conan import ConanFile

class OpenBLAS(ConanFile):
    name = "openblas"
    version = "1.0"
    provides = "cblas", "lapack"
```

If the attribute is omitted, the value of the attribute is assumed to be equal to the current package name. Thus, it's redundant for `libjpeg` recipe to declare that it provides `libjpeg`, it's already implicitly assumed by Conan.

package_type_traits

This attribute declares that the package has runtime artifacts that consumers should be aware of. This is useful for `package_type="static-library"` recipes that nevertheless also contain shared libraries, or by `package_type="header-library"` that have some kind of resource that library needs at runtime, so that at runtime their folders are exposed to the consumers.

It is important to still declare the `package_type` that best describes the package for linkage and `package_id` purposes (see *the effect of dependencies on package_id* and *package_type trait inferring*), and use `package_type_traits` only to fix the specific trait that `package_type` got wrong, instead of changing the `package_type` itself. For example, a package that is fundamentally a static library, meant to be embedded and to force a rebuild of its direct consumers like any other static library, should keep `package_type = "static-library"`, even if it also happens to ship a shared library or some other resource needed at runtime. Declaring it instead as `package_type = "shared-library"` would also change the `package_id` computation mode and the linkage information propagated to consumers, when the only thing that actually needs fixing is the `run` trait.

By default, a `static-library` dependency gets the `run` requirement trait set to `False` (see *package_type trait inferring*), so its binary can be skipped when only linked transitively, and its folders are not added to the runtime environment by generators like *VirtualRunEnv*, nor collected by deployers like *runtime_deploy*. Declaring `package_type_traits = {"run": True}` forces `run=True` for every consumer of the package, regardless of its `package_type`:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "mypkg"
    version = "1.0"
    # This is a static library, but it also ships a shared library (e.g. a plugin) that
    # consumers need available at runtime
    package_type = "static-library"
    package_type_traits = {"run": True}
```

Note: At the moment, `run` is the only key read from `package_type_traits`, and `True` is the only value that has an effect. Setting any other trait or value currently does nothing, so it shouldn't be relied upon: unsupported keys are silently ignored today, but might be validated, or start having an effect, in future Conan versions.

See also:

- *The run requirement trait*.
- *How package_type infers the run trait*.

Other

dependencies

Conan recipes provide access to their dependencies via the `self.dependencies` attribute.

```
class Pkg(ConanFile):
    requires = "openssl/0.1"

    def generate(self):
        openssl = self.dependencies["openssl"]
        # access to members
        openssl.ref.version
        openssl.ref.revision # recipe revision
        openssl.options
        openssl.settings
```

See also:

Read here *the complete reference of self.dependencies*.

subgraph

(Experimental) A read-only dependency graph of the recipe. The `dependencies` attribute should be used to access the dependencies of the recipe, as this attribute is intended to be passed to other Conan APIs and exposed for advanced usages like *SBOM generation*.

conf

In the `self.conf` attribute we can find all the conf entries declared in the *[conf]* section of the profiles. in addition of the declared *self.conf_info* entries from the first level tool requirements. The profile entries have priority.

```
from conan import ConanFile

class MyConsumer(ConanFile):

  tool_requires = "my_android_ndk/1.0"

  def generate(self):
      # This is declared in the tool_requires
      self.output.info("NDK host: %s" % self.conf.get("tools.android:ndk_path"))
      # This is declared in the profile at [conf] section
      self.output.info("Custom var1: %s" % self.conf.get("user.custom.var1"))
```

Note: The `conf` attribute is a **read-only** attribute. It can only be defined in profiles and command lines, but it should never be set by recipes. Recipes can only read its value via `self.conf.get()` method.

Output

Output contents

Use the `self.output` attribute to print contents to the output.

```
self.output.success("This is good, should be green")
self.output.info("This is neutral, should be white")
self.output.warning("This is a warning, should be yellow")
self.output.error("Error, should be red")
```

Additional output methods are available and you can produce different outputs with different colors. See *the output documentation* for the list of available output methods.

revision_mode

This attribute allow each recipe to declare how the revision for the recipe itself should be computed. It can take three different values:

- `"hash"` (by default): Conan will use the checksum hash of the recipe manifest to compute the revision for the recipe.
- `"scm"`: if the project is inside a Git repository the commit ID will be used as the recipe revision. If there is no repository it will raise an error.
- `"scm_folder"`: This configuration applies when you have a mono-repository project, but still want to use *scm* revisions. In this scenario, the revision of the exported *conanfile.py* will correspond to the commit ID of the folder where it's located. This approach allows multiple *conanfile.py* files to exist within the same Git repository, with each file exported under its distinct revision.

When `scm` or `scm_folder` is selected, the Git commit will be used, but by default the repository must be clean, otherwise it would be very likely that there are uncommitted changes and the build wouldn't be reproducible. So if there are dirty files, Conan will raise an error.

If there are files that can be dirty in the repo, but do not belong at all to the recipe or the package, then it is possible to exclude them from the check with the `revision_mode_excluded` recipe attribute or the `core.scm:excluded` configuration, which is a list of patterns (fnmatch) to exclude.

```
from conan import ConanFile

class MyConsumer(ConanFile):

  revision_mode = "scm"
  # the .tmp files are excluded from revision and dirty check
  revision_mode_excluded = ["*.tmp"]
```

upload_policy

Controls when the current package built binaries are uploaded or not

- `"skip"`: The precompiled binaries are not uploaded. This is useful for "installer" packages that just download and unzip something heavy (e.g. android-ndk), and is useful together with the `build_policy = "missing"`

```
class Pkg(ConanFile):
    upload_policy = "skip"
```

required_conan_version

Recipes can define a module level `required_conan_version` that defines a valid version range of Conan versions that can load and understand the current `conanfile.py`. The syntax is:

```
from conan import ConanFile

required_conan_version = ">=2.0"

class Pkg(ConanFile):
    pass
```

Version ranges as in `requires` are allowed. Also there is a `global.conf` file `core:required_conan_version` configuration that can define a global minimum, maximum or exact Conan version to run, which can be very convenient to maintain teams of developers and CI machines to use the desired range of versions.

implements

A list is used to define a series of option configurations that Conan will handle automatically. This is especially handy for avoiding boilerplate code that tends to repeat in most of the recipes. The syntax is as follows:

```
from conan import ConanFile

class Pkg(ConanFile):
    implements = ["auto_shared_fpic", "auto_header_only", ...]
```

Currently these are the automatic implementations provided by Conan:

- `"auto_shared_fpic"`: automatically manages `fPIC` and `shared` options. Adding this implementation will have both effect in the *configure* and *config_options* steps when those methods are not explicitly defined in the recipe.
- `"auto_header_only"`: automatically manages the package ID clearing settings. Adding this implementation will have effect in the *package_id* step when the method is not explicitly defined in the recipe.

> **Warning:** This is a 2.0-only feature, and it will not work in 1.X

alias

Warning: While aliases can technically still be used in Conan 2, their usage is not recommended and they may be fully removed in future releases. Users are encouraged to adapt to the *newer versioning features* for a more standardized and efficient package management experience.

In Conan 2, the `alias` attribute remains a part of the recipe, allowing users to define an alias for a package version. Normally, you would create one using the `conan new` command with the `alias` template and the exporting the recipe with conan export:

```
$ conan new alias -d name=mypkg -d version=latest -d target=1.0
$ conan export .
```

Note that when requiring the alias, you must place the version in parentheses `()` to explicitly declare the use of an alias as a requirement:

```
class Consumer(ConanFile):

    ...
    requires = "mypkg/(latest)"
    ...
```

extension_properties

The `extensions_properties` attribute is a dictionary intended to define and pass information from the recipes to the Conan extensions.

At the moment, the only defined properties are `compatibility_cppstd` and `compatibility_cstd`, that allows disabling the behavior of *the default compatibility.py extension*, that considers binaries built with different `compiler.cppstd` and `compiler.cstd` values ABI-compatible among them. To disable this behavior for the current package, it is possible to do it with:

```
class Pkg(ConanFile):
    extension_properties = {"compatibility_cppstd": False}
```

If it is necessary to do it conditionally, it is also possible to define its value inside recipe `compatibility()` method:

```
class Pkg(ConanFile):

    def compatibility(self):
        self.extension_properties = {"compatibility_cppstd": False}
```

Note: The value of `extension_properties` is not transitive from the dependencies to the consumers by default, but can be propagated manually by iterating the `self.dependencies` and checking the desired values of their `extension_properties`.

9.3.2 Methods

What follows is a list of methods that you can define in your recipes to customize the package creation & consumption processes:

build()

The `build()` method is used to define the build from source of the package. In practice this means calling some build system, which could be done explicitly or using any of the build helpers provided by Conan:

```
from conan.tools.cmake import CMake

class Pkg(ConanFile):

    def build(self):
        # Either using some of the Conan built-in helpers
        cmake = CMake(self)
        cmake.configure()  # equivalent to self.run("cmake . <other args>")
        cmake.build() # equivalent to self.run("cmake --build . <other args>")
        cmake.test()  # equivalent to self.run("cmake --target=RUN_TESTS")

        # Or it could run your own build system or scripts
        self.run("mybuildsystem . --configure")
        self.run("mybuildsystem . --build")
```

For more information about the existing built-in build system integrations, visit *Recipe tools*.

The `build()` method should be as simple as possible, just wrapping the command line invocations that a developer would do in the simplest possible way. The `generate()` method is the one responsible for preparing the build, creating toolchain files, CMake presets, or any other files which are necessary so developers could easily call the build system by hand. This allows for much better integrations with IDEs and improves the developer experience. The result is that in practice the `build()` method should be relatively simple.

The `build()` method runs once per unique configuration, so if there are some source operations like applying patches that are done conditionally to different configurations, they could be also applied in the `build()` method, before the actual build. It is important to note that in this case the *no_copy_source* attribute cannot be set to `True`.

The `build()` method is the right place to build and run unit tests, before packaging, and raising errors if those tests fail, interrupting the process, and not even packaging the final binaries. The built-in helpers will skip the unit tests if the `tools.build:skip_test` configuration is defined. For custom integrations, it is expected that the method checks this `conf` value in order to skip building and running tests, which can be useful for some CI scenarios.

Running Tests in Cross-Building Scenarios: There may be some cases where you want to build tests but cannot run them, such as in cross-building scenarios. For these rare situations, you can use the *conan.tools.build.can_run* tool as follows:

```
...

def build(self):
    cmake = CMake(self)
    cmake.configure()
    cmake.build()
    if can_run(self):
        cmake.test()
```

Note: Best practices

- The `build()` method should be as simple as possible, the heavy lifting of preparing the build should happen in the `generate()` method in order to achieve a good developer experience that can easily build locally with just `conan install .`, plus directly calling the build system or opening their IDE.

See also:

Follow the *tutorial about building packages* for more information about building from sources.

build_id()

The `build_id()` method allows you to **reuse a single build** to create multiple binary packages in the Conan cache, saving time by avoiding unnecessary rebuilds.

It is primarily an optimization tool for situations where **building each configuration separately isn't feasible**.

There are a couple of scenarios where this could be useful, for example, when a package build:

- **Generates multiple configurations in a single build run**: Some build scripts always produce both Debug and Release artifacts together, without a way to build them separately.
- **Produces one configuration but different sets of artifacts**: The build could generate the main library plus some test executables, and you want to create:
  - one package with just the library (for general use), and
  - another package that includes both the library and the test binaries (for compliance, debugging, or reproducibility).

In these scenarios, **reusing the same build folder avoids recompiling the same sources multiple times** just because you need slightly different packaging.

How does the build folder relate to the package ID and the build ID?

By default, Conan creates **one build folder per unique package ID**, where:

- Generally, the **package ID** depends on the combination of *settings*, *options*, and dependencies.
- Each different **package ID** triggers a separate `build()` execution and generates a separate build folder.

When you define the `build_id()` method, you can **force different package IDs to share the same build folder** by customizing *self.info_build*:

- `self.info_build` is like `self.info`, but it only affects the computation of the **build ID**, not the final package ID.
- Any package IDs with the same build ID will reuse the same build folder and the same build step.

Example: sharing the build for Debug and Release

```
settings = "os", "compiler", "arch", "build_type"

def build_id(self):
    self.info_build.settings.build_type = "Any"
```

- With this recipe, Debug and Release will each produce their own package IDs (and thus their own binary packages), but they will **share the same build folder**, because the build ID ignores the `build_type` setting.
- **However, you still need to run one `conan create` command per configuration** (e.g., once for Debug, once for Release). Conan will check if the build folder already exists (based on the shared build ID) and skip the actual compilation if it's already been built, only executing *package()* to create the corresponding package.

Example workflow:

```
# First build: creates the build folder + packages the Debug package
$ conan create . -s build_type=Debug

# Second build: reuses the previous build folder + packages the Release package without␣
↪rebuilding
$ conan create . -s build_type=Release
```

This way, although we called **`conan create`** twice (once per package ID), the actual build will only happen once.

Note: You can also customize `build_id()` based on options:

```
def build_id(self):
    self.info_build.options.myoption = "MyValue"
    self.info_build.options.fullsource = "Always"
```

Conditional usage of the build ID

If the `build_id()` method does not modify the `self.info_build` data, and produces the same build ID as the package ID, then the standard behavior will be applied. For example:

```
settings = "os", "compiler", "arch", "build_type"

def build_id(self):
    if self.settings.os == "Windows":
        self.info_build.settings.build_type = "Any"
```

This will only produce a different **build ID** if the package is for Windows, so it will only run the `build()` method once for all the `build_type` values.

For any other OS, Conan will behave as usual (as if the `build_id()` method was not defined), running the `build()` method for every `build_type` configuration.

Note: Best practices

The goal of the `build_id()` method is to deal with legacy build scripts that cannot easily be changed to compile one configuration at a time. We strongly recommend to just package **one package binary per package ID** for each different

configuration.

build_requirements()

The `build_requirements()` method is functionally equivalent to the `requirements()` one, it is executed just after it. It is not strictly necessary, in theory everything that is inside this method, could be done in the end of the `requirements()` one. Still, `build_requirements()` is good for having a dedicated place to define `tool_requires` and `test_requires`:

```
def build_requirements(self):
    self.tool_requires("cmake/3.23.5")
    self.test_requires("gtest/1.13.0")
```

For simple cases the attribute syntax can be enough, like `tool_requires = "cmake/3.23.5"` and `test_requires = "gtest/1.13.0"`. The method form can be necessary for conditional or parameterized requirements.

The `tool_requires` and `test_requires` methods are just a specialized instance of `requires` with some predefined trait values. See the *requires() reference* for more information about traits.

There are 2 **experimental** `confs` that can be used to avoid the expansion of these types of requirements:

- `tools.graph:skip_build` allows to skip `tool_requires` dependencies completely. This can be done if two conditions are met: the packages requiring these tools do not need to be built from sources, and the tool requirements do not affect the consumers `package_id`. If this happens, Conan will raise an error.
- `tools.graph:skip_test` allows to skip `test_requires` dependencies completely. If these dependencies are skipped, but then some package needs to be built from source and `tools.build:skip_test` was not activated, it will fail to locate the `test_requires`. So in most cases, the `tools.build:skip_test` should also be defined.

Note that if tool and/or test requirements are skipped they will not be part of the dependency graph, and they will not become part of possible generated lockfiles or package lists, with a potential impact on future reproducibility. Also, in most cases, Conan is able to mark tool and test requirements as unnecessary (`Skip`), avoiding the download of the heavy binaries, just downloading the recipe which is usually very fast. That means that in most cases these `confs` are not necessary and the Conan defaults are good, please use them being aware of the tradeoffs.

tool_requires()

The `tool_requires` is equivalent to `requires()` with the following traits:

- `build=True`. This dependency is in the "build" context, being necessary at build time, but not at application runtime, and will receive the "build" profile and configuration.
- `visible=False`. The dependency to a tool requirement is not propagated downstream. For example, one package can call `tool_requires("cmake/3.23.5")`, but that doesn't mean that the consumer packages also use `cmake`, they could even use a different build system, or a different version, without causing conflicts.
- `run=True`. This dependency has some executables or runtime that needs to be ran at build time.
- `headers=False` A tool requirement does not have headers.
- `libs=False`: A tool requirement does not have libraries to be linked by the consumer (if it had libraries they would be in the "build" context and could be incompatible with the "host" context of the consumer package).

Recall that `tool_requires` are intended exclusively for depending on tools like `cmake` or `ninja`, which run in the "build" context, but not for library-like dependencies that would be linked into binaries. For libraries or library-like dependencies, use `requires` or `test_requires`.

<host_version>

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This syntax is useful when you're using the same package recipe as a *requires* and as a *tool_requires* and you want to avoid version mismatches if any user decides to override the original *requires* version in the *host* context, i.e., the user could end up with two different versions in the host and build contexts of the same dependency.

In a nutshell, the <host_version> specifier allows us to ensure that the version resolved for the *tool_requires* always matches the one for the host requirement.

For instance, let's show a simple recipe using *protobuf*:

```
from conan import ConanFile

class mylibRecipe(ConanFile):
    name = "mylib"
    version = "0.1"
    def requirements(self):
        self.requires("protobuf/3.18.1")
    def build_requirements(self):
        self.tool_requires("protobuf/<host_version>")
```

Then, if any user wants to use *mylib/0.1*, but another version of *protobuf*, there shouldn't be any problems overriding it:

```
from conan import ConanFile

class myappRecipe(ConanFile):
    name = "myapp"
    version = "0.1"
    def requirements(self):
        self.requires("protobuf/3.21.9", override=True)
        self.requires("mylib/0.1")
```

The <host_version> defined upstream is ensuring that the host and build contexts are using the same version of that requirement.

Additionally, the syntax <host_version:mylib> can be used to specify the name of the package to be tracked, should the *requires* and *tool_requires* have different names. For instance:

```
from conan import ConanFile

class mylibRecipe(ConanFile):
    name = "mylib"
    version = "0.1"
    def requirements(self):
        self.requires("gettext/2.31")
    def build_requirements(self):
        self.tool_requires("libgettext/<host_version:gettext>")
```

Warning: It's important to note that the reference match is only performed over the package name, not the full reference, so variations on the `user` and `channel` fields are allowed, for example, having `self.requires("protobuf/3.18.1@mycompany/fork")` and `self.tool_requires("protobuf/<host_version>")` will work and look for a `protobuf/3.18.1` package in the build context, without `user` nor `channel` fields.

If we want to also keep the same `user` and `channel` fields, we'd need to specify it in the tool requirements reference as well, i.e., `self.tool_requires("protobuf/<host_version>@mycompany/fork")`.

The `<host_version>` feature also works when the requirement is replaced using the *[replace_requires]* section in your profile, so that the replaced version would be used in both contexts at once.

Note: If your `[replace_requires]` is replacing not only the version, but also the `user/channel` fields, (so for example replacing `protobuf/*: protobuf/3.18.1@mycompany/fork`) and you would like to also use the same `user` and `channel` fields in the build context, you should use the *[replace_tool_requires]* to replace it in the build context as well, otherwise the `<host_version>` will look for `protobuf/3.18.1` without the `user` and `channel` fields in the build context as explained in the previous warning, which could lead to unexpected results.

---

test_requires

The `test_requires` is equivalent to `requires()` with the following traits:

- `test=True`. This dependency is a "test" dependency, existing in the "host" context, but not aiming to be part of the final product.
- `visible=False`. The dependency to a test requirement is not propagated downstream. For example, one package can call `self.test_requires("gtest/1.13.0")`, but that doesn't mean that the consumer packages also use `gtest`, they could even use a different test framework, or the same `gtest` with a different version, without causing conflicts.

Warning: As the `test_requires` defines a `visible=False` trait, care must be taken to avoid having transitive dependencies with a normal requirement to the same packages used in the `test_requires`. This is because having direct `visible=False` requirements can create conflicts if a transitive dependency has a `visible=True` requirement to the same package that the current recipe is requiring as a test dependency. In these cases where different visibility rules reach the same package, the visible transitive dependency will be used and propagated downstream.

It is possible to further modify individual traits of `tool_requires()` and `test_requires()` if necessary, for example:

```
def build_requirements(self):
    self.tool_requires("cmake/3.23.5", options={"shared": False})
```

Warning: Defining options values for dependencies in recipes does not have strong guarantees, please check *this FAQ about options values for dependencies*. The recommended way to define options values for dependencies is in **profile files**.

For the `tool_requires/test_requires` defining the `options` trait is more feasible than with regular requires, because they are not visible and not propagated, but don't apply `options` trait to regular requires if possible, and use **profile files** instead.

Still, both `tool_requires` and `test_requires` are private (`visible=False`), only the recipe that declares them has visibility and can use them. The consumers of the package will not see or know about their existence. Consequently, they cannot be affected by consumers `options``values definitions, it doesn't matter that a consumer of the package defines options like ``cmake*:some_option=somevalue`, because `cmake` is `visible=False` and it will never receive that value from downstream consumers.

The `test_requires()` allows the `force=True` trait in case there are transitive test requirements with conflicting versions, and likewise `tool_requires()` support the `override=True` trait, for overriding possible transitive dependencies of the direct tool requirements.

Note: Best practices

- `tool_requires` are exclusively for build time **tools**, not for libraries that would be included and linked into the consumer package. For libraries with some special characteristics, use a `requires()` with custom trait values.
- The `self.test_requires()` and `self.tool_requires()` methods should exclusively be used in the `build_requirements()` method, with the only possible exception being the `requirements()` method. Using them in any other method is forbidden. To access information about dependencies when necessary in some methods, the *self.dependencies* attribute should be used.

See also:

- Follow the *tutorial about consuming Conan packages as tools*.
- Read the *tutorial about creating tool_requires packages*.
- *Using the same requirement as a requires and as a tool_requires*

build_system_requirements()

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `build_system_requirements()` method is a companion to `system_requirements()` intended specifically for system packages that are only needed when **building from source** (i.e., when the package binary is being compiled). It is not called when a pre-built binary is retrieved from a server or cache.

The key difference from `system_requirements()` is that `build_system_requirements()` installs packages using the **build machine architecture** (`settings_build.arch`) rather than the host architecture. This is especially important in cross-compilation scenarios where the build and host machines have different architectures.

Then, these requirements are intended to be "tools", executables used at build time, but not libraries that would be linked in the consumers libraries or applications.

```
from conan import ConanFile
from conan.tools.system.package_manager import Apt

class MyPkg(ConanFile):
    name = "mypkg"
    version = "1.0"
    settings = "os", "arch"
```

(continues on next page)

(continued from previous page)

```
    def build_system_requirements(self):
        # Installed using build machine arch (settings_build.arch),
        # only when building from source
        apt = Apt(self)
        apt.install(["build-essential", "pkg-config"])

    def system_requirements(self):
        # Installed using host arch (settings.arch), always
        apt = Apt(self)
        apt.install(["libssl-dev"])
```

Note: `build_system_requirements()` is only invoked when the package is being built from source. If the binary is already available in the cache or a remote, this method is skipped.

See also:

- *system_requirements()*
- *conan.tools.system.package_manager*

compatibility()

Warning: This is a **preview** feature

The `compatibility()` method implements the same binary compatibility mechanism than the *compatibility plugin*, but at the recipe level. In general, the global compatibility plugin should be good for most cases, and only require the recipe method for exceptional cases.

This method can be used in a *conanfile.py* to define packages that are compatible between each other. If there are no binaries available for the requested settings and options, this mechanism will retrieve the compatible package's binaries if they exist. This method should return a list of compatible configurations.

For example, if we want that binaries built with gcc versions 4.8, 4.7 and 4.6 to be considered compatible with the ones compiled with 4.9 we could declare a `compatibility()` method like this:

```
def compatibility(self):
    if self.settings.compiler == "gcc" and self.settings.compiler.version == "4.9":
        return [{"settings": [("compiler.version", v)]}
                for v in ("4.8", "4.7", "4.6")]
```

The format of the list returned is as shown below:

```
[
    {
        "settings": [(<setting>, <value>), (<setting>, <value>), ...],
        "options": [(<option>, <value>), (<option>, <value>), ...]
    },
    {
        "settings": [(<setting>, <value>), (<setting>, <value>), ...],
        "options": [(<option>, <value>), (<option>, <value>), ...]
```

(continues on next page)

(continued from previous page)

```
    },
    ...
]
```

See also:

- Read the *binary model reference* for a full view of the Conan binary model.
- See more about *customizing the binary compatibility of your packages*

configure()

The `configure()` method should be used for the configuration of settings and options in the recipe for later use in the different methods like `generate()`, `build()` or `package()`. This method executes while building the dependency graph and expanding the packages dependencies, which means that when this method executes the dependencies are still not there, they do not exist, and it is not possible to access `self.dependencies`.

For example, for a C (not C++) library, the `compiler.libcxx` and `compiler.cppstd` settings shouldn't even exist during the `build()`. It is not only that they are not part of the `package_id`, but they shouldn't be used in the build process at all. They will be defined in the profile, because other packages in the graph can be C++ packages and need them, but it is the responsibility of this recipe to remove them so they are not used in the recipe:

```
settings = "os", "compiler", "build_type", "arch"

def configure(self):
    # Not all compilers have libcxx subsetting, so we use rm_safe
    # to avoid exceptions
    self.settings.rm_safe("compiler.libcxx")
    self.settings.rm_safe("compiler.cppstd")

def package_id(self):
    # No need to delete those settings here, they were already deleted
    pass
```

Note: From Conan 2.4, the above `configure()` is not necessary if defined `languages = "C"` recipe attribute (experimental).

For packages where you want to remove every subsetting of a setting, you can use the `rm_safe` method with a wildcard:

```
settings = "os", "compiler", "build_type", "arch"

def configure(self):
    self.settings.rm_safe("compiler.*")
```

This will remove all the subsettings of the `compiler` setting, like `compiler.libcxx` or `compiler.cppstd`, but keep the `compiler` setting itself (Which `self.settings.rm_safe("compiler")` would remove).

Likewise, for a package containing a library, the `fPIC` option really only applies when the library is compiled as a static library, but otherwise, the `fPIC` option doesn't make sense, so it should be removed:

```
options = {"shared": [True, False], "fPIC": [True, False]}
default_options = {"shared": False, "fPIC": True}
```

(continues on next page)

(continued from previous page)

```
def configure(self):
    if self.options.shared:
        # fPIC might have been removed in config_options(), so we use rm_safe
        self.options.rm_safe("fPIC")
```

Available automatic implementations

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

When the `configure()` method is not defined, Conan can automatically manage some conventional options if specified in the *implements* ConanFile attribute:

auto_shared_fpic

Options automatically managed:

- `fPIC` (True, False).
- `shared` (True, False).
- `header_only` (True, False).

It can be added to the recipe like this:

```
from conan import ConanFile

class Pkg(ConanFile):
    implements = ["auto_shared_fpic"]
    ...
```

Then, if no `configure()` method is specified in the recipe, Conan will automatically manage the fPIC setting in the `configure` step like this:

```
if conanfile.options.get_safe("header_only"):
    conanfile.options.rm_safe("fPIC")
    conanfile.options.rm_safe("shared")
elif conanfile.options.get_safe("shared"):
    conanfile.options.rm_safe("fPIC")
```

Be aware that adding this implementation to the recipe may also affect the *configure* step.

If you need to implement custom behaviors in your recipes but also need this logic, it must be explicitly declared:

```
def configure(self):
    if conanfile.options.get_safe("header_only"):
        conanfile.options.rm_safe("fPIC")
        conanfile.options.rm_safe("shared")
    elif conanfile.options.get_safe("shared"):
        conanfile.options.rm_safe("fPIC")
```

(continues on next page)

(continued from previous page)

```
    self.settings.rm_safe("compiler.libcxx")
    self.settings.rm_safe("compiler.cppstd")
```

Recipes can suggest values for their dependencies options as `default_options = {"*:shared": True}`, but it is not possible to do that conditionally. For this purpose, it is also possible to use the `configure()` method:

```
def configure(self):
    if something:
        self.options["*"].shared = True
```

Warning: Defining options values for dependencies in recipes does not have strong guarantees, please check *this FAQ about options values for dependencies*. The recommended way to define options values for dependencies is in **profile files**.

Note: Best practices

- Recall that it is **not** possible to define `settings` or `conf` values in recipes, they are read only.
- The definition of `options` values is only a "suggestion", depending on the graph computation, priorities, etc., the final value of `options` can be different from the one set by the recipe.

See also:

- Follow the *tutorial about recipe configuration methods*.

config_options()

The `config_options()` method is used to configure or constrain the available options in a package **before** assigning them a value. A typical use case is to remove an option in a given platform. For example, the `SSE2` flag doesn't exist in architectures different than 32 bits, so it should be removed in this method like so:

```
def config_options(self):
    if self.settings.arch != "x86_64":
        del self.options.with_sse2
```

The `config_options()` method executes:

- Before calling the `configure()` method.
- Before assigning the `options` values.
- After `settings` are already defined.

Available automatic implementations

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

When the `config_options()` method is not defined, Conan can automatically manage some conventional options if specified in the *implements* ConanFile attribute:

auto_shared_fpic

Options automatically managed:

- `fPIC` (True, False).

It can be added to the recipe like this:

```
from conan import ConanFile

class Pkg(ConanFile):
    implements = ["auto_shared_fpic"]
    ...
```

Then, if no `config_options()` method is specified in the recipe, Conan will automatically manage the fPIC setting in the `config_options` step like this:

```
if conanfile.settings.get_safe("os") == "Windows":
    conanfile.options.rm_safe("fPIC")
```

Be aware that adding this implementation to the recipe may also affect the *configure* step.

If you need to implement custom behaviors in your recipes but also need this logic, it must be explicitly declared:

```
def config_options(self):
    if conanfile.settings.get_safe("os") == "Windows":
        conanfile.options.rm_safe("fPIC")
    if self.settings.arch != "x86_64":
        del self.options.with_sse2
```

See also:

- Follow the *tutorial about recipe configuration methods*.

deploy()

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `deploy()` method is intended to deploy (copy) artifacts from the current package. It only executes at `conan install` time, when the `--deployer-package` argument is provided, otherwise `deploy()` is ignored.

Artifacts should be deployed to the `self.deploy_folder`, by default the current folder. A custom destination can be defined with `--deployer-folder`. A basic `deploy()` method would copy all files from the package folder to the deploy folder:

```
from conan import ConanFile
from conan.tools.files import copy

class Pkg(ConanFile):

    def deploy(self):
        copy(self, "*", src=self.package_folder, dst=self.deploy_folder)
```

Refer to the documentation of the *conan install command* for more information.

If you need to run binaries from your build dependencies, the recommended approach is to apply the env from a `VirtualBuildEnv`, such as:

```
def deploy(self):
    venv = VirtualBuildEnv(self)
    with venv.vars().apply():
        self.run("mytool")
```

Note: Best practices

- Only "binary" package artifacts can be deployed, copying from the `self.package_folder`. It is recommended to copy only from the package folder, not other folders.
- The `deploy()` method is intended for final production deployments or the installation of binaries on the machine, as it extracts the files from the Conan cache. It is not intended for normal development operations, nor to build Conan packages against deployed binaries. The recommendation is to build against packages in the Conan cache.
- The `self.deploy_folder` should only be used from within the `deploy()` method.

export()

Equivalent to the `exports` attribute, but in method form. This method will be called at `export` time, which happens in the `conan export`, `conan export-pkg` and `conan create` commands, and it is intended to allow copying files from the user folder to the Conan cache folders, thus making files becoming part of the recipe. These sources will be uploaded to the servers together with the recipe and are always downloaded together with the recipe.

The current working directory will be `self.recipe_folder`, and it can use the `self.export_folder` as the destination folder for using `copy()` or your custom copy.

```
from conan import ConanFile
from conan.tools.files import copy

class Pkg(ConanFile):
    def export(self):
        # This LICENSE file is intended to be the license of the current conanfile.py␣
↪recipe
        # and go with it. It is not intended to be the license of the final package (for␣
↪that
        # purpose export_sources() would be recommended)
        copy(self, "LICENSE.md", self.recipe_folder, self.export_folder)
```

There are 2 files that are always exported to the cache, without being explicitly defined in the recipe: the `conanfile.py` recipe, and the `conandata.yml` file if it exists. The `conandata.yml` file is automatically loaded whenever the `conanfile.py` is loaded, becoming the `self.conan_data` attribute, so it is a intrinsic part of the recipe, so it is part of the "exported" recipe files, not of the "exported" source files.

Note: Best practices

- The recipe files must be configuration independent. Those files are common for all configurations, thus it is not possible to do conditional `export()` to different settings, options, or platforms. Do not try to do any kind of conditional export. If necessary export all the files necessary for all configurations at once.
- The `export()` method does not receive any information from profiles, not even `conf`. Only the `global.conf` will be available, and in any case it is not possible to use that `conf` to define conditionals.
- Keep the `export()` method simple. Its intention is to copy files from the user folder to the cache to store those files together with the recipe.
- The exported files must be small. Exporting big files with the recipe will make the resolution of dependencies much slower the resolution.
- Only files that are necessary for the evaluation of the `conanfile.py` recipe must be exported with this method. Files necessary for building from sources should be exported with the `exports_sources` attribute or the *export_source()* method.

export_sources()

Equivalent to the `exports_sources` attribute, but in method form. This method will be called at `export` time, which happens in `conan export`, `conan export-pkg` and `conan create` commands, and it is intended to allow copying files from the user folder to the Conan cache folders, those files becoming part of the recipe sources. These sources will be uploaded to the servers together with the recipe, but are typically not downloaded unless the package is being built from source.

The current working directory will be `self.recipe_folder`, and it can use the `self.export_sources_folder` as the destination folder for using `copy()` or your custom copy.

```
from conan import ConanFile
from conan.tools.files import copy

class Pkg(ConanFile):
    def export_sources(self):
        # This LICENSE.md is a source file intended to be part of the final package
        # it is not the license of the current recipe
        copy(self, "LICENSE.md", self.recipe_folder, self.export_sources_folder)
```

The method might be able to read files in the recipe folder and do something with it:

```
import os
from conan import ConanFile
from conan.tools.files import load, save

class Pkg(ConanFile):

    def export_sources(self):
        content = load(self, os.path.join(self.recipe_folder, "data.txt"))
        save(self, os.path.join(self.export_sources_folder, "myfile.txt"), content)
```

The `export_conandata_patches()` is a high-level helper function that does the export of the patches defined in the `conandata.yml` file, which could be later applied with `apply_conandata_patches()` in the `source()` method.

```
from conan.tools.files import export_conandata_patches

class Pkg(ConanFile):

    def export_sources(self):
        export_conandata_patches(self)
```

Note: Best practices

- The recipe sources must be configuration independent. Those sources are common for all configurations, thus it is not possible to do conditional `export_sources()` to different settings, options, or platforms. Do not try to do any kind of conditional export. If necessary export all the files necessary for all configurations at once.
- The `export_sources()` method does not receive any information from profiles, not even `conf`. Only the `global.conf` will be available, and in any case it is not possible to use that `conf` to define conditionals.
- Keep the `export_source()` method simple. Its intention is to copy files from the user folder to the cache to store those files together with the recipe.

finalize()

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Package immutability is an important concept in Conan. It ensures that the package is not modified after it has been built and packaged, so that the package id is consistent and the package can be reused in different machines.

This method is intended for customization of the package in the running machine, allowing modifications to the package that will be used by the consumers of the package, but not modifying the original package inside the Conan cache. This method is called after the package has been installed in the local cache and the modifications are not uploaded to any remote server, they are only local.

The main use-cases of this method include:

- Having different packages that get modified when they are run (Like python tools that generate pycache files as part of their execution)
- Modifying the package to be used in the local machine (Like creating a `conf` file with the necessary environment variables for the package to be used in the local machine)

These changes are transparent to the consumers of the package, they will use the customized package as if they were dealing with the original package, so changes made in this method should work.

finalize() example usage

The most common use-case of this method is to avoid corrupting the immutability of the package.

For example, if a package generates some files during its execution, like a python package that generates pycache files, you can use this method to generate those files in the local machine, without affecting the original package. This is the case for tools like Meson.

```
from conan import ConanFile, conan_version
from conan.tools.files import copy


class MesonPackage(ConanFile):
    ...

    def package(self):
        copy(self, "*", src=self.source_folder, dst=os.path.join(self.package_folder,
↪"bin"))
        ...

    def finalize(self):
        copy(self, "*", src=self.immutable_package_folder, dst=self.package_folder)
```

Here we are copying the files from the immutable package folder to the finalized package folder, which inside the finalize method (and everywhere after that point) will be the new package folder, so that any modifications done by the package are done in the local finalized folder, without affecting the original package.

For cases where it's necessary to access the original package, the `immutable_package_folder` attribute is available both in the same recipe's `self.immutable_package_folder` and thru the `self.dependencies[<package_name>].immutable_package_folder` attribute in the dependants' recipe. This info is also serialized as part of the graph information in **`conan graph info`** etc.

As this method must have a 1 to 1 correspondence to the generated package id, access to `self.settings`, `self.options` and `self.cpp_info` is forbidden inside the `finalize()` method, and **must** be done thru the `self.info` attribute.

Note: Without using this approach, the package would generate the pycache files in the package folder, and thus there would be a need to set `PYTHONDONTWRITEBYTECODE` to avoid mutating the package, but this would affect performance, and performing cache integrity checks either thru **`conan cache check-integrity`** or as part of the upload processes in **`conan upload ... --check`** would raise errors if the modified package was ever checked.

Warning: This is not a replacement for the *post_package* hook. The hook runs after the creation of the package for a chance to modify it before the package_id is computed, but it is not intended for modifications of the package for a particular running machine.

generate()

This method will run after the computation and installation of the dependency graph. This means that it will run after a **conan install** command, or when a package is being built in the cache, it will be run before calling the `build()` method.

The purpose of `generate()` is to prepare the build, generating the necessary files. These files would typically be:

- Files containing information to locate the dependencies, as `xxxx-config.cmake` CMake config scripts, or `xxxx.props` Visual Studio property files.
- Environment activation scripts, like `conanbuild.bat` or `conanbuild.sh`, that define all the necessary environment variables necessary for the build.
- Toolchain files, like `conan_toolchain.cmake`, that contains a mapping between the current Conan settings and options, and the build system specific syntax. `CMakePresets.json` for CMake users using modern versions.
- General purpose build information, as a `conanbuild.conf` file that could contain information for some toolchains like autotools to be used in the `build()` method.
- Specific build system files, like `conanvcvars.bat`, that contains the necessary Visual Studio *vcvars.bat* call for certain build systems like Ninja when compiling with the Microsoft compiler.

The idea is that the `generate()` method implements all the necessary logic, making both the user manual builds after a **conan install** very straightforward, and also the `build()` method logic simpler. The build produced by a user in their local flow should result in exactly the same one as the build done in the cache with a `conan create` without effort.

Generation of files happens in the *generators_folder* as defined by the current layout.

In many cases, the `generate()` method might not be necessary, and declaring the `generators` attribute could be enough:

```
from conan import ConanFile

class Pkg(ConanFile):
    generators = "CMakeDeps", "CMakeToolchain"
```

But the `generate()` method can explicitly instantiate those generators, use them conditionally (like using one build system in Windows, and another build system integration in other platforms), customize them, or provide a complete custom generation.

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain

class Pkg(ConanFile):

    def generate(self):
        tc = CMakeToolchain(self)
        # customize toolchain "tc"
        tc.generate()
        # Or provide your own custom logic
```

The current working directory for the `generate()` method will be the `self.generators_folder` defined in the current layout.

For custom integrations, putting code in a common `python_require` would be a good way to avoid repetition in multiple recipes:

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain

class Pkg(ConanFile):

    python_requires = "mygenerator/1.0"

    def generate(self):
        mygen = self.python_requires["mygenerator"].module.MyGenerator(self)
        # customize mygen behavior, like mygen.something= True
        mygen.generate()
```

In case it is necessary to collect or copy some files from the dependencies, it is also possible to do it in the `generate()` method, accessing `self.dependencies`. Listing the different include directories, lib directories from a dependency "mydep" would be possible like this:

```
from conan import ConanFile

class Pkg(ConanFile):

    def generate(self):
        info = self.dependencies["mydep"].cpp_info
        self.output.info("**includedirs:{}**".format(info.includedirs))
        self.output.info("**libdirs:{}**".format(info.libdirs))
        self.output.info("**libs:{}**".format(info.libs))
```

And copying the shared libraries in Windows and OSX to the current build folder, could be done like:

```
from conan import ConanFile

class Pkg(ConanFile):

    def generate(self):
        # NOTE: In most cases it is not necessary to copy the shared libraries
        # of dependencies to use them. Conan environment generators that create
        # environment scripts allow to use the shared dependencies without copying
        # them to the current location
        for dep in self.dependencies.values():
            # This code assumes dependencies will only have 1 libdir/bindir, if for some
            # reason they have more than one, it will fail. Use ``dep.cpp_info.libdirs``
            # and ``dep.cpp_info.bindirs`` lists for those cases.
            copy(self, "*.dylib", dep.cpp_info.libdir, self.build_folder)
            # In Windows, dlls are in the "bindir", not "libdir"
            copy(self, "*.dll", dep.cpp_info.bindir, self.build_folder)
```

Note: Best practices

- Copying shared libraries to the current project in `generate()` is not a necessary in most cases, and shouldn't be done as a general approach. Instead, the Conan environment generators, which are enabled by default, will automatically generate environment scripts like `conanbuild.bat|.sh` or `conanrun.bat|.sh` with the necessary environment variables (`PATH`, `LD_LIBRARY_PATH`, etc), to correctly locate and use the shared libraries of dependencies at runtime.
- Accessing dependencies `self.dependencies["mydep"].package_folder` is possible, but it can be `None`

(see the `package_folder` field below). Always check for `None` before using it, and prefer referencing the `cpp_info.xxxdirs` instead, which are always defined and are resolved transparently for both packages in the cache and "editable" packages.

See also:

- Follow the *tutorial about preparing build from source in recipes*.

self.dependencies

Conan recipes provide access to their dependencies via the `self.dependencies` attribute. This attribute is generally used by generators like `CMakeDeps` or `MSBuildDeps` to generate the necessary files for the build.

This section documents the `self.dependencies` attribute, as it might be used by users both directly in recipe or indirectly to create custom build integrations and generators.

Dependencies interface

It is possible to access each one of the individual dependencies of the current recipe, with the following syntax:

```
class Pkg(ConanFile):
    requires = "openssl/0.1"

    def generate(self):
        openssl = self.dependencies["openssl"]
        # access to members
        openssl.ref.version
        openssl.ref.revision # recipe revision
        openssl.options
        openssl.settings

        if "zlib" in self.dependencies:
            # do something
```

Some **important** points:

- All the information is **read only**. Any attempt to modify dependencies information is an error and can raise at any time, even if it doesn't raise yet.
- It is not possible either to call any methods or any attempt to reuse code from the dependencies via this mechanism.
- This information does not exist in some recipe methods, only in those methods that evaluate after the full dependency graph has been computed. It will not exist in `configure()`, `config_options`, `export()`, `export_source()`, `set_name()`, `set_version()`, `requirements()`, `build_requirements()`, `system_requirements()`, `source()`, `init()`, `layout()`. Any attempt to use it in these methods can raise an error at any time.
- At the moment, this information should only be used in `generate()` and `validate()` methods. For any other use, please submit a Github issue.

Not all fields of the dependency conanfile are exposed, the current fields are:

- **package_folder**: The folder location of the dependency package binary. It can be `None` in some cases:

  - When the dependency binary is skipped because it is not necessary (for example, a static library that is a transitive dependency of a shared library). The binary is not even downloaded from the server, so there is no folder to point to.
  - It is **not** `None` for packages in *editable mode*. In that case it points to the local project root folder (where the `conanfile.py` of the editable package is). Note that there is no actual "package" there: the files are not laid out as in a package, and the folder can contain sources, build files and any other project file. Consumers that need to locate artifacts should use `cpp_info` (defined in the `layout()` method), which works transparently for editable and cache packages.

  Any recipe, generator or deployer accessing `package_folder` must be prepared for these cases.

- **recipe_folder**: The folder containing the `conanfile.py` (and other exported files) of the dependency
- **recipe_metadata_folder**: The folder containing optional recipe metadata files of the dependency
- **package_metadata_folder**: The folder containing optional package metadata files of the dependency
- **immutable_package_folder**: The folder containing the immutable artifacts when `finalize()` method exists
- **ref**: A *RecipeReference* object that contains `name`, `version`, `user`, `channel` and `revision` (recipe revision)
- **pref**: An object that contains `ref`, `package_id` and `revision` (package revision)
- **buildenv_info**: `Environment` object with the information of the environment necessary to build
- **runenv_info**: `Environment` object with the information of the environment necessary to run the app
- **cpp_info**: includedirs, libdirs, etc for the dependency.
- **settings**: The actual settings values of this dependency
- **settings_build**: The actual build settings values of this dependency
- **options**: The actual options values of this dependency
- **context**: The context (build, host) of this dependency
- **conf_info**: Configuration information of this dependency, intended to be applied to consumers.
- **dependencies**: The transitive dependencies of this dependency
- **is_build_context**: Return `True` if `context == "build"`.
- **conan_data**: The `conan_data` attribute of the dependency that comes from its `conandata.yml` file
- **license**: The `license` attribute of the dependency
- **description**: The `description` attribute of the dependency
- **homepage**: The `homepage` attribute of the dependency
- **url**: The `url` attribute of the dependency
- **package_type**: The `package_type` of the dependency
- **languages**: The `languages` of the dependency.
- **extension_properties**: The `extension_properties` of the dependency. Should be treated as read-only.
- **recipe**: The `recipe` type of the dependency (e.g., "Cache", "Editable"). This should only be used for informational or reporting purposes. For most cases, the contents of the dependency are abstracted from the type of the dependency, so using it for any kind of conditional logic on the consumers side is discouraged in most cases, except for checking when a dependency is `Editable` from a custom `deployer`, to prevent deploying such editable package.

Iterating dependencies

It is possible to iterate in a dict-like fashion all dependencies of a recipe. Take into account that `self.dependencies` contains all the current dependencies, both direct and transitive. Every upstream dependency of the current one that has some effect on it, will have an entry in this `self.dependencies`.

Iterating the dependencies can be done as:

```
requires = "zlib/1.3.1", "poco/1.9.4"

def generate(self):
    for require, dependency in self.dependencies.items():
        self.output.info("Dependency is direct={}: {}".format(require.direct, dependency.
↪ref))
```

will output:

```
conanfile.py (hello/0.1): Dependency is direct=True: zlib/1.3.1
conanfile.py (hello/0.1): Dependency is direct=True: poco/1.9.4
conanfile.py (hello/0.1): Dependency is direct=False: pcre/8.44
conanfile.py (hello/0.1): Dependency is direct=False: expat/2.4.1
conanfile.py (hello/0.1): Dependency is direct=False: sqlite3/3.35.5
conanfile.py (hello/0.1): Dependency is direct=False: openssl/1.1.1k
conanfile.py (hello/0.1): Dependency is direct=False: bzip2/1.0.8
```

Where the `require` dictionary key is a "requirement", and can contain specifiers of the relation between the current recipe and the dependency. At the moment they can be:

- `require.direct`: boolean, `True` if it is direct dependency or `False` if it is a transitive one.
- `require.build`: boolean, `True` if it is a `build_require` in the build context, as `cmake`.
- `require.test`: boolean, `True` if its a `build_require` in the host context (defined with `self.test_requires()`), as `gtest`.

The `dependency` dictionary value is the read-only object described above that access the dependency attributes.

The `self.dependencies` contains some helpers to filter based on some criteria:

- `self.dependencies.host`: Will filter out requires with `build=True`, leaving regular dependencies like `zlib` or `poco`.
- `self.dependencies.direct_host`: Will filter out requires with `build=True` or `direct=False`
- `self.dependencies.build`: Will filter out requires with `build=False`, leaving only `tool_requires` in the build context, as `cmake`.
- `self.dependencies.direct_build`: Will filter out requires with `build=False` or `direct=False`
- `self.dependencies.test`: Will filter out requires with `build=True` or with `test=False`, leaving only test requirements as `gtest` in the host context.

They can be used in the same way:

```
requires = "zlib/1.3.1", "poco/1.9.4"

def generate(self):
    cmake = self.dependencies.direct_build["cmake"]
    for require, dependency in self.dependencies.build.items():
        # do something, only build deps here
```

Note: By default, indexing into `self.dependencies[...]` returns dependencies in the *host* context. If you need to access dependencies that belong to the *build* context (e.g. `cmake`), use the `self.dependencies.build[...]` filter helper, as shown in the example above.

Dependencies `cpp_info` interface

The `cpp_info` interface is heavily used by build systems to access the data. This object defines global and per-component attributes to access information like the include folders:

```
def generate(self):
    cpp_info = self.dependencies["mydep"].cpp_info
    cpp_info.includedirs
    cpp_info.libdirs

    cpp_info.components["mycomp"].includedirs
    cpp_info.components["mycomp"].libdirs
```

All the paths declared in the `cppinfo` object (like `cpp_info.includedirs`) are absolute paths and works whether the dependency is in the cache or is an *editable package*.

See also:

- *CppInfo* model.

init()

This is an optional method for initializing conanfile values, designed for inheritance from `python_requires`. Assuming we have a `base/1.1` recipe:

Listing 29: base/conanfile.py

```
from conan import ConanFile

class MyConanfileBase:
    license = "MyLicense"
    settings = "os", # tuple!


class PyReq(ConanFile):
    name = "base"
    version = "1.1"
    package_type = "python-require"
```

We could reuse and inherit from it with:

Listing 30: pkg/conanfile.py

```
from conan import ConanFile

class Pkg(ConanFile):
    license = "MIT"
```

(continues on next page)

(continued from previous page)

```
    settings = "arch", # tuple!
    python_requires = "base/1.1"
    python_requires_extend = "base.MyConanfileBase"

    def init(self):
        base = self.python_requires["base"].module.MyConanfileBase
        self.settings = base.settings + self.settings  # Note, adding 2 tuples = tuple
        self.license = base.license  # License is overwritten
```

The final `Pkg` conanfile will have both `os` and `arch` as settings, and `MyLicense` as license.

To extend the `options` of the base class, it is necessary to call the `self.options.update()` method:

Listing 31: base/conanfile.py

```
from conan import ConanFile

class BaseConan:
    options = {"base": [True, False]}
    default_options = {"base": True}

class PyReq(ConanFile):
    name = "base"
    version = "1.0.0"
    package_type = "python-require"
```

When the `init()` is called, the `self.options` object is already initialized. Then, updating the `self.default_options` is useless, and it is necessary to update the `self.options` with both the base class options and the base class default options values:

Listing 32: pkg/conanfile.py

```
from conan import ConanFile

class DerivedConan(ConanFile):
    name = "derived"
    python_requires = "base/1.0.0"
    python_requires_extend = "base.BaseConan"
    options = {"derived": [True, False]}
    default_options = {"derived": False}

    def init(self):
        base = self.python_requires["base"].module.BaseConan
        # Note we pass the base options and default_options
        self.options.update(base.options, base.default_options)
```

This method can also be useful if you need to unconditionally initialize class attributes like `license` or `description` or any other from datafiles other than *conandata.yml*. For example, you can have a *json* file containing the information about the `license`, `description` and `author` for the library:

Listing 33: data.json

```
{"license": "MIT", "description": "This is my awesome library.", "author": "Me"}
```

Then, you can load that information from the `init()` method:

```
import os
import json
from conan import ConanFile
from conan.tools.files import load


class Pkg(ConanFile):
    exports = "data.json" # Important that it is exported with the recipe

    def init(self):
        data = load(self, os.path.join(self.recipe_folder, "data.json"))
        d = json.loads(data)
        self.license = d["license"]
        self.description = d["description"]
        self.author = d["author"]
```

Note: Best practices

- Try to keep your `python_requires` as simple as possible, and do not reuse attributes from them (the main need for the `init()` method), trying to avoid the complexity of this `init()` method. In general inheritance can have more issues than composition (or in other words "use composition over inheritance" as a general programming good practice), so try to avoid it if possible.
- Do not abuse `init()` for other purposes other than listed here, nor use the Python private `ConanFile.__init__` constructor.
- The `init()` method executes at recipe load time. It cannot contain conditionals on settings, options, conf, or use any dependencies information other than the above `python_requires`.

layout()

In the layout() method you can adjust `self.folders` and `self.cpp`.

self.folders

- **self.folders.source** (Defaulted to ""): Specifies a subfolder where the sources are. The `self.source_folder` attribute inside the `source(self)` and `build(self)` methods will be set with this subfolder. The *current working directory* in the `source(self)` method will include this subfolder. The *export_sources* and *exports* sources will also be copied to the root source directory. It is used in the cache when running **`conan create`** (relative to the cache source folder) as well as in a local folder when running **`conan build`** (relative to the local current folder).
- **self.folders.build** (Defaulted to ""): Specifies a subfolder where the files from the build are. The `self.build_folder` attribute and the *current working directory* inside the `build(self)` method will be set with this subfolder. It is used in the cache when running **`conan create`** (relative to the cache source folder) as well as in a local folder when running **`conan build`** (relative to the local current folder).

- **self.folders.generators** (Defaulted to ""): Specifies a subfolder in which to write the files from the generators and the toolchains. In the cache, when running **`conan create`**, this subfolder will be relative to the root build folder and when running the **`conan install`** command it will be relative to the current working directory.
- **self.folders.root** (Defaulted to `None`): Specifies a parent directory where the sources, generators, etc., are located specifically when the `conanfile.py` is located in a separated subdirectory. Check *this example* on how to use **self.folders.root**.
- **self.folders.subproject** (Defaulted to `None`): Specifies a subfolder where the `conanfile.py` is relative to the project root. This is particularly useful for *layouts with multiple subprojects*
- **self.folders.build_folder_vars** (Defaulted to `None`): Use settings, options and/or `self.name` and `self.version` to produce a different build folder and different CMake presets names.

self.cpp

The `layout()` method allows to declare `cpp_info` objects not only for the final package (like the classic approach with the `self.cpp_info` in the `package_info(self)` method) but for the `self.source_folder` and `self.build_folder`.

The fields of the `cpp_info` objects at `self.cpp.build` and `self.cpp.source` are the same described *here*. Components are also supported.

Properties to declare all the information needed by the consumers of a package: include directories, library names, library paths... Used both for *editable packages* and regular packages in the cache.

There are three objects available in the `layout()` method:

- **self.cpp.package**: For a regular package being used from the Conan cache. Describes the contents of the final package. Exactly the same as in the `package_info()` `self.cpp_info`, but in the `layout()` method.
- **self.cpp.source**: For "editable" packages, to describe the artifacts under `self.source_folder`. These can cover:
  - `self.cpp.source.includedirs`: To specify where the headers are at development time, like the typical `src` folder, before being packaged in the `include` package folder.
  - `self.cpp.source.libdirs` and `self.cpp.source.libs` could describe the case where libraries are committed to source control (hopefully exceptional case), so they are not part of the build results, but part of the source.
- **self.cpp.build**: For "editable" packages, to describe the artifacts under `self.build_folder`.
  - `self.cpp.build.libdirs` will express the location of the built libraries before being packaged. They can often be found in a folder like `x64/Release`, or `release64` or similar.
  - `self.cpp.build.includedirs` can define the location of headers that are generated at build time, like headers stubs generated by some tools.

```
def layout(self):
    ...
    self.folders.source = "src"
    self.folders.build = "build"

    # In the local folder (when the package is in development, or "editable") the␣
↪artifacts can be found:
    self.cpp.source.includedirs = ["my_includes"]
    self.cpp.build.libdirs = ["lib/x86_64"]
    self.cpp.build.libs = ["foo"]
```

(continues on next page)

(continued from previous page)

```
    # In the Conan cache, we packaged everything at the default standard directories,␣
↪the library to link
    # is "foo"
    self.cpp.package.libs = ["foo"]
```

When defining `.location` or `.link_location` in `self.cpp.build` or `self.cpp.source` for editable packages, Conan automatically relativizes these paths with respect to the build/source folder. This ensures that the location correctly resolves relative to the package's build or source directory:

```
def layout(self):
    cmake_layout(self)
    # location is relative to self.build_folder for editable packages
    self.cpp.build.location = "lib/myexe"
```

See also:

- Read more about the usage of the `layout()` in *this tutorial* and Conan package layout
- *here*.

Ordering and `conan source` compatibility

The `layout()` method is also called by the **`conan source`** command, which runs without a host profile. This has two implications for how `layout()` should be written:

- **Order matters**: define the inputs, source-related parts first (`self.folders.source`, `self.folders.root`, `self.folders.subproject`). These are independent of settings and options, so they will always be correctly defined even when **`conan source`** is invoked.
- **Do not fail when settings/options are not defined**: the build-related parts (`self.folders.build`, `self.folders.generators`, `self.cpp.build...`) typically depend on `self.settings` or `self.options`, which are empty during **`conan source`**. The `layout()` method must not raise in that case. There are two common approaches:
  - Use `self.settings.get_safe(...)` / `self.options.get_safe(...)`, which return `None` when the value is not defined instead of raising. This is the approach used by the built-in `cmake_layout()`, which produces a build folder like `build/None` when **`conan source`** is invoked. That value is not used by **`conan source`**, so it is harmless.
  - Early-return from `layout()` after defining the source parts when settings are not available, for example `if not self.settings.get_safe(): return`.

```
def layout(self):
    # 1) Source parts: independent of settings/options, define them first
    self.folders.source = "src"

    # 2) Build parts: may depend on settings/options, use get_safe to avoid
    #    failing during `conan source` when settings are empty
    build_type = self.settings.get_safe("build_type")
    # might be "build/None", but won't really be used in "conan source" commands
    self.folders.build = f"build/{build_type}"
    self.folders.generators = f"build/{build_type}/generators"
    self.cpp.build.libdirs = [f"build/{build_type}"]
```

(continues on next page)

(continued from previous page)

```
    # The "cpp_info" editable information needs also configuration, goes
    # together with the build parts
    self.cpp.source.includedirs = ["include"]
```

Environment variables and configuration

There are some packages that might define some environment variables in their `package_info()` method via `self.buildenv_info`, `self.runenv_info`. Other packages can also use `self.conf_info` to pass configuration to their consumers.

This is not an issue as long as the value of those environment variables or configuration do not require using the `self.package_folder`. If they do, then their values will not be correct for the "source" and "build" layouts. Something like this will be **broken** when used in `editable` mode:

```
import os
from conan import ConanFile

class SayConan(ConanFile):
    ...
    def package_info(self):
        # This is BROKEN if we put this package in editable mode
        self.runenv_info.define_path("MYDATA_PATH",
                                     os.path.join(self.package_folder, "my/data/path"))
```

When the package is in editable mode, for example, `self.package_folder` is `None`, as obviously there is no package yet. The solution is to define it in the `layout()` method, in the same way the `cpp_info` can be defined there:

```
from conan import ConanFile

class SayConan(ConanFile):
    ...
    def layout(self):
        # The final path will be relative to the self.source_folder
        self.layouts.source.buildenv_info.define_path("MYDATA_PATH", "my/source/data/path
↪")
        # The final path will be relative to the self.build_folder
        self.layouts.build.buildenv_info.define_path("MYDATA_PATH2", "my/build/data/path
↪")
        # The final path will be relative to the self.build_folder
        self.layouts.build.conf_info.define_path("user.myconf:my_path", "my_conf_folder")
        # Both for user defined confs, or in the case of tool-requires, also built-in␣
↪confs
        self.layouts.build.conf_info.define_path("tools.android:ndk_path", "local/path/
↪to/ndk")
```

The `layouts` object contains `source`, `build` and `package` scopes, and each one contains one instance of `buildenv_info`, `runenv_info` and `conf_info`.

package()

The `package()` method is in charge of copying files from the `source_folder` and the temporary `build_folder` to the `package_folder`, copying only those files and artifacts that will be part of the final package, like headers, compiler static and shared libraries, executables, license files, etc.

The `package()` method will be called once per different configuration that is creating a new package binary, which happens with `conan install --build=pkg*`, `conan create` and `conan export-pkg` commands.

There are 2 main ways the `package()` method can do such a copy. The first one is an explicit `copy()` from the origin `source_folder` and `build_folder` to the `package folder`:

```
from conan import ConanFile
from conan.tools.files import copy

class Pkg(ConanFile):

    def package(self):
        # copying headers from source_folder
        copy(self, "*.h", join(self.source_folder, "include"), join(self.package_folder,
↪"include"))
        # copying compiled .lib from build folder
        copy(self, "*.lib", self.build_folder, join(self.package_folder, "lib"), keep_
↪path=False)
```

The second way is to use the `install` functionality of some build systems, provided that the build scripts implement such functionality. For example if the `CMakeLists.txt` of a package implements the correct CMake `INSTALL` instructions, it is possible to do:

```
def package(self):
    cmake = CMake(self)
    cmake.install()
```

Also, it is possible to combine both approaches, doing `cmake.install()` and also adding some `copy()` calls, for example to make sure some "License.txt" file is packaged that was not taken into account by the CMakeLists.txt script.

It is also possible to use conditionals in the `package()` method, because different platforms might have different artifacts in different locations:

```
def package(self):
    if self.settings.os == "Windows":
        copy(self, "*.lib", src=os.path.join(self.build_folder, "libs"), ...)
        copy(self, "*.dll", ....)
    else:
        copy(self, "*.lib", src=os.path.join(self.build_folder, "build", "libs"), ...)
```

Though in most situations it might not be necessary, because pattern based copy will likely not find wrong artifacts like `*.dll` in a non-Windows build.

The `package()` method is also the one called when packaging precompiled binaries with `conan export-pkg`. In this case the `self.source_folder` and `self.build_folder` refer to user space folders, as defined by the `layout()` method and the only folder in the Conan cache will be `self.package_folder`.

Note: Best practices

The `cmake.install()` functionality should be called in the `package()` method, not in the `build()` method. It is not

necessary to reuse the `CMake(self)` object, it shouldn't be reused among methods. Creating a new instance in every method is the recommended approach.

See also:

See *the package() method tutorial* for more information.

package_id()

Conan computes a unique `package_id` reference for each configuration, including `settings`, `options` and `dependencies` versions. This `package_id()` method allows some customizations and changes over the computed `package_id`, in general with the goal to relax some of the global binary compatibility assumptions.

The general rule is that every different value of `settings` and `options` creates a different `package_id`. This rule can be relaxed or expanded following different approaches:

- A given package recipe can decide in its `package_id()` that the final binary is independent of some settings, for example if it is a header-only library, that uses input settings to build some tests, it might completely clear all configuration, so the resulting `package_id` is always the same irrespective of the inputs. Likewise a C library might want to remove the effect of `compiler.cppstd` and/or `compiler.libcxx` from its binary `package_id`, because as a C library, its binary will be independent.
- A given package recipe can implement some partial erasure of information, for example to obtain the same `package_id` for a range of compiler versions. This type of binary compatibility is in general better addressed with the global `compatibility` plugin, or with the `compatibility()` method if the global plugin is not enough.
- A package recipe can decide to inject extra variability in its computed `package_id`, adding `conf` items or "target" settings.

Available automatic implementations

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

When the `package_id()` method is not defined, the following automatic implementation can be specified in the *implements* ConanFile attribute:

auto_header_only

Conan will automatically manage the package ID clearing settings and options when the recipe declares an option `header_only=True` or when `package_type` is `"header-library"`. It can be added to the recipe like this:

```
from conan import ConanFile

class Pkg(ConanFile):
    implements = ["auto_header_only"]
    ...
```

Then, if no `package_id()` method is specified in the recipe, Conan will automatically manage it and call `self.info.clear()` in the `package_id()` automatically, to make the `package_id` independent of settings, options, configuration and requirements.

If you need to implement custom behaviors in your recipes but also need this logic, it must be explicitly declared, for example, something like this:

```
def package_id(self):
    if self.package_type == "header-library":
        self.info.clear()
    else:
        self.info.settings.rm_safe("compiler.libcxx")
        self.info.settings.rm_safe("compiler.cppstd")
```

Information erasure

This is a `package_id` relaxing strategy. Let's check the first case: a header-only library, that has input `settings`, because it still wants to use them for some unit-tests in its `build()` method. In order to have exactly one final binary for all configurations, because the final artifact should be identical in all cases (just the header files), it would be necessary to do:

```
settings = "os", "compiler", "arch", "build_type"

def build(self):
    cmake = CMake(self) # need specific settings to build
    ...
    cmake.test()  # running unit tests for the current configuration

def package_id(self):
    # Completely clear all the settings from the ``package_id`` information ("info"␣
↪object)
    # All resulting ``package_id`` will be the same, irrespective of configuration
    self.info.settings.clear()
```

Warning: The modifications of the information always happen over the `self.info` object, not on `self.settings` or `self.options`

If a package is just a C library, but it couldn't remove the `compiler.cppstd` and `compiler.libcxx` in the `configure()` method (the recommended approach for most cases, to guarantee those flags are not used in the build), because there are C++ unit tests to the C library, then as the tests are not packaged and the final binary will be independent of C++, those could be removed with:

```
settings = "os", "compiler", "arch", "build_type"

def build(self):
    # building C++ tests for a C library

def package_id(self):
    del self.info.settings.compiler.cppstd
    # Some compilers might not declare libcxx subsetting
    self.info.settings.rm_safe("compiler.libcxx")
```

If a package is building an executable to be used as a tool, and only 1 executable for each OS and architecture is desired to be more efficient, the `package_id()` could remove the other settings and options if existing:

```
# this will be a "tool_require"
package_type = "application"
settings = "os", "compiler", "arch", "build_type"

def package_id(self):
    del self.info.settings.compiler
    del self.info.settings.build_type
```

Note that this doesn't mean that the `compiler` and `build_type` should be removed for every application executable. For other things that are not tools, but final products to release, the most common situation is that maintaining the different builds for the different compilers, compiler versions, build types, etc. is the best approach. It also means that we are erasing some information. We will not have the information of the compiler and build type that was used for the binary that we are using (it will not be in the `conan list` output, and it will not be in the server metadata either). If we compile a new binary with a different compiler or build type, it will create a new package revision under the same `package_id`.

Partial information erasure

It is also possible to partially erase information for given subsets of values. For example, if we want to have the same `package_id` for all the binaries compiled with `gcc` between versions 4.5 and 5.0, we can do:

```
def package_id(self):
    v = Version(str(self.info.settings.compiler.version))
    if self.info.settings.compiler == "gcc" and (v >= "4.5" and v < "5.0"):
        # The assigned string can be arbitrary
        self.info.settings.compiler.version = "GCC 4 between 4.5 and 5.0"
```

This will result in all other compilers rather than `gcc` and other versions outside of that range to have a different `package_id`, but there will be only 1 `package_id` binary for all `gcc 4.5-5.0` versions. This also has the disadvantage mentioned above about losing the information that created this binary.

This approach is not recommended in the general case, and it would be better approached with the global `compatibility` plugin or the recipe `compatibility()` method.

Note: Not only `settings` can be erased, but every other type of information such as `options` and `conf` items.

Adding information

There is some information not added by default to the `package_id`. If we are creating a package for a tool, to be used as a `tool_require`, and it happens that such package binary will be different for each "target" configuration, like it is the case for some cross-compilers, if the compiler itself might be different for the different architectures that it is targeting, it will be necessary to add the `settings_target` to the `package_id` with:

```
def package_id(self):
    self.info.settings_target = self.settings_target
```

The `conf` items do not affect the `package_id` by default. It is possible to explicitly make them part of it at the recipe level with:

```
def package_id(self):
    self.info.conf.define("user.myconf:myitem", self.conf.get("user.myconf:myitem"))
```

Although this can be achieved for all recipes without the `package_id()` method, using the `tools.info.package_id:confs = ["user.myconf:myitem"]` configuration.

Using regex patterns: You can use regex patterns in the *tools.info.package_id:confs*. This means that instead of specifying each individual configuration item, you can use a regex pattern to match multiple configurations. This is particularly useful when dealing with a large number of configurations or when configurations follow a predictable naming pattern. For instance:

- `tools.info.package_id:confs=[".*"]` matches all configurations.
- `tools.info.package_id:confs=["tools\..*"]` matches configurations starting with "tools.".
- `tools.info.package_id:confs=["(tools\.deploy|core)"]` matches configurations starting with "tools.deploy" or "core".

See also:

- See *the tutorial about header-only packages* for explanations about the `package_id()` method.
- Read the *binary model reference* for a full view of the Conan binary model.

package_info()

The `package_info()` method is the one responsible of defining the information to the consumers of the package, so those consumers can easily and automatically consume this package. The `generate()` method of the consumers is the place where the information defined in the `package_info()` will be mapped to the specific build system of the consumer. Then, if we want a package to be consumed by different build systems (like it happens with ConanCenter recipes for the community), it is very important that this information is complete.

Important: This method defines information exclusively for **consumers** of this package, not for itself. This method executes after the binary has been built and packaged. The information that is consumed in the build should be processed in `generate()` method.

---

cpp_info: Library and build information

Each package has to specify certain build information for its consumers. This can be done in the `cpp_info` attribute.

```
# Binaries to link
self.cpp_info.libs = []  # The libs to link against
self.cpp_info.system_libs = []  # System libs to link against
self.cpp_info.frameworks = []  # OSX frameworks that consumers will link against
self.cpp_info.objects = []  # precompiled objects like .obj .o that consumers will link
# Directories
self.cpp_info.includedirs = ['include']  # Ordered list of include paths
self.cpp_info.libdirs = ['lib']  # Directories where libraries can be found
self.cpp_info.bindirs = ['bin']  # Directories where executables and shared libs can be␣
↪found
self.cpp_info.resdirs = []  # Directories where resources, data, etc. can be found
self.cpp_info.srcdirs = []  # Directories where sources can be found (debugging, reusing␣
↪sources)
self.cpp_info.builddirs = []  # Directories where build scripts for consumers can be␣
↪found
self.cpp_info.frameworkdirs = []  # Directories where OSX frameworks can be found
# Flags
```

(continues on next page)

(continued from previous page)

```
self.cpp_info.defines = []  # preprocessor definitions
self.cpp_info.cflags = []  # pure C flags
self.cpp_info.cxxflags = []  # C++ compilation flags
self.cpp_info.sharedlinkflags = []  # linker flags
self.cpp_info.exelinkflags = []  # linker flags
# Properties
self.cpp_info.set_property("property_name", "property_value")
# Structure
self.cpp_info.components # Dictionary-like structure to define the different components␣
↪a package may have
self.cpp_info.requires # List of components from requirements that need to be propagated␣
↪downstream
```

Binaries to link:

- **libs**: Ordered list of compiled libraries (contained in the package) the consumers should link. Empty by default.
- **system_libs**: Ordered list of system libs (not contained in the package) the consumers should link. Empty by default.
- **frameworks**: Ordered list of OSX frameworks (contained or not in the package), the consumers should link. Empty by default.
- **objects**: Ordered list of precompiled objects (.obj, .o) contained in the package the consumers should link. Empty by default

Directories:

- **includedirs**: List of relative paths (starting from the package root) of directories where headers can be found. By default it is initialized to `['include']`, and it is rarely changed.
- **libdirs**: List of relative paths (starting from the package root) of directories in which to find library object binaries (*.lib, *.a, *.so, *.dylib). By default it is initialized to `['lib']`, and it is rarely changed.
- **bindirs**: List of relative paths (starting from the package root) of directories in which to find library runtime binaries (like executable Windows .dlls). By default it is initialized to `['bin']`, and it is rarely changed.
- **resdirs**: List of relative paths (starting from the package root) of directories in which to find resource files (images, xml, etc). By default it is empty.
- **srcdirs**: List of relative paths (starting from the package root) of directories in which to find sources (like .c, .cpp). By default it is empty. It might be used to store sources (for later debugging of packages, or to reuse those sources building them in other packages too).
- **builddirs**: List of relative paths (starting from package root) of directories that can contain build scripts that could be used by the consumers. Empty by default.
- **frameworkdirs**: List of relative paths (starting from the package root), of directories containing OSX frameworks.

There is one exception that would allow using absolute paths in `self.cpp_info.xxxxdirs` fields: for recipes that serve as a "wrapper" of a library installed in the system, but not by Conan. See the *example of a recipe wrapping a library installed in the system*.

Flags:

- **defines**: Ordered list of preprocessor directives. It is common that the consumers have to specify some sort of defines in some cases, so that including the library headers matches the binaries.

- **cflags**, **cxxflags**, **sharedlinkflags**, **exelinkflags**: List of flags that the consumer should activate for proper behavior. Rarely used. These flags can be handled by the *compiler flags plugin* when mixing different compilers that create compatible binaries.

Properties:

- **set_property()** allows to define some built-in and user general properties to be propagated with the `cpp_info` model for consumers. They might contain build-system specific information. Some built-in properties are `cmake_file_name`, `cmake_target_name`, `pkg_config_name`, `qbs_file_name`, that can define specific behavior for `CMakeDeps`, `PkgConfigDeps` or `QbsDeps` generators. For more information about these, read the specific build system integration documentation.

Structure:

- **components**: Dictionary with names as keys and a component object as value to model the different components a package may have: libraries, executables...
- **requires**: **Experimental** List of components from the requirements this package (and its consumers) should link with. It will be used by generators that add support for components features.

It is common that different configurations will produce different `package_info`, for example, the library names might change in different OSs, or different `system_libs` will be used depending on the compiler and OS:

```
settings = "os", "compiler", "arch", "build_type"
options = {"shared": [True, False]}

def package_info(self):
    if not self.settings.os == "Windows":
        self.cpp_info.libs = ["zmq-static"] if not self.options.shared else ["zmq"]
    else:
        ...

    if not self.options.shared:
        self.cpp_info.defines = ["ZMQ_STATIC"]
    if self.settings.os == "Windows" and self.settings.compiler == "msvc":
        self.cpp_info.system_libs.append("ws2_32")
```

Properties

Any CppInfo object can declare "properties" that can be read by the generators. The value of a property can be of any type. Check each generator reference to see the properties used on it.

```
def set_property(self, property_name, value)
def get_property(self, property_name, check_type=None):
```

Example:

```
def package_info(self):
    self.cpp_info.set_property("cmake_find_mode", "both")
    self.cpp_info.get_property("cmake_find_mode", check_type=str)
```

Components

If your package is composed by more than one library, it is possible to declare components that allow to define a `CppInfo` object per each of those libraries and also requirements between them and to components of other packages (the following case is not a real example):

```
def package_info(self):
    self.cpp_info.set_property("cmake_file_name", "OpenSSL")

    self.cpp_info.components["crypto"].set_property("cmake_target_name", "OpenSSL::Crypto
↪")
    self.cpp_info.components["crypto"].libs = ["libcrypto"]
    self.cpp_info.components["crypto"].defines = ["DEFINE_CRYPTO=1"]
    self.cpp_info.components["crypto"].requires = ["zlib::zlib"]  # Depends on all␣
↪components in zlib package

    self.cpp_info.components["ssl"].set_property("cmake_target_name", "OpenSSL::SSL")
    self.cpp_info.components["ssl"].includedirs = ["include/headers_ssl"]
    self.cpp_info.components["ssl"].libs = ["libssl"]
    self.cpp_info.components["ssl"].requires = ["crypto",
                                                "boost::headers"]  # Depends on headers␣
↪component in boost package

    obj_ext = "obj" if platform.system() == "Windows" else "o"
    self.cpp_info.components["ssl-objs"].objects = [os.path.join("lib", "ssl-object.{}".
↪format(obj_ext))]
```

Dependencies among components and to components of other requirements can be defined using the `requires` attribute and the name of the component. The dependency graph for components will be calculated and values will be aggregated in the correct order for each field.

Conan validates that every direct dependency of a recipe is used in some `(cpp_info/components).requires`, raising an error otherwise.

If a dependency really has no relationship at all with the current package artifacts (not linked, not embedded, not exposing any of its headers), it can be added to `self.cpp_info.ignored_requires` to silence the validation:

```
def requirements(self):
    self.requires("other/1.0")
    self.requires("another/1.0")  # not linked or embedded in this package in any way

def package_info(self):
    self.cpp_info.components["cmp1"].requires = ["other::other"]
    self.cpp_info.ignored_requires = ["another"]
```

The definition of `ignored_requires` should not be necessary in most cases, as Conan already avoids propagating information that is not needed (for example, it doesn't propagate headers downstream unless `transitive_headers=True`).

If there is some relationship between the dependency and the package, even if it looks like a build-time only one (for example, a static library linked into the current shared library), it is recommended to still declare it via `requires` instead. Otherwise, changes in that dependency, like building it with a different `shared` value, could break the package without Conan being able to detect it.

buildenv_info, runenv_info

The `buildenv_info` and `runenv_info` attributes are `Environment` objects that allow to define information for the consumers in the form of environment variables. They can use any of the `Environment` methods to define such information:

```
settings = "os", "compiler", "arch", "build_type"

def package_info(self):
    self.buildenv_info.define("MYVAR", "1")
    self.buildenv_info.prepend_path("MYPATH", "my/path")
    if self.settings.os == "Android":
        arch = "myarmarch" if self.settings.arch=="armv8" else "otherarch"
        self.buildenv_info.append("MY_ANDROID_ARCH", f"android-{arch}")

    self.runenv_info.append_path("MYRUNPATH", "my/run/path")
    if self.settings.os == "Windows":
        self.runenv_info.define_path("MYPKGHOME", "my/home")
```

Note that these objects are not tied to either regular `requires` or `tool_requires`, any package recipe can use both. The difference between `buildenv_info` and `runenv_info` is that the former is applied when Conan is building something from source, like in the `build()` method, while the later would be used when executing something in the "host" context that would need the runtime activated.

Conan `VirtualBuildEnv` generator will be used by default in consumers, collecting the information from `buildenv_info` (and some `runenv_info` from the "build" context) to create the `conanbuild` environment script, which runs by default in all `self.run(cmd, env="conanbuild")` calls. The `VirtualRunEnv` generator will also be used by default in consumers collecting the `runenv_info` from the "host" context creating the `conanrun` environment script, which can be explicitly used with `self.run(<cmd>, env="conanrun")`.

Note: Best practices

It is not necessary to add `bindirs` to the `PATH` environment variable, this will be automatically done by the consumer `VirtualBuildEnv` and `VirtualRunEnv` generators. Likewise, it is not necessary to add `includedirs`, `libdirs` or any other dirs to environment variables, as this information will be typically managed by other generators.

---

conf_info

`tool_requires` packages in the "build" context can transmit some `conf` configuration to its immediate consumers, with the `conf_info` attribute. For example, one Conan package packaging the AndroidNDK could do:

```
def package_info(self):
    self.conf_info.define_path("tools.android:ndk_path", "path/to/ndk/in/package")
```

`conf_info` from packages can still be overwritten from profiles values, because user profiles will have higher priority.

`Conf.define`(*name*, *value*)

Define a value for the given configuration name.

Parameters

- **name** – Name of the configuration.

- **value** – Value of the configuration.

```
def package_info(self):
    # Setting values
    self.conf_info.define("tools.build:verbosity", "verbose")
    self.conf_info.define("tools.system.package_manager:sudo", True)
    self.conf_info.define("tools.microsoft.msbuild:max_cpu_count", 2)
    self.conf_info.define("user.myconf.build:ldflags", ["--flag1", "--flag2"])
    self.conf_info.define("tools.microsoft.msbuildtoolchain:compile_options", {
↪"ExceptionHandling": "Async"})
```

Conf.**append**(*name*, *value*)

Append a value to the given configuration name.

Parameters

- **name** – Name of the configuration.
- **value** – Value to append.

```
def package_info(self):
    # Modifying configuration list-like values
    self.conf_info.append("user.myconf.build:ldflags", "--flag3")  # == ["--flag1",
↪"--flag2", "--flag3"]
```

Conf.**prepend**(*name*, *value*)

Prepend a value to the given configuration name.

Parameters

- **name** – Name of the configuration.
- **value** – Value to prepend.

```
def package_info(self):
    self.conf_info.prepend("user.myconf.build:ldflags", "--flag0")  # == ["--flag0",
↪ "--flag1", "--flag2", "--flag3"]
```

Conf.**update**(*name*, *value*)

Update the value to the given configuration name.

Parameters

- **name** – Name of the configuration.
- **value** – Value of the configuration.

```
def package_info(self):
    # Modifying configuration dict-like values
    self.conf_info.update("tools.microsoft.msbuildtoolchain:compile_options", {
↪"ExpandAttributedSource": "false"})
```

Conf.**remove**(*name*, *value*)

Remove a value from the given configuration name.

Parameters

- **name** – Name of the configuration.
- **value** – Value to remove.

```
def package_info(self):
    # Remove
    self.conf_info.remove("user.myconf.build:ldflags", "--flag1")  # == ["--flag0",
↪"--flag2", "--flag3"]
```

Conf.**unset**(*name*)

Clears the variable, equivalent to a unset or set XXX=

Parameters
name – Name of the configuration.

```
def package_info(self):
    # Unset any value
    self.conf_info.unset("tools.microsoft.msbuildtoolchain:compile_options")
```

It is possible to define configuration in packages that are `tool_requires`. For example, assuming there is a package that bundles the *AndroidNDK*, it could define the location of such NDK to the `tools.android:ndk_path` configuration as:

```
import os
from conan import ConanFile

class Pkg(ConanFile):
    name = "android_ndk"

    def package_info(self):
        self.conf_info.define("tools.android:ndk_path", os.path.join(self.package_folder,
↪ "ndk"))
```

Note that this only propagates from the immediate, direct `tool_requires` of a recipe.

generator_info

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

`tool_requires` in the build context can inject generators into the recipe, by adding them to the `generator_info` attribute inside the `package_info` method. This is useful to inject custom generators into the recipe, that will be used by the consumers of the package, just as if they were declared in their `generators` attribute.

```
class MyGenerator:
    def __init__(self, conanfile):
        self._conanfile = conanfile

    def generate(self):
        self.output.info(f"Calling custom generator for {conanfile}")

def package_info(self):
    self.generator_info = [MyGenerator]
```

Note that this only propagates from the immediate, direct `tool_requires` of a recipe, and that by default `self.generator_info` is `None`.

Note: Best practices

- The `package_info()` method is not strictly necessary if you have other means of propagating information for consumers. For example, if your package creates `xxx-config.cmake` files at build time, and they are put in the final package, it might not be necessary to define `package_info()` at all, and in the consumer side the `CMakeDeps` would not be necessary either, as `CMakeToolchain` is able to inject the paths to locate the `xxx-config.cmake` files inside the packages. This approach can be good for private usage of Conan, albeit some limitations of CMake, like not being able to manage multi-configuration projects (like Visual Studio switching Debug/Release in the IDE, that `CMakeDeps` can provide), limitations in some cross-build scenarios using packages that are both libraries and build tools (like `protobuf`, that also `CMakeDeps` can handle).
- Providing a `package_info()` is very necessary if consumers can use different build systems, like in ConanCenter. In this case, it is necessary a bit of repetition, and coding the `package_info()` might feel duplicating the package `xxx-config.cmake`, but automatically extracting the info from CMake is not feasible at this moment.
- If you plan to use editables or the local development flow, there's a need to check the `layout()` and define the information for `self.cpp.build` and `self.cpp.source`.
- It is not necessary to add `bindirs` to the `PATH` environment variable, this will be automatically done by the consumer `VirtualBuildEnv` and `VirtualRunEnv` generators.
- The **paths** defined in `package_info()` shouldn't be converted to any specific format (like the one required by Windows subsystems). Instead, it is the responsibility of the consumer to translate these paths to the adequate format.

See also:

See *the defining package information tutorial* for more information.

requirements()

The `requirements()` method is used to specify the dependencies of a package.

```
def requirements(self):
    self.requires("zlib/1.3.1")
```

For simple cases the attribute syntax can be used, like `requires = "zlib/1.3.1"`.

Requirement traits

Traits are properties of a requires clause. They determine how various parts of a dependency are treated and propagated by Conan. Values for traits are usually computed by Conan based on the dependency's *package_type*, but can also be specified manually.

A good introduction to traits is provided in the Advanced Dependencies Model in Conan 2.0 presentation.

In the example below `headers` and `libs` are traits.

```
self.requires("math/1.0", headers=True, libs=True)
```

headers

Indicates that there are headers that are going to be `#included` from this package at compile time. The dependency will be in the host context.

libs

The dependency contains some library or artifact that will be used at link time of the consumer. This trait will typically be `True` for direct shared and static libraries, but could be false for indirect static libraries that are consumed via a shared library. The dependency will be in the host context.

build

This dependency is a build tool, an application or executable, like cmake, that is used exclusively at build time. It is not linked/embedded into binaries, and will be in the build context.

> **Warning:** Build time requirements (`tool_requires`, `build_requires`) that define `build=True` are designed to work with their default `visible=False`, and at the moment it is very strongly recommended to keep them as `visible=False`. If you think you might have a use case, it would be better to discuss first in https://github.com/conan-io/conan/issues and ask about it than trying to enable `visible=True`.
>
> For some very exceptional cases, there is **experimental** support for build/tool requires with `build=True` that also define `visible=True`, but experimental and subject to possible breaking changes in future Conan versions. It is also known and designed to not propagate all traits, for example `headers/libs` will not be propagated, because headers and libs from the "build" context cannot be linked in the host context.

run

This dependency contains some executables, either apps or shared libraries that need to be available to execute (typically in the path, or other system env-vars). This trait can be `True` for `build=False`, in that case, the package will contain some executables that can run in the host system when installing it, typically like an end-user application. This trait can be `True` for `build=True`, the package will contain executables that will run in the build context, typically while being used to build other packages.

visible

This `require` will be propagated downstream, even if it doesn't propagate `headers`, `libs` or `run` traits. Requirements that propagate downstream can cause version conflicts. This is typically `True`, because in most cases, having 2 different versions of the same library in the same dependency graph is at least complicated, if not directly violating ODR or causing linking errors. It can be set to `False` in advanced scenarios, when we want to use different versions of the same package during the build.

> **Warning:** The `visible` trait can create conflicts if a transitive dependency has a `visible=True` requirement to the same package that the current recipe is declaring as `visible=False`. In these cases where different visibility rules reach the same package, the visible transitive dependency will be used and propagated downstream.

Important: Best practices

In general it is recommended to use the `visible` trait defaults, that is, `visible=True` for normal host `requires()` and `visible=False` for `test_requires()` and `tool_requires()`:

- Changing that default visibility can create different issues and problems if not handle properly.
- It is not recommended nor necessary to define `visible=False` for shared libraries packages that requires other static library packages, or similar C, C++ libraries usages, the default Conan behavior will already skip downloading that transitive static library if not needed and will not propagate downstream the linkage requirements. In general, you can use the default `visible=True` for most regular `requires()` dependencies.
- Using `visible=False` in regular `requires` must guarantee complete API and ABI encapsulation. If those are not met, issues from compilation errors, to linkage errors, to runtime errors can happen.
- Using `visible=True` for `tool_requires` or `test_requires` is also discouraged. The recommended way to inject `tool_requires` from outside a recipe is using profiles.

---

consistent

This is a new trait introduced in Conan 2.28, and **experimental**. It takes effect only when `visible=False`, it is not possible to have a requirement with `visible=True` and `consistent=False`. It was introduced to dissambiguate the intention of the `visible=False` trait.

Its main purpose is to model and provide an opt-in and opt-out for the graph computation behavior regarding how transitive dependencies of the same package name can be considered the same node in the dependency graph or be different nodes in the dependency graph.

Previous to Conan 2.28, when defining `visible=False` for requirements in the "host" context, the behavior was not well defined, depending on different factors like if using version ranges or the order of requirements. From Conan 2.28 the behavior is the following:

- If the policy `required_conan_version = ">=2.28"` is defined in the recipe or the equivalent global `core:policies` is defined, then the default `consistent` trait will be:
  - `consistent=True` for the "host" context. In general, dependencies in the host context are libraries, and the libraries must be consistent among them, even if they are not propagated downstream. That means that it is not expected to have multiple instances (nodes) of the same package in the dependencies.
  - `consistent=False` for the "build" context. In general dependencies in the build context are tools, so it doesn't matter much if they have different transitive dependencies.
  - Users can always explicitly define the `self.[tool_]requires("pkg/version", visible=False, consistent=True|False)` in their recipes to change this default.
- Test requires with the `test=True` trait also default the `consistent=True` trait, as test-requires must be consistent, even if the policy is not defined.
- It is possible for recipes to opt-in the new trait explicitly without activating the policy with `self.requires("pkg/version", visible=False, consistent=True)` for the host context and `self.tool_requires("tool/version", consistent=True)` for the build context.
- Even if `consistent=False`, if Conan detects that some transitive libraries are propagating conflicting traits such as `headers`, `libs` or `run`, then, those transitive dependencies will still be considered overlapping and conflict or be considered the same dependency, closing a diamond structure in the graph, instead of being separate nodes in the graph for the same package, keeping a tree structure.

See pull request 19286 for discusion and more information.

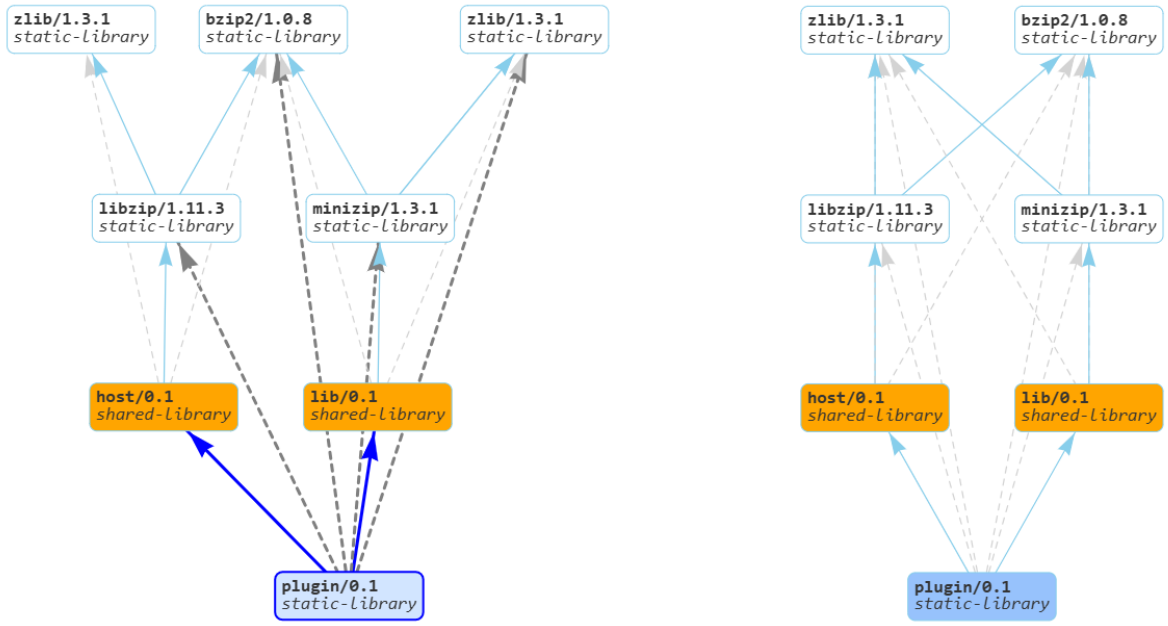


Fig. 1: `consistent=False` (left) `consistent=True` (right)

transitive_headers

If `True` the headers of the dependency will be visible downstream. Read more about this trait in the *tutorial for headers transitivity*.

transitive_libs

If `True` the libraries to link with of the dependency will be visible downstream.

test

This requirement is a test library or framework, like Catch2 or gtest. It is mostly a library that needs to be included and linked, but that will not be propagated downstream.

package_id_mode

If the recipe wants to specify how the dependency version affects the current package `package_id`, can be directly specified here.

While it could be also done in the `package_id()` method, it seems simpler to be able to specify it in the `requires` while avoiding some ambiguities.

```
# We set the package_id_mode so it is part of the package_id
self.tool_requires("tool/1.1.1", package_id_mode="minor_mode")
```

Which would be equivalent to:

```
def package_id(self):
  self.info.requires["tool"].minor_mode()
```

force

This `requires` will force its version in the dependency graph upstream, overriding other existing versions even of transitive dependencies, and also solving potential existing conflicts. The downstream consumer's `force` traits always have higher priority.

override

The same as the `force` trait, but not adding a `direct` dependency. **If there is no transitive dependency to override, this ``require`` will be discarded**. This trait only exists at the time of defining a `requires`, but it will not exist as an actual `requires` once the graph is fully evaluated

---

Note: Best practices

- The `force` and `override` traits to solve conflicts are not recommended as a general versioning solution, just as a temporary workaround to solve a version conflict. Its usage should be avoided whenever possible, and updating versions or version ranges in the graph to avoid the conflicts without overrides and forces is the recommended approach.
- A key takeaway is that the `override` trait does not create a direct dependency from your package, while the `force` trait does. This means that the `override` trait is only useful when you want to override the version of one of your transitive dependencies, while not adding a direct dependency to it.

---

direct

If the dependency is a direct one, that is, it has explicitly been declared by the current recipe, or if it is a transitive one.

options

It is possible to define options values for dependencies as a trait:

```
self.requires("mydep/0.1", options={"dep_option": "value"})
```

Warning: Defining options values in recipes does not have strong guarantees, please check *this FAQ about options values for dependencies*. The recommended way to define options values is in profile files.

no_skip

This trait is an **experimental** feature introduced in Conan 2.16, and subject to breaking changes. See *the Conan stability* section for more information.

Conan is able to avoid the download of the package binaries of the transitive dependencies when they are not needed. For example if a `package_type = "application"` package that contains an executable depends (`requires`) another package that is a `package_type = "static-library"` (or a regular library, but with option `shared=False`), then, installing the application package binary doesn't require the binaries of the static libraries dependencies to work. Conan will then "skip" the download of those binaries, saving the time and transfer cost of such download and installation. These binaries are marked as "Skipped binaries" in the Conan commands output.

The `tools.graph:skip_binaries` conf can change the default behavior and if `False` it will avoid skipping binaries, which can be useful in some scenarios.

The `no_skip=True` trait can be defined in a dependency like:

```
name = "mypkg"

def requirements(self):
  self.requires("mydep/0.1", no_skip=True)
```

And that will force the download of the binary for `mydep/0.1` when the binary for `mypkg` is necessary.

Note: Best practices

The usage of `no_skip=True` should be exceptional, for very limited and extraordinary use cases, the default Conan "skipping binaries" behavior should be good for the vast majority of cases. Typically, it wouldn't make sense in isolation, but if used jointly with other traits such as `visible=False`. Avoid using it except when absolutely necessary, and it should only be used in very particular recipes. If used in many recipes, it is most likely an abuse.

See also:

If a dependency should always be treated as needed at runtime by every consumer, for example a `static-library` that also ships a shared library, consider declaring `"run": True` inside *package_type_traits* in that dependency's own recipe instead. Unlike `no_skip`, which is opted into per `requires()` and only forces the binary download, `package_type_traits` is declared once by the dependency and also sets `run=True`, so its folders are exposed in the runtime environment and it is correctly picked up by deployers like *runtime_deploy*.

package_type trait inferring

Some traits are automatically inferred based on the value of the `package_type` of the dependency if not explicitly set by the recipe.

Note: The `libs`, `headers` and `visible` traits are set to `True` by default unless otherwise stated in the lists shown below, or if they are manually set by the user to `False`.

The inferring rules are:

- `application`: `headers=False`, `libs=False`, `run=True`
- `shared-library`: `run=True`
- `static-library`: `run=False`

- header-library: headers=True, libs=False, run=False
- build-scripts: headers=False, libs=False, run=True, visible=False

---

Note: A dependency that declares "run": True inside *package_type_traits* in its own recipe always gets run=True, overriding the package_type-based default above. This is meant for packages that ship extra artifacts needed at runtime regardless of their package_type, like a static-library that also contains a shared library.

This means that if in your recipe you have self.requires("mypkg/1.0"), and mypkg/1.0 has package_type="application", then the effective traits for that requires will be headers=False, libs=False, run=True. These can then be overridden by explicitly setting them in the requires.

Additionally, some additional traits are inferred on top of the above mentioned choices, based on the package_type of your recipe:

- header-library: transitive_headers=True, transitive_libs=True

This means that if your package is a header-library, then all its requirements will have transitive_headers=True and transitive_libs=True by default.

Default traits for each kind of requires

Each kind of requires sets some additional traits by default on top of the ones stated in the last section. Those are:

- requires: build=False
- build_requires: headers=False, libs=False, build=True, visible=False
- tool_requires: headers=False, libs=False, build=True, run=True, visible=False
- test_requires: headers=True, libs=True, build=False, visible=False, test=True

For example, taking all the logic shown into account, this means that if in your library that has package_type="header-library" you have a requirement of the form self.requires("mypkg/1.0") and mypkg/1.0 has package_type="shared-library", the effective traits for that requires will be:

- Inferred from mypkg's package_type: run=True
- Inferred from your package's package_type: transitive_headers=True, transitive_libs=True
- By the requires kind: build=False
- By default: headers=True, libs=True

set_name()

Dynamically define name attribute. This method would be rarely needed, as the only use case that makes sense is when a recipe is shared and used to create different packages with the same recipe. In most cases the recommended approach is to define the name = "mypkg" attribute in the recipe.

This method is executed only when the recipe is exported to the cache with conan create or conan export, or when the recipe is being locally used, like with conan install .. In all other cases, the name of the package is fully defined, and set_name() will not be called, so do not rely on it for any other functionality different than defining the self.name value.

If the current package name was defined in a *name.txt* file, it would be possible to do:

```
from conan import ConanFile
from conan.tools.files import load

class Pkg(ConanFile):
    def set_name(self):
        # This will execute relatively to the current user directory (name.txt in cwd)
        self.name = load(self, "name.txt")
        # if "name.txt" is located relative to the conanfile.py better do:
        self.name = load(self, os.path.join(self.recipe_folder, "name.txt"))
```

The package name can also be defined in command line for some commands with `--name=xxxx` argument. If we want to prioritize the command line argument we should do:

```
from conan import ConanFile
from conan.tools.files import load

class Pkg(ConanFile):
    def set_name(self):
        # Command line ``--name=xxxx`` will be assigned first to self.name and have␣
↪priority
        self.name = self.name or load(self, "name.txt")
```

The `set_name()` method can decide to define the `name` value, irrespective of the potential `--name=xxx` command line argument, that can be even completely ignored by `set_name()`. It is the responsibility of the developer to provide a correct `set_name()`:

```
def set_name(self):
    # This will always assign "pkg" as name, ignoring ``--name`` command line argument
    # and without erroring or warning
    self.name = "pkg"
```

If a command line argument `--name=xxx` is provided, it will be initialized in the `self.name` attribute, so `set_name()` method can read and use it:

```
def set_name(self):
    # Takes the provided command line ``--name`` argument and creates a name appending to
    # it the ".extra" string
    self.name = self.name + ".extra"
```

Warning: The `set_name()` method is an alternative to the `name` attribute. It is not advised or supported to define both a `name` attribute and a `set_name()` method.

set_version()

Dynamically define `version` attribute. This method might be needed when the same recipe is being used to create different versions of the same package, and such version is defined elsewhere, like in the git branch or in a text or build script file. This would be a common situation.

This method is executed only when the recipe is exported to the cache `conan create` and `conan export`, and when the recipe is being locally used, like with `conan install ..`. In all other cases, the version of the package is fully defined, and `set_version()` will not be called, so do not rely on it for any other functionality different than defining the `self.version` value.

If the current package version was defined in a *version.txt* file, it would be possible to do:

```
from conan import ConanFile
from conan.tools.files import load

class Pkg(ConanFile):
    def set_version(self):
        # This will execute relatively to the current user directory (version.txt in cwd)
        self.version = load(self, "version.txt")
        # if "version.txt" is located relative to the conanfile.py better do:
        self.version = load(self, os.path.join(self.recipe_folder, "version.txt"))
```

The package version can also be defined in command line for some commands with `--version=xxxx` argument. If we want to prioritize the command line argument we should do:

```
from conan import ConanFile
from conan.tools.files import load

class Pkg(ConanFile):
    def set_version(self):
        # Command line ``--version=xxxx`` will be assigned first to self.version and have␣
↪priority
        self.version = self.version or load(self, "version.txt")
```

A common use case could be to define the `version` dynamically from some version control mechanism, like the current git tag. This could be done with:

```
from conan import ConanFile
from conan.tools.scm import Git

class Pkg(ConanFile):
    name = "pkg"

    def set_version(self):
        git = Git(self, self.recipe_folder)
        self.version = git.run("describe --tags")
```

The `set_version()` method can decide to define the `version` value, irrespective of the potential `--version=xxx` command line argument, that can be even completely ignored by `set_version()`. It is the responsibility of the developer to provide a correct `set_version()`:

```
def set_version(self):
    # This will always assign "2.1" as version, ignoring ``--version`` command line␣
↪argument
```

(continues on next page)

(continued from previous page)

```
    # and without erroring or warning
    self.version = "2.1"
```

If a command line argument `--version=xxx` is provided, it will be initialized in the `self.version` attribute, so `set_version()` method can read and use it:

```
def set_version(self):
    # Takes the provided command line ``--version`` argument and creates a version␣
↪appending to
    # it the ".extra" string
    self.version = self.version + ".extra"
```

Warning: The `set_version()` method is an alternative to the `version` attribute. It is not advised or supported to define both a `version` class attribute and a `set_version()` method.

source()

The `source()` method can be used to retrieve the necessary source code to build a package from source, and to apply patches to such source code if necessary. It will be called when a package is being built from source, like with `conan create` or `conan install --build=pkg*`, but it will not be called if a package pre-compiled binary is being used. That means that the source code will not be downloaded if a pre-compiled binary exists.

The `source()` method can implement different strategies for retrieving the source code:

- Fetching the source code for a third party library:
  - Using a `Git(self).clone()` to clone a Git repository
  - Executing a `download()` + `unzip()` or a combined `get()` (internally does download + unzip) to download a tarball, tgz, or zip archive.
- Fetching the source code for itself, from its repository, whose coordinates have been captured in the `conandata.yml` file in the `export()` method. This is the strategy that would be used to manage the source code for packages in which the `conanfile.py` lives in the package itself, but that for some reason we don't want to put the source code in the recipe (like not distributing our source code, but being able to distribute our package binaries).

The `source()` method executes in the `self.source_folder`, the current working directory will be equal to that folder (which value is derived from `layout()` method).

A `source()` implementation might use the convenient `get()` helper, or use its own mechanisms or other Conan helpers for the task, something like:

```
import os
import shutil

from conan import ConanFile
from conan.tools.files import download, unzip, check_sha1


class PocoConan(ConanFile):
    name = "poco"
    version = "1.6.0"
```

(continues on next page)

(continued from previous page)

```
    def source(self):
        zip_name = f"poco-{self.version}-release.zip"
        # Immutable source .zip
        download(self, f"https://github.com/pocoproject/poco/archive/poco-{self.version}-
↪release.zip", zip_name)
        # Recommended practice, always check hashes of downloaded files
        check_sha1(self, zip_name, "8d87812ce591ced8ce3a022beec1df1c8b2fac87")
        unzip(self, zip_name)
        shutil.move(f"poco-poco-{self.version}-release", "poco")
        os.unlink(zip_name)
```

Important: Please read the note in *conan.tools.files.unzip()* regarding Python 3.14 breaking changes and the new tar archive extract filters.

Applying patches to downloaded sources can be done (and should be done) in the `source()` method if those patches apply to all possible configurations. As explained below, it is not possible to introduce conditionals in the `source()` method. If the patches are in file form, those patches must be exported together with the recipe, so they can be used whenever a build from source is fired.

It is possible to apply patches with:

- Your own or `git` patches utilities
- The Conan built-in `patch()` utility to explicitly apply patches one by one
- Apply the `apply_conandata_patches()` Conan utility to automatically apply all patches defined in `conandata.yml` file following some conventions.

Source caching

Once the `source()` method has been called, its result will be cached and reused for any build from source, for any configuration. That means that the retrieval of sources from the `source()` method should be completely independent of the configuration. It is not possible to implement conditionals on the `settings`, and in general, any attempt to apply any conditional logic to the `source()` method is wrong.

```
def source(self):
    if self.settings.compiler == "gcc":  # ERROR, will raise
        # download some source
```

Trying to bypass the Conan exception by using some other mechanism like:

```
def source(self):
    # Might work, but NOT recommended, try to avoid as much as possible
    if platform.system() == "Windows":
        # download something
    else:
        # download something different
```

Might apparently work if not doing any cross-build, and not recollecting sources in a different OS, but could be problematic otherwise.

To be completely safe, if different source code is necessary for different configurations, the recommended approach would be to retrieve that code conditionally in the `build()` method.

Forced retrieval of sources

When working with a recipe in a user folder, it is easy to call the `source()` method and force the retrieval of the source code, that will be done in the same user folder, according to the `layout()` definition:

```
$ conan source .
```

Calling the `source()` method and forcing the retrieval of source code in the cache, for all or some dependencies, even if they are not being built from sources, is possible with the `tools.build:download_source=True` configuration. For example:

```
$ conan graph info . -c tools.build:download_source=True
```

Will compute the dependency graph, then call the `source()` method for all "host" packages in the graph (as the configuration by default is a "host" configuration, if you want also the sources for the "build" context `tool_requires`, you could use `-c:b tools.build:download_source=True`). It is possible to collect all the source folders from the json formatted output, or to automate recollection of all sources, a `deployer` could be used.

Likewise, it is possible to retrieve the sources for packages in other `create` and `install` commands, just by passing the configuration. Finally, as also configuration can be defined per-package, using `-c mypkg*:tools.build:download_source=True` would only retrieve the sources of packages matching the `mypkg*` pattern.

Note that `tools.build:download_source=True` will not have any effect on packages in **editable** mode. Downloading sources in that case could easily overwrite and destroy local developer changes over that code. The `conan source` command must be used on packages in editable mode to download the sources.

Note: Best practices

- The `source()` method should be the same for all configurations, it cannot be conditional to any configuration.
- The `source()` method should retrieve immutable sources. Using some branch name, HEAD, or a tarball whose URL is not immutable and is being overwritten is a bad practice and will lead to broken packages. Using a Git commit, a frozen Git release tag, or a fixed and versioned release tarballs is the expected input.
- Applying patches should be done by default in the `source()` method, except if the patches are exclusive for one configuration, in that case they could be applied in `build()` method.
- The `source()` method should not access nor manipulate files in other folders different to the `self.source_folder`. All the "exported" files are copied to the `self.source_folder` before calling it.

See also:

See *the tutorial about managing recipe sources* for more information.

system_requirements()

The `system_requirements()` method can be used to call the system package managers to install packages at the system level. In general, this should be reduced to a minimum, system packages are not modeled dependencies, but it can be sometimes convenient to automate the installation of some system packages that are necessary for some Conan packages. For example, when creating a recipe to package the `opencv` library, we could realize that it needs in Linux the `gtk` libraries, but it might be undesired to create a package for them, because we want to make sure we use the system ones. We code

```
from conan import ConanFile
from conan.tools.system.package_manager import Apt
```

(continues on next page)

(continued from previous page)

```
class OpenCV(ConanFile):
    name = "opencv"
    version = "4.0"

    def system_requirements(self):
        apt = Apt(self)
        apt.install(["libgtk-3-dev"], update=True, check=True)
```

For full reference of the built-in helpers for different system package managers read the *tools.system.package_manager documentation*.

Collecting system requirements

When `system_requirements()` uses some built-in `package_manager` helpers, it is possible to collect information about the installed or required system requirements. If we have the following `conanfile.py`:

```
from conan import ConanFile
from conan.tools.system.package_manager import Apt

class MyPkg(ConanFile):
    settings = "arch"

    def system_requirements(self):
        apt = Apt(self)
        apt.install(["pkg1", "pkg2"])
```

It is possible to display the installed system packages (with the default `tools.system.package_manager:mode` requirements will be checked, but not installed) with:

```
 # Assuming apt is the default or using explicitly
 #   -c tools.system.package_manager:tool=apt-get
 $ conan install . --format=json
"graph": {
     "nodes": [
         {
             "ref": "",
             "id": 0,
             "settings": {
                 "arch": "x86_64"
             },
             "system_requires": {
                 "apt-get": {
                     "install": [
                         "pkg1",
                         "pkg2"
                     ],
                     "missing": []
                 }
             },
```

A similar result can be obtained without even installing binaries, we could use the `report` or `report-installed`

modes. The `report` mode displays the `install` packages, those are the packages that are required to be installed, irrespective of whether they are actually installed or not. The `report` mode does not check the system for those package, so it could even be ran in another OS:

```
$ conan graph info . -c tools.system.package_manager:mode=report --format=json
...
"system_requires": {
    "apt-get": {
        "install": [
            "pkg1",
            "pkg2"
        ]
    }
}
```

On the other hand, the `report-installed` mode will do a check if the package is installed in the system or not, but not failing nor raising any error if it is not found:

```
$ conan graph info . -c tools.system.package_manager:mode=report-installed --format=json
...
"system_requires": {
    "apt-get": {
        "install": [
            "pkg1",
            "pkg2"
        ],
        "missing": [
            "pkg1",
            "pkg2"
        ]
    }
}
```

Build time system requirements

In some scenarios, it might be possible that some system-requirements are only necessary exclusively at build time. For those scenarios, there are several possibilities:

- Use the `build_system_requirements()` method (from Conan 2.30), that is similar to this `system_requirements()` but exclusively installed when it is necessary to build from source (and using the "build" architecture). See *docs for build_system_requirements()*
- Wrap the installation and logic for the build-time system requirement in its own package recipe, and use that recipe as a `tool_requires`.

There are examples *for build-time system requirements in this section*

test()

The `test()` method is only used for **test_package/conanfile.py**. It will execute immediately after `build()` has been called, and its goal is to run some executable or tests on binaries to prove the package is correctly created. Note that it is intended to be used as a test of the package: the headers are found, the libraries are found, it is possible to link, etc. But it is **not intended** to run unit, integration or functional tests.

It usually takes the form of:

```
def test(self):
    if can_run(self):
        cmd = os.path.join(self.cpp.build.bindir, "example")
        self.run(cmd, env="conanrun")
```

See also:

- See *the "testing packages" tutorial* for more information.
- The *test_package_folder attribute* allows defining a different default location of the test-package instead of the default `test_package` folder.

validate()

The `validate()` method can be used to mark a package binary as "invalid", or not working for the current configuration. For example, if we have a header-only library that doesn't work in Windows, we could have the following `conanfile.py`:

```
from conan import ConanFile
from conan.errors import ConanInvalidConfiguration

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0"
    package_type = "header-library"
    settings = "os"

    def validate(self):
        if self.settings.os == "Windows":
            raise ConanInvalidConfiguration("Windows not supported")

    def package_id(self):
        self.info.clear()  # header-only
```

If we try to create this package in Windows, it will fail, but if we do it in Linux, it will succeed:

```
$ conan create . -s os=Windows # FAILS
...
ERROR: There are invalid packages:
pkg/1.0: Invalid: Windows not supported
$ conan create . -s os=Linux # WORKS
```

And if we try to use it in Windows, it will fail again:

```
$ conan install --requires=pkg/1.0 -s os=Windows # FAILS
...
```

(continues on next page)

(continued from previous page)

```
ERROR: There are invalid packages:
pkg/1.0: Invalid: Windows not supported
```

When the `ConanInvalidConfiguration` causes an error, Conan application exit code will be `6`

It is possible to check the validity of a given graph without raising errors with the `conan graph info` command:

```
$ conan graph info --requires=pkg/1.0 -s os=Windows --filter=binary
conanfile:
ref: conanfile
binary: None
pkg/1.0#cfc18fcc7a50ead278a7c1820be74e56:
ref: pkg/1.0#cfc18fcc7a50ead278a7c1820be74e56
binary: Invalid
```

The `validate()` method is evaluated after the whole graph has been computed. This means that it can use the `self.dependencies` information to raise errors:

```
from conan import ConanFile
from conan.errors import ConanInvalidConfiguration

class Pkg(ConanFile):
    requires = "dep/0.1"

    def validate(self):
        if self.dependencies["dep"].options.myoption == 2:
            raise ConanInvalidConfiguration("Option 2 of 'dep' not supported")
```

Note: Best practices

The `configure()` method evaluates before the graph is complete, so it doesn't have the real values of the dependencies `options`. The `validate()` method is the one that should be checking those dependencies options values if necessary, not `configure()`.

See also:

- Follow the *tutorial about preparing build from source in recipes*.

validate_build()

The `validate_build()` method is used to verify if a package binary can be **built** with the current configuration. It is different than the `validate()` method which raises when the package cannot be **used** with the current configuration.

The `validate_build()` method can check the `self.settings` and `self.options` values to raise `ConanInvalidConfiguration` if necessary.

```
from conan import ConanFile
from conan.errors import ConanInvalidConfiguration

class Pkg(ConanFile):
    name = "pkg"
    version = "1.0"
```

(continues on next page)

(continued from previous page)

```
    settings = "os", "arch", "compiler", "build_type"

    def package_id(self):
        # For this package, it doesn't matter the compiler used for the binary package
        del self.info.settings.compiler

    def validate_build(self):
        # But we know this cannot be build with "gcc"
        if self.settings.compiler == "gcc":
            raise ConanInvalidConfiguration("This doesn't build in GCC")
```

This package cannot be created with the `gcc` compiler, but it can be created with other:

```
$ conan create . -s compiler=gcc
...
ERROR: There are invalid packages:
pkg/1.0: Cannot build for this configuration: This doesn't build in GCC

$ conan create . -s compiler=clang  # WORKS!
```

Once the package has been built, it can be consumed with that compiler:

```
$ conan install --requires=pkg/1.0 -s compiler=gcc # WORKS!
```

- *build()*: Contains the build instructions to build a package from source
- *build_id()*: Allows reusing the same build to create different package binaries
- *build_requirements()*: Defines `tool_requires` and `test_requires`
- *build_system_requirements()*: Installs system packages when building from source (uses build arch)
- *compatibility()*: Defines binary compatibility at the recipe level
- *configure()*: Allows configuring settings and options while computing dependencies
- *config_options()*: Configure options while computing dependency graph
- *deploy()*: Deploys (copy from package to user folder) the desired artifacts
- *export()*: Copies files that are part of the recipe
- *export_sources()*: Copies files that are part of the recipe sources
- *finalize()*: Customizes the package for using it in the running machine without affecting the original package
- *generate()*: Generates the files that are necessary for building the package
- *init()*: Special initialization of recipe when extending from `python_requires`
- *layout()*: Defines the relative project layout, source folders, build folders, etc.
- *package()*: Copies files from build folder to the package folder.
- *package_id()*: Defines special logic for computing the binary `package_id` identifier
- *package_info()*: Provide information for consumers of this package about libraries, folders, etc.
- *requirements()*: Define the dependencies of the package
- *set_name()*: Dynamically define the name of a package
- *set_version()*: Dynamically define the version of a package.

- *source()*: Contains the commands to obtain the source code used to build
- *system_requirements()*: Call system package managers like Apt to install system packages
- *test()*: Run some simple package test (exclusive of `test_package`)
- *validate()*: Define if the current package is invalid (cannot work) with the current configuration.
- *validate_build()*: Define if the current package cannot be created with the current configuration.

9.3.3 Running and output

Output text from recipes

Use the `self.output` attribute to output text from the recipes. Do **not** use Python's `print()` function.

error(*self*, *msg: str*, *error_type: str = None*)

Indicates that a serious issue has occurred that prevents the system or application from continuing to function correctly.

Typically, this represents a failure in the normal flow of execution, such as a service crash or a critical exception. Notice that if the user has set the `core:warnings_as_errors` configuration, this will raise an exception when the output is printed, so that the error does not pass unnoticed.

warning(*self*, *msg: str*, *warn_tag: str = None*)

Highlights a potential issue that, while not stopping the system, could cause problems in the future or under certain conditions.

Warnings signal abnormal situations that should be reviewed but don't necessarily cause an immediate halt in operations. Notice that if the tag matches the pattern in the `core:warnings_as_errors` configuration, and is not skipped, this will be upgraded to an error, and raise an exception when the output is printed, so that the error does not pass unnoticed.

success(*self*, *msg: str*)

Shows that an operation has been completed successfully.

This type of message is useful to confirm that key processes or tasks have finished correctly, which is essential for good application monitoring.

highlight(*self*, *msg: str*)

Marks or emphasizes important events or processes that need to stand out but don't necessarily indicate success or error.

These messages draw attention to key points that may be relevant for the user or administrator.

info(*self*, *msg: str*, *fg: str = None*, *bg: str = None*, *newline: bool = True*)

Provides general information about the system or ongoing operations.

Info messages are basic and used to inform about common events, like the start or completion of processes, without implying specific problems or achievements.

status(*self*, *msg: str*, *fg: str = None*, *bg: str = None*, *newline: bool = True*)

Provides general information about the system or ongoing operations.

Info messages are basic and used to inform about common events, like the start or completion of processes, without implying specific problems or achievements.

The following three methods are not shown by default and are usually reserved for scenarios that require a higher level of verbosity. You can display them using the arguments `-v`, `-vv`, and `-vvv` respectively.

verbose(*self*, *msg: str*, *fg: str = None*, *bg: str = None*)

Displays additional and detailed information that, while not critical, can be useful for better understanding how the system is working.

This message won't be printed unless the user has set the log level to verbose (e.g., using the `-v` option in the command line).

It's appropriate for gaining more context without overloading the logs with excessive detail. Useful when more clarity is needed than a simple info.

debug(*self*, *msg: str*, *fg: str = '\x1b[35m'*, *bg: str = None*)

With a high level of detail, it is mainly used for debugging code.

This message won't be printed unless the user has set the log level to debug (e.g., using the `-vv` option in the command line).

These messages provide useful information for developers, such as variable values or execution flow details, to trace errors or analyze the program's behavior.

trace(*self*, *msg: str*)

This is the most extreme level of detail.

Trace messages log every little step the system takes, including function entries and exits, variable changes, and other very specific events.

This message won't be printed unless the user has set the log level to trace (e.g., using the `-vvv` option in the command line).

It's used when full visibility of everything happening in the system is required, but should be used carefully due to the large amount of information it can generate.

These output functions will only output if the verbosity level with which Conan was launched is the same or higher than the message, so running with `-vwarning` will output calls to `warning()` and `error()`, but not `info()` (Additionally, the `highlight()` and `success()` methods have a `-vnotice` verbosity level)

Note that these methods return the output object again, so that you can chain output calls if needed.

Using the `core:warnings_as_errors` conf, you can make Conan raise an exception when either errors or a tagged warning matching any of the given patterns is printed. This is useful to make sure that recipes are not printing unexpected warnings or errors. Additionally, you can skip which warnings trigger an exception *with the core:skip_warnings conf*.

```
# Raise an exception if any warning or error is printed
core:warnings_as_errors=['*']
# But skip the deprecation warnings
core:skip_warnings=['deprecated']
```

Both confs accept a list of patterns to match against the warning tags. A special `unknown` value can be used to match any warning without a tag.

To tag a warning, use the `warn_tag` argument of the `warning()` method in your recipes:

```
self.output.warning("Extra warning", warn_tag="custom_tag")
```

Note: Custom commands and tools are free to instantiate their own `ConanOutput` object.

Some methods have optional `fg` and `bg` arguments, these are colour codes for the foreground and background of the text, available in the `conan.api.output.Color` class.

```
self.output.info("This is a message", fg=Color.BLUE, bg=Color.YELLOW)
```

Running commands

Recipes and helpers can use the `self.run()` method to run system commands while injecting the calls to activate the appropriate environment, and throw exceptions when errors occur so that command errors do not pass unnoticed. It also wraps the commands with the results of the *command wrapper plugin*.

run(*self*, *command: str*, *stdout=None*, *cwd=None*, *ignore_errors=False*, *env=''*, *quiet=False*, *shell=True*, *scope='build'*, *stderr=None*)

Run a command in the current package context.

Parameters

- **command** – The command to run formatted as a plain string
- **stdout** – The output stream to write the command output. If `None`, it defaults to the standard output stream.
- **stderr** – The error output stream to write the command error output. If `None`, it defaults to the standard error stream.
- **cwd** – The current working directory to run the command in.
- **ignore_errors** – If `True`, do not raise an error if the command returns a non-zero exit code.
- **env** – The environment file to use. If empty, it defaults to `"conanbuild"` for when `scope` is `build` or `"conanrun"` for `run`. If set to `None` explicitly, no environment file will be applied, which is useful for commands that do not require any environment.
- **quiet** – If `True`, suppress the output of the command.
- **shell** – If `True`, run the command in a shell. This is passed to the underlying `Popen` function. If set to `False`, `env` parameter should be set to `None` (a shell is needed in order to source files)
- **scope** – The scope of the command, either `"build"` or `"run"`.

Use the `stdout` and `stderr` arguments to redirect the output of the command to a file-like object instead of the console.

```
# Redirect stdout to a file
with open("ninja_stdout.log", "w") as stdout:
    # Redirect stderr to a StringIO object to be able to read it later
    stderr = StringIO()
    self.run("ninja ...", stdout=stdout, stderr=stderr)
```

9.4 conanfile.txt

The `conanfile.txt` file is a simplified version of `conanfile.py`, aimed at simple consumption of dependencies, but it cannot be used to create a package. Also, it is not necessary to have a `conanfile.txt` for consuming dependencies, a `conanfile.py` is perfectly suited for simple consumption of dependencies.

It also provides a simplified functionality, for example it is not possible to express conditional requirements in `conanfile.txt`, and it will be necessary to use a `conanfile.py` for that. Read *Understanding the flexibility of using conanfile.py vs conanfile.txt* for more information about this.

9.4.1 [requires]

List of requirements, specifying the full reference. Equivalent to `self.requires(<ref>)` in `conanfile.py`.

```
[requires]
poco/1.9.4
zlib/1.3.1
```

This section supports references with version-ranges too:

```
[requires]
poco/[>1.0 <1.9]
zlib/1.3.1
```

And specific recipe revisions can be pinned too:

```
[requires]
zlib/1.2.13#revision1
boost/1.70.0#revision2
```

Note: Note that pinning a revision when using version ranges has not effect and Conan will warn about it.

9.4.2 [tool_requires]

List of tool requirements (executable tools) specifying the full reference. Equivalent to `self.tool_requires()` in `conanfile.py`.

```
[tool_requires]
7zip/16.00
cmake/3.23.0
```

This section also supports version ranges and pinned recipe revisions, as above.

In practice the `[tool_requires]` will be always installed (same as `[requires]`) as installing from a *conanfile.txt* means that something is going to be built, so the tool requirements are indeed needed. Note however, that by default `tool_requires` live in the "build" context, they cannot be libraries to built with, just executable tools, and for example, using the `CMakeDeps` generator, they will not create CMake config files for them (an exception is possible, but it requires using a `conanfile.py`, read the *CMakeDeps reference* for more information).

9.4.3 [test_requires]

List of test requirements specifying the full reference. Equivalent to `self.test_requires()` in `conanfile.py`.

```
[test_requires]
gtest/1.12.1
```

This section also supports version ranges and pinned recipe revisions, as above. The behavior of `test_requires` is totally equivalent to the `[requires]` section above, as the only difference is that `test_requires` are not propagated to consumers, but as a `conanfile.txt` is never creating a package that can be consumed, it is irrelevant. It is provided to maintain the equivalence with `conanfile.py`

9.4.4 [generators]

List of built-in generators to be used, equivalent to the `conanfile.py` `generators = "CMakeDeps", ...` attribute.

```
[requires]
poco/1.9.4
zlib/1.2.13

[generators]
CMakeDeps
CMakeToolchain
```

9.4.5 [options]

List of options scoped for each package with a pattern like **package_name*:option = Value**.

```
[requires]
poco/1.9.4
zlib/1.3.1

[generators]
CMakeDeps
CMakeToolchain

[options]
poco/*:shared=True
openssl/*:shared=True
```

For example using `*:shared=True` will define `shared=True` for all packages in the dependency graph that have this option defined.

> **Warning:** Defining options values in conanfile.txt does not have strong guarantees, please check *this FAQ about options values for dependencies*. The recommended way to define options values is in **profile files**.

9.4.6 [layout]

You can specify one name of a predefined layout. The available values are:

- cmake_layout
- vs_layout
- bazel_layout (experimental)

```
[layout]
cmake_layout
```

See also:

Read *Understanding the flexibility of using conanfile.py vs conanfile.txt* for more information about conanfile.txt vs conanfile.py.

9.5 Conan Server

Important: This server is mainly used for testing (though it might work fine for small teams). We recommend using the free *Artifactory Community Edition for C/C++* for private development or **Artifactory Pro** as Enterprise solution.

9.5.1 Configuration

By default your server configuration is saved under `~/.conan_server/server.conf`, however you can modify this behaviour by either setting the `CONAN_SERVER_HOME` environment variable or launching the server with `-d` or `--server_dir` command line argument followed by desired path. In case you use one of the options your configuration file will be stored under `server_directory/server.conf` Please note that command line argument will override the environment variable. You can change configuration values in `server.conf`, prior to launching the server. Note that the server does not support hot-reload, and thus in order to see configuration changes you will have to manually relaunch the server.

The server configuration file is by default:

```
[server]
jwt_secret: IJKhyoioUINMXCRTytrR
jwt_expire_minutes: 120

ssl_enabled: False
port: 9300

public_port:
host_name: localhost

authorize_timeout: 1800

disk_storage_path: ./data
disk_authorize_timeout: 1800
updown_secret: HJhjujkjkjkJKLUYyuuyHJ

[write_permissions]
# "opencv/2.3.4@lasote/testing": default_user,default_user2

[read_permissions]
*/*@*/*: *

[users]
demo: demo
```

Server Parameters

Note: The Conan server supports relative URLs, allowing you to avoid setting `host_name`, `public_port` and `ssl_enabled`. The URLs used to upload/download packages will be automatically generated in the client following the URL of the remote. This allows accessing the Conan server from different networks.

- `port`: Port where **conan_server** will run.
- The client server authorization is done with JWT. `jwt_secret` is a random string used to generate authentication tokens. You can change it safely anytime (in fact it is a good practice). The change will just force users to log in again. `jwt_expire_minutes` is the amount of time that users remain logged-in within the client without having to introduce their credentials again.
- `host_name`: If you set `host_name`, you must use the machine's IP where you are running your server (or domain name), something like **host_name: 192.168.1.100**. This IP (or domain name) has to be visible (and resolved) by the Conan client, so take it into account if your server has multiple network interfaces.
- `public_port`: Might be needed when running virtualized, Docker or any other kind of port redirection. File uploads/downloads are served with their own URLs, generated by the system, so the file storage backend is independent. Those URLs need the public port they have to communicate from the outside. If you leave it blank, the `port` value is used.

  Example: Use conan_server in a Docker container that internally runs in the 9300 port but exposes the 9999 port (where the clients will connect to):

  ```
  docker run ... -p9999:9300 ... # Check Docker docs for that
  ```

  server.conf

  ```
  [server]

  ssl_enabled: False
  port: 9300
  public_port: 9999
  host_name: localhost
  ```

- `ssl_enabled` Conan doesn't handle the SSL traffic by itself, but you can use a proxy like *Nginx to redirect the SSL traffic to your Conan server*. If your Conan clients are connecting with "https", set *ssl_enabled* to True. This way the conan_server will generate the upload/download urls with "https" instead of "http".

Note: **Important**: The Conan client, by default, will validate the server SSL certificates and won't connect if it's invalid. If you have self signed certificates you have two options:

1. Use the **`conan remote`** command to disable the SSL certificate checks. E.g., *conan remote add/update myremote https://somedir False*
2. If using the *core.net.http:cacert_path* configuration in the Conan client, append the server *.crt* file contents to the *cacert.pem* location.

The folder in which the uploaded packages are stored (i.e., the folder you would want to backup) is defined in the `disk_storage_path`. The storage backend might use a different channel, and uploads/downloads are authorized up to a maximum of `authorize_timeout` seconds. The value should sufficient so that large downloads/uploads are not rejected, but not too big to prevent hanging up the file transfers. The value `disk_authorize_timeout` is not currently used. File transfers are authorized with their own tokens, generated with the secret `updown_secret`. This value should be different from the above `jwt_secret`.

Permissions Parameters

By default, the server configuration when set to Read can be done anonymous, but uploading requires you to be registered users. Users can easily be registered in the `[users]` section, by defining a pair of `login: password` for each one. Plain text passwords are used at the moment, but as the server is on-premises (behind firewall), you just need to trust your sysadmin :)

If you want to restrict read/write access to specific packages, configure the `[read_permissions]` and `[write_permissions]` sections. These sections specify the sequence of patterns and authorized users, in the form:

```
# use a comma-separated, no-spaces list of users
package/version@user/channel: allowed_user1,allowed_user2
```

E.g.:

```
*/*@*/*: * # allow all users to all packages
PackageA/*@*/*: john,peter # allow john and peter access to any PackageA
*/*@project/*: john # Allow john to access any package from the "project" user
```

The rules are evaluated in order. If the left side of the pattern matches, the rule is applied and it will not continue searching for matches.

Authentication

By default, Conan provides a simple `user: password` users list in the `server.conf` file.

There is also a plugin mechanism for setting other authentication methods. The process to install any of them is a simple two-step process:

1. Copy the authenticator source file into the `.conan_server/plugins/authenticator` folder.
2. Add `custom_authenticator: authenticator_name` to the `server.conf` [server] section.

This is a list of available authenticators, visit their URLs to retrieve them, but also to report issues and collaborate:

- **htpasswd**: Use your server Apache htpasswd file to authenticate users. Get it: https://github.com/d-schiffner/conan-htpasswd
- **LDAP**: Use your LDAP server to authenticate users. Get it: https://github.com/uilianries/conan-ldap-authentication

Create Your Own Custom Authenticator

If you want to create your own Authenticator, create a Python module in `~/.conan_server/plugins/authenticator/my_authenticator.py`

Example:

```
def get_class():
    return MyAuthenticator()


class MyAuthenticator(object):
    def valid_user(self, username, plain_password):
        return username == "foo" and plain_password == "bar"
```

The module has to implement:

- A factory function `get_class()` that returns a class with a `valid_user()` method instance.
- The class containing the `valid_user()` that has to return True if the user and password are valid or False otherwise.

Authorizations

By default, Conan uses the contents of the `[read_permissions]` and `[write_permissions]` sections to authorize or reject a request.

A plugin system is also available to customize the authorization mechanism. The installation of such a plugin is a simple two-step process:

1. Copy the authorizer's source file into the `.conan_server/plugins/authorizer` folder.
2. Add `custom_authorizer: authorizer_name` to the `server.conf` [server] section.

Create Your Own Custom Authorizer

If you want to create your own Authorizer, create a Python module in `~/.conan_server/plugins/authorizer/my_authorizer.py`

Example:

```
from conan.internal.errors import AuthenticationException, ForbiddenException

def get_class():
    return MyAuthorizer()

class MyAuthorizer(object):
    def _check_conan(self, username, ref):
        if ref.user == username:
            return

        if username:
            raise ForbiddenException("Permission denied")
        else:
            raise AuthenticationException()

    def _check_package(self, username, pref):
       self._check(username, pref.ref)

    check_read_conan = _check_conan check_write_conan = _check_conan
    check_delete_conan = _check_conan check_read_package = _check_package
    check_write_package = _check_package check_delete_package = _check_package
```

The module has to implement:

- A factory function `get_class()` that returns an instance of a class conforming to the Authorizer's interface.
- **A class that implements all the methods defined in the Authorizer interface:**
  - `check_read_conan()` is used to decide whether to allow read access to a recipe.
  - `check_write_conan()` is used to decide whether to allow write access to a recipe.
  - `check_delete_conan()` is used to decide whether to allow a recipe's deletion.

- `check_read_package()` is used to decide whether to allow read access to a package.
- `check_write_package()` is used to decide whether to allow write access to a package.
- `check_delete_package()` is used to decide whether to allow a package's deletion.

The `check_*_conan()` methods are called with a username and `conans.model.ref.ConanFileReference` instance as their arguments. Meanwhile the `check_*_package()` methods are passed a username and `conans.model.ref.PackageReference` instance as their arguments. These methods should raise an exception, unless the user is allowed to perform the requested action.

9.5.2 Running the Conan Server with SSL using Nginx

server.conf

```
[server] port: 9300
```

nginx conf file

```
server {
    listen 443; server_name myservername.mydomain.com;

    location / {
      proxy_pass http://0.0.0.0:9300;
    } ssl on; ssl_certificate /etc/nginx/ssl/server.crt; ssl_certificate_key
    /etc/nginx/ssl/server.key;
}
```

remote configuration in Conan client

```
$ conan remote add myremote https://myservername.mydomain.com
```

9.5.3 Running the Conan Server with SSL using Nginx in a Subdirectory

server.conf

```
[server] port: 9300
```

nginx conf file

```
server {

       listen 443; ssl on; ssl_certificate /usr/local/etc/nginx/ssl/server.crt;
       ssl_certificate_key /usr/local/etc/nginx/ssl/server.key; server_name
       myservername.mydomain.com;

       location /subdir/ {
          proxy_pass http://0.0.0.0:9300/;
       }
  }
```

remote configuration in Conan client

```
$ conan remote add myremote https://myservername.mydomain.com/subdir/
```

9.5.4 Running Conan Server using Apache

You need to install `mod_wsgi`. If you want to use Conan installed from `pip`, the conf file should be similar to the following example:

Apache conf file (e.g., /etc/apache2/sites-available/0_conan.conf)

```
<VirtualHost *:80>
    WSGIScriptAlias /
    /usr/local/lib/python3.6/dist-packages/conans/server/server_launcher.py
    WSGICallableObject app WSGIPassAuthorization On

    <Directory /usr/local/lib/python3.6/dist-packages/conans>
        Require all granted
    </Directory>
</VirtualHost>
```

If you want to use Conan checked out from source in, for example in */srv/conan*, the conf file should be as follows:

Apache conf file (e.g., /etc/apache2/sites-available/0_conan.conf)

```
<VirtualHost *:80>
    WSGIScriptAlias / /srv/conan/conans/server/server_launcher.py
    WSGICallableObject app WSGIPassAuthorization On

    <Directory /srv/conan/conans>
        Require all granted
    </Directory>
</VirtualHost>
```

The directive `WSGIPassAuthorization On` is needed to pass the HTTP basic authentication to Conan.

Also take into account that the server config files are located in the home of the configured Apache user, e.g., var/www/.conan_server, so remember to use that directory to configure your Conan server.

See also:

- *Setting-up a Conan Server*

9.6 Configuration files

These are the most important configuration files, used to customize conan.

9.6.1 global.conf

The **global.conf** file is located in the Conan user home directory, e.g., *[CONAN_HOME]/global.conf*. If it does not already exist, a default one is automatically created.

Introduction to configuration

global.conf is aimed to save some core/tools/user configuration variables that will be used by Conan. For instance:

- Package ID modes.
- General HTTP(python-requests) configuration.
- Number of retries when downloading/uploading recipes.
- Related tools configurations (used by toolchains, helpers, etc.)
- *Policies*, which are a set of rules to enforce certain behaviors from Conan.
- Others (required Conan version, CLI non-interactive, etc.)

Let's briefly explain the three types of existing configurations:

- `core.*`: aimed to configure values of Conan core behavior (download retries, package ID modes, etc.). Only definable in *global.conf* file.
- `tools.*`: aimed to configure values of Conan tools (toolchains, build helpers, etc.) used in your recipes. Definable in both *global.conf* and *profiles*.
- `user.*`: aimed to define personal user configurations. They can define whatever user wants. Definable in both *global.conf* and *profiles*.

To list all the possible configurations available, run **`conan config list`**:

```
$ conan config list
core.cache:storage_path: Absolute path where the packages and database are stored
core.download:download_cache: Define path to a file download cache. Files uploaded to a␣
↪server are also stored here, so a later download of the same artifact can be skipped
core.download:parallel: Number of concurrent threads to download packages
core.download:retry:  (int, default: 2) Number of retries in case of failure when␣
↪downloading from Conan server
core.download:retry_wait: (int, default: 1s) Seconds to wait between download attempts␣
↪from Conan server
core.graph:compatibility_mode: (Experimental) Set this to 'optimized' to enable the␣
↪improved compatibility behaviour when querying multiple compatible binaries in remotes
core.gzip:compresslevel: The Gzip compression level for Conan artifacts (default=9)
core.net.http:cacert_path: Path containing a custom Cacert file
core.net.http:clean_system_proxy: If defined, the proxies system env-vars will be␣
↪discarded
core.net.http:client_cert: Path or tuple of files containing a client cert (and key)
core.net.http:max_retries: Maximum number of connection retries (requests library)
core.net.http:no_proxy_match: List of urls to skip from proxies configuration
core.net.http:proxies: Dictionary containing the proxy configuration
core.net.http:timeout: Number of seconds without response to timeout (requests library)
core.net.http:trust_store: (Experimental) If True, additionally verify HTTPS␣
↪certificates against the OS-native trust store via the 'truststore' (needs Python>=3.
↪10) package (used together with, not instead of, the certifi CA bundle).
core.package_id:config_mode: How the 'config_version' affects binaries. By default 'None'
```

(continues on next page)

(continued from previous page)

```
core.package_id:default_build_mode: By default, 'None'
core.package_id:default_embed_mode: By default, 'full_mode'
core.package_id:default_non_embed_mode: By default, 'minor_mode'
core.package_id:default_python_mode: By default, 'minor_mode'
core.package_id:default_unknown_mode: By default, 'semver_mode'
core.scm:excluded: List of excluded patterns for builtin git dirty checks
core.scm:local_url: By default allows to store local folders as remote url, but not␣
↪upload them. Use 'allow' for allowing upload and 'block' to completely forbid it
core.sources.patch:extra_path: Extra path to search for patch files for conan create
core.sources:download_cache: Folder to store the sources backup
core.sources:download_urls: List of URLs to download backup sources from
core.sources:exclude_urls: URLs which will not be backed up
core.sources:upload_url: Remote URL to upload backup sources to
core.upload:compression_format: The compression format used when uploading Conan␣
↪packages. Possible values: 'zst', 'xz', 'gz' (default=gz)
core.upload:parallel: Number of concurrent threads to upload packages
core.upload:retry: (int, default: 1) Number of retries in case of failure when uploading␣
↪to Conan server
core.upload:retry_wait: (int, default: 5s) Seconds to wait between upload attempts to␣
↪Conan server
core.version_ranges:resolve_prereleases: Whether version ranges can resolve to pre-
↪releases or not
core:allow_uppercase_pkg_names: Temporarily (will be removed in 2.X) allow uppercase␣
↪names
core:compresslevel: The compression level for Conan artifacts (default zstd=3, gz=9)
core:default_build_profile: Defines the default build profile ('default' by default)
core:default_profile: Defines the default host profile ('default' by default)
core:non_interactive: Disable interactive user input, raises error if input necessary
core:policies: A list of opt-in behaviors that can be defined in the configuration to␣
↪control specific aspects of Conan's behavior,
such as keeping deprecated behaviours:
   - If required_conan_version>=2.28, bugfix https://github.com/conan-io/conan/pull/
↪19705 for transitive static libraries package_id
   - If required_conan_version>=2.28, bugfix https://github.com/conan-io/conan/pull/
↪19849 for VirtualBuildEnv bindir path propagation based on requirement run trait
   - If required_conan_version>=2.28, https://github.com/conan-io/conan/pull/19286␣
↪defaults the new 'consistent' trait to True for the host context, even when
↪'visible=False'
   - If required_conan_version>=2.30, bugfix https://github.com/conan-io/conan/pull/
↪20073 for propagation of the 'transitive_header' trait
core:required_conan_version: Raise if current version does not match the defined range.
core:skip_warnings: Do not show warnings matching any of the patterns in this list.␣
↪Current warning tags are 'network', 'deprecated', 'experimental'
core:update_policy: (Legacy). If equal 'legacy' when multiple remotes, update based on␣
↪order of remotes, only the timestamp of the first occurrence of each revision counts.
core:warnings_as_errors: Treat warnings matching any of the patterns in this list as␣
↪errors and then raise an exception. Current warning tags are 'network', 'deprecated'
tools.android:cmake_legacy_toolchain: Define to explicitly pass ANDROID_USE_LEGACY_
↪TOOLCHAIN_FILE in CMake toolchain
tools.android:ndk_path: Argument for the CMAKE_ANDROID_NDK
tools.apple:enable_arc: (boolean) Enable/Disable ARC Apple Clang flags
tools.apple:enable_bitcode: (boolean) Enable/Disable Bitcode Apple Clang flags
```

(continues on next page)

(continued from previous page)

```
tools.apple:enable_visibility: (boolean) Enable/Disable Visibility Apple Clang flags
tools.apple:sdk_path: Path to the SDK to be used
tools.build.cross_building:can_run: (boolean) Indicates whether is possible to run a non-
↪native app on the same architecture. It's used by 'can_run' tool
tools.build.cross_building:cross_build: (boolean) Decides whether cross-building or not␣
↪regardless of arch/OS settings. Used by 'cross_building' tool
tools.build:add_rpath_link: Add -Wl,-rpath-link flags pointing to all lib directories␣
↪for host dependencies (CMake and Meson toolchains)
tools.build:asmflags: List of extra ASM flags used by CMakeToolchain
tools.build:cflags: List of extra C flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:compiler_executables: Defines a Python dict-like with the compilers path to␣
↪be used. Allowed keys {'c', 'cpp', 'cuda', 'objc', 'objcxx', 'rc', 'fortran', 'asm',
↪'hip', 'ispc'}
tools.build:cxxflags: List of extra CXX flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:defines: List of extra definition flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:download_source: Force download of sources for every package
tools.build:exelinkflags: List of extra flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:install_strip: (boolean or list) True/False to strip on install for every␣
↪CMake, Meson and Autotools integration, or a list of 'cmake', 'meson', 'autotools' to␣
↪strip only for those.
tools.build:jobs: Default compile jobs number -jX Ninja, Make, /MP VS (default: max CPUs)
tools.build:linker_scripts: List of linker script files to pass to the linker used by␣
↪different toolchains like CMakeToolchain, AutotoolsToolchain, and MesonToolchain
tools.build:rcflags: List of extra RC (resource compiler) flags used by different␣
↪toolchains like CMakeToolchain, MSBuildToolchain and MesonToolchain
tools.build:sharedlinkflags: List of extra flags used by different toolchains like␣
↪CMakeToolchain, AutotoolsToolchain and MesonToolchain
tools.build:skip_test: Do not execute CMake.test() and Meson.test() when enabled
tools.build:sysroot: Pass the --sysroot=<tools.build:sysroot> flag if available. (None␣
↪by default)
tools.build:verbosity: Verbosity of build systems if set. Possible values are 'quiet'␣
↪and 'verbose'
tools.cmake.cmake_layout:build_folder: (Experimental) Allow configuring the base folder␣
↪of the build for local builds
tools.cmake.cmake_layout:build_folder_vars: Settings and Options that will produce a␣
↪different build folder and different CMake presets names
tools.cmake.cmake_layout:test_folder: (Experimental) Allow configuring the base folder␣
↪of the build for test_package
tools.cmake.cmakedeps:new: Use the new CMakeDeps generator
tools.cmake.cmaketoolchain:enabled_blocks: Select the specific blocks to use in the␣
↪conan_toolchain.cmake
tools.cmake.cmaketoolchain:extra_variables: Dictionary with variables to be injected in␣
↪CMakeToolchain (potential override of CMakeToolchain defined variables)
tools.cmake.cmaketoolchain:find_package_prefer_config: Argument for the CMAKE_FIND_
↪PACKAGE_PREFER_CONFIG
tools.cmake.cmaketoolchain:generator: User defined CMake generator to use instead of␣
↪default
tools.cmake.cmaketoolchain:presets_environment: String to define wether to add or not␣
```

(continues on next page)

(continued from previous page)

```
↪the environment section to the CMake presets. Empty by default, will generate the␣
↪environment section in CMakePresets. Can take values: 'disabled'.
tools.cmake.cmaketoolchain:system_name: Define CMAKE_SYSTEM_NAME in CMakeToolchain
tools.cmake.cmaketoolchain:system_processor: Define CMAKE_SYSTEM_PROCESSOR in␣
↪CMakeToolchain
tools.cmake.cmaketoolchain:system_version: Define CMAKE_SYSTEM_VERSION in CMakeToolchain
tools.cmake.cmaketoolchain:toolchain_file: Use other existing file rather than conan_
↪toolchain.cmake one
tools.cmake.cmaketoolchain:toolset_arch: Toolset architecture to be used as part of␣
↪CMAKE_GENERATOR_TOOLSET in CMakeToolchain
tools.cmake.cmaketoolchain:toolset_cuda: (Experimental) Path to a CUDA toolset to use,␣
↪or version if installed at the system level
tools.cmake.cmaketoolchain:user_presets: (Experimental) Select a different name or␣
↪relative filepath instead of CMakeUserPresets.json, empty to disable. A relative␣
↪filepath (with directory component) enables CMake 4.4+ --preset-file mode
tools.cmake.cmaketoolchain:user_toolchain: Inject existing user toolchains at the␣
↪beginning of conan_toolchain.cmake
tools.cmake:cmake_program: Path to CMake executable
tools.cmake:configure_args: Add extra arguments to CMake.configure() command line
tools.cmake:ctest_args: Add extra arguments to CMake.ctest() runner command line
tools.cmake:install_strip: (Deprecated) Add --strip to cmake.install(). Use tools.
↪build:install_strip instead
tools.compilation:verbosity: Verbosity of compilation tools if set. Possible values are
↪'quiet' and 'verbose'
tools.deployer:symlinks: Set to False to disable deployers copying symlinks
tools.env.virtualenv:powershell: If specified, it generates PowerShell launchers (.ps1).␣
↪Use this configuration setting the PowerShell executable you want to use (e.g.,
↪'powershell.exe' or 'pwsh')
tools.env:deactivation_mode: (Experimental) If 'function', generate a deactivate␣
↪function instead of a script to unset the environment variables
tools.env:dotenv: (Experimental) Generate dotenv environment files
tools.files.download:retry: (int, default: 2) Number of retries in case of failure when␣
↪downloading
tools.files.download:retry_wait: (int, default: 5s) Seconds to wait between download␣
↪attempts
tools.files.download:verify: If set, overrides recipes on whether to perform SSL␣
↪verification for their downloaded files. Only recommended to be set while testing
tools.files.unzip:filter: Define tar extraction filter: 'fully_trusted', 'tar', 'data'
tools.gnu:build_triplet: Custom build triplet to pass to Autotools scripts
tools.gnu:define_libcxx11_abi: Force definition of GLIBCXX_USE_CXX11_ABI=1 for␣
↪libstdc++11
tools.gnu:disable_flags: Disable the automatic addition of flags to some build systems.␣
↪List of possible values: ['arch', 'arch_link', 'libcxx', 'build_type', 'build_type_link
↪', 'threads','cppstd', 'cstd']
tools.gnu:extra_configure_args: List of extra arguments to pass to configure when using␣
↪AutotoolsToolchain and GnuToolchain
tools.gnu:host_triplet: Custom host triplet to pass to Autotools scripts
tools.gnu:make_program: Indicate path to make program
tools.gnu:pkg_config: Path to pkg-config executable used by PkgConfig build helper
tools.google.bazel:bazelrc_path: List of paths to bazelrc files to be used as 'bazel --
↪bazelrc=rcpath1 ... build'
tools.google.bazel:configs: List of Bazel configurations to be used as 'bazel build --
```

(continues on next page)

(continued from previous page)

```
↪config=config1 ...'
tools.graph:skip_binaries: Allow the graph to skip binaries not needed in the current␣
↪configuration (True by default)
tools.graph:skip_build: (Experimental) Do not expand build/tool_requires
tools.graph:skip_test: (Experimental) Do not expand test_requires. If building it might␣
↪need 'tools.build:skip_test=True'
tools.graph:vendor: (Experimental) If 'build', enables the computation of dependencies␣
↪of vendoring packages to build them
tools.info.package_id:confs: List of existing configuration to be part of the package ID
tools.intel:installation_path: Defines the Intel oneAPI installation root path
tools.intel:setvars_args: Custom arguments to be passed onto the setvars.sh|bat script␣
↪from Intel oneAPI
tools.meson.mesontoolchain:backend: Any Meson backend: ninja, vs, vs2010, vs2012, vs2013,
↪ vs2015, vs2017, vs2019, xcode
tools.meson.mesontoolchain:extra_machine_files: List of paths for any additional native/
↪cross file references to be appended to the existing Conan ones
tools.meson.mesontoolchain:extra_variables: Dict of dicts defining extra variables per␣
↪meson file section: 'properties', 'binaries', 'project_options'
tools.microsoft.bash:active: Set True only when Conan runs in a POSIX Bash (MSYS2/
↪Cygwin) where Python's subprocess (shell=True) uses a POSIX-compatible shell (e.g., /
↪bin/sh). Do not set when using Conan from cmd/PowerShell or with native Windows Python␣
↪('win32').
tools.microsoft.bash:path: The path to the shell to run when conanfile.win_bash==True
tools.microsoft.bash:subsystem: The subsystem to be used when conanfile.win_bash==True.␣
↪Possible values: msys2, msys, cygwin, wsl, sfu
tools.microsoft.msbuild:installation_path: VS install path, to avoid auto-detect via␣
↪vswhere, like C:/Program Files (x86)/Microsoft Visual Studio/2019/Community. Use empty␣
↪string to disable
tools.microsoft.msbuild:max_cpu_count: Argument for the /m when running msvc to build␣
↪parallel projects
tools.microsoft.msbuild:vs_version: Defines the IDE version (15, 16, 17) when using the␣
↪msvc compiler. Necessary if compiler.version specifies a toolset that is not the IDE␣
↪default
tools.microsoft.msbuilddeps:exclude_code_analysis: Suppress MSBuild code analysis for␣
↪patterns
tools.microsoft.msbuildtoolchain:compile_options: Dictionary with MSBuild compiler␣
↪options
tools.microsoft:msvc_update: Force the specific update irrespective of compiler.update␣
↪(CMakeToolchain and VCVars)
tools.microsoft:winsdk_version: Use this winsdk_version in vcvars
tools.system.package_manager:mode: Mode for package_manager tools: 'check', 'report',
↪'report-installed' or 'install'
tools.system.package_manager:sudo: Use 'sudo' when invoking the package manager tools in␣
↪Linux (False by default)
tools.system.package_manager:sudo_askpass: Use the '-A' argument if using sudo in Linux␣
↪to invoke the system package manager (False by default)
tools.system.package_manager:tool: Default package manager tool: 'apk', 'apt-get', 'yum',
↪ 'dnf', 'brew', 'pacman', 'choco', 'zypper', 'pkg' or 'pkgutil'
tools.system.pipenv:python_interpreter: (Deprecated) Use 'tools.system.pyenv:python_
↪interpreter' instead. Path to the Python interpreter to be used to create the␣
↪virtualenv
tools.system.pyenv:python_interpreter: (Experimental) Path to the Python interpreter to␣
```

(continues on next page)

(continued from previous page)

```
↪be used to create the virtualenv
```

User/Tools configurations

Tools and user configurations can be defined in both the *global.conf* file and *Conan profiles*. They look like:

Listing 34: *global.conf*

```
tools.build:verbosity=verbose
tools.microsoft.msbuild:max_cpu_count=2
tools.microsoft.msbuild:vs_version = 16
tools.build:jobs=10
# User conf variable
user.confvar:something=False
```

Important: Profiles values will have priority over globally defined ones in global.conf.

These are some hints about configuration items scope and naming:

- `core.xxx` and `tools.yyy` are Conan built-ins, users cannot define their own ones in these scopes.
- `core.xxx` can be defined in `global.conf` or via the `--core-conf` CLI argument only, but not in profiles.
- `tools.yyy` can be defined in `global.conf`, in profiles `[conf]` section and as CLI `-c` arguments
- **`user.zzz` can be defined everywhere, and they are totally at the user discretion, no established naming convention. However this would be more or less expected:**
  - For open source libraries, specially those in conancenter, `user.packagename:conf` might be expected, like the `boost` recipe defining `user.boost:conf` conf
  - For private usage, the recommendation could be to use something like `user.orgname:conf` for global org configuration across all projects, `user.orgname.project:conf` for project or package configuration, though `user.project:conf` might be also good if the project name is unique enough.
  - They _must_ have one `:` separator, like `user.myorg:conf`, but not `user.myorg.conf` or `user.myorg`. This is to disambiguate from patterns, which are discussed below.

Configuration file template

It is possible to use **jinja2** template engine for *global.conf*. When Conan loads this file, it immediately parses and renders the template, which must result in a standard tools-configuration text.

```
# Using all the cores automatically
tools.build:jobs={{os.cpu_count()}}
# Using the current OS
user.myconf.system:name = {{platform.system()}}
```

Conan also injects `detect_api` (non-stable, read the reference) to the jinja rendering context. You can use it like this:

```
user.myteam:myconf1={{detect_api.detect_os()}}
user.myteam:myconf2={{detect_api.detect_arch()}}
```

For more information on how to use it, please check *the detect_api section* in the profiles reference.

The Python packages passed to render the template are `os`, `platform` and `hashlib` for all platforms and `distro` in Linux platforms. Additionally, the variables `conan_version` and `conan_home_folder` are also available.

The `os`, `platform` and `distro` can be useful to perform different system checks, while the `hashlib` library can be convenient to compute unique hashes based on the `conan_home_folder` to define unique strings, for example for unique shorter paths in Windows in CI systems when sometimes the path length can be an issue, for example:

```
# compute a unique hash based on the current home folder
{% set h = hashlib.new("sha256", conan_home_folder.encode(),
                       usedforsecurity=False).hexdigest() %}
# and use the first 6 characters to compose a short path for package storage
core.cache:storage_path=C:/conan_{{h[:6]}}
```

Configuration data types

All the values will be interpreted by Conan as the result of the python built-in *eval()* function:

```
# String
tools.build:verbosity=verbose
# Boolean
tools.system.package_manager:sudo=True
# Integer
tools.microsoft.msbuild:max_cpu_count=2
# List of values
user.myconf.build:ldflags=["--flag1", "--flag2"]
# Dictionary
tools.microsoft.msbuildtoolchain:compile_options={"ExceptionHandling": "Async"}
```

Warning: Because the value is passed through `eval()`, an input that looks like a numeric literal will be evaluated as such. For example, a version string like `1.10` will be evaluated to the float `1.1`, silently losing the trailing zero:

```
# This becomes the float 1.1, NOT the string "1.10"
user.myorg:docker_image_tag=1.10
```

Note that this mostly affects `user.*` confs, since built-in `core.*` and `tools.*` confs internally declare their type and would not convert a float. For `user.*` confs no type is enforced by default, so the evaluated value is passed through as-is.

The recommended workaround is to prefix the value with a non-numeric character (for example `v`) so `eval()` cannot turn it into a number, and then strip that character in the recipe:

```
# In the profile / global.conf / -c CLI argument
user.myorg:docker_image_tag=v1.10
```

```
# In the recipe
tag = self.conf.get("user.myorg:docker_image_tag", check_type=str).lstrip("v")
```

Wrapping the value in quotes (`"1.10"`) also works from a *global.conf* or profile `[conf]` section (`eval()` will return the string `1.10`), but this typically does not survive shell quoting when passed on the command line with `-c`.

Configuration data operators

It's also possible to use some extra operators when you're composing tool configurations in your *global.conf* or any of your profiles:

- += == `append`: appends values at the end of the existing value (only for lists).
- =+ == `prepend`: puts values at the beginning of the existing value (only for lists).
- *= == `update`: updates the specified keys only, leaving the rest unmodified (only for dictionaries)
- =! == `unset`: gets rid of any configuration value. The =~ operator is an alias for =!.

Listing 35: *global.conf*

```
# Define the value => ["-f1"]
user.myconf.build:flags=["-f1"]

# Append the value ["-f2"] => ["-f1", "-f2"]
user.myconf.build:flags+=["-f2"]

# Prepend the value ["-f0"] => ["-f0", "-f1", "-f2"]
user.myconf.build:flags=+["-f0"]

# Unset the value (both notations are equivalent)
user.myconf.build:flags=!
user.myconf.build:flags=~

# Define the value => {"a": 1, "b": 2}
user.myconf.build:other={"a": 1, "b": 2}

# Update b = 4 => {"a": 1, "b": 4}
user.myconf.build:other*={"b": 4}
```

Configuration patterns

You can use package patterns to apply the configuration in those dependencies which are matching:

```
*:tools.cmake.cmaketoolchain:generator=Ninja
zlib/*:tools.cmake.cmaketoolchain:generator=Visual Studio 16 2019
```

This example shows you how to specify a general `generator` for all your packages except for `zlib` which is defining `Visual Studio 16 2019` as its generator.

Besides that, it's quite relevant to say that **the order matters**. So, if we change the order of the configuration lines above:

```
zlib/*:tools.cmake.cmaketoolchain:generator=Visual Studio 16 2019
*:tools.cmake.cmaketoolchain:generator=Ninja
```

The result is that you're specifying a general `generator` for all your packages, and that's it. The `zlib` line has no effect because it's the first one evaluated, and after that, Conan is overriding that specific pattern with the most general one, so it deserves to pay special attention to the order.

9.6.2 global_user.conf

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Similar to how the `settings_user.yml` can complete the default `settings.yml`, the (new in Conan 2.29) `global_user.conf` allow developers to have their own `[CONAN_HOME]/global_user.conf` file in the Conan cache, with the following properties:

- It also allows `jinja2` templating syntax, with the same inputs as the `global.conf` file.
- Its contents compose and overwrite the existing values in `global.conf` with higher precedence.
- It is designed for developers to be able to customize and deviate from the organization/team common `global.conf`.
- This file is not intended to be managed or installed by `conan config install/install-pkg` commands.
- The existence of this file allows developers to sync and update their organization `global.conf` with `conan config install/install-pkg` without losing their customization
- A `conan config clean` will remove the `global_user.conf` file, as its purpose is to completely reset the Conan home configuration.

9.6.3 Configuration precedence

There are different places where a configuration such as `tools.build:verbosity` can be defined:

- Globally, in the `global.conf` file
- In a `tool_requires` recipe that defines a `self.conf_info` in their `package_info()` method. (Recall that only **direct** `tool_requires` propagate `conf_info` to their consumers).
- In a profile file `[conf]` section
- In the command line `-c tools.build:verbosity=<value>`

In general, the rule is that the "closest to the user" has higher precedence.

That means that:

- The command line arguments like `-c tools.build:verbosity=<value>` will have higher precedence and overwrite possible values already defined in `tool_requires`, in `global.conf` or profiles `[conf]` section. The idea is that the user explicitly requested that typing it in the command line, so they want that value to prevail.
- Then, the profile `[conf]` section will have precedence over the `tool_requires` and `global.conf` definitions, as the profiles are also inputs by the user.
- Then, the `global.conf` will have precedence over `tool_requires` defined `conf_info`. The idea is that the user can define the behavior they want without having to modify or rewrite recipes.
- Finally, the one with less precedence is the `tool_requires` configuration defined in `package_info()` method with `self.conf_info`.

The core configurations such as `core:skip_warnings` can be defined in:

- Globally, in the `global.conf`, with less precedence
- In the command line, with `-cc/--core-conf core:skip_warnings=<value>` with higher precedence over the `global.conf`.

Note that `core` configurations cannot be defined in profiles or in recipes.

Important configurations with `!` specifier

There are some scenarios when it is desired that a recipe defined configuration in `package_info()` via the `conf_info` has higher precedence over a value defined downstream by the user in profiles or command line.

The "important configuration" definition allows this, specifying with the `!` qualifier over the configuration name that such value should have relatively higher priority. Imagine we are writing a `tool_requires` for the Msys2 subsystem, and we would like that recipe to define the `tools.microsoft.bash:path` value so it points to itself. But for some reason we also have `tools.microsoft.bash:path` in our profiles, pointing to a Msys2 that still some packages that do not use the new `msys2/1.0` still need. We could define a recipe like:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "msys2"
    version = "1.0"

    def package_info(self):
      bash = os.path.join(self.package_folder, "bash.exe")
      # Note the ! after the name of the configuration
      # That makes this definition "important", and have higher
      # precedence than in profiles
      self.conf_info.define("tools.microsoft.bash:path!", bash)
      # You can apply the same ! specifier in other "conf_info"
      # operations, for paths, append/prepend, etc
```

with some consumers that requires it:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "mylib"
    version = "1.0"

    def build_requirements(self):
      if self.settings_build.os == "Windows":
        self.tool_requires("msys2/1.0")
```

And then have a profile like

```
[conf]
tools.microsoft.bash:path=<point/to/system/msys2/installation>
```

Then, the `mylib/1.0` will get the `tools.microsoft.bash:path` pointing to the `msys2` path, while other recipes that do not `tool_requires` the `msys2` will still get the system one.

If for some reason a profile or command line would still want to force an override also the important configurations from packages upstream, they can do it using the same syntax:

```
[conf]
# This will force all packages to use the system msys2, even if they
# are tool-requiring the ``msys2/1.0`` package
```

(continues on next page)

(continued from previous page)

```
# Note the ! after the configuration name
tools.microsoft.bash:path!=<point/to/system/msys2/installation>
```

Important: Best practices

The usage of important ! configuration should be exceptional, and reduced to limited cases when there are no other alternatives. Modifying the default precedence, in which users expects their inputs from command line or profiles to be always applied can be confusing for them. Please use this feature sparingly and being aware of these implications.

9.6.4 Information about built-in confs

This section provides extra information about specific confs.

Policies

The `core:policies` conf allows to define policies that will be applied globally to modify the behaviour of Conan in certain aspects. Check *the policies section* for more information.

Networking confs

Configuration of client certificates

Conan supports client TLS certificates. You can configure the path to your existing *Cacert* file and/or your client certificate (and the key) using the following configuration variables:

- `core.net.http:cacert_path`: Path containing a custom Cacert file. When multiple certificates are necessary for different remotes, it is possible to aggregate them, for example adding your own `my-ca.crt` certificate:

  ```
  sudo cp my-ca.crt /usr/local/share/ca-certificates/my-ca.crt
  sudo update-ca-certificates
  ```

  Then, the certificate storage can be defined with `core.net.http:cacert_path=/etc/ssl/certs/ca-certificates.crt`. The `cacert_path` Conan configuration is forwarded to the `python-requests verify` argument, see Python-requests SSL certificates. That means that if the `REQUESTS_CA_BUNDLE` environment variable is defined, it might be taken into account too.

- `core.net.http:trust_store`: (Experimental) If `True`, additionally verify HTTPS certificates against the OS-native trust store (the certificate store managed by your operating system) using the truststore package. It is used *together with*, not instead of, the `certifi` CA bundle that Conan uses by default, so a certificate is accepted if it validates against either one.

  Listing 36: **[CONAN_HOME]/global.conf**

  ```
  core.net.http:trust_store=True
  ```

  This requires Python >= 3.10, and it is not compatible with `core.net.http:cacert_path`: Conan raises an error if both are enabled at the same time, or if this conf is enabled with an older Python version. The `truststore` package itself is installed automatically as a Conan dependency when installing Conan with `pip` on Python >= 3.10; if it is missing, Conan raises an error.

- `core.net.http:client_cert`: Path or tuple of files containing a client certificate (and the key). See more details in Python requests and Client Side Certificates

  For instance:

Listing 37: **[CONAN_HOME]/global.conf**

```
core.net.http:client_cert=('/path/client.cert', '/path/client.key')
```

- `tools.files.download:verify`: Setting `tools.files.download:verify=False` constitutes a security risk if enabled, as it disables certificate validation. Do not use it unless you understand the implications (And even then, properly scoping the conf to only the required recipes is a good idea) or if you are using it for development purposes

See also:

If you need to use both a corporate remote (with a private CA) and a public remote like ConanCenter, see *Using Conan with both corporate and public remotes (SSL certificates)* for step-by-step instructions on creating a combined CA bundle.

Proxies

There are 3 `confs` that can define proxies information:

```
$ conan config list proxies
core.net.http:clean_system_proxy: If defined, the proxies system env-vars will be␣
↪discarded
core.net.http:no_proxy_match: List of urls to skip from proxies configuration
core.net.http:proxies: Dictionary containing the proxy configuration
```

The `core.net.http:proxies` dictionary is passed to the underlying `python-requests` library, to the "proxies" argument as described in the python-requests documentation

The `core.net:no_proxy_match` is a list of URL patterns, like:

```
core.net.http:no_proxy_match = ["http://someurl.com/*"]
```

for URLs to be excluded from the `proxies` configuration. That means that all URLs that are referenced that matches any of those patterns will not receive the `proxies` definition. Note the * in the pattern is necessary for the match.

If `core.net.http:clean_system_proxy` is `True`, then the environment variables `"http_proxy"`, `"https_proxy"`, `"ftp_proxy"`, `"all_proxy"`, `"no_proxy"`, will be temporary removed from the environment, so they are not taken into account when resolving proxies.

MSYS2 subsystem environments

The `tools.microsoft.bash:subsystem` configuration accepts `msys2` as a generic value, but also supports explicit MSYS2 environment names using the `msys2-<environment>` syntax. This allows selecting a specific MSYS2 environment (and its associated `MSYSTEM` variable) rather than relying on the architecture-based default.

This specific `MSYSTEM` variable can become important for recipes using autotools in Windows with `win_bash=True`, and compiling with different `msys2` compilers. For other build systems such as CMake, explicitly defining the path to the compiler used could be enough.

Supported values:

- `msys2` (default): uses architecture-based MSYSTEM (`MINGW64` for x86_64, `MINGW32` for x86).

- `msys2-ucrt64`: selects the UCRT64 environment (`UCRT64`).
- `msys2-clang64`: selects the CLANG64 environment (`CLANG64`).
- `msys2-clangarm64`: selects the CLANGARM64 environment.
- `msys2-mingw64`: selects the MINGW64 environment explicitly.
- `msys2-mingw32`: selects the MINGW32 environment explicitly.
- `msys2-clang32`: selects the CLANG32 environment.

Example profile:

Listing 38: *windows_ucrt64*

```
[settings]
os=Windows
arch=x86_64
...

[conf]
tools.microsoft.bash:subsystem=msys2-ucrt64
tools.microsoft.bash:path=C:\msys64\usr\bin\bash.exe
```

The `msys2-<environment>` format sets the `MSYSTEM` environment variable accordingly when running bash commands, enabling proper activation of the selected MSYS2 toolchain.

Recall that the `tools.microsoft.bash:subsystem` conf doesn't have any effect on the `package_id` computation. If users want to model binaries for the different environments, it is necessary to define accordingly the `os.subsystem` setting. At the moment it only accepts `msys2` value, so users wanting more control can extend it via the custom `settings_user.yml` file.

Storage configurations

core.cache:storage_path

Absolute path to a folder where the Conan packages and the database of the packages will be stored. This folder will be the heaviest Conan storage folder, as it stores the binary packages downloaded or created.

Listing 39: *global.conf*

```
core.cache:storage_path = C:\Users\danielm\my_conan_storage_folder
```

Default value: `<CONAN_HOME>/p`

core.download:download_cache

Absolute path to a folder where the Conan packages will be stored *compressed*. This is useful to avoid recurrent downloads of the same packages, especially in CI.

Listing 40: *global.conf*

```
core.download:download_cache = C:\Users\danielm\my_download_cache
```

Default value: Not defined.

Note: When this conf is defined and an artifact is uploaded to a remote, Conan will also populate the download cache of the client with the same artifact.

UX confs

Skip warnings

There are several warnings that Conan outputs in certain cases which can be omitted via the `core:skip_warnings` conf, by adding the warning tag to its value.

Those warnings are:

- `deprecated`: Messages for deprecated features such as legacy generators
- `network`: Messages related to network issues, such as retries

Parallel download

By default the download and unzip of pre-compiled package binaries from remote servers will happen in parallel, defaulting to the number of cpu-cores. The configuration `core.download:parallel=<int-number>` can change this behavior. If `core.download:parallel=0`, then the behavior will be to not use parallelism and do a sequential download and unzip of precompiled package binaries. This `core.download:parallel` configuration also affects the `conan download` command, but for that command the default at the moment is not to use parallelism, but sequential download.

Environment deactivation functions

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

When setting the configuration `tools.env:deactivation_mode` to `function` in your profile or in `global.conf`, the deactivation scripts will no longer be generated.

Instead, an *in-memory* deactivation function will be available in the current shell session as soon as you activate the conan environment.

Moving from the classical Conan workflow:

Bash

PowerShell

Batchfile

```
$ source conanbuild.sh
$ ...
$ source deactivate_conanbuild.sh
```

```
$ .\conanbuild.ps1
$ ...
$ .\deactivate_conanbuild.ps1
```

```
$ .\conanbuild.bat
$ ...
$ .\deactivate_conanbuild.bat
```

To the new workflow,

Bash

PowerShell

Batchfile

```
$ source conanbuild.sh
$ ...
$ deactivate_conanbuild # from anywhere in the shell
```

```
$ .\conanbuild.ps1
$ ...
$ deactivate_conanbuild # from anywhere in the shell
```

```
$ .\conanbuild.bat
$ ...
$ deactivate_conanbuild # from anywhere in the shell
```

By executing this function, the environment will be restored and the function will no longer be available in the current shell session. This behavior emulates the well known `virtualenv` Python tool.

Listing 41: *global.conf*

```
tools.env:deactivation_mode=function
```

Default value: None.

9.6.5 profiles

Introduction to profiles

Conan profiles allow users to set a complete configuration set for **settings**, **options**, **environment variables** (for build time and runtime context), **tool requirements**, and **configuration variables** in a file.

They have this structure:

```
[settings]
arch=x86_64
build_type=Release
```

(continues on next page)

(continued from previous page)

```
os=Macos

[options]
mylib/*:shared=True

[tool_requires]
tool1/0.1@user/channel
*: tool4/0.1@user/channel

[buildenv]
VAR1=value

[runenv]
EnvironmentVar1=My Value

[conf]
tools.build:jobs=2

[replace_requires]
zlib/1.2.12: zlib/[*]

[replace_tool_requires]
7zip/*: 7zip/system

[platform_requires]
dlib/1.3.22

[platform_tool_requires]
cmake/3.24.2
```

Profiles can be created with the `detect` option in *conan profile* command, and edited later. If you don't specify a *name*, the command will create the `default` profile:

Listing 42: *Creating the Conan default profile*

```
$ conan profile detect
apple-clang>=13, using the major as version
Detected profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos

WARN: This profile is a guess of your environment, please check it.
WARN: Defaulted to cppstd='gnu17' for apple-clang.
WARN: The output of this command is not guaranteed to be stable and can change in future␣
↪Conan versions.
WARN: Use your own profile files for stability.
Saving detected profile to [CONAN_HOME]/profiles/default
```

Note: A note about the detected C++ standard by Conan

Conan will always set the default C++ standard as the one that the detected compiler version uses by default, except for the case of macOS using apple-clang. In this case, for apple-clang>=11, it sets `compiler.cppstd=gnu17`. If you want to use a different C++ standard, you can edit the default profile file directly.

Listing 43: *Creating another profile: myprofile*

```
$ conan profile detect --name myprofile
Found apple-clang 14.0
apple-clang>=13, using the major as version
Detected profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos

WARN: This profile is a guess of your environment, please check it.
WARN: Defaulted to cppstd='gnu17' for apple-clang.
WARN: The output of this command is not guaranteed to be stable and can change in future␣
↪Conan versions.
WARN: Use your own profile files for stability.
Saving detected profile to [CONAN_HOME]/profiles/myprofile
```

Profile files can be used with `-pr/--profile` option in many commands like **`conan install`** or **`conan create`** commands. If you don't specify any profile at all, the `default` profile will be always used:

Listing 44: Using the *default* profile

```
$ conan create .
```

Listing 45: Using a *myprofile* profile

```
$ conan create . -pr=myprofile
```

Profile files can have comments, but only in new lines, with the first character being #, trailing comments are not supported:

```
[settings]
# Valid comment
arch=x86_64
build_type=Release  # INVALID comment, do not use
```

Using profiles

Profiles can be located in different folders:

```
$ conan install . -pr /abs/path/to/myprofile   # abs path
$ conan install . -pr ./relpath/to/myprofile   # resolved to current dir
$ conan install . -pr ../relpath/to/myprofile  # resolved to relative dir
$ conan install . -pr myprofile  # resolved to [CONAN_HOME]/profiles/myprofile
```

Listing existing profiles in the *profiles* folder can be done like this:

```
$ conan profile list
Profiles found in the cache:
default
myprofile1
myprofile2
...
```

You can also show the profile's content per context:

```
$ conan profile show -pr myprofile
Host profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos

Build profile:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos
```

See also:

- Manage your profiles and share them using *conan config install*.
- Check the command and its sub-commands of *conan profile*.

Profile sections

These are the available sections in profiles:

[settings]

List of settings available from *settings.yml*:

Listing 46: *myprofile*

```
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=14
os=Macos
```

See also:

- This section allows to use patterns to limit which packages are affected by the setting. See *this section* for more details.

[options]

List of options available from your recipe and its dependencies:

Listing 47: *myprofile*

```
[options]
mypkg/*:my_pkg_option=True
*:shared=True
```

See also:

- This section allows to use patterns to limit which packages are affected by the options. See *this section* for more details.

[tool_requires]

List of `tool_requires` required by your recipe or its dependencies:

Listing 48: *myprofile*

```
[tool_requires]
cmake/3.25.2
```

See also:

- This section allows to use patterns to limit which packages are affected by the tool requirement. See *this section* for more details.
- Read more about tool requires in this section: *Using build tools as Conan packages*.

[system_tools] (DEPRECATED)

Note: This section is **deprecated** and has been replaced by *[platform_requires]* and *[platform_tool_requires]* sections.

[buildenv]

List of environment variables that will be injected to the environment every time the ConanFile `run(cmd, env="conanbuild")` method is invoked (build time context is automatically run by *VirtualBuildEnv*).

Besides that, it is able to apply some additional operators to each variable declared when you're composing profiles or even local variables:

- `+=` == `append`: appends values at the end of the existing value.
- `=+` == `prepend`: puts values at the beginning of the existing value.
- `=!` == `unset`: gets rid of any variable value.

Note that it is different to define an empty variable, like `MyVar1=`, which defines it with a value of an empty string, that requesting it to be explicitly unset with the `MyVar1=!` syntax.

Another essential point to mention is the possibility of defining variables as *PATH* ones by simply putting `(path)` as the prefix of the variable. It is useful to automatically get the append/prepend of the *PATH* in different systems (Windows uses `;` as separation, and UNIX `:`).

Listing 49: *myprofile*

```
[buildenv]
# Define a variable "MyVar1"
MyVar1=My Value; other

# Append another value to "MyVar1"
MyVar1+=MyValue12

# Define a variable with an empty string value
MyVar2=

# Define a PATH variable "MyPath1"
MyPath1=(path)/some/path11

# Prepend another PATH to "MyPath1"
MyPath1=+(path)/other path/path12

# Unset the variable "MyPath2"
MyPath2=!
```

Then, the result of applying this profile is:

- `MyVar1`: `My Value; other MyValue12`
- `MyVar2`: An empty string value
- **`MyPath1:`**
  - Unix: `/other path/path12:/some/path11`
  - Windows: `/other path/path12;/some/path11`
- `MyPath2`: if system environment had defined `MyPath2`, this will be unset by Conan

Warning: Note that `[buildenv]` and `[runenv]` environment variables definition keeps user blank spaces in values. `MYVAR = MyValue` will produce a `" MyValue"` value, which will be different than `MYVAR=MyValue` that will produce `"MyValue"`. avoid using extra spaces around = in profiles, use the syntax shown above.

See also:

- This section allows to use patterns to limit which packages are affected by the buildenv. See *this section* for more details.

For example:

```
[buildenv]
MyPath=MyValue
mypkg/*:MyPath=MyOtherValue
```

This will result in `MyPath=MyValue` for all the packages, and `MyPath=MyOtherValue` only for the `mypkg` package.

[runenv]

List of environment variables that will be injected to the environment every time the ConanFile `run(cmd, env="conanrun")` method is invoked (runtime context is automatically run by *VirtualRunEnv*).

All the operators/patterns explained for *[buildenv]* applies to this one in the same way:

Listing 50: *myprofile*

```
[runenv]
MyVar1=My Value; other
MyVar1+=MyValue12
MyPath1=(path)/some/path11
MyPath1=+(path)/other path/path12
MyPath1=!
```

See also:

- This section allows to use patterns to limit which packages are affected by the runenv. See *this section* for more details.

[conf]

Note: It's recommended to have previously read the *global.conf* section.

List of user/tools configurations:

Listing 51: *myprofile*

```
[conf]
tools.build:verbosity=verbose
tools.microsoft.msbuild:max_cpu_count=2
tools.microsoft.msbuild:vs_version = 16
tools.build:jobs=10
# User conf variable
user.confvar:something=False
```

Recall some hints about configuration scope and naming:

- `core.xxx` configuration can only be defined in `global.conf` file, but not in profiles
- `tools.yyy` and `user.zzz` can be defined in `global.conf` and they will affect both the "build" and the "host" context. But configurations defined in a profile `[conf]` will only affect the respective "build" or "host" context of the profile, not both.

They can also be used in *global.conf*, but **profiles values will have priority over globally defined ones in global.conf**, so let's see an example that is a bit more complex, trying different configurations coming from the *global.conf* and another profile *myprofile*:

Listing 52: *global.conf*

```
# Defining several lists
user.myconf.build:ldflags=["--flag1 value1"]
user.myconf.build:cflags=["--flag1 value1"]
```

Listing 53: *myprofile*

```
[settings]
...

[conf]
# Appending values into the existing list
user.myconf.build:ldflags+=["--flag2 value2"]

# Unsetting the existing value (it'd be like we define it as an empty value)
user.myconf.build:cflags=!

# Prepending values into the existing list
user.myconf.build:ldflags=+["--prefix prefix-value"]
```

Running, for instance, **conan install . -pr myprofile**, the configuration output will be something like:

```
...
Configuration:
[settings]
[options]
[tool_requires]
[conf]
user.myconf.build:cflags=!
user.myconf.build:ldflags=['--prefix prefix-value', '--flag1 value1', '--flag2 value2']
...
```

See also:

- This section allows to use patterns to limit which packages are affected by the confs. See *this section* for more details.

[replace_requires]

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This section allows the user to redefine requires of recipes. This can be useful when a package can be changed by a similar one like *zlib* and *zlibng*. It is also useful to solve conflicts, or to replace some dependencies by system alternatives wrapped in another Conan package recipe.

References listed under this section work as a **literal replacement of requires in recipes**, and is done as the very first step before any other processing of recipe requirements, without processing them or checking for conflicts.

As an example, we could define *zlibng* as a replacement for the typical ZLIB

Listing 54: *myprofile*

```
[replace_requires]
zlib/*: zlibng/*
```

Using the * pattern for the `zlibng` reference means that `zlib` will be replaced by the exact same version of `zlibng`.

Other examples are:

Listing 55: *myprofile*

```
[replace_requires]
# To override dep/[>=1.0 <2] in recipes to a specific version dep/1.1)
dep/*: dep/1.1
# To override a dep/1.3 in recipes to dep/1.3@system
dep/*: dep/*@system
# To override every dep requirement in recipes to a specific version range
dep/*: dep/[>=1 <2]
# To override "dep/1.3@comp/stable" in recipes to the same version with other user but␣
↪same channel
dep/*@*/*: dep/*@system/*
# To replace exact reference in recipes by the same one in the system
dep/1.1: dep/1.1@system
# To replace dep/[>=1.0 <2]@comp version range in recipes by 1.1 version in stable␣
↪channel
dep/1.1@*: dep/1.1@*/stable
```

When composing multiple profiles (e.g., `-pr=base -pr=override`), it is possible to unset replacement rules defined in earlier profiles using the `!` marker as the replacement target:

Listing 56: *override_profile*

```
[replace_requires]
# Remove a specific replacement rule inherited from a base profile
# The "zlib/*" expression must match exactly the previous rule, it will not invalidate a
↪"zlib/1.3.1" rule.
zlib/*: !
# Clear all replacement rules
*: !
```

Note: Best practices

- Please make rational use of this feature. It is not a versioning mechanism and is not intended to replace actual requires in recipes.
- The usage of this feature is intended for **temporarily** solving conflicts or replacing a specific dependency by a system one in some cross-build scenarios.

Important: `[replace_requires]` does **not** apply to requires that are explicitly specified via command-line arguments (e.g., `conan install --requires=dep/1.0` or `conan create`). CLI-specified requirements have higher priority and are never replaced. Transitive dependencies of CLI-specified requires can still be replaced.

[replace_tool_requires]

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Same usage as the *replace_requires* section but in this case for *tool_requires*.

Listing 57: *myprofile*

```
[replace_tool_requires]
cmake/*: cmake/3.25.2
```

In this case, whatever version of `cmake` declared in recipes, will be replaced by the reference *cmake/3.25.2*.

Replacement rules can be unset in profile composition using `!` as the replacement target:

Listing 58: *override_profile*

```
[replace_tool_requires]
# Remove a specific rule from a base profile
# The "cmake/*" expression must match exactly the previous rule, it will not invalidate␣
↪a "cmake/3.30" rule.
cmake/*: !
# Clear all tool replacement rules
*: !
```

Note:

- This section should be added to the profile whose context is the one that requires the tool, i.e., if the tool is required in the host context, then it should be added to the host profile, so that the requirement itself can be replaced. For example, if a `zlib` recipe in the host context has a `tool_requires("cmake/xxx")`, a `replace_tool_requires` in the **host profile** will replace it.
- If what you want to replace are transitive dependencies of the tools that live inside `tool_requires` packages, those live in the **build context**. To replace them, you must add the replacements to the **build profile**. Both `[replace_requires]` and `[replace_tool_requires]` in the build profile will affect the build context in the same way, replacing `requires` and `tool_requires` of the tools themselves.

Important: `[replace_tool_requires]` does **not** apply to requires that are explicitly specified via command-line arguments (e.g., `conan install --tool-requires=dep/1.0` or `conan create`). CLI-specified requirements have higher priority and are never replaced. Transitive dependencies of CLI-specified requires can still be replaced.

[platform_requires]

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This section allows the user to redefine requires of recipes replacing them with platform-provided dependencies, which means that Conan will not try to download the reference or look for it in the cache and will assume that it is installed in your system and ready to be used.

For example, if the `zlib/1.3.1` library is already installed in your system or it is part of your build toolchain and you would like Conan to use it, you could specify so as:

Listing 59: *myprofile*

```
[platform_requires]
zlib/1.3.1
```

Important: In practice, it can be very challenging to achieve a full transparent replacement of a package in the Conan dependency graph by a system or platform installed alternative, because there can be information missing for correctly using it, like dependencies and transitive dependencies, build-system integrations specifics such as the CMake `find_package()` file names or CMake targets, etc.

That means that it might not be possible to achieve such a `[platform_requires]` for regular packages, as opposed to the `[platform_tool_requires]` because for many tools, it is enough that they are just in the system path. But this is not the case for regular libraries.

For these cases, the recommendation is to write a "wrapper" Conan `conanfile.py` recipe that models the platform installed dependency, defines if it has dependencies and provides the necessary information to consume it in its `package_info()` method. Then, use the `[replace_requires]` feature instead.

A recipe revision can also be used in the reference, to help keep track of changes in the system dependency, but it is not mandatory, and Conan will use the default `#platform` revision if not specified.

Listing 60: *myprofile*

```
[platform_requires]
zlib/1.3.1#myrevision
```

This will ensure that lockfiles are able to track changes over this platform dependency, and it will be easier to specify when the system dependency has changed and needs to be re-evaluated.

[platform_tool_requires]

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Same usage as the *platform_requires* section but in this case for *tool_requires* such as *cmake*, *meson*...

As an example, let's say you have already installed `cmake==3.24.2` in your system:

```
$ cmake --version
cmake version 3.24.2

CMake suite maintained and supported by Kitware (kitware.com/cmake).
```

And you have in your recipe (or the transitive dependencies) declared a **tool_requires**, i.e., something like this:

Listing 61: **conanfile.py**

```
from conan import ConanFile

class PkgConan(ConanFile):
    name = "pkg"
    version = "2.0"
    # ....

    # Exact version
    def build_requirements(self):
        self.tool_requires("cmake/3.24.2")

    # Or even version ranges
    def build_requirements(self):
        self.tool_requires("cmake/[>=3.20.0]")
```

Given this situation, it could make sense to want to use your already installed CMake version, so it's enough to declare it as a `platform_tool_requires` in your profile:

Listing 62: *myprofile*

```
...

[platform_tool_requires]
cmake/3.24.2
```

Whenever you want to create the package, you'll see that build requirement is already satisfied because of the platform tool declaration:

```
$ conan create . -pr myprofile --build=missing
...
-------- Computing dependency graph --------
Graph root
    virtual
Requirements
    pkg/2.0#3488ec5c2829b44387152a6c4b013767 - Cache
Build requirements
    cmake/3.24.2#platform - Platform

-------- Computing necessary packages --------

-------- Computing necessary packages --------
pkg/2.0: Forced build from source
Requirements
    pkg/2.0#3488ec5c2829b44387152a6c4b013767:20496b332552131b67fb99bf425f95f64d0d0818 -␣
↪Build
```

(continues on next page)

(continued from previous page)

```
Build requirements
    cmake/3.24.2#platform - Platform
```

A recipe revision can also be used in the reference, to help keep track of changes in the system dependency, but it is not mandatory, and Conan will use the default `#platform` revision if not specified.

Listing 63: *myprofile*

```
[platform_tool_requires]
cmake/3.24.2#myrevision
```

This will ensure that lockfiles are able to track changes over this platform dependency, and it will be easier to specify when the system dependency has changed and needs to be re-evaluated.

Note:

- If the `platform_tool_requires` declared **does not make a strict match** with the `tool_requires` one (version or version range), then Conan will try to bring them remotely or locally as usual.
- This section should be added to the profile whose context is the one that requires the tool, i.e., if the tool is required in the host context, then it should be added to the host profile, so that the requirement itself can be replaced by the platform one.

Profile rendering

The profiles are rendered as **jinja2** templates by default. When Conan loads a profile, it immediately parses and renders the template, which must result in a standard text profile.

Some of the capabilities of the profile templates are:

- Using the platform information, like obtaining the current OS, is possible because the Python `platform` module is added to the render context:

Listing 64: *profile_vars*

```
[settings]
os = {{ {"Darwin": "Macos"}.get(platform.system(), platform.system()) }}
```

- Reading environment variables can be done because the Python `os` module is added to the render context:

Listing 65: *profile_vars*

```
[settings]
build_type = {{ os.getenv("MY_BUILD_TYPE") }}
```

- Defining your own variables and using them in the profile:

Listing 66: *profile_vars*

```
{% set os = "FreeBSD" %}
{% set clang = "my/path/to/clang" %}

[settings]
```

(continues on next page)

(continued from previous page)

```
os = {{ os }}

[conf]
tools.build:compiler_executables={'c': '{{ clang }}', 'cpp': '{{ clang + '++' }}' }
```

- Joining and defining paths, including referencing the current profile directory. For example, defining a toolchain whose file is located besides the profile can be done. Besides the `os` Python module, the variable `profile_dir` pointing to the current profile folder is added to the context.

Listing 67: *profile_vars*

```
[conf]
tools.cmake.cmaketoolchain:toolchain_file = {{ os.path.join(profile_dir, "toolchain.
↪cmake") }}
```

- Getting settings from a filename, including referencing the current profile name. For example, defining a generic profile which is configured according to its file name. The variable `profile_name` pointing to the current profile file name is added to the context.

Listing 68: *Linux-x86_64-gcc-12*

```
{% set os, arch, compiler, compiler_version = profile_name.split('-') %}
[settings]
os={{ os }}
arch={{ arch }}
compiler={{ compiler }}
compiler.version={{ compiler_version }}
```

If you want to use the original profile file name, instead of the current profile name, for example, if the current profile file is called `common`, and another profile called `windows-arm` was doing an `include(common)`, then the `root_profile_name` variable will be equal to `windows-arm`, even inside the `common` file.

- Executing external commands and using their output. The `subprocess` module is added to the context, so you can use it to execute commands and capture their output. Note that Conan startup times for some commands can be affected if the command takes a long time to execute, so use this feature with caution. For example, to get the version of the installed compiler (But you should use `detect_api.detect_default_compiler()` instead for this case):

Listing 69: *profile_vars*

```
{% set version = subprocess.check_output(['clang++', "-dumpversion"]).strip() %}
[settings]
compiler.version={{ version }}
```

- Branching based on the context that the profile is being rendered for. The `context` variable is injected and can take the values `host`, `build` or be `None`. For example, you can define different settings for the host and build contexts without having to create two different profiles:

Listing 70: *profile_vars*

```
[settings]
os=Linux
compiler=gcc
compiler.version=12
```

(continues on next page)

(continued from previous page)

```
{% if context == "host" %}
compiler.cppstd=gnu17
{% else %}
compiler.cppstd=gnu20
{% endif %}
```

- Including or importing other files from `profiles` folder:

Listing 71: *profile_vars*

```
{% set a = "Debug" %}
```

Listing 72: *myprofile*

```
{% import "profile_vars" as vars %}
[settings]
build_type = {{ vars.a }}
```

When including or importing other files using relative paths, the Jinja renderer uses the base path of the current profile file as the first location to look for. If this search fails, the Jinja renderer will also start looking in the Conan home profiles folder (typically in `<userhome>/.conan2/profiles`).

- Any other feature supported by *jinja2* is possible: for loops, if-else, etc. This would be useful to define custom per-package settings or options for multiple packages in a large dependency graph.

As a summary, this is the rendering context for profile files (all the items Conan injects):

- `platform`, `os`, `subprocess`: The Python `platform`, `os` and `subprocess` modules
- `profile_dir`: The folder where the current profile file is located
- `profile_name`: The current profile file filename
- `root_profile_name`: The initial profile that was loaded. It is different of the current `profile_name` when there is an `include(otherprofile)`
- `conan_version`: The current Conan version
- `detect_api`: This is a full automatic detection API, see below
- `context`: The current Conan context, can be `host`, `build` or be `None`

Profile Rendering with ``detect_api``

> **Warning: Stability Guarantees**: The detect_api, similar to `conan profile detect`, does not offer strong stability guarantees.
>
> **Usage Recommendations**: The detect_api is not a regular API meant for creating new commands or similar functionalities. While auto-detection can be convenient, it's not the recommended approach for all scenarios. This API is internal to Conan and is only exposed for profile and *global.conf* rendering. It's advised to use it judiciously.

Conan also injects `detect_api` to the jinja rendering context. With it, it's possible to use Conan's auto-detection capabilities directly within Jinja profile templates. This provides a way to dynamically determine certain settings based on the environment.

`detect_api` can be invoked within the Jinja template of a profile. For instance, to detect the operating system and architecture, you can use:

```
[settings]
os={{detect_api.detect_os()}}
arch={{detect_api.detect_arch()}}
```

Similarly, for more advanced detections like determining the compiler, its version, and the associated runtime, you can use:

```
{% set compiler, version, compiler_exe = detect_api.detect_default_compiler() %}
{% set runtime, _ = detect_api.default_msvc_runtime(compiler) %}
[settings]
compiler={{compiler}}
compiler.version={{detect_api.default_compiler_version(compiler, version)}}
compiler.runtime={{runtime}}
compiler.cppstd={{detect_api.default_cppstd(compiler, version)}}
compiler.libcxx={{detect_api.detect_libcxx(compiler, version, compiler_exe)}}
```

detect_api reference:

- `detect_os()`: returns operating system as a string (e.g., "Windows", "Macos").
- `detect_arch()`: returns system architecture as a string (e.g., "x86_64", "armv8").
- `detect_libc(ldd="/usr/bin/ldd")`: **experimental** returns a tuple with the name (e.g., "gnu", "musl") and version (e.g., "2.39", "1.2.4") of the C library.
- `detect_libcxx(compiler, version, compiler_exe=None)`: returns C++ standard library as a string (e.g., "libstdc++", "libc++").
- `default_msvc_runtime(compiler)`: returns tuple with runtime (e.g., "dynamic") and its version (e.g., "v143").
- `default_cppstd(compiler, compiler_version)`: returns default C++ standard as a string (e.g., "gnu14").
- `detect_default_compiler()`: returns tuple with compiler name (e.g., "gcc"), its version and the executable path.
- `detect_msvc_update(version)`: returns the MSVC update version as a string (e.g., "12" for VS 17.12.1). Note that in Conan profiles, the `compiler.update` setting accepts values from 0 to 10. To convert the result from `detect_msvc_update` into the format required by profiles, you can do something like this:

  Example:

  ```
  ...
  [settings]
  compiler=msvc
  compiler=194 # for msvc toolset starting in 14.40 (VS 17.10)
  # If we are using VS 17.12 we convert 12 to 2 so it's 194 with update 2
  compiler.update = "{{ (detect_api.detect_msvc_update(version) | int) % 10 }}
  ↪"
  ...
  ```

- `default_msvc_ide_version(version)`: returns MSVC IDE version as a string (e.g., "17").
- `default_compiler_version(compiler, version)`: returns the default version that Conan uses in profiles, typically dropping some of the minor or patch digits, that do not affect binary compatibility.
- `detect_gcc_compiler(compiler_exe="gcc")`: Return the tuple ('gcc', version, executable) for `gcc`
- `detect_intel_compiler(compiler_exe="icx")`: Return the tuple ('intel-cc', version, executable) for `intel-cc`

- `detect_suncc_compiler(compiler_exe="cc")`: Return the tuple ('sun-cc', version, executable) for `sun-cc`
- `detect_clang_compiler(compiler_exe="clang")`: Return the tuple ('clang'|'apple-clang', version, executable) for `clang` or `apple-clang`.
- `detect_msvc_compiler()`: Detect the compiler ('msvc', version, None) default version of the latest VS IDE installed
- `detect_cl_compiler(compiler_exe="cl")`: Detect the compiler ('msvc', version, executable) for the `cl.exe` compiler
- `detect_emcc_compiler(compiler_exe="emcc")`: Return the tuple ('emcc', version, executable) for the `emcc` Emscripten compiler
- `detect_sdk_version(sdk)`: Detect the Apple SDK version (`None` for non Apple platforms), for the given `sdk`. Equivalent to `xcrun -sdk {sdk} --show-sdk-version`

Profile patterns

Profiles (and everywhere where a setting or option is defined) also support patterns definition, so you can override some settings, configuration variables, etc. for some specific packages:

Listing 73: *zlib_clang_profile*

```
[settings]
# Only for zlib
zlib/*:compiler=clang
zlib/*:compiler.version=3.5
zlib/*:compiler.libcxx=libstdc++11

# For the all the dependency tree
compiler=gcc
compiler.version=4.9
compiler.libcxx=libstdc++11

[options]
# shared=True option only for zlib package
zlib/*:shared=True

[buildenv]
# For the all the dependency tree
*:MYVAR=my_var

[conf]
# Only for zlib
zlib/*:tools.build:compiler_executables={'c': '/usr/bin/clang', 'cpp': '/usr/bin/clang++
↪'}
```

Your build tool will locate **clang** compiler only for the **zlib** package and **gcc** (default one) for the rest of your dependency tree.

Important: Putting only `zlib:` is deprecated behaviour and won't work, you have to always put a pattern-like expression, e.g., `zlib*:`, `zlib/*:`, `zlib/1.*:`, etc.

They accept patterns too, like `-s *@myuser/*`, which means that packages that have the username "myuser" will use clang 3.5 as compiler, and gcc otherwise:

Listing 74: *myprofile*

```
[settings]
*@myuser/*:compiler=clang
*@myuser/*:compiler.version=3.5
*@myuser/*:compiler.libcxx=libstdc++11
compiler=gcc
compiler.version=4.9
compiler.libcxx=libstdc++11
```

Also `&` can be specified as the package name. It will apply only to the consumer conanfile (.py or .txt). This is a special case because the consumer conanfile might not declare a `name` so it would be impossible to reference it.

Listing 75: *myprofile*

```
[settings]
&:compiler=gcc
&:compiler.version=4.9
&:compiler.libcxx=libstdc++11
```

Partial matches are also supported, so you can define a pattern like `zlib*` to match all the zlib like libraries, so it will match everything starting with zlib, like `zlib`, `zlibng`, `zlib/1.2.8@user/channel`, etc.

Listing 76: *myprofile*

```
[settings]
zlib*:compiler=clang
zlib*:compiler.version=3.5
zlib*:compiler.libcxx=libstdc++11
```

If a pattern begins with `!`, it's considered an exclusion pattern, meaning that it will match everything except the specified pattern. This also works for the consumer `&` pattern mentioned above.

Care must be taken when using exclusion patterns, as they can lead to unexpected results if not used carefully.

For example, to define a `shared=True` option for all packages except to *zlib*, you can use:

Listing 77: *myprofile*

```
[options]
!zlib/*:shared=True
```

From Conan 2.27 it is also possible to do an exclusion of multiple patterns, with the `!(<pattern1>|<pattern2>)` **experimental** syntax, exclusively for `[tool_requires]` (it will not work in other sections), like:

Listing 78: *myprofile*

```
[tool_requires]
!(gcc/*|ninja/*): cmake/3.20
```

That means that all packages except `gcc` and `ninja` will get a tool-requires to `cmake`. This feature is intended to avoid circular dependencies in the build-context (try to avoid abusing it for other use cases), that is, it should be mostly in the "build" profile, but likely not in the "host" profile.

Note that for the msvc compiler, the compiler.runtime_type setting is automatically initialized from the build_type setting value by the profile.py plugin, as it is a good default for the vast majority of cases. That means that a user defining -s build_type=MinSizeRel will by default get a compiler.runtime_type=Release value for msvc compiler. However, this is not the case for package specific settings, and the profile.py plugin won't automatically define the compiler.runtime_type for msvc, so doing something like mydep/*:build_type=<build_type> doesn't automatically define mydep/*:compiler.runtime_type.

In some circumstances it might be desired to undefine some specific package settings, assuming that such a setting allows to be undefined. For this case, the syntax ~ can be used as:

Listing 79: myprofile

```
[settings]
build_type=Release
mypkg/*:build_type=~
```

This syntax will define build_type=Release for all packages, except for mypkg (any version), that will leave build_type undefined, as it was never assigned a value. In recipe code that means that self.settings.build_type will be None.

Profile includes

You can include other profile files using the include() statement. The path can be relative to the current profile, absolute, or a profile name from the default profile location in the local cache.

The include() statement has to be at the top of the profile file:

Listing 80: *gcc_49*

```
[settings]
compiler=gcc
compiler.version=4.9
compiler.libcxx=libstdc++11
```

Listing 81: *myprofile*

```
include(gcc_49)

[settings]
zlib/*:compiler=clang
zlib/*:compiler.version=3.5
zlib/*:compiler.libcxx=libstdc++11
```

Note: Cache profiles have more priority than the ones in the current working directory, so if you have a profile named myprofile in the cache, it will be used instead of the one in the current working directory.

To use the profile in the current working directory, you can use:

- -pr ./myprofile option in the command line or
- include(./myprofile) in the profile file itself.

The final result of using *myprofile* is:

Listing 82: *myprofile (virtual result)*

```
[settings]
compiler=gcc
compiler.libcxx=libstdc++11
compiler.version=4.9
zlib/*:compiler=clang
zlib/*:compiler.libcxx=libstdc++11
zlib/*:compiler.version=3.5
```

See also:

- *How to compose two or more profiles*

9.6.6 settings.yml

This configuration file is located in the Conan user home, i.e., `[CONAN_HOME]/settings.yml`. It looks like this:

```
# This file was generated by Conan. Remove this comment if you edit this file or Conan
# will destroy your changes.
os:
    Windows:
        subsystem: [null, cygwin, msys, msys2, wsl]
    WindowsStore:
        version: ["8.1", "10.0"]
    WindowsCE:
        platform: [ANY]
        version: ["5.0", "6.0", "7.0", "8.0"]
    Linux:
    iOS:
        version: &ios_version
                    ["7.0", "7.1", "8.0", "8.1", "8.2", "8.3", "8.4", "9.0", "9.1", "9.2",
↪ "9.3",
                     "10.0", "10.1", "10.2", "10.3",
                     "11.0", "11.1", "11.2", "11.3", "11.4",
                     "12.0", "12.1", "12.2", "12.3", "12.4", "12.5",
                     "13.0", "13.1", "13.2", "13.3", "13.4", "13.5", "13.6", "13.7",
                     "14.0", "14.1", "14.2", "14.3", "14.4", "14.5", "14.6", "14.7", "14.8
↪",
                     "15.0", "15.1", "15.2", "15.3", "15.4", "15.5", "15.6", "15.7", "15.8
↪",
                     "16.0", "16.1", "16.2", "16.3", "16.4", "16.5", "16.6", "16.7",
                     "17.0", "17.1", "17.2", "17.3", "17.4", "17.5", "17.6", "17.7", "17.8
↪",
                     "18.0", "18.1", "18.2", "18.3", "18.4", "18.5", "18.6", "18.7",
                     "26.0", "26.1", "26.2", "26.3", "26.4", "26.5", "26.6",
                     "27.0"]
        sdk: ["iphoneos", "iphonesimulator"]
        sdk_version: [null, "11.3", "11.4", "12.0", "12.1", "12.2", "12.4",
                           "13.0", "13.1", "13.2", "13.3", "13.4", "13.5", "13.6", "13.7",
                           "14.0", "14.1", "14.2", "14.3", "14.4", "14.5", "15.0", "15.2",
↪"15.4",
```

(continues on next page)

(continued from previous page)

```
                      "15.5", "16.0", "16.1", "16.2", "16.4", "17.0", "17.1", "17.2",
↪"17.4", "17.5",
                      "18.0", "18.1", "18.2", "18.4", "18.5",
                      "26.0", "26.1", "26.2", "26.4", "26.5",
                      "27.0"]
    watchOS:
        version: ["4.0", "4.1", "4.2", "4.3", "5.0", "5.1", "5.2", "5.3", "6.0", "6.1",
↪"6.2", "6.3",
                  "7.0", "7.1", "7.2", "7.3", "7.4", "7.5", "7.6",
                  "8.0", "8.1", "8.3", "8.4", "8.5", "8.6", "8.7", "8.8",
                  "9.0","9.1", "9.2", "9.3", "9.4", "9.5", "9.6",
                  "10.0", "10.1", "10.2", "10.3", "10.4", "10.5", "10.6",
                  "11.0", "11.1", "11.2", "11.3", "11.4", "11.5", "11.6",
                  "26.0", "26.1", "26.2", "26.3", "26.4", "26.5", "26.6",
                  "27.0"]
        sdk: ["watchos", "watchsimulator"]
        sdk_version: [null, "4.3", "5.0", "5.1", "5.2", "5.3", "6.0", "6.1", "6.2",
                      "7.0", "7.1", "7.2", "7.3", "7.4", "8.0", "8.0.1", "8.3", "8.5",
↪"9.0", "9.1",
                      "9.4", "10.0", "10.1", "10.2", "10.4", "10.5",
                      "11.0", "11.1", "11.2", "11.4", "11.5",
                      "26.0", "26.1", "26.2", "26.4", "26.5", "26.6",
                      "27.0"]
    tvOS:
        version: ["11.0", "11.1", "11.2", "11.3", "11.4",
                  "12.0", "12.1", "12.2", "12.3", "12.4",
                  "13.0", "13.2", "13.3", "13.4",
                  "14.0", "14.2", "14.3", "14.4", "14.5", "14.6", "14.7",
                  "15.0", "15.1", "15.2", "15.3", "15.4", "15.5", "15.6",
                  "16.0", "16.1", "16.2", "16.3", "16.4", "16.5", "16.6",
                  "17.0", "17.1", "17.2", "17.3", "17.4", "17.5", "17.6",
                  "18.0", "18.1", "18.2", "18.3", "18.4", "18.5", "18.6",
                  "26.0", "26.1", "26.2", "26.3", "26.4", "26.5", "26.6",
                  "27.0"]
        sdk: ["appletvos", "appletvsimulator"]
        sdk_version: [null, "11.3", "11.4", "12.0", "12.1", "12.2", "12.4",
                      "13.0", "13.2", "13.3", "13.4", "14.0", "14.2", "14.3", "14.4",
↪"14.5", "15.0",
                      "15.2", "15.4", "15.5", "16.0", "16.1", "16.4", "17.0", "17.1",
↪"17.2", "17.4", "17.5",
                      "18.0", "18.1", "18.2", "18.4", "18.5",
                      "26.0", "26.1", "26.2", "26.4", "26.5",
                      "27.0"]
    visionOS:
        version: ["1.0", "1.1", "1.2", "1.3", "2.0", "2.1", "2.2", "2.3", "2.4", "2.5",
↪"2.6",
                  "26.0", "26.1", "26.2", "26.3", "26.4", "26.5", "26.6",
                  "27.0"]
        sdk: ["xros", "xrsimulator"]
        sdk_version: [null, "1.0", "1.1", "1.2", "1.3", "2.0", "2.1", "2.2", "2.4", "2.5
↪",
                      "26.0", "26.1", "26.2", "26.4", "26.5",
```

(continues on next page)

(continued from previous page)

```
                      "27.0"]
    Macos:
        version: [null, "10.6", "10.7", "10.8", "10.9", "10.10", "10.11", "10.12", "10.13
↪", "10.14", "10.15",
                  "11.0", "11.1", "11.2", "11.3", "11.4", "11.5", "11.6", "11.7",
                  "12.0", "12.1", "12.2", "12.3", "12.4", "12.5", "12.6", "12.7",
                  "13.0", "13.1", "13.2", "13.3", "13.4", "13.5", "13.6", "13.7",
                  "14.0", "14.1", "14.2", "14.3", "14.4", "14.5", "14.6", "14.7", "14.8
↪",
                  "15.0", "15.1", "15.2", "15.3", "15.4", "15.5", "15.6", "15.7", "15.8
↪",
                  "26.0", "26.1", "26.2", "26.3", "26.4", "26.5", "26.6", "26.7",
                  "27.0"]
        sdk_version: [null, "10.13", "10.14", "10.15", "11.0", "11.1", "11.2", "11.3",
↪"12.0", "12.1",
                      "12.3", "12.4", "13.0", "13.1", "13.3", "14.0", "14.2", "14.4",
↪"14.5",
                      "15.0", "15.1", "15.2", "15.4", "15.5",
                      "26.0", "26.1", "26.2", "26.4", "26.5", "26.6",
                      "27.0"]
        subsystem:
            null:
            catalyst:
                ios_version: *ios_version
    Android:
        api_level: [ANY]
        ndk_version: [null, ANY]
    FreeBSD:
    SunOS:
    AIX:
    Arduino:
        board: [ANY]
    Emscripten:
    Neutrino:
        version: ["6.4", "6.5", "6.6", "7.0", "7.1", "8.0"]
        variant: [null, "safe"]
    baremetal:
    VxWorks:
        version: ["7"]
        variant: [null, "certified"]
arch: [x86, x86_64, ppc32be, ppc32, ppc64le, ppc64,
       armv4, armv4i, armv5el, armv5hf, armv6, armv7, armv7hf, armv7s, armv7k, armv8,␣
↪armv8_32, armv8.3, arm64ec,
       sparc, sparcv9,
       mips, mips64, avr, s390, s390x, asm.js, wasm, wasm64, sh4le,
       e2k-v2, e2k-v3, e2k-v4, e2k-v5, e2k-v6, e2k-v7,
       riscv64, riscv32,
       loongarch64,
       xtensalx6, xtensalx106, xtensalx7,
       tc131, tc16, tc161, tc162, tc18]
compiler:
    sun-cc:
```

(continues on next page)

(continued from previous page)

```
        version: ["5.10", "5.11", "5.12", "5.13", "5.14", "5.15"]
        threads: [null, posix]
        libcxx: [libCstd, libstdcxx, libstlport, libstdc++]
    gcc:
        version: ["4.1", "4.4", "4.5", "4.6", "4.7", "4.8", "4.9",
                  "5", "5.1", "5.2", "5.3", "5.4", "5.5",
                  "6", "6.1", "6.2", "6.3", "6.4", "6.5",
                  "7", "7.1", "7.2", "7.3", "7.4", "7.5",
                  "8", "8.1", "8.2", "8.3", "8.4", "8.5",
                  "9", "9.1", "9.2", "9.3", "9.4", "9.5",
                  "10", "10.1", "10.2", "10.3", "10.4", "10.5",
                  "11", "11.1", "11.2", "11.3", "11.4", "11.5",
                  "12", "12.1", "12.2", "12.3", "12.4", "12.5",
                  "13", "13.1", "13.2", "13.3", "13.4",
                  "14", "14.1", "14.2", "14.3", "14.4",
                  "15", "15.1", "15.2", "15.3",
                  "16", "16.1", "16.2"]
        libcxx: [libstdc++, libstdc++11]
        threads: [null, posix, win32, mcf]  # Windows MinGW
        exception: [null, dwarf2, sjlj, seh]  # Windows MinGW
        cppstd: [null, 98, gnu98, 11, gnu11, 14, gnu14, 17, gnu17, 20, gnu20, 23, gnu23,␣
↪26, gnu26]
        cstd: [null, 99, gnu99, 11, gnu11, 17, gnu17, 23, gnu23]
    msvc:
        version: [170, 180, 190, 191, 192, 193, 194, 195]
        update: [null, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
        runtime: [static, dynamic]
        runtime_type: [Debug, Release]
        cppstd: [null, 14, 17, 20, 23]
        toolset: [null, v110_xp, v120_xp, v140_xp, v141_xp]
        cstd: [null, 11, 17]
    clang:
        version: ["3.3", "3.4", "3.5", "3.6", "3.7", "3.8", "3.9", "4.0",
                  "5.0", "6.0", "7.0", "7.1",
                  "8", "9", "10", "11", "12", "13", "14", "15", "16", "17",
                  "18", "19", "20", "21", "22", "23"]
        libcxx: [null, libstdc++, libstdc++11, libc++, c++_shared, c++_static]
        cppstd: [null, 98, gnu98, 11, gnu11, 14, gnu14, 17, gnu17, 20, gnu20, 23, gnu23,␣
↪26, gnu26]
        runtime: [null, static, dynamic]
        runtime_type: [null, Debug, Release]
        runtime_version: [null, v140, v141, v142, v143, v144, v145]
        cstd: [null, 99, gnu99, 11, gnu11, 17, gnu17, 23, gnu23]
    apple-clang:
        version: ["5.0", "5.1", "6.0", "6.1", "7.0", "7.3", "8.0", "8.1", "9.0", "9.1",
                  "10.0", "11.0", "12.0", "13", "13.0", "13.1", "14", "14.0", "15", "15.0
↪",
                  "16", "16.0", "17", "17.0", "21", "21.0"]
        libcxx: [libstdc++, libc++]
        cppstd: [null, 98, gnu98, 11, gnu11, 14, gnu14, 17, gnu17, 20, gnu20, 23, gnu23,␣
↪26, gnu26]
        cstd: [null, 99, gnu99, 11, gnu11, 17, gnu17, 23, gnu23]
```

(continues on next page)

(continued from previous page)

```
    intel-cc:
        version: ["2021.1", "2021.2", "2021.3", "2021.4", "2022.1", "2022.2",
                  "2022.3", "2023.0", "2023.1", "2023.2", "2024.0", "2024.1",
                  "2025.0", "2025.1", "2025.2", "2025.3",
                  "2026.0", "2026.1"]
        update: [null, ANY]
        mode: ["icx", "classic", "dpcpp"]
        libcxx: [null, libstdc++, libstdc++11, libc++]
        cppstd: [null, 98, gnu98, "03", gnu03, 11, gnu11, 14, gnu14, 17, gnu17, 20,␣
↪gnu20, 23, gnu23]
        runtime: [null, static, dynamic]
        runtime_type: [null, Debug, Release]
    qcc:
        version: ["4.4", "5.4", "8.3", "12.2"]
        libcxx: [cxx, gpp, cpp, cpp-ne, accp, acpp-ne, ecpp, ecpp-ne]
        cppstd: [null, 98, gnu98, 11, gnu11, 14, gnu14, 17, gnu17]
    mcst-lcc:
        version: ["1.19", "1.20", "1.21", "1.22", "1.23", "1.24", "1.25"]
        libcxx: [libstdc++, libstdc++11]
        cppstd: [null, 98, gnu98, 11, gnu11, 14, gnu14, 17, gnu17, 20, gnu20, 23, gnu23]
    emcc:
        # From https://github.com/emscripten-core/emscripten/blob/main/ChangeLog.md
        # There is no ABI compatibility guarantee between versions
        version: [ANY]
        libcxx: [null, libstdc++, libstdc++11, libc++]
        threads: [null, posix, wasm_workers]
        cppstd: [null, 98, gnu98, 11, gnu11, 14, gnu14, 17, gnu17, 20, gnu20, 23, gnu23,␣
↪26, gnu26]
        cstd: [null, 99, gnu99, 11, gnu11, 17, gnu17, 23, gnu23]

build_type: [null, Debug, Release, RelWithDebInfo, MinSizeRel]
```

As you can see, the possible values of `settings` are defined in the same file. This is done to ensure matching naming and spelling as well as defining a common settings model among users and the OSS community. Some general information about settings:

- If a setting is allowed to be set to any value, you can use `ANY`.
- If a setting is allowed to be set to any value or it can also be unset, you can use `[null, ANY]`.

However, this configuration file can be modified to any needs, including new settings or sub-settings and their values. If you want to distribute an unified *settings.yml* file you can use the *conan config install command*.

See also:

- *Conan packages binary compatibility: the package ID*
- *settings*

Operating systems

`baremetal` operating system is a convention meaning that the binaries run directly on the hardware, without an operating system or equivalent layer. This is to differentiate to the `null` value, which is associated to the "this value is not defined" semantics. `baremetal` is a common name convention for embedded microprocessors and microcontrollers' code. It is expected that users might customize the space inside the `baremetal` setting with further subsettings to specify their specific hardware platforms, boards, families, etc. At the moment the `os=baremetal` value is still not used by Conan builtin toolchains and helpers, but it is expected that they can evolve and start using it.

Compilers

Some notes about different compilers:

msvc

The `msvc` compiler setting uses the actual `cl.exe` compiler version, that is 190 (19.0), 191 (19.1), etc, instead of the Visual Studio IDE version(15, 16, etc).

When using the `msvc` compiler, the Visual Studio toolset version (the actual `vcvars` activation and `MSBuild` location) will be defined by the default provided by that compiler version:

- `msvc` compiler version '190': Visual Studio 14 2015 (toolset v140)
- `msvc` compiler version '191': Visual Studio 15 2017 (toolset v141)
- `msvc` compiler version '192': Visual Studio 16 2019 (toolset v142)
- `msvc` compiler version '193': Visual Studio 17 2022 (toolset v143, compiler versions up to 19.39, toolset version 14.3X)
- `msvc` compiler version '194': Visual Studio 17 2022 (toolset v143, compiler versions from 19.40, toolset version 14.4X, Visual Studio update 17.10)
- `msvc` compiler version '195': Visual Studio 18 2026 (toolset v145, compiler versions from 19.50, toolset version 14.5X)

Note that both `compiler.version=193` and `compiler.version=194` map to the `v143` toolset, but to different toolset versions `14.3X` and `14.4X`, due to the versioning scheme change done from Visual Studio update 17.10 that introduced compiler version 19.40 and toolset version 14.40 while keeping the toolset `v143` nomenclature. Visual Studio 18 jumped to `v145` toolset, so the `v144` doesn't really exist.

If you want to explicitly force a specific Visual Studio IDE version, you can do it with the `tools.microsoft.msbuild:vs_version` configuration:

```
[settings]
compiler=msvc
compiler.version=190

[conf]
tools.microsoft.msbuild:vs_version = 16
```

In this case, the `vcvars` will activate the Visual Studio 16 installation, but the `190` compiler version will still be used because the necessary `toolset=v140` will be set.

The settings define the last digit `update: [null, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`, which by default is `null` and means that Conan assumes binary compatibility for the compiler patches, which works in general for the Microsoft compilers. For cases where finer control is desired, you can just add the `update` part to your profiles:

```
[settings]
compiler=msvc
compiler.version=191
compiler.update=3
```

This will be equivalent to the full version `1913 (19.13)`. If even further details are desired, you could even add your own digits to the `update` subsetting in `settings.yml`.

Note:

- When using `cmake`, it is necessary to have a modern version to ensure CMake correctly finds and processes the right toolset version depending on the update.
- Be aware that even when installing the latest compiler update with the MS VS installer, this might not be the default one that CMake uses when it is not specified. CMake takes the information from the files `Microsoft.VCToolsVersion.v143.default` and `Microsoft.VCToolsVersion.v143.default.props` located in `C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Auxiliary\Build`. Check the output logs to verify the toolset and compiler version that is being used.

intel-cc

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This compiler is aimed to handle the new Intel oneAPI DPC++/C++/Classic compilers. Instead of having *n* different compilers, you have 3 different **modes** of working:

- `icx` for Intel oneAPI C++.
- `dpcpp` for Intel oneAPI DPC++ (deprecated by Intel, prefer `icx` with `sycl` linkage, see *SYCL docs*)
- `classic` for Intel C++ Classic ones.

Besides that, Intel releases some versions with revisions numbers so the `update` field is supposed to be any possible minor number for the Intel compiler version used, e.g, `compiler.version=2026.0` and `compiler.update=1` mean Intel version is `2026.0.1`. Supported versions include the `2021.x`, `2022.x`, `2023.x`, `2024.x`, `2025.x`, and `2026.x` families. Conan does not use the `compiler.update` number in the toolchain definitions, it is just a declaration of its value when users want that specific udpate to be part of the packages `package_id` and have different binaries for different compiler updates.

Architectures

Here you can find a brief explanation of each of the architectures defined as `arch`, `arch_build` and `arch_target` settings.

- **x86**: The popular 32 bit x86 architecture.
- **x86_64**: The popular 64 bit x64 architecture.
- **ppc64le**: The PowerPC 64 bit Big Endian architecture.
- **ppc32**: The PowerPC 32 bit architecture.
- **ppc64le**: The PowerPC 64 bit Little Endian architecture.

- **ppc64**: The PowerPC 64 bit Big Endian architecture.
- **armv5el**: The ARM 32 bit version 5 architecture, soft-float.
- **armv5hf**: The ARM 32 bit version 5 architecture, hard-float.
- **armv6**: The ARM 32 bit version 6 architecture.
- **armv7**: The ARM 32 bit version 7 architecture.
- **armv7hf**: The ARM 32 bit version 7 hard-float architecture.
- **armv7s**: The ARM 32 bit version 7 *swift* architecture mostly used in Apple's A6 and A6X chips on iPhone 5, iPhone 5C and iPad 4.
- **armv7k**: The ARM 32 bit version 7 *k* architecture mostly used in Apple's WatchOS.
- **armv8**: The ARM 64 bit and 32 bit compatible version 8 architecture. It covers only the `aarch64` instruction set.
- **armv8_32**: The ARM 32 bit version 8 architecture. It covers only the `aarch32` instruction set (a.k.a. `ILP32`).
- **armv8.3**: The ARM 64 bit and 32 bit compatible version 8.3 architecture. Also known as `arm64e`, it is used on the A12 chipset added in the latest iPhone models (XS/XS Max/XR).
- **arm64ec**: Windows 11 ARM64EC (Emulation Compatible). This architecture support is **experimental** and incomplete. Supported in CMake for VS and MSBuild integrations.. Report new issues in Github if necessary.
- **sparc**: The SPARC (Scalable Processor Architecture) originally developed by Sun Microsystems.
- **sparcv9**: The SPARC version 9 architecture.
- **mips**: The 32 bit MIPS (Microprocessor without Interlocked Pipelined Stages) developed by MIPS Technologies (formerly MIPS Computer Systems).
- **mips64**: The 64 bit MIPS (Microprocessor without Interlocked Pipelined Stages) developed by MIPS Technologies (formerly MIPS Computer Systems).
- **avr**: The 8 bit AVR microcontroller architecture developed by Atmel (Microchip Technology).
- **s390**: The 32 bit address Enterprise Systems Architecture 390 from IBM.
- **s390x**: The 64 bit address Enterprise Systems Architecture 390 from IBM.
- **asm.js**: The subset of JavaScript that can be used as low-level target for compilers, not really a processor architecture, it's produced by Emscripten. Conan treats it as an architecture to align with build systems design (e.g. GNU auto tools and CMake).
- **wasm**: The Web Assembly, not really a processor architecture, but byte-code format for Web, it's produced by Emscripten. Conan treats it as an architecture to align with build systems design (e.g. GNU auto tools, CMake, etc).
- **wasm64**: The Web Assembly 64 bit, same as **wasm** but for 64 bit. Conan will add the necessary flags to the compiler to produce 64 bit Web Assembly code.
- **sh4le**: The Hitachi SH-4 SuperH architecture.
- **e2k-v2**: The Elbrus 2000 v2 512 bit VLIW (Very Long Instruction Word) architecture (Elbrus 2CM, Elbrus 2C+ CPUs) originally developed by MCST (Moscow Center of SPARC Technologies).
- **e2k-v3**: The Elbrus 2000 v3 512 bit VLIW (Very Long Instruction Word) architecture (Elbrus 2S, aka Elbrus 4C, CPU) originally developed by MCST (Moscow Center of SPARC Technologies).
- **e2k-v4**: The Elbrus 2000 v4 512 bit VLIW (Very Long Instruction Word) architecture (Elbrus 8C, Elbrus 8C1, Elbrus 1C+ and Elbrus 1CK CPUs) originally developed by MCST (Moscow Center of SPARC Technologies).

- **e2k-v5**: The Elbrus 2000 v5 512 bit VLIW (Very Long Instruction Word) architecture (Elbrus 8C2 ,aka Elbrus 8CB, CPU) originally developed by MCST (Moscow Center of SPARC Technologies).
- **e2k-v6**: The Elbrus 2000 v6 512 bit VLIW (Very Long Instruction Word) architecture (Elbrus 2C3, Elbrus 12C and Elbrus 16C CPUs) originally developed by MCST (Moscow Center of SPARC Technologies).
- **e2k-v7**: The Elbrus 2000 v7 512 bit VLIW (Very Long Instruction Word) architecture (Elbrus 32C CPU) originally developed by MCST (Moscow Center of SPARC Technologies).
- **xtensalx6**: Xtensa LX6 DPU for ESP32 microcontroller.
- **xtensalx106**: Xtensa LX6 DPU for ESP8266 microcontroller.
- **xtensalx7**: Xtensa LX7 DPU for ESP32-S2 and ESP32-S3 microcontrollers.

C++ standard libraries (aka compiler.libcxx)

`compiler.libcxx` sub-setting defines C++ standard libraries implementation to be used. The sub-setting applies only to certain compilers, e.g. it applies to *clang*, *apple-clang* and *gcc*, but doesn't apply to *Visual Studio*.

- **libstdc++** (gcc, clang, apple-clang, sun-cc, intel-cc, mcst-lcc, emcc): The GNU C++ Library. NOTE that this implicitly defines **_GLIBCXX_USE_CXX11_ABI=0** to use old ABI. Might be a wise choice for old systems, such as CentOS 6. On Linux systems, you may need to install libstdc++-dev (package name could be different in various distros) in order to use the standard library. NOTE that on Apple systems usage of **libstdc++** has been deprecated.
- **libstdc++11** (gcc, clang, intel-cc, mcst-lcc, emcc): The GNU C++ Library. NOTE that this implicitly defines **_GLIBCXX_USE_CXX11_ABI=1** to use new ABI. Might be a wise choice for newer systems, such as Ubuntu 20. On Linux systems, you may need to install libstdc++-dev (package name could be different in various distros) in order to use the standard library. NOTE that on Apple systems usage of **libstdc++** has been deprecated.
- **libc++** (clang, apple-clang, intel-cc, emcc): LLVM libc++. On Linux systems, you may need to install libc++-dev (package name could be different in various distros) in order to use the standard library.
- **c++_shared** (clang, Android only): use LLVM libc++ as a shared library. Refer to the C++ Library Support for the additional details.
- **c++_static** (clang, Android only): use LLVM libc++ as a static library. Refer to the C++ Library Support for the additional details.
- **libCstd** (sun-cc): Rogue Wave's stdlib. See Comparing C++ Standard Libraries libCstd, libstlport, and libstdcxx.
- **libstlport** (sun-cc): STLport. See Comparing C++ Standard Libraries libCstd, libstlport, and libstdcxx.
- **libstdcxx** (sun-cc): Apache C++ Standard Library. See Comparing C++ Standard Libraries libCstd, libstlport, and libstdcxx.
- **gpp** (qcc): GNU C++ lib. See QCC documentation.
- **cpp** (qcc): Dinkum C++ lib. See QCC documentation.
- **cpp-ne** (qcc): Dinkum C++ lib (no exceptions). See QCC documentation.
- **acpp** (qcc): Dinkum Abridged C++ lib. See QCC documentation.
- **acpp-ne** (qcc): Dinkum Abridged C++ lib (no exceptions). See QCC documentation.
- **ecpp** (qcc): Embedded Dinkum C++ lib. See QCC documentation.
- **ecpp-ne** (qcc): Embedded Dinkum C++ lib (no exceptions). See QCC documentation.
- **cxx** (qcc): LLVM C++. See QCC documentation.

Customizing settings

Settings are also customizable to add your own ones:

Adding new settings

It is possible to add new settings at the root of the *settings.yml* file, something like:

```
os:
    Windows:
        subsystem: [null, cygwin, msys, msys2, wsl]
distro: [null, RHEL6, CentOS, Debian]
```

If we want to create different binaries from our recipes defining this new setting, we would need to add to our recipes that:

```
class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch", "distro"
```

The value `null` allows for not defining it (which would be a default value, valid for all the other distros). It is also possible to define values for it in the profiles:

```
[settings]
os = "Linux"
distro = "CentOS"
compiler = "gcc"
```

And use their values to affect our build if desired:

```
class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch", "distro"

    def generate(self):
        tc = CMakeToolchain(self)
        if self.settings.distro == "CentOS":
            tc.cache_variables["SOME_CENTOS_FLAG"] = "Some CentOS Value"
            ...
```

Adding new sub-settings

The above approach requires modification to all recipes to take it into account. It is also possible to define kind of incompatible settings, like `os=Windows` and `distro=CentOS`. While adding new settings is totally suitable, it might make more sense to add it as a new sub-setting of the `Linux` OS:

```
os:
    Windows:
        subsystem: [null, cygwin, msys, msys2, wsl]
    Linux:
        distro: [null, RHEL6, CentOS, Debian]
```

With this definition we could define our profiles as:

```
[settings]
os = "Linux"
os.distro = "CentOS"
compiler = "gcc"
```

And any attempt to define `os.distro` for another `os` value rather than `Linux` will raise an error.

As this is a sub-setting, it will be automatically taken into account in all recipes that declare an `os` setting. Note that having a value of `distro=null` possible is important if you want to keep previously created binaries, otherwise you would be forcing to always define a specific distro value, and binaries created without this sub-setting, won't be usable anymore.

The sub-setting can also be accessed from recipes:

```
class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch"  # Note, no "distro" defined here

    def generate(self):
        tc = CMakeToolchain(self)
        if self.settings.os == "Linux" and self.settings.os.distro == "CentOS":
            tc.cache_variables["SOME_CENTOS_FLAG"] = "Some CentOS Value"
```

It is possible to have `ANY` to define nested subsettings, being the `ANY` the fallback for any value not matching the defined ones:

```
os:
    ANY:
        version: [null, ANY]
    Ubuntu:
        version: ["18.04", "20.04"]
```

This will allow settings like `-s os=MyOS -s os.version=1.2.3`, because the version can be `ANY` for `os!=Ubuntu`, but if we try `-s os=Ubuntu -s os.version=1.2.3` it will error because `Ubuntu` only accept those defined versions.

Add new values

In the same way we have added a new `distro` sub-setting, it is possible to add new values to existing settings and sub-settings. For example, if some compiler version is not present in the range of accepted values, you can add those new values.

You can also add a completely new compiler:

```
os:
    Windows:
        subsystem: [null, cygwin, msys, msys2, wsl]
   ...
compiler:
    gcc:
        ...
    mycompiler:
        version: [1.1, 1.2]
    msvc:
```

This works as the above regarding profiles, and the way they can be accessed from recipes. The main issue with custom compilers is that the builtin build helpers, like `CMake`, `MSBuild`, etc, internally contains code that will check for those

values. For example, the `MSBuild` build helper will only know how to manage the `msvc` setting and sub-settings, but not the new compiler. For those cases, custom logic can be implemented in the recipes:

```
class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch"

    def build(self):
        if self.settings.compiler == "mycompiler":
            my_custom_compile = ["some", "--flags", "for", "--my=compiler"]
            self.run(["mycompiler", "."] + my_custom_compile)
```

Note: You can remove items from *settings.yml* file: compilers, OS, architectures, etc. Do that only in the case you really want to protect against creation of binaries for other platforms other than your main supported ones. In the general case, you can leave them, the binary configurations are managed in **profiles**, and you want to define your supported configurations in profiles, not by restricting the *settings.yml*

Note: If you customize your *settings.yml*, you can share, distribute and sync this configuration with your team and CI machines with the *conan config install* command.

settings_user.yml

The previous section explains how to customize the Conan *settings.yml*, but you could also create your *settings_user.yml*. This file will contain only the new fields-values that you want to use in your recipes, so the final result will be a composition of both files, the *settings.yml* and the *settings_user.yml*.

See also:

- *Customize your settings: create your settings_user.yml*

9.6.7 remotes.json

The **remotes.json** file is located in the Conan user home directory, e.g., *[CONAN_HOME]/remotes.json*.

The default file created by Conan looks like this:

Listing 83: **remotes.json**

```
{
 "remotes": [
  {
   "name": "conancenter",
   "url": "https://center2.conan.io",
   "verify_ssl": true
  }
 ]
}
```

Note: Default Remote Update in Conan 2.9.2

Starting from **Conan version 2.9.2**, the default remote has been changed to *https://center2.conan.io*. The previous default remote *https://center.conan.io* is now frozen and will no longer receive updates. It is recommended to update

your remote configuration to use the new default remote to ensure access to the latest recipes and package updates (for more information, please read this post).

If you still have the deprecated remote configured as the default, please update using the following command:

```
conan remote update conancenter --url="https://center2.conan.io"
```

Essentially, it tells Conan where to list/upload/download the recipes/binaries from the remotes specified by their URLs.

The fields for each remote are:

- `name` (Required, `string` value): Name of the remote. This name will be used in commands like *conan list*, e.g., **`conan list zlib/1.3.1 --remote my_remote_name`**.
- `url` (Required, `string` value): indicates the URL to be used by Conan to search for the recipes/binaries.
- `verify_ssl` (Required, `bool` value): Verify SSL certificate of the specified url.
- `disabled` (Optional, `bool` value, `false` by default): If the remote is enabled or not to be used by commands like search, list, download and upload. Notice that a disabled remote can be used to authenticate against it even if it's disabled.
- `allowed_packages`: (Optional, `list` of `string` values): List of recipes that are allowed to be downloaded from this remote. If the list is empty or not present, all packages are allowed. Uses fnmatch rules.
- `recipes_only`: (Optional, `bool` value, `false` by default): If true, only recipes will be downloaded from this remote, no binaries will be downloaded.
- `force_auth`: (Optional, `bool` value, `false` by default): If true, Conan will not attempt anonymous access to this remote, going directly for the authenticated credentials instead. This is useful to force authentication on remotes that might return a different response depending on the user (like hiding information for anonymous users).

See also:

- *How to manage SSL (TLS) certificates*
- *How to manage remotes.json through CLI: conan remotes*.
- *How to use your own secrets manager for Conan remotes logins*.

9.6.8 source_credentials.json

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

When a `conanfile.py` recipe downloads some sources from other servers with the `download()` or the `get()` helpers like:

```
def source(self):
    # Immutable source .zip
    download(self, f"https://server/that/need/credentials/files/tarballname-{self.
↪version}.zip", "downloaded.zip")
    # Also the ``get()`` function, as it internally calls ``download()``
```

These downloads would be typically anonymous for open-source third party libraries in the internet, but it is also possible that some proprietary code in a private organization or provided by a vendor would require some kind of authentication.

For this purpose the `source_credentials.json` file can be provided in the Conan cache. This file has the following format, in which every `credentials` entry should have a `url` that defines the URL that should match the recipe one. If the recipe URL starts with the given one in the credentials files, then the credentials will be injected. If the file provides multiple credentials for multiple URLs, they will be evaluated in order until the first match happens. If no match is found, no credentials will be injected. A custom *auth plugin* can also be used to retrieve credentials directly from your own secrets manager.

It has to be noted that the `source_credentials` applies only to files downloaded with the `tools.files` `download()` and `get()` helpers, but it won't be used in other cases. To provide credentials for Conan repos, the `credentials.json` file should be used instead, see *credentials.json*.

```
{
    "credentials": [
        {
            "url": "https://server/that/need/credentials",
            "token": "mytoken"
        }
    ]
}
```

Using the `token` field, will add an `Authorization = Bearer {token}` header. This would be the preferred way of authentication, as it is typically more secure than using user/password.

If for some reason HTTP-Basic auth with user/password is necessary it can be provided with the `user` and `password` fields:

```
{
    "credentials": [
        {
            "url": "https://server/that/need/credentials",
            "user": "myuser",
            "password": "mypassword"
        }
    ]
}
```

As a general rule, hardcoding secrets like passwords in files is strongly discouraged. To avoid it, the `source_credentials.json` file is always rendered as a jinja template, so it can do operations like getting environment variables `os.getenv()`, allowing the secrets to be configured at the system or CI level:

```
{% set mytk = os.getenv('mytoken') %}
{
    "credentials": [
        {
            "url": "https://server/that/need/credentials",
            "token": "{{mytk}}"
        }
    ]
}
```

Note that `mytoken` environment variable must be defined, otherwise `mytk=None`, and that will translate to a literal `token="None"` that is obviously an invalid token and will cause an authentication failure. If you want to condition the

existence of the credential itself, you need to protect the whole credential entry (both `url`, and `token`) with a `{% if mytk %}-{% endif %}` block.

In some special cases, the server might need some specific custom headers. You can also specify them using a `headers` dictionary, either on its own, or together with the `token` or `user/password` fields:

```
{
    "credentials": [
        {
            "url": "https://server/that/need/credentials",
            "token": "mytoken",
            "headers": {"my-header-1": "my-value-1", "my-header-2": "my-value-2"}
        }
    ]
}
```

Note: Best practices

- Avoid using URLs that encode tokens or user/password authentication in the `conanfile.py` recipes. These URLs can easily leak into logs, and can be more difficult to fix in case of credentials changes (this is also valid for Git repositories URLs and clones, better use other Git auth mechanisms like ssh-keys)

See also:

- *How to use your own secrets manager for your source server logins*.

9.6.9 credentials.json

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Conan can authenticate against its Conan remote servers with the following:

- Interactive command line, when some server launches an unauthorized error, the Conan client will ask for user/password interactively and retry.
- With the `conan remote login` command, authentication can be done with argument passing, or interactively.
- With the environment variables `CONAN_LOGIN_USERNAME` for all remotes (`CONAN_LOGIN_USERNAME_{REMOTE}` for an individual remote) and `CONAN_PASSWORD` (`CONAN_PASSWORD_{REMOTE}` for an individual remote), Conan will not request interactively in the command line when necessary, but will take the values from the environment variables as if they were provided by the user.
- With a `credentials.json` file put in the Conan cache.
- With a custom *auth plugin*.

This section describes the usage of `credentials.json` file.

This file has the following format, in which every `credentials` entry should have a `remote` name, matching the name defined in `conan remote list`. Then, the `user` and `password` fields.

```
{
    "credentials": [
        {
            "remote": "default",
            "user": "admin",
            "password": "password"
        }
    ]
}
```

Conan will be able to extract the credentials from this file automatically when necessary and requested by the server.

Note: Conan does not pre-emptively use the credentials to force a login automatically in every remote defined at every Conan command. By default Conan uses the previously stored tokens or anonymous usage, until an explicit `conan remote login` command is done, or until a remote server launches an authentication error. When that happens, authentication against that server will be done, using the `credentials.json` file, the environment variables or the user interactive inputs.

The priority of credentials origins is as follows:

- If the `credentials.json` file exist, it has higher priority, if an entry for the remote exists, it will be used. If it doesn't work, it will be an error.
- If an entry in the `credentials.json` for that remote does not exist, it will look for defined environment variables
- If environment variables don't exist, it will request interactively the credentials. If `core:non_interactive=True`, it will error.

The `credentials.json` file is jinja-rendered with injected `platform` and `os` imports, so it allows to use jinja syntax. For example it could do something like the following to get the credentials from environment variables:

```
{% set myuser = os.getenv('myuser') %}
{% set mytk = os.getenv('mytoken') %}
{
    "credentials": [
        {
            "remote": "myremote",
            "user": "{{myuser}}"
            "password": "{{mytk}}"
        }
    ]
}
```

See also:

- *How to use your own secrets manager for Conan remotes logins*.

9.6.10 .conanrc

Warning: This feature is in **preview**. See *the Conan stability* section for more information.

The **.conanrc** file can be placed in the folder where you are running Conan or any parent folder. This file is used to set up the Conan user home directory by defining the *conan_home* value. This value will take precedence over the *CONAN_HOME* environment variable in case it's also defined. Below are some examples of how you can define the Conan user home in the **.conanrc** file:

Set the Conan home to an absolute folder:

```
# accepts comments
conan_home=/absolute/folder
```

Set the Conan home to a relative folder inside the current folder:

```
conan_home=./relative/folder/inside/current/folder
```

Set the Conan home to a relative folder outside the current folder:

```
conan_home=../relative/folder/outside/current/folder
```

Set the Conan home to a path containing the ~ symbol, which will be expanded to the system's user home:

```
conan_home=~/use/the/user/home/to/expand/it
```

Be aware that the **.conanrc** file is searched for in all parent folders. For example, in this structure:

```
.
.conanrc
|-- project1
|-- project2
```

If you are running from the folder *project1*, the parent folders are traversed recursively until a **.conanrc** file is found, in case it exists.

9.7 Environment variables

These are very few environment variables that can be used to configure some of the Conan behavior. These variables are the exception, for customization and configuration control, Conan uses the *global.conf configuration* and the *profile [conf] section*

9.7.1 CONAN_HOME

This variable controls the location of the Conan home folder. By default, if it is not defined, it will be <username>/.conan2.

Note: Recall that the Conan package cache, contained in the Conan home, is not concurrent. Different parallel tasks like those that can happen in CI, need to use a separate cache, and defining CONAN_HOME is the way to do it.

9.7.2 CONAN_DEFAULT_PROFILE, CONAN_DEFAULT_BUILD_PROFILE

The default profile will be the "default" file in the Conan cache. These environment variables allow to define a different default for the host and build profiles respectively. There are also equivalent conf items core:default_profile and core:default_build_profile to define such default profile names. In general, env-vars should be used only when the conf is not enough.

9.7.3 Remote login variables

CONAN_LOGIN_USERNAME, CONAN_LOGIN_USERNAME_{REMOTE_NAME} define the login username for a given remote. CONAN_PASSWORD, CONAN_PASSWORD_{REMOTE_NAME} define the login password for a given remote.

These environment variables are just a substitute of the interactive input of the username or password when Conan CLI requests it. They do not perform any kind of authentication unless the remote server throws an authentication challenge. That means that for some remote servers configured to allow anonymous usage, these will not be used, and the user will remain as an unauthenticated user, unless a conan remote login or conan remote auth is done first.

When the Conan CLI is about to ask the user for the remote password, it will check the variable CONAN_LOGIN_USERNAME_{REMOTE_NAME} or CONAN_PASSWORD_{REMOTE_NAME} first, if the variable is not declared Conan will try to use the variable CONAN_LOGIN_USERNAME and CONAN_PASSWORD respectively, if the variable is not declared either, Conan will request to the user to input a password or fail.

The remote name is transformed to all uppercase. If the remote name contains "-", you have to replace it with "_" in the variable name.

Note:

- These variables are useful for unattended executions like CI servers or automated tasks, as CI secrets
- These variables are not recommended for developer machines.
- Recall that these variables do not perform authentication unless the remote server requests it.
- The core:non_interactive conf can be defined in global.conf to force Conan to fail if any interactive prompt is requested, to avoid CI process being stuck.

9.7.4 Terminal color variables

Conan default behavior is try to autodetect the output. If the output is redirected to a file, or other support not `tty`, that cannot print colors, it will disable colored output. For regular terminals, it will try to do colored output, unless some of the following change that behavior:

- `CLICOLOR_FORCE` Forces the generation of terminal color escape characters, no matter what the autodetection of terminal is.
- `NO_COLOR` disables the generation of color escape characters. This will be ignored if `CLICOLOR_FORCE` is activated.
- `CONAN_COLOR_DARK` will revert the color scheme for white/light background terminals (default assumes dark background).

9.7.5 Logging

The environment variable `CONAN_LOG_LEVEL` can define the Conan command line verbosity in the same way that the `-v` command line argument, with the same values (`error`, `verbose`, etc.). It also has priority over the value of the command line arg if both are present. This can be useful to temporarily change the log level in CI pipelines, in automation, etc., without needing to modify the command line arguments.

9.8 Extensions

Conan can be extended in a few ways, with custom user code:

- `python_requires` allow to put common recipe code in a recipe package that can be reused by other recipes by declaring a `python_requires = "mypythoncode/version"`
- You can create your own custom Conan commands to solve self-needs thanks to Python and Conan public API powers altogether.
- It's also possible to make your own custom Conan generators in case you are using build systems that are not supported by the built-in Conan tools. Those can be used from `python_requires` or installed globally.
- `hooks` are "pre" and "post" recipe methods (like `pre_build()` and `post_build()`) extensions that can be used to complement recipes with orthogonal functionality, like quality checks, binary analyzing, logging, etc.
- Binary compatibility `compatibility.py` extension allows to write custom rules for defining custom binary compatibility across different settings and options
- The `cmd_wrapper.py` extension allows to inject arbitrary command wrappers to any `self.run()` recipe command invocation, which can be useful to inject wrappers as parallelization tools
- The package signing extension allows to sign and verify packages at upload and install time respectively
- Deployers, a mechanism to facilitate copying files from one folder, usually the Conan cache, to user folders

---

Note: Besides the built-in Conan extensions listed in this document, there is a repository that contains extensions for Conan, such as custom commands and deployers, useful for different purposes like artifactory tasks, Conan Center Index, etc.

You can find more information on how to use those extensions in the GitHub repository.

Contents:

9.8.1 Python requires

Introduction

The `python_requires` feature is a very convenient way to share files and code between different recipes. A python require is a special recipe that does not create packages and it is just intended to be reused by other recipes.

A very simple recipe that we want to reuse could be:

```
from conan import ConanFile

myvar = 123

def myfunct():
    return 234

class Pkg(ConanFile):
    name = "pyreq"
    version = "0.1"
    package_type = "python-require"
```

And then we will make it available to other packages with `conan create .`. Note that a `python-require` package does not create binaries, it is just the recipe part.

```
$ conan create .
# It will only export the recipe, but will NOT create binaries
# python-requires do NOT have binaries
```

We can reuse the above recipe functionality declaring the dependency in the `python_requires` attribute and we can access its members using `self.python_requires["<name>"].module`:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "0.1"
    python_requires = "pyreq/0.1"

    def build(self):
        v = self.python_requires["pyreq"].module.myvar  # v will be 123
        f = self.python_requires["pyreq"].module.myfunct()  # f will be 234
        self.output.info(f"{v}, {f}")
```

```
$ conan create .
...
pkg/0.1: 123, 234
```

Python requires can also use version ranges, and this can be recommended in many cases if those `python-requires` need to evolve over time:

```
from conan import ConanFile

class Pkg(ConanFile):
    python_requires = "pyreq/[>=1.0 <2]"
```

It is also possible to require more than 1 `python-requires` with `python_requires = "pyreq/0.1", "other/1.2"`

Extending base classes

A common use case would be to declare a base class with methods we want to reuse in several recipes via inheritance. We'd write this base class in a python-requires package:

```
from conan import ConanFile

class MyBase:
    def source(self):
        self.output.info("My cool source!")
    def build(self):
        self.output.info("My cool build!")
    def package(self):
        self.output.info("My cool package!")
    def package_info(self):
        self.output.info("My cool package_info!")

class PyReq(ConanFile):
    name = "pyreq"
    version = "0.1"
    package_type = "python-require"
```

And make it available for reuse with:

```
$ conan create .
```

Note that there are two classes in the recipe file:

- `MyBase` is the one intended for inheritance and doesn't extend `ConanFile`.
- `PyReq` is the one that defines the current package being exported, it is the recipe for the reference `pyreq/0.1`.

Once the package with the base class we want to reuse is available we can use it in other recipes to inherit the functionality from that base class. We'd need to declare the `python_requires` as we did before and we'd need to tell Conan the base classes to use in the attribute `python_requires_extend`. Here our recipe will inherit from the class `MyBase`:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "0.1"
    python_requires = "pyreq/0.1"
    python_requires_extend = "pyreq.MyBase"
```

The resulting inheritance is equivalent to declare our `Pkg` class as `class Pkg(pyreq.MyBase, ConanFile)`. So creating the package we can see how the methods from the base class are reused:

```
$ conan create .
...
pkg/0.1: My cool source!
pkg/0.1: My cool build!
pkg/0.1: My cool package!
```

(continues on next page)

(continued from previous page)

```
pkg/0.1: My cool package_info!
...
```

In general, base class attributes are not inherited, and should be avoided as much as possible. There are method alternatives to some of them like `export()` or `set_version()`. For exceptional situations, see the `init()` method documentation for more information to extend inherited attributes.

It is possible to re-implement some of the base class methods, and also to call the base class method explicitly, with the Python `super()` syntax:

```
from conan import ConanFile

class Pkg(ConanFile):
    name = "pkg"
    version = "0.1"
    python_requires = "pyreq/0.1"
    python_requires_extend = "pyreq.MyBase"

    def source(self):
        super().source()  # call the base class method
        self.output.info("MY OWN SOURCE") # Your own implementation
```

It is not mandatory to call the base class method, a full overwrite without calling `super()` is possible. Also the call order can be changed, and calling your own code, then `super()` is possible.

Reusing files

It is possible to access the files exported by a recipe that is used with `python_requires`. We could have this recipe, together with a *myfile.txt* file containing the "Hello" text.

```
from conan import ConanFile

class PyReq(ConanFile):
    name = "pyreq"
    version = "1.0"
    package_type = "python-require"
    exports = "*"
```

```
$ echo "Hello" > myfile.txt
$ conan create .
```

Now that the python-require has been created, we can access its path (the place where *myfile.txt* is) with the `path` attribute:

```
import os

from conan import ConanFile
from conan.tools.files import load

class Pkg(ConanFile):
    python_requires = "pyreq/0.1"
```

(continues on next page)

(continued from previous page)

```
    def build(self):
        pyreq_path = self.python_requires["pyreq"].path
        myfile_path = os.path.join(pyreq_path, "myfile.txt")
        content = load(self, myfile_path)  # content = "Hello"
        self.output.info(content)
        # we could also copy the file, instead of reading it
```

Note that only `exports` works for this case, but not `exports_sources`.

Testing python-requires

It is possible to test with `test_package` a `python_require`, by adding a `test_package/conanfile.py`:

Listing 84: conanfile.py

```
from conan import ConanFile

def mynumber():
    return 42

class PyReq(ConanFile):
    name = "pyreq"
    version = "1.0"
    package_type = "python-require"
```

Listing 85: test_package/conanfile.py

```
from conan import ConanFile

class Tool(ConanFile):

    # Literal "tested_reference_str", Conan will dynamically replace it
    python_requires = "tested_reference_str"

    def test(self):
        pyreq = self.python_requires["pyreq"].module
        mynumber = pyreq.mynumber()
        self.output.info("{}!!!".format(mynumber))
```

The `python_requires = "tested_reference_str"` is mandatory from Conan 2.1. Automatic injection of `python_requires` without this declaration is deprecated and it will be removed in future versions.

Note that the `test_package/conanfile.py` does not need any type of declaration of the `python_requires`, this is done automatically and implicitly. We can now create and test it with:

```
$ conan create .
...
pyreq/0.1 (test package): 42!!!
```

Effect in package_id

The `python_requires` will affect the `package_id` of the **consumer packages** using those dependencies. By default, the policy is `minor_mode`, which means:

- Changes to the **patch** version of the **revision** of a python-require will not affect the package ID. So depending on `"pyreq/1.2.3"` or `"pyreq/1.2.4"` will result in identical package ID (both will be mapped to `"pyreq/1.2.Z"` in the hash computation). Bump the patch version if you want to change your common code, but you don't want the consumers to be affected or to fire a re-build of the dependants.
- Changes to the **minor** version will produce a different package ID. So if you depend on `"pyreq/1.2.3"`, and you bump the version to `"pyreq/1.3.0"`, then, you will need to build new binaries that are using that new python-require. Bump the minor or major version if you want to make sure that packages requiring this python-require will be built using these changes in the code.

In most cases using a version-range `python_requires = "pyreq/[>=1.0 <2.0]"` is the right approach, because that means the **major** version bumps are not included because they would require changes in the consumers themselves. It is then possible to release a new major version of the `pyreq/2.0`, and have consumers gradually change their requirements to `python_requires = "pyreq/[>=2.0 <3.0]"`, fix the recipes, and move forward without breaking the whole project.

As with the regular `requires`, this default can be customized with the `core.package_id:default_python_mode` configuration.

It is also possible to customize the effect of `python_requires` per package, using the `package_id()` method:

```
from conan import ConanFile

class Pkg(ConanFile):
    python_requires ="pyreq/[>=1.0]"
    def package_id(self):
        self.info.python_requires.patch_mode()
```

Resolution of python_requires

There are few important things that should be taken into account when using `python_requires`:

- Python requires recipes are loaded by the interpreter just once, and they are common to all consumers. Do not use any global state in the `python_requires` recipes.
- Python requires are private to the consumers. They are not transitive. Different consumers can require different versions of the same `python-require`. Being private, they cannot be overridden from downstream in any way.
- `python_requires` cannot use regular `requires` or `tool_requires` themselves. Having a `requirements()` (and similar) methods to be inherited by recipes is possible and allowed, but the `python_requires` class itself cannot use them.
- `python_requires` cannot be "aliased".
- `python_requires` can use native python `import` to other python files, as long as these are exported together with the recipe.
- `python_requires` can be used as editable packages too.
- `python_requires` are locked in lockfiles, to guarantee reproducibility, in the same way that other `requires` and `tool_requires` are locked.

Note: Best practices

- Even if `python-requires` can `python_requires` transitively other `python-requires` recipes, this is discouraged. Multiple level inheritance and reuse can become quite complex and difficult to manage, it is recommended to keep the hierarchy flat.
- Do not try to mix Python inheritance with `python_requires_extend` inheritance mechanisms, they are incompatible and can break.
- Do not use multiple inheritance for `python-requires`

9.8.2 Custom commands

It's possible to create your own Conan commands to solve self-needs thanks to Python and Conan public API powers altogether.

Location and naming

All the custom commands must be located in `[YOUR_CONAN_HOME]/extensions/commands/` folder. If you don't know where `[YOUR_CONAN_HOME]` is located, you can run **`conan config home`** to check it.

If _commands_ sub-directory is not created yet, you will have to create it. Those custom commands files must be Python files and start with the prefix `cmd_[your_command_name].py`. The call to the custom commands is like any other existing Conan one: **`conan your_command_name`**.

Scoping

It's possible to have another folder layer to group some commands under the same topic.

For instance:

```
| - [YOUR_CONAN_HOME]/extensions/commands/greet/
      | - cmd_hello.py
      | - cmd_bye.py
```

The call to those commands change a little bit: **`conan [topic_name]:your_command_name`**. Following the previous example:

```
$ conan greet:hello
$ conan greet:bye
```

Note: It's possible for only one folder layer, so it won't work to have something like `[YOUR_CONAN_HOME]/extensions/commands/topic1/topic2/cmd_command.py`

Decorators

conan_command(group=None, formatters=None)

Main decorator to declare a function as a new Conan command. Where the parameters are:

- `group` is the name of the group of commands declared under the same name. This grouping will appear executing the **conan -h** command.
- `formatters` is a dict-like Python object where the `key` is the formatter name and the value is the function instance where will be processed the information returned by the command one.

Listing 86: cmd_hello.py

```
import json

from conan.api.conan_api import ConanAPI
from conan.api.output import ConanOutput, cli_out_write
from conan.cli.command import conan_command

def output_json(msg):
    myjson = json.dumps({"greet": msg})
    cli_out_write(myjson)


@conan_command(group="Custom commands", formatters={"json": output_json})
def hello(conan_api: ConanAPI, parser, *args):
    """
    Simple command to print "Hello World!" line
    """
    msg = "Hello World!"
    ConanOutput().info(msg)
    return msg
```

Important: The function decorated by `@conan_command(....)` must have the same name as the suffix used by the Python file. For instance, the previous example, the file name is `cmd_hello.py`, and the command function decorated is `def hello(....)`.

conan_subcommand(formatters=None)

Similar to `conan_command`, but this one is declaring a sub-command of an existing custom command. For instance:

Listing 87: cmd_hello.py

```
from conan.api.conan_api import ConanAPI
from conan.api.output import ConanOutput
from conan.cli.command import conan_command, conan_subcommand


@conan_subcommand()
def hello_moon(conan_api, parser, subparser, *args):
    """
```

(continues on next page)

(continued from previous page)

```
    Sub-command of "hello" that prints "Hello Moon!" line
    """
    ConanOutput().info("Hello Moon!")


@conan_command(group="Custom commands")
def hello(conan_api: ConanAPI, parser, *args):
    """
    Simple command "hello"
    """
```

The command call looks like **conan hello moon**.

Note: Notice that to declare a sub-command is required an empty Python function acts as the main command.

Argument definition and parsing

Commands can define their own arguments with the `argparse` Python library.

```
@conan_command(group='Creator')
def build(conan_api, parser, *args):
    """
    Command help
    """
    parser.add_argument("path", nargs="?", help='help for command')
    ...
    args = parser.parse_args(*args)
    # Use args.path
```

When there are sub-commands, the base command cannot define arguments, only the sub-commands can do it. If you have a set of common arguments to all sub-commands, you can define a function that adds them.

```
@conan_command(group="MyGroup")
def mycommand(conan_api, parser, *args):
    """
    Command help
    """
    # Do not define arguments in the base command
    pass

@conan_subcommand()
def mycommand_mysubcommand(conan_api: ConanAPI, parser, subparser, *args):
    """
    Subcommand help
    """
    # Arguments are added to "subparser"
    subparser.add_argument("reference", help="Recipe reference or Package reference")
    # You can add common args with your helper
    # add_my_common_args(subparser)
    # But parsing all of them happens to "parser"
```

(continues on next page)

(continued from previous page)

```
    args = parser.parse_args(*args)
    # use args.reference
```

Formatters

The return of the command will be passed as argument to the formatters. If there are different formatters that require different arguments, the approach is to return a dictionary, and let the formatters chose the arguments they need. For example, the `graph info` command uses several formatters like:

```
def format_graph_html(result):
    graph = result["graph"]
    conan_api = result["conan_api"]
    ...

def format_graph_info(result):
    graph = result["graph"]
    field_filter = result["field_filter"]
    package_filter = result["package_filter"]
    ...

@conan_subcommand(formatters={"text": format_graph_info,
                              "html": format_graph_html,
                              "json": format_graph_json,
                              "dot": format_graph_dot})
def graph_info(conan_api, parser, subparser, *args):
    ...
    return {"graph": deps_graph,
            "field_filter": args.filter,
            "package_filter": args.package_filter,
            "conan_api": conan_api}
```

Commands parameters

These are the passed arguments to any custom command and its sub-commands functions:

Listing 88: cmd_command.py

```
from conan.cli.command import conan_command, conan_subcommand


@conan_subcommand()
def command_subcommand(conan_api, parser, subparser, *args):
    """
    subcommand information. This info will appear on ``conan command subcommand -h``.

    :param conan_api: <object conan.api.conan_api.ConanAPI> instance
    :param parser: root <object argparse.ArgumentParser> instance (coming from main␣
↪command)
    :param subparser: <object argparse.ArgumentParser> instance for sub-command
    :param args: ``list`` of all the arguments passed after sub-command call
```

(continues on next page)

(continued from previous page)

```
    :return: (optional) whatever is returned will be passed to formatters functions (if␣
↪declared)
    """
    # ...


@conan_command(group="Custom commands")
def command(conan_api, parser, *args):
    """
    command information. This info will appear on ``conan command -h``.

    :param conan_api: <object conan.api.conan_api.ConanAPI> instance
    :param parser: root <object argparse.ArgumentParser> instance
    :param args: ``list`` of all the arguments passed after command call
    :return: (optional) whatever is returned will be passed to formatters functions (if␣
↪declared)
    """
    # ...
```

- `conan_api`: instance of `ConanAPI` class. See more about it in *conan.api.conan_api.ConanAPI section*
- `parser`: root instance of Python `argparse.ArgumentParser` class to be used by the main command function. See more information in argparse official website.
- `subparser` (only for sub-commands): child instance of Python `argparse.ArgumentParser` class for each sub-command function.
- `*args`: list of all the arguments passed via command line to be parsed and used inside the command function. Normally, they'll be parsed as `args = parser.parse_args(*args)`. For instance, running **`conan mycommand arg1 arg2 arg3`**, the command function will receive them as a Python list-like `["arg1", "arg2", "arg3"]`.

See also:

- *Custom command to remove recipe and package revisions but the latest package one from the latest recipe one*.
- You can check more examples of Conan custom command in the *conan-extensions* repository https://github.com/conan-io/conan-extensions

9.8.3 Custom Conan generators

In the case that you need to use a build system or tool that is not supported by Conan off-the-shelf, you could create your own custom integrations using a custom generator. This can be done in three different ways.

Custom generators as python_requires

One way of having your own custom generators in Conan is by using them as *python_requires*. You could declare a *MyGenerator* class with all the logic to generate some files inside the *mygenerator/1.0 python_requires* package:

Listing 89: mygenerator/conanfile.py

```
from conan import ConanFile
from conan.tools.files import save

```

(continues on next page)

(continued from previous page)

```
class MyGenerator:
    def __init__(self, conanfile):
        self._conanfile = conanfile

    def generate(self):
        deps_info = ""
        for dep, _ in self._conanfile.dependencies.items():
            deps_info += f"{dep.ref.name}, {dep.ref.version}\n"
        save(self._conanfile, "deps.txt", deps_info)


class PyReq(ConanFile):
    name = "mygenerator"
    version = "1.0"
    package_type = "python-require"
```

And then `conan create mygenerator` and use it in the generate method of your own packages like this:

Listing 90: pkg/conanfile.py

```
from conan import ConanFile


class MyPkg(ConanFile):
    name = "pkg"
    version = "1.0"

    python_requires = "mygenerator/1.0"
    requires = "zlib/1.3.1", "bzip2/1.0.8"

    def generate(self):
        mygenerator = self.python_requires["mygenerator"].module.MyGenerator(self)
        mygenerator.generate()
```

Then, doing a `conan install pkg` on this `pkg` recipe, will create a `deps.txt` text file containing:

```
zlib, 1.2.11
bzip2, 1.0.8
```

This has the advantage that you can version your own custom generators as packages and also that you can share those generators as Conan packages.

Using global custom generators

You can also use your custom generators globally if you store them in the `[CONAN_HOME]/extensions/generators` folder. You can place them directly in that folder or install with the `conan config install` command.

Listing 91: [CONAN_HOME]/extensions/generators/mygen.py

```
from conan.tools.files import save


class MyGenerator:
    def __init__(self, conanfile):
        self._conanfile = conanfile

    def generate(self):
        deps_info = ""
        for dep, _ in self._conanfile.dependencies.items():
            deps_info = f"{dep.ref.name}, {dep.ref.version}"
        save(self._conanfile, "deps.txt", deps_info)
```

Then you can use them by name in the recipes or in the command line using the *-g* argument:

```
conan install --requires=zlib/1.2.13 -g MyGenerator
```

Generators from tool_requires

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

A direct dependency tool requires can also be used to provide custom generators. The following example shows how to create a custom generator that generates a file with the dependencies of the package, just like the example above, but using a `tool_require` instead of a `python_require` to inject the generator into the recipe, by adding them to the `self.generator_info` attribute inside the `package_info` method. Note that this attribute is `None` by default, so you need to set it explicitly to a list of generators.

Listing 92: mygenerator/conanfile.py

```
from conan import ConanFile
from conan.tools.files import save

class MyGenerator:
    def __init__(self, conanfile):
        self._conanfile = conanfile

    def generate(self):
        deps_info = ""
        for dep, _ in self._conanfile.dependencies.items():
            deps_info = f"{dep.ref.name}, {dep.ref.version}"
        save(self._conanfile, "deps.txt", deps_info)

class MyToolReq(ConanFile):
```

(continues on next page)

(continued from previous page)

```
    name = "mygenerator-tool"
    version = "1.0"

    def package_info(self):
        self.generator_info = [MyGenerator]
```

And then having a `tool_requires` in your recipe for the `mygenerator-tool` package will automatically inject the generator into the recipe.

Note: Note that built-in generators can also be injected using tool_requires, by adding them by name: `self.generator_info = ["CMakeDeps"]`. `tool_require`'ing this package will inject the `CMakeDeps` generator into the recipe just as if it was declared in its `generators` attribute.

9.8.4 Python API

Warning: The full Python API is **experimental**. See *the Conan stability* section for more information.

The Python API is a set of Python classes that allow you to interact with Conan programmatically. It's designed to be used as part of the custom commands extension point, or in Python scripts or applications, providing a more flexible and powerful way to work with Conan than the command line interface.

It is organized in submodules, each one providing a specific set of functionalities.

Note: Only the **documented** public members of these classes are guaranteed to be stable, and the rest of the members are considered private and can change without notice.

Note: Although instantiating the `ConanAPI` class is possible in standalone scripts, the recommended way to use the Python API is through *custom commands*, which provide a more stable and controlled environment for interacting with Conan.

Conan API Reference

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

class ConanAPI(*cache_folder=None*)

This is the main object to interact with the Conan API. It provides all the subapis to work with recipes, packages, remotes, etc., which are exposed as attributes of this class, and should not be created directly.

Parameters

cache_folder – Absolute path to the Conan home folder. If `None`, it will be resolved as explained in *home_folder*.

config: *ConfigAPI*

Used to interact with the local Conan configuration

remotes: *RemotesAPI*

Used to interact with remotes

command: *CommandAPI*

Used to call other commands

list: *ListAPI*

Used to get latest refs and list refs of recipes and packages

profiles: *ProfilesAPI*

Used to process and load Conan profiles

install: *InstallAPI*

Used to install binaries, sources, deploy packages and more

export: *ExportAPI*

Used to export recipes and pre-compiled package binaries to the Conan cache

upload: *UploadAPI*

Used to upload recipes and packages to remotes

download: *DownloadAPI*

Used to download recipes and packages from remotes

cache: *CacheAPI*

Used to interact wit the packages storage cache

lockfile: *LockfileAPI*

Used to read and manage lockfile files

local: *LocalAPI*

Local flow helpers for developer "source", "build", "editable" commands

audit: *AuditAPI*

Used to check vulnerabilities of dependencies

workspace: **WorkspaceAPI**

Used to manage workspaces

property home_folder: **str**

Where the Conan user home is located. Read only. Can be modified by the `CONAN_HOME` environment variable or by the `.conanrc` file in the current directory or any parent directory when Conan is called.

reinit()

Reinitialize the Conan API. This is useful when the configuration changes.

See also:

- *Creating Conan custom commands*

Audit API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class AuditAPI(*conan_api*)

This class provides the functionality to scan references for vulnerabilities.

static scan(*deps_graph*, *provider*, *context=None*)

Scan a given recipe for vulnerabilities in its dependencies.

Parameters

- **deps_graph** – Dependency graph as returned by the *Graph API*
- **provider** – Provider object as returned by *get_provider()*
- **context** – Context to filter the dependencies (e.g., `"host"` or `"build"`). If `None`, all contexts are considered.

Returns

A `dict` mapping each scanned reference to its vulnerability information, together with some metadata about the request, with the following shape:

```
{
    "data": {
        # One entry per scanned reference
        "openssl/3.2.0": {
            "vulnerabilities": {
                "totalCount": 2,
                # One "node" per vulnerability, holding its name,
                # description, severity, cvss, references,
                # advisories, fixed versions, etc.
                "edges": [{"node": {...}}, ...]
            }
        },
        # References that could not be scanned carry an error instead
        "unknown/1.0": {"error": {"details": "Package 'unknown/1.0'␣
↪not scanned: Not found."}},
    },
    # URL of the provider that produced the data, or None
    "provider_url": "https://...",
    # Only present if the whole request failed (authentication,
    # rate limit, server error...). When set, the scan is aborted.
    "conan_error": "...",
}
```

static list(*references: List[str]*, *provider*)

List the vulnerabilities of the given reference.

Parameters

- **references** – List of reference strings
- **provider** – Provider object as returned by *get_provider()*

Returns
A `dict` with the vulnerability information for each reference, with the same structure as the one returned by *scan()*.

`get_provider`(*provider_name: str*)

Get the provider opaque object by name. This object is only meant to be used as arguments for other methods in this class, and should not be used/modified directly.

Parameters
provider_name – Provider name

Returns
Provider opaque object

`list_providers`()

Get all available providers.

Returns
The list of available providers

`add_provider`(*name: str*, *url: str*, *provider_type: str*)

Add a provider.

Parameters

- **name** – Provider name
- **url** – Provider url
- **provider_type** – Provider type, either `conan-center-proxy` or `private`

`remove_provider`(*provider_name: str*)

Remove a provider.

Parameters
provider_name – Provider name

`auth_provider`(*provider_name*, *token: str*)

Set authentication token for the provider. Note that this does not perform an authentication attempt, it just stores the token for future use.

Parameters

- **provider_name** – Provider name
- **token** – Provider token

Cache API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

`class CacheAPI(conan_api, api_helpers)`

This CacheAPI is used to interact with the packages storage cache

Note that the Conan packages cache is exclusively **read-only** for user code. Only Conan can write or modify the folders and files in the Conan cache. In general, when a method returns a folder, it is mostly for debugging purposes and read-only access, but never to modify the contents of the cache.

`export_path(ref: RecipeReference)`

Returns the path of the recipe conanfile and exported files in the Conan cache

This folder is exclusively for **read-only** access, typically for debugging purposes, it is completely forbidden to modify any of its contents.

Parameters
`ref` – RecipeReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`recipe_metadata_path(ref: RecipeReference)`

Returns the path of the recipe metadata files in the Conan cache

Exceptionally, adding or modifying the files within this folder is allowed, as the metadata files are not taken into account into the computation of the recipe hash (recipe revision).

Parameters
`ref` – RecipeReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`export_source_path(ref: RecipeReference)`

Returns the path of the exported sources in the Conan cache

Note that the exported sources only exist in the cache when the package has been created locally or built from source.

This folder is exclusively for **read-only** access, typically for debugging purposes, it is completely forbidden to modify any of its contents.

Parameters
ref – RecipeReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`source_path`(*ref:* RecipeReference)

Returns the path of the temporary source folder in the Conan cache

Note that the source folder only exist in the cache when the package has been created locally or built from source.

This folder is exclusively for **read-only** access, typically for debugging purposes, it is completely forbidden to modify any of its contents.

Parameters
ref – RecipeReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`build_path`(*pref:* PkgReference)

Returns the path of the temporary build folder in the Conan cache

Note that the build folder only exist in the cache when the package has been created locally or built from source.

This folder is exclusively for **read-only** access, typically for debugging purposes, it is completely forbidden to modify any of its contents.

Parameters
pref – PkgReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one. Exactly same behavior for the package revision.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`package_metadata_path`(*pref:* PkgReference)

Returns the path of the package metadata folder in the Conan cache

Exceptionally, adding or modifying the files within this folder is allowed, as the metadata files are not taken into account into the computation of the package hash (package revision).

Parameters
pref – PkgReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one. Exactly same behavior for the package revision.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`package_path`(*pref:* PkgReference)

Returns the path of the package folder in the Conan cache

If the recipe implements the *finalize()* method and it has already been executed, the path of the finalized folder is returned instead.

This folder is exclusively for **read-only** access, typically for debugging purposes, it is completely forbidden to modify any of its contents.

Parameters
pref – PkgReference. If it includes recipe revision, that exact revision will be returned, if it doesn't include recipe revision, it will return the latest revision one. Exactly same behavior for the package revision.

Returns
path to the folder, as a string

Raises
ConanExcepcion if the folder doesn't exist

`check_integrity`(*package_list*, *return_pkg_list=False*)

Check if the recipes and packages are corrupted

Parameters

- **package_list** – PackagesList to check
- **return_pkg_list** – If True, return a PackagesList with corrupted artifacts

Returns
PackagesList with corrupted artifacts if return_pkg_list is True

Raises
ConanExcepcion if there are corrupted artifacts and return_pkg_list is False

`sign`(*package_list*)

Sign packages with the package signing plugin

`verify`(*package_list*)

Verify packages with the package signing plugin

`clean`(*package_list=None*, *source=True*, *build=True*, *download=True*, *temp=True*, *backup_sources=False*) → None

Remove non critical folders from the cache, like source, build and download (.tgz store) folders.

Parameters

- **package_list** – the package list that should be cleaned. If `None`, the whole cache is cleaned, and with `backup_sources` all the backup sources files are removed, including the ones that do not belong to any recipe in the cache
- **source** – boolean, remove the "source" folder if True
- **build** – boolean, remove the "build" folder if True
- **download** – boolean, remove the "download (.tgz)" folder if True
- **temp** – boolean, remove the temporary folders

- **backup_sources** – boolean, remove the backup sources files if True. If a `package_list` is given, only the ones downloaded by its recipes

Returns

save(*package_list:* PackagesList, *path*, *no_source=False*) → None

Create a compressed archive with recipes and packages from the Conan cache that can be later restored in another cache.

Do not manipulate the contents of the resulting archive, as it also contains metadata, and modifying the contents would be equivalent to modify the Conan package cache, which is forbidden.

Parameters

- **package_list** – PackagesList containing the recipes and packages to add to the compressed archive
- **path** – The archive file to generate. Based on the extension of the file, different compression formats can be used (.tgz, .txz and .tzst, the latter only for Python>=3.14).
- **no_source** – If True, the source folders in the cache will not be added to the archive.

Returns

restore(*path*) → *PackagesList*

Restore a compressed archive with recipes and packages previously saved from another Conan cache into the currently active Conan cache.

Parameters

path – The archive file to restore. Based on the extension of the file, different compression formats can be used (.tgz, .txz and .tzst, the latter only for Python>=3.14).

Returns

a PackageLists with the recipes and packages that have been restored to the cache

get_backup_sources(*package_list=None*, *exclude=True*, *only_upload=True*)

Get list of backup source files currently present in the cache, either all of them if no argument, or filtered by those belonging to the references in the package_list

Parameters

- **package_list** – a PackagesList object to filter backup files from (The files should have been downloaded form any of the references in the package_list)
- **exclude** – if True, exclude the sources that come from URLs present the core.sources:exclude_urls global conf
- **only_upload** – if True, only take into account the recipes and packages of the package_list that are set to be uploaded. Not used if no package_list is given

Returns

A list with the paths of the backup source files and their associated json files

Command API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class CommandAPI(*conan_api*)

This CommandAPI is useful to be able to launch full commands from the ConanAPI

Sometimes some commands are built using several calls to the ConanAPI. If we want to reuse the same functionality, then we would have to copy all that code into our own commands. Instead of doing that, it is possible to call Conan commands using this API, via the `run()` method.

run(*cmd*, *raise_on_errors=True*)

Runs another Conan command via API

Parameters

- **cmd** – Conan command to run. It can be either a string, or a list of strings.
- **raise_on_errors** – If True, it will raise an exception on errors. By default, some command will return errors as `conan_error` entries in the result, and it will be the caller's responsibility to check for them and raise/report them if desired.

Returns

It will return what that command returns. Note that different commands can return different things, so the caller needs to process it accordingly.

Note: When consuming this sub-API from a standalone script where the `ConanAPI` class is instantiated manually (which is not the recommended way to use the Python API, and *custom commands* should be preferred), an extra step is required, because the `ConanAPI` class does not automatically set the CLI interface needed for this sub-API to work properly. In that case, the following code snippet should be used:

```
from conan.api.conan_api import ConanAPI
from conan.cli.cli import Cli

api = ConanAPI()
cli = Cli(api)
cli.add_commands()
result = api.command.run(["remote", "list"])
```

The `Cli` class should be treated as a private opaque object, and its usage limited to the above snippet.

Config API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class ConfigAPI(*conan_api*, *helpers*)

This API provides methods to manage the Conan configuration in the Conan home folder. It allows installing configurations from various sources, retrieving global configuration values, and listing available configurations. It also provides methods to clean the Conan home folder, resetting it to a clean state.

home()

return the current Conan home folder containing the configuration files like remotes, settings, profiles, and the packages cache. It is provided for debugging purposes. Recall that it is not allowed to write, modify or remove packages in the packages cache, and that to automate tasks that uses packages from the cache Conan provides mechanisms like deployers or custom commands.

install(*path_or_url: str*, *verify_ssl*, *config_type=None*, *args=None*, *source_folder=None*, *target_folder=None*) → None

install Conan configuration from a git repo, from a zip file in an http server or a local folder

Calling this method will cause a reinitilization of the full ConanAPI, with possible invalidation of cached information, and references to objects from the ConanAPI might become dangling or outdated.

Parameters

- **path_or_url** – path or url to install. It can be a http://…/somefile.zip, a git repository URL, or a local folder
- **verify_ssl** – Argument passed to python-requests library for SSL verification
- **config_type** – type of configuration to install: “git”, “dir”, “file”, “url”
- **args** – additional arguments to pass to git repositories cloning
- **source_folder** – If specified, install files from that folder of the origin only
- **target_folder** – If the files are to be installed in a specific folder in the Conan home. For example, if it is desired to install only profiles from a configuration and using source_folder=”profiles”, it might be expected to use target_folder=”profiles” to keep the correct profile files location in the local home.

install_package(*require*, *lockfile=None*, *force=False*, *remotes=None*, *profile=None*)

install Conan configuration from a Conan package

Calling this method will cause a reinitilization of the full ConanAPI, with possible invalidation of cached information, and references to objects from the ConanAPI might become dangling or outdated.

Parameters

- **require** – The package requirement to be installed. It can contain version range expressions. If the revision is not specified, as a recipe `requires`, it will also resolve to the latest recipe-revision

- **lockfile** – Lockfile to be used to constrain and lock the versions and recipe-revisions from the input requirements, to the exact versions and revisions specified in the lockfile
- **force** – If the package has already been installed, nothing will be done unless force is True
- **remotes** – Remotes to look for the configuration package
- **profile** – If specified, use that profile to resolve for profile-specific different configurations, like depending on different settings.

Returns
list of RecipeReferences of all the configuration packages installed in the Conan home after this operation, including the previously installed ones

install_conanconfig(*path*, *lockfile=None*, *force=False*, *remotes=None*, *profile=None*)

install Conan configuration from a Conan "conanconfig.yml" file

Calling this method will cause a reinitilization of the full ConanAPI, with possible invalidation of cached information, and references to objects from the ConanAPI might become dangling or outdated.

Parameters

- **path** – Path to the conanconfig.yml file containing the configuration packages requirement definitions
- **lockfile** – Lockfile to be used to constrain and lock the versions and recipe-revisions from the input requirements, to the exact versions and revisions specified in the lockfile
- **force** – If the package has already been installed, nothing will be done unless force is True
- **remotes** – Remotes to look for the configuration package
- **profile** – If specified, use that profile to resolve for profile-specific different configurations, like depending on different settings.

Returns
list of RecipeReferences of all the configuration packages installed in the Conan home after this operation, including the previously installed ones

fetch_packages(*requires*, *lockfile=None*, *remotes=None*, *profile=None*)

get and download configuration packages into the Conan cache, without installing such configuration in the current Conan home.

This shouldn't be necessary for regular Conan configuration, and used at the moment exclusively for the "conan lock upgrade-config" experimental command.

get(*name*, *default=None*, *check_type=None*)

get the value of a global.conf item

Parameters

- **name** – configuration value to return
- **default** – default value to return if the configuration doesn't contain a value
- **check_type** – check if value is of type check_type, only if the value is defined

show(*pattern*) → dict

get the values of global.conf for those configurations that matches the pattern that have an actual user definition.

Values with no user definitions will be skipped from the returned value, defaults for those confs won't be shown.

Parameters
pattern – pattern to match against

Returns
dict of configuration values

`static conf_list()` → dict

list all the available built-in configurations

Returns
A sorted dictionary with all possible built-in configurations

`clean()` → None

reset the Conan home folder to a clean state, removing all the user custom configuration, custom files, and resetting modified files

`property settings_yml`

Get the contents of the settings.yml and user_settings.yml files, which define the possible values for settings.

Note that this is different from the settings present in a conanfile, which represent the actual values for a specific package, while this property represents the possible values for each setting.

This is intended to be a **read-only** value, do not try to attempt to modify, inject or remove settings with this attribute.

Returns
A read-only object representing the settings scheme, with a `possible_values()` method that returns a dictionary with the possible values for each setting, and a `fields` property that returns an alphabetically sorted list with the names of the settings. Note that it's possible to access nested settings using attribute access, such as `settings_yml.compiler.possible_values()`.

Download API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

`class DownloadAPI(conan_api, api_helpers)`

This API is used to download recipes and packages from a remote server.

`recipe(ref: RecipeReference, remote: Remote, metadata: List[str] | None = None)`

Download the recipe specified in the ref from the remote. If the recipe is already in the cache it will be skipped, but the specified metadata will be downloaded.

`package(pref: PkgReference, remote: Remote, metadata: List[str] | None = None)`

Download the package specified in the pref from the remote. The recipe for this package binary must already exist in the cache. If the package is already in the cache it will be skipped, but the specified metadata will be downloaded.

recipe_metadata(*ref:* RecipeReference, *remote:* Remote, *metadata: List[str]*, *folder: str*)

Download only the recipe metadata files into `folder`, without using the Conan cache. Files are always fetched from the server and written under `<folder>/metadata/`.

package_metadata(*pref:* PkgReference, *remote:* Remote, *metadata: List[str]*, *folder: str*)

Download only the package metadata files into `folder`, without using the Conan cache. Files are always fetched from the server and written under `<folder>/metadata/`.

download_full(*package_list:* PackagesList, *remote:* Remote, *metadata: List[str] | None = None*)

Download the recipes and packages specified in the `package_list` from the remote, parallelized based on `core.download:parallel`

Export API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class ExportAPI(*conan_api*, *helpers*)

This API provides methods to export artifacts, both recipes and pre-compiled package binaries from user folders to the Conan cache, as Conan recipes and Conan package binaries

export(*path*, *name: str = None*, *version: str = None*, *user: str = None*, *channel: str = None*, *lockfile=None*, *remotes: List[*Remote*] = None*) → Tuple[*RecipeReference*, ConanFile]

Exports a `conanfile.py` recipe, together with its associated files to the Conan cache. A "recipe-revision" will be computed and assigned.

Parameters

- **path** – Path to the conanfile to be exported
- **name** – Optional package name. Typically not necessary as it is defined by the recipe attribute or dynamically with the `set_name()` method. If it is defined in recipe and as an argument, but they don't match, an error will be raised.
- **version** – Optional version. It can be defined in the recipe with the version attribute or dynamically with the 'set_version()' method. If it is defined in recipe and as an argument, but they don't match, an error will be raised.
- **user** – Optional user. Can be defined by recipe attribute. If it is defined in recipe and as an argument, but they don't match, an error will be raised.
- **channel** – Optional channel. Can be defined by recipe attribute. If it is defined in recipe and as an argument, but they don't match, an error will be raised.
- **lockfile** – Optional, only relevant if the recipe has 'python-requires' to be locked
- **remotes** – Optional, only relevant to resolve 'python-requires' in remotes

Returns

A tuple of the exported RecipeReference and a ConanFile object

export_pkg_graph(*path*, *ref:* RecipeReference, *profile_host*, *profile_build*, *remotes: List[*Remote*]*, *lockfile=None*, *is_build_require=False*, *skip_binaries=False*, *output_folder=None*)

Computes a dependency graph for a given configuration, for an already existing (previously exported) recipe in the Conan cache. This method computes the full dependency graph, using the profiles, lockfile and remotes information as any other install/graph/create command. This is necessary in order to compute the "package_id" of the binary being exported into the Conan cache. The resulting dependency graph can be passed to `export_pkg()` method

Parameters

- **path** – Path to the conanfile.py in the user folder
- **ref** – full RecipeReference, including recipe-revision
- **profile_host** – Profile for the host context
- **profile_build** – Profile for the build context
- **lockfile** – Optional lockfile
- **remotes** – List of Remotes
- **is_build_require** – In case a package intended to be used as a tool-requires
- **skip_binaries** –
- **output_folder** – The folder containing output files, like potential environment scripts

Returns

A Graph object that can be passed to `export_pkg()` method

export_pkg(*graph*, *output_folder=None*) → None

Executes the `package()` method of the exported recipe in order to copy the artifacts from user folder to the Conan cache package folder

Parameters

- **graph** – A Graph object
- **output_folder** – Optional folder where generated files like environment scripts of dependencies have been installed

Graph API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

Note: Most methods below consume or produce a dependency graph object. This object is *documented as opaque*: unless explicitly shown otherwise in the documentation of a specific API method, it must not be inspected or modified directly, and `serialize()` is its only method guaranteed to remain stable.

class GraphAPI(*conan_api*, *helpers*)

load_root_test_conanfile(*path*, *tested_reference*, *profile_host*, *profile_build*, *update=None*, *remotes=None*, *lockfile=None*, *tested_python_requires=None*)

Create and initialize a root node from a test_package/conanfile.py consumer

Parameters

- **tested_python_requires** – the reference of the python_require to be tested
- **lockfile** – Might be good to lock python-requires, build-requires
- **path** – The full path to the test_package/conanfile.py being used
- **tested_reference** – The full RecipeReference of the tested package
- **profile_host** –
- **profile_build** –
- **update** –
- **remotes** –

Returns

a graph Node, recipe=RECIPE_CONSUMER

load_graph(*root_node*, *profile_host*, *profile_build*, *lockfile=None*, *remotes=None*, *update=None*, *check_update=False*)

Compute the dependency graph, starting from a root package, evaluation the graph with the provided configuration in profile_build, and profile_host. The resulting graph is a graph of recipes, but packages are not computed yet (package_ids) will be empty in the result. The result might have errors, like version or configuration conflicts, but it is still possible to inspect it. Only trying to install such graph will fail

Parameters

- **root_node** – the starting point, an already initialized Node structure, as returned by *load_root_test_conanfile()*
- **profile_host** – The host profile
- **profile_build** – The build profile
- **lockfile** – A valid lockfile (None by default, means no locked)
- **remotes** – list of remotes we want to check
- **update** – (False by default), if Conan should look for newer versions or revisions for already existing recipes in the Conan cache
- **check_update** – For "graph info" command, check if there are recipe updates

analyze_binaries(*graph*, *build_mode=None*, *remotes=None*, *update=None*, *lockfile=None*, *build_modes_test=None*, *tested_graph=None*)

Given a dependency graph, will compute the package_ids of all recipes in the graph, and evaluate if they should be built from sources, downloaded from a remote server, of if the packages are already in the local Conan cache

Parameters

- **lockfile** –
- **graph** – a Conan dependency graph, as returned by "load_graph()"
- **build_mode** – TODO: Discuss if this should be a BuildMode object or list of arguments

- **remotes** – list of remotes
- **update** – (`False` by default), if Conan should look for newer versions or revisions for already existing recipes in the Conan cache. It also accepts an array of reference patterns to limit the update to those references if any of the items match. Eg. `False`, `None` or `[]` *means no update*, `True` or `["*"]` *means update all*, and `["pkgA/*", "pkgB/1.0@user/channel"]` *means to update only specific packages*.
- **build_modes_test** – the –build-test argument
- **tested_graph** – In case of a "test_package", the graph being tested

static find_first_missing_binary(*graph*, *missing=None*)

(Experimental) Given a dependency graph, will return the first node with a missing binary package

Install API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class InstallAPI(*conan_api*, *helpers*)

This is the InstallAPI.

It provides methods to install binaries, sources, prepare the consumer folder with generators and deploy, etc., all of them based on an already resolved dependency graph.

install_binaries(*deps_graph*, *remotes: List[*Remote*] = None*, *return_install_error=False*)

Install binaries of a dependency graph.

This is the equivalent to the `conan install` command, but working with an already resolved dependency graph, usually obtained from the corresponding `GraphAPI` methods.

It will download the available packages from the given remotes, and then build the ones that were marked for build from source.

System requirements will be installed as well, taking into account the `tools.system.package_manager:mode` conf to determine whether to install, check or skip them.

Parameters

- **deps_graph** – Dependency graph to install packages for
- **remotes** – List of remotes to fetch packages from if necessary.
- **return_install_error** – If `True`, do not raise an exception, but return it

install_system_requires(*graph*, *only_info=False*)

Install only the system requirements of a dependency graph.

This is a subset of `install_binaries` which only deals with system requirements of an already resolved dependency graph, usually obtained from the corresponding `GraphAPI` methods.

The `tools.system.package_manager:mode` conf will be taken into account to determine whether to install, check or skip system requirements.

Parameters

- **graph** – Dependency graph to install system requirements for
- **only_info** – If `True`, only reporting and checking of whether the system requirements are installed is performed.

install_sources(*graph*, *remotes: List[*Remote*]*)

Download sources in the given dependency graph.

If the `tools.build:download_source` conf is `True`, sources will be downloaded for every package in the graph, otherwise only the packages marked for build from source will have their sources downloaded.

`tools.build:download_source=True` is useful when users want to inspect the source code of all dependencies, even the ones that are not built from source.

After this method, the `conanfile.source_folder` on each node of the dependency graph for which the sources have been downloaded will be set to the folder where sources have been downloaded.

Parameters

- **remotes** – List of remotes where the `exports_sources` of the packages might be located
- **graph** – Dependency graph to download sources from

install_consumer(*deps_graph*, *generators: List[str] = None*, *source_folder=None*, *output_folder=None*, *deploy=False*, *deploy_package: List[str] = None*, *deploy_folder=None*, *envs_generation=None*)

Prepare the folder of the root consumer of a dependency graph after installation of the dependencies.

This ensures that the requested generators are created in the consumer folder, and also handles deployment if requested.

Parameters

- **deps_graph** – Dependency graph whose root is the consumer we want to prepare
- **generators** – List of generators to be used in addition to the ones defined in the root conanfile, if any
- **source_folder** – Source folder of the consumer
- **output_folder** – Output folder of the consumer
- **deploy** – List of deployers to be used for deployment
- **deploy_package** – Only execute the `deploy()` method of the packages matching these patterns (`None` or empty to not execute any)
- **deploy_folder** – Folder where to deploy, by default the build folder
- **envs_generation** – If `None` (default), the virtual environment files of the root conanfile are generated. Any other value disables their generation.

deploy(*graph*, *deployer: List[str]*, *deploy_package: List[str] = None*, *deploy_folder=None*) → None

Run the given deployer in the dependency graph.

No checks are performed in the graph, it is assumed to be already resolved and in a valid state to be deployed from.

Parameters

- **graph** – The dependency graph to deploy
- **deployer** – List of deployers to be used

- **deploy_package** – Only execute the `deploy()` method of the packages matching these patterns (`None` or empty to not execute any)
- **deploy_folder** – Folder where to deploy. It must be provided

List API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class ListAPI(*conan_api*, *api_helpers*)

Get references from the recipes and packages in the cache or a remote

latest_recipe_revision(*ref:* RecipeReference, *remote:* Remote = *None*)

For a given recipe reference, return the latest revision of the recipe in the remote, or in the local cache if no remote is specified.

Raises if the reference is not found in the remote or cache.

recipe_revisions(*ref:* RecipeReference, *remote:* Remote = *None*)

For a given recipe reference, return all the revisions of the recipe in the remote, or in the local cache if no remote is specified

select(*pattern:* ListPattern, *package_query=None*, *remote:* Remote = *None*, *lru=None*, *profile=None*) → *PackagesList*

For a given pattern, return a list of recipes and packages matching the provided filters.

Parameters

- **pattern** (`ListPattern`) – Search criteria
- **package_query** (*str*) – When returning packages, expression of the form `"os=Windows AND (arch=x86 OR compiler=gcc)"` to filter packages by. If `None`, all packages will be returned if requested.
- **remote** (`Remote`) – Remote to search in, if `None`, it will search in the local cache.
- **lru** (*str*) – If set, it will filter the results to only include packages/binaries that have **not** been used in the last 'lru' time. It can be a string like `"2d"` (2 days) or `"3h"` (3 hours).
- **profile** (*Profile*) – Profile to filter the packages by settings and options.

explain_missing_binaries(*ref*, *conaninfo*, *remotes*)

(Experimental) Explain why a binary is missing in the cache

find_remotes(*package_list*, *remotes*)

(Experimental) Find the remotes where the current package lists can be found

Local API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class LocalAPI(*conan_api*, *helpers*)

This `LocalAPI` contains several helpers related to the local development flow, i.e., locally calling `source()` or `build()` methods, or adding and removing editable packages

static get_conanfile_path(*path*, *cwd*, *py*)

Obtain the full path to a conanfile file, either .txt or .py, from the current working directory.

If `path` is a folder containing both `conanfile.py` and `conanfile.txt`, and `py` is `False`, it will raise an error.

Parameters

- **path** – Relative path to look for the file. Can be a folder or a file.
- **cwd** – The current working directory.
- **py** – If True, a conanfile.py must exist, a .txt is not valid in this case

editable_add(*path*, *name=None*, *version=None*, *user=None*, *channel=None*, *cwd=None*, *output_folder=None*, *remotes: List[*Remote*] = None*) → *RecipeReference*

Add the conanfile in the given path as an editable package

Note that for automation over editables it might be recommended to use the `WorkspaceAPI` instead of this API.

Parameters

- **path** – Relative path to look for it. Can be a folder or a file.
- **name** – The name of the package. If not defined, it is taken from conanfile
- **version** – The version of the package. If not defined, it is taken from conanfile
- **user** – The user of the package. If not defined, it is taken from conanfile
- **channel** – The channel of the package. If not defined, it is taken from conanfile
- **cwd** – The current working directory
- **output_folder** – The output folder. If not defined, the recipe layout will be used.
- **remotes** – The remotes to resolve possible `python-requires` for this recipe if needed.

Returns

RecipeReference of the added package

editable_remove(*path=None*, *requires=None*, *cwd=None*)

Remove the editable packages matching the given path or requirements

Note that for automation over editables it might be recommended to use the `WorkspaceAPI` instead of this API.

Parameters

- **path** – Relative path to the folder containing the conanfile.py of the editable package to remove
- **requires** – Remove the editables matching these reference patterns (instead of by path)
- **cwd** – The current working directory

Returns
A dict whose keys are the RecipeReference of the removed packages

source(*path*, *name=None*, *version=None*, *user=None*, *channel=None*, *remotes: List[*Remote*] = None*, *lockfile=None*)

Calls the `source()` method of the current (user folder) `conanfile.py`

This method does not require computing a dependency graph, because the `source()` method is assumed to be invariant with respect to settings, options and dependencies.

Parameters

- **path** – Path to the conanfile.py file, like the one returned by `get_conanfile_path()`
- **name** – The name of the package. If not defined, it is taken from conanfile
- **version** – The version of the package. If not defined, it is taken from conanfile
- **user** – The user of the package. If not defined, it is taken from conanfile
- **channel** – The channel of the package. If not defined, it is taken from conanfile
- **remotes** – The remotes to resolve possible `python-requires` for this recipe if needed.
- **lockfile** – The lockfile to use for the `python-requires` resolution, if needed.

build(*conanfile*) → None

Calls the `build()` method of the current (user folder) `conanfile.py`

This method does require computing a dependency graph, because the `build()` method needs all dependencies and transitive dependencies. Then, the `conanfile` argument must be the one obtaind from a full dependency graph install operation, including both the graph comptutation and the binary installation.

Parameters
conanfile – `Conanfile` object representing the "root" node in the dependency graph, corresponding to a `conanfile.py` in the user folder, containing the `build()` method to be called. This `conanfile` object must have all of its dependencies computed and installed in the current Conan package cache to work.

static test(*conanfile*) → None

Calls the `test()` method of the current (user folder) `test_package/conanfile.py`

This method does require computing a dependency graph, because the `test()` method needs all dependencies and transitive dependencies. Then, the `conanfile` argument must be the one obtaind from a full dependency graph install operation, including both the graph comptutation and the binary installation.

Typically called after a `build()` one.

Parameters
conanfile – `Conanfile` object representing the "root" node in the dependency graph, corresponding to a conanfile.py in the user "test_package" folder, containing the `test()` method to be called. This `conanfile` object must have all of its dependencies computed and installed in the current Conan package cache to work.

Lockfile API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class LockfileAPI(*conan_api*)

Loads and saves Lockfiles from disk, modifies and manipulates them.

At the moment Lockfile objects are "opaque" objects, they are not intended to be used independently, only to be retrieved from this API and passed as arguments to methods of the API.

static get_lockfile(*lockfile=None*, *conanfile_path=None*, *cwd=None*, *partial=False*, *overrides=None*) → Lockfile

obtain a lockfile, following this logic:

If lockfile is explicitly defined, it would be either absolute or relative to cwd and the lockfile file must exist. If lockfile="" (empty string) the default "conan.lock" lockfile will not be automatically used even if it is present.

If lockfile is not defined, it will still look for a default conan.lock:

- if conanfile_path is defined, it will be besides it
- if conanfile_path is not defined, the default conan.lock should be in cwd
- if the default conan.lock cannot be found, it is not an error

Parameters

- **partial** – If the obtained lockfile will allow partial resolving
- **cwd** – the current working dir, if None, os.getcwd() will be used
- **conanfile_path** – The full path to the conanfile, if existing
- **lockfile** – the name of the lockfile file
- **overrides** – Dictionary of overrides {overriden: [new_ref1, new_ref2]}

check_lockfile_config(*lockfile: Lockfile*)

Verify that installed configurations are aligned with lockfile config_requires.

Parameters
lockfile – The lockfile to check, can be None

update_lockfile_export(*lockfile*, *conanfile*, *ref*, *is_build_require=False*) → Lockfile

Update the lockfile or create a new one with the information resulting from a conan export operation, so the recently exported version and revision can be locked and prioritized.

Parameters

- **lockfile** – The lockfile to update. Can be None and a new lockfile will be created
- **conanfile** – The exported conanfile
- **ref** – The reference of the exported conanfile, including its recipe revision

- **is_build_require** – If True, the exported conanfile is for a tool used as tool_requires

Returns
The updated lockfile

static update_lockfile(*lockfile*, *graph*, *lock_packages=False*, *clean=False*) → Lockfile

Update the lockfile with information from the dependency graph

Parameters

- **lockfile** – The lockfile to update. It can be None, and a new lockfile will be created.
- **graph** – The dependency graph
- **lock_packages** – Unused, do not use or define it.
- **clean** – If true, completely clean the lockfile, computing a new lockfile from graph

static merge_lockfiles(*lockfiles*) → Lockfile

Merge multiple lockfiles into a single lockfile.

Parameters
lockfiles – list of paths to the lockfile files to merge

Returns
the merged lockfile

static add_lockfile(*lockfile=None*, *requires=None*, *build_requires=None*, *python_requires=None*, *config_requires=None*) → Lockfile

Add requires to a lockfile. If the lockfile doesn't exist, it will be created

Parameters

- **lockfile** – The lockfile to add to. Can be None.
- **requires** – The list of requirements to add. Can be None.
- **build_requires** – The list of build requirements to add. Can be None.
- **python_requires** – The list of Python requirements to add. Can be None.
- **config_requires** – The list of configuration requirements to add. Can be None.

Returns
The lockfile with the added information.

static remove_lockfile(*lockfile: Lockfile*, *requires=None*, *build_requires=None*, *python_requires=None*, *config_requires=None*) → Lockfile

Remove entries from lockfile

Parameters

- **lockfile** – The lockfile to remove entries. It will be mutated in place.
- **requires** – The list of requires to remove
- **build_requires** – The list of build requires to remove
- **python_requires** – The list of python_requires to remove
- **config_requires** – The list of config_requires to remove

Returns
The modified lockfile

static save_lockfile(*lockfile: Lockfile*, *lockfile_out*, *path=None*)

Save lockfile to disk

Parameters

- **lockfile** – The lockfile object to save
- **lockfile_out** – The output lockfile filename. If None, nothing will be saved
- **path** – The path of the output lockfile, if None, it will be cwd

New API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class NewAPI(*conan_api*)

save_template(*template: str*, *defines=None*, *output_folder=None*, *force=False*)

Save the `template` files in the `output_folder`, replacing the template variables with the `defines`

Parameters

- **template** – The template to use: a path to a folder containing the template files (absolute or relative to the current working directory), or the name of a template, either those available under `<conan_home>/templates/command/new/<template>` or the built-in ones
- **defines** – A list with the `k=v` variables to replace in the template
- **output_folder** – The folder where the template files will be saved, cwd if `None`
- **force** – If `True`, overwrite the files if they already exist, otherwise raise an error

Profiles API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class ProfilesAPI(*conan_api*, *api_helpers*)

This ProfilesAPI is used to list, manage and load Conan profiles

`get_default_host()`

Returns
the path to the default "host" profile, either in the cache or as defined by the user in configuration

`get_default_build()`

Returns
the path to the default "build" profile, either in the cache or as defined by the user in configuration

`get_profile(profiles, settings=None, options=None, conf=None, cwd=None, context=None)`

Computes a Profile as the result of aggregating all the user arguments, first it loads the "profiles", composing them in order (last profile has priority), and finally adding the individual settings, options (priority over the profiles)

Parameters

- **profiles** – the list of profiles to load
- **settings** – list of "key=value" settings to define the profile. Patterns allowed as "pkg-pattern:key=value"
- **options** – list of "key=value" options. Patterns allowed as "pkg-pattern:key=value"
- **conf** – list of "key=value" configurations. Following "conf" definitions, patterns are allowed as "pkg-pattern:key=value", values that are lists or dictionaries might be allowed, and configuration operations like += for appending are allowed.
- **cwd** – the current working directory. If None, os.getcwd() will be used.
- **context** – the context, "build" or "host" to which this profile belongs

`get_path(profile, cwd=None, exists=True)`

Returns
the resolved path of the given profile name, that could be in the cache, or local, depending on the "cwd"

`list()`

List all the profiles files in the cache

Returns
an alphabetically ordered list of profile files in the default cache location

`static detect()`

Detects a possible default profile.

The output of this detection is not guaranteed to be complete or stable, it might change in future releases, following the same rules as the "conan profile detect" command.

Returns
an automatically detected Profile, with a "best guess" of the system settings

Remotes API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class RemotesAPI(*conan_api*, *api_helpers*)

The `RemotesAPI` manages the definition of remotes, contained in the "remotes.json" file in the Conan home, supporting addition, removal, update, rename, enable, disable of remotes. These operations do not contact the servers or check their existence at all. If they are not available, they will fail later when used.

The `user_xxx` methods perform authentication related tasks, and some of them will contact the servers to perform such authentication

list(*pattern=None*, *only_enabled=True*)

Obtain a list of *Remote* objects matching the pattern.

Parameters

- **pattern** – `None`, single `str` or list of `str`. If it is `None`, all remotes will be returned (equivalent to `pattern="*"`).
- **only_enabled** – boolean, by default return only enabled remotes

Returns
A list of *Remote* objects

disable(*pattern*)

Disable all remotes matching `pattern`

Parameters
pattern – single `str` or list of `str`. If the pattern is an exact name without wildcards like "*" and no remote is found matching that exact name, it will raise an error.

Returns
the list of disabled *Remote* objects (even if they were already disabled)

enable(*pattern*)

Enable all remotes matching `pattern`.

Parameters
pattern – single `str` or list of `str`. If the pattern is an exact name without wildcards like "*" and no remote is found matching that exact name, it will raise an error.

Returns
the list of enabled *Remote* objects (even if they were already enabled)

get(*remote_name*)

Obtain a *Remote* object

Parameters
remote_name – the exact name of the remote to be returned

Returns
the *Remote* object, or raise an Exception if the remote does not exist.

add(*remote:* Remote, *force=False*, *index=None*)

Add a new *Remote* object to the existing ones

Parameters

- **remote** – a *Remote* object to be added
- **force** – do not fail if the remote already exist (but default it fails)
- **index** – if not defined, the new remote will be last one. Pass an integer to insert the remote in that position instead of the last one

remove(*pattern*)

Remove the remotes matching the `pattern`

Parameters

pattern – single `str` or list of `str`. If the pattern is an exact name without wildcards like "*" and no remote is found matching that exact name, it will raise an error.

Returns

The list of removed *Remote* objects

update(*remote_name: str*, *url=None*, *secure=None*, *disabled=None*, *index=None*, *allowed_packages=None*, *recipes_only=None*, *force_auth=None*)

Update an existing remote

Parameters

- **remote_name** – The name of the remote to update, must exist
- **url** – optional url to update, if not defined it will not be updated
- **secure** – optional ssl secure connection to update
- **disabled** – optional disabled state
- **index** – optional integer to change the order of the remote
- **allowed_packages** – optional list of packages allowed from this remote
- **recipes_only** – optional boolean to only allow recipe downloads from this remote, never package binaries
- **force_auth** – optional boolean to force Conan to skip anonymous access and go directly for authenticated credentials against this remote

rename(*remote_name: str*, *new_name: str*)

Change the name of an existing remote

Parameters

- **remote_name** – The previous existing name
- **new_name** – The new name

user_login(*remote:* Remote, *username: str*, *password: str*)

Perform user authentication against the given remote with the provided username and password

Parameters

- **remote** – a *Remote* object
- **username** – the user login as `str`
- **password** – password `str`

user_logout(*remote:* Remote)

Logout from the given *Remote*

Parameters

remote – The *Remote* object to logout

Remove API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class RemoveAPI(*conan_api*, *api_helpers*)

This API is used to remove artifacts from either remotes or the Conan cache. It can either remove specific package references, or whole recipe references with all its associated packages

recipe(*ref:* RecipeReference, *remote:* Remote | *None* = *None*)

Removes the specified recipe reference alongside all its associated packages.

If `remote` is specified, the recipe will be removed from the remote, otherwise they will be removed from the local cache.

Parameters

- **ref** – Recipe reference to remove
- **remote** – Optional remote to remove references from

recipes(*refs: List[*RecipeReference*]*, *remote:* Remote | *None* = *None*)

Removes the specified recipe reference alongside all its associated packages.

If `remote` is specified, the packages will be removed from the remote, otherwise they will be removed from the local cache.

Warning:

This method is not atomic wit respect to each of the given references

Parameters

- **refs** – List of recipe references to delete, must contain recipe revisions
- **remote** – Optional remote to remove references from

package(*pref:* PkgReference, *remote:* Remote | *None* = *None*)

Removes the specified package reference.

If `remote` is specified, the packages will be removed from the remote, otherwise they will be removed from the local cache.

Parameters

- **pref** – Package reference to remove
- **remote** – Optional remote to remove references from

packages(*prefs: List[PkgReference]*, *remote:* Remote | *None = None*)

Removes all the specified package references.

If `remote` is specified, the packages will be removed from the remote, otherwise they will be removed from the local cache.

Warning:

This method is not atomic when performed in the local cache with respect to each of the given references, nor are remotes guaranteed to implement this call atomically either.

Parameters

- **prefs** – List of package references to delete, must contain package revisions
- **remote** – Optional remote to remove references from

Report API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class ReportAPI(*conan_api*, *helpers*)

Used to compute the differences (the "diff") between two versions or revisions, for both the recipe and source code.

diff(*old_reference*, *new_reference*, *remotes*, *old_path=None*, *new_path=None*, *cwd=None*)

Compare two recipes and return the differences.

Parameters

- **old_reference** – The reference of the old recipe.
- **new_reference** – The reference of the new recipe.
- **remotes** – List of remotes to search for the recipes.
- **old_path** – Optional path to the old recipe's conanfile.py.
- **new_path** – Optional path to the new recipe's conanfile.py.
- **cwd** – Current working directory, used to resolve paths.

Returns

A dictionary with the differences between the two recipes.

Upload API

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

class UploadAPI(*conan_api*, *api_helpers*)

This API is used to upload recipes and packages to a remote server.

check_upstream(*package_list: PackagesList*, *remote: Remote*, *enabled_remotes: List[Remote]*, *force=False*)

Checks `remote` for the existence of the recipes and packages in `package_list`. An `upload` key will be added to each entry, with the value `True` for the items that are not present in the remote and need to be uploaded, and `False` otherwise.

If the recipe has an upload policy of `skip`, its packages will be discarded from the upload list, and only the recipe will be considered.

Parameters

- **package_list** – A `PackagesList` object with the recipes and packages to check.
- **remote** – Remote to check.
- **enabled_remotes** – List of enabled remotes. This is used to possibly load python_requires from the listed recipes if necessary.
- **force** – If `True`, the items already present in the remote will also be marked to be uploaded, adding a `force_upload` key with the value `True` to their entries.

prepare(*package_list: PackagesList*, *enabled_remotes: List[Remote]*, *metadata: List[str] = None*)

Compress the recipes and packages and fill the `package_list` entries with the complete information of the files to upload. It doesn't perform the upload nor checks upstream to see if the recipe is still there

Parameters

- **package_list** – A PackagesList object with the recipes and packages to upload.
- **enabled_remotes** – A list of remotes that are enabled in the client. Recipe sources will attempt to be fetched from these remotes.
- **metadata** – A list of patterns of metadata that should be uploaded. Default `None` means all metadata will be uploaded together with the package artifacts. If metadata contains an empty string (`""`), it means that no metadata files should be uploaded.

upload_full(*package_list: PackagesList*, *remote: Remote*, *enabled_remotes: List[Remote]*, *check_integrity=False*, *force=False*, *metadata: List[str] = None*, *dry_run=False*, *is_prepared=False*)

Does the whole process of uploading, including the possibility of parallelizing per recipe based on the `core.upload:parallel` conf.

The steps that this method performs are:

- if `check_integrity` is `True`, calls `conan_api.cache.check_integrity` to ensure the packages are not corrupted

- **checks the upload policy of the recipes**
  - (if it is "skip", it will not upload the binaries, but will still upload the recipe)
- checks which revisions already exist in the server so that it can skip the upload
- prepares the artifacts to upload (compresses the conan_package.tgz)
- executes the actual upload
- uploads associated sources backups if any

Parameters

- **package_list** – A PackagesList object with the recipes and packages to upload.
- **remote** – The remote to upload the packages to.
- **enabled_remotes** – A list of remotes that are enabled in the client. Recipe sources will attempt to be fetched from these remotes, and to possibly load python_requires from the listed recipes if necessary.
- **check_integrity** – If True, it will check the integrity of the cache packages before uploading them. This is useful to ensure that the packages are not corrupted.
- **force** – If True, it will force the upload of the recipes and packages, even if they already exist in the remote. Note that this might update the timestamps
- **metadata** – A list of patterns of metadata that should be uploaded. Default None means all metadata will be uploaded together with the package artifacts. If metadata contains an empty string (""), it means that no metadata files should be uploaded.
- **dry_run** – If True, it will not perform the actual upload, but will still prepare the artifacts and check the upstream.

upload_backup_sources(*files: List*) → None

Upload to the server the backup sources files, that have been typically gathered by CacheAPI.get_backup_sources()

Parameters

files – The list of files that must be uploaded

Warning: Subapis **must not** be initialized by themselves. They are intended to be accessed only through the main *ConanAPI* attributes.

There are also some model classes that represent the data structures used in the API. Note that as with the API, only the **documented** public members are guaranteed to be stable, and the rest of the members are considered private and can change without notice.

Models

There are some model classes that represent the data structures used in the API. Note that as with the API, only the **documented** public members are guaranteed to be stable, and the rest of the members are considered private and can change without notice.

Remote model

class Remote(*name*, *url*, *verify_ssl=True*, *disabled=False*, *allowed_packages=None*, *remote_type=None*, *recipes_only=False*, *force_auth=False*)

The `Remote` class represents a remote registry of packages.

A Remote object can be constructed to be passed as an argument to RemotesAPI methods. When possible, it is better to use Remote objects returned by the API, but for the `RemotesAPI.add()` method, for which a new constructed object is necessary. It is recommended to use named arguments like `Remote(..., verify_ssl=False)` in the constructor.

Parameters

- **name** – The name of the remote.
- **url** – The URL of the remote repository (or local folder for "local-recipes-index").
- **verify_ssl** – Enable SSL Certificate validation.
- **disabled** – Disable the remote repository.
- **allowed_packages** – List of patterns of allowed packages from this remote
- **remote_type** – Type of the remote repository, use "local-recipes-index" or `None`
- **recipes_only** – If True, binaries form this remote will be ignored and never used
- **force_auth** – If True, Conan will not attempt anonymous access to this remote, going directly for the authenticated credentials instead.

invalidate_cache()

If external operations might have modified the remote since it was instantiated, this method can be called to invalidate the cache. Note that this is done automatically when the remote is used in any operation by Conan, such as uploading packages, so this method is not usually needed when only interacting with the Conan API

List classes

class PackagesList

A collection of recipes, revisions and packages.

split()

Returns a list of PackageList, split one per reference. This can be useful to parallelize things like upload, parallelizing per-reference

only_recipes() → None

Filter out all the packages and package revisions, keep only the recipes and recipe revisions in self._data.

add_ref(*ref:* RecipeReference) → None

Adds a new RecipeReference to a package list

add_pref(*pref:* PkgReference, *pkg_info: dict = None*) → None

Add a PkgReference to an already existing RecipeReference inside a package list

items() → Iterable[Tuple[*RecipeReference*, Dict[*PkgReference*, Dict]]]

Iterate over the contents of the package list.

Yields tuples containing a recipe reference and a dictionary of its associated package references content.

Returns:
An iterable of tuples where:

- The first element is a `RecipeReference` (representing the recipe revision).
- The second element is a dictionary mapping a `PkgReference` to a dictionary containing the package revision information.

Warning:
Missing Revisions Behavior: This method filters out results that lack revision information.

- It will ONLY yield items if they contain at least a **recipe revision**.
- The nested package dictionary will be empty unless it contains a **package revision**.

When to use serialize instead: If you perform a general search that does not fetch revisions (e.g., running `conan list *`), this method will yield nothing because no artifact references are created. In these cases, use the `serialize()` method to access the results.

To successfully use `items()`, your query must explicitly request revisions (e.g., running `conan list pkg/version#*:*#*`).

recipe_dict(*ref:* RecipeReference)

Gives read/write access to the dictionary containing a specific RecipeReference information.

package_dict(*pref:* PkgReference)

Gives read/write access to the dictionary containing a specific PkgReference information

serialize()

Serialize the instance to a dictionary.

static deserialize(*data*)

Loads the data from a serialized dictionary.

class MultiPackagesList

A collection of PackagesList by remote name.

serialize()

Serialize object to a dictionary.

static load(*file*)

Create an instance of the class from a serialized JSON file path pointed by `file`.

static load_graph(*graphfile*, *graph_recipes=None*, *graph_binaries=None*, *context=None*)

Create an instance of the class from a graph file path, which is the json format returned by a few commands like `conan graph info` or `conan create/install.`

Parameters

- **graphfile** (*str*) – Path to the graph file
- **graph_recipes** (*list[str]*) – List for kinds of recipes to return. For example `"cache"` will return only recipes in the local cache, `"download"` will return only recipes that have been downloaded, and passing `"*"` will return all recipes.

- **graph_binaries** (*list[str]*) – List for kinds of binaries to return. For example "cache" will return only binaries in the local cache, "download" will return only binaries that have been downloaded, "build" will return only binaries that are built, "missing" will return only binaries that are missing, "invalid" will return only binaries that are invalid, and passing "*" will return all binaries.
- **context** (*str*) – Context to filter the graph, can be "host", "build", "host-only" or "build-only"

class ListPattern(*expression*, *rrev='latest'*, *package_id=None*, *prev='latest'*, *only_recipe=False*)

Object holding a pattern that matches recipes, revisions and packages.

Parameters

- **expression** – The pattern to match, e.g. "name/*:*"
- **rrev** – The recipe revision to match, defaults to "latest", can also be "!latest" or "~latest" to match all but the latest revision, a pattern like "1234*" to match a specific revision, or a specific revision like "1234".
- **package_id** – The package ID to match, defaults to None, which matches all package IDs.
- **prev** – The package revision to match, defaults to "latest", can also be "!latest" or "~latest" to match all but the latest revision, a pattern like "1234*" to match a specific revision, or a specific revision like "1234".
- **only_recipe** – If True, only the recipe part of the expression is parsed, ignoring package_id and prev. This is useful for commands that only operate on recipes, like conan search.

Reference models

class RecipeReference(*name=None*, *version=None*, *user=None*, *channel=None*, *revision=None*, *timestamp=None*)

An exact (no version-range) reference of a recipe, it represents a reference of the form name/version[@user/channel][#revision][%timestamp]. Should be enough to locate a recipe in the cache or in a server, and validation will be external to this class, at specific points (export, api, etc).

The attributes should be regarded as immutable, and should not be modified by the user.

name: str

Name of the reference

version: *Version*

Version of the reference

user

User of the reference, if any

channel

Channel of the reference, if any

revision

Revision of the reference, if any

timestamp

Timestamp of the reference, if any

static loads(*rref*)

Instantiates an object from a string, in the form: `name/version[@user/channel][#revision][%timestamp]`

validate_ref(*allow_uppercase=False*)

Check that the reference is valid, and raise a `ConanException` if not.

matches(*pattern*, *is_consumer*)

fnmatches the reference against the provided pattern.

Parameters

- **pattern** (*str*) – the pattern to match against, it can contain wildcards, and can start with `!` or `~` to negate the match. A special value of `&` will return a match only of `is_consumer` is `True`
- **is_consumer** (*bool*) – if `True`, the pattern `&` will match this reference.

class PkgReference(*ref=None*, *package_id=None*, *revision=None*, *timestamp=None*)

An exact (no version-range) reference of a package binary, it represents a reference of the form `name/version[@user/channel][#revision][%timestamp]:package_id[#revision][%timestamp]`. Should be enough to locate a package binary in the cache or in a server

The attributes should be regarded as immutable, and should not be modified by the user.

ref

The base RecipeReference for this package reference

package_id

The unique identifier within this RecipeReference of a package binary

revision

The package revision of this binary for this specific package_id

timestamp

The timestamp of the current package revision, float in Unix seconds UTC

static loads(*pkg_ref*)

Instantiates a PkgReference object from a string. Does not implement validation of the members.

Dependency graph class

A dependency graph is an opaque object, only meant to be produced and consumed by the *GraphAPI* methods. It should not be created, inspected, or modified directly.

Unless explicitly shown otherwise in the documentation of a specific API method (for example, how to obtain the `conanfile` to pass to *LocalAPI.build()*), the only supported operation on it is calling its `serialize()` method, which returns a `dict` representation of the graph. That representation follows the same JSON schema documented for the *"conan graph info" JSON formatter*.

9.8.5 Deployers

Deployers are a mechanism to facilitate copying files from one folder, usually the Conan cache, to user folders. While Conan provides three built-in ones (`full_deploy`, `direct_deploy` and `runtime_deploy`), users can easily manage their own with `conan config install`.

Deployers run before generators, and they can change the target folders. For example, if the `--deployer=full_deploy` deployer runs before `CMakeDeps`, the files generated by `CMakeDeps` will point to the local copy in the user folder done by the `full_deploy` deployer, and not to the Conan cache. Multiple deployers can be specified by supplying more than one `--deployer=` argument, and they will be ran in order of appearance.

Deployers can be multi-configuration. Running `conan install . --deployer=full_deploy` repeatedly for different profiles can achieve a fully self-contained project, including all the artifacts, binaries, and build files. This project will be completely independent of Conan and no longer require it at all to build. Use the `--deployer-folder` argument to change the base folder output path for the deployer as desired.

Built-in deployers

Warning: The built-in deployers are in **preview**. See *the Conan stability* section for more information.

full_deploy

Deploys each package folder of every dependency to your recipe's `output_folder` in a subfolder tree based on:

1. The build context
2. The dependency name and version
3. The build type
4. The build arch

Then every dependency will end up in a folder such as:

`[OUTPUT_FOLDER]/full_deploy/host/dep/0.1/Release/x86_64`

See a full example of the usage of `full_deploy` deployer in *Creating a Conan-agnostic deploy of dependencies for developer use*.

Note: Editable packages and skipped binaries

- Dependencies whose `package_folder` is `None` (for example, skipped binaries that are not necessary) are not deployed.
- Dependencies in *editable mode* are **not** skipped. Their `package_folder` is the local project root folder, so the whole contents of that folder (sources, build outputs, etc.) will be copied to the deployed folder. This is generally harmless, but it is usually not what is wanted, and the general recommendation is to not deploy editable packages.
- If a different behavior is needed, such as skipping editables or raising an error, use a custom deployer. See *Customize the full_deploy deployer: handling editable packages* for the full implementation of `full_deploy` and how to adapt it.
- The built-in deployers are mostly a convenience and an example. For real production needs, the expectation is to provide custom deployers and install them with `conan config install` or `conan config install-pkg`.

direct_deploy

Same as `full_deploy`, but only processes your recipe's *direct* dependencies. This deployer will output your dependencies in a tree folder such as:

`[OUTPUT_FOLDER]/direct_deploy/dep`

runtime_deploy

New since Conan 2.5.0

Copies all shared libraries and executables from dependencies (such as .so, .dll, or .dylib files) into a flattened directory structure.

Since Conan 2.20.0, subdirectories are maintained and preserved as-is. Files are only included in environment generators when correctly specified through `cpp_info.bindirs` and `cpp_info.libdirs` configuration.

Note: `runtime_deploy` only copies dependencies whose requirement has the `run` trait set to `True`. A `static-library` dependency is skipped by default, even if it internally ships a shared library needed at runtime. To include such a dependency, declare `"run": True` inside *package_type_traits* in its recipe.

---

cyclone_1.6 and cyclone_1.4

The `cyclone_1.6` and `cyclone_1.4` deployers are available to generate a CycloneDX 1.6 or 1.4 SBOM file respectively, with the information of the dependencies in the graph. The generated file will be named `sbom-cyclonedx-1.{6, 4}.json` and it will be located in the deployer output folder. These deployers will use the `cyclonedx_1_6` and `cyclonedx_1_4` functions from the *conan.tools.sbom.cyclonedx* module with default arguments.

If you want to customize the generated SBOM, you can either create your own custom deployer that calls these functions with the desired arguments, or use the documented hook approach from the previous link to generate the SBOM.

configuration

The `full_deploy`, `direct_deploy` and `runtime_deploy` understand when the conf `tools.deployer:symlinks` is set to `False` to disable deployers copying symlinks. This can be convenient in systems that do not support symlinks and could fail if deploying packages that contain symlinks.

Custom deployers

Custom deployers can be managed via `conan config install`. When looking for a specific deployer, Conan will look in these locations for the deployer in the following order:

1. Absolute paths
2. Relative to cwd
3. In the `[CONAN_HOME]/extensions/deployers` folder
4. As built-in deployers

Conan will look for a `deploy()` method to call for each installed file. The function signature of your custom deployers should be as follows:

Listing 93: **my_custom_deployer.py**

```
def deploy(graph, output_folder: str, **kwargs):
```

(Note that the arguments are passed as named parameters, so both the `graph` and `output_folder` names are mandatory)

The `**kwargs` is mandatory even if not used, as new arguments can be added in future Conan versions, and those would break if `**kwargs` is not defined.

You can access your conanfile object with `graph.root.conanfile`. See *ConanFile.dependencies* for information on how to iterate over its dependencies.

If you need to run binaries from your build dependencies, the recommended approach is to apply the env from a `VirtualBuildEnv`, such as:

```
from conan.tools.env import VirtualBuildEnv

def deploy(graph, output_folder: str, **kwargs):
    venv = VirtualBuildEnv(graph.root.conanfile)
    with venv.vars().apply():
        self.run("mytool")
```

Your custom deployer can now be invoked as if it were a built-in deployer using the filename in which it's found, in this case `conan install . --deployer=my_custom_deployer`. Note that supplying the **.py** extension is optional.

See the *custom deployers* section for examples on how to implement your own deployers.

9.8.6 Hooks

The Conan hooks is a feature intended to extend the Conan functionalities to perform certain orthogonal operations, like some quality checks, in different stages of a package creation process, like pre-build and post-build.

Hook structure

A hook is a Python function that will be executed at certain points of Conan workflow to customize the client behavior without modifying the client sources or the recipe ones.

Here is an example of a simple hook:

Listing 94: *hook_example.py*

```
from conan.tools.files import load

def pre_export(conanfile):
    for field in ["url", "license", "description"]:
        field_value = getattr(conanfile, field, None)
        if not field_value:
            conanfile.output.error(f"[REQUIRED ATTRIBUTES] Conanfile doesn't have '
↪{field}'.
                                   It is recommended to add it as attribute.")
```

This hook checks the recipe content prior to it being exported. Basically the `pre_export()` function checks the attributes of the `conanfile` object to see if there is an URL, a license and a description and if missing, warns the user with a message through the `conanfile.output`. This is done **before** the recipe is exported to the local cache.

Any kind of Python script can be executed. You can create global functions and call them from different hook functions, import from a relative module and warn, error or even raise to abort the Conan client execution.

Importing from a module

The hook interface should always be placed inside a Python file with the name of the hook starting by *hook_* and with the extension *.py*. It also should be stored in the *<conan_home>/extensions/hooks* folder. However, you can use functionalities from imported modules if you have them installed in your system or if they are installed with Conan:

Listing 95: hook_example.py

```
import requests
from conan.tools.files import replace_in_file

def post_package(conanfile):
    if not os.path.isdir(os.path.join(conanfile.package_folder, "licenses")):
        response = requests.get('https://api.github.com/repos/company/repository/
↪contents/LICENSE')
```

You can also import functionalities from a relative module:

```
hooks
├── custom_module
│   ├── custom.py
│   └── __init__.py
└── hook_printer.py
```

Inside the *custom.py* from my *custom_module* there is:

Listing 96: custom.py

```
def my_printer(conanfile):
    conanfile.output.info("my_printer(): CUSTOM MODULE")
```

And it can be used in the hook importing the module, just like regular Python:

Listing 97: hook_printer.py

```
from custom_module.custom import my_printer

def pre_export(conanfile):
    my_printer(conanfile)
```

Hook interface

Here you can see a complete example of all the hook functions available:

Listing 98: *hook_full.py*

```
def pre_export(conanfile):
    conanfile.output.info("Running before to execute export() method.")

def post_export(conanfile):
```

(continues on next page)

(continued from previous page)

```
    conanfile.output.info("Running after of executing export() method.")

def pre_validate(conanfile):
    conanfile.output.info("Running before executing the validate() method.")

def post_validate(conanfile):
    conanfile.output.info("Running after executing the validate() method.")

def pre_source(conanfile):
    conanfile.output.info("Running before to execute source() method.")

def post_source(conanfile):
    conanfile.output.info("Running after of executing source() method.")

def pre_generate(conanfile):
    conanfile.output.info("Running before to execute generate() method.")

def post_generate(conanfile):
    conanfile.output.info("Running after of executing generate() method.")

def pre_build(conanfile):
    conanfile.output.info("Running before to execute build() method.")

def post_build(conanfile):
    conanfile.output.info("Running after of executing build() method.")

def post_build_fail(conanfile):
    conanfile.output.info("Running after failed execution of build() method.")

def pre_package(conanfile):
    conanfile.output.info("Running before to execute package() method.")

def post_package(conanfile):
    conanfile.output.info("Running after of executing package() method.")

def pre_package_info(conanfile):
    conanfile.output.info("Running before to execute package_info() method.")

def post_package_info(conanfile):
    conanfile.output.info("Running after of executing package_info() method.")

# Note that pre_package_id() hook doesn't exist yet, so far there hasn't been
# a use case
def post_package_id(conanfile):
    conanfile.output.info("Running after executing package_id() method.")
```

Functions of the hooks are intended to be self-descriptive regarding to the execution of them. For example, the `pre_package()` function is called just before the `package()` method of the recipe is executed.

All hook methods are filled only with the same single object:

- **conanfile**: It is a regular `ConanFile` object loaded from the recipe that received the Conan command. It has its normal attributes and dynamic objects such as `build_folder`, `package_folder`, `output`, `dependencies`, `options` ...

Storage, activation and sharing

Hooks are Python files stored under *<conan_home>/extensions/hooks* folder and **their file name should start with hook_ and end with the .py extension**.

The activation of the hooks is done automatically once the hook file is stored in the hook folder. In case storing in subfolders, it works automatically too.

To deactivate a hook, its file should be removed from the hook folder. There is no configuration which can deactivate but keep the file stored in hooks folder.

Official Hooks

There are some officially maintained hooks in its own repository in Conan hooks GitHub, but mostly are only compatible with Conan 1.x, so please, check first the README to have information which hooks are compatible with Conan v2.

9.8.7 Binary compatibility

This plugin, located in the cache `extensions/plugins/compatibility/compatibility.py` allows defining custom rules for the binary compatibility of packages across settings and options. It has some built-in logic implemented, but can be customized.

The interface is a single function called `def compatibility(conanfile)` that receives a single `conanfile` object as argument. Its return will be equal to the `compatibility()` recipe method, an ordered list of variations over `settings`, `options` that is considered to be binary compatible.

Note: Conan will check that list in order for binary existence until one binary is found.

The following would be valid syntax (but not an useful or working one, as it will fail in Windows, for example):

```
def compatibility(conanfile):
    result = []
    if conanfile.settings.build_type == "Debug":
        result.append({"settings": [("build_type", "Release")]})
    return result
```

Conan provides a default `compatibility.py` that implements binary compatibility for different `compiler.cppstd` and `compiler.cstd` values. That is, by default it assumes that binaries built with different `cppstd` and `cstd` values (for the same compiler and compiler version) are binary compatible, and can be linked together without issues.

The `compiler.cppstd` must be defined in profiles in most C++ scenarios. If a binary for a given `compiler.cppstd` value doesn't exist (that means, a binary built with exactly that setting), Conan default `compatibility.py` will iterate the supported `cppstd` values by that compiler version. It is possible to disable this behavior for any specific package, adding to the package `conanfile.py` recipe the `extension_properties = {"compatibility_cppstd": False}` attribute, read the *extension_properties docs*.

From Conan 2.4, the `compiler.cstd` setting is available. It will only be taken into account in the computation of packages `package_id` when their recipes explicitly declare the `languages = "C"` attribute.

There are some cases where the default `compatibility.py` will not be enough, and users will need to customize it to their needs. Some rules and tips for that customization are explained below.

- Importantly, the built-in `compatibility.py` **is subject to changes in future releases**. To avoid being updated in the future, please remove the first comment `# This file was generated by Conan.`

- Removing values from `settings` or `options` in the returned list is possible when they allow being unset, by specifying the value of the item to `None`.

Warning: The `compatibility.py` feature is in **preview**. The current default `compatibility.py` is **experimental**. See *the Conan stability* section for more information.

See also:

Read the *binary model reference* for a full view of the Conan binary model.

9.8.8 Profile plugin

The `profile.py` extension plugin is a Python script that receives one profile and allow checking and modifying it.

This plugin is located in the `extensions/plugins/profile.py` cache folder.

This `profile.py` contains a default implementation that does:

- Will try to define `compiler.runtime_type` for `msvc` and `clang` compilers (in Windows) if it is not defined, and it will define it to match the `settings.build_type`. That allow users to let it undefined in profiles, and switch it conveniently in command line just with `-s build_type=Debug`
- Will check the `compiler.cppstd` value if defined to validate if the current compiler version has support for it. For example, if a developer tries to use `-s compiler=gcc -s compiler.version=5 -s compiler.cppstd=20`, it will raise an error.
- Even though the `profile.py` plugin has some provision to handle `compiler.cstd` checks, they are not implemented yet, so the plugin will be permissive regarding definition errors, please make sure the `compiler.cstd` is actually supported by your compiler version.

Users can customize this `profile.py` and distribute it via `conan config install`, in that case, the first lines should be removed:

```
# This file was generated by Conan. Remove this comment if you edit this file or Conan
# will destroy your changes.
```

And `profile.py` should contain one function with the signature:

```
def profile_plugin(profile):
    settings = profile.settings
    print(settings)
```

When a profile is computed, it will display something like:

```
OrderedDict([('arch', 'x86_64'), ('build_type', 'Release'), ('compiler', 'msvc'), (
↪'compiler.cppstd', '14'), ('compiler.runtime', 'dynamic'), ('compiler.runtime_type',
↪'Release'), ('compiler.version', '192'), ('os', 'Windows')])
```

See also:

- See the documentation about the *Conan profiles*.

9.8.9 Authorization plugins

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Regarding authorization, we have two plugins: one focused on remote *Conan servers* authorization, `auth_remote.py`, and another focused on authorization for source file servers, `auth_source.py`.

The idea behind these plugins is to create custom integrations with each user's secrets managers.

Auth remote plugin

This first plugin is a Python script that receives a *Remote* object and an optional parameter: `user`. If the user is provided, the expected output is the credentials that use that username. The output should be a tuple of the username that we want to use for that remote, or `None` if no credentials are specified for that remote and we want Conan to follow the normal login flow.

This plugin is located at the path `<CONAN_HOME>/extensions/plugins/auth_remote.py` and must be manually created with the name `auth_remote.py`, containing a function named `auth_remote_plugin(remote, user=None, **kwargs)`.

The order for retrieving credentials is as follows:

- First, an attempt is made to obtain the credentials from the `auth_remote_plugin`.
- If it doesn't exist or returns `None`, the next step is to check `credentials.json`.
- After that, the environment variables are searched.
- Finally, the credentials are obtained through an interactive prompt.

Here we can see an example of a plugin implementation.

```
def auth_remote_plugin(remote, user=None, **kwargs):
    if remote.url.startswith("https://artifactory.my-org/"):
        return "admin", "password"
```

Auth source plugin

This one is a Python script that receives an `url` as a parameter and outputs a dictionary with the credentials or access token. It can also return `None` to indicate that Conan should proceed with its normal login flow.

This plugin is located at the path `<CONAN_HOME>/extensions/plugins/auth_source.py` and must be manually created with the name `auth_source.py`, containing a function named `auth_source_plugin(url, **kwargs)`.

The order for retrieving the credentials is as follows:

- First, an attempt is made to obtain the credentials from the `auth_source_plugin`.
- If it doesn't exist or returns `None`, an attempt is made to retrieve them from `source_credentials.json`.

Here we can see an example of a plugin implementation.

```
def auth_source_plugin(url, **kwargs):
    if url.startswith("https://my-sources-user-password.my-org/"):
        return {'user': 'my-user', 'password': 'my-password'}
```

(continues on next page)

(continued from previous page)

```
    elif url.startswith("https://my-private-token-sources.my-org/"):
        return {'token': 'my-secure-token'}
```

Additionally, returning a `headers` dictionary will add the contents as HTTP headers for the sources request, as some servers might need some specific custom headers:

```
def auth_source_plugin(url, **kwargs):
    if url.startswith("https://my-private-token-sources-with-headers.my-org/"):
        return {'token': 'my-secure-token', 'headers': {'my-header-1': 'my-value-1'}}
```

Note: These plugins can be shared and installed using `conan config install` or `conan config install-pkg`

Important: Ensure that your plugins and configurations do **not** contain hardcoded secrets or sensitive data. Instead, passwords should be retrieved using your implementation with a secret manager.

9.8.10 Command wrapper

The `cmd_wrapper.py` extension plugin is a Python script that receives the command line argument provided by `self.run()` recipe calls, and allows intercepting them and returning a new one.

This plugin must be located in the `extensions/plugins` cache folder, and can be installed with the `conan config install` command.

For example:

```
def cmd_wrapper(cmd, **kwargs):
    return 'echo "{}"'.format(cmd)
```

Would just intercept the commands and display them to terminal, which means that all commands in all recipes `self.run()` will not execute, but just be echoed.

The `**kwargs` is a mandatory generic argument to be robust against future changes and injection by Conan of new keyword arguments. Not adding it, even if not used could make the extension fail in future Conan versions.

A more common use case would be the injection of a parallelization tools over some commands, which could look like:

```
def cmd_wrapper(cmd, **kwargs):
    # lets parallelize only CMake invocations
    if cmd.startswith("cmake"):
        return 'parallel-build "{}"  --parallel-argument'.format(cmd)
    # otherwise return same command, not modified
    return cmd
```

The `conanfile` object is passed as an argument, so it is possible to customize the behavior depending on the caller:

```
def cmd_wrapper(cmd, conanfile, **kwargs):
    # Let's parallelize only CMake invocations, for a few specific heavy packages
    name = conanfile.ref.name
    heavy_pkgs = ["qt", "boost", "abseil", "opencv", "ffmpeg"]
    if cmd.startswith("cmake") and name in heavy_pkgs:
        return 'parallel-build "{}"  --parallel-argument'.format(cmd)
    # otherwise return same command, not modified
    return cmd
```

9.8.11 Package signing

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This plugin is a Conan extension mechanism to sign and verify packages. For example using Cosign (Sigstore), OpenSSL, GPG, etc.

This capability allows users to:

- Sign and verify packages stored in the local cache.
- Verify packages when installed.
- Upload signatures alongside packages to a remote server.

Configuration

To enable package signing, the plugin should be placed in the cache at `extensions/plugins/sign/sign.py`.

Implementation

The plugin is a Python file that must define the following two functions:

For signing packages

```
def sign(ref, artifacts_folder, signature_folder, **kwargs):
    """
    :param ref: The package reference being signed.
    :param artifacts_folder: Path to the folder containing the package artifacts␣
↪(conaninfo.txt, conan_package.tgz, etc).
    :param signature_folder: Path to the folder where signatures should be written␣
↪(metadata/sign).
    :return: A list of signature dictionaries with their signing method and provider and␣
↪the signing artifacts generated that are included in the package
    """
    # Logic to compute signatures of files in artifacts_folder
    # and write them to signature_folder.
    #
    # The signature_folder should only store the files that are part of the signature.
    # These files will be uploaded with the package and downloaded when
    # it is installed (usually the metadata is not downloaded by default,
    # but the metadata/sign folder is) so the signature verification
    # is always done.
    #
    return [
      {
          "method": "openssl-dgst",  # The signing method indicates the tool used to␣
↪sign the package so it can be verified later
          "provider": "my-organization",  # The signing provider indicates the␣
↪organization that signed the package, so the correct public key is used for␣
```

(continues on next page)

(continued from previous page)

```
↪verification
         "sign_artifacts": {
             "manifest": "pkgsign-manifest.json",  # This manifest file is generated by␣
↪Conan and should be added here
             "signature": "pkgsign-manifest.json.sig"  # This is the signature file,␣
↪typically signing the manifest file
         }
     }
   ]
```

The manifest file: `pkgsign-manifest.json`

Conan will **generate a JSON manifest file** called `pkgsign-manifest.json` in the signature folder right before calling the `sign()` method of the plugin. This file is useful to check the integrity of the package and to **perform the sign over it by just signing this file (recommended)**.

Here is an example of the contents:

Listing 99: *<signature_folder>/pkgsign-manifest.json*

```
{
  "files":[
    {
      "file":"conan_export.tgz",
      "sha256":"45c89ebf7d37d05f53472535db0d347019263bb5c6b990319108db6e66b85fe3"
    },
    {
      "file":"conan_sources.tgz",
      "sha256":"e757aa62bf6ac3cc5e36ecbd3c82c04d0faff46a61223189ee07e16489ef7c99"
    },
    {
      "file":"conanfile.py",
      "sha256":"12cc1f6477f6512158914c1041833db0b417d07d8f4f2538b07e5e4661d0139b"
    },
    {
      "file":"conanmanifest.txt",
      "sha256":"8d001ac33f5ea7e818d7c2e1aa2eb920d87472ada0b3abfeb0d8b2aaa1248653"
    }
  ]
}
```

The signatures file: `pkgsign-signatures.json`

The `sign()` function **should return a list of signatures** (see the example above) that will be **automatically saved by Conan into a JSON file** called `pkgsign-signatures.json`. This file is intended to list the signature metadata like the `provider` or `method` and the files that are part of the signature in the `sign_artifacts` field.

- `method`: The signing method **indicates the tool used to sign the package so it can be verified later**. For example: gpg, cosign, x509, openssl…
- `provider`: The **name of the organization** that signed the package. This is useful to later **verify the package with a matching public key** provided by the organization.

- `sign_artifacts`: A dictionary of the **files that are part of the signature**. Normally this should include a `manifest` key with the manifest `pkgsign-manifest.json` file, and a `signature` key with the signature file `pkgsign-manifest.json.sig` (or any other signature format). This is useful to identify the files involved in the signature so that they can be used later in the `verify()` function.

Additionally, **more than one signature is supported**, in case you need to sign with different formats or transition from one signature method to another.

Here is an example of the contents:

Listing 100: *<signature_folder>/pkgsign-signatures.json*

```
{
  "signatures": [
    {
      "method": "openssl-dgst",
      "provider": "my-organization",
      "sign_artifacts": {
        "manifest": "pkgsign-manifest.json",
        "signature": "pkgsign-manifest.json.sig"
      }
    }
  ]
}
```

For verifying packages

The following function should be implemented in the plugin:

```
def verify(ref, artifacts_folder, signature_folder, files, **kwargs):
    """
    :param ref: The package reference being verified.
    :param artifacts_folder: Path to the folder containing the package artifacts.
    :param signature_folder: Path to the folder containing the signatures.
    :param files: Dictionary of names and paths of the files in the package.
    """
    # Logic to verify that files in artifacts_folder match the
    # signatures in signature_folder.
    pass
```

This function does not return any value. Instead, it is expected that it raises a `ConanException` if the verification of the signature fails.

Before calling the `verify()` function, **Conan will perform an integrity check** calculating the checksums of the package files and checking against the manifest `pkgsign-manifest.json` file (if the file is present).

Note: Note that the `**kwargs` argument in both functions is required to ensure forward compatibility. Future versions of Conan may pass additional parameters, and omitting `**kwargs` could break your plugin.

Commands

The **conan cache sign** command will trigger the `sign()` method of the configured signing plugin and sign the recipe and packages:

```
$ conan cache sign mypkg/1.0.0
[Package sign] Results:

mypkg/1.0
  revisions
    294e801a0e1da10084441487e95b80e8
      packages
        7b737f1649e252dd60b2aefeabe5b6ef5e1a4750
          revisions
            7722a3748274ae073736f20d84428fe9
        dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b
          revisions
            829868ff6774b7da8c1eace8d76e71f1

[Package sign] Summary: OK=3, FAILED=0
```

And the **conan cache verify** command will trigger the `verify()` method of the plugin and verify the recipe and packages as well:

```
$ conan cache verify mypkg/1.0.0
[Package sign] Results:

mypkg/1.0
  revisions
    294e801a0e1da10084441487e95b80e8
      packages
        7b737f1649e252dd60b2aefeabe5b6ef5e1a4750
          revisions
            7722a3748274ae073736f20d84428fe9
        dee9f7f985eb1c20e3c41afaa8c35e2a34b5ae0b
          revisions
            829868ff6774b7da8c1eace8d76e71f1

[Package sign] Summary: OK=3, FAILED=0
```

Here is a usual flow for signing and verifying packages:

```
$ conan create --name=mypkg --version=1.0.0
$ conan cache sign mypkg/1.0.0
$ conan upload mypkg/1.0.0 --remote=myremote
...
$ conan install --requires=mypkg/1.0.0  # This will trigger verify() when the package is␣
↪downloaded from a remote
# When the package is signed, the verify() can be done at anytime with:
$ conan cache verify mypkg/1.0.0
```

Caution: The **conan upload** command **will not automatically sign** the packages since Conan 2.26.0. Please make sure to use the **conan cache sign** command to **sign the packages before uploading them**, and **update**

your plugin to conform to the new implementation.

Plugin implementation examples

Here you can find some implementation examples of the plugin so they can serve as guidance to develop your own:

- *Signing packages with Sigstore (Cosign)*.
- *Signing packages with OpenSSL*.

9.8.12 Compiler flags mapper plugin

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `compiler_flags.py` extension plugin is a Python script that receives the list of compiler flags provided by dependencies `cpp_info` information, and allows to process it.

This plugin must be located in the `extensions/plugins` cache folder, and can be installed with the `conan config install` and `conan config install-pkg` commands.

A possible use case would be when packages built with a different compiler, for example `msvc` define some `msvc` specific compiler flags for their consumers assuming it will be the same compiler. But if the consumer is using LLVM/Clang, assuming binary compatibility, then that clang compiler could be receiving `msvc` flags that is not expecting and that it doesn't understand. This plugin allows to convert, map, remove, translate or add compiler flags.

For example, a case that exists in ConanCenter recipes is some packages defining `/Zc:__cplusplus` compiler flag for `msvc` that `clang` compiler won't understand. In this case, it is possible to write a pluging like:

```
def flags_map(conanfile, item, flags, **kwargs):
    if item != "cxxflags":
        return flags
    result = []
    compiler = conanfile.settings.get_safe("compiler")
    for d in flags:
        if d.startswith("/Zc:"):
            if compiler == "msvc":
                result.append(d)
        else:
            result.append(d)
    return result
```

The `conanfile` object is passed as an argument, so it is possible to check the `compiler` that the consumer will be using. Only for `msvc` the `/Zc` flags will be propagated, otherwise they will be ignored.

The possible `item` values are `cflags`, `cxxflags`, `sharedlinkflags` and `exelinkflags`.

Warning: Do not abuse this feature for other purposes rather than the binary compatibility among different compilers exemplified above. If you have some other use case that could be addressed by this plugin, please submit a ticket to https://github.com/conan-io/conan/issues first to discuss.

9.9 Policies

Policies are a set of rules to enforce certain behaviors from Conan. Policies are handled by the `core:policies` configuration in your `global.conf`, which is a list of strings, where each string is the name of a policy to be enabled.

Listing 101: `global.conf`

```
core:policies = ["required_conan_version>=2.28"]
```

9.9.1 List of current policies

required_conan_version>=version

Introduced in Conan 2.28

This policy is unique, as the version specified in the policy is used to enable different behaviors based on the version. This allows to opt-in to bugfixes that can be considered breaking changes, without having to wait for a new Conan release to include them by default.

The same behaviours are also enabled recipe-wise when the `required_conan_version` attribute is defined in the recipe, such that the policy can be enabled for specific recipes, without having to enable it globally. If both the policy and the recipe attribute are defined, the behavior will be enabled if either of them matches the required version range.

- **If using `required_conan_version>=2.28` or later, the following bugfixes will be enabled:**
  - **Bugfix https://github.com/conan-io/conan/pull/19705:**
    - The computation of `package_id` for static libraries and non-embed mode was taking into account transitive (non-direct) dependencies, even if they were not being embedded and not contributing headers at all. See the docs for the *effect of dependencies in the package_id*.
  - **Bugfix https://github.com/conan-io/conan/pull/19849:**
    - The `VirtualBuildEnv` generator used to include the `bindir` paths of tool requires regardless of their `run` trait in the generated environment. With the bugfix enabled, only tool requires with the `run` trait set to `True` will have their `bindir` paths propagated.
  - **Behaviour change https://github.com/conan-io/conan/pull/19286:**
    - For the new `consistent` trait, its default value currently keeps the old graph expansion behaviour, which had some inconsistencies regarding the handling of private dependencies. With the new behaviour enabled, the graph expansion is more consistent and private dependencies are handled in a more intuitive way, but some graphs can be expanded differently. For a detailed explanation of the changes, see *the trait documentation section*.
- **If using `required_conan_version>=2.30` or later, the following bugfixes will be enabled:**
  - **Bugfix https://github.com/conan-io/conan/pull/20073:**
    - The `transitive_headers` trait propagation of some diamond structures was not working correctly and unexpected headers were being propagated to consumers. With the bugfix enabled, the propagation of transitive headers is fixed and works as expected in all cases, but the consumer package_id can change if it was previously affected by this bug.

Note: This policy is independent of the `core:required_conan_version` conf, which is exclusively used to define the minimum required Conan version.

9.10 Recipe tools

Tools are all things that can be imported and used in Conan recipes.

The import path is always like:

```
from conan.tools.cmake import CMakeToolchain, CMakeDeps, CMake
from conan.tools.microsoft import MSBuildToolchain, MSBuildDeps, MSBuild
```

The main guidelines are:

- Everything that recipes can import belong to `from conan.tools`. Any other thing is private implementation and shouldn't be used in recipes.
- Only documented, public (not preceded by _) tools can be used in recipes.

Contents:

9.10.1 conan.tools.android

android_abi()

android_abi(*conanfile*, *context='host'*)

Returns Android-NDK ABI

Parameters

- **conanfile** – ConanFile instance
- **context** – either "host", "build" or "target"

Returns

Android-NDK ABI

This function might not be necessary when using Conan built-in integrations, as they already manage it, but can be useful if developing your own build system integration.

`android_abi()` function returns the Android standard ABI name based on Conan `settings.arch` value, something like:

```
def android_abi(conanfile, context="host"):
  ...
  return {
        "armv5el": "armeabi",
        "armv5hf": "armeabi",
        "armv5": "armeabi",
        "armv6": "armeabi-v6",
        "armv7": "armeabi-v7a",
        "armv7hf": "armeabi-v7a",
        "armv8": "arm64-v8a",
        }.get(conanfile.settings.arch)
```

As it can be seen, the default is the "host" ABI, but it is possible to select also the "build" or "target" ones if necessary.

```
from conan.tools.android import android_abi

class Pkg(ConanFile):
```

(continues on next page)

(continued from previous page)

```
    def generate(self)
        abi = android_abi(self)
```

9.10.2 conan.tools.apple

XcodeDeps

The `XcodeDeps` tool is the dependency information generator for *Xcode*. It will generate multiple *.xcconfig* configuration files, the can be used by consumers using *xcodebuild* or *Xcode*. To use them just add the generated configuration files to the Xcode project or set the `-xcconfig` argument from the command line.

The `XcodeDeps` generator can be used by name in conanfiles:

Listing 102: conanfile.py

```
class Pkg(ConanFile):
    generators = "XcodeDeps"
```

Listing 103: conanfile.txt

```
[generators]
XcodeDeps
```

And it can also be fully instantiated in the conanfile `generate()` method:

Listing 104: conanfile.py

```
from conan import ConanFile
from conan.tools.apple import XcodeDeps

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    requires = "libpng/1.6.37@" # Note libpng has zlib as transitive dependency

    def generate(self):
        xcode = XcodeDeps(self)
        xcode.generate()
```

When the `XcodeDeps` generator is used, every invocation of `conan install` will generate several configuration files, per direct dependency and configuration. For the *conanfile.py* above, for example:

```
$ conan install conanfile.py # default is Release
$ conan install conanfile.py -s build_type=Debug
```

This generator is multi-configuration. It will generate different files for the different *Debug/Release* configurations for each direct requirement. It will also generate one single file (*conandeps.xcconfig*) aggregating all the files for the direct dependencies (just *libpng* in this case). The above commands generate the following files:

```
.
├── conan_config.xcconfig
├── conan_libpng.xcconfig
├── conan_libpng_libpng.xcconfig
```

(continues on next page)

(continued from previous page)

```
├── conan_libpng_libpng_debug_x86_64.xcconfig
├── conan_libpng_libpng_release_x86_64.xcconfig
└── conandeps.xcconfig
```

Note that even though *libpng* depends on *zlib*, no *conan_zlib** files are generated. The *XcodeDeps* generator only emits *xcconfig* files for **direct** dependencies, and inlines all transitive data (include directories, library names, flags, etc.) into the props file of the direct dependency.

The first `conan install` with the default *Release* and *x86_64* configuration generates:

- *conan_libpng_libpng_release_x86_64.xcconfig*: declares variables with conditional logic to be considered only for the active configuration in *Xcode* or the one passed by command line to *xcodebuild*. The values inlined here include the *includedirs*, *libs*, *libdirs*, flags, etc., from *libpng* and from all of its transitive dependencies (in this case, also *zlib*), merged into a single set of variables named after the direct dependency.
- *conan_libpng_libpng.xcconfig*: includes *conan_libpng_libpng_release_x86_64.xcconfig* and declares the following *Xcode* build settings using the inlined values: `SYSTEM_HEADER_SEARCH_PATHS`, `GCC_PREPROCESSOR_DEFINITIONS`, `OTHER_CFLAGS`, `OTHER_CPLUSPLUSFLAGS`, `FRAMEWORK_SEARCH_PATHS`, `LIBRARY_SEARCH_PATHS`, `OTHER_LDFLAGS`. This file no longer includes any *xcconfig* file from transitive dependencies, since the data is already inlined in the props file.
- *conan_libpng.xcconfig*: in this case it only includes *conan_libpng_libpng.xcconfig*, but in the case that the required package has components, this file will include all of the components of the package.
- *conandeps.xcconfig*: configuration file including all direct dependencies, in this case, it just includes `conan_libpng.xcconfig`.
- The main *conan_config.xcconfig* file, to be added to the project. Includes both the files from this generator and the generated by the *XcodeToolchain* in case it was also set.

The second `conan install -s build_type=Debug` generates:

- *conan_libpng_libpng_debug_x86_64.xcconfig*: same variables as the one above for *Debug* configuration, also containing the inlined data from transitive dependencies.
- *conan_libpng_libpng.xcconfig*: this file has been already created by the previous command, now it's modified to add the include for *conan_libpng_libpng_debug_x86_64.xcconfig*.
- *conan_libpng.xcconfig*: this file will remain the same.
- *conandeps.xcconfig*: configuration file including all direct dependencies, in this case, it just includes `conan_libpng.xcconfig`.
- The main *conan_config.xcconfig* file, to be added to the project. Includes both the files from this generator and the generated by the *XcodeToolchain* in case it was also set.

If you want to add this dependencies to you Xcode project, you just have to add the *conan_config.xcconfig* configuration file for all of the configurations you want to use (usually *Debug* and *Release*).

Additional variables defined

Besides the variables that define the *Xcode* build settings mentioned above, there are additional variables declared that may be useful to use in your *Xcode* project:

- *PACKAGE_ROOT_<package_name>*: Set to the location of the *package_folder attribute*.

Components support

This generator supports packages with components. That means that:

- If a **dependency** `package_info()` declares `cpp_info.requires` on some components, only the data from those components (and their transitive components) will be inlined into the props file of the direct dependency. Components that are not required by anyone are not inlined.
- The current package `requires` will be fully dependent on and all components. Recall that the `package_info()` only applies for consumers, but not to the current package.

Custom configurations

If your Xcode project defines custom configurations, like `ReleaseShared`, or `MyCustomConfig`, it is possible to define it into the `XcodeDeps` generator, so different project configurations can use different set of dependencies. Let's say that our current project can be built as a shared library, with the custom configuration `ReleaseShared`, and the package also controls this with the `shared` option:

```
from conan import ConanFile
from conan.tools.apple import XcodeDeps

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    options = {"shared": [True, False]}
    default_options = {"shared": False}
    requires = "zlib/1.3.1"

    def generate(self):
        xcode = XcodeDeps(self)
        # We assume that -o *:shared=True is used to install all shared deps too
        if self.options.shared:
            xcode.configuration = str(self.settings.build_type) + "Shared"
        xcode.generate()
```

This will manage to generate new *.xcconfig* files for this custom configuration, and when you switch to this configuration in the IDE, the build system will take the correct values depending whether we want to link with shared or static libraries.

XcodeToolchain

The `XcodeToolchain` is the toolchain generator for Xcode. It will generate *.xcconfig* configuration files that can be added to Xcode projects. This generator translates the current package configuration, settings, and options, into Xcode *.xcconfig* files syntax.

The `XcodeToolchain` generator can be used by name in conanfiles:

Listing 105: conanfile.py

```
class Pkg(ConanFile):
    generators = "XcodeToolchain"
```

Listing 106: conanfile.txt

```
[generators]
XcodeToolchain
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.apple import XcodeToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = XcodeToolchain(self)
        tc.generate()
```

The `XcodeToolchain` will generate three files after a `conan install` command. As explained above for the XcodeDeps generator, each different configuration will create a set of files with different names. For example, running `conan install` for *Release* first and then *Debug* configuration:

```
$ conan install conanfile.py # default is Release
$ conan install conanfile.py -s build_type=Debug
```

Will create these files:

```
.
├── conan_config.xcconfig
├── conantoolchain_release_x86_64.xcconfig
├── conantoolchain_debug_x86_64.xcconfig
├── conantoolchain.xcconfig
└── conan_global_flags.xcconfig
```

Those files are:

- The main *conan_config.xcconfig* file, to be added to the project. Includes both the files from this generator and the generated by the *XcodeDeps* in case it was also set.
- *conantoolchain_<debug/release>_x86_64.xcconfig*: declares `CLANG_CXX_LIBRARY`, `CLANG_CXX_LANGUAGE_STANDARD` and `MACOSX_DEPLOYMENT_TARGET` (when `os` is set to `Macos`) variables with conditional logic depending on the build configuration, architecture and sdk set.
- *conantoolchain.xcconfig*: aggregates all the *conantoolchain_<config>_<arch>.xcconfig* files for the different installed configurations.

- *conan_global_flags_<config>_<arch>.xcconfig*: declares `GCC_PREPROCESSOR_DEFINITIONS`, `OTHER_CFLAGS`, `OTHER_CPLUSPLUSFLAGS` and `OTHER_LDFLAGS` variables with conditional logic depending on the build configuration, architecture and sdk set. This file will only be generated in case any configuration variables related to compiler or linker flags are set. Check *the configuration section* below for more details.
- *conan_global_flags.xcconfig*: aggregates all the *conan_global_flags_<config>_<arch>.xcconfig* files for the different installed configurations (*since Conan 2.32*).

Every invocation to `conan install` with different configuration will create a new *conantoolchain_<config>_<arch>.xcconfig* and, if any compiler or linker flags are set, a new *conan_global_flags_<config>_<arch>.xcconfig* file. Both are aggregated in *conantoolchain.xcconfig* and *conan_global_flags.xcconfig* respectively, so you can have different configurations included in your Xcode project.

The XcodeToolchain files can declare the following Xcode build settings based on Conan settings values:

- `*_DEPLOYMENT_TARGET` is based on the value of the `os` + `os.version` settings and will make the build system pass respective flag (e.g. `MACOSX_DEPLOYMENT_TARGET` translates to `-mmacosx-version-min` for macOS) with that value (if set). It defines the minimum operating system version the binary should run on.
- `CLANG_CXX_LANGUAGE_STANDARD` is based on the value of the `compiler.cppstd` setting that sets the C++ language standard.
- `CLANG_CXX_LIBRARY` is based on the value of the compiler.libcxx setting and sets the version of the C++ standard library to use.

One of the advantages of using toolchains is that they can help to achieve the exact same build with local development flows, than when the package is created in the cache.

conf

This toolchain is also affected by these **[conf]** variables:

- `tools.build:cxxflags` list of C++ flags.
- `tools.build:cflags` list of pure C flags.
- `tools.build:sharedlinkflags` list of flags that will be used by the linker when creating a shared library.
- `tools.build:exelinkflags` list of flags that will be used by the linker when creating an executable.
- `tools.build:defines` list of preprocessor definitions.

If you set any of these variables, the toolchain will use them to generate the `conan_global_flags.xcconfig` file that will be included from the `conan_config.xcconfig` file.

build_settings

The `build_settings` attribute lets a recipe set any Xcode build setting directly. Settings already covered by a **[conf]** variable, such as `OTHER_CFLAGS`, `OTHER_CPLUSPLUSFLAGS`, `OTHER_LDFLAGS` and `GCC_PREPROCESSOR_DEFINITIONS`, should instead be set with the `tools.build:*` variables from the *conf section* above:

```
def generate(self):
    tc = XcodeToolchain(self)
    tc.build_settings["OTHER_SWIFT_FLAGS"] = "$(inherited) -cxx-interoperability-
↪mode=default"
    tc.build_settings["GCC_WARN_UNUSED_VARIABLE"] = "YES"
    tc.generate()
```

XcodeBuild

The `XcodeBuild` build helper is a wrapper around the command line invocation of Xcode. It will abstract the calls like `xcodebuild -project app.xcodeproj -configuration <config> -arch <arch> ...`

The `XcodeBuild` helper can be used like:

```
from conan import conanfile
from conan.tools.apple import XcodeBuild

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def build(self):
        xcodebuild = XcodeBuild(self)
        xcodebuild.build("app.xcodeproj")
```

Reference

class XcodeBuild(*conanfile*)

__init__(*conanfile*)

XcodeBuild.**build**(*xcodeproj*, *target=None*, *configuration=None*, *cli_args=None*)

Call to `xcodebuild` to build a Xcode project.

Parameters

- **xcodeproj** – the *xcodeproj* file to build.
- **target** – the target to build, in case this argument is passed to the `build()` method it will add the `-target` argument to the build system call. If not passed, it will build all the targets passing the `-alltargets` argument instead.
- **configuration** – Build configuration to use (e.g., `Debug`, `Release`). Defaults to the recipe's `settings.build_type`.
- **cli_args** – Extra options to pass directly to `xcodebuild` (list of strings). Examples: `["-xcconfig", "<path/to/file.xcconfig>"]` or custom Xcode build settings like `["BUILD_LIBRARY_FOR_DISTRIBUTION=YES"]`.

Returns

the return code for the launched `xcodebuild` command.

The `XcodeBuild.build()` method internally implements a call to `xcodebuild` like:

```
$ xcodebuild -project app.xcodeproj -configuration <configuration> -arch <architecture>
↪<sdk> <verbosity> -target <target>/-alltargets *_DEPLOYMENT_TARGET=settings.os.version
↪<cli_args>
```

Where:

- `configuration` is the configuration, typically *Release* or *Debug*, which will be obtained from `settings.build_type` unless you pass it explicitly via the `configuration` parameter.
- `architecture` is the build architecture, a mapping from the `settings.arch` to the common architectures defined by Apple 'i386', 'x86_64', 'armv7', 'arm64', etc.

- `sdk` is set based on the values of the `os.sdk` and `os.sdk_version` defining the `SDKROOT` Xcode build setting according to them. For example, setting `os.sdk=iOS` and *os.sdk_version=8.3* ` will pass `SDKROOT=iOS8.3` to the build system. In case you defined the `tools.apple:sdk_path` in your **[conf]** this value will take preference and will directly pass `SDKROOT=<tools.apple:sdk_path>` so **take into account** that for this case the skd located in that path should set your `os.sdk` and `os.sdk_version` settings values.
- `verbosity` is the verbosity level for the build and can take value 'verbose' or 'quiet' if set by `tools.build:verbosity` in your **[conf]**
- `cli_args` are the additional command line arguments passed via the `cli_args` parameter. These can include custom build settings like `BUILD_LIBRARY_FOR_DISTRIBUTION=YES`. You can also redirect build artifacts to the Conan build folder by passing `SYMROOT` and `OBJROOT` settings:

```
def build(self):
  xcodebuild = XcodeBuild(self)
  xcodebuild.build("app.xcodeproj", cli_args=[f"SYMROOT={self.build_folder}",
                                              f"OBJROOT={self.build_folder}"])
```

Additional parameters that are passed to `xcodebuild` (but before `cli_args`):

- Deployment target setting according to the values of `os` and `os.version` from profile, e.g. `MACOSX_DEPLOYMENT_TARGET=10.15` or `IPHONEOS_DEPLOYMENT_TARGET=15.0`

conf

- `tools.build:verbosity` (or `tools.compilation:verbosity` as fallback) which accepts `quiet` or `verbose`, and sets the `-verbose` or `-quiet` flags in `XcodeBuild.install()`
- `tools.apple:sdk_path` path for the sdk location, will set the `SDKROOT` value with preference over composing the value from the `os.sdk` and `os.sdk_version` settings.

conan.tools.apple.fix_apple_shared_install_name()

fix_apple_shared_install_name(*conanfile*)

Search for all the *dylib* files in the conanfile's *package_folder* and fix both the `LC_ID_DYLIB` and `LC_LOAD_DYLIB` fields on those files using the *install_name_tool* utility available in macOS to set `@rpath`.

This tool will search for all the *dylib* files in the conanfile's *package_folder* and fix the library *install names* (the `LC_ID_DYLIB` header). Libraries and executables inside the package folder will also have the `LC_LOAD_DYLIB` fields updated to reflect the patched install names. Executables inside the package will also get an `LC_RPATH` entry pointing to the relative location of the libraries inside the package folder. This is done using the *install_name_tool* utility available in macOS, as outlined below:

- For `LC_ID_DYLIB` which is the field containing the install name of the library, it will change the install name to one that uses the `@rpath`. For example, if the install name is `/path/to/lib/libname.dylib`, the new install name will be `@rpath/libname.dylib`. This is done by internally executing something like:

```
install_name_tool /path/to/lib/libname.dylib -id @rpath/libname.dylib
```

- For `LC_LOAD_DYLIB` which is the field containing the path to the library dependencies, it will change the path of the dependencies to one that uses the `@rpath`. For example, if a binary has a dependency on `/path/to/lib/dependency.dylib`, this will be updated to be `@rpath/dependency.dylib`. This is done for both libraries and executables inside the package folder, invoking *install_name_tool* as below:

```
install_name_tool /path/to/lib/libname.dylib -change /path/to/lib/dependency.dylib␣
↪@rpath/dependency.dylib
```

- For `LC_RPATH`, in those cases in which the packages also contain binary executables that depend on libraries within the same package, entries will be added to reflect the location of the libraries relative to the executable. If a package has executables in the *bin* subfolder and libraries in the *lib* subfolder, this can be performed with an invocation like this:

```
install_name_tool /path/to/bin/my_executable -add_rpath @executable_path/../lib
```

This tool is typically needed by recipes that use Autotools as the build system and in the case that the correct install names are not fixed in the library being packaged. Use this tool, if needed, in the conanfile's `package()` method like:

```
from conan.tools.apple import fix_apple_shared_install_name

class HelloConan(ConanFile):

  ...

  def package(self):
      autotools = Autotools(self)
      autotools.install()
      fix_apple_shared_install_name(self)
```

conan.tools.apple.is_apple_os()

is_apple_os(*conanfile*, *build_context=False*)

returns True if OS is Apple one (Macos, iOS, watchOS, tvOS or visionOS)

conan.tools.apple.to_apple_arch()

to_apple_arch(*conanfile*, *default=None*)

converts conan-style architecture into Apple-style arch

conan.tools.apple.XCRun()

class XCRun(*conanfile*, *sdk=None*, *use_settings_target=False*)

XCRun is a wrapper for the Apple **xcrun** tool used to get information for building.

Parameters

- **conanfile** – Conanfile instance.
- **sdk** – Will skip the flag when `False` is passed and will try to adjust the sdk it automatically if `None` is passed.
- **use_settings_target** – Try to use `settings_target` in case they exist (`False` by default)

find(*tool*)

find SDK tools (e.g. clang, ar, ranlib, lipo, codesign, etc.)

property sdk_path
obtain sdk path (aka apple sysroot or -isysroot

property sdk_version
obtain sdk version

property sdk_platform_path
obtain sdk platform path

property sdk_platform_version
obtain sdk platform version

property cc
path to C compiler (CC)

property cxx
path to C++ compiler (CXX)

property ar
path to archiver (AR)

property ranlib
path to archive indexer (RANLIB)

property strip
path to symbol removal utility (STRIP)

property libtool
path to libtool

property otool
path to otool

property install_name_tool
path to install_name_tool

9.10.3 conan.tools.build

Building

conan.tools.build.build_jobs()

build_jobs(*conanfile*)
Returns the number of CPUs available for parallel builds. It returns the configuration value for `tools.build:jobs` if exists, otherwise, it defaults to the helper function `_cpu_count()`. `_cpu_count()` reads cgroup to detect the configured number of CPUs. Currently, there are two versions of cgroup available.

In the case of cgroup v1, if the data in cgroup is invalid, processor detection comes into play. Whenever processor detection is not enabled, `build_jobs()` will safely return 1.

In the case of cgroup v2, if no limit is set, processor detection is used. When the limit is set, the behavior is as described in cgroup v1.

Parameters
conanfile – The current recipe object. Always use `self`.

Returns
`int` with the number of jobs

conan.tools.build.cross_building()

cross_building(*conanfile=None*, *skip_x64_x86=False*)

Check if we are cross building comparing the *build* and *host* settings. Returns `True` in the case that we are cross-building.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **skip_x64_x86** – Do not consider cross building when building to 32 bits from 64 bits: x86_64 to x86, sparcv9 to sparc or ppc64 to ppc32

Returns

`bool` value from `tools.build.cross_building:cross_build` if exists, otherwise, it returns `True` if we are cross-building, else, `False`.

conan.tools.build.can_run()

can_run(*conanfile*)

Validates whether is possible to run a non-native app on the same architecture. It's a useful feature for the case your architecture can run more than one target. For instance, Mac M1 machines can run both *armv8* and *x86_64*.

Parameters

conanfile – The current recipe object. Always use `self`.

Returns

`bool` value from `tools.build.cross_building:can_run` if exists, otherwise, it returns `False` if we are cross-building, else, `True`.

conan.tools.build.architecture_bits()

architecture_bits(*conanfile*)

Detect the architecture bits based on the `conanfile.settings.arch` and those known architectures in Conan.

Parameters

conanfile – The current recipe object. Always use `self`.

Returns

`"64"` string for 64-bit architectures, `"32"` string for 32-bit architectures, or `None` if the architecture is unknown.

Cppstd

conan.tools.build.check_min_cppstd()

check_min_cppstd(*conanfile*, *cppstd*, *gnu_extensions=False*)

Check if current cppstd fits the minimal version required.

In case the current cppstd doesn't fit the minimal version required by cppstd, a ConanInvalidConfiguration exception will be raised.

settings.compiler.cppstd must be defined, otherwise ConanInvalidConfiguration is raised

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cppstd** – Minimal cppstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17)

conan.tools.build.check_max_cppstd()

check_max_cppstd(*conanfile*, *cppstd*, *gnu_extensions=False*)

Check if current cppstd fits the maximum version required.

In case the current cppstd doesn't fit the maximum version required by cppstd, a ConanInvalidConfiguration exception will be raised.

settings.compiler.cppstd must be defined, otherwise ConanInvalidConfiguration is raised

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cppstd** – Maximum cppstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17)

conan.tools.build.valid_min_cppstd()

valid_min_cppstd(*conanfile*, *cppstd*, *gnu_extensions=False*)

Validate if current cppstd fits the minimal version required.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cppstd** – Minimal cppstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17). This option ONLY works on Linux.

Returns

True, if current cppstd matches the required cppstd version. Otherwise, False.

conan.tools.build.valid_max_cppstd()

valid_max_cppstd(*conanfile*, *cppstd*, *gnu_extensions=False*)

Validate if current cppstd fits the maximum version required.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cppstd** – Maximum cppstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17). This option ONLY works on Linux.

Returns

True, if current cppstd matches the required cppstd version. Otherwise, False.

conan.tools.build.default_cppstd()

default_cppstd(*conanfile*, *compiler=None*, *compiler_version=None*)

Get the default `compiler.cppstd` for the "conanfile.settings.compiler" and "conanfile settings.compiler_version" or for the parameters "compiler" and "compiler_version" if specified.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **compiler** – Name of the compiler e.g. gcc
- **compiler_version** – Version of the compiler e.g. 12

Returns

The default `compiler.cppstd` for the specified compiler

conan.tools.build.supported_cppstd()

supported_cppstd(*conanfile*, *compiler=None*, *compiler_version=None*)

Get a list of supported `compiler.cppstd` for the "conanfile.settings.compiler" and "conanfile.settings.compiler_version" or for the parameters "compiler" and "compiler_version" if specified.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **compiler** – Name of the compiler e.g: gcc
- **compiler_version** – Version of the compiler e.g: 12

Returns

a list of supported `cppstd` values.

conan.tools.build.cppstd_flag()

cppstd_flag(*conanfile*) → str

Returns flags specific to the C++ standard based on the `conanfile.settings.compiler`, `conanfile.settings.compiler.version` and `conanfile.settings.compiler.cppstd`.

It also considers when using GNU extension in `settings.compiler.cppstd`, reflecting it in the compiler flag. Currently, it supports GCC, Clang, AppleClang, MSVC, Intel, MCST-LCC.

In case there is no `settings.compiler` or `settings.cppstd` in the profile, the result will be an **empty string**.

Parameters

conanfile – The current recipe object. Always use `self`.

Returns

`str` with the standard C++ flag used by the compiler. e.g. "-std=c++11", "/std:c++latest"

cstd

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

conan.tools.build.check_min_cstd()

check_min_cstd(*conanfile*, *cstd*, *gnu_extensions=False*)

Check if current cstd fits the minimal version required.

In case the current cstd doesn't fit the minimal version required by cstd, a ConanInvalidConfiguration exception will be raised.

1. If settings.compiler.cstd, the tool will use settings.compiler.cstd to compare
2. It not settings.compiler.cstd, the tool will use compiler to compare (reading the default from cstd_default)
3. If not settings.compiler is present (not declared in settings) will raise because it cannot compare.
4. If can not detect the default cstd for settings.compiler, a exception will be raised.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cstd** – Minimal cstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17)

conan.tools.build.check_max_cstd()

check_max_cstd(*conanfile*, *cstd*, *gnu_extensions=False*)

Check if current cstd fits the maximum version required.

In case the current cstd doesn't fit the maximum version required by cstd, a ConanInvalidConfiguration exception will be raised.

1. If settings.compiler.cstd, the tool will use settings.compiler.cstd to compare
2. It not settings.compiler.cstd, the tool will use compiler to compare (reading the default from cstd_default)
3. If not settings.compiler is present (not declared in settings) will raise because it cannot compare.
4. If can not detect the default cstd for settings.compiler, a exception will be raised.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cstd** – Maximum cstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17)

conan.tools.build.valid_min_cstd()

valid_min_cstd(*conanfile*, *cstd*, *gnu_extensions=False*)

Validate if current cstd fits the minimal version required.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cstd** – Minimal cstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17). This option ONLY works on Linux.

Returns

True, if current cstd matches the required cstd version. Otherwise, False.

conan.tools.build.valid_max_cstd()

valid_max_cstd(*conanfile*, *cstd*, *gnu_extensions=False*)

Validate if current cstd fits the maximum version required.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **cstd** – Maximum cstd version required
- **gnu_extensions** – GNU extension is required (e.g gnu17). This option ONLY works on Linux.

Returns

True, if current cstd matches the required cstd version. Otherwise, False.

conan.tools.build.default_cstd()

default_cstd(*conanfile*, *compiler=None*, *compiler_version=None*)

Get the default `compiler.cstd` for the "conanfile.settings.compiler" and "conanfile settings.compiler_version" or for the parameters "compiler" and "compiler_version" if specified.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **compiler** – Name of the compiler e.g. gcc
- **compiler_version** – Version of the compiler e.g. 12

Returns

The default `compiler.cstd` for the specified compiler

conan.tools.build.supported_cstd()

supported_cstd(*conanfile*, *compiler=None*, *compiler_version=None*)

Get a list of supported `compiler.cstd` for the "conanfile.settings.compiler" and "conanfile.settings.compiler_version" or for the parameters "compiler" and "compiler_version" if specified.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **compiler** – Name of the compiler e.g: gcc
- **compiler_version** – Version of the compiler e.g: 12

Returns

a list of supported `cstd` values.

Compiler

conan.tools.build.check_min_compiler_version()

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

check_min_compiler_version(*conanfile*, *compiler_restrictions*)

(Experimental) Checks if the current compiler and its version meet the minimum requirements.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **compiler_restrictions** – A list of tuples, where each tuple contains:
  - **compiler** (*str*): The name of the compiler (e.g., "gcc", "msvc").
  - **min_version** (*str*): The minimum required version as a string (e.g., "14", "19.0").
  - **reason** (*str*): A string explaining the reason for the version requirement.

Raises

- **ConanException** – If the 'compiler' or 'compiler.version' settings are not defined.
- **ConanInvalidConfiguration** – If the found compiler version is less than the specified minimum version for that compiler.

Example

```
def validate(self):
    compiler_restrictions = [
        ("clang", "14", "requires C++20 coroutines support"),
        ("gcc", "12", "requires C++20 modules support")
    ]
    check_min_compiler_version(self, compiler_restrictions)
```

9.10.4 conan.tools.cmake

CMakeDeps

Note: There is a new experimental `CMakeConfigDeps` generator that implements many improvements and fixes over this `CMakeDeps` generator. Fixes, support, new features and maintenance is expected mainly in `CMakeConfigDeps`, so if you have issues with `CMakeDeps`, give a try to `CMakeConfigDeps` first.

To simplify the testing and validation of the `CMakeDeps` -> `CMakeConfigDeps` migration, the `-c tools.cmake.cmakedeps:new=will_break_next` configuration can be used, that does a hot replacement of every `CMakeDeps` in recipes by `CMakeConfigDeps`, without needing to edit the recipes at all.

See *CMakeConfigDeps generator*

The `CMakeDeps` generator produces the necessary files for each dependency to be able to use the cmake `find_package()` function to locate the dependencies. It can be used like:

```
from conan import ConanFile

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"
    generators = "CMakeDeps"
```

The full instantiation, that allows custom configuration can be done in the `generate()` method:

```
from conan import ConanFile
from conan.tools.cmake import CMakeDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"

    def generate(self):
        cmake = CMakeDeps(self)
        cmake.generate()
```

Listing 107: **CMakeLists.txt**

```
cmake_minimum_required(VERSION 3.15)
project(compressor C)

find_package(hello REQUIRED)

add_executable(${PROJECT_NAME} src/main.c)
target_link_libraries(${PROJECT_NAME} hello::hello)
```

By default, for a `hello` requires, you need to use `find_package(hello)` and link with the target `hello::hello`. Check *the properties affecting CMakeDeps* like `cmake_target_name` to customize the file and the target names in the conanfile.py of the dependencies and their components.

Note: The `CMakeDeps` is intended to run with the `CMakeToolchain` generator. It will set `CMAKE_PREFIX_PATH` and `CMAKE_MODULE_PATH` to the right folder (`conanfile.generators_folder`) so CMake can locate the generated

config/module files.

Generated files

- **XXX-config.cmake**: By default, the CMakeDeps generator will create config files declaring the targets for the dependencies and their components (if declared).
- **FindXXX.cmake**: Only when the property `cmake_find_mode` is set by the dependency with "module" or "both". See *The properties affecting CMakeDeps* is set in the dependency.
- **Other necessary *.cmake**: files like version, flags and directory data or configuration.

Note that it will also generate a **conandeps_legacy.cmake** file. This is a file that provides a behavior similar to the Conan 1 `cmake` generator, allowing to include this file with `include(${CMAKE_BINARY_DIR}/generators/conandeps_legacy.cmake)`, and providing a single CMake `CONANDEPS_LEGACY` variable that allows to link with all the direct and transitive dependencies without explicitly enumerating them like: `target_link_libraries(app ${CONANDEPS_LEGACY})`. This is a convenience provided for Conan 1.X users to upgrade to Conan 2 without changing their overall developer flow, but it is not recommended otherwise, and using the CMake canonical flow of explicitly using `find_package()` and `target_link_libraries(... pkg1::pkg1 pkg2::pkg2)` with targets is the correct approach.

Customization

There are some attributes you can adjust in the created `CMakeDeps` object to change the default behavior:

configuration

Allows to define custom user CMake configuration besides the standard Release, Debug, etc ones.

```
def generate(self):
    deps = CMakeDeps(self)
    # By default, ``deps.configuration`` will be ``self.settings.build_type``
    if self.options["hello"].shared:
        # Assuming the current project ``CMakeLists.txt`` defines the ReleasedShared␣
↪configuration.
        deps.configuration = "ReleaseShared"
    deps.generate()
```

The `CMakeDeps` is a *multi-configuration* generator, it can correctly create files for Release/Debug configurations to be simultaneously used by IDEs like Visual Studio. In single configuration environments, it is necessary to have a configuration defined, which must be provided via the `cmake ... -DCMAKE_BUILD_TYPE=<build-type>` argument in command line (Conan will do it automatically when necessary, in the `CMake.configure()` helper).

build_context_activated

When you have a **build-require**, by default, the config files (*xxx-config.cmake*) files are not generated. But you can activate it using the **build_context_activated** attribute:

```
tool_requires = ["my_tool/0.0.1"]

def generate(self):
    cmake = CMakeDeps(self)
    # generate the config files for the tool require
    cmake.build_context_activated = ["my_tool"]
    cmake.generate()
```

build_context_suffix

When you have the same package as a **build-require** and as a **regular require** it will cause a conflict in the generator because the file names of the config files will collide as well as the targets names, variables names etc.

For example, this is a typical situation with some requirements (capnproto, protobuf...) that contain a tool used to generate source code at build time (so it is a **build_require**), but also providing a library to link to the final application, so you also have a **regular require**. Solving this conflict is specially important when we are cross-building because the tool (that will run in the building machine) belongs to a different binary package than the library, that will "run" in the host machine.

You can use the **build_context_suffix** attribute to specify a suffix for a requirement, so the files/targets/variables of the requirement in the build context (tool require) will be renamed:

```
tool_requires = ["my_tool/0.0.1"]
requires = ["my_tool/0.0.1"]

def generate(self):
    cmake = CMakeDeps(self)
    # generate the config files for the tool require
    cmake.build_context_activated = ["my_tool"]
    # disambiguate the files, targets, etc
    cmake.build_context_suffix = {"my_tool": "_BUILD"}
    cmake.generate()
```

build_context_build_modules

Also there is another issue with the **build_modules**. As you may know, the recipes of the requirements can declare a *cppinfo.build_modules* entry containing one or more **.cmake** files. When the requirement is found by the cmake `find_package()` function, Conan will include automatically these files.

By default, Conan will include only the build modules from the `host` context (regular requires) to avoid the collision, but you can change the default behavior.

Use the **build_context_build_modules** attribute to specify require names to include the **build_modules** from **tool_requires**:

```
tool_requires = ["my_tool/0.0.1"]
```

(continues on next page)

(continued from previous page)

```
def generate(self):
    cmake = CMakeDeps(self)
    # generate the config files for the tool require
    cmake.build_context_activated = ["my_tool"]
    # Choose the build modules from "build" context
    cmake.build_context_build_modules = ["my_tool"]
    cmake.generate()
```

check_components_exist

Warning: The `check_components_exist` attribute is **experimental** and subject to change.

This property is `False` by default. Use this property if you want to add a check when you require specifying components in the consumers' `find_package()`. For example, if we are consuming a Conan package like Boost that declares several components. If we set the attribute to `True`, the `find_package()` call of the consumer, will check that the required components exist and raise an error otherwise. You can set this attribute in the `generate()` method:

```
requires = "boost/1.81.0"

...

def generate(self):
    deps = CMakeDeps(self)
    deps.check_components_exist = True
    deps.generate()
```

Then, when consuming Boost the `find_package()` will raise an error as *fakecomp* does not exist:

```
cmake_minimum_required(VERSION 3.15)
...
find_package(Boost COMPONENTS random regex fakecomp REQUIRED)
...
```

Reference

class CMakeDeps(*conanfile*)

generate()

This method will save the generated files to the `conanfile.generators_folder` folder

set_property(*dep*, *prop*, *value*, *build_context=False*)

Using this method you can overwrite the *property* values set by the Conan recipes from the consumer. This can be done for `cmake_file_name`, `cmake_target_name`, `cmake_find_mode`, `cmake_module_file_name`, `cmake_module_target_name`, `cmake_additional_variables_prefixes`, `cmake_config_version_compat`, `system_package_version`, `cmake_build_modules`, `nosoname`, `cmake_target_aliases` and `cmake_extra_variables`.

Parameters

- **dep** – Name of the dependency to set the *property*. For components use the syntax: `dep_name::component_name`.
- **prop** – Name of the *property*.
- **value** – Value of the property. Use `None` to invalidate any value set by the upstream recipe.
- **build_context** – Set to `True` if you want to set the property for a dependency that belongs to the build context (`False` by default).

get_cmake_package_name(*dep*, *module_mode=None*)

Get the name of the file for the `find_package(XXX)` call

get_find_mode(*dep*)

Parameters
dep – requirement

Returns
One of `"none"`, `"config"`, `"module"` or `"both"`. Defaults to `"config"` when not set

Properties

The following properties affect the CMakeDeps generator:

- **cmake_file_name**: The config file generated for the current package will follow the `<VALUE>-config.cmake` pattern, so to find the package you write `find_package(<VALUE>)`.
- **cmake_target_name**: Name of the target to be consumed.
- **cmake_target_aliases**: List of aliases that Conan will create for an already existing target.
- **cmake_find_mode**: Defaulted to `config`. Possible values are:
  - `config`: The CMakeDeps generator will create config scripts for the dependency.
  - `module`: Will create module config (FindXXX.cmake) scripts for the dependency.
  - `both`: Will generate both config and modules.
  - `none`: Won't generate any file. It can be used, for instance, to create a system wrapper package so the consumers find the config files in the CMake installation config path and not in the generated by Conan (because it has been skipped).
- **cmake_module_file_name**: Same as **cmake_file_name** but when generating modules with `cmake_find_mode=module/both`. If not specified it will default to **cmake_file_name**.
- **cmake_module_target_name**: Same as **cmake_target_name** but when generating modules with `cmake_find_mode=module/both`. If not specified it will default to **cmake_target_name**.
- **cmake_build_modules**: List of `.cmake` files (route relative to root package folder) that are automatically included when the consumer run the `find_package()`. This property cannot be set in the components, only in the root `self.cpp_info`.
- **cmake_set_interface_link_directories**: (**deprecated** in 2.14). It was used for `#pragma comment(lib, "foo")` a long while ago, but it is no longer necessary for `CMakeDeps` generator. In the new `CMakeConfigDeps` generator, it will be necessary to correctly specify the library details in the `cpp_info`, aligned with the CPS practices.
- **nosoname**: boolean value that should be used only by dependencies that are defined as `SHARED` and represent a library built without the `soname` flag option.

- **cmake_config_version_compat**: By default `SameMajorVersion`, it can take the values `"AnyNewerVersion"`, `"SameMajorVersion"`, `"SameMinorVersion"`, `"ExactVersion"`. It will use that policy in the generated `<PackageName>ConfigVersion.cmake` file
- **system_package_version**: version of the package used to generate the `<PackageName>ConfigVersion.cmake` file. Can be useful when creating system packages or other wrapper packages, where the conan package version is different to the eventually referenced package version to keep compatibility to `find_package(<PackageName> <Version>)` calls.
- **cmake_additional_variables_prefixes**: List of prefixes to be used when creating CMake variables in the config files. These variables are created with `file_name` as prefix by default, but setting this property will create additional variables with the specified prefixes alongside the default `file_name` one.
- **cmake_extra_variables**: Dictionary of extra variables to be added to the generated config file. The keys of the dictionary are the variable names and the values are the variable values, which can be a plain string, a number or a dict-like python object which must specify the `value` (string/number) , `cache` (boolean), `type` (CMake cache type) and optionally, `docstring` (string: defaulted to variable name) and `force` (boolean) keys. Note that this has less preference over those values defined in the `tools.cmake.cmaketoolchain:extra_variables` conf.
- **cmake_extra_dependencies**: List of extra package names for which a `find_dependency()` call will be added to the generated `<pkg>-config.cmake` file. Useful when the package depends on libraries located via CMake's `find_package()` but not declared as Conan `requires` (for example, system-provided dependencies like `OpenMPI`). This property is only honored at the package level (setting it on a component has no effect).
- **cmake_extra_interface_libs**: List of extra libraries to be appended to the generated CMake target's interface link libraries. Useful to declare additional libraries (such as system libraries, or libraries located by the packages declared in `cmake_extra_dependencies`) that have to be linked together with the package's own libraries. Can be defined both at the package level and per-component.

Example:

```
def package_info(self):
    ...
    # MyFileName-config.cmake
    self.cpp_info.set_property("cmake_file_name", "MyFileName")
    # Names for targets are absolute, Conan won't add any namespace to the target names␣
↪automatically
    self.cpp_info.set_property("cmake_target_name", "Foo::Foo")
    # Automatically include the lib/mypkg.cmake file when calling find_package()
    # This property cannot be set in a component.
    self.cpp_info.set_property("cmake_build_modules", [os.path.join("lib", "mypkg.cmake
↪")])

    # Create a new target "MyFooAlias" that is an alias to the "Foo::Foo" target
    self.cpp_info.set_property("cmake_target_aliases", ["MyFooAlias"])

    self.cpp_info.components["mycomponent"].set_property("cmake_target_name", "Foo::Var")

    # Create a new target "VarComponent" that is an alias to the "Foo::Var" component␣
↪target
    self.cpp_info.components["mycomponent"].set_property("cmake_target_aliases", [
↪"VarComponent"])

    # Skip this package when generating the files for the whole dependency tree in the␣
↪consumer
    # note: it will make useless the previous adjustments.
```

(continues on next page)

(continued from previous page)

```
    # self.cpp_info.set_property("cmake_find_mode", "none")

    # Generate both MyFileNameConfig.cmake and FindMyFileName.cmake
    self.cpp_info.set_property("cmake_find_mode", "both")

    # Add extra variables to the generated config file
    self.cpp_info.set_property("cmake_extra_variables", {
                                    "FOO": 42,
                                    "CMAKE_GENERATOR_INSTANCE": "${GENERATOR_INSTANCE}/
↪buildTools/",
                                    "CACHE_VAR_DEFAULT_DOC": {"value": "hello world",
                                                              "cache": True, "type":
↪"STRING"}
                                })

    # Emit ``find_dependency(MyOpenMPI REQUIRED)`` in the generated <pkg>-config.cmake,
    # so consumers can locate dependencies not provided as Conan requires.
    self.cpp_info.set_property("cmake_extra_dependencies", ["MyOpenMPI"])

    # Append extra libraries to the generated target's interface link libraries.
    # Can also be set per-component, for example:
    # self.cpp_info.components["mycomponent"].set_property("cmake_extra_interface_libs",
    #                                                      ["MyOpenMPI::MyOpenMPI"])
    self.cpp_info.set_property("cmake_extra_interface_libs", ["MyOpenMPI::MyOpenMPI"])
```

Overwrite properties from the consumer side using CMakeDeps.set_property()

Using `CMakeDeps.set_property()` method you can overwrite the property values set by the Conan recipes from the consumer. This can be done for every property listed above.

Imagine we have a *compressor/1.0* package that depends on *zlib/1.3.1*. The *zlib* recipe defines some properties:

Listing 108: Zlib conanfile.py

```
class ZlibConan(ConanFile):
    name = "zlib"

    ...

    def package_info(self):
        self.cpp_info.set_property("cmake_find_mode", "both")
        self.cpp_info.set_property("cmake_file_name", "ZLIB")
        self.cpp_info.set_property("cmake_target_name", "ZLIB::ZLIB")
        ...
```

This recipe defines several properties. For example the `cmake_find_mode` property is set to `both`. That means that module and config files are generated for Zlib. Maybe we need to alter this behaviour and just generate config files. You could do that in the compressor recipe using the `CMakeDeps.set_property()` method:

Listing 109: compressor conanfile.py

```
class Compressor(ConanFile):
    name = "compressor"

    requires = "zlib/1.3.1"
    ...

    def generate(self):
        deps = CMakeDeps(self)
        deps.set_property("zlib", "cmake_find_mode", "config")
        deps.generate()
        ...
```

You can also use the `set_property()` method to invalidate the property values set by the upstream recipe and use the values that Conan assigns by default. To do so, set the value `None` to the property like this:

Listing 110: compressor conanfile.py

```
class Compressor(ConanFile):
    name = "compressor"

    requires = "zlib/1.3.1"
    ...

    def generate(self):
        deps = CMakeDeps(self)
        deps.set_property("zlib", "cmake_target_name", None)
        deps.generate()
        ...
```

After doing this the generated target name for the Zlib library will be `zlib::zlib` instead of `ZLIB::ZLIB`

Additionally, *CMakeDeps.set_property()* can also be used for packages that have components. In this case, you will need to provide the package name along with its component separated by a double colon (*::*). Here's an example:

```
def generate(self):
    deps = CMakeDeps(self)
    deps.set_property("pkg::component", "cmake_target_name", <new_component_target_name>)
    deps.generate()
    ...
```

Disable CMakeDeps For Installed CMake configuration files

Some projects may want to disable the `CMakeDeps` generator for downstream consumers. This can be done by settings `cmake_find_mode` to `none`. If the project wants to provide its own configuration targets, it should append them to the `buildirs` attribute of `cpp_info`.

This method is intended to work with downstream consumers using the `CMakeToolchain` generator, which will be populated with the `builddirs` attribute.

Example:

```
def package(self):
    ...
    cmake.install()

def package_info(self):
    self.cpp_info.set_property("cmake_find_mode", "none") # Do NOT generate any files
    self.cpp_info.builddirs.append(os.path.join("lib", "cmake", "foo"))
```

Map from project configuration to imported target's configuration

As mentioned above, `CMakeDeps` provides support for multiple configuration environments (Debug, Release, etc.) This is achieved by populating properties on the imported targets according to the `build_type` setting when installing dependencies. When a consumer project is configured with a single-configuration CMake generator, however, it is necessary to define the `CMAKE_BUILD_TYPE` with a value that matches that of the installed dependencies.

If the consumer CMake project is configured with a different build type than the dependencies, it is necessary to tell CMake how to map the configurations from the current project to the imported targets by setting the `CMAKE_MAP_IMPORTED_CONFIG_<CONFIG>` CMake variable.

```
cd build-coverage/
conan install .. -s build_type=Debug
cmake .. -DCMAKE_BUILD_TYPE=Coverage -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake -
↪DCMAKE_MAP_IMPORTED_CONFIG_COVERAGE=Debug
```

CMakeConfigDeps

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

This generator (available as experimental from Conan 2.25) is designed as a replacement of the current `CMakeDeps` generator, with multiple pending fixes and improvements that couldn't easily be done in the current one without breaking.

Note: To simplify the testing and validation of the `CMakeDeps` -> `CMakeConfigDeps` migration, the `-c tools.cmake.cmakedeps:new=will_break_next` configuration can be used. It performs a hot replacement of every `CMakeDeps` in recipes by `CMakeConfigDeps`, without needing to edit the recipes at all. Note that this configuration is not necessary if the recipe explicitly uses `CMakeConfigDeps`.

That means that it can be used as any other generator in conanfiles, and in command line `-g CMakeConfigDeps`:

Listing 111: conanfile.txt

```
[requires]
hello/0.1

[generators]
CMakeConfigDeps
```

Listing 112: conanfile.py

```
class Pkg(ConanFile):
    ...
    requires = "hello/0.1"
    generators = "CMakeConfigDeps"
```

Or:

Listing 113: conanfile.py

```
from conan import ConanFile
from conan.tools.cmake import CMakeConfigDeps

class TestConan(ConanFile):
    ...
    requires = "hello/0.1"

    def generate(self):
        deps = CMakeConfigDeps(self)
        deps.generate()
```

The `CMakeConfigDeps` generator produces the necessary files for each dependency to be able to use the cmake `find_package()` function to locate the dependencies. It can be used like:

Listing 114: **CMakeLists.txt**

```
cmake_minimum_required(VERSION 3.15)
project(compressor C)

find_package(hello CONFIG REQUIRED)

add_executable(${PROJECT_NAME} src/main.c)
target_link_libraries(${PROJECT_NAME} hello::hello)
```

By default, for a `hello` requires, you need to use `find_package(hello)` and link with the target `hello::hello`.

Generated files

- **xxx-config.cmake**: By default, the `CMakeConfigDeps` generator will create config files declaring the targets for the dependencies and their components (if declared).
- This generator is only intended to generate `xxx-config.cmake` config files, it will not generate `Find*.cmake` find modules, and support for it is not planned. Use the `CMakeDeps` generator for that, or patch the consumers to use CMake config files instead of modules
- **Other necessary *.cmake**: files like version, flags and directory data or configuration.
- **conan_cmakedeps_paths.cmake**: file containing the definition of `xxx_DIR` variables that point to the folder containing the `xxx-config.cmake` files to be located when doing the `find_package()`. This file will be included by default by the `conan_toolchain.cmake`, but it might also be useful in other tools like `cmake-conan` integration. This file will also contain other path variables such as `CMAKE_VS_DEBUGGER_ENVIRONMENT`, `CMAKE_PROGRAM_PATH`, `CMAKE_LIBRARY_PATH`, etc., that help consumers to locate dependencies for different

scenarios like runtime or with other CMake `find_program()` utilities. It also prepends the package folders of the dependencies to `CMAKE_PREFIX_PATH`, as a fallback for the cases not covered by the `<pkg>_DIR` variables.

Improvements over `CMakeDeps`

This is a brief summary of the improvements and fixes of `CMakeConfigDeps` over `CMakeDeps`:

- Creates real `SHARED/STATIC/INTERFACE IMPORTED` targets, no more artificial interface targets. The `CONAN_LIB::` and other similar targets do not exist anymore.
- Defines `IMPORTED_CONFIGURATIONS` for targets.
- `CONFIG` definition of dependencies matching the dependency `Release/Debug/etc build_type`, no longer using the consumer one.
- Definition of `IMPORTED_LOCATION` and `IMPORTED_IMPLIB` for library targets.
- Definition of `LINK_LANGUAGES` based on the recipe `languages` and `cpp_info/component languages` properties.
- All these allows better propagation of linkage requirement and visibility, avoiding some linkage error of transitive shared libraries in Linux.
- Better definition of `requires` relationships across components inside the same package and with respect to other packages.
- It doesn't need any `build_context_activated` or `build_context_suffix` to use `tool_requires` dependencies.
- Checking CMake `COMPONENTS` definition by default.
- Definition of `cpp_info/components .exe` information (should include the `.location` definition too), to define `EXECUTABLE` targets that can be run.
- Executables from `requires` can also be used in non cross-build scenarios. When a `tool_requires` to the same dependency exists, then those executables will have priority.
- Creation of a new `conan_cmakedeps_paths.cmake` that contains definitions of `<pkg>_DIR` paths for direct finding of the dependencies. This file is also planned to be used in `cmake-conan` to extend its usage and avoid some current limitations due to the fact that a CMake driven installation cannot inject a toolchain later.
- Better management of the system OSX Frameworks through `cpp_info.frameworks`.
- Definition of `cpp_info/component.package_framework` information (should include the `.location` definition too, e.g., `os.path.join(self.package_folder, "MyFramework.framework", "MyFramework")`) to define the custom OSX Framework library to be linked against.
- Definition of `self.cpp_info.sources = ["src/mylib.cpp", "src/other.cpp"]` that will add `INTERFACE_SOURCES` to the CMake target to be consumed downstream.

Targets generation

CMakeConfigDeps can both generate CMake files from the dependencies package_info() information or use the in-package xxx-config.cmake files (when self.cpp_info.set_property("cmake_find_mode", "none") is defined, that indicates that the generator will not create any files.

This section explains how CMakeConfigDeps generates CMake targets from the information of the package_info() dependencies method.

cpp_info

CMakeConfigDeps will use the common *self.cpp_info* fields, like includedirs, libs, etc. to create and populate the targets the consumers will use.

But besides these common fields, there are a few new fields, exclusively used at the moment by CMakeConfigDeps, that implement some of the new capabilities of this generator that can be defined in the cpp_info or cpp_info.components:

```
# EXPERIMENTAL FIELDS, used exclusively by new CMakeConfigDeps
self.cpp_info.type  # The type of this artifact "shared-library", "static-library", etc␣
↪(same as package_type)
self.cpp_info.location # full location (path and filename with extension) of the␣
↪artifact or the Apple Framework library one
self.cpp_info.link_location  # Location of the import library for Windows .lib␣
↪associated to a dll
self.cpp_info.languages # same as "languages" attribute, it can be "C", "C++"
self.cpp_info.exe  # Definition of an executable artifact
self.cpp_info.package_framework  # Definition of an Apple Framework (new since Conan 2.
↪14)
self.cpp_info.sources  # List of paths to source files in the package (for packages that␣
↪provide source code to consumers)
```

These fields will be auto-deduced from the other cpp_info and components definitions, like the libs or libdirs fields, but the automatic deduction might have limitations. Defining them explicitly will inhibit the auto deduction and use the value as provided by the recipe.

Declare sources for targets with:

```
self.cpp_info.sources = ["src/mylib.cpp", "src/other.cpp"]
```

This allows packages to provide source code as a dependency to consumers. The paths should be relative to the package folder. The source files will be added as *INTERFACE_SOURCES* property to the CMake library (or component) target and consumers that link with this target will compile the sources when building their own targets.

Note: A package (or component) that only declares cpp_info.sources, without any other field like includedirs or libs, will still get an INTERFACE CMake target created for it (since Conan 2.33). Previously, packages that exclusively provided sources and nothing else were skipped, and no target was generated for them at all.

INTERFACE_SOURCES are only set on the target when the package declaring them is a **direct** dependency of the consumer for which the files are being generated (since Conan 2.33). If the package is only reachable transitively (for example, through another dependency using transitive_headers/transitive_libs), its target can still be generated, but without the INTERFACE_SOURCES property, to avoid the same sources being unexpectedly compiled more than once through different paths of the dependency graph.

Filtering of the target information based on requires traits

Since Conan 2.34, `CMakeConfigDeps` filters the information injected in the generated targets according to the *traits* (`headers`, `libs`, `run`...) of the requirement as seen from the consumer, so that consumers do not get compile and link information from dependencies that they do not really need to compile or link against. This is the same kind of filtering that `CMakeDeps` does.

- If `headers=False`, the header-related information is not injected: `includedirs` and `defines`.
- If `headers=False` and `libs=False`, the remaining compile and link information is not injected either: `cxxflags`, `cflags`, `sharedlinkflags`, `exelinkflags` and `system_libs`. This is typically the case of a requirement that is only needed at runtime, like `run=True`.
- If `libs=False`, but `headers=True`, the `system_libs` and the link flags of the dependency are still injected.
- For Apple `package_framework`, the framework information is only injected if `headers=True` or `libs=True`.

Some information is **not** filtered by the traits:

- The target is always declared, even if it ends up with no compile or link information, so that it can still be depended upon by other targets.
- The `IMPORTED_LOCATION` (and `IMPORTED_IMPLIB`) and the type of the library targets are always defined, even for `libs=False`. For example, if `engine` is a shared library that privately links `matrix` (`transitive_libs=False`), an application requiring `engine` doesn't link `matrix` directly, but the Linux linker still needs to locate `libmatrix.so` when linking against `libengine.so`, so the location of `matrix` is necessary.
- The `cpp_info.sources` are not filtered by `headers` or `libs`; they follow the direct dependency rule explained above.

Properties

The following properties affect the `CMakeConfigDeps` generator:

- **cmake_file_name**: The config file generated for the current package will follow the `<VALUE>-config.cmake` pattern, so to find the package you write `find_package(<VALUE>)`.
- **cmake_file_names**: Dictionary to split a package into **several** CMake config files, each one containing a subset of the package components. The keys are the names of the CMake packages (the `<KEY>-config.cmake` files, found with `find_package(<KEY>)`) and the values are dictionaries with the following fields:
  - `components`: list of the package component names that belong to that config file.
  - `properties` (optional): dictionary of properties that apply only to that config file, overriding the package-level ones. For example `system_package_version`, `cmake_config_version_compat`, `cmake_build_modules`, `cmake_extra_variables`, `cmake_additional_variables_prefixes`, `cmake_extra_dependencies` or `cmake_file_name_variants`.

  See *CMakeConfigDeps multiple config files* for the details and limitations. This property is only available since Conan 2.34, it takes precedence over **cmake_file_name** (which is ignored if both are defined), and it cannot be set from the consumer for `build_context=True` dependencies.
- **cmake_file_name_variants**: List of upper/lower case variants of the cmake file name that consumers can use to find the package. This is useful to increase the compatibility with different CMake `find_package()` calls that might be used in the consumers. An error will be raised if the list contains words that are not variants of the `cmake_file_name` value unless the consumer explicitly sets a custom `cmake_file_name`, in which case the variants will be ignored.
- **cmake_target_name**: Name of the target to be consumed.
- **cmake_target_aliases**: List of aliases that Conan will create for an already existing target.

- **cmake_find_mode**: Defaulted to `config`. Possible values are:
  - `config`: The `CMakeConfigDeps` generator will create config scripts for the dependency.
  - `module`: Not supported, will be ignored and generate only `xxx-config.cmake` files (with a warning).
  - `both`: Not supported, will be ignored and generate only `xxx-config.cmake` files (with a warning).
  - `none`: Won't generate any file. It can be used, for instance, to create a system wrapper package so the consumers find the config files in the CMake installation config path and not in the generated by Conan (because it has been skipped).
- **cmake_build_modules**: List of `.cmake` files (path relative to root package folder) that are automatically included when the consumer run the `find_package()`. This property cannot be set in the components, only in the root `self.cpp_info`.
- **cmake_config_version_compat**: By default `SameMajorVersion`, it can take the values `"AnyNewerVersion"`, `"SameMajorVersion"`, `"SameMinorVersion"`, `"ExactVersion"`. It will use that policy in the generated `<PackageName>ConfigVersion.cmake` file
- **system_package_version**: version of the package used to generate the `<PackageName>ConfigVersion.cmake` file. Can be useful when creating system packages or other wrapper packages, where the conan package version is different to the eventually referenced package version to keep compatibility to `find_package(<PackageName> <Version>)` calls.
- **cmake_additional_variables_prefixes**: List of prefixes to be used when creating CMake variables in the config files. These variables are created with `file_name` as prefix by default, but setting this property will create additional variables with the specified prefixes alongside the default `file_name` one.
- **cmake_extra_variables**: Dictionary of extra variables to be added to the generated config file. The keys of the dictionary are the variable names and the values are the variable values, which can be a plain string, a number or a dict-like python object which must specify the `value` (string/number) , `cache` (boolean), `type` (CMake cache type) and optionally, `docstring` (string: defaulted to variable name) and `force` (boolean) keys. Note that this has less preference over those values defined in the `tools.cmake.cmaketoolchain:extra_variables` conf.
- **cmake_extra_dependencies**: List of extra package names for which a `find_dependency()` call will be added to the generated `<pkg>-config.cmake` file. Useful when the package depends on libraries located via CMake's `find_package()` but not declared as Conan `requires` (for example, system-provided dependencies like `OpenMPI`). This property is only honored at the package level (setting it on a component has no effect).
- **cmake_extra_interface_libs**: List of extra libraries to be appended to the generated CMake target's `INTERFACE_LINK_LIBRARIES`. Useful to declare additional libraries (such as system libraries, or libraries located by the packages declared in `cmake_extra_dependencies`) that have to be linked together with the package's own libraries. Can be defined both at the package level and per-component.
- **cmake_link_feature**: Sets the link feature for the generated target. This can be any of the built-in link features supported by CMake like `WHOLE_ARCHIVE` etc., or a custom one, provided you also set the expected `CMAKE_LINK_LIBRARY_USING_<FEATURE>_SUPPORTED` and `CMAKE_LINK_LIBRARY_USING_<FEATURE>` variables. (Possibly using the **cmake_extra_variables** property). Supported when using CMake 3.24 or newer. This property performs **no** checks on the given feature, it is up to the recipe author to ensure that the feature is usable.
- **nosoname**: boolean value that should be used only by dependencies that are defined as `SHARED` and represent a library built without the `soname` flag option. When set to `True`, the generated target will have the `IMPORTED_NO_SONAME_<CONFIG>` CMake property set to `TRUE`, so CMake skips the SONAME check when linking. Can be set at the package level or per-component. Ignored for non-shared library types. Available since Conan 2.31.

Example:

```
def package_info(self):
    ...
    # MyFileName-config.cmake
    self.cpp_info.set_property("cmake_file_name", "MyFileName")
    # Names for targets are absolute, Conan won't add any namespace to the target names␣
↪automatically
    self.cpp_info.set_property("cmake_target_name", "Foo::Foo")
    # Automatically include the lib/mypkg.cmake file when calling find_package()
    # This property cannot be set in a component.
    self.cpp_info.set_property("cmake_build_modules", [os.path.join("lib", "mypkg.cmake
↪")])

    # Create a new target "MyFooAlias" that is an alias to the "Foo::Foo" target
    self.cpp_info.set_property("cmake_target_aliases", ["MyFooAlias"])

    # Skip this package when generating the files for the whole dependency tree in the␣
↪consumer
    # note: it will make useless the previous adjustments.
    # self.cpp_info.set_property("cmake_find_mode", "none")

    # Add extra variables to the generated config file
    self.cpp_info.set_property("cmake_extra_variables", {
                                    "FOO": 42,
                                    "CMAKE_GENERATOR_INSTANCE": "${GENERATOR_INSTANCE}/
↪buildTools/",
                                    "CACHE_VAR_DEFAULT_DOC": {"value": "hello world",
                                                              "cache": True, "type":
↪"STRING"}
                                })

    self.cpp_info.set_property("cmake_link_feature", "WHOLE_ARCHIVE")
    # Additional names that consumers of this library might usually use in their find_
↪package() calls,
    # so they can find the same config file with different lower/upper case variants.
    self.cpp_info.set_property("cmake_file_name_variants", ["myfilename", "MYFILENAME"])

    # Emit ``find_dependency(MyOpenMPI REQUIRED)`` in the generated <pkg>-config.cmake,
    # so consumers can locate dependencies not provided as Conan requires.
    self.cpp_info.set_property("cmake_extra_dependencies", ["MyOpenMPI"])

    # Append extra libraries to the generated target's INTERFACE_LINK_LIBRARIES
    self.cpp_info.set_property("cmake_extra_interface_libs", ["MyOpenMPI::MyOpenMPI"])

# Or if using components
def package_info(self):
    self.cpp_info.components["mycomponent"].set_property("cmake_target_name", "Foo::Var")

    # Create a new target "VarComponent" that is an alias to the "Foo::Var" component␣
↪target
    self.cpp_info.components["mycomponent"].set_property("cmake_target_aliases", [
↪"VarComponent"])

    # ``cmake_extra_interface_libs`` can also be set on a single component,
```

(continues on next page)

(continued from previous page)

```
    # in which case it is appended only to that component's INTERFACE_LINK_LIBRARIES.
    self.cpp_info.components["mycomponent"].set_property("cmake_extra_interface_libs",
                                                         ["MyOpenMPI::MyOpenMPI"])
```

Using `CMakeConfigDeps.set_property()` method you can overwrite the property values set by the Conan recipes from the consumer. This can be done for every property listed above.

Listing 115: conanfile.py

```
class Pkg(ConanFile):

    requires = "zlib/1.3.1"
    ...

    def generate(self):
        deps = CMakeConfigDeps(self)
        # This will require in CMakeLists.txt a
        # target_link_libraries(mytarget PUBLIC MyZlib::MyZlib)
        deps.set_property("zlib", "cmake_target_name", "MyZlib::MyZlib")
        deps.generate()
```

You can also use the `set_property()` method to invalidate the property values set by the upstream recipe and use the values that Conan assigns by default. To do so, set the value `None` to the property like `deps.set_property("zlib", "cmake_target_name", None)`.

After doing this the generated target name for the Zlib library will be `zlib::zlib` instead of `ZLIB::ZLIB` which is the upstream default target name.

Additionally, *CMakeConfigDeps.set_property()* can also be used for packages that have components. In this case, you will need to provide the package name along with its component separated by a double colon (*::*). Here's an example:

```
def generate(self):
    deps = CMakeConfigDeps(self)
    deps.set_property("pkg::component", "cmake_target_name", <new_component_target_name>)
    deps.generate()
```

Multiple CMake config files per package

By default, `CMakeConfigDeps` generates a single `<name>-config.cmake` file per package, containing the targets of all its components. However, some libraries distribute **several CMake packages** from a single project, for example `find_package(SPIRV-Tools)` and `find_package(SPIRV-Tools-opt)`. To be able to model that from a single Conan package, use the `cmake_file_names` property (since Conan 2.34) to define which components belong to which CMake config file:

Listing 116: conanfile.py

```
def package_info(self):
    self.cpp_info.set_property("cmake_file_names", {
        "SPIRV-Tools": {"components": ["spirv-tools-core"]},
        "SPIRV-Tools-opt": {"components": ["spirv-tools-opt"]},
        "SPIRV-Tools-link": {
            "components": ["spirv-tools-link"],
```

(continues on next page)

(continued from previous page)

```
            # Properties that only apply to the SPIRV-Tools-link config files
            "properties": {"cmake_config_version_compat": "ExactVersion"},
        },
    })

    self.cpp_info.components["spirv-tools-core"].libs = ["SPIRV-Tools"]
    self.cpp_info.components["spirv-tools-opt"].libs = ["SPIRV-Tools-opt"]
    self.cpp_info.components["spirv-tools-opt"].requires = ["spirv-tools-core"]
    self.cpp_info.components["spirv-tools-link"].libs = ["SPIRV-Tools-link"]
    self.cpp_info.components["spirv-tools-link"].requires = ["spirv-tools-core", "spirv-
↪tools-opt"]
```

The generator will create a complete set of files for each key (`SPIRV-ToolsConfig.cmake`, `SPIRV-Tools-optConfig.cmake`, `SPIRV-Tools-linkConfig.cmake`, their `*Targets.cmake`, version files, etc.), and a `<KEY>_DIR` variable for each of them in `conan_cmakedeps_paths.cmake`. Consumers then use the `find_package()` of the CMake packages they need, and link the component targets directly:

Listing 117: CMakeLists.txt

```
find_package(SPIRV-Tools-link CONFIG REQUIRED)

add_executable(example src/main.cpp)
target_link_libraries(example PRIVATE SPIRV-Tools-link)
```

When a component of one config file requires components that live in other config files (from the same or from other Conan packages), the generated `<KEY>-config.cmake` automatically calls `find_dependency()` on the right config files.

When `cmake_file_names` is defined, these rules apply:

- Every component of the package must be listed in exactly one of the files. A `ConanException` is raised if a component does not exist, or if it is not added to any of the files.
- There is **no root config file** named after the package, and no root `pkg::pkg` target is created, so `default_components` are ignored. The targets are not aggregated either, because each config file declares only the targets of its own components.
- Consumers that require a package using `cmake_file_names` must declare which component they link with, using `self.cpp_info.requires = ["pkg::component"]` in their `package_info()`. Otherwise, a `ConanException` is raised, because there is no `pkg::pkg` target to link against.
- Each component must define at most one library in `libs`.
- It takes precedence over `cmake_file_name`, which is ignored if both properties are defined.
- Properties like `cmake_build_modules`, `cmake_extra_variables` or `system_package_version` defined in the `properties` field of a file, apply only to that file. The rest of properties are read as usual.
- The legacy variables of each config file (like `<KEY>_INCLUDE_DIRS` or `<KEY>_LIBRARIES`) only contain the information of the components of that file. The `<KEY>_VERSION_STRING` is also generated for each file.

Consumers can also redefine the file names and their components with `CMakeConfigDeps.set_property()`, in the same way as any other property:

Listing 118: conanfile.py

```
def generate(self):
    deps = CMakeConfigDeps(self)
    deps.set_property("spirv-tools", "cmake_file_names", {
        "SPIRV-Tools": {"components": ["spirv-tools-core", "spirv-tools-opt"]},
        "SPIRV-Tools-link": {"components": ["spirv-tools-link"]},
    })
    deps.generate()
```

Note: `cmake_file_names` cannot be set with `deps.set_property(..., build_context=True)` for dependencies in the build context.

Configuration

Allows to define custom user CMake configuration besides the standard Release, Debug, etc ones.

```
def generate(self):
    deps = CMakeConfigDeps(self)
    # By default, ``deps.configuration`` will be ``self.settings.build_type``
    if self.options["hello"].shared:
        # Assuming the current project ``CMakeLists.txt`` defines the ReleasedShared␣
↪configuration.
        deps.configuration = "ReleaseShared"
    deps.generate()
```

The `CMakeConfigDeps` is a *multi-configuration* generator, it can correctly create files for Release/Debug configurations to be simultaneously used by IDEs like Visual Studio. In single configuration environments, it is necessary to have a configuration defined, which must be provided via the `cmake ... -DCMAKE_BUILD_TYPE=<build-type>` argument in command line (Conan will do it automatically when necessary, in the `CMake.configure()` helper).

Using "host" and "build" context dependencies

With `CMakeConfigDeps`, it is possible to use executable `IMPORTED` targets from regular "host" dependencies. This can only be done in case where platform binary compatibility is possible, and in other cases like cross-compiling, it will fail.

The `IMPORTED` executable targets will be automatically created from the `self.cpp_info.exe` and `self.cpp_info.components["mycomponent"].exe` information. These targets can be directly used by consumers in their own builds as tools.

When a Conan package recipe depends on the same package both as regular `requires` and `tool-requires`, to be able to implement scenarios like cross-building, the "build" context will have priority when defining the executable `IMPORTED` targets. A common example is when using `protobuf` in a cross-building scenario, the `protoc` compiler inside the package that is used for the "host" architecture cannot be executed in the current "build" machine. So the recipe defines a `self.requires("protobuf/version")` and a `self.tool_requires("protobuf/version")`. In this case, the library (headers, static/shared lib) targets will be obtained from the dependency in the "host" context, but the executable target for `protoc` will be defined from the "build" context.

Disable CMakeConfigDeps to use xxx-config.cmake files inside the package

Some projects may want to disable the CMakeConfigDeps generator for downstream consumers. This can be done by setting cmake_find_mode to "none". If the project wants to provide its own configuration targets, it should append them to the builddirs attribute of cpp_info.

This method is intended to work with downstream consumers using the CMakeToolchain generator or by including the generated conan_cmakedeps_paths.cmake file, which will be populated with the builddirs attribute.

Example:

```
def package(self):
    ...
    cmake.install()

def package_info(self):
    self.cpp_info.set_property("cmake_find_mode", "none") # Do NOT generate any files
    self.cpp_info.builddirs.append(os.path.join("lib", "cmake", "foo"))
```

Reference

class CMakeConfigDeps(*conanfile*)

Parameters
conanfile – < ConanFile object > The current recipe object. Always use self.

generate()

This method will save the generated files to the conanfile.generators_folder folder

set_property(*dep*, *prop*, *value*, *build_context=False*)

Using this method you can overwrite the *property* values set by the Conan recipes from the consumer.

Parameters

- **dep** – Name of the dependency to set the *property*. For components use the syntax: dep_name::component_name.
- **prop** – Name of the *property*.
- **value** – Value of the property. Use None to invalidate any value set by the upstream recipe.
- **build_context** – Set to True if you want to set the property for a dependency that belongs to the build context (False by default). cmake_file_names cannot be set when this is True.

CMakeToolchain

The CMakeToolchain is the toolchain generator for CMake. It produces the toolchain file that can be used in the command line invocation of CMake with the -DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake. This generator translates the current package configuration, settings, and options, into CMake toolchain syntax.

It can be declared as:

```
from conan import ConanFile

```

(continues on next page)

(continued from previous page)

```
class Pkg(ConanFile):
    generators = "CMakeToolchain"
```

Or fully instantiated in the `generate()` method:

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"
    generators = "CMakeDeps"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    def generate(self):
        tc = CMakeToolchain(self)
        tc.variables["MYVAR"] = "MYVAR_VALUE"
        tc.preprocessor_definitions["MYDEFINE"] = "MYDEF_VALUE"
        tc.generate()
```

Note: The `CMakeToolchain` is intended to run with the `CMakeDeps` dependencies generator. Please do not use other CMake legacy generators (like `cmake`, or `cmake_paths`) with it.

Generated files

This will generate the following files after a `conan install` (or when building the package in the cache) with the information provided in the `generate()` method as well as information translated from the current `settings`:

- **conan_toolchain.cmake**: containing the translation of Conan settings to CMake variables. Some things that will be defined in this file:
  - Definition of the CMake generator platform and generator toolset
  - Definition of the `CMAKE_POSITION_INDEPENDENT_CODE`, based on `fPIC` option.
  - Definition of the C++ standard as necessary
  - Definition of the standard library used for C++
  - Deactivation of rpaths in OSX
  - Definition of `CMAKE_VS_DEBUGGER_ENVIRONMENT` when on Windows with Visual Studio. This sets up the `PATH` environment variable to point to directories containing DLLs, to allow debugging directly from the Visual Studio IDE without copying DLLs (requires CMake 3.27).
  - Definition of `CONAN_RUNTIME_LIB_DIRS` to allow collecting runtime dependencies (shared libraries), see below for details.
- **conanvcvars.bat**: In some cases, the Visual Studio environment needs to be defined correctly for building, like when using the Ninja or NMake generators. If necessary, the `CMakeToolchain` will generate this script, so defining the correct Visual Studio prompt is easier.

- **CMakePresets.json**: This toolchain generates a standard *CMakePresets.json* file. For more information, refer to the documentation here. It currently uses version "3" of the JSON schema. Conan adds *configure*, *build*, and *test* preset entries to the JSON file:
  - ***configurePresets* storing the following information:**
    - The *generator* to be used.
    - The path to the *conan_toolchain.cmake*.
    - Cache variables corresponding to the specified settings that cannot work if specified in the toolchain.
    - The *CMAKE_BUILD_TYPE* variable for single-configuration generators.
    - The *BUILD_TESTING* variable set to *OFF* when the configuration *tools.build:skip_test* is true.
    - An environment section, setting all the environment information related to the *VirtualBuildEnv*, if applicable. This environment can be modified in the *generate()* method of the recipe by passing an environment through the *CMakeToolchain.presets_build_environment* attribute. Generation of this section can be skipped by using the *tools.cmake.cmaketoolchain:presets_environment* configuration.
    - By default, preset names will be *conan-xxxx*, but the "conan-" prefix can be customized with the *CMakeToolchain.presets_prefix = "conan"* attribute.
    - Preset names are controlled by the *layout() self.folders.build_folder_vars* definition, which can contain a list of settings, options, `self.name` and `self.version` and constants `const.xxx` like *["settings.compiler", "settings.arch", "options.shared", "const.myname"]*.
    - If CMake is found as a direct *tool_requires* dependency, or if *tools.cmake:cmake_program* is set, the configure preset will include a *cmakeExecutable* field. This field represents the path to the CMake executable to be used for this preset. As stated in the CMake documentation, this field is reserved for use by IDEs and is not utilized by CMake itself.
  - ***buildPresets* storing the following information:**
    - The *configurePreset* associated with this build preset.
  - ***testPresets* storing the following information:**
    - The *configurePreset* associated with this build preset.
    - An environment section, setting all the environment information related to the *VirtualRunEnv*, if applicable. This environment can be modified in the *generate()* method of the recipe by passing an environment through the *CMakeToolchain.presets_run_environment* attribute. Please note that since this preset inherits from a *configurePreset*, it will also inherit its environment. Generation of this section can be skipped by using the`tools.cmake.cmaketoolchain:presets_environment` configuration.
- **CMakeUserPresets.json**: If you declare a `layout()` in the recipe and your `CMakeLists.txt` file is found at the `conanfile.source_folder` folder, a `CMakeUserPresets.json` file will be generated (if doesn't exist already) including automatically the `CMakePresets.json` (at the `conanfile.generators_folder`) to allow your IDE (Visual Studio, Visual Studio Code, CLion...) or `cmake` tool to locate the `CMakePresets.json`. The location of the generated `CMakeUserPresets.json` can be further tweaked by the `user_presets_path` attribute, as documented below. The version schema of the generated `CMakeUserPresets.json` is "4" and requires CMake `>=` 3.23. The file name of this file can be configured with the `CMakeToolchain.user_presets_path = "CMakeUserPresets.json"` attribute, so if you want to generate a "ConanPresets.json" instead to be included from your own file, you can define `tc.user_presets_path = "ConanPresets.json"` in the `generate()` method. See *extending your own CMake presets* for a full example.

Note: Conan will skip the generation of the `CMakeUserPresets.json` if it already exists and was not generated by Conan.

Note: To list all available presets, use the `cmake --list-presets` command:

Note: The version schema of the generated `CMakeUserPresets.json` is 4 (compatible with CMake>=3.23) and the schema for the `CMakePresets.json` is 3 (compatible with CMake>=3.21).

CONAN_RUNTIME_LIB_DIRS

This variable in the generated `conan_toolchain.cmake` file contains a list of directories that contain runtime libraries (like DLLs) from all dependencies in the host context. This is intended to be used when relying on CMake functionality to collect shared libraries to create a relocatable bundle, as per the example below.

Just pass the `CONAN_RUNTIME_LIB_DIRS` variable to the `DIRECTORIES` argument in the `install(RUNTIME_DEPENDENCY_SET ...)` invocation.

```
install(RUNTIME_DEPENDENCY_SET my_app_deps
    PRE_EXCLUDE_REGEXES
        [[api-ms-win-.*]]
        [[ext-ms-.*]]
        [[kernel32\.dll]]
        [[libc\.so\..*]] [[libgcc_s\.so\..*]] [[libm\.so\..*]] [[libstdc\+\+\.so\..*]]
    POST_EXCLUDE_REGEXES
        [[.*/system32/.*\.dll]]
        [[^/lib.*]]
        [[^/usr/lib.*]]
    DIRECTORIES ${CONAN_RUNTIME_LIB_DIRS}
)
```

Customization

preprocessor_definitions

This attribute allows defining compiler preprocessor definitions, for multiple configurations (Debug, Release, etc).

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.preprocessor_definitions["MYDEF"] = "MyValue"
    tc.preprocessor_definitions.debug["MYCONFIGDEF"] = "MyDebugValue"
    tc.preprocessor_definitions.release["MYCONFIGDEF"] = "MyReleaseValue"
    # Setting to None will add the definition with no value
    tc.preprocessor_definitions["NOVALUE_DEF"] = None
    tc.generate()
```

This will be translated to:

- One `add_compile_definitions()` definition for `MYDEF` in `conan_toolchain.cmake` file.
- One `add_compile_definitions()` definition, using a cmake generator expression in `conan_toolchain.cmake` file, using the different values for different configurations.

cache_variables

This attribute allows defining CMake cache-variables. These variables, unlike the `variables`, are single-config. They will be stored in the `CMakePresets.json` file (at the *cacheVariables* in the *configurePreset*) and will be applied with `-D` arguments when calling `cmake.configure` using the *CMake() build helper*.

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.cache_variables["foo"] = True
    tc.cache_variables["foo2"] = False
    tc.cache_variables["var"] = "23"
```

The booleans assigned to a cache_variable will be translated to `ON` and `OFF` symbols in CMake.

variables

This attribute allows defining CMake variables, for multiple configurations (Debug, Release, etc). These variables should be used to define things related to the toolchain and for the majority of cases *cache_variables* is what you probably want to use. Also, take into account that as these variables are defined inside the *conan_toolchain.cmake* file, and the toolchain is loaded several times by CMake, the definition of these variables will be done at those points as well.

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.variables["MYVAR"] = "MyValue"
    tc.variables.debug["MYCONFIGVAR"] = "MyDebugValue"
    tc.variables.release["MYCONFIGVAR"] = "MyReleaseValue"
    tc.generate()
```

This will be translated to:

- One `set()` definition for `MYVAR` in `conan_toolchain.cmake` file.
- One `set()` definition, using a cmake generator expression in `conan_toolchain.cmake` file, using the different values for different configurations.

The booleans assigned to a variable will be translated to `ON` and `OFF` symbols in CMake:

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.variables["FOO"] = True
    tc.variables["VAR"] = False
    tc.generate()
```

Will generate the sentences: `set(FOO ON ...)` and `set(VAR OFF ...)`.

user_presets_path

This attribute allows specifying the location of the generated `CMakeUserPresets.json` file. Accepted values:

- An absolute path
- A path relative to `self.source_folder`
- The boolean value `False`, to suppress the generation of the file altogether.

For example, we can prevent the generator from creating `CMakeUserPresets.json` in the following way:

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.user_presets_path = False
    tc.generate()
```

It's also possible to use the `tools.cmake.cmaketoolchain:user_presets` experimental configuration to change the name and location of the `CMakeUserPresets.json` file. Assigning it to an empty string will disable the generation of the file. Please check the *conf section* for more information.

If the value assigned to `tools.cmake.cmaketoolchain:user_presets` contains a directory component, e.g. `presets/CMakeUserPresets.json`, Conan will rely on the CMake `--preset-file` mechanism (available since CMake 4.4) to let the user presets file live outside the CMake root directory. In that case, the commands suggested by Conan to configure and build the project will be like:

```
$ cmake --preset-file presets/CMakeUserPresets.json --preset conan-release
```

presets_build_environment, presets_run_environment

These attributes enable the modification of the build and run environments associated with the presets, respectively, by assigning an *Environment*. This can be accomplished in the *generate()* method.

For example, you can override the value of an environment variable already set in the build environment:

```
def generate(self):
    buildenv = VirtualBuildEnv(self)
    buildenv.environment().define("MY_BUILD_VAR", "MY_BUILDVAR_VALUE_OVERRIDDEN")
    buildenv.generate()

    tc = CMakeToolchain(self)
    tc.presets_build_environment = buildenv.environment()
    tc.generate()
```

Or generate a new environment and compose it with an already existing one:

```
def generate(self):
    runenv = VirtualRunEnv(self)
    runenv.environment().define("MY_RUN_VAR", "MY_RUNVAR_SET_IN_GENERATE")
    runenv.generate()

    env = Environment()
    env.define("MY_ENV_VAR", "MY_ENV_VAR_VALUE")
    env = env.vars(self, scope="run")
    env.save_script("other_env")
```

(continues on next page)

(continued from previous page)

```
    tc = CMakeToolchain(self)
    tc.presets_run_environment = runenv.environment().compose_env(env)
    tc.generate()
```

Extra compilation flags

You can use the following attributes to append extra compilation flags to the toolchain:

- **extra_cxxflags** (defaulted to `[]`) for additional cxxflags
- **extra_cflags** (defaulted to `[]`) for additional cflags
- **extra_sharedlinkflags** (defaulted to `[]`) for additional shared link flags
- **extra_exelinkflags** (defaulted to `[]`) for additional exe link flags

---

Note: flags order of preference: Flags specified in the *tools.build* configuration, such as *cxxflags*, *cflags*, *sharedlinkflags* and *exelinkflags*, will always take precedence over those set by the CMakeToolchain attributes.

---

presets_prefix

By default it is `"conan"`, and it will generate CMake presets named "conan-xxxx". This is done to avoid potential name clashes with users own presets.

absolute_paths

By default, `CMakeToolchain` will generate relative paths. For example the `CMakeUserPresets.json` will have a relative path to the included `CMakePresets.json` (both files generated by `CMakeToolchain`), and the `CMakePresets.json` file will have a relative path to the `conan_toolchain.cmake` file defined in its `toolchainFile` field, that will be relative to the build folder, as specified by the CMake presets documentation.

If for some reason using absolute paths was desired, it is possible to do it with:

```
def generate(self):
    tc = CMakeToolchain(self)
    tc.absolute_paths = True
    tc.generate()
```

add_rpath_link

Setting this attribute to `True` will add the `-Wl,-rpath-link,` flag to the linker flags, pointing to all library directories for all dependencies in the host context. Should only be added on platforms where the linker supports the `-rpath-link` flag, like Linux. This is needed in some cases when linking executables (typically when cross-building), when there are indirect shared library dependencies. This attribute will unconditionally add the `-Wl,-rpath-link` flags, so if the recipe is intended to be built on other platforms different to Linux/gcc (or more correctly, the Gnu `ld` linker), then it would be necessary to define this attribute conditionally to the platform and compiler, otherwise it can produce errors in other compilers that don't recognize this flag.

Note: Should not be needed when using the newer *CMakeConfigDeps* generator.

Using a custom toolchain file

There are two ways of providing custom CMake toolchain files:

- The `conan_toolchain.cmake` file can be completely skipped and replaced by a user one, defining the `tools.cmake.cmaketoolchain:toolchain_file=<filepath>` configuration value. Note this approach will translate all the toolchain responsibility to the user provided toolchain, but things like locating the necessary `xxx-config.cmake` files from dependencies can be challenging without some help. For this reason, using the following `tools.cmake.cmaketoolchain:user_toolchain` is recommended in most cases, and if necessary, using `tools.cmake.cmaketoolchain:enabled_blocks` can be used.
- A custom user toolchain file can be added (included from) to the `conan_toolchain.cmake` one, by using the `user_toolchain` block described below, and defining the `tools.cmake.cmaketoolchain:user_toolchain=["<filepath>"]` configuration value.

  The configuration `tools.cmake.cmaketoolchain:user_toolchain=["<filepath>"]` can be defined in the `global.conf`. but also creating a Conan package for your toolchain and using `self.conf_info` to declare the toolchain file:

  ```
  import os
  from conan import ConanFile
  class MyToolchainPackage(ConanFile):
      ...
      def package_info(self):
          f = os.path.join(self.package_folder, "mytoolchain.cmake")
          self.conf_info.define("tools.cmake.cmaketoolchain:user_toolchain",␣
  ↪[f])
  ```

  If you declare the previous package as a `tool_require`, the toolchain will be automatically applied.

- If you have more than one `tool_requires` defined, you can easily append all the user toolchain values together using the `append` method in each of them, for instance:

  ```
  import os
  from conan import ConanFile
  class MyToolRequire(ConanFile):
      ...
      def package_info(self):
          f = os.path.join(self.package_folder, "mytoolchain.cmake")
          # Appending the value to any existing one
          self.conf_info.append("tools.cmake.cmaketoolchain:user_toolchain",␣
  ↪f)
  ```

  So, they'll be automatically applied by your `CMakeToolchain` generator without writing any extra code:

  ```
  from conan import ConanFile
  from conan.tools.cmake import CMake
  class Pkg(ConanFile):
      settings = "os", "compiler", "arch", "build_type"
      exports_sources = "CMakeLists.txt"
      tool_requires = "toolchain1/0.1", "toolchain2/0.1"
  ```

(continues on next page)

(continued from previous page)

```
    generators = "CMakeToolchain"

    def build(self):
        cmake = CMake(self)
        cmake.configure()
```

Note: Important notes

- In most cases, `tools.cmake.cmaketoolchain:user_toolchain` will be preferred over `tools.cmake.cmaketoolchain:toolchain_file`
- The `user_toolchain` files can define variables for cross-building, such as `CMAKE_SYSTEM_NAME`, `CMAKE_SYSTEM_VERSION` and `CMAKE_SYSTEM_PROCESSOR`. If these variables are defined in the user toolchain file, they will be respected, and the `conan_toolchain.cmake` deduced ones will not overwrite the user defined ones. If those variables are not defined in the user toolchain file, then the Conan automatically deduced ones will be used. Those variables defined in the `user_toolchain` files will also have higher precedence than the configuration defined ones like `tools.cmake.cmaketoolchain:system_name`.
- The usage of `tools.cmake.cmaketoolchain:enabled_blocks` can be used together with `tools.cmake.cmaketoolchain:user_toolchain` to enable only certain blocks but avoid CMakeToolchain to override CMake values defined in the user toolchain file.

Extending and advanced customization

`CMakeToolchain` implements a powerful capability for extending and customizing the resulting toolchain file.

The contents are organized by `blocks` that can be customized. The following predefined blocks are available, and added in this order:

- **user_toolchain**: Allows to include user toolchains from the `conan_toolchain.cmake` file. If the configuration `tools.cmake.cmaketoolchain:user_toolchain=["xxxx", "yyyy"]` is defined, its values will be `include(xxx)\ninclude(yyyy)` as the first lines in `conan_toolchain.cmake`.
- **generic_system**: Defines `CMAKE_SYSTEM_NAME`, `CMAKE_SYSTEM_VERSION`, `CMAKE_SYSTEM_PROCESSOR`, `CMAKE_GENERATOR_PLATFORM`, `CMAKE_GENERATOR_TOOLSET`
- **compilers**: Defines `CMAKE_<LANG>_COMPILER` for different languages, as defined by `tools.build:compiler_executables` configuration.
- **android_system**: Defines `ANDROID_PLATFORM`, `ANDROID_STL`, `ANDROID_ABI` and includes `ANDROID_NDK_PATH/build/cmake/android.toolchain.cmake` where `ANDROID_NDK_PATH` comes defined in `tools.android:ndk_path` configuration value.
- **apple_system**: Defines `CMAKE_OSX_ARCHITECTURES` (see the *universal binaries section*), `CMAKE_OSX_SYSROOT` for Apple systems. It also defines a default for `CMAKE_Swift_COMPILER_TARGET` from the Apple settings, so Swift code can be cross-built for iOS, watchOS, tvOS, visionOS and Mac Catalyst. This only takes effect with the `Ninja` generator, as the Xcode generator ignores this variable, so `Ninja` is the recommended generator for Swift.
- **fpic**: Defines the `CMAKE_POSITION_INDEPENDENT_CODE` when there is a `options.fPIC`
- **arch_flags**: Defines C/C++ flags like `-m32, -m64` when necessary.
- **linker_scripts**: Defines the flags for any provided linker scripts.
- **libcxx**: Defines `-stdlib=libc++` flag when necessary as well as `_GLIBCXX_USE_CXX11_ABI`.

- **vs_runtime**: Defines the `CMAKE_MSVC_RUNTIME_LIBRARY` variable, as a generator expression for multiple configurations.
- **vs_debugger_environment**: Defines `CMAKE_VS_DEBUGGER_ENVIRONMENT` from "bindirs" folders of dependencies, exclusively for Visual Studio.
- **cppstd**: defines `CMAKE_CXX_STANDARD`, `CMAKE_CXX_EXTENSIONS`
- **parallel**: defines `/MP` parallel build flag for Visual.
- **extra_flags**: Adds extra definitions, compile and link flags from `tools.build:cxxflags`, `tools.build:cflags`, `tools.build:defines`, `tools.build:sharedlinkflags`, `tools.build:rcflags`, etc.
- **cmake_flags_init**: defines `CMAKE_XXX_FLAGS` variables based on previously defined Conan variables. The blocks above only define `CONAN_XXX` variables, and this block will define CMake ones like `set(CMAKE_CXX_FLAGS_INIT "${CONAN_CXX_FLAGS}" CACHE STRING "" FORCE)`.
- **extra_variables**: Definition of extra CMake variables from `tools.cmake.cmaketoolchain:extra_variables`
- **try_compile**: Stop processing the toolchain, skipping the blocks below this one, if `IN_TRY_COMPILE` CMake property is defined.
- **find_paths**: Defines `CMAKE_FIND_PACKAGE_PREFER_CONFIG`, `CMAKE_MODULE_PATH`, `CMAKE_PREFIX_PATH` so the generated files from `CMakeDeps` are found.
- **pkg_config**: Defines `PKG_CONFIG_EXECUTABLE` based on `tools.gnu:pkg_config` and adds `CMAKE_CURRENT_LIST_DIR` to `ENV{PKG_CONFIG_PATH}` to let pkg-config find generated .pc files.
- **rpath**: Defines `CMAKE_SKIP_RPATH`. By default it is disabled, and it is needed to define `self.blocks["rpath"].skip_rpath=True` if you want to activate `CMAKE_SKIP_RPATH`
- **shared**: defines `BUILD_SHARED_LIBS`.
- **output_dirs**: Define the `CMAKE_INSTALL_XXX` variables.
  - **CMAKE_INSTALL_PREFIX**: Is set with the `package_folder`, so if a "cmake install" operation is run, the artifacts go to that location.
  - **CMAKE_INSTALL_BINDIR**, **CMAKE_INSTALL_SBINDIR** and **CMAKE_INSTALL_LIBEXECDIR**: Set by default to `bin`.
  - **CMAKE_INSTALL_LIBDIR**: Set by default to `lib`.
  - **CMAKE_INSTALL_INCLUDEDIR** and **CMAKE_INSTALL_OLDINCLUDEDIR**: Set by default to `include`.
  - **CMAKE_INSTALL_DATAROOTDIR**: Set by default to `res`.

  If you want to change the default values, adjust the `cpp.package` object at the `layout()` method:

```
def layout(self):
    ...
    # For CMAKE_INSTALL_BINDIR, CMAKE_INSTALL_SBINDIR and CMAKE_
↪INSTALL_LIBEXECDIR, takes the first value:
    self.cpp.package.bindirs = ["mybin"]
    # For CMAKE_INSTALL_LIBDIR, takes the first value:
    self.cpp.package.libdirs = ["mylib"]
    # For CMAKE_INSTALL_INCLUDEDIR, CMAKE_INSTALL_OLDINCLUDEDIR,␣
↪takes the first value:
    self.cpp.package.includedirs = ["myinclude"]
    # For CMAKE_INSTALL_DATAROOTDIR, takes the first value:
    self.cpp.package.resdirs = ["myres"]
```

> **Note:** It is **not valid** to change the self.cpp_info at the `package_info()` method, the `self.cpp.package` needs to be defined instead.

- **variables**: Define CMake variables from the `CMakeToolchain.variables` attribute.
- **preprocessor**: Define preprocessor directives from `CMakeToolchain.preprocessor_definitions` attribute

Customizing the content blocks

Every block can be customized in different ways (recall to call `tc.generate()` after the customization):

```
# tc.generate() should be called at the end of every one

# remove an existing block, the generated conan_toolchain.cmake
# will not contain code for that block at all
def generate(self):
    tc = CMakeToolchain(self)
    tc.blocks.remove("generic_system")

# remove several blocks
def generate(self):
    tc = CMakeToolchain(self)
    tc.blocks.remove("generic_system", "cmake_flags_init")

# LEGACY: keep one block, remove all the others
# If you want to generate conan_toolchain.cmake with only that
# block. Use "tc.blocks.enabled()" instead
def generate(self):
    tc = CMakeToolchain(self)
    # this still leaves blocks "variables" and "preprocessor"
    # use "tc.blocks.enabled()"" instead
    tc.blocks.select("generic_system")

# LEGACY: keep several blocks, remove the other blocks
# Use "tc.blocks.enabled()" instead
def generate(self):
    tc = CMakeToolchain(self)
    # this still leaves blocks "variables" and "preprocessor"
    # use "tc.blocks.enabled()" instead
    tc.blocks.select("generic_system", "cmake_flags_init")

# keep several blocks, remove the other blocks
# This can be done from configuration with
# tools.cmake.cmaketoolchain:enabled_blocs
def generate(self):
    tc = CMakeToolchain(self)
    # Discard all the other blocks except ``generic_system``
    tc.blocks.enabled("generic_system")

# iterate blocks
def generate(self):
```

(continues on next page)

(continued from previous page)

```
    tc = CMakeToolchain(self)
    for block_name in tc.blocks.keys():
        # do something with block_name
    for block_name, block in tc.blocks.items():
        # do something with block_name and block

# modify the template of an existing block
def generate(self):
    tc = CMakeToolchain(self)
    tmp = tc.blocks["generic_system"].template
    new_tmp = tmp.replace(...)  # replace, fully replace, append...
    tc.blocks["generic_system"].template = new_tmp

# modify one or more variables of the context
def generate(self):
    tc = CMakeToolchain(conanfile)
    # block.values is the context dictionary
    toolset = tc.blocks["generic_system"].values["toolset"]
    tc.blocks["generic_system"].values["toolset"] = "other_toolset"

# modify the whole context values
def generate(self):
    tc = CMakeToolchain(conanfile)
    tc.blocks["generic_system"].values = {"toolset": "other_toolset"}

# modify the context method of an existing block
import types

def generate(self):
    tc = CMakeToolchain(self)
    generic_block = toolchain.blocks["generic_system"]

    def context(self):
        assert self  # Your own custom logic here
        return {"toolset": "other_toolset"}
    generic_block.context = types.MethodType(context, generic_block)

# completely replace existing block
from conan.tools.cmake import CMakeToolchain

def generate(self):
    tc = CMakeToolchain(self)
    # this could go to a python_requires
    class MyGenericBlock:
        template = "HelloWorld"

        def context(self):
            return {}

    tc.blocks["generic_system"] = MyGenericBlock

# add a completely new block
```

(continues on next page)

(continued from previous page)

```
from conan.tools.cmake import CMakeToolchain
def generate(self):
    tc = CMakeToolchain(self)
    # this could go to a python_requires
    class MyBlock:
        template = "Hello {{myvar}}!!!"

        def context(self):
            return {"myvar": "World"}

    tc.blocks["mynewblock"] = MyBlock
```

It is possible to select which blocks are active from configuration in profiles, using the `tools.cmake.cmaketoolchain:enabled_blocks` configuration. This is a list of blocks, so doing:

```
[conf]
tools.cmake.cmaketoolchain:enabled_blocks=["generic_system"]
```

Will leave only the `generic_system` block, and discard all others. This feature can be used for example when users are providing their own toolchain files, and they don't need Conan `CMakeToolchain` to define any flags or CMake variables, except for the necessary paths so dependencies can be found. For this case, it should be possible to do something like:

```
[conf]
tools.cmake.cmaketoolchain:user_toolchain+=my_user_toolchain.cmake
tools.cmake.cmaketoolchain:enabled_blocks=["find_paths"]
```

For more information about these blocks, please have a look at the source code.

Finding dependencies paths

The generated `conan_toolchain.cmake` contains information in its `find_paths` block for variables such as `CMAKE_PROGRAM_PATH`, `CMAKE_LIBRARY_PATH`, `CMAKE_INCLUDE_PATH` and others, that allow CMake to run `find_program()`, `find_file()` and other special "finder" routines that find artifacts without a explicit package and targets definition via the overall recommended `find_package()`.

With the new `CMakeConfigDeps` generator, the `conan_toolchain.cmake` block `find_paths` no longer defines the information itself, but it just loads a new file generated by the `CMakeConfigDeps` generator, the `conan_cmakedeps_paths.cmake` file. This way, the responsibility for creating information about dependencies is the `CMakeConfigDeps` generator, and that new file can be used in some scenarios in which passing a toolchain is not possible.

Cross building

The `generic_system` block contains some basic cross-building capabilities. In the general case, the user would want to provide their own user toolchain defining all the specifics, which can be done with the configuration `tools.cmake.cmaketoolchain:user_toolchain`. If this conf value is defined, the `generic_system` block will include the provided file or files, but no further define any CMake variable for cross-building.

If `user_toolchain` is not defined and Conan detects it is cross-building, because the build and host profiles contain different OS or architecture, it will try to define the following variables:

- CMAKE_SYSTEM_NAME: tools.cmake.cmaketoolchain:system_name configuration if defined, otherwise, it will try to autodetect it. This block will consider cross-building if Android systems (that is managed by other blocks), and not 64bits to 32bits builds in x86_64, sparc and ppc systems.
- CMAKE_SYSTEM_VERSION: tools.cmake.cmaketoolchain:system_version conf if defined, otherwise os.version subsetting (host) when defined. On Apple systems, this os.version is converted to the corresponding Darwin version.
- CMAKE_SYSTEM_PROCESSOR: tools.cmake.cmaketoolchain:system_processor conf if defined, otherwise arch setting (host) if defined

Support for Universal Binaries in macOS

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Starting in Conan 2.2.0, there's preliminary support for building universal binaries on macOS using CMakeToolchain. To specify multiple architectures for a universal binary in Conan, use the | separator when defining the architecture in the settings. This approach enables passing a list of architectures. For example, running:

```
conan create . --name=mylibrary --version=1.0 -s="arch=armv8|x86_64"
```

will create a universal binary for *mylibrary* containing both armv8 and x86_64 architectures, by setting CMAKE_OSX_ARCHITECTURES with a value of arm64;x86_64 in the *conan_toolchain.cmake* file.

Warning: It is important to note that this method is not applicable to build systems other than CMake or Autotools via CMakeToolchain and AutotoolsToolchain.

Be aware that this feature is primarily beneficial for building final universal binaries for release purposes. The default Conan behavior of managing one binary per architecture generally provides a more reliable and trouble-free experience. Users should be cautious and not overly rely on this feature for broader use cases.

Reference

class CMakeToolchain(*conanfile*, *generator=None*)

generate()

This method will save the generated files to the conanfile.generators_folder

conf

CMakeToolchain is affected by these [conf] variables:

- **tools.cmake.cmaketoolchain:toolchain_file** user toolchain file to replace the conan_toolchain.cmake one.
- **tools.cmake.cmaketoolchain:user_toolchain** list of user toolchains to be included from the conan_toolchain.cmake file.
- **tools.android:ndk_path** value for ANDROID_NDK_PATH.

- **tools.android:cmake_legacy_toolchain**: boolean value for `ANDROID_USE_LEGACY_TOOLCHAIN_FILE`. It will only be defined in `conan_toolchain.cmake` if given a value. This is taken into account by the CMake toolchain inside the Android NDK specified in the `tools.android:ndk_path` config, for versions `r23c` and above. It may be useful to set this to `False` if compiler flags are defined via `tools.build:cflags` or `tools.build:cxxflags` to prevent Android's legacy CMake toolchain from overriding the values. If setting this to `False`, please ensure you are using CMake 3.21 or above.
- **tools.cmake.cmaketoolchain:system_name** is not necessary in most cases and is only used to force-define `CMAKE_SYSTEM_NAME`.
- **tools.cmake.cmaketoolchain:system_version** is not necessary in most cases and is only used to force-define `CMAKE_SYSTEM_VERSION`.
- **tools.cmake.cmaketoolchain:system_processor** is not necessary in most cases and is only used to force-define `CMAKE_SYSTEM_PROCESSOR`.
- **tools.cmake.cmaketoolchain:enabled_blocks** define which blocks are enabled and discard the others.
- **tools.cmake.cmaketoolchain:extra_variables**: dict-like python object which specifies the CMake variable name and value. The value can be a plain string, a number or a dict-like python object which must specify the `value` (string/number) , `cache` (boolean), `type` (CMake cache type) and optionally, `docstring` (string: defaulted to variable name) and `force` (boolean) keys. It can override CMakeToolchain defined variables, for which users are at their own risk. E.g.

```
[conf]
tools.cmake.cmaketoolchain:extra_variables={'MY_CMAKE_VAR': 'MyValue'}
```

Resulting in:

```
set(MY_CMAKE_VAR "MyValue")
```

Which will be injected later so it can override default Conan variables.

Another advanced usage:

```
tools.cmake.cmaketoolchain:extra_variables={'MyIntegerVariable': 42, 'CMAKE_GENERATOR_
↪INSTANCE': '${ENV}/buildTools/'}
tools.cmake.cmaketoolchain:extra_variables*={'CACHED_VAR': {'value': '/var/run', 'cache
↪': True, 'type': 'PATH', 'docstring': 'test cache var', 'force': True}}
```

Resulting in:

```
set(MyIntegerVariable 42)
set(CMAKE_GENERATOR_INSTANCE "${ENV}/buildTools/")
set(CACHED_VAR "/var/run" CACHE BOOL "test cache var" FORCE)
```

This block injects `$` which will be expanded later. It also defines a cache variable of type `PATH`.

Tip: Use the *configuration data operator* `*=` to **update** (instead of redefining) conf variables already set in profiles or the global configuration.

- **tools.cmake.cmaketoolchain:toolset_arch**: Will add the `,host=xxx` specifier in the `CMAKE_GENERATOR_TOOLSET` variable of `conan_toolchain.cmake` file.
- **tools.cmake.cmaketoolchain:toolset_cuda**: (Experimental) Will add the `,cuda=xxx` specifier in the `CMAKE_GENERATOR_TOOLSET` variable of `conan_toolchain.cmake` file.

- **tools.cmake.cmake_layout:build_folder_vars**: Settings, Options, `self.name` and `self.version` and constants `const.uservalue` that will produce a different build folder and different CMake presets names.
- **tools.cmake.cmaketoolchain:presets_environment**: Set to `'disabled'` to prevent the addition of the environment section to the generated CMake presets.
- **tools.cmake.cmaketoolchain:user_presets**: (Experimental) Allows setting a custom name or subfolder for the *CMakeUserPresets.json* file. An empty string disables file generation entirely. A relative path with a directory component enables the CMake >= 4.4 `--preset-file` mode.
- **tools.build:cxxflags** list of extra C++ flags that will be appended to `CMAKE_CXX_FLAGS_INIT`.
- **tools.build:cflags** list of extra of pure C flags that will be appended to `CMAKE_C_FLAGS_INIT`.
- **tools.build:sharedlinkflags** list of extra linker flags that will be appended to `CMAKE_SHARED_LINKER_FLAGS_INIT`.
- **tools.build:exelinkflags** list of extra linker flags that will be appended to `CMAKE_EXE_LINKER_FLAGS_INIT`.
- **tools.build:rcflags** list of extra RC flags that will be appended to `CMAKE_RC_FLAGS_INIT`.
- **tools.build:defines** list of preprocessor definitions that will be used by `add_definitions()`.
- **tools.build:tools.build:add_rpath_link**: add `-Wl,-rpath-link,` linker flag. Set this to `True` to pass this flag pointing to all library directories of all host dependencies. Notice that it should not be needed when using the newer *CMakeConfigDeps* generator.
- **tools.apple:sdk_path** value for `CMAKE_OSX_SYSROOT`. In the general case it's not needed and will be passed to CMake by the settings values.
- **tools.apple:enable_bitcode** boolean value to enable/disable Bitcode Apple Clang flags, e.g., `CMAKE_XCODE_ATTRIBUTE_ENABLE_BITCODE`.
- **tools.apple:enable_arc** boolean value to enable/disable ARC Apple Clang flags, e.g., `CMAKE_XCODE_ATTRIBUTE_CLANG_ENABLE_OBJC_ARC`.
- **tools.apple:enable_visibility** boolean value to enable/disable Visibility Apple Clang flags, e.g., `CMAKE_XCODE_ATTRIBUTE_GCC_SYMBOLS_PRIVATE_EXTERN`.
- **tools.build:sysroot** defines the value of `CMAKE_SYSROOT`.
- **tools.microsoft:winsdk_version** Defines the `CMAKE_SYSTEM_VERSION` or the `CMAKE_GENERATOR_PLATFORM` according to CMake policy `CMP0149`.
- **tools.microsoft:msvc_update** allows defining the latest digits of `CMAKE_GENERATOR_TOOLSET` without being part of the package-id. For example defining it to `tools.microsoft:msvc_update=0.35717` for `compiler.version=195` will generate `CMAKE_GENERATOR_TOOLSET = "v145,version=14.50.35717"`.
- **tools.build:compiler_executables** dict-like Python object which specifies the compiler as key and the compiler executable path as value. Those keys will be mapped as follows:
  - `c`: will set `CMAKE_C_COMPILER` in *conan_toolchain.cmake*.
  - `cpp`: will set `CMAKE_CXX_COMPILER` in *conan_toolchain.cmake*.
  - `rc`: will set `CMAKE_RC_COMPILER` in *conan_toolchain.cmake*.
  - `objc`: will set `CMAKE_OBJC_COMPILER` in *conan_toolchain.cmake*.
  - `objcpp`: will set `CMAKE_OBJCXX_COMPILER` in *conan_toolchain.cmake*.
  - `cuda`: will set `CMAKE_CUDA_COMPILER` in *conan_toolchain.cmake*.
  - `fortran`: will set `CMAKE_Fortran_COMPILER` in *conan_toolchain.cmake*.
  - `asm`: will set `CMAKE_ASM_COMPILER` in *conan_toolchain.cmake*.

- `hip`: will set `CMAKE_HIP_COMPILER` in *conan_toolchain.cmake*.
- `ispc`: will set `CMAKE_ISPC_COMPILER` in *conan_toolchain.cmake*.

Since Conan 2.32, keys not listed above (including uppercase variants such as `RC`) are reported with a `risk` warning.

CMake

The `CMake` build helper is a wrapper around the command line invocation of cmake. It will abstract the calls like `cmake --build . --config Release` into Python method calls. It will also add the argument `-DCMAKE_TOOLCHAIN_FILE=conan_toolchain.cmake` (from the generator `CMakeToolchain`) to the `configure()` call, as well as other possible arguments like `-DCMAKE_BUILD_TYPE=<config>`. The arguments that will be used are obtained from a generated `CMakePresets.json` file.

The helper is intended to be used in the `build()` method, to call CMake commands automatically when a package is being built directly by Conan (create, install)

```
from conan import ConanFile
from conan.tools.cmake import CMake, CMakeToolchain, CMakeDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    def generate(self):
        tc = CMakeToolchain(self)
        tc.generate()
        deps = CMakeDeps(self)
        deps.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()
```

Experimental feature. `CMake.configure()`, `CMake.build()` and `CMake.install()` methods have the **subfolder** parameter in case you have more than one `CMakeLists.txt` in different folders. This feature allows you to call the **configure build** and **install** method of each **CMakeLists.txt** separately and without mixing the generated files and artifacts, also creating these folders in the **build folder** and **package folder**.

In the following example, we can see what it would look like if we had two different **CMakeLists.txt** (**cmake_1/CMakeLists.txt** and **cmake_2/CMakeLists.txt**) in the **folder1** and **folder2** folders.

```
def build(self):
    cmake = CMake(self)

    # Configure and build source_folder/cmake_1/CMakeLists.txt in folder1
    cmake.configure(build_script_folder="cmake_1", subfolder="folder1")
    cmake.build(subfolder="folder1")

    # Configure and build source_folder/cmake_2/CMakeLists.txt in folder2
```

(continues on next page)

(continued from previous page)

```
    cmake.configure(build_script_folder="cmake_2" subfolder="folder2")
    cmake.build(subfolder="folder2")
```

Reference

class CMake(*conanfile*)

CMake helper to use together with the CMakeToolchain feature

Parameters
conanfile – The current recipe object. Always use `self`.

configure(*variables=None*, *build_script_folder=None*, *cli_args=None*, *stdout=None*, *stderr=None*, *subfolder=None*)

Reads the `CMakePresets.json` file generated by the *CMakeToolchain* to get:

- The generator, to append `-G="xxx"`.
- Toolchain path to append `-DCMAKE_TOOLCHAIN_FILE=/path/conan_toolchain.cmake`
- The declared `cache variables` to append `-Dxxx`.

and call `cmake`.

Parameters

- **variables** – Should be a dictionary of CMake variables and values, that will be mapped to command line `-DVAR=VALUE` arguments. Recall that in the general case information to CMake should be passed in `CMakeToolchain` to be provided in the `conan_toolchain.cmake` file. This `variables` argument is intended for exceptional cases that wouldn't work in the toolchain approach.
- **build_script_folder** – Path to the CMakeLists.txt in case it is not in the declared `self.folders.source` at the `layout()` method.
- **cli_args** – List of arguments `[arg1, arg2, ...]` that will be passed as extra CLI arguments to pass to cmake invocation
- **subfolder** – (Experimental): subfolder to be created inside the `build_folder` and the `package_folder`. If not provided, files will be placed in the `build_folder` and `package_folder` root.
- **stdout** – Use it to redirect stdout to this stream
- **stderr** – Use it to redirect stderr to this stream

build(*build_type=None*, *target=None*, *cli_args=None*, *build_tool_args=None*, *stdout=None*, *stderr=None*, *subfolder=None*)

Parameters

- **build_type** – Use it only to override the value defined in the `settings.build_type` for a multi-configuration generator (e.g. Visual Studio, XCode). This value will be ignored for single-configuration generators, they will use the one defined in the toolchain file during the install step.
- **target** – The name of a single build target as a string, or names of multiple build targets in a list of strings to be passed to the `--target` argument.

- **cli_args** – A list of arguments `[arg1, arg2, ...]` that will be passed to the `cmake --build ... arg1 arg2` command directly.
- **build_tool_args** – A list of arguments `[barg1, barg2, ...]` for the underlying build system that will be passed to the command line after the `--` indicator: `cmake --build . .. -- barg1 barg2`
- **subfolder** – (Experimental): subfolder to be created inside the `build_folder` and the `package_folder`. If not provided, files will be placed in the `build_folder` and `package_folder` root.
- **stdout** – Use it to redirect stdout to this stream
- **stderr** – Use it to redirect stderr to this stream

install(*build_type=None*, *component=None*, *cli_args=None*, *stdout=None*, *stderr=None*, *subfolder=None*)

Equivalent to running `cmake --install`

Parameters

- **component** – The specific component to install, if any
- **build_type** – Use it only to override the value defined in the settings.build_type. It can fail if the build is single configuration (e.g. Unix Makefiles), as in that case the build type must be specified at configure time, not build type.
- **cli_args** – A list of arguments `[arg1, arg2, ...]` for the underlying build system that will be passed to the command line: `cmake --install ... arg1 arg2`
- **subfolder** – (Experimental): subfolder to be created inside the `build_folder` and the `package_folder`. If not provided, files will be placed in the `build_folder` and `package_folder` root.
- **stdout** – Use it to redirect stdout to this stream
- **stderr** – Use it to redirect stderr to this stream

test(*build_type=None*, *target=None*, *cli_args=None*, *build_tool_args=None*, *env=''*, *stdout=None*, *stderr=None*)

Equivalent to running cmake –build . –target=RUN_TESTS.

Parameters

- **build_type** – Use it only to override the value defined in the `settings.build_type`. It can fail if the build is single configuration (e.g. Unix Makefiles), as in that case the build type must be specified at configure time, not build time.
- **target** – Name of the build target to run, by default `RUN_TESTS` or `test`
- **cli_args** – Same as above `build()`, a list of arguments `[arg1, arg2, ...]` to be passed as extra arguments for the underlying build system
- **build_tool_args** – Same as above `build()`
- **stdout** – Use it to redirect stdout to this stream
- **stderr** – Use it to redirect stderr to this stream

ctest(*cli_args=None*, *env=''*, *stdout=None*, *stderr=None*)

Equivalent to running ctest ...

Parameters

- **cli_args** – List of arguments `[arg1, arg2, ...]` to be passed as extra ctest command line arguments

- **env** – the environment files to activate, by default conanbuild + conanrun
- **stdout** – Use it to redirect stdout to this stream
- **stderr** – Use it to redirect stderr to this stream

conf

The `CMake()` build helper is affected by these `[conf]` variables:

- `tools.build:verbosity` will accept one of `quiet` or `verbose` to be passed to the `CMake.build()` command, when a Visual Studio generator (MSBuild build system) is being used for CMake. It is passed as an argument to the underlying build system via the call `cmake --build . --config Release -- /verbosity:Diagnostic`
- `tools.compilation:verbosity` will accept one of `quiet` or `verbose` to be passed to CMake, which sets `-DCMAKE_VERBOSE_MAKEFILE` if `verbose`
- `tools.build:jobs` argument for the `--jobs` parameter when running Ninja generator.
- `tools.microsoft.msbuild:max_cpu_count` argument for the `/m` (`/maxCpuCount`) when running `MSBuild`. If `max_cpu_count=0`, then it will use `/m` without arguments, which means use all available cpus.
- `tools.cmake:cmake_program` specify the location of the CMake executable, instead of using the one found in the `PATH`.
- `tools.cmake:ctest_args` (Since Conan 2.23.0) will inject a list of arguments to the `CMake.ctest()` runner. For example, `tools.cmake:ctest_args=["--debug", "--output-junit myfile"]` will add these arguments to the ctest command.
- `tools.cmake:install_strip` (**deprecated** use `tools.build:install_strip`) will pass `--strip` to the `cmake --install` call if set to `True`.
- `tools.build:install_strip` (Since Conan 2.18.0; list values since Conan 2.28.0): when enabled for CMake, passes `--strip` to `cmake --install`. Use `True` so every integration that reads this configuration may strip; use a list such as `["cmake"]` so only the CMake helper strips. `False` or an unset value disables stripping in this helper.
- `tools.cmake:configure_args` (Since Conan 2.27) allows injecting extra arguments (list) to the `cmake ...` configure command line executed by `cmake.configure()`. This allows to inject arguments such as `--fresh` to force a fresh build when doing local development (`conan build -c tools.cmake:configure_args="['--fresh']"` command), or to inject arbitrary CMake variables, specifying `-DMYCMAKE_VAR=VALUE` as it would be specified in the command line.

cmake_layout

The `cmake_layout()` sets the *folders* and *cpp* attributes to follow the structure of a typical CMake project.

```
from conan.tools.cmake import cmake_layout

def layout(self):
    cmake_layout(self)
```

Note: To try it you can use the `conan new cmake_lib -d name=hello -d version=1.0` template.

The assigned values depend on the CMake generator that will be used. It can be defined with the `tools.cmake.cmaketoolchain:generator` [conf] entry or passing it in the recipe to the `cmake_layout(self,`

cmake_generator) function. The assigned values are different if it is a multi-config generator (like Visual Studio or Xcode), or a single-config generator (like Unix Makefiles).

These are the values assigned by the cmake_layout:

- conanfile.folders.source: *src_folder* argument or . if not specified.
- **conanfile.folders.build:**
  - build: if the cmake generator is multi-configuration.
  - build/Debug or build/Release: if the cmake generator is single-configuration, depending on the build_type.
  - The "build" string, can be defined to other value by the build_folder argument.
- conanfile.folders.generators: build/generators
- conanfile.cpp.source.includedirs: ["include"]
- **conanfile.cpp.build.libdirs and conanfile.cpp.build.bindirs:**
  - ["Release"] or ["Debug"] for a multi-configuration cmake generator.
  - . for a single-configuration cmake generator.

Reference

cmake_layout(*conanfile*, *generator=None*, *src_folder='.'*, *build_folder='build'*)

Parameters

- **conanfile** – The current recipe object. Always use self.
- **generator** – Allow defining the CMake generator. In most cases it doesn't need to be passed, as it will get the value from the configuration tools.cmake.cmaketoolchain:generator, or it will automatically deduce the generator from the settings
- **src_folder** – Value for conanfile.folders.source, change it if your source code (and CMakeLists.txt) is in a subfolder.
- **build_folder** – Specify the name of the "base" build folder. The default is "build", but if that folder name is used by the project, a different one can be defined

conf

cmake_layout is affected by these [conf] variables:

- **tools.cmake.cmake_layout:build_folder_vars** list of settings, options, self.name and self.version and constants const.xxx to customize the conanfile.folders.build folder. See section *Multi-setting/option cmake_layout* below.
- **tools.cmake.cmake_layout:build_folder** (*new since Conan 2.2.0*)(*experimental*) uses its value as the base folder of the conanfile.folders.build for local builds.
- **tools.cmake.cmake_layout:test_folder** (*new since Conan 2.2.0*)(*experimental*) uses its value as the base folder of the conanfile.folders.build for test_package builds. If that value is $TMP, Conan will create and use a temporal folder.

Multi-setting/option cmake_layout

The `folders.build` and `conanfile.folders.generators` can be customized to take into account the `settings` and `options` and not only the `build_type`. Use the `tools.cmake.cmake_layout:build_folder_vars` conf to declare a list of settings, options and/or `self.name` and `self.version` and user constants:

```
conan install . -c tools.cmake.cmake_layout:build_folder_vars="['settings.compiler',
↪'options.shared']"
```

For the previous example, the values assigned by the `cmake_layout` (installing the Release/static default configuration) would be:

- **`conanfile.folders.build`:**
  - `build/apple-clang-shared_false`: if the cmake generator is multi-configuration.
  - `build/apple-clang-shared_false/Debug`: if the cmake generator is single-configuration.
- `conanfile.folders.generators`: `build/generators`

If we repeat the previous install with a different configuration:

```
conan install . -o shared=True -c tools.cmake.cmake_layout:build_folder_vars="['settings.
↪compiler', 'options.shared']"
```

The values assigned by the `cmake_layout` (installing the Release/shared configuration) would be:

- **`conanfile.folders.build`:**
  - `build/apple-clang-shared_true`: if the cmake generator is multi-configuration.
  - `build/apple-clang-shared_true/Debug`: if the cmake generator is single-configuration.
- `conanfile.folders.generators`: `build-apple-clang-shared_true/generators`

So we can keep separated folders for any number of different configurations that we want to install. Recipe attributes like name and version and user constants can also be used:

```
$ conan install . -c tools.cmake.cmake_layout:build_folder_vars="['const.myvalue, 'self.
↪name']"
```

And it will create folders like `build/myvalue-pkgname`.

The `CMakePresets.json` file generated at the *CMakeToolchain* generator, will also take this `tools.cmake.cmake_layout:build_folder_vars` config into account to generate different names for the presets, being very handy to install N configurations and building our project for any of them by selecting the chosen preset.

By default, `settings.build_type` is always taken into account to compute the `conanfile.folders.build` folder for single-configuration generators, even if it is not explicitly listed in `build_folder_vars`. It is possible to opt out of this default behavior with the `!settings.build_type` exclusion marker:

```
conan install . -c tools.cmake.cmake_layout:build_folder_vars="['!settings.build_type']"
```

With this configuration, the `build_type` won't be added to the `conanfile.folders.build` folder nor to the generated preset name, even for single-configuration generators, resulting in a plain `build` folder and a `conan-default` preset regardless of the `build_type` value. It can be combined with other settings and options:

```
conan install . -c tools.cmake.cmake_layout:build_folder_vars="['settings.compiler.cppstd
↪', '!settings.build_type']"
```

producing a `build/17` folder and a `conan-17` preset name, still excluding the `build_type` from both. This is useful to avoid having different `Debug/Release` folders and presets when that separation is not desired. Currently, `!settings.build_type` is the only supported exclusion marker.

9.10.5 conan.tools.CppInfo

The `CppInfo` class represents the basic C++ usage information of a given package, like the `includedirs`, `libdirs`, library names, etc. It is the information that the consumers of the package need in order to be able to find the headers and link correctly with the libraries.

The `self.cpp_info` object in the `package_info()` is a `CppInfo` object, so in most cases it will not be necessary to explicitly instantiate it, and using it as explained in *the package_info()* section would be enough.

This section describes the other, advanced uses cases of the `CppInfo`.

Aggregating information in custom generators

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Some generators, like the built-in `NMakeDeps`, contains the equivalent to this code, that collapses all information from all dependencies into one single `CppInfo` object that aggregates all the information

```
from conan.tools import CppInfo

...

def generate(self):
    aggregated_cpp_info = CppInfo(self)
    deps = self.dependencies.host.topological_sort
    deps = [dep for dep in reversed(deps.values())]
    for dep in deps:
        # We don't want independent components management, so we collapse
        # the "dep" components into one CppInfo called "dep_cppinfo"
        dep_cppinfo = dep.cpp_info.aggregated_components()
        # Then we merge and aggregate this dependency "dep" into the final result
        aggregated_cpp_info.merge(dep_cppinfo)

    aggregated_cpp_info.includedirs  # All include dirs from all deps, all components
    aggregated_cpp_info.libs  # All library names from all deps, all components
    aggregated_cpp_info.system_libs # All system-libs from all deps
    ....
    # Creates a file with this information that the build system will use
```

This aggregation could be useful in cases where the build system cannot easily use independent dependencies or components. For example `NMake` or `Autotools` mechanism to provide dependencies information would be via `LIBS`, `CXXFLAGS` and similar variables. These variables are global, so passing all the information from all dependencies is the only possibility.

CppInfo interface

The public documented interface (besides the defined one in *the package_info()*) is:

- `CppInfo(conanfile)`: Constructor. Receives a `conanfile` as argument, typically `self`
- `aggregated_components(components=None)`: return a new `CppInfo` object resulting from the aggregation of all the components. If the optional `components` argument is passed (a list of component names), only those components are aggregated (new in Conan 2.34).
- `get_sorted_components()`: Get the ordered components of a package, prioritizing those with fewer dependencies within the same package. Returns an `OrderedDict` of sorted components in the format `{component_name: component}`.
- `merge(other_cppinfo: CppInfo)`: modifies the current `CppInfo` object, updating it with the information of the parameter `other_cppinfo`, allowing to aggregate information from multiple dependencies.
- `save(path)` serializes the `CppInfo` object into json and saves it to the `path`.
- `CppInfo.load(path)` static method returns a `CppInfo` object result of loading the `path` file and deserializing its json contents (result of a `cpp_info.save(path)`)

Note: Best practices

Regular recipes should not use these special `CppInfo` methods in most cases. Most of them are intended for custom generators only, not recipes. The `save()/load()` operations shouldn't be needed in recipes in most cases either, only for some exceptional circumstances like the *PkgConfig fill_cpp_info example*.

9.10.6 conan.tools.env

Environment

`Environment` is a generic class that helps to define modifications to the environment variables. This class is used by other tools like the *conan.tools.gnu Autotools* helpers and the *VirtualBuildEnv* and *VirtualRunEnv* generator. It is important to highlight that this is a generic class, to be able to use it, a specialization for the current context (shell script, bat file, path separators, etc), a `EnvVars` object needs to be obtained from it.

Variable declaration

```
from conan.tools.env import Environment

def generate(self):
    env = Environment()
    env.define("MYVAR1", "MyValue1")  # Overwrite previously existing MYVAR1 with new␣
↪value
    env.append("MYVAR2", "MyValue2")  # Append to existing MYVAR2 the new value
    env.prepend("MYVAR3", "MyValue3") # Prepend to existing MYVAR3 the new value
    env.remove("MYVAR3", "MyValue3")  # Remove the MyValue3 from MYVAR3
    env.unset("MYVAR4")               # Remove MYVAR4 definition from environment

    # And the equivalent with paths
    env.define_path("MYPATH1", "path/one")  # Overwrite previously existing MYPATH1 with␣
↪new value
```

(continues on next page)

(continued from previous page)

```
    env.append_path("MYPATH2", "path/two")  # Append to existing MYPATH2 the new value
    env.prepend_path("MYPATH3", "path/three") # Prepend to existing MYPATH3 the new value
```

The "normal" variables (the ones declared with `define`, `append` and `prepend`) will be appended with a space, by default, but the `separator` argument can be provided to define a custom one.

The "path" variables (the ones declared with `define_path`, `append_path` and `prepend_path`) will be appended with the default system path separator, either `:` or `;`, but it also allows defining which one.

Generation of environment files

The generation of environment script files (like `envfile.bat|.sh|.ps1|.env`) can be done indirectly by the `EnvVars` class, which can be obtained with:

```
from conan.tools.env import Environment

env1 = Environment()
...
envvars = env1.vars(self)  # An EnvVars object
# Generate a .bat|.sh|.ps1|.env file depending on current
# settings and Conan configuration
envars.save_script("mybuild")
# or decide to be explicit and generate some of the files:
envvars.save_dotenv("myenv.env")
```

These files can be used also automatically by subsequent `self.run()` calls. For more information see the `EnvVars` class documentation.

Composition

Environments can be composed:

```
from conan.tools.env import Environment

env1 = Environment()
env1.define(...)
env2 = Environment()
env2.append(...)

env1.compose_env(env2) # env1 has priority, and its modifications will prevail
```

Obtaining environment variables

You can obtain an `EnvVars` object with the `vars()` method like this:

```
from conan.tools.env import Environment

def generate(self):
    env = Environment()
    env.define("MYVAR1", "MyValue1")
    envvars = env.vars(self, scope="build")
    # use the envvars object
```

The default `scope` is equal `"build"`, which means that if this `envvars` generate a script to activate the variables, such script will be automatically added to the `conanbuild.sh|bat` one, for users and recipes convenience. Conan generators use `build` and `run` scope, but it might be possible to manage other scopes too.

Environment definition

There are some other places where `Environment` can be defined and used:

- In recipes `package_info()` method, in new `self.buildenv_info` and `self.runenv_info`, this environment will be propagated via `VirtualBuildEnv` and `VirtualRunEnv` respectively to packages depending on this recipe.
- In generators like `AutootoolsDeps`, `AutotoolsToolchain`, that need to define environment for the current recipe.
- In profiles `[buildenv]` section.
- In profiles `[runenv]` section.

The definition in `package_info()` is as follow, taking into account that both `self.buildenv_info` and `self.runenv_info` are objects of `Environment()` class.

```
from conan import ConanFile

class App(ConanFile):
    name = "mypkg"
    version = "1.0"
    settings = "os", "arch", "compiler", "build_type"

    def package_info(self):
        # This is information needed by consumers to build using this package
        self.buildenv_info.append("MYVAR", "MyValue")
        self.buildenv_info.prepend_path("MYPATH", "some/path/folder")

        # This is information needed by consumers to run apps that depends on this␣
↪package
        # at runtime
        self.runenv_info.define("MYPKG_DATA_DIR", os.path.join(self.package_folder,
                                                               "datadir"))
```

Reference

class Environment

Generic class that helps to define modifications to the environment variables.

dumps()

Returns
A string with a profile-like original definition, not the full environment values

define(*name*, *value*, *separator=' '*)

Define *name* environment variable with value *value*

Parameters

- **name** – Name of the variable
- **value** – Value that the environment variable will take
- **separator** – The character to separate appended or prepended values

unset(*name*)

clears the variable, equivalent to a unset or set XXX=

Parameters
name – Name of the variable to unset

append(*name*, *value*, *separator=None*)

Append the *value* to an environment variable *name*

Parameters

- **name** – Name of the variable to append a new value
- **value** – New value
- **separator** – The character to separate the appended value with the previous value. By default it will use a blank space.

append_path(*name*, *value*)

Similar to "append" method but indicating that the variable is a filesystem path. It will automatically handle the path separators depending on the operating system.

Parameters

- **name** – Name of the variable to append a new value
- **value** – New value

prepend(*name*, *value*, *separator=None*)

Prepend the *value* to an environment variable *name*

Parameters

- **name** – Name of the variable to prepend a new value
- **value** – New value
- **separator** – The character to separate the prepended value with the previous value

prepend_path(*name*, *value*)

Similar to "prepend" method but indicating that the variable is a filesystem path. It will automatically handle the path separators depending on the operating system.

Parameters

- **name** – Name of the variable to prepend a new value
- **value** – New value

remove(*name*, *value*)

Removes the *value* from the variable *name*.

Parameters

- **name** – Name of the variable
- **value** – Value to be removed.

compose_env(*other*)

Compose an Environment object with another one. `self` has precedence, the "other" will add/append if possible and not conflicting, but `self` mandates what to do. If `self` has `define()`, without placeholder, that will remain.

Parameters
other (class:*Environment*) – the "other" Environment

vars(*conanfile*, *scope='build'*)

Parameters

- **conanfile** – Instance of a conanfile, usually `self` in a recipe
- **scope** – Determine the scope of the declared variables.

Returns
An EnvVars object from the current Environment object

deploy_base_folder(*package_folder*, *deploy_folder*)

Make the paths relative to the deploy_folder

EnvVars

`EnvVars` is a class that represents an instance of environment variables for a given system. It is obtained from the generic *Environment* class.

This class is used by other tools like the *conan.tools.gnu* autotools helpers and the *VirtualBuildEnv* and *VirtualRunEnv* generator.

Creating environment files

`EnvVars` object can generate environment files (shell, bat or powershell scripts):

```
def generate(self):
    env1 = Environment()
    env1.define("foo", "var")
    envvars = env1.vars(self)
    envvars.save_script("my_env_file")
```

Although it potentially could be used in other methods, this functionality is intended to work in the `generate()` method.

It will generate automatically a `my_env_file.bat` for Windows systems or `my_env_file.sh` otherwise.

PowerShell `.ps1` scripts are also generated next to the `.bat` (Windows) or `.sh` (Linux, Mac and other platforms) ones, on all platforms (Windows, Linux and Mac), for the consumer (a local `conanfile`, like in `conan install .`) and for `test_package`, so developers working natively in PowerShell can activate the environment. For other packages, the `.ps1` files are only generated if the configuration `tools.env.virtualenv:powershell` is defined. This configuration should be set with the value corresponding to the desired PowerShell executable: `powershell.exe` for versions up to 5.1, and `pwsh` for PowerShell versions starting from 7. Setting `tools.env.virtualenv:powershell` to `True` or `False` is no longer supported (it was deprecated in Conan 2.11.0).

Conan only uses the `.ps1` files when running subprocesses, like `self.run()`, if `tools.env.virtualenv:powershell` is defined. Otherwise, it always uses the `.bat` files.

You can also include additional arguments in the `tools.env.virtualenv:powershell` configuration. For example, you can set the value to `powershell.exe -NoProfile` or `pwsh -NoProfile` by including the arguments as part of the configuration value. These arguments will be considered when executing the generated `.ps1` launchers.

Also, by default, Conan will automatically append that launcher file path to a list that will be used to create a `conanbuild.bat|sh|ps1` file aggregating all the launchers in order. The `conanbuild.sh|bat|ps1` launcher will be created after the execution of the `generate()` method.

The `scope` argument (`"build"` by default) can be used to define different scope of environment files, to aggregate them separately. For example, using a `scope="run"`, like the `VirtualRunEnv` generator does, will aggregate and create a `conanrun.bat|sh|ps1` script:

```
def generate(self):
    env1 = Environment()
    env1.define("foo", "var")
    envvars = env1.vars(self, scope="run")
    # Will append "my_env_file" to "conanrun.bat|sh|ps1"
    envvars.save_script("my_env_file")
```

From Conan 2.21, if the **experimental** `tools.env:dotenv` configuration is active, then `.env` files will also be generated. These files are not intended to be activated as scripts, but loaded by tools such as IDEs.

You can also use `scope=None` argument to avoid appending the script to the aggregated `conanbuild.bat|sh|ps1`:

```
env1 = Environment()
env1.define("foo", "var")
# Will not append "my_env_file" to "conanbuild.bat|sh|ps1"
envvars = env1.vars(self, scope=None)
envvars.save_script("my_env_file")
```

Running with environment files

The `conanbuild.bat|sh|ps1` launcher will be executed by default before calling every `self.run()` command. This would be typically done in the `build()` method.

You can change the default launcher with the `env` argument of `self.run()`:

```
...
def build(self):
    # This will automatically wrap the "foo" command with the correct environment:
    # source my_env_file.sh && foo
    # my_env_file.bat && foo
    # powershell my_env_file.ps1 ; cmd c/ foo
    self.run("foo", env=["my_env_file"])
```

Applying the environment variables

As an alternative to running a command, environments can be applied in the python environment:

```
from conan.tools.env import Environment

env1 = Environment()
env1.define("foo", "var")
envvars = env1.vars(self)
with envvars.apply():
   # Here os.getenv("foo") == "var"
   ...
```

Iterating the variables

You can iterate the environment variables of an `EnvVars` object like this:

```
env1 = Environment()
env1.append("foo", "var")
env1.append("foo", "var2")
envvars = env1.vars(self)
for name, value in envvars.items():
    assert name == "foo":
    assert value == "var var2"
```

The current value of the environment variable in the system is replaced in the returned value. This happens when variables are appended or prepended. If a placeholder is desired instead of the actual value, it is possible to use the `variable_reference` argument with a jinja template syntax, so a string with that resolved template will be returned instead:

```
env1 = Environment()
env1.append("foo", "var")
envvars = env1.vars(self)
for name, value in envvars.items(variable_reference="$penv{{{name}}}""):
    assert name == "foo":
    assert value == "$penv{{foo}} var"
```

Warning: In Windows, there is a limit to the size of environment variables, a total of 32K for the whole environment, but specifically the PATH variable has a limit of 2048 characters. That means that the above utils could hit that limit, for example for large dependency graphs where all packages contribute to the PATH env-var.

This can be mitigated by:

- Putting the Conan cache closer to C:/ for shorter paths
- Better definition of what dependencies can contribute to the PATH env-var
- Other mechanisms for things like running with many shared libraries dependencies with too many .dlls, like `deployers`

Reference

class EnvVars(*conanfile*, *values*, *scope*)

Represents an instance of environment variables for a given system. It is obtained from the generic Environment class.

get(*name*, *default=None*, *variable_reference=None*)

get the value of a env-var

Parameters

- **name** – The name of the environment variable.
- **default** – The returned value if the variable doesn't exist, by default None.
- **variable_reference** – if specified, use a variable reference instead of the pre-existing value of environment variable, where {name} can be used to refer to the name of the variable.

items(*variable_reference=None*)

returns {str: str} (varname: value)

Parameters

variable_reference – if specified, use a variable reference instead of the pre-existing value of environment variable, where {name} can be used to refer to the name of the variable.

apply()

Context manager to apply the declared variables to the current `os.environ` restoring the original environment when the context ends.

save_script(*filename*)

Saves a script file (bat, sh, ps1) with a launcher to set the environment. If the extension is not provided, it generates the .bat/.sh and also the .ps1 launchers for the consumer or if the conf "tools.env.virtualenv:powershell" is defined.

Parameters

filename – Name of the file to generate. If the extension is provided, it will generate the launcher script for that extension, otherwise the format will be deduced checking if we are running inside Windows (checking also the subsystem) or not.

VirtualBuildEnv

VirtualBuildEnv is a generator that produces a *conanbuildenv* .bat, .ps1 or .sh script containing the environment variables of the build time context:

- From the `self.buildenv_info` of the direct `tool_requires` in "build" context.
- From the `self.runenv_info` of the transitive dependencies of those `tool_requires`.

It can be used by name in conanfiles:

Listing 119: conanfile.py

```
class Pkg(ConanFile):
    generators = "VirtualBuildEnv"
```

Listing 120: conanfile.txt

```
[generators]
VirtualBuildEnv
```

And it can also be fully instantiated in the conanfile `generate()` method:

Listing 121: conanfile.py

```
from conan import ConanFile
from conan.tools.env import VirtualBuildEnv

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    requires = "zlib/1.3.1", "bzip2/1.0.8"

    def generate(self):
        ms = VirtualBuildEnv(self)
        ms.generate()
```

Note that instantiating the `VirtualBuildEnv()` generator without later calling the `generate()` method, which is intended only for the `generate()` recipe method, will inhibit the creation of environment files.

So something like:

```
ms = VirtualBuildEnv(self)
my_env_var = ms.vars().get("MY_ENV_VAR")
# does not create conanbuildenv.sh|.bat files
```

will stop creating the `conanbuild.sh|.bat` and `conanbuildenv.sh|.bat` files that are created by default, even when `VirtualBuildEnv` is not instantiated.

In order to keep creating those files, the `auto_generate=True` argument can be passed to the constructor, as:

```
ms = VirtualBuildEnv(self, auto_generate=True)
my_env_var = ms.vars().get("MY_ENV_VAR")
# does create conanbuildenv.sh|.bat files
```

Generated files

This generator (for example the invocation of `conan install --tool-require=cmake/3.20.0@ -g VirtualBuildEnv`) will create the following files:

- conanbuildenv-release-x86_64.(bat|ps1|sh): This file contains the actual definition of environment variables like PATH, LD_LIBRARY_PATH, etc, and any other variable defined in the dependencies `buildenv_info` corresponding to the `build` context, and to the current installed configuration. If a repeated call is done with other settings, a different file will be created. After the execution or sourcing of this file, a new deactivation script will be generated, capturing the current environment, so the environment can be restored when desired. The file will be named also following the current active configuration, like `deactivate_conanbuildenv-release-x86_64.bat`.
- conanbuild.(bat|ps1|sh): Accumulates the calls to one or more other scripts, in case there are multiple tools in the generate process that create files, to give one single convenient file for all. This only calls the latest specific configuration one, that is, if `conan install` is called first for Release build type, and then for Debug, `conanbuild.(bat|ps1|sh)` script will call the Debug one.

- deactivate_conanbuild.(bat|ps1|sh): Accumulates the deactivation calls defined in the above `conanbuild.(bat|ps1|sh)`. This file should only be called after the accumulated activate has been called first.

Note: To create `.ps1` files required for PowerShell, you need to set the `tools.env.virtualenv:powershell` configuration with the value of the PowerShell executable (e.g., `powershell.exe` or `pwsh`). Note that setting it to `True` or `False` is no longer supported. The `.ps1` files are always generated, next to the `.bat` or `.sh` ones in Windows, Linux and Mac, for the consumer (`conan install <path>`) and `test_package`, but Conan only uses them for its own subprocesses, like `self.run()`, if this configuration is defined, otherwise it uses the `.bat` files.

Note: To create `.env` dotenv files, use the **experimental** (new in Conan 2.21) `tools.env:dotenv` configuration. These files are not intended to be activated as scripts, but loaded by tools such as IDEs. The configuration specific files such as `conanbuildenv-Release.env` will be generated, as the environment can be different for Release and Debug configurations. These files at the moment do not use variable interpolation due to some VScode limitations, a warning is printed pointing to https://github.com/microsoft/vscode-cpptools/issues/13781 to track progress. Please open a Github ticket to report any feedback about this feature.

Note: The `VirtualBuildEnv` generator will not propagate the bindirs of tool requires which have the `run` trait set to `False`, for either recipes that have `required_conan_version=">=2.28"` or greater, or globally with the `core:policies=["required_conan_version>=2.28"]` conf.

Disabling VirtualBuildEnv

It is possible to disable the generation of the `VirtualBuildEnv` and `VirtualRunEnv` files with different mechanisms:

- By passing `--envs-generation=false` to the `conan install` command, it will disable the generation of environment files (both `VirtualBuildEnv` and `VirtualRunEnv`) for the current consumer only (dependencies that need to be built from source will generate their own environment files as needed). **This feature is experimental and subject to change**.
- Recipes can instantiate a `VirtualBuildEnv(self)` in their `generate()` method, without calling their `generate()` method. That will inhibit the creation of environment files for that specific recipe:

```
def generate(self):
    VirtualBuildEnv(self)
    # do not call its generate() method
    # discouraged, in most cases, it is desired
    # to call generate()
    VirtualRunEnv(self)
    # Same for VirtualRunEnv
```

- Recipes can directly define their `virtualbuildenv = False` attribute to inhibit the automatic default creation of `VirtualBuildEnv` files for this recipe:

```
class Pkg(ConanFile):
    virtualbuildenv = False
    # Also for VirtualRunEnv
    virtualrunenv = False
```

Warning: In general, disabling the generation of environment files is **discouraged**. Environment files are the mechanism use for things like `[tool_requires]` defined in the profiles to be able to inject those tools dynamically in dependencies, even if those dependencies didn't directly declare such `tool_requires`. Without proper `VirtualBuildEnv` files in the recipe, the `tool_requires` will fail to apply.

Reference

class VirtualBuildEnv(*conanfile*, *auto_generate=False*)

Calculates the environment variables of the build time context and produces a conanbuildenv .bat or .sh script

environment()

Returns an `Environment` object containing the environment variables of the build context.

Returns
an `Environment` object instance containing the obtained variables.

vars(*scope='build'*)

Parameters
scope – Scope to be used.

Returns
An `EnvVars` instance containing the computed environment variables.

generate(*scope='build'*)

Produces the launcher scripts activating the variables for the build context.

Parameters
scope – Scope to be used.

VirtualRunEnv

`VirtualRunEnv` is a generator that produces a launcher *conanrunenv* .bat, .ps1 or .sh script containing environment variables of the run time environment.

The launcher contains the runtime environment information, anything that is necessary in the environment to actually run the compiled executables and applications. The information is obtained from:

- The `self.runenv_info` of the dependencies corresponding to the `host` context.
- Also automatically deduced from the `self.cpp_info` definition of the package to define `PATH`.
- `LD_LIBRARY_PATH`, `DYLD_LIBRARY_PATH`, and `DYLD_FRAMEWORK_PATH` are similarly deduced on non-Windows hosts if the `os` setting is set.

Note: Only dependencies whose `run` requirement trait is `True` contribute their folders to this environment. This is normally inferred from the dependency's `package_type` (e.g. `shared-library` or `application`), but a `static-library` that also ships runtime artifacts, like a shared library, can opt in by declaring `"run": True` inside *package_type_traits* in its recipe.

It can be used by name in conanfiles:

Listing 122: conanfile.py

```
class Pkg(ConanFile):
    generators = "VirtualRunEnv"
```

Listing 123: conanfile.txt

```
[generators]
VirtualRunEnv
```

And it can also be fully instantiated in the conanfile `generate()` method:

Listing 124: conanfile.py

```
from conan import ConanFile
from conan.tools.env import VirtualRunEnv

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    requires = "zlib/1.3.1", "bzip2/1.0.8"

    def generate(self):
        ms = VirtualRunEnv(self)
        ms.generate()
```

Generated files

- conanrunenv-release-x86_64.(bat|ps1|sh): This file contains the actual definition of environment variables like PATH, LD_LIBRARY_PATH, etc, and `runenv_info` of dependencies corresponding to the `host` context, and to the current installed configuration. If a repeated call is done with other settings, a different file will be created.
- conanrun.(bat|ps1|sh): Accumulates the calls to one or more other scripts to give one single convenient file for all. This only calls the latest specific configuration one, that is, if `conan install` is called first for Release build type, and then for Debug, `conanrun.(bat|ps1|sh)` script will call the Debug one.

After the execution of one of those files, a new deactivation script will be generated, capturing the current environment, so the environment can be restored when desired. The file will be named also following the current active configuration, like `deactivate_conanrunenv-release-x86_64.bat`.

Note: To create `.ps1` files required for PowerShell, you need to set the `tools.env.virtualenv:powershell` configuration with the value of the PowerShell executable (e.g., `powershell.exe` or `pwsh`). Note that setting it to `True` or `False` is no longer supported. The `.ps1` files are always generated, next to the `.bat` or `.sh` ones in Windows, Linux and Mac, for the consumer (`conan install <path>`) and `test_package`, but Conan only uses them for its own subprocesses, like `self.run()`, if this configuration is defined, otherwise it uses the `.bat` files.

Note: To create `.env` dotenv files, use the **experimental** (new in Conan 2.21) `tools.env:dotenv` configuration. These files are not intended to be activated as scripts, but loaded by tools such as IDEs. The configuration specific files such as `conanrunenv-Release.env` will be generated, as the environment can be different for Release and Debug configurations. These files at the moment do not use variable interpolation due to some VScode limitations, a warning is printed pointing to https://github.com/microsoft/vscode-cpptools/issues/13781 to track progress. Please open a Github

ticket to report any feedback about this feature.

Note: For disabling the automatic generation of environment files, check *Disabling VirtualBuildEnv*

On Windows, `VirtualRunEnv` can also copy the runtime binaries of dependencies into a single folder and point `PATH` to that folder instead of to every dependency `bindirs` location. This can help avoid `PATH` overflow limits for large dependency graphs. Use the `win_copy_folder` argument with a relative path when instantiating the generator:

```
from conan import ConanFile
from conan.tools.env import VirtualRunEnv

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"

    def generate(self):
        runenv = VirtualRunEnv(self, win_copy_folder="imported-bin")
        runenv.generate()
```

Reference

class VirtualRunEnv(*conanfile*, *auto_generate=False*, *win_copy_folder=None*)

Calculates the environment variables of the runtime context and produces a conanrunenv .bat or .sh script

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **auto_generate** – Automatically generate .bat or .sh files
- **win_copy_folder** – Copy the "bindirs" folders contents from dependencies into this folder, and point the PATH environment variable to it, instead of the dependencies ones. Only for Windows, to avoid PATH and env-var overflow limits.

environment()

Returns an `Environment` object containing the environment variables of the run context.

Returns
an `Environment` object instance containing the obtained variables.

vars(*scope='run'*)

Parameters
scope – Scope to be used.

Returns
An `EnvVars` instance containing the computed environment variables.

generate(*scope='run'*)

Produces the launcher scripts activating the variables for the run context.

Parameters
scope – Scope to be used.

9.10.7 conan.tools.files

conan.tools.files basic operations

conan.tools.files.copy()

copy(*conanfile*, *pattern*, *src*, *dst*, *keep_path=True*, *excludes=None*, *ignore_case=True*, *overwrite_equal=False*)

Copy the files matching the pattern (fnmatch) at the src folder to a dst folder.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **pattern** – (Required) An fnmatch file pattern, or a list/tuple of fnmatch patterns, of the files that should be copied. When a list is given the src folder is walked only once, and every file matching any of the patterns is copied exactly once, even if several patterns match it. Patterns must not start with `..` relative path or an exception will be raised.
- **src** – (Required) Source folder in which those files will be searched. This folder will be stripped from the dst parameter. E.g., lib/Debug/x86.
- **dst** – (Required) Destination local folder. It must be different from src value or an exception will be raised.
- **keep_path** – (Optional, defaulted to `True`) Means if you want to keep the relative path when you copy the files from the src folder to the dst one.
- **excludes** – (Optional, defaulted to `None`) A tuple/list of fnmatch patterns or even a single one to be excluded from the copy.
- **ignore_case** – (Optional, defaulted to `True`) If enabled, it will do a case-insensitive pattern matching. will do a case-insensitive pattern matching when `True`
- **overwrite_equal** – (Optional, default `False`). If the file to be copied already exists in the destination folder, only really copy it if it seems different (different size, different modification time)

Returns

list of copied files

Usage:

```
def package(self):
    copy(self, "*.h", self.source_folder, os.path.join(self.package_folder, "include"))
    copy(self, "*.lib", self.build_folder, os.path.join(self.package_folder, "lib"))
```

Since Conan 2.32, the `pattern` argument also accepts a list or tuple of patterns. When a list is provided, the source folder is walked only once and every file matching any of the patterns is copied, which can be significantly faster than calling `copy()` several times for large trees:

```
def package(self):
    copy(self, ["*.h", "*.hpp"], self.source_folder,
         os.path.join(self.package_folder, "include"))
```

Note: The files that are **symlinks to files** or **symlinks to folders** with be treated like any other file, so they will only be copied if the specified pattern matches with the file.

At the destination folder, the symlinks will be created pointing to the exact same file or folder, absolute or relative, being the responsibility of the user to manipulate the symlink to, for example, transform the symlink into a relative path before copying it so it points to the destination folder.

Check *here* the reference of tools to manage symlinks.

conan.tools.files.load()

load(*conanfile*, *path*, *encoding='utf-8'*)

Utility function to load files in one line. It will manage the open and close of the file, and load binary encodings. Returns the content of the file.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **path** – Path to the file to read
- **encoding** – (Optional, Defaulted to `utf-8`): Specifies the input file text encoding.

Returns

The contents of the file

Usage:

```
from conan.tools.files import load

content = load(self, "myfile.txt")
```

conan.tools.files.save()

save(*conanfile*, *path*, *content*, *append=False*, *encoding='utf-8'*)

Utility function to save files in one line. It will manage the open and close of the file and creating directories if necessary.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **path** – Path of the file to be created.
- **content** – Content (str or bytes) to be write to the file.
- **append** – (Optional, Defaulted to False): If `True` the contents will be appended to the existing one.
- **encoding** – (Optional, Defaulted to utf-8): Specifies the output file text encoding.

Usage:

```
from conan.tools.files import save

save(self, "path/to/otherfile.txt", "contents of the file")
```

conan.tools.files.rename()

rename(*conanfile*, *src*, *dst*)

Utility functions to rename a file or folder src to dst with retrying. `os.rename()` frequently raises "Access is denied" exception on Windows. This function renames file or folder using robocopy to avoid the exception on Windows.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **src** – Path to be renamed.
- **dst** – Path to be renamed to.

Usage:

```
from conan.tools.files import rename

def source(self):
    rename(self, "lib-sources-abe2h9fe", "sources")  # renaming a folder
```

conan.tools.files.replace_in_file()

replace_in_file(*conanfile*, *file_path*, *search*, *replace*, *strict=True*, *encoding='utf-8'*, *regex=False*, *flags=RegexFlag.MULTILINE*)

Replace a string `search` in the contents of the file `file_path` with the string replace.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **file_path** – File path of the file to perform the replacing.
- **search** – String you want to be replaced. With `regex=True`, treated as a regular expression.
- **replace** – String to replace the searched string. With regex mode, backreferences (`\1`, `\g<name>`) are supported.
- **strict** – (Optional, Defaulted to `True`) If `True`, it raises an error if the searched string is not found, so nothing is actually replaced.
- **encoding** – (Optional, Defaulted to utf-8): Specifies the input and output files text encoding.
- **regex** – (Optional, Defaulted to `False`) If `True`, `search` is treated as a regular expression and applied with `re.sub`.
- **flags** – (Optional, Defaulted to `re.MULTILINE`) Flags passed to `re.sub` when `regex=True`.

Returns

`True` if the pattern was found, `False` otherwise if *strict* is `False`.

Usage:

```
from conan.tools.files import replace_in_file

replace_in_file(self, os.path.join(self.source_folder, "folder", "file.txt"), "foo", "bar
↪")
```

Regular expressions and backreferences can be used by enabling `regex`:

```
replace_in_file(self, "file.txt", r"version=(\d+)", r"version=v\1", regex=True)
```

conan.tools.files.chmod()

chmod(*conanfile*, *path: str*, *read: bool | None = None*, *write: bool | None = None*, *execute: bool | None = None*, *recursive: bool = False*)

Change file or directory permissions cross-platform.

New in version 2.15.

This function is a simple wrapper around the chmod Unix command, but it is cross-platform supported. It is indicated to use it instead of os.stat + os.chmod, as it only changes the permissions of the directory or file for the owner and avoids issues with the umask. On Windows is limited to changing write permission only.

Parameters

conanfile
[object] The current recipe object. Always use `self`.

path
[str] Path to the file or directory whose permissions will be changed.

read
[bool, optional] If `True`, the file or directory will be given read permissions for owner user. If `False`, the read permission will be removed. If `None`, the read permission will be left unchanged. Defaults to None.

write
[bool, optional] If `True`, the file or directory will be given write permissions for owner user. If `False`, the write permission will be removed. If `None`, the file or directory will not be changed. Defaults to None.

execute
[bool, optional] If `True`, the file or directory will be given execute permissions for owner user. If `False`, the execution permission will be removed. If `None`, the file or directory will not be changed. Defaults to None.

recursive
[bool] If `True`, the permissions will be applied recursively to all files and directories inside the specified directory. If `False`, only the specified file or directory will be changed. Defaults to False.

Returns

None

Examples

Listing 125: Add execution permission to a packaged bash script

```
from conan.tools.files import chmod
chmod(self, os.path.join(self.package_folder, "bin", "script.sh"), execute=True)
```

conan.tools.files.rm()

This function removes files from the filesystem. It can be used to remove a single file or a pattern based on fnmatch. It's indicated to use it instead of `os.remove` because it's cross-platform and may avoid permissions issues.

Listing 126: *Remove all files finished in .tmp in build_folder and recursively*

```
from conan.tools.files import rm

rm(self, "*.tmp", self.build_folder, recursive=True)
```

Listing 127: *Remove all files from bin_folder, except for any finished by .dll*

```
from conan.tools.files import rm

rm(self, "*", self.bin_folder, recursive=False, excludes="*.dll")
```

rm(*conanfile*, *pattern*, *folder*, *recursive=False*, *excludes=None*)

Utility functions to remove files matching a `pattern` in a `folder`.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **pattern** – Pattern that the files to be removed have to match (fnmatch).
- **folder** – Folder to search/remove the files.
- **recursive** – If `recursive` is specified it will search in the subfolders.
- **excludes** – (Optional, defaulted to None) A tuple/list of fnmatch patterns or even a single one to be excluded from the remove pattern.

conan.tools.files.mkdir()

mkdir(*conanfile*, *path*)

Utility functions to create a directory. The existence of the specified directory is checked, so mkdir() will do nothing if the directory already exists.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **path** – Path to the folder to be created.

Usage:

```
from conan.tools.files import mkdir

mkdir(self, "mydir") # Creates mydir if it does not already exist
mkdir(self, "mydir") # Does nothing
```

conan.tools.files.rmdir()

rmdir(*conanfile*, *path*)

Usage:

```
from conan.tools.files import rmdir

rmdir(self, "mydir") # Remove mydir if it exist
rmdir(self, "mydir") # Does nothing
```

conan.tools.files.chdir()

chdir(*conanfile*, *newdir*)

This is a context manager that allows to temporary change the current directory in your conanfile

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **newdir** – Directory path name to change the current directory.

Usage:

```
from conan.tools.files import chdir

def build(self):
    with chdir(self, "./subdir"):
        do_something()
```

conan.tools.files.unzip()

This function extract different compressed formats (`.tar`, `.tar.gz`, `.tgz`, `.tar.bz2`, `.tbz2`, `.tar.xz`, `.txz`, and `.zip`) into the given destination folder.

It also accepts gzipped files, with extension `.gz` (not matching any of the above), and it will unzip them into a file with the same name but without the extension, or to a filename defined by the `destination` argument.

```
from conan.tools.files import unzip

unzip(self, "myfile.zip")
# or to extract in "myfolder" sub-folder
unzip(self, "myfile.zip", "myfolder")
```

You can keep the permissions of the files using the `keep_permissions=True` parameter.

```
from conan.tools.files import unzip

unzip(self, "myfile.zip", "myfolder", keep_permissions=True)
```

Use the `pattern` argument if you want to filter specific files and paths to decompress from the archive.

```
from conan.tools.files import unzip

# Extract only files inside relative folder "small"
unzip(self, "bigfile.zip", pattern="small/*")
# Extract only txt files
unzip(self, "bigfile.zip", pattern="*.txt")
```

Important: In Conan 2.8 `unzip()` provides a new `extract_filter=None` argument and a new `tools.files.unzip:filter` configuration was added to prepare for future Python 3.14 breaking changes, in which the `data` filter for extracting tar archives will be made the default. The recommendation is to start using the `data` filter as soon as possible (the conf can be defined in `global.conf`, or it can be explicitly added as argument in recipes `unzip()` and `get()` helpers) as that is the current security recommendation while downloading sources from the internet.

Warning: By default, if no `extract_filter` argument or `tools.files.unzip:filter` conf is defined, Conan extracts `tar` files using the `fully_trusted` filter, also with Python 3.14 or later. This is done to avoid breaking existing packages (the `data` filter, for example, removes setuid/setgid/sticky bits and group/other write permissions) and because the extracted contents (source code or binaries) are going to be compiled or executed anyway, so archives must always be fully trusted. This default is not applied to `.zip` files, which are extracted with Python `zipfile` and are not affected by these filters. Read more in *the security guidelines*.

unzip(*conanfile*, *filename*, *destination='.'*, *keep_permissions=False*, *pattern=None*, *strip_root=False*, *extract_filter=None*, *excludes=None*)

Extract different compressed formats

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **filename** – Path to the compressed file.
- **destination** – (Optional, Defaulted to `.`) Destination folder (or file for .gz files)
- **keep_permissions** – (Optional, Defaulted to `False`) Keep the zip permissions. WARNING: Can be dangerous if the zip was not created in a NIX system, the bits could produce undefined permission schema. Use this option only if you are sure that the zip was created correctly.
- **pattern** – (Optional, Defaulted to `None`) Extract only paths matching the pattern. This should be a Unix shell-style wildcard, see fnmatch documentation for more details.
- **strip_root** – (Optional, Defaulted to False) If True, and all the unzipped contents are in a single folder it will flat the folder moving all the contents to the parent folder.
- **extract_filter** – (Optional, defaulted to None). When extracting a tar file, use the tar extracting filters define by Python in https://docs.python.org/3/library/tarfile.html
- **excludes** – (Optional, defaulted to None). When extracting a file, exclude paths matching any of the patterns. This should be a Unix shell-style wildcard, see fnmatch documentation

for more details.

conan.tools.files.update_conandata()

This function reads the `conandata.yml` inside the exported folder in the conan cache, if it exists. If the `conandata.yml` does not exist, it will create it. Then, it updates the conandata dictionary with the provided `data` one, which is updated recursively, prioritizing the `data` values, but keeping other existing ones. Finally the `conandata.yml` is saved in the same place.

This helper can only be used within the `export()` method, it can raise otherwise. One application is to capture in the `conandata.yml` the scm coordinates (like Git remote url and commit), to be able to recover it later in the `source()` method and have reproducible recipes that can build from sources without actually storing the sources in the recipe.

update_conandata(*conanfile*, *data*)

Tool to modify the `conandata.yml` once it is exported. It can be used, for example:

- To add additional data like the "commit" and "url" for the scm.
- To modify the contents cleaning the data that belong to other versions (different from the exported) to avoid changing the recipe revision when the changed data doesn't belong to the current version.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **data** – (Required) A dictionary (can be nested), of values to update

conan.tools.files.trim_conandata()

trim_conandata(*conanfile*, *raise_if_missing=True*)

Tool to modify the `conandata.yml` once it is exported, to limit it to the current version only

> **Warning:** The `conan.tools.files.trim_conandata()` function is in **preview**. See *the Conan stability* section for more information.

This function modifies the `conandata.yml` inside the exported folder in the conan cache, if it exists, and keeps only the information related to the currently built version.

This helper can only be used within the `export()` method or `post_export()` *hook*, it may raise in the future otherwise. One application is to ensure changes in the `conandata.yml` file related to some versions do not affect the generated recipe revisions of the rest.

Usage:

```
from conan import ConanFile
from conan.tools.files import trim_conandata

class Pkg(ConanFile):
    name = "pkg"

    def export(self):
        # any change to other versions in the conandata.yml
        # won't affect the revision of the version that is built
        trim_conandata(self)
```

conan.tools.files.collect_libs()

collect_libs(*conanfile*, *folder=None*)

Returns a sorted list of library names from the libraries (files with extensions *.so*, *.lib*, *.a* and *.dylib*) located inside the `conanfile.cpp_info.libdirs` (by default) or the **folder** directory relative to the package folder. Useful to collect not inter-dependent libraries or with complex names like `libmylib-x86-debug-en.lib`.

For UNIX libraries staring with **lib**, like *libmath.a*, this tool will collect the library name **math**.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **folder** – (Optional, Defaulted to `None`): String indicating the subfolder name inside `conanfile.package_folder` where the library files are.

Returns

A list with the library names

Warning: This tool collects the libraries searching directly inside the package folder and returns them in no specific order. If libraries are inter-dependent, then `package_info()` method should order them to achieve correct linking order.

Usage:

```
from conan.tools.files import collect_libs

def package_info(self):
    self.cpp_info.libdirs = ["lib", "other_libdir"]  # Default value is 'lib'
    self.cpp_info.libs = collect_libs(self)
```

For UNIX libraries starting with **lib**, like *libmath.a*, this tool will collect the library name **math**. Regarding symlinks, this tool will keep only the "most generic" file among the resolved real file and all symlinks pointing to this real file. For example among files below, this tool will select *libmath.dylib* file and therefore only append *math* in the returned list:

```
-rwxr-xr-x libmath.1.0.0.dylib lrwxr-xr-x libmath.1.dylib -> libmath.1.0.0.dylib
lrwxr-xr-x libmath.dylib -> libmath.1.dylib
```

conan.tools.files downloads

conan.tools.files.get()

get(*conanfile*, *url*, *md5=None*, *sha1=None*, *sha256=None*, *destination='.'*, *filename=''*, *keep_permissions=False*, *pattern=None*, *verify=True*, *retry=None*, *retry_wait=None*, *auth=None*, *headers=None*, *strip_root=False*, *extract_filter=None*, *excludes=None*)

High level download and decompressing of a tgz, zip or other compressed format file. Just a high level wrapper for download, unzip, and remove the temporary zip file once unzipped. You can pass hash checking parameters: `md5`, `sha1`, `sha256`. All the specified algorithms will be checked. If any of them doesn't match, it will raise a `ConanException`.

Parameters

- **conanfile** – The current recipe object. Always use `self`.

- **destination** – (Optional defaulted to `.`) Destination folder
- **filename** – (Optional defaulted to ‘’) If provided, the saved file will have the specified name, otherwise it is deduced from the URL
- **url** – forwarded to `tools.file.download()`.
- **md5** – forwarded to `tools.file.download()`.
- **sha1** – forwarded to `tools.file.download()`.
- **sha256** – forwarded to `tools.file.download()`.
- **keep_permissions** – forwarded to `tools.file.unzip()`.
- **pattern** – forwarded to `tools.file.unzip()`.
- **verify** – forwarded to `tools.file.download()`.
- **retry** – forwarded to `tools.file.download()`.
- **retry_wait** – S forwarded to `tools.file.download()`.
- **auth** – forwarded to `tools.file.download()`.
- **headers** – forwarded to `tools.file.download()`.
- **strip_root** – forwarded to `tools.file.unzip()`.
- **extract_filter** – forwarded to `tools.file.unzip()`.
- **excludes** – forwarded to `tools.file.unzip()`.

Important: `get()` calls internally `unzip()`. Please read the note in *conan.tools.files.unzip()* regarding Python 3.14 breaking changes and the new tar archive extract filters.

conan.tools.files.ftp_download()

ftp_download(*conanfile*, *host*, *filename*, *login=''*, *password=''*, *secure=False*)

Ftp download of a file. Retrieves a file from an FTP server.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **host** – IP or host of the FTP server.
- **filename** – Path to the file to be downloaded.
- **login** – Authentication login.
- **password** – Authentication password.
- **secure** – Set to True to use FTP over TLS/SSL (FTPS). Defaults to False for regular FTP.

Usage:

```
from conan.tools.files import ftp_download

def source(self):
    ftp_download(self, 'ftp.debian.org', "debian/README")
    self.output.info(load("README"))
```

conan.tools.files.download()

download(*conanfile*, *url*, *filename*, *verify=True*, *retry=None*, *retry_wait=None*, *auth=None*, *headers=None*, *md5=None*, *sha1=None*, *sha256=None*)

Retrieves a file from a given URL into a file with a given filename. It uses certificates from a list of known verifiers for https downloads, but this can be optionally disabled.

You can pass hash checking parameters: `md5`, `sha1`, `sha256`. All the specified algorithms will be checked. If any of them doesn't match, the downloaded file will be removed and it will raise a `ConanException`.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **url** – URL to download. It can be a list, which only the first one will be downloaded, and the follow URLs will be used as mirror in case of download error. Files accessible in the local filesystem can be referenced with a URL starting with `file:///` followed by an absolute path to a file (where the third `/` implies `localhost`).
- **filename** – Name of the file to be created in the local storage
- **verify** – When False, disables https certificate validation
- **retry** – Number of retries in case of failure. Default is overridden by "tools.files.download:retry" conf
- **retry_wait** – Seconds to wait between download attempts. Default is overriden by "tools.files.download:retry_wait" conf.
- **auth** – A tuple of user and password to use HTTPBasic authentication
- **headers** – A dictionary with additional headers
- **md5** – MD5 hash code to check the downloaded file
- **sha1** – SHA-1 hash code to check the downloaded file
- **sha256** – SHA-256 hash code to check the downloaded file

Usage:

```
download(self, "http://someurl/somefile.zip", "myfilename.zip")

# to disable verification:
download(self, "http://someurl/somefile.zip", "myfilename.zip", verify=False)

# to retry the download 2 times waiting 5 seconds between them
download(self, "http://someurl/somefile.zip", "myfilename.zip", retry=2, retry_wait=5)

# Use https basic authentication
download(self, "http://someurl/somefile.zip", "myfilename.zip", auth=("user", "password
↪"))

# Pass some header
download(self, "http://someurl/somefile.zip", "myfilename.zip", headers={"Myheader": "My␣
↪value"})

# Download and check file checksum
download(self, "http://someurl/somefile.zip", "myfilename.zip", md5=
↪"e5d695597e9fa520209d1b41edad2a27")
```

(continues on next page)

(continued from previous page)

```
# to add mirrors
download(self, ["https://ftp.gnu.org/gnu/gcc/gcc-9.3.0/gcc-9.3.0.tar.gz",
                "http://mirror.linux-ia64.org/gnu/gcc/releases/gcc-9.3.0/gcc-9.3.0.tar.gz
↪"],
                "gcc-9.3.0.tar.gz",
               sha256="5258a9b6afe9463c2e56b9e8355b1a4bee125ca828b8078f910303bc2ef91fa6")
```

conf

It uses these *configuration entries*:

- `tools.files.download:retry`: number of retries in case some error occurs.
- `tools.files.download:retry_wait`: seconds to wait between retries.

conan.tools.files patches

conan.tools.files.patch()

patch(*conanfile*, *base_path=None*, *patch_file=None*, *patch_string=None*, *strip=0*, *fuzz=False*, ***kwargs*)

Applies a diff from file (patch_file) or string (patch_string) in the conanfile.source_folder directory. The folder containing the sources can be customized with the self.folders attribute in the layout(self) method.

Parameters

- **conanfile** – the current recipe, always pass 'self'
- **base_path** – The path is a relative path to conanfile.export_sources_folder unless an absolute path is provided.
- **patch_file** – Patch file that should be applied. The path is relative to the conanfile.source_folder unless an absolute path is provided.
- **patch_string** – Patch string that should be applied.
- **strip** – Number of folders to be stripped from the path.
- **fuzz** – Should accept fuzzy patches.
- **kwargs** – Extra parameters that can be added and will contribute to output information

Usage:

```
from conan.tools.files import patch

def build(self):
    for it in self.conan_data.get("patches", {}).get(self.version, []):
        patch(self, **it)
```

conan.tools.files.apply_conandata_patches()

apply_conandata_patches(*conanfile*)

Applies patches stored in `conanfile.conan_data` (read from `conandata.yml` file). It will apply all the patches under `patches` entry that matches the given `conanfile.version`. If versions are not defined in `conandata.yml` it will apply all the patches directly under `patches` keyword.

The key entries will be passed as kwargs to the `patch` function.

Usage:

```
from conan.tools.files import apply_conandata_patches

def source(self):
    apply_conandata_patches(self)
```

Examples of `conandata.yml`:

```
patches:
- patch_file: "patches/0001-buildflatbuffers-cmake.patch"
- patch_file: "patches/0002-implicit-copy-constructor.patch"
  base_path: "subfolder"
  patch_type: backport
  patch_source: https://github.com/google/flatbuffers/pull/5650
  patch_description: Needed to build with modern clang compilers.
```

With different patches for different versions:

```
patches:
  "1.11.0":
    - patch_file: "patches/0001-buildflatbuffers-cmake.patch"
    - patch_file: "patches/0002-implicit-copy-constructor.patch"
      base_path: "subfolder"
      patch_type: backport
      patch_source: https://github.com/google/flatbuffers/pull/5650
      patch_description: Needed to build with modern clang compilers.
  "1.12.0":
    - patch_file: "patches/0001-buildflatbuffers-cmake.patch"
    - patch_string: |
        --- a/tests/misc-test.c
        +++ b/tests/misc-test.c
        @@ -1232,6 +1292,8 @@ main (int argc, char **argv)
              g_test_add_func ("/misc/pause-cancel", do_pause_cancel_test);
              g_test_add_data_func ("/misc/stealing/async", GINT_TO_POINTER (FALSE), do_
↪stealing_test);
              g_test_add_data_func ("/misc/stealing/sync", GINT_TO_POINTER (TRUE), do_
↪stealing_test);
        +     g_test_add_func ("/misc/response/informational/content-length", do_
↪response_informational_content_length_test);
        +

        ret = g_test_run ();
    - patch_file: "patches/0003-fix-content-length-calculation.patch"
```

For each patch, a `patch_file`, a `patch_string` or a `patch_user` field must be provided. The first two are automat-

ically applied by `apply_conandata_patches()`, while `patch_user` are ignored, and must be handled by the user directly in the `conanfile.py` recipe.

conan.tools.files.export_conandata_patches()

export_conandata_patches(*conanfile*)

Exports patches stored in 'conanfile.conan_data' (read from 'conandata.yml' file). It will export all the patches under 'patches' entry that matches the given 'conanfile.version'. If versions are not defined in 'conandata.yml' it will export all the patches directly under 'patches' keyword.

Example of `conandata.yml` without versions defined:

```
from conan.tools.files import export_conandata_patches
def export_sources(self):
    export_conandata_patches(self)
```

core.sources.patch:extra_path

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `export_conandata_patches()` tool can automatically inject patches from an external path at package creation time using the `core.sources.patch:extra_path` core configuration.

That means that `conan create` commands in `conan-center-index` repository could inject and apply patches without necessarily putting the patches in the same repository and without modifying the `conandata.yml` files.

The `core.sources.patch:extra_path` configuration should point to a folder containing all possible extra patches for all possible packages, structured by package name, following the same conventions as `conan-center-index` repository:

```
extra_folder
    pkgname1
       conandata.yml
       patches
          mypatch.path
    pkgname2
       ...
```

The `conandata.yml` should also follow the same structure:

```
patches:
    "1.0":
        - patch_file: "patches/mypatch.patch"
```

Note: It is impossible to apply patches to arbitrary dependencies when installing them (`conan install --build=xxx`), as the possible injected patches are part of the "source" identity of the package, and must be represented in their recipe revision. Already existing packages in the cache or in the remote servers have already exported its files and computed a recipe revision, so patches cannot be applied there without violating the identity (and as such

the reproducibility and traceability) of packages. As a conclusion, it means that `core.sources.patch:extra_path` can only work at `conan create` time.

conan.tools.files checksums

conan.tools.files.check_md5()

check_md5(*conanfile*, *file_path*, *signature*)

Check that the specified `MD5` hash of the `file_path` matches the actual hash. If doesn't match it will raise a `ConanException`.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **file_path** – Path of the file to check.
- **signature** – Expected MD5 hash.

conan.tools.files.check_sha1()

check_sha1(*conanfile*, *file_path*, *signature*)

Check that the specified `SHA-1` hash of the `file_path` matches the actual hash. If doesn't match it will raise a `ConanException`.

Parameters

- **conanfile** – Conanfile object.
- **file_path** – Path of the file to check.
- **signature** – Expected SHA-1 hash.

conan.tools.files.check_sha256()

check_sha256(*conanfile*, *file_path*, *signature*)

Check that the specified `SHA-256` hash of the `file_path` matches the actual hash. If doesn't match it will raise a `ConanException`.

Parameters

- **conanfile** – Conanfile object.
- **file_path** – Path of the file to check.
- **signature** – Expected SHA-256 hash.

conan.tools.files.symlinks

conan.tools.files.symlinks.absolute_to_relative_symlinks()

absolute_to_relative_symlinks(*conanfile*, *base_folder*)

Convert the symlinks with absolute paths into relative ones if they are pointing to a file or directory inside the `base_folder`. Any absolute symlink pointing outside the `base_folder` will be ignored.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **base_folder** – Folder to be scanned.

conan.tools.files.symlinks.remove_external_symlinks()

remove_external_symlinks(*conanfile*, *base_folder*)

Remove the symlinks to files that point outside the `base_folder`, no matter if relative or absolute.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **base_folder** – Folder to be scanned.

conan.tools.files.symlinks.remove_broken_symlinks()

remove_broken_symlinks(*conanfile*, *base_folder=None*)

Remove the broken symlinks, no matter if relative or absolute.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **base_folder** – Folder to be scanned.

9.10.8 conan.tools.gnu

AutotoolsDeps

The `AutotoolsDeps` is the dependencies generator for Autotools. It will generate shell scripts containing environment variable definitions that the autotools build system can understand.

It can be used by name in conanfiles:

Listing 128: conanfile.py

```
class Pkg(ConanFile):
    generators = "AutotoolsDeps"
```

Listing 129: conanfile.txt

```
[generators]
AutotoolsDeps
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = AutotoolsDeps(self)
        tc.generate()
```

Generated files

It will generate the file `conanautotoolsdeps.sh` or `conanautotoolsdeps.bat`:

```
$ conan install conanfile.py # default is Release
$ source conanautotoolsdeps.sh
# or in Windows
$ conanautotoolsdeps.bat
```

These launchers will define aggregated variables `CPPFLAGS`, `LIBS`, `LDFLAGS`, `CXXFLAGS`, `CFLAGS` that accumulate all dependencies information, including transitive dependencies, with flags like `-I<path>`, `-L<path>`, etc.

Important: On Windows with `msvc`, `AutotoolsDeps` emits linker flags in `_LINK_` rather than `LDFLAGS`, so the Autotools wrapper can process the flags correctly. `_LINK_` is used instead of `LINK` to avoid a conflict with the `LINK` variable that Automake defines for C targets.

At this moment, only the `requires` information is generated, the `tool_requires` one is not managed by this generator yet.

Customization

To modify the computed values, you can access the `.environment` property that returns an *Environment* class.

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = AutotoolsDeps(self)
        tc.environment.remove("CPPFLAGS", "undesired_value")
        tc.environment.append("CPPFLAGS", "var")
```

(continues on next page)

(continued from previous page)

```
        tc.environment.define("OTHER", "cat")
        tc.environment.unset("LDFLAGS")
        tc.generate()
```

Reference

class AutotoolsDeps(*conanfile*)

property environment

Returns
An `Environment` object containing the computed variables. If you need to modify some of the computed values you can access to the `environment` object.

AutotoolsToolchain

The `AutotoolsToolchain` is the toolchain generator for Autotools. It will generate shell scripts containing environment variable definitions that the autotools build system can understand.

Warning: `AutotoolsToolchain` and *GnuToolchain* are mutually exclusive alternatives. Do not use both in the same recipe: choose one or the other.

This generator can be used by name in conanfiles:

Listing 130: conanfile.py

```
class Pkg(ConanFile):
    generators = "AutotoolsToolchain"
```

Listing 131: conanfile.txt

```
[generators]
AutotoolsToolchain
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = AutotoolsToolchain(self)
        tc.generate()
```

Generated files

It will generate the file `conanautotoolstoolchain.sh` or `conanautotoolstoolchain.bat` files:

```
$ conan install conanfile.py # default is Release
$ source conanautotoolstoolchain.sh
# or in Windows
$ conanautotoolstoolchain.bat
```

This launchers will append information to the `CPPFLAGS`, `LDFLAGS`, `CXXFLAGS`, `CFLAGS` environment variables that translate the settings and options to the corresponding build flags like `-stdlib=libstdc++`, `-std=gnu14`, architecture flags, etc. It will also append the folder where the Conan generators are located to the `PKG_CONFIG_PATH` environment variable. On Apple platforms, `ASFLAGS` is also set with architecture and sysroot flags to ensure the assembler uses the same cross-compilation settings as the C/C++ compilers.

Since Conan 2.4.0, in a cross-building context, the environment variables `CC_FOR_BUILD` and `CXX_FOR_BUILD` are also set if the build profile defines the `c` and `cpp` values in the configuration variable `tools.build:compiler_executables`. See more info in the *conf section*.

This generator will also generate a file called `conanbuild.conf` containing two keys:

- **configure_args**: Arguments to call the `configure` script.
- **make_args**: Arguments to call the `make` script.
- **autoreconf_args**: Arguments to call the `autoreconf` script.

The *Autotools build helper* will use that `conanbuild.conf` file to seamlessly call the configure and make script using these precalculated arguments.

Customization

You can change some attributes before calling the `generate()` method if you want to change some of the precalculated values:

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = AutotoolsToolchain(self)
        tc.configure_args.append("--my_argument")
        tc.generate()
```

- **configure_args: Additional arguments to be passed to the configure script.**
  - **By default the following arguments are passed:**
    - `--prefix`: Takes `/` as default value.
    - `--bindir=${prefix}/bin`
    - `--sbindir=${prefix}/bin`
    - `--libdir=${prefix}/lib`
    - `--includedir=${prefix}/include`

        * `--oldincludedir=${prefix}/include`
        * `--datarootdir=${prefix}/res`
    - **Also if the shared option exists it will add by default:**
        * `--enable-shared`, `--disable-static` if `shared==True`
        * `--disable-shared`, `--enable-static` if `shared==False`

- **make_args** (Defaulted to `[]`): Additional arguments to be passed to he make script.
- **autoreconf_args** (Defaulted to `["--force", "--install"]`): Additional arguments to be passed to he make script.
- **extra_defines** (Defaulted to `[]`): Additional defines.
- **extra_cxxflags** (Defaulted to `[]`): Additional cxxflags.
- **extra_cflags** (Defaulted to `[]`): Additional cflags.
- **extra_ldflags** (Defaulted to `[]`): Additional ldflags.
- **ndebug**: "NDEBUG" if the `settings.build_type` != *Debug*.
- **gcc_cxx11_abi**: "_GLIBCXX_USE_CXX11_ABI" if `gcc/libstdc++`.
- **libcxx**: Flag calculated from `settings.compiler.libcxx`.
- **fpic**: True/False from `options.fpic` if defined.
- **cppstd**: Flag from `settings.compiler.cppstd`
- **arch_flag**: Flag from `settings.arch`
- **build_type_flags**: Flags from `settings.build_type`
- **sysroot_flag**: To pass the `--sysroot` flag to the compiler.
- **apple_arch_flag**: Only when cross-building with Apple systems. Flags from `settings.arch`. For universal binaries, contains multiple `-arch` flags.
- **apple_isysroot_flag**: Only when cross-building with Apple systems. Path to the root sdk.
- **apple_min_version_flag**: Only for Apple systems, defines the `-m{ios,macosx,watchos,tvos,...}-version-min=xxx` flag.
- **msvc_runtime_flag**: Flag from `settings.compiler.runtime_type` when compiler is `msvc` or `settings.compiler.runtime` when using the deprecated `Visual Studio`.
- **asflags**: List of assembler flags set in the `ASFLAGS` environment variable. On Apple platforms, populated with `arch_flag`, `sysroot_flag`, `apple_isysroot_flag`, `apple_arch_flag`, and `apple_min_version_flag`. Empty on non-Apple platforms. Applies to both `AutotoolsToolchain` and `GnuToolchain`.

The following attributes are ready-only and will contain the calculated values for the current configuration and customized attributes. Some recipes might need to read them to generate custom build files (not strictly Autotools) with the configuration:

- **defines**
- **cxxflags**
- **cflags**
- **ldflags**

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    def generate(self):
        tc = AutotoolsToolchain(self)
        # Customize the flags
        tc.extra_cxxflags = ["MyFlag"]
        # Read the computed flags and use them (write custom files etc)
        tc.defines
        tc.cxxflags
        tc.cflags
        tc.ldflags
```

If you want to change the default values for `configure_args`, adjust the `cpp.package` object at the `layout()` method:

```
def layout(self):
    ...
    # For bindir and sbindir takes the first value:
    self.cpp.package.bindirs = ["mybin"]
    # For libdir takes the first value:
    self.cpp.package.libdirs = ["mylib"]
    # For includedir and oldincludedir takes the first value:
    self.cpp.package.includedirs = ["myinclude"]
    # For datarootdir takes the first value:
    self.cpp.package.resdirs = ["myres"]
```

Note: It is **not valid** to change the self.cpp_info at the `package_info()` method.

Customizing the environment

If your `Makefile` or `configure` scripts need some other environment variable rather than `CPPFLAGS`, `LDFLAGS`, `CXXFLAGS` or `CFLAGS`, you can customize it before calling the `generate()` method. Call the `environment()` method to calculate the mentioned variables and then add the variables that you need. The `environment()` method returns an *Environment* object:

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        at = AutotoolsToolchain(self)
        env = at.environment()
        env.define("FOO", "BAR")
        at.generate(env)
```

The AutotoolsToolchain also sets CXXFLAGS, CFLAGS, LDFLAGS and CPPFLAGS reading variables from the [conf] section in the profiles. *See the conf reference below*.

Managing the configure_args, make_args and autoreconf_args attributes

AutotoolsToolchain provides some help methods so users can add/update/remove values defined in configure_args, make_args and autoreconf_args (all of them lists of strings). Those methods are:

- update_configure_args(updated_flags): will change AutotoolsToolchain.configure_args.
- update_make_args(updated_flags): will change AutotoolsToolchain.make_args.
- update_autoreconf_args(updated_flags): will change AutotoolsToolchain.autoreconf_args.

Where updated_flags is a dict-like Python object defining all the flags to change. It follows the next rules:

- Key-value are the flags names and their values, e.g., {"--enable-tools": no} will be translated as --enable-tools=no.
- If that key has no value, then it will be an empty string, e.g., {"--disable-verbose": ""} will be translated as --disable-verbose.
- If the key value is None, it means that you want to remove that flag from the xxxxxx_args (notice that it could be configure_args, make_args or autoreconf_args), e.g., {"--force": None} will remove that flag from the final result.

In a nutshell, you will:

- **Add arguments**: if the given flag in updated_flags does not already exist in xxxxxx_args.
- **Update arguments**: if the given flag in updated_flags already exists in attribute xxxxxx_args.
- **Remove arguments**: if the given flag in updated_flags already exists in xxxxxx_args and it's passed with None as value.

For instance:

```
from conan import ConanFile
from conan.tools.gnu import AutotoolsToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        at = AutotoolsToolchain(self)
        at.update_configure_args({
            "--new-super-flag": "",  # add new flag '--new-super-flag'
            "--host": "my-gnu-triplet",  # update flag '--host=my-gnu-triplet'
            "--force": None  # remove existing '--force' flag
        })
        at.generate()
```

The AutotoolsToolchain will listen to tools.gnu:extra_configure_args from the *global.conf* to extend the configure_args attribute.

Support for Universal Binaries in macOS

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Starting in Conan 2.21.0, there's support for building universal binaries on macOS using AutotoolsToolchain. To specify multiple architectures for a universal binary in Conan, use the | separator when defining the architecture in the settings. This approach enables passing a list of architectures. For example, running:

```
conan create . --name=mylibrary --version=1.0 -s="arch=armv8|x86_64"
```

will create a universal binary for *mylibrary* containing both `armv8` and `x86_64` architectures, by setting multiple `-arch` flags in the generated toolchain script.

Warning: It is important to note that this method is not applicable to build systems other than CMake or Autotools via `CMakeToolchain` and `AutotoolsToolchain`.

Attempting to use universal architecture settings on non-Apple platforms will result in an error.

Be aware that this feature is primarily beneficial for building final universal binaries for release purposes. The default Conan behavior of managing one binary per architecture generally provides a more reliable and trouble-free experience. Users should be cautious and not overly rely on this feature for broader use cases.

Reference

class AutotoolsToolchain(*conanfile*, *namespace=None*, *prefix='/'*)

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **namespace** – This argument avoids collisions when you have multiple toolchain calls in the same recipe. By setting this argument, the *conanbuild.conf* file used to pass information to the build helper will be named as *<namespace>_conanbuild.conf*. The default value is `None` meaning that the name of the generated file is *conanbuild.conf*. This namespace must be also set with the same value in the constructor of the Autotools build helper so that it reads the information from the proper file.
- **prefix** – Folder to use for `--prefix` argument ("/" by default).

update_configure_args(*updated_flags*)

Helper to update/prune flags from `self.configure_args`.

Parameters

updated_flags – `dict` with arguments as keys and their argument values. Notice that if argument value is `None`, this one will be pruned.

update_make_args(*updated_flags*)

Helper to update/prune arguments from `self.make_args`.

Parameters

updated_flags – `dict` with arguments as keys and their argument values. Notice that if argument value is `None`, this one will be pruned.

`update_autoreconf_args`(*updated_flags*)

Helper to update/prune arguments from `self.autoreconf_args`.

Parameters

`updated_flags` – `dict` with arguments as keys and their argument values. Notice that if argument value is `None`, this one will be pruned.

conf

- `tools.build:cxxflags` list of extra C++ flags that will be used by `CXXFLAGS`.
- `tools.build:cflags` list of extra of pure C flags that will be used by `CFLAGS`.
- `tools.build:sharedlinkflags` list of extra linker flags that will be used by `LDFLAGS`.
- `tools.build:exelinkflags` list of extra linker flags that will be used by `LDFLAGS`.
- `tools.build:rcflags```list of extra RC flags to define ``RCFLAGS`
- `tools.build:defines` list of preprocessor definitions that will be used by `CPPFLAGS`.
- `tools.build:linker_scripts` list of linker scripts, each of which will be prefixed with `-T` and added to `LDFLAGS`. Only use this flag with linkers that supports specifying linker scripts with the `-T` flag, such as `ld`, `gold`, and `lld`.
- `tools.build:sysroot` defines the `--sysroot` flag to the compiler.
- `tools.android:ndk_path` (*new since Conan 2.5.0*) argument for NDK path in case of Android cross-compilation. It is used to set some environment variables like `CC`, `CXX`, `LD`, `STRIP`, `RANLIB`, `AS`, `AR`, and, since *Conan 2.11.0*, `ADDR2LINE`, `NM`, `OBJCOPY`, `OBJDUMP`, `READELF`, and `ELFEDIT` in the *conanautotoolstoolchain.sh|bat* script, as long as they are not already defined in the `buildenv` environment. If they are defined in the `buildenv` environment, the `conanautotoolstoolchain` file will not define them, leaving their definition to the `VirtualBuildEnv` generator.
- `tools.build:compiler_executables` dict-like Python object which specifies the compiler as key and the compiler executable path as value. Those keys will be mapped as follows:
- `tools.gnu:extra_configure_args` list of arguments to extend the `configure_args` attribute of `AutotoolsToolchain`.
  - `c`: will set `CC` (and `CC_FOR_BUILD` if cross-building) in *conanautotoolstoolchain.sh|bat* script.
  - `cpp`: will set `CXX` (and `CXX_FOR_BUILD` if cross-building) in *conanautotoolstoolchain.sh|bat* script.
  - `rc`: will set `RC` in *conanautotoolstoolchain.sh|bat* script.
  - `cuda`: will set `NVCC` in *conanautotoolstoolchain.sh|bat* script.
  - `fortran`: will set `FC` in *conanautotoolstoolchain.sh|bat* script.
  - `objc` *(since Conan 2.32)*: will set `OBJC` in *conanautotoolstoolchain.sh|bat* script.
  - `objcpp` *(since Conan 2.32)*: will set `OBJCXX` in *conanautotoolstoolchain.sh|bat* script.
  - `asm` *(since Conan 2.32)*: will set `AS` in *conanautotoolstoolchain.sh|bat* script.

  Since Conan 2.32, keys not listed above are reported with a `risk` warning.

Note: flags order of preference: Flags specified in the *tools.build* configuration, such as *cxxflags*, *cflags*, *sharedlinkflags*, *exelinkflags*, and *defines*, will always take precedence over those set by the AutotoolsToolchain attributes.

GnuToolchain

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `GnuToolchain` is the toolchain generator for Autotools and other GNU-like build systems. It is a more modern and usable alternative to the *AutotoolsToolchain*, based on it, but with an improved user experience, like the dict-like `configure_args`, `make_args` and `autoreconf_args` attributes or the `extra_env` attribute to customize the generated environment. It will generate shell scripts containing environment variable definitions that the autotools build system can understand.

Warning: `GnuToolchain` and *AutotoolsToolchain* are mutually exclusive alternatives. Do not use both in the same recipe: choose one or the other.

This generator can be used by name in conanfiles:

Listing 132: conanfile.py

```
class Pkg(ConanFile):
    generators = "GnuToolchain"
```

Listing 133: conanfile.txt

```
[generators]
GnuToolchain
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.gnu import GnuToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = GnuToolchain(self)
        tc.generate()
```

Generated files

It will generate either the `conangnutoolchain.sh` or `conangnutoolchain.bat` files:

```
$ conan install # there is a conanfile here, default is Release
$ source conangnutoolchain.sh
# or in Windows
$ conangnutoolchain.bat
```

These launchers will append information to the `CPPFLAGS`, `LDFLAGS`, `CXXFLAGS`, `CFLAGS` environment variables that translate the settings and options to the corresponding build flags like `-stdlib=libstdc++`, `-std=gnu14`, architecture flags, etc. It will also append the folder where the Conan generators are located to the `PKG_CONFIG_PATH` environment

variable. On Apple platforms, `ASFLAGS` is also set with architecture and sysroot flags to ensure the assembler uses the same cross-compilation settings as the C/C++ compilers.

In a cross-building context, the environment variables `CC_FOR_BUILD` and `CXX_FOR_BUILD` are also set if the build profile defines the `c` and `cpp` values in the configuration variable `tools.build:compiler_executables`. See more info in the *conf section*.

This generator will also generate a file called `conanbuild.conf` containing three keys:

- **configure_args**: Arguments to call the `configure` script.
- **make_args**: Arguments to call the `make` script.
- **autoreconf_args**: Arguments to call the `autoreconf` script.

The *Autotools build helper* will use that `conanbuild.conf` file to seamlessly call the configure and make script using these precalculated arguments.

Customization

You can change some attributes before calling the `generate()` method if you want to change some of the precalculated values:

```
from conan import ConanFile
from conan.tools.gnu import GnuToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = GnuToolchain(self)
        tc.configure_args["--my_argument"] = None
        tc.generate()
```

The constructor of `GnuToolchain` accepts the following arguments:

- **namespace** (Defaulted to `None`): Avoids collisions when there are multiple toolchain calls in the same recipe. When set, the generated file will be named *<namespace>_conanbuild.conf* instead of *conanbuild.conf*. The same value must be passed to the constructor of the *Autotools build helper*.
- **prefix** (Defaulted to `"/"`): Folder to use for the `--prefix` argument.

These are the attributes that can be customized:

- **configure_args**: Dict-like object with the arguments to be passed to the configure script. The keys are the argument names and the values their values, or `None` for arguments without value.
  - By default the following arguments are passed:
    - `--prefix`: Takes the `prefix` constructor argument, `/` as default value.
    - `--bindir=${prefix}/bin`
    - `--sbindir=${prefix}/bin`
    - `--libdir=${prefix}/lib`
    - `--includedir=${prefix}/include`
    - `--oldincludedir=${prefix}/include`
    - `--datarootdir=${prefix}/res`

  - Also, depending on the `package_type` of the recipe, it will add by default:
    * `--enable-shared`, `--disable-static` if `package_type` is `shared-library`
    * `--disable-shared`, `--enable-static` if `package_type` is `static-library`
  - When cross-building, `--host` and `--build` are added with the calculated GNU triplets.

- **make_args** (Defaulted to `{}`): Dict-like object with the additional arguments to be passed to the make script.
- **autoreconf_args** (Defaulted to `{"--force": None, "--install": None}`): Dict-like object with the additional arguments to be passed to the autoreconf script.
- **extra_defines** (Defaulted to `[]`): Additional defines.
- **extra_cxxflags** (Defaulted to `[]`): Additional cxxflags.
- **extra_cflags** (Defaulted to `[]`): Additional cflags.
- **extra_ldflags** (Defaulted to `[]`): Additional ldflags.
- **extra_env**: *Environment* object with the extra environment variables to define in the generated script. It is initialized with some default values (like the `CC`, `CXX`, `LD`, `AR`, `NM`, `OBJDUMP`, `RANLIB` and `STRIP` variables for MSVC, the Android NDK ones if `tools.android:ndk_path` is defined, or the ones defined with `tools.build:compiler_executables`) and the values defined here have precedence over the ones calculated by the toolchain.
- **ndebug**: "NDEBUG" if the `settings.build_type` is `Release`, `RelWithDebInfo` or `MinSizeRel`.
- **gcc_cxx11_abi**: "_GLIBCXX_USE_CXX11_ABI" if `gcc/libstdc++`.
- **libcxx**: Flag calculated from `settings.compiler.libcxx`.
- **fpic**: True/False from `options.fPIC` if defined.
- **cppstd**: Flag from `settings.compiler.cppstd`
- **arch_flag**: Flag from `settings.arch`
- **arch_ld_flag**: Linker flag from `settings.arch`
- **threads_flags**: List of flags to enable threads, if applicable for the compiler.
- **build_type_flags**: Flags from `settings.build_type`
- **build_type_link_flags**: Linker flags from `settings.build_type`
- **sysroot_flag**: To pass the `--sysroot` flag to the compiler.
- **apple_arch_flag**: Only when cross-building with Apple systems. Flags from `settings.arch`. For universal binaries, contains multiple `-arch` flags.
- **apple_isysroot_flag**: Only when cross-building with Apple systems. Path to the root sdk.
- **apple_min_version_flag**: Only for Apple systems, defines the `-m{ios,macosx,watchos,tvos,...}-version-min=xxx` flag.
- **msvc_runtime_flag**: Flag from `settings.compiler.runtime_type` when compiler is `msvc`.
- **msvc_extra_flags**: Extra flags for MSVC (like `-FS`) that are automatically added to the `CXXFLAGS` and `CFLAGS`.
- **msvc_runtime_link_flags**: Extra linker flags when using `clang` with the MSVC runtime.
- **asflags**: List of assembler flags set in the `ASFLAGS` environment variable. On Apple platforms, populated with `arch_flag`, `sysroot_flag`, `apple_isysroot_flag`, `apple_arch_flag`, and `apple_min_version_flag`. Empty on non-Apple platforms.

The following attributes are read-only and will contain the calculated values for the current configuration and customized attributes. Some recipes might need to read them to generate custom build files (not strictly Autotools) with the configuration:

- **defines**
- **cxxflags**
- **cflags**
- **ldflags**
- **rcflags**

```
from conan import ConanFile
from conan.tools.gnu import GnuToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = GnuToolchain(self)
        # Customize the flags
        tc.extra_cxxflags = ["MyFlag"]
        # Read the computed flags and use them (write custom files etc)
        tc.defines
        tc.cxxflags
        tc.cflags
        tc.ldflags
```

If you want to change the default values for `configure_args`, adjust the `cpp.package` object at the `layout()` method:

```
def layout(self):
    ...
    # For bindir and sbindir takes the first value:
    self.cpp.package.bindirs = ["mybin"]
    # For libdir takes the first value:
    self.cpp.package.libdirs = ["mylib"]
    # For includedir and oldincludedir takes the first value:
    self.cpp.package.includedirs = ["myinclude"]
    # For datarootdir takes the first value:
    self.cpp.package.resdirs = ["myres"]
```

Note: It is **not valid** to change the self.cpp_info at the `package_info()` method.

Managing the configure_args, make_args and autoreconf_args attributes

In `GnuToolchain`, `configure_args`, `make_args` and `autoreconf_args` are dict-like Python objects, so they can be managed with the regular Python dictionary operations. This is the main difference with the `AutotoolsToolchain`, that uses lists of strings and some helper methods to modify them. The rules are:

- Key-value are the flags names and their values, e.g., `{"--enable-tools": "no"}` will be translated as `--enable-tools=no`.
- If that flag has no value, then the value must be `None`, e.g., `{"--disable-verbose": None}` will be translated as `--disable-verbose`.
- Setting a key that already exists will update its value, and deleting a key will remove the flag from the final result.

For instance:

```
from conan import ConanFile
from conan.tools.gnu import GnuToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = GnuToolchain(self)
        # add new flag '--new-super-flag'
        tc.configure_args["--new-super-flag"] = None
        # update flag '--host=my-gnu-triplet'
        tc.configure_args["--host"] = "my-gnu-triplet"
        # remove existing '--force' flag
        del tc.autoreconf_args["--force"]
        tc.generate()
```

The `GnuToolchain` will listen to `tools.gnu:extra_configure_args` from the *global.conf* to extend the `configure_args` attribute.

The `yes_no()` helper method returns `"yes"` or `"no"` depending on the value of a recipe option, which is useful for the `--enable-xxx=yes|no` arguments:

```
from conan import ConanFile
from conan.tools.gnu import GnuToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    options = {"with_tools": [True, False]}
    default_options = {"with_tools": True}

    def generate(self):
        tc = GnuToolchain(self)
        # --enable-tools=yes or --enable-tools=no
        tc.configure_args["--enable-tools"] = tc.yes_no("with_tools")
        # Negated, --disable-tools=no or --disable-tools=yes
        tc.configure_args["--disable-tools"] = tc.yes_no("with_tools", negated=True)
        tc.generate()
```

Customizing the environment

If your `Makefile` or `configure` scripts need some other environment variable rather than `CPPFLAGS`, `LDFLAGS`, `CXXFLAGS` or `CFLAGS`, you can define them in the `extra_env` attribute before calling the `generate()` method. It is an *Environment* object, and the values defined there have precedence over the ones that the toolchain computes by default:

```
from conan import ConanFile
from conan.tools.gnu import GnuToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = GnuToolchain(self)
        tc.extra_env.define("FOO", "BAR")
        # Prepend a compiler wrapper
        tc.extra_env.prepend("CC", "compile")
        # Do not define RANLIB in the generated script
        tc.extra_env.unset("RANLIB")
        tc.generate()
```

The `GnuToolchain` also sets `CXXFLAGS`, `CFLAGS`, `LDFLAGS` and `CPPFLAGS` reading variables from the `[conf]` section in the profiles. *See the conf reference below*.

Support for Universal Binaries in macOS

Starting in Conan 2.21.0, there's support for building universal binaries on macOS using `GnuToolchain`. To specify multiple architectures for a universal binary in Conan, use the | separator when defining the architecture in the settings. This approach enables passing a list of architectures. For example, running:

```
conan create . --name=mylibrary --version=1.0 -s="arch=armv8|x86_64"
```

will create a universal binary for *mylibrary* containing both `armv8` and `x86_64` architectures, by setting multiple `-arch` flags in the generated toolchain script.

> **Warning:** Attempting to use universal architecture settings on non-Apple platforms will result in an error.
>
> Be aware that this feature is primarily beneficial for building final universal binaries for release purposes. The default Conan behavior of managing one binary per architecture generally provides a more reliable and trouble-free experience. Users should be cautious and not overly rely on this feature for broader use cases.

Reference

class GnuToolchain(*conanfile*, *namespace=None*, *prefix='/'*)

GnuToolchain generator.

Note: it's based on legacy AutotoolsToolchain but with a more modern and usable UX

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **namespace** – This argument avoids collisions when you have multiple toolchain calls in the same recipe. By setting this argument, the *conanbuild.conf* file used to pass information to the build helper will be named as *<namespace>_conanbuild.conf*. The default value is `None` meaning that the name of the generated file is *conanbuild.conf*. This namespace must be also set with the same value in the constructor of the Autotools build helper so that it reads the information from the proper file.
- **prefix** – Folder to use for `--prefix` argument ("/" by default).

yes_no(*option_name*, *default=None*, *negated=False*)

Simple wrapper to return "yes" or "no" depending on whether option_name is evaluated as True or False.

Parameters

- **option_name** – option name.
- **default** – Default value to return.
- **negated** – Negates the option value if True.

Returns

"yes" or "no" depending on whether option_name is True or False.

conf

- `tools.build:cxxflags` list of extra C++ flags that will be used by `CXXFLAGS`.
- `tools.build:cflags` list of extra of pure C flags that will be used by `CFLAGS`.
- `tools.build:sharedlinkflags` list of extra linker flags that will be used by `LDFLAGS`.
- `tools.build:exelinkflags` list of extra linker flags that will be used by `LDFLAGS`.
- `tools.build:rcflags` list of extra RC flags to define `RCFLAGS`
- `tools.build:defines` list of preprocessor definitions that will be used by `CPPFLAGS`.
- `tools.build:linker_scripts` list of linker scripts, each of which will be prefixed with `-T` and added to `LDFLAGS`. Only use this flag with linkers that supports specifying linker scripts with the `-T` flag, such as `ld`, `gold`, and `lld`.
- `tools.build:sysroot` defines the `--sysroot` flag to the compiler.
- `tools.gnu:host_triplet` and `tools.gnu:build_triplet` define the GNU triplets used for the `--host` and `--build` arguments when cross-building, instead of the automatically calculated ones.
- `tools.android:ndk_path` argument for NDK path in case of Android cross-compilation. It is used to set some environment variables like `CC`, `CXX`, `LD`, `STRIP`, `RANLIB`, `AS`, `AR`, `ADDR2LINE`, `NM`, `OBJCOPY`, `OBJDUMP`, `READELF`, and `ELFEDIT` in the *conangnutoolchain.sh|bat* script, as long as they are not already defined in the `buildenv` environment. If they are defined in the `buildenv` environment, the `conangnutoolchain` file will not define them, leaving their definition to the `VirtualBuildEnv` generator.

- `tools.build:compiler_executables` dict-like Python object which specifies the compiler as key and the compiler executable path as value. Those keys will be mapped as follows:
  - `c`: will set `CC` (and `CC_FOR_BUILD` if cross-building) in *conangnutoolchain.sh|bat* script.
  - `cpp`: will set `CXX` (and `CXX_FOR_BUILD` if cross-building) in *conangnutoolchain.sh|bat* script.
  - `rc`: will set `RC` in *conangnutoolchain.sh|bat* script.
  - `cuda`: will set `NVCC` in *conangnutoolchain.sh|bat* script.
  - `fortran`: will set `FC` in *conangnutoolchain.sh|bat* script.
  - `objc`: will set `OBJC` in *conangnutoolchain.sh|bat* script.
  - `objcpp`: will set `OBJCXX` in *conangnutoolchain.sh|bat* script.
  - `asm`: will set `AS` in *conangnutoolchain.sh|bat* script.
  - `nm`: will set `NM` in *conangnutoolchain.sh|bat* script.
  - `ranlib`: will set `RANLIB` in *conangnutoolchain.sh|bat* script.
  - `objdump`: will set `OBJDUMP` in *conangnutoolchain.sh|bat* script.
  - `strip`: will set `STRIP` in *conangnutoolchain.sh|bat* script.

  Keys not listed above are reported with a `risk` warning.
- `tools.gnu:extra_configure_args` list of arguments to extend the `configure_args` attribute of `GnuToolchain`.
- `tools.apple:enable_arc` boolean to add `-fobjc-arc` (or `-fno-objc-arc` if `False`) to `OBJCFLAGS` and `OBJCXXFLAGS`.

Note: flags order of preference: Flags specified in the *tools.build* configuration, such as *cxxflags*, *cflags*, *sharedlinkflags*, *exelinkflags*, and *defines*, will always take precedence over those set by the GnuToolchain attributes.

Autotools

The `Autotools` build helper is a wrapper around the command line invocation of autotools. It will abstract the calls like `./configure` or `make` into Python method calls.

Usage:

```
from conan import ConanFile
from conan.tools.gnu import Autotools

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def build(self):
        autotools = Autotools(self)
        autotools.configure()
        autotools.make()
```

It will read the `conanbuild.conf` file generated by the *AutotoolsToolchain* to know read the arguments for calling the configure and make scripts:

- **configure_args**: Arguments to call the `configure` script.

- **make_args**: Arguments to call the `make` script.

Reference

`class Autotools(conanfile, namespace=None)`

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **namespace** – this argument avoids collisions when you have multiple toolchain calls in the same recipe. By setting this argument, the *conanbuild.conf* file used to pass information to the toolchain will be named as: *<namespace>_conanbuild.conf*. The default value is `None` meaning that the name of the generated file is *conanbuild.conf*. This namespace must be also set with the same value in the constructor of the AutotoolsToolchain so that it reads the information from the proper file.

`configure(build_script_folder=None, args=None)`

Call the configure script.

Parameters

- **args** – List of arguments to use for the `configure` call.
- **build_script_folder** – Subfolder where the *configure* script is located. If not specified conanfile.source_folder is used.

`make(target=None, args=None, makefile=None)`

Call the make program.

Parameters

- **target** – (Optional, Defaulted to `None`): Choose which target to build. This allows building of e.g., docs, shared libraries or install for some AutoTools projects
- **args** – (Optional, Defaulted to `None`): List of arguments to use for the `make` call.
- **makefile** – (Optional, Defaulted to `None`): Allow specifying a custom makefile to use instead of default "Makefile"

`install(args=None, target=None, makefile=None)`

This is just an "alias" of `self.make(target="install")` or `self.make(target="install-strip")`

Parameters

- **args** – (Optional, Defaulted to `None`): List of arguments to use for the `make` call. By default an argument `DESTDIR=unix_path(self.package_folder)` is added to the call if the passed value is `None`. See more information about *tools.microsoft.unix_path() function*
- **target** – (Optional, Defaulted to `None`): Choose which target to install.
- **makefile** – (Optional, Defaulted to `None`): Allow specifying a custom makefile to use instead of default "Makefile"

`autoreconf(build_script_folder=None, args=None)`

Call `autoreconf`

Parameters

- **args** – (Optional, Defaulted to `None`): List of arguments to use for the `autoreconf` call.
- **build_script_folder** – Subfolder where the *configure* script is located. If not specified conanfile.source_folder is used.

conf

The `Autotools` build helper is affected by these `[conf]` variables:

- `tools.gnu:make_program` allows to define which `make` executable is being used. This will default for `mingw32-make` for MinGW builds or to `make` for any other build.
- `tools.build:install_strip` (Since Conan 2.20.0; list values since Conan 2.28.0): when enabled for Autotools, `Autotools.install()` uses the `install-strip` make target instead of `install`. Use `True` so every integration that reads this configuration may strip; use a list such as `["autotools"]` so only the Autotools helper strips. `False` or an unset value keeps the plain `install` target.

A note about relocatable shared libraries in macOS built the Autotools build helper

When building a shared library with Autotools in macOS a section `LC_ID_DYLIB` and another `LC_LOAD_DYLIB` are added to the `.dylib`. These sections store `install_name` information, which is the location of the folder where the library or its dependencies are installed. You can check the install_name of your shared libraries using the otool command:

```
$ otool -l path/to/libMyLib.dylib
...
cmd LC_ID_DYLIB
    cmdsize 48
        name path/to/libMyLib.dylib (offset 24)
time stamp 1 Thu Jan  1 01:00:01 1970
    current version 1.0.0
compatibility version 1.0.0
...
Load command 11
        cmd LC_LOAD_DYLIB
    cmdsize 48
        name path/to/dependency.dylib (offset 24)
time stamp 2 Thu Jan  1 01:00:02 1970
    current version 1.0.0
compatibility version 1.0.0
...
```

Why is this a problem when using Conan?

When using Conan the library will be built in the local cache and this means that this location will point to Conan's local cache folder where the library was installed. This location is where the library tells any other binaries using it where to load it at runtime. This is a problem since you can build the shared library in one machine, then upload it to a server and install it in another machine to use it. In this case, as Autotools behaves by default, you would have a library storing an `install_name` pointing to a folder that does not exist in your current machine so you would get linker errors when building.

How to address this problem in Conan

The only thing Conan can do to make these shared libraries relocatable is to patch the built binaries after installation. To do this, when using the `Autotools` build helper and after running the Makefile's `install()` step, you can use the *fix_apple_shared_install_name()* tool to search for the built `.dylib` files and patch them by running the `install_name_tool` macOS utility, like this:

```
from conan.tools.apple import fix_apple_shared_install_name
class HelloConan(ConanFile):
  ...
  def package(self):
      autotools = Autotools(self)
      autotools.install()
      fix_apple_shared_install_name(self)
```

This will change the value of the `LC_ID_DYLIB` and `LC_LOAD_DYLIB` sections in the `.dylib` file to:

```
$ otool -l path/to/libMyLib.dylib
...
cmd LC_ID_DYLIB
    cmdsize 48
        name @rpath/libMyLib.dylib (offset 24)
time stamp 1 Thu Jan  1 01:00:01 1970
    current version 1.0.0
compatibility version 1.0.0
...
Load command 11
        cmd LC_LOAD_DYLIB
    cmdsize 48
        name @rpath/dependency.dylib (offset 24)
time stamp 2 Thu Jan  1 01:00:02 1970
    current version 1.0.0
compatibility version 1.0.0
```

The `@rpath` special keyword will tell the loader to search a list of paths to find the library. These paths can be defined by the consumer of that library by defining the `LC_RPATH` field. This is done by passing the `-Wl,-rpath -Wl,/path/to/libMyLib.dylib` linker flag when building the consumer of the library. Then if Conan builds an executable that consumes the `libMyLib.dylib` library, it will automatically add the `-Wl,-rpath -Wl,/path/to/libMyLib.dylib` flag so that the library is correctly found when building.

MakeDeps

> **Warning:** This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

`MakeDeps` is the dependencies generator for make. It generates a Makefile file named `conandeps.mk` containing a valid make file syntax with all dependencies listed, including their components.

This generator can be used by name in conanfiles:

Listing 134: conanfile.py

```
class Pkg(ConanFile):
    generators = "MakeDeps"
```

Listing 135: conanfile.txt

```
[generators]
MakeDeps
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.gnu import MakeDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "zlib/1.2.13"

    def generate(self):
        pc = MakeDeps(self)
        pc.generate()
```

Generated files

make format file named `conandeps.mk`, containing a valid makefile file syntax. The `prefix` variable is automatically adjusted to the `package_folder`:

```
CONAN_DEPS = zlib

# zlib/1.2.13
CONAN_NAME_ZLIB = zlib
CONAN_VERSION_ZLIB = 1.2.13
CONAN_REFERENCE_ZLIB = zlib/1.2.13
CONAN_ROOT_ZLIB = /home/conan/.conan2/p/b/zlib273508b343e8c/p
CONAN_INCLUDE_DIRS_ZLIB = $(CONAN_INCLUDE_DIR_FLAG)$(CONAN_ROOT_ZLIB)/include
CONAN_LIB_DIRS_ZLIB = $(CONAN_LIB_DIR_FLAG)$(CONAN_ROOT_ZLIB)/lib
CONAN_BIN_DIRS_ZLIB = $(CONAN_BIN_DIR_FLAG)$(CONAN_ROOT_ZLIB)/bin
CONAN_LIBS_ZLIB = $(CONAN_LIB_FLAG)z

CONAN_INCLUDE_DIRS = $(CONAN_INCLUDE_DIRS_ZLIB)
CONAN_LIB_DIRS = $(CONAN_LIB_DIRS_ZLIB)
CONAN_BIN_DIRS = $(CONAN_BIN_DIRS_ZLIB)
CONAN_LIBS = $(CONAN_LIBS_ZLIB)
```

Properties

Makefile variables will be generated for each property set in *package_info()* of all dependencies and their components. Let's take following receipt:

```
from conan import ConanFile

class MyLib(ConanFile):

    name = "mylib"
    version = "1.0"

    def package_info(self):
        self.cpp_info.set_property("my.prop", "some vale")
        self.cpp_info.components["mycomp"].set_property("comp_prop", "comp_value")
```

The resulting makefile variable assignments would look like this:

```
# mylib/1.0

#[...]
CONAN_PROPERTY_MYLIB_MY_PROP = some value
CONAN_PROPERTY_MYLIB_MYCOMP_COMP_PROP = comp_value
```

When substituting package names, component names and property names into makefile variable names, the names are converted to uppercase and all characters except *A-Z*, *0-9* and _ are replaced with _ (see example above with a dot in the property name). The property value is not modified, it is put to the right side of the variable assignment literally. Any whitespace and special character remain unchanged, no quotation or escaping is applied, because GNU Make is not consistent in escaping spaces and cannot handle whitespaces in path names anyway. Because values with newlines would break the makefile they are skipped and a warning is displayed.

Customization

Flags

By default, the `conandeps.mk` will contain all dependencies listed, including their `cpp_info` information, but will not pass any flags to the compiler.

Thus, the consumer should pass the following flags to the compiler:

- **CONAN_LIB_FLAG**: Add a prefix to all libs variables, e.g. `-l`
- **CONAN_DEFINE_FLAG**: Add a prefix to all defines variables, e.g. `-D`
- **CONAN_SYSTEM_LIB_FLAG**: Add a prefix to all system_libs variables, e.g. `-l`
- **CONAN_INCLUDE_DIR_FLAG**: Add a prefix to all include dirs variables, e.g. `-I`
- **CONAN_LIB_DIR_FLAG**: Add a prefix to all lib dirs variables, e.g. `-L`
- **CONAN_BIN_DIR_FLAG**: Add a prefix to all bin dirs variables, e.g. `-L`

Those flags should be appended as prefixes to flags variables. For example, if the `CONAN_LIB_FLAG` is set to `-l`, the `CONAN_LIBS` variable will be set to `-lz`.

Reference

class MakeDeps(*conanfile*)

Generates a Makefile with the variables needed to build a project with the specified.

Parameters
conanfile – < ConanFile object > The current recipe object. Always use self.

generate() → None

Collects all dependencies and components, then, generating a Makefile

PkgConfigDeps

The PkgConfigDeps is the dependencies generator for pkg-config. Generates pkg-config files named <PKG-NAME>.pc containing a valid pkg-config file syntax.

This generator can be used by name in conanfiles:

Listing 136: conanfile.py

```
class Pkg(ConanFile):
    generators = "PkgConfigDeps"
```

Listing 137: conanfile.txt

```
[generators]
PkgConfigDeps
```

And it can also be fully instantiated in the conanfile generate() method:

```
from conan import ConanFile
from conan.tools.gnu import PkgConfigDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "zlib/1.3.1"

    def generate(self):
        pc = PkgConfigDeps(self)
        pc.generate()
```

Generated files

pkg-config format files named <PKG-NAME>.pc, containing a valid pkg-config file syntax. The prefix variable is automatically adjusted to the package_folder:

```
prefix=/Users/YOUR_USER/.conan/data/zlib/1.3.1/_/_/package/
↪647afeb69d3b0a2d3d316e80b24d38c714cc6900
libdir=${prefix}/lib
includedir=${prefix}/include
bindir=${prefix}/bin
```

(continues on next page)

(continued from previous page)

```
Name: zlib
Description: Conan package: zlib
Version: 1.2.11
Libs: -L"${libdir}" -lz -F Frameworks
Cflags: -I"${includedir}"
```

Customization

Naming

By default, the `*.pc` files will be named following these rules:

- For packages, it uses the package name, e.g., package `zlib/1.3.1` -> `zlib.pc`.
- For components, the package name + hyphen + component name, e.g., `openssl/3.0.0` with `self.cpp_info.components["crypto"]` -> `openssl-crypto.pc`.

You can change that default behavior with the `pkg_config_name` and `pkg_config_aliases` properties. See *Properties section below*.

If a recipe uses **components**, the files generated will be `<[PKG-NAME]-[COMP-NAME]>.pc` with their corresponding flags and require relations.

Additionally, a `<PKG-NAME>.pc` is generated to maintain compatibility for consumers with recipes that start supporting components. This `<PKG-NAME>.pc` file declares all the components of the package as requires while the rest of the fields will be empty, relying on the propagation of flags coming from the components `<[PKG-NAME]-[COMP-NAME]>.pc` files.

If you want to disable the generation of the `<PKG-NAME>.pc` file, you can set the `pkg_config_name` property to the `none` string value:

```
def package_info(self):
    self.cpp_info.set_property("pkg_config_name", "none")
    self.cpp_info.components["crypto"].set_property("pkg_config_name", "mylib-crypto")
```

This will generate only the `mylib-foo.pc` file, but not the `mylib.pc` one. This can only be done at the global `cpp_info` level, not at component level.

Reference

class PkgConfigDeps(*conanfile*)

generate()

Save all the **.pc* files

set_property(*dep*, *prop*, *value*)

Using this method you can overwrite the *property* values set by the Conan recipes from the consumer. This can be done for *pkg_config_name*, *pkg_config_aliases* and *pkg_config_custom_content* properties.

Parameters

- **dep** – Name of the dependency to set the *property*. For components use the syntax: `dep_name::component_name`.
- **prop** – Name of the *property*.

- **value** – Value of the property. Use `None` to invalidate any value set by the upstream recipe.

Attributes

build_context_activated

When you have a **build-require**, by default, the `*.pc` files are not generated. But you can activate it using the **build_context_activated** attribute:

```
tool_requires = ["my_tool/0.0.1"]
def generate(self):
    pc = PkgConfigDeps(self)
    # generate the *.pc file for the tool require
    pc.build_context_activated = ["my_tool"]
    pc.generate()
```

build_context_folder

New since Conan 2.2.0

When you have the same package as a **build-require** and as a **regular require** it will cause a conflict in the generator because the file names of the `*.pc` files will collide as well as the names, requires names, etc.

For example, this is a typical situation with some requirements (capnproto, protobuf...) that contain a tool used to generate source code at build time (so it is a **build_require**), but also providing a library to link to the final application, so you also have a **regular require**. Solving this conflict is specially important when we are cross-building because the tool (that will run in the building machine) belongs to a different binary package than the library, that will "run" in the host machine.

You can use the `build_context_folder` attribute to specify a folder to save the **.pc* files created by all those build requirements listed in the `build_context_activated` one:

```
tool_requires = ["my_tool/0.0.1"]
requires = ["my_tool/0.0.1"]
def generate(self):
    pc = PkgConfigDeps(self)
    # generate the *.pc file for the tool require
    pc.build_context_activated = ["my_tool"]
    # save all the *.pc files coming from the "my_tool" build context and its␣
↪requirements
    pc.build_context_folder = "build"  # [generators_folder]/build/
    pc.generate()
```

build_context_suffix

DEPRECATED: use build_context_folder attribute instead

Same concept as the quoted `build_context_folder` attribute above, but this is meant to specify a suffix for a requirement, so the files/requires/names of the requirement in the build context (tool require) will be renamed:

```
tool_requires = ["my_tool/0.0.1"]
requires = ["my_tool/0.0.1"]
def generate(self):
    pc = PkgConfigDeps(self)
    # generate the *.pc file for the tool require
    pc.build_context_activated = ["my_tool"]
    # disambiguate the files, requires, names, etc
    pc.build_context_suffix = {"my_tool": "_BUILD"}
    pc.generate()
```

Important: This attribute should not be used simultaneously with the `build_context_folder` attribute.

Properties

The following properties affect the `PkgConfigDeps` generator:

- **pkg_config_name** property will define the name of the generated `*.pc` file (`xxxxx.pc`), or the string `none` to disable the generation of the global `*.pc` file for the package.
- **pkg_config_aliases** property sets some aliases of any package/component name for *pkg_config* generator. This property only accepts list-like Python objects.
- **pkg_config_custom_content** property will add user defined content to the *.pc* files created by this generator as freeform variables. That content can be a string or a dict-like Python object. Notice that the variables declared here will overwrite those ones already defined by Conan. Click here for more information about the type of variables in a `*.pc` file.
- **system_package_version**: property sets a custom version to be used in the `Version` field belonging to the created `*.pc` file for the package.
- **component_version** property sets a custom version to be used in the `Version` field belonging to the created `*.pc` file for that component (takes precedence over the **system_package_version** property).

These properties can be defined at global `cpp_info` level or at component level.

Example:

```
def package_info(self):
    custom_content = {"datadir": "${prefix}/share"}  # or "datadir=${prefix}/share"
    self.cpp_info.set_property("pkg_config_custom_content", custom_content)
    self.cpp_info.set_property("pkg_config_name", "myname")
    self.cpp_info.components["mycomponent"].set_property("pkg_config_name",
↪"componentname")
    self.cpp_info.components["mycomponent"].set_property("pkg_config_aliases", ["alias1",
↪ "alias2"])
    self.cpp_info.components["mycomponent"].set_property("component_version", "1.14.12")
```

PkgConfig

This tool can execute pkg_config executable to extract information from existing .pc files. This can be useful for example to create a "system" package recipe over some system installed library, as a way to automatically extract the .pc information from the system. Or if some proprietary package has a build system that only outputs .pc files.

Usage:

Read a pc file and access the information:

```
pkg_config = PkgConfig(conanfile, "libastral", pkg_config_path=<somedir>)

print(pkg_config.provides) # something like"libastral = 6.6.6"
print(pkg_config.version) # something like"6.6.6"
print(pkg_config.includedirs) # something like['/usr/local/include/libastral']
print(pkg_config.defines) # something like['_USE_LIBASTRAL']
print(pkg_config.libs) # something like['astral', 'm']
print(pkg_config.libdirs) # something like['/usr/local/lib/libastral']
print(pkg_config.linkflags) # something like['-Wl,--whole-archive']
print(pkg_config.variables['prefix']) # something like'/usr/local'
```

Use the pc file information to fill a cpp_info object:

```
def package_info(self):
    pkg_config = PkgConfig(conanfile, "libastral", pkg_config_path=tmp_dir)
    pkg_config.fill_cpp_info(self.cpp_info, is_system=False, system_libs=["m", "rt"])
```

If pkg_config_path is not provided, PkgConfig relies on the PKG_CONFIG_PATH environment variable to locate the .pc files. This variable can be defined in the profile [buildenv] section, a common approach for cross-compilation with a sysroot that already provides .pc files. See *Using PkgConfig.fill_cpp_info with PKG_CONFIG_PATH from a profile* for a full example.

Reference

class PkgConfig(*conanfile*, *library*, *pkg_config_path=None*)

Parameters

- **conanfile** – The current recipe object. Always use self.
- **library** – The library which .pc file is to be parsed. It must exist in the pkg_config path.
- **pkg_config_path** – If defined it will be prepended to PKG_CONFIG_PATH environment variable, so the execution finds the required files.

fill_cpp_info(*cpp_info*, *is_system=True*, *system_libs=None*)

Method to fill a cpp_info object from the PkgConfig configuration

Parameters

- **cpp_info** – Can be the global one (self.cpp_info) or a component one (self.components["foo"].cpp_info).
- **is_system** – If True, all detected libraries will be assigned to cpp_info.system_libs, and none to cpp_info.libs.
- **system_libs** – If True, all detected libraries will be assigned to cpp_info.system_libs, and none to cpp_info.libs.

conf

This helper will listen to `tools.gnu:pkg_config` from the *global.conf* to define the `pkg_config` executable name or full path. It will by default it is `pkg-config`.

9.10.9 conan.tools.google

Bazel

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `Bazel` build helper is a wrapper around the command line invocation of bazel. It will abstract the calls like `bazel <rcpaths> build <configs> <targets>` into Python method calls.

The helper is intended to be used in the *conanfile.py* `build()` method, to call Bazel commands automatically when a package is being built directly by Conan (create, install)

```
from conan import ConanFile
from conan.tools.google import Bazel

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def build(self):
        bz = Bazel(self)
        bz.build(target="//main:hello-world")
```

Reference

class Bazel(*conanfile*)

Parameters
conanfile – < `ConanFile object` > The current recipe object. Always use `self`.

build(*args=None*, *target='//...'*, *clean=True*)

Runs "bazel <rcpaths> build <configs> <args> <targets>" command where:

- `rcpaths`: adds `--bazelrc=xxxx` per rc-file path. It listens to `BazelToolchain` (`--bazelrc=conan_bzl.rc`), and `tools.google.bazel:bazelrc_path` conf.
- `configs`: adds `--config=xxxx` per bazel-build configuration. It listens to `BazelToolchain` (`--config=conan-config`), and `tools.google.bazel:configs` conf.
- `args`: they are any extra arguments to add to the `bazel build` execution.
- `targets`: all the target labels.

Parameters

- **target** – It is the target label. By default, it's "//..." which runs all the targets.
- **args** – list of extra arguments to pass to the CLI.

- **clean** – boolean that indicates to run a "bazel clean" before running the "bazel build". Notice that this is important to ensure a fresh bazel cache every

test(*target=None*)

Runs "bazel test <targets>" command.

Properties

The following properties affect the `Bazel` build helper:

- `tools.build:skip_test=<bool>` (boolean) if `True`, it runs the `bazel test <target>`.

conf

`Bazel` is affected by these *[conf]* variables:

- `tools.google.bazel:bazelrc_path`: List of paths to other bazelrc files to be used as **`bazel --bazelrc=rcpath1 ... build`**.
- `tools.google.bazel:configs`: List of Bazel configurations to be used as **`bazel build --config=config1 ...`**.

See also:

- *Build a simple Bazel project using Conan*
- *Build a simple Bazel 7.x project using Conan*

BazelDeps

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `BazelDeps` is the dependencies generator for Bazel. Generates a *<REPOSITORY>/BUILD.bazel* file per dependency, where the *<REPOSITORY>/* folder is the Conan recipe reference name by default, e.g., *mypkg/BUILD.bazel*. Apart from that, it also generates Bazel 6.x compatible file like *dependencies.bzl*, Bazel >= 7.1 compatible files like *conan_deps_module_extension.bzl* and *conan_deps_repo_rules.bzl*, and Bazel >= 7.2 / 9.x compatible files like *conan_deps.MODULE.bazel* and *BUILD.rules_cc.bazel* (with explicit `rules_cc` `load()` statements required by Bazel 9.x). All of them contain the logic to load all your Conan dependencies through your *WORKSPACE* | *MODULE.bazel*.

Warning: Since Conan 2.34, the Bazel 6.x support (the *dependencies.bzl* file loaded from a *WORKSPACE*) is deprecated and will be removed in a future release. `BazelDeps` still generates *dependencies.bzl*, but it now warns about the deprecation when there are dependencies. Use Bazel >= 7.2 and add `include("//conan:conan_deps.MODULE.bazel")` to your *MODULE.bazel* file instead.

The `BazelDeps` generator can be used by name in conanfiles:

Listing 138: conanfile.py

```
class Pkg(ConanFile):
    generators = "BazelDeps"
```

Listing 139: conanfile.txt

```
[generators]
BazelDeps
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.google import BazelDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "zlib/1.3.1"

    def generate(self):
        bz = BazelDeps(self)
        bz.generate()
```

Generated files

When the `BazelDeps` generator is used, every invocation of `conan install` will generate several bazel files. For the *conanfile.py* above, for example:

```
$ conan install .
.
├── BUILD.bazel
├── conanfile.py
├── conan_deps.MODULE.bazel
├── conan_deps_module_extension.bzl
├── conan_deps_module_extension_rules_cc.bzl
├── conan_deps_repo_rules.bzl
├── dependencies.bzl
└── zlib
    ├── BUILD.bazel
    └── BUILD.rules_cc.bazel
```

Every **`conan install`** generates these files:

- *BUILD.bazel*: An empty file aimed to be alongside the *dependencies.bzl* one. More information here.
- *zlib/BUILD.bazel*: contains all the targets that you can load from any of your *BUILD* files. More information in *Customization*.
- *zlib/BUILD.rules_cc.bazel*: same targets as *BUILD.bazel*, but with `rules_cc` `load()` statements (Bazel 9.x compatible).
- *dependencies.bzl*: (Bazel 6.x compatible, deprecated since Conan 2.34) this file tells your Bazel *WORKSPACE* how to load the dependencies.
- *conan_deps_module_extension.bzl*: (since Conan 2.4.0)(Bazel >= 7.1 compatible) This file is used to load each dependency as repository.
- *conan_deps_module_extension_rules_cc.bzl*: (Bazel >= 7.2 / 9.x compatible) module extension that references the *BUILD.rules_cc.bazel* dependency files.

- *conan_deps.MODULE.bazel*: (Bazel >= 7.2 / 9.x compatible) generated snippet to include in your *MODULE.bazel* file. Declares the `rules_cc` dependency and loads the module extension above.
- *conan_deps_repo_rules.bzl*: (since Conan 2.4.0)(Bazel >= 7.1 compatible) The rule provided by this file is used to create a repository. It is not intended to be used by consumers but by *conan_deps_module_extension.bzl*.

Let's check the content of the files created:

Bazel 6.x compatible (deprecated)

Listing 140: dependencies.bzl

```
# This Bazel module should be loaded by your WORKSPACE file.
# Add these lines to your WORKSPACE one (assuming that you're using the "bazel_layout"):
# load("@//conan:dependencies.bzl", "load_conan_dependencies")
# load_conan_dependencies()

def load_conan_dependencies():
    native.new_local_repository(
        name="zlib",
        path="/path/to/conan/package/folder/",
        build_file="/your/current/working/directory/zlib/BUILD.bazel",
    )
```

Bazel >= 7.1 compatible

Listing 141: conan_deps_repo_rules.bzl

```
# This bazel repository rule is used to load Conan dependencies into the Bazel workspace.
# It's used by a generated module file that provides information about the conan packages.
# Each conan package is loaded into a bazel repository rule, with having the name of the
# package. The whole method is based on symlinks to not copy the whole package into the
# Bazel workspace, which is expensive.
def _conan_dependency_repo(rctx):
    package_path = rctx.workspace_root.get_child(rctx.attr.package_path)

    child_packages = package_path.readdir()
    for child in child_packages:
        rctx.symlink(child, child.basename)

    rctx.symlink(rctx.attr.build_file_path, "BUILD.bazel")

conan_dependency_repo = repository_rule(
    implementation = _conan_dependency_repo,
    attrs = {
        "package_path": attr.string(
            mandatory = True,
            doc = "The path to the Conan package in conan cache.",
        ),
        "build_file_path": attr.string(
            mandatory = True,
            doc = "The path to the BUILD file.",
        ),
    },
)
```

Listing 142: conan_deps_module_extension.bzl

```
# This module provides a repo for each requires-dependency in your conanfile.
# It's generated by the BazelDeps, and should be used in your Module.bazel file.
load(":conan_deps_repo_rules.bzl", "conan_dependency_repo")

def _load_dependencies_impl(mctx):
    conan_dependency_repo(
        name = "zlib",
        package_path = "/path/to/conan/package/folder/",
        build_file_path = "/your/current/working/directory/zlib/BUILD.bazel",
    )

    return mctx.extension_metadata(
        # It will only warn you if any direct
        # dependency is not imported by the 'use_repo' or even it is imported
        # but not created. Notice that root_module_direct_dev_deps can not be None as we
        # are giving 'all' value to root_module_direct_deps.
        # Fix the 'use_repo' calls by running 'bazel mod tidy'
        root_module_direct_deps = 'all',
        root_module_direct_dev_deps = [],

        # Prevent writing function content to lockfiles:
        # - https://bazel.build/rules/lib/builtins/module_ctx#extension_metadata
        # Important for remote build. Actually it's not reproducible, as local paths will
        # be different on different machines. But we assume that conan works correctly␣
↪here.
        # IMPORTANT: Not compatible with bazel < 7.1
        reproducible = True,
    )

conan_extension = module_extension(
    implementation = _load_dependenies_impl,
    os_dependent = True,
    arch_dependent = True,
)
```

Given the examples above, and imagining that your *WORKSPACE* | *MODULE.bazel* is at the same directory, you would have to add these lines in there:

Bazel 6.x compatible (deprecated)

Listing 143: WORKSPACE

```
load("@//:dependencies.bzl", "load_conan_dependencies")
load_conan_dependencies()
```

Bazel >= 7.1 compatible

Listing 144: MODULE.bazel

```
load_conan_dependencies = use_extension("//:conan_deps_module_extension.bzl", "conan_
↪extension")
# use_repo(load_conan_dependencies, "dep1", "dep2", ..., "depN")
```

(continues on next page)

(continued from previous page)

```
use_repo(load_conan_dependencies, "zlib")
```

Bazel >= 7.2 compatible (recommended for Bazel 9.x)

Listing 145: MODULE.bazel

```
include("//:conan_deps.MODULE.bazel")
```

Important: Bazel 9.x requires C++ rules to be loaded from `rules_cc`. Include the generated *conan_deps.MODULE.bazel* snippet in your *MODULE.bazel* file — it declares the `rules_cc` dependency and uses the *BUILD.rules_cc.bazel* dependency files.

As you can observe, the *zlib/BUILD.bazel* defines these global targets:

Listing 146: zlib/BUILD.bazel

```
# Components precompiled libs
# Root package precompiled libs
cc_import(
    name = "z_precompiled",
    static_library = "lib/libz.a",
)

# Components libraries declaration
# Package library declaration
cc_library(
    name = "zlib",
    hdrs = glob([
        "include/**",
    ]),
    includes = [
        "include",
    ],
    visibility = ["//visibility:public"],
    deps = [
        ":z_precompiled",
    ],
)

# Filegroup library declaration
filegroup(
    name = "zlib_binaries",
    srcs = glob([
        "bin/**",
    ]),
    visibility = ["//visibility:public"],
)
```

- `zlib`: bazel library target. The label used to depend on it would be `@zlib//:zlib`.
- `zlib_binaries`: bazel filegroup target. The label used to depend on it would be `@zlib//:zlib_binaries`.

Note: For Bazel 9.x, the equivalent *BUILD.rules_cc.bazel* file includes `load("@rules_cc//cc:cc_import.bzl", "cc_import")` and `load("@rules_cc//cc:cc_library.bzl", "cc_library")` at the top.

You can put all the files generated by `BazelDeps` into another folder using the `bazel_layout`:

Listing 147: conanfile.py

```
from conan import ConanFile
from conan.tools.google import BazelDeps, bazel_layout

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "zlib/1.3.1"

    def layout(self):
        bazel_layout(self)

    def generate(self):
        bz = BazelDeps(self)
        bz.generate()
```

Running again the **conan install** command, we now get this structure:

```
$ conan install .
.
├── conan
│   ├── BUILD.bazel
│   ├── conan_deps.MODULE.bazel
│   ├── conan_deps_module_extension.bzl
│   ├── conan_deps_module_extension_rules_cc.bzl
│   ├── conan_deps_repo_rules.bzl
│   ├── dependencies.bzl
│   └── zlib
│       ├── BUILD.bazel
│       └── BUILD.rules_cc.bazel
└── conanfile.py
```

Now your Conan-bazel files were generated in the *conan/* folder, and your MODULE.bazel will look like:

Listing 148: MODULE.bazel

```
include("//conan:conan_deps.MODULE.bazel")
```

Or, with the deprecated Bazel 6.x support, your WORKSPACE:

Listing 149: WORKSPACE

```
load("@//conan:dependencies.bzl", "load_conan_dependencies")
load_conan_dependencies()
```

Customization

Naming

The <REPOSITORY>/BUILD.bazel file contains all the targets declared by the dependency. Both the <REPOSITORY>/ folder and the targets declared in there will be named following these rules by default:

- **For packages, it uses the package name as folder/target name, e.g., package `zlib/1.3.1` will have:**
  - Folder: `zlib/BUILD.bazel`.
  - Global target: `zlib`.
  - How it can be consumed: `@zlib//:zlib`.
- **For components, the package name + hyphen + component name, e.g., package `openssl/3.1.4` will have:**
  - Folder: `openssl/BUILD.bazel`.
  - Global target: `openssl`.
  - Components targets: `openssl-ssl`, and `openssl-crypto`.
  - **How it can be consumed:**
    - `@openssl//:openssl` (global one which includes all the components)
    - `@openssl//:openssl-ssl` (component one)
    - `@openssl//:openssl-crypto` (component one)

You can change that default behavior with the `bazel_target_name` and the `bazel_repository_name` properties. See *Properties section below*.

Reference

class BazelDeps(*conanfile*)

Parameters
conanfile – < ConanFile object > The current recipe object. Always use `self`.

build_context_activated

Activates the build context for the specified Conan package names.

generate()

Generates all the targets <DEP>/BUILD.bazel files, a dependencies.bzl (for bazel<7), a conan_deps_repo_rules.bzl and a conan_deps_module_extension.bzl file (for bazel>=7.1) one in the build folder.

In case of bazel < 7, it's important to highlight that the `dependencies.bzl` file should be loaded by your WORKSPACE Bazel file:

```
load("@//[BUILD_FOLDER]:dependencies.bzl", "load_conan_dependencies")
load_conan_dependencies()
```

In case of Bazel >= 7.2, include the generated module snippet in your MODULE.bazel file:

```
include("//conan:conan_deps.MODULE.bazel")
```

build_context_activated

When you have a **build-requirement**, by default, the Bazel files are not generated. But you can activate it using the **build_context_activated** attribute:

```
def build_requirements(self):
    self.tool_requires("my_tool/0.0.1")

def layout(self):
    bazel_layout(self)

def generate(self):
    bz = BazelDeps(self)
    # generate the build-mytool/BUILD.bazel file for the tool require
    bz.build_context_activated = ["my_tool"]
    bz.generate()
```

Running the **conan install** command, the structure created is as follows:

```
$ conan install . -pr:b default
.
├── conan
│   ├── BUILD.bazel
│   ├── build-my_tool
│   │   ├── BUILD.bazel
│   │   └── BUILD.rules_cc.bazel
│   ├── conan_deps.MODULE.bazel
│   ├── conan_deps_module_extension.bzl
│   ├── conan_deps_module_extension_rules_cc.bzl
│   ├── conan_deps_repo_rules.bzl
│   └── dependencies.bzl
└── conanfile.py
```

Notice that *my_tool* Bazel folder is prefixed with `build-` which indicates that it's being used in the build context.

Properties

The following properties affect the `BazelDeps` generator:

- **bazel_target_name** property will define the name of the target declared in the `<REPOSITORY>/BUILD.bazel`. This property can be defined at both global and component `cpp_info` level.
- **bazel_repository_name** property will define the name of the folder where the dependency *BUILD.bazel* will be allocated. This property can only be defined at global `cpp_info` level.

Example:

```
def package_info(self):
    self.cpp_info.set_property("bazel_target_name", "my_target")
    self.cpp_info.set_property("bazel_repository_name", "my_repo")
    self.cpp_info.components["mycomponent"].set_property("bazel_target_name", "component_
↪name")
```

See also:

- *Build a simple Bazel project using Conan*
- *Build a simple Bazel 7.x project using Conan*

BazelToolchain

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `BazelToolchain` is the toolchain generator for Bazel. It will generate a `conan_bzl.rc` file that contains a build configuration `conan-config` to inject all the parameters into the **`bazel build`** command.

The `BazelToolchain` generator can be used by name in conanfiles:

Listing 150: conanfile.py

```
class Pkg(ConanFile):
    generators = "BazelToolchain"
```

Listing 151: conanfile.txt

```
[generators]
BazelToolchain
```

And it can also be fully instantiated in the conanfile `generate()` method:

Listing 152: conanfile.py

```
from conan import ConanFile
from conan.tools.google import BazelToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = BazelToolchain(self)
        tc.generate()
```

Generated files

After running **conan install** command, the `BazelToolchain` generates the *conan_bzl.rc* file that contains Bazel build parameters (it will depend on your current Conan settings and options from your *default* profile):

Listing 153: conan_bzl.rc

```
# Automatic bazelrc file created by Conan

build:conan-config --cxxopt=-std=gnu++17

build:conan-config --dynamic_mode=off
build:conan-config --compilation_mode=opt
```

The *Bazel build helper* will use that `conan_bzl.rc` file to perform a call using this configuration. The outcoming command will look like this **bazel --bazelrc=/path/to/conan_bzl.rc build --config=conan-config <target>**.

Reference

class BazelToolchain(*conanfile*)

Parameters
conanfile – < `ConanFile object` > The current recipe object. Always use `self`.

force_pic

Boolean used to add –force_pic=True. Depends on self.options.shared and self.options.fPIC values

dynamic_mode

String used to add –dynamic_mode=[“fully”|”off”]. None if self.options.shared is undefined.

cppstd

String used to add –cxxopt=[FLAG]. Depends on your settings.

copt

List of flags used to add –copt=flag1 … –copt=flagN

conlyopt

List of flags used to add –conlyopt=flag1 … –conlyopt=flagN

`cxxopt`
: List of flags used to add –cxxopt=flag1 … –cxxopt=flagN

`linkopt`
: List of flags used to add –linkopt=flag1 … –linkopt=flagN

`compilation_mode`
: String used to add –compilation_mode=[“opt”|”dbg”]. Depends on self.settings.build_type

`compiler`
: String used to add –compiler=xxxx.

`cpu`
: String used to add –cpu=xxxxx. At the moment, it’s only added if cross-building.

`crosstool_top`
: String used to add –crosstool_top.

`generate()`
: Creates a `conan_bzl.rc` file with some bazel-build configuration. This last mentioned is put as `conan-config`.

conf

`BazelToolchain` is affected by these *[conf]* variables:

- `tools.build:cxxflags` list of extra C++ flags that will be used by `cxxopt`.
- `tools.build:cflags` list of extra of pure C flags that will be used by `conlyopt`.
- `tools.build:sharedlinkflags` list of extra linker flags that will be used by `linkopt`.
- `tools.build:exelinkflags` list of extra linker flags that will be used by `linkopt`.
- `tools.build:linker_scripts` list of linker scripts, each of which will be prefixed with `-T` and added to `linkopt`.

See also:

- *Build a simple Bazel project using Conan*
- *Build a simple Bazel 7.x project using Conan*

9.10.10 conan.tools.intel

IntelCC

This tool helps you to manage the Intel oneAPI DPC++/C++ and Classic ecosystem in Conan.

> **Warning:** This generator is **experimental** and subject to breaking changes.

> **Warning:** macOS is not supported for the Intel oneAPI DPC++/C++ (icx/icpx or dpcpp) compilers. For macOS or Xcode support, you’ll have to use the Intel C++ Classic Compiler.

Note: Remember, you need to have installed previously the Intel oneAPI software.

This generator creates a `conanintelsetvars.sh|bat` wrapping the Intel script `setvars.sh|bat` that sets the Intel oneAPI environment variables needed. That script is the first step to start using the Intel compilers because it's setting some important variables in your local environment.

When your profile specifies `compiler=intel-cc`, Conan automatically invokes the `IntelCC` generator. You don't need to declare it explicitly in your conanfile.

Note: If you explicitly set `tools.intel:installation_path=""` configuration (empty string), Conan will **not generate** the `conanintelsetvars` script. In this case, you are expected to have already activated the Intel oneAPI environment manually.

At first, ensure you are using a *profile* like this one:

Linux profile

Windows profile

```
[settings]
os=Linux
arch=x86_64
compiler=intel-cc
compiler.mode=icx
compiler.version=2026.0
compiler.libcxx=libstdc++
compiler.cppstd=gnu17
build_type=Release

[conf]
tools.intel:installation_path=/opt/intel/oneapi
```

```
[settings]
os=Windows
arch=x86_64
compiler=intel-cc
compiler.mode=icx
compiler.version=2026.0
compiler.runtime=dynamic
compiler.cppstd=17
build_type=Release

[conf]
tools.intel:installation_path=C:\Program Files (x86)\Intel\oneAPI
```

Compiler mode

Conan selects the appropriate compiler executables based on the `compiler.mode` setting:

- **icx mode**: Uses `icx`/`icpx` on Linux, or `icx-cl` on Windows (for MSVC compatibility)
- **dpcpp mode**: Uses `icx`/`dpcpp`
- **classic mode**: Uses `icc`/`icpc`

You typically don't need to specify `tools.build:compiler_executables` in your profile. If you do specify it, it will take precedence over the auto-detected values.

SYCL support

Warning: The `dpcpp` compiler is **deprecated** by Intel. The recommended way to compile SYCL code is to use `icpx` with the `-fsycl` flag.

To enable SYCL compilation, use the `compiler.mode=icx` setting and add the `-fsycl` flag via the build configuration:

Linux profile

Windows profile

```
[settings]
os=Linux
arch=x86_64
compiler=intel-cc
compiler.mode=icx
compiler.version=2026.0
compiler.libcxx=libstdc++
compiler.cppstd=gnu17
build_type=Release

[conf]
tools.build:cxxflags=["-fsycl"]
tools.build:exelinkflags=["-fsycl"]
tools.build:sharedlinkflags=["-fsycl"]
tools.intel:installation_path=/opt/intel/oneapi
```

```
[settings]
os=Windows
arch=x86_64
compiler=intel-cc
compiler.mode=icx
compiler.version=2026.0
compiler.runtime=dynamic
compiler.cppstd=17
build_type=Release

[conf]
tools.build:cxxflags=["-fsycl"]
tools.build:exelinkflags=["-fsycl"]
```

(continues on next page)

(continued from previous page)

```
tools.build:sharedlinkflags=["-fsycl"]
tools.intel:installation_path=C:\Program Files (x86)\Intel\oneAPI
```

Custom configurations

Apply different installation paths and command arguments simply by changing the `[conf]` entries. For instance:

Listing 154: intelprofile

```
[settings]
...
compiler=intel-cc
compiler.mode=icx
compiler.version=2026.0
compiler.libcxx=libstdc++
build_type=Release

[conf]
tools.intel:installation_path=/opt/intel/oneapi
tools.intel:setvars_args=--config="full/path/to/your/config.txt" --force
```

Run again a **`conan install . -pr intelprofile`**, then the `conanintelsetvars.sh` script (if we are using Linux OS) will contain something like:

Listing 155: conanintelsetvars.sh

```
. "/opt/intel/oneapi/setvars.sh" --config="full/path/to/your/config.txt" --force
```

The `IntelCC` generator can also be fully instantiated in the conanfile `generate()` method:

Listing 156: *conanfile.py*

```
from conan import ConanFile
from conan.tools.intel import IntelCC

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        intelcc = IntelCC(self)
        intelcc.generate()
```

Reference

class IntelCC(*conanfile*)

Class that manages Intel oneAPI DPC++/C++/Classic Compilers vars generation

arch

arch setting

property ms_toolset

Get Microsoft Visual Studio Toolset depending on the mode selected

generate(*scope=None*)

Generate the Conan Intel file to be loaded in build and run environments.

Parameters
scope – The scope(s) for which to generate. Can be a string or list of strings. Defaults to ["build", "run"] to enable both compilation and runtime.

property installation_path

Get the Intel oneAPI installation root path

property command

The Intel oneAPI DPC++/C++ Compiler includes environment configuration scripts to configure your build and development environment variables:

- On Linux, the file is a shell script called setvars.sh.
- On Windows, the file is a batch file called setvars.bat.
- Linux -> >> `. /<install-dir>/setvars.sh <arg1> <arg2> ... <argn><arg1> <arg2> ... <argn>` The compiler environment script file accepts an optional target architecture argument <arg>: - intel64: Generate code and use libraries for Intel 64 architecture-based targets. - ia32: Generate code and use libraries for IA-32 architecture-based targets.
- Windows -> >> `call <install-dir>\setvars.bat [<arg1>] [<arg2>]` Where <arg1> is optional and can be one of the following: - intel64: Generate code and use libraries for Intel 64 architecture (host and target). - ia32: Generate code and use libraries for IA-32 architecture (host and target).

  With the dpcpp compiler, <arg1> is intel64 by default.

  The <arg2> is optional. If specified, it is one of the following: - vs2019: Microsoft Visual Studio* 2019 - vs2017: Microsoft Visual Studio 2017

Returns
str setvars.sh|bat command to be run

conf

`IntelCC` uses these *configuration entries*:

- `tools.intel:installation_path`: **(required)** argument to tell Conan the installation path, if it's not defined, Conan will try to find it out automatically. If it is explicitly set to the empty string (""), Conan will **skip the generation** of the `conanintelsetvars` script, assuming the Intel environment has already been activated manually.
- `tools.intel:setvars_args`: **(optional)** it is used to pass whatever we want as arguments to our *setvars.sh|bat* file. You can check out all the possible ones from the Intel official documentation.

9.10.11 conan.tools.layout

Predefined layouts

There are some pre-defined common *layouts*, ready to be simply used in recipes:

- `cmake_layout()`: *a layout for a typical CMake project*
- `vs_layout()`: a layout for a typical Visual Studio project
- `basic_layout()`: *a very basic layout for a generic project*

The pre-defined layouts define the Conanfile `.folders` and `.cpp` attributes with typical values. To check which values are set by these pre-defined layouts please check the reference for the *layout()* method. For example in the `cmake_layout()` the source folder is set to `"."`, meaning that Conan will expect the sources in the same directory where the conanfile is (most likely the project root, where a `CMakeLists.txt` file will be typically found). If you have a different folder where the `CMakeLists.txt` is located, you can use the `src_folder` argument:

```
from conan.tools.cmake import cmake_layout

def layout(self):
    cmake_layout(self, src_folder="mysrcfolder")
```

Even if this pre-defined layout doesn't suit your specific projects layout, checking how they implement their logic shows how you could implement your own logic (and probably put it in a common `python_require` if you are going to use it in multiple packages).

To learn more about the layouts and how to use them while developing packages, please check the Conan package layout *tutorial*.

basic_layout

Usage:

```
from conan.tools.layout import basic_layout

def layout(self):
    basic_layout(self)
```

The current layout implementation is very simple, basically sets a different build folder for different build_types and sets the generators output folder inside the build folder. This way we avoid to clutter our project while working locally. If you prefer, you can define the *build_folder* to take control over the destination folder, so the temporary build files do not pollute the source tree.

```
def basic_layout(conanfile, src_folder=".", build_folder=None):
    subproject = conanfile.folders.subproject

    conanfile.folders.source = src_folder if not subproject else os.path.join(subproject,
↪ src_folder)
    if build_folder:
        conanfile.folders.build = build_folder if not subproject else os.path.
↪join(subproject, build_folder)
    else:
        conanfile.folders.build = "build" if not subproject else os.path.join(subproject,
↪ "build")
        if conanfile.settings.get_safe("build_type"):
            conanfile.folders.build += "-{}".format(str(conanfile.settings.build_type).
↪lower())
    conanfile.folders.generators = os.path.join(conanfile.folders.build, "conan")
    conanfile.cpp.build.bindirs = ["."]
    conanfile.cpp.build.libdirs = ["."]

    if not conanfile.cpp.source.includedirs:
        conanfile.cpp.source.includedirs = ["include"]
```

9.10.12 conan.tools.meson

MesonToolchain

Important: This class will generate files that are only compatible with Meson versions >= 0.55.0

The `MesonToolchain` is the toolchain generator for Meson and it can be used in the `generate()` method as follows:

```
from conan import ConanFile
from conan.tools.meson import MesonToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"
    options = {"shared": [True, False]}
    default_options = {"shared": False}

    def generate(self):
        tc = MesonToolchain(self)
        tc.preprocessor_definitions["MYDEFINE"] = "MYDEF_VALUE"
        tc.generate()
```

Important: When your recipe has dependencies `MesonToolchain` only works with the `PkgConfigDeps` generator.

Please, do not use other generators, as they can have overlapping definitions that can conflict.

Generated files

The `MesonToolchain` generates the following files after a **`conan install`** (or when building the package in the cache) with the information provided in the `generate()` method as well as information translated from the current `settings`, `conf`, etc.:

- *conan_meson_native.ini*: if doing a native build.
- *conan_meson_cross.ini*: if doing a cross-build (*conan.tools.build*).

conan_meson_native.ini

This file contains the definitions of all the Meson properties related to the Conan options and settings for the current package, platform, etc. This includes but is not limited to the following:

- Detection of `default_library` from Conan settings.
  - Based on existence/value of an option named `shared`.
- Detection of `buildtype` from Conan settings.
- Definition of the C++ standard as necessary.
- The Visual Studio runtime (`b_vscrt`), obtained from Conan input settings.

conan_meson_cross.ini

This file contains the same information as the previous *conan_meson_native.ini*, but with additional information to describe host, target, and build machines (such as the processor architecture).

Check out the meson documentation for more details on native and cross files:

- Machine files
- Native environments
- Cross compilation

Default directories

`MesonToolchain` manages some of the directories used by Meson. These are variables declared under the `[project options]` section of the files *conan_meson_native.ini* and *conan_meson_cross.ini* (see more information about Meson directories):

`bindir`: value coming from `self.cpp.package.bindirs`. Defaulted to None. `sbindir`: value coming from `self.cpp.package.bindirs`. Defaulted to None. `libexecdir`: value coming from `self.cpp.package.bindirs`. Defaulted to None. `datadir`: value coming from `self.cpp.package.resdirs`. Defaulted to None. `localedir`: value coming from `self.cpp.package.resdirs`. Defaulted to None. `mandir`: value coming from `self.cpp.package.resdirs`. Defaulted to None. `infodir`: value coming from `self.cpp.package.resdirs`. Defaulted to None. `includedir`: value coming from `self.cpp.package.includedirs`. Defaulted to None. `libdir`: value coming from `self.cpp.package.libdirs`. Defaulted to None.

Notice that it needs a `layout` to be able to initialize those `self.cpp.package.xxxxx` variables. For instance:

```
from conan import ConanFile
from conan.tools.meson import MesonToolchain
class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    def layout(self):
        self.folders.build = "build"
        self.cpp.package.resdirs = ["res"]
    def generate(self):
        tc = MesonToolchain(self)
        self.output.info(tc.project_options["datadir"])  # Will print '["res"]'
        tc.generate()
```

Note: All of them are saved only if they have any value. If the values are``None``, they won't be mentioned in `[project options]` section.

Customization

Attributes

project_options

This attribute allows defining Meson project options:

```
def generate(self):
    tc = MesonToolchain(self)
    tc.project_options["MYVAR"] = "MyValue"
    tc.generate()
```

This is translated to:

- One project options definition for `MYVAR` in `conan_meson_native.ini` or `conan_meson_cross.ini` file.

The `wrap_mode: nofallback` is defined by default as a project option, to make sure that dependencies are found in Conan packages. It is possible to change or remove it with:

```
def generate(self):
    tc = MesonToolchain(self)
    tc.project_options.pop("wrap_mode")
    tc.generate()
```

Note that in this case, Meson might be able to find dependencies in "wraps", it is the responsibility of the user to check the behavior and make sure about the dependencies origin.

subproject_options

This attribute allows defining Meson subproject options:

```
def generate(self):
    tc = MesonToolchain(self)
    tc.subproject_options["SUBPROJECT"] = [{'MYVAR': 'MyValue'}]
    tc.generate()
```

This is translated to:

- One subproject `SUBPROJECT` and option definition for `MYVAR` in the `conan_meson_native.ini` or `conan_meson_cross.ini` file.

Note that in contrast to `project_options`, `subproject_options` is a dictionary of lists of dictionaries. This is because Meson allows multiple subprojects, and each subproject can have multiple options.

preprocessor_definitions

This attribute allows defining compiler preprocessor definitions, for multiple configurations (Debug, Release, etc).

```
def generate(self):
    tc = MesonToolchain(self)
    tc.preprocessor_definitions["MYDEF"] = "MyValue"
    tc.generate()
```

This is translated to:

- One preprocessor definition for `MYDEF` in `conan_meson_native.ini` or `conan_meson_cross.ini` file.

conf

`MesonToolchain` is affected by these `[conf]` variables:

- `tools.meson.mesontoolchain:backend`. the meson backend to use. Possible values: `ninja`, `vs`, `vs2010`, `vs2015`, `vs2017`, `vs2019`, `xcode`.
- `tools.meson.mesontoolchain:extra_variables` dict of dicts to inject extra variables into the generated Meson files at generation time. Supported keys are `properties`, `binaries`, and `project_options`. This configuration takes priority over attributes set in the Conanfile. For example:

  ```
  [conf]
  tools.meson.mesontoolchain:extra_variables={"binaries": {"exe_wrapper": "/usr/bin/
  ↪qemu-aarch64"}, "properties": {"needs_exe_wrapper": "true"}}
  ```

- `tools.apple:sdk_path` argument for SDK path in case of Apple cross-compilation. It is used as value of the flag `-isysroot`.
- `tools.android:ndk_path` argument for NDK path in case of Android cross-compilation. It is used to get some binaries like `c`, `cpp` and `ar` used in `[binaries]` section from *conan_meson_cross.ini*.
- `tools.build:cxxflags` list of extra C++ flags that is used by `cpp_args`.
- `tools.build:cflags` list of extra of pure C flags that is used by `c_args`.
- `tools.build:sharedlinkflags` list of extra linker flags that is used by `c_link_args` and `cpp_link_args`.

- `tools.build:exelinkflags` list of extra linker flags that is used by `c_link_args` and `cpp_link_args`.
- `tools.build:linker_scripts` list of linker scripts, each of which will be prefixed with `-T` and passed to `c_link_args` and `cpp_link_args`. Only use this flag with linkers that supports specifying linker scripts with the `-T` flag, such as `ld`, `gold`, and `lld`.
- `tools.build:tools.build:add_rpath_link`: add `-Wl,-rpath-link,` linker flag. Set this to `True` to pass this flag pointing to all library directories of all host dependencies.
- `tools.build:defines` list of preprocessor definitions, each of which will be prefixed with `-D` and passed to `cpp_args` and `c_args`.
- `tools.build:compiler_executables` dict-like Python object which specifies the compiler as key and the compiler executable path as value. Those keys will be mapped as follows:
- `tools.build:sysroot` which accepts a path to the system root directory and sets the `--sysroot` flag that is used by `c_args`, `cpp_args`, `c_link_args` and `cpp_link_args`.
  - `c`: will set `c` in `[binaries]` section from *conan_meson_xxxx.ini*.
  - `cpp`: will set `cpp` in `[binaries]` section from *conan_meson_xxxx.ini*.
  - `objc`: will set `objc` in `[binaries]` section from *conan_meson_xxxx.ini*.
  - `objcpp`: will set `objcpp` in `[binaries]` section from *conan_meson_xxxx.ini*.

Using Proper Data Types for Conan Options in Meson

Always transform Conan options into valid Python data types before assigning them as Meson values:

```
options = {{"shared": [True, False], "fPIC": [True, False], "with_msg": ["ANY"]}}
default_options = {{"shared": False, "fPIC": True, "with_msg": "Hi everyone!"}}

def generate(self):
    tc = MesonToolchain(self)
    tc.project_options["DYNAMIC"] = bool(self.options.shared)  # shared is bool
    tc.project_options["GREETINGS"] = str(self.options.with_msg)  # with_msg is str
    tc.subproject_options["SUBPROJECT"] = [{'MYVAR': str(self.options.with_msg)}]  #␣
↪with_msg is str
    tc.subproject_options["SUBPROJECT"].append({'MYVAR': bool(self.options.shared)})  #␣
↪shared is bool
    tc.generate()
```

In contrast, directly assigning a Conan option as a Meson value is strongly discouraged:

```
options = {{"shared": [True, False], "fPIC": [True, False], "with_msg": ["ANY"]}}
default_options = {{"shared": False, "fPIC": True, "with_msg": "Hi everyone!"}}
# ...
def generate(self):
    tc = MesonToolchain(self)
    tc.project_options["DYNAMIC"] = self.options.shared  # == <PackageOption object>
    tc.project_options["GREETINGS"] = self.options.with_msg  # == <PackageOption object>
    tc.subproject_options["SUBPROJECT"] = [{'MYVAR': self.options.with_msg}]  # ==
↪<PackageOption object>
    tc.subproject_options["SUBPROJECT"].append({'MYVAR': self.options.shared})  # ==
↪<PackageOption object>
    tc.generate()
```

These are not boolean or string values but an internal Conan class representing such option values. If you assign these values directly, upon executing the *generate()* function, you should receive a warning in your console stating, `WARN: deprecated: Please, do not use a Conan option value directly.` This method is considered bad practice as it can result in unexpected errors during your project's build process.

Cross-building for Apple and Android

The `MesonToolchain` generator adds all the flags required to cross-compile for Apple (MacOS M1, iOS, etc.) and Android.

Apple

It adds link flags `-arch XXX`, `-isysroot [SDK_PATH]` and the minimum deployment target flag, e.g., `-mios-version-min=8.0` to the `MesonToolchain` `c_args`, `c_link_args`, `cpp_args`, and `cpp_link_args` attributes, given the Conan settings for any Apple OS (iOS, watchOS, etc.) and the `tools.apple:sdk_path` configuration value like it's shown in this example of host profile:

Listing 157: **ios_host_profile**

```
[settings]
os = iOS
os.version = 10.0
os.sdk = iphoneos
arch = armv8
compiler = apple-clang
compiler.version = 12.0
compiler.libcxx = libc++

[conf]
tools.apple:sdk_path=/my/path/to/iPhoneOS.sdk
```

Android

It initializes the `MesonToolchain` `c`, `cpp`, and `ar` attributes, which are needed to cross-compile for Android, given the Conan settings for Android and the `tools.android:ndk_path` configuration value like it's shown in this example of host profile:

Listing 158: **android_host_profile**

```
[settings]
os = Android
os.api_level = 21
arch = armv8

[conf]
tools.android:ndk_path=/my/path/to/NDK
```

Cross-building and native=true

New since Conan 2.3.0

When you are cross-building, sometimes you need to build a tool which is used to generate source files. For this you would want to build some targets with the system's native compiler. Then, you need Conan to create both context files:

```
def generate(self):
    tc = MesonToolchain(self)
    tc.generate()
    # Forcing to create the native context too
    if cross_building(self):
        tc = MesonToolchain(self, native=True)
        tc.generate()
```

See also this reference from the Meson documentation for more information.

Objective-C arguments

In Apple OS's there are also specific Objective-C/Objective-C++ arguments: `objc`, `objcpp`, `objc_args`, `objc_link_args`, `objcpp_args`, and `objcpp_link_args`, as public attributes of the `MesonToolchain` class, where the variables `objc` and `objcpp` are initialized as `clang` and `clang++` respectively by default.

See also:

- *Getting started with Meson*

Reference

class MesonToolchain(*conanfile*, *backend=None*, *native=False*)

MesonToolchain generator

Parameters

- **conanfile** – < ConanFile object > The current recipe object. Always use `self`.
- **backend** – (**DEPRECATED**, use `self.backend` instead) `str` backend Meson variable value. By default, `ninja`.
- **native** – `bool` Indicates whether you want Conan to create the `conan_meson_native.ini` in a cross-building context. Notice that it only makes sense if your project's `meson.build` uses the `native=true` (see also https://mesonbuild.com/Cross-compilation.html#mixing-host-and-build-targets).

backend

Backend to use. Defined by the conf `tools.meson.mesontoolchain:backend`. By default, `ninja`.

buildtype

Build type to use.

b_ndebug

Disable asserts.

b_staticpic

Build static libraries as position independent. By default, `self.options.get_safe("fPIC")`

default_library

Default library type, e.g., "shared.

cpp_std

C++ language standard to use. Defined by `to_cppstd_flag()` by default.

c_std

C language standard to use. Defined by `to_cstd_flag()` by default.

b_vscrt

VS runtime library to use. Defined by `msvc_runtime_flag()` by default.

extra_cxxflags

List of extra `CXX` flags. Added to `cpp_args`

extra_cflags

List of extra `C` flags. Added to `c_args`

extra_ldflags

List of extra linker flags. Added to `c_link_args` and `cpp_link_args`

extra_defines

List of extra preprocessor definitions. Added to `c_args` and `cpp_args` with the format `-D[FLAG_N]`.

arch_flag

Architecture flag deduced by Conan and added to `c_args`, `cpp_args`, `c_link_args` and `cpp_link_args`

arch_link_flag

Architecture link flag deduced by Conan and added to `c_link_args` and `cpp_link_args`

threads_flags

Threads flags deduced by Conan and added to `c_args`, `cpp_args`, `c_link_args` and `cpp_link_args`

properties

Dict-like object that defines Meson `properties` with `key=value` format

binaries

Dict-like object that defines Meson `binaries` with `key=value` format. If any dict key matches a public attribute binary name, e.g., "c", "cpp", etc., it will override that one.

project_options

Dict-like object that defines Meson `project options` with `key=value` format

preprocessor_definitions

Dict-like object that defines Meson `preprocessor definitions`

subproject_options

Dict-like object that defines Meson `subproject options`.

pkg_config_path

Defines the Meson `pkg_config_path` variable

cross_build

Dict-like object with the build, host, and target as the Meson machine context

c

Sets the Meson `c` variable, defaulting to the `CC` build environment value. If provided as a blank-separated string, it will be transformed into a list. Otherwise, it remains a single string.

cpp

Sets the Meson `cpp` variable, defaulting to the `CXX` build environment value. If provided as a blank-separated string, it will be transformed into a list. Otherwise, it remains a single string.

ld

Sets the Meson `ld` variable, defaulting to the `LD` build environment value. If provided as a blank-separated string, it will be transformed into a list. Otherwise, it remains a single string.

c_ld

Defines the Meson `c_ld` variable. Defaulted to `CC_LD` environment value

cpp_ld

Defines the Meson `cpp_ld` variable. Defaulted to `CXX_LD` environment value

ar

Defines the Meson `ar` variable. Defaulted to `AR` build environment value

strip

Defines the Meson `strip` variable. Defaulted to `STRIP` build environment value

as_

Defines the Meson `as` variable. Defaulted to `AS` build environment value

windres

Defines the Meson `windres` variable. Defaulted to `WINDRES` build environment value

pkgconfig

Defines the Meson `pkgconfig` variable. Defaulted to `PKG_CONFIG` build environment value

c_args

Defines the Meson `c_args` variable. Defaulted to `CFLAGS` build environment value

c_link_args

Defines the Meson `c_link_args` variable. Defaulted to `LDFLAGS` build environment value

cpp_args

Defines the Meson `cpp_args` variable. Defaulted to `CXXFLAGS` build environment value

cpp_link_args

Defines the Meson `cpp_link_args` variable. Defaulted to `LDFLAGS` build environment value

apple_arch_flag

Apple arch flag as a list, e.g., `["-arch", "i386"]`

apple_isysroot_flag

Apple sysroot flag as a list, e.g., `["-isysroot", "./Platforms/MacOSX.platform"]`

apple_min_version_flag

Apple minimum binary version flag as a list, e.g., `["-mios-version-min", "10.8"]`

apple_extra_flags

Apple bitcode, visibility and arc flags

objc

Defines the Meson `objc` variable. Defaulted to `None`, if if any Apple OS `clang`

objcpp

Defines the Meson `objcpp` variable. Defaulted to `None`, if if any Apple OS `clang++`

objc_args

Defines the Meson `objc_args` variable. Defaulted to `OBJCFLAGS` build environment value

objc_link_args

Defines the Meson `objc_link_args` variable. Defaulted to `LDFLAGS` build environment value

objcpp_args

Defines the Meson `objcpp_args` variable. Defaulted to `OBJCXXFLAGS` build environment value

objcpp_link_args

Defines the Meson `objcpp_link_args` variable. Defaulted to `LDFLAGS` build environment value

generate()

Creates a `conan_meson_native.ini` (if native builds) or a `conan_meson_cross.ini` (if cross builds) with the proper content. If Windows OS, it will be created a `conanvcvars.bat` as well.

Meson

The `Meson()` build helper is intended to be used in the `build()` and `package()` methods, to call Meson commands automatically.

```
from conan import ConanFile
from conan.tools.meson import Meson

class PkgConan(ConanFile):

    def build(self):
        meson = Meson(self)
        meson.configure()
        meson.build()

    def package(self):
        meson = Meson(self)
        meson.install()
```

Reference

class Meson(*conanfile*)

This class calls Meson commands when a package is being built. Notice that this one should be used together with the `MesonToolchain` generator.

Parameters
conanfile – < `ConanFile object` > The current recipe object. Always use `self`.

configure(*reconfigure=False*)

Runs `meson setup [FILE] "BUILD_FOLDER" "SOURCE_FOLDER" [-Dprefix=/]` command, where `FILE` could be `--native-file conan_meson_native.ini` (if native builds) or `--cross-file conan_meson_cross.ini` (if cross builds).

Parameters
reconfigure – `bool` value that adds `--reconfigure` param to the final command.

build(*target=None*)

Runs `meson compile -C . -j[N_JOBS] [TARGET]` in the build folder. You can specify `N_JOBS` through the configuration line `tools.build:jobs=N_JOBS` in your profile `[conf]` section.

Parameters
target – `str` Specifies the target to be executed.

install(*cli_args=None*)

Runs `meson install -C "." --destdir ..` in the build folder.

Parameters
cli_args – List of arguments to be added to the command: `meson install -C "." --destdir ... arg1 arg2`

test()

Runs `meson test -v -C "."` in the build folder.

conf

The `Meson` build helper is affected by these `[conf]` variables:

- `tools.meson.mesontoolchain:extra_machine_files=[<FILENAME>]` configuration to add your machine files at the end of the command using the correct parameter depending on native or cross builds. See this Meson reference for more information.
- `tools.compilation:verbosity` which accepts one of `quiet` or `verbose` and sets the `--verbose` flag in `Meson.build()`
- `tools.build:verbosity` which accepts one of `quiet` or `verbose` and sets the `--quiet` flag in `Meson.install()`
- `tools.build:install_strip` (Since Conan 2.18.0; list values since Conan 2.28.0): when enabled for Meson, passes `--strip` to `meson install`. Use `True` so every integration that reads this configuration may strip; use a list such as `["meson"]` so only the Meson helper strips. `False` or an unset value disables stripping in this helper.

9.10.13 conan.tools.microsoft

MSBuild

The `MSBuild` build helper is a wrapper around the command line invocation of MSBuild. It abstracts the calls like `msbuild "MyProject.sln" /p:Configuration=<conf> /p:Platform=<platform>` into Python method ones.

This helper can be used like:

```
from conan import ConanFile
from conan.tools.microsoft import MSBuild

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def build(self):
        msbuild = MSBuild(self)
        msbuild.build("MyProject.sln")
```

The `MSBuild.build()` method internally implements a call to `msbuild` like:

```
$ <vcvars-cmd> && msbuild "MyProject.sln" /p:Configuration=<configuration> /p:Platform=
↪<platform>
```

Where:

- `<vcvars-cmd>` calls the Visual Studio prompt that matches the current recipe `settings`.
- `configuration`, typically Release, Debug, which will be obtained from `settings.build_type` but this can be customized with the `build_type` attribute.
- `<platform>` is the architecture, a mapping from the `settings.arch` to the common 'x86', 'x64', 'ARM', 'ARM64', 'ARM64EC'. This can be customized with the `platform` attribute.

Customization

attributes

You can customize the following attributes in case you need to change them:

- **build_type** (default `settings.build_type`): Value for the `/p:Configuration`.
- **platform** (default based on `settings.arch` to select one of these values: `('x86', 'x64', 'ARM', 'ARM64', 'ARM64EC')`: Value for the `/p:Platform`.

Example:

```
from conan import ConanFile
from conan.tools.microsoft import MSBuild
class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    def build(self):
        msbuild = MSBuild(self)
        msbuild.build_type = "MyRelease"
        msbuild.platform = "MyPlatform"
        msbuild.build("MyProject.sln")
```

conf

MSBuild is affected by these [conf] variables:

- tools.build:verbosity accepts one of quiet or verbose to be passed to the MSBuild.build() call as msbuild .... /verbosity:{Quiet,Detailed}.
- tools.microsoft.msbuild:max_cpu_count maximum number of CPUs to be passed to the MSBuild.build() call as msbuild .... /m:N. If max_cpu_count=0, then it will use /m without arguments, which means use all available cpus.

Reference

class MSBuild(*conanfile*)

MSBuild build helper class

Parameters
conanfile – < ConanFile object > The current recipe object. Always use self.

command(*sln*, *targets=None*)

Gets the msbuild command line. For instance, **msbuild.exe "MyProject.sln" -p:Configuration=<conf> -p:Platform=<platform>**.

Parameters

- **sln** – str name of Visual Studio *.sln file
- **targets** – targets is an optional argument, defaults to None, and otherwise it is a list of targets to build

Returns
str msbuild command line.

build(*sln*, *targets=None*)

Runs the msbuild command line obtained from self.command(sln).

Parameters

- **sln** – str name of Visual Studio *.sln file
- **targets** – targets is an optional argument, defaults to None, and otherwise it is a list of targets to build

MSBuildDeps

The MSBuildDeps is the dependency information generator for Microsoft MSBuild build system. It will generate multiple *xxxx.props* properties files, one per dependency of a package, to be used by consumers using MSBuild or Visual Studio, just adding the generated properties files to the solution and projects.

The MSBuildDeps generator can be used by name in conanfiles:

Listing 159: conanfile.py

```
class Pkg(ConanFile):
    generators = "MSBuildDeps"
```

Listing 160: conanfile.txt

```
[generators]
MSBuildDeps
```

And it can also be fully instantiated in the conanfile `generate()` method:

Listing 161: conanfile.py

```
from conan import ConanFile
from conan.tools.microsoft import MSBuildDeps

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    requires = "zlib/1.3.1", "bzip2/1.0.8"

    def generate(self):
        ms = MSBuildDeps(self)
        ms.generate()
```

Generated files

The `MSBuildDeps` generator is a multi-configuration generator, and generates different files for any different Debug/Release configuration. For instance, running these commands:

```
$ conan install .  # default is Release
$ conan install . -s build_type=Debug
```

It generates the next files:

- *conan_zlib_vars_release_x64.props*: `Conanzlibxxxx` variables definitions for the `zlib` dependency, Release config, like `ConanzlibIncludeDirs`, `ConanzlibLibs`, etc.
- *conan_zlib_vars_debug_x64.props*: Same `Conanzlib``variables for ``zlib` dependency, Debug config
- *conan_zlib_release_x64.props*: Activation of `Conanzlibxxxx` variables in the current build as standard C/C++ build configuration, Release config. This file contains also the transitive dependencies definitions.
- *conan_zlib_debug_x64.props*: Same activation of `Conanzlibxxxx` variables, Debug config, also inclusion of transitive dependencies.
- *conan_zlib.props*: Properties file for `zlib`. It conditionally includes, depending on the configuration, one of the two immediately above Release/Debug properties files.
- Same 5 files are generated for every dependency in the graph, in this case `conan_bzip.props` too, which conditionally includes the Release/Debug bzip properties files.
- *conandeps.props*: Properties files that includes all direct dependencies, for this case `conan_zlib.props` and `conan_bzip2.props`

Add the *conandeps.props* to your solution project files if you want to depend on all the declared dependencies. For single project solutions, this is probably the way to go. For multi-project solutions, you might be more efficient and add properties files per project. You could add *conan_zlib.props* properties to "project1" in the solution and *conan_bzip2.props* to "project2" in the solution for example.

The above files are generated when the package doesn't have components. If the package has defined components, the following files will be generated:

- *conan_pkgname_compname_vars_release_x64.props*: Definition of variables for the component `compname` of the package `pkgname`
- *conan_pkgname_compname_release_x64.props*: Activation of the above variables into VS effective variables to be used in the build
- *conan_pkgname_compname.props*: Properties file for component `compname` of package `pkgname`. It conditionally includes, depending on the configuration, the specific activation property files.
- *conan_pkgname.props*: Properties file for package `pkgname`. It includes and aggregates all the components of the package.
- *conandeps.props*: Same as above, aggregates all the direct dependencies property files for the packages (like `conan_pkgname.props`)

If your project depends only on certain components, the specific `conan_pkgname_compname.props` files can be added to the project instead of the global or the package ones.

Requirement traits support

The above generated files, more specifically the files containing the variables (`conan_pkgname_vars_release_x64.props/conan_pkgname_compname_vars_release_x64.props`), will not contain all the information if the requirement traits have excluded them. For example, by default, the `includedirs` of transitive dependencies will be empty, as those headers shouldn't be included by the user unless a specific `requires` to that package is defined.

Configurations

If your Visual Studio project defines custom configurations, like `ReleaseShared`, or `MyCustomConfig`, it is possible to define it into the `MSBuildDeps` generator, so different project configurations can use different set of dependencies. Let's say that our current project can be built as a shared library, with the custom configuration `ReleaseShared`, and the package also controls this with the `shared` option:

```
from conan import ConanFile
from conan.tools.microsoft import MSBuildDeps

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    options = {"shared": [True, False]}
    default_options = {"shared": False}
    requires = "zlib/1.3.1"

    def generate(self):
        ms = MSBuildDeps(self)
        # We assume that -o *:shared=True is used to install all shared deps too
        if self.options.shared:
            ms.configuration = str(self.settings.build_type) + "Shared"
        ms.generate()
```

This generates new properties files for this custom configuration, and switching it in the IDE allows to gather dependencies configuration like Debug/Release, and even static and/or shared libraries.

Platform

By default, the `Platform` is computed from the Conan `arch` setting as:

Conan `arch`	MSBuild `Platform`
x86	Win32
x86_64	x64
armv7	ARM
armv8	ARM6

This default platform can be overriden if necessary, for example for Wix projects that want to use `Platform=x86` instead of `Win32`, by defining the `platform` attribute:

```
def generate(self):
  deps = MSBuildDeps(self)
  if self.settings.arch == "x86":
      deps.platform = "x86"  # Override the "Win32" default value
  deps.generate()
```

Dependencies

`MSBuildDeps` uses the `self.dependencies` to access to the dependencies information. The following dependencies are translated to properties files:

- All the direct dependencies, which are the ones declared by the current `conanfile`, live in the `host` context: all regular `requires`, plus the `tool_requires`, that are in the host context, e.g. test frameworks like `gtest` or `catch`.
- All transitive `requires` of those direct dependencies (all in the host context)
- Tool requires, in the build context, that is, application and executables that run in the build machine irrespective of the destination platform, are added exclusively to the <ExecutablePath> property, taking the value from `$(Conan{{name}}BinaryDirectories)` defined properties. This allows to define custom build commands, invoke code generation tools, with the <CustomBuild> and <Command> elements.

Customization

conf

`MSBuildDeps` is affected by these `[conf]` variables:

- `tools.microsoft.msbuilddeps:exclude_code_analysis` list of packages names patterns to be added to the Visual Studio `CAExcludePath` property.

Reference

class MSBuildDeps(*conanfile*)

MSBuildDeps class generator conandeps.props: unconditional import of all *direct* dependencies only

Parameters
conanfile – < ConanFile object > The current recipe object. Always use self.

configuration

Defines the build type. By default, the value of settings.build_type.

configuration_key

Defines the configuration key used to conditionally select which property sheet to import (defaults to "Configuration").

platform

Platform name, e.g., Win32 if settings.arch == "x86".

platform_key

Defines the platform key used to conditionally select which property sheet to import (defaults to "Platform").

exclude_code_analysis

List of packages names patterns to add Visual Studio CAExcludePath property to each match as part of its conan_[DEP]_[CONFIG].props. By default, value given by tools.microsoft.msbuilddeps:exclude_code_analysis configuration.

generate()

Generates conan_<pkg>_<config>_vars.props, conan_<pkg>_<config>.props, and conan_<pkg>.props files into the conanfile.generators_folder.

MSBuildToolchain

The MSBuildToolchain is the toolchain generator for MSBuild. It will generate MSBuild properties files that can be added to the Visual Studio solution projects. This generator translates the current package configuration, settings, and options, into MSBuild properties files syntax.

This generator can be used by name in conanfiles:

Listing 162: **conanfile.py**

```
class Pkg(ConanFile):
    generators = "MSBuildToolchain"
```

Listing 163: **conanfile.txt**

```
[generators]
MSBuildToolchain
```

And it can also be fully instantiated in the conanfile generate() method:

```
from conan import ConanFile
from conan.tools.microsoft import MSBuildToolchain

class App(ConanFile):
```

(continues on next page)

(continued from previous page)

```
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = MSBuildToolchain(self)
        tc.generate()
```

The `MSBuildToolchain` will generate three files after a `conan install` command:

```
$ conan install . # default is Release
$ conan install . -s build_type=Debug
```

- The main *conantoolchain.props* file, to be added to the project.
- A *conantoolchain_<config>.props* file, that will be conditionally included from the previous *conantoolchain.props* file based on the configuration and platform, e.g., *conantoolchain_release_x86.props*.
- A *conanvcvars.bat* file with the `vcvars` invocation to define the build environment from the command line, or any other automated tools (might not be required if opening the IDE). This file will be automatically called by the `MSBuild.build()` method.

Every invocation with different configuration creates a new properties `.props` file, that is also conditionally included. That allows to install different configurations, then switch among them directly from the Visual Studio IDE.

The MSBuildToolchain files can configure:

- The Visual Studio runtime (*MT/MD/MTd/MDd*), obtained from Conan input settings.
- The C++ standard, obtained from Conan input settings.

One of the advantages of using toolchains is that they help to achieve the exact same build with local development flows, than when the package is created in the cache.

Customization

conf

`MSBuildToolchain` is affected by these `[conf]` variables:

- `tools.microsoft.msbuildtoolchain:compile_options` dict-like object of extra compile options to be added to <ClCompile> section. The dict will be translated as follows: <[KEY]>[VALUE]</[KEY]> and inserted inside the <ClCompile> element. If the `compile_options` attribute is defined, it will be updated with the values from this dict.
- `tools.microsoft:winsdk_version` value will define the <WindowsTargetPlatformVersion> element in the toolchain file.
- `tools.build:cxxflags` list of extra C++ flags that will be appended to <AdditionalOptions> section from <ClCompile> and <ResourceCompile> one.
- `tools.build:cflags` list of extra of pure C flags that will be appended to <AdditionalOptions> section from <ClCompile> and <ResourceCompile> one.
- `tools.build:sharedlinkflags` list of extra linker flags that will be appended to <AdditionalOptions> section from <Link> one.
- `tools.build:exelinkflags` list of extra linker flags that will be appended to <AdditionalOptions> section from <Link> one.

- `tools.build:rcflags` list of extra RC flags that will be appended to `<AdditionalOptions>` section in `<ResourceCompile>` one.
- `tools.build:defines` list of preprocessor definitions that will be appended to `<PreprocessorDefinitions>` section from `<ResourceCompile>` one.

Reference

class MSBuildToolchain(*conanfile*)

MSBuildToolchain class generator

Parameters
conanfile – < ConanFile object > The current recipe object. Always use `self`.

generate()

Generates a `conantoolchain.props`, a `conantoolchain_<config>.props`, and, if `compiler=msvc`, a `conanvcvars.bat` files. In the first two cases, they'll have the valid XML format with all the good settings like any other VS project `*.props` file. The last one emulates the `vcvarsall.bat` env script. See also *VCVars*.

Attributes

- **properties**: Additional properties added to the generated `.props` files. You can define the properties in a key-value syntax like:

```
from conan import ConanFile
from conan.tools.microsoft import MSBuildToolchain

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        msbuild = MSBuildToolchain(self)
        msbuild.properties["IncludeExternals"] = "true"
        msbuild.generate()
```

Then, the generated *conantoolchain_<config>.props* file will contain the defined property in its contents:

```
<?xml version="1.0" encoding="utf-8"?>
<Project xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
<ItemDefinitionGroup>
...
</ItemDefinitionGroup>
<PropertyGroup Label="Configuration">
    ...
    <IncludeExternals>true</IncludeExternals>
    ...
</PropertyGroup>
</Project>
```

- **compile_options**: Additional compile options added to the generated `.props` files. You can define the properties in a key-value syntax like:

```
def generate(self):
    msbuild = MSBuildToolchain(self)
    msbuild.compile_options = {'ExceptionHandling': 'Async'}
    msbuild.generate()
```

Then, the generated *conantoolchain_<config>.props* file will contain the defined compile option in its contents:

```
<ClCompile>
    ...
    <LanguageStandard>...</LanguageStandard>
    <ExceptionHandling>Async</ExceptionHandling>
</ClCompile>
```

Note this attribute will be updated with the `conf` from `tools.microsoft.msbuildtoolchain:compile_options`.

VCVars

Generates a file called `conanvcvars.bat` that activates the Visual Studio developer command prompt according to the current settings by wrapping the vcvarsall Microsoft bash script.

The `VCVars` generator can be used by name in conanfiles:

Listing 164: conanfile.py

```
class Pkg(ConanFile):
    generators = "VCVars"
```

Listing 165: conanfile.txt

```
[generators]
VCVars
```

And it can also be fully instantiated in the conanfile `generate()` method:

Listing 166: conanfile.py

```
from conan import ConanFile
from conan.tools.microsoft import VCVars

class Pkg(ConanFile):
    settings = "os", "compiler", "arch", "build_type"
    requires = "zlib/1.3.1", "bzip2/1.0.8"

    def generate(self):
        ms = VCVars(self)
        ms.generate()
```

Customization

conf

`VCVars` is affected by these `[conf]` variables:

- `tools.microsoft.msbuild:installation_path` indicates the path to Visual Studio installation folder. For instance: `C:\Program Files (x86)\Microsoft Visual Studio\2019\Community`, `C:\Program Files (x86)\Microsoft Visual Studio 14.0`, etc.
- `tools.microsoft:winsdk_version` defines the specific winsdk version in the vcvars command line.
- `tools.env.virtualenv:powershell` generates an additional `conanvcvars.ps1` for all packages, so it can be run from the Powershell console. The `conanvcvars.ps1` is always generated for the consumer (`conan install <path>`) and `test_package`, but Conan only uses it in its own subprocesses if this conf is defined.

Reference

class VCVars(*conanfile*)

VCVars class generator to generate a `conanvcvars.bat` script that activates the correct Visual Studio prompt.

This generator will be automatically called by other generators such as `CMakeToolchain` when considered necessary, for example if building with Visual Studio compiler using the CMake `Ninja` generator, which needs an active Visual Studio prompt. Then, it is not necessary to explicitly instantiate this generator in most cases.

Parameters

conanfile – `ConanFile object` The current recipe object. Always use `self`.

generate(*scope='build'*)

Creates a `conanvcvars.bat` file that calls Visual `vcvars` with the necessary args to activate the correct Visual Studio prompt matching the Conan settings.

Parameters

scope – `str` activation scope, by default "build". It means it will add a call to this `conanvcvars.bat` from the aggregating general `conanbuild.bat`, which is the script that will be called by default in `self.run()` calls and build helpers such as `cmake.configure()` and `cmake.build()`.

NMakeDeps

This generator can be used as:

```
from conan import ConanFile

class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch"

    requires = "mydep/1.0"
    # attribute declaration
    generators = "NMakeDeps"

    # OR explicit usage in the generate() method
    def generate(self):
        deps = NMakeDeps(self)
        deps.generate()

    def build(self):
        self.run(f"nmake /f makefile")
```

The generator will create a `conannmakedeps.bat` environment script that defines `CL`, `LIB` and `_LINK_` environment variables, injecting necessary flags to locate and link the dependencies declared in `requires`. This generator should most likely be used together with `NMakeToolchain` one.

NMakeToolchain

This generator can be used as:

```
from conan import ConanFile

class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch"
    generators = "NMakeToolchain"

    def build(self):
        self.run("nmake /f makefile")
```

Or it can be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.microsoft import NMakeToolchain

class Pkg(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = NMakeToolchain(self)
        tc.generate()

    def build(self):
        self.run("nmake /f makefile")
```

NMakeToolchain generator will create a `conannmaketoolchain.bat` environment script injecting flags deduced from profile (build_type, runtime, cppstd, build flags from conf) into environment variables NMake can understand: `CL` and `_LINK_`. It will also generate a `conanvcvars.bat` script that activates the correct VS prompt matching the Conan host settings `arch`, `compiler` and `compiler.version`, and build settings `arch`.

constructor

```
def __init__(self, conanfile):
```

- `conanfile`: the current recipe object. Always use `self`.

Attributes

You can change some attributes before calling the `generate()` method if you want to inject more flags:

```
from conan import ConanFile
from conan.tools.microsoft import NMakeToolchain

class Pkg(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = NMakeToolchain(self)
        tc.extra_cflags.append("/my_flag")
        tc.extra_defines.append("FOO=BAR")
        tc.generate()
```

- **extra_cflags** (Defaulted to `[]`): Additional cflags.
- **extra_cxxflags** (Defaulted to `[]`): Additional cxxflags.
- **extra_defines** (Defaulted to `[]`): Additional defines.
- **extra_ldflags** (Defaulted to `[]`): Additional ldflags.

conf

`NMaketoolchain` is affected by these `[conf]` variables:

- `tools.build:cflags` list of extra pure C flags that will be used by `CL`.
- `tools.build:cxxflags` list of extra C++ flags that will be used by `CL`.
- `tools.build:defines` list of preprocessor definitions that will be used by `CL`.
- `tools.build:sharedlinkflags` list of extra linker flags that will be used by `_LINK_`.
- `tools.build:exelinkflags` list of extra linker flags that will be used by `_LINK_`.
- `tools.build:rcflags` list of extra RC flags that will define a `RCFLAGS` env variable
- `tools.build:compiler_executables` dict-like Python object which specifies the compiler as key and the compiler executable path as value. Those keys will be mapped as follows:
  - `asm`: will set `AS` in *conannmaketoolchain.sh|bat* script.
  - `c`: will set `CC` in *conannmaketoolchain.sh|bat* script.

- cpp: will set CPP and CXX in *conannmaketoolchain.sh|bat* script.
- rc: will set RC in *conannmaketoolchain.sh|bat* script.

Customizing the environment

If your Makefile script needs some other environment variable rather than CL and _LINK_, you can customize it before calling the generate() method. Call the environment() method to calculate the mentioned variables and then add the variables that you need. The environment() method returns an *Environment* object:

```
from conan import ConanFile
from conan.tools.microsoft import NMakeToolchain

class Pkg(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = NMakeToolchain(self)
        env = tc.environment()
        env.define("FOO", "BAR")
        tc.generate(env)
```

You can also inspect default environment variables NMakeToolchain will inject in *conannmaketoolchain.sh|bat* script:

```
from conan import ConanFile
from conan.tools.microsoft import NMakeToolchain

class Pkg(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = NMakeToolchain(self)
        env_vars = tc.vars()
        cl_env_var = env_vars.get("CL")
```

vs_layout

vs_layout(*conanfile*)

Initialize a layout for a typical Visual Studio project.

Parameters

conanfile – < ConanFile object > The current recipe object. Always use self.

check_min_vs

check_min_vs(*conanfile*, *version*, *raise_invalid=True*)

This is a helper method to allow the migration of 1.X -> 2.0 and VisualStudio -> msvc settings without breaking recipes. The legacy "Visual Studio" with different toolset is not managed, not worth the complexity.

Parameters

- **raise_invalid** – `bool` Whether to raise or return False if the version check fails
- **conanfile** – `< ConanFile object >` The current recipe object. Always use `self`.
- **version** – `str` Visual Studio or msvc version number.

Example:

```
def validate(self):
    check_min_vs(self, "192")
```

msvc_runtime_flag

msvc_runtime_flag(*conanfile*)

Gets the MSVC runtime flag given the `compiler.runtime` value from the settings.

Parameters

conanfile – `< ConanFile object >` The current recipe object. Always use `self`.

Returns

`str` runtime flag.

is_msvc

is_msvc(*conanfile*, *build_context=False*)

Validates if the current compiler is `msvc`.

Parameters

- **conanfile** – `< ConanFile object >` The current recipe object. Always use `self`.
- **build_context** – If True, will use the settings from the build context, not host ones

Returns

`bool` True, if the host compiler is `msvc`, otherwise, False.

is_msvc_static_runtime

is_msvc_static_runtime(*conanfile*)

Validates when building with Visual Studio or msvc and MT on runtime.

Parameters

conanfile – `< ConanFile object >` The current recipe object. Always use `self`.

Returns

`bool` True, if `msvc + runtime MT`. Otherwise, False.

msvs_toolset

msvs_toolset(*conanfile*)

Returns the corresponding platform toolset based on the compiler setting. In case no toolset is configured in the profile, it will return a toolset based on the compiler version, otherwise, it will return the toolset from the profile. When there is no compiler version neither toolset configured, it will return None It supports msvc, intel-cc and clang compilers. For clang, is assumes the ClangCl toolset, as provided by the Visual Studio installer.

Parameters
conanfile – Conanfile instance to access settings.compiler

Returns
A toolset when compiler.version is valid or compiler.toolset is configured. Otherwise, None.

unix_path

unix_path(*conanfile*, *path*, *scope='build'*)

9.10.14 conan.tools.qbs

Qbs

The `Qbs` build helper is a wrapper around the command line invocation of the Qbs build tool. It will abstract the calls like `qbs resolve`, `qbs build` and `qbs install` into Python method calls.

The helper is intended to be used in the `build()` and `package()` methods, to call Qbs commands automatically when a package is being built directly by Conan (create, install).

```
from conan import ConanFile
from conan.tools.qbs import Qbs, QbsProfile, QbsDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    exports_sources = "*.h", "*.cpp", "*.qbs",
    requires = "hello/0.1"
    options = {"shared": [True, False], "fPIC": [True, False]}
    default_options = {"shared": False, "fPIC": True}

    def generate(self):
        profile = QbsProfile(self)
        profile.generate()
        deps = QbsDeps(self)
        deps.generate()

    def build(self):
        qbs = Qbs(self)
        qbs.resolve()
        qbs.build()

    def package(self):
        qbs = Qbs(self)
        qbs.install()
```

Reference

class Qbs(*conanfile*, *project_file=None*)

Qbs helper to use together with the QbsDeps feature. This class provides helper methods that wraps calls to the Qbs tool.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **project_file** – The name to the main project file. If not set, Qbs will try to autodetect the project file.

add_configuration(*name*, *values*)

Adds a build configuration for the multi-configuration build. This Qbs feature is rarely needed since each conan package can contain only one configuration, however might be useful when creating multiple versions of the same product that should be put in the same Conan package.

Parameters

- **name** – the name of the configuration. This corresponds to the `config` parameter of `qbs resolve`, `qbs build` and `qbs install` commands.
- **values** – the dict containing Qbs properties and their values for this configuration.

resolve(*parallel=True*)

Wraps the `qbs resolve` call. If QbsDeps generator is used, this will also set the necessary properites of the Qbs "conan" module provider automatically adding dependencies to the project. :param parallel: Whether to use multi-threaded resolving. Defaults to `True`.

build(*products=None*)

Wraps the `qbs build` call.

Parameters

products – The list of product names to build. If not set, builds all products that have builtByDefault set to true. This parameter corresponds to the `--products` option of the `qbs build` command.

The resolve() method should be called before calling this method.

build_all()

Wraps the `qbs build --all-products` call. This method builds all products, even if their builtByDefault property is false. The resolve() method should be called before calling this method.

install()

Wraps the `qbs install` call. Perfoms the installation of files marked as installable in the Qbs project. The build() or build_all() methods should be called before calling this method.

QbsDeps

The `QbsDeps` generator produces the necessary files for each dependency to be able to use the qbs `Depends` item to locate the dependencies. It can be used like:

```
from conan import ConanFile

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
```

(continues on next page)

(continued from previous page)

```
    requires = "hello/0.1"
    generators = "QbsDeps"
```

It is also possible to use `QbsDeps` manually in the `generate()` method:

```
from conan import ConanFile
from conan.tools.qbs import QbsDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"

    def generate(self):
        deps = QbsDeps(self)
        deps.generate()
```

The dependency to the application can be added using the `Depends` item:

Listing 167: **project.qbs**

```
CppApplication {
    Depends { name: "hello"; version: "0.1" }
    files: "main.c"
    qbs.installPrefix: ""
    install: true
    qbsModuleProviders: "conan"
    moduleProviders.conan.installDirectory: "build"
}
```

By default, for a `hello` requires, you need to use `Depends { name: "hello" }`.

Note that we are setting the `qbsModuleProviders` property to `"conan"` in order to tell Qbs that dependencies are generated by Conan. We also set the `installDirectory` property of the `conan` module provider to `"build"` to tell the provider to look for generated files in the `build` directory within the source directory. It is also possible to set this value from command line.

We install dependencies using the `conan install` command.

```
$ conan install . --output-folder=build --build missing
```

Finally, we can build the project by simply calling Qbs:

```
$ qbs
```

Note, that Qbs helper will automatically set the `conan.installDirectory` property when QbsDeps generator is used.

See also:

- Check the *Qbs helper* for details.

Properties

The following properties affect the `QbsDeps` generator:

- **qbs_file_name**: Name of the generated Qbs module for the current package. The generator creates a `<VALUE>.json` file, and consumers use `Depends { name: "<VALUE>" }`. By default this is the package name, or `pkg_config_name` if that property is set. Use `qbs_file_name` to override the module name.

Example:

```
def package_info(self):
    ...
    # Generates MyFirstLib.json, consumed as: Depends { name: "MyFirstLib" }
    self.cpp_info.set_property("qbs_file_name", "MyFirstLib")
```

Reference

class QbsDeps(*conanfile*)

This class will generate a JSON file for each dependency inside the "conan-qbs-deps" folder. Each JSON file contains information necesary for Qbs `"conan" module provider` to be able to generate Qbs module files.

Parameters
conanfile – The current recipe object. Always use `self`.

property content

Returns all dependency information as a Python dict object where key is the dependency name and value is a dict with dependency properties.

generate()

This method will save the generated files to the "conan-qbs-deps" directory inside the `conanfile.generators_folder` directory. Generates a single JSON file per dependency or component.

QbsProfile

The `QbsProfile` generator produces the settings file that contains toolchain information. This file can be imported into Qbs. The `QbsProfile` generator can be used like:

```
from conan import ConanFile

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"
    generators = "QbsProfile"
```

It is also possible to use `QbsProfile` manually in the `generate()` method:

```
from conan import ConanFile
from conan.tools.qbs import QbsProfile

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "hello/0.1"
```

(continues on next page)

(continued from previous page)

```
    def generate(self):
        profile = QbsProfile(self)
        profile.generate()
```

Now we can generate the file using the `conan install` command.

```
$ conan install . --output-folder=build --build missing
```

And import it into Qbs:

```
$ qbs config import qbs_settings.txt --settings-dir qbs
```

Note that to acutually use the imported file, Qbs should be called with `--settings-dir`:

```
$ qbs resolve --settings-dir qbs
```

Those commands are called automatically when using the `Qbs` helper class. .. seealso:

```
- Check the :ref:`Qbs helper <_conan_tools_qbs_helper>` for details.
```

Reference

class QbsProfile(*conanfile*, *profile='conan'*, *default_profile='conan'*)

Qbs profiles generator.

This class generates file with the toolchain information that can be imported by Qbs.

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **profile** – The name of the profile in settings. Defaults to `"conan"`.
- **default_profile** – The name of the default profile. Defaults to `"conan"`.

property filename

The name of the generated file. Returns `qbs_settings.txt`.

property content

Returns the content of the settings file as dict of Qbs properties.

render()

Returns the content of the settings file as string.

generate()

This method will save the generated files to the conanfile.generators_folder.

Generates the "qbs_settings.txt" file. This file contains Qbs settings such as toolchain properties and can be imported using `qbs config --import`.

9.10.15 conan.tools.ros

ROSEnv

The ROSEnv generator is an environment generator that, in conjunction with *CMakeDeps* and *CMakeToolchain*, allows to consume Conan packages from a ROS package.

Listing 168: conanfile.txt

```
[requires]
fmt/11.0.2

[generators]
CMakeDeps
CMakeToolchain
ROSEnv
```

This generator will create a *conanrosenv.sh* script with the required environment variables that allow CMake and Colcon to locate the packages installed by Conan.

This script needs to be *sourced* before the **colcon build** command:

```
$ cd workspace
$ conan install ...
$ source conanrosenv.sh
$ colcon build
```

Reference

class ROSEnv(*conanfile*)

Generator to serve as integration for Robot Operating System 2 development workspaces.

IMPORTANT: This generator should be used together with CMakeDeps and CMakeToolchain generators.

Parameters
conanfile – < ConanFile object > The current recipe object. Always use self.

generate()

Creates a conanrosenv.sh with the environment variables that are needed to build and execute ROS packages with Conan dependencies.

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

9.10.16 conan.tools.sbom

CycloneDX

The CycloneDX tool is available in the `conan.tools.sbom` module.

It provides the `cyclonedx_1_4` and `cyclonedx_1_6` functions which receive a `conanfile` and return a dictionary with the SBOM data in the CycloneDX 1.4/1.6 JSON format.

cyclonedx_1_4(*conanfile*, *name=None*, *add_build=False*, *add_tests=False*, *extra_info=None*, ***kwargs*)

(Experimental) Generate cyclone 1.4 SBOM with JSON format

Creates a CycloneDX 1.4 Software Bill of Materials (SBOM) from a given dependency graph.

Parameters:
conanfile: The conanfile instance. name (str, optional): Custom name for the metadata field. add_build (bool, optional, default=False): Include build dependencies. add_tests (bool, optional, default=False): Include test dependencies. extra_info (dict, optional): Extra CycloneDX component fields, keyed by name, name/version, name/version@user/channel, purl or bom-ref.

Returns:
The generated CycloneDX 1.4 document as a string.

Example usage: ` cyclonedx_1_4(conanfile, name="custom_name", add_build=True, add_test=True, **kwargs) `

cyclonedx_1_6(*conanfile*, *name=None*, *add_build=False*, *add_tests=False*, *extra_info=None*, ***kwargs*)

(Experimental) Generate cyclone 1.6 SBOM with JSON format

Creates a CycloneDX 1.6 Software Bill of Materials (SBOM) from a given dependency graph.

Parameters:
conanfile: The conanfile instance. name (str, optional): Custom name for the metadata field. add_build (bool, optional, default=False): Include build dependencies. add_tests (bool, optional, default=False): Include test dependencies. extra_info (dict, optional): Extra CycloneDX component fields, keyed by name, name/version, name/version@user/channel, purl or bom-ref.

Returns:
The generated CycloneDX 1.6 document as a string.

Example usage: ` cyclonedx_1_6(conanfile, name="custom_name", add_build=True, add_test=True, **kwargs) `

Both functions share an interface and are very similar; the main difference is the version of CycloneDX that each of them supports. The options `add_build` and `add_test` allow you to include the build and test packages, respectively, resolved by the graph.

Remember to enable the option if you wish to add any of them to your SBOM!

See also:

- *Software Bills of Materials (SBOM)*.
- *Generate SBOMs with the built-in deployers*.

9.10.17 conan.tools.scm

Git

The `Git` helper is a thin wrapper over the `git` command. It can be used for different purposes:

- Obtaining the current tag in the `set_version()` method to assign it to `self.version`
- Clone sources in third-party or open source package recipes in the `source()` method (in general, doing a `download()` or `get()` to fetch release tarballs will be preferred)
- Capturing the "scm" coordinates (url, commit) of your own package sources in the `export()` method, to be able to reproduce a build from source later, retrieving the code in the `source()` method. See the *example of git-scm capture*.

The `Git()` constructor receives the current folder as argument, but that can be changed if necessary, for example, to clone the sources of some repo in `source()`:

```
def source(self):
    git = Git(self)  # by default, the current folder "."
    git.clone(url="<repourl>", target="target") # git clone url target
    # we need to cd directory for next command "checkout" to work
    git.folder = "target"                       # cd target
    git.checkout(commit="<commit>")             # git checkout commit
```

An alternative, equivalent approach would be:

```
def source(self):
    git = Git(self, "target")
    # Cloning in current dir, not a children folder
    git.clone(url="<repourl>", target=".")
    git.checkout(commit="<commit>")
```

class Git(*conanfile*, *folder='.'*, *excluded=None*)

Git is a wrapper for several common patterns used with *git* tool.

Parameters

- **conanfile** – Conanfile instance.
- **folder** – Current directory, by default `.`, the current working directory.
- **excluded** – Files to be excluded from the "dirty" checks. It will compose with the configuration `core.scm:excluded` (the configuration has higher priority). It is a list of patterns to `fnmatch`.

run(*cmd*, *hidden_output=None*)

Executes `git <cmd>`

Returns

The console output of the command.

get_commit(*repository=False*)

Parameters

repository – By default gets the commit of the defined folder, use repo=True to get the commit of the repository instead.

Returns
The current commit, with `git rev-list HEAD -n 1 -- <folder>`. The latest commit is returned, irrespective of local not committed changes.

`get_remote_url`(*remote='origin'*)

Obtains the URL of the remote git remote repository, with `git remote -v`

Warning! Be aware that This method will get the output from `git remote -v`. If you added tokens or credentials to the remote in the URL, they will be exposed. Credentials shouldn't be added to git remotes definitions, but using a credentials manager or similar mechanism. If you still want to use this approach, it is your responsibility to strip the credentials from the result.

Parameters
remote – Name of the remote git repository ('origin' by default).

Returns
URL of the remote git remote repository.

`commit_in_remote`(*commit*, *remote='origin'*)

Checks that the given commit exists in the remote, with `branch -r --contains <commit>` and checking an occurrence of a branch in that remote exists.

Parameters

- **commit** – Commit to check.
- **remote** – Name of the remote git repository ('origin' by default).

Returns
True if the given commit exists in the remote, False otherwise.

`is_dirty`(*repository=False*)

Returns if the current folder is dirty, running `git status -s` The `Git(..., excluded=[])` argument and the `core.scm:excluded` configuration will define file patterns to be skipped from this check.

Parameters
repository – By default checks if the current folder is dirty. If repository=True it will check the root repository folder instead, not the current one.

Returns
True, if the current folder is dirty. Otherwise, False.

`get_url_and_commit`(*remote='origin'*, *repository=False*)

This is an advanced method, that returns both the current commit, and the remote repository url. This method is intended to capture the current remote coordinates for a package creation, so that can be used later to build again from sources from the same commit. This is the behavior:

- If the repository is dirty, it will raise an exception. Doesn't make sense to capture coordinates of something dirty, as it will not be reproducible. If there are local changes, and the user wants to test a local conan create, should commit the changes first (locally, not push the changes).
- If the repository is not dirty, but the commit doesn't exist in the given remote, the method will return that commit and the URL of the local user checkout. This way, a package can be conan create created locally, testing everything works, before pushing some changes to the remote.
- If the repository is not dirty, and the commit exists in the specified remote, it will return that commit and the url of the remote.

Warning! Be aware that This method will get the output from `git remote -v`. If you added tokens or credentials to the remote in the URL, they will be exposed. Credentials shouldn't be added to git remotes definitions, but using a credentials manager or similar mechanism. If you still want to use this approach, it is your responsibility to strip the credentials from the result.

Parameters

- **remote** – Name of the remote git repository ('origin' by default).
- **repository** – By default gets the commit of the defined folder, use repo=True to get the commit of the repository instead.

Returns
(url, commit) tuple

`get_repo_root()`

Get the current repository top folder with `git rev-parse --show-toplevel`

Returns
Repository top folder.

`clone(url, target='', args=None, hide_url=True)`

Performs a `git clone <url> <args> <target>` operation, where target is the target directory.

Parameters

- **url** – URL of remote repository.
- **target** – Target folder.
- **args** – Extra arguments to pass to the git clone as a list.
- **hide_url** – Hides the URL from the log output to prevent accidental credential leaks. Can be disabled by passing `False`.

`fetch_commit(url, commit, hide_url=True)`

Experimental: does a single commit fetch and checkout, instead of a full clone, should be faster.

Parameters

- **url** – URL of remote repository.
- **commit** – The commit ref to checkout.
- **hide_url** – Hides the URL from the log output to prevent accidental credential leaks. Can be disabled by passing `False`.

`checkout(commit)`

Checkouts the given commit using `git checkout <commit>`.

Parameters
commit – Commit to checkout.

`included_files()`

Run `git ls-files --full-name --others --cached --exclude-standard` to the get the list of files not ignored by `.gitignore`

Returns
List of files.

`coordinates_to_conandata(repository=False)`

Capture the "url" and "commit" from the Git repo, calling `get_url_and_commit()`, and then store those in the `conandata.yml` under the "scm" key. This information can be used later to clone and checkout the exact source point that was used to create this package, and can be useful even if the recipe uses `exports_sources` as mechanism to embed the sources.

Parameters
repository – By default gets the commit of the defined folder, use repository=True to get the commit of the repository instead.

checkout_from_conandata_coordinates()

Reads the "scm" field from the `conandata.yml`, that must contain at least "url" and "commit" and then do a `clone(url, target=".")`, `fetch <commit>`, followed by a `checkout(commit)`.

Version

This is a helper class to work with versions, it splits the version string based on dots and hyphens. It exposes all the version components as properties and offers total ordering through compare operators.

Note: This is the reference for `Version`. For more detailed explanations and usage, visit the section about *semantic versionining in the tutorial*

Listing 169: Comparing versions

```
compiler_lower_than_12 = Version(self.settings.compiler.version) < "12.0"

is_legacy = Version(self.version) < 2
```

class Version(*value*, *qualifier=False*)

This is NOT an implementation of semver, as users may use any pattern in their versions. It is just a helper to parse "." or "-" and compare taking into account integers when possible

Attributes

The `Version` class offers ways to access the different parts of the version number:

main

Get all the main digits.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert [str(i) for i in v.main] == ['1', '2', '3', '4', '5']
```

major

Get the major digit.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert str(v.major) == "1"
```

minor

Get the minor digit.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert str(v.minor) == "2"
```

patch

Get the patch digit.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert str(v.patch) == "3"
```

micro

Get the micro digit.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert str(v.micro) == "4"
```

pre

Get the pre-release digit.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert str(v.pre) == "alpha.3"
```

build

Get the build digit.

```
v = Version("1.2.3.4-alpha.3+b.1")
assert str(v.build) == "b.1"
```

Note: If any field in the Version contains letters, it will be treated alphabetically, so the relative order of numbers within those field will not be correct. Make sure to separate with dots the digits, like `alpha.3` and `build.1` instead of `alpha3` and `build1` for correct ordering.

Methods

The `Version` class implements the following methods:

in_range

Check if the version is in the specified range.

```
assert Version("1.0").in_range(">=1.0 <2")
assert not Version("1.0").in_range(">1.0 <2")

assert not Version("1.0-rc").in_range(">=1.0 <2.0")
assert Version("1.0-rc").in_range(">=1.0 <2.0", resolve_prerelease=True)
```

9.10.18 conan.tools.scons

SConsDeps

The `SConsDeps` is the dependency generator for SCons. It will generate a *SConscript_conandeps* file containing the necessary information for SCons to build against the desired dependencies.

The `SConsDeps` generator can be used by name in conanfiles:

Listing 170: conanfile.py

```
from conan import ConanFile

class Pkg(ConanFile):
    generators = "SConsDeps"
```

Listing 171: conanfile.txt

```
[generators]
SConsDeps
```

It can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.scons import SConsDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        tc = SConsDeps(self)
        tc.generate()
```

After executing the `conan install` command, the `SConsDeps` generator will create the *SConscript_conandeps* file. This file will provide the following information for *SCons*: `CPPPATH`, `LIBPATH`, `BINPATH`, `LIBS`, `FRAMEWORKS`, `FRAMEWORKPATH`, `CPPDEFINES`, `CXXFLAGS`, `CCFLAGS`, `SHLINKFLAGS`, and `LINKFLAGS`. This information is generated for the accumulated list of all dependencies and also for each one of the requirements. You can load it in your consumer *SConscript* like this:

Listing 172: consumer *SConscript*

```
...
info = SConscript('./SConscript_conandeps')
# You can use conandeps to get the information
# for all the dependencies.
flags = info["conandeps"]

# Or use the name of the requirement if
# you only want the information about that one.
flags = info["zlib"]

env.MergeFlags(flags)
...
```

9.10.19 conan.tools.premake

PremakeDeps

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `PremakeDeps` is the dependencies generator for Premake. The generator can be used by name in conanfiles:

Listing 173: conanfile.py

```
class Pkg(ConanFile):
    generators = "PremakeDeps"
```

Listing 174: conanfile.txt

```
[generators]
PremakeDeps
```

And it can also be fully instantiated in the conanfile `generate()` method:

```
from conan import ConanFile
from conan.tools.premake import PremakeDeps

class App(ConanFile):
    settings = "os", "arch", "compiler", "build_type"
    requires = "zlib/1.3.1"

    def generate(self):
        deps = PremakeDeps(self)
        deps.generate()
```

Important: The `PremakeDeps` generator must be used in conjunction with the *PremakeToolchain* generator, as it will generate a `include('conandeps.premake5.lua')` that will be automatically included by the toolchain.

Generated files

PremakeDeps will generate a conandeps.premake5.lua script file which will be injected later by the toolchain and the following files per dependency in the conanfile.generators_folder:

- conan_<pkg>.premake5.lua: will be including the proper script depending on the build_type and architecture.
- conan_<pkg>_vars_<config>.premake5.lua: will contain essentially the following information for the specific dependency, architecture and build_type:
  - includedirs
  - libdirs
  - bindirs
  - sysincludedirs
  - frameworkdirs
  - frameworks
  - libs
  - syslibs
  - defines
  - cxxflags
  - cflags
  - sharedlinkflags
  - exelinkflags

All this information will be loaded in the conandeps.premake5.lua script and injected later to the main premake script, allowing a transparent and easy to use dependency management with conan.

PremakeToolchain

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Important: This class will generate files that are only compatible with Premake versions >= 5.0.0

The PremakeToolchain generator can be used by name in conanfiles:

Listing 175: conanfile.py

```
class Pkg(ConanFile):
    generators = "PremakeToolchain"
```

Listing 176: conanfile.txt

```
[generators]
PremakeToolchain
```

And it can also be fully instantiated in the conanfile `generate()` method:

Usage Example:

```
from conan.tools.premake import PremakeToolchain

class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch"

    def generate(self):
        tc = PremakeToolchain(self)
        tc.extra_defines = ["VALUE=2"]                      # Add define to main premake␣
↪workspace
        tc.extra_cflags = ["-Wextra"]                       # Add cflags to main premake␣
↪workspace
        tc.extra_cxxflags = ["-Wall", "-Wextra"]            # Add cxxflags to main␣
↪premake workspace
        tc.extra_ldflags = ["-lm"]                          # Add ldflags to main␣
↪premake workspace
        tc.project("main").extra_defines = ["TEST=False"]   # Add define to "main"␣
↪project (overriding possible value)
        tc.project("main").extra_cxxflags = ["-FS"]         # Add cxxflags to "main"␣
↪project
        tc.project("test").disable = True                   # A way of disabling "test"␣
↪project compilation
        tc.project("foo").kind = "StaticLib"                # Override library type of
↪"foo"
        tc.generate()
```

Generated files

The `PremakeToolchain` generates `conantoolchain.premake5.lua` file after a **`conan install`** (or when building the package in the cache) with the information provided in the `generate()` method as well as information translated from the current `settings`, `conf`, etc.

Premake does not expose any kind of API to interact inject/modify the build scripts. Furthermore, premake does not have a concept of toolchain so following premake maintainers instructions, (see premake discussion) as premake is built in top of Lua scripts, Conan generated file is a Lua script that will override the original premake script adding the following information:

- Detection of `buildtype` from Conan settings.
- Definition of the C++ standard as necessary.
- Definition of the C standard as necessary.
- Detection of the premake `action` based on conan profile and OS.
- Definition of the build folder in order to avoid default premake behavior of creating build folder and object files in the source repository (which goes against conan good practices).

- Definition of compiler and linker flags and defines based on user configuration values.
- Definition of proper target architecture when cross building.
- Definition of `fPIC` flag based on conan options.
- Based on `shared` conan option, `PremakeToolchain` will set the `kind` of the `workspace` to `SharedLib` or `StaticLib`.

Note: `PremakeToolchain` is not able to override the `kind` of a project if that project has already define the `kind` attribute (typical case). It can only override top-level `workspace.kind`, which will only affect projects without a defined `kind`.

Recomendation: as premake does not offer any mechanism like CMake's BUILD_SHARED_LIBS to externally manage the creation type of libraries, it is recommended while using conan to **AVOID** defining the `kind` attribute on library project.

Reference

class PremakeToolchain(*conanfile*)

PremakeToolchain generator

Parameters
conanfile – < `ConanFile object` > The current recipe object. Always use `self`.

extra_cxxflags

List of extra `CXX` flags. Added to `buildoptions`.

extra_cflags

List of extra `C` flags. Added to `buildoptions`.

extra_ldflags

List of extra linker flags. Added to `linkoptions`.

extra_defines

List of extra preprocessor definitions. Added to `defines`.

project(*project_name*)

The returned object will also have the same properties as the workspace but will only affect the project with the name. :param project_name: The name of the project inside the workspace to be updated. :return: <`PremakeProject`> object which allow to set project specific flags.

generate()

Creates a `conantoolchain.premake5.lua` file which will properly configure build paths, binary paths, configuration settings and compiler/linker flags based on toolchain configuration.

Premake

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

The `Premake` build helper is a wrapper around the command line invocation of Premake. It will abstract the project configuration and build command.

The helper is intended to be used in the *conanfile.py* `build()` method, to call Premake commands automatically when a package is being built directly by Conan (create, install)

Usage Example:

```
from conan.tools.premake import Premake

class Pkg(ConanFile):
    settings = "os", "compiler", "build_type", "arch"

    # The PremakeToolchain generator is always needed to use premake helper
    generators = "PremakeToolchain"

    def build(self):
        p = Premake(self)

        # Set the main Lua configuration file (default: premake5.lua)
        p.luafile = "myproject.lua"

        # Pass custom arguments to Premake (translates to --{key}={value})
        p.arguments["myarg"] = "myvalue"

        # Automatically determines the correct action:
        # - For MSVC, selects vs<version> based on the compiler version
        # - Defaults to "gmake" for other compilers
        # p.configure() will run: premake5 --file=myproject.lua <action> --{key}={value}␣
↪...
        p.configure()
        # p.build() will invoke proper compiler depending on action (automatically␣
↪detected by profile)
        p.build("HelloWorld.sln")
```

Reference

class Premake(*conanfile*)

This class calls Premake commands when a package is being built. Notice that this one should be used together with the `PremakeToolchain` generator.

This premake generator is only compatible with `premake5`.

Parameters
conanfile – < `ConanFile object` > The current recipe object. Always use `self`.

luafile

Path to the root premake5 lua file (default is `premake5.lua`)

arguments

Key value pairs. Will translate to "–{key}={value}"

configure()

Runs `premake5 <action> [FILE]` which will generate respective build scripts depending on the `action`.

build(*workspace*, *targets=None*, *configuration=None*, *msbuild_platform=None*)

Depending on the action, this method will run either `msbuild` or `make` with `N_JOBS`. You can specify `N_JOBS` through the configuration line `tools.build:jobs=N_JOBS` in your profile `[conf]` section.

Parameters

- **workspace** – `str` Specifies the solution to be compiled (only used by `MSBuild`).
- **targets** – `List[str]` Declare the projects to be built (None to build all projects).
- **configuration** – `str` Specify the configuration build type, default to build_type ("Release" or "Debug"), but this allow setting custom configuration type.
- **msbuild_platform** – `str` Specify the platform for the internal MSBuild generator (only used by `MSBuild`).

conf

The `Premake` build helper is affected by these `[conf]` variables:

- `tools.build:verbosity` which accepts one of `quiet` or `verbose` and sets the `--quiet` flag in `Premake.configure()`
- `tools.compilation:verbosity` which accepts one of `quiet` or `verbose` and sets the `--verbose` flag in `Premake.build()`

Extra configuration

By default, typical Premake configurations are `Release` and `Debug`. This configurations could vary depending on the used Premake script.

For example,

```
workspace "MyProject"
    configurations { "Debug", "Release", "DebugDLL", "ReleaseDLL" }
```

If you wish to use a different configuration than `Release` or `Debug`, you can override the configuration from the `Premake` generator.

If the project also have dependencies, you will also need to override the `configuration` property of the `PremakeDeps` generator accordingly, with the same value.

```
class MyRecipe(Conanfile):
    ...
    def _premake_configuration(self):
        return str(self.settings.build_type) + ("DLL" if self.options.shared else "")

    def generate(self):
        deps = PremakeDeps(self)
```

(continues on next page)

(continued from previous page)

```
        deps.configuration = self._premake_configuration
        deps.generate()
        tc = PremakeToolchain(self)
        tc.generate()

    def build(self):
        premake = Premake(self)
        premake.configure()
        premake.build(workspace="MyProject", configuration=self._premake_configuration)
```

9.10.20 conan.tools.system

conan.tools.system.package_manager

The tools under *conan.tools.system.package_manager* are wrappers around some of the most popular system package managers for different platforms. You can use them to invoke system package managers in recipes and perform the most typical operations, like installing a package, updating the package manager database or checking if a package is installed. By default, when you invoke them they will not try to install anything on the system, to change this behavior you can set the value of the `tools.system.package_manager:mode` *configuration*.

You can use these tools inside the `system_requirements()` method of your recipe, like:

Listing 177: conanfile.py

```
from conan.tools.system.package_manager import Apt, Yum, PacMan, Zypper

def system_requirements(self):
    # depending on the platform or the tools.system.package_manager:tool configuration
    # only one of these will be executed
    Apt(self).install(["libgl-dev"])
    Yum(self).install(["libglvnd-devel"])
    PacMan(self).install(["libglvnd"])
    Zypper(self).install(["Mesa-libGL-devel"])
```

Conan will automatically choose which package manager to use by looking at the Operating System name. In the example above, if we are running on Ubuntu Linux, Conan will ignore all the calls except for the `Apt()` one and will only try to install the packages using the `apt-get` tool.

The `package_manager` tool allows you to specify the package version of the system package to be installed using the format `<package-name>=<package-version>`, for example `Apt(self).install(["libgl-dev=0.0.1"])`. The system package managers that support versions are `Apt`, `Yum`, `Dnf`, `Brew`, `Pkg`, `Chocolatey`, `Apk` and `Zypper`. On systems where package versioning is not supported by the package manager, in our case `PacMan` and `PkgUtil`, the provided version will be ignored.

Conan uses the following mapping by default:

- *Apt* for **Linux** with distribution names: *ubuntu*, *debian*, *raspbian* or *linuxmint*
- *Yum* for **Linux** with distribution names: *pidora*, *scientific*, *xenserver*, *amazon*, *oracle*, *amzn*, *almalinux* or *rocky*
- *Dnf* for **Linux** with distribution names: *fedora*, *rhel*, *centos*, *mageia*
- *Apk* for **Linux** with distribution names: *alpine*
- *Brew* for **macOS**

- *PacMan* for **Linux** with distribution names: *arch*, *manjaro* and when using **Windows** with *msys2*
- *Chocolatey* for **Windows**
- *Zypper* for **Linux** with distribution names: *opensuse*, *sles*
- *Pkg* for **FreeBSD**
- *PkgUtil* for **Solaris**

You can override this default mapping and set the package manager tool you want to use by default setting the configuration property *tools.system.package_manager:tool*.

Methods available for system package manager tools

All these wrappers share three methods that represent the most common operations with a system package manager. They take the same form for all of the package managers except for *Apt* that also accepts the *recommends* argument for the *install method*.

- `install(self, packages, update=False, check=True, host_package=True)`: try to install the list of packages passed as a parameter. If the parameter `check` is `True` it will check if those packages are already installed before installing them. If the parameter `update` is `True` it will try to update the package manager database before checking and installing. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*. If the parameter `host_package` is `True` it will install the packages for the host machine architecture (the machine that will run the software), it has an effect when cross building. When `host_package` is `False`, packages are installed for the build machine. In both cases you can include an explicit architecture suffix in the package name (see *Explicit architecture suffixes*). This method will return the return code of the executed commands.
- `install_substitutes(packages_substitutes, update=False, check=True)`: try to install one of the lists of substitutes packages passed as a parameter, e.g., `install_substitutes(["pkg1", "pkg2"], ["pkg3"])`. It succeeds if one of the substitutes list is completely installed, so it's intended to be used when you have different packages for different distros. In this example, it will first try to install `pkg1` and `pkg2`, if it succeeds to install both packages, it will stop. If it fails, it will proceed to the next list and try to install `pkg3`. Internally, it's calling the previous `install(packages, update=update, check=check)` method, so `update` and `check` have the same purpose as above.
- `update()` update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.
- `check(packages)` check if the list of packages passed as parameter are already installed. It will return a list with the packages that are missing.

Configuration properties that affect how system package managers are invoked

As explained above there are several `[conf]` that affect how these tools are invoked:

- `tools.system.package_manager:tool`: to choose which package manager tool you want to use by default: `"apk"`, `"apt-get"`, `"yum"`, `"dnf"`, `"brew"`, `"pacman"`, `"choco"`, `"zypper"`, `"pkg"` or `"pkgutil"`
- `tools.system.package_manager:mode`: mode to use when invoking the package manager tool. There are two possible values:
  - `"check"`: it will just check for missing packages at most and will not try to update the package manager database or install any packages in any case. It will raise an error if required packages are not installed in the system. This is the default value.

  - `"report"`: Just capture the `.install()` calls to capture packages, but do not check nor install them. Never raises an error. Mostly useful for `conan graph info` commands.
  - `"report-installed"`: Report, without failing which packages are needed (same as `report`) and also check which of them are actually installed in the current system.
  - `"install"`: it will allow Conan to perform update or install operations.
- `tools.system.package_manager:sudo`: Use *sudo* when invoking the package manager tools in Linux (`False` by default)
- `tools.system.package_manager:sudo_askpass`: Use the `-A` argument if using sudo in Linux to invoke the system package manager (`False` by default)

There are some specific arguments for each of these tools. Here is the complete reference:

conan.tools.system.package_manager.Apk

Will invoke the *apk* command. Enabled by default for **Linux** with distribution names: *alpine*.

Reference

class Apk(*conanfile*, *_arch_names=None*)

Constructor method. Note that *Apk* does not support architecture names since Alpine Linux does not support multiarch. Therefore, the `arch_names` argument is ignored.

Parameters
conanfile – the current recipe object. Always use `self`.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

Alpine Linux does not support multiple architectures in the same repository, so there is no mapping from Conan architectures to Alpine architectures.

conan.tools.system.package_manager.Apt

Will invoke the *apt-get* command. Enabled by default for **Linux** with distribution names: *ubuntu*, *debian*, *raspbian* and *linuxmint*.

Reference

class Apt(*conanfile*, *arch_names=None*)

Parameters

- **conanfile** – The current recipe object. Always use `self`.
- **arch_names** – This argument maps the Conan architecture setting with the package manager tool architecture names. It is `None` by default, which means that it will use a default mapping for the most common architectures. For example, if you are using `x86_64` Conan architecture setting, it will map this value to `amd64` for *Apt* and try to install the `<package_name>:amd64` package.

install(*packages*, *update=False*, *check=True*, *recommends=False*, *host_package=True*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.
- **host_package** – install the packages for the host machine architecture (the machine that will run the software), it has an effect when cross building.

- **recommends** – if the parameter `recommends` is `False` it will add the `'--no-install-recommends'` argument to the *apt-get* command call.

Returns
the return code of the executed apt command.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

You can pass the `arch_names` argument to override the default Conan mapping like this:

Listing 178: conanfile.py

```
...
def system_requirements(self):
    apt = Apt(self, arch_names={"<conan_arch_setting>": "apt_arch_setting"})
    apt.install(["libgl-dev"])
```

The default mapping that Conan uses for *APT* packages architecture is:

```
self._arch_names = {"x86_64": "amd64",
                    "x86": "i386",
                    "ppc32": "powerpc",
                    "ppc64le": "ppc64el",
```

(continues on next page)

(continued from previous page)

```
                    "armv7": "arm",
                    "armv7hf": "armhf",
                    "armv8": "arm64",
                    "s390x": "s390x"} if arch_names is None else arch_names
```

Explicit architecture suffixes

When cross-building, Conan can automatically append the architecture suffix to package names based on the Conan `settings`. You can also include the architecture directly in the package name. This is useful when you need packages for a foreign architecture on the native build machine — for example, 32-bit libraries on an `x86_64` system to run 32-bit build tools — using `host_package=False`:

Listing 179: conanfile.py

```
def system_requirements(self):
    apt = Apt(self)
    apt.install(["libc6:i386", "libstdc++6:i386"], check=True, host_package=False)
```

Conan parses the `:` suffix and uses it for both `check()` and `install()`, so already installed packages are detected correctly. Version constraints are also supported, for example `libc6:i386=2.39-0ubuntu8.7`.

An explicit suffix in the package name takes precedence over the automatic mapping described above.

conan.tools.system.package_manager.Yum

Will invoke the *yum* command. Enabled by default for **Linux** with distribution names: *pidora*, *scientific*, *xenserver*, *amazon*, *oracle*, *amzn* and *almalinux*.

Reference

class Yum(*conanfile*, *arch_names=None*)

Parameters

- **conanfile** – the current recipe object. Always use `self`.
- **arch_names** – this argument maps the Conan architecture setting with the package manager tool architecture names. It is `None` by default, which means that it will use a default mapping for the most common architectures. For example, if you are using `x86` Conan architecture setting, it will map this value to `i?86` for *Yum* and try to install the `<package_name>.i?86` package.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns

the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns

the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns

the return code of the executed package manager update command.

The default mapping Conan uses for *Yum* packages architecture is:

```
self._arch_names = {"x86_64": "x86_64",
                    "x86": "i?86",
                    "ppc32": "powerpc",
                    "ppc64le": "ppc64le",
                    "armv7": "armv7",
                    "armv7hf": "armv7hl",
                    "armv8": "aarch64",
                    "s390x": "s390x"} if arch_names is None else arch_names
```

Yum and *Dnf* also support *explicit architecture suffixes* using `.` as separator (for example `glibc.i686`). The name is only split when the suffix matches a known architecture from the mapping above, so package names that contain dots (such as `libfoo.bar`) are not affected.

conan.tools.system.package_manager.Dnf

Will invoke the *dnf* command. Enabled by default for **Linux** with distribution names: *fedora*, *rhel*, *centos* and *mageia*. This tool has exactly the same default values, constructor and methods than the *Yum* tool.

conan.tools.system.package_manager.PacMan

Will invoke the *pacman* command. Enabled by default for **Linux** with distribution names: *arch*, *manjaro* and when using **Windows** with *msys2*

Reference

class PacMan(*conanfile*, *arch_names=None*)

Parameters

- **conanfile** – the current recipe object. Always use `self`.
- **arch_names** – this argument maps the Conan architecture setting with the package manager tool architecture names. It is `None` by default, which means that it will use a default mapping for the most common architectures. If you are using `x86` Conan architecture setting, it will map this value to `lib32` for *PacMan* and try to install the `<package_name>-lib32` package.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

The default mapping Conan uses for *PacMan* packages architecture is:

```
self._arch_names = {"x86": "lib32"} if arch_names is None else arch_names
```

conan.tools.system.package_manager.Zypper

Will invoke the *zypper* command. Enabled by default for **Linux** with distribution names: *opensuse*, *sles*.

Reference

class Zypper(*conanfile*)

Parameters
conanfile – The current recipe object. Always use `self`.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

`install_substitutes`(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns

the return code of the executed package manager command.

`update`(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns

the return code of the executed package manager update command.

conan.tools.system.package_manager.Brew

Will invoke the *brew* command. Enabled by default for **macOS**.

Reference

`class Brew`(*conanfile*)

Parameters

conanfile – The current recipe object. Always use `self`.

`check`(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters

packages – list of packages to check.

Returns

list of packages from the packages argument that are not installed in the system.

`install`(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.

- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

conan.tools.system.package_manager.Pkg

Will invoke the *pkg* command. Enabled by default for **Linux** with distribution names: *freebsd*.

Reference

class Pkg(*conanfile*)

Parameters
conanfile – The current recipe object. Always use `self`.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

conan.tools.system.package_manager.PkgUtil

Will invoke the *pkgutil* command. Enabled by default for **Solaris**.

Reference

class PkgUtil(*conanfile*)

Parameters
conanfile – The current recipe object. Always use `self`.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

conan.tools.system.package_manager.Chocolatey

Will invoke the *choco* command. Enabled by default for **Windows**.

Reference

class Chocolatey(*conanfile*)

Parameters
conanfile – The current recipe object. Always use `self`.

check(**args*, ***kwargs*)

Check if the list of packages passed as parameter are already installed.

Parameters
packages – list of packages to check.

Returns
list of packages from the packages argument that are not installed in the system.

install(**args*, ***kwargs*)

Will try to install the list of packages passed as a parameter. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Parameters

- **packages** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

install_substitutes(**args*, ***kwargs*)

Will try to call the install() method with several lists of packages passed as a variable number of parameters. This is useful if, for example, the names of the packages are different from one distro or distro version to another. For example, `libxcb` for `Apt` is named `libxcb-util-dev` in Ubuntu >= 15.0 and `libxcb-util0-dev` for other versions. You can call to:

```
# will install the first list of packages that succeeds in the␣
↪installation
Apt.install_substitutes(["libxcb-util-dev"], ["libxcb-util0-dev"])
```

Parameters

- **packages_alternatives** – try to install the list of packages passed as a parameter.
- **update** – try to update the package manager database before checking and installing.
- **check** – check if the packages are already installed before installing them.

Returns
the return code of the executed package manager command.

update(**args*, ***kwargs*)

Update the system package manager database. Its behaviour is affected by the value of `tools.system.package_manager:mode` *configuration*.

Returns
the return code of the executed package manager update command.

PyEnv

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Important: This is **only** for: - **Executable Python packages and its Python dependencies.** (for example, meson build system) - **Python package used during the source build process.** (for example, html5lib as a build dependency) This approach doesn't work for Python library packages that you would typically use via `import` inside your recipe.

The `PyEnv` helper installs executable Python packages with **pip** inside a dedicated virtual environment (**venv**), keeping them isolated so they don't interfere with system packages or the Conan package itself. It is designed to use a Python CLI tool inside a recipe during the build step.

By default, it attempts to create the virtualenv using the Python you have set on your system Path. To use a different one, you can set a Python path in the `tools.system.pyenv:python_interpreter` *configuration*.

`class PyEnv(`*conanfile*, *folder=None*, *name=''*, *py_version=None*`)`

Parameters

- **conanfile** – The current conanfile `self`
- **folder** – Optional folder, by default the `build_folder`
- **name** – Optional name for the virtualenv, by default `conan_pyenv`
- **py_version** – Optional python version to create the virtualenv using UV

`property env_dir`

Root directory of the virtual environment.

`property env_exe`

Path to the Python executable inside the virtual environment.

`property bin_path`

Path to the bin or Scripts directory inside the virtual environment.

`generate()`

Create a conan environment to use the python venv in the next steps of the conanfile.

`install(`*packages*, *pip_args=None*`)`

Will try to install the list of pip packages passed as a parameter.

Parameters

- **packages** – try to install the list of pip packages passed as a parameter.
- **pip_args** – additional argument list to be passed to the 'pip install' command, e.g.: ['–no-cache-dir', '–index-url', 'https://my.pypi.org/simple']. Defaults to `None`.

Returns

the return code of the executed pip command.

Using a Python package in a recipe

To install a python package or use a tool installed with Python, we have to install it using the `PyEnv.install()` method.

When the `py_version` parameter is defined, Conan will automatically use UV to create and manage a temporary virtual environment with that specific Python version. **UV requires Python 3.8 or higher**, make sure you use a compatible Python version if you want to define the `py_version`.

We also have to call the `PyEnv.generate()` method to create a **Conan Environment** that adds the **Python virtualenv path** to the system path.

These two steps appear in the following recipe in the `generate()` method. Calling it in this method ensures that the **Python package** and the **Conan Environment** will be available in the following steps. In this case, in the build step, which is where we will use the installed tool.

Listing 180: conanfile.py

```
from conan import ConanFile
from conan.tools.system import PyEnv
from conan.tools.layout import basic_layout


class PipPackage(ConanFile):
    name = "pip_install"
    version = "0.1"

    def layout(self):
        basic_layout(self)

    def generate(self):
        pyenv = PyEnv(self)
        pyenv.install(["meson==1.9.1"])
        pyenv.generate()

    def build(self):
        self.run("meson --version")
```

If we run a `conan build` we can see how our Python package is installed when the generate step, and how it is called in the build step as if it were installed on the system.

```
$ conan build .

...

======== Finalizing install (deploy, generators) ========
conanfile.py (pip_install/0.1): Calling generate()
conanfile.py (pip_install/0.1): Generators folder: /Users/user/pip_install/build/conan
conanfile.py (pip_install/0.1): RUN: /Users/user/pip_install/build/pip_venv_pip_install/
↪bin/python -m pip install --disable-pip-version-check meson==1.9.1
Collecting meson==1.9.1
  Using cached meson-1.9.1-py3-none-any.whl.metadata (1.8 kB)
Using cached meson-1.9.1-py3-none-any.whl (1.0 MB)
Installing collected packages: meson
Successfully installed meson-1.9.1
```

(continues on next page)

(continued from previous page)

```
conanfile.py (pip_install/0.1): Generating aggregated env files
conanfile.py (pip_install/0.1): Generated aggregated env files: ['conanbuild.sh',
↪'conanrun.sh']

======== Calling build() ========
conanfile.py (pip_install/0.1): Calling build()
conanfile.py (pip_install/0.1): RUN: meson --version
1.9.1
```

Using the PyEnv Python from CMake

Calling `PyEnv.generate()` adds the **bin** (or **Scripts** on Windows) folder of the virtual environment to the `PATH` of the Conan Environment. This is enough to run the installed tools from a `self.run()` call, but it is **not always enough** for CMake's `find_package(Python)`:

- CMake's FindPython does not always prioritize the `python` found in the `PATH`, so it may prefer a different Python installation.
- The `VirtualBuildEnv` is generated automatically by Conan *after* the `generate()` method. If the profile modifies the `PATH` with something like `[buildenv] PATH=+(path)C:/Python313`, that folder will have a higher priority than the virtual environment. To avoid it, generate the `VirtualBuildEnv` explicitly **before** calling `PyEnv.generate()`.

To make sure that CMake uses the Python from the virtual environment, pass its path explicitly with *CMakeToolchain*, using the `env_exe` attribute of `PyEnv`:

Listing 181: conanfile.py

```
from conan import ConanFile
from conan.tools.cmake import CMakeToolchain
from conan.tools.env import VirtualBuildEnv
from conan.tools.system import PyEnv


class Pkg(ConanFile):
    settings = "os", "arch", "compiler", "build_type"

    def generate(self):
        # Generate it first, so the PyEnv path has priority over the profile [buildenv]␣
↪PATH
        buildenv = VirtualBuildEnv(self)
        buildenv.generate()

        pyenv = PyEnv(self)
        pyenv.install(["html5lib~=1.0"])
        pyenv.generate()

        tc = CMakeToolchain(self)
        tc.cache_variables["Python_EXECUTABLE"] = pyenv.env_exe
        tc.generate()
```

Alternatively, `Python_ROOT_DIR` can be set to `pyenv.env_dir`, the root folder of the virtual environment.

9.11 Runners

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Runners provide a seamless method to execute Conan on remote build environments like Docker ones, directly from your local setup by simply configuring your host profile.

- Installing a version of Conan with runner dependencies `pip install conan[runners]`.
- Install the tools to run each of the runners (`docker`).
- Add the `[runner]` section defined in the documentation of each runner to the host profile.

Runners:

9.11.1 Docker runner

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

How to use a docker runner

To run Conan inside a Docker container, a `[runner]` section must be defined in the host profile with the following fields:

- `type` **(mandatory)**: Specifies the runner to use, in this case, `docker`.
- `dockerfile` **(optional, default: None)**: Absolute path to a Dockerfile, if a Docker image needs to be built.
- `image` **(optional, default: conan-runner-default)**: Name of the Docker image to download from a registry or the name of the locally built image if a Dockerfile path is provided.
- `cache` **(optional, default: clean)**: Determines how the Docker container interacts with the host's Conan cache.
  - `clean`: Uses an empty cache.
  - `copy`: Copies the host cache into the container using the *conan cache save/restore* command.
  - `shared`: Mounts the host's Conan cache as a shared volume.
- `remove` **(optional, default: false)**: Specifies whether to remove the container after executing the Conan command (true or false).
- `configfile` **(optional, default: None)**: Absolute path to a configuration file with additional parameters (see **extra configuration** section for details).
- `build_context` **(optional, default: None)**: Defines the Docker build context (see **extra configuration** section for details).
- `platform` **(optional, default: None)**: Specifies the platform for building the container (e.g., `linux/amd64`). This is particularly useful for Mac Silicon users.
- `copy_config_files` **(optional, default: empty)**: Comma-separated list of fnmatch patterns, relative to the Conan home folder, of extra host files/folders to copy into the container's Conan home, for the `clean` and `copy` cache modes. Disabled by default.

```
[runner]
type=docker
copy_config_files=extensions/*
```

A few files are never copied, even if a pattern matches them: the package cache storage folder, the sources backup cache, `version.txt`, `.local_recipes_index/*`, and the local remote-login database. `credentials.json` and `source_credentials.json` are also skipped by default for containing sensitive data, but are copied if their exact name is listed in `copy_config_files`.

Note:

- The `shared` cache option may cause permission issues depending on the user inside and outside the container. It is recommended to use the `copy` cache for greater stability, despite a slight increase in setup time.
- The runner profile section does not impact the package ID.

Extra configuration

For greater control over the build and execution of the container, additional parameters can be defined within a `configfile` YAML file.

```
image: image_name # The image to build or run.
build:
    dockerfile: /dockerfile/path # Dockerfile path.
    build_context: /build/context/path # Path within the build context to the Dockerfile.
    build_args: # A dictionary of build arguments
        foo: bar
    cacheFrom: # A list of images used for build cache resolution
        - image_1
run:
    name: container_name # The name for this container.
    containerEnv: # Environment variables to set inside the container.
        env_var_1: env_value
    containerUser: user_name # Username or UID to run commands as inside the container.
    privileged: False # Run as privileged
    capAdd: # Add kernel capabilities.
        - SYS_ADMIN
        - MKNOD
    securityOpt: # A list of string values to customize labels for MLS systems, such as␣
↪SELinux.
        - opt_1
    mounts: # A dictionary to configure volumes mounted inside the container.
        /home/user1/: # The host path or a volume name
            bind: /mnt/vol2 # The path to mount the volume inside the container
            mode: rw # rw to mount the volume read/write, or ro to mount it read-only.
    network: my-network # Specifies the network for the container.
```

How to run a *conan create* in a runner

Note: The docker runner feature is only supported by `conan create` command. The `conan install --build` command is not supported.

The following links provide examples of how to use a Conan Docker runner:

- *Creating a Conan package using a Docker runner*
- *Using a docker runner configfile to parameterize the Dockerfile base image*

9.12 Workspace

Warning: This feature is experimental and subject to breaking changes. See *the Conan stability* section for more information.

Workspaces are defined by the `conanws.yml` and/or `conanws.py` files that will define the "root" workspace folder.

9.12.1 conanws.yml

The most basic implementation of a workspace is a `conanws.yml` file. It defines the workspace's `packages` (editable packages). For instance, a workspace `conanws.yml` defining 2 `packages` could be:

Listing 182: conanws.yml

```
packages:
   - path: dep1
     ref: dep1/0.1
   - path: dep2
     ref: dep2/0.1
```

Moreover, it could not have the `ref` field, and let Conan read the *name/version* from the respective *path/conanfile.py*:

Listing 183: conanws.yml

```
packages:
   - path: dep1
   - path: dep2
```

Warning: Support for `python_requires` in a workspace is limited and highly **experimental**.

Discovering the workspace packages does not resolve `python_requires`, as remotes and the Conan cache are not necessarily available at that point. If a package's `name/version` is declared directly (or via its own `set_name()/set_version()`), it works out of the box. If it is inherited from a `python_requires`, it cannot be deduced automatically, and a `get_ref()` method must be defined in `conanws.py` (see below).

Python-requires are also expected to declare `package_type = "python-require"` directly as a class attribute, not inherited, so they can be filtered out from the workspace packages.

Note: Best practices

Workspaces are in general not expected to contain `python-require` recipes. The purpose of a workspace is to be able to develop the C/C++ code of several packages simultaneously in a convenient way. The recipes should be rarely modified in this flow, and it is not expected that the common Python code that lives in a `python-require` needs to be modified by developers in a Workspace together with the C/C++ code.

By the time a developer is putting several different packages into a Workspace to work on them, the common recipe logic living in a `python-require` is expected to be read-only, not editable, and managed as an external dependency, not an editable one within the workspace.

A `python-require` might still be added to a workspace, with some limitations and caveats, but still the intention is that this is a very exceptional scenario, and not something that regular developers would do.

9.12.2 conanws.py

A `conanws.yml` can be extended with a way more powerful `conanws.py` that follows the same relationship as a `ConanFile` does with its `conandata.yml`. If we want to dynamically define the `packages`, for example based on the existence of some `name.txt` and `version.txt` files in folders, the packages could be defined in `conanws.py` as:

Listing 184: conanws.py

```
import os
from conan import Workspace

class MyWorkspace(Workspace):

   def packages(self):
      result = []
      for f in os.listdir(self.folder):
         if os.path.isdir(os.path.join(self.folder, f)):
            with open(os.path.join(self.folder, f, "name.txt")) as fname:
               name = fname.read().strip()
            with open(os.path.join(self.folder, f, "version.txt")) as fversion:
               version = fversion.read().strip()
            result.append({"path": f, "ref": f"{name}/{version}"})
      return result
```

It is also possible to re-use the `conanfile.py` logic in `set_name()` and `set_version()` methods, using the `Workspace.load_conanfile()` helper:

Listing 185: conanws.py

```
import os
from conan import Workspace

class MyWorkspace(Workspace):
   def packages(self):
      result = []
      for f in os.listdir(self.folder):
         if os.path.isdir(os.path.join(self.folder, f)):
            conanfile = self.load_conanfile(f)
```

(continues on next page)

(continued from previous page)

```
            result.append({"path": f, "ref": f"{conanfile.name}/{conanfile.version}"})
    return result
```

get_ref()

If a package's `name/version` cannot be deduced because it is inherited from a `python_requires` (which is not resolved while discovering the workspace packages), `conanws.py` can define a `get_ref(folder)` method to provide the reference explicitly:

Listing 186: conanws.py

```
from conan import Workspace

class MyWorkspace(Workspace):
   def get_ref(self, folder):
      return {"pkga": "pkga/1.0", "pkgb": "pkgb/2.0"}.get(folder)
```

`get_ref()` can return a `name/version[@user/channel]` string, a `RecipeReference`, or `None` if it doesn't apply to that `folder`. When the reference for a package cannot be deduced by any of these means (`ref` in `conanws.yml`, `name/version` attributes, `set_name()`/`set_version()`, or `get_ref()`), Conan raises an error listing all the available alternatives.

conanws.py super-build `ConanFile`

The `conanws.py` file can contain the definition of a `ConanFile` that represents the super-build. When the workspace dependency graph is computed, all packages in the workspace are collapsed into a single node in the dependency graph, and that node will have dependencies to the other packages external to the workspace, that is, installed in the Conan cache.

Note: Collapsing the workspace packages into a single node means that an external dependency can also only be represented once in the super-build graph, even if it would otherwise appear as more than one node, for example once in the `host` context and once in the `build` context, or once as a private (`visible=False`) requirement and once as a public one. Since Conan 2.33, Conan resolves this ambiguity by preferring, in order, the `host` context node and then the visible node, warning when it does so. See *the monolithic build tutorial* for more details.

The `ConanFile` that represents the workspace super-build project is defined as:

Listing 187: conanws.py

```
from conan import ConanFile, Workspace
from conan.tools.cmake import cmake_layout

class MyWs(ConanFile):
   settings = "os", "compiler", "arch", "build_type"
   generators = "CMakeToolchain", "CMakeDeps"

   def layout(self):
      cmake_layout(self)
```

(continues on next page)

(continued from previous page)

```
class Ws(Workspace):
    def root_conanfile(self):
        return MyWs
```

It defines that our super-build project will be a CMake project that uses the `CMakeToolchain` and `CMakeDeps` generators to integrate and find the external package dependencies. It is not necessary that the `ConanFile` defines `requires` at all, they will be computed by aggregating the requires of all packages in the workspace.

Important: The `ConanFile` inside `conanws.py` is a special conanfile, used exclusively for the workspace super-build definition of layout and generators. It shouldn't have any kind of requirements, not regular `requires`, `tool_requires` or `test_requires`. It obtains its dependencies collecting and aggregating the workspace packages requirements. It shouldn't have `build()` or `package()` methods either.

---

conanws.py super-build workspace packages

With the `workspace_packages` attribute, the `conanws.py` super-build `ConanFile` can have access to the workspace packages, to be able to reuse their functionality. For example, if we wanted to collect the behavior of workspace packages toolchain definitions we could do the following. Let's imagine that we have a workspace with 2 packages, `pkga/1.2.3` and `pkgb/2.3.4`

Listing 188: pkga/conanfile.py

```
from conan import ConanFile

class PkgA(ConanFile):
    name = "pkga"
    version = "1.2.3"

    def configure_toolchain(self, tc):
        tc.preprocessor_definitions["PKGA_SOME_DEFINITION"] = self.version

    def generate(self):
        tc = CMakeToolchain(self)
        self.configure_toolchain(tc)
        tc.generate()
```

Listing 189: pkgb/conanfile.py

```
from conan import ConanFile
class PkgB(ConanFile):
    name = "pkgb"
    version = "2.3.4"

    def configure_toolchain(self, tc):
        tc.preprocessor_definitions["SOME_PKGB_DEFINE"] = self.version

    def generate(self):
        tc = CMakeToolchain(self)
        self.configure_toolchain(tc)
        tc.generate()
```

Listing 190: conanws.py

```
from conan import ConanFile
from conan import Workspace
from conan.tools.cmake import CMakeToolchain

class MyWs(ConanFile):
   settings = "arch", "build_type"
   def generate(self):
      tc = CMakeToolchain(self)
      for ref, dep in self.workspace_packages.items():
         dep.configure_toolchain(tc)
      tc.generate()

class Ws(Workspace):
   def root_conanfile(self):
      return MyWs
```

Then, the `workspace_packages.items()` iteration will be able to call every package in the workspace `configure_toolchain()` and collect all their behavior in the current super-build `CMakeToolchain`. The resulting toolchain will contain the definitions for `SOME_PKGB_DEFINE=2.3.4` and `PKGA_SOME_DEFINITION=1.2.3`.

Warning: Important

The access of `workspace_packages` to the workspace packages `ConanFiles` must be **read-only** and **pure**. It cannot modify the workspace `pkga` and `pkgb` packages data, and it cannot have any side effect. For example it is forbidden to call any method such as `.build()` or even the `.generate()` method. If there is logic to be reused, it is the responsibility of the developer to define some convention, such as the `configure_toolchain()` method that if called from the `conanws.py` will not modify at all (pure) the `pkga` or `pkgb` data.

conanws.py super-build options

A particular case of the above `workspace_packages` access could be reading the individual workspace packages options. A `conanws.py` used for a super-build workspaces file can manage options in two different ways:

- It can define its own `options` with the normal `conanfile.py` syntax, so the generated `conan_toolchain.cmake` for the super-project uses those inputs.
- It can collect the options of the workspace's packages with the `workspace_packages` and process them in any user-custom way.

super-project options

A `conanws.py` must define the options for the super-build in the `ConanFile` class, and use those options in the `generate()` method, as it usually happens with `conanfile.py` files, something like:

```
from conan import ConanFile, Workspace

class MyWs(ConanFile):
   settings = "arch", "build_type"
   options = {"myoption": [1, 2, 3]}

   def generate(self):
```

(continues on next page)

(continued from previous page)

```
        self.output.info(f"Generating with my option {self.options.myoption}!!!!!")

class Ws(Workspace):
    def root_conanfile(self):
        return MyWs
```

Then, options can be provided with the usual syntax, via profiles or command line:

```
$ conan workspace super-install -of=build -o "*:myoption=1"
> conanws.py base project Conanfile: Generating with my option 1!!!!
```

Note there can be overlap with the `options` defined in the workspace packages, as for super-projects those options are simply ignored, and only the options of the super-project are taken into account to generate the `conan_toolchain.cmake`. For example, the `conanws.py` can define a `shared` option if it is desired that the `conan_toolchain.cmake` will correctly define `BUILD_SHARED_LIBS` or not when defining something like `-o "*:shared=True"`, as the workspace packages having the `shared` option information is discarded when the workspace packages are collapsed in the dependency graph to model the super-project.

packages options

The second alternative is to collect the `options` of the workspace packages that have been collapsed. Recall that in the final dependency graph, the workspace packages are no longer represented, as they are no longer individual packages but part as the current super-build. The way to access their options information is via the `workspace_packages`, and that information can be used in the `generate()` method to do any desired action at the super-build project level.

So let's say that a workspace containing a `dep/0.1` package that contains the standard `shared` options defines the following super-build `ConanFile`:

```
from conan import ConanFile, Workspace

class MyWs(ConanFile):
    def generate(self):
        for pkg, dep in self.workspace_packages.items():
            for k, v in dep.options.items():
                self.output.info(f"Generating with opt {pkg}:{k}={v}!!!!")

class Ws(Workspace):
    def root_conanfile(self):
        return MyWs
```

Then, when the workspace package options are defined, the workspace `ConanFile` can collect them.

```
$ conan workspace super-install -of=build -o "*:shared=True"
> conanws.py base project Conanfile: Generating with opt dep/0.1:shared=True!!!!!!!!
```

Note: In practice it is the responsibility of the workspace creator to define what to do with the options, either by defining its own options or collecting the workspace packages ones. Note it is not possible to automatically map workspace packages options to the super-project, as options are defined per-package. Two different packages could have different `shared=True` and `shared=False` values. Also, very often, the effect on the generated toolchain files is custom and implemented in each package `generate()` method. This effect is programmatic (not declarative), it would be extremely challenging to aggregate all these effects in a single toolchain.

See also:

Read *the Workspace tutorial* section.

CHAPTER

TEN

KNOWLEDGE

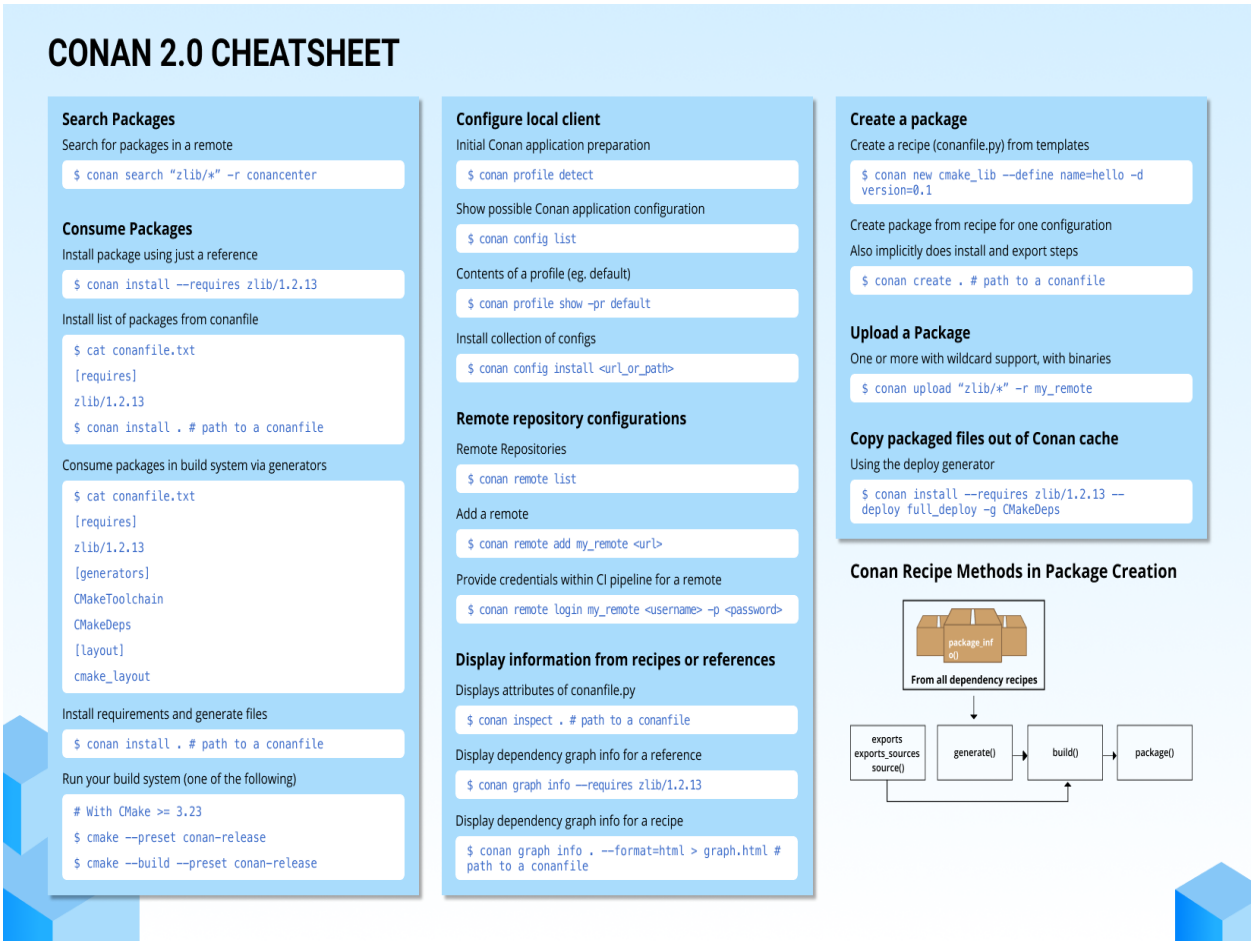
10.1 Cheat sheet

This is a visual cheat sheet for basic Conan commands and concepts which users can print out and use as a handy reference. It is available as both a PDF and PNG.

Tip: There is a blog post which goes over the changes from Conan 1.x and 2.0 as well.

PDF Format

PNG Format



10.2 Core guidelines

10.2.1 Good practices

- **build() should be simple, prepare the builds in generate() instead**: The recipes' `generate()` method purpose is to prepare the build as much as possible. Users calling `conan install` will execute this method, and the generated files should allow users to do "native" builds (calling directly "cmake", "meson", etc.) as easy as possible. Thus, avoiding as much as possible any logic in the `build()` method, and moving it to the `generate()` method helps developers achieve the same build locally as the one that would be produced by a `conan create` build in the local cache.
- **Always use your own profiles in production**, instead of relying on the auto-detected profile, as the output of such auto detection can vary over time, resulting in unexpected results. Profiles (and many other configuration), can be managed with `conan config install`.
- **Developers should not be able to upload to "development" and "production" repositories** in the server. Only CI builds have write permissions in the server. Developers should only have read permissions and at most to some "playground" repositories used to work and share things with colleagues, but which packages are never used, moved or copied to the development or production repositories.
- **The test_package purpose is to validate the correct creation of the package, not for functional testing**. The `test_package` purpose is to check that the package has been correctly created (that is, that it has correctly packaged the headers, the libraries, etc, in the right folders), not that the functionality of the package is correct. Then, it should be kept as simple as possible, like building and running an executable that uses the headers and links against a packaged library should be enough. Such execution should be as simple as possible too. Any kind of unit and functional tests should be done in the `build()` method.
- **All input sources must be common for all binary configurations**: All the "source" inputs, including the `conanfile.py`, the `conandata.yml`, the `exports` and `exports_source`, the `source()` method, patches applied in the `source()` method, cannot be conditional to anything, platform, OS or compiler, as they are shared among all configurations. Furthermore, the line endings for all these things should be the same, it is recommended to use always just line-feeds in all platforms, and do not convert or checkout to `crlf` in Windows, as that will cause different recipe revisions.
- **Keep ``python_requires`` as simple as possible**. Avoid transitive `python_requires`, keep them as reduced as possible, and at most, require them explicitly in a "flat" structure, without `python_requires` requiring other `python_requires`. Avoid inheritance (via `python_requires_extend`) if not strictly necessary, and avoid multiple inheritance at all costs, as it is extremely complicated, and it does not work the same as the built-in Python one.
- At the moment the **Conan cache is not concurrent**. Avoid any kind of concurrency or parallelism, for example different parallel CI jobs should use different caches (with CONAN_HOME env-var). This might change in the future and we will work on providing concurrency in the cache, but until then, use isolated caches for concurrent tasks.
- **Avoid 'force' and 'override' traits as a versioning mechanism.** The `force` and `override` traits to solve conflicts are not recommended as a general versioning solution, just as a temporary workaround to solve a version conflict. Its usage should be avoided whenever possible, and updating versions or version ranges in the graph to avoid the conflicts without overrides and forces is the recommended approach.
- **Please, do not abuse 'tool_requires'**. Those are intended only for executables like `cmake` and `ninja` running in the "build" context, not for libraries or library-like dependencies, that must use `requires` or `test_requires`.
- Positional arguments when invoking Conan should be specified first, before any named argument. For example, `conan install . -s="os=Windows"` is correct, but `conan install -s="os=Windows" .` is not. Likewise, it's recommended to use = instead of spaces between the name and value of named arguments. This is to avoid some ambiguity scenarios when parsing the command line arguments.

- **It is strongly discouraged to use user/channel** for any quality, stage, maturity or variable information The `channel` part is very legacy, and should be avoided in most cases, or use a fixed string as `stable`. The `user` might be used for intra-organization private packages, while the recommendation for packages coming from ConanCenter or forks of `conan-center-index` Github repo is to use them without any user or channel, like the `zlib/1.3.1` ConanCenter references, even for customization of the recipes and packages for those third party libraries.
- The way to manage package quality, stage or maturity **promotions is by using different server repositories**, and the well known developer's best practices recommend to manage the pipelines by doing promotions (copying) immutable artifacts or packages between those different server repositories, for example copying packages from a `staging` repository to a `production` repository once they have passed some quality checks. But it is very important that this promotion does not change in any way those packages, which must be completely immutable, not even changing its `user/channel`, this is why the above point discourages using user and channel, packages and artifacts must be immutable.
- Define **dependencies options values** in **profile files**, not in recipes. Do not define `default_options` for dependencies like `default_options = {"mydep*:myoption": "value"}`. Do not use `configure()` to define options values for dependencies, and avoid using the `options` requirement trait as much as possible. Use the dependencies default options, if necessary change those defaults directly in the dependencies to default to your most used configurations and use **profile** files whenever is necessary to diverge from those defaults. See *this FAQ about options values for dependencies* for more information.
- Do not export packages with versions that have an alphanumeric major version when you want to use minor versions too. That is, do not use `v1.3`, etc. (but `system`, `develop`, etc. are fine). The reason is that the checks for package binary compatibility that Conan performs special-case the major as alphanumeric cases, and will consider that `v1.0` and `v1.3` to be the same version regardless of the minor version, which might lead to unexpected results, such as a missing binary not being built when it should, and a package releasing without a critical bugfix. This can be mitigated by ensuring that both the `package_id_non_embed_mode` and `package_id_unknown_mode` are explicitly set in the recipe to something other than `major_mode`, `minor_mode`, `patch_mode` or `semver_mode`, which are the affected modes. But it is simpler to just avoid alphanumeric major versions if you want to use minor versions too.

10.2.2 Forbidden practices

- **Conan is not re-entrant**: Calling the Conan process from Conan itself cannot be done. That includes calling Conan from recipe code, hooks, plugins, and basically every code that already executes when Conan is called. Doing it will result in undefined behavior. For example it is not valid to run `conan search` from a `conanfile.py`. This includes indirect calls, like running Conan from a build script (like `CMakeLists.txt`) while this build script is already being executed as a result of a Conan invocation. For the same reason **Conan Python API cannot be used from recipes**: The Conan Python API can only be called from Conan custom commands or from user Python scripts, but never from `conanfile.py` recipes, hooks, extensions, plugins, or any other code executed by Conan.
- **Settings and configuration (conf) are read-only in recipes**: The settings and configuration cannot be defined or assigned values in recipes. Something like `self.settings.compiler = "gcc"` in recipes shouldn't be done. That is undefined behavior and can crash at any time, or just be ignored. Settings and configuration can only be defined in profiles, in command line arguments or in the `profile.py` plugin.
- **Recipes reserved names**: Conan `conanfile.py` recipes user attributes and methods should always start with `_`. Conan reserves the "public" namespace for all attributes and methods, and `_conan` for private ones. Using any non-documented Python function, method, class, attribute, even if it is "public" in the Python sense, is undefined behavior if such element is not documented in this documentation.
- **Conan artifacts are immutable**: Conan packages and artifacts, once they are in the Conan cache, they are assumed to be immutable. Any attempt to modify the exported sources, the recipe, the conandata.yml or any of the exported or the packaged artifacts, is undefined behavior. For example, it is not possible to modify the

contents of a package inside the `package_info()` method or the `package_id()` method, those methods should never modify, delete or create new files inside the packages. If you need to modify some package, you might use your own custom `deployer`.

- **Conan cache paths are internal implementation detail**: The Conan cache paths are an internal implementation detail. Conan recipes provide abstractions like `self.build_folder` to represent the necessary information about folders, and commands like `conan cache path` to get information of the current folders. The Conan cache might be checked while debugging, as read-only, but it is not allowed to edit, modify or delete artifacts or files from the Conan cache by any other means that Conan command line or public API.
- **Sources used in recipes must be immutable**. Once a recipe is exported to the Conan cache, it is expected that the sources are immutable, that is, that retrieving the sources in the future will always retrieve the exact same sources. It is not allowed to use moving targets like a `git` branch or a download of a file that is continuously rewritten in the server. `git` checkouts must be of an immutable tag or a commit, and `download()/get()` must use checksums to verify the server files doesn't change. Not using immutable sources will be undefined behavior.

10.3 FAQ

See also:

There is a great community behind Conan with users helping each other in Cpplang Slack. Please join us in the `#conan` channel!

10.3.1 ERROR: Missing prebuilt package

When installing packages (with **`conan install`** or **`conan create`**) it is possible that you get an error like the following one:

```
ERROR: Missing binary: zlib/1.3.1:b1d267f77ddd5d10d06d2ecf5a6bc433fbb7eeed

zlib/1.3.1: WARN: Can't find a 'zlib/1.3.1' package binary
↪'b1d267f77ddd5d10d06d2ecf5a6bc433fbb7eeed' for the configuration:
[settings]
arch=x86_64
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu11
compiler.libcxx=libc++
compiler.version=14
os=Macos
[options]
fPIC=True
shared=False

ERROR: Missing prebuilt package for 'zlib/1.3.1'. You can try:
    - List all available packages using 'conan list "{ref}:*" -r=remote'
    - Explain missing binaries: replace 'conan install ...' with 'conan graph explain ...
↪'
    - Try to build locally from sources using the '--build=zlib/1.3.1' argument

More Info at 'https://docs.conan.io/en/2/knowledge/faq.html#error-missing-prebuilt-
↪package'
```

This means that the package recipe `zlib/1.3.1` exists, but for some reason there is no precompiled package for your current settings or options. Maybe the package creator didn't build and shared pre-built packages at all and only uploaded the package recipe, or they are only providing packages for some platforms or compilers. E.g. the package creator built packages from the recipe for apple-clang 11, but you are using apple-clang 14. Also you may want to check your *package ID mode* as it may have an influence on the packages available for it.

These would be the things to check:

- Run a `conan graph explain` with exactly the same arguments as you did. It will give you hints about the differences between the binaries that exists in the cache and remotes and the binaries you are requesting to install.
- You an also use a `conan list <pkgname/version>:*` command to list binaries in your cache and in your remotes and manually compare differences
- If the binaries are missing because dependencies versions diverge, maybe your current Conan cache is using older revisions or versions, try adding an `--update` argument to the `conan install` command.
- If finally the binary that is being requested doesn't really exists anywhere, you might want to build it from sources.

By default, Conan doesn't build packages from sources. There are several possibilities to overcome this error:

- You can try to build the package for your settings from sources, indicating some build policy as argument, like **`--build zlib*`** or **`--build missing`**. If the package recipe and the source code work for your settings you will have your binaries built locally and ready for use.
- If building from sources fails, and you are using the *conancenter* remote, you can open an issue in the Conan Center Index repository

10.3.2 ERROR: Invalid setting

It might happen sometimes, when you specify a setting not present in the defaults that you receive a message like this:

```
$ conan install . -s compiler.version=4.19 ...

ERROR: Invalid setting '4.19' is not a valid 'settings.compiler.version' value.
Possible values are ['4.4', '4.5', '4.6', '4.7', '4.8', '4.9', '5.1', '5.2', '5.3', '5.4
↪', '6.1', '6.2']
```

This doesn't mean that such compiler version is not supported by Conan, it is just that it is not present in the actual defaults settings. You can find in your user home folder `~/.conan2/settings.yml` a settings file that you can modify, edit, add any setting or any value, with any nesting if necessary. See *settings.yml* to learn how you can customize your settings to model your binaries at your will.

As long as your team or users have the same settings (`settings.yml` and `settings_user.yml` an be easily shared with the `conan config install` command), everything will work. The *settings.yml* file is just a mechanism so users agree on a common spelling for typical settings. Also, if you think that some settings would be useful for many other conan users, please submit it as an issue or a pull request, so it is included in future releases.

It is possible that some built-in helper or integrations, like `CMake` or `CMakeToolchain` will not understand the new added settings, don't use them or even fail if you added some new unexpected value to existing settings. Such helpers as `CMake` are simple utilities to translate from conan settings to the respective build system syntax and command line arguments, so they can be extended or replaced with your own one that would handle your own private settings.

10.3.3 ERROR: AuthenticationException:

This error can happen, if there are no or false authentication credentials in the HTTP request from conan. To get more information try enabling the debug level for HTTP connections:

```
import http.client
http.client.HTTPConnection.debuglevel = 1
```

One source of error can be the `.netrc` file, which is honored by the requests library.

10.3.4 ERROR: Obtaining different revisions in Linux and Windows

Git will (by default) checkout files in Windows systems using `CRLF` line endings, effectively producing different files than in Linux that files will use `LF` line endings. As files are different, the Conan recipe revision will be different from the revisions computed in other platforms such as Linux, resulting in missing the respective binaries in the other revision.

Conan will not normalize or change in any way the source files, it is not its responsibility and there are risks of breaking things. The source control is the application changing the files, so that is a more correct place to handle this. It is necessary to instruct Git to do the checkout with the same line endings. This can be done several ways, for example, by adding a `.gitattributes` file to the project repository with something like:

```
* text eol=lf
```

Above will mark all files in repo with `text` attribute and force `lf` as end of line. Treating binary files as `text` lead to data corruption although. If there are binary files alongside, make sure to exclude them back:

```
* text eol=lf

*.png binary
*.jpg binary
*.jpeg binary
```

Other approach would be to change the `.gitconfig` to change it globally. Modern editors (even Notepad) in Windows can perfectly work with files with `LF`, it is no longer necessary to change the line endings.

10.3.5 Defining options for dependencies in conanfile.py recipes doesn't work

Conan expands the dependency graph depth-first, this is important to be able to implement many of the very special C/C++ propagation logic (headers, static and shared libraries, applications, tool-requires, test-requires, conflicts, overrides, etc.).

This means that when a `conanfile.py` declares something like:

```
class MyPkg(ConanFile):
    name = "mypkg"
    version = "0.1"
    default_options = {"zlib/*:shared": True}
    # Or
    def requirements(self):
        self.requires("zlib/1.3", options={"shared": True})
```

it cannot be always honored, and the `zlib` dependency might end with different `shared=False` option value. This in-recipe options values definition for dependencies only works if:

- There are no other packages depending on `zlib` in the graph
- There are other packages depending on `zlib` in the graph, but `mypkg/0.1` is the first require (the first branch in the dependency graph) that is required. That means that `requires = "mypkg/0.1", "zlib/1.3"` will work and will have `zlib` as shared, but `requires = "zlib/1.3", "mypkg/0.1"` will expand first `zlib` with its default, which is `shared=False` and when the `mypkg/0.1` is computed it will be too late to change `zlib` to be `shared=True`.

In case there are some recipe that won't work at all with some option of the dependency, the recommendation is to define a `validate()` method in the recipe to guarantee that it will raise an error if for some reason the upstream dependency doesn't have the right options values.

Conan might be able to show some (not guaranteed to be exhaustive) of these issues in the output of the Conan commands, please read it carefully.

```
Options conflicts
    liba/0.1:myoption=1 (current value)
        libc/0.1->myoption=2
    It is recommended to define options values in profiles, not in recipes
```

In general, it is more recommended to define options values in profile files, not in recipes. Recipe defined options always have less precedence than options defined in profiles.

Important: Defining options values for dependencies in recipes does not have strong guarantees:

- This applies to any recipe definition of dependencies options, via `default_options`, `configure()`, or requirements `options=` trait.
- It is not possible to change the options of non-visible transitive dependencies, for example `test_requires` or `tool_requires` of dependencies cannot be affected by any definition of options values in downstream recipes, because they are private.
- The "locality" of the definition makes it a bad location for large projects with developers working on different packages in the dependency graph.

So in general, defining options values for dependencies in recipes is **discouraged**. The strongly recommended way to define options values for dependencies is in **profile files**.

10.3.6 Getting version conflicts even when using version ranges

It is possible that when installing dependencies, there are version conflict error messages like:

```
...
Version conflict: Conflict between math/1.0.1 and math/1.0 in the graph
```

This *tutorial about version conflicts* summarizes how different versions of the same package dependency can conflict in a dependency graph and how to resolve those conflicts.

However, there are some situations in which the conflict is not that evident, for example when there are some mixed version ranges and fixed dependencies, something like:

```
def requirements(self):
  self.requires("libb/1.0")  # requires liba/[>=1.0 <2]
  self.requires("libc/1.0")  # requires liba/1.0
```

And it happens that `libb/1.0` has a transitive requirement to `liba/[>=1.0 <2]`, and `libc/1.0` requires `liba/1.0`, and there exist the `liba/1.1` or higher packages. In this case, Conan might also throw a "version conflict" error.

The root cause is that resolving the joint compatibility for all the possible constraints that version-ranges define in a graph is a known NP-hard problem, known as SAT-solver. Evaluating each hypothesis in this NP-hard problem in Conan is very expensive, because it usually requires to look for a version/revision in all remotes defined, then download such version/revision compressed files, unzip them, load and Python-parse and evaluate them and finally to do all the graph computation processing, which involves a full propagation down the already expanded graph to propagate the C/C++ requirement traits that can produce the conflicts. This would make the problem intractable in practice, that would require to wait for many hours to finish.

So instead of doing that, Conan uses a "greedy" algorithm that does not require backtracking, but still will try to reconcile version-ranges with fixed versions when possible. The most important point to know about this is that Conan implements a "depth-first" graph expansion, evaluating the `requires` in the order they are declared. Knowing this can help to solve this conflict. In the case above the error happens because `libb/1.0` is expanded first, it finds a requirement of `liba/[>=1.0 <2]`, and as no other constraint to `liba` has been found before, it freely resolves to the latest `liba/1.1`. When later `libc/1.0` is expanded, it finds a requirement to `liba/1.0`, but it is already too late, as it will conflict with the previous `liba/1.1`. Going back in the previous hypothesis is the "backtracking" part that converts the problem in NP-hard, so the algorithm stops there and raises the conflict.

This can be solved just by swapping the order of `requires`:

```
def requirements(self):
  self.requires("libc/1.0")  # requires liba/1.0
  self.requires("libb/1.0")  # requires liba/[>=1.0 <2]
```

If `libc/1.0` is expanded first, it resolves to `liba/1.0`. When later `libb/1.0` is expanded, its transitive requirement `liba/[>=1.0 <2]` can be successfully satisfied by the previous `libb/1.0`, so it can resolve the graph successfully.

The general best practices are:

- For the same dependency, try to use the same approach everywhere: use version ranges everywhere, or fixed versions everywhere for that specific dependency.
- Keep the versions aligned. If using a version range try to use the same version range everywhere.
- Declare first dependencies that use fixed version, not version ranges
- Use the `conan graph info ... --format=html > graph.html` graphical interactive output to understand and navigate conflicts.

10.3.7 Conan is redirecting its output to stderr

As explained *in the commands reference*, by design Conan redirects its logging information to the standard error output (stderr), while the actual results of the commands are sent to the standard output (stdout).

This is done to allow users to easily redirect the output of Conan commands to files or other processes without having the logging information mixed with the actual results.

For example, running a command like:

```
$ conan graph info --requires=zlib/1.3.1 --format=json > graph.json

...

======== Computing dependency graph ========
Graph root
    cli
```

(continues on next page)

(continued from previous page)

```
Requirements
    zlib/1.3.1#b8bc2603263cf7eccbd6e17e66b0ed76 - Cache

======== Computing necessary packages ========
Connecting to remote 'conancenter' anonymously
Requirements
    zlib/1.3.1#b8bc2603263cf7eccbd6e17e66b0ed76:dbb40f41e6e9a5c4a9a1fd8d9e6ccf6d92676c92
↪#8976086f07d37e3f6288e2fccf9650ae - Cache
```

will create a file named `graph.json` with the JSON output of the command, but it will not include any logging information, which will be printed to the console.

Note that this approach is common in many command-line tools such as `git` and `curl`, and it is not specific to Conan.

10.3.8 Missing binary for a (tool) package that was just created with `conan create`

There is sometimes the case for a package, intended to be used as a `tool_requires` by other packages, whose recipe contains a `test_package` folder that requires such tool as:

Listing 1: test_package/conanfile.py

```
from conan import ConanFile


class secure_scannerTestConan(ConanFile):
    settings = "os", "compiler", "build_type", "arch"

    def build_requirements(self):
        self.tool_requires(self.tested_reference_str)
```

Then, users doing the creation of the package with:

```
$ conan create
...
mytool/0.1: Package created successfully

======== Launching test_package ========
...
======== Computing necessary packages ========
...
ERROR: Missing binary: mytool/0.1
```

will find a "Missing binary" error in the `test_package` step. How is this possible, if the package binary was created a few lines above?

The `conan create` command by default creates packages for the "host" context, using the "host" profile. But if the package we are creating is intended to be used as a tool, that is, as a `tool_requires`, then it needs to be built for the "build" context.

If for any reason, the "host" and the "build" context are not identical, then the binary that is built in the initial package creation will be a binary for the "host" context, but then the `test_package` will require it as `tool_requires()`, requiring it in the "build" context, and such binary will be missing, as it hasn't been built.

This is evident in the case of a cross-compilation, from example, building on a Windows laptop a binary for the RaspberryPI with a cross-compiler. The "build" context will be the "Windows" one, while the "host" context will be the

"RaspberryPI" one. A regular `conan create .` for "mytool/0.1", will create by default a binary for "RaspberryPI"! So when later, the `test_package` wants to use it as a `tool_requires()` it will look for the "Windows" binary, as it needs to run it in the current Windows machine, the "build" context.

The `--build-require` argument specifies this. When this argument is provided, the current recipe binary will be built to run in the "build" context

---

Note: When doing `conan create` for a package intended to be used as a `tool_requires`, always specify the `conan create ... --build-require` argument.

10.3.9 Using Conan with both corporate and public remotes (SSL certificates)

In corporate environments it is common to need access to both a private Artifactory remote (secured with a self-signed or internal CA certificate) and a public remote such as ConanCenter at the same time.

The problem appears because:

- Without any custom CA configuration Conan may reject the corporate remote (`CERTIFICATE_VERIFY_FAILED`).
- Setting `core.net.http:cacert_path` to point **only** to the corporate CA makes ConanCenter unreachable, because the public root CAs are no longer in the trust bundle.

The solution is to build a **single combined CA bundle** that contains both the public root certificates **and** the corporate CA, and then configure Conan to use it via `core.net.http:cacert_path` (see *Configuration of client certificates*).

The combined bundle can be created by appending the corporate CA (in PEM format) to the default public CA bundle. The `certifi` Python package ships the Mozilla root CA bundle that Conan uses by default:

```
# 1. Copy the default public CA bundle to a working location
cp "$(python -m certifi)" combined-ca-bundle.pem

# 2. Append the corporate CA (PEM format) to the bundle
cat my-corporate-ca.crt >> combined-ca-bundle.pem
```

You can append as many additional CAs as needed. Then point Conan at the combined file:

Listing 2: **[CONAN_HOME]/global.conf**

```
core.net.http:cacert_path=/path/to/combined-ca-bundle.pem
```

See also:

Configuration of client certificates for the full reference on `core.net.http:cacert_path`, `core.net.http:client_cert`, and alternative ways to aggregate certificates (including `update-ca-certificates` on Debian/Ubuntu).

10.3.10 Conan doesn't skip failing remotes

This frequent question happens when users have defined some remotes, some of them are not available for any reasons, and Conan fails with an error reporting about that unresponsive or disconnected server remote.

There are different scenarios in which silently skipping a server can cause production issues, for example when trying to build a new binary of an application that had some new versions of some new dependencies in that server. If that server doesn't work properly and the pipeline continues it will result in a binary without the new versions without the bug fixes on upgrades. There could be extra checks later, validation gates, etc, to try to avoid that binary without the fixes in production, but those are often too late or just non existing.

So Conan will always treat the defined and available servers as "must be alive", and fail immediately if they do not work. Conan already provides some mechanisms to control the remote servers that are used for package resolution:

- `conan remote enable/disable` to temporarily enable, disable servers
- Explicit listing of the remotes you want to use conan install `-r=remote1 -r=remote8 -r=remote3`, in the order you want the precedence
- `conan config install/install-pkg` for easy configuration of remotes for different projects or needs.

10.4 Videos

Warning: This section presents some conference talks and presentations regarding Conan. While they can be very informative and educational, please note that some of them might be outdated. Always use the documentation and reference as the source of truth, not the videos.

10.4.1 Using std::cpp 2026

"Using std::cpp 2026: Cross-Platform C++ AI Development with Conan, CMake, and CUDA" - Luis Caro

> Every year, the ISO C++ survey delivers the same verdict: dependency management is the #1 pain point for developers. For AI and machine learning development, introducing CUDA into the mix can cause that "pain point" to become a bigger bottleneck when developers these days are demanding "one-line, cross-platform" solutions.
>
> Setting up a cross-platform AI development environment can involve a fragile "ritual" of manual installs, environment variables, and platform-specific hacks. Moving from a Windows dev-box to a Linux CI server or a Jetson Orin at the edge? That's often a week of lost productivity, especially for large teams that need to support multiple versions of CUDA on multiple platforms.

This talk demonstrates how to use Conan and CMake to model the CUDA compatibility matrix directly in your code, achieving an ideal pipeline that works both on development machines, and CI, regardless of your target platform: One source checkout. One command. Identical builds on every platform.

https://youtu.be/jnKeUE2C8_I

10.4.2 ACCU 2025

"Continuous Integration for Large Scale C/C++ Projects With Conan2" - Diego Rodriguez-Losada

There are two main paradigms to develop large scale C, C++ projects: using mono-repos and package-based development, both have different pros and cons. Using package managers such as Conan allows easy retrieval of dependencies, including binaries, avoiding continuous and expensive rebuilds from source. While this works easily for third party dependencies, when dependencies are very stable, quick evolution in a large dependency graph when there are different developers working concurrently in different packages, can be more challenging.

With Conan 2's new tools it is possible to implement an efficient Continuous Integration process for large scale projects, computing what packages need to be built, and in what specific order and parallelism.

The concept of "products pipeline" will be introduced as an scalable approach that allows to focus on the business value while allowing efficiency. The dependency graph build order for every different product can be computed, for every different configuration (platform, compiler, build_type, etc). This is done efficiently, taking into account the versioning scheme and the package types (header-only, static library, shared library, application) with a explicit novel model of the dependencies effect in the packages binaries.

Then all the different computed graph "build-orders" can be merged into a single one to avoid duplicated builds. The final graph merged "build-order" can be distributed efficiently to different build agents, as it also specifies which package builds can be done in parallel. For projects with concurrent changes, using lockfiles is very important in this stage to guarantee consistency and reproducibility of the dependencies.

Finally, the concept of stages in the CI process using multiple server repositories and copying of packages across those repositories will be presented. This process is known as package promotions and is a known best practice in DevOps for other technologies, and is critical to avoid disruptions to developers and production systems that the CI builds could introduce.

The talk will present both the theoretical foundations and a full real example with available source code to demonstrate the presented functionalities.

https://youtu.be/A3X1MpvYTrM

10.4.3 Using std::cpp 2025

"An introduction to the Common Package Specification (CPS) for C and C++" - Diego Rodriguez-Losada

The Common Package Specification format is an effort to create a standard description of binary packages to improve the reuse of C and C++ software and the interoperability of different build systems and tools.

This talk briefly introduces the specification and its main concepts, the project, and summarize the efforts of the team so far.

This specification already has some experimental support in different open source tools, like CMake and Conan C++ package manager. This will be demonstrated, showing how it can achieve such interoperability of build systems.

https://youtu.be/C1OCKEl7x_w

"Open source C++ packages vulnerabilities and tools" - Luis Caro Campos

The highest priority of the C++ language is security in the language itself. But what happens after a security issue is identified in some existing open source C or C++ library?

This talk gives an introduction of CVEs for C and C++ packages, report the state of the art and introduce different tools in this domain.

It also presents the Software Build of Materials (SBOMs) concept, and the main standards such as CycloneDX and SPDX.

The talk finish demonstrating some of these concepts with Conan C++ package manager and the JFrog platform security tools.

https://youtu.be/sTqbfdiOSUY

10.4.4 Using std::cpp 2024

"Unlock the power of Conan 2 - 15 new features you didn't know about!" - Luis Caro & Diego Rodriguez-Losada

Conan 2.0 was released over a year ago with a large number of new features and improvements. Since its release, the team has continued to add improvements based on user feedback, releasing many more new features than in the previous years combined.
Join Diego and Luis from the Conan team for an overview of practical examples of what Conan 2 can do for your C and C++ package management development workflows. Some highlights include: transparent fall back to system-provided dependencies, managing metadata files, and the flexible and fully transparent CMake integrations, and more!

https://youtu.be/yC3ERwB-Njc

10.4.5 ACCU 2022

"Advanced Dependencies Model in Conan 2.0 C, C++ Package Manager" - Diego Rodriguez-Losada

Conan 2.0 introduces a new dependencies model with requirements "traits" like visibility, definition and propagation of headers and libraries independently, and more that allow modeling all these advanced use cases. This talk will present this new model, and apply it to solve different advanced use cases, with real life examples

https://youtu.be/kKGglzm5ous

10.4.6 CppCon 2022

"What's New in Conan 2.0 C/C++ Package Manager" - Diego Rodriguez-Losada

During the years since Conan 1.0 was released, we have continued to learn from the C++ ecosystem as we watched it grow; learning many lessons, challenges and trends in the industry from the feedback from tens of thousands of conversations with users and customers, including many of the largest C++ related companies in the world. This talk summarizes some of these lessons and how they have been used to create the new major version of Conan.

https://youtu.be/NM-xp3tob2Q

10.4.7 Meeting C++ 2023

"CMake and Conan: past, present and future" - Diego Rodriguez-Losada

This talk will quickly review the past approaches, their pitfalls, and how modern CMake and Conan integrations have improved over them:

From variables, to targets, to transparent targets integration with modern Conan generators

Better separation of concerns to align binary configurations using CMake toolchains

Improving the developer experience with CMake presets

The new CMake-Conan integration using CMake's new dependency providers feature for transparent installation of dependencies

https://youtu.be/s0q6s5XzIrA

10.4.8 Using std::cpp 2021

"Why you shouldn't write your own C++ package manager" - Luis Caro Campos

This talk will provide a quick overview of how Conan deals with intrinsic C++ complexities:

Headers vs binary symbols

Shared and static library

Symbol visibility

Binary compatibility: is there a one-size fits all approach to modeling it?

Build-time dependency resolution is only half the battle, what about runtime dependencies?

https://youtu.be/8Go5g2jVJWo

10.4.9 Meeting C++ online book & tool fair

"Conan 2.0 demo" - Chris McArthur

https://youtu.be/1q5oIOupwjg

10.5 JFrog Academy: Conan 2 Training

Conan 2 training is available on the JFrog Academy in two paths: **Conan 2 Essentials** and **Conan 2 Advanced**. There are 29 video lessons across 7 modules (16 in Essentials, 13 in Advanced). The courses are practical (with code you can follow along), free, and self-paced.

The examples in the courses use the Conan Training 2 GitHub repository. If you run into problems with the code or have feedback, open an issue there.

10.5.1 Conan 2 Essentials

Fundamentals of consuming and creating packages. Good starting point if you have little or no Conan experience.

Register here: Conan 2 Essentials

Modules:

- **Module 1: Fundamentals of Package Usage** (Lessons 1–7, ~59 min) — Building a simple CMake project, multiple configurations (Release/Debug, static/shared), *conanfile.py* for consumption, build tools as Conan packages, cross-compilation (host/build profiles), versioning (revisions, version ranges, lockfiles), Conan Audit.
- **Module 2: Package Creation and Uploading** (Lessons 8–13, ~48 min) — First package, dependencies and generators, `package()` and `package_info()`, settings and options (`package_id`), testing packages, remotes and uploading.
- **Module 3: Advanced Package Creation Scenarios** (Lessons 14–16, ~21 min) — Header-only libraries, prebuilt binaries, tool requires.

10.5.2 Conan 2 Advanced

For users who already know Conan 2 basics. Local development, dependency graph, extensibility, and the advanced binary model.

Register here: Conan 2 Advanced

Modules:

- **Module 1: Developing Packages Locally** (Lessons 17–18, ~16 min) — Package development flow (source, install, build, export-pkg), editable mode.
- **Module 2: The Dependency Graph** (Lessons 19–21, ~25 min) — Package types, `test_requires`, advanced versioning and lockfiles.
- **Module 3: Extensibility & Customization** (Lessons 22–26, ~47 min) — Config and extensibility (config install), deployers, custom commands, hooks, `python_requires`.
- **Module 4: Advanced Binary Model** (Lessons 27–29, ~28 min) — Binary model, extending the model (custom settings, conf in `package_id`), defining binary compatibility.

10.6 Community Resources

In this section, you can find documentation and Conan resources created by the community. We hope you find them useful.

If you have your own resources that you believe could help the community grow, don't hesitate to share them with us on our GitHub.

- The Conan Cookbook
- Localization of Conan Documentation

CHAPTER

ELEVEN

INCUBATING FEATURES

This section is dedicated to new features that are under development, looking for user testing and feedback. They are generally behind a flag to enable them to be explicitly opted-in at this testing stage. They require the very latest Conan version (sometimes recommended running from the `develop2` source branch), and explicitly setting those flags.

11.1 New CMakeConfigDeps generator

This generator is not incubating anymore, but already generally available. See *CMakeConfigDeps generator*

11.2 Workspaces

Workspaces are not incubating anymore, but generally available as experimental.

Moved to *Workspaces*

11.2.1 Workspace files syntax

Moved to *Workspace*

11.2.2 Workspace commands

Moved to *conan workspace*

11.2.3 Workspace monolithic builds

Moved to *Monolithic build*

For any feedback, please open new tickets in https://github.com/conan-io/conan/issues.

CHAPTER

TWELVE

WHAT'S NEW IN CONAN 2

Conan 2 comes with many exciting improvements based on the lessons learned in the last years with Conan 1.X. Also, a lot of effort has been made to backport necessary things to Conan 1.X to make the upgrade easier: recipes using latest 1.X integrations will be compatible with Conan 2, and binaries for both versions will not collide and be able to live in the same server repositories.

12.1 Conan 2 migration guide

If you are using Conan 1.X, please read the Conan 2 Migration guide to start preparing your package recipes for 2.0 and be aware of some changes while you still work in Conan 1.X. That guide summarizes the above mentioned backports to make the upgrade easier.

12.2 New graph model

Conan 2 defines new requirement traits (headers, libs, build, run, test, package_id_mode, options, transitive_headers, transitive_libs) and package types (static, shared, application, header-only) to better represent the relations that happen with C and C++ binaries, like executables or shared libraries linking static libraries or shared libraries.

See also:

- https://www.youtube.com/watch?v=kKGglzm5ous
- https://github.com/conan-io/tribe/blob/main/design/026-requirements_traits.md
- https://github.com/conan-io/tribe/blob/main/design/027-package_types.md

12.3 New public Python API

A new modular Python API is made available, public and documented. This is a real API, with building blocks that are already used to build the Conan built-in commands, but that will allow further extensions. There are sub-APIs for different functional groups, like `api.list`, `api.search`, `api.remove`, `api.profile`, `api.graph`, `api.upload`, `api.remotes`, etc. that will allow to implement advanced user flows, functionality and automation.

See also:

- *Python API reference*

12.4 New build system integrations

Introduced in latest Conan 1.X, Conan 2 will use modern build system integrations like `CMakeDeps` and `CMakeToolchain` that are fully transparent CMake integrations (i.e. the consuming `CMakeLists.txt` doesn't need to be aware at all about Conan). These integrations can also achieve a better IDE integration, for example via CMakePresets.json.

See also:

- *Tools reference*

12.5 New custom user commands

Conan 2 allows extending Conan with custom user commands, written in Python that can be called as `conan xxxx`. These commands can be shared and installed with `conan config install`, and have layers of commands and subcommands. The custom user commands use the new 2.0 public Python API to implement their functionality.

12.6 New CLI

Conan 2 has redesigned the CLI for better consistency, removing ambiguities, and improving the user experience. The new CLI also sends all the information, warning, and error messages to stderr, while keeping the final result in stdout, allowing multiple output formats like `--format=html` or `--format=json` and using redirects to create files `--format=json > myfile.json`. The information provided by the CLI will be more structured and thorough so that it can be used more easily for automation, especially in CI/CD systems.

See also:

- *Commands reference*

12.7 New deployers

Conan 2 implements "deployers" which can be called on the command-line as `conan install .... --deployer=mydeploy`, typically to perform copy operations from the Conan cache to user folders. Such deployers can be built-in ("full_deploy", "direct_deploy" and "runtime_deploy" are provided so far), or user-defined, which can be shared and managed with `conan config install`. Deployers run before generators, and they can change the target folders. For example, if the `--deployer=full_deploy` deployer runs before `CMakeDeps`, the files generated by `CMakeDeps` will point to the local copy in the user folder done by the `full_deploy` deployer, and not to the Conan cache.

Deployers can be multi-configuration. Running `conan install . --deployer=full_deploy` repeatedly for different profiles, can achieve a fully self-contained project, including all the artifacts, binaries, and build files that is completely independent of Conan and no longer requires Conan at all to build.

12.8 New package_id

Conan 2 defines a new, dynamic `package_id` that is a great improvement over the limitations of Conan 1.X. This `package_id` will take into account the package types and types of requirements to implement a more meaningful strategy, depending on the scenario. For example, it is well known that when an application `myapp` is linking a static library `mylib`, any change in the binary of the static library `mylib` requires re-building the application `myapp`. So Conan will default to a mode like `full_mode` that will generate a new `myapp` `package_id`, for every change in the `mylib` recipe or binary. While a dependency between a static library `mylib_a` that is used by `mylib_b` in general does not imply that a change in `mylib_b` always needs a rebuild of `mylib_a`, and that relationship can default to a `minor_mode` mode. In Conan 2, the one doing modifications to `mylib_a` can better express whether the consumer `mylib_b` needs to rebuild or not, based on the version bump (patch version bump will not trigger a rebuild while a minor version bump will trigger it)

Furthermore, the default versioning scheme in Conan has been generalized to any number of digits and letters, as opposed to the official "semver" that uses just 3 fields.

12.9 compatibility.py

Conan 2 features a new extension mechanism to define binary compatibility at a global level. A `compatibility.py` file in the Conan cache will be used to define which fallbacks of binaries should be used in case there is some missing binary for a given package. Conan will provide a default one to account for `cppstd` compatibility, and executables compatibility, but this extension is fully configurable by the user (and can also be shared and managed with `conan config install`).

12.10 New lockfiles

Lockfiles in Conan 2 have been greatly simplified and made way more flexible. Lockfiles are now modeled as lists of sorted references, which allow one single lockfile being used for multiple configurations, merging lockfiles, applying partially defined lockfiles, being strict or non-strict, adding user-defined constraints to lockfiles, and much more.

See also:

- *Tutorial introduction to lockfiles*
- https://github.com/conan-io/tribe/blob/main/design/034-new_lockfiles.md
- *Tutorial about versioning and lockfiles*

12.11 New configuration and environment management

The new configuration system called `[conf]` in profiles and command line, and introduced experimentally in Conan 1.X, is now the primary mechanism to configure and control Conan behavior. The idea is that the configuration system is used to transmit information from Conan (a Conan profile) to Conan (a Conan recipe, or a Conan build system integration like `CMakeToolchain`). This new configuration system can define strings, booleans, lists, and is cleaner and more structured and powerful than environment variables. A better, more explicit environment management, also introduced in Conan 1.X is now the way to pass information from Conan (profiles) to tools (like compilers, build systems).

See also:

- *Reference of environment tools*

12.12 Multi-revision cache

The Conan cache has been completely redesigned to allow storing more than one revision at a time. It has also shortened the paths, using hashes, removing the need to use `short_paths` in Windows. Note that the cache is still not concurrent, so parallel jobs or tasks should use independent caches.

12.13 New extension plugins

Several extension points, named "plugins" have been added, to provide advanced and typically orthogonal functionality to what the Conan recipes implement. These plugins can be shared, managed and installed via `conan config install`

12.13.1 Profile checker

A new `profile.py` extension point is provided that can be used to perform operations on the profile after it has been processed. A default implementation that checks that the given compiler version is capable of supporting the given compiler `cppstd` is provided, but this is fully customizable by the user.

12.13.2 Command wrapper

A new `cmd_wrapper.py` extension provides a way to wrap any `conanfile.py` command (i.e., anything that runs inside `self.run()` in a recipe), in a new command. This functionality can be useful for wrapping build commands in build optimization tools such as IncrediBuild or compile caches.

12.13.3 Package signing

A new `sign.py` extension has been added to implement signing and verifying of packages. With growing awareness about the importance of software supply chain security, the ability to sign and verify software packages is becoming more critical. This extension point will soon get a plugin implementation based on Sigstore.

12.14 Package immutability optimizations

The thorough use of `revisions` in Conan 2 (already introduced in Conan 1.X as opt-in in https://docs.conan.io/en/latest/versioning/revisions.html), together with the declaration of artifacts **immutability** allows for improved processes when downloading, installing, updating and uploading dependencies.

`Revisions` allow accurate traceability of artifacts, and thus allow better update flows. For example, it will be easier to get different binaries for different configurations from different repositories, as long as they were created from the same recipe revision.

Package transfers, uploads and downloads will also be more efficient, based on `revisions`. As long as a given revision exists on the server or in the cache, Conan will not transfer artifacts at all for that package.

12.15 Package lists

Conan 2 allows bulk operations over multiple recipes and packages with the "Package Lists" feature. This feature allows to upload, download, remove and list multiple recipes and packages with one single command.

Package lists can also be created from a dependency graph resulting from a `conan create` or `conan install` command, so it is possible to upload to a server all packages that belong to a given dependency graph by just chaining two commands.

See also:

- *Read the example usages*
- Package lists blog post

12.16 Metadata files

Conan 2 allows to store, upload, download and modify metadata files associated to recipes and packages. This feature can be very useful to manage build logs, test executables, test results, coverage data and various other files needed for traceability, compliance and business purposes.

See also:

- Metadata files blog post

12.17 Third-party backup sources

When building packages for third parties with sources in the internet, those sources can be removed or changed. The "backup sources" can automatically store a copy of those sources on your own server, so your builds are always fully reproducible, no matter what happens to the original internet sources.

See also:

- Blog post about backup sources

12.18 Installing configuration from Conan packages

From Conan 2.2, it is possible to install configuration not only from git repos and http servers, but also from Conan packages. Running `conan config install-pkg myconf/myversion` for a Conan package `myconf/myversion` stored on a Conan server, will install the configuration files inside that package. It also allows to use version ranges to update easily to the latest one within the range, and lockfiles to achieve reproducibility.

See also:

- *Read the conan config install-pkg command reference*

CHAPTER

THIRTEEN

CHANGELOG

This page lists the changes made to Conan in each version, with links to each pull request for more details.

13.1 2.34.0 (28-Oct-2026)

- Feature: Define */MP* in *CMakeToolchain* for *clang-cl* with Visual Studio generators (*ClangCL* toolset), like *msvc*. #20439
- Feature: Reduce local cache sqlite3 DB calls (33% fewer for regular installs with cached packages). #20434
- Feature: Simplify the build step terminal output, removing duplicated package references. #20426
- Feature: Add `CMAKE_PREFIX_PATH` as fallback to `CMakeConfigDeps` for cases beyond the `<pkg>_DIR` variable. #20420 . Docs here
- Feature: Pass original URLs to backup source remotes in the *X-Conan-Origin-Urls* header. #20385 . Docs here
- Feature: Update *conan new bazel_lib* and *conan new bazel_exe* templates to Bazel 7. #20367 . Docs here
- Feature: Add a `command_alias.conf` file to define command aliases and shortcuts without writing a custom command. #20365
- Feature: Support the new CPS `dyld_requires` (CMake>=4.4) for transitive shared libraries on Linux. #20362
- Feature: Add *cmake_file_names* dict property to *CMakeConfigDeps* to generate several CMake CONFIG files per package. #20298 . Docs here
- Feature: Add an optional *components* argument to *aggregated_components()* to aggregate only the given components. #20298 . Docs here
- Feature: Always generate `.ps1` environment launchers, alongside the `.bat/.sh` ones, for the consumer, editables and `test_package`. #19649 . Docs here
- Fix: Clarify in the `PackagesList.items()` docstring that it only returns revision-specific information. #20432
- Fix: Add docstrings to the `PkgReference` class of the Python API. #20431 . Docs here
- Fix: Remove the legacy warning about ConanCenter being frozen for Conan 1 clients. #20425
- Fix: Remove the `python-dateutil` dependency for Python>=3.11. #20419
- Fix: Resolve `LocalAPI.editable_add` relative `output_folder` against the `cwd` argument. #20416
- Fix: Make `conan cache clean --backup-sources` only clean the backup sources of the selected recipes. #20415
- Fix: Allow several negated patterns, and mixing them with inclusions, in remote `--allowed-packages`. #20412

- Fix: Avoid a `VirtualRunEnv` generation crash with `win_bash=True` when cross-building from Linux to Windows. #20407
- Fix: Correct Python API docstrings for `ListAPI`, `CacheAPI` and other subapis that did not match their behavior. #20405
- Fix: Correct the `ListAPI.latest_recipe_revision` docstring, as it always raises if the reference is not found. #20400
- Fix: Report a proper error for non-integer `--lru` time values. #20399
- Fix: Avoid duplicating *tools.build:sharedlinkflags* from *conf* in *BazelToolchain*. #20390
- Fix: Join multiple SBOM license expressions with `AND` to comply with the CycloneDX standard. #20382
- Fix: Allow running *conan remote logout* for disabled remotes. #20381
- Fix: Replace stale `CONAN_RUNTIME_LIB_DIRS` entries in `CMakeConfigDeps` when installing a new package revision. #20378
- Fix: Raise an error for `--requires=pkg/version@_`, an undocumented and unsupported Conan 1 legacy syntax. #20371
- Fix: Remove Conan 1.X legacy `alias` (undocumented and unsupported) code. #20370
- Fix: Make the `workspace` experimental warning skippable with `core:skip_warnings`. #20369
- Fix: Add deprecation warnings for Bazel 6 support in the *BazelDeps* generator. #20367 . Docs here
- Fix: Filter `CMakeConfigDeps` target data based on require traits, aligned with `CMakeDeps`. #20277 . Docs here
- Fix: `tools.env.virtualenv:powershell` no longer replaces the `.bat/.sh` launchers. `self.run()` only uses `.ps1` launchers when this conf is defined. #19649 . Docs here
- Bugfix: Avoid accumulating variables when calling *GnuToolchain._environment* several times. #20446
- Bugfix: Avoid internal `rmdir` failures when removing Windows folders with read-only permissions. #20440
- Bugfix: Do not install host profile `[tool_requires]` in **`conan install`** with only `--tool-requires` arguments. #20422 . Docs here
- Bugfix: Do not remove pre-existing recipe revisions in `ReportAPI.diff`. #20417
- Bugfix: Make `AuditAPI.auth_provider` receive a provider name instead of a provider object, as documented. #20414
- Bugfix: Do not write *–dynamic_mode=off* in *BazelToolchain* when the recipe has no *shared* option. #20390
- Bugfix: Avoid random `dictionary changed size during iteration` errors in uploads with `core.upload:parallel`. #20389
- Bugfix: Avoid HTTP header cross-pollution between consecutive calls to the backup sources server. #20386
- Bugfix: Use `tools.microsoft.msbuild:max_cpu_count` for the `-m` argument in `CMakeToolchain` Visual Studio presets. #20360

13.2 2.33.0 (29-Sept-2026)

- Feature: Allow `cpp_info.sources`-only packages in CMakeConfigDeps. #20297 . Docs here
- Feature: *CMakeConfigDeps.get_cmake_file_name* returns a dict object instead of a string. #20300
- Feature: Support Clang 23. #20330
- Feature: Support Apple OS 27.0. #20330
- Feature: Add *copy_config_files* to the *[runner]* profile to copy extra host config files into Docker runners. #20332 . Docs here
- Feature: New remotes `force_auth` capability. #20341 . Docs here
- Feature: Add *core.net.http:trust_store* conf to verify HTTPS certs against the OS trust store. #20343 . Docs here
- Feature: Download recipe and package metadata in parallel with regular files instead of after them. #20351
- Feature: Use extra_info to provide CPEs to the component. #20353
- Feature: Support for CPS components configurations. #20354
- Fix: Do not inject a full lockfile due to `export` operations. #19404
- Fix: Implements the *find_package* verbosity level in CMakeConfigDeps. #20275
- Fix: Prioritize host and visible nodes in workspace monolithic builds when ambiguity arises. #20305 . Docs here
- Fix: Propagate errors from a *.bat* script called by another *.bat* script. #20317
- Fix: Avoid `CMakeDeps` generating empty generator expressions, improving performance. #20322
- Fix: Split skipped binaries summary per host/build/test group. #20326
- Fix: Only spawn `min(threads, download_count)` threads in parallel downloads. #20335
- Fix: Do not force `DebuggerFlavor` from generated prop files in MsBuildDeps, it's already the default. #20340
- Fix: Add support for `CMakeConfigDeps` for `cpp_info.objects` imported object libraries. #20347
- Fix: Error messages inside the *source()*, *generate()* and *package()* steps missing the package reference. #20348
- Fix: Remove support for Python 3.7, EOL since 2024. #20357
- Bugfix: Limit propagation of `cpp_info.sources` from `INTERFACE_SOURCES` to only direct dependencies. #20297 . Docs here
- Bugfix: Avoid reporting the cached username when authentication was attempted with a different user. #20316
- Bugfix: *QbsDeps* dependency names now match *pkg_config_name*/*qbs_file_name*. #20329
- Bugfix: Correct computation of `options` in the `conan graph build-order` command. #20331
- Bugfix: Consider a server revision found via `--update` as latest, even if its `prev` isn't latest locally. #20336
- Bugfix: *conan graph build-order* now honors *–update* when resolving version ranges (same as *graph info* / *install*) #20350
- Bugfix: Fix *conan graph build-order –update* ignoring the *–update* flag. #20356

13.3 2.32.0 (31-Aug-2026)

- Feature: Add *!settings.build_type* syntax to exclude *build_type* from *cmake_layout*'s *build_folder_vars*. #17331 . Docs here
- Feature: Accept CMake 4.4 *–preset-file* presets generated outside the CMake root. #20158 . Docs here
- Feature: Populate the download cache with uploaded artifacts if *core.download:download_cache* is set. #20163 . Docs here
- Feature: Rework the *conan graph build-order* HTML formatter with search and full offline support. #20174
- Feature: Allow **`conan run`** without a conanfile to run executables from the profile's *[tool_requires]*. #20214 . Docs here
- Feature: Add *qbs_file_name* property to override the generated Qbs module name and avoid collisions. #20215 . Docs here
- Feature: Feature: Allow *copy()* to accept multiple patterns, also used by *exports*/*exports_sources*, for speedup. #20217 . Docs here
- Feature: Add *–ref* argument to *conan editable add <path>* as a shortcut to define name, version, etc. #20218
- Feature: Add *package_type_traits* attribute to force requirement traits by *package_type* (*run* only for now). #20228 . Docs here
- Feature: Make *conan report diff -f=html* use a dark theme by default. #20229
- Feature: Improve `source` step output. #20234
- Feature: Add *recipe_metadata()*/*package_metadata()* API to download metadata to a user folder. #20240 . Docs here
- Feature: Compute *CMAKE_Swift_COMPILER_TARGET* in *CMakeToolchain* for Apple Swift cross-builds. #20243 . Docs here
- Feature: Add *architecture_bits* build helper to detect when the target architecture is 64-bit. #20246 . Docs here
- Feature: `conan workspace open` with no arguments will read the `conanws` file and try to open the defined references there into their respective folders if they are not present. #20247 . Docs here
- Feature: `conan workspace open` and `conan workspace add --ref` learned a new `--folder` argument to indicate the relative location wrt the workspace root for the cloning of the repository. #20247 . Docs here
- Feature: *XcodeToolchain* now accepts arbitrary extra xcconfig settings via *tc.build_settings*, for build settings with no dedicated conf. #20256 . Docs here
- Feature: Add `raise_on_errors=True` arg to `CommandAPI.run`, it now raises on all errors returned by commands. #20266
- Feature: Parse reference attributes from *–ref* in *workspace add*. #20269
- Feature: Add support for *Xcode 26.6*. #20280
- Feature: Add support for *gcc 16.2*. #20280
- Feature: Add basic LoongArch64 architecture support and host detection. #20288
- Fix: Remove the deprecated *–order-by* behavior and empty version ranges, as planned for Conan 2.32. #20207 . Docs here
- Fix: Make *conan lock upgrade-config*'s *path* argument point to a *conanconfig.yml* file, not a *conanfile.py*. #20208 . Docs here

- Fix: Warn when *tools.build:compiler_executables* keys are unknown or unused by *CMakeToolchain/AutotoolsToolchain*. #20219 . Docs here
- Fix: Add *objc*, *objcpp* and *asm* as *tools.build:compiler_executables* for *AutotoolsToolchain*. #20219 . Docs here
- Fix: Improve *copy()* performance when using lists of patterns. #20226
- Fix: Remove the Vigenère cipher used to store JWTs, since it provided no real security benefit. #20227
- Fix: Correct filename parsing in *conan report diff -f=html* for non-ASCII filenames. #20229
- Fix: *XcodeToolchain*'s extra flags (*tools.build:cflags/cxxflags/ldflags/defines*) no longer get overwritten when installing more than one configuration for the same project. #20255 . Docs here
- Fix: Make *test_linker_script* test resilient to the exact linker error text, which changed with Meson >= 1.12. #20257
- Fix: Match the *dll.a* extension (MinGW) in *deduce_full_cpp_info*. #20259
- Fix: Generate valid CycloneDX SBOMs for recipes without a *description* attribute. #20263
- Fix: Add support for lockfile reading in **`conan source`** for Python requires. #20264
- Fix: Add support for python requires in remotes for *conan workspace create*. #20265
- Fix: Failed creates in `workspace` subcommands now raise errors as expected. #20266
- Fix: Correct a typo in the CycloneDX component recipe revision. #20271
- Fix: Do not force *CONFIG* requires for transitive dependencies in *CMakeConfigDeps*. #20283
- Bugfix: Make *CMakeConfigDeps* generate common files only for the host context, avoiding lost CMake variables. #20198
- Bugfix: Keep the local recipe index cache folder when *conan remote add* fails on a duplicate without *–force*. #20210
- Bugfix: intel-cc default compilers on Windows. Classic mode now defaults to *icl* (not *icc*/*icpc*), icx mode to *icx* (not *icx-cl*, which is absent in older oneAPI), and CMakeToolchain now honors *CC*/*CXX* from *[buildenv]* over the hard-coded defaults. #20233

13.4 2.31.2 (04-Aug-2026)

- Bugfix: `conan upload .. --dry-run -f=json > to_upload.json` now allows round-trip for `conan upload -l=to_upload.json` without re-preparing the artifacts. #20237

13.5 2.31.1 (24-Jul-2026)

- Fix: Fixed *replace_in_file()* raising a false "pattern not found" error when the replacement produced the same content as the original (no-op replace). #20211

13.6 2.31.0 (23-Jul-2026)

- Feature: Get Workspace feature out of incubating. #20202 . Docs here
- Feature: *replace_in_file* now supports regex patterns. #20194 . Docs here
- Feature: Warn when *conan config install* is used after *conan config install-pkg*. #20188 . Docs here
- Feature: Support GCC 14.4 and 15.3 in `settings.yml`. #20184
- Feature: Support intel-cc 2025.2, 2025.3 and 2026.1 in `settings.yml`. #20184
- Feature: Change the `Workspace` approach to not process `python_requires` just to derive `name/version`. Proposed a new `get_ref(folder)` method in the `conanws.py` file to be able to define versions dynamically. #20176 . Docs here
- Feature: Define new *cpp_info.ignored_requires* attribute to avoid downstream propagation of dependencies. #20171 . Docs here
- Feature: Add *NO_SONAME* support to *CMakeConfigDeps* to skip SONAME generation. #20168 . Docs here
- Feature: Assert invalid parameters in *ConanFile.run()* method. #20156
- Feature: Add *.tar.zst* (Zstandard) archive support to the *unzip()* tool function. #20140
- Feature: Reduce verbose terminal output produced during the *package()* step. #20134
- Feature: Add *tools.build:asmflags* config and *CMakeToolchain.extra_asmflags* to populate *CMAKE_ASM_FLAGS*. #20127
- Feature: Add `micro` modes to `package_id_modes` to model changes over MAJOR.MINOR.PATCH.MICRO `micro` digit. #20102 . Docs here
- Feature: New `win_runtime_copy` capability of `VirtualRunEnv`, so it can copy on Windows the runtime files of dependencies (the `cpp_info.bindirs` directories contents) to a single location and point to it, avoiding path overflows (like in Windows) for large graphs. #18638 . Docs here
- Fix: Update remaining *audit.conan.io/<page>* links to *conan.io/audit/<page>*. #20203
- Fix: Fix upload with empty uploads from pattern, to make it equal to package lists behavior #20197
- Fix: Avoid conflict with Automake's *LINK* var when using *AutotoolsDeps* with MSVC by using *_LINK_* for linker flags. #20192 . Docs here
- Fix: Add support for VS 2026 in the *Premake* generator (requires Premake 5.0 beta 8). #20189
- Fix: Fix rpath flags from *AutotoolsDeps* being dropped by Libtool on macOS, causing missing *LC_RPATH* in executables. #20179
- Fix: Avoid option names shadowing internal *Options* implementation methods. #20172 . Docs here
- Fix: Fix deprecation warning for emcc/em++ >= 6.0.1, changing `-sMEMORY64` flag with the modern `-m64` one. #20152
- Fix: Avoid a corner-case in component requires when *replace_requires* creates diamond divergent requirements. #20139
- Fix: Fix Artifactory search patterns not matching **@user/channel*, now requiring explicit **/*@user/channel*. #20136
- Fix: Use *LINK* env-var in Windows for AutotoolsDeps `msvc` compiler. #20130 . Docs here
- Bugfix: Require *default_components*, or all existing components, in *PkgConfigDeps* when root *pkg_config_name="none"*. #20169
- Bugfix: Apply sorting to `include()` profiles settings patterns. #20164

- Bugfix: Raise an exception when *layout()* fails in the Docker runner. #20154
- Bugfix: Fix early return in *workspace complete* that could skip necessary packages. #20150
- Bugfix: Fix crash in *replace_requires* when the graph had an override to a replaced requirement that did not apply. #20146

13.7 2.30.0 (29-Jun-2026)

- Feature: Add SPDX expression support in SBOM generation. #20112
- Feature: Add type annotations to `LockfileAPI` for improved IDE support. #20104
- Feature: Add missing favicon to Conan HTML output. #20087
- Feature: Implement the `conf=~` for the "unset" operation (alias for `conf=!`). #20084 . Docs here
- Feature: Add `ASFLAGS` to `AutotoolsToolchain/GnuToolchain`, including architecture and sysroot flags. #20078 . Docs here
- Feature: Add *IntelCC* support in Meson, Autotools and Premake toolchains. #20075 . Docs here
- Feature: Model explicitly the msys2 environments (clang64, ucrt64, etc) in `tools.microsoft.bash:subsystem`. #20074 . Docs here
- Feature: Add `build_system_requirements()` method for installing system tools only in the build context. #20070 . Docs here
- Feature: Add support for `cmake_extra_interface_libs` and `cmake_extra_dependencies` properties to the `CMakeDeps` generator. #20066 . Docs here
- Feature: Support explicit architecture suffixes in system package names for native builds with `host_package=False`. #20047 . Docs here
- Feature: Prevent multiple `.libs` or `.package_framework` entries in CMakeConfigDeps. #20046
- Feature: Add support for Bazel 9.x to BazelDeps. #20042 . Docs here
- Feature: Adapt `bazel_7_lib/bazel_7_exe` templates to support Bazel 9.x (**`conan new`** command). #20042 . Docs here
- Feature: Allow `[replace_requires/tool_requires]` unset in profile composition and extension. #20040 . Docs here
- Feature: Add `tools.meson.mesontoolchain:extra_variables` configuration to `MesonToolchain`. #20032 . Docs here
- Fix: Fix `conan install <path>` failing with absolute paths ending in a dot. #20092
- Fix: Relativize *cpp_info.location* for editable layouts. #20088 . Docs here
- Fix: Improve output messages for the *generate()* step. #20085
- Fix: Make `conan config install` maintain permission bits. #20059
- Fix: Improve error message for broken Jinja2 syntax in `global.conf`. #20056
- Fix: Add Intel `2026.0` version and Intel compiler configuration check. #20049 . Docs here
- Fix: Set *IMPORTED_CONFIGURATIONS* property to package frameworks in CMakeConfigDeps. #20041
- Fix: Remove `build` context suffix from host requirements in CMakeConfigDeps. #20038
- Bugfix: Version-based policy for fixed propagation of `transitive_headers` trait. #20073

- Bugfix: Fix `workspace info` with `python_requires` and a custom user `packages()` method in `conanws.py`. #20064
- Bugfix: Fix some cases where `IMPORTED_IMPLIB` was not being declared when extension was set in `cpp_info.libs`. #20062
- Bugfix: Fix caching of different revisions of the same package while resolving the graph. #20061
- Bugfix: Fix `replace_requires` applying when command line arguments explicitly define a package reference. #20048 . Docs here
- Bugfix: Check whole git repository for dirtiness when using *revision_mode=scm*. #18630

13.8 2.29.1 (12-Jun-2026)

- Bugfix: Handle special case where only one possible option is defined for shared. #20082

13.9 2.29.0 (28-May-2026)

- Feature: Limited support for `python_requires` in workspace, only in `conanws.yml` file. #20028 . Docs here
- Feature: Added new public attribute *binaries* to the MesonToolchain generator. #20017
- Feature: Add support for Apple OS 26.5 release. #19976
- Feature: Document *RemoveAPI*. #19930
- Feature: New `global_user.conf` file to locally customize `global.conf`. #19923 . Docs here
- Feature: Add support for GCC 16. #19921
- Fix: Forward `-vxxx` verbosity argument to `conan workspace build/install` packages. #20015
- Fix: Add message when no packages found in conan list command. #20013
- Fix: `source_credentials.json` now supports supplying only headers #20004 . Docs here
- Fix: *conan workspace add –output-folder* now properly updates the *output_folder* of an existing package entry in *conanws.yml* instead of ignoring it. #19988
- Fix: Default `variant` for QNX and VxWorks in default `settings.yml` should be `null`, not `None`. Also, add `qcc=12.2` version. #19981
- Fix: *runtime_deploy*: avoid crashing when overriding existing symlinks. #19977
- Fix: *AutotoolsDeps* will link with *-rpath* on shared package types and components correctly. #19975
- Fix: Warn and re-fetch when requested URLs conflict with existing ones with the same SHA256. #19969
- Fix: *cppstd_flag* for *clang-cl* now correctly maps C++23, C++26 and *gnu++* standards instead of producing invalid */std:* flags. #19965
- Fix: Correct the `conan workspace init` docstring. #19950
- Fix: Allow workspaces root Conanfile to have a name to apply `conf` with patterns that would match that name. #19927
- Fix: Fix *meson_lib* missing deps, *meson_exe* layout & install verbosity. #19918
- Bugfix: Raise an error if *package_type* and *shared* option mismatch (`package-type` is set to `shared-library` and `shared=False` or `package_type=static-library` and `shared=True`) #20023

- BugFix: Fix the computation of `options` in `graph build-order`. #19950
- Bugfix: Fix requirement definition for multilib components without explicit requirements for `CMakeConfigDeps`. #19942
- Bugfix: *MSBuildDeps* no longer generates duplicate include and library directory paths in multi-component packages. #19937

13.10 2.28.1 (30-Apr-2026)

- Bugfix: Fix regression for downloads without sha256. #19934

13.11 2.28.0 (28-Apr-2026)

- Feature: `conan upload --allow-disabled` to allow uploading to a disabled remote. #19916
- Feature: Allow using patch-ng 1.19 to incorporate fixes #19913
- Feature: Use a OR policy between `core:policies` and recipe `required_conan_version`. #19907 . Docs here
- Feature: `conan config install-pkg --insecure` new feature. Also supported for `conanconfig.yml` files #19900 . Docs here
- Feature: Introduce policies in `core:policies` conf to control Conan behaviour. #19892 . Docs here
- Feature: Colorize output of *conan config list* and *conan config show* #19889
- Feature: Add ability to show transitive requires in *conan graph info ... -f=html* output #19884
- Feature: Add ability to show node subgraph in *conan graph info ... -f=html* output #19884
- Feature: Add ability to filter by file extensions in *conan report diff ... -f=html* output #19884
- Feature: New compiler flags `flags_map()` Python plugin to be able to translate, remove or handle compiler flags coming from compatible binaries built with a different compiler trying to inject compiler flags for that compiler. #19879 . Docs here
- Feature: *tools.build:install_strip* now accepts a list of possible build systems #19874 . Docs here
- Feature: Add new "certified" variant to settings `VxWorks` and new "safe" variant to `Neutrino` #19861
- Feature: Allow patterns for recipe names in *–update* flag #19856
- Feature: Add –strict flag to *conan remote auth*. #19848 . Docs here
- Feature: Avoid detecting default package manager when overridden from profile #19847
- Feature: Add new experimental contextual output for **`conan export`** command #19836
- Feature: Added default package manager for CachyOS. #19788
- Feature: Introduce new `consistent=True` requirement trait to be able to have diamond structures for `visible=False` requirements. #19286 . Docs here
- Fix: Test publishing Conan wheels to test PyPi. #19906
- Fix: Ignore local package lockfiles in `conan workspace install/build` and use consistently a global lockfile if provided or found by default. #19896
- Fix: Force `--order-by` in `conan graph build-order`. Old deprecated behaviour can be restored until Conan 2.32 with the `deprecated_build_order_args` policy #19892 . Docs here

- Fix: Remove support for empty version ranges. Old deprecated behaviour can be restored until Conan 2.32 with the `deprecated_empty_version_range` policy #19892 . Docs here
- Fix: Remove deprecated `system_tools` profile section #19877
- Fix: Remove deprecated `detect_compiler` method in detect api #19877
- Fix: Remove deprecated `deploy` folder in conan home #19877
- Fix: Remove deprecated methods from `PackagesList` #19877
- Fix: Remove deprecated `cmake_set_interface_link_directories` property #19877
- Fix: Remove deprecated `Node::dependencies` method #19877
- Fix: Warn when credentials environment variables are set but not used because the server accepted anonymous access and point users to add *–force* to force authentication. #19872
- Fix: Documenting `CommandAPI` and better docs for `ProfilesAPI` #19871
- Fix: Fix output of options with "error" in its name in *conan list* command. #19867
- Fix: Forward underlying system package manager error messages #19858
- Fix: Deprecate `build_requires`, use `tool_requires` instead #19849 . Docs here
- Fix: Inline transitive dependencies to avoid Xcode recursion crashes. #19844 . Docs here
- Fix: *finalize()* output folder should be printed only once #19834
- Fix: Change default `core.download:retry_wait` from 0 to 1 second. Document `retry` conf defaults for `conan config list` #19830
- Fix: Use user locale in *conan list ... -f=html* output. #19828
- Fix: Avoid subtle errors with casing errors like `requires("myPkg/[*]")` using version ranges. #19799
- Fix: Add support for Xcode 26.4 with Apple Clang 21. #19795
- Fix: Add `-ur/-ubr/-upr` for `conan lock upgrade` as short forms for `--update-requires`, etc. #19791
- Fix: Propagate build requirement run trait to upstream shared dependency. #19751
- Fix: Make components from same package full link in `CMakeConfigDeps`. #19641
- Bugfix: Solve crash of `conan workspace install/build --lockfile=mylock --lockfile-partial` #19896
- Bugfix: `conan install --lockfile=xxxx` will raise if the lockfile contains `config_requires` and the current installed configuration packages do not align with it #19875 . Docs here
- Bugfix: Change propagation on bindirs for `VirtualBuildEnv` respecting requirement `run` trait, based on the new `required_conan_version=">=2.28"` recipe version or `global.conf` using `core:policies=["required_conan_version>=2.28"]` #19849 . Docs here
- Bugfix: Solve incorrect `.ps1` file generation when unsetting the conf with `-c tools.env.virtualenv:powershell=!`. #19820
- Bugfix: Fix `transitive_libs=True` when using `CMakeConfigDeps` for shared libraries. #19815
- Bugfix: Change computation of `package_id` for transitive static libraries, based on the new `required_conan_version=">=2.28"` recipe version or `global.conf` using `core:policies=["required_conan_version>=2.28"]`. #19705 . Docs here

13.12 2.27.1 (13-Apr-2026)

- Bugfix: Revert quote escaping changes in defines for *NMake* integrations. #19859

13.13 2.27.0 (25-Mar-2026)

- Feature: Feature: Allow negated OR patterns `!(<pattern1>|<pattern2>|...)` in profile `[tool_requires]` for breaking cycles in build context. #19780 . Docs here
- Feature: Feature: Add CVE version info to `conan audit` results. #19774
- Feature: Disable CMake user package registry exports by default (*cmake_policy(SET CMP0090 NEW)* and *CMAKE_EXPORT_PACKAGE_REGISTRY OFF* when unset). #19766
- Feature: Legacy Conan 1.X alias support has been removed. #19740
- Feature: PyEnv output based on verbosity level #19731
- Feature: The HTML graph representation learned to show transitive requirements. #19725
- Feature: Add support for Clang 22. #19709
- Feature: Show cycles/loops in `conan graph info --format=html` with a red arrow. #19694
- Feature: Add `tools.build:rcflags` configuration to inject flags for RC. #19693 . Docs here
- Feature: Add support for Apple OS 26.3 releases. #19691
- Feature: Feature: New `tools.cmake:configure_args` configuration to inject arbitrary arguments into the CMake `configure` step via the command line, allowing users to inject CMake variables and arguments such as `--fresh`. #19639 . Docs here
- Fix: Allow `NMake` integrations to handle defines such as `WINVER=0x0601` as numeric, not strings. #19779
- Fix: Correct definition of `set_property()` in `CMakeConfigDeps` for build context. #19760
- Fix: Show a clear error message when defining `[platform_xxx_requires]` with a version range. #19750
- Fix: Only group build packages in `conan graph info .. -f=html`. #19744
- Fix: Solve issue with overrides and lockfiles. #19739
- Fix: Improve the error message for authentication with source credentials. #19737 . Docs here
- Fix: Fix missing libraries in legacy `<packagename>_LIBRARIES` variable definition in `CMakeConfigDeps` generator. #19724
- Fix: Relax the `CMakeConfigDeps` requirement to explicitly declare the CMake `C` language in `CXX` projects when linking C dependencies, since this is already implicit in CMake. #19704
- Fix: Create stubs correctly for user CMake presets when `user_presets_path` is specified. #19251
- Bugfix: Solve assertion with `--build=editable` and usage of `tools.build:download_source=True` conf. #19758
- Bugfix: Fix `detect_emcc_compiler` on first run and on Windows. #19735
- Bugfix: Define correct bash usage for `win_bash_run=True` with `self.run(..., scope="run")`. #19703
- Bugfix: Solve path issues with `tools.gnu:make_program` in Windows subsystems. #15047

13.14 2.26.2 (05-Mar-2026)

- Bugfix: Fix exception when *conan cache check-integrity* finds a corrupted recipe. #19713
- Bugfix: Revert regression in escaping `CMakeToolchain.variables`, those variables will not be automatically escaped. #19706

13.15 2.26.1 (27-Feb-2026)

- Bugfix: Avoid missing binaries due to default platform requires revision. #19680

13.16 2.26.0 (25-Feb-2026)

- Feature: Expose PyEnv *env_dir* (venv root), *env_exe* (venv python executable), and *bin_path* (bin/Scripts directory). #19628
- Feature: Document publicly `LocalAPI`. #19623 . Docs here
- Feature: Create deployers that generate CycloneDX SBOMs. #19611 . Docs here
- Feature: New `important!` conf that allows `tool-requires` `conf_info` to have higher relative priority over profiles `conf`. #19610 . Docs here
- Feature: Don't check for user/channel match in <`host_version`>. #19599 . Docs here
- Feature: Allow disabling environment script generation from recipes using *virtualxxxenv = False*. #19594 . Docs here
- Feature: Add *detect_api.detect_emcc_compiler* to detect EMSDK Emscripten compiler version. #19592 . Docs here
- Feature: Generate *.sh* scripts with variable existence checks to harden scripts and avoid extra separators for empty variables. #19591
- Feature: Optimize LRU database updates by using filesystem folder mtimes. #19582
- Feature: Enable access to the *author* attribute in the *ConanFileInterface* class. #19577
- Feature: Add *tools.build:add_rpath_link* conf (Meson and CMake toolchains) to pass *-rpath-link* with all directories of host dependencies #19574 . Docs here
- Feature: Add *cmake_file_name_variants* support to *CMakeConfigDeps* to allow packages to define additional lower/upper-case variants that consumers may use when calling *find_package*. #19530 . Docs here
- Feature: Add *.bat* support for `tools.env:deactivation_mode=function`. #19474 . Docs here
- Feature: New *package_id_abi_options* to allow specific dependency options to affect the consumer *package_id* when headers variability (e.g. *shared*) can impact consumer binaries, even in non-embed cases. #19438 . Docs here
- Feature: New *conan cache sign* and *conan cache verify* commands for signing and verifying packages. #19345 . Docs here
- Fix: Use lazy imports to avoid circular dependency so PyInstaller bundles *conan.tools.system*. #19670
- Fix: Fix legacy definitions syntax for `CMakeConfigDeps` #19662
- Fix: Add support for *.txz* and *.tzst* extensions to conan cache save help output. #19660 . Docs here

- Fix: Allow requires-only components to create a target with `CMakeConfigDeps` #19645
- Fix: `MSBuildDeps` bug with transitive build requirements and components. #19625
- Fix: Improve the `-DCMAKE_TOOLCHAIN_FILE` tip in CMakeToolchain generator to abstract it to a `<output_folder>`. #19602
- Fix: Fix CPS parsing of package preprocessor definitions. #19539
- Fix: Deprecate Python 3.7 warning for Conan. #19535 . Docs here
- Fix: Update terminology to use "hash" instead of "signature". #19522 . Docs here
- Bugfix: Solve `CMakeConfigDeps` issue with in-package config.cmake files that were ignoring `cmake_file_name_variants`. #19669
- Bugfix: Fix `conan list --graph-context={build,host}-only` for consumer recipes without a name #19657
- Bugfix: Force parsing of `conf` like `tools.microsoft:msvc_update` as a string, to avoid parsing it as float and dropping trailing zero. #19647
- Bugfix: Avoid `--build=compatible` to rebuild an already existing binary #19643
- Bugfix: Correctly escape `CMakeToolchain.variables` for CMake syntax. #19642
- Bugfix: Fix serialization of *cpp_info* when it uses the type field. #19604
- Bugfix: Ensure `CPS` component Cmake targets follow expected name pattern. #19584
- Bugfix: Fix corruption of *[buildenv]* information when using per-package patterns across multiple packages. #19571
- Bugfix: Add default `#platform` revision to `platform_{tool_}requires`. #19561 . Docs here
- Bugfix: Add missing *riscv64* mappings for *yum* and *apt*. #19560

13.17 2.25.2 (04-Feb-2026)

- Fix: Revert atomic `os.replace` for package binary downloads due to antivirus Windows issues. #19565

13.18 2.25.1 (29-Jan-2026)

- Fix: Do "retry" over the `os.replace()` in Windows to avoid antivirus blocking issues. #19532

13.19 2.25.0 (28-Jan-2026)

- Feature: Make the download of package binaries more atomic to make cancellations more robust. #19510
- Feature: Add public docs for *InstallAPI* subapi. #19497
- Feature: **`conan new`** with no positional arguments creates a default CMake basic conanfile. #19496 . Docs here
- Feature: Further optimize the number of DB calls for upload, download, and package builds. #19485
- Feature: Add public documentation for `CacheAPI` and `ConfigAPI` subapis. #19479
- Feature: Deprecate *MesonToolchain.preprocessor_definitions* in favor of *extra_defines*. #19468

- Feature: Add *conan require* command to add/remove requirements to/from your local conanfile. #19457 . Docs here
- Feature: Support for CPS CMake round trip with components with `requires`. #19446
- Feature: Add a way to specify link features (*$<LINK_LIBRARY:...>*) in *CMakeConfigDeps*. #19444 . Docs here
- Feature: Update settings with new versions of supported tools. #19442
- Feature: CPS CMake-Conan round trip support for components. #19428
- Feature: Move `CMakeConfigDeps` from incubating to experimental. #19421 . Docs here
- Feature: Support CPS shared libs from CMake. #19417
- Feature: Support full CPS CMake round trip in `CMakeConfigDeps`. #19410
- Feature: Optimize package cache DB access, reducing connections and queries by half for the dependency graph construction. #19398
- Feature: Add *root_profile_name* to the profile jinja2 context to allow tracing back the root profile from included profiles. #19393 . Docs here
- Feature: Optimize package cache DB access by doing batch updates of the LRU recipes and packages. #19392
- Feature: Define the Python version used by PipEnv using UV #19388 . Docs here
- Feature: (Experimental) Support built-in *xz* and *zstd* compression for Conan artifacts. #19337 . Docs here
- Fix: Check Python version in *PyEnv* init. #19520
- Fix: Populate *<library>_DEFINITIONS* legacy variables in *CMakeConfigDeps* for compatibility with old *check_symbol_exists* and similar #19519
- Fix: Use `dnf` as the default system package manager for Almalinux, Rocky and Oracle Linux instead of legacy `yum`. #19487
- Fix: Display packages even when a system package manager install is a no-op. #19483
- Fix: Remove `--lockfile-out` generation arguments in `workspace` commands that cannot generate a lockfile (orchestrated `conan workspace install/build/complete`). #19475
- Fix: Reduce the sqlite DB connection scope to try to optimize DB locking. Remove the `yield` DB return that could create operational issues. #19394
- Bugfix: Avoid potential *None* and *PackageType* comparison when deducing *cpp_info*. #19494
- Bugfix: Solve `PipEnv` failure when using version ranges. #19478
- Bugfix: Correct pattern comparison for symlink with *files.copy* function. #19437
- Bugfix: Solve issue in `CMakeConfigDeps` when building transitive libraries in the "build" context. #19429
- Bugfix: Allow updating to newer remote revisions that already exist in the Conan cache with an older timestamp. #19402
- Bugfix: Fix unintended packages showing up when using *conan list ... –graph-context={build-only,host-only}* when there are package binary mismatches #19368

13.20 2.24.0 (15-Dec-2025)

- Feature: MesonToolchain *needs_exe_wrapper* property now listens to *can_run()* function. #19382
- Feature: Workspace super-install now follows *layout()*. #19376 . Docs here
- Feature: Make diff symbols in *conan report diff ...* *-f=html* non-selectable. #19375
- Feature: Add more public sub Python APIs, some more typing and fix docstrings. #19370 . Docs here
- Feature: Represent missing packages in *graph.html* output. #19360
- Feature: *conan cache check-integrity*: new JSON output format. #19343 . Docs here
- Feature: New `conan workspace complete` command to open/add intermediate packages to the workspace. #19331 . Docs here
- Feature: Inject `hashlib` in the `global.conf` jinja2 rendering to be able to compute hashes (for paths, for example). #19319 . Docs here
- Feature: Introduce new `conanconfig.yml` file that can store packages for multiple automatic `conan config install-pkg`. #17793 . Docs here
- Fix: *defines* and *frameworks* now also generate *CMakeConfigDeps* targets. #19357
- Fix: Warn on version ranges in reference pattern, which have no effect (ie `-o="foo/[>1]:shared=True"`). #19356
- Fix: Improve `MSBuildToolchain` docstrings for `compile_options`. #19351 . Docs here
- Fix: Conan profile detect can detect Visual Studio 18 (2026). #19348
- Fix: Avoid `AutotoolsToolchain/GnuToolchain` to define `--sysroot` for QNX `qcc` compiler, define `-Wc,-isysroot` instead. #18897
- Fix: Fixed `conan_config.json` storage to use only RecipeReferences (not PackageReferences). #17793 . Docs here
- Fix: `conan config install-pkg` will report for order-changing updates to existing configuration installs, allowing to `-force` to change the order. #17793 . Docs here
- Bugfix: Do correct scoping of `conan install --requires=dep/[*] -o myoption=value`. #19367
- Bugfix: fix new `core.graph:compatibility_mode=optimized` with multiple repos. #19349
- Bugfix: Fix crash for `conan workspace build` with external global editables. #19338
- Bugfix: Bugfix: Fix CMakeConfigDeps flags handling of generator expressions with separators. #19330
- Bugfix: Fix logic in `EnvVars` generation of `.sh` and `.ps1` scripts for "unset" vs "empty" definition. #19328 . Docs here
- Bugfix: Fix `Workspace` crash when passing per-package configuration. #19327
- Bugfix: Fix version range pattern replacement in *replace_requires*. #19324

13.21 2.23.0 (25-Nov-2025)

- Feature: Adding optional `cli_args` to meson install. #19301
- Feature: implement `VCVars` support for latest VS 18 2026. #19294
- Feature: Implement `ClangCL` support for VS 18 2026, add `v145` to the clang vs-runtime. #19289
- Feature: Improve `conan pkglist find-remote` to handle partial information, like not providing revisions. #19265
- Feature: Add message when compatibility does not find a matching package. #19262
- Feature: Improve `Workspace` error reporting for user code, and allow workspace conanfile to execute `self.run()` commands. #19260
- Feature: Introduce `workspace_packages` so `workspace super-install` can collect information from workspace packages. #19245 . Docs here
- Feature: New parameter to support custom Premake configuration names. #19242 . Docs here
- Feature: Add collapse all and expand level buttons to *conan report diff html* output. #19240
- Feature: Add support for apple os versions 26.1 and corresponding SDKs. #19239
- Feature: Show *provides* conflicts in *conan graph info -f=html*. #19222
- Feature: Allow `source_credentials.json` and the source auth plugin to define headers. #19206 . Docs here
- Feature: Let `conan install/create` output the resulting `graph.json/html` even when there are build failures, to allow listing possible packages that have been built. #19204
- Feature: Document the `Remote()` constructor as public API. #19200
- Feature: `CMake.ctest()` runner new `tools.cmake:ctest_args` conf. #19198 . Docs here
- Feature: Add some more useful output to *conan audit*. #19197
- Feature: Allow `distro 1.19` python pip package dependency for broader compatibility. #19192
- Feature: Support metadata files addition without previously downloading other metadata files. #19185
- Feature: Add rename info for files in *conan report diff*. #19171
- Feature: Add *conan run* command to run commands from packages #18972 . Docs here
- Feature: Compatibility checks are now performed in a single request to each remote #18396
- Fix: Propagate *verbosity* confs to *CMakeToolchain*. #19296
- Fix: Fix 15.7 version in settings.yml. #19250
- Fix: Remove empty value from *–verbose* choice help string. #19244
- Fix: Improve error message referencing non existing `git_excluded`, use `core.scm:excluded` instead. #19232 . Docs here
- Fix: [MesonToolchain] Omits the *'sys_root'* property field. #19229
- Fix: CMakeConfigDeps management of `cmake_extra_interface_libs` per component. #19187
- Fix: Added deployer arguments to *workspace super-install* command. #18792
- Fix: Added *format_graph_json* formatter to *workspace super-install* command. #18792
- Bugfix: Make `layout()` have higher precedence for `self.layouts.build.xxxenv_info/conf_info`. #19268

- Bugfix: Fix bug in `conan new cmake_lib` template without arguments, incorrect function name. #19257
- Bugfix: *Conanfile.run*'s *quiet* parameter now silences the output of the command #18972 . Docs here
- Bugfix: Verbosity level *quiet* now also silences the output of tools ran by Conan #18972 . Docs here

13.22 2.22.2 (07-Nov-2025)

- Bugfix: Fix assert when finding compatible binaries of a package that exists in both contexts with different settings. #19208

13.23 2.22.1 (30-Oct-2025)

- Bugfix: Revert "Error out if components miss requiring direct dependencies in more cases". #19168

13.24 2.22.0 (29-Oct-2025)

- Feature: Make the remote name used by `conan config install-pkg --url=<url>` public. #19132 . Docs here
- Feature: Add support for macOS 15.7 and iOS 18.7. #19130
- Feature: Group arguments in CLI help. #19126
- Feature: Suggest possible typos for CLI arguments declared as string choices. #19126
- Feature: Add opt-in conf to control making *.sh* and *.ps1* env *deactivate* functionality into in-memory functions instead of files. #19105 . Docs here
- Feature: Document `ExportAPI`. #19103
- Feature: Enable parallel download of packages by default, by defaulting `core.download:parallel` to the available CPU cores. #19099 . Docs here
- Feature: Support `conan graph info --package-filter=&` pattern. #19080 . Docs here
- Feature: Add new environment variable *CONAN_DEFAULT_BUILD_PROFILE* for default build profile. #19040 . Docs here
- Feature: Allow nullifying settings from profiles and command line. #19035 . Docs here
- Feature: Support compatibility plugin removal of nullable settings. #19031 . Docs here
- Feature: New `tools.gnu:disable_flags` configuration to allow disabling the injection of some build system flags. #19014
- Feature: Expose *recipe* in the *ConanFileInterface* for information purpose only. #18995 . Docs here
- Feature: New `CMakeConfigDeps` properties to inject extra dependencies and targets. #18316
- Fix: Better error message for `CMakeConfigDeps` when the package_type or component type is not defined for something with `.location` defined. #19096
- Fix: Change order of `build_modules` inclusion in `CMakeConfigDeps` so it happens after legacy-vars, to support some ConanCenter recipes abusing those variables. #19094
- Fix: Reintroduce *settings.yml* access to *config* Sub-API. #19078

- Fix: Raise an error if `conan list * --lru=xx`, recommending the `#<rev-pattern>` argument. #19077
- Fix: Sanitize XcodeDeps file and variable names to use only valid xcconfig characters. #19075
- Fix: Explicitly set `allow_empty=True` in `glob()` function in `BazelDeps` (Bazel 8.x compatible). #19068
- Fix: Fix `CMakeConfigDeps` when a regular library `requires()` an application, using components. #19052
- Fix: Add missing final newline when saving lockfiles to disk. #19043
- Fix: Warn when adding requirement to version range with pinned revision, it has no effect. #19041 . Docs here
- Fix: Fix `CMakeConfigDeps` escaping. #19034
- Fix: Improved Python virtual environment creation in `PipEnv` by using the system-installed interpreter or a user-defined one via `tools.system.pipenv:python_interpreter`. #19030 . Docs here
- Fix: Add VS2026 CMake generator mapping. #19024
- Fix: Avoid referencing xcconfig from skipped dependencies required in components in XcodeDeps. #19023
- Fix: Add `execution["jobs"]` to the generated CMake `testPresets` with same logic and value as `buildPresets`. #19021
- Fix: Improve the error message when a workspace `super-install` defines intermediate packages in the cache depending on workspace packages. #19013
- Fix: Improve support for huge diffs in *conan report diff* HTML output. #19012 . Docs here
- Bugfix: `MSBuildToolchain` explicitly adds the specific toolset .props file when `compiler.update` is defined, otherwise, activating `vcvars` is not enough. #19137
- Bugfix: Fixes an issue where the Apt packages for the build arch would be reported missing, in cross-compiling scenarios, even though they are installed. #19074
- Bugfix: Solve unexpected conflict when pinning a `recipe-revision` directly in a conanfile that is not the latest, and having other dependencies resolving first to the latest recipe revision. #19038
- Bugfix: Fix <`host_version`> resolution in certain transitive cases. #18947
- Bugfix: Fix orphan nodes being created when expanding the dependency graph for some cases. #18947
- Bugfix: Error out if components miss requiring direct dependencies in more cases. #18830

13.25 2.21.0 (29-Sept-2025)

- Feature: Add support for universal binaries to AutotoolsToolchain. #18992 . Docs here
- Feature: Add support for universal binaries to GnuToolchain. #18992 . Docs here
- Feature: Add *–context={build,host}* filter to *conan audit scan*. #18976 . Docs here
- Feature: Default recipe paths to cwd if not specified when calling Conan. #18964
- Feature: Implement a new `post_package_id()` hook. #18960 . Docs here
- Feature: Add support for Xcode 26 and related. #18953
- Feature: Add support for MSVC VS 2026 (insiders at the moment). #18948 . Docs here
- Feature: Add the PipEnv tool to install python tools using pip in an isolated virtual environment. #18923 . Docs here
- Feature: Allow separate build and package directories for multiple CMake builds from single conanfile. #18905 . Docs here

- Feature: Add *recipes_only* field to remote to control whether a remote can be used to download binaries. #18896 . Docs here
- Feature: Add *cmake_extra_variables* property for *CMakeConfigDeps*. #18822 . Docs here
- Feature: Add *cmake_extra_variables* property for *CMakeDeps*. #18822 . Docs here
- Feature: Improve *conan report diff* html UX. #18686 . Docs here
- Feature: Allow passing build configuration in *XcodeBuild* explicitly via *configuration* parameter. #18668 . Docs here
- Feature: Allow passing arbitrary command line arguments to *XcodeBuild* via *cli_args*. #18668 . Docs here
- Feature: Add generation of `dotenv` environment files with `tools.env:dotenv=True` conf. #18266 . Docs here
- Fix: Avoid `CMakePresets` adding the `jobs` field when `tools.build:jobs=0`. #18984
- Fix: Warn when exporting recipes with versions containing alphanumeric majors. #18980 . Docs here
- Fix: Set *cpp.source.includedirs* to *include* in *basic_layout*. #18958 . Docs here
- Fix: `workspace add` can update the package version of an existing package in the workspace. #18955
- Fix: Move macOS bitcode flag testing to integration test. #18930
- Fix: Move *test_requires* to *build_requirements* method in tests. #18929
- Fix: Always run *validate* hooks even if recipe does not define *validate()* method. #18928
- Fix: Remove deprecated, old and undocumented features, marked for deprecations for a long time. #18920 . Docs here
- Fix: Ensure `VCVars` generated `conanvcvars.bat` has normalized path with backward slash (Windows). #18907
- Fix: Fix *compatibility.py* migration overwrite when no changes were necessary. #18882
- Fix: Cleaning `files` and `upload-urls` from "package lists" after a download or when skipping uploads. #18878
- Fix: Improve the error message and avoid the traceback when a `build-scripts` package tries to depend on a `library` in the "host" context. #18869 . Docs here
- Fix: Move legacy CMake vars from `xxx-target-<config>.cmake` to `xxxx-config.cmake` file. #18860
- Bugfix: Added Apple frameworks support for BazelDeps. #19004
- Bugfix: Ensure *conan graph build-order* errors out when passing both a recipe path and a `--requires` reference. #18964
- Bugfix: MesonToolchain no longer add quotes to `linker_script` definition. #18922
- Bugfix: Fix missing double quotes for MSBuild's commands. #18911
- Bugfix: Omit the `test_requires` when `cyclonedx(..., add_test=False))` in all cases, including **`conan install`** consumption. #18873
- Bugfix: Command `conan list` with version-ranges can now listen to `core.version_ranges:resolve_prereleases=True` to list pre-releases. #18868
- Bugfix: Stabilize *PackagesList* methods. #18833

13.26 2.20.1 (04-Sept-2025)

- BugFix: Fix Apt not detecting the correct architecture in multiarch setups. #18872
- BugFix: Apt correctly detects arch-independent (all) packages during cross-building to avoid unnecessary reinstalls. #18872

13.27 2.20.0 (01-Sept-2025)

- Feature: new *check_min_compiler_version* validator which simplify compiler restriction description in recipes. #18849 . Docs here
- Feature: Adding Clang 21 to the default `settings.yml`. #18846
- Feature: Add new Apple OS's versions. #18845
- Feature: Avoid the generation of `conanintelsetvars` script by `IntelCC` if the `tools.intel:installation_path=""`, similarly to `VCVars` generation. The user should have already activated the IntelCC environment on their own before running. #18840 . Docs here
- Feature: Improved the CMakeLists.txt file created by the *conan new workspace* command for super-builds. #18838
- Feature: Add `CMAKE_FIND_PACKAGE_PREFER_CONFIG=ON` for `CMakeConfigDeps` generator. #18832
- Feature: Add *excludes* pattern support for *get* and *unzip* methods. #18831
- Feature: Add support for GCC 15.2. #18735
- Feature: Add iOS 18.5 and tvOS 18.5 to *default_settings.yml* template. #18722
- Feature: Stabilize and document `ConfigAPI` public Python sub-API. #18709
- Feature: Print build-order in the `conan workspace super-install` command #18693 . Docs here
- Feature: Workspace super-build options aggregation. #18608 . Docs here
- Feature: Implement *tools.build:install_strip* for Autotools. #18606 . Docs here
- Feature: Added support to the system_package tool for defining the system package version to be installed. #18517 . Docs here
- Fix: Better error message when there is an existing file called "build" in the same location as the "build" build-folder is expected to be created. #18842
- Fix: The `cmake_layout` was not taking into account the Apple multi-arch/universal separator when creating folders named after the `arch` setting. #18823
- Fix: Avoiding issues if passing non-string objects to `ConanOutput` methods. Still, the input to several `.info()` and similar methods must be "text", passing arbitrary objects and expecting them to convert to strings internally is not supported. #18782 . Docs here
- Fix: Document public interface for `ConanOutput` class. #18782 . Docs here
- Fix: Remove Python 3.6 support, End Of Life since 2021. #18779 . Docs here
- Fix: Make all non-documented subapi attributes private. #18736
- Fix: Remove *SearchAPI* in favour of *ListAPI*'s *select()*. #18726 . Docs here
- Fix: Relax the "risk" warning for conflicting visibility in `test_requires`. #18723

- Fix: Fixed an issue that caused APT packages without a defined architecture to be detected if one with the same name was installed for a different architecture. #18517 . Docs here
- Fix: Preserve subfolders for *runtime_deploy* deployer. #17848 . Docs here
- Bugfix: Made *ConanAPI*'s *home_folder* read-only. #18726 . Docs here

13.28 2.19.1 (30-Jul-2025)

- Fix: Remove the definition of `CMAKE_TRY_COMPILE_CONFIGURATION` in `CMakeToolchain` to avoid issues with `check_function_exists()` legacy code in MSVC. #18707

13.29 2.19.0 (23-Jul-2025)

- Feature: Changed some private attributes in MesonToolchain as public ones, e.g., *b_ndebug*, *b_staticpic*. #18676
- Feature: PremakeDeps will now correctly propagate libraries, headers and binaries depending on the requirement traits. #18663
- Feature: Add *cmake_target_aliases* support for *CMakeConfigDeps*. #18662
- Feature: Add `self.conan_data` to the information serialized by ConanFile, so it is printed in `conan graph info` and other commands. #18661 . Docs here
- Feature: Let **`conan source`** reference the backup sources it generates in more cases. #18655
- Feature: Add user channel to CycloneDX SBOM `sbom_ref` field. #18649
- Feature: Enable `test_package_folder` attribute for **`conan export-pkg`** command. #18621 . Docs here
- Feature: Add support for GCC 12.5. #18587 . Docs here
- Feature: New *makefile* parameter in Autotools *make/install* methods to allow specifying the name of the Makefile file. #18578
- Feature: Let graph html focus on searched package when pressing Intro in search box. #18575
- Feature: Allow defining a custom platform on Premake generator for Windows. #18572
- Feature: Allow profile composition while using conan runners. #18534
- Fix: Add explicit error when trying to export a reference with *channel* but no *user*. #18646 . Docs here
- Fix: Check *required_conan_version* before loading hooks. #18644
- Fix: Avoid logging levels hiding the login username/password request messages. #18642
- Fix: Allow `cc` compiler to be defined with spaces for profile auto detection. #18628
- Fix: Fixed untargz when the destination path uses the Windows long paths prefix \?`. `#18612
- Fix: *CMakeConfigDeps* filter *requires()* to *package_type=application*. #18611
- Fix: Fix multithreading for self-contained Conan binaries. #18603
- Fix: Improve version detection for *cc* compilers. #18600
- Fix: Pass deployment target from profile to `XcodeBuild`. #18496 . Docs here
- Fix: Project path and target name are quoted now for `XcodeBuild`. #18496 . Docs here
- Bugfix: Make *package_type="configuration"* packages independent of the *config_mode* for their *package_id*. #18671

- Bugfix: PremakeDeps: ensure correct linkage on dependent libraries. #18631

13.30 2.18.1 (04-Jul-2025)

- Bugfix: Revert remote caching for missing packages #18586

13.31 2.18.0 (30-Jun-2025)

- Feature: Allow consuming meson libname.a libs in `MSBuildDeps`. #18557
- Feature: Avoid library renames when using Meson + MSVC + static builds. #18533
- Feature: Added *threads* subsetting in *emcc* compiler model. #18520 . Docs here
- Feature: New `conan cache ref <path>` to reverse look the Conan cache, with a path argument will return the reference of the artifact in that folder. Intended exclusively for debugging purposes. #18518 . Docs here
- Feature: New linker flags autodetected by conan based on profile architecture. #18498
- Feature: Changed *conanws.yml* format. Now, *packages* is a list of dict-like objects. #18493 . Docs here
- Feature: Added support for `.exe` in editables packages in `CMakeConfigDeps`. #18489
- Feature: Add *build_folder* parameter in *basic_layout*. #18442 . Docs here
- Feature: Using *pkg_config_name = 'none'* to skip the **.pc* file creation. #18439 . Docs here
- Feature: Add support for sbom and lockfiles to *conan audit list*. #18437 . Docs here
- Feature: Added first class citizen emscripten support (new wasm64 architecture + emcc). #18432 . Docs here
- Feature: Replace *tools.cmake:install_strip* by *tools.install:strip*. Affect both CMake and Meson tool helpers. #18429 . Docs here
- Feature: Add *open* to *TestClient* to open files locally. #18399
- Feature: New `conan workspace create` orchestrated. #18390 . Docs here
- Feature: Add `context` variable to profile jinja2 rendering (can be "build", "host" and `None`). #18383 . Docs here
- Feature: Implement `cpp_info.sources` to support source targets. #18350 . Docs here
- Feature: Add support for source targets in CMakeConfigDeps generator. #18350 . Docs here
- Feature: New *conan report diff* command to inspect diffs between versions and revisions. #18247 . Docs here
- Feature: Add premake toolchain and improved premake integration in conan with new premake5. #17898 . Docs here
- Fix: Better error message in `CMakeConfigDeps` for incorrect component requires. #18562
- Fix: Avoid incorrect absolute path inputs in `-of` for relativize paths in generators. #18561
- Fix: Better error message when an incorrect `cpp_info.requires` is defined. #18552
- Fix: Avoid hyphens for msbuild verbosity argument passed to CMake after – by powershell. #18548
- Fix: Improve *conan cache check-integrity* output. #18544
- Fix: Raise an error for incorrect definition of `conf_info` items. #18541
- Fix: Fix `qcc` `cppstd` support for latest QNX 8.0 with c++20. #18538

- Fix: SBOM component *bom-ref* should not use *has_special_root_node*. #18515
- Fix: Add a deprecated warning message for `Node.dependencies`, now renamed to `Node.edges`. #18472
- Fix: Fix issue with missing folder in local-recipes-index. #18449
- Fix: *Git.get_remote_url* now returns only the URL when using treeless repository. #18444
- Fix: Improvement over ill-formed graphs with different *visible=True/False* for the same dependency. #18440 . Docs here
- Fix: Fixing CMake presets on Windows with backslash. #18435
- Fix: Do not output upload-urls on basic text **`conan upload`** output. #18430
- Fix: Create folders if they don't exist when using *–out-file*. #18427
- Fix: Fix AutotoolsToolchain/GnuToolchain with LLVM/Clang in Windows for dynamic runtime in Debug. #18422
- Fix: Test `NMake` integration with `clang-cl`. #18422
- Fix: Ensure old gcc version are detected up to minor version only. #18419
- Fix: Fixing source retrieval when resetting local-index remote. #18418
- Fix: Allow minors greater than 9 in *detect_api*. #18410
- Fix: Removed `Workspaces` product definition and make `conan workspace build` work computing the right build-order. #18390 . Docs here
- Fix: Forward *ConanInvalidConfiguration* when raised in hooks. #18385
- Bugfix: Avoid crash when installing packages with tuple *generators* attribute and requirements to tool requires that provide *self.generator_info* generators. #18503
- Bugfix: Fix detection of riscv64 cpu in Meson toolchain. #18495
- Bugfix: Redirected Apple ARC flags to the ObjC/C++ ones. #18485
- Bugfix: Fix *TestClient* mocked *HEAD* requests. #18477
- Bugfix: Avoid leak of `global.conf` and `-cc` configuration for `core.xxx` items in Conan profiles, the `core` conf is exclusively for Conan internals, not for recipes neither for profiles. #18474
- Bugfix: XcodeToolchain sets correct . . . *_DEPLOYMENT_TARGET* for all Apple OSs. #18471 . Docs here
- Bugfix: **`conan export-pkg`** now correctly passes a *str* as the conanfile version. #18456
- Bugfix: Fix conan cache backup-upload ignoring *-cc* arguments. #18447
- Bugfix: Fixed *CMakeConfigDeps* behavior with multiple *find_package* in folders and subfolders. #18407
- Bugfix: Fixes issue where conanfile's *source()* method doesn't use *folders.root* when present. #18377

13.32 2.17.1 (23-Jun-2025)

- Bugfix: add support for `Git()` for git<2.36, for operations that check if a commit exists in a remote. #18501

13.33 2.17.0 (28-May-2025)

- Feature: Add support for gcc 13.4 #18374 . Docs here
- Feature: Renamed 'editables' to 'packages'. #18359 . Docs here
- Feature: Putting a folder named *conanws* as the top limit search if it exists. #18343 . Docs here
- Feature: Removed the *home_folder* definition mechanism from the *conanws.[yml | py]* file. #18339 . Docs here
- Feature: Packages/products do not need to be within the *workspace* folder. #18334 . Docs here
- Feature: Add *tools.gnu:configure_args* conf to GnuToolchain and Autotoolchain generator to allow extra arguments to be added to the configure command. #18333 . Docs here
- Feature: Add gcc 14.3 support. #18322 . Docs here
- Feature: Auto detection of C standard. #18290 . Docs here
- Feature: define CMAKE_C/CXX_COMPILER in `CMakeToolchain` generated presets, only for MSVC cl-like compilers, automatically only for Ninja generator. #18280
- Feature: Add *header_lib* template to **`conan new`**. #18249 . Docs here
- Feature: *to_cppstd_flag*/*to_cstd_flag* methods are not using fixed values. #18246
- Feature: Add `subprocess` to the profile jinja rendering. #18244 . Docs here
- Feature: New `conan cache save ... --no-source` to avoid storing downloaded sources in the *.tgz*. #18243 . Docs here
- Feature: Add verbose logs for *conan cache clean*. #18228
- Feature: Add *–list* inputs to *conan cache clean* and *conan cache check-integrity*. #18219 . Docs here
- Feature: Add *allowed_packages* info to remote json output. #18206
- Feature: Add URL information to json output format for conan upload. #18166 . Docs here
- Feature: New `conan workspace clean` command, removes the `output-folder` of editables if defined, otherwise nothing. Can be custom implemented by users in the `conanws.py` file. #17763 . Docs here
- Fix: Fix PyInstaller *–exclude-module* adding wildcard for *conan.test*. #18381
- Fix: Fix urls for conan audit. #18360
- Fix: Validate if the licenses in the SBOM are SPDX compatible. #18358
- Fix: Autotools in Windows working for both LLVM/Clang both clang and clang-cl frontends. #18347 . Docs here
- Fix: Change wording on unzip tool when uncompressing file. #18327
- Fix: Avoid duplicate component requirement names in *PkgConfigDeps* and *BazelDeps*. #18324
- Fix: Avoid grafted commits in `Git` helper for `commit_in_remote()` affecting also `coordinates_to_conandata()`, `get_url_and_commit()`. #18315
- Fix: *copy()* now is capable of excluding symlinks to folders. #18304
- Fix: Better error message in *conan list –graph=file.json* when using filtered graph. #18303
- Fix: Always sort overrides serialization. #18274
- Fix: Allow composition of conf values that are different categories of numbers. #18265
- Fix: Avoid incorrect warning in `test_package` of `python_requires` about "tested_reference_str". #18226

- Fix: CycloneDX 1.6 authors field. #18208
- Fix: Make `CMakeConfigDeps` incubating generator paths relative for `deployers`. #18197
- Fix: Add the full conan package in PyInstaller bundle. #18195
- Bugfix: Remove `LT_INIT` from `conan new autotools_exe` template `configure.ac`. #18378
- Bugfix: Fix CMakeConfigDeps link flags. #18367
- BugFix: Fix `conan audit` producing *_parse_error_threshold* crash when some package was not found in the catalog. #18363
- Bugfix: The first edge on *conan graph info ... -f=html* now shows require information. #18245
- Bugfix: `conan cache save` no longer zips downloaded artifacts like `conan_export.tgz` and `conan_sources.tgz`. #18243 . Docs here
- Bugfix: Allow to **`conan create`** a `python-requires` package with a profile that contains tool-requires. #18226
- Bugfix: Let *conan config install* walk the fs tree looking for a *.conanignore*. #18170

13.34 2.16.1 (29-Apr-2025)

- Feature: Add missing GCC 15 key to *settings.yml* #18193 . Docs here

13.35 2.16.0 (29-Apr-2025)

- Feature: Add support for GCC 15.1. #18175 . Docs here
- Feature: Allow `CMakeConfigDeps` to support components with multilibs (with deprecation warning). #18172
- Feature: add CMAKE_MODULE_PATH to CMakeConfigDeps for include(module). #18162
- Feature: Add threshold for severity level in the conan audit scan command. #18160 . Docs here
- Feature: *GnuToolchain* including the latest changes from *AutotoolsToolchain*. #18159
- Feature: Add *CycloneDx 1.6* support. #18108 . Docs here
- Feature: Introduce a new `no_skip=True` requirement trait for exceptional cases like one application depending on another application privately with `requires` to avoid it being skipped. #18101 . Docs here
- Feature: Raise error early if `conf_info` is assigned with raw settings/options etc #18083
- Feature: Moving functionality from `Command` layer to the `ConanAPI` for clearing old private imports `from conans`. #18079
- Feature: Document publicly the `MSBuildDeps.platform` attribute to allow customization for wix projects needing `x86` value. #18078 . Docs here
- Feature: Add missing intel-cc releases #18054 . Docs here
- Feature: Add information about the configuration each package is building for #18019
- Feature: Add *-vv* information for the configuration of each dependency in the graph #18019
- Fix: Some improvements in conan audit reports. #18171
- Fix: Fix ordering by severity value in audit html output. #18161
- Fix: Fix column overflow in audit html output. #18161

- Fix: Make *audit_providers.json* read/writeable only by owner. #18158
- Fix: Remove bogus SDK versions for some Apple OS's. #18152 . Docs here
- Fix: Make the `conan.cli` command layer fully independent of legacy `from conans` imports that will break. #18127
- Fix: Explicit `git fetch commit` in `Git.checkout_from_conandata_coordinates()`, for cases like Azure DevOps creating commits that are not fetched by default in `git clone`. #18110
- Fix: Add `ARM64EC` platform in `MSBuild`, it was missing. #18100 . Docs here
- Fix: Allow `conan graph build-order` to output `build_args` for "editable" packages. #18097
- Fix: Improve error message when private audit providers don't have curation. #18094
- Fix: Making some `Command` formatter helpers private (only the ones in `printers` are common for reusage), and making some ConanAPI attributes private. #18079
- Bugfix: Raise a not-found error if "local recipes index" `user/channel` doesn't match requested one. #18153
- Bugfix: Fixed bug using *MesonToolchain* and *visionOS*. #18151
- Bugfix: Add IMPORTED_CONFIGURATIONS to INTERFACE libraries to in `CMakeConfigDeps` #18088
- Bugfix: Apply Apple `bitcode`, `visibility` and `arc` confs to `Autootools/Gnu/Meson Toolchains` #18085

13.36 2.15.1 (14-Apr-2025)

- Feature: Update Apple products supported versions. #18122 . Docs here

13.37 2.15.0 (31-Mar-2025)

- Feature: Improve error messages when dealing with incorrect JSON input file formats. #18037
- Feature: Added new *–graph-context* to *conan list* command. #18015 . Docs here
- Feature: Add version-ranges patterns defined with `[1.2.3.4.*]` with the `*` at the end of the string. #18012 . Docs here
- Feature: Added *subsystem* field in MesonToolchain if cross-compiling between Apple OSs. #17985
- Feature: Added new kwarg *build_context`to `is_apple_os* helper function. #17985
- Feature: Integrate chmod feature in *tools.files*. #17800 . Docs here
- Fix: Remove backup sources from unknown refs when calling *conan cache clean*. #18018
- Fix: Fix SBOM author field. #18014
- Fix: Avoid resolving the symlinks path by default if they match the library name. #17964
- Fix: Make some `from conan.internal` and `from conans` usages from `CLI` commands private, moving to ConanAPI. #17961
- Fix: Add warning for `deprecated` attribute in recipes. #17957 . Docs here
- Fix: Improve relative paths in generators to be as short as possible. #17945
- Fix: *_Component()* has no *package_type* property. #17943
- Bugfix: Fix `global.conf` precedence over profiles `[conf]` and order change of per-package pattern confs. #18028

- Bugfix: Solve issue with `update_policy=legacy` and using lockfiles. #18009
- Bugfix: *untargz()* method was failing if directories had a more restrictive mode. #17998
- Bugfix: *CppInfo.auto_deduce_location* method gives more prio to exact match. #17975
- Bugfix: Avoid crash of `--format=json` serialization when custom generators inside tool-requires are referenced by class, not by name. #17954
- BugFix: Add correct info in metadata + prevent crash when no component is associated to root_node. #17925

13.38 2.14.0 (12-Mar-2025)

- Feature: Add **`conan audit`** command for scanning Conan packages for CVE's #17951 . Docs here
- Feature: Add clang 20 support. #17920 . Docs here
- Feature: Allow partial `workspace install <path1> ... <pathN>` installation of workspace. #17887 . Docs here
- Feature: Add hooks for validate method: *pre_validate* and *post_validate*. #17856 . Docs here
- Feature: Added complete Apple Frameworks management to *CMakeConfigDeps*. #17725 . Docs here
- Feature: Added new *cpp_info.package_framework* to *cpp_info*. #17725 . Docs here
- Feature: Fix several bugs in docker runner, added new configuration options and improved logging system #17542 . Docs here
- Fix: Improve error message when `jinja2` profile rendering fails due to unexpected syntax. #17940
- Fix: Do not warn in auto-deduce location for exact library matches. #17923
- Fix: the `cmake_set_interface_link_directories` property is not really necessary at all in `CMakeDeps` and it becomes invalid in `CMakeConfigDeps` as it will require full `package_info()` definition. #17917 . Docs here
- Fix: Do not convert \\ to / in `MSBuildDeps` generator as some MSBuild functionality needs Windows \\ paths. #17901
- Fix: Avoid workspace incorrectly defining a "local-recipes-index" auxiliary cache. #17883
- Fix: Improve the output of the profile dumping for environment, with correct prepend order. #17863
- Fix: Fixes VCVars vcvarsall.bat generation if OS is set to WindowsStore. #17849
- Bugfix: Avoid self-requirement and loop when a `[tool_requires]` in the build profile depends on itself with a version range. #17931
- Bugfix: Fix `conan graph build-order --reduce --order-by=recipe` that was not filtering all packages != "Build". #17909
- Bugfix: Solve conflict not raised when version-ranges have different user. #17877

13.39 2.13.0 (26-Feb-2025)

- Feature: `CMakeDeps` generated `Findxxxx.cmake` files now can define `{prefix}_FOUND` and `{prefix}_VERSION` for the `cmake_additional_variables_prefixes`. #17838
- Feature: Make available in conanfiles the new incubating `CMakeConfigDeps` generator, still under the incubating "conf" feature flag. #17831 . Docs here
- Feature: Add a warning if a specific revision different than the current one is requested to a `local-recipes-index` remote. #17819
- Feature: Forward repository parameter (with same default) from *coordinates_to_conandata()* to *get_url_and_commit()*. #17722
- Feature: Add `mcf` threading for `gcc` MinGW compiler *settings.yml*. #17704
- Feature: Improve `conanws.py` file definition following same patterns as `ConanFile`. #17688 . Docs here
- Feature: Workspace new `workspace install` command for monolithic super-projects containing multiple `editables`. #17675 . Docs here
- Feature: New `conan new workspace` template contains CMake-based monolithic super-project that works with `conan workspace install`. #17675 . Docs here
- Feature: Added *CMAKE_LIBRARY_PATH* to *conan_cmakedeps_paths.cmake* (new CMakeDeps). #17668
- Feature: Added *CMAKE_INCLUDE_PATH* to *conan_cmakedeps_paths.cmake* (new CMakeDeps). #17668
- Feature: Add *extension_properties* access to conanfile dependencies. #17659 . Docs here
- Feature: Introducing *in_range* method in Version which allows comparing against version ranges. #17658 . Docs here
- Feature: Upgrade dependency `urllib3` to `2.0`. #17655
- Feature: New *lock upgrade* command to automatically upgrade desired dependencies resolving the graph. #17577 . Docs here
- Feature: Enhanced `Premake` CLI wrapper with configurable Lua file path, and support for custom command-line arguments. #17398 . Docs here
- Fix: Docstring for `conan remote auth` regarding CONAN_LOGIN env-var. #17834
- Fix: runtime_deploy preserves symbolic links along with their libraries. #17824 . Docs here
- Fix: Better message for incubating CMakeDeps about `target_link_libraries()` from tool-requires. #17821
- Fix: Fix the *_calculate_licenses* SBOM method bug and add a small test. #17801
- Fix: Allow build context information from `conf` in `AutotoolsToolchain`. #17794
- Fix: Allow msys2 subsystem path inheriting from environment variables #17781
- Fix: Improve error messages for components definition errors and for runtime conflicts. #17771
- Fix: Update the message for client migration. #17751
- Fix: Improve untar performance. #17708
- Fix: Protect erroneous assignment of `cpp_info/components.required_components = xxx`, for `required_components` property. Now it will raise a proper error. #17692
- Fix: New `CMakeDeps` transitive linking of shared libs. #17459
- Bugfix: Fix self-contained `pyinstaller` executable to also include the new `conan.tools.sbom` tools. #17809

13.40 2.12.2 (12-Feb-2025)

- Fix: Fix default name and let cycloneDX define a custom name. #17760 . Docs here
- Fix: Add cycloneDX *add_tests* and *add_build* parameters. #17760 . Docs here
- Bugfix: Fix cycloneDX tool parameters. #17760 . Docs here

13.41 2.12.1 (28-Jan-2025)

- Bugfix: Fix *conan config clean* not regenerating every necessary file. #17649
- Bugfix: Avoid `compatibility.py` migration if any of the files are modified by users. #17647

13.42 2.12.0 (27-Jan-2025)

- Feature: Make public documented (and experimental) the `--build=compatible:[pattern]` build mode, to allow building other configurations different than the current one when the current one is invalid and binary compatibility defines compatible binaries. #17637 . Docs here
- Feature: Define new `tools.cmake.cmaketoolchain:user_presets` to customize the name of the generated `CMakeUserPresets.json`, disabling its generation. Also can generate it in a subfolder. #17613 . Docs here
- Feature: Serialize in `--format=json` graph output the original requirements version range, not only the resolved one. #17603
- Feature: Add cycloneDX as a Conan tool and implement subgraph for conanfile. #17559 . Docs here
- Feature: Initial `conan workspace build` command to build the full workspace, based on the definition of `products`. #17538 . Docs here
- Feature: Allow applying patches on "create" time for conan-center-index like layouts from an external centralized folder. #17520 . Docs here
- Feature: Add report progress while unpacking tarball files. #17519
- Feature: *conan profile show* can now select which context's profile to show. #17518
- Feature: Better logging, printing the username for repositories, successful auth event and trace-level messages including full URL requests. #17517
- Feature: Adds *conan config clean* command that will remove all custom config from conan home, excluding the generated packages. #17514 . Docs here
- Feature: Add *reinit* method to *ConanApi*, which reinitializes every *subapi*. #17514 . Docs here
- Feature: Allow defining `--out-file=file.ext` instead of `--format=ext > file.ext` to write to files directly and avoid issues with redirects. #17507 . Docs here
- Feature: Cache HTTP request sessions between API calls. #17455
- Feature: Implement caching in the `Remote` objects for `RemoteManager` calls, saving repeated calls to the server for the duration of the life of the Remote objects. #17449 . Docs here
- Fix: Added *arch_flag* as a public attribute to the *MesonToolchain* generator. #17629
- Fix: Increase sqlite timeout from 10 to 20 seconds for very heavily loaded CI servers. #17616
- Fix: Make `remotes.json` saving transactional to avoid corruption for hard killed processes. #17588

- Fix: Improve error message for **`conan create`** when `test_package` has missing binaries. #17581
- Fix: Fix *Git is_dirty* detection of excluded files with paths. #17571
- Fix: Allow latest bottle 0.13 release for `conan_server` to work with Python 3.13. #17534
- Fix: GnuToolchain's make_args handle empty values correctly. #17532
- Fix: Fix inconsistency in `replace_in_file`, that returned *False* if the pattern was not found (with strict off), otherwise *None*. #17531
- Fix: *conan profile show* does not pollute stdout with information titles. #17518
- Fix: Error out when unknown language is used in languages attribute. #17512
- Fix: Fix `Workspace` when using the `workspace_api.load()` and using `self.run()` inside `set_version()`. #17501
- Bugfix: *conf_build* does not exist for *cli* and *conanfile.txt* contexts. #17640
- Bugfix: Make possible to use *pattern* and *strip_root* at the same time for *conan.tools.files.unzip()*. #17591
- Bugfix: Solve incubating `CMakeDeps` issues with transitive `[replace_requires]`. #17566
- Bugfix: Solve `PkgConfigDeps` issues with transitive `[replace_requires]`. #17566

13.43 2.11.0 (18-Dec-2024)

- Feature: Only warn on frozen conan v1 remote if enabled. #17482
- Feature: *AutotoolsToolchain* uses user's variables when Android cross-compilation at first. #17470 . Docs here
- Feature: *AutotoolsToolchain* checks if Android cross-compilation paths exist. #17470 . Docs here
- Feature: Adding the Conan cache "profiles" folder to the jinja2 search path, so profiles can be included/imported from jinja syntax even for parent and sibling folders. #17432 . Docs here
- Feature: Updated *tools.env.virtualenv:powershell* conf to allow specifying the PowerShell executable (e.g., powershell.exe or pwsh) and passing additional arguments. #17416 . Docs here
- Feature: Deprecate use of *tools.env.virtualenv:powershell=True/False*. #17416 . Docs here
- Fix: Do not show powershell deprecation message if value is None. #17500
- Fix: Fix `LocalAPI` definition of editables when calling `editable_add`. #17498
- Fix: Clarify debug message in CMakeDeps. #17453
- Fix: Added explicitly *allow_empty = True* to *glob()* function in BazelDeps (bazel 8.x compatible). #17444
- Fix: Fix broken *cpp_info.location* deduction due to unsanitized regex. #17430
- Fix: Trusting the real path coming from a symlink is a good one. #17421
- Fix: Fix user/channel when searching patterns in a local-recipes-index. #17408
- Fix: Add warning for empty version ranges. #17405
- Bugfix: Fix bogus duplication of component properties #17503
- Bugfix: Fix running commands in powershell with single quotes. #17487
- Bugfix: Fix issues with unsetting some types of confs. #17445

13.44 2.10.3 (18-Dec-2024)

- Bugfix: Integrate Conan 2.9.3 missing fix https://github.com/conan-io/conan/pull/17338 #17496

13.45 2.10.2 (10-Dec-2024)

- Fix: Solve performance issue in large graphs computing the "skip" binaries. #17436

13.46 2.10.1 (04-Dec-2024)

- Bugfix: Fix *[replace_requires]* for replacements of same reference name. #17409

13.47 2.10.0 (02-Dec-2024)

- Feature: Add *–force* option to *conan remote auth* to force authentication even for remotes that have anonymous access enabled. #17377 . Docs here
- Feature: Add *–output* option to **`conan new`** command. #17359
- Feature: Let the new `CMakeDeps` always define components and check them with `find_package(COMPONENTS)`, listening to new property `cmake_components`. #17302
- Feature: Allow `tools.microsoft.msbuild:max_cpu_count=0` to use `/m` to use all available cores. #17301 . Docs here
- Feature: define * as default argument if no args specified for `conan list`. #17300 . Docs here
- Feature: Improved auto deduce location function. #17296
- Feature: BazelDeps using the new *deduce_location* mechanism to find the libraries. #17296
- Feature: Initial `conan workspace` initial proposal to manage local set of editables. Introduced only as a dev/maintainers feature, behind an environment variable. #17272 . Docs here
- Feature: Allow `--settings` in `conan config install-pkg` to create and install different configurations in different platforms. #17217 . Docs here
- Feature: Add network to configfile for Docker runners. #17069 . Docs here
- Fix: Fix help message for PowerShell conf. #17389 . Docs here
- Fix: Fixed an error that occurred when using *conan.tools.scm.Git.fetch_commit()* in a subfolder. #17369
- Fix: Adding a "risk" warning for options conflicts, so users can do warn-as-error to raise when they happen. #17366
- Fix: New `CMakeDeps` generator allow `fooConfig.cmake` for in-package files besides `foo-config.cmake`. #17330
- Fix: Add a warning for editable dependencies when building in the cache. #17325
- Fix: Raise ConanException if source patch does not exist in *export_conandata_patches*. #17294
- Fix: Improve the UX for *CONAN_LOG_LEVEL* env-var incorrect values. #17280
- Fix: Meson aligns with other build systems considering *x86_64*->`x86` as cross building. #17266

- Fix: Avoid `colorama` bug crashing for large outputs. #17259
- Fix: Fix arch for docker runner tests. #17069 . Docs here
- Bugfix: Add correct flags when `compiler=clang` and `compiler_executables={"c": "clang-cl"}` to not inject incorrect flags when cross-building from Linux to Windows. #17387
- Bugfix: Solve `Choco().check()` bug using legacy `choco search --local-only`, replaced by `choco list`. #17382
- Bugfix: Fix adding *tools.android:ndk_path* with spaces in path. #17379
- BugFix: Fix `Premake` integration. #17350 . Docs here
- Bugfix: Solve problem with misdetection of consumer packages for the & pattern. #17346
- Bugfix: Fix *conan graph info ... -f=html* in Safari. #17335
- Bugfix: Allow multiple `[replace_requires]` by the same dependency. #17326
- Bugfix: BazelDeps failed to find OpenSSL shared libraries. #17296
- Bugfix: Solve bug in `CMake` not using the correct value from `tools.microsoft.msbuild:max_cpu_count`. #17292
- Bugfix: Fix `cpp_info` properties overwriting instead of merging for properties with list values. Necessary for `cmake_build_modules` to work in `editable` mode. #17214

13.48 2.9.3 (21-Nov-2024)

- Bugfix: Fixing `is_test` computation affecting to components checks. #17338

13.49 2.9.2 (07-Nov-2024)

- Feature: Use center2.conan.io as new default remote and warn about having the old one. #17284 . Docs here
- Bugfix: Fix ROSEnv quotes for CMAKE_TOOLCHAIN_FILE variable. #17270

13.50 2.9.1 (30-Oct-2024)

- Bugfix: Fix *deduce_subsystem* when *scope=None* assuming the scope is *build*. #17251
- Bugfix: Fixed false positives of `profile.py` plugin checks over c++26 for latest compiler versions #17250

13.51 2.9.0 (29-Oct-2024)

- Feature: Add missing major OS/compiler version support in *settings.yml*. #17240 . Docs here
- Feature: **`conan new`** learned defaults `-d name=mypkg -d version=0.1` for simpler UX. #17186 . Docs here
- Feature: Warn when patching files and the recipe has *no_copy_source = True*, which could lead to unforseen issues #17162
- Feature: Add *self.generator_info* for *tool_requires* to propagate generators to their direct dependencies. #17129 . Docs here

- Feature: Add support for including paths that are ignored in *.conanignore*. #17123 . Docs here
- Feature: New `tools.graph:skip_build` conf to be able to skip the expansion of `tool_requires`. #17117 . Docs here
- Feature: New `tools.graph:skip_test` conf to be able to skip the expansion of `test_requires`. #17117 . Docs here
- Feature: Add ROSEnv generator integration for ROS2 (Robot Operating System). #17110
- Feature: Add profile arguments information to `conan graph build-order` to improve UX and usage in CI systems. #17102 . Docs here
- Feature: Add C++26 support for *gcc*, *clang*, and *apple-clang*. #17092 . Docs here
- Feature: Add Configuration and Platform keys for MSBuildDeps property sheets. #17076 . Docs here
- Feature: New `CMakeDeps` generator activated by `tools.cmake.cmakedeps:new` conf with value `will_break_next` for evaluation. This new generator deduces or use `cpp_info.location/link_location` to define STATIC; SHARED, INTERFACE imported targets. It will also define the IMPORTED_LOCATION, the IMPORTED_CONFIGURATION, etc. #16964
- Feature: Use `cpp_info.languages`, that default to the recipe `languages` to propagate "link-language" requirements to consumers of the packages. #16964
- Feature: Define `cpp_info.default_components` for the new `CMakeDeps` generator only. #16964
- Feature: Model `cpp_info.exes` field for executable applications, used only by the new `CMakeDeps` generator, that generate IMPORTED executable targets in `CMakeDeps` for `cpp_info.exes`. #16964
- Fix: Use a valid prefix path for *meson.configure()* on Windows, to avoid failures in Python 3.13. #17206
- Fix: Allow *cmake_target_aliases* to be set in CMakeDeps. #17200 . Docs here
- Fix: Adding the startup options to each Bazel command. #17183
- Fix: Add remote name to login prompt. #17178
- Fix: Get credentials and re-authenticate when an expired token gives AuthenticationException. #17127
- Fix: Moved exceptions from the legacy `from conans.error` to documented `from conan.error`. #17126 . Docs here
- Fix: `Pacman` as package manager shouldn't be used for `tools.microsoft.bash:subsystem=msys2`, but when the target platform is actually msys2 `os.subsystem=msys2` (as a setting). #17103
- Fix: Properly deduce RuntimeLibrary from profile in MSBuildToolchain. #17100
- Fix: Set C++20 flag to *{gnu}c++20* for *gcc* >= 10 instead of *c++2a* until *gcc* 12. #17092 . Docs here
- Fix: Set C++23 flag to *{gnu}c++23* for *gcc* >= 11 instead of *c++2b*. #17092 . Docs here
- Fix: Avoid repeated login attempts to the server for 401 when the credentials come from env-vars or `credentials.json` file, only repeated login attempts for user interactive prompt. #17083
- Fix: Align CMakeToolchain and AutotoolsToolchain to automatically define `cl` compiler for `compiler=msvc` if not defined (only when necessary, as when using Ninja generator in CMake). #16875 . Docs here
- Fix: Quote *build_args* in *conan graph build-order -f=json* to avoid issues with options with spaces. #16594
- Bugfix: Improved *bazeldeps._get_libs()* mechanism. #17233
- Bugfix: Improve cstd check for different compiler versions at profile load time. #17157
- Bugfix: Fix cppstd/cstd *variable_watch* when they are not defined. #17156
- Bugfix: Fix cstd error reporting when a recipe does not support the required version. #17156

- Bugfix: Drop the username permission validation bypass in `conan_server`, it could be a potential security issue. #17132
- Bugfix: Listing recipes with equal versions under semver rules but different representation (ie *1.0* & *1.0.0*) now returns both references. #17121
- Bugfix: Conan Server: Do not return duplicated references for each revision of the same recipe reference when searching them. #17121
- Bugfix: Empty version range results in empty condition set. #17116
- Bugfix: Adding the *# do not sort* comment to *deps* section. Regression since Conan 1.61. #17109
- Bugfix: Restore ConanOutput global state when using *Commands* API. #17095
- Bugfix: *build_args* options in *graph build-order* now respect the context of the reference. #16594

13.52 2.8.1 (17-Oct-2024)

- Bugfix: Avoid raising an error for required components for `test_requires` also required as transitive `requires`. #17174

13.53 2.8.0 (30-Sept-2024)

- Feature: Add support for iOS 18, watchOS 11, tvOS 18, visionOS 2 & macos 15. #17012 . Docs here
- Feature: Add Clang 19 support. #17010 . Docs here
- Feature: `conan config list <pattern>` to filter available configurations. #17000 . Docs here
- Feature: New `auth_remote.py` plugin for custom user authentication to Conan remotes. #16942 . Docs here
- Feature: New `auth_source.py` plugin for custom user authentication for generic downloads of sources. #16942 . Docs here
- Feature: Add *–envs-generation={false}* to **`conan install`** and **`conan build`** to disable the generation of virtualenvs (`conanbuildenv.sh|bat` and `conanrunenv.sh|bat`). #16935 . Docs here
- Feature: New `tools.files.unzip:filter` conf that allows to define `data`, `tar` and `fully_trusted` extraction policies for tgz files. #16918 . Docs here
- Feature: `get()` and `unzip()` tools for `source()` learned a new `extract_filter` argument to define `data`, `tar` and `fully_trusted` extraction policies for tgz files. #16918 . Docs here
- Feature: Add progress updates for large uploads (>100Mbs) every 10 seconds. #16913
- Feature: Implement `conan config install-pkg --url=<repo-url>` for initial definition of remote URL when no remotes are defined yet. #16876 . Docs here
- Feature: Allow building a compatible package still of the current profile one. #16871
- Feature: Allow bootstrapping (depending on another variant of yourself), even for the same version. #16870
- Feature: Allow `[replace_requires]` to replace the package name and `self.dependencies` still works with the old name. #16443
- Fix: Let `CMakeToolchain` defining `CMAKE_SYSTEM_XXX` even if `user_toolchain` is defined, but protected in case the toolchain really defines them. #17036 . Docs here
- Fix: Replace | character in generated CMake and Environment files. #17024

- Fix: Redirect the `PkgConfig` `stderr` to the exception raised. #17020
- Fix: Use always forward slashes in Windows subsystems `bash` path. #16997
- Fix: Better error messages when `conan list --graph=<graph-json-file>` file has issues. #16936
- Bugfix: *PkgConfigDeps.set_property()* was not setting properly all the available properties. #17051
- Bugfix: BazelDeps did not find DLL files as Conan does not model them in the Windows platform. #17045
- Bugfix: Do not skip dependencies of a package if it is not going to be skipped due to `tools.graph:skip_binaries=False`. #17033
- Bugfix: Allow `requires(..., package_id_mode)` trait in case of diamonds to always use the recipe defined one irrespective of `requires()` order. #16987
- Bugfix: Propagate include_prerelease flag to intersection of VersionRange. #16986
- Bugfix: Raise error if invalid value passed to conf.get(check_type=bool). #16976
- Bugfix: Allow *remote_login* accept patterns. #16942 . Docs here

13.54 2.7.1 (11-Sept-2024)

- Feature: Add support apple-clang 16. #16972
- Fix: Add test for #19960. #16974
- Bugfix: Revert "Define compiler variables in CMakePresets.json" commit 60df72cf75254608ebe6a447106e60be4d8c05a4. #16971

13.55 2.7.0 (28-Aug-2024)

- Feature: Added `Git.is_dirty(repository=False)` new argument #16892
- Feature: Add variable_watch for *CMAKE_{C,CXX}_STANDARD* in *conan_toolchain.cmake*. #16879
- Feature: Add *check_type* to *get_property* for CMakeDeps. #16854 . Docs here
- Feature: Propagate *run* trait requirement information in the "build" context downstream when *visible* trait is *True*. #16849 . Docs here
- Feature: Add *check_type* on components *get_property*. #16848 . Docs here
- Feature: Add *set_property* for PkgConfigDeps to set properties for requirements from consumer recipes. #16789
- Feature: Define *CMAKE_<LANG>_COMPILER* variables in CMakePresets.json. #16762
- Feature: Add support for gcc 14.2. #16760
- Feature: Rework QbsProfile to support Conan 2. #16742
- Feature: Add *finalize()* method for local cache final adjustments of packages. #16646 . Docs here
- Feature: Add `tricore` compiler architecture support. #16317 . Docs here
- Feature: Describe here your pull request #16317 . Docs here
- Fix: Propagate `repository` argument from `Git.get_url_and_commit(repository=True)` to `Git.is_dirty()`. #16892
- Fix: Improve error when accessing *cpp_info* shorthand methods. #16847

- Fix: Improve error message when a lockfile fails to lock a requirement, specifying its type. #16841
- Fix: Update patch-ng 1.18.0 to avoid SyntaxWarning spam. #16766
- Bugfix: Avoid `CMakeToolchain` error when both architecture flags and `tools.build:linker_scripts` are defined, due to missing space. #16883
- Bugfix: When using Visual Studio's llvm-clang, set the correct Platform Toolset in *MSBuildToolchain*. #16844
- Bugfix: Fix *export_sources* for non-existent recipes in a local recipes index. #16776

13.56 2.6.0 (01-Aug-2024)

- Feature: Add `build_folder_vars=["const.myvalue"]` to create presets for user "myvalue" arbitrary values. #16633 . Docs here
- Feature: Added *outputRootDir* as an optional definition in Bazel new templates. #16620
- Feature: MakeDeps generator generates make variables for dependencies and their components. #16613 . Docs here
- Feature: Add html output for graph build-order and graph build-order-merge #16611 . Docs here
- Feature: Introduce `core.scm:local_url=allow|block` to control local folder URLs in conandata `scm`. #16597 . Docs here
- Feature: Added *XXX_FOR_BUILD* flags and Android extra ones to *extra_env* attribute in *GnuToolchain*. #16596
- Feature: Support `python_requires` in `local-recipes-index`. #16420 . Docs here
- Fix: Avoid `runtime_deployer` to deploy dependencies with `run=False` trait. #16724
- Fix: Improve error message when passing a `--deployer-folder=xxx` argument of an incorrect folder. #16723
- Fix: Change `requirements.txt` so it install `distro` package in FreeBSD too. #16700
- Fix: Better error messages when loading an inexistent or broken "package list" file. #16685
- Fix: Remove unsupported *ld* and *ar* entries from *tools.build:compiler_executables*, they had no effect in every Conan integration. #16647
- Fix: Ensure direct tool requires conflicts are properly reported instead of hanging. #16619
- Fix: Update `docker` dependency version for the `runners` feature #16610
- Fix: Raise an error when trying to upload a package with a local folder URL in `conandata scm`. #16597 . Docs here
- Bugfix: Fix profile `include()` with per-package `[settings]` with partial definition. #16720
- Bugfix: The `MakeDeps` generator was missing some aggregated variables when packages have components. #16715
- Bugfix: Avoid *settings.update_values()* failing when deducing compatibility. #16642
- Bugfix: Fix handling of *tools.build:defines* for Ninja Multi-Config CMake. #16637
- Bugfix: Make conan graph <subcommand> a real "install" dry-run. Return same errors and give same messages #16415

13.57 2.5.0 (03-Jul-2024)

- Feature: New `tools.cmake.cmaketoolchain:enabled_blocks` configuration to define which blocks of `CMakeToolchain` should be active or not. #16563 . Docs here
- Feature: Allow *–filter-options* in *conan list* to use *&:* scope to refer to all packages being listed. #16559
- Feature: Highlight missing or invalid requirements while computing dependency graph. #16520
- Feature: New `conan lock update` subcommand to remove + add a reference in the same command. #16511 . Docs here
- Feature: Add support for GCC 12.4. #16506 . Docs here
- Feature: Honoring *tools.android:ndk_path* conf. Setting the needed flags to cross-build for Android. #16502 . Docs here
- Feature: Add `os.ndk_version` for `Android`. #16494 . Docs here
- Feature: Qbs helper now invokes Conan provider automatically. #16486
- Feature: Added force option to *tools.cmake.cmaketoolchain:extra_variables*. #16481 . Docs here
- Feature: Raising a ConanException if any section is duplicated in the same Conan profile file. #16454
- Feature: Added *resolve()* method to the Qbs toolchain. #16449
- Feature: Make `MSBuildDeps` generation with deployer relocatable. #16441
- Feature: Add QbsDeps class to be used with Qbs Conan module provider. #16334
- Feature: Add built in *runtime_deploy* deployer. #15382 . Docs here
- Fix: Fix provides conflict error message not showing conflicting provides when a named reference matches a provider. #16562
- Fix: Set correct *testpaths* for pytest. #16530
- Fix: Allow `.conanrc` file in the root of a filesystem. #16514
- Fix: Add support for non default docker hosts in conan runners #16477
- Fix: Don't fail when we can't overwrite the summary file in the backup upload. #16452
- Fix: Make `source_credentials.json` do not apply to Conan server repos protocol. #16425 . Docs here
- Fix: Allow packages to have empty folders. #16424
- Bugfix: Fix `detect_msvc_compiler()` from `detect_api` to properly detect VS 17.10 with `compiler.version=194`. #16581
- Bugfix: Fix unexpected error when a recipe performs *package_id() info* erasure and is used as a compatibility candidate. #16575
- Bugfix: Ensure msvc cppstd compatibility fallback does not ignore 194 binaries. #16573
- Bugfix: *XXXDeps* generators did not include an absolute path in their generated files if *–deployer-folder=xxxx* param was used. #16552
- Bugfix: Fix *conan list ... –format=compact* for package revisions. #16490
- Bugfix: Fix XcodeToolchain when only defines are set. #16429

13.58 2.4.1 (10-Jun-2024)

- Fix: Avoid *find_package*'s of transitive dependencies on *test_package* generated by *cmake_lib* template. #16451
- Fix: Fix back migration of default compatibility.py from a clean install. #16417
- Bugfix: Solve issue with setuptools (distributed Conan packages in Python) packaging the "test" folder. #16446
- Bugfix: Fixed regression in `CMakeToolchain` with `--deployer=full_deploy` creating wrong escaping. #16434

13.59 2.4.0 (05-Jun-2024)

- Feature: Added support for MacOS sdk_version 14.5 #16400 . Docs here
- Feature: Added *CC_FOR_BUILD* and *CXX_FOR_BUILD* environment variable to AutotoolsToolchain. #16391 . Docs here
- Feature: Added *extra_xxxx* flags to MesonToolchain as done in other toolchains like AutotoolsToolchain, CMakeToolchain, etc. #16389
- Feature: Add new `qbs_lib` template for the **`conan new`** command. #16382
- Feature: new `detect_api.detect_sdk_version()` method #16355 . Docs here
- Feature: Add excludes parameter to tools.files.rm to void removing pattern. #16350 . Docs here
- Feature: Allow multiple `--build=missing:~pattern1 --build=missing:~pattern2` patterns. #16327
- Feature: Deprecate use of path accessors in ConanFile. #16247
- Feature: add support for setting *tools.cmake.cmaketoolchain:extra_variables* #16242 . Docs here
- Feature: Add *cmake_additional_variables_prefixes* variable to CMakeDeps generator to allow adding extra names for declared CMake variables. #16231 . Docs here
- Feature: Allow GNUInstallDirs definition in `CMakeToolchain` for the local `conan install/build` flow too. #16214
- Feature: Let `conan cache save` listen to the `core.gzip:compresslevel` conf. #16211
- Feature: Add support for Bazel >= 7.1. #16196 . Docs here
- Feature: Add new `revision_mode` including everything down to the `recipe-revision`, but not the `package_id`. #16195 . Docs here
- Feature: Allow a recipe to `requires(..., visible=False)` a previous version of itself without raising a loop error. #16132
- Feature: New `vendor=True` package creation and build isolation strategy #16073 . Docs here
- Feature: New `compiler.cstd` setting for C standard #16028 . Docs here
- Feature: Implemented `compatibility.py` default compatibility for different C standards #16028 . Docs here
- Feature: Implement `check_min_cstd`, `check_max_cstd`, `valid_max_cstd`, `valid_min_cstd`, `supported_cstd` tools #16028 . Docs here
- Feature: New `languages = "C", "C++"` class attribute to further automate settings management #16028 . Docs here
- Feature: Add *CONAN_RUNTIME_LIB_DIRS* variable to the *conan_toolchain.cmake*. #15914 . Docs here
- Fix: Implement a back migration to <2.3 for default `compatibility.py` plugin. #16405

- Fix: Add missing *[replace_requires]* and *[platform_requires]* to serialization/dump of profiles. #16401
- Fix: Fix handling spaces in paths in `Qbs` helper. #16382
- Fix: Make cc version detection more robust #16362
- Fix: Allow `--build=missing:&` pattern to build only the consumer if missing, but not others. #16344
- Fix: Allow "local-recipes-index" to `conan list` packages with custom `user/channel`. #16342
- Fix: Fixing docstrings for `cppstd` functions. #16341
- Fix: Change autodetect of *CMAKE_SYSTEM_VERSION* to use the Darwin version. #16335 . Docs here
- Fix: Fix *require* syntax in output in *graph build-order*. #16308
- Fix: Improve some commands help documentation strings by adding double quotes. #16292
- Fix: Better error message for incorrect version-ranges definitions. #16289
- Fix: Only print info about cached recipe revision being newer when it truly is. #16275
- Fix: Warn when using `options` without pattern scope, to improve UX of users expecting `-o shared=True` to apply to dependencies. #16233 . Docs here
- Fix: Fix CommandAPI usage when not used by Conan custom commands. #16213
- Fix: Avoid `datetime` deprecated calls in Python 3.12. #16095
- Fix: Handle *tools.build:sysroot* on Meson toolchain. #16011 . Docs here
- Bugfix: Fix `LLVM/Clang` enablement of `vcvars` for latest `v14.4` toolset version after VS 17.10 update #16374 . Docs here
- Bugfix: Fix profile errors when using a docker runner of *type=shared* #16364
- Bugfix: `conan graph info .. --build=pkg` doesn't download `pkg` sources unless `tools.build:download_source` is defined. #16349
- Bugfix: Solved problem with relativization of paths in CMakeToolchain and CMakeDeps. #16316
- Bugfix: Avoid sanitizing *tools.build:compiler_executables* value in MesonToolchain. #16307
- Bugfix: Solved incorrect paths in `conan cache save/restore` tgz files that crashed when using `storage_path` custom configuration. #16293
- Bugfix: Fix stacktrace with nonexistent graph file in *conan list*. #16291
- Bugfix: Let `CMakeDeps` generator overwrite the `xxxConfig.cmake` when it already exists. #16279
- Bugfix: Disallow *self.info* access in *source()* method. #16272

13.60 2.3.2 (28-May-2024)

- Feature: New `tools.microsoft:msvc_update` configuration to define the MSVC compiler `update` even when `compiler.update` is not defined. Can be used to use `compiler.version=193` once VS2022 is updated to 17.10, which changes the default compiler to `compiler.version=194`. #16332
- Bugfix: Allow default `compatibility.py` plugin to fallback from MSVC `compiler.version=194->193` and to other `cppstd` values. #16346
- Bugfix: Skip dot folders in local recipe index layouts. #16345
- Bugfix: Remove extra backslash in generated *conanvcvars.ps1*. #16322

13.61 2.3.1 (16-May-2024)

- Feature: Add GCC 13.3 support. #16246 . Docs here
- Feature: Allow opt-out for `CMakeToolchain` default use of absolute paths for CMakeUserPresets->CMakePreset and CMakePresets->toolchainFile path. #16244 . Docs here
- Fix: Fix config container name for Docker runner. #16243
- Bugfix: Make compatibility checks understand update flag patterns. #16252
- Bugfix: Solve bug with `overrides` from `lockfiles` in case of diamond structures. #16235
- Bugfix: Allow `export-pkg --version=xxx` to be passed to recipes with `python_requires` inheriting `set_version` from base class. #16224

13.62 2.3.0 (06-May-2024)

- Feature: Allow * wildcard as subsetting in in *rm_safe*. #16105 . Docs here
- Feature: Show recipe and package sizes when running **`conan upload`**. #16103
- Feature: Extend *conan version* to report current python and system for troubleshooting. #16102 . Docs here
- Feature: Add `detect_xxxx_compiler()` for mainstream compilers as gcc, msvc, clang. to the public `detect_api`. #16092 . Docs here
- Feature: Add comment support for *.conanignore* file. #16087
- Feature: In graph *html* search bar now takes in multiple search patterns separated by commas. #16083
- Feature: In graph *html* added 'filter packages' bar that takes in multiple search patterns separated by comma and hides filters them from graph. #16083
- Feature: Add an argument *hide_url* to Git operations to allow logging of the repository URL. By default, URLs will stay <*hidden*>, but users may opt-out of this. #16038
- Feature: Allow `.conf` access (exclusively to `global.conf` information, not to profile information) in the `export()` and `export_sources()` methods. #16034 . Docs here
- Feature: Avoid copying identical existing files in `copy()`. #16031
- Feature: New `conan pkglist merge` command to merge multiple package lists. #16022 . Docs here
- Feature: New `conan pkglist find-remote` command to find matching in remotes for list of packages in the cache. #16022 . Docs here
- Feature: Relativize paths in *CMakePresets* generation. #16015
- Feature: Add new `test_package_folder` attribute to `conanfile.py` to define a different custom location and name rather than `test_package` default. #16013 . Docs here
- Feature: New `conan create --test-missing` syntax to optionally run the `test_package` only when the package is actually created (useful with `--build=missing`). #15999 . Docs here
- Feature: Add *tools.gnu:build_triplet* to conf. #15965
- Feature: Add `--exist-ok` argument to `conan profile detect` to not fail if the profile already exists, without overwriting it. #15933
- Feature: MesonToolchain can generate a native file if native=True (only makes sense when cross-building). #15919 . Docs here

- Feature: Meson helper injects native and cross files if both exist. #15919 . Docs here
- Feature: Add support for meson subproject. #15916 . Docs here
- Feature: Added transparent support for running Conan within a Docker container. #15856 . Docs here
- Fix: Allow defining `CC=/usr/bin/cc` (and for CXX) for `conan profile detect` auto-detection. #16187
- Fix: Solve issue in `pyinstaller.py` script, it will no longer install `pip install pyinstaller`, having it installed will be a precondition #16186
- Fix: Use backslash in `CMake` helper for the CMakeLists.txt folder, fixes issue when project is in the drive root, like `X:` #16180
- Fix: Allowing `conan editable remove <path>` even when the path has been already deleted. #16170
- Fix: Fix *conan new –help* formatting issue. #16155
- Fix: Improved error message when there are conflicts in the graph. #16137
- Fix: Improve error message when one URL is not a valid server but still returns 200-ok under a Conan "ping" API call. #16126
- Fix: Solve `sqlite3` issues in FreeBSD due to queries with double quotes. #16123
- Fix: Clean error message for `conan cache restore <non-existing-file>`. #16113
- Fix: Improve UX and error messages when a remotes or credentials file in the cache is invalid/empty. #16091
- Fix: Use `cc` executable in Linux systems for autodetect compiler (`conan profile detect` and `detect_api`). #16074
- Fix: Improve the definition of version ranges UX with better error message for invalid `==, ~=, ^=` operators. #16069
- Fix: Improve error message UX when incorrect `settings.yml` or `settings_user.yml`. #16065
- Fix: Print a warning for Python 3.6 usage which is EOL since 2021. #16003
- Fix: Remove duplicated printing of command line in `Autotools` helper. #15991
- Fix: Add response error message output to HTTP Status 401 Errors in FileDownloader. #15983
- Fix: Add gcc 14 to default `settings.yml`. #15958
- Fix: Make `VCVars` use the `compiler.update` to specify the toolset. #15947
- Fix: Add `rc` to `AutotoolsToolchain` mapping of `compiler_executables` for cross-build Linux->Windows. #15946
- Fix: Add `Pop!_OS` to the distros using `apt-get` as system package manager. #15931
- Fix: Do not warn with package names containing the - character. #15920
- Fix: Fix html escaping of new `--format=html` graph output, and pass the graph serialized object instead of the string. #15915
- Bugfix: Make MesonToolchain listen to *tools.build:defines* conf variable. #16172 . Docs here
- Bugfix: Disallow *self.cpp_info* access in *validate_build()* method. #16135
- Bugfix: Don't show a trace when *.conanrc*'s *conan_home* is invalid. #16134
- Bugfix: Avoid the propagation of transitive dependencies of `tool_requires` to generators information even if they are marked as `visible=True`. #16077
- Bugfix: *BazelDeps* now uses the *requirement.build* property instead of *dependency.context* one. #16025

- Bugfix: Make *conan cache restore* work correctly when restoring over a package already in the local cache. #15950

13.63 2.2.3 (17-Apr-2024)

- Fix: Fix *to_apple_archs* method when using architectures from settings_user. #16090

13.64 2.2.2 (25-Mar-2024)

- Fix: Avoid issues with recipe `print(..., file=fileobj)`. #15934
- Fix: Fix broken calls to *print(x, file=y)* with duplicate keyword arguments. #15912
- Bugfix: Fix handling of *tools.build:defines* for multiconfig CMake. #15924

13.65 2.2.1 (20-Mar-2024)

- Fix: Add *copytree_compat* method for compatibility with Python>=3.12 after distutils removal. #15906

13.66 2.2.0 (20-Mar-2024)

- Feature: Raise for toolchains different than CMakeToolchain if using universal binary syntax. #15896
- Feature: Warn on misplaced requirement function calls #15888
- Feature: Print options conflicts in the graph caused by different branches recipes defining options values. #15876 . Docs here
- Feature: Add macOS versions 14.2, 14.3, 14.4 to *settings.yml*. #15859 . Docs here
- Feature: New graph `html`: more information, test-requires, hiding/showing different packages (build, test). #15846 . Docs here
- Feature: Add *–backup-sources* flag to *conan cache clean*. #15845
- Feature: Add *conan graph outdated* command that lists the dependencies that have newer versions in remotes #15838 . Docs here
- Feature: Set *CMAKE_VS_DEBUGGER_ENVIRONMENT* from CMakeToolchain to point to all binary directories when using Visual Studio. This negates the need to copy DLLs to launch executables from the Visual Studio IDE (requires CMake 3.27 or newer). #15830 . Docs here
- Feature: Add a parameter to *trim_conandata* to avoid raising an exception when conandata.yml file doesn't exist. #15829 . Docs here
- Feature: Added build_context_folder ` to PkgConfigDeps. `#15813 . Docs here
- Feature: Included build.pkg_config_path ` in the built-in options section in the MesonToolchain template. `#15813 . Docs here
- Feature: Update *_meson_cpu_family_map* to support *arm64ec*. #15812
- Feature: Added support for Clang 18. #15806 . Docs here
- Feature: Add basic support in CMakeToolchain for universal binaries. #15775 . Docs here

- Feature: New `tools.cmake.cmake_layout:build_folder` config that allows re-defining `cmake_layout` local build-folder. #15767 . Docs here
- Feature: New `tools.cmake.cmake_layout:test_folder` config that allows re-defining `cmake_layout` output build folder for `test_package`, including a `$TMP` placeholder to create a temporary folder in system `tmp`. #15767 . Docs here
- Feature: (Experimental) Add `conan config install-pkg myconf/[*]` new configuration inside Conan packages with new `package_type = "configuration"`. #15748 . Docs here
- Feature: (Experimental) New `core.package_id:config_mode` that allows configuration package reference to affect the `package_id` of all packages built with that configuration. #15748 . Docs here
- Feature: Make *cppstd_flag* public to return the corresponding C++ standard flag based on the settings. #15710 . Docs here
- Feature: Allow `self.name` and `self.version` in `build_folder_vars` attribute and conf. #15705 . Docs here
- Feature: Add `conan list --filter-xxx` arguments to list package binaries that match settings+options. #15697 . Docs here
- Feature: Add *detect_libc* to the *detect_api* to get the name and version of the C library. #15683 . Docs here
- Feature: New `CommandAPI` subapi in the `ConanAPI` that allows calling other commands. #15630 . Docs here
- Fix: Avoid unnecessary build of `tool_requires` when `--build=missing` and repeated `tool_requires`. #15885
- Fix: Fix `CMakeDeps` `set_property(... APPEND` argument order. #15877
- Fix: Raising an error when an infinite loop is found in the install graph (ill-formed dependency graph with loops). #15835
- Fix: Make sure *detect_default_compiler()* always returns a 3-tuple. #15832
- Fix: Print a clear message for `conan graph explain` when no binaries exist for one revision. #15823
- Fix: Add `package_type="static-library"` to the `conan new msbuild_lib` template. #15807
- Fix: Avoid `platform_requires` to fail when explicit options are being passed via `requires(.., options={})`. #15804
- Fix: Make `CMakeToolchain` end with newline. #15788
- Fix: Do not allow `conan list` binary filters or package query if a binary pattern is not provided. #15781
- Fix: Avoid `CMakeToolchain.preprocessor_definition` definitions to `"None"` literal string when it has no value (Python `None`). #15756
- Fix: Improved `conan install <path> --deployer-package=*` case that was crashing when using `self.package_folder`. #15737
- Fix: Fix *conan graph info –format=html* for large dependency graphs. #15724
- Fix: Make all recipe and plugins python file `print()` to `stderr`, so json outputs to `stdout` are not broken. #15704
- Fix: Fix getting the gnu triplet for Linux x86. #15699
- Bugfix: Solve backslash issues with `conan_home_folder` in `global.conf` when used in strings inside lists. #15870
- Bugfix: Fix `CMakeDeps` multi-config when there are conditional dependencies on the `build_type`. #15853
- Bugfix: Move *get_backup_sources()* method to expected *CacheAPI* from *UploadAPI*. #15845

- Bugfix: Avoid TypeError when a version in conandata.yml lists no patches. #15842
- Bugfix: Solve `package_type=build-scripts` issue with lockfiles and **`conan create`**. #15802
- Bugfix: Allow `--channel` command line argument if the recipe specifies `user` attribute. #15794
- Bugfix: Fix cross-compilation to Android from Windows when using `MesonToolchain`. #15790
- Bugfix: Fix `CMakeToolchain` GENERATOR_TOOLSET when `compiler.update` is defined. #15789
- Bugfix: Solved evaluation of `conf` items when they matched a Python module #15779
- Bugfix: Fix `PkgConfigDeps` generating .pc files for its `tool_requires` when it is in the build context already. #15763
- Bugfix: Adding *VISIBILITY* flags to *CONAN_C_FLAGS* too. #15762
- Bugfix: Fix *conan profile show –format=json* for profiles with scoped confs. #15747
- Bugfix: Fix legacy usage of *update* argument in Conan API. #15743
- Bugfix: Solve broken profile `[conf]` when strings contains Windows backslash. #15727
- Bugfix: Fix version precendence for metadata version ranges. #15653

13.67 2.1.0 (15-Feb-2024)

- Feature: Implement multi-config `tools.build:xxxx` flags in `CMakeToolchain`. #15654
- Feature: Add ability to pass patterns to *–update* flag. #15652 . Docs here
- Feature: Add *–format=json* formatter to **`conan build`**. #15651
- Feature: Added *tools.build.cross_building:cross_build* to decide whether cross-building or not regardless of the internal Conan mechanism. #15616
- Feature: Add *–format=json* option to *conan cache path*. #15613
- Feature: Add the –order-by argument for conan graph build-order. #15602 . Docs here
- Feature: Provide a new `graph build-order --reduce` argument to reduce the order exclusively to packages that need to be built from source. #15573 . Docs here
- Feature: Add configuration to specify desired CUDA Toolkit in CMakeToolchain for Visual Studio CMake generators. #15572 . Docs here
- Feature: New "important" options values definition, with higher precedence over regular option value definitions. #15571 . Docs here
- Feature: Display message when calling *deactivate_conanvcvars*. #15557
- Feature: Add `self.info` information of `package_id` to serialized output in the graph, and forward it to package-lists. #15553 . Docs here
- Feature: Log Git tool commands when running in verbose mode. #15514
- Feature: Add verbose debug information (with `-vvv`) for `conan.tools.files.copy()` calls. #15513
- Feature: Define `python_requires = "tested_reference_str"` for explicit `test_package` of `python_requires`. #15485 . Docs here
- Feature: Adding *CMakeToolchain.presets_build/run_environment* to modify *CMakePresets* environment in *generate()* method. #15470 . Docs here
- Feature: Add *–allowed-packages* to remotes to limit what references a remote can supply. #15464 . Docs here

- Feature: Initial documentation to make `RemotesAPI` publicly available (experimental). #15462
- Feature: Add support for use of vcvars env variables when calling from powershell. #15461 . Docs here
- Feature: New `Git(..., excluded=[])` feature to avoid "dirty" errors in `Git` helper. #15457 . Docs here
- Feature: New `core.scm:excluded` feature to avoid "dirty" errors in `Git` helper and `revision_mode = "scm"`. #15457 . Docs here
- Feature: Recipe `python_package_id_mode` for `python_requires` recipes, to define per-recipe effect on consumers `package_id`. #15453 . Docs here
- Feature: Add cmakeExecutable to configure preset. #15447 . Docs here
- Feature: Add new `--core-conf` command line argument to allow passing *core.* confs via CLI. #15441 . Docs here
- Feature: Add `detect_api.detect_msvc_update(version)` helper to `detect_api`. #15435 . Docs here
- Feature: `CMakeToolchain` defines `jobs` in generated `CMakePresets.json` buildPresets. #15422
- Feature: Allow nested "ANY" definitions in `settings.yml`. #15415 . Docs here
- Feature: Helpers `Git().coordinates_to_conandata()` and `Git().checkout_from_conandata_coordinates()` to simplify scm based flows. #15377
- Feature: `AutotoolsToolchain` automatically inject `-FS` for VS. #15375
- Feature: New **`conan upload`** `core.upload:parallel` for faster parallel uploads. #15360 . Docs here
- Feature: Intel oneAPI compiler detection improvement. #15358
- Feature: Display progress for long `conan list` commands. #15354
- Feature: Add *extension_properties* attribute to pass information to extensions from recipes. #15348 . Docs here
- Feature: Implement *compatibility_cppstd* in *extension_properties* for the `compatibility.py` plugin to disable fallback to other cppstd for the recipe. #15348 . Docs here
- Feature: Add `Git.get_commit(..., repository=True)` to obtain the repository commit, not the folder commit. #15304
- Feature: Ensure `--build=editable` and `--build=cascade` works together. #15300 . Docs here
- Feature: New `conan graph build-order --order=configuration` to output a different order, sorted by package binaries/configurations, not grouped by recipe revisions. #15270 . Docs here
- Feature: Allow copy&paste of recipe revisions with timestamps from `--format=compact` into `conan lock add`. #15262 . Docs here
- Fix: Guarantee order of *generators* attribute execution. #15678
- Fix: Solve issue with `[platform_tool_requires]` in the build profile and context. Discard `[platform_requires]` in build profile. #15665
- Fix: Fix gcc detection in conda environments. #15664
- Fix: Improve handling of *.dirty* download files when uploading backup sources. #15601
- Fix: Fix relativize paths in generated files. #15592
- Fix: Allow `None` values for `CMakeToolchain.preprocessor_definitions` that will map to definitions without values. #15545 . Docs here
- Fix: Fix *graph build-order –order=configuration* text format output. #15538
- Fix: Raise a helpful error when the remote is not reachable in case the user wants to work in offline mode. #15516

- Fix: Avoid missing file stacktrace when no metadata exists for a source backup. #15501
- Fix: Remove `--lockfile-packages` argument, it was not documented as it is was not intended for public usage. #15499 . Docs here
- Fix: Raise if *check_type=int* and conf value is set to *bool*. #15378
- Fix: Add *pkg-config* entry to machine file generated by MesonToolchain, due to *pkgconfig* entry being deprecated since Meson 1.3.0. #15369
- Fix: Fix *graph explain* not showing some differences in requirements if missing. #15355
- Fix: Fix *tools.info.package_id:confs* when pattern did not match any defined conf. #15353
- Fix: Fix `upload_policy=skip` with `--build=missing` issues. #15336
- Fix: Accept `conan download/upload --list=.. --only-recipe` to download only the recipes. #15312
- Fix: Allow `cmake.build(build_type="Release")` for recipes built with multi-config systems but without `build_type` setting. #14780
- Bugfix: Fix `MSBuildDeps` with components and skipped dependencies. #15626
- Bugfix: Avoid `provides` raising an error for packages that self `tool_requires` to themselves to cross-build. #15575
- Bugfix: Fix build scope OS detection in *tools.microsoft.visual.VCVars*. #15568
- Bugfix: Fix wrong propagation over `visible=False` when dependency is header-only. #15564
- Bugfix: Store the temporary cache folders inside `core.cache:storage_path`, so `conan cache clean` also finds and clean them correctly. #15505
- Bugfix: The `conan export-pkg --format=json` output now returns `recipe = "cache"` status, as the recipe is in the cache after the command. #15504
- Bugfix: The **`conan export-pkg`** command stores the lockfile excluding the `test_package`, following the same behavior as **`conan create`**. #15504
- Bugfix: Avoid **`conan test`** failing for `python_requires` test-package. #15485 . Docs here
- Bugfix: MesonToolchain calculates a valid *apple_min_version_flag*. #15465
- Bugfix: Allow to limit `os`, `compiler` and other settings with subsettings in `build_id()` and `package_id()` methods. #15439
- Bugfix: Fix getting environment variable CONAN_LOGIN_USERNAME_REMOTE. #15388
- Bugfix: Don't take . folder into consideration for *tools.files.copy() excludes* patterns. #15349
- Bugfix: Disable creating editables without name and version. #15337
- Bugfix: Fix *Git.get_url_and_commit* raising for some Git configs. #15271
- Bugfix: Direct dependencies in the "host" context of packages being built shouldn't be skipped. This allows for non C/C++ libraries artifacts, like images, in the "host" context, to be used as build-time resources. #15128

13.68 2.0.17 (10-Jan-2024)

- Fix: Automatically create folder if `conan cache save --file=subfolder/file.tgz` subfolder doesn't exist. #15409
- Bugfix: Fix libcxx detection when using *CC/CXX* env vars. #15418 . Docs here
- Bugfix: Solve `winsdk_version` bug in `CMakeToolchain` generator for `cmake_minimum_required(3.27)`. #15373
- Bugfix: Fix visible trait propagation with `build=True` trait. #15357
- Bugfix: Fix *package_id* calculation when including conf values thru *tools.info.package_id:confs*. #15356
- Bugfix: Order *conf* items when dumping them to allow reproducible *package_id* independent of the order the confs were declared. #15356

13.69 2.0.16 (21-Dec-2023)

- Bugfix: Revert the default of `source_buildenv`, make it `False` by default. #15319 . Docs here

13.70 2.0.15 (20-Dec-2023)

- Feature: New `conan lock remove` command to remove requires from lockfiles. #15284 . Docs here
- Feature: New `CMake.ctest()` helper method to launch directly `ctest` instead of via `cmake --target=RUN_TEST`. #15282
- Feature: Add tracking syntax in <*host_version*> for different references. #15274 . Docs here
- Feature: Adding `tools.microsoft:winsdk_version` conf to make `VCVars` generator to use the given `winsdk_version`. #15272 . Docs here
- Feature: Add *pkglist* formatter for conan export command. #15266 . Docs here
- Feature: Define `CONAN_LOG_LEVEL` env-var to be able to change verbosity at a global level. #15263 . Docs here
- Feature: *conan cache path xxx –folder xxxx* raises an error if the folder requested does not exist. #15257
- Feature: Add *in* operator support for ConanFile's *self.dependencies*. #15221 . Docs here
- Feature: Make `CMakeDeps` generator create a `conandeps.cmake` that aggregates all direct dependencies in a `cmake`-like generator style. #15207 . Docs here
- Feature: Add build environment information to CMake configure preset and run environment information to CMake test presets. #15192 . Docs here
- Feature: Removed a warning about a potential issue with conan migration that would print every time a build failed. #15174
- Feature: New `deploy()` method in recipes for explicit per-recipe deployment. #15172 . Docs here
- Feature: Allow `tool-requires` to be used in `source()` method injecting environment. #15153 . Docs here
- Feature: Allow accessing the contents of *settings.yml* (and *settings_user*!) from *ConfigAPI*. #15151
- Feature: Add builtin conf access from *ConfigAPI*. #15151
- Feature: Add *redirect_stdout* to CMake integration methods. #15150

- Feature: Add *core:warnings_as_errors* configuration option to make Conan raise on warnings and errors. #15149 . Docs here
- Feature: Added *FTP_TLS* option using *secure* argument in *ftp_download* for secure communication. #15137
- Feature: New `[replace_requires]` and `[replace_tool_requires]` in profile for redefining requires, useful for package replacements like `zlibng/zlib`, to solve conflicts, and to replace some dependencies by system alternatives wrapped in another Conan package recipe. #15136 . Docs here
- Feature: Add *stderr* capture argument to conanfile's *run()* method. #15121 . Docs here
- Feature: New `[platform_requires]` profile definition to be able to replace Conan dependencies by platform-provided dependencies. #14871 . Docs here
- Feature: New `conan graph explain` command to search, compare and explain missing binaries. #14694 . Docs here
- Feature: Global `cpp_info` can be used to initialize components values. #13994
- Fix: Make *core:warnings_as_errors* accept a list #15297
- Fix: Fix *user* confs package scoping when no separator was given #15296
- Fix: Fix range escaping in conflict reports involving ranges. #15222
- Fix: Allow hard `set_name()` and `set_version()` to mutate name and version provided in command line. #15211 . Docs here
- Fix: Make *conan graph info –format=text* print to stdout. #15170
- Fix: Avoid warning in CMake output due to *CMAKE_POLICY_DEFAULT_CMP0091* unused variable. #15127
- Fix: Deprecate `[system_tools]` in favor of `[platform_tool_requires]` to align with `[platform_requires]` for regular dependencies. Changed output from "System tool" to "Platform". #14871 . Docs here
- Bugfix: Ensure *user* confs have at least 1 *:* separator #15296
- Bugfix: `Git.is_dirty()` will use `git status . -s` to make sure it only process the current path, not the whole repo, similarly to other `Git` methods. #15289
- Bugfix: Make `self.info.clear()` and header-only packages to remove `python_requires` and `tool_requires`. #15285 . Docs here
- Bugfix: Make `conan cache save/restore` portable across Windows and other OSs. #15253
- Bugfix: Do not relativize absolute paths in `deployers`. #15244
- Bugfix: Add `architecture` to `CMakePresets` to avoid cmake ignoring toolchain definitions when using presets. #15215
- Bugfix: Fix *conan graph info –format=html* reporting misleading conflicting nodes. #15196
- Bugfix: Fix serialization of tool_requires in *conan profile show –format=json*. #15185
- Bugfix: Fix NMakeDeps quoting issues. #15140
- Bugfix: Fix the 2.0.14 migration to add LRU data to the cache when `storage_path` conf is defined. #15135
- Bugfix: Fix definition of `package_metadata_folder` for **`conan export-pkg`** command. #15126
- Bugfix: *pyinstaller.py* was broken for Python 3.12 due to a useless *distutils* import. #15116
- Bugfix: Fix backup sources error when no *core.sources:download_cache* is set. #15109

13.71 2.0.14 (14-Nov-2023)

- Feature: Added `riscv64, riscv32` architectures to default `settings.yml` and management of them in Meson and Autotools. #15053
- Feature: Allow only one simultaneous database connection. #15029
- Feature: Add *conan cache backup-upload* to upload all the backup sources in the cache, regardless of which references they are from #15013 . Docs here
- Feature: New `conan list --format=compact` for better UX. #15011 . Docs here
- Feature: Ignore metadata upload by passing –metadata="" #15007 . Docs here
- Feature: Better output messages in **`conan upload`** #14984
- Feature: Add extra flags to CMakeToolchain. #14966 . Docs here
- Feature: Implement package load/restore from the cache, for CI workflows and moving packages over air-gaps. #14923 . Docs here
- Feature: Compute version-ranges intersection to avoid graph version conflicts for compatible ranges #14912
- Feature: CMake helper can use multiple targets in target argument. #14883
- Feature: Add Macos 13.6 to settings.yml. #14858 . Docs here
- Feature: Add CMakeDeps and PkgConfigDeps generators listening to system_package_version property. #14808 . Docs here
- Feature: Add shorthand syntax in cli to specify host and build in 1 argument #14727 . Docs here
- Feature: Implement cache LRU control for cleaning of unused artifacts. #14054 . Docs here
- Fix: Avoid `CMakeToolchain` overwriting user `CMakePresets.json` when no layout nor output-folder is defined #15058
- Fix: Add `astra`, `elbrus` and `altlinux` as distribution using `apt` in SystemPackageManager #15051
- Fix: Default to apt-get package manager in Linux Mint #15026 . Docs here
- Fix: Make `Git()` check commits in remote server even for shallow clones. #15023
- Fix: Add new Apple OS versions to settings.yml #15015
- Fix: Fix AutotoolsToolchain extraflags priority. #15005 . Docs here
- Fix: Remove colors from `conan --version` output #15002
- Fix: Add an error message if the sqlite3 version is unsupported (less than 3.7.11 from 2012) #14950
- Fix: Make cache DB always use forward slash for paths, to be uniform across Windows and Linux #14940
- Fix: Solve re-build of an existing package revision (like forcing rebuild of a an existing header-only package), while previous folder was still used by other projects. #14938
- Fix: Avoid a recipe mutating a `conf` to affect other recipes. #14932 . Docs here
- Fix: The output of system packages via `Apt.install()` or `PkgConfig.fill_cpp_info`, like `xorg/system` was very noisy to the Conan output, making it more quiet #14924
- Fix: Serialize the `path` information of `python_requires`, necessary for computing buildinfo #14886
- Fix: Define remotes in **`conan source`** command in case recipe has `python_requires` that need to be downloaded from remotes. #14852
- Fix: Fix min target flag for xros and xros-simulator. #14776

- Bugfix: `--build=missing` was doing unnecessary builds of packages that were not needed and could be skipped, in the case of `tool_requires` having transitive dependencies. #15082
- BugFix: Add package revision to format=json in 'conan export-pkg' command #15042
- Bugfix: `tools.build:download_source=True` will not fail when there are editable packages. #15004 . Docs here
- Bugfix: Transitive dependencies were incorrectly added to conandeps.xcconfig. #14898
- Bugfix: Fix integrity-check (`upload --check` or `cache check-integrity`) when the `export_source` has not been downloaded #14850
- Bugfix: Properly lock release candidates of python requires #14846
- BugFix: Version ranges ending with - do not automatically activate pre-releases resolution in the full range. #14814 . Docs here
- BugFix: Fix version ranges so pre-releases are correctly included in the lower bound and excluded in the upper bound. #14814 . Docs here

13.72 2.0.13 (28-Sept-2023)

- Bugfix: Fix wrong cppstd detection for newer apple-clang versions introduced in 2.0.11. #14837

13.73 2.0.12 (26-Sept-2023)

- Feature: Add support for Clang 17. #14781 . Docs here
- Feature: Add *–dry-run* for **`conan remove`**. #14760 . Docs here
- Feature: Add *host_tool* to *install()* method in *package_manager* to indicate whether the package is a host tool or a library. #14752 . Docs here
- Fix: Better error message when trying to `export-pkg` a `python-require` package, and avoid it being exported and then failing. #14819
- Fix: `CMakeDeps` allows `set_property()` on all properties. #14813
- Fix: Add minor version for Apple clang 15.0. #14797 . Docs here
- Fix: **`conan build`** command prettier error when <path> argument not provided. #14787
- Bugfix: fix `compatibility()` over `settings_target` making it None #14825
- Bugfix: `compatible` packages look first in the cache, and only if not found, the servers, to allow offline installs when there are compatible packages. #14800
- BugFix: Reuse session in ConanRequester to speed up requests. #14795
- Bugfix: Fix relative paths of `editable` packages when they have components partially defining directories. #14782

13.74 2.0.11 (18-Sept-2023)

- Feature: Add `--format=json` formatter to `conan profile show` command #14743 . Docs here
- Feature: add new –deployer –generators args to 'conan build' cmd #14737 . Docs here
- Feature: Better `CMakeToolchain` blocks interface. Added new `.blocks.select()`, `.blocks.keys()`. #14731 . Docs here
- Feature: Add message when copying large files from download cache instead of downloading from server. #14716
- Feature: MesonToolchain shows a warning message if any options are used directly. #14692 . Docs here
- Feature: Support clang-cl in default profile plugin. #14682 . Docs here
- Feature: Added mechanism to transform *c*, *cpp*, and/or *ld* binaries variables from Meson into lists if declared blank-separated strings. #14676
- Feature: Add *nobara* distro to *dnf* package manager mapping. #14668
- Feature: Ensure meson toolchain sets *b_vscrt* with clang-cl. #14664
- Feature: Supporting regex pattern for conf *tools.info.package_id:confs* #14621 . Docs here
- Feature: MakeDeps: Provide "require" information, and more styling tweaks. #14605
- Feature: New `detect_api` to be used in profiles jinja templates. #14578 . Docs here
- Feature: Allow access to *settings_target* in compatibility method. #14532
- Fix: Add missing minor macos versions #14740 . Docs here
- Fix: Improve error messages in *ConanApi* init failures, #14735
- Fix: CMakeDeps: Remove "Target name ... already exists" warning about duplicating aliases. #14644
- Bugfix: Fix regression in `Git.run()` when `win_bash=True`. #14756
- Bugfix: Change the default *check=False* in *conan.tools.system.package_manager.Apt* to *True* as the other package manager tools. #14728 . Docs here
- Bugfix: Solved propagation of transitive shared dependencies of `test_requires` with diamonds. #14721
- Bugfix: Solve crash with **`conan export-pkg`** with `test_package` doing calls to remotes. #14712
- Bugfix: Do not binary-skip packages that have transitive dependencies that are not skipped, otherwise the build chain of build systems to those transitive dependencies like `CMakeDeps` generated files are broken. #14673
- Bugfix: Fix detected CPU architecture when running `conan profile detect` on native ARM64 Windows. #14667
- Bugfix: `conan lock create --update` now correctly updates references from servers if newer than cache ones. #14643
- Bugfix: Fix unnecessarily decorating command stdout with escape sequences. #14642
- Bugfix: `tools.info.package_id:confs` shouldn't affect header-only libraries. #14622

13.75 2.0.10 (29-Aug-2023)

- Feature: Allow `patch_user` in `conandata.yml` definition for user patches, not handled by `apply_conandata_patches()`. #14576 . Docs here
- Feature: Support for Xcode 15, iOS 17, tvOS 17, watchOS 10, macOS 14. #14538
- Feature: Raise an error if users are adding incorrect ConanCenter web URL as remote. #14531
- Feature: Serialization of graph with `--format=json` adds information to `python_requires` so `conan list --graph` can list `python_requires` too. #14529
- Feature: Add `rrev`, `rrev_timestamp` and `prev_timestamp` to the graph json serialization. #14526
- Feature: Allow `version-ranges` to resolve to editable packages too. #14510
- Feature: Add *tools.files.download:verify*. #14508 . Docs here
- Feature: Add support for Apple visionOS. #14504
- Feature: Warn of unknown version range options. #14493
- Feature: Add *tools.graph:skip_binaries* to control binary skipping in the graph. #14466 . Docs here
- Feature: New `tools.deployer:symlinks` configuration to disable symlinks copy in deployers. #14461 . Docs here
- Feature: Allow remotes to automatically resolve missing `python_requires` in 'editable add'. #14413 . Docs here
- Feature: Add `cli_args` argument for `CMake.install()`. #14397 . Docs here
- Feature: Allow `test_requires(..., force=True)`. #14394 . Docs here
- Feature: New `credentials.json` file to store credentials for Conan remotes. #14392 . Docs here
- Feature: Added support for *apk* package manager and Alpine Linux #14382 . Docs here
- Feature: *conan profile detect* can now detect the version of msvc when invoked within a Visual Studio prompt where *CC* or *CXX* are defined and pointing to the *cl* compiler executable #14364
- Feature: Properly document `--build=editable` build mode. #14358 . Docs here
- Feature: `conan create --build-test=missing` new argument to control what is being built as dependencies of the `test_package` folder. #14347 . Docs here
- Feature: Provide new `default_build_options` attribute for defining options for `tool_requires` in recipes. #14340 . Docs here
- Feature: Implement `...@` as a pattern for indicating matches with packages without user/channel. #14338 . Docs here
- Feature: Add support to Makefile by the new MakeDeps generator #14133 . Docs here
- Fix: Allow *–format=json* in **`conan create`** for *python-requires* #14594
- Fix: Remove conan v2 ready conan-center link. #14593
- Fix: Make **`conan inspect`** use all remotes by default. #14572 . Docs here
- Fix: Allow extra hyphens in versions pre-releases. #14561
- Fix: Allow confs for `tools.cmake.cmaketoolchain` to be used if defined even if `tools.cmake.cmaketoolchain:user_toolchain` is defined. #14556 . Docs here
- Fix: Serialize booleans of `dependencies` in `--format=json` for graphs as booleans, not strings. #14530 . Docs here

- Fix: Avoid errors in **conan upload** when `python_requires` are not in the cache and need to be downloaded. #14511
- Fix: Improve error check of `lock add` adding a full package reference instead of a recipe reference. #14491
- Fix: Better error message when a built-in deployer failed to copy files. #14461 . Docs here
- Fix: Do not print non-captured stacktraces to `stdout` but to `stderr`. #14444
- Fix: Serialize `conf_info` in `--format=json` output. #14442
- Fix: *MSBuildToolchain/MSBuildDeps*: Avoid passing C/C++ compiler options as options for *ResourceCompile*. #14378
- Fix: Removal of plugin files result in a better error message instead of stacktrace. #14376
- Fix: Fix CMake system processor name on armv8/aarch64. #14362
- Fix: Make backup sources `core.sources` conf not mandate the final slash. #14342
- Fix: Correctly propagate options defined in recipe `default_options` to `test_requires`. #14340 . Docs here
- Fix: Invoke XCRun using conanfile.run() so that environment is injected. #14326
- Fix: Use `abspath` for `conan config install` to avoid symlinks issues. #14183
- Bugfix: Solve `build_id()` issues, when multiple different `package_ids` reusing same build-folder. #14555
- Bugfix: Avoid DB errors when timestamp is not provided to **conan download** when using package lists. #14526
- Bugfix: Print exception stacktrace (when *-vtrace* is set) into stderr instead of stdout #14522
- Bugfix: Print only packages confirmed interactively in **conan upload**. #14512
- Bugfix: 'conan remove' was outputting all entries in the cache matching the filter not just the once which where confirmed by the user. #14478
- Bugfix: Better error when passing *–channel* without *–user*. #14443
- Bugfix: Fix `settings_target` computation for `tool_requires` of packages already in the "build" context. #14441
- Bugfix: Avoid `DB is locked` error when `core.download:parallel` is defined. #14410
- Bugfix: Make generated powershell environment scripts relative when using deployers. #14391
- Bugfix: fix profile [tool_requires] using revisions that were ignored. #14337

13.76 2.0.9 (19-Jul-2023)

- Feature: Add *implements* attribute in ConanFile to provide automatic management of some options and settings. #14320 . Docs here
- Feature: Add *apple-clang* 15. #14302
- Fix: Allow repository being dirty outside of *conanfile.py* folder when using *revision_mode = "scm_folder"*. #14330
- Fix: Improve the error messages and provide Conan traces for errors in *compatibility.py* and *profile.py* plugins. #14322
- Fix: `flush()` output streams after every message write. #14310
- Bugfix: Fix Package signing plugin not verifying the downloaded sources. #14331 . Docs here
- Bugfix: Fix `CMakeUserPresets` inherits from conan generated presets due to typo. #14325

- Bugfix: ConanPresets.json contains duplicate presets if multiple user presets inherit from the same conan presets. #14296
- Bugfix: Meson *prefix* param is passed as UNIX path. #14295
- Bugfix: Fix *CMake Error: Invalid level specified for –loglevel* when *tools.build:verbosity* is set to *quiet*. #14289

13.77 2.0.8 (13-Jul-2023)

- Feature: Add GCC 10.5 to default settings.yml. #14252
- Feature: Let *pkg_config_custom_content* overwrite default **.pc* variables created by *PkgConfigDeps*. #14233 . Docs here
- Feature: Let *pkg_config_custom_content* be a dict-like object too. #14233 . Docs here
- Feature: The *fix_apple_shared_install_name* tool now uses *xcrun* to resolve the *otool* and *install_name_tool* programs. #14195
- Feature: Manage shared, fPIC, and header_only options automatically. #14194 . Docs here
- Feature: Manage package ID for header-library package type automatically. #14194 . Docs here
- Feature: New `cpp_info.set_property("cmake_package_version_compat" , "AnyNewerVersion")` for `CMakeDeps` generator. #14176 . Docs here
- Feature: Metadata improvements. #14152
- Fix: Improve error message when missing binaries with **`conan test`** command. #14272
- Fix: Make **`conan download`** command no longer need to load conanfile, won't fail for 1.X recipes or missing `python_requires`. #14261
- Fix: Using *upload* with the *–list* argument now permits empty recipe lists. #14254
- Fix: Guarantee that `Options.rm_safe` never raises. #14238
- Fix: Allow *tools.gnu:make_program* to affect every CMake configuration. #14223
- Fix: Add missing *package_type* to **`conan new`** lib templates. #14215
- Fix: Add clarification for the default folder shown when querying a package reference. #14199 . Docs here
- Fix: Enable existing status-message code in the *patch()* function. #14177
- Fix: Use `configuration` in `XcodeDeps` instead of always `build_type`. #14168
- Fix: Respect symlinked path for cache location. #14164
- Fix: `PkgConfig` uses `conanfile.run()` instead of internal runner to get full Conan environment from profiles and dependencies. #13985
- Bugfix: Fix leaking of `CMakeDeps` `CMAKE_FIND_LIBRARY_SUFFIXES` variable. #14253
- Bugfix: Fix conan not finding generator by name when multiple custom global generators are detected. #14227
- Bugfix: Improve display of graph conflicts in *conan graph info* in html format. #14190
- Bugfix: Fix `CMakeToolchain` cross-building from Linux to OSX. #14187
- Bugfix: Fix KeyError in backup sources when no package is selected. #14185

13.78 2.0.7 (21-Jun-2023)

- Feature: Add new `arm64ec` architecture, used to define CMAKE_GENERATOR_PLATFORM. #14114 . Docs here
- Feature: Make `CppInfo` a public, documented, importable tool for generators that need to aggregate them. #14101 . Docs here
- Feature: Allow `conan remove --list=pkglist` to remove package-lists. #14082 . Docs here
- Feature: Output for `conan remove --format` both text (summary of deleted things) and json. #14082 . Docs here
- Feature: Add *core.sources:excluded_urls* to backup sources. #14020
- Feature: **`conan test`** command learned the `--format=json` formatter. #14011 . Docs here
- Feature: Allow `pkg/[version-range]` expressions in `conan list` (and download, upload, remove) patterns. #14004 . Docs here
- Feature: Add `conan upload --dry-run` equivalent to 1.X `conan upload --skip-upload`. #14002 . Docs here
- Feature: Add new command *conan version* to format the output. #13999 . Docs here
- Feature: Small UX improvement to print some info while downloading large files. #13989
- Feature: Use `PackagesList` as input argument for `conan upload --list=pkglist.json`. #13928 . Docs here
- Feature: Use `--graph` input for `conan list` to create a `PackagesList` that can be used as input for **`conan upload`**. #13928 . Docs here
- Feature: New metadata files associated to recipes and packages that can be uploaded, downloaded and added after the package exists. #13918
- Feature: Allow to specify a custom deployer output folder. #13757 . Docs here
- Feature: Split build & compilation verbosity control to two confs. #13729 . Docs here
- Feature: Added *bindir* to generated *.pc* file in *PkgConfigDeps*. #13623 . Docs here
- Fix: Deprecate `AutoPackage` remnant from Conan 1.X. #14083 . Docs here
- Fix: Fix description for the conf entry for default build profile used. #14075 . Docs here
- Fix: Allow spaces in path in `Git` helper. #14063 . Docs here
- Fix: Remove trailing `.` in `conanfile.xxx_folder` that is breaking subsystems like msys2. #14061
- Fix: Avoid caching issues when some intermediate package in the graph calls `aggregated_components()` over some dependency and using `--deployer`, generators still pointed to the Conan cache and not deployed copy. #14060
- Fix: Allow internal `Cli` object to be called more than once. #14053
- Fix: Force `pyyaml>=6` for Python 3.10, as previous versions broke. #13990
- Fix: Improve graph conflict message when Conan can't show one of the conflicting recipes. #13946
- Bugfix: Solve bug in timestamp of non-latest revision download from the server. #14110
- Bugfix: Fix double base path setup in editable packages. #14109
- Bugfix: Raise if `conan graph build-order` graph has any errors, instead of quietly doing nothing and outputting and empty json. #14106

- Bugfix: Avoid incorrect path replacements for `editable` packages when folders have overlapping matching names. #14095
- Bugfix: Set clang as the default FreeBSD detected compiler. #14065
- Bugfix: Add prefix var and any custom content (through the *pkg_config_custom_content* property) to already generated pkg-config root .pc files by *PkgConfigDeps*. #14051
- Bugfix: **`conan create`** command returns always the same output for `--format=json` result graph, irrespective of test_package existence. #14011 . Docs here
- Bugfix: Fix problem with `editable` packages when defining `self.folders.root=".."` parent directory. #13983
- Bugfix: Removed *libdir1* and *includedir1* as the default index. Now, *PkgConfigDeps* creates the *libdir* and *includedir* variables by default in *.pc* files. #13623 . Docs here

13.79 2.0.6 (26-May-2023)

- Feature: Add a *tools.cmake:cmake_program* configuration item to allow specifying the location of the desired CMake executable. #13940 . Docs here
- Fix: Output "id" property in graph json output as str instead of int. #13964 . Docs here
- Fix: Fix custom commands in a layer not able to do a local import. #13944
- Fix: Improve the output of download + unzip. #13937
- Fix: Add missing values to *package_manager:mode* in *conan config install*. #13929
- Bugfix: Ensuring the same graph-info JSON output for *graph info*, *create*, *export-pkg*, and *install*. #13967 . Docs here
- Bugfix: `test_requires` were affecting the `package_id` of consumers as regular `requires`, but they shouldn't. #13966
- Bugfix: Define `source_folder` correctly in the json output when `-c tools.build:download_source=True`. #13953
- Bugfix: Fixed and completed the *graph info xxxx –format json* output, to publicly document it. #13934 . Docs here
- Bugfix: Fix "double" absolute paths in premakedeps. #13926
- Bugfix: Fix regression from 2.0.5 https://github.com/conan-io/conan/pull/13898, in which overrides of packages and components specification was failing #13923

13.80 2.0.5 (18-May-2023)

- Feature: *-v* argument defaults to the *VERBOSE* level. #13839
- Feature: Avoid showing unnecessary skipped dependencies. Now, it only shows a list of reference names if exists skipped binaries. They can be completely listed by adding *-v* (verbose mode) to the current command. #13836
- Feature: Allow step-into dependencies debugging for packages built locally with `--build` #13833 . Docs here
- Feature: Allow non relocatable, locally built packages with `upload_policy="skip"` and `build_policy="missing"` #13833 . Docs here

- Feature: Do not move "build" folders in cache when `package-revision` is computed to allow locating sources for dependencies debuggability with step-into #13810
- Feature: New `settings.possible_values()` method to query the range of possible values for a setting. #13796 . Docs here
- Feature: Optimize and avoid hitting servers for binaries when `upload_policy=skip` #13771
- Feature: Partially relativize generated environment .sh shell scripts #13764
- Feature: Improve settings.yml error messages #13748
- Feature: Auto create empty `global.conf` to improve UX looking for file in home. #13746 . Docs here
- Feature: Render the profile file name as profile_name #13721 . Docs here
- Feature: New global custom generators in cache "extensions/generators" that can be used by name. #13718 . Docs here
- Feature: Improve **`conan inspect`** output, it now understands *set_name/set_version*. #13716 . Docs here
- Feature: Define new `self.tool_requires("pkg/<host_version>")` to allow some tool-requires to follow and use the same version as the "host" regular requires do. #13712 . Docs here
- Feature: Introduce new `core:skip_warns` configuration to be able to silence some warnings in the output. #13706 . Docs here
- Feature: Add info_invalid to graph node serialization #13688
- Feature: Computing and reporting the `overrides` in the graph, and in the `graph build-order` #13680
- Feature: New `revision_mode = "scm_folder"` for mono-repo projects that want to use `scm` revisions. #13562 . Docs here
- Feature: Demonstrate that it is possible to `tool_requires` different versions of the same package. #13529 . Docs here
- Fix: *build_scripts* now set the *run* trait to *True* by default #13901 . Docs here
- Fix: Fix XcodeDeps includes skipped dependencies. #13880
- Fix: Do not allow line feeds into `pkg/version` reference fields #13870
- Fix: Fix `AutotoolsToolchain` definition of `tools.build:compiler_executable` for Windows subsystems #13867
- Fix: Speed up the CMakeDeps generation #13857
- Fix: Fix imported library config suffix. #13841
- Fix: Fail when defining an unknown conf #13832
- Fix: Fix incorrect printing of "skipped" binaries in the `conan install/create` commands, when they are used by some other dependencies. #13778
- Fix: Renaming the cache "deploy" folder to "deployers" and allow `-d, --deployer` cli arg. ("deploy" folder will not break but will warn as deprecated). #13740 . Docs here
- Fix: Omit `-L` libpaths in `CMakeDeps` for header-only libraries. #13704
- Bugfix: Fix when a `test_requires` is also a regular transitive "host" requires and consumer defines components. #13898
- Bugfix: Fix propagation of options like `*:shared=True` defined in recipes #13855
- Bugfix: Fix `--lockfile-out` paths for 'graph build-order' and 'test' commands #13853
- Bugfix: Ensure backup sources are uploaded in more cases #13846

- Bugfix: fix `settings.yml` definition of `intel-cc` `cppstd=03` #13844
- Bugfix: Fix **`conan upload`** with backup sources for exported-only recipes #13779
- Bugfix: Fix `conan lock merge` of lockfiles containing alias #13763
- Bugfix: Fix python_requires in transitive deps with version ranges #13762
- Bugfix: fix CMakeToolchain CMAKE_SYSTEM_NAME=Generic for baremetal #13739
- Bugfix: Fix incorrect environment scripts deactivation order #13707
- Bugfix: Solve failing lockfiles when graph has requirements with `override=True` #13597

13.81 2.0.4 (11-Apr-2023)

- Feature: extend `--build-require` to more commands (`graph info`, `lock create`, `install`) and cases. #13669 . Docs here
- Feature: Add *-d tool_requires* to **`conan new`**. #13608 . Docs here
- Feature: Make CMakeDeps, CMakeToolchain and Environment (.bat, Windows only) generated files have relative paths. #13607
- Feature: Adding preliminary (non documented, dev-only) support for premake5 deps (PremakeDeps). #13390
- Fix: Update old **`conan user`** references to `conan remote login`. #13671
- Fix: Improve dependencies options changed in `requirements()` error msg. #13668
- Fix: [system_tools] was not reporting the correct resolved version, but still the original range. #13667
- Fix: Improve *provides* conflict message error. #13661
- Fix: When server responds Forbidden to the download of 1 file in a recipe/package, make sure other files and DB are cleaned. #13626
- Fix: Add error in **`conan remove`** when using *–package-query* without providing a pattern that matches packages. #13622
- Fix: Add `direct_deploy` subfolder for the `direct_deploy` deployer. #13612 . Docs here
- Fix: Fix html output when pattern does not list package revisions, like: `conan list "*#*:*"`. #13605
- Bugfix: `conan list -p <package-query>` failed when a package had no settings or options. #13662
- Bugfix: *python_requires* now properly loads remote requirements. #13657
- Bugfix: Fix crash when `override` is used in a node of the graph that is also the closing node of a diamond. #13631
- Bugfix: Fix the `--package-query` argument for `options`. #13618
- Bugfix: Add `full_deploy` subfolder for the `full_deploy` deployer to avoid collision with “build” folder. #13612 . Docs here
- Bugfix: Make *STATUS* the default log level. #13610
- Bugfix: Fix double delete error in *conan cache clean*. #13601

13.82 2.0.3 (03-Apr-2023)

- Feature: `conan cache clean` learned the `--all` and `--temp` to clean everything (sources, builds) and also the temporary folders. #13581 . Docs here
- Feature: Introduce the `conf` dictionary update semantics with *= operator. #13571 . Docs here
- Feature: Support MacOS SDK 13.1 (available in Xcode 14.2). #13531
- Feature: The `full_deploy` deployer together with `CMakeDeps` generator learned to create relative paths deploys, so they are relocatable. #13526
- Feature: Introduce the `conan remove *#!latest` (also for package-revisions), to remove all revisions except the latest one. #13505 . Docs here
- Feature: New `conan cache check-integrity` command to replace 1.X legacy `conan upload --skip-upload --check`. #13502 . Docs here
- Feature: Add filtering for options and settings in conan list html output. #13470
- Feature: Automatic server side source backups for third parties. #13461
- Feature: Add *tools.android:cmake_legacy_toolchain* configuration useful when building CMake projects for Android. If defined, this will set the value of *ANDROID_USE_LEGACY_TOOLCHAIN_FILE*. It may be useful to set this to *False* if compiler flags are defined via *tools.build:cflags* or *tools.build:cxxflags* to prevent Android's legacy CMake toolchain from overriding the values. #13459 . Docs here
- Feature: Default `tools.files.download:download_cache` to `core.download:download_cache`, so it is only necessary to define one. #13458
- Feature: Authentication for `tools.files.download()`. #13421 . Docs here
- Fix: Define a way to update `default_options` in `python_requires_extend` extension. #13487 . Docs here
- Fix: Allow again to specify `self.options["mydep"].someoption=value`, equivalent to `"mydep/*"`. #13467
- Fix: Generate *cpp_std=vc++20* for c++20 with meson with VS2019 and VS2022, rather than *vc++latest*. #13450
- Bugfix: Fixed `CMakeDeps` not clearing `CONAN_SHARED_FOUND_LIBRARY` var in `find_library()`. #13596
- Bugfix: Do not allow adding more than 1 remote with the same remote name. #13574
- Bugfix: `cmd_wrapper` added missing parameter `conanfile`. #13564 . Docs here
- Bugfix: Avoid generators errors because dependencies binaries of editable packages were "skip". #13544
- Bugfix: Fix subcommands names when the parent command has underscores. #13516
- Bugfix: Fix `python-requires` in remotes when running **`conan export-pkg`**. #13496
- Bugfix: Editable packages now also follow `build_folder_vars` configuration. #13488
- Bugfix: Fix `[system_tools]` profile composition. #13468

13.83 2.0.2 (15-Mar-2023)

- Feature: Allow relative paths to the Conan home folder in the `global.conf`. #13415 . Docs here
- Feature: Some improvements for html formatter in **`conan list`** command. #13409 . Docs here
- Feature: Adds an optional "build_script_folder" argument to the *autoreconf* method of the *Autotools* class. It mirrors the same argument and behavior of the *configure* method of the same class. That is, it allows one to override where the tool is run (by default it runs in the *source_folder*. #13403
- Feature: Create summary of cached content. #13386
- Feature: Add *conan config show* <*conf*> command. #13354 . Docs here
- Feature: Allow `global.conf` jinja2 inclusion of other files. #13336
- Feature: Add `conan export-pkg --skip-binaries` to allow exporting without binaries of dependencies. #13324 . Docs here
- Feature: Add *core.version_ranges:resolve_prereleases* conf to control whether version ranges can resolve to prerelease versions #13321
- Fix: Allow automatic processing of `package_type = "build-scripts"` in **`conan create`** as `--build-require`. #13433
- Fix: Improve the detection and messages of server side package corruption. #13432
- Fix: Fix conan download help typo. #13430
- Fix: Remove profile arguments from *conan profile path*. #13423 . Docs here
- Fix: Fix typo in _detect_compiler_version. #13396
- Fix: Fix `conan profile detect` detection of `libc++` for `clang` compiler on OSX. #13359
- Fix: Allow internal `vswhere` calls to detect and use VS pre-releases too. #13355
- Fix: Allow **`conan export-pkg`** to use remotes to install missing dependencies not in the cache. #13324 . Docs here
- Fix: Allow conversion to `dict` of `settings.yml` lists when `settings_user.yml` define a `dict`. #13323
- Fix: Fix flags passed by AutotoolsToolchain when cross compiling from macOS to a non-Apple OS. #13230
- BugFix: Fix issues in `MSBuild` with custom configurations when custom configurations has spaces. #13435
- Bugfix: Solve bug in `conan profile path <nonexisting>` that was crashing. #13434
- Bugfix: Add global verbosity conf *tools.build:verbosity* instead of individual ones. #13428 . Docs here
- Bugfix: Avoid raising fatal exceptions for malformed custom commands. #13365
- Bugfix: Do not omit `system_libs` from dependencies even if they are header-only. #13364
- Bugfix: Fix `VirtualBuildEnv` environment not being created when `MesonToolchain` is instantiated. #13346
- Bugfix: Nicer error in the compatibility plugin with custom compilers. #13328
- Bugfix: adds qcc cppstd compatibility info to allow dep graph to be calculated. #13326

13.84 2.0.1 (03-Mar-2023)

- Feature: Add *–insecure* alias to *–verify-ssl* in config install. #13270 . Docs here
- Feature: Add *.conanignore* support to *conan config install*. #13269 . Docs here
- Feature: Make verbose tracebacks on exception be shown for `-vv` and `-vvv`, instead of custom env-var used in 1.X. #13226
- Fix: Minor improvements to **`conan install`** and 2.0-readiness error messages. #13299
- Fix: Remove `vcvars.bat` VS telemetry env-var, to avoid Conan hanging. #13293
- Fix: Remove legacy `CMakeToolchain` support for `CMakePresets` schema2 for `CMakeUserPresets.json`. #13288 . Docs here
- Fix: Remove `--logger` json logging and legacy traces. #13287 . Docs here
- Fix: Fix typo in *conan remote auth* help. #13285 . Docs here
- Fix: Raise arg error if `conan config list unexpected-arg`. #13282
- Fix: Do not auto-detect `compiler.runtime_type` for `msvc`, rely on profile plugin. #13277
- Fix: Fix conanfile.txt options parsing error message. #13266
- Fix: Improve error message for unified patterns in options. #13264
- Fix: Allow `conan remote add --force` to force re-definition of an existing remote name. #13249
- Fix: Restore printing of profiles for build command. #13214
- Fix: Change **`conan build`** argument description for "path" to indicate it is only for conanfile.py and explicitly state that it does not work with conanfile.txt. #13211 . Docs here
- Fix: Better error message when dependencies `options` are defined in `requirements()` method. #13207
- Fix: Fix broken links to docs from error messages and readme. #13186
- Bugfix: Ensure that *topics* are always serialized as lists. #13298
- Bugfix: Ensure that *provides* are always serialized as lists. #13298
- Bugfix: Fixed the detection of certain visual c++ installations. #13284
- Bugfix: Fix supported `cppstd` values for `msvc` compiler. #13278
- Bugfix: CMakeDeps generate files for `tool_requires` with the same `build_type` as the "host" context. #13267
- Bugfix: Fix definition of patterns for dependencies options in configure(). #13263
- Bugfix: Fix CMakeToolchain error when output folder in different Win drive. #13248
- Bugfix: Do not raise errors if a `test_requires` is not used by components `.requires`. #13191

13.85 2.0.0 (22-Feb-2023)

- Feature: Change default profile cppstd for apple-clang to gnu17. #13185
- Feature: New `conan remote auth` command to force authentication in the remotes #13180
- Fix: Allow defining options trait in `test_requires(..., options={})` #13178
- Fix: Unifying Conan commands help messages. #13176
- Bugfix: Fix MesonToolchain wrong cppstd in apple-clang #13172
- Feature: Improved global Conan output messages (create, install, export, etc.) #12746

13.86 2.0.0-beta10 (16-Feb-2023)

- Feature: Add basic html output to *conan list* command. #13135
- Feature: Allow `test_package` to process `--build` arguments (computing –build=never for the main, non test_package graph). #13117
- Feature: Add *–force* argument to remote add. #13112
- Feature: Validate if the input configurations exist, to avoid typos. #13110
- Feature: Allow defining `self.folders.build_folder_vars` in recipes `layout()`. #13109
- Feature: Block settings assignment. #13099
- Feature: Improve *conan editable* ui. #13093
- Feature: Provide the ability for users to extend Conan generated CMakePresets. #13090
- Feature: Add error messages to help with the migration of recipes to 2.0, both from ConanCenter and from user repos. #13074
- Feature: Remove option.fPIC for shared in **`conan new`** templates. #13066
- Feature: Add *conan cache clean* subcommand to clean build and source folders. #13050
- Feature: Implement customizable `CMakeToolchain.presets_prefix` so presets name prepend this. #13015
- Feature: Add *[system_tools]* section to profiles to use your own installed tools instead of the packages declared in the requires. #10166
- Fix: Fixes in powershell escaping. #13084
- Fix: Define `CMakeToolchain.presets_prefix="conan"` by default, to avoid conflict with other users presets. #13015

13.87 2.0.0-beta9 (31-Jan-2023)

- Feature: Add package names in Conan cache hash paths. #13011
- Feature: Implement `tools.build:download_source` conf to force the installation of sources in **`conan install`** or `conan graph info`. #13003
- Feature: Users can define their own settings in *settings_user.yml* that will be merged with the Conan *settings.yml*. #12980
- Feature: List disabled remotes too. #12937

- Fix: PkgConfiDeps is using the wrong `dependencies.host` from dependencies instead of `get_transitive_requires()` computation. #13013
- Fix: Fixing transitive shared linux libraries in CMakeDeps. #13010
- Fix: Fixing issues with test_package output folder. #12992
- Fix: Improve error messages for wrong methods. #12962
- Fix: Fix fail in parallel packages download due to database concurrency issues. #12930
- Fix: Enable authentication against disabled remotes. #12913
- Fix: Improving system_requirements. #12912
- Fix: Change tar format to PAX, which is the Python3.8 default. #12899

13.88 2.0.0-beta8 (12-Jan-2023)

- Feature: Add *unix_path_package_info_legacy* function for those cases in which it is used in *package_info* in recipes that require compatibility with Conan 1.x. In Conan 2, path conversions should not be performed in the *package_info* method. #12886
- Feature: New serialization json and printing for `conan list`. #12883
- Feature: Add requirements to *conan new cmake_{lib,exe}* #12875
- Feature: Allow `--no-remotes` to force temporal disabling of remotes #12808
- Feature: Add barebones template option to conan new. #12802
- Feature: Avoid requesting package configuration if PkgID is passed. #12801
- Feature: Implemented *conan list *#latest* and *conan list *:*#latest*. Basically, this command can show the latest RREVs and PREVs for all the matching references. #12781
- Feature: Allow chaining of *self.output* write methods #12780
- Fix: Make `graph info` filters to work on json output too #12836
- Bugfix: Fix bug to pass a valid GNU triplet when using AutotoolsToolchain and cross-building on Windows. #12881
- Bugfix: Ordering if same ref.name but different versions. #12801

13.89 2.0.0-beta7 (22-Dec-2022)

- Feature: Raise an error when a generator is both defined in generators attribute and instantiated in generate() method #12722
- Feature: *test_requires* improvements, including allowing it in conanfile.txt #12699
- Feature: Improve errors for when required_conan_version has spaces between the operator and the version #12695
- Feature: ConanAPI cleanup and organization #12666

13.90 2.0.0-beta6 (02-Dec-2022)

- Feature: Use `--confirm` to not request confirmation when removing instead of `--force` #12636
- Feature: Simplify loading conaninfo.txt for search results #12616
- Feature: Renamed ConanAPIV2 to ConanAPI #12615
- Feature: Refactor ConanAPI #12615
- Feature: Improve conan cache path command #12554
- Feature: Improve #latest and pattern selection from remove/upload/download #12572
- Feature: Add build_modules to provided deprecated warning to allow migration from 1.x #12578
- Feature: Lockfiles alias support #12525

13.91 2.0.0-beta5 (11-Nov-2022)

- Feature: Improvements in the remotes management and API #12468
- Feature: Implement env_info and user_info as fake attributes in Conan 2.0 #12351
- Feature: Improve settings.rm_safe() #12379
- Feature: New RecipeReference equality #12506
- Feature: Simplifying compress and uncompress of .tgz files #12378
- Feature: conan source command does not require a default profile #12475
- Feature: Created a proper LockfileAPI, with detailed methods (update, save, etc), instead of several loose methods #12502
- Feature: The conan export can also produce lockfiles, necessary for users doing a 2 step (export + install–build) process #12502
- Feature: Drop compat_app #12484
- Fix: Fix transitive propagation of transitive_headers=True #12508
- Fix: Fix transitive propagation of transitive_libs=False for static libraries #12508
- Fix: Fix test_package for python_requires #12508

13.92 2.0.0-beta4 (11-Oct-2022)

- Feature: Do not allow doing conan create/export with uncommitted changes using revision_mode=scm #12267
- Feature: Simplify conan inspect command, removing path subcommand #12263
- Feature: Add –deploy argument to graph info command #12243
- Feature: Pass graph object to deployers instead of ConanFile #12243
- Feature: Add included_files method to conan.tools.scm.Git #12246
- Feature: Improve detection of clang libcxx #12251
- Feature: Remove old profile variables system in favor of Jinja2 syntax in profiles #12152

- Fix: Update command to follow Conan 2.0 conventions about CLI output #12235
- Fix: Fix aggregation of test trait in diamonds #12080

13.93 2.0.0-beta3 (12-Sept-2022)

- Feature: Decouple test_package from create. #12046
- Feature: Warn if special chars in exported refs. #12053
- Feature: Improvements in MSBuildDeps traits. #12032
- Feature: Added support for CLICOLOR_FORCE env var, that will activate the colors in the output if the value is declared and different to 0. #12028
- Fix: Call source() just once for all configurations. #12050
- Fix: Fix deployers not creating output_folder. #11977
- Fix: Fix build_id() removal of require. #12019
- Fix: If Conan fails to load a custom command now it fails with a useful error message. #11720
- Bugfix: If the 'os' is not specified in the build profile and a recipe, in Windows, wanted to run a command. #11728

13.94 2.0.0-beta2 (27-Jul-2022)

- Feature: Add traits support in MSBuildDeps. #11680
- Feature: Add traits support in XcodeDeps. #11615
- Feature: Let dependency define package_id modes. #
- Feature: Add `conan.conanrc` file to setup the conan user home. #11675
- Feature: Add `core.cache:storage_path` to declare the absolute path where you want to store the Conan packages. #11672
- Feature: Add tools for checking max cppstd version. #11610
- Feature: Add a `post_build_fail` hook that is called when a build fails. #11593
- Feature: Add `pre_generate` and `post_generate` hook, covering the generation of files around the `generate()` method call. #11593
- Feature: Brought `conan config list` command back and other conf improvements. #11575
- Feature: Added two new arguments for all commands -v for controlling the verbosity of the output and –logger to output the contents in a json log format for log processors. #11522

13.95 2.0.0-beta1 (20-Jun-2022)

- Feature: New graph model to better support C and C++ binaries relationships, compilation, and linkage.
- Feature: New documented public Python API, for user automation
- Feature: New build system integrations, more flexible and powerful, and providing transparent integration when possible, like `CMakeDeps` and `CMakeToolchain`
- Feature: New custom user commands, that can be built using the public PythonAPI and can be shared and installed with `conan config install`
- Feature: New CLI interface, with cleaner commands and more structured output
- Feature: New deployers mechanism to copy artifacts from the cache to user folders, and consume those copies while building.
- Feature: Improved `package_id` computation, taking into account the new more detailed graph model.
- Feature: Added `compatibility.py` extension mechanism to allow users to define binary compatibility globally.
- Feature: Simpler and more powerful `lockfiles` to provide reproducibility over time.
- Feature: Better configuration with `[conf]` and better environment management with the new `conan.tools.env` tools.
- Feature: Conan cache now can store multiple revisions simultaneously.
- Feature: New extensions plugins to implement profile checking, package signing, and build commands wrapping.
- Feature: Used the package immutability for an improved update, install and upload flows.

INDEX

Symbols

__init__() (*XcodeBuild method*), 824

A

absolute_to_relative_symlinks() (*in module conan.tools.files.symlinks*), 903
add() (*RemotesAPI method*), 793
add_configuration() (*Qbs method*), 973
add_lockfile() (*LockfileAPI static method*), 790
add_pref() (*PackagesList method*), 799
add_provider() (*AuditAPI method*), 772
add_ref() (*PackagesList method*), 799
analyze_binaries() (*GraphAPI method*), 783
android_abi() (*in module conan.tools.android*), 818
Apk (*class in conan.tools.system.package_manager*), 993
append() (*Conf method*), 674
append() (*Environment method*), 878
append_path() (*Environment method*), 878
apple_arch_flag (*MesonToolchain attribute*), 955
apple_extra_flags (*MesonToolchain attribute*), 956
apple_isysroot_flag (*MesonToolchain attribute*), 955
apple_min_version_flag (*MesonToolchain attribute*), 956
apply() (*EnvVars method*), 882
apply_conandata_patches() (*in module conan.tools.files.patches*), 900
Apt (*class in conan.tools.system.package_manager*), 994
ar (*MesonToolchain attribute*), 955
ar (*XCRun property*), 827
arch (*IntelCC attribute*), 945
arch_flag (*MesonToolchain attribute*), 954
arch_link_flag (*MesonToolchain attribute*), 954
architecture_bits() (*in module conan.tools.build.flags*), 828
arguments (*Premake attribute*), 989
as_ (*MesonToolchain attribute*), 955
audit (*ConanAPI attribute*), 770
AuditAPI (*class in conan.api.subapi.audit*), 771
auth_provider() (*AuditAPI method*), 772
autoreconf() (*Autotools method*), 920
Autotools (*class in conan.tools.gnu.autotools*), 920
AutotoolsDeps (*class in conan.tools.gnu.autotoolsdeps*), 905
AutotoolsToolchain (*class in conan.tools.gnu.autotoolstoolchain*), 910

B

b_ndebug (*MesonToolchain attribute*), 954
b_staticpic (*MesonToolchain attribute*), 954
b_vscrt (*MesonToolchain attribute*), 954
backend (*MesonToolchain attribute*), 953
Bazel (*class in conan.tools.google*), 930
BazelDeps (*class in conan.tools.google*), 937
BazelToolchain (*class in conan.tools.google*), 940
bin_path (*PyEnv property*), 1005
binaries (*MesonToolchain attribute*), 954
Brew (*class in conan.tools.system.package_manager*), 1000
build() (*Bazel method*), 930
build() (*CMake method*), 869
build() (*LocalAPI method*), 788
build() (*Meson method*), 957
build() (*MSBuild method*), 959
build() (*Premake method*), 990
build() (*Qbs method*), 973
build() (*XcodeBuild method*), 824
build_all() (*Qbs method*), 973
build_context_activated (*BazelDeps attribute*), 937
build_jobs() (*in module conan.tools.build.cpu*), 827
build_path() (*CacheAPI method*), 774
buildtype (*MesonToolchain attribute*), 954

C

c (*MesonToolchain attribute*), 955
c_args (*MesonToolchain attribute*), 955
c_ld (*MesonToolchain attribute*), 955
c_link_args (*MesonToolchain attribute*), 955
c_std (*MesonToolchain attribute*), 954
cache (*ConanAPI attribute*), 770
CacheAPI (*class in conan.api.subapi.cache*), 773
can_run() (*in module conan.tools.build.cross_building*), 828
cc (*XCRun property*), 827

channel (*RecipeReference attribute*), 801
chdir() (*in module conan.tools.files.files*), 893
check() (*Apk method*), 993
check() (*Apt method*), 995
check() (*Brew method*), 1000
check() (*Chocolatey method*), 1003
check() (*PacMan method*), 998
check() (*Pkg method*), 1001
check() (*PkgUtil method*), 1002
check() (*Yum method*), 996
check() (*Zypper method*), 999
check_integrity() (*CacheAPI method*), 775
check_lockfile_config() (*LockfileAPI method*), 789
check_max_cppstd() (*in module conan.tools.build.cppstd*), 829
check_max_cstd() (*in module conan.tools.build.cstd*), 831
check_md5() (*in module conan.tools.files.files*), 902
check_min_compiler_version() (*in module conan.tools.build.compiler*), 833
check_min_cppstd() (*in module conan.tools.build.cppstd*), 828
check_min_cstd() (*in module conan.tools.build.cstd*), 831
check_min_vs() (*in module conan.tools.microsoft*), 971
check_sha1() (*in module conan.tools.files.files*), 902
check_sha256() (*in module conan.tools.files.files*), 902
check_upstream() (*UploadAPI method*), 797
checkout() (*Git method*), 981
checkout_from_conandata_coordinates() (*Git method*), 982
chmod() (*in module conan.tools.files.files*), 891
Chocolatey (*class in conan.tools.system.package_manager*), 1003
clean() (*CacheAPI method*), 775
clean() (*ConfigAPI method*), 780
clone() (*Git method*), 981
CMake (*class in conan.tools.cmake.cmake*), 869
cmake_layout() (*in module conan.tools.cmake.layout*), 872
CMakeConfigDeps (*class in conan.tools.cmake.cmakeconfigdeps.cmakeconfigdeps*), 852
CMakeDeps (*class in conan.tools.cmake.cmakedeps.cmakedeps*), 837
CMakeToolchain (*class in conan.tools.cmake.toolchain.toolchain*), 865
collect_libs() (*in module conan.tools.files*), 896
command (*ConanAPI attribute*), 770
command (*IntelCC property*), 945
command() (*MSBuild method*), 959
CommandAPI (*class in conan.api.subapi.command*), 777
commit_in_remote() (*Git method*), 980
compilation_mode (*BazelToolchain attribute*), 941
compiler (*BazelToolchain attribute*), 941
compose_env() (*Environment method*), 879
ConanAPI (*class in conan.api.conan_api*), 769
conf_list() (*ConfigAPI static method*), 780
config (*ConanAPI attribute*), 769
ConfigAPI (*class in conan.api.subapi.config*), 778
configuration (*MSBuildDeps attribute*), 963
configuration_key (*MSBuildDeps attribute*), 963
configure() (*Autotools method*), 920
configure() (*CMake method*), 869
configure() (*Meson method*), 957
configure() (*Premake method*), 990
conlyopt (*BazelToolchain attribute*), 940
content (*QbsDeps property*), 975
content (*QbsProfile property*), 976
coordinates_to_conandata() (*Git method*), 981
copt (*BazelToolchain attribute*), 940
copy() (*in module conan.tools.files.copy_pattern*), 888
cpp (*MesonToolchain attribute*), 955
cpp_args (*MesonToolchain attribute*), 955
cpp_ld (*MesonToolchain attribute*), 955
cpp_link_args (*MesonToolchain attribute*), 955
cpp_std (*MesonToolchain attribute*), 954
cppstd (*BazelToolchain attribute*), 940
cppstd_flag() (*in module conan.tools.build.flags*), 830
cpu (*BazelToolchain attribute*), 941
cross_build (*MesonToolchain attribute*), 955
cross_building() (*in module conan.tools.build.cross_building*), 828
crosstool_top (*BazelToolchain attribute*), 941
ctest() (*CMake method*), 870
cxx (*XCRun property*), 827
cxxopt (*BazelToolchain attribute*), 940
cyclonedx_1_4() (*in module conan.tools.sbom.cyclonedx*), 978
cyclonedx_1_6() (*in module conan.tools.sbom.cyclonedx*), 978

D

debug() (*in module conan.api.output.ConanOutput*), 694
default_cppstd() (*in module conan.tools.build.cppstd*), 830
default_cstd() (*in module conan.tools.build.cstd*), 832
default_library (*MesonToolchain attribute*), 954
define() (*Conf method*), 673
define() (*Environment method*), 878
deploy() (*InstallAPI method*), 785
deploy_base_folder() (*Environment method*), 879
deserialize() (*PackagesList static method*), 800
detect() (*ProfilesAPI static method*), 792
diff() (*ReportAPI method*), 796

disable() (*RemotesAPI method*), 793
download (*ConanAPI attribute*), 770
download() (*in module conan.tools.files.files*), 898
download_full() (*DownloadAPI method*), 781
DownloadAPI (*class in conan.api.subapi.download*), 780
dumps() (*Environment method*), 878
dynamic_mode (*BazelToolchain attribute*), 940

E

editable_add() (*LocalAPI method*), 787
editable_remove() (*LocalAPI method*), 787
enable() (*RemotesAPI method*), 793
env_dir (*PyEnv property*), 1005
env_exe (*PyEnv property*), 1005
environment (*AutotoolsDeps property*), 905
Environment (*class in conan.tools.env.environment*), 878
environment() (*VirtualBuildEnv method*), 885
environment() (*VirtualRunEnv method*), 887
EnvVars (*class in conan.tools.env.environment*), 882
error() (*in module conan.api.output.ConanOutput*), 693
exclude_code_analysis (*MSBuildDeps attribute*), 963
explain_missing_binaries() (*ListAPI method*), 786
export (*ConanAPI attribute*), 770
export() (*ExportAPI method*), 781
export_conandata_patches() (*in module conan.tools.files.patches*), 901
export_path() (*CacheAPI method*), 773
export_pkg() (*ExportAPI method*), 782
export_pkg_graph() (*ExportAPI method*), 781
export_source_path() (*CacheAPI method*), 773
ExportAPI (*class in conan.api.subapi.export*), 781
extra_cflags (*MesonToolchain attribute*), 954
extra_cflags (*PremakeToolchain attribute*), 988
extra_cxxflags (*MesonToolchain attribute*), 954
extra_cxxflags (*PremakeToolchain attribute*), 988
extra_defines (*MesonToolchain attribute*), 954
extra_defines (*PremakeToolchain attribute*), 988
extra_ldflags (*MesonToolchain attribute*), 954
extra_ldflags (*PremakeToolchain attribute*), 988

F

fetch_commit() (*Git method*), 981
fetch_packages() (*ConfigAPI method*), 779
filename (*QbsProfile property*), 976
fill_cpp_info() (*PkgConfig method*), 929
find() (*XCRun method*), 826
find_first_missing_binary() (*GraphAPI static method*), 784
find_remotes() (*ListAPI method*), 786
fix_apple_shared_install_name() (*in module conan.tools.apple*), 825
force_pic (*BazelToolchain attribute*), 940
ftp_download() (*in module conan.tools.files.files*), 897

G

generate() (*BazelDeps method*), 937
generate() (*BazelToolchain method*), 941
generate() (*CMakeConfigDeps method*), 852
generate() (*CMakeDeps method*), 837
generate() (*CMakeToolchain method*), 865
generate() (*IntelCC method*), 945
generate() (*MakeDeps method*), 925
generate() (*MesonToolchain method*), 956
generate() (*MSBuildDeps method*), 963
generate() (*MSBuildToolchain method*), 965
generate() (*PkgConfigDeps method*), 926
generate() (*PremakeToolchain method*), 988
generate() (*PyEnv method*), 1005
generate() (*QbsDeps method*), 975
generate() (*QbsProfile method*), 976
generate() (*ROSEnv method*), 977
generate() (*VCVars method*), 967
generate() (*VirtualBuildEnv method*), 885
generate() (*VirtualRunEnv method*), 887
get() (*ConfigAPI method*), 779
get() (*EnvVars method*), 882
get() (*in module conan.tools.files.files*), 896
get() (*RemotesAPI method*), 793
get_backup_sources() (*CacheAPI method*), 776
get_cmake_package_name() (*CMakeDeps method*), 838
get_commit() (*Git method*), 979
get_conanfile_path() (*LocalAPI static method*), 787
get_default_build() (*ProfilesAPI method*), 792
get_default_host() (*ProfilesAPI method*), 791
get_find_mode() (*CMakeDeps method*), 838
get_lockfile() (*LockfileAPI static method*), 789
get_path() (*ProfilesAPI method*), 792
get_profile() (*ProfilesAPI method*), 792
get_provider() (*AuditAPI method*), 772
get_remote_url() (*Git method*), 980
get_repo_root() (*Git method*), 981
get_url_and_commit() (*Git method*), 980
Git (*class in conan.tools.scm.git*), 979
GnuToolchain (*class in conan.tools.gnu.gnutoolchain*), 918
GraphAPI (*class in conan.api.subapi.graph*), 782

H

highlight() (*in module conan.api.output.ConanOutput*), 693
home() (*ConfigAPI method*), 778
home_folder (*ConanAPI property*), 770

I

included_files() (*Git method*), 981

info() (*in module conan.api.output.ConanOutput*), 693
install (*ConanAPI attribute*), 770
install() (*Apk method*), 993
install() (*Apt method*), 994
install() (*Autotools method*), 920
install() (*Brew method*), 1000
install() (*Chocolatey method*), 1004
install() (*CMake method*), 870
install() (*ConfigAPI method*), 778
install() (*Meson method*), 957
install() (*PacMan method*), 998
install() (*Pkg method*), 1001
install() (*PkgUtil method*), 1003
install() (*PyEnv method*), 1005
install() (*Qbs method*), 973
install() (*Yum method*), 996
install() (*Zypper method*), 999
install_binaries() (*InstallAPI method*), 784
install_conanconfig() (*ConfigAPI method*), 779
install_consumer() (*InstallAPI method*), 785
install_name_tool (*XCRun property*), 827
install_package() (*ConfigAPI method*), 778
install_sources() (*InstallAPI method*), 785
install_substitutes() (*Apk method*), 993
install_substitutes() (*Apt method*), 995
install_substitutes() (*Brew method*), 1001
install_substitutes() (*Chocolatey method*), 1004
install_substitutes() (*PacMan method*), 998
install_substitutes() (*Pkg method*), 1002
install_substitutes() (*PkgUtil method*), 1003
install_substitutes() (*Yum method*), 997
install_substitutes() (*Zypper method*), 999
install_system_requires() (*InstallAPI method*), 784
InstallAPI (*class in conan.api.subapi.install*), 784
installation_path (*IntelCC property*), 945
IntelCC (*class in conan.tools.intel*), 945
invalidate_cache() (*Remote method*), 799
is_apple_os() (*in module conan.tools.apple*), 826
is_dirty() (*Git method*), 980
is_msvc() (*in module conan.tools.microsoft*), 971
is_msvc_static_runtime() (*in module conan.tools.microsoft*), 971
items() (*EnvVars method*), 882
items() (*PackagesList method*), 800

L

latest_recipe_revision() (*ListAPI method*), 786
ld (*MesonToolchain attribute*), 955
libtool (*XCRun property*), 827
linkopt (*BazelToolchain attribute*), 941
list (*ConanAPI attribute*), 770
list() (*AuditAPI static method*), 771
list() (*ProfilesAPI method*), 792
list() (*RemotesAPI method*), 793
list_providers() (*AuditAPI method*), 772
ListAPI (*class in conan.api.subapi.list*), 786
ListPattern (*class in conan.api.model*), 801
load() (*in module conan.tools.files.files*), 889
load() (*MultiPackagesList static method*), 800
load_graph() (*GraphAPI method*), 783
load_graph() (*MultiPackagesList static method*), 800
load_root_test_conanfile() (*GraphAPI method*), 783
loads() (*PkgReference static method*), 802
loads() (*RecipeReference static method*), 801
local (*ConanAPI attribute*), 770
LocalAPI (*class in conan.api.subapi.local*), 787
lockfile (*ConanAPI attribute*), 770
LockfileAPI (*class in conan.api.subapi.lockfile*), 789
luafile (*Premake attribute*), 989

M

make() (*Autotools method*), 920
MakeDeps (*class in conan.tools.gnu*), 925
matches() (*RecipeReference method*), 802
merge_lockfiles() (*LockfileAPI static method*), 790
Meson (*class in conan.tools.meson*), 957
MesonToolchain (*class in conan.tools.meson*), 953
mkdir() (*in module conan.tools.files.files*), 892
ms_toolset (*IntelCC property*), 945
MSBuild (*class in conan.tools.microsoft*), 959
MSBuildDeps (*class in conan.tools.microsoft*), 963
MSBuildToolchain (*class in conan.tools.microsoft*), 965
msvc_runtime_flag() (*in module conan.tools.microsoft*), 971
msvs_toolset() (*in module conan.tools.microsoft*), 972
MultiPackagesList (*class in conan.api.model*), 800

N

name (*RecipeReference attribute*), 801
NewAPI (*class in conan.api.subapi.new*), 791

O

objc (*MesonToolchain attribute*), 956
objc_args (*MesonToolchain attribute*), 956
objc_link_args (*MesonToolchain attribute*), 956
objcpp (*MesonToolchain attribute*), 956
objcpp_args (*MesonToolchain attribute*), 956
objcpp_link_args (*MesonToolchain attribute*), 956
only_recipes() (*PackagesList method*), 799
otool (*XCRun property*), 827

P

package() (*DownloadAPI method*), 780
package() (*RemoveAPI method*), 795
package_dict() (*PackagesList method*), 800

package_id (*PkgReference attribute*), 802
package_metadata() (*DownloadAPI method*), 781
package_metadata_path() (*CacheAPI method*), 774
package_path() (*CacheAPI method*), 775
packages() (*RemoveAPI method*), 795
PackagesList (*class in conan.api.model*), 799
PacMan (*class in conan.tools.system.package_manager*), 998
patch() (*in module conan.tools.files.patches*), 899
Pkg (*class in conan.tools.system.package_manager*), 1001
pkg_config_path (*MesonToolchain attribute*), 955
PkgConfig (*class in conan.tools.gnu*), 929
pkgconfig (*MesonToolchain attribute*), 955
PkgConfigDeps (*class in conan.tools.gnu*), 926
PkgReference (*class in conan.api.model*), 802
PkgUtil (*class in conan.tools.system.package_manager*), 1002
platform (*MSBuildDeps attribute*), 963
platform_key (*MSBuildDeps attribute*), 963
Premake (*class in conan.tools.premake*), 989
PremakeToolchain (*class in conan.tools.premake*), 988
prepare() (*UploadAPI method*), 797
prepend() (*Conf method*), 674
prepend() (*Environment method*), 878
prepend_path() (*Environment method*), 878
preprocessor_definitions (*MesonToolchain attribute*), 954
profiles (*ConanAPI attribute*), 770
ProfilesAPI (*class in conan.api.subapi.profiles*), 791
project() (*PremakeToolchain method*), 988
project_options (*MesonToolchain attribute*), 954
properties (*MesonToolchain attribute*), 954
PyEnv (*class in conan.tools.system*), 1005

Q

Qbs (*class in conan.tools.qbs.qbs*), 973
QbsDeps (*class in conan.tools.qbs.qbsdeps*), 975
QbsProfile (*class in conan.tools.qbs.qbsprofile*), 976

R

ranlib (*XCRun property*), 827
recipe() (*DownloadAPI method*), 780
recipe() (*RemoveAPI method*), 795
recipe_dict() (*PackagesList method*), 800
recipe_metadata() (*DownloadAPI method*), 780
recipe_metadata_path() (*CacheAPI method*), 773
recipe_revisions() (*ListAPI method*), 786
RecipeReference (*class in conan.api.model*), 801
recipes() (*RemoveAPI method*), 795
ref (*PkgReference attribute*), 802
reinit() (*ConanAPI method*), 770
Remote (*class in conan.api.model*), 799
remotes (*ConanAPI attribute*), 770
RemotesAPI (*class in conan.api.subapi.remotes*), 793
remove() (*Conf method*), 674
remove() (*Environment method*), 879
remove() (*RemotesAPI method*), 794
remove_broken_symlinks() (*in module conan.tools.files.symlinks*), 903
remove_external_symlinks() (*in module conan.tools.files.symlinks*), 903
remove_lockfile() (*LockfileAPI static method*), 790
remove_provider() (*AuditAPI method*), 772
RemoveAPI (*class in conan.api.subapi.remove*), 795
rename() (*in module conan.tools.files.files*), 890
rename() (*RemotesAPI method*), 794
render() (*QbsProfile method*), 976
replace_in_file() (*in module conan.tools.files.files*), 890
ReportAPI (*class in conan.api.subapi.report*), 796
resolve() (*Qbs method*), 973
restore() (*CacheAPI method*), 776
revision (*PkgReference attribute*), 802
revision (*RecipeReference attribute*), 801
rm() (*in module conan.tools.files.files*), 892
rmdir() (*in module conan.tools.files.files*), 893
ROSEnv (*class in conan.tools.ros*), 977
run() (*CommandAPI method*), 777
run() (*Git method*), 979
run() (*in module conan.internal.model.conan_file.ConanFile*), 695

S

save() (*CacheAPI method*), 776
save() (*in module conan.tools.files.files*), 889
save_lockfile() (*LockfileAPI static method*), 790
save_script() (*EnvVars method*), 882
save_template() (*NewAPI method*), 791
scan() (*AuditAPI static method*), 771
sdk_path (*XCRun property*), 826
sdk_platform_path (*XCRun property*), 827
sdk_platform_version (*XCRun property*), 827
sdk_version (*XCRun property*), 827
select() (*ListAPI method*), 786
serialize() (*MultiPackagesList method*), 800
serialize() (*PackagesList method*), 800
set_property() (*CMakeConfigDeps method*), 852
set_property() (*CMakeDeps method*), 837
set_property() (*PkgConfigDeps method*), 926
settings_yml (*ConfigAPI property*), 780
show() (*ConfigAPI method*), 779
sign() (*CacheAPI method*), 775
source() (*LocalAPI method*), 788
source_path() (*CacheAPI method*), 774
split() (*PackagesList method*), 799

status() (*in module conan.api.output.ConanOutput*), 693
strip (*MesonToolchain attribute*), 955
strip (*XCRun property*), 827
subproject_options (*MesonToolchain attribute*), 954
success() (*in module conan.api.output.ConanOutput*), 693
supported_cppstd() (*in module conan.tools.build.cppstd*), 830
supported_cstd() (*in module conan.tools.build.cstd*), 833

T

test() (*Bazel method*), 931
test() (*CMake method*), 870
test() (*LocalAPI static method*), 788
test() (*Meson method*), 957
threads_flags (*MesonToolchain attribute*), 954
timestamp (*PkgReference attribute*), 802
timestamp (*RecipeReference attribute*), 801
to_apple_arch() (*in module conan.tools.apple*), 826
trace() (*in module conan.api.output.ConanOutput*), 694
trim_conandata() (*in module conan.tools.files.conandata*), 895

U

unix_path() (*in module conan.tools.microsoft*), 972
unset() (*Conf method*), 675
unset() (*Environment method*), 878
unzip() (*in module conan.tools.files.files*), 894
update() (*Apk method*), 994
update() (*Apt method*), 995
update() (*Brew method*), 1001
update() (*Chocolatey method*), 1004
update() (*Conf method*), 674
update() (*PacMan method*), 999
update() (*Pkg method*), 1002
update() (*PkgUtil method*), 1003
update() (*RemotesAPI method*), 794
update() (*Yum method*), 997
update() (*Zypper method*), 1000
update_autoreconf_args() (*AutotoolsToolchain method*), 910
update_conandata() (*in module conan.tools.files.conandata*), 895
update_configure_args() (*AutotoolsToolchain method*), 910
update_lockfile() (*LockfileAPI static method*), 790
update_lockfile_export() (*LockfileAPI method*), 789
update_make_args() (*AutotoolsToolchain method*), 910
upload (*ConanAPI attribute*), 770
upload_backup_sources() (*UploadAPI method*), 798
upload_full() (*UploadAPI method*), 797
UploadAPI (*class in conan.api.subapi.upload*), 797
user (*RecipeReference attribute*), 801
user_login() (*RemotesAPI method*), 794
user_logout() (*RemotesAPI method*), 794

V

valid_max_cppstd() (*in module conan.tools.build.cppstd*), 829
valid_max_cstd() (*in module conan.tools.build.cstd*), 832
valid_min_cppstd() (*in module conan.tools.build.cppstd*), 829
valid_min_cstd() (*in module conan.tools.build.cstd*), 832
validate_ref() (*RecipeReference method*), 802
vars() (*Environment method*), 879
vars() (*VirtualBuildEnv method*), 885
vars() (*VirtualRunEnv method*), 887
VCVars (*class in conan.tools.microsoft*), 967
verbose() (*in module conan.api.output.ConanOutput*), 693
verify() (*CacheAPI method*), 775
Version (*class in conan.tools.scm*), 982
version (*RecipeReference attribute*), 801
VirtualBuildEnv (*class in conan.tools.env.virtualbuildenv*), 885
VirtualRunEnv (*class in conan.tools.env.virtualrunenv*), 887
vs_layout() (*in module conan.tools.microsoft*), 970

W

warning() (*in module conan.api.output.ConanOutput*), 693
windres (*MesonToolchain attribute*), 955
workspace (*ConanAPI attribute*), 770

X

XcodeBuild (*class in conan.tools.apple.xcodebuild*), 824
XCRun (*class in conan.tools.apple*), 826

Y

yes_no() (*GnuToolchain method*), 918
Yum (*class in conan.tools.system.package_manager*), 996

Z

Zypper (*class in conan.tools.system.package_manager*), 999